Femap API Guide

SIEMENS

Unpublished work. © 2025 Siemens

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, “Siemens”), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer’s applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at **https://www.sw.siemens.com/en-US/sw-terms/base/uca/**, as supplemented by the product specific terms which may be viewed at **https://www.sw.siemens.com/en-US/sw-terms/supplements/**.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: **https://www.plm.automation.siemens.com/global/en/legal/trademarks.html**. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow’s idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: **https://support.sw.siemens.com**

Send Feedback on Documentation: **https://support.sw.siemens.com/doc_feedback_form**

Table of Contents

Femap API Guide....1-1
Objects, Methods and Properties....1-1
The Application Object....1-2
Other Objects....1-2
Data Types....1-3
Using Visual Basic .NET....1-4
Arrays and Variants....1-5
An Array Shortcut....1-7
Arrays and Visual Basic .NET....1-7
Arrays and Visual C++....1-8
Using Multi-dimensional Arrays....1-8
Multiple Definitions of Array Properties....1-10
Memory Allocation....1-10
Global Constants....1-11
Other Considerations....1-11

Using the Femap API....2-1
Using Visual Basic .NET....2-1
Loading Data into Excel....2-2
Loading Data from Excel....2-6
Working with Output in Femap v2020.1 and above....2-10
Using the ReadFile Object....2-18
Storing Your Own Data in the FEMAP Database....2-20
Creating a Rigid Element....2-23
Working with Loads and Constraints....2-26
Creating Multiple Loads at Once in a Single Load Set....2-27
Walking Through Mesh-Based Loads and Constraints....2-28
Charting Femap Results in Excel....2-31
Calling the API from Visual C++....2-38
Calling the API from Python....2-42
API Selection Tool - Select Property(s) To Get Nodes....2-58
Femap API Code....2-62
Creating Output in Femap prior to v2020.1 (OBSOLETE)....2-64

The Femap Application Object....3-1
Getting Started....3-1
Using the Femap API Programming Window....3-1
Creating a Femap Object....3-2
Using the Femap Type Library....3-4
Working with the User Interface....3-7
Running as two separate applications....3-7
Running Femap as a hidden server....3-7

Unpublished work. © 2025 Siemens

Running your application as a Femap add-in..... 3-8
Embedding Femap in your application..... 3-9
API Performance..... 3-15
Femap Object Properties..... 3-15
Global Properties - **File** menu commands..... 3-16
Global Integer Properties..... 3-80
Global Real Properties..... 3-90
Global Boolean Properties..... 3-94
Global String Properties..... 3-98
Entity Types..... 3-99
Femap Constants..... 3-103
File Methods..... 3-173
File Open/Exit..... 3-174
File Save..... 3-178
Import Geometry..... 3-182
Import Analysis Model..... 3-217
Import Analysis Results..... 3-224
Attach To Results File..... 3-230
Import/Export Neutral File..... 3-240
Export Geometry..... 3-258
Export Analysis Model..... 3-269
Notes..... 3-279
Print and Copy..... 3-281
Rebuild and Preferences..... 3-302
Version Number..... 3-307
Tool Methods..... 3-308
Units..... 3-308
Variables..... 3-312
Layers..... 3-315
Text..... 3-317
Measure..... 3-330
Checks..... 3-352
Charting Pane..... 3-390
Geometry Methods..... 3-393
Lines..... 3-393
Arcs..... 3-402
Circles..... 3-410
Splines..... 3-419
Curves from Surfaces Methods..... 3-427
Curve Modification Methods..... 3-445
Boundaries..... 3-454
Surfaces..... 3-462
Midsurfaces..... 3-487
Solids..... 3-503
Geometry and Mesh Generation Methods..... 3-554
Meshing Toolbox - Geometry Editing Tool Methods..... 3-574
Creating FEMAP geometry from Parasolid Geometry..... 3-578
Nodal Area on Geometry..... 3-581
Connection Methods..... 3-583

Unpublished work. © 2025 Siemens

Generating Connections..... 3-584
Model Methods..... 3-587
Model Load Methods..... 3-587
Model Constraint Methods..... 3-590
Model Output Methods..... 3-591
Model Information Methods..... 3-632
Meshing Methods..... 3-634
Mesh Control..... 3-634
Meshing Geometry..... 3-654
Mesh Editing..... 3-680
Other Meshing..... 3-685
Meshing Utilities..... 3-739
Mesh Copy/**Radial Copy**/**Scale**/**Rotate**/**Reflect** commands..... 3-744
Modification Methods..... 3-744
Projection..... 3-744
Move, Rotate and Scale..... 3-753
Edit..... 3-767
Color and Layer..... 3-769
Renumbering..... 3-776
Associativity..... 3-782
Other Updates..... 3-788
Element Updates..... 3-798
Delete Methods..... 3-831
feDeleteAll..... 3-832
feDelete..... 3-833
feDeleteMesh..... 3-834
feDeleteLoads..... 3-835
feDeleteConstraints..... 3-837
feDeleteOutputV2..... 3-838
feDeleteOutput2V2..... 3-839
feDeleteOutputEntryV2..... 3-840
View, Window and Visibility Methods..... 3-842
feViewVisible (or feWindowVisible)..... 3-842
feViewRedraw (or feWindowRedraw)..... 3-842
feViewRegenerate (or feWindowRegenerate)..... 3-843
feViewAutoscaleAll..... 3-844
feViewAutoscaleVisible..... 3-845
feViewAllViews..... 3-845
feViewShow (or feWindowShow)..... 3-846
feViewShow2 (or feWindowShow2)..... 3-847
feViewTile (or feWindowTile)..... 3-848
feWindowTileHorz..... 3-849
feViewCascade (or feWindowCascade)..... 3-849
feWindowMinimize..... 3-850
feWindowMaximize..... 3-851
feWindowRestore..... 3-851
feWindowSetRect..... 3-852
feWindowSetSize..... 3-853
feWindowTitle..... 3-854

Unpublished work. © 2025 Siemens

feEntitySetVisibility....3-855
feEntitySetVisibility2....3-856
feEntityGetVisibility....3-857
feAppUpdateModelBox....3-859
feViewShowMultiple....3-859
Group Methods....3-861
feGroupEvaluate....3-861
feGroupBoolean....3-861
feGroupBoolean2....3-863
feGroupCombine....3-864
feGroupGenerate....3-864
feGroupGenerate2....3-865
feGroupGenSolid....3-867
feGroupGenProp....3-868
feGroupGenMatl....3-868
feGroupGenElemType....3-869
feGroupGenElemShape....3-870
feGroupPeel....3-870
feGroupMoveToLayer....3-871
feGroupsContaining....3-872
feGroupGenVisible....3-873
feGroupFromCriteria....3-874
feGroupCondense....3-875
feGroupReduceToExisting....3-876
Standard Dialog Boxes....3-876
Coordinate Methods....3-876
Vector Methods....3-899
Plane Methods....3-917
Entity Selection Methods....3-927
Other Methods....3-942
User Interface Methods....3-954
Free Edge and Free Face Methods....3-954
Model Activation and Management....3-957
Window Management....3-966
View Management....3-970
Toolbar and Menu Customization....3-973
Embedding Applications....3-996
Locking Femap....3-1009
Status Bar....3-1015
Message Window....3-1017
Model Info Tree....3-1021
Real Number Text Formatting....3-1025
Library Management....3-1027
API Selection Tools....3-1030
Other Utilities....3-1033
User Graphics Methods....3-1043
Managing User Graphics Data....3-1043
Displaying User Graphics Data....3-1047
File Utility Methods....3-1049

Unpublished work. © 2025 Siemens

feFileGetName..... 3-1049
feFileExecute..... 3-1050
feFileProgramRun..... 3-1051
feFileProgramRunning..... 3-1052
feFileCurrentDirectory..... 3-1053
Set Utility Methods..... 3-1054
feSetFree..... 3-1054
feSetFreeNotInSet..... 3-1054
Data Management..... 3-1055
feLoginToTeamcenter..... 3-1055
feIsUserLoggedInToTeamcenter..... 3-1056
feOpenModelFromTeamcenter..... 3-1057
feSaveModelToTeamcenter..... 3-1058
feGetCurrentModelPDMInfo..... 3-1059
feGetTeamcenterDatasetInfo..... 3-1059

Femap Tool Objects..... 4-1
DataTable Object..... 4-5
DataTable Object Properties..... 4-5
Specifying Row and Column Indexes..... 4-6
DataTable Object Row Methods..... 4-7
DataTable Object Column Methods..... 4-10
DataTable Object Data Methods..... 4-21
DataTable Object Grouping Methods..... 4-39
DataTable Object Sorting Methods..... 4-43
DataTable Object Selection Methods..... 4-45
DataTable Object Other Methods..... 4-47
Element Quality Objects..... 4-52
Element Quality Object Properties..... 4-53
Element Quality Methods..... 4-57
Geometry Preparation and Meshing Objects..... 4-118
Geometry Preparation and Meshing Object Properties..... 4-119
Geometry Preparation and Meshing Object Methods..... 4-126
Geometry Interface Objects..... 4-135
Geometry Interface Object Properties..... 4-135
Geometry Interface Object Methods..... 4-143
ReadFile Objects..... 4-146
ReadFile Object Properties..... 4-146
ReadFile Object Methods..... 4-147
Selector Objects..... 4-172
Selector Object Properties..... 4-173
Selector Object Methods..... 4-173
Set Objects..... 4-187
Set Object Properties..... 4-188
Set Object Methods..... 4-189
Set Object SavedSet Methods..... 4-285
Sort Objects..... 4-299
Sort Object Properties..... 4-300

Unpublished work. © 2025 Siemens

Sort Object Methods....4-300
UserData Objects....4-328
UserData Object Properties....4-328
UserData Object Methods....4-329
Results Browsing Objects....4-359
Results Browsing Object Properties....4-360
Limitations and Usage of Methods....4-360
Output Vectors....4-361
Results Browsing General Object Methods....4-364
Results Browsing Study Methods....4-367
Results Browsing Set Methods....4-373
Results Browsing Vector Methods....4-383
Results Browsing Entity Value Methods....4-394
Results Browsing Bulk Data Review Methods....4-395
Results Browsing Update and Save Methods....4-443
MapData Objects....4-474
MapData Object Methods....4-474
Entity Tracking Objects....4-493
TrackData Object Methods....4-494
Beam Calculator Objects....4-501
Beam Calculator Object Properties....4-501
Beam Calculator Object Methods....4-502
Draw/Erase Objects....4-513
Draw/Erase Object Properties....4-513
Draw/Erase Object Methods....4-514
User Defined Graphics Objects....4-521
User Defined Graphics Object Methods....4-522
CopyTool Objects....4-609
CopyTool Object Properties....4-610
CopyTool Object Options Methods....4-611
CopyTool Object Operation Methods....4-620
MoveTool Objects....4-631
MoveTool Object Properties....4-631
MoveTool Object Options Methods....4-632
MoveTool Object Operation Methods....4-634
MergeTool Objects....4-643
MergeTool Object Properties....4-644
MergeTool Object Model Selection Methods....4-645
MergeTool Object Entity Selection and Status Methods....4-648
MergeTool Object Operation Methods....4-666
Results Vector ID Query Objects....4-677
Results Vector ID Query Constants (Enumerations)....4-677
Results Vector ID Query Object Methods....4-695
Stress Linearization (StressLinear) Objects....4-720
StressLinear Object Properties....4-720
StressLinear Object Methods....4-721
PublishTool Objects....4-724
PublishTool Object Methods....4-724

Unpublished work. © 2025 Siemens

PublishTable Objects **4-738**
PublishTable Object Methods 4-739
BodyMesher Objects **4-754**
BodyMesher Object Properties 4-754
BodyMesher Object Option Methods 4-758
BodyMesher Object Operation Methods 4-765
SolidCleanupTool Objects **4-768**
Solid Cleanup Tool Object Properties 4-768
Solid Cleanup Tool Object Methods 4-770

Femap Entity Objects **5-1**
Common Entity Properties and Methods **5-7**
Common Entity Properties 5-7
Common Entity Methods 5-8
Aero Panel/Body Objects **5-31**
Aero Panel/Body Object Properties 5-31
Aero Panel/Body Object Methods 5-33
Aero Property Objects **5-40**
Aero Property Object Properties 5-40
Aero Property Object Methods 5-41
Aero Spline Objects **5-50**
Aero Spline Object Properties 5-50
Aero Spline Object Methods 5-51
Aero Surface Objects **5-53**
Aero Surface Object Properties 5-53
Aero Surface Object Methods 5-55
AnalysisCase Objects **5-59**
AnalysisCase Object Properties 5-59
AnalysisCase Object Methods 5-99
AnalysisCase Object - ANSYS Specific Methods 5-112
AnalysisMgr Objects **5-126**
AnalysisMgr Object Properties 5-126
AnalysisMgr Object Methods 5-213
AnalysisMgr Objects - ANSYS Specific Methods 5-255
AnalysisMgr Objects - LS-Dyna Specific Methods 5-275
AnalysisStep Objects **5-281**
AnalysisStep Object Properties 5-281
AnalysisStep Object Methods 5-290
AnalysisStudy Objects **5-293**
AnalysisStudy Object Properties 5-293
AnalysisStudy Object Methods 5-294
BCDefinition Objects **5-299**
BCDefinition Object Properties 5-299
BCDefinition Object Methods 5-300
BCEqn Objects **5-303**
BCEqn Object Properties 5-303
BCEqn Object Methods 5-304

Unpublished work. © 2025 Siemens

BCGeom Objects **5-308**
BCGeom Object Properties 5-308
BCGeom Object Methods 5-310
BCNode Objects **5-319**
BCNode Object Properties 5-319
BCNode Object Methods 5-320
BCSet Objects **5-327**
BCSet Object Properties 5-328
BCSet Object Methods 5-328
Chart Objects **5-337**
Chart Object Properties 5-337
Chart Object Methods 5-345
Chart Data Series Objects **5-348**
Chart Data Series Object Properties 5-348
Chart Data Series Object Methods 5-354
Computed Vector Objects **5-357**
Computed Vector Object Properties 5-357
Computed Vector Object Methods 5-359
Connection Property Objects **5-370**
Connection Property Object Properties 5-370
Accessing Connection Property Properties 5-371
Connection Property Object Methods 5-381
Connection Region Objects **5-382**
Connection Region Object Properties 5-383
Connection Region Object Methods 5-387
Connection Objects **5-409**
Connection Object Properties 5-409
Connection Object Methods 5-409
CSys Objects **5-417**
CSys Object Properties 5-417
CSys Object Methods 5-418
Curve Objects **5-422**
Curve Object Properties 5-422
Curve Object Methods 5-426
Data Surface Objects **5-485**
Data Surface Object Properties 5-485
Data Surface Methods 5-486
Design Equation Objects **5-521**
FlexibleSlider Object Properties 5-521
Direct Matrix Input Objects **5-523**
Direct Matrix Input Object Properties 5-523
Direct Matrix Input Object Methods 5-524
Discrete Value Set Object **5-533**
Discrete Value Set Object Properties 5-534
Discrete Value Set Object Methods 5-534
Element Add | Remove Objects **5-546**
Element Add | Remove Properties 5-547
Element Add | Remove Object Methods 5-548

Unpublished work. © 2025 Siemens

Element Objects **5-558**
Element Object Properties 5-558
Element Object Methods 5-563
Flexible Slider Objects **5-609**
Flexible Slider Object Properties 5-610
Flexible Slider Object Methods 5-612
Freebody Objects **5-614**
Freebody Object Properties 5-614
Freebody Object Methods 5-619
Frequency Objects **5-632**
Frequency Object Properties 5-632
Frequency Object Methods 5-633
Function Objects **5-663**
Function Object Properties 5-663
Function Object Methods 5-664
GFXArrow Objects **5-667**
GFXArrow Object Properties 5-668
GFXArrow Object Methods 5-669
GFXLine Objects **5-673**
GFXLine Object Properties 5-674
GFXLine Object Methods 5-675
GFXPoint Objects **5-678**
GFXPoint Object Properties 5-679
GFXPoint Object Methods 5-679
GFXQuad4 Objects **5-683**
GFXQuad4 Object Properties 5-684
GFXQuad4 Object Methods 5-684
GFXTria3 Objects **5-690**
GFXTria3 Object Properties 5-690
GFXTria3 Object Methods 5-691
GlobalPly Objects **5-696**
GlobalPly Object Properties 5-696
GlobalPly Object Methods 5-697
GlobalStep Objects **5-697**
GlobalStep Object Properties 5-697
GlobalStep Object Methods 5-705
Group Objects **5-708**
Group Object Properties 5-708
Group Object Methods 5-716
Kinematic Joint and Joint Connection Objects **5-739**
Kinematic Joint and Joint Connection Object Properties 5-739
Joint Object Methods 5-746
Layer Objects **5-750**
Layer Object Properties 5-750
Layer Object Methods 5-750
Layup Objects **5-761**
Layup Object Properties 5-761
Layup Object Methods 5-764

Unpublished work. © 2025 Siemens

LoadBolt Object..... 5-789
LoadBolt Object Properties.....5-789
LoadBolt Object Methods.....5-789
LoadDefinition Objects..... 5-794
LoadDefinition Object Properties.....5-794
LoadDefinition Object Methods.....5-795
LoadETemp Objects..... 5-799
LoadETemp Object Properties.....5-800
LoadETemp Object Methods.....5-800
LoadGeom Objects..... 5-805
LoadGeom Object Properties.....5-805
LoadGeom Object Methods.....5-811
LoadGeom Load Types.....5-813
LoadMesh Objects..... 5-816
LoadMesh Object Properties..... 5-816
LoadMesh Object Methods..... 5-820
LoadMesh Load Types.....5-831
LoadNTemp Objects.....5-832
LoadNTemp Object Properties.....5-832
LoadNTemp Object Methods.....5-833
LoadSet Objects.....5-838
LoadSet Object Properties.....5-838
LoadSet Object Methods.....5-845
MapOutput Objects..... 5-855
MapOutput Object Properties.....5-856
MapOutput Object Methods.....5-858
Material Objects..... 5-870
Material Object Properties.....5-870
Material Object Methods.....5-881
Mesh Point Objects.....5-893
MeshHardPoint Object Properties.....5-893
MeshHardPoint Object Methods.....5-894
Mesh Point Definition Objects.....5-903
MeshHardPointDefinition Object Properties.....5-904
MeshHardPointDefinition Object Methods.....5-904
Monitor Point Objects.....5-906
Monitor Point Object Properties..... 5-906
Monitor Point Object Methods..... 5-909
Node Objects..... 5-921
Node Object Properties.....5-921
Node Object Methods.....5-922
Optim Objects.....5-934
Optim Object Properties.....5-934
Optim Object Methods.....5-936
Optimization Manufacturing Constraints Objects.....5-936
Optimization Manufacturing Constraint Object Properties..... 5-936
Optimization Manufacturing Constraints Object Methods.....5-938
Optimization Relationship (Variable) Objects..... 5-950

Unpublished work. © 2025 Siemens

Optimization Relationship Object Properties....5-950
Optimization Relationship Object Methods....5-951
Optimization Response (Limits) Objects....5-959
Optimization Response (Limits) Object Properties....5-960
Optimization Response (Limits) Object Methods....5-961
Output Objects (OBSOLETE)....5-965
Output Object Properties (OBSOLETE)....5-965
Output Object Methods (OBSOLETE)....5-968
OutputSet Objects....5-995
OutputSet Object Properties....5-996
OutputSet Object Methods....5-997
Plane Objects....5-1002
Plane Object Properties....5-1002
Plane Object Methods....5-1003
Ply Material Objects....5-1004
Ply Material Object Properties....5-1004
Point Objects....5-1005
Point Object Properties....5-1005
Point Object Methods....5-1006
Property Objects....5-1017
Property Object Properties....5-1017
Accessing Property Properties....5-1021
Property Object Methods....5-1050
Reference Objects....5-1077
Reference Object Properties....5-1077
Reference Object Methods....5-1078
Report Objects....5-1079
Report Object Properties....5-1079
Report Object Methods....5-1080
Superelement Reference Objects....5-1080
Superelement Reference Object Properties....5-1080
Solid Objects....5-1081
Solid Object Properties....5-1081
Solid Object Methods....5-1083
Surface Objects....5-1107
Surface Object Properties....5-1107
Surface Object Methods....5-1111
Table Data Objects....5-1179
Table Data Object Properties....5-1180
Table Data Object Methods....5-1181
Text Objects....5-1205
Text Object Properties....5-1205
Text Object Methods....5-1206
TMGBC Objects....5-1208
TMGBC Object Properties....5-1208
TMGBC Object Methods....5-1209
TMGCtrl Objects....5-1212
TMGCtrl Object Properties....5-1213

Unpublished work. © 2025 Siemens

TMGCtrl Object Methods....5-1213
TMGInt Objects....5-1217
TMGInt Object Properties....5-1217
TMGInt Object Methods....5-1218
TMGOpt Objects....5-1219
TMGOpt Object Properties....5-1220
TMGOpt Object Methods....5-1220
TMGReal Objects....5-1222
TMGReal Object Properties....5-1222
TMGReal Object Methods....5-1223
Variable Objects....5-1224
Variable Object Properties....5-1225
Variable Object Methods....5-1225
Vector Objects....5-1228
Vector Object Properties....5-1228
Vector Object Methods....5-1229
View Objects....5-1230
View Object Properties....5-1230
View Object Methods....5-1282
ViewOrient Objects....5-1303
ViewOrient Object Properties....5-1304
ViewOrient Object Methods....5-1305

Femap Events....6-1
Receiving and Processing Events....6-1
Alternate Method - Event Callbacks....6-1
Format of Event Messages....6-2
The Femap Events....6-2
Using Femap Events....6-5

Proprietary and Restricted Rights Notice....7-1

Unpublished work. © 2025 Siemens

1. Femap API Guide

Welcome to the Femap API Guide. Femap provides a robust set of finite element modeling and post-processing functionality. At times, however, you may need a specific capability that is not included in the standard product. The Femap Application Programming Interface (API) lets you customize Femap to meet your specific needs. The following chapters will guide you through the Femap API interface and its hundreds of functions.

Introduction to the Femap API

The Femap Application Programming Interface (API) lets you customize Femap to meet your specific needs or add capabilities not included in the standard product.

The Femap API is an OLE/COM-based programming interface to Femap. It contains hundreds of functions that can be called from Visual Basic, VBA (Excel, Word, Access, ...), C, or C++. The Femap API Reference documents the objects and functions that are available in the Femap API.

To use the Femap API successfully, you need to understand:

- Objects, Methods and Properties
- Data Types
- Memory Allocation
- Global Constants
- Other Considerations

> **Note**
> Undocumented API calls are subject to change without notice.

For examples of how to use the Femap API, see **Using the Femap API**.

Objects, Methods and Properties

The Femap API is an object-oriented system. If you are familiar with object-oriented programming, you should have no problem understanding the concepts discussed. If you are not, this section briefly introduces those concepts. You should refer to other sources for more complete information.

Throughout the Femap API Reference, you will see references to objects, methods, and properties. Access to functionality and data is through the *objects* that are defined by the API. There are only two levels of objects in the current API: the application object, and other objects.

Unpublished work. © 2025 Siemens

The Application Object

The first level includes a single object: the Femap application object. In every application that you write, you will always define and reference a Femap application object, which forms the connection between your application and Femap. For more information, see **The Femap Application Object**.

The application object contains properties (data) and methods (functionality) that are global in nature. For example, all of the global data for your Femap session (defined under **File → Preferences**) can be accessed as properties of the application object (see **Femap Object Properties**). Almost all of the same functionality in the Femap menus is available as methods in the application object. These are described in the other sections under **The Femap Application Object**, and are organized according to their locations in the menu.

Other Objects

In addition to the main Femap application object, the API also defines many other objects that provide detailed access to the entities in your models. Since most of the model entities represent the underlying modeling data, these objects are very rich in property definitions. In addition, the API includes methods that are used to find, retrieve, traverse and store the entities. For a full listing of objects, see **Femap Entity Objects**.

As an important starting point to using any of the entity objects, refer to **Common Entity Properties and Methods**. It describes the properties and methods that are common to all of the objects. This information is not repeated in each of the individual objects. You should understand these options before proceeding to the detailed description of the unique capabilities of each object.

Unpublished work. © 2025 Siemens

Data Types

To use the API, you must understand the types of data that it requires and returns. Because of differences in the data type definitions depending on whether you will call the API from Basic, C++, or some other source, all of the documentation is presented in a *language-neutral*, more descriptive form. The actual data types that you use will change from language to language, however, the underlying format of those data types must match what the API requires. The following table summarizes what you will find in the manual:

API Definition in the Manual	Description	From Basic		From C++
		Visual Basic 6	Visual Basic .NET	
`BOOL`	Single byte, True/ False value	Boolean	Boolean	Unsigned Char
`INT2`	2-byte integer	Integer	Integer	short
`INT4`	4-byte integer	Long	Integer	long, int
`INT8`	8-byte integer	n/a	Int64	longlong, int64
`REAL4`	4-byte real	Single	Single	float
`REAL8`	8-byte real	Double	Double	double
`STRING`	character string, null terminated	String	String	char[..]

Unpublished work. © 2025 Siemens

Using Visual Basic .NET

The INT4 data type, which is widely used throughout the API, must be declared differently if you are using Visual Basic 6 or Visual Basic .NET. In Visual Basic 6, the type *Long* maps to the 4-byte integers used by Femap. In Visual Basic .NET, the *Integer* type must be used to map to the same 4-byte integers, as the *Long* type has changed to map to 8-byte integers which are incompatible with the Femap API.

One other difference with Visual Basic .NET is that you must always initialize String variables before they can be passed. Even for methods, like `feNotesGet` in the Femap object, which return a string to you, the variable must be initialized prior to the call. For example, you can not use the following:

```
Dim s as String
rc = femap.feNotesGet( 1, s )
```

This will generate a type mismatch error, because the String variable has not been initialized. If instead you use:

```
Sub Main             'Full Subroutine usable in FEMAP API window

    Dim App As femap.model      ' Declare "App" model object
   Set App = GetObject(,"femap.model") ' Instantiate "App" object
  Dim s As String
  s = ""

   rc = App.feNotesGet( 1, s ) ' Puts model notes into String "s"

   MsgBox(s)  ' Shows String "s" in modal dialog box

End Sub
```

then, everything will work properly.

One final difference is that the *Set* command is no longer used in Visual Basic .NET. Throughout this document, you will see examples that still include *Set*, as is required for the older Visual Basic 6. For example,

```
Dim femap as Object
Set femap = GetObject(, "femap.model")
```

If you are using Visual Basic .NET, you must simply ignore (or remove) the *Set*, leaving the correct syntax:

```
Dim femap as Object
femap = GetObject(, "femap.model")
```


Unpublished work. © 2025 Siemens

Arrays and Variants

In addition to the basic data types, you will often see references to arrays of values, for instance:

```
INT4 x[0..9]
```

This implies that an array of 10, 4-byte integers is required. Through the API (COM) interface, however, all arrays must be passed as Variant data types. In many current languages (for example, Visual Basic .NET, Visual Basic for Applications in Microsoft Office 2007, and WinWrap Basic embedded in Femap) the details of this can be hidden from you, and you can write code that directly passes array variables.

```
Dim x(10) as Long
x(0) = 5
x(1) = 4
...
```

and then x can be passed directly in calls that require an array.

In older languages like Visual Basic 6, where you need to manually declare the Variants, you can create it fairly easily. For instance:

```
Dim x(10) as Long
Dim vx as Variant
x(0) = 5
x(1) = 4
...
vx = x
```

This will define a variant, vx, that contains all of the values assigned to the x array. This variant can then be passed through the API to Femap in place of the array.

For any method that returns an array, you do not need to define an array variable in Basic. You simply define the variant, pass it to the API, and then reference it like an array afterward. This applies to both current and older languages. For instance,

```
Sub Main
 Dim App As femap.model
 Set App = feFemap()


  Dim rc As Long
 Dim a(3) As Double
 Dim va As Variant


   ' Create and store some userdata
```


Unpublished work. © 2025 Siemens

```
  Dim e As UserData
 Set e = App.feUserData
 a(0) = 1.290
 a(1) = 2*a(0)
 a(2) = 2*a(1)

  va = a                              ' Variant to Array
  rc = e.WriteDoubleArray(3, va)  ' Write Variant as Array
 rc = e.Put(1)


   ' Retrieve userdata
 Dim vb As Variant
 Dim i As Long
 Dim f As UserData
 Set f = App.feUserData

 rc = f.Get (1)
  rc = f.ReadDoubleArray ( i, vb ) ' Read Array from DB

                                    'into Variant

  Dim mstr As String  'Convert Doubles in Variant to Strings
 mstr =Str$(vb(0)) + " " + Str$(vb(1))+" "+Str$(vb(2))
 MsgBox(mstr)

End Sub
```


Unpublished work. © 2025 Siemens

An Array Shortcut

In current languages where you can pass arrays directly without manually declaring a Variant, there is a non-standard programming shortcut that you can use to simplify some of your coding. If there is an array that is used as an input parameter to a function, and you want to specify all entries in the array to have the same value, rather than declaring an array at all, you can simply pass a single value - either as a constant or a single variable. Femap will automatically treat this single value if it were an array of the proper size with each entry in the array equal to the value you passed. Of course, if you need to specify different values for the array entries, then the normal method must be used. For instance, the node object has a method called `PutAllArray()` that requires multiple input arrays, so you could use....

```
Sub Main
 Dim App As femap.model
 Set App = feFemap()


Dim rc As Long
Dim nd as femap.node
Set nd = App.feNode
Dim numNode as Long
Dim ndID(10) as Long
Dim xyz(30) as Long
Dim layer as Long
layer = 1
...
rc = nd.PutAllArray( numNode, ndID, xyz, layer, 43, 0, 0, 0, 0 )


End Sub
```

Note

All the arguments including and after `layer` actually require arrays, but are being passed as single values because we are assigning the same constant value to all of the nodes being defined in the call to this method.

Arrays and Visual Basic .NET

If you were using Visual Basic 6, then as stated, all array variables are passed via the Variant type. With the release of Visual Basic .NET, the Variant data type is no longer available in Basic. To use Visual Basic .NET, simply declare all of the variables that you would have declared as *Variant* in Visual Basic 6, as the *Object* data type. With this change, the remainder of your code will work as it did previously.

Unpublished work. © 2025 Siemens

Arrays and Visual C++

If you are callinig the API from Visual C++, probably the easiest approach to accessing arrays is to use the ATL capability to create Safe Arrays and Variants. To create Safe Arrays, use:

```
CComSafeArray<data_type> ldof(size);
```

Where *data_type* is the type of data in the array, and size is the number of entries. You should use the following *data_type*s to be compatible with the API:

API Definition in the Manual	Safe Array Creation
BOOL	CComSafeArray<BOOL> ...
INT4	CComSafeArray<LONG> ...
REAL8	CComSafeArray<DOUBLE> ...

To create Variants, you can use:

```
CComVariant vdir;
```

You can fill the Variant by simply setting it equal to the Safe Array, and then pass it to the API. Refer to **Calling the API from Visual C++**, for an example of using this approach.

Using Multi-dimensional Arrays

Even though this documents shows arrays as being dimensioned as

```
REAL8 xyz[0..11]
```

It may be more convenient is some cases to think of them in terms of multi-dimensional arrays. For example, the above array may really contain 4 sets of coordinates (3 values for each coordinate - x, y and z), in which case referencing it as

```
REAL8 xyz[4][3]
```

may simplify your coding. With some care and understanding this is possible. There are two basic issues that need to be properly handled to use multi-dimensional arrays

Unpublished work. © 2025 Siemens

Array Storage Order

Depending on the language that you are using to call the API, multi-dimensional arrays store data in different order. Femap requires data stored in these arrays to be in a specific order that is described with the individual property or method - it is up to you to properly arrange the array indices so that data is stored as needed - and this varies by language. For example, in the above array Femap expected to have the data arranged as follows (X1,Y1,Z1, X2,Y2,Z2, X3,Y3,Z3, X4,Y4,Z4). If you were calling the API from C or C++, the rightmost index varies first, so declaring the array as:

```
double xyz[4][3] (in C or C++)
```

would be appropriate. On the other hand, if you were using Basic, then the leftmost index varies first and you would have to declare the array as:

```
Dim xyz(2,3) as double (in Basic)
```

to get the data into the proper locations. Also notice in Basic, that the dimensions are the values of the largest index, not the size to allocate, so the numbers decreased by 1 each.

Proper Dimensioning

To get all of the data in the proper locations, you can not *over-size* a multi-dimensional array - except in the final dimension. In general, you will want to make your arrays just big enough to hold the data that you are using, and no larger. In some cases however, it is convenient to have one or more large arrays that simply collect data and may not be completely filled. When using multi-dimensional arrays however, care must be taken. In the above case, from C, when you declared the array as

```
double xyz[4][3] (in C or C++)
```

everything was fine and data is stored as (X1,Y1,Z1, X2,Y2,Z2, X3,Y3,Z3, X4,Y4,Z4). If however you declared the array as

```
double xyz[4][4] (in C or C++)
```

then data would be stored as (X1,Y1,Z1,HOLE1, X2,Y2,Z2,HOLE2, X3,Y3,Z3,HOLE3 X4,Y4,Z4,HOLE4), and you can see that the “holes” will move the data to the wrong locations. Similarly, if you over-dimension the first index in Basic, the same thing will happen.

You can successfully over-dimension the final index however. In C, for example,

```
double xyz[100][3] (in C or C++)
```


Unpublished work. © 2025 Siemens

would be fine, since data would be stored as (X1,Y1,Z1, X2,Y2,Z2, X3,Y3,Z3, X4,Y4,Z4, ... X100, Y100, Z100), and even if only 4 sets of X,Y,Z values were needed, they will still be in their proper locations.

Multiple Definitions of Array Properties

As you look through the documentation for the various entity types, you will see definitions of properties that look like the following:

Property	Description
BOOL dof[0..5] or VARIANT vdof	An array of flags telling whether or not the corresponding degrees of freedom are constrained (True) or free (False) in this set.

Both of these properties provide access to the same data. The first (in this case *dof*) lets you access individual array entries, one at a time (for instance, *dof[2]*). The second Variant property lets you access the entire array in one OLE/COM call. To access these Variant properties, you must create a *Variant* variable in your application, load the entire array into it (by setting it equal to the property), manipulate your local variable as needed, then put the entire array back into the Femap object (by setting it equal to your local Variant). Once again, in Visual Basic .NET, use the generic *Object* data type instead of *Variant* which is no longer supported.

Memory Allocation

Like any OLE/COM implementation, when using the Femap API, it is the responsibility of the calling program (the one you write) to free all memory when it is no longer needed. This means if you call an API method that returns a variant (a string or array), you must free the memory used by that variable when you no longer need it.

If you are using Basic, as soon as a variable goes out of scope (your program finishes, or the procedure where the variable was defined finishes), it is automatically freed. If you are using another language, like C or C++, you need to explicitly free the memory.

It is not just variables that are returned from the API that you need to free. If you allocate memory for a variable and pass it to an API method, the memory will not be freed (you might still need it). It is your responsibility to free all memory for your variables.

Unpublished work. © 2025 Siemens

Global Constants

Throughout this manual are references to certain constant values: for example, error return values, entity type values, group definition types, events, and other options. Please see the following sections for more information:

- **Entity Types**
- **Femap Constants**
- **Group Selection Rules**
- **Group Range Types**
- **The Femap Events**

Other Considerations

When using API calls, methods, properties, etc with most programming languages, the capitalization of the item found in the *Type Library* MUST be used, regardless of how the item is documented. The exception to this rule is when using basic languages, such as Visual Basic or WinWrap (language used in the API Programming dockable pane available within the Femap user interface), which often have the ability to auto-correct capitalization.

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

2. Using the Femap API

This section of the Femap API guide includes some fairly simple examples that can be implemented using the Femap API. These examples show some of the basic setup and syntax of using the API, but do not attempt to show the breadth of the API's capabilities.

Using Visual Basic .NET

These examples were created to use Visual Basic 6. As such they use the *Long* and *Variant* data types that are required in that version of Visual Basic. If you want to use them with Visual Basic .NET, the *Long* data types must be changed to *Integer* and the *Variant* data types to *Object*.

Unpublished work. © 2025 Siemens

Loading Data into Excel

This program shows a simple example of connecting to Femap from Excel, and loading all of the nodal data into the worksheet. Although this program loads nodal data, the approach is the same for other data types. To load other data types, you would simply dimension and access other Femap objects and reference their properties as you are storing the data.

```
_____________________________________________________________________________
__
Private Sub LoadNodalData()

'1.      Attach to the model in a FEMAP session that is already running.
(The comma before femap.model is used externally for a FEMAP model already
open.)

Dim femap As Object

Set femap = GetObject(, "femap.model")

'2.     Create a Node object.

Dim nd As Object

Set nd = femap.feNode

'3.     Make the titles in the first row of the worksheet.

   Row = 1

   Worksheets(1).Cells(Row, 1).Value = "ID"

   Worksheets(1).Cells(Row, 2).Value = "Layer"

   Worksheets(1).Cells(Row, 3).Value = "Color"

   Worksheets(1).Cells(Row, 4).Value = "Def CSys"

   Worksheets(1).Cells(Row, 5).Value = "Out CSys"

   Worksheets(1).Cells(Row, 6).Value = "X"

   Worksheets(1).Cells(Row, 7).Value = "Y"

   Worksheets(1).Cells(Row, 8).Value = "Z"

'4.     Loop over all of the nodes in the model.

While nd.Next

   Row = Row + 1

'5.     Store properties for each node in successive worksheet rows.
```


Unpublished work. © 2025 Siemens

```
   Worksheets(1).Cells(Row, 1).Value = nd.ID
   Worksheets(1).Cells(Row, 2).Value = nd.layer
   Worksheets(1).Cells(Row, 3).Value = nd.Color
   Worksheets(1).Cells(Row, 4).Value = nd.defCSys
   Worksheets(1).Cells(Row, 5).Value = nd.outCSys
   Worksheets(1).Cells(Row, 6).Value = nd.x
   Worksheets(1).Cells(Row, 7).Value = nd.y
   Worksheets(1).Cells(Row, 8).Value = nd.z
Wend
End Sub
_____________________________________________________________________________
__
```

While the previous program is fairly simple, it does require calling Femap for each of the nodal quantities. For example, each time you reference nd.x, nd.y, or nd.z, your program must call Femap to retrieve that information. This approach provides the simplicity of the object-oriented approach, but does not necessarily perform as well as making just one call for multiple pieces of data. The following program does exactly the same thing as the previous one, but makes far fewer calls, and therefore performs better.

```
_____________________________________________________________________________
__
Private Sub LoadNodalDataFast()
'1.      Attach to the model in a FEMAP session that is already running.
Dim femap As Object
Set femap = GetObject(, "femap.model")
'2.     Create a Node object.
Dim nd As Object
Set nd = femap.feNode
'3.     Dimension local variables to receive the data.
Dim x As Double
Dim y As Double
Dim z As Double
```


Unpublished work. © 2025 Siemens

```
Dim l As Long

Dim co As Long

Dim d As Long

Dim oc As Long

Dim e As Long

Dim rc As Long

Dim ID As Long

Dim pb As Variant

'4.     Make the titles in the first row of the worksheet.

Row = 1

   Worksheets(1).Cells(Row, 1).Value = "ID"

   Worksheets(1).Cells(Row, 2).Value = "Layer"

   Worksheets(1).Cells(Row, 3).Value = "Color"

   Worksheets(1).Cells(Row, 4).Value = "Def CSys"

   Worksheets(1).Cells(Row, 5).Value = "Out CSys"

   Worksheets(1).Cells(Row, 6).Value = "X"

   Worksheets(1).Cells(Row, 7).Value = "Y"

   Worksheets(1).Cells(Row, 8).Value = "Z"

'5.     Loop over all of the nodes in the model.

ID = nd.nextID

While ID < 2147000000

   Row = Row + 1

'6.     Call the GetAll method of the Node object to retrieve all of the
required properties in one call. Store the retrieved data into the
worksheet.

   rc = nd.GetAll(ID, x, y, z, l, co, e, d, oc, pb)

   Worksheets(1).Cells(Row, 1).Value = ID

   Worksheets(1).Cells(Row, 2).Value = l

   Worksheets(1).Cells(Row, 3).Value = co
```


Unpublished work. © 2025 Siemens

```
   Worksheets(1).Cells(Row, 4).Value = d

   Worksheets(1).Cells(Row, 5).Value = oc

   Worksheets(1).Cells(Row, 6).Value = x

   Worksheets(1).Cells(Row, 7).Value = y

   Worksheets(1).Cells(Row, 8).Value = z

'(These fields were not loaded by the first example, but show the use of
the Variant data type to access array data.)

   Worksheets(1).Cells(Row, 9).Value = pb(0)

   Worksheets(1).Cells(Row, 10).Value = pb(1)

   Worksheets(1).Cells(Row, 11).Value = pb(2)

   Worksheets(1).Cells(Row, 12).Value = pb(3)

   Worksheets(1).Cells(Row, 13).Value = pb(4)

   Worksheets(1).Cells(Row, 14).Value = pb(5)

   ID = nd.nextID

Wend

End Sub
_____________________________________________________________________________
__
```


Unpublished work. © 2025 Siemens

Loading Data from Excel

This program reverses the programs shown in the previous example. It takes the same nodal data that was stored in the worksheet, and updates the nodes in Femap to have the worksheet values. Used together, these two examples allow you to use Excel as an editor. Running the first example loads data into your worksheet, where you can edit it using the tools available in Excel. When your edits are complete, this program puts the changes back into Femap.

```
_____________________________________________________________________________
__


Private Sub UpdateNodalData()
'1.      Attach to the model in a FEMAP session that is already running.
Dim femap As Object
Set femap = GetObject(, "femap.model")
'2.     Create a Node object.
Dim nd As Object
Set nd = femap.feNode
'3.     Loop over all rows of the worksheet that have an ID value.
Row = 2
ID = Worksheets(1).Cells(Row, 1).Value
While ID > 0
'4.     Retrieve the node to load all of its properties.
    rc = nd.Get(ID)
'5.     Update the properties that you have edited.
   nd.layer = Worksheets(1).Cells(Row, 2).Value
   nd.Color = Worksheets(1).Cells(Row, 3).Value
   nd.defCSys = Worksheets(1).Cells(Row, 4).Value
   nd.outCSys = Worksheets(1).Cells(Row, 5).Value
   nd.x = Worksheets(1).Cells(Row, 6).Value
   nd.y = Worksheets(1).Cells(Row, 7).Value
```


Unpublished work. © 2025 Siemens

```
   nd.z = Worksheets(1).Cells(Row, 8).Value
'6.     Put the changes back into the database.
   rc = nd.Put(ID)
'7.     Go to the next row.
   Row = Row + 1
   ID = Worksheets(1).Cells(Row, 1).Value

Wend
End Sub
```

Just as before, you can also do this function with a single call approach to reduce the number of times that you reference the Node object.

```
Private Sub UpdateNodalDataFast()
'1.      Attach to the model in a FEMAP session that is already running.
Dim femap As Object
Set femap = GetObject(, "femap.model")
'2.     Create a Node object.
Dim nd As Object
Set nd = femap.feNode
'3.     Dimension local variables to receive the data.
Dim x As Double
Dim y As Double
Dim z As Double
Dim l As Long
Dim co As Long
Dim d As Long
```


Unpublished work. © 2025 Siemens

```
Dim oc As Long

Dim e As Long

Dim rc As Long

Dim ID As Long

'4.     Arrays are passed as variants. Dimension both, assign the array to
the variant, and pass the variant.

Dim pb As Variant

Dim p(6) As Long

pb = p


Row = 2

ID = Worksheets(1).Cells(Row, 1).Value

While ID > 0

'5.     Load the local variables with worksheet values.

   l = Worksheets(1).Cells(Row, 2).Value

   co = Worksheets(1).Cells(Row, 3).Value

   d = Worksheets(1).Cells(Row, 4).Value

   oc = Worksheets(1).Cells(Row, 5).Value

   x = Worksheets(1).Cells(Row, 6).Value

   y = Worksheets(1).Cells(Row, 7).Value

   z = Worksheets(1).Cells(Row, 8).Value

   pb(0) = Worksheets(1).Cells(Row, 9).Value

   pb(1) = Worksheets(1).Cells(Row, 10).Value

   pb(2) = Worksheets(1).Cells(Row, 11).Value

   pb(3) = Worksheets(1).Cells(Row, 12).Value

   pb(4) = Worksheets(1).Cells(Row, 13).Value

   pb(5) = Worksheets(1).Cells(Row, 14).Value

'6.     Put all data back into FEMAP with one call.
```


Unpublished work. © 2025 Siemens

```
   rc = nd.PutAll(ID, x, y, z, l, co, e, d, oc, pb)
'7.     Go to the next row.
   Row = Row + 1
   ID = Worksheets(1).Cells(Row, 1).Value
Wend

End Sub
_____________________________________________________________________________
__
```


Unpublished work. © 2025 Siemens

Working with Output in Femap v2020.1 and above

The static ranges of Output Vector IDs in previous versions of Femap cannot accommodate the large numbers of new output quantities and element types that are found in today's solutions. To accommodate the ongoing addition of output quantities, some of the existing output vectors have been moved to new ID ranges.

All API object methods which required a vector ID as an argument in version 2019.1 and below have been deprecated. Identical ...*V2* methods have been added to work with the *version 2020.1 and above vector IDs* (**New Vector IDs**). Because of this, the deprecated properties and methods should no longer be used, however, these properties and methods will still work, they simply require the use of *version 2019.1 and below vector IDs* (**Old Vector IDs**). Internally, Femap will convert any old vector ID to the appropriate new vector ID and execute the API call in the same manner it would have been executed in Femap 2019.1 and below.

For example, the `.AddColumn` method on the Results Browsing Object is now `.AddColumnV2`. It is used to add a column containing data from a given output vector. To fill a column with *Plate Bottom von Mises Stress* data, the original method used the vector ID of 7433 as the second argument.

```
rc = RBO.AddColum(nSetID, 7433, False, nCol, cIndex)
```

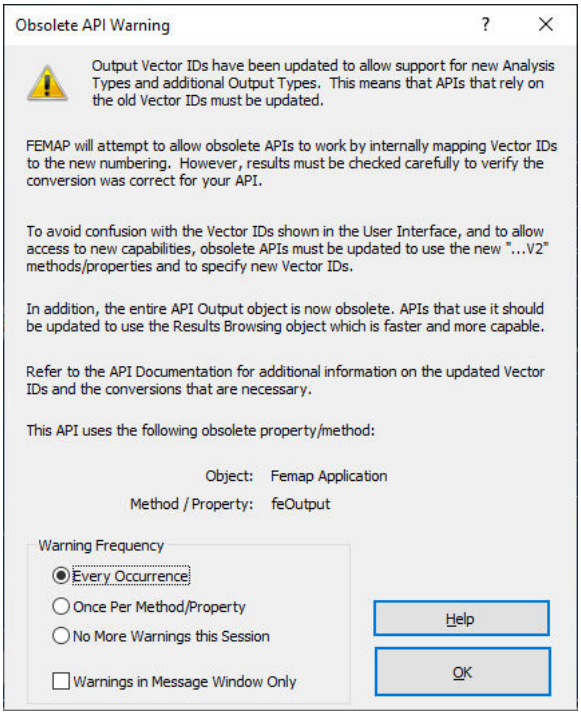
In Femap 2020.1 and beyond, the ID in the user interface for *Plate Bottom von Mises Stress* is 9033. This is because the *offset* of plate results has changed from 200 to 1000.

For example, in Femap 2019.1 and previous, the vector ID for *Plate Top von Mises Stress* was 7033. Adding an offset of 200 places *Plate Mid von Mises Stress* at 7233 and *Plate Bottom von Mises Stress* at 7433. For Femap 2020.1 and above, the offset is now 1000, which places *Plate Mid von Mises Stress* at 8033 and *Plate Bottom von Mises Stress* at 9033. Therefore, Using the `.AddColumn` call from above in Femap 2020.1 and above will internally convert 7433 to the new ID of 9033 and retrieve the results of *Plate Bottom von Mises Stress*.

Note

If the `.AddColumn` method is used, an Obsolete API Warning message will appear. The user is advised to update scripts to the new ...*V2* properties and methods at the earliest convenience to avoid confusion of output vector IDs shown in the user interface going forward.

Unpublished work. © 2025 Siemens



The **Warning Frequency** section can be used to control how often Femap warns the user about using deprecated API calls and/or how the user will be warned for a given Femap session:

Every Occurrence
This is the default setting and issues a warning message every time a deprecated call, method, and/or property involving output vector IDs or the Output Object (`feOutput`) is encountered in an API script or program.

Once Per Method/Property
When selected, a warning will be issued a single time for each deprecated for each call, method, or property involving output vector IDs or the Output Object (`feOutput`) is encountered in an API script or program.

No More Warnings this Session
When selected, no more warning messages will be issued while the current Femap session is open for any deprecated call, method, and/or property involving output vector IDs or the Output Object (`feOutput`) is encountered in an API script or program.

Warnings in Message Window Only
When activated, the **Warnings in Message Window Only** option will no longer display the Obsolete API Warning dialog box and only list these warning messages to the **Messages** window until the Femap session is closed. Once Femap is closed, all settings will return to the default settings and the next time Femap is opened, the user can select their options again.

There are also two API calls, `Pref_API_WarningLevel` and `Pref_API_WarningInDialog`, which can be used to control the warning messages.

To see a full list of the remapped vector IDs, see **Output Vector IDs**.

Unpublished work. © 2025 Siemens

When updating to the ...*V2* methods, the new vector IDs must be used. There is no conversion happening internally. If 7433 is used as an argument in a ...*V2* version of a method, that method will execute with whatever output vector now uses that ID or fail if the vector does not exist (i.e. 7433 no longer represents Plate Bottom von Mises Stress in Femap 2020.1 and beyond).

Additionally, the entire Output Object has been deprecated. This Object should no longer be used going forward. To accommodate the deprecation of the Output Object, the Results Browsing Object has been enhanced to match the functionality of the Output Object. As a benefit, the Results Browsing Object is a more capable and efficient way of retrieving and creating output data.

To help mitigate the ID changes, a new *ResultsIDQuery Object* has been created to help the user determine the new vector ID of a specific output quantity. This provides the means to acquire vector IDs using a more descriptive *keyword* approach.

For example, suppose a user wants to retrieve both *Plate Top von Mises Stress* and *Plate Bottom von Mises Stress* output values to export to Microsoft Excel. Instead of hard-coding the new vector IDs directly in the method arguments, the user can now fill a variable with the new vector ID using the *ResultsIDQuery Object*. The ResultsIDQuery Object has several different methods. Each method can be used to locate the new vector ID(s) for a specific type of output, such as *Nodal* for nodal output, *Line* for line element output, *Plate* for plate element output, and so on. The required arguments for each method depend on the entity type, location on the entity, etc. of the output.

To learn more about the *ResultsIDQuery Object*, see **Results Vector ID Query Objects**.

The example below fills two variables with the respective centroid IDs of the Plate Top and Plate Bottom von Mises Stress output vectors and echoes them to the messages window.

```
'1.    Attach to the model in a FEMAP session that is already running.

Sub Main

Dim App as femap.model

Set App = feFemap()

'2.    Create a ResultsIDQuery Object

Dim lookUpID As femap.ResultsIDQuery

Set lookUpID = App.feResultsIDQuery

'3.    Initialize variables for output vector IDs

Dim topVonMisesID As Long

Dim botVonMisesID As Long

'4.    Use the Plate method on the ResultsIDQuery Object to fill the
variables with the vector IDs.

topVonMisesID = lookUpID.Plate(VPV_STRESS, VPT_VON_MISES, VPP_TOP,
```


Unpublished work. © 2025 Siemens

```
VPL_CENTROID)

botVonMisesID = lookUpID.Plate(VPV_STRESS, VPT_VON_MISES, VPP_BOT,
VPL_CENTROID)

’5.    Write the IDs to the messages window.

App.feAppMessage(FCM_NORMAL, "Plate Top Von Mises ID:" +

Str$(topVonMisesID))

App.feAppMessage(FCM_NORMAL, "Plate Bottom Von Mises ID:" +

Str$(botVonMisesID))


End Sub
```

Below are two examples of how to create output using the Results Browsing Object.

The first example creates nodal data with component vectors. The script prompts to select nodes to create output on and fills 3 arrays of the corresponding size with dummy data for X, Y and Z. It then initializes columns to fill the data with via `.AddVectorAtNodeColumnsV2`. Finally, it sets the data in the columns and saves them as output using the `.SetVectorAtNodeColumnsV2` and `.Save` methods, respectively.

```
Sub Main

’1.    Attach to the model in a FEMAP session that is already running.

   Dim App as femap.model

   Set App = feFemap()

’2.    Create Objects

Dim rbo As femap.Results

Set rbo = App.feResults


Dim os As femap.OutputSet

Set os = App.feOutputSet


Dim ndSet As femap.Set

Set ndSet = App.feSet

’3.     Create variables for later use
```


Unpublished work. © 2025 Siemens

```
'Variable to store newly created Output Set ID

Dim oSetID As Long

'Array to store selected Element IDs for creating output

Dim nIDs As Variant

'Variable for how many nodes are selected use for creating output method

Dim listcount As Long

'Array to store the column indices of where the nodal output vectors are
initialized

Dim cIndex As Variant

'Arrays for output data

Dim xVals As Variant

Dim yVals As Variant

Dim zVals As Variant


’4.    Initialize Output Set and store it

oSetID = os.NextEmptyID

os.title = "Create Output on Nodes Example"

os.value = 35.4

os.analysis = 5

os.Put(oSetID)

’5.    Select Nodes to create results on

ndSet.Select(FT_NODE, True, "Select Nodes to create output on")

'Find the count of nodes selected

listcount = ndSet.Count

'Fill array of the selected node IDs

ndSet.GetArray(listcount, nIDs)

'Redim data arrays to size of the nodes selected

ReDim xVals(listcount-1) As Variant
```


Unpublished work. © 2025 Siemens

```
ReDim yVals(listcount-1) As Variant

ReDim zVals(listcount-1) As Variant


’6.    Initialize the Result Browsing Object to create nodal vector output

'In this case, you will create total vector and x, y, z, component vectors.

'These vectors will be stored in the 24,000,000 IDs for user defined output.

rbo.AddVectorAtNodeColumnsV2(oSetID, 24000000, 24000001, 24000002,
24000003, "Displacement", FOT_DISP, True, cIndex)


’7.    Create some data the the nodal results

For i = 0 To (listcount-1)

xVals(i) = 0.001 * i

yVals(i) = -(0.002 * i - 0.001)

zVals(i) = 0.0

Next


’8.    Set the data into the columns of the results browsing object and
then save it to store as an output vector

rbo.SetVectorAtNodeColumnsV2(cIndex, listcount, nIDs, xVals, yVals, zVals)

rbo.Save( )


End Sub
```

The second example creates scalar elemental data. Similarly the script prompts to select elements to create output on and fills an array with the corresponding size with dummy data. It then initializes a column to fill the data with via `.AddColumnsV2`. Finally, it sets the columns and saves it as output using the `.SetColumnV2` and `.Save` methods.

```
Sub Main

’1.    Attach to the model in a FEMAP session that is already running.

Dim App as femap.model
```


Unpublished work. © 2025 Siemens

```
Set App = feFemap()


’2.    Create Objects
Dim rbo As femap.Results
Set rbo = App.feResults


Dim os As femap.OutputSet
Set os = App.feOutputSet


Dim eSet As femap.Set
Set eSet = App.feSet


’3.    Create variables for later use
'Variable to store newly created Output Set ID
Dim oSetID As Long
'Array to store selected Element IDs for creating output
Dim eIDs As Variant
'Variable for how many nodes are selected use for creating output method
Dim listcount As Long
'Variable to store the column index of where the element output vector is
initialized
Dim cIndex As Long
'Arrays for output data
Dim eVals As Variant


’4.    Initialize Output Set and store it
oSetID = os.NextEmptyID
os.title = "Create Output on Elements Example"
os.value = 35.4
```


Unpublished work. © 2025 Siemens

```
os.analysis = 5

os.Put(oSetID)


’5.    Select Elements to create results on

eSet.Select(FT_ELEM, True, "Select Elements to create output on")

'Find the count of nodes selected

listcount = eSet.Count

'Fill array of the selected element IDs

eSet.GetArray(listcount, eIDs)

'Redim data arrays to size of the elements selected

ReDim eVals(listcount-1) As Variant

’6.    Initialize the Result Browsing Object to create a column to store elemental data

'In this case, you will create total vector and x, y, z, component vectors.

'These vectors will be stored in the 24,000,000 IDs for user defined output.

rbo.AddScalarAtElemColumnV2(oSetID, 24000000, "Element Stress", FOT_STRESS, False, cIndex)


’7.    Create some data the the nodal results

For i = 0 To (listcount-1)

eVals(i) = i *100 + 1200

Next


’8.    Set the data into the columns of the results browsing object and then save it to store as an output vector

rbo.SetColumn(cIndex, listcount, eIDs, eVals)

rbo.Save( )

End Sub
```


Unpublished work. © 2025 Siemens

Using the ReadFile Object

The Femap API provides an object that lets you read data lines from text files and parse their contents. While you can use the native file read capabilities of the programming language that you are using, this object is often faster than those routines. The extra speed comes from the internal buffering of data that the object provides.

This example illustrates using this object to read and extract all of the GRID (node) lines from a Simcenter Nastran file. It is not intended to be a general Simcenter Nastran reader. It can only handle small-field (or free format) lines, and only GRID statements in already defined coordinate systems; however, it could be easily modified for other purposes. The results of this reader are used to create nodes in the Femap model, but they could just as easily be used for some other purpose.

```
_____________________________________________________________________________
__
Sub Main
'1.      Attach to the model in a FEMAP session that is already running.
Dim App As Object
Set App = GetObject(, "femap.model")
'2.     Create ReadFile Object.
Dim f As Object
Set f = App.feRead
'3.     Create a Node object.
Dim nd As Object
Set nd = App.feNode


Dim rc As Long
'4.     Set up field widths: 10 fields, 8 characters wide.
Dim w(10) As Long
For i = 0 To 9
  w(i) = 8
Next i
Dim wVar As Variant
```


Unpublished work. © 2025 Siemens

```
wVar = w

'5.     Prepare to read the file. Read only "GRID" lines in fixed or free
format.

rc = f.SetSearchString("GRID", "")

rc = f.FixedFormat(10, wVar)

rc = f.SetAutoFormat()

rc = f.Open("d:\temp\grid.nas", 100)

'6.     Begin reading.

 Do

   rc = f.Read()

'7.      Put the data into the nodes.

   nd.x = f.RealField(4, 0#)

   nd.y = f.RealField(5, 0#)

   nd.z = f.RealField(6, 0#)

   nd.defCSys = f.IntField(3, 0)

   nd.outCSys = f.IntField(7, 0)

'8.     Save the nodes.

   rc = nd.Put(f.IntField(2, 0))

'9.     Keep reading until you get to the end of the file.

 Loop Until f.AtEOF()

 rc = f.Close()


End Sub
```


Unpublished work. © 2025 Siemens

Storing Your Own Data in the FEMAP Database

The Femap API provides the UserData object, which lets you store your own data in the Femap database. You can think of this storage approach as basically a binary file. You write whatever data you want to the file, then, at a later time, you can read the data back using the same approach. The format and order of the data is up to you. The only restriction is that your code must remember what format you used when writing the data, or you will be unable to read it later.

The following example shows writing a number of data types to a UserData object.

```
_____________________________________________________________________________
__
Dim App As femap.object  'For use by all local subroutines

'1.       Simple driver to load and then retrieve userData


Sub Main()

'2.     Attach to the model in a FEMAP session that is already running.

 Set App = GetObject(, "femap.model")

 Call SaveUserData()
 Call LoadUserData()
End Sub




Private Sub SaveUserData()

'3.     Create the UserData object.

  Dim d As Object
  Set d = App.feUserData()

  Dim a(5) As Double
  Dim va As Variant
  Dim rc As Long
  Dim b(4) As Boolean
  Dim vb As Variant
  Dim p As Long


'4.     Start writing data to the UserData storage object.

  d.WriteDouble (8.3)
  d.WriteDouble (76.34)
  d.WriteLong (-345)
```


Unpublished work. © 2025 Siemens

```
  d.writestring ("This is my text")
  d.WriteDouble (3457000000000#)
  a(0) = 0.1234
  a(1) = 0.5432
  a(2) = 0.9876
  a(3) = 0.234
  a(4) = 13.44
  va = a
  rc = d.WriteDoubleArray(5, va)
  b(0) = True
  b(1) = False
  b(2) = True
  b(3) = True
  b(4) = 3
  vb = b
  rc = d.WriteBoolArray(5, vb)


'5.     Assign a title and store object.

  d.Title = "ABCProduct"
  d.Put (1)

End Sub

'_____________________________________________________________________________
___



Private Sub LoadUserData() ' Reloads and displays data

'6.     Create the UserData object.

  Dim e As Object
  Set e = App.feUserData()


'7.     Reload the UserData object.

  e.Get (1)
  MsgBox (e.Title)

  Dim x As Double
  Dim i As Long
  Dim s As String
  Dim vb As Variant
  Dim cc As Variant


'8.     Start reading and displaying the data.

  rc = e.ReadDouble(x)
  MsgBox (Str$(x))
  rc = e.ReadDouble(x)
  MsgBox (Str$(x))
```


Unpublished work. © 2025 Siemens

```
  rc = e.ReadLong(i)
  MsgBox (Str$(i))
  rc = e.readstring(s)
  MsgBox (s)
  rc = e.ReadDouble(x)
  MsgBox (Str$(x))
  rc = e.ReadDoubleArray(i, vb)
  MsgBox (Str$(i))
  MsgBox (Str$(vb(0)))
  MsgBox (Str$(vb(4)))
  rc = e.ReadBoolArray(i, cc)
  MsgBox (Str$(i))
  MsgBox (Str$(cc(0)))
  MsgBox (Str$(cc(1)))
  MsgBox (Str$(cc(2)))
  MsgBox (Str$(cc(3)))
  MsgBox (Str$(cc(4)))


End Sub

_____________________________________________________________________________
__
```


Unpublished work. © 2025 Siemens

Creating a Rigid Element

Another application for the Femap API is to extend the functionality that is already present in Femap. As an example, you may sometimes need to create a rigid, *spider* element inside a hole so that you can distribute a load, or make another connection. This normally involves multiple steps. First you must find and create a node at the center of the hole or node pattern, then you must reselect that node and the surrounding nodes to create the element. With a little bit of work with the API, this routine can be automated to simply let you select the nodes that you want to connect.

The following example shows how to automate creating a rigid spider:

```
_____________________________________________________________________________
__
Sub Main()
'1.      Initialize variables.
Dim App As femap.model
Dim feNode As Object
Dim fN As Object
Dim nodeSet As Object
Dim feElem As Object
Dim nodeCount As Long
Dim nodeX As Double
Dim nodeY As Double
Dim nodeZ As Double
Dim nodeID As Long
Dim elemID As Long

Dim vDOF As Variant
Dim vNodeArray As Variant
Dim vDum As Variant

vDOF = DOF
```


Unpublished work. © 2025 Siemens

```
nodeX = 0#
nodeY = 0#
nodeZ = 0#
'2.     Attach to FEMAP.
Set App = GetObject(, "femap.model")
'3.     Ask the user to select nodes for the rigid element.
Set nodeSet = App.feSet()
rc = nodeSet.Select(7, True, "Select Nodes for Rigid Spider")

If rc = -1 Then
'4.     Identify how many nodes were selected.
   nodeCount = nodeSet.Count()
    If nodeCount > 0 Then
'5.     Make some local arrays to hold the data that was collected.
       ReDim nodeArray(nodeCount) As Long
       ReDim faceArray(nodeCount) As Long
       ReDim Weight(nodeCount) As Long
       ReDim DOFArray(nodeCount * 6) As Long

       vDOF = DOFArray
       vWeight = Weight
       vfaceArray = faceArray

       Dim passCount As Long
        passCount = 0
'6.     Walk the nodes.
       Set fN = App.feNode()
       rc = nodeSet.Reset()
```


Unpublished work. © 2025 Siemens

```
        nodeID = nodeSet.Next()
'7.      Store the IDs.
        Do While nodeID <> 0
            nodeArray(passCount) = nodeID
            passCount = passCount + 1
            rc = fN.Get(nodeID)

            nodeX = nodeX + fN.x
            nodeY = nodeY + fN.y
            nodeZ = nodeZ + fN.z

            nodeID = nodeSet.Next()
        Loop
'8.      Compute the centroid and create the Node at the centroid.
        Set feNode = App.feNode()
        vNodeArray = nodeArray

        nodeID = feNode.NextEmptyID

        feNode.x = nodeX / nodeCount
        feNode.y = nodeY / nodeCount
        feNode.z = nodeZ / nodeCount
'9.      Store the node.
        rc = feNode.Put(nodeID)

         If rc = -1 Then 'return code FE_OK
'10.  Create the element.
            Set feElem = App.feElem()
```


Unpublished work. © 2025 Siemens

```
            elemID = feElem.NextEmptyID


             feElem.Type = 29 'Rigid
             feElem.topology = 13 'Rigid
             feElem.node(0) = nodeID 'Independent Node
            feElem.Release(0, 0) = 1
            feElem.Release(0, 1) = 1
            feElem.Release(0, 2) = 1


            vDum = nodeArray
           rc = feElem.PutNodeList(0, nodeCount, vNodeArray, _
                                      vfaceArray, vWeight, vDOF )
'11.  Store the element.
            rc = feElem.Put(elemID)


       End If
      End If
  End If


End Sub
```

Working with Loads and Constraints

Working with Loads and Constraints includes:

Unpublished work. © 2025 Siemens

Creating Multiple Loads at Once in a Single Load Set

The numbered list below is the work flow for creating loads from scratch that is needed to assure the load creation is successful.

Procedure

1. Create Load Set Object, Load Definition Object, and LoadMesh, LoadBolt, Load ETemp, or LoadNTemp to be created.
2. Assign Load Set Object properties.
3. Use the `Put` method to save Load Set in the model.
4. Assign Load Definition Object properties. The setID property of the load definition object should correspond with the ID of the Load Set Entity that was created in Step 3.
5. Use the `Put` method to save Load Definition in the model.
6. Assign the setID property on the Load Mesh Object (or other) corresponding to the Load Set that was created in Step 3. When using the `PutArray` method, the LoadDefinitionID property on the Load Mesh Object (or other) needs to be set beforehand. If using the `PutAllArray` method, the load definition ID will be assigned from an argument of the call. In both cases the ID(s) must correspond with the load definition ID that was created in Step 5.

> **Note**
>
> When using the `PutArray` or `PutAllArray` methods, the `Put` method on the Load Mesh Object (or other) is not needed. These calls are made to save multiple loads to the database with a single call.

Unpublished work. © 2025 Siemens

Walking Through Mesh-Based Loads and Constraints

This example demonstrates how to walk through all of the mesh-based loads and constraints in your model. If any, expanded geometry-based loads or constraints are found, they are converted to permanent loads or constraints.

Warning

RUNNING THIS PROGRAM WILL PREVENT YOU FROM COMPRESSING THESE LOADS BACK TO PURELY GEOMETRIC LOADS.

This script connects to an open Femap session, and starts to query the database.

```
'------------------------------------------------------------------------------
Sub Main


Dim App As Object
Set App = GetObject(, "femap.model")


Dim rc as Integer
Dim BCSet As Object
Set BCSet = App.feBCSet
Dim BC As Object
Set BC = App.feBCNode


Dim LdSet As Object
Set LdSet = App.feLoadSet
Dim Ld As Object
Set Ld = App.feLoadMesh
Dim Nt As Object
Set Nt = App.feLoadNTemp
Dim Et As Object
```


Unpublished work. © 2025 Siemens

```
Set Et = App.feLoadETemp
'1.      Do the Constraints.
While (BCSet.Next())
   BC.setID = BCSet.ID


   BC.ID = -1
   While (BC.Next())
     If (BC.expanded) Then
         BC.expanded = False
         BC.Put (BC.ID)
     End If
   Wend
Wend
'2.     Do the Loads.
While (LdSet.Next())
'3.       Mesh-based loads.
   Ld.setID = LdSet.ID
   Ld.ID = -1
   While (Ld.Next())
      If (Ld.expanded) Then
          Ld.expanded = False
          Ld.Put (Ld.ID)
      End If
   Wend
'4.       Elemental Temps.
   Et.setID = LdSet.ID
   Et.ID = -1
   While (Et.Next())
```


Unpublished work. © 2025 Siemens

```
      If (Et.expanded) Then
          Et.expanded = False
          Et.Put (Et.ID)
      End If
    Wend
'5.       Nodal Temps.
   Nt.setID = LdSet.ID
   Nt.ID = -1
   While (Nt.Next())
      If (Nt.expanded) Then
          Nt.expanded = False
          Nt.Put (Nt.ID)
      End If
   Wend
 Wend
End Sub
```


Unpublished work. © 2025 Siemens

Charting Femap Results in Excel

This example demonstrates how Femap communicates with other programs. Here you will create a table of data and an associated chart in Microsoft Excel using the data that is currently being used in a deformed display in Femap.

This script connects to an open Femap session, pulls the values of the vector selected for the deformed plot and places them into Sheet 1. The script then sub-graphs the values.

```
_____________________________________________________________________________
__
'Before running this script, you must have a deformed model displayed in
the active view.

Sub Button3_Click()

’1.    Attach to the model in the FEMAP session that is already running.

Dim femap As Object

Set femap = GetObject(, "femap.model")

’2.    Create the View objects and collect the view information.

Dim View As Object 'The FEMAP view object

Set View = femap.feView

’3.     Use the feOutputSet method of the FEMAP Application object to work
with Results Sets.

Dim outset As Object

Set outset = femap.feOutputSet

’4.    Use the Results Browsing Object to work with Results Vector data

Dim RBO As Object

Set RBO = femap.feResults


Dim v As Long

Dim setID As Long

Dim def_vecID As Long

Dim Nodal As Boolean

Dim vec_title As String
```


Unpublished work. © 2025 Siemens

```
Dim LCol As Long

Dim vCol As Variant



'*********************************************************************

’5.    Use the feAppGetActiveView User Interface Method to return Active
View of the active 'Model.

ret = femap.feAppGetActiveView(v)

’6.     Use the Get property to retrieve the record of the FEMAP view
number that is entered. In this case, you are getting the active view.

View.Get (v)

’7.    Use this If statement to make sure that an XY display is not being '
displayed.

If (View.Mode < 6) Then

’8.    Use the Deformed property to return the deformed style: 0=Off,
1=Deformed, 2=Animate. This If statement makes sure that you are using a
deformed display.

   If Not (View.Deformed = 0) Then

’9.    Use the OutputSet property to return the ID of the active output
'set.

        If (View.OutputSet > 0) Then

           setID = View.OutputSet

’10.'Use the DeformData property to return the ID of the vector selected
for the deformed display.

           def_vecID = View.DeformDataV2

           ret = outset.Get(setID)

           ret = RBO.AddColumnV2(setID, def_vecID, False, LCol, vCol)

           ret = RBO.Populate

            'ret = RBO.SendToDataTable 'Useful for debugging. (*Note: 'Data
Table unlocking omitted)

       End If

   Else

           MsgBox "Model needs to be deformed."
```


Unpublished work. © 2025 Siemens

```
           End

   End If

End If



'11.' Use the DataLocation method to return how the output is computed and
stored. Nodal = 7, so this checks if the deformed output vector is Nodal or
Elemental.



Dim dLocation As Long

ret = RBO.DataLocation(dLocation)



Nodal = False

If dLocation = 7 Then

   Nodal = True

End If



'12.' Get the title of the output vector.

ret = RBO.VectorTitleV2(setID, def_vecID, vec_title)

'13.' Next, fill the Worksheet Sheet1 with the node IDs, nodal coordinates,
and values. If the data is elemental, use instead the element IDs and
values.

'14.  Use the feNode method of the FEMAP Application object to create 'Node
objects.

Dim Node As Object

Set Node = femap.feNode

'15.' Use the feElement method of the FEMAP Application object to create
Element objects.

Dim Element As Object

Set Element = femap.feElem



Dim Counter As Long
```


Unpublished work. © 2025 Siemens

```
’16.' Turn off Excel’s screen updating so that the data from FEMAP will be
retrieved faster.

Application.ScreenUpdating = False

’17.' Clear the desired Excel Range

Range("A:E").Clear

’18.' Name the column 1 header "Elements" if the data is elemental, or
"Nodes" if the data is nodal.

If Not (Nodal) Then

   Worksheets("Sheet1").Cells(1, 1).Formula = "Elements"

Else

   Worksheets("Sheet1").Cells(1, 1).Formula = "Nodes"

   Worksheets("Sheet1").Cells(1, 2).Formula = "X"

   Worksheets("Sheet1").Cells(1, 3).Formula = "Y"

   Worksheets("Sheet1").Cells(1, 4).Formula = "Z"

End If

Worksheets("Sheet1").Cells(1, 5).Formula = vec_title


Counter = 1


’19.  Use the Results Browsing Object to return a column of Deformed data

   Dim vIDs As Variant

   Dim vValues As Variant

   Dim endRow As Long


   ret = RBO.GetColumn(0, vIDs, vValues)

   endRow = UBound(vIDs) + 2

’20.' Send the column of Node or Element IDs to Excel all at once

   Range(Cells(2, 1), Cells(endRow, 1)).Value2 = _

                                    Application.Transpose(vIDs)
```


Unpublished work. © 2025 Siemens

```
'21.' Retrieve the coordinate values if using Nodal data.

If Nodal Then

'22.' Use GetAllArray method on Node Objects to reduce FEMAP calls

Dim LnumNode As Long

Dim vEntID As Variant, vXYZ As Variant, vLayer As Variant

Dim vColor As Variant, vNdType As Variant, vDefCSys As Variant

Dim vOutCSys As Variant, vPermBC As Variant



    ret = Node.GetAllArray(0, LnumNode, vEntID, vXYZ, vLayer,
vColor,_                                            vNdType, vDefCSys,
vOutCSys, vPermBC)

'23.' Use a For Loop to create a matrix array to send all coordinate values
at once to Excel

   countEntities = UBound(vEntID) + 1



   Dim vCoords() As Double

    ReDim vCoords(1 To countEntities, 1 To 3) 'Rows, Cols



   a = 1

   b = 0

   For i = 0 To UBound(vEntID)

       vCoords(a, 1) = vXYZ(b)

       vCoords(a, 2) = vXYZ(b + 1)

       vCoords(a, 3) = vXYZ(b + 2)

       b = b + 3

       a = a + 1

   Next

'24.' Send Coordinate values all at once to Excel
```


Unpublished work. © 2025 Siemens

```
   Range(Cells(2, 2), Cells(2 + countEntities, 4)).Value2 = vCoords


End If


’25.' Send values, either Elemental or Nodal all at once to Column E

Range(Cells(2, 5), Cells(endRow, 5)).Value2 = Application.Transpose(vValues)

’26.' Now that Sheet1 is populated, use the Create_chart sub to create an
Excel chart.

Create_chart

’27.' Turn screen updating back on.

Application.ScreenUpdating = True


End Sub



'*********************************************************************


Sub Create_chart()

’28.' Make column 5 the active selection. The Deformed data.

Columns(5).Select

’29.' Assign the address of the selected column of cells to ValueColumn.

ValueColumn = Selection.Address

’30.' Make column 1 the active selection.

Columns(1).Select

’31.' Assign the address of the selected column of cells to xcolumn.

XColumn = Selection.Address


ChartTitle = Worksheets("Sheet1").Cells(1, 5)

XTitle = Worksheets("Sheet1").Cells(1, 1)
```


Unpublished work. © 2025 Siemens

```
'32.' Create and format the chart.
Charts.Add
ActiveChart.ChartType = xlXYScatter
ActiveChart.SetSourceData Source:= _
Sheets("Sheet1").Range(ValueColumn)
ActiveChart.Location where:=xlLocationAsObject, Name:="Sheet1"
ActiveChart.SeriesCollection(1).XValues = _
Sheets("Sheet1").Range(XColumn)

With ActiveChart
   .HasTitle = True
   .ChartTitle.Characters.Text = ChartTitle
   .Axes(xlCategory, xlPrimary).HasTitle = True
   .Axes(xlCategory, xlPrimary).AxisTitle.Characters.Text = XTitle
   .Axes(xlValue, xlPrimary).HasTitle = False
End With

ActiveChart.HasLegend = False

End Sub


_____________________________________________________________________________
__
```


Unpublished work. © 2025 Siemens

Calling the API from Visual C++

The examples in this manual use Visual Basic to call the API because, in general, it is much easier to do. If you want to, however, the API can be called using Visual C++. For example, to simply tell Femap to exit using Visual Basic, you could use the following Basic program:

```
Dim femap As Object

Set femap = GetObject(, "femap.model")

femap.feFileExit
```

If you want to use Visual C++ to accomplish the same thing, without using the supplied Femap Type Library, the code is much more complex:

```
{
   // Get the CLSID for the FEMAP Application Object

   CLSID   clsid;

   CLSIDFromProgID( L"femap.model", &clsid );


   // Create an instance of the Applications IUnknown interface

   IUnknown* pUnk;

   HRESULT      hr = ::CoCreateInstance( clsid, NULL, CLSCTX_SERVER, IID_IUn►
known, (void **) &pUnk );


   // Query IUnknown to retrieve a pointer to the IDispatch interface

   IDispatch*   pDispApp;

   hr = pUnk->QueryInterface( IID_IDispatch, (void**)&pDispApp );


   OLECHAR      FAR* szFunction;

   DISPID      dispid;

   DISPPARAMS   dispparamsNoArgs = { NULL, NULL, 0, 0 };

   VARIANT      retcode;
```


Unpublished work. © 2025 Siemens

```
   szFunction = OLESTR("feFileExit");

   hr = pDispApp->GetIDsOfNames( IID_NULL, &szFunction, 1, LOCALE_US►
ER_DEFAULT, &dispid );

   hr = pDispApp->Invoke( dispid, IID_NULL, LOCALE_USER_DEFAULT, DIS►
PATCH_METHOD, &dispparamsNoArgs, &retcode, NULL, NULL );

}
```

The example above calls a function with no arguments. Calling functions with array arguments can be even more complex since these need to be passed and accessed as VARIANT data types, which must be properly handled in C++.

Using the Femap Type Library and ATL makes this function much easier, and more readable. To do that however, you must first import the Type Library into your application. In your program's header file, add the following line:

```
#import "d:\\v9\run\femap.tlb" named_guids
```

This will automatically create and include header files that make the API functionality available to your application.

In addition, you should include the ATL and OLE Automation header files:

```
#include "atlsafe.h"

#include "oleauto.h"
```

Once you have done this, you can write the above code in the following format.

```
CLSID       clsid;
HRESULT hr=AfxGetClassIDFromString( "femap.model", &clsid);
IUnknown    FAR *punk;
if ( hr == S_OK )
{
   hr=GetActiveObject(clsid,NULL,&punk);
   if ( hr == S_OK )
   {
       CComQIPtr<femap::Imodel> pModel;
       pModel=punk;
       pModel->feFileExit();
   }
}
```


Unpublished work. © 2025 Siemens

Furthermore, once you have the *pModel* pointer, you can then simply use it like you would use the *Femap* object in Visual Basic to provide direct access to the remainder of the API.

```
CLSID       clsid;
HRESULT hr=AfxGetClassIDFromString( "femap.model", &clsid);
IUnknown    FAR *punk;
if ( hr == S_OK )
{
   hr=GetActiveObject(clsid,NULL,&punk);
   if ( hr == S_OK )
   {
       CComQIPtr<femap::Imodel> pModel;
       pModel=punk;

       CComQIPtr<femap::IDBase> pEntity;
       CComQIPtr<femap::INode> pNode;

       pEntity = pModel->feNode;
       pNode = pEntity;
       pEntity->Get(23);

       double xyz[3];

       xyz[0] = pNode->x;
       xyz[1] = pNode->y;
       xyz[2] = pNode->z;
    }

}
```

If you are working with an object that contains array properties you can typically either use the Variant properties, or direct access to the arrays. For the Variant property - you can use code that looks like the following:

```
CComQIPtr<femap::IBCGeom> pBCGeom;
pBCGeom=pModel->feBCGeom;
...
CComVariant vldof;
CComSafeArray<BOOL> ldof(6);
ldof[0]=TRUE;
ldof[1]=TRUE;
ldof[2]=TRUE;
ldof[3]=FALSE;
ldof[4]=FALSE;
ldof[5]=FALSE;
vldof = ldof;
...
pBCGeom->type = 0;
pBCGeom->geomType = 5;
pBCGeom->Layer = 1;
pBCGeom->color = 4;
pBCGeom->vdof = vldof;
...
```


Unpublished work. © 2025 Siemens

```
pBCGeom->setID = 1;
pBCGeom->Put(pBCGeom->NextEmptyID());
```

To retrieve values from an array property, you can simply reverse this:

```
CComVariant vldof;
CComSafeArray<BOOL> ldof(6);
BOOL dof;
...
vldof = pBCGeom->vdof;
ldof = vldof.parray;
dof = ldof.GetAt(0);
...


Or the simpler sequence, which bypasses the Variant...

CComSafeArray<BOOL> ldof(6);
BOOL dof;
...
ldof = pBCGeom->vdof.parray;
dof = pBCGeom->vdof.GetAt(0);
...
```

Or the even simpler approach which uses direct access to the arrays...

```
...
pBCGeom->dof[0] = TRUE;
pBCGeom->dof[1] = TRUE;
pBCGeom->dof[2] = TRUE;
pBCGeom->dof[3] = FALSE;
pBCGeom->dof[4] = FALSE;
pBCGeom->dof[5] = FALSE;
...
dof = pBCGeom->dof[0];
...


_____
__
```


Unpublished work. © 2025 Siemens

Calling the API from Python

This section is intended to introduce core concepts for writing Femap API scripts through Python. Part 1 outlines installation and setup. Part 2 covers Python basics. Part 3 goes into detail of Python-Femap specifics. Example problems are provided in Parts 2 and 3, with sample solutions given at the end.

Versioning and Acknowledgments

This guide utilizes Python version 3.4, with PythonWin version 3.4.4 as the IDE, operating on a Windows 10 machine running Femap version 2019.1. Steps outlined here are based on this guide found on the Siemens Femap Community Forum and personal user experience. Code was formatted using an online resource to automate the process.

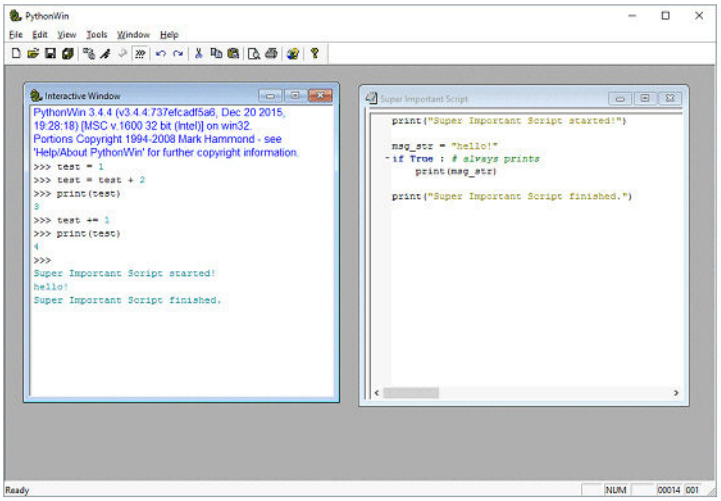
Part 1 - Installation and Setup

If you have a working installation of Python, skip to Part 2 - Python Basics. If you are already proficient with Python and have a working Python installation, skip to Part 3 - Femap API through Python.

Setting up Python

The first step is to download and install Python. **Python.org** is naturally the best option for doing so. While the most up-to-date version is recommended, this guide utilizes Python 3.4.0, and may contain minor syntactical differences to the most recent release.

Next, an interpreter is required to compile and run Python scripts. While many viable options exist, an interpreter with a live command window is tremendously useful for testing small amounts of code at a time. Using an IDE (Integrated Development Environment) is a good start for new users, as it encompasses both a text editor (where scripts are written) and an interpreter (where scripts are run) in a single installation. This guide utilizes PythonWin 3.4.4 as an IDE for convenience, as it is bundled in the Pywin32 installation covered in the following section. Programs like Visual Studio and Notepad++ can even be set to run Python scripts by calling an external interpreter.



Unpublished work. © 2025 Siemens

The screenshot above shows a sample IDE, PythonWin. The left window is the Live Interpreter, which allows the user to test small amounts of code, as shown with the "test" variable. The right window is the Script Editor. Here scripts are written and saved as **.Py* files, which are called using the *Run* command from the main window.

Transferring the Femap Type Library

The Femap API can be written in just about any programming language, so long as the type library is transferred.

Python communicates with Femap via OLE/COM objects, so in order to transfer the type library, Pywin32 must first be downloaded. Select the proper version from **https://github.com/mhammond/pywin32/releases** corresponding to your Python/Windows versions, close all Python applications (including interpreters and IDEs), and run the Pywin32 executable. During installation, verify that the correct Python directory is identified by the installer.

Next, reopen the Python interpreter/IDE (i.e. PythonWin) and execute the following code. Ensure that the file path in the `sys.argv` call is directed to the *Femap.tlb* of the desired version of Femap, wherever that is installed on your computer:

```
import sys

   from win32com.client import makepy

   sys.argv = ["makepy", "-o Pyfemap.py", r"C:\ Program Files\Sie►
mens\FEMAPv2021.2\femap.tlb"]

   makepy.main()
```

The installation directory of the desired Python version should now contain a newly created file, *Pyfemap.py*. Move this file into the *\Lib* folder contained in the Python installation directory.

The *Pyfemap.py* library contains the entire Femap type library imported for use in Python. Part 3 will cover how to access and utilize this library, as well as how to browse the file's contents to troubleshoot unexpected behavior.

Differences across Interpreters

Some commands may function slightly differently across interpreters. For example, calling:

```
sys.exit('femap not open')
```

in Visual Studio has been known to raise errors. This line is included at the start of some examples throughout this guide. Be aware of the quirks in your chosen interpreter as you become more familiar with it, and adjust accordingly. A Python environment in Visual Studio will exit successfully with:

```
raise SystemExit(0)
```


Unpublished work. © 2025 Siemens

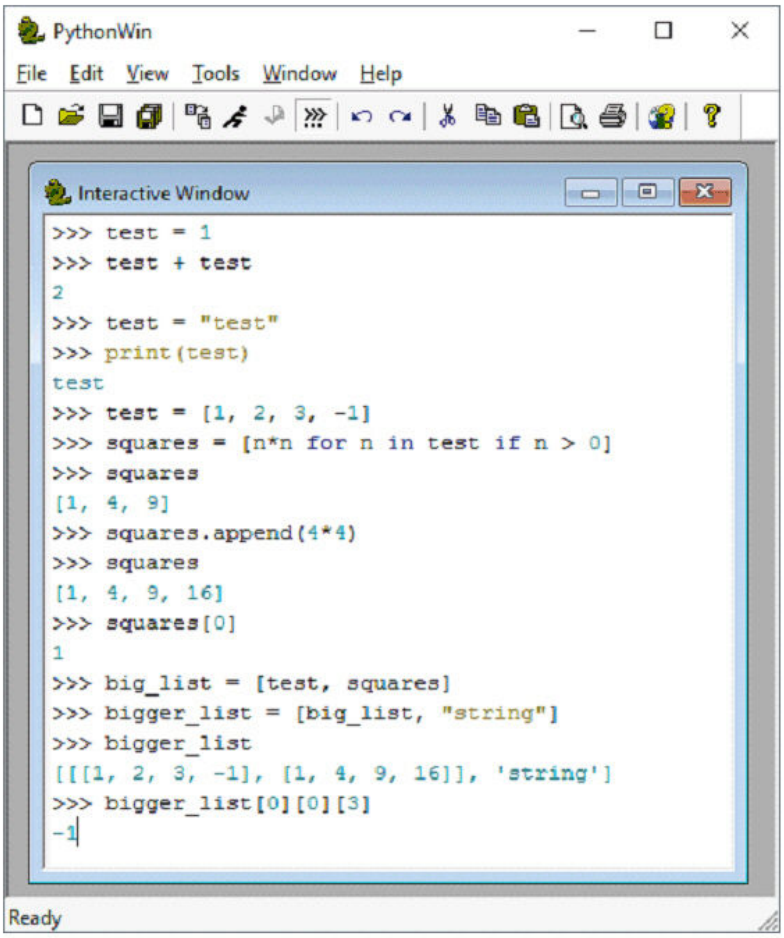
Part 2 - Python Basics

If you already have a working knowledge of Python, feel free to skip ahead to Part 3, which covers specific issues of writing the Femap API through Python.

Learning Python is not so much mastering a set of skills as it is familiarizing oneself with the language. Designed around natural readability, Python concepts and syntax become simple to self-teach once the basics are grasped. Below is a selection of such basics. For more comprehensive coverage, see guides such as **https://docs.python.org/3/tutorial/index.html**, or the myriad other online resources available.

Creating Variables

A major component of Python is flexibility. Variables do not need to be defined ahead of assignment, and data type is never specified - any data can be stored to a variable. Every variable in Python is an object. The example below shows the variable *test* being written as a number, a string, and a list.



The Flexibility of Lists

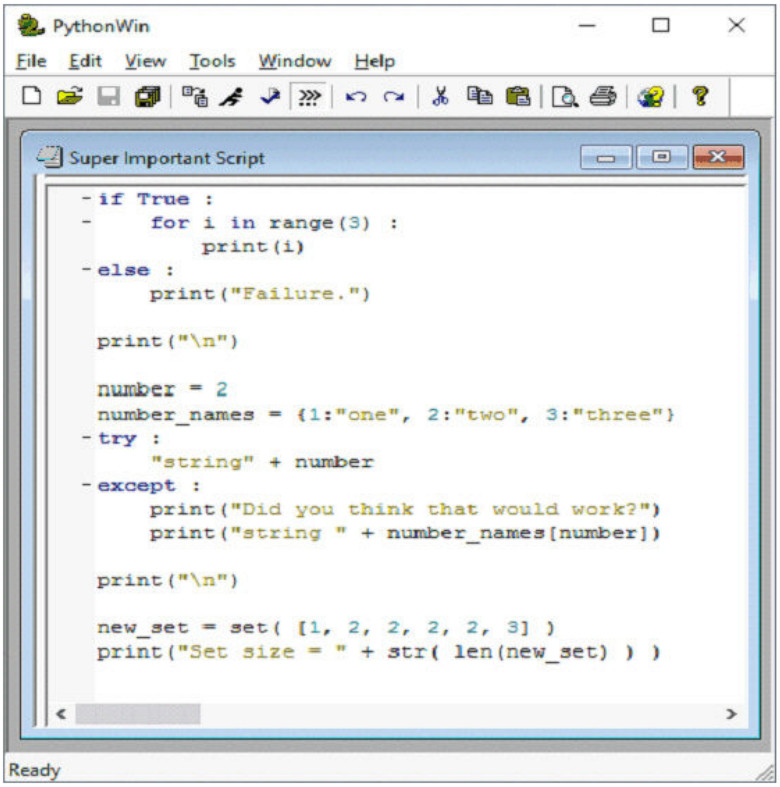
Lists are zero-indexed data structures. Beyond that, there are very few limits on lists. The use of *squares* above is built using a list comprehension, a set of logic packaged into an assignment statement. In this case, *squares* is a list of the square of all items in *test* that are greater than 0. Lists can be appended to as shown using `.append`.

Femap API Guide
Unpublished work. © 2025 Siemens

`big_list` and `bigger_list` are simply lists of lists. Notice that list items need not be the same size - they can even be different data types. The final call in the example returns the third entry of the zeroth entry of the zeroth entry of `bigger_list`. Although complex, this shows how flexible data storage in lists can be. A word of caution: this flexibility leaves ample room for user error. Be careful in data assignment and querying!

Control Flow Tools - if, for, etc.

Like most languages Python utilizes common loops such as if, for, and while. Python denotes the extent of loops, if statements, and other Control Flow Tools by indentation. There are no *end* statements needed to resume normal coding, simply reduce the indentation. The line containing the Control Flow statement itself is always ended with a : (colon).



The example above shows several Control Flow Tools in use. The `try` / `except` pair may be unfamiliar: this is used to have Python attempt a section of code, and specify a fallback set of code if the `try` statement raises any errors, known as exceptions. Specific exceptions can be handled, with an `except [type]:` syntax, specifying one of the predefined Python exception types.

More Data Structures

The example above makes use of two data structures that may be unfamiliar. `number_names` is a Python dictionary, which is a set of `key:value` pairings. Within the `except` statement, `number_names` is indexed by `number` to return a string. Note that dictionaries can contain only one value for every key, but can contain multiple keys that all return the same value. Values can even be functions, which is important because Python does not have a case structure built in. Dictionaries or conditional statements provide all the functionality and more that a case structure has to offer.

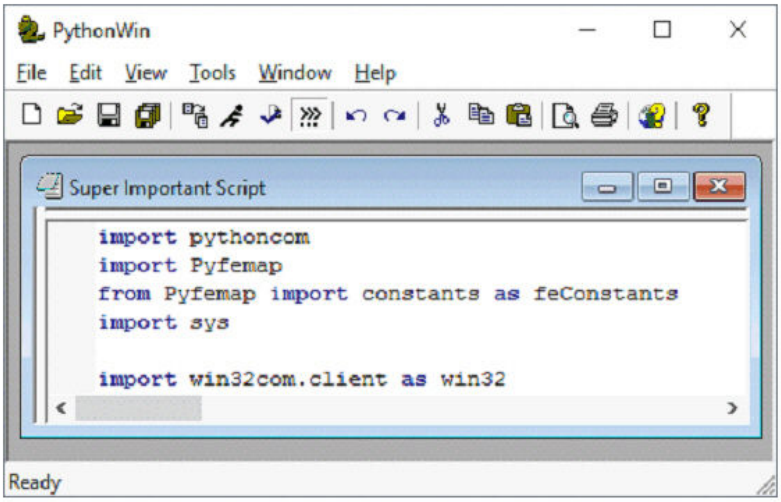
Unpublished work. © 2025 Siemens

There is additionally a set object included in the example above; `new_set` is defined as a set created from a list of 6 entries. Set objects behave like a mathematical set, supporting operations such as union, intersect, and many others from set theory.

Familiarity with the various data structures within Python can often save the hassle of writing complex logic to keep lists in order, or help shrink bloated syntax down to a concise solution.

Modules

Delving into increasingly complex tasks will often require tools beyond Python's standard functionality. For this, modules present an excellent solution. Modules are essentially toolkits containing functions and object classes designed for specific applications. There are several modules built into Python, but the most notable for the purpose of this guide is the Pyfemap module.



The example above contains several import statements with a sampling of styles. The typical `import [module name]` simply imports that module into the script, allowing items within the module to be accessed by typing `[module name].[object name]`. An example of this would be `Pyfemap.constants`, which contains the enumerated constants defined in Femap.

At times the dot-indexed syntax can become unruly. In these cases, the `import as` format shown in the example above is a good way to cut out unnecessary syntactical bulk. In the example above, Femap enumerated constants can be accessed by typing:

```
feConstants.FT_ELEM
```

If no dot notation is desired, the `from [module] import *` syntax can be used. However, be wary of using `import *` as this imports a potentially large number of variables, whose names may not all be known. This may overwrite existing variables in the project namespace, or if the module's creator was not careful, existing reserved Python names, eliminating entire functions.

Example Problem 1

A program that prints all integer factors of an integer. For example, inputting 6 into the program would return something like, *The factors of 6 are: 1, 2, 3, 6*.

Unpublished work. © 2025 Siemens

For Added Challenge

1. *Foolproof* the program - handle the cases where a user inputs a decimal, a string, or something else that might disrupt the script.
2. Design a GUI for the program using Python's tkinter module

For the solution, see **Example Problem 1 - Solution**

Part 3 - Femap API through Python

With a working grasp of both Python and the Femap API, merging the two is quite simple. As mentioned in Part 2, importing the Pyfemap module is accomplished in a single line. This section outlines the additional legwork to connect to an active Femap session, as well as some common pitfalls and recommendations for avoiding them.

Connecting to a Femap Model

Connection to an active Femap session is accomplished via OLE/COM between Python and Femap. While certainly not the only way to do this, the following code is a useful starting point for getting a Femap Python API started:

```
import pythoncom
import Pyfemap
from Pyfemap import constants as feConstants
import sys
#next lines may be needed for PythonWin or other IDEs
#import win32com.client as win32
#win32.gencache.is_readonly=False

#See if there is a current session of Femap open
try:
    existObj = pythoncom.connect(Pyfemap.model.CLSID)
    App = Pyfemap.model(existObj)
    App.feAppMessage(0, "Python API Started")
#Throw an error if no Femap instance opened
except:
    sys.exit('Femap not open')
```

This code template imports all required modules, including Femap constants as `feConstants.[desired constant]`. Next, it attempts to make a connection with the active Femap session, saving this reference as *App*. If this fails, the `except` statement notifies a failure in the IDE and exits the script. On a successful connection, *Python API Started* is printed to the Femap **Messages** window, and an undo point is set.

Pass by Reference - Output Specification

Python does not support the pass by reference output assignment utilized in the native Femap API methods. Pass by reference means that output from a method is stored into entities whose IDs are given

Unpublished work. © 2025 Siemens

as arguments to the method. For example, the following code demonstrates calling the `Curves` method on a Solid object `sol`:

```
rc = sol.Curves(FCC_BOTH, numCurves, curveID) # Native FEMAP API

   [rc, numCurves, curveID] = sol.Curves(constants.FCC_BOTH) # Python
```

In either case, `FCC_BOTH` is the only input value, with `rc`, `numCurves`, and `curveID` all being outputs. In Python, only the method's inputs are passed as arguments, with all of the outputs on the left side of the assignment. In the native Femap API, outputs and inputs are passed as arguments in an order specified by the method's definition.

One benefit of not requiring pass by reference coding is that specific values can be queried without wasting effort dimensioning variables for, and saving, the rest of the outputs. For example, if all that is really needed is the Curve IDs:

```
 curveID = sol.Curves(constants.FCC_BOTH)[2] # Only return the IDs
```

Since the method returns a list of three items, the list can be indexed as the method is called. Output variables need not be defined ahead of time in Python, since methods do not require references to existing, properly defined output variables in order to execute.

Setting Properties

In the native Femap API, properties are accessed in the same way for both reading and writing, an ability not available in Python. A call such as `feElem.Node(0) = 2` in the native Femap API would set the reference node of the rigid element `feElem` to Node ID 2, while in Python this would crash the script.

```
feElem.Node(0) = 2 # Setting a FEMAP property with native WinWrap API

   feElem.SetNode(0,2) # Setting a FEMAP property with Python API
```

The Pyfemap library comes equipped with a set of functions to handle exactly this. For every Femap property `[property]`, there is an associated method `Set[property]`. It's important to note that these methods exist solely within Python, and are not Femap API methods. They are created purely to bridge the syntactical gap between the two programs, so that API scripts can be written similarly in either.

Custom User Dialogs

While Femap's built-in dialog box designer is inaccessible from Python, the *tkinter* module supports Python GUI design. Tkinter offers tremendous flexibility in design and operation of custom GUIs.

Overall functionality between the two platforms is quite similar. The native API dialogue boxes are overall easier to setup on the fly, and WinWrap-savvy users can write event-handling into functions within the script so dialogs can respond in real time. However, this functionality is far easier to achieve in Python, since custom dialogs are designed around event callbacks to begin with. While a simple dialog box with a few choices and one or two buttons is far faster to create in the native Femap API, more complicated

Unpublished work. © 2025 Siemens

UI requirements are far simpler to lay out in Python, once the basic structure of windows, frames, and widget types is solidified, and event callback can be bound in-line as the widget is created.

Considerations of Speed

As further usage will show, certain operations are performed slower via Python than via Femap WinWrap BASIC. Operations such as a `.Get()` can often take over twice as long to perform via Python compared to the native Femap API, due to the Windows OLE/COM overhead. It's important to consider performance factors, especially when designing an algorithm for a large data set. Python modules like *time* will be useful for measuring performance.

Generally speaking, the fastest algorithm will minimize the number of queries between Python and Femap, as communication across the two platforms is often the primary performance bottleneck. Best practice is to gather large amounts of data in a single call from Femap, perform processing in Python, then export results to Femap. Methods like `GetAllArray()` are hugely beneficial for minimizing the sheer number of calls between Python and Femap.

Issues to Note

Given that the Pyfemap library is auto-generated by makepy, it is possible for errors to occur. An example is with the Pyfemap function `.feSelectOutput()`. This method is expected to return a populated Set object (Femap Set, not Python set). Instead, it returns a COM object that references a Femap Set. The difference is subtle, but significant (both written in Python syntax):

```
feSet.Count() # Set object, only works if parentheses present

   feSet.Count # COM object, will raise error if parentheses present
```

The difference is that the COM object behaves like a native Python class object, whereas the FEMAP Set (defined through the Pyfemap library) must call a method to return a property from the FEMAP database. Take caution to look for this issue whenever using the following methods:

```
NextBC, NextBCDef, NextLoad, NextLoadDef, feFileAttachInfo,

feSelectOutput, feSelectOutputSets
```

Browsing the Pyfemap Library

Part 1 of this document outlines how to generate the Pyfemap library, an import of the Femap type library for use in Python. This file, a **.Py* file like any other Python module, can be browsed to investigate unexpected behavior of converted Femap methods, properties, or objects. The Pyfemap library outlines every imported class, along with all associated methods and properties. Consider the example method definition below:

```
  def SetNode(self, nVectorIndex=defaultNamedNotOptArg, arg1=defaultUn▶
namedArg):

            return self._oleobj_.InvokeTypes(18002, LCID, 4, (24, 0), ((3,
1), (3, 1)),nVectorIndex

                , arg1)
```


Unpublished work. © 2025 Siemens

This is the definition for the `SetNode` method discussed in Part 3 - Setting Properties. Recall that this method was created by makepy for setting the *Node* property on Elements. The definition is truly just a wrapper on a single OLE/COM call querying the database of the connected Femap model. The set of tuples ((3,1),(3,1)) specifies the data type of the two inputs. In this case, 3 specifies an integer, and 1 specifies that they are both inputs. With the Femap API manual in-hand and a bit of internet investigation, these tuples can typically grant most of the information about a given definition.

An example of browsing the Pyfemap library is the issue mentioned in the preceding section about Set objects returning as COM objects. The underlying link between the methods listed as potential issues is that all of their function definitions specify a (16393, 2) tuple for their output values, meaning they all return the same type of object. This information was uncovered simply by searching the Pyfemap.Py file for the first method encountered that gave this issue, examining the relevant argument tuple, searching the library for that tuple, then testing the resultant methods in scripts.

Example Problem 2

A Python program that counts all Elements of a specific type in a Femap model.

For Added Challenge:

1. 1. Count Elements of all types in the model, storing the count for each type separately
2. 2. Export the Element count data to Excel

For solution, see **Example Problem 2 - Solution**.

Solutions to Example Problems

All example solutions given address all parts of the Example Problem, including items under *For Added Challenge*" Note that many solutions exist for a given problem, but a valid solution functions for all expected inputs, and is not tripped up by reasonable deviations from expected input, such as a user keystroke error.

The solutions presented here are merely for example, and make no claim of being optimized.

Example Problem 1 - Solution

The prompt itself is addressed almost entirely within the "findFactors" function. The rest of the code is setup for the GUI, or callback definitions for GUI widgets and specific events.

> **Note**
>
> Notice the use of the `lambda` function to define the pushbutton callback command. `lambda` is a Python utility function that essentially creates a function reference to whatever follows. Removing the `lambda` function in this case would instead bind the return value of `cbButton(entryBox)` to the button click, rather than having the button run the `cbButton` function with the argument `entryBox` every time.

```
import tkinter as tk


title = "Number Factorizer"
```


Unpublished work. © 2025 Siemens

```
# Set up main window

root = tk.Tk()

root.title(title)

root.geometry("300x150")


# String Variables for dynamic Labels

strStatus = tk.StringVar()

strStatus.set("Ready")


# Use frames to organize the window

frameTop = tk.LabelFrame(root, text = "Input an Integer")

frameTop.pack(side = tk.TOP, fill = tk.BOTH, expand = True, padx = 5,
pady = 5)


frameBottom = tk.Frame(root)

frameBottom.pack(side = tk.TOP, fill = tk.X, expand = True)


# Entry widget for users to input data

entryBox = tk.Entry(frameTop, width = 20)

entryBox.pack(side = tk.TOP, pady = 5)


# Label widget to display info to the user

label = tk.Label(frameBottom, textvariable = strStatus, wraplength =
300)

label.pack(side = tk.LEFT, expand = False)


# Callback for the button widget

def cbButton(widget):
```


Unpublished work. © 2025 Siemens

```
    # Get Entry value
    val = widget.get()

    # Reset text color, info string
    widget.config(fg = 'black')
    strStatus.set("Processing...")

    # Exit function if no input given
    if val == "" :
            strStatus.set("Ready")
            return

    # Pre-Screen user input
    try :
        if len(val) > 14 : # Toss out big numbers
            strStatus.set("Number too large!")
            raise

        if val != str( int( float(val) ) ): # Integers only
            strStatus.set("Not an integer!")
            raise

        if int(val) < 0 : # No negativity
            strStatus.set("No negative numbers!")
            raise

        # Convert from string and run processing function
        val = int(val)
```


Unpublished work. © 2025 Siemens

```
        findFactors(val)


    # Make text red to indicate bad input if the above fails
    except :
        widget.config(fg = 'red')
        return


 # Processing Function - finds all factors of pre-screened input value
'val'
def findFactors(val):


    # Create list of all factors as tuples
facs = [(i, int(val/i)) for i in range (1, int(val**0.5) + 1 )\
            if val % i == 0 ]


    # Write results to Status String
strStatus.set( "Factors of " + str(val) + " : " + \
                str(facs)[1:len( str(facs) ) - 1] )


# Define pushbutton
button = tk.Button(frameTop, text = "GO!", command = lambda: cbBut▶
ton(entryBox) )
button.pack(side = tk.TOP, pady = 5)


# Bind 'Return' key to mimic "GO!" Button press
root.bind('<Return>', lambda self: cbButton(entryBox) )


# Launch UI
root.mainloop()
```


Unpublished work. © 2025 Siemens

Example Problem 2 - Solution

Barring the 42-line Dictionary definition and the 15 lines of setup code, this example is really quite concise. The base problem prompt is solved, where *i* can be substituted for a user-specified Element type (based on the Femap enumerated constant for Element type).

Here a Python Dictionary is used to simplify the process of appending the Element type string into Excel, eliminating the need for a lengthy `if`/`else` statement.

This solution is a minor revision of an example given in the guide referenced under **Versioning and Acknowledgements** in this document.

```
import pythoncom

import Pyfemap

from Pyfemap import constants

import sys


import win32com.client as win32

win32.gencache.is_readonly=False


try:

    existObj = pythoncom.connect(Pyfemap.model.CLSID) # Grabs active
model

    App = Pyfemap.model(existObj)

except:

    sys.exit("femap not open") # Exits program if there is no active
femap model


# Create checkpoint for FEMAP "undo" to revert to

App.feAppUndoCheckpoint("Undo Python Script")


rc = App.feAppMessage(constants.FCM_NORMAL,"Python API Started")


# Package code into function to enable easy exit with "return" call
```


Unpublished work. © 2025 Siemens

```
def main() :
    # Define FEMAP Set to hold relevant Elements
    pElemSet = App.feSet

    # Exit API if no Elements exist
    pElemSet.AddAll(constants.FT_ELEM)
    if pElemSet.Count() <= 0:
       App.feAppMessage(constants.FCM_NORMAL,'No Elements found!')
        return

    # Calls and creates new excel application
   appExcel = win32.Dispatch('Excel.Application')

    # Add sheet to workBook
    workBook = appExcel.Workbooks.Add()
    workSheet = workBook.Worksheets(1)

    # Set column headers
    workSheet.Cells(1,1).Value = "Element Type"
    workSheet.Cells(1,2).Value = "Element Count"

    # Initialize row counter
    row = 2

    # Define dicitonary of all Element types by FEMAP Enum
typeDict = {1:"L_ROD elements", \
2:"L_BAR elements", \
3:"L_TUBE elements", \
```


Unpublished work. © 2025 Siemens

```
4:"L_LINK elements", \
5:"L_BEAM elements", \
6:"L_SPRING elements", \
7:"L_DOF_SPRING elements", \
8:"L_CURVED_BEAM elements", \
9:"L_GAP elements", \
10:"L_PLOT elements", \
11:"L_SHEAR elements", \
12:"P_SHEAR elements", \
13:"L_MEMBRANE elements", \
14:"P_MEMBRANE elements", \
15:"L_BENDING elements", \
16:"P_BENDING elements", \
17:"L_PLATE elements", \
18:"P_PLATE elements", \
19:"L_PLANE_STRAIN elements", \
20:"P_PLANE_STRAIN elements", \
21:"L_LAMINATE_PLATE elements", \
22:"P_LAMINATE_PLATE elements", \
23:"L_AXISYM elements", \
24:"P_AXISYM elements", \
25:"L_SOLID elements", \
26:"P_SOLID elements", \
27:"L_MASS elements", \
28:"L_MASS_MATRIX elements", \
29:"L_RIGID elements", \
30:"L_STIFF_MATRIX elements", \
31:"L_CURVED_TUBE elements", \
```


Unpublished work. © 2025 Siemens

```
32:"L_PLOT_PLATE elements", \
33:"L_SLIDE_LINE elements", \
34:"L_CONTACT elements", \
35:"L_AXISYM_SHELL elements", \
36:"P_AXISYM_SHELL elements", \
37:"P_BEAM elements", \
38:"L_WELD elements", \
39:"L_SOLID_LAMINATE elements", \
40:"P_SOLID_LAMINATE elements", \
41:"L_SPRING_to_GROUND elements", \
42:"L_DOF_SRPING_to_GROUND elements", \
                }

    # Check for Elements of all types
    for i in range(1,42):
        pElemSet.clear() # Clear out old Elements found
# Add all Elements of new type
        pElemSet.AddRule(i,constants.FGD_ELEM_BYTYPE)
        # If any Elements of type "i" found, add to spreadsheet
        if pElemSet.Count() > 0:
            workSheet.Cells(row, 1).Value = typeDict[i]
            workSheet.Cells(row, 2).Value = str(pElemSet.Count())
            row += 1

       # Make Excel visible
       appExcel.Visible = True

   # Invoke main function to run API
```


Unpublished work. © 2025 Siemens

```
   main()


   rc = App.feAppMessage(constants.FCM_NORMAL,"Python API Example Fin▶
ished")
```

API Selection Tool - Select Property(s) To Get Nodes

In this example, you will use an API that you can create by copying the API code provided. It will demonstrate how to select a property or properties to get Nodes using the Femap **API Selection Tools**.

Procedure

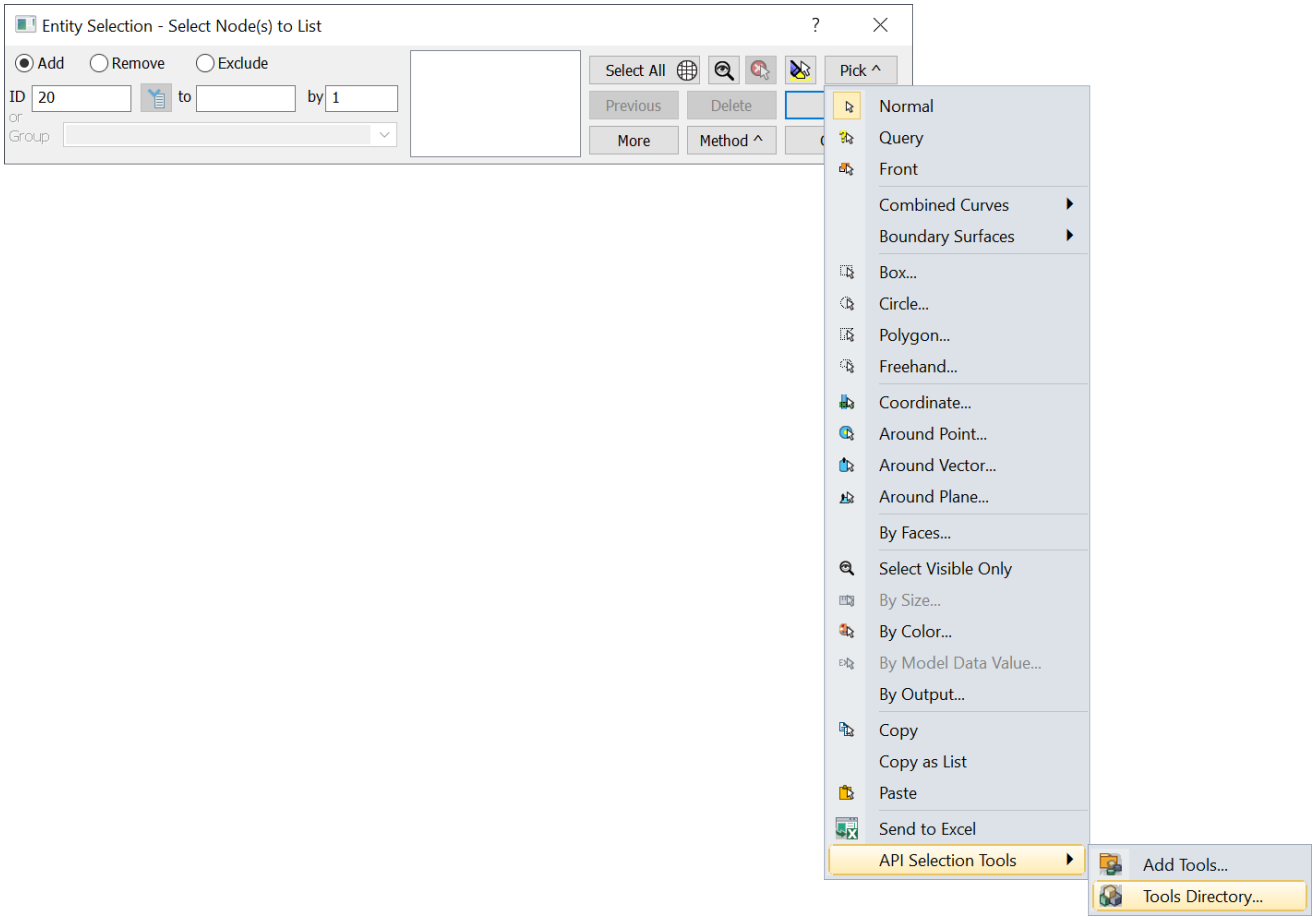
1. Select a command which has a “Node Selection Entity,” such as the **List** → **Model** → **Node** command.

   Note

   The command prompts you to “select nodes”. The **API command** in this procedure will only work with commands that have Entity Selection with Nodes.

2. From the Entity Selection dialog, choose **Pick ^** (upper right corner), and then select **API Selection Tools** → **Tools Directory** from the pop up menu.

Unpublished work. © 2025 Siemens

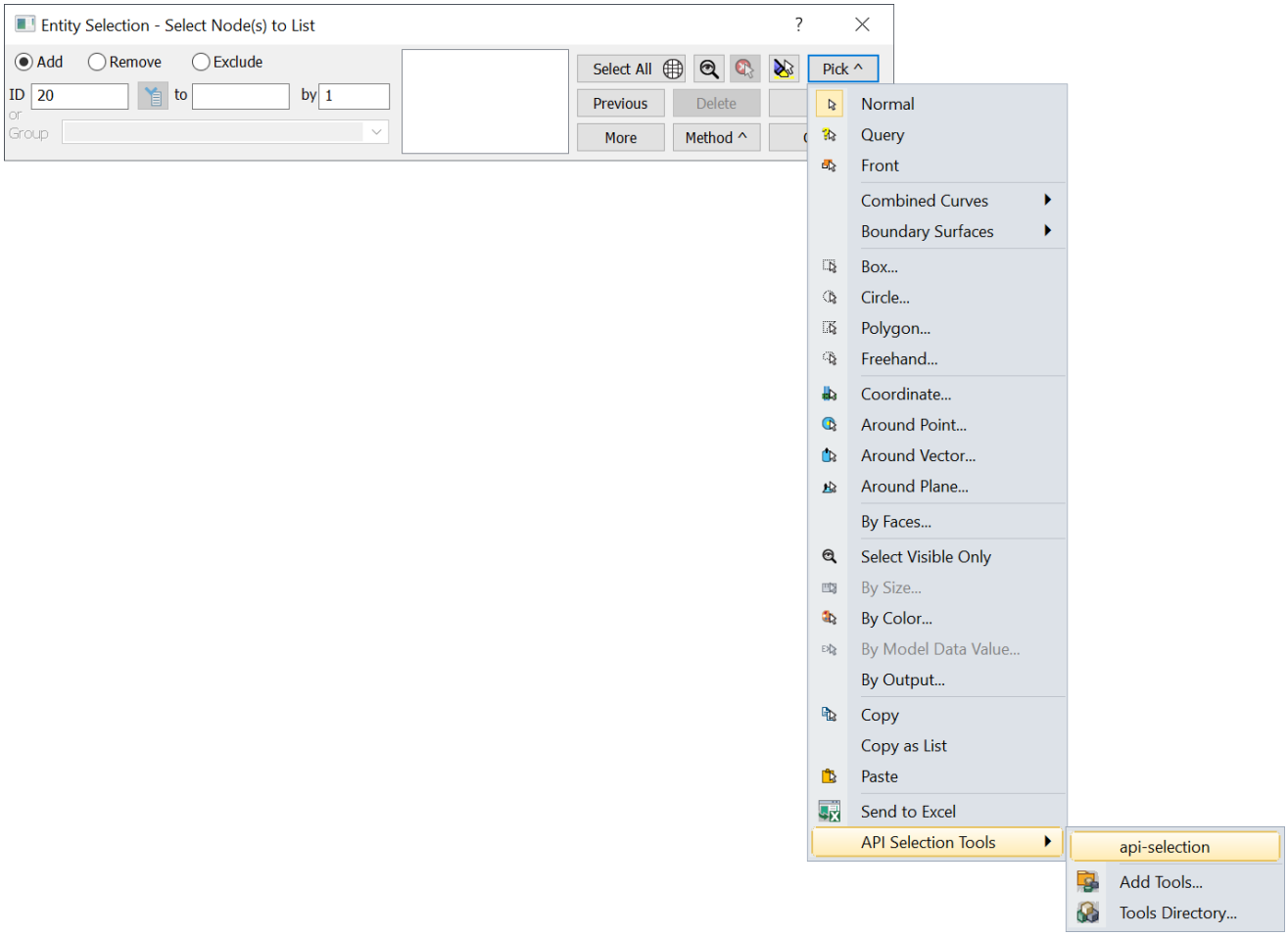


3. Select a directory containing the API file. If it is not there, copy the API to the directory and, repeat this step.

4. The next time you start a node selection command, when you choose **Pick ^**, and then select from the **API Selection Tools** sub menu, the API will appear on the menu.

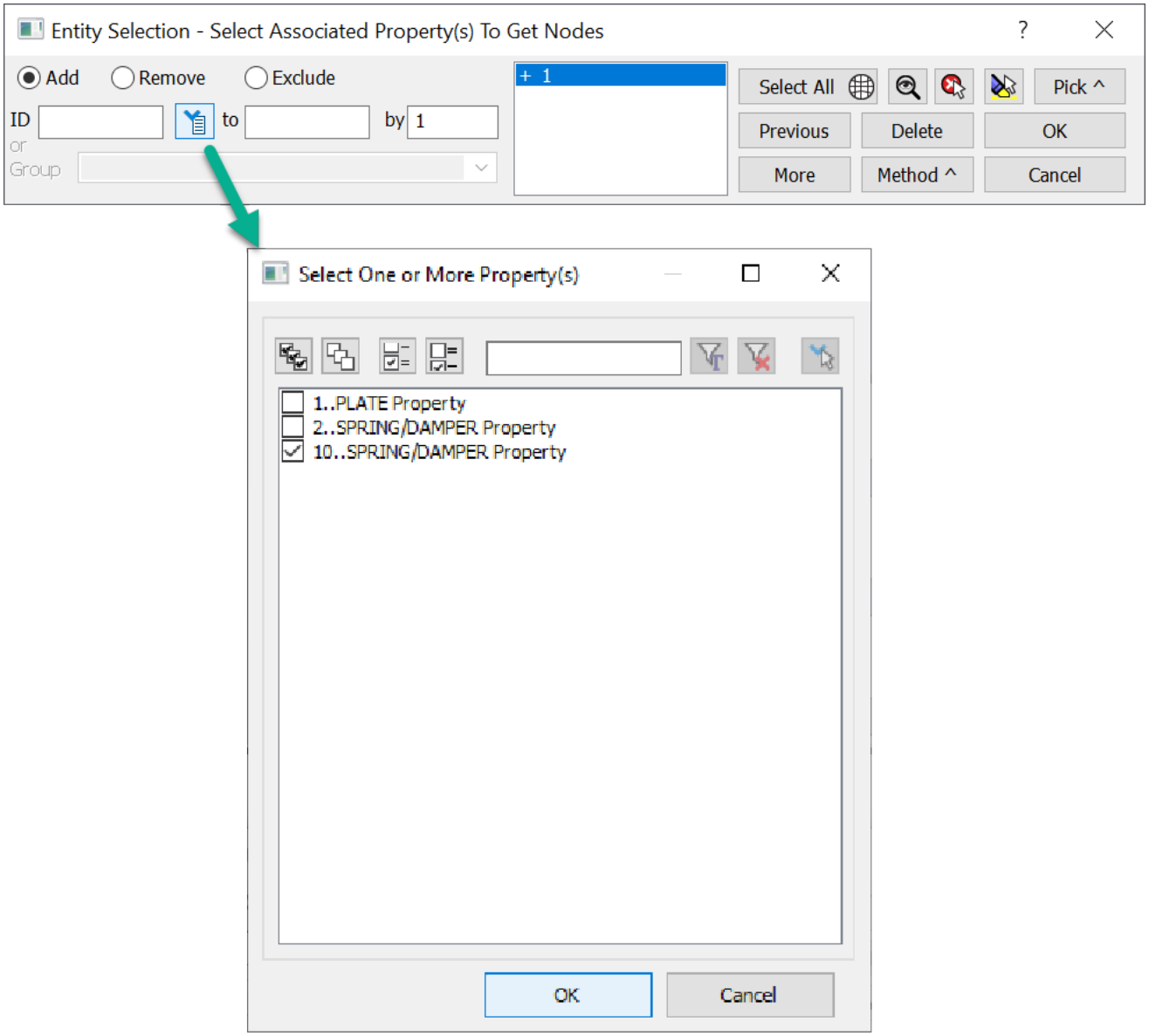
   From the Entity Selection dialog,

Unpublished work. © 2025 Siemens



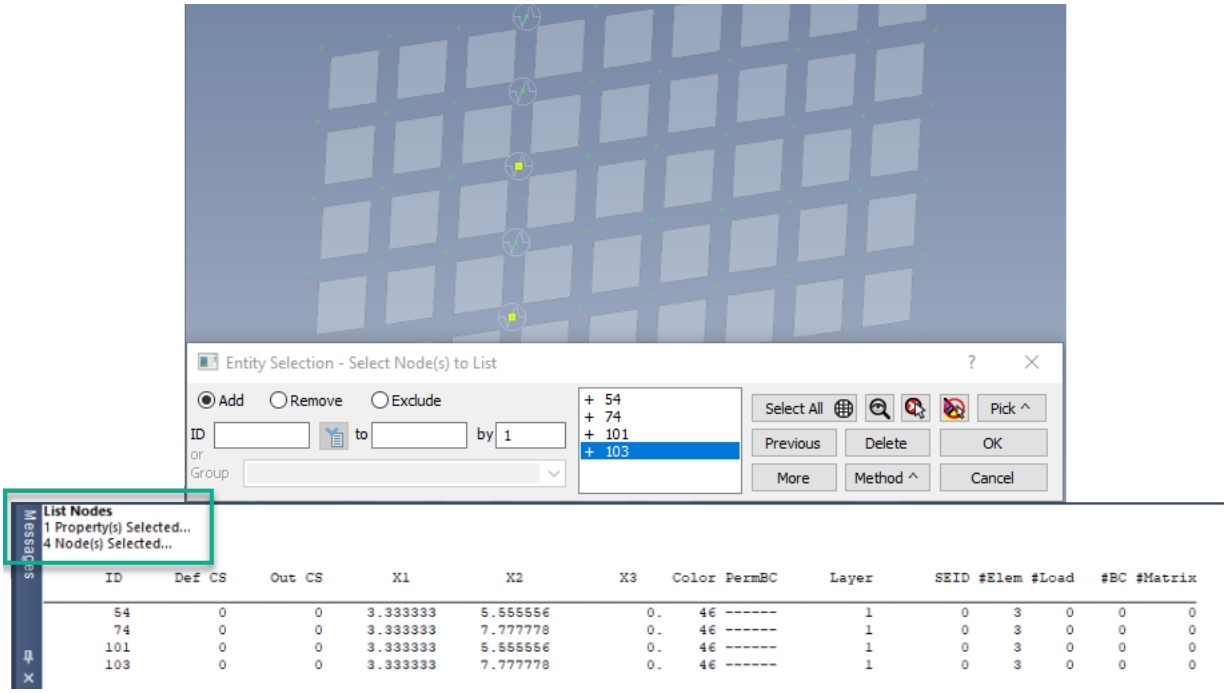
5. This API prompts you to select one or more properties. Do this by clicking in the graphics window or click the Select from List of Titled Entities icon to select properties.

Unpublished work. © 2025 Siemens



6. With the properties selected, the node IDs will appear in the Entity Selection dialog.

Unpublished work. © 2025 Siemens



Femap API Code

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Dim enType As femap.zDataType

    Dim SelectionSet As femap.Set
    Set SelectionSet = App.feSet

    Dim SelectionSet2 As femap.Set
    Set SelectionSet2 = App.feSet

    Dim feProp As femap.Set
    Set feProp = App.feSet

    Dim ElemSet As femap.Set
    Set ElemSet = App.feSet

    Dim NodeSet As femap.Set
    Set NodeSet = App.feSet

    Dim PropIDs As Long

    App.feStartSelectionTool(SelectionSet.ID,enType)

    rc = feProp.Select(FT_PROP,True,"Select Associated Property(s) To Get
Nodes")
```


Unpublished work. © 2025 Siemens

```
    If rc = FE_CANCEL Then
        End
    End If

    'Check to make sure the user is a node entity type command selection
    If enType = FT_NODE Then
        'Get all elements associated to selected property(s)
        ElemSet.AddSetRule(feProp.ID,FGD_ELEM_BYPROP)

        'Get nodes on the elements for the property(s)
        NodeSet.AddSetRule(ElemSet.ID,FGD_NODE_ONELEM)

        'If current selection is empty, then add nodes to the selection
        If SelectionSet.IsEmpty Then
            App.feEndSelectionTool(NodeSet.ID,0)
        End If

    Else
        App.feAppMessageBox(0,"Not On Node Selection Entity Command")
        App.feAbortSelectionTool

    End If


End Sub
```


Unpublished work. © 2025 Siemens

Creating Output in Femap prior to v2020.1 (OBSOLETE)

Note

Starting with Femap 2020.1, the entire Output Object has been deprecated and the Results Browsing Object MUST be used in place of the Output Object. The Results Browsing Object provides more capability when reviewing existing results and methods were added for Femap 2020.1 to provide superior functionality to the Output Object.

For an example of using the Results Browsing Object to create output, see **Working with Output in Femap v2020.1 and above**.

This example demonstrates the use of several API functions that could be used to create your own output data in Femap. The example simply loads dummy results as output, but you could modify it to be part of a larger program that computes its own real output, uses existing output from Femap to do further processing, or reads output from another program.

This example assumes that the existing model in Femap already contains 100,000 nodes and 90,000 planar elements. To be useful as a real program, the sections of code that generate the output must be rewritten to check the model and get or compute real results from some other source.

```
'-------------------------------------------------------------------------------
Sub Main

'1.         Attach to the model in a FEMAP session that is already running.

   Dim App As Object
   Set App = GetObject(, "femap.model")

'2.     Create an Output Set object.

   Dim os As Object
   Set os = App.feOutputSet

'3.     Create an Output object.

   Dim o As Object
   Set o = App.feOutput
   Dim rc As Long
   Dim i As Long

'4.        Initialize an Output Set and store it.

   os.title = "My Test Set"
   os.value = 35.4
   os.analysis = 5
   os.Put (1)

'5.        Initialize the Output object to create nodal vector output.
```


Unpublished work. © 2025 Siemens

```
'   (In this case, you will create a total vector and x, y and z component
vectors.)

   rc = o.InitVectorAtNode(1, 300000, 300001, 300002, 300003, _
      "Displacement", 1, True)
   Dim ID(100000) As Long
   Dim x(100000) As Double
   Dim y(100000) As Double
   Dim z(100000) As Double
   Dim xV As Variant
   Dim yV As Variant
   Dim zV As Variant
   Dim idV As Variant

'6.     Make up some output values for 100,000 nodes.
    ' (Replace this section with some code that creates or reads     real
results values.)

   For i = 0 To 99999
      ID(i) = i + 1
      x(i) = 0.001 * i
      y(i) = 0#
      z(i) = 0.02 * Sin(360# / 99999# * i * 3.14 / 180#)
   Next i


   xV = x
   yV = y
   zV = z
   idV = ID

'7.     Save the displacement output in the database.

   rc = o.PutVectorAtNode(100000, idV, xV, yV, zV)
   rc = o.Put(-1)

'8.     Make some elemental stress output. Reinitialize the output object.


   rc = o.InitElemWithCorner(1, 100, 101, 102, 103, 104, 0, 0, 0, 0, _
       "Stress", 2, True)
   Dim c(100000) As Double
   Dim c1(100000) As Double
   Dim c2(100000) As Double
   Dim c3(100000) As Double
   Dim c4(100000) As Double
   Dim cV As Variant
   Dim c1V As Variant
   Dim c2V As Variant
   Dim c3V As Variant
   Dim c4V As Variant

'9.     Make output for 90,000 elements. Make both the center and corner
results.

   For i = 0 To 89999
```


Unpublished work. © 2025 Siemens

```
      ID(i) = i + 1
      c(i) = i
      c1(i) = i + 0.1
      c2(i) = i + 0.2
      c3(i) = i - 0.2
      c4(i) = i - 0.1
   Next i

   cV  = c
   c1V = c1
   c2V = c2
   c3V = c3
   c4V = c4


'10.  Store the elemental results in the database.

   rc = o.PutElemWithCorner(90000, 4, idV, cV,   c1V, c2V, _
   c3V, c4V, c4V, c4V, c4V, c4V)
   rc = o.Put(-1)


End Sub

_____________________________________________________________________________
__
```


Unpublished work. © 2025 Siemens

3. The Femap Application Object

Getting Started

The first thing that you must do before you can use any of the other functionality in the Femap API is to access the Femap model, or application object. This object provides access to all of the other methods and properties, including the methods used to create other model objects.

Normally, when you define your *Femap* object, you will want to make it a global object. Once you *get* or *create* the object, you will want it to remain available for the entire duration of your application. In most cases, you will not want to repeatedly create and destroy this object.

Using the Femap API Programming Window

The quickest way to get started using the Femap API is to open the API Programming window. This provides access to WinWrap® Basic from Polar Engineering and Consulting. It provides a complete editing, debugging and runtime environment with a language that closely follows Visual Basic. More importantly, since it is integrated with Femap, it is always readily available.

Unpublished work. © 2025 Siemens

Creating a Femap Object

To link your application to Femap, you must declare an object and then connect it to Femap. In Basic, this is done via the `GetObject` or `CreateObject` functions. To decide which of these to use, you must decide whether your application needs to access a Femap session that is already in progress - in which case you would use `GetObject`. If you need to start a new Femap session, you must use `CreateObject`.

Example of Creating a Femap Application Object

The following shows a sample (in Basic) of how to create a Femap Application Object, and connect to a running Femap session.

```
Dim App As Object
Set App = GetObject(, "femap.model")
```

If instead, you wanted to start a new Femap session, the proper approach would be:

```
Dim App As Object
Set App = CreateObject("femap.model")
```

This call to `CreateObject` starts a new Femap session, but it will not be visible. If you want Femap to be visible, you must then call the `feAppVisible` method.

If you have an application that creates a new Femap session and then want to exit Femap and no longer have it running as a process, you need to:

Set the Femap instance to `null` or `nothing`.

```
femap = nothing
```

Also, if you are working on most .net platforms, you will also want to call the .net *Garbage Collector*.

```
System.GC.Collect()
```

If you have an application that connects to a running Femap session and then want to exit Femap and no longer have it running as a process, you need to call `feFileExit ()` first, then also perform the steps above.

Issues When Connecting to an Existing Session

When you are connecting to an existing session there are some special considerations. If you are programming in an environment outside of Femap (anything but the Femap API Programming Window), then you must use `GetObject` to connect. This uses standard OLE/COM mechanisms to find the Femap

Unpublished work. © 2025 Siemens

object, but imposes a limitation. If you have multiple copies of Femap running (multiple processes, not multiple models open in the same Femap), `GetObject` will always connect you to the process that was started first. This is simply a limitation of the OLE/COM interface and can not be avoided.

If you are using the integrated WinWrap environment (API Programming) however, you can overcome this limitation by never using `GetObject`. Instead, you can connect to the current Femap session, no matter how many Femap processes are running by using:

```
Dim App As Object
Set App = feFemap()
```

or more appropriately, to access the object information provided by the Type Library (as described below)

```
Dim App As femap.model
Set App = feFemap()
```

The `feFemap()` method does the same thing as `GetObject`, but since the programming environment is fully integrated with Femap, it can always determine the session that launched your application and communicate directly with that process. This will work even if you save your Basic script from the API Programming window and connect it as a User Command to a toolbar button or menu command.

Unpublished work. © 2025 Siemens

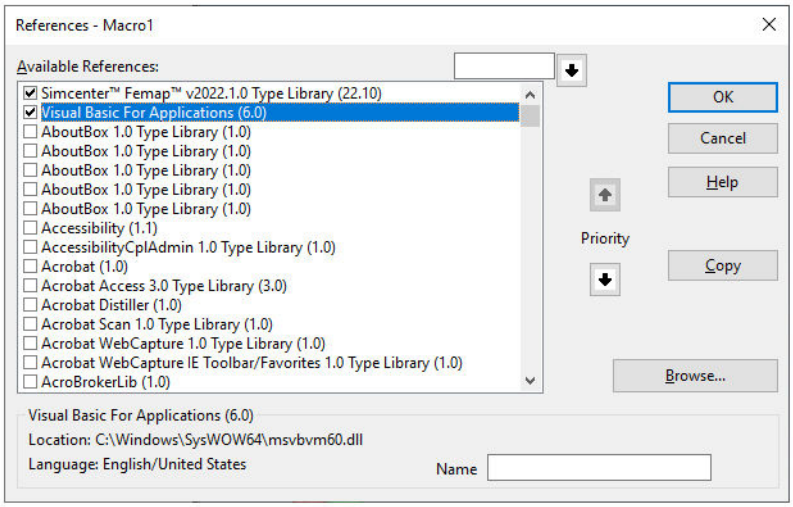
Using the Femap Type Library

As shown above, you can declare your Femap object as a generic *Object*, whether you are programming in Visual Basic or any other Basic Editor outside of Femap (such as in Excel). If you do work in Visual Basic however, you have the option of referencing the Femap Type library and explicitly using the objects that it defines (The API Programming Dockable Pane inside of Femap automatically uses the Femap Type Library). Using the Type Library provides you with several very helpful benefits. First you will be able to use the Object Browser to view the available properties and methods. Even more convenient however when you are working with an object or method, you will see popup tooltips that give you information regarding the available properties, or the required parameter lists.

If you are using a different language, like C++, you will always want to use the Type Library, since it provides much simpler access to the API functionality.

Referencing the Type Library

Before you can use the Type Library, you must reference it. The command to do this can be different depending on where you are working.



In Visual Basic

In Microsoft Visual Basic, go to the Project → References command (or Project → Add Reference in VB .NET). Look through the list of available references. If you find Femap, make sure it is checked. If not, press the Browse button, and find the file *Femap.tlb*, which should be in the directory where you installed Femap. Once you select that file, it should appear in the list of references (possibly at the bottom, and not in alphabetical order). Find the reference to Femap, make sure it is checked and press OK. Although the list may be somewhat different, you should end up with Femap checked as an available reference, as shown.

In Visual Basic for Applications

If you are programming with Microsoft Excel, Word or other Office applications, you will be using Visual Basic for Applications. The process to access the Type Library is the same, except that instead of using the Project → References command, you must use the Tools → References command.

In Visual C++

Unpublished work. © 2025 Siemens

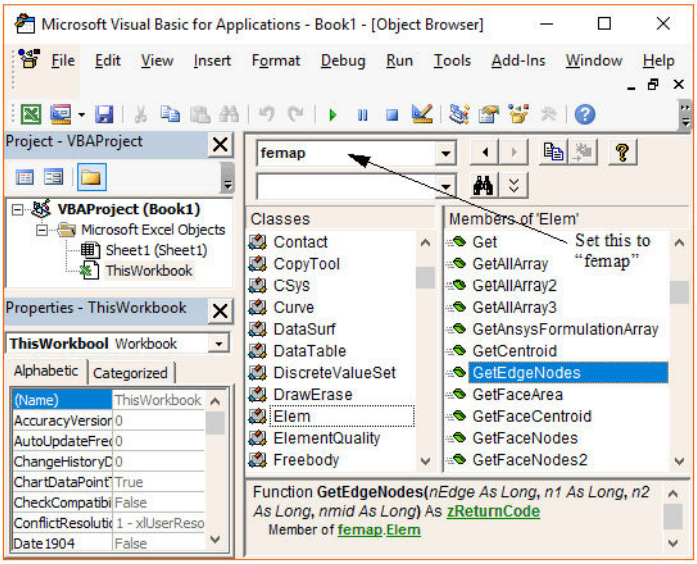
In a Visual C++ project, you must add a reference to the Type Library by adding a reference to it in your header file. Add a line similar to:

```
#import "path\femap.tlb" named_guids
```

into your header file. Modify *path* to accurately locate the Type Library that you want to reference.

Using the Object Browser

Once you have referenced the Femap Type Library, you will be able to use the Object Browser to review the objects, properties and methods that are available. Simply go to the View → Object Browser (F2) command. You will see a form that looks something like the one shown here. To limit the information shown to the Femap classes, make sure you change the Project/Library to Femap in the drop-down control near the top of the window. You can then browse through the list of available classes, and their associated properties and methods. When you select one of these items, a description of the item, along with any parameter lists is displayed at the bottom of the browser.



Using the Type Library

To use the information from the Type library, you must declare your objects using one of the available classes - not using the more generic *Object* definition. For example, instead of using:

```
Dim App as Object
```

you must code,

```
Dim App as femap.model
```


Unpublished work. © 2025 Siemens

Once you have declared the Femap object as shown, you still must `Get`/`Set` it in the same way that you did above. Therefore, to use the type library, and access an existing Femap session, the code required would be:

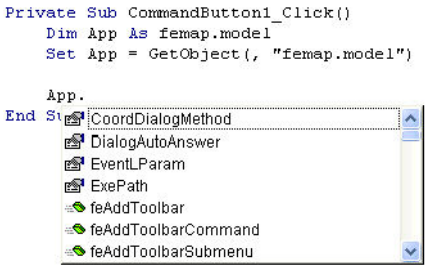
```
Dim App as femap.model
Set App = GetObject(,"femap.model")
```

or once again, if you are programming in the API Programming window, you should use

```
Dim App as femap.model
Set App = feFemap()
```

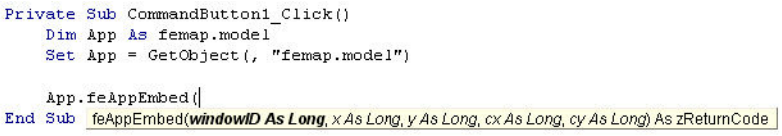
to always connect to the current Femap session.

To create a new Femap session, using the type library use:




```
Dim App as femap.model
Set App = CreateObject("femap.model")
```

As stated previously, there are definite advantages to using the Type Library. One of which is the popup list of available properties and methods. For example, after having defined your Femap/App object, as you type App, you will see a popup that shows the available properties and methods. Further, as you continue to type you will see a popup tooltip that shows you the parameter list for the method that you are working on.



The various classes from the Type library can also be used to define your other objects (not just App). This is described later in **Femap Entity Objects**.

Unpublished work. © 2025 Siemens

Working with the User Interface

In addition to the basic decision about whether or not to connect to a running Femap session, you must also decide how you want the user interface of your application to interact with the Femap user interface. There are several approaches that you might want to use, including:

- Running as two separate applications (no interaction)
- Running Femap as a hidden server
- Running your application as a Femap add-in
- Embedding Femap graphics and messages in your application

Proper setup for each of these options is described in the following sections.

Running as two separate applications

As you might expect, this is possibly the simplest of the approaches to interaction. Since neither Femap, nor your application, need to really know anything about the user interface of the other, there is no additional setup required. Simply start your application, then *get* or *create* your Femap object as described above. If you used `GetObject` to attach to a running session, there is nothing more to do. If you used `CreateObject` however, a new Femap session will be started, but Femap will not be visible. To make your Femap session visible, after calling `CreateObject`, you must call `femap.feAppVisible`. You do not need to call `feAppVisible` if you used `GetObject`, because Femap will already be visible.

In summary, if you want to run as two separate, on-screen, visible applications, use

```
Dim App as Object
Set App = GetObject(,"femap.model")
```

to work with an existing Femap session. Or to start a new Femap, use

```
Dim App as Object
Dim rc as Long
Set App = CreateObject("femap.model")
rc = App.feAppVisible(True)
```

Running Femap as a hidden server

If you want your application to be the only user interface that is visible to the user, then the process is very similar to running as two separate applications. In this case however, you will never want to use

Unpublished work. © 2025 Siemens

`GetObject` to attach to an existing Femap - you will use `CreateObject`, to create a new, hidden Femap object. Simply use:

```
Dim App as Object
Set App = CreateObject("femap.model")Dim App as Object
Set App = CreateObject("femap.model")
```

At this point a new Femap session will be started and through the API you can access any of the available functionality. You will have to develop your own user interface to present the capabilities that you want to the user, and then call Femap API functions to implement that functionality.

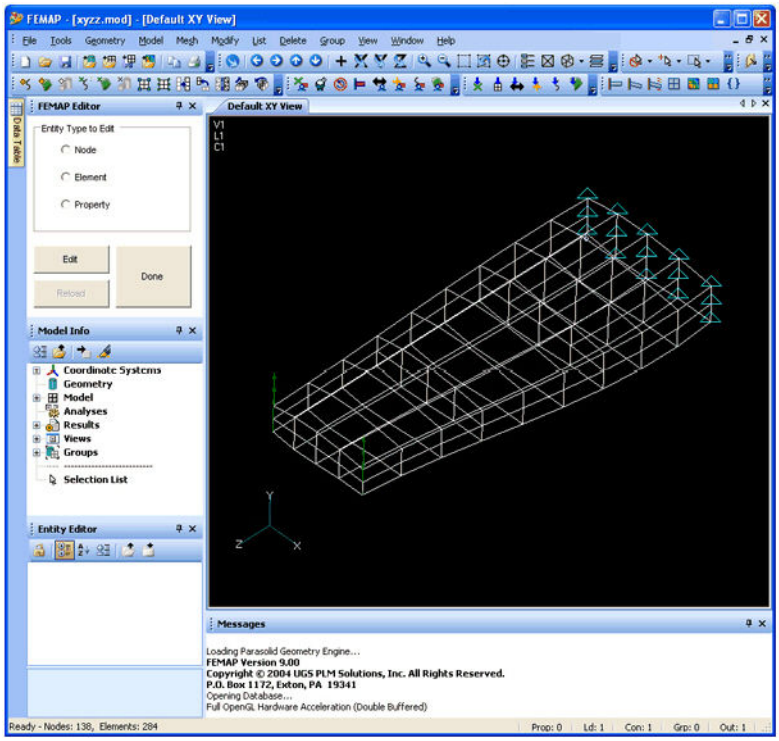
When you are running Femap in this mode, in general you will not want to call the numerous Femap API calls that optionally display dialog boxes to ask for user input. If you must call methods that display dialog boxes or message boxes, you can control whether or not they are displayed using the **DialogAutoAnswer** variable. By default the boxes will just be cancelled, however if you choose to show the dialog boxes, these methods will still work, and dialog boxes will be displayed, (although their placement may be somewhat random), even though Femap is hidden. If you want to work in a totally hidden mode, with nothing displayed, you simply have to set the return code you want - dialog boxes that collect additional user input must be displayed if you want the data to be properly defined.

Running your application as a Femap add-in

If you want your application to become a Femap *add-in*, then you need to follow several additional steps. In this case, the overall interface that will be visible to the user will be Femap. Your application will become part of that interface. You will probably want to build your application with its main window as a dialog box.

To begin, simply follow the steps described above for running as two separate, visible applications. Call `GetObject` or `CreateObject` to attach to Femap, and call `feAppVisible` if you created a new Femap session. Finally, you will want to call `feAppRegisterAddInPane` (see **feAppRegisterAddInPane**). In this call you will specify the handles to your application window, and where in the Femap user interface you want it placed. Using this capability, you can build a user interface that looks something like:

Unpublished work. © 2025 Siemens



In this case the *add-in* application (Femap Editor) has been added to a pane in the Femap user interface in the upper-left corner, using the following method:

```
rc = App.feAppRegisterAddInPane(True, Form1.DefInstance.Handle.ToInt32, 0,
False, False, 3, 2)
```

> **Note**
>
> The messageID window was not specified, because events are not being used.

If you have registered your application as a Femap add-in, remember to call `feAppRegisterAddInPane` again to *unregister* your application before it exits.

Embedding Femap in your application

Instead of embedding your application in the Femap user interface, you might also want to embed parts of Femap into your interface. To do this, you embed the entire Femap window, and then you can selectively turn off/hide various portions of the user interface, until you are left with the parts you want. Probably the most useful capability is that the Femap graphics windows can be embedded directly into one of your windows. This can provide you all of the graphical capabilities of Femap without having to reimplement them yourself, and with a very simple setup.

Unpublished work. © 2025 Siemens

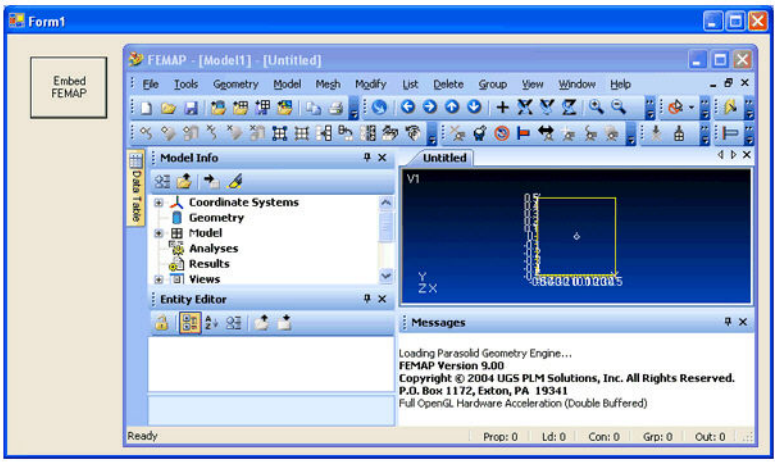
Once again, you will connect to the Femap session using either `GetObject` or `CreateObject`. You will probably want to use `CreateObject` in most cases, however an existing Femap session can be embedded into your application. Even though this is possible however, you should not normally *grab* a user's session and change it that dramatically.

Embedding Femap in your Application

To embed the Femap windows into your application, you simply call the `feAppEmbed` method. Here you specify the handle to your application window where you want the graphics embedded, the portion of that window to use for Femap. Here is a sample that embeds Femap when the button is pushed:

```
Dim femap As Object
Dim rc As Long

Private Sub Command1_Click()
   Set femap = CreateObject("femap.model")
   rc = femap.feAppEmbed(Me.Handle, 120, 10, 660, 420)
End Sub
```

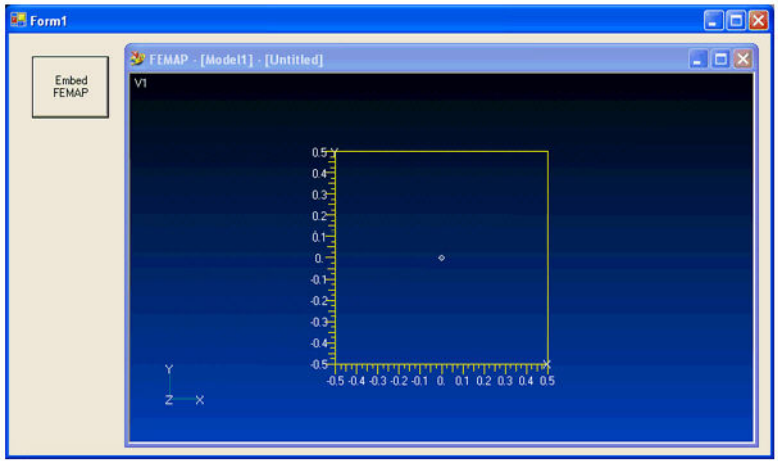


Alternatively, with a few more calls, you can turn off various parts of the Femap interface to leave just the graphics window.

```
Dim femap As Object
Dim rc As Long

Private Sub Command1_Click()
   Set femap = CreateObject("femap.model")
   rc = femap.feAppManageToolbars("", False)
   rc = femap.feAppManageStatusBar(False)
   rc = femap.feAppManagePanes("", 0)
   rc = femap.feAppManageGraphicsTabs(False)
   rc = femap.feAppEmbed(Me.Handle, 120, 10, 660, 420)
End Sub
```


Unpublished work. © 2025 Siemens



As your application window is moved around the screen the Femap windows will move with it, staying visible in the same relative location. If you want the Femap windows to resize as you resize your window, then you need to update the Femap window as you are resizing. During resizing, simply keep calling `feAppEmbed` with new coordinates and sizes. You can still maximize Femap, and if you do, it will fill the client area of your window, possibly covering other items (like the button above). You can control the look of the Femap graphics windows by modifying the various view options prior to making a view visible.

> **Note**
>
> You do not want to call `feAppVisible` if you want to run in this mode. Calling `feAppEmbed` will automatically make the windows visible, and calling `feAppVisible` first will just cause unnecessary flashing of the windows.

Capturing the Femap Messages

In addition to embedding graphics or messages windows, you may want to capture the text that is displayed in the Femap message window and display it in one of your own windows.

Capturing the Femap message stream is a bit more involved than a single method call, however it is not too difficult. It is very similar to the approach needed for capturing Femap events. In addition, you can turn message capture on and off at any time. The procedure to capture messages is as follows:

- Get or Create a Femap object
- Call `feAppRegisterMessageHandler` to provide the handle to a window which will process the messages. This does not have to be the window where the messages will be displayed, but it must be a window with a message loop that you can watch and modify.
- Override the standard Windows procedure for the window that you registered. This requires a call to the Windows function `SetWindowsLong`, setting the `GWL_WNDPROC` constant. You need to Declare this Windows function and define the constant before accessing it.

Unpublished work. © 2025 Siemens

- Write the new Windows procedure to process the WM_COPYDATA messages that you will be receiving from Femap. You should also pass all other messages along to the standard Windows procedure that you overrode. In this procedure, every time you receive a WM_COPYDATA message, you will extract the text string (and possibly the color) from the message, process it in any way that you choose, and display it wherever you want.
- To stop capturing messages, or when you exit your application, call feAppRegisterMessage-Handler again to *unregister* your window and stop Femap from sending WM_COPYDATA messages. You should also replace the original Window message procedure.

Format of the WM_COPYDATA messages

Since Windows only passes messages with wParam and lParam values, both of which are *long* values, it is difficult to pass a general text string through a message. The WM_COPYDATA message however was created to allow interprocess communication of other data types. In this message, the lParam value contains the address of a data structure defined as follows:

```
Type COPYDATASTRUCT
  dwData As Long
  cbData As Long
  lpData As Long
End Type
```

cbData contains the number of bytes of data in lpData. Femap uses this to pass the text string for each message in the lpData portion of the structure. You can use the Windows CopyMemory function to move the data in the structure to a variable defined as a COPYDATASTRUCT, and then again to extract the string data from lpData.

In addition to the string data, Femap also passes the message color/status information in the dwData field. Refer to the feAppMessage method (see **feAppMessage**) for more information regarding color values.

Sample Program to Capture and Display Femap Messages

The following program is a fairly simple example of what is required to capture and display the messages from Femap. As you can see, it does require quite a bit more work than simply embedding a messages window, but does provide the flexibility of modifying and displaying the messages in any way you would like.

Each of the declarations in the beginning of this program are necessary for its proper operation. You must have proper access to these global Windows constants and functions. The MsgWindowProc function is used to override the standard Windows message loop and process the WM_COPYDATA messages coming from Femap. In this case, once the message is extracted, it is simply added to a listbox control on the main form.

Unpublished work. © 2025 Siemens

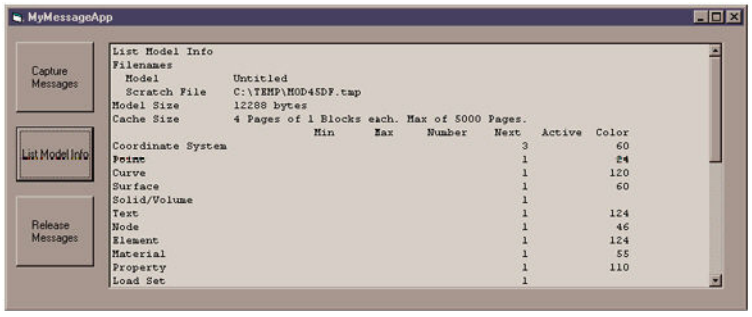
`CaptureMessages` connects to Femap and sets up the message capture, including defining `MsgWindowProc` as the procedure to override message processing. `ReleaseMessages` terminates message capture and can be called either at shutdown or if you simply want to stop capturing messages.

```
Global femap As Object
Global Const GWL_WNDPROC = (-4&)
Global Const GWL_HWNDPARENT = (-8)
Global Const WM_COPYDATA = &H4A
Global PreviousWindowProc As Long
'=============================================================
Type COPYDATASTRUCT
  dwData As Long
  cbData As Long
  lpData As Long
End Type
'=============================================================
' Public Declarations to provide access to Windows API functions
'-------------------------------------------------------------
Public Declare Function GetWindowLong Lib "user32" _
       Alias "GetWindowLongA" (ByVal hwnd As Long, _
       ByVal nIndex As Long) As Long
Public Declare Function RegisterWindowMessage Lib "user32" _
      Alias "RegisterWindowMessageA" (ByVal lpString As String) _
       As Long
Public Declare Function SetWindowLong Lib "user32" _
       Alias "SetWindowLongA" (ByVal hwnd As Long, _
       ByVal nIndex As Long, ByVal dwNewLong As Long) _
       As Long
Public Declare Function CallWindowProc Lib "user32" _
       Alias "CallWindowProcA" (ByVal lpPrevWndFunc As Long, _
       ByVal hwnd As Long, ByVal Msg As Long, _
       ByVal wParam As Long, ByVal lParam As Long) As Long
Public Declare Sub CopyMemory Lib "kernel32" _
       Alias "RtlMoveMemory" _
       (hpvDest As Any, hpvSource As Any, ByVal cbCopy As Long)
'=============================================================
' Override Window procedure and process WM_COPYDATA messages
'-------------------------------------------------------------
Public Function MsgWindowProc(ByVal hwnd As Long, _
       ByVal uMsg As Long, ByVal wParam As Long, _
       ByVal lParam As Long) As Long
  Dim rc As Long
  Dim cds As COPYDATASTRUCT
  Dim buf(1 To 255) As Byte
  If uMsg = WM_COPYDATA Then
     Call CopyMemory(cds, ByVal lParam, Len(cds))
     Call CopyMemory(buf(1), ByVal cds.lpData, cds.cbData)
     a$ = StrConv(buf, vbUnicode)
     a$ = Left$(a$, InStr(1, a$, Chr$(0)) - 1)
      'Add to Listbox
     Form1.List1.AddItem (a$)
     MsgWindowProc = True
  Else
     MsgWindowProc = CallWindowProc(PreviousWindowProc, _
```


Unpublished work. © 2025 Siemens

```
                                hwnd, uMsg, wParam, lParam)
  End If
End Function
'================================================================
' Initialize FEMAP object and capture messages
'----------------------------------------------------------------
Public Sub CaptureMessages()
   Dim rc As Long
   Set femap = GetObject(, "femap.model")
   rc = femap.feAppRegisterMessageHandler(True, Form1.hwnd)
   PreviousWindowProc = SetWindowLong(Form1.hwnd, GWL_WNDPROC, _
                                      AddressOf MsgWindowProc)
End Sub
'================================================================
' Stop capturing messages
'----------------------------------------------------------------
Public Sub ReleaseMessages()
   Dim rc As Long
   If PreviousWindowProc Then
       rc = femap.feAppRegisterMessageHandler(False, Form1.hwnd)
       rc = SetWindowLong(Form1.hwnd, GWL_WNDPROC, _
                                  PreviousWindowProc)
       PreviousWindowProc = 0
   End If
End Sub
'================================================================
```

When combined with a Basic Form, this code drives an application that looks like:



In addition to the code shown above, the following code connects the buttons on the form to the previous code.

```
Private Sub Capture_Click()
  Call CaptureMessages
End Sub
Private Sub ListInfo_Click()
  rc = femap.feFileProgramRun(False, True, False, "{LI}")
End Sub
Private Sub Release_Click()
   Call ReleaseMessages
End Sub
Private Sub Form_Unload(Cancel As Integer)
```


Unpublished work. © 2025 Siemens

```
   Call ReleaseMessages
End Sub
```

API Performance

As described in the previous sections, there are many ways to integrate your API application with Femap. No matter which of these methods you use however, the overall performance of your application can vary greatly depending on how you implement the integration. Whenever it is visible, the Femap user interface is being continually updated to reflect the changing states of your models. While this is very desirable in most situations, and certainly desirable when running Femap interactively, it can result in substantial overhead during API calls. There are two basic approaches that you can take to prevent this overhead, and substantially improve the performance of your application:

1. Hide the Femap user interface using `feAppVisible( )`. The user interface is not updated if it is not visible and therefore the API calls will run faster. If you started Femap from your API, then it will start in the invisible state, so if you never call `feAppVisible( True )`, to make it visible, you will not have to hide it. If you attach to a running Femap session, then calling `feAppVisible( False )` will hide it and improve performance. Using this hide/show approach certainly works, however it will result in Femap disappearing and reappearing on the screen which might not be desirable.
2. Lock the Femap user interface using `feAppLock( )`. This is the method that you will typically want to use unless you are launching your own Femap session and running it totally in the invisible state (without calling `feAppVisible( True )`). Using this function simply locks the user interface so that it is not usable, nor updated. This function also has the added benefit of preventing the user from accidentally picking commands or changing anything while your application is running. Depending on what your application does, you will either want to call `feAppLock( )` at the beginning of your application, and `feAppUnlock( )` at the end, or place `feAppLock( )` / `feAppUnlock( )` pairs around API intensive portions of your application (like commands), so that the Femap user interface updates at the end of that portion, but does not slow down the work being performed. You must always pair `feAppLock( )` and `feAppUnlock( )` calls - there must be an equal number of `feAppUnlock( )` calls as there were `feAppLock( )` calls.

A few simple tests using these methods show that using them can improve the performance of your application by a factor of 10 or more in areas where many API calls are being made, so it is worth the time to implement these simple methods.

Femap Object Properties

Methods and Properties

The following sections describe the methods and properties that are available directly from the Femap application object. In general, these provide access to “global” data that applies to your entire model, and access to functionality that is equivalent to many of the interactive menu commands. Access to lower level functionality, like querying and editing of individual entity data, is available through the other objects that you can create from the application object.

Unpublished work. © 2025 Siemens

The Femap object properties are global in nature. Changing their values will have an immediate impact on the Femap session. Use care when changing these (or any other) properties.

In these tables, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

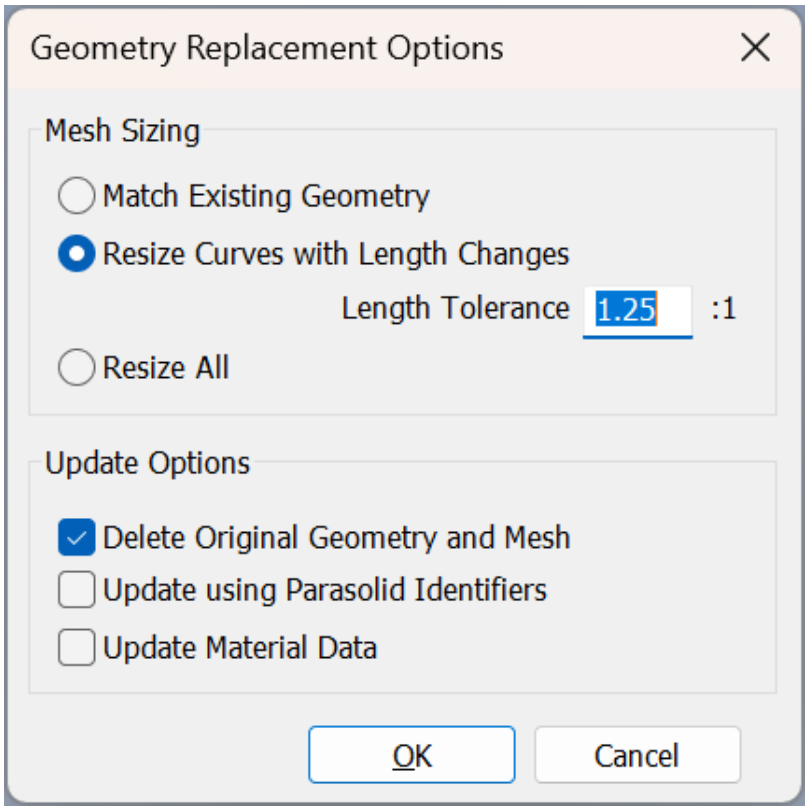
Global Properties - File menu commands

These Global Properties are used to set options for various commands on the **File** menu in Femap. These commands are **File → Import → Geometry**; **File → Page Setup**; and **File → Preferences** in Femap. Depending on the Global property, it may require an Integer (`INT4`), Real (`REAL8`), Boolean (`BOOL`), or String (`STRING`). The type is noted for each Global Property. Each tab of the Preferences dialog box is discussed in greater detail in the following topics.

Unpublished work. © 2025 Siemens

File → Import → Geometry - Geometry Replacement Options

These Global Properties set the options on the Geometry Replacement Options dialog box after a file has been selected for import and the **Update Existing Geometry** option is available:



Geometry Replacement Options		
Property	**Description**	**Type**
`Solid_UpdateActive`	If True, active Parasolid geometry will be updated with the geometry being imported. This option is actually on the Solid Model Read Options dialog box and allows access to the other options.	`BOOL`
Mesh Sizing		
`Solid_UpdateResizeMode`	The type of mesh resizing to do if you are updating Parasolid geometry on import: 0=Keep existing sizes, 1=Update sizes on curves that exceed a certain length change tolerance, 2=Resize all.	`INT4`

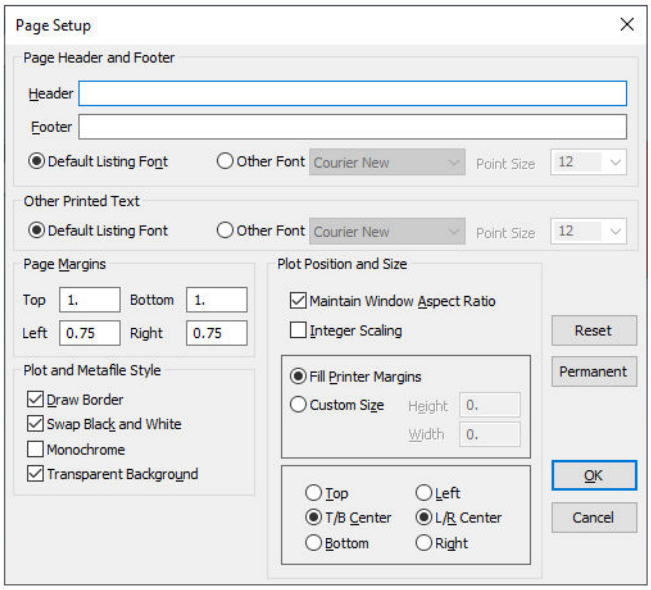
Unpublished work. © 2025 Siemens

Geometry Replacement Options		
Property	**Description**	**Type**
`Solid_UpdateResizeTol`	The tolerance above which curve mesh sizes will be recomputed when curves change length during import of updated Parasolid geometry. Only used if the appropriate `Solid_UpdateResizeMode` is used.	`REAL8`
Update Options		
`Solid_UpdateDelete`	If True, active Parasolid geometry and any mesh on that geometry will be deleted if updated geometry is imported. Only used if `Solid_UpdateActive` is also True.	`BOOL`
`Solid_UpdateParasolidIdent`	If True, the geometry will be updated using Parasolid Identifiers. Only used if `Solid_UpdateActive` is also True.	`BOOL`
`Solid_UpdateMaterialData`	If True, the existing geometry will be updated to use the Material Data of the imported solid. Only used if `Solid_UpdateActive` is also True.	`BOOL`

Unpublished work. © 2025 Siemens

File → Page Setup

These Global Properties are used to set the options on the Page Setup dialog box:



Page Setup Properties		
Property	**Description**	**Type**
Page Header and Footer		
`PgSetup_Header`	Text for Print header.	`STRING`
`PgSetup_Footer`	Text for Print footer.	`STRING`
`PgSetup_DefaultHdrFont`	If True, use the default system font for header/footer.	`BOOL`
`PgSetup_HdrFont`	Name of font to use for header/footer. Only used if `PgSetup_DefaultHdrFont`=False.	`STRING`
`PgSetup_HdrFontSize`	Size in points of the header/footer font.	`INT4`
Other Printed Text		
`PgSetup_DefaultTextFont`	If True, use the default system font for header/footer.	`BOOL`
`PgSetup_TextFont`	Name of font to use for other text. Only used if `PgSetup_DefaultTextFont`=False.	`STRING`
`PgSetup_TextFontSize`	Size in points of the font for printed text.	`INT4`

Unpublished work. © 2025 Siemens

Page Setup Properties		
Property	**Description**	**Type**
Page Margins		
PgSetup_LeftMargin	Page margins.	REAL8
PgSetup_RightMargin		REAL8
PgSetup_TopMargin		REAL8
PgSetup_BottomMargin		REAL8
Plot and Metafile Style		
PgSetup_PrintBorder	If True, a border is drawn around the plot.	BOOL
PgSetup_SwapBW	If True, black and white colors are reversed for printing and metafiles.	BOOL
PgSetup_Monochrome	If True, print is done in monochrome.	BOOL
PgSetup_TransparentBkgd	If True, the background is not filled in a printer resolution print or Metafile.	BOOL
PgSetup_PenScale	Has been removed. Use Pref_ResPenMethod =1 and Pref_ResPenScale instead	INT4
PgSetup_RenderScale	Has been removed. Use Pref_ResCopySaveMethod =1 and Pref_ResCopySaveScale instead	REAL8
Plot Position and Size		
PgSetup_MaintainAspect	If True, plots will maintain the same aspect ratio as the on-screen window.	BOOL
PgSetup_IntegerScaling	If True, the plotting dimensions in device units (pixels) must be an integer multiple of the dimensions of the on-screen window. This is primarily used for printing bitmaps with the highest possible accuracy.	BOOL
PgSetup_FillMargins	If True, plot will fill the printer margins. If False, use the custom height and width.	BOOL
PgSetup_CustomHeight	Height and width for custom print size (only used if PgSetup_FillMargins = False).	REAL8
PgSetup_CustomWidth		REAL8

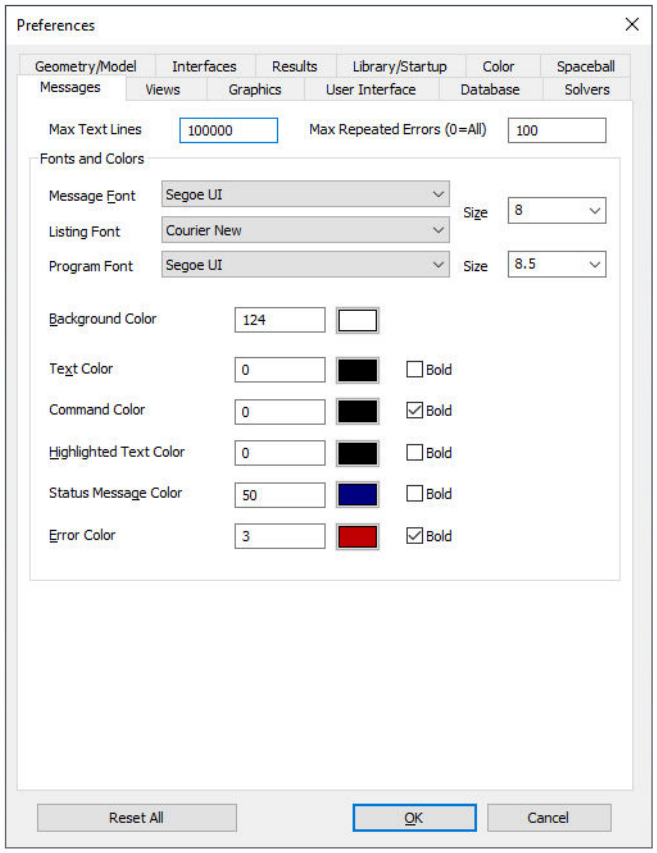
Unpublished work. © 2025 Siemens

Page Setup Properties		
Property	**Description**	**Type**
`PgSetup_VertAlign`	Vertical alignment: T/B Center=0, Top=1, Bottom=2.	`INT4`
`PgSetup_HorzAlign`	Horizontal alignment: L/R Center=0, Left=1, Right=2.	`INT4`

Unpublished work. © 2025 Siemens

File → Preferences - Messages tab

These Global Properties are used to set the options on the **Messages** tab of the Preferences dialog box:



Messages Preference Properties		
Property	**Description**	**Type**
General		
`Pref_ScrollBackLines`	Number of lines of text to be saved for *scroll-back* in the Messages and Lists window.	`INT4`
`Pref_MsgWndRepeatedErrors`	Number of times an error message will be repeated before listing stops. A final message will be listed with an error count.	`INT4`
Fonts and Colors		
`Pref_MessageFont`	Name of font used for messages (other than listings) in the Message Window.	`STRING`

Unpublished work. © 2025 Siemens

Messages Preference Properties		
Property	**Description**	**Type**
`Pref_MsgWndFontSize`	Message window font size in points.	`REAL8`
`Pref_ListingFont`	Name of font (fixed-pitch) used for listings in the Message Window.	`STRING`
`Pref_ProgramFont`	Name of font used for API and Program File windows.	`STRING`
`Pref_ProgramFontSize`	Font size of the font used for the API and Program File panes	`REAL8`
`Pref_MsgWndBackColor`	Message window **Background Color**.	`INT4`
`Pref_MsgWndTextColor`	Message window **Text Color**.	`INT4`
`Pref_MsgWndTextBold`	If True, normal Text written to the Message Window uses a bold font	`BOOL`
`Pref_MsgWndCmdColor`	Message window **Command Color**	`INT4`
`Pref_MsgWndCmdBold`	If True, Command names written to the Message Window use a bold font	`BOOL`
`Pref_MsgWndHighColor`	Message window **Highlighted Text Color**.	`INT4`
`Pref_MsgWndHighBold`	If True, Highlighted Text written to the Message Window uses a bold font	`BOOL`
`Pref_MsgWndWarnColor`	Message window **Status Message Color**.	`INT4`
`Pref_MsgWndWarnBold`	If True, Status Messages written to the Message Window use a bold font	`BOOL`
`Pref_MsgWndErrorColor`	Message window **Error Color**.	`INT4`

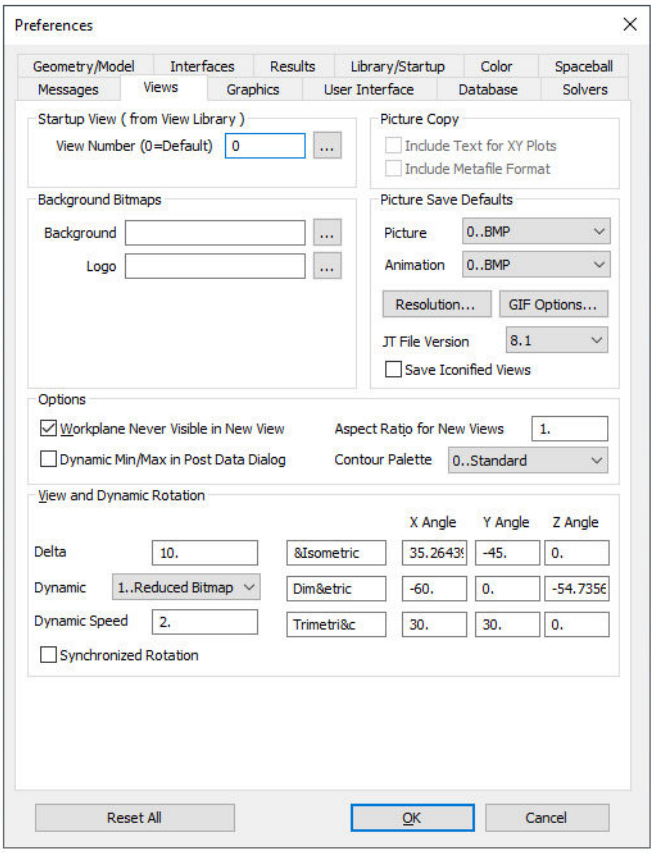
Unpublished work. © 2025 Siemens

Messages Preference Properties		
Property	**Description**	**Type**
`Pref_MsgWndErrorBold`	If True, Error messages written to the Message Window use a bold font	`BOOL`
`Pref_MsgWndFont`	Name of fixed-pitch font to be used in Messages and Lists window.	`STRING`

Unpublished work. © 2025 Siemens

File → Preferences - Views tab

These Global Properties are used to set the options on the **Views** tab of the Preferences dialog box:



Views Preference Properties		
Property	**Description**	**Type**
Startup View (from View Library)		
`Pref_LibViewNumber`	Index of default view in the view library.	`INT4`
Background Bitmaps (Render Only)		
`Pref_ViewBkgdBitmap`	File name for the **Background** bitmap.	`STRING`
`Pref_ViewLogo`	File name for the **Logo** bitmap.	`STRING`
`Pref_ViewLogoScreenRes`	If True, the View **Logo** is defined in Screen Resolution, and will be scaled when printed. If False, it is in Print Resolution and is scaled for Screen display.	`BOOL`

Unpublished work. © 2025 Siemens

Views Preference Properties		
Property	**Description**	**Type**
Picture Copy		
`Pref_PictureCopyTextForXY`	Controls whether comma-separated text is sent to the clipboard when an XY plot is copied. True = **Include Text for XY Plots**.	`BOOL`
`Pref_MetafileToClip`	Controls whether a Windows Metafile is sent to the clipboard when the graphics area is copied. True = **Include Metafile Format**.	`BOOL`
Picture Save Defaults		
`Pref_PictureFormat`	Controls the default picture format for **Picture** save. 0=BMP, 1=JPG, 2=PNG, 3=GIF, 4=TIF	`INT4`
`Pref_AnimationFormat`	Controls the default animation format for **Picture** save. 0=BMP, 1=BMP series, 2=AVI, 3=GIF	`INT4`
`Pref_JTFileVersion`	Sets the default file version for JT files	`INT4`
`Pref_RenderCaptureIconified`	If True, Iconified views can be captured when saving a picture.	`BOOL`
Picture Save Defaults (Resolution... Options)		
`Pref_ResPrintMethod`	Specifies the **Print Resolution** method (0=Printer, 1=Screen, 2=Screen Scaled By)	`INT4`
`Pref_ResPrintScale`	Scale factor used by **Screen Scaled By** method in the **Print Resolution** section	`REAL8`
`Pref_ResCopySaveMethod`	Specifies the **Copy/Save Resolution** method (0=Screen, 1=Screen Scaled By, 2=Screen Scaled With Width, 3=Screen Scaled With Height, 4=Fixed Size)	`INT4`
`Pref_ResCopySaveScale`	Scale factor used by **Screen Scaled By** method in the **Copy/Save Resolution** section	`REAL8`

Unpublished work. © 2025 Siemens

Views Preference Properties		
Property	**Description**	**Type**
`Pref_ResScaleWithWidth`	Resolution used by **Screen Scaled With Width** method in the **Copy/Save Resolution** section	`INT4`
`Pref_ResScaleWithHeigh t`	Resolution used by **Screen Scaled With Height** method in the **Copy/Save Resolution** section.	`INT4`
`Pref_ResFixedWidth`	Resolution used by **Fixed Size** method to set the **Width** in the **Copy/Save Resolution** section	`INT4`
`Pref_ResFixedHeight`	Resolution used by **Fixed Size** method to set the **Height** in the **Copy/Save Resolution** section	`INT4`
`Pref_ResPenMethod`	Specifies the **Pen Width** method (0=Auto, 1=Manual Factor)	`INT4`
`Pref_ResPenScale`	Sets **Line** value in the **Pen Width** and **Font Size** section	`INT4`
`Pref_ResFontScale`	Sets **Font** value in the **Pen Width** and **Font Size** section	`INT4`
`Pref_ResScreenLogoScal e`	Sets the value for **Screen Scale** in the **Logo and Background Bitmap Scaling** section	`REAL8`
`Pref_ResPrintLogoScale`	Sets the value for **Print Scale** in the **Logo and Background Bitmap Scaling** section	`REAL8`
`Pref_ResCopySaveLogoSc ale`	Sets the value for **Copy/Save Scale** in the **Logo and Background Bitmap Scaling** section	`REAL8`
Picture Save Defaults (GIF Options...)		
`Pref_GIFColorOpt`	The color options for creating a GIF picture file. 1=Network, 2=Octree, 3=Color Diffusion (Dither)	`INT4`

Unpublished work. © 2025 Siemens

Views Preference Properties		
Property	**Description**	**Type**
`Pref_GIFAnimationDelay`	The delay between frames of an animated GIF file	`INT4`
`Pref_GIFOptimized`	Toggles the **Optimized** check box on/off	`BOOL`
`Pref_GIFFrameSeries`	Controls whether Animations will be saved in GIF formats as a series of files or a single animated GIF. True=Save GIF Frame Series.	`BOOL`
Options		
`Pref_WorkplaneNotVisible`	Controls whether the workplane will be visible when you start a new model. True = Workplane Never Visible in New View	`BOOL`
`Pref_AltContourPalette`	Obsolete - see `Pref_DefContourPalette`	`BOOL`
`Pref_DefContourPalette`	**Default Contour Palette** 0=Standard, 1=No Magenta, 2=Temperature, 3=Red,Yellow,Green, 4=Gray	`INT4`
`Pref_AspectRatio`	Aspect ratio for new views.	`REAL8`
(Obsolete) `Pref_2DTensorPlotOverride`	This option has been removed for 11.3 and no longer does anything.	`BOOL`
`Pref_ViewPostDynamicMaxMin`	If True, Max/Min data is automatically shown and updated as you change output sets/vectors in the Post Data dialog box	`BOOL`
View and Dynamic Rotation		
`Pref_DeltaAngle`	Angle for incremental rotation in the **View → Rotate** command.	`REAL8`
`Pref_ViewDynamicMode`	Dynamic rotation mode: 0=Fast Redraw, 1=Reduced Bitmap, 2=Full Bitmap.	`INT4`
`Pref_ViewDynamSpeed`	Speed of dynamic rotation.	`REAL8`

Unpublished work. © 2025 Siemens

Views Preference Properties		
Property	**Description**	**Type**
`Pref_ViewDimetric`	Text for **Dimetric** button in **View → Rotate**.	`STRING`
`Pref_ViewIsometric`	Text for **Isometric** button in **View → Rotate**.	`STRING`
`Pref_ViewTrimetric`	Text for **Trimetric** button in **View → Rotate**.	`STRING`

`Pref_AngleIsometric[0..2]`
or
`VARIANT vPref_AngleIsometric`
	The rotation angles assigned to the corresponding buttons in the **View → Rotate** command.	`REAL8`

`Pref_AngleDimetric[0..2]`
or
`VARIANT vPref_AngleDimetric`
		`REAL8`

`Pref_AngleTrimetric[0..2]`
or
`VARIANT vPref_AngleTrimetric`
		`REAL8`

`Pref_SynchronizedRotation`	If True, the **Synchronized Rotation** option is activated.	`BOOL`
Default View Font (not currently in GUI)		
`Pref_GraphicsFont`	Name of font used for the view (the default is Microsoft Sans Serif)	`STRING`

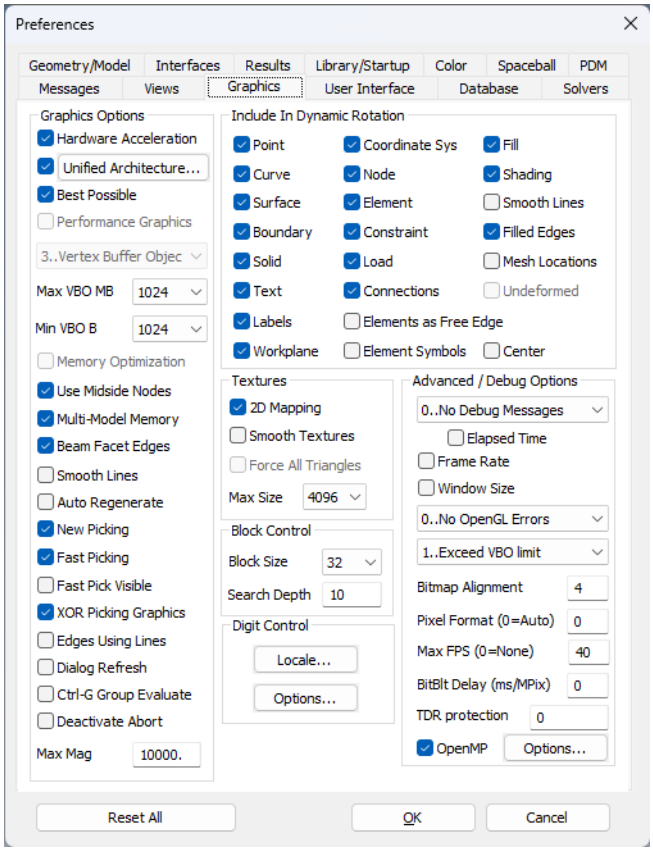
Unpublished work. © 2025 Siemens

Views Preference Properties		
Property	**Description**	**Type**
`Pref_GraphicsFontSize`	View font size in points, between 6 and 72 (allows half sizes)	`REAL8`
Suppress Screen Entities (not currently in GUI)		
`Pref_SuppressScreenEntities`	If True, deactivates active screen entities (View Legend, View Axes, Post Titles, and Contour Legend)	`BOOL`

Unpublished work. © 2025 Siemens

File → Preferences - Graphics tab

These Global Properties are used to set the options on the **Graphics** tab of the Preferences dialog box:



Graphics Preference Properties		
Property	**Description**	**Type**
Graphics Options		
`Pref_RenderHW`	If True, OpenGL will attempt to use **Hardware Acceleration**.	`BOOL`
`Pref_RenderBestAvailable*`	If True, automatically queries the computer and automatically specifies option for **Performance Graphics** and **Memory Optimization**, as well as selected values for the Vertex Arrays drop-down, **Max VBO MB**, and **Min VBO B**	`BOOL`
`Pref_PerformanceGraphics*`	If True, **Performance Graphics** mode is enabled	`BOOL`

Unpublished work. © 2025 Siemens

Graphics Preference Properties		
Property	**Description**	**Type**
`Pref_UnifiedArchitecture`	If True, **Unified Architecture** option is turned on to draw picking marker symbols quicker.	`BOOL`
`Pref_UnifiedArchitectureOpenGL`	Specifies the options in the Unified Architecture Options dialog box accessed by the **Unified Architecture** button on the **Graphics** tab of the **File → Preferences** command.	`INT4`
`Pref_RenderVertexArrays*`	0=No use of Vertex Arrays, 1=Partial, 2=Full for all graphics	`BOOL`
`Pref_RenderMaxVBOMB`	Sets the maximum amount of memory that can be used by VBOs – if this is exceeded, any remaining graphics data is drawn using vertex arrays	`INT4`
`Pref_RenderMinVBOB`	Any block of graphics data smaller than this value will not use VBOs as large numbers of small VBOs is not more efficient than vertex arrays	`INT4`
`Pref_RenderMemoryOpt*`	If True, Graphics **Memory Optimization** is on	`BOOL`
`Pref_RenderMidside`	If True, Render mode will **Use Midside Nodes** when drawing elements.	`BOOL`
`Pref_RenderMultiModelMem`	If True, **Multi-Model Memory** is on	`BOOL`
`Pref_RenderBeamEdges`	Controls drawing of **Beam Facet Edges** between cross sections of beams	`BOOL`
`Pref_RenderSmoothLines`	If True, uses antialiasing to render smooth lines	`BOOL`

Unpublished work. © 2025 Siemens

Graphics Preference Properties		
Property	**Description**	**Type**
`Pref_RenderAutoRegen`	If True, graphics are completely regenerated after every command	`BOOL`
`Pref_NewPick`	If True, activates picking technology implemented for version 2021.1 and above	`BOOL`
`Pref_RenderFastPick`	If True, activates faster, in-memory selection of entities	`BOOL`
`Pref_RenderFastPickVisible`	If True, activates fast front pick and also allows front picking to work when area picking.	`BOOL`
`Pref_RenderXORPicking`	If True, **XOR Picking Graphics** in on	`BOOL`
`Pref_RenderEdgesAsLines`	If True, element and surface edges are drawn as lines rather than using edge flags.	`BOOL`
`Pref_RenderDialogRefresh`	If True, **Dialog Refresh** in on	`BOOL`
`Pref_RenderCtrlGGroupEvaluate`	If True, **Ctrl-G Group Evaluate** option is activated, which performs **Group → Operations → Evaluate** on all visible groups in the active model	`BOOL`
`Pref_RenderDisableAbort`	If True, **Disable Abort** is activated, which disables the ability to abort a redraw of the graphics window, which can improve redraw performance	`BOOL`
`Pref_RenderMaxMagnification`	Sets the value for **Max Mag**, must be between 10,000 and 1.0E+15	`REAL8`
`Pref_UnifiedGraphics`	If True, Unified Architecture is activated, which draws marker symbols quicker. Implemented geometry in Unified Architecture includes Points, Curves, Composite Curves, Surfaces, Boundaries, volumes/	`BOOL`

Unpublished work. © 2025 Siemens

Graphics Preference Properties		
Property	**Description**	**Type**
	solids and geometry-based loads and constraints.	
Include In Dynamic Rotation		

`Pref_RenderRotate[0..22]*`
 	
or
 	
`VARIANT vPref_RenderRotate`
	The flags used to indicate which entities will be on for dynamic rotation. 0-Point, 1-Curve, 2-Surface, 3-Boundary, 4-Solid, 5-Text, 6-Coordinate Sys, 7-Node, 8-Element, 9-Constraint, 10-Load, 11-Connections, 12-Fill, 13-Filled Edges, 14-Shading, 15-Undeformed, 16-Labels, 17-Workplane, 18-Mesh Locations, 19-Smooth Lines, 20-Elements as Free Edge, 21-Element Symbols, 22-Center	`BOOL`

Textures		
`Pref_RenderTex2D`	If True, uses **2D Mapping**	`BOOL`
`Pref_RenderTexSmooth`	If True, uses **Smooth Textures**	`BOOL`
`Pref_RenderTexAllTri*`	If True, activates **Force All Triangles** for texture mapping	`BOOL`
`Pref_RenderTexNumber`	Max Number of Textures	`INT4`
Block Control		
`Pref_RenderBlockSize`	Sets the **Block Size**. Available values are 4, 8, 16, 32, 64, 128, 256, 512, and 1024	`INT4`
`Pref_RenderSearchDepth`	**Search Depth** for adding entities to graphics system	`INT4`
Digit Control - Locale...		
`Pref_DL_UseIntIAll`	Sets On/Off for User Region Separator Style/(Negative/ Values).	`BOOL`
`Pref_DL_DecimalSeparator`	Sets the option for Decimal Separator.	`INT4`

Unpublished work. © 2025 Siemens

Graphics Preference Properties		
Property	**Description**	**Type**
	0 = 1 234 1 = 1.234 2 = 1,234 3 = 1’234	
`Pref_DL_DigitGroupSeparator`	Sets the option for Digit Group Separator. 0 = 123 456 1 = 123.456 2 = 123,456 3 = 123’456	`INT4`
`Pref_DL_DigitGroupOption`	Sets the option for Separator Style. 0 = 1234567 1 = 1,234,567 2 = 1234,567 3 = 12,34,567	`INT4`
`Pref_DL_DigitGroupFlag`	Sets On/Off for Separator after the decimal.	`BOOL`
`Pref_DL_ExponentChar`	Sets the Exponent Character. 0 = E 1 = e 2 = D 3 = d 4 = F 5 = f	`INT4`
`Pref_DL_ForceSingleZero`	Sets On/Off for Force Single Zero.	`BOOL`
`Pref_DL_ForcePlusSign`	Sets On/Off for + Sign.	`BOOL`
`Pref_DL_NegativeBraces`	Sets On/Off for (Negative Value).	`BOOL`
`Pref_DL_TrailingZeros`	Sets On/Off for Remove Trailing Zeros.	`BOOL`
Digit Control - Options		

Unpublished work. © 2025 Siemens

Graphics Preference Properties		
Property	**Description**	**Type**
`Pref_DO_Option`	Sets the Method in Digit Options. 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic	`INT4`
`Pref_DO_RoundToZero`	Sets On/Off for Round To Zero.	`BOOL`
`Pref_DO_RoundToZeroValue`	Sets value for Round To Zero.	`REAL8`
`Pref_DO_BeforeDecimalSeparator`	Sets the before Exponent Style. 0 = .123 1 = 0.123 2 = 1.123	`INT4`
`Pref_DO_MinimumExponentSize`	Sets the minimum exponent size. 0 = No E+0 1 = e+1 2 = e+01 3 = e+001	`INT4`
`Pref_DO_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.	`INT4`
`Pref_DO_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15	`INT4`
`Pref_DO_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.	`INT4`
`Pref_DO_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.	`INT4`
`Pref_DO_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.	`INT4`

Unpublished work. © 2025 Siemens

Graphics Preference Properties		
Property	**Description**	**Type**
`Pref_DO_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.	`INT4`
`Pref_DO_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.	`INT4`
`Pref_DO_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.	`INT4`
`Pref_DO_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.	`BOOL`
Advanced/Debug Options		
`Pref_RenderTiming`	If True, prints render timings	`BOOL`
`Pref_RenderDebugElapsedTime`	If True, prints elapsed times	`BOOL`
`Pref_RenderDebugAllTime`	If True, prints All times	`BOOL`
`Pref_RenderDebugOGLErrors`	If True, prints OpenGL errors (if there are any)	`BOOL`
`Pref_RenderBitmapAlignment`	**Bitmaps Alignment** stored for OpenGL - do not change.	`INT4`
`Pref_RenderPixelFormat`	Controls selection of Graphics Descriptors - 0=Automatic	`INT4`
`Pref_RenderBitBltDelay`	Value needed for Delay in synchronizing OpenGL and GDI on some graphics cards (0..1000)	`INT4`
`Pref_RenderDebugFrameRate`	This option writes a frames per second calculated from the display time without having to have any of the debug messages on. It gives a	`BOOL`

Unpublished work. © 2025 Siemens

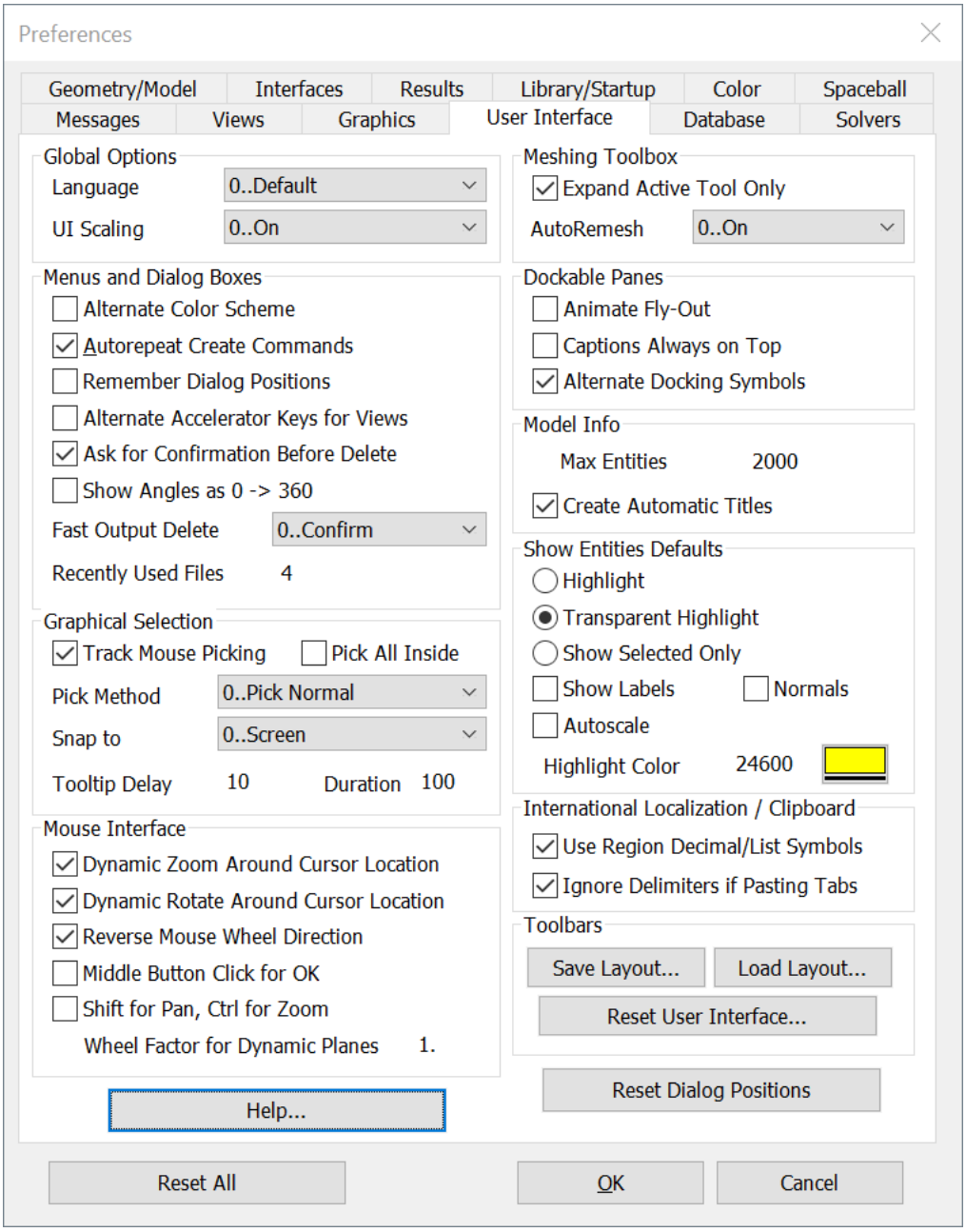
Graphics Preference Properties		
Property	**Description**	**Type**
	more accurate timing while dynamically rotating. Used to determine which graphics preferences give the best performance.	
`Pref_RenderVBOUsage`	Outputs information about memory used by VBOs 0 – No messages 1 – Only when VBO memory exceeded 2 – Output detailed VBO memory usage	`INT4`
`Pref_RenderTDRProtect`	Sets value for **TDR protection**, which allows the graphics driver to have more time to receive messages from Femap.	`INT4`
`Pref_OpenMP`	If True, enables **OpenMP** functionality in Femap (currently only used by functionality which has been coded specifically to take advantage of OpenMP architecture)	`BOOL`
`PrefOpenMPThreads`	Number or threads for **OpenMP** to attempt to use	`INT4`

The `feFilePreferencesSave` call MUST be used after using this option for the preference to be changed correctly. See **feFilePreferencesSave** for more information.

Unpublished work. © 2025 Siemens

File → Preferences - User Interface tab

These Global Properties are used to set the options on the **User Interface** tab of the Preferences dialog box:



User Interface Preference Properties		
Property	**Description**	**Type**
Global Options		
`Pref_GlobalLanguage`	Sets the Language option.	`INT4`

Unpublished work. © 2025 Siemens

User Interface Preference Properties		
Property	**Description**	**Type**
	0=Default (FLNG_AUTO) 1=English (FLNG_EN) 2=German (FLNG_DE) 3=Japanese (FLNG_JP) 4=Chinese, Traditional (FLNG_TW) 5=Chinese, Simplified (FLNG_CN)	
Pref_GlobalUIScaling	Sets the **UI Scaling** option. 0=Enabled, 1=Disabled, 2=Windows	INT4
Menus and Dialog Boxes		
Pref_AltTheme	If True, use an **Alternate Color Scheme** and style for the user interface	BOOL
Pref_RepeatCreatec	If True, Entity Create Commands will repeat (Previously, this was misspelled Pref_RepeateCreate, which has identical functionality, but should not be used)	BOOL
Pref_RememberDialogPos	If True, each dialog box position is remembered during a session and if a dialog box is redisplayed it will be located at the same position as when it was last closed.	BOOL
Pref_AltAccelerator	If True, redefine certain accelerator keys to access Top, Bottom, ... Views.	BOOL
Pref_ConfirmDelete	If True, **Ask for Confirmation Before Delete** is on.	BOOL
Pref_Show0To360Angles	If True, shows angles from 0 to 360 instead of -180 to 180.	BOOL
Pref_FastOutputDelete	Sets the **Fast Output Delete** option. 0=Confirm, 1=Fast (No Undo), and 2=Preserve Undo.	INT4
Pref_RecentlyUsedFiles	Number of files on the most **Recently Used Files** list.	INT4

Unpublished work. © 2025 Siemens

User Interface Preference Properties		
Property	**Description**	**Type**
Graphical Selection		
`Pref_TrackMousePick`	Activates dynamic selection tracking.	`BOOL`
`Pref_PickAllInside`	Controls selection of entities when screen area picking is used.	`BOOL`
`Pref_PickMethod`	Controls the default **Pick Method**. 0=Pick Normal, 1=Pick Query, and 2=Pick Front	`INT4`
`Pref_SnapTo`	Controls the default **Snap Method**. 0=Screen, 1=Grid, 2=Point, 3=Node, 4=Smart	`INT4`
`Pref_TooltipDelay`	Delay before selector tooltips appear (in tenths of a second).	`INT4`
`Pref_TooltipDuration`	Length of time before selector tooltips automatically disappear (in tenths of a second).	`INT4`
Mouse Interface		
`Pref_ZoomAroundCursor`	When on, zooming by scrolling the mouse wheel will zoom about the cursor location.	`BOOL`
`Pref_RotateAroundCursor`	When on, rotation will occur about a location on the model, close to the cursor location.	`BOOL`
`Pref_ReverseMouseWheel`	Controls the direction of zoom when you turn the mouse wheel.	`BOOL`
`Pref_MiddleMouseButtonOK`	When on, the middle mouse button (wheel) can be clicked without moving the mouse to OK dialog boxes.	`BOOL`
`Pref_ReversePanZoom`	If True, hold down Shift to Zoom, and Ctrl for Pan in Dynamic Rotation. If False, reverse the keys.	`BOOL`
`Pref_PlanceWheelFactor`	Adjusts how much the cutting plane moves when holding Ctrl+scrolling the mouse for	`REAL8`

Unpublished work. © 2025 Siemens

User Interface Preference Properties		
Property	**Description**	**Type**
	Dynamic Cutting Plane and Model Cutting Plane.	
Meshing Toolbox		
`Pref_TbxExpandActive`	If True, only the active tool will be expanded	`BOOL`
`Pref_TbxAutomesh`	0=AutoRemesh, 1=Track Changes, 2=Disable Remesh	`INT4`
Dockable Panes		
`Pref_AnimatePanes`	If True, the docking panes will animate when you click on their tab to make them visible.	`BOOL`
`Pref_PaneCaptionsOnTop`	If True, the docking panes will always have their captions placed on the top of the pane. Otherwise they are either on the top or left, whichever takes less space.	`BOOL`
`Pref_PaneAltDockSymbols`	If True, the docking position indicators will be shown using the *original* symbols instead of the *new* symbols.	`BOOL`
`Pref_LegacyXYPlotting`	If True, the **Legacy XY plotting in Views** will be enabled.	`BOOL`
Model Info		
`Pref_ModelInfoMaxEntity`	Limits the max number of items of each type to show in the Model Info tree.	`INT4`
`Pref_AutomaticTitles`	Controls whether automatic titles will be given to otherwise untitled entities.	`BOOL`
Show Entities Defaults		
`Pref_ShowMode`	Sets the Mode for the **Show Entities Defaults**. 0 = Highlight, 1 = Transparent Highlight, and 2 = Show Selected Only	`INT4`

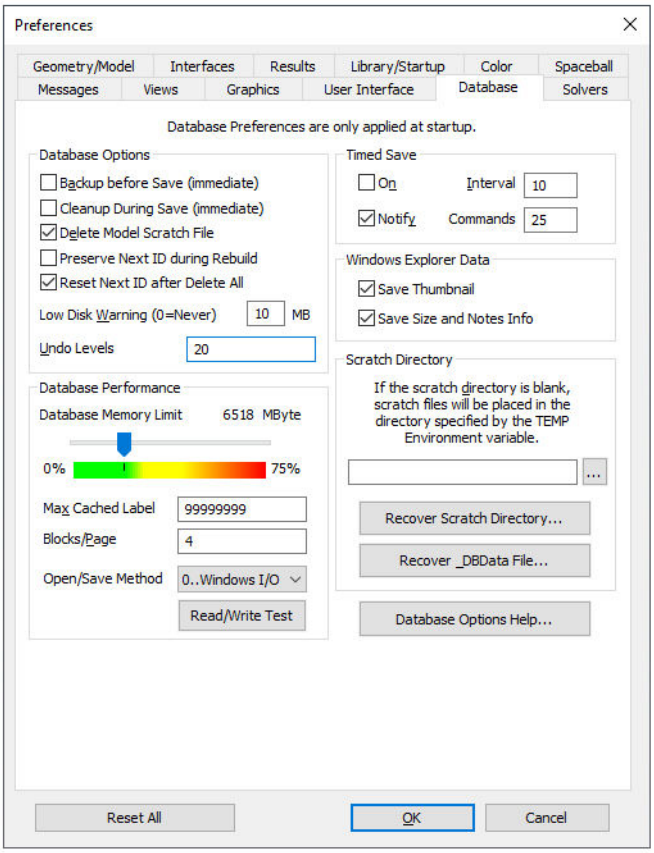
Unpublished work. © 2025 Siemens

User Interface Preference Properties		
Property	**Description**	**Type**
`Pref_ShowLabels`	When True, **Show Labels** option will be on	`BOOL`
`Pref_ShowNormals`	When True, **Show Normals** option will be on	`BOOL`
`Pref_ShowColor`	Sets color value for **Highlight Color**	`INT4`
`Pref_ShowAutoscale`	When True, **Autoscale** option will be on	`BOOL`
International Localization		
`Pref_UseRegionDecimalChar`	When on, uses the appropriate settings "." and "," based on the region being used by Windows OS	`BOOL`
`Pref_TabPasteIgnoreDelimeter`	When on, ignores delimiters when pasting from the clipboard, provided a tab exists between values	`BOOL`
Help... button		
`Pref_OnlineHelp_Mode`	Specifies option for Help Type: 0=Online HTML 1=Local HTML (Requires Siemens Documentation Server)	`INT4`
`Pref_OnlineHelp_Path`	Specifies the text for **Help Location**	`STRING`

Unpublished work. © 2025 Siemens

File → Preferences - Database tab

These Global Properties are used to set the options on the **Database** tab of the Preferences dialog box:



Database Preference Properties		
Property	**Description**	**Type**
Database Options		
`Pref_BackupModel`	If True, any existing model will be copied before each save.	BOOL
`Pref_CleanupModell`	If True, any model will undergo a **File → Rebuild** before being saved to remove unused space.	BOOL
`Pref_DeleteScratchFile`	If True, scratch files are automatically deleted whenever you exit a model.	BOOL
`Pref_PreserveNextID`	When True, **Preserve Next ID during Rebuild** is on	BOOL
`Pref_ResetNextID`	When True, **Reset Next ID after Delete All** is on	BOOL

Unpublished work. © 2025 Siemens

Database Preference Properties		
Property	**Description**	**Type**
`Pref_DBLowDiskWarning`	Issues a warning when free space on the model scratch disk falls below this amount (in **MByte**). 0=Disable.	`INT4`
`Pref_DBUndoLevels`	Maximum number of undo levels.	`INT4`
Database Performance		
`Pref_DBCacheMaxMemoryPct`	Sets max percentage value to use for **Database Memory Limit**	`INT4`
`Pref_DBCacheBlocksPerPage`	Number of blocks per page in the database cache.	`INT4`
`Pref_DBCacheMaxLabel`	Maximum cached database node/element label.	`INT4`
`Pref_OpenSaveMethod`	Choose an **Open/Save Method**. 0=Window I/O, 1=C I/O, 2=Windows 64K I/O, and 3=C 64K I/O. See feRunIOTest for information about running the Read/Write Test to determine the preferred value for **Open/Save Method**	`INT4`
Timed Save		
`Pref_TimedSaveOn`	Turns on **Timed Save** automatically.	`BOOL`
`Pref_TimedSaveNotify`	Sets **Timed Save** to notify you before saving.	`BOOL`
`Pref_TimedSaveInterval`	Number of minutes between timed saves.	`INT4`
`Pref_TimedSaveCommands`	Number of commands between timed saves.	`INT4`
Windows Explorer Data		
`Pref_ModelThumbnail`	When True, thumbnail image shown with model file in Windows Explorer saved with model.	`BOOL`

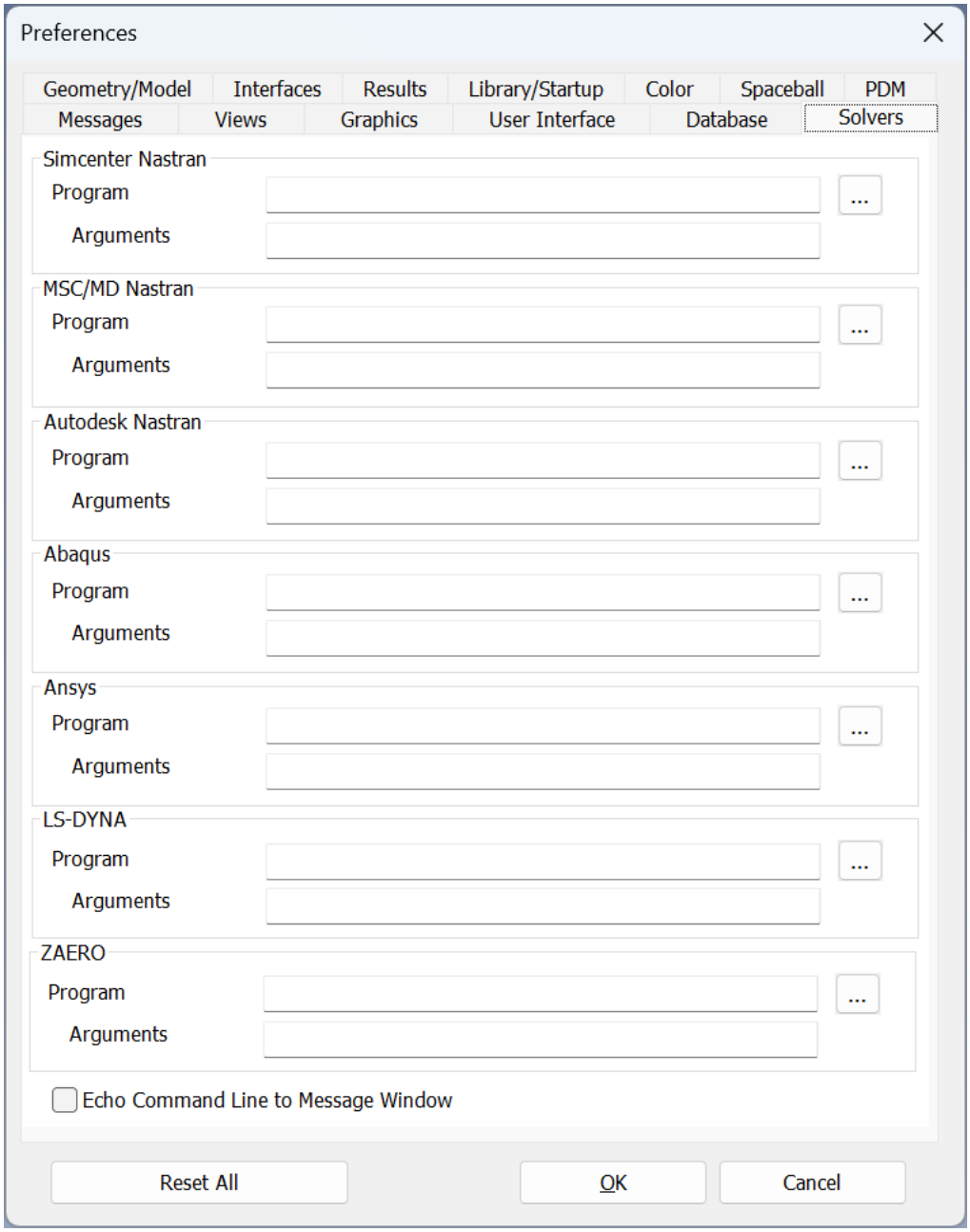
Unpublished work. © 2025 Siemens

Database Preference Properties		
Property	**Description**	**Type**
`Pref_ModelTooltip`	When True, model info for Tooltip available when hovering over model file in Windows Explorer saved with model.	`BOOL`
Scratch Directory		
`Pref_DBScratch`	Path (no filename) for model scratch file.	`STRING`

Unpublished work. © 2025 Siemens

File → Preferences - Solvers tab

These Global Properties are used to set the options on the **Solvers** tab of the Preferences dialog box:



Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
NX Nastran		
Pref_SolNXNastProgram	Specifies path to NX Nastran solver	STRING
Pref_SolNXNastCmd	Command line arguments for NX Nastran	STRING
MSC/MD Nastran		
Pref_SolMSCNastProgram	Specifies path to MSC/MD Nastran solver	STRING
Pref_SolMSCNastCmd	Command line arguments for MSC/MD Nastran	STRING
Autodesk Nastran		
Pref_SolNEINastProgram	Specifies path to Autodesk Nastran solver	STRING
Pref_SolNEINastCmd	Command line arguments for Autodesk Nastran	STRING
Abaqus		
Pref_SolAbaqusProgram	Specifies path to Abaqus solver	STRING
Pref_SolAbaqusCmd	Command line arguments for Abaqus Nastran	STRING
Ansys		
Pref_SolAnsysProgram	Specifies path to Ansys solver	STRING
Pref_SolAnsysCmd	Command line arguments for Ansys Nastran	STRING
LS-DYNA		
Pref_LSDynaProgram	Specifies path to LS-DYNA solver	STRING
Pref_LSDynaCmd	Command line arguments for LS-DYNA Nastran	STRING

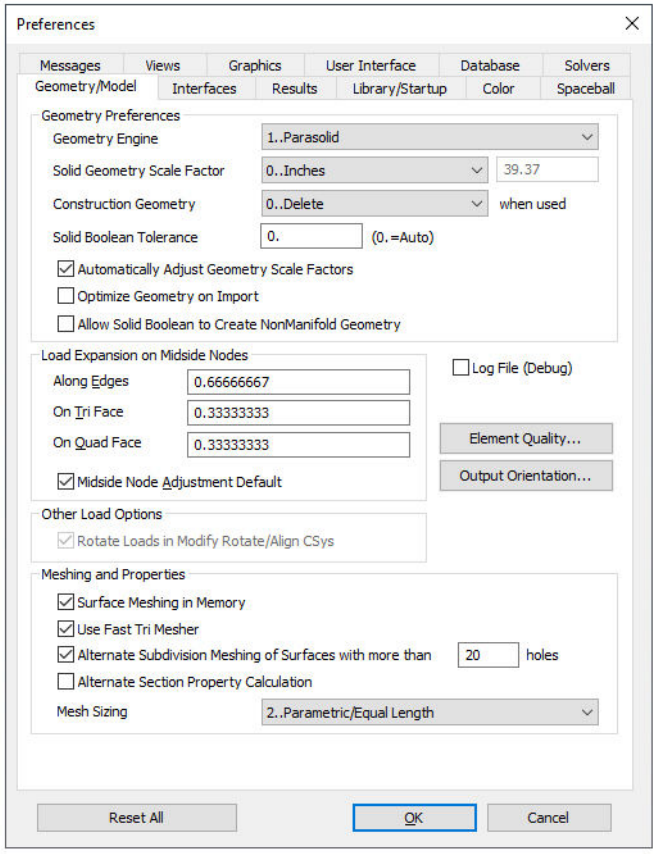
Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
All Solvers		
`Pref_SolEchoCmd`	When on, turns on Echo Command Line to Message Window option	`BOOL`

Unpublished work. © 2025 Siemens

File → Preferences - Geometry/Model tab

These Global Properties are used to set the options on the **Geometry/Model** tab of the Preferences dialog box:



Geometry/Model Preference Properties		
Property	**Description**	**Type**
Geometry Preferences		
`Pref_GeomEngine`	Default geometry engine: 0=Std, 1=Parasolid.	`INT4`
`Pref_GeometryScale`	Solid geometry scale factor used to make sure that geometry fits inside the limited coordinate spaces used by the solid modeling engines. This value is copied to `Info_GeometryScale` when a model is started, and is used throughout that model. Setting this value to:	`REAL8`

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
	39.37 will change the drop-down to 0..Inches 1.0 will change the drop-down to 1..Meters 1000.0 will change the drop-down to 2..Millimeters All other values change the drop-down to 3..Other	
`Pref_ConstructionGeometry`	Default for construction geometry: 0=Delete, 1=Move To No Pick Layer, 2=Do Nothing.	`INT4`
`Pref_GeometryBooleanTolerance`	Tolerance Parasolid uses when performing Boolean operations. A value of 0.0 causes the tolerance to be automatically determined. This value should only be changed from 0.0 when instructed to do so by Femap development or GTAC.	`REAL8`
`Pref_GeomScaleAdjust`	If True, geometry scale factors are automatically adjusted to match the modeling scale factor.	`BOOL`
`Pref_GeomOptimize`	If True, imported geometry is run through additional options in Parasolid to try and simplify the geometry.	`BOOL`
`Pref_NonmanifoldBooleans`	If True, imported geometry is run through additional options in Parasolid to try and simplify the geometry.	`BOOL`
Load Expansion on Midside Nodes		
`Pref_GeomLoadEdgeFactor`	Adjustment factor for load expansion on midside nodes along edges.	`REAL8`
`Pref_GeomLoadTriFactor`	Adjustment factor for load expansion on midside nodes on tri faces.	`REAL8`

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_GeomLoadQuadFactor`	Adjustment factor for load expansion on midside nodes on quad faces.	`REAL8`
`Pref_AdjMidsideGeomLoad`	Enables default adjustment of nodal loads at the midside of parabolic elements during geometry load expansion.	`BOOL`
Log File		
`Pref_GeomLogFile`	Enables a log file for all solid modeling operations (for debugging purposes only).	`BOOL`
Other Load Options		
`Pref_LoadModifyRotate`	Controls whether loads should be rotated or otherwise modified when the mesh or geometry the load is applied on has been copied, scaled, rotated, or reflected. Also, if the model has been modified using **Modify → Move By → CSys**; **Modify → Rotate By → CSys**; **Modify → Align → CSys**; **Modify → Scale → CSys**	`BOOL`
Meshing and Properties		
`Pref_MeshInMemory`	Allows the boundary mesher to allocate additional memory to speed up its operation.	`BOOL`
`Pref_FastTriMesh`	Chooses the **Use Fast Tri Mesher** as the default triangular mesher.	`BOOL`
`Pref_FastHoleMeshing`	Used to enable and set value, “n”, for the **Alternate Subdivision Meshing of Surfaces with more than** *n* holes option. When set to 0, the option is deactivated. When set to > 0, activates the option and the specified integer value is used for *n*.	`INT4`
`Pref_AltSectionProp`	If True, uses the alternate section property calculator.	`BOOL`

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_LengthBasedMeshSize`	0 = 0..Parametric, 1 = 1..Equal Length, 2 = 2..Parametric/Equal Length	`INT4`
Element Quality button - Femap tab		
`Pref_ElemQualAspectRatio`	If True, **Aspect Ratio** element check is on	`BOOL`
`Pref_ElemQualAspectRatioVal`	**Value for Aspect Ratio** element check	`REAL8`
`Pref_ElemQualTaper`	If True, **Taper** element check is on	`BOOL`
`Pref_ElemQualTaperVal`	Value for **Taper** element check	`REAL8`
`Pref_ElemQualAltTaper`	If True, **Alternate Taper** element check is on	`BOOL`
`Pref_ElemQualAltTaperVal`	Value for **Alternate Taper** element check	`REAL8`
`Pref_ElemQualIntAngles`	If True, **Internal Angles** element check is on	`BOOL`
`Pref_ElemQualIntAnglesVal`	Value for **Internal Angles** element check	`REAL8`
`Pref_ElemQualSkew`	If True, **Skew** element check is on	`BOOL`
`Pref_ElemQualSkewVal`	Value for **Skew** element check	`REAL8`
`Pref_ElemQualWarping`	If True, **Warping** element check is on	`BOOL`
`Pref_ElemQualWarpingVal`	Value for **Warping** element check	`REAL8`
`Pref_ElemQualNastranWarping`	If True, **Nastran Warping** element check is on	`BOOL`
`Pref_ElemQualNastranWarpingVal`	Value for **Nastran Warping** element check	`REAL8`

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_ElemQualTetCollapse`	If True, **Tet Collapse** element check is on	BOOL
`Pref_ElemQualTetCollapseVal`	Value for **Tet Collapse** element check	REAL8
`Pref_ElemQualJacobian`	If True, **Jacobian** element check is on	BOOL
`Pref_ElemQualJacobianVal`	Value for **Jacobian** element check	REAL8
`Pref_ElemQualCombined`	If True, **Combined Quality** element check is on	BOOL
`Pref_ElemQualCombinedVal`	Value for **Combined Quality** element check	REAL8
`Pref_ElemQualExplicitTime`	If True, **Explicit Time Step** element check is on	BOOL
`Pref_ElemQualExplicitTimeVal`	Value for **Explicit Time Step** element check	REAL8
Element Quality button - NX Nastran tab		
`Pref_ElemQualQuadSkew`	If True, **QUAD SKEW** element check is on	BOOL
`Pref_ElemQualQuadSkewVal`	Value for **QUAD SKEW** element check	REAL8
`Pref_ElemQualQuadTaper`	If True, **QUAD TAPER** element check is on	BOOL
`Pref_ElemQualQuadTaperVal`	Value for **QUAD TAPER** element check	REAL8
`Pref_ElemQualQuadWarp`	If True, **QUAD WARP** element check is on	BOOL
`Pref_ElemQualQuadWarpVal`	Value for **QUAD WARP** element check	REAL8

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_ElemQualQuadIAMin`	If True, **QUAD IAMIN** element check is on	BOOL
`Pref_ElemQualQuadIAMinVal`	Value for **QUAD IAMIN** element check	REAL8
`Pref_ElemQualQuadAR`	If True, **QUAD AR** element check is on	BOOL
`Pref_ElemQualQuadARVal`	Value for **QUAD AR** element check	REAL8
`Pref_ElemQualQuadIAMax`	If True, **QUAD IAMAX** element check is on	BOOL
`Pref_ElemQualQuadIAMaxVal`	Value for **QUAD IAMAX** element check	REAL8
`Pref_ElemQualTriaSkew`	If True, **TRIA SKEW** element check is on	BOOL
`Pref_ElemQualTriaSkewVal`	Value for **TRIA SKEW** element check	REAL8
`Pref_ElemQualTriaIAMax`	If True, **TRIA IAMAX** element check is on	BOOL
`Pref_ElemQualTriaIAMaxVal`	Value for **TRIA IAMAX** element check	REAL8
`Pref_ElemQualTriaAR`	If True, **TRIA AR** element check is on	BOOL
`Pref_ElemQualTriaARVal`	Value for **TRIA AR** element check	REAL8
`Pref_ElemQualTriaEPLR`	If True, **TRIA AR** element check is on	BOOL
`Pref_ElemQualTriaEPLRVal`	Value for **TRIA AR** element check	REAL8
`Pref_ElemQualTetAspectRatio`	If True, **TETRA AR** element check is on	BOOL

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_ElemQualTetAspectRatioVal`	Value for **TETRA AR** element check	REAL8
`Pref_ElemQualTetEPLR`	If True, **TETRA EPLR** element check is on	BOOL
`Pref_ElemQualTetEPLRVal`	Value for **TETRA EPLR** element check	REAL8
`Pref_ElemQualTetDetJ`	If True, **TETRA DETJ** element check is on	BOOL
`Pref_ElemQualTetDetJVal`	Value for **TETRA DETJ** element check	REAL8
`Pref_ElemQualTetEPIA`	If True, **TETRA EPIA** element check is on	BOOL
`Pref_ElemQualTetEPIAVal`	Value for **TETRA EPIA** element check	REAL8
`Pref_ElemQualHexAspectRatio`	If True, **HEX AR** element check is on	BOOL
`Pref_ElemQualHexAspectRatioVal`	Value for **HEX AR** element check	REAL8
`Pref_ElemQualHexEPLR`	If True, **HEX EPLR** element check is on	BOOL
`Pref_ElemQualHexEPLRVal`	Value for **HEX EPLR** element check	REAL8
`Pref_ElemQualHexDetJ`	If True, **HEX DETJ** element check is on	BOOL
`Pref_ElemQualHexDetJVal`	Value for **HEX DETJ** element check	REAL8
`Pref_ElemQualHexEPIA`	If True, **HEX EPIA** element check is on	BOOL
`Pref_ElemQualHexEPIAVal`	Value for **HEX EPIA** element check	REAL8

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_ElemQualHexWarp`	If True, **HEX WARP** element check is on	BOOL
`Pref_ElemQualHexWarpVal`	Value for **HEX WARP** element check	REAL8
`Pref_ElemQualPenAspectRatio`	If True, **PENTA AR** element check is on	BOOL
`Pref_ElemQualPenAspectRatioVal`	Value for **PENTA AR** element check	REAL8
`Pref_ElemQualPenEPLR`	If True, **PENTA EPLR** element check is on	BOOL
`Pref_ElemQualPenEPLRVal`	Value for **PENTA EPLR** element check	REAL8
`Pref_ElemQualPenDetJ`	If True, **PENTA DETJ** element check is on	BOOL
`Pref_ElemQualPenDetJVal`	Value for **PENTA DETJ** element check	REAL8
`Pref_ElemQualPenEPIA`	If True, **PENTA EPIA** element check is on	BOOL
`Pref_ElemQualPenEPIAVal`	Value for **PENTA EPIA** element check	REAL8
`Pref_ElemQualPenWarp`	If True, **PENTA WARP** element check is on	BOOL
`Pref_ElemQualPenWarpVal`	Value for **PENTA WARP** element check	REAL8
`Pref_ElemQualPyrAspectRatio`	If True, **PYR AR** element check is on	BOOL
`Pref_ElemQualPyrAspectRatioVal`	Value for **PYR AR** element check	REAL8
`Pref_ElemQualPyrEPLR`	If True, **PYR EPLR** element check is on	BOOL

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_ElemQualPyrEPLRVal`	Value for **PYR EPLR** element check	`REAL8`
`Pref_ElemQualPyrDetJ`	If True, **PYR DETJ** element check is on	`BOOL`
`Pref_ElemQualPyrDetJVal`	Value for **PYR DETJ** element check	`REAL8`
`Pref_ElemQualPyrEPIA`	If True, **PYR EPIA** element check is on	`BOOL`
`Pref_ElemQualPyrEPIAVal`	Value for **PYR EPIA** element check	`REAL8`
`Pref_ElemQualPyrWarp`	If True, **PYR WARP** element check is on	`BOOL`
`Pref_ElemQualPyrWarpVal`	Value for **PYR WARP** element check	`REAL8`
Output Orientation button		
`Pref_OrientSolidIsoOutput`	Defines raw orientation of isotropic material solid element output vector data 0 = Material Direction 1 = Global Rectangular 3 = Element	`INT4`
`Pref_OrientSolidAnisoOutput`	Defines raw orientation of anisotropic material solid element output vector data 0 = Material Direction 1 = Global Rectangular 3 = Element	`INT4`
`Pref_OrientSolidHyperOutput`	Defines raw orientation of hyperelastic material solid element output vector data 0 = Material Direction 1 = Global Rectangular 3 = Element	`INT4`

Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
`Pref_OrientTria3StressOutput`	Defines raw orientation of Tria3 stress output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction	`INT4`
`Pref_OrientTria3StrainOutput`	Defines raw orientation of Tria3 strain output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction	`INT4`
`Pref_OrientTria3ForceOutput`	Defines raw orientation of Tria3 force output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction	`INT4`
`Pref_OrientTria6StressOutput`	Defines raw orientation of Tria6 stress output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction	`INT4`
`Pref_OrientTria6StrainOutput`	Defines raw orientation of Tria6 strain output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction	`INT4`
`Pref_OrientTria6ForceOutput`	Defines raw orientation of Tria6 force output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction	`INT4`
`Pref_OrientQuad4StressOutput`	Defines raw orientation of Quad4 stress output vector data 0 = First Edge 1 = Midside Location	`INT4`

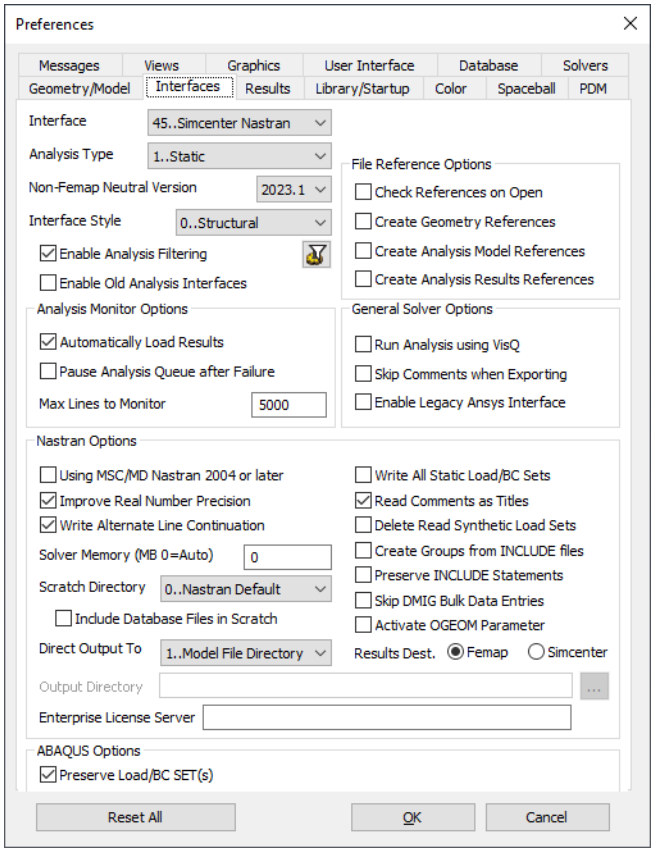
Unpublished work. © 2025 Siemens

Geometry/Model Preference Properties		
Property	**Description**	**Type**
	2 = Diagonal Bisector 3 = Material Direction	
`Pref_OrientQuad4StrainOutput`	Defines raw orientation of Quad4 strain output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction	`INT4`
`Pref_OrientQuad4ForceOutput`	Defines raw orientation of Quad4 force output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction	`INT4`
`Pref_OrientQuad8StressOutput`	Defines raw orientation of Quad8 stress output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction	`INT4`
`Pref_OrientQuad8StrainOutput`	Defines raw orientation of Quad8 strain output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction	`INT4`
`Pref_OrientQuad8ForceOutput`	Defines raw orientation of Quad8 force output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction	`INT4`

Unpublished work. © 2025 Siemens

File → Preferences - Interfaces tab

These Global Properties are used to set the options on the **Interfaces** tab of the Preferences dialog box:



Interfaces Preference Properties		
Property	**Description**	**Type**
Interface Options		
`Pref_AnalysisProg`	Default analysis program Interface. It is easiest to use the `FTR_` enums to select a default solver.	`INT4`
`Pref_AnalysisType`	Default Analysis Type.	`INT4`
`Pref_NeutralVersion`	Neutral file version number when writing Neutral files for third party analysis programs.	`REAL8`
`Pref_InterfaceStyle`	Default user **Interface Style**: 0=Structural, 1=Thermal, 2=Adv. Thermal.	`INT4`

Unpublished work. © 2025 Siemens

Interfaces Preference Properties		
Property	**Description**	**Type**
`Pref_UseOldAnalysisInterfaces`	If True, older, unsupported analysis program interfaces will still be available.	`BOOL`
`Pref_NeutralDigits`	Number of significant digits for real numbers in Neutral files. As of FEMAP 11.4, this item has been removed from the User Interface and should only be used if a specific number of digits is required.	`INT4`
Analysis Filtering Options		
`Pref_Filter_Analysis`	If True, the **Enable Analysis Filtering** option is activated.	`BOOL`
`Pref_Filter_SC_Nastran_Linear`	If True, the **Linear** option under **Simcenter Nastran** in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_SC_Nastran_MSNL`	If True, the **Mutistep Structural** option under **Simcenter Nastran** in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_SC_Nastran_MSNLK`	If True, the **Multistep Kinematic** option under **Simcenter Nastran** in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_SC_Nastran_ADVNL`	If True, the Advanced Nonlinear option under Simcenter Nastran in the Select Solvers to Filter dialog box is enabled.	`BOOL`
`Pref_Filter_SC_Nastran_ADVNLE`	If True, the **Explicit** option under **Simcenter Nastran** in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_MSC_Nastran`	If True, the **MSC Nastran** option in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_Autodesk_Nastran`	If True, the **Autodesk Nastran** option in the Select Solvers to Filter dialog box is activated.	`BOOL`

Unpublished work. © 2025 Siemens

Interfaces Preference Properties		
Property	**Description**	**Type**
`Pref_Filter_DYNA`	If True, the **LS-DYNA** option in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_ABAQUS`	If True, the **ABAQUS** option in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_ANSYS`	If True, the **ANSYS** option in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_SINDA`	If True, the **SINDA** option in the Select Solvers to Filter dialog box is activated.	`BOOL`
`Pref_Filter_MARC`	If True, the **MARC** option in the Select Solvers to Filter dialog box is activated.	`BOOL`
Analysis Monitor Options		
`Pref_AnalysisMonAutoLoad`	If True, will automatically load results when using the Analysis Monitor	`BOOL`
`Pref_AnalysisMonMaxLines`	Default Max number of lines to include in the Analysis Monitor	`INT4`
File Reference Options		
`Pref_FileRefCheck`	If True, checks file references when the model is opened	`BOOL`
`Pref_FileRefGeometry`	If True, creates geometry references when files are read	`BOOL`
`Pref_FileRefModel`	If True, creates analysis model references when files are read	`BOOL`
`Pref_FileRefResults`	If True, creates results file references when files are read.	`BOOL`
General Solver Options		
`Pref_UseVisQ`	For an integerated solver, turning this on will start the solver with VisQ (if it is available). When on, the default option for	`BOOL`

Unpublished work. © 2025 Siemens

Interfaces Preference Properties		
Property	**Description**	**Type**
	Solve Using is set to **VisQ** when creating a new analysis set.	
`Pref_WriteComments`	If True, comments will NOT be written to NASTRAN. This can get around some character set issues in entity titles.	`BOOL`
`Pref_EnableLegacyAnsysInterface`	If True, the Analysis Set Manager will use the dialog boxes and translator for ANSYS from versions of FEMAP prior to version 12.	`BOOL`
Nastran Options		
`Pref_UseMSCNastran2004`	If True, activates the **MSC.Nastran 2004** interface options	`BOOL`
`Pref_ImproveSinglePrecision`	If True, Femap writes real numbers to Nastran files using a nonstandard exponential format that improves precision.	`BOOL`
`Pref_NasAlternateContinue`	If True Femap will write an alternate line continuation using **+**.	`BOOL`
`Pref_NastranMemory`	Amount of memory specified for Nastran solver	`INT4`
`Pref_NastranScratchLocation`	Specifies option for **Scratch Directory**. 0 = Nastran Default, 1 = FEMAP Scratch, and 2 = Output Directory	`INT4`
`Pref_NasDballScratch`	If True Femap will use the dbs command line to specify a alternate location for user database files.	`BOOL`
`Pref_NastranOutputTo`	Location of Nastran Output Files (0=Current Directory, 1=Model File Directory, 2=Specified Directory)	`INT4`

Unpublished work. © 2025 Siemens

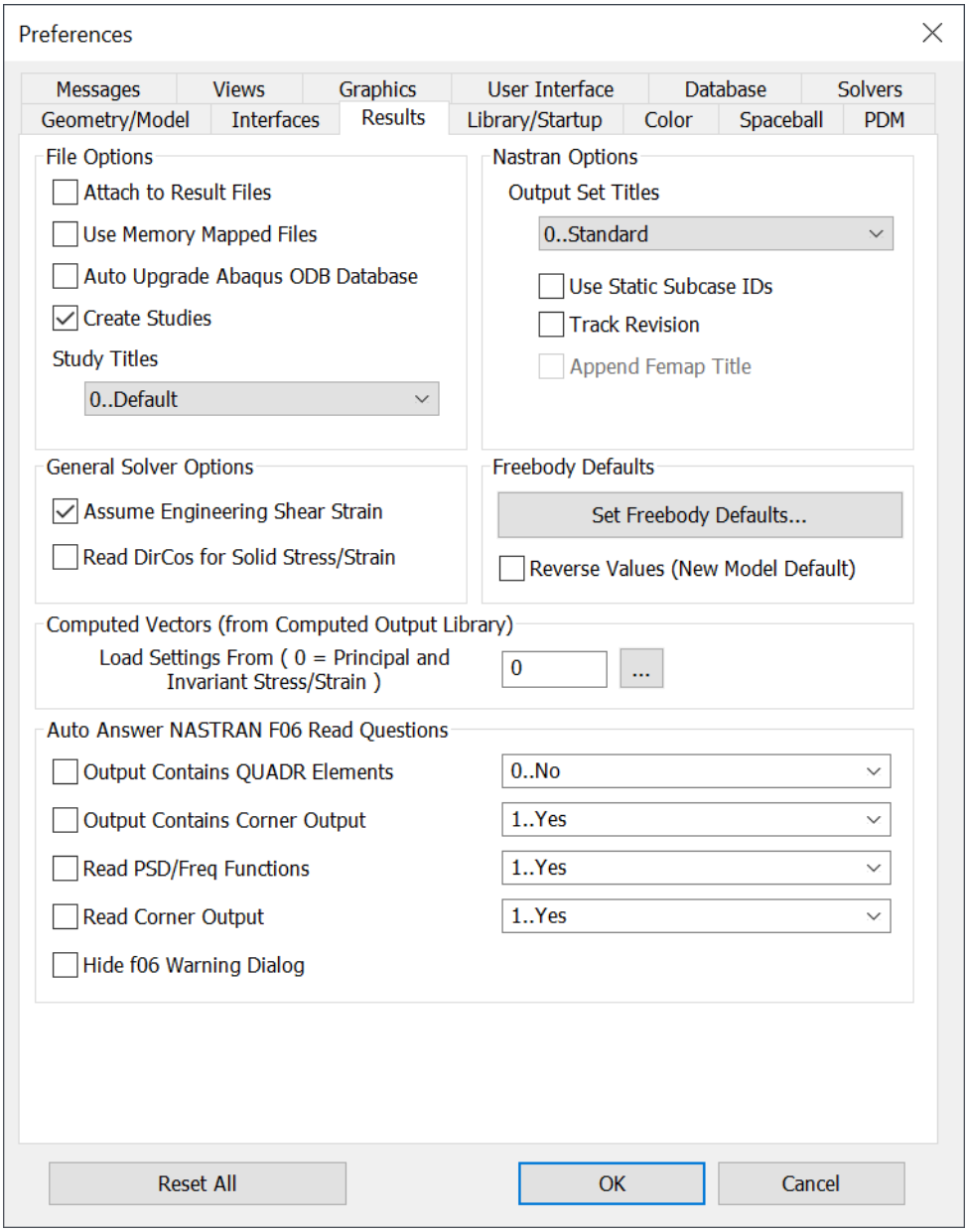
Interfaces Preference Properties		
Property	**Description**	**Type**
`Pref_NastranOutputPath`	Name/path of specified **Output Directory** for Nastran solver output	BOOL
`Pref_NXNastEntLicenseServer`	Used to specify a license server if using an enterprise version of NX Nastran. Simply enter the @ symbol followed by the name of the license server (i.e., @Nastran_Server_Name)	BOOL
`Pref_NastranWriteAllLdbcSets`	Default for Analysis Set to write all Static Loads/Boundary conditions.	BOOL
`Pref_ReadComments`	If True, an attempt will be made to read NASTRAN comment cards as entity titles.	BOOL
`Pref_DeleteRdScratchLdSets`	If True, Femap will delete *Synthetic* load sets that it creates during reading of NASTRAN LOAD cards.	BOOL
`Pref_GroupIncludeFiles`	If True, Femap will create groups based on entities in INCLUDE files found in a Nastran input file.	BOOL
`Pref_PreserveIncludes`	If True, then Nastran INCLUDE statements are read as Bulk Data text rather than reading their contents into the model	BOOL
`Pref_nastran_param_ogeom`	If True, the **Activate OGEOM Parameter** option is turned on. If False, it is not.	BOOL
`Pref_SC3DResultDest`	If True, the **Simcenter** option is selected for **Results Dest**. If False, it is set to **Femap**.	BOOL
ABAQUS Options		
`Pref_Abaqus_PreserveCompressedLoads`	If True, loads and boundary conditions which are defined with *SET(s) in the ABAQUS input file are used to automatically create Load Definition(s) and Constraint Definition(s) during import	BOOL

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

File → Preferences - Results tab

These Global Properties are used to set the options on the **Results** tab of the Preferences dialog box:



Results Preference Properties		
Property	**Description**	**Type**
File Options		
`Pref_AutoAttachResults`	If True, Femap will automatically attach to external results files rather than reading the results into the database.	`BOOL`

Unpublished work. © 2025 Siemens

Results Preference Properties		
Property	**Description**	**Type**
`Pref_MemoryMappedFiles`	If True, when attaching to external results files, memory mapped files will be utilized.	`BOOL`
`Pref_AutoConvertOdb`	If True, when attaching to an ODB results file, the database will be updated automatically	`BOOL`
`Pref_CreateResultStudy`	If True, an analysis study will be created for each imported or attached output file.	`BOOL`
`Pref_ResultStudyTitle`	Specifies option for **Study Titles**. 0=Default, 1=Analysis Set Title, 2=File Name, 3=Nastran TITLE	`INT4`
Nastran Options		
`Pref_OutputSetTitles`	Used by the Nastran OP2 file reader to set **Output Set Titles**. 0=Default, 1=TITLE, 2=SUBTITLE, 3=LABEL, 4=SUBTITLE, LABEL	`INT4`
`Pref_UseSubcaseIDForOutput`	If True, an attempt will be made to match the Femap Output Set ID to the Nastran Subcase ID	`BOOL`
`Pref_NasResTrackRevision`	If True, any output set with the same Subcase ID (Case ID) as an existing output set will be given a revision number.	`BOOL`
`Pref_StudyAppendFemapTitles`	If True, each output set title will be appended with any value associated with the output set, such as time or frequency.	`BOOL`
`Pref_SkipDMIGEntries`	If True, DMIG entries will be skipped when reading input files.	`BOOL`
General Solver Options		
`Pref_ComputePrincipal`	If True, Femap will automatically **Compute Principal Stress/Strain** and other quantities if they have not been read from results.	`BOOL`
`Pref_ComputeAverageMidResults`	If True, Femap automatically computes average results values from top and bottom results if	`BOOL`

Unpublished work. © 2025 Siemens

Results Preference Properties		
Property	**Description**	**Type**
	they have not been read from results.	
`Pref_AssumeEngrStrain`	If True, Femap Assumes Shear Strain results are Engineering shear strain.	`BOOL`
`Pref_ReadDirCos`	If True, Femap will read direction cosine matrices for stress/strain results.	`BOOL`
Freebody Defaults		
`Pref_DefaultFreebodyReverse`	If True, the **Reverse Freebody Values** option will be activated for all new models.	`BOOL`
`Pref_FBDispMode`	Sets default setting for Freebody Display Mode: 0=Freebody (`FBD_DISPLAYMODE_FREEBODY`) 1=Interface Load (`FBD_DISPLAYMODE_INTERFACE`) 2=Section Cut (`FBD_DISPLAYMODE_SECTION`)	`INT4`

`Pref_FBDefContributions[0..7]`
 	
or
 	
`VARIANT vPref_FBDefContributions [`
	Sets defaults for Freebody Contributions:
	
	0=Applied (`FBL_APPLIED`)
	1=Reaction (`FBL_SPC`)
	2=MultiPoint Reaction (`FBL_MPC`)
	3=Freebody Elements (`FBL_EL_FB`)
	4=Peripheral Elements (`FBL_EL_PER`)
	5=Nodal Summation (`FBL_TOTAL`)
	6=Contact (`FBL_CONTACT`)
	7=Glue (`FBL_GLUE`)
	True = contribution included
		`BOOL`

Unpublished work. © 2025 Siemens

Results Preference Properties		
Property	**Description**	**Type**

`Pref_FBDefNodalVectorMode[0..1]`
 	
or
 	
`VARIANT vPref_FBDefNodalVectorMode`
	Sets defaults for **Vector Display (Nodal)**:
	
	0=Nodal Forces
	1=Nodal Moments
	
	Set value to:
	
	0=Off (`FBD_VEC_MODE_OFF`)
	1=Display Components (`FBD_VEC_MODE_COMPONENT`)
	2=Display Resultant (`FBD_VEC_MODE_RESULTANT`)
		`INT4`

`Pref_FBDefTotalVectorMode[0..1]`
 	
or
 	
`VARIANT vPref_FBDefTotalVectorMode`
	Sets defaults for **Vector Display (Total)**:
	
	0=Total Forces
	1=Total Moments
	
	Set option:
	
	0=Off (`FBD_VEC_MODE_OFF`)
	1=Display Components (`FBD_VEC_MODE_COMPONENT`)
	2=Display Resultant (`FBD_VEC_MODE_RESULTANT`)
		`INT4`

`Pref_FBDefComponents[0..5]`
 	
or
 	
`VARIANT vPref_FBDefComponents`
	Sets defaults for **Load Components in Total Summation**:
	
	0=Fx (FBC_FX)
	1=Fy (FBC_FY)
	2=Fz (FBC_FZ)
	3=Mx (FBC_MX)
	4=My (FBC_MY)
	5=Mz (FBC_MZ)
	True = component included
		`BOOL`

Computed Vectors (from Computed Output Library)		

Unpublished work. © 2025 Siemens

Results Preference Properties		
Property	**Description**	**Type**
`Pref_LibComputedResultsVectors`	Selects library for computed vectors.	`String`
`Pref_LibComputedVectorNumber`	Displays vector library number.	`INT4`
Auto Answer Post Questions		

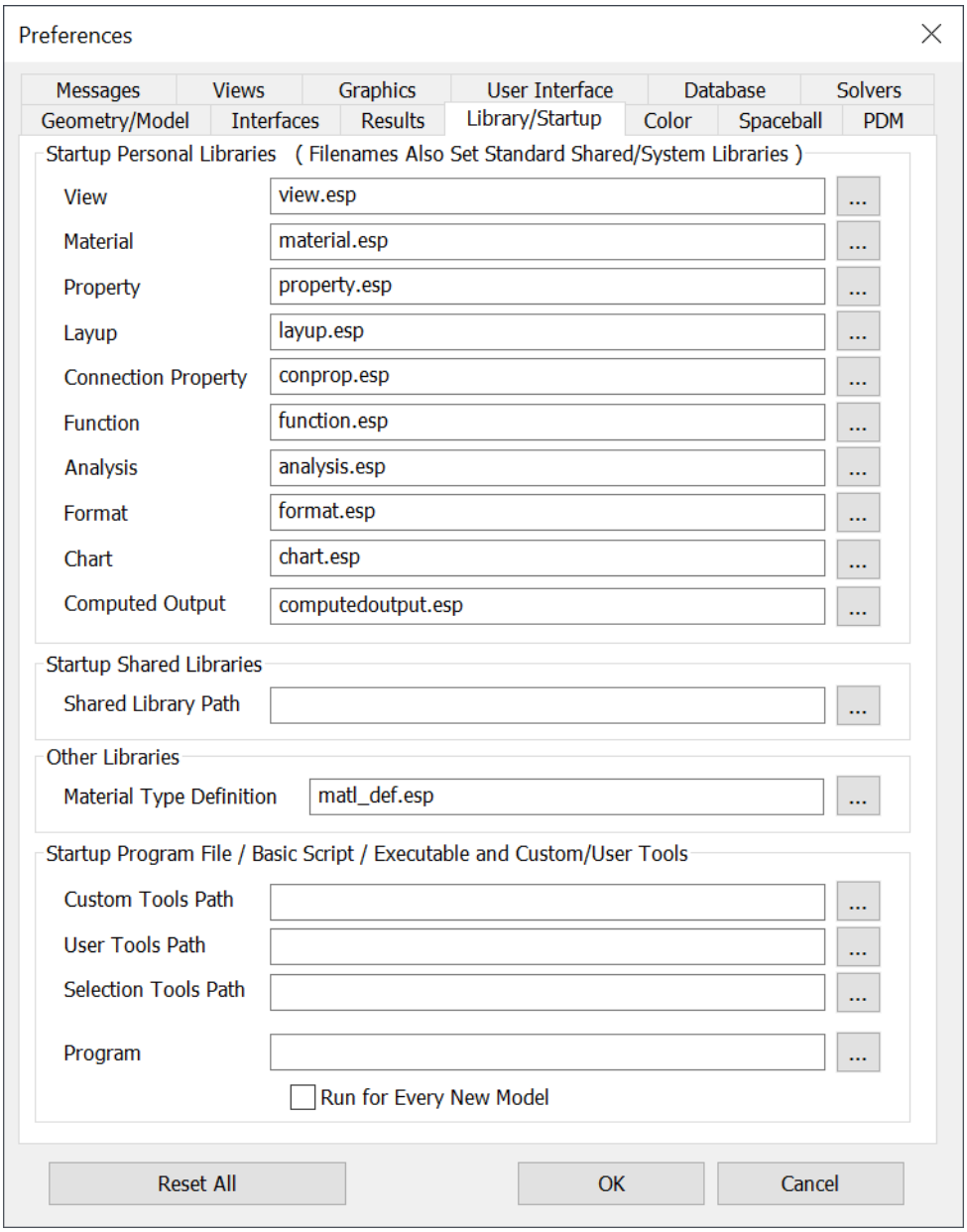
`Pref_NasQstOn[0..5]`
 	
or
 	
`VARIANT vPrefNasQstOn`
	`Pref_NasQstOn` activates Auto Answer for each question.
	
	Questions:
	
	0-Not Used, 1-Output Contains QUADR Elements, 2-Output Contains Corner Output, 3-Read PSD/Freq Functions, 4-Read Corner Output, 5-Hide f06 Warning dialog box
	
	`Pref_NasQstVal` sets the question answer.
	
	Answers:
	
	0 - No, 1 - Yes
		`BOOL`

`Pref_NasQstVal[0..5]`
 	
or
 	
`VARIANT vPrefNasQstVal`
		`BOOL`

Unpublished work. © 2025 Siemens

File → Preferences - Library/Startup tab

These Global Properties are used to set the options on the **Library/Startup** tab of the Preferences dialog box:



Library/Startup Preference Properties		
Property	**Description**	**Type**
Libraries		
`Pref_LibView`	Filename for **View** Library.	`STRING`

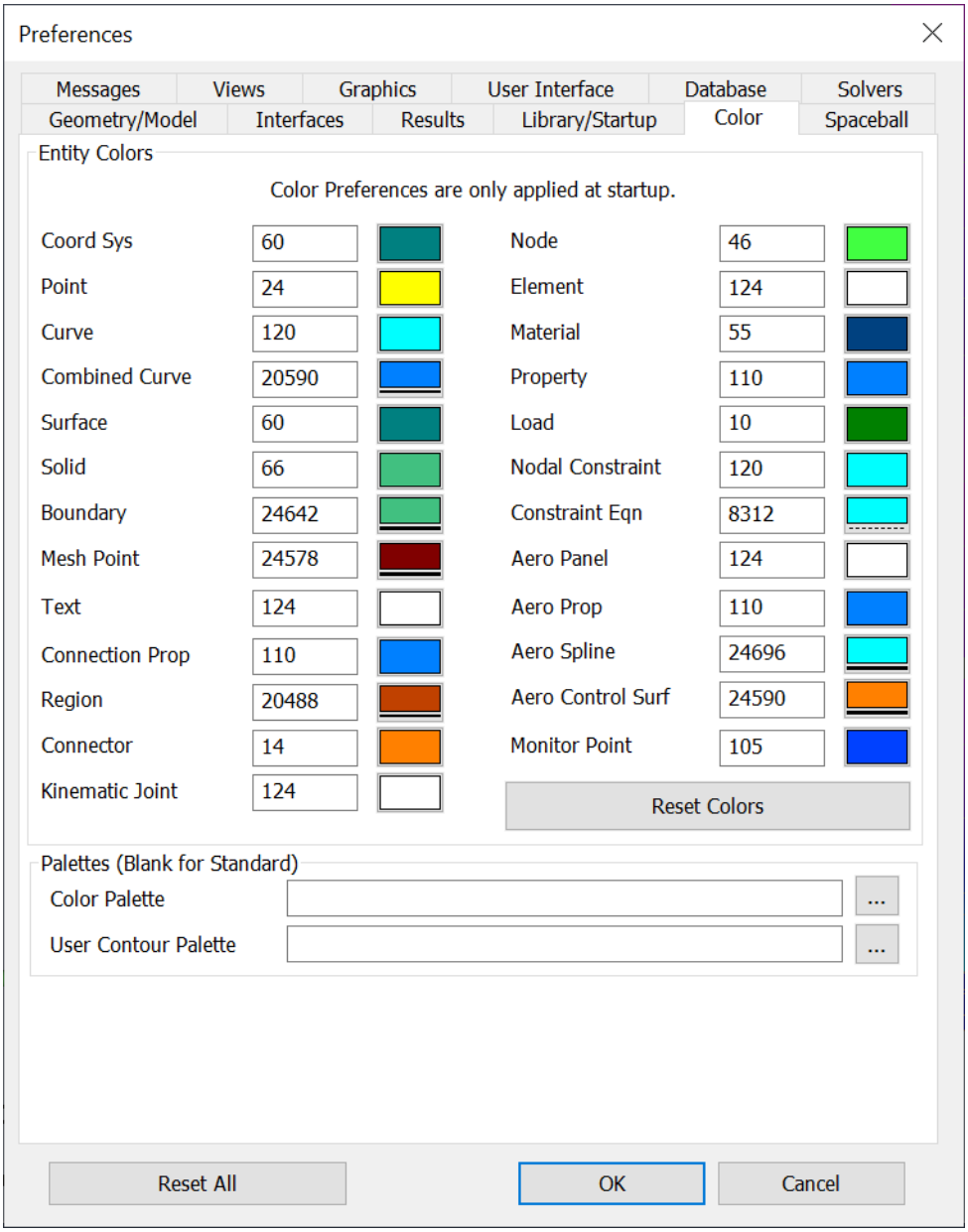
Unpublished work. © 2025 Siemens

Library/Startup Preference Properties		
Property	**Description**	**Type**
`Pref_LibMatl`	Filename for **Material** Library.	`STRING`
`Pref_LibProp`	Filename for **Property** Library.	`STRING`
`Pref_LibLayup`	Filename for **Layup** Library	`STRING`
`Pref_LibConnectionProp`	Filename for **Connection Property** Library	`STRING`
`Pref_LibFunction`	Filename for **Function** Library.	`STRING`
`Pref_LibAnalysis`	Filename for **Analysis** Library.	`STRING`
`Pref_LibReport`	Filename for report **Format** Library.	`STRING`
`Pref_LibChart`	Filename for **Chart** Library.	`STRING`
Shared Libraries		
`Pref_SharedLibPath`	Directory Path for Shared Libraries	`STRING`
Other Libraries		
`Pref_LibMatlDef`	Filename for **Material Type Definition** Library.	`STRING`
Startup Program File/Basic Script/Executable and Custom Tools		
`Pref_CustomToolsPath`	Name/path of specified directory for Custom Tools	`STRING`
`Pref_UserToolsPath`	Name/path of specified directory for User Tools	`STRING`
`Pref_StartupProg`	Name/path of a startup program (Program File or API)	`STRING`
`Pref_RunProgNewModels`	If True, the startup program will be run every time a new model is opened.	`BOOL`
`Pref_SelectionToolsPath`	Name/path that specifies the path for the selection tools directory.	`STRING`

Unpublished work. © 2025 Siemens

File → Preferences - Color tab

These Global Properties are used to set the options on the **Color** tab of the Preferences dialog box:



Color Preference Properties		
Property	**Description**	**Type**
Entity Colors		
`Pref_EntityColor[0..24]`	Default colors used for various entity types when starting a new database.	`INT4`

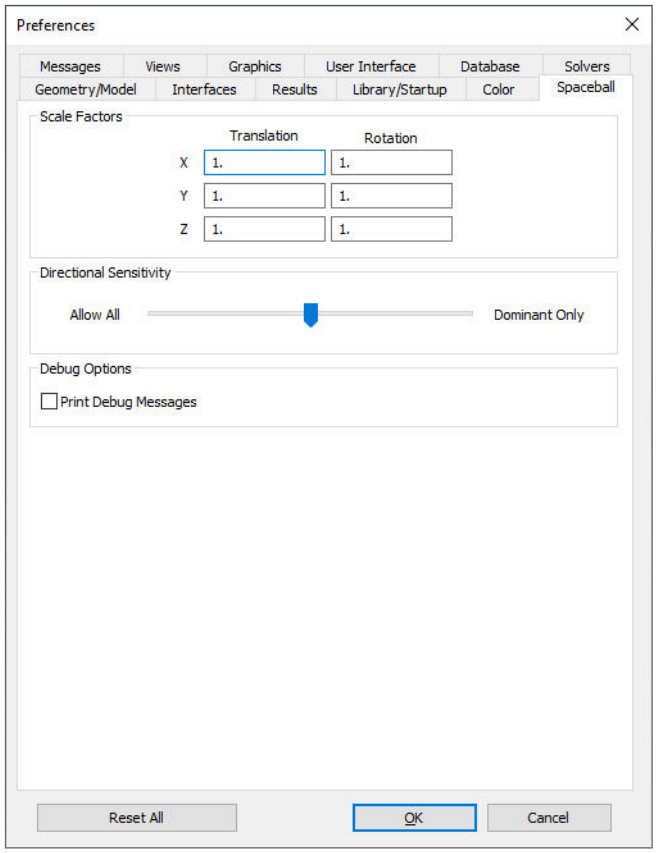
Unpublished work. © 2025 Siemens

Color Preference Properties		
Property	**Description**	**Type**
or `VARIANT vPref_EntityColor`	0 - Coord Sys 1 - Point 2 - Curve 3 - Surface 4 -Volume/Solid 5 - Boundary 6 - Text 7 - Node 8 - Element 9 - Material 10 - Property 11 - Load 12 - Nodal Constraint 13 - Constraint Eqn, 14 - Region 15 - Connector 16 - Connection Property 17 - Combined Curve 18 - Aero Panel 19 - Aero Prop 20 - Aero Spline 21 - Aero Control Surface 22 - Mesh Point 23 - Monitor Point 24 - Kinematic Joint	
Palettes (Blank for Standard)		
`Pref_LibPalette`	Filename for **Color Palette** Library.	`STRING`
`Pref_LibContPal`	Filename for **User Contour Palette** Library.	`STRING`

Unpublished work. © 2025 Siemens

File → Preferences - Spaceball tab

These Global Properties are used to set the options on the **Spaceball** tab of the Preferences dialog box:



Spaceball Preference Properties		
Property	**Description**	**Type**
Scale Factors		

`Pref_Spaceball Factors[6]`
 	
or
 	
`VARIANT vPref_SpaceballFactors`
	Allows specification for each degree of freedom.
	
	[0] = Translation X
	[1] = Translation Y
	[2] = Translation Z
	[3] = Rotation X
	[4] = Rotation Y
	[5] = Rotation Z
		REAL8

`Pref_SpaceballSensitivity`	Used to set the **Directional Sensitivity**. Value must be between 0.0 and 100.0	REAL8

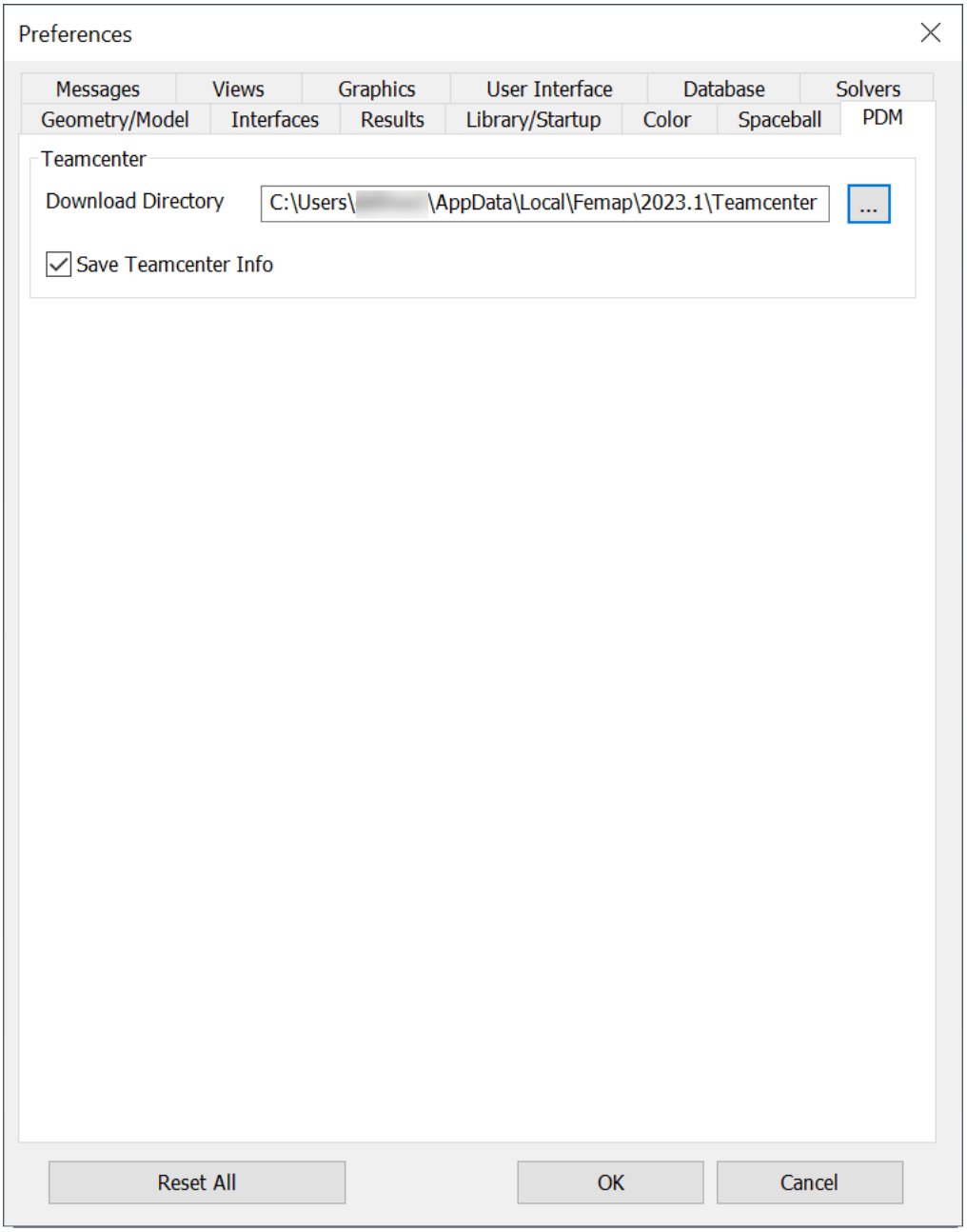
Unpublished work. © 2025 Siemens

Spaceball Preference Properties		
Property	**Description**	**Type**
Debug Options		
`Pref_SpaceballDebug`	When True, prints debug messages for Spaceball	`BOOL`

Unpublished work. © 2025 Siemens

File → Preferences - PDM tab

These Global Properties are used to set the options on the **Spaceball** tab of the Preferences dialog box:



PDM Preference Properties		
Property	**Description**	**Type**
Teamcenter		
`Pref_TCDownloadDir`	Sets the location or directory on your system where Teamcenter	`REAL8`

Femap API Guide
Unpublished work. © 2025 Siemens

PDM Preference Properties		
Property	**Description**	**Type**
	will download files when you are interacting with Teamcenter.	
`Pref_SaveTeamcenterInfo`	When True, the **Download Directory** information is saved regardless of where you set it when working with the Teamcenter interactive Open and Save commands in Femap.	`BOOL`

Unpublished work. © 2025 Siemens

Global Integer Properties

Global Integer Properties	
Property	**Description**
Window Handles	
`hMainWnd`	The handle to the Femap main window (read-only). **Note** This is an 8-byte integer, not a 4-byte integer like most other properties.
`hListWnd`	The handle to the Femap Messages and Lists window (read-only). **Note** This is an 8-byte integer, not a 4-byte integer like most other properties.
Event Properties	
`EventLParam`	The `lParam` value for the active event. This is not used for many events, but can be accessed from any event callback. **Note** This is an 8-byte integer, not a 4-byte integer like most other properties.
Deprecated API Warning	
`Pref_API_WarningLevel`	The warning level for deprecated API properties and methods. 0 = `APIWARN_EVERY`, 1 = `API_ONCEPERITEM`, 2 = `APIWARN_ONCEPERSESSION`3 = `APIWARN_NEVER`. This works in conjunctions with `Pref_API_WarningInDialog Boolean`.
File Print	
`Print_Copies`	Number of copies to print (only works with printers that support setting a number of copies).
Message Window	

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
`List_Destination`	A combination of one or more destinations (Screen, Printer, File) where listings are sent. 1=Messages Window only, 16=Printer only, 256=File only, 17=Messages Window and Printer, 257=Messages Window and File, 272=Printer and File, 273=Messages Window, Printer, and File. (INT4)
`List_FileName`	The path to a file to use for listing when the File destination is chosen.(STRING)
Model Info	
`Info_CommandNumber`	The number of commands that have been performed in this database (read-only).

`Info_MaxID[entity]`
or
`VARIANT vInfo_MaxID`
	The maximum ID of the selected entity (read-only). For information on entities, see **Entity Types**.

`Info_MinID[entity]`
or
`VARIANT vInfo_MinID`
	The minimum ID of the selected entity (read-only). For information on entities, see **Entity Types**.

`Info_Increment[entity]`
or
`VARIANT vInfo_Increment`
	The ID creation increment for the selected entity. For information on entities, see **Entity Types**. This provides similar capability as Tools-Parameters.

`Info_Count[entity]`
or
`VARIANT vInfo_Count`
	The number of selected entities in the database (read-only). For information on entities, see **Entity Types**.

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**

`Info_NextID[entity]`
 	
or
 	
`VARIANT vInfo_NextID`
	The ID of the next entity of a given type to be created. This provides similar capability as Tools-Parameters.

`Info_ActiveID[entity]`
 	
or
 	
`VARIANT vInfo_ActiveID`
	The Active ID of the selected entity. For information on entities, see **Entity Types**.

`Info_ActiveType[entity]`
 	
or
 	
`VARIANT vInfo_ActiveType`
	The Active Type of the selected entity. For information on entities, see **Entity Types**.

`Info_Color[entity]`
 	
or
 	
`VARIANT vInfo_Color`
	The default color for the selected entity. For information on entities, see **Entity Types**.

`Info_OutputCSys`	The default output coordinate system for node creation.
`Info_NodeType`	The default node type for node creation: 0=Node, 1=Scalar, 2=Extra.
`Info_TextBorderColor`	The default color for the border of text objects that are created.
`Info_WorkplaneOnSurface`	The surface ID where the workplane is located. The value is 0 if the workplane is simply located somewhere in model space.
`Info_WorkplaneOnSurfacePrev`	The previous value of `Info_WorkplaneOnSurface`.

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
`Info_SnapTo`	Current snap mode: 0=Screen, 1=Grid, 2=Point, 3=Node, 4=Smart.
`Info_SnapStyle`	The snap grid style: 0=Invisible, 1=Dots, 2=Lines.
`Info_SnapSpacingMode`	The snap grid spacing mode: 0=Automatic, 1=Uniform, 2=Nonuniform.
`Info_WorkplaneTics`	The number of divisions between labels along the workplane rulers.
`Info_MatlAngleMethod`	Sets the method for defining material angles for planar elements: 0=None - Default Orientation, 1=Vector Direction, 2=Coordinate Axis, 3=Angle Value, 4=X Axis of CSys
`Info_MatlAngleDir`	Sets the direction (0=X, 1=Y, 2=Z, 3=-X, 4=-Y, 5=-Z) in the specified coordinate system to be used for material angles. Only used if `Info_MatlAngleMethod`=Coordinate Axis.
`Info_MatlAngleCSys`	Sets the coordinate system ID to be used for the definition of material angles for planar elements if `InfoMatlAngleMethod`=Coordinate Direction or X Axis of CSys.
`Info_MeshSizeMinElem`	Sets the default for the minimum number of elements on any curves where a mesh size has not been explicitly specified. From **Mesh → Mesh Control → Default Size** command.

`Info_SurfDivisionsLinear[0..1]`
or
`VARIANT vInfo_SurfDivisionsLinear`
	Sets the default number of divisions to be displayed in the parametric directions of surfaces along linear edges: 0=s dir, 1=t dir

`Info_SurfDivisionsCurved[0..1]`
or
`VARIANT vInfo_SurfDivisionsCurved`
	Sets the default number of divisions to be displayed in the parametric directions of surfaces along curved edges. Unlike linear edges, which specify the total number of divisions, this number is the total to be displayed for a closed surface. The actual number displayed depends on the total curvature of the surface.

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
`Info_ViewShowColor`	The color to be used by the **View → Show** command.
`Info_ViewShowEntity`	The default type of entity to be shown by the **View → Show** command.
`Info_ViewShowLabel`	If nonzero, the **View → Show** command will label all selected entities.
`Info_GroupAutomaticAdd`	Controls the setting of the **Group → Automatic Add** command: 0=Off, -1=Active Group, or specify an existing group ID to automatically add to that group.
`Info_SplineOrder`	The default order of splines created with the standard geometry engine.
`Info_LayerNoPick`	Specifies the ID of the NoPick Layer. Set to 0 for None.
`Info_MatlUserType`	The default type of user-defined material to be created.

`Info_ElemFormulation1[0..34]`
or
`VARIANT vInfo_ElemFormulation1`
	The formulation options which are available for the various types of elements. Data is stored as bits inside these values.

`Info_ElemFormulation2[0..34]`
or
`VARIANT vInfo_ElemFormulation2`	

`Info_SuppressedCurveColor`	The color used to draw curves that are suppressed for meshing
`Info_SuppressedSurfaceColor`	The color used to draw surfaces that are suppressed for meshing
User Interface	
`Info_OrientSolidIsoOutput`	Defines raw orientation of isotropic material solid element output vector data

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
	0 = Material Direction 1 = Global Rectangular 3 = Element
`Info_OrientSolidAnisoOutput`	Defines raw orientation of anisotropic material solid element output vector data 0 = Material Direction 1 = Global Rectangular 3 = Element
`Info_OrientSolidHyperOutput`	Defines raw orientation of hyperelastic material solid element output vector data 0 = Material Direction 1 = Global Rectangular 3 = Element
`Info_OrientTria3StressOutput`	Defines raw orientation of `Tria3` stress output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction
`Info_OrientTria3StrainOutput`	Defines raw orientation of `Tria3` strain output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction
`Info_OrientTria3ForceOutput`	Defines raw orientation of `Tria3` force output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction
`Info_OrientTria6StressOutput`	Defines raw orientation of `Tria6` stress output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
`Info_OrientTria6StrainOutput`	Defines raw orientation of `Tria6` strain output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction
`Info_OrientTria6ForceOutput`	Defines raw orientation of `Tria6` force output vector data 0 = First Edge 1 = Midside Location 3 = Material Direction
`Info_OrientQuad4StressOutput`	Defines raw orientation of `Quad4` stress output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction
`Info_OrientQuad4StrainOutput`	Defines raw orientation of `Quad4` strain output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction
`Info_OrientQuad4ForceOutput`	Defines raw orientation of `Quad4` force output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction
`Info_OrientQuad8StressOutput`	Defines raw orientation of `Quad8` stress output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
`Info_OrientQuad8StrainOutput`	Defines raw orientation of `Quad8` strain output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction
`Info_OrientQuad8ForceOutput`	Defines raw orientation of `Quad8` force output vector data 0 = First Edge 1 = Midside Location 2 = Diagonal Bisector 3 = Material Direction
User Interface - Other Items	
`DialogAutoAnswer`	When you are running Femap in hidden mode (user interface not visible), then you may not want dialog boxes or message boxes popping up. Set this option to control the automatic return codes from boxes in this condition. 0=No/Cancel, 1=Yes/OK, 2=Show dialog boxes and message boxes even though main user interface is hidden
`DialogAutoSkip`	Same as `DialogAutoAnswer`, but works even if the user interface is not hidden
`DialogAutoSkipMsg`	Automatically *answers* any messages that are displayed while an API is running. The value specifies the answer that is selected. 0=Display Message Box and wait for user answer 1=Press First Button 2=Press Second Button 3=Press Third Button -1, -2 and -3 are the same as 1,2,3 except that the message is not echoed to the Message Window.
`CoordDialogMethod`	The current method used by the standard coordinate dialog box. Refer to **Coordinate Dialog Methods** for possible values.

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
`VectorDialogMethod`	The current method used by the standard vector dialog box. Refer to **Vector Dialog Methods** for possible values.
`PlaneDialogMethod`	The current method used by the standard plane dialog box. Refer to **Plane Dialog Methods** for possible values.
`PickBoundaryInternalMode`	The current mode of the Combined/Boundary Surface Picking option 0=Default, Pick Combined Surfaces, no Internal Surfaces 1=Pick Both Combined and Internal Surfaces 2=Pick Only Internal Surfaces 3=Pick Only Combined Surfaces
`PickCombinedCurveInternalMode`	The current mode of the Combined Curve Picking Option 0=Default, Pick Combined Curves, no Internal Curves 1=Pick Both Combined and Internal Curves 2=Pick Only Internal Curves 3=Pick Only Combined Curves 4=Never Pick Combined Curves 5=Never Pick Combined nor Internal Curves
Mesh Propagation Defaults	
`Info_MeshSizePropagation_SelMode`	Default Mode for option in Limit Selection section of the Mesh Size Propagation Options dialog box 0=All Solids / Surfaces (MSPS_ALL) 1=Meshed Solids/Surfaces Only (MSPS_MESHED)
`Info_MeshSizePropagation_EdgeOption`	Default Mode for option in Limit Selection section of the Mesh Size Propagation Options dialog box 0=None (MSPE_OFF) 1=Free Surface Edges Only (MSPE_FREEONLY)

Unpublished work. © 2025 Siemens

Global Integer Properties	
Property	**Description**
	2=Free Surface Edges and Other Edges (MSPE_FREEPLUS) 3=All Edges (MSPE_ALL)
Complex Results - Model Options	
`Info_ComplexSyncMethod`	Sets the option in Model Options of Complex Results dialog box 0=Selected Results - Ignore Phase Differences 1=Synchronize Phase. Used in conjunction with `Info_ComplexSyncPhase` and `Info_ComplexSyncIncrement`

Unpublished work. © 2025 Siemens

Global Real Properties

Global Real Properties	
Property	**Description**
Model and Session	
`ModelMigratedFromVersion`	Version of Femap the current model was migrated from by automatic neutral file conversion during **File → Open** (read-only)
Model Info	
`Info_Version`	The current database version. This may not equal the software version - for example, Femap v10.1.1 uses a v10.1 database. (read-only).

`Info_WorkplaneOrigin[0..2]`
or
`VARIANT vInfo_WorkplaneOrigin`
	The coordinates of the origin of the workplane. Specified in global rectangular model coordinates.

`Info_WorkplaneX[0..2]`
or
`VARIANT vInfo_WorkplaneX`
	The direction cosines of the workplane axes. Like the origin, these are specified in global rectangular model coordinates.

`Info_WorkplaneY[0..2]`
or
`VARIANT vInfo_WorkplaneY`	

`Info_WorkplaneZ[0..2]`
or
`VARIANT vInfo_WorkplaneZ`	

`Info_WorkplanePrevOrigin [0..2]`	The coordinates of the previous origin of the workplane. These are saved values of

Unpublished work. © 2025 Siemens

Global Real Properties	
Property	**Description**
or `VARIANT vInfo_WorkplanePrevOrigin`	`Info_WorkplaneOrigin`, used by the Previous Workplane command.

`Info_WorkplanePrevX[0..2]`
or
`VARIANT vInfo_WorkplanePrevX`
	The direction cosines of the workplane axes. Like the origin, these are specified in global rectangular model coordinates.

`Info_WorkplanePrevY[0..2]`
or
`VARIANT vInfo_WorkplanePrevY`	

`Info_WorkplanePrevZ[0..2]`
or
`VARIANT vInfo_WorkplanePrevZ`	

`Info_SnapSpacing[0..1]`
or
`VARIANT vInfo_SnapSpacing`
	The snap grid spacing X and Y values.

`Info_WorkplaneSizeX[0..1]`
or
`VARIANT vInfo_WorkplaneSizeX`
	The minimum and maximum dimensions of the workplane in the X and Y directions.

`Info_WorkplaneSizeY[0..1]`	

Unpublished work. © 2025 Siemens

Global Real Properties	
Property	**Description**
or `VARIANT vInfo_WorkplaneSizeY`	
`Info_WorkplaneLabels`	The length between labels on the workplane rulers.
`Info_DefaultX`	The default values for the coordinate dialog box. Stored in global rectangular model coordinates.
`Info_DefaultY`	
`Info_DefaultZ`	
`Info_LineLength`	The default length used to create horizontal and vertical lines.

`Info_ModelSizeX[0..1]`
 	
or
 	
`VARIANT vInfo_ModelSizeX`
	The minimum and maximum coordinates of the current model in the corresponding global rectangular direction. These values are only valid if the *model box* has been computed and `Info_ModelNeedBox` is False (read_only).

`Info_ModelSizeY[0..1]`
 	
or
 	
`VARIANT vInfo_ModelSizeY`	

`Info_ModelSizeZ[0..1]`
 	
or
 	
`VARIANT vInfo_ModelSizeZ`	

`Info_MatlAngleVec[0..2]`	The vector to be used for definition of material angles for planar elements (only used if `Info_MatlAngleMethod`=Vector).

Unpublished work. © 2025 Siemens

Global Real Properties	
Property	**Description**
or `VARIANT vInfo_MatlAngleVec`	
`Info_MatlAngle`	Sets the angle to be used for the definition of material angles for planar elements when `InfoMatlAngleMethod`=Angle.
`Info_MeshSizeDefault`	The default mesh size for geometry where mesh sizes have not been explicitly defined. From **Mesh → Mesh Control → Default Size** command.
`Info_GeometryScale`	Solid geometry scale factor. This is copied from `Pref_GeometryScale` when a model is started, and is used to make sure that solid geometry fits inside the limited coordinate space allowed by the solid modeling engines (read-only).
`Info_FacetAngleTolerance`	Factors used for Curve Facetting - found in Tools Parameters
`Info_FacetChordTolerance`	
`Info_FacetCurveFactor`	
Tools	
`Tool_MergeTolerance`	The default coincident location merge tolerance for the model. Only used if `Tool_MergeToleranceAuto` is False
Mesh Propagation Defaults	
`Info_MeshSizePropagation_Tolerance`	Specifies a default value for **Tolerance** when Specified option is selected in the **Edge Pairing Tolerance** section of the Mesh Size Propagation Options dialog box. Used in conjunction with `Info_MeshSizePropagation_UseTolerance`.
Complex Results - Model Options	
`Info_ComplexSyncPhase`	Specifies value for **Display Phase Angle** in **Model** Options
`Info_ComplexSyncIncrement`	Specifies value for **Animation Phase Increment** in **Model Options**

Unpublished work. © 2025 Siemens

Global Boolean Properties

Global Boolean Properties	
Property	**Description**
Deprecated API Warning	
`Pref_API_WarningInDialog`	Controls whether the deprecated API warning dialog box will appear when using deprecated API properties and methods. If True, this works in conjunction with `Pref_API_WarningLevel`.
File Preferences	
`Pref_CompressBitmaps`	Controls whether Windows Bitmaps will be compressed during Picture Save
Model and Session	
`ModelFromPreviousVersion`	If True, current model was migrated from a previous version into the version currently in use via automatic neutral file conversion invoked by the **File** → **Open** command (read-only)
Model Info	

`Info_PermBC[0..5]`
or
`VARIANT vInfo_PermBC`
	The default permanent boundary conditions for node creation.

`Info_WorkplaneModelAdjust`	Automatically adjust the size of the workplane to the model size.
`Info_WorkplaneSurfaceAdjust`	Automatically adjust the size of the workplane to the size of any surface where it is located.

`Info_WorkplaneRuler[0..1]`
or
`VARIANT vInfo_WorkplaneRuler`
	If True, draws the workplane X and Y rulers respectively.

`Info_SnapFullPrecision`	Controls whether coordinates or equations functions are loaded into the dialog boxes when a user picks coordinates.

Unpublished work. © 2025 Siemens

Global Boolean Properties	
Property	**Description**
`Info_SnapCoordOnly`	If True, the snap mode will only be used during coordinate picking.
`Info_ModelNeedBox`	If True the model box has been computed, otherwise the box coordinates are not valid.
`Info_UpdateAllViews`	The current setting of the **View → All Views** command.
`Info_TranslateNotes`	If True, the Notes translation text will be written during a model export.
`Info_ViewShowErase`	If True, the **View → Show** command will erase the window before showing entities.
`Info_ViewShowUseColor`	If True, **View → Show** will use the color specified by `Info_ViewShowColor`, otherwise entity colors will be used.
`Info_ViewShowNormal`	If True, the **View → Show** command will show element normal vectors when highlighting
`Info_ViewShowTranparent`	If True, the **View → Show** command makes the model transparent when highlighting
`Info_ViewShowAutoscale`	If True, the **View → Show** command *autoscales* to *zoom in* to the entities when highlighting.
`Info_MeshSizeAutoDefault`	If True, default mesh sizing will be automatically calculated when it is used.
`Info_FreebodyReverse`	If True, the **Reverse Freebody Values** option in the **Freebody Tool** of the **PostProcessing** Toolbox is activated.
Geometry	
(Do Not Use) `SolidAllowNonManifold`	If True, allows other Solid Modeling APIs to work with/create nonmanifold bodies (i.e., when "Ok to create nonmanifold geometry?" appears because of a Solid Modeling command, setting `SolidAllowNonManifold` to True answers that question Yes, False answers that question No.
Tools	
`Tool_CursorXYZ[0..2]`	If True, the corresponding coordinate direction will be picked whenever you select a coordinate with the mouse. Automatically updated with every pick.

Unpublished work. © 2025 Siemens

Global Boolean Properties	
Property	**Description**
or `VARIANT vTool_CursorXYZ`	
`Tool_CursorSingle`	If True, each cursor pick selects and updates only a single coordinate value.
`Tool_MergeToleranceAuto`	If True, the merge tolerance is automatically calculated, if False, then `Tool_MergeTolerance` is used.
`UndoGrayed`	If True, the **Tools** → **Undo** command is not available for selection. Set to False to activate the **Tools** → **Undo** command again.
Mesh Propagation Defaults	
`Info_MeshSizePropagation_On`	If True, the **Propagate Sizing** option is activated in all dialog boxes it exists and the **Turn On/Off Mesh Sizing Propagation** icon in the **Mesh Control Explorer** will be activated by default.
`Info_MeshSizePropagation_Slaved►Mapped`	If True, the **Matched and Mapped Surfaces** option in the **On Solids** section of the Mesh Size Propagation Options dialog box will be activated by default.
`Info_MeshSizePropagation_SameSolid`	If True, the **Include Edges on Same Solid** option in the **Edges to Pair** section of the Mesh Size Propagation Options dialog box will be activated by default.
`Info_MeshSizePropagation_UseToler►ance`	If True, the **Specified** option in the **Edge Pairing Tolerance** section of the Mesh Size Propagation Options dialog box will be activated by default.
User Interface - Other Items	
`PickVisibleOnly`	If True, the **Select Visible Only** option will be activated in the **Pick^** menu of the standard selection dialog box. This also activates the **Select Visible Only** option on the **Selector Actions** menu of the **Select** toolbar.
`SelectOutputSetsFromStudies`	If True, any dialog box which allows selection of Output Sets will display any existing Analysis Studies containing appropriate Output Sets under

Unpublished work. © 2025 Siemens

Global Boolean Properties	
Property	**Description**
	each Study. Set to False to show all available Output Sets without Analysis Studies.
`Pref_API_UseAlternateFileDialog`	If True, uses the alternate File Open dialog box to open files.
`Pref_API_HonorWorkingDirectory`	If True, allows setting initial directory when File Open dialog box is opened.

Unpublished work. © 2025 Siemens

Global String Properties

Global String Properties	
Property	**Description**
Initialization and Configuration File Paths	
`InitializationPath`	Name/path for the *Femap.ini* file.
`ConfigurationPath`	Name/path for the *config.ini* file.
File Preferences	
`Pref_NetKeyDir`	Name/path of network license file (only used when FlexLM network licensing is in use).
Model and Session	
`ModelName`	The name of the current model file. This will be blank if the model has not yet been saved (read-only)
`ModelMigratedFromDB`	The name of the original model file which went through neutral file conversation during the **File → Open** command and is now the current model (read-only)
`ScratchName`	The name of the working scratch file (OEM character set) (read-only)
`FullScratchName`	The full path to the working scratch file (ANSI character set) (read-only)
`ExePath`	The path to the Femap executable (without the filename) (read-only)
`SerialNumber`	The current Femap license serial number. (read_only)
Message Window	
`List_FileName`	The path to a file to use for listing when the File destination is chosen.

Unpublished work. © 2025 Siemens

Entity Types

The following table list the values to be used for each of the entity types. These values are typically used as an index into one of the global arrays such as `Info_MaxID`, `Info_ActiveID`...

Note

Some of the abbreviations for the entity types are very obvious, while others are not. For more information about the entity types marked with a *, please see the explanations after the table.

Entity Type	Numeric Value	Entity Type	Numeric Value
FT_POINT	3	FT_AMGR_DIR	60
FT_CURVE	4	FT_AMGR_CASE	61
FT_SURFACE	5	FT_REFERENCE *	68
FT_VOLUME	6	FT_DATA_SURF	69
FT_NODE	7	FT_CONNECTION	71
FT_ELEM	8	FT_CONNECTION_PROP	72
FT_CSYS	9	FT_BC_DEFINITION *	80
FT_MATL	10	FT_LAYUP	81
FT_PROP	11	FT_LAYUP_PLY	82
FT_LOAD_DIR *	12	FT_LAYUP_GLOBALPLY	83
FT_SURF_LOAD *	13	FT_BOLT_PRELOAD	84
FT_GEOM_LOAD *	14	FT_FREQUENCY*	88
FT_NTHERM_LOAD *	15	FT_AERO_PANEL	95
FT_ETHERM_LOAD *	16	FT_AERO_PROP	96

Unpublished work. © 2025 Siemens

Entity Type	Numeric Value	Entity Type	Numeric Value
FT_BC_DIR *	17	FT_AERO_SPLINE	98
FT_BCO *	18	FT_AERO_SURF	99
FT_BCO_GEOM *	19	FT_FREEBODY	102
FT_BEQ *	20	FT_RES_ATTACH*	103
FT_TEXT	21	FT_CHART	106
FT_VIEW	22	FT_CHART_SERIES*	107
FT_GROUP	24	FT_ANALYSIS_STUDY	113
FT_VAR *	27	FT_TABLE_DATA*	124
FT_OUT_CASE *	28	FT_VECTOR	132
FT_OUT_DIR *	29	FT_PLANE	133
FT_OUT_DATA *	30	FT_DISCRETE_VALUE_DIR	134
FT_REPORT *	31	FT_MESH_POINT_DEFINITION	136
FT_BOUNDARY	32	FT_MESH_POINT	137
FT_LAYER	33	FT_OPT_REL*	138
FT_FUNCTION_DIR	35	FT_OPT_RESP*	140
FT_FUNCTION_TABLE	36	FT_OPT_MANCON*	146
FT_SOLID	39	FT_MONITOR_POINT	149
FT_COLOR	40	FT_MATRIX_INPUT*	150

Unpublished work. © 2025 Siemens

Entity Type	Numeric Value	Entity Type	Numeric Value
FT_OUT_CSYS	41	FT_DESIGN_EQUATION	159
FT_DESIGN_OPT *	45	FT_JOINT	162
FT_LOAD_DEFINITION	46	FT_FLEXIBLE_SLIDER	164
FT_MODEL_NOTES *	54	FT_DRIVER_LOAD*	165
FT_CONTACT *	58		

*FT_LOAD_DIR = Femap Load Set

FT_SURF_LOAD = Nodal or Elemental Nodes

FT_NTHERM_LOAD = Nodal Temperatures

FT_ETHERM_LOAD = Elemental Temperatures

FT_BC_DIR = Femap Constraint Set

FT_BCO = Nodal Constraint

FT_BCO_GEOM = Constraint on geometric entity

FT_BEQ = Constraint Equation

FT_VAR = Femap Variable (**Tools** → **Varibales**...command in Femap)

FT_OUT_CASE = Femap Output Set

FT_OUT_DIR = Output Vector Header (includes Output Vector ID, Title, Type - Nodal or Elemental, Max and Min Values)

FT_OUT_DATA = Output Vector Data (Data at each node or element)

FT_REPORT = **List** → **Output** Format

FT_DESIGN_OPT = Design Optimization

FT_MODEL_NOTES = Notes saved in **File** → **Notes**

Unpublished work. © 2025 Siemens

`FT_CONTACT` = Femap Region

`FT_CONNECTION` = Femap Connector (contact pair)

`FT_REFERENCE` = References saved in **PDM → References**

`FT_BC_DEFINITION` = Femap Constraint Definition.

`FT_FREQUENCY` = Solution Frequencies for Dynamic Analysis

`FT_RES_ATTACH` = Attached Results Files

`FT_CHART_SERIES` = Chart Data Series

`FT_TABLE_DATA` = Table entity created via **Function/Table Editor**

`FT_OPT_REL` = Optimization Variable/Topology Region

`FT_OPT_RESP` = Optimization Limit

`FT_OPT_MANCON` = Topology Optimization Manufacturing Constraint

`FT_MATRIX_INPUT` = Direct Matrix Input (Simulation Entities)

`FT_DRIVER_LOAD` = Kinematic Driver (Load)

TMG Objects			
Entity Type	**Numeric Value**	**Entity Type**	**Numeric Value**
`FT_TMG_BCO`	127	`FT_TMG_REAL`	130
`FT_TMG_CONTROL`	128	`FT_TMG_OPTION`	131
`FT_TMG_INTEGER`	129		

Unpublished work. © 2025 Siemens

Femap Constants

The following tables list the constants which can be used for each of the Femap Objects. These constants always begin with “F” then a two letter abbreviation of the particular Femap Object where they can be used, followed by what the constant represents.

For Example, if you are looking for the Constant to use for a Ruled surface, then the constant would be FSU_RULED. In this case, SU represents “Surface” and RULED represents the type of surface to create.

Model Merge Renumbering Methods

The following table lists constants to used to set the renumbering method for the Merge Tool.

Constant Name	Constant Name
FMRRNM_NONE	FMRRNM_OFFSET
FMRRNM_MINIMAL	FMRRNM_COMPRESS
FMRRNM_BLOCK	

Analysis Types

The following table lists constants to be used for Analysis Types.

Constant Name	Constant Name
FAT_UNKNOWN	FAT_FATIGUE
FAT_STATIC	FAT_DATA_RECOVERY
FAT_MODES	FAT_CFD
FAT_TRANSIENT	FAT_STEADY_STATE_HEAT
FAT_FREQUENCY_RESPONSE	FAT_TRANSIENT_HEAT
FAT_RESPONSE_SPECTRUM	FAT_601_STATIC
FAT_RANDOM	FAT_601_TRANSIENT

Unpublished work. © 2025 Siemens

FAT_LINEAR_BUCKLING	FAT_701_EXPLICIT
FAT_DESIGN_OPTIMIZATION	FAT_LSDYNA_IMPLICIT
FAT_EXPLICIT	FAT_AERO_FLUTTER
FAT_NONLINEAR_STATIC	FAT_MULTISTEP_NONLINEAR
FAT_AERO_STATIC	FAT_MULTISTEP_KINEMATIC
FAT_NONLINEAR_BUCKLING	FAT_IMPLICIT_NONLINEAR
FAT_NONLINEAR_TRANSIENT	

Analysis Manager Solvers

The following table lists constants to be used for Analysis Manager Solvers.

Constant Name	Constant Name
FAM_UNKNOWN	FAM_MARC
FAM_MSC_NASTRAN	FAM_NE_NASTRAN
FAM_ANSYS	FAM_FEMAP_STRUCTURAL
FAM_ABAQUS	FAM_NX_NASTRAN

Analysis Manager Solve Using Options

The following table lists constants to be used for the **Solve Using** section of the Analysis Set dialog box in the Analysis Manager.

Constant Name	Constant Name
FS_INTEGRATED	FS_VISQ
FS_LINKED	

Unpublished work. © 2025 Siemens

Analysis Programs

The following table lists constants to be used for Analysis Programs.

Constant Name	Constant Name
FAP_UNKNOWN	FAP_CDA_SPRINT
FAP_FEMAP_GEN	FAP_CAEFEM
FAP_PAL	FAP_IDEAS
FAP_PAL2	FAP_ME_NASTRAN
FAP_MSC_NASTRAN	FAP_ADINA
FAP_ANSYS	FAP_GENESIS
FAP_STARDYNE	FAP_ODBC
FAP_COSMOS	FAP_CSA_NASTRAN
FAP_PATRAN	FAP_CFDESIGN
FAP_NEUTRAL	FAP_LS_DYNA
FAP_ALGOR	FAP_MARC
FAP_PC_NASTRAN	FAP_SINDA
FAP_TALE	FAP_NE_NASTRAN
FAP_UAI_NASTRAN	FAP_SATOOLKIT
FAP_COSMIC_NASTRAN	FAP_TMG
FAP_STAAD	FAP_FEMAP_STRUCTURAL
FAP_ABAQUS	FAP_VR_NASTRAN

Unpublished work. © 2025 Siemens

FAP_WECAN	FAP_NX_NASTRAN
FAP_SAP	FAP_AP_SATOOLKIT

Analysis Boundary Condition Set Types

The following table lists constants to be used for Analysis Boundary Condition Set Types.

Constant Name	Constant Name
FABT_CONSTRAINTS	FABT_SUPORT1_DOF
FABT_CONSTRAINT_EQUATIONS	FABT_OMIT_DOF
FABT_LOADS	FABT_QSET_DOF
FABT_INITIAL_CONDITIONS	FABT_CSET_DOF
FABT_MASTER_DOF	FABT_BSET_DOF
FABT_SUPORT_DOF	FABT_BOLT_PRELOADS

Analysis - Superelement Creation

The following table lists constants to be used for Superelement Creation.

Constant Name	Constant Name
EXTSEOUTTO_DMIGPCH	EXTSEOUTTO_MATOP4
EXTSEOUTTO_DMIGOP2	

Analysis - Superelement Reference

The following table lists constants to be used for Superelement Reference

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
SEREF_PCH	SEREF_OP4
SEREF_OP2	

Analysis - Superelement Assign Form

The following table lists constants to be used for the `NasExtSEOutAssignForm` property on the `AnalysisMgr` object

Constant Name	Constant Name
AAF_NONE	AAF_LITTLEENDIAN
AAF_FORMATTED	AAF_BIGENDIAN
AAF_UNFORMATTED	

Analysis - Solution Frequency List Types

The following table lists constants used when specifying the type of list when creating a list of solution frequencies.

Constant Name	Constant Name
FREQ	FREQ3
FREQ1	FREQ4
FREQ2	FREQ5

Analysis Output Requests

The following table lists constants to be used for Analysis Output Requests.

Constant Name	Constant Name
FABC_ND_APPLIED LOAD	FABC_EL_PLASTIC_STRAIN

Unpublished work. © 2025 Siemens

FABC_ND_REACTION_FORCE	FABC_EL_STRAIN_ENERGY
FABC_ND_DISPLACEMENT	FABC_EL_HEAT_FLUX
FABC_ND_VELOCITY	FABC_EL_ENTHALPY
FABC_ND_ACCELERATION	FABC_EL_ENTHALPY_RATE
FABC_ND_TEMPERATURE	FABC_EL_CONTACT
FABC_ND_HEAT_FLUX	FABC_EL_TEMPERATUR
FABC_ND_MPC_FORCES	FABC_EL_KINETIC_ENERGY
FABC_ND_FORCE_BALANCE	FABC_EL_ENERGY_LOSS
FABC_ND_KINETIC_ENERGY	FABC_EL_FLUID PRESSURE
FABC_EL_FORCE	FABC_EL_THERMAL_STRAIN
FABC_EL_STRESS	FABC_EL_ELASTIC_STRAIN
FABC_EL_STRAIN	

Point Types

The following table lists constants to be used for Point Types.

Constant Name	Constant Name
FPT_DEFAULT	FPT_SOLID

Curve Types

The following table lists constants to be used for Curve Types.

Constant Name	Constant Name
FCU_LINE	FCU_BSPLINE

Unpublished work. © 2025 Siemens

FCU_ARC	FCU_SOLID
FCU_CIRCLE	FCU_COMPOSITE
FCU_SPLINE	

Curve Orient Methods

The following table lists constants to be used for Curve Orient methods.

Constant Name	Constant Name
FCR_BY_VECTOR	FCR_BY_VECTOR_REV
FCR_BY_LOCATION	FCR_BY_LOCATION_REV

Curve Offset Methods

The following table lists constants to be used for Curve Offset methods.

Constant Name	Constant Name
FCO_BY_VECTOR	FCO_POINT
FCO_RADIAL	

Curve - From Surface, Pad Alignment Methods

The following table lists constants to be used to set the Alignment method for the Pad command.

Constant Name	Constant Name
FPAD_AUTO	FPAD_CURVE
FPAD_VECTOR	

Curve - From Surface, Pad Offset Types

The following table lists constants to be used to set the Offset type for the Pad command.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FPAD_SCALE	FPAD_DIST

Curve Project Onto Options

The following table lists constants to be used as **Project Onto** options with the **Modify → Project** commands.

Constant Name	Constant Name
FPO_CURVE	FPO_VECTOR
FPO_SURFACE	FPO_PLANE

Curve Project Along Options

The following table lists constants to be used as **Project Along** options with the **Modify → Project** commands.

Constant Name	Constant Name
FPA_CLOSEST	FPA_RADIAL_VECTOR
FPA_VECTOR	FPA_RADIAL_POINT

Curve Remove Options

The following table lists constants to be used as **Project Along** options with the **Modify → Project** commands.

Constant Name	Constant Name
FCRO_NORMAL	FCRO_COMBINE
FCRO_AGRESSIVE	

Curve Manifold Types

The following table lists constants to be used when determining the manifold type of a curve.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FCMT_UNKNOWN	FCMT_MANIFOLD
FCMT_WIREEDGE	FCMT_NONMANIFOLD
FCMT_FREEEDGE	

Combined Curve Methods

The following table lists constants to be used for Combined Curve methods.

Constant Name	Constant Name
FCC_OFF	FCC_BOTH
FCC_ON	

Surface Types

The following table lists constants to be used for Surface Types.

Constant Name	Constant Name
FSU_BILINEAR	FSU_BEZIER
FSU_RULED	FSU_SOLID
FSU_REVOLUTION	FSU_BSPLINE
FSU_COONS	FSU_BOUNDARY

Surface - Mesh Approaches

The following table lists constants to be used for Surface Mesh Apporaches.

Constant Name	Constant Name
FMAP_NONE	FMAP_MAP_4CORNER

Unpublished work. © 2025 Siemens

FMAP_FREE_PARAMETRIC	FMAP_MAP_3CORNER
FMAP_FREE_PLANAR	FMAP_MAP_3CORNERFAN
FMAP_TRI_PARAMETRIC	FMAP_MAP_LINKED
FMAP_TRI_PLANAR	

Surface - Mesh Offset Attribute

The following table lists constants to be used for Surface Mesh Offset Attribute.

Constant Name	Constant Name
FMOF_CENTERLINE	FMOF_BOTTOM
FMOF_TOP	

Surface - Triangle Mesher Attribute

The following table lists constants to be used for Surface Mesh *Triangle Mesher* Attribute.

Constant Name	Constant Name
FMSH_AUTO	FMSH_FASTTRI
FMSH_SUBDIVISION	FMSH_3DTRI

Solid - Alignment Options

The following table lists constants to be used for alignment options for `feSolidAlignSplits`.

Constant Name	Constant Name
FSAV_VECAUTO	FSAV_VECASK
FSAV_VECINPUT	

Solid - Tet Meshing Approach

Unpublished work. © 2025 Siemens

The following table lists constants to be used for meshing approach option for `feMeshTetSolid3`.

Constant Name	Constant Name
FTP_SURFACEONLY	FTP_PYR
FTP_TET	

Volume Types

The following table lists constants to be used for Volume Types.

Constant Name	Constant Name
FVO_BRICK	FVO_PYRAMID
FVO_WEDGE	FVO_TETRA

Node Types

The following table lists constants to be used for Node Types.

Constant Name	Constant Name
FND_NODE	FND_EPOINT
FND_SPOINT	FND_FLPOINT

Material Types

The following table lists constants to be used for Material Types.

Constant Name	Constant Name
FMT_ISOTROPIC	FMT_ANISOTROPIC_3D
FMT_ORTHOTROPIC_2D	FMT_HYPERELASTIC

Unpublished work. © 2025 Siemens

FMT_ORTHOTROPIC_3D	FMT_GENERAL
FMT_ANISOTROPIC_2D	FMT_FLUID

Element Topology Types

The following table lists constants to be used for Element Topology Types.

Constant Name	Constant Name
FTO_POINT1	FTO_WEDGE15
FTO_LINE2	FTO_PYRAMID5
FTO_LINE3	FTO_PYRAMID13
FTO_TRIA3	FTO_BRICK8
FTO_TRIA6	FTO_BRICK20
FTO_QUAD4	FTO_MULTILIST2
FTO_QUAD8	FTO_RIGIDLIST
FTO_TETRA4	FTO_RIGIDLIST2
FTO_TETRA10	FTO_CONTACT2
FTO_WEDGE5	FTO_WELD2
	FTO_RIGIDRODLINE2
	FTO_RIGIDBARLINE2

Element Types

The following table lists constants to be used for Element Types.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FET_NONE	FET_L_PLANE_STRAIN
FET_L_ROD	FET_P_PLANE_STRAIN
FET_L_TUBE	FET_L_AXISYM_SHELL
FET_L_CURVED_TUBE	FET_P_AXISYM_SHELL
FET_L_BAR	FET_L_PLOT_PLATE
FET_L_BEAM	FET_L_AXISYM
FET_P_BEAM	FET_P_AXISYM
FET_L_LINK	FET_L_SOLID
FET_L_CURVED_BEAM	FET_P_SOLID
FET_L_SPRING	FET_L_LAMINATE_SOLID
FET_L_DOF_SPRING	FET_P_LAMINATE_SOLID
FET_L_GAP	FET_L_COHESIVE_SOLID
FET_L_PLOT	FET_P_COHESIVE_SOLID
FET_L_SHEAR	FET_L_MASS
FET_P_SHEAR	FET_L_MASS_MATRIX
FET_L_MEMBRANE	FET_L_SPRING_TO_GROUND
FET_P_MEMBRANE	FET_L_DOF_SPRING_TO_GROUND
FET_L_BENDING	FET_L_RIGID
FET_P_BENDING	FET_L_STIFF_MATRIX

Unpublished work. © 2025 Siemens

FET_L_PLATE	FET_L_SLIDE_LINE
FET_P_PLATE	FET_L_WELD
FET_L_LAMINATE_PLATE	FET_L_NASTRAN_MATRIX
FET_P_LAMINATE_PLATE	FET_L_CONTACT

Beam Section Calculator Orientation

The following table lists constants used to set the orientation for the beam section calculator.

Constant Name	Constant Name
FSOR_RIGHT	FSOR_LEFT
FSOR_UP	FSOR_DOWN

Beam Section Calculator Shape

The following table lists constants to be used to set the shape for the beam section calculator.

Constant Name	Constant Name
FSHP_RECT_BAR	FSHP_NAS_I
FSHP_RECT_TUBE	FSHP_NAS_CHAN
FSHP_TRAP_BAR	FSHP_NAS_T
FSHP_TRAP_TUBE	FSHP_NAS_BOX
FSHP_CIRC_BAR	FSHP_NAS_BAR
FSHP_CIRC_TUBE	FSHP_NAS_CROSS
FSHP_HEX_BAR	FSHP_NAS_H
FSHP_HEX_TUBE	FSHP_NAS_T1

Unpublished work. © 2025 Siemens

FSHP_I	FSHP_NAS_I1
FSHP_C	FSHP_NAS_CHAN1
FSHP_L	FSHP_NAS_Z
FSHP_T	FSHP_NAS_CHAN2
FSHP_Z	FSHP_NAS_T2
FSHP_HAT	FSHP_NAS_BOX1
FSHP_NAS_ROD	FSHP_NAS_HEXA
FSHP_NAS_TUBE	FSHP_NAS_HAT
FSHP_NAS_L	FSHP_NAS_HAT1

Beam Section Evaluation Method

The following table lists constants used to set the beam section calculator evaluation method.

Constant Name	Constant Name
FSEV_AUTO	FSEV_ALT
FSEV_ORIG	FSEV_PBEAML

Spring/Damper Element Orientation

The following table lists constants to be used for Spring/Damper Orientation options.

Constant Name	Constant Name
FESO_NONE	FESO_ELCID
FESO_NODE	FESO_PRCID
FESO_VECTOR	

Unpublished work. © 2025 Siemens

Spring/Damper Element Offset

The following table lists constants to be used for Spring/Damper Offset options.

Constant Name	Constant Name
FESL_DEFAULT	FESL_PRLOC
FESL_ELLOC	FESL_OFFSET

Modify Element Thickness/Offset - Updating

The following table lists constants to select what type of update will be performed by a `feModify-PlateThickOffset...` call.

Constant Name	Constant Name
FPTO_ELEM_THICKNESS	FPTO_ELEM_OFFSET
FPTO_PROP_THICKNESS	

Modify Element Thickness/Offset - Method

The following table lists constants to select the method used to perform an update by a `feModify-PlateThickOffset...` call.

Constant Name	Constant Name
FPTM_SETTOVALUE	FPTM_TOPTONODE
FPTM_INCREMENT	FPTM_BOTTOMTONODE
FPTM_ADJUSTPCT	

Coordinate System Types

The following table lists constants to be used for Coordinate System Types.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FCS_RECTANGULAR	FCS_SPHERICAL
FCS_CYLINRICAL	

Connection Region Types

The following table lists constants to be used for Connection Region Types.

Constant Name	Constant Name
FRT_CONNECTION	FRT_ROTOR
FRT_FLUID	FRT_NSM
FRT_BOLT	

Connection Property Types

The following table lists constants to be used for Connection Property Types.

Constant Name	Constant Name
FCPT_CONTACT	FCPT_GLUED

Geometric Boundary Condition Types

The following table lists constants to be used for Geometric Boundary Condition Types.

Constant Name	Constant Name
FBG_DOF	FBG_SURFNONSLIDING
FBG_GENERAL	FBG_SURFDIRSLIDING
FBG_SURFSLIDING	FBG_CYLINDER

Load Direction Methods

The following table lists constants to be used for Load Direction Methods.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FLD_NONE	FLD_NORMALTOPLANE
FLD_VECTOR	FLD_NORMALTOSURFACE
FLD_ALONGCURVE	

Load Variation Methods

The following table lists constants to be used for Load Variation Methods.

Constant Name	Constant Name
FLV_NONE	FLV_FUNCTION
FLV_EQUATION	FLV_INTERPOLATION
FLV_DATASURFACE	

Load Types

The following table lists constants to be used for Nodal Load Types except for FLT_NBODY which represents a Body Load.

Constant Name	Constant Name
FLT_NBODY	FLT_NHEATGEN
FLT_NFORCE	FLT_NTRANSIENT
FLT_NFOLLOWERFORCE	FLT_NPRESSURE
FLT_NMOMENT	FLT_NTOTALPRESSURE
FLT_NFOLLOWERMOMENT	FLT_NSCALAR
FLT_NDISPLACEMENT	FLT_NSTEAMQUALITY
FLT_NROTDISPLACEMENT	FLT_NHUMIDITY

Unpublished work. © 2025 Siemens

FLT_NVELOCITY	FLT_NFLUIDHEIGHT
FLT_NROTVELOCITY	FLT_NUNKNOWNCONDITION
FLT_NACCELERATION	FLT_NSLIPCONDITION
FLT_NROTACCELERATION	FLT_NFANCURVE
FLT_NTEMPERATURE	FLT_NPERIODIC
FLT_NHEATFLUX	

The following table lists constants to be used for Elemental Load Types.

Constant Name	Constant Name
FLT_ELINELOAD	FLT_ECONVECTION
FLT_EPRESSURE	FLT_ERADIATION
FLT_ETEMPERATURE	FLT_EHEATGEN
FLT_EHEATFLUX	FLT_EBOLTPRELOAD

The following table lists constants to be used for Loads Types on Geometric Points which are expanded to nodes.

Constant Name	Constant Name
FLT_PNFORCE	FLT_PNHEATGEN
FLT_PNFOLLOWERFORCE	FLT_PNPRESSURE
FLT_PNMOMENT	FLT_PNTOTALPRESSURE
FLT_PNFOLLOWERMOMENT	FLT_PNSCALAR
FLT_PNDISPLACEMENT	FLT_PNSTEAMQUALITY

Unpublished work. © 2025 Siemens

FLT_PNROTDISPLACEMENT	FLT_PNHUMIDITY
FLT_PNVELOCITY	FLT_PNFLUIDHEIGHT
FLT_PNROTVELOCITY	FLT_PNUNKNOWNCONDITION
FLT_PNACCELERATION	FLT_PNSLIPCONDITION
FLT_PNROTACCELERATION	FLT_PNFANCURVE
FLT_PNTEMPERATURE	FLT_PNPERIODICFLT_NPRESSURE
FLT_PNHEATFLUX	

The following table lists constants to be used for Loads Types on Geometric Curves which are expanded to nodes.

Constant Name	Constant Name
FLT_CNFORCE	FLT_CNHEATGEN
FLT_CNFORCEPERLEN	FLT_CNPRESSURE
FLT_CNFORCEATNODE	FLT_CNTOTALPRESSURE
FLT_CNMOMENT	FLT_CNSCALAR
FLT_CNMOMENTPERLEN	FLT_CNSTEAMQUALITY
FLT_CNMOMENTATNODE	FLT_CNHUMIDITY
FLT_CNDISPLACEMENT	FLT_CNFLUIDHEIGHT
FLT_CNROTDISPLACEMENT	FLT_CNUNKNOWNCONDITION
FLT_CNVELOCITY	FLT_CNSLIPCONDITION
FLT_CNROTVELOCITY	FLT_CNFANCURVE

Unpublished work. © 2025 Siemens

FLT_CNACCELERATION	FLT_CNPERIODIC
FLT_CNROTACCELERATION	FLT_CNTORQUE
FLT_CNTEMPERATURE	FLT_CNBEARING
FLT_CNHEATFLUX	FLT_CNFOLLOWERFORCE
FLT_CNHEATFLUXPERLEN	FLT_CNFOLLOWERMOMENT
FLT_CNHEATFLUXATNODE	

The following table lists constants to be used for Loads Types on Geometric Curves which are expanded to elements.

Constant Name	Constant Name
FLT_CEPRESSURE	FLT_CECONVECTION
FLT_CETEMPERATURE	FLT_CERADIATION
FLT_CEHEATFLUX	FLT_CEHEATGEN

The following table lists constants to be used for Loads Types on Geometric Surfaces which are expanded to nodes.

Constant Name	Constant Name
FLT_SNFORCE	FLT_SNHEATGEN
FLT_SNFORCEPERAREA	FLT_SNPRESSURE
FLT_SNFORCEATNODE	FLT_SNTOTALPRESSURE
FLT_SNMOMENT	FLT_SNSCALAR
FLT_SNMOMENTPERAREA	FLT_SNSTEAMQUALITY
FLT_SNMOMENTATNODE	FLT_SNHUMIDITY

Unpublished work. © 2025 Siemens

FLT_SNDISPLACEMENT	FLT_SNFLUIDHEIGHT
FLT_SNROTDISPLACEMENT	FLT_SNUNKNOWNCONDITION
FLT_SNVELOCITY	FLT_SNSLIPCONDITION
FLT_SNROTVELOCITY	FLT_SNFANCURVE
FLT_SNACCELERATION	FLT_SNPERIODIC
FLT_SNROTACCELERATION	FLT_SNTORQUE
FLT_SNTEMPERATURE	FLT_SNBEARING
FLT_SNHEATFLUX	FLT_SNFOLLOWERFORCE
FLT_SNHEATFLUXPERAREA	FLT_SNFOLLOWERMOMENT
FLT_SNHEATFLUXATNODE	

The following table lists constants to be used for Loads Types on Geometric Surfaces which are expanded to elements.

Constant Name	Constant Name
FLT_SEPRESSURE	FLT_SECONVECTION
FLT_SETEMPERATURE	FLT_SERADIATION
FLT_SEHEATFLUX	FLT_SEHEATGEN

Load Selection Types

The following table lists constants to be used when selecting particular load types to perform an operation on them, such as deleting loads.

Constant Name	Constant Name
FLS_ANY	FLS_NTEMP

Unpublished work. © 2025 Siemens

FLS_BODY	FLS_ETEMP
FLS_NONLINEAR	FLS_ECONVECTION
FLS_NHTGEN	FLS_ERADIATION
FLS_NHTFLUX	FLS_NTRANSIENT
FLS_FORCE	FLS_FLPRESSURE
FLS_DISP	FLS_FLTRACKING
FLS_ACCEL	FLS_FLUNKNOWN
FLS_LINELOAD	FLS_FLSLIP
FLS_VELOCITY	FLS_FLFAN
FLS_PRESS	FLS_FLPERIODIC
FLS_EHTGEN	FLS_NONE
FLS_EHTFLUX	

Function/Table Type

The following table lists constants to be used for Type of Function or Table in the Function/Table Editor Type.

Constant Name	Constant Name
FTBL_NONE	FTBL_LOADSETCOMBINATION
FTBL_FUNCTION	FTBL_RESULTSETPROCESS
FTBL_VECTORFUNCTION	

Unpublished work. © 2025 Siemens

Function Types

The following table lists constants to be used for Function Types.(i.e., when entity type in the Function/Table Editor Type is `FTBL_FUNCTION`).

Constant Name	Constant Name
FTB_NONE	FTB_ROTFREQ
FTB_TIME	FTB_VELFREQ
FTB_TEMP	FTB_ANGVELFREQ
FTB_FREQ	FTB_ACCFREQ
FTB_STRESSSTRAIN	FTB_ANGACCFREQ
FTB_FUNCTEMP	FTB_FORCEFREQ
FTB_GDAMP	FTB_MOMENTFREQ
FTB_CRITDAMP	FTB_STRESSFREQ
FTB_QDAMP	FTB_STRAINFREQ
FTB_STRAINRATE	FTB_MACHFREQ
FTB_FUNCSTRAINRATE	FTB_AEROFACTOR
FTB_CURVELENGTH	FTB_ACCELLOC
FTB_CURVEPARAM	FTB_VELOCITY
FTB_STRAINSTRESS	FTB_PRESSURE
FTB_PLASTICSTRAIN	FTB_WAVELENGTH
FTB_FUNCVAL	FTB_TEMPDIFF
FTB_DAMPFUNC	FTB_HEATFLOWRATE

Unpublished work. © 2025 Siemens

FTB_ANGLEINCIDENCE	FTB_MASSFLOWRATE
FTB_DIRECTIONINCIDENCE	FTB_VOLUMEFLOWRATE
FTB_TBLM1LINLIN	FTB_THERMALCAPACITANCE
FTB_TBLM1LOGLIN	FTB_LATITUDE
FTB_TBLM1LINLOG	FTB_TRUESTRAINSTRESS
FTB_TBLM1LOGLOG	FTB_TRUEPLASTICSTRAIN
FTB_MODE	FTB_TBLM5DAMAGEFORCE
FTB_DISPFREQ	

Table Type

The following table lists constants to be used for the Type of Table (i.e., when entity type in the Function/Table Editor Type is FTBL_VECTORFUNCTION).

Constant Name	Constant Name
FTBV_NONE	FTBV_SUNPLANETALTITUDE
FTBV_ACCELERATION	FTBV_SPHERICALVECTORSUNPLANET
FTBV_SUNPLANET	FTBV_SPHERICALVECTORSUNPLANETALTITUDE

MapOutput Constants

The following table lists constants to be used for “LoadType” that can be mapped from output using MapFromModelToSet method

Constant Name	Constant Name
FMLT_NTEMPERATURE	FMLT_NACCELERATION
FMLT_NFORCE	FMLT_NROTACCELERATION

Unpublished work. © 2025 Siemens

FMLT_NMOMENT	FMLT_ETEMPERATURE
FMLT_NDISPLACEMENT	FMLT_EHEATFLUX
FMLT_NROTDISPLACEMENT	FMLT_EHEATGEN
FMLT_NVELOCITY	FMLT_EPRESSURE
FMLT_NROTVELOCITY	FMLT_NPRESSURE

The following table lists constants to be used for `MapOption` that can be used to specify algorithm to map output to loads where meshes do not coincide.

These constants are used for the `MapFromModelToSet` method.

Constant Name	Constant Name
FMO_ZERO	FMO_INTERP
FMO_VALUE	FMO_NONE
FMO_EXTEND	

The following table lists constants to be used for `MapToType` that can be used to specify target mapping type. FMP_STANDARD is used if source output is from same entity type (Node or Element) as the desired target load.

These constants are used for `MapFromModelToSet` method.

Constant Name	Constant Name
FMP_STANDARD	FMP_ELEMENT
FMP_NODE	

The following table lists constants to be used for `DataConversion` that can be used to specify the data conversion method used to interpret the output data from the source model.

These constants are used for the `MapFromModelToSet` method.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FMDC_VU	FMDC_AVG_SKIP_CORNER
FMDC_AVG	FMDC_MAX_SKIP_CORNER
FMDC_MAX	FMDC_MIN_SKIP_CORNER
FMDC_MIN	

Data Surface Constants

The following table lists constants to be used for Data Surface Types.

Constant Name	Constant Name
FDSU_TABULAR	FDSU_4PT_BILINEAR
FDSU_EQ	FDSU_8PT_TRILINEAR
FDSU_NODE	FDSU_2PT_LINEAR
FDSU_ELEM	FDSU_ARB_3D
FDSU_ALG_CURVE	FDSU_OUTPUT_MAP
FDSU_ALG_SURF	FDSU_LOADSET_COMBINE
FDSU_ALG_COORD	FDSU_RESULTSET_COMBINE

The following table lists constants to be used for Data Surface Cell Types.

Constant Name	Constant Name
FDT_VALUE_X	FDT_PAGEVALUE
FDT_VALUE_Y	FDT_BASECOLUMNTITLE
FDT_VALUE_Z	FDT_COLUMNTITLE

Unpublished work. © 2025 Siemens

FDT_ROWVALUE	FDT_PAGETITLE
FDT_COLUMNVALUE	FDT_PAGESIZE

The following table lists constants to be used for Data Surface Dimension.

Constant Name	Constant Name
FDM_1D	FDM_3D
FDM_2D	

Data Table Constants

The Data Table is used to create tables of data extracted from Model(s) currently open in Femap which are presented in a spread sheet format.

The following table lists constants to be used for the Data Table Column Type.

Constant Name	Constant Name
FCT_BOOL	FCT_DOUBLE
FCT_INT	FCT_STRING

The following table lists constants to be used for the Data Table Alignment.

Constant Name	Constant Name
FAL_LEFT	FAL_RIGHT
FAL_CENTER	

The following table lists constants to be used for the Data Table Save to a File Format

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FSF_TEXT	FSF_RTF
FSF_CSV	FSF_HTML

Results ID Query

The constants for the Results ID Query Object are located in the documentation for the object itself, as it was logical to put all of that information in single section. For more information, see **Results Vector ID Query Constants (Enumerations)**.

Output Types

The following table lists constants to be used for Output Types.

Constant Name	Constant Name
FOT_ANY	FOT_STRESS
FOT_DISP	FOT_STRAIN
FOT_ACCEL	FOT_TEMP
FOT_FORCE	

Output - Complex Types

The following table lists constants to be used for Complex Output Types.

Constant Name	Constant Name
FOC_ANY	FOC_REAL
FOC_MAGNITUDE	FOC_IMAGINARY
FOC_PHASE	

Output - Transformation Types

The following table lists constants to be used for Output Transformation Types.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FOD_NONE	FOD_PLATE_TO_VECTOR
FOD_VECTOR_TO_CSYS	FOD_SOLID_TO_CSYS
FOD_VECTOR_TO_NODE_OUTPUT_CSYS	FOD_SOLID_TO_MATL
FOD_PLATE_TO_MATL	FOD_LINE_TO_CSYS
FOD_PLATE_TO_CSYS	

Output - Invariant Types

The following table lists constants to be used for Output Invariant Types.

Constant Name	Constant Name
FIVT_MAX_PRIN	FIVT_MEAN_PRIN
FIVT_MIN_PRIN	FIVT_MAX_SHEAR
FIVT_MID_PRIN	FIVT_VON_MISES

Output Set - Output Process Operation (Combination) Types

The following table lists constants to be used for Output Processing Operation Types.

Constant Name	Constant Name
FRPROC_NONE	FRPROC_ENV_ABSMAX
FRPROC_LINEAR	FRPROC_ENV_MAX_SETID
FRPROC_RSS	FRPROC_ENV_MIN_SETID
FRPROC_ENV_MAX	FRPROC_ENV_ABSMAX_SETID
FRPROC_ENV_MIN	

Unpublished work. © 2025 Siemens

View Modes

The following table lists constants to be used for View Modes.

Constant Name	Constant Name
FVM_DRAW	FVM_XY_VS_ID
FVM_FEATURE	FVM_XY_VS_CASE
FVM_SORT	FVM_XY_VS_VALUE
FVM_HIDE	FVM_XY_VS_POSITION
FVM_FREE	FVM_XY_OF_FUNCTION
FVM_FREE_FACE	

Entity Visibility

The following table lists constants to be used for Entity Visibility.

Constant Name	Constant Name
FVIS_POINT	FVIS_ELEM_TYPE
FVIS_CURVE	FVIS_ELEM_SHAPE
FVIS_SURFACE	FVIS_MATL
FVIS_SOLID	FVIS_PROP
FVIS_CSYS	FVIS_AERO_PANEL
FVIS_CONNECTION_REGION	FVIS_AERO_SPLINE
FVIS_CONNECTION	FVIS_AERO_SURFACE
FVIS_ELEM	FVIS_FREEBODY

Unpublished work. © 2025 Siemens

View Options

A list of the View Options constants for use with Draw, Label, Color, and ColorMode can be found in **View Option Types** with Text in View Options and Index number.

Deformed Formats

The following table lists constants to be used for Deformed Formats.

Constant Name	Constant Name
FDF_NONE	FDF_ANIMATE_MULTICASE
FDF_MODEL_UNDEF	FDF_ARROW
FDF_DEFORMED	FDF_TRACE
FDF_ANIMATE	FDF_STREAMLINE

Contour Formats

The following table lists constants to be used for Contour Formats.

Constant Name	Constant Name
FCF_NONE	FCF_BEAM_DIAGRAM
FCF_MODEL_COLOR	FCF_ISOSURFACE
FCF_CONTOUR	FCF_SECTION_CUT
FCF_CRITERIA	FCF_CONTOUR_VECTOR

Freebody Display Modes

The following table lists constants to be used for Freebody Display Mode.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FBD_DISPLAYMODE_FREEBODY	FBD_DISPLAYMODE_SECTION
FBD_DISPLAYMODE_INTERFACE	

Freebody Vector Modes

The following table lists constants to be used for Freebody Vector Display Mode.

Constant Name	Constant Name
FBD_VEC_MODE_OFF	FBD_VEC_MODE_RESULTANT
FBD_VEC_MODE_COMPONENT	

Freebody Components

The following table lists constants to be used for Freebody Components to Display.

Constant Name	Constant Name
FBC_FX	FBC_MX
FBC_FY	FBC_MY
FBC_FZ	FBC_MZ

Freebody Contributions

The following table lists constants to be used for Freebody Contributions.

Constant Name	Constant Name
FBL_APPLIED	FBL_EL_PER
FBL_SPC	FBL_TOTAL

Unpublished work. © 2025 Siemens

FBL_MPC	FBL_CONTACT
FBL_EL_FB	FBL_GLUE

Freebody Section Cut Entity Selection Modes

The following table lists constants to be used for Freebody Entity Selection Mode when Freebody Display Mode is set to Section Cut.

Constant Name	Constant Name
FBD_SECTMODE_PLANENORMAL	FBD_SECTMODE_VECTOR
FBD_SECTMODE_PLANEVECTOR	FBD_SECTMODE_CURVE

Freebody Section Cut Summation Location

The following table lists constants to be used for Summation Location when Freebody Display Mode is set to Section Cut.

Constant Name	Constant Name
FBD_SECTION_SUMLOC_BASE	FBD_SECTION_SUMLOC_LOCATION
FBD_SECTION_SUMLOC_CENTROID	

Group Data Types

The following table lists constants to be used when selecting DataTypes for Groups.

Constant Name		Constant Name	
FGR_CSYS	0	FGR_BEQ	14
FGR_POINT	1	FGR_PLOAD	15
FGR_CURVE	2	FGR_CLOAD	16
FGR_SURFACE	3	FGR_SLOAD	17

Unpublished work. © 2025 Siemens

FGR_VOLUME	4	FGR_PBCO	18
FGR_TEXT	5	FGR_CBCO	19
Not Used	6	FGR_SBCO	20
FGR_NODE	7	FGR_SOLID	21
FGR_ELEM	8	FGR_CONNECTION	22
FGR_MATL	9	FGR_CONTACT	23
FGR_PROP	10	FGR_CONNECTIONPROP	24
FGR_NLOAD	11	FGR_REGLOAD	25
FGR_ELOAD	12	FGR_LAYUP	26
FGR_BCO	13	FGR_MESHPOINT	27

Group Definition Types

The following table lists constants to be used for adding Text to Groups.

Constant Name	Constant Name
FGD_TEXT_ID	FGD_TEXT_BYLAYER
FGD_TEXT_BYCOLOR	

The following table lists constants to be used for adding Points to Groups.

Constant Name	Constant Name
FGD_POINT_ID	FGD_POINT_ONSOLID
FGD_POINT_BYDEFCSYS	FGD_POINT_BYCOLOR

Unpublished work. © 2025 Siemens

FGD_POINT_ONCURVE	FGD_POINT_BYLAYER
FGD_POINT_ONSURFACE	FGD_POINT_BYPROP

The following table lists constants to be used for adding Curves to Groups.

Constant Name	Constant Name
FGD_CURVE_ID	FGD_CURVE_BYLAYER
FGD_CURVE_BYPOINT	FGD_CURVE_BYPROP
FGD_CURVE_ONSURFACE	FGD_CURVE_ONSOLID
FGD_CURVE_BYCOLOR	

The following table lists constants to be used for adding Surfaces to Groups.

Constant Name	Constant Name
FGD_SURFACE_ID	FGD_SURFACE_BYLAYER
FGD_SURFACE_BYCURVE	FGD_SURFACE_BYPROP
FGD_SURFACE_ONVOLUME	FGD_SURFACE_ONSOLID
FGD_SURFACE_BYCOLOR	

The following table lists constants to be used for adding Volumes to Groups.

Constant Name	Constant Name
FGD_VOLUME_ID	FGD_VOLUME_BYLAYER
FGD_VOLUME_BYSURFACE	FGD_VOLUME_BYPROP
FGD_VOLUME_BYCOLOR	

The following table lists constants to be used for adding Solids to Groups.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FGD_SOLID_ID	FGD_SOLID_BYLAYER
FGD_SOLID_BYCURVE	FGD_SOLID_BYPROP
FGD_SOLID_BYSURFACE	FGD_SOLID_BYTYPE
FGD_SOLID_BYCOLOR	

The following table lists constants to be used for adding Mesh Points to Groups.

Constant Name	Constant Name
FGD_MESHPOINT_ID	FGD_MESHPOINT_BYSOLID
FGD_MESHPOINT_BYPOINT	FGD_MESHPOINT_BYCOLOR
FGD_MESHPOINT_BYCURVE	FGD_MESHPOINT_BYLAYER
FGD_MESHPOINT_BYSURFACE	

The following table lists constants to be used for adding Connection Properties to Groups.

Constant Name	Constant Name
FGD_CONNECTIONPROP_ID	FGD_CONNECTIONPROP_BYCOLOR
FGD_CONNECTIONPROP_BYCOLOR	FGD_CONNECTIONPROP_ONCONNECTION

The following table lists constants to be used for adding Regions to Groups.

Constant Name	Constant Name
FGD_CONNECTION_ID	FGD_CONNECTION_BYPROP
FGD_CONNECTION_BYCOLOR	FGD_CONNECTION_BYCONTACT
FGD_CONNECTION_BYLAYER	

Unpublished work. © 2025 Siemens

The following table lists constants to be used for adding Connectors (contact pairs) to Groups.

Constant Name	Constant Name
FGD_CONTACT_ID	FGD_CONTACT_BYELEM
FGD_CONTACT_BYCOLOR	FGD_CONTACT_BYNODE
FGD_CONTACT_BYLAYER	FGD_CONTACT_BYCURVE
FGD_CONTACT_ONCONNECTION	FGD_CONTACT_BYSURFACE
FGD_CONTACT_ONSOLID	FGD_CONTACT_BYPROP

The following table lists constants to be used for adding Coordinate Systems to Groups.

Constant Name	Constant Name
FGD_CSYS_IDF	FGD_CSYS_ONNODE
FGD_CSYS_BYDEFCSYS	FGD_CSYS_ONELEM
FGD_CSYS_BYTYPE	FGD_CSYS_ONPOINT
FGD_CSYS_BYCOLOR	FGD_CSYS_ONCSYS
FGD_CSYS_BYLAYER	FGD_CSYS_ONPROP

The following table lists constants to be used for adding Nodes to Groups.

Constant Name	Constant Name
FGD_NODE_ID	FGD_NODE_BYLAYER
FGD_NODE_BYDEFCSYS	FGD_NODE_ONELEMASORIENT
FGD_NODE_BYOUTCSYS	FGD_NODE_BYSUPERELEMENT
FGD_NODE_ONELEM	FGD_NODE_BYTYPE

Unpublished work. © 2025 Siemens

FGD_NODE_ATPOINT	FGD_NODE_ONREGION
FGD_NODE_ATCURVE	FGD_NODE_ATPOINTSOFSOLID
FGD_NODE_ATSURFACE	FGD_NODE_ATCURVESOFSOLID
FGD_NODE_ATSOLID	FGD_NODE_ATSURFACESOFSOLID
FGD_NODE_BYCOLOR	

The following table lists constants to be used for adding Elements to Groups.

Constant Name	Constant Name
FGD_ELEM_ID	FGD_ELEM_BYLAYER
FGD_ELEM_BYMATL	FGD_ELEM_BYSHAPE
FGD_ELEM_BYPROP	FGD_ELEM_BYLAYUP
FGD_ELEM_BYTYPE	FGD_ELEM_BYALLNODES
FGD_ELEM_BYNODE	FGD_ELEM_BYGLOBALPLY
FGD_ELEM_ATPOINT	FGD_ELEM_BYORIENTNODE
FGD_ELEM_ATCURVE	FGD_ELEM_ALLNODESONSOLID
FGD_ELEM_ATSURFACE	FGD_ELEM_ATPOINTSOFSOLID
FGD_ELEM_ATSOLID	FGD_ELEM_ATCURVESOFSOLID
FGD_ELEM_BYCOLOR	FGD_ELEM_ATSURFACESOFSOLID

The following table lists constants to be used for adding Materials to Groups.

Constant Name	Constant Name
FGD_MATL_ID	FGD_MATL_BYTYPE

Unpublished work. © 2025 Siemens

FGD_MATL_ONPROP	FGD_MATL_BYCOLOR
FGD_MATL_ONELEM	FGD_MATL_BYLAYER

The following table lists constants to be used for adding Properties to Groups.

Constant Name	Constant Name
FGD_PROP_ID	FGD_PROP_BYCOLOR
FGD_PROP_ONELEM	FGD_PROP_BYLAYER
FGD_PROP_BYMATL	FGD_PROP_BYLAYUP
FGD_PROP_BYTYPE	

The following table lists constants to be used for adding Layups to Groups.

Constant Name	Constant Name
FGD_LAYUP_ID	FGD_LAYUP_ONPROP
FGD_LAYUP_BYMATL	FGD_LAYUP_BYGLOBALPLY

The following table lists constants to be used for adding Loads to Groups.

Constant Name	Constant Name
FGD_LOAD_BYNODE	FGD_LOAD_BYCURVE
FGD_LOAD_BYELEM	FGD_LOAD_BYSURFACE
FGD_LOAD_BYPOINT	FGD_LOAD_BYREGION

The following table lists constants to be used for adding Boundary Conditions to Groups.

Constant Name	Constant Name
FGD_BCO_ID	FGD_BCO_BYCURVE

Unpublished work. © 2025 Siemens

FGD_BEQ_BYNODE	FGD_BCO_BYSURFACE
FGD_BCO_BYPOINT	

The following table lists constants to be used for adding Aero Entities to Femap Sets

Although Aero entities are not currently supported in Femap groups the group rules can be used for adding Aero entities to Femap Set Objects.

Constant Name	Constant Name
FGD_AEROPROP_ID	FGD_AEROSPLINE_BYCOLOR
FGD_AEROPROP_BYCOLOR	FGD_AEROSPLINE_BYLAYER
FGD_AEROPROP_BYLAYER	FGD_AEROSPLINE_BYPANEL
FGD_AEROPANEL_ID	FGD_AEROCTRLSURF_ID
FGD_AEROPANEL_BYCOLOR	FGD_AEROCTRLSURF_BYCOLOR
FGD_AEROPANEL_BYLAYER	FGD_AEROCTRLSURF_BYLAYER
FGD_AEROSPLINE_ID	

Print Source Options

The following table lists constants to be used for Print Source Options.

Constant Name	Constant Name
FPS_VIEW	FPS_MESSAGES
FPS_DESKTOP	FPS_PROGRAMFILE
FPS_FILE	FPS_ENTITYINFO
FPS_LAYOUT	FPS_APIPROGRAM

Unpublished work. © 2025 Siemens

Return Codes

The following table lists constants to be used for Return Codes. These can be helpful when testing and debugging a program created with the Femap API.

Constant Name		Constant Name	
FE_OK	-1	FE_BAD_DATA	9
FE_FAIL	0	FE_NO_MEMORY	10
FE_CANCEL	2	FE_NEGATIVE_MASS_VOLUME	11
FE_INVALID	3	FE_INVALID_DEVELOPER	12
FE_NOT_EXIST	4	FE_NO_VALID_GRAPHICS_WINDOW	13
FE_SECURITY	5	FE_NO_VALID_GRAPHIC_VIEW	14
FE_NOT_AVAILABLE	6	FE_FILE_OPEN_FAILED	15
FE_TOO_SMALL	7	FE_NO_FILENAME	16
FE_BAD_TYPE	8	FE_FILE_WRITE_FAILED	17

Femap Events

The following table lists constants to be used for Femap Events.

Constant Name		Constant Name	
FEVENT_INITIALIZE	1	FEVENT_ANALYSISEND	10
FEVENT_NEWMODEL	2	FEVENT_RESULTSEND	11
FEVENT_ENDMODEL	3	FEVENT_SWITCHMODEL	12
FEVENT_SHUTDOWN	4	FEVENT_SELECTOR	13
FEVENT_COMMAND	5	FEVENT_UPDATEDSAVEDSET	14

Unpublished work. © 2025 Siemens

FEVENT_PROGSTART	6	FEVENT_CLOSINGMODEL	15
FEVENT_PROGEND	7	FEVENT_LEAVINGMODEL	16
FEVENT_DRAWSTART	8	FEVENT_FILEOPENEND	17
FEVENT_DRAWEND	9	FEVENT_FILEIMPORTEND	18

Femap Colors

The following table lists constants to be used for Colors located in the First and Second Rows of the Femap color palette. Color Number listed in Column.

Constant Name		Constant Name	
FCL_BLACK	0	FCL_MIDNIGHTBLUE	25
FCL_SEPIA	1	FCL_EGGPLANT	26
FCL_DARKRED	2	FCL_DARKRASPBERRY	27
FCL_MAROON	3	FCL_RASPBERRY	28
FCL_RED	4	FCL_CRIMSON	29
FCL_DARKESTGREEN	5	FCL_DARKSLATEGRAY	30
FCL_DARKOLIVEGREEN	6	FCL_DARKGRAY	31
FCL_BROWN	7	FCL_CAMOUFLAGEGREEN	32
FCL_SIENNA	8	FCL_FIREBRICK	33
FCL_ORANGERED	9	FCL_TOMATO	34
FCL_GREEN	10	FCL_PINEGREEN	35
FCL_FERNGREEN	11	FCL_FORESTGREEN	36
FCL_OLIVE	12	FCL_PALEOLIVE	37

Unpublished work. © 2025 Siemens

FCL_DARKGOLDENROD	13	FCL_PERU	38
FCL_ORANGE	14	FCL_CORAL	39
FCL_LIMEGREEN	15	FCL_DARKPASTELGREEN	40
FCL_YELLOWLIMEGREEN	16	FCL_PALELIMEGREEN	41
FCL_AVOCADO	17	FCL_YELLOWGREEN	42
FCL_OLIVEYELLOW	18	FCL_OLDGOLD	43
FCL_GOLD	19	FCL_GOLDENROD	44
FCL_LIME	20	FCL_BLUELIME	45
FCL_BRIGHTLIME	21	FCL_PALELIME	46
FCL_CHARTREUSE	22	FCL_PALEBRIGHTGREEN	47
FCL_BRIGHTCHARTREUSE	23	FCL_GREENYELLOW	48
FCL_YELLOW	24	FCL_NOTEYELLOW	49

The following table lists constants to be used for Colors located in the Third and Fourth Rows of the Femap color palette. Color Number listed in Column.

Constant Name		Constant Name	
FCL_DARKBLUE	50	FCL_MEDIUMBLUE	75
FCL_INDIGO	51	FCL_BLUEINDIGO	76
FCL_PURPLE	52	FCL_DARKVIOLET	77
FCL_MEDIUMVIOLETRED	53	FCL_VIOLETEGGPLANT	78
FCL_DEEPPINK	54	FCL_FUCHSIA	79

Unpublished work. © 2025 Siemens

FCL_DARKCERULEAN	55	FCL_COBALT	80
FCL_DARKSLATEBLUE	56	FCL_PALEBLUEPURPLE	81
FCL_PALEPURPLE	57	FCL_MEDIUMPURPLE	82
FCL_MAUVE	58	FCL_ORCHID	83
FCL_CERISE	59	FCL_PINKCERISE	84
FCL_DARKCYAN	60	FCL_CERULEAN	85
FCL_VIRIDIAN	61	FCL_STEELBLUE	86
FCL_GRAY	62	FCL_SLATEBLUE	87
FCL_ROSYBROWN	63	FCL_PLUM	88
FCL_LIGHTSALMON	64	FCL_REDPINK	89
FCL_JADE	65	FCL_DARKTURQUOISE	90
FCL_SEAGREEN	66	FCL_MEDIUMTURQUOISE	91
FCL_DARKSEAGREEN	67	FCL_MEDIUMAQUAMARINE	92
FCL_DARKKHAKI	68	FCL_SILVER	93
FCL_SANDYBROWN	69	FCL_PALEREDVIOLET	94
FCL_SPRINGGREEN	70	FCL_MEDIUMSPRINGGREEN	95
FCL_LIGHTSPRINGGREEN	71	FCL_TURQUOISE	96
FCL_LIGHTGREEN	72	FCL_AQUAMARINE	97
FCL_TEAGREEN	73	FCL_MINTCREAM	98
FCL_PALEYELLOW	74	FCL_LEMONCHIFFON	99

Unpublished work. © 2025 Siemens

The following table lists constants to be used for Colors located in the Fifth and Sixth Rows of the Femap color palette. Color Number listed in Column.

Constant Name		Constant Name	
FCL_BLUE	100	FCL_GRAYBLUE_EXTRADARK	150
FCL_PERSIANBLUE	101	FCL_GRAYBLUE_DARK	151
FCL_VIOLET	102	FCL_GRAYBLUE	152
FCL_BRIGHTVIOLET	103	FCL_GRAYBLUE_LIGHT	153
FCL_MAGENTA	104	FCL_GRAYBLUE_EXTRALIGHT	154
FCL_BRIGHTCOBALT	105	FCL_GRAYGREEN_EXTRADARK	155
FCL_PALECOBALT	106	FCL_GRAYGREEN_DARK	156
FCL_BLUEVIOLET	107	FCL_GRAYGREEN	157
FCL_MEDIUMORCHID	108	FCL_GRAYGREEN_LIGHT	158
FCL_HOTPINK	109	FCL_GRAYGREEN_EXTRALIGHT	159
FCL_DODGERBLUE	110	FCL_GRAYBEIGE_EXTRADARK	160
FCL_ROYALBLUE	111	FCL_GRAYBEIGE_DARK	161
FCL_MEDIUMSLATEBLUE	112	FCL_GRAYBEIGE	162
FCL_WISTERIA	113	FCL_GRAYBEIGE_LIGHT	163
FCL_PINK	114	FCL_GRAYBEIGE_EXTRALIGHT	164
FCL_DEEPSKYBLUE	115	FCL_USER1	165
FCL_SKYBLUE	116	FCL_USER2	166
FCL_PALESKYBLUE	117	FCL_USER3	167

Unpublished work. © 2025 Siemens

FCL_THISTLE	118	FCL_USER4	168
FCL_PALEPINK	119	FCL_USER5	169
FCL_CYAN	120	FCL_USER6	170
FCL_BRIGHTTURQUOISE	121	FCL_USER7	171
FCL_PALEBLUE	122	FCL_USER8	172
FCL_POWDERBLUE	123	FCL_USER9	173
FCL_WHITE	124	FCL_USER10	174

The following table lists constants to be used for Colors located in the Grayscale Row of the Femap color palette. Color Number listed in Column.

Constant Name	
GRAY10	125
FCL_GRAY20	126
FCL_GRAY30	127
FCL_GRAY40	128
FCL_GRAY50	129
FCL_GRAY60	130
FCL_GRAY70	131
FCL_GRAY80	132
FCL_GRAY90	133
FCL_GRAY100	134

Unpublished work. © 2025 Siemens

FCL_GRAY110	135
FCL_GRAY120	136
FCL_GRAY130	137
FCL_GRAY140	138
FCL_GRAY150	139
FCL_GRAY160	140
FCL_GRAY170	141
FCL_GRAY180	142
FCL_GRAY190	143
FCL_GRAY200	144
FCL_GRAY210	145
FCL_GRAY220	146
FCL_GRAY230	147
FCL_GRAY240	148
FCL_GRAY250	149

Femap Brush Patterns

The following table lists constants to be used for Femap Brush Patterns. These can be added to a Femap color using an “or” between the Color Name and the Brush Pattern Constant. For Example, to make a green color with level 4 transparency, you would put `FCL_GREEN or FPF_TRANSPARENT4`.

Constant Name	Constant Name
FPF_SOLID	FPF_TRANSPARENT7

Unpublished work. © 2025 Siemens

FPF_HORIZONTAL	FPF_TRANSPARENT6
FPF_VERTICAL	FPF_TRANSPARENT5
FPF_FDIAGONAL	FPF_TRANSPARENT4
FPF_BDIAGONAL	FPF_TRANSPARENT3
FPF_CROSS	FPF_TRANSPARENT2
FPF_DIAGCROSS	FPF_TRANSPARENT1
FPF_TRANSPARENT8	

Femap Pen Linestyles

The following table lists constants to be used for Femap Pen Linestyles. These can be added to a Femap color using an “or” between the Brush Pattern Constant and the Pen Linestyle.

Constant Name	Constant Name
FPL_SOLID	FPL_DASHDOTDOT
FPL_DASH	FPL_THICK2
FPL_DOT	FPL_THICK3
FPL_DASHDOT	FPL_THICK4

Femap Picture Regions

The following table lists constants to be used to determine which portion of the Femap interface to copy or save.

Constant Name	Constant Name
FPRG_WINDOW	FPRG_DESKTOP
FPRG_LAYOUT	

Unpublished work. © 2025 Siemens

Femap Picture Formats

The following table lists constants to be used for Femap Picture Formats.

Constant Name	Constant Name
FPM_BMP	FPM_JT
FPM_METAFILE	(Constant Reserved)
FPM_PLACEMF	FPM_GIF
FPM_JPEG	FPM_GIFANIM
FPM_BMPSERIES	FPM_TIF
FPM_AVI	FPM_PNG
FPM_AVI_UNCOMPRESSED	

Femap Picture Formats

The following table lists constants to be used for Femap Picture Formats used by the `feCrossSectionReport` function.

Constant Name	Constant Name
FPM2_BMP	FPM2_TIF
FPM2_JPEG	FPM2_PNG
FPM2_GIF	

Femap Default Picture Formats

The following table lists constants to be used for Femap Default Picture Formats which can be set via the Preferences.

Constant Name	Constant Name
PPFM_BMP	PPFM_GIF

Unpublished work. © 2025 Siemens

PPFM_JPEG	PPFM_TIF
PPFM_PNG	

Femap Default Animation Formats

The following table lists constants to be used for Femap Default Animation Formats which can be set via the Preferences.

Constant Name	Constant Name
PAFM_BMP	PAFM_AVI
PAFM_BMPSERIES	PAFM_GIF

Femap GIF Options

The following table lists constants to be used for GIF Formats which can be set via the Preferences.

Constant Name	Constant Name	Constant Name
GIFC_NET	GIFC_OCTREE	GIFC_DITHER

Femap Language Options

The following table lists constants to be used for the Language preference on **User Interface** tab.

Constant Name	Constant Name
FLNG_AUTO	FLNG_JP
FLNG_EN	FLNG_TW
FLNG_DE	FLNG_CN

Messages Window Text Colors

The following table lists constants to be used for Messages Window Text Colors.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FCM_NORMAL	FCM_ERROR
FCM_HIGHLIGHT	FCM_COMMAND
FCM_WARNING	

PublishTool Object

The following table lists constants used by `AddPicture` method.

Constant Name	Constant Name
PUBIF_BITMAP	PUBIF_GIF
PUBIF_JPEG	PUBIF_TIF
PUBIF_PNG	

The following table lists constants used by `SetFormat` method.

Constant Name	Constant Name
PUB_RTF	PUB_HTML

The following table lists constants used by SetParagraphAlignment method.

Constant Name	Constant Name	Constant Name
PUBALN_LEFT	PUBALN_CENTER	PUBALN_RIGHT

Femap Default Solver/Translator

The following table lists constants used to set a default solver (Interface option on the **Interfaces** tab in the Preferences). There are also constants for the geometry translators.

Constant Name	Constant Name
FTR_NONE	FTR_ODBC

Unpublished work. © 2025 Siemens

FTR_MSC_NASTRAN	FTR_CSA_NASTRAN
FTR_PC_NASTRAN	FTR_SLA
FTR_PAL_2	FTR_CFDESIGN
FTR_STARDYNE	FTR_ACIS
FTR_ANSYS	FTR_PARASOLID
FTR_ANSYS_PC_LINEAR	FTR_LS_DYNA
FTR_ALGOR	FTR_MARC
FTR_COSMOS	FTR_SINDA
FTR_ATRAN	FTR_NE_NASTRAN
FTR_FEMAP_NEUTRAL	FTR_SATOOLKIT
FTR_DXF	FTR_TMG
FTR_IGES	FTR_MODEL_SOLUTION
FTR_TABLE	FTR_CATIA
FTR_UAI_NASTRAN	FTR_PROE
FTR_COSMIC_NASTRAN	FTR_VDA
FTR_STAAD	FTR_SOLIDEDGE
FTR_ABAQUS	FTR_STEP
FTR_WECAN	FTR_UNIGRAPHICS
FTR_SAP	FTR_VR_NASTRAN
FTR_CDA_SPRINT	FTR_NX_NASTRAN

Unpublished work. © 2025 Siemens

FTR_CAEFEM	FTR_CATIA_V5
FTR_IDEAS	FTR_SOLIDWORKS
FTR_ME_NASTRAN	FTR_TOSCA
FTR_ADINA	FTR_JT
FTR_GENESIS	FTR_FIBERSIM

Coordinate dialog box Methods

The following table lists constants to be used for Coordinate dialog box Methods. These can be helpful when testing and debugging a program created with the Femap API.

Constant Name	Constant Name
FCD_COORDINATES	FCD_BETWEEN
FCD_WORKPLANE_COORDINATES	FCD_INTERSECTION
FCD_USE_POINTS	FCD_MIDPOINT
FCD_USE_NODES	FCD_PROJECT_ON_CURVE
FCD_OFFSET	FCD_PROJECT_ON_SURFACE
FCD_DISTANCE	FCD_ON_SURFACE
FCD_CENTER	FCD_INTERSECTCURVESURFACE
FCD_ALONG	FCD_CG_SURFACE
FCD_MESH_LOCATION	FCD_ALONGLENGTH

Vector dialog box Methods

The following table lists constants to be used for Vector dialog box Methods.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FVD_POSITION	FVD_BISECT
FVD_POSITION_LENGTH	FVD_NORMAL
FVD_USE_POINTS	FVD_DEFCS_AXIS
FVD_USE_NODES	FVD_GLOBAL_AXIS
FVD_COMPONENTS	FVD_NORMAL_TO_VIEW
FVD_DIRECTION	FVD_AXIS_OF_SURFACE
FVD_TANGENT	FVD_SAVED
FVD_NORMAL_TO_SURFACE	

Plane dialog box Methods

The following table lists constants to be used for Plane dialog box Methods.

Constant Name	Constant Name
FPD_POSITION	FPD_DEFCS_AXIS
FPD_USE_POINTS	FPD_GLOBAL_AXIS
FPD_USE_NODES	FPD_NORMAL_TO_VIEW
FPD_COMPONENTS	FPD_NORMAL_TO_CURVE
FPD_BISECT	FPD_NORMAL_TO_SURFACE
FPD_NORMAL	FPD_SAVED

Selector Constants

The Selector (Select Toolbar) is used to create *Selection sets* in Femap which may contain different entity types and can also be used to load entities into the **Data Table** and **Entity Editor**.

Unpublished work. © 2025 Siemens

The following table lists constants to be used for the Selector Entity Type.

Constant Name	Constant Name
FS_POINT	FS_LOAD_NODAL
FS_CURVE	FS_LOAD_ELEMENTAL
FS_SURFACE	FS_LOAD_ON_POINT
FS_SOLID	FS_LOAD_ON_CURVE
FS_CONNECTION_PROP	FS_LOAD_ON_SURFACE
FS_CONNECTION_REGION	FS_BC_NODAL
FS_CONNECTION	FS_BC_EQUATION
FS_CSYS	FS_BC_ON_POINT
FS_NODE	FS_BC_ON_CURVE
FS_ELEM	FS_BC_ON_SURFACE
FS_MATL	FS_TEXT
FS_PROP	

The following table lists constants to be used for Selector Pick (Drill) Method.

Constant Name	Constant Name
FSD_OFF	FSD_FRONT
FSD_QUERY	

Femap Library Options

The following table lists constants to be used to specify a tab in the various Femap Library dialog boxes.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FLIB_DIALOG	FLIB_SHARED
FLIB_PERSONAL	FLIB_SYSTEM

Real Number Formats

The following table lists formats available for writing out real numbers.

Constant Name	Constant Name
FRFM_NORMAL	FRFM_NASTRAN
FRFM_EXPONENTIAL	

Chart Data Series Types

The following table lists constants to be used for Chart Data Series Type in the **Charting** pane.

Constant Name	Constant Name
FCHD_TYPE_ID	FCHD_TYPE_VECVEC
FCHD_TYPE_SET	FCHD_TYPE_CPLXEXP
FCHD_TYPE_SETVAL	FCHD_TYPE_COMBO
FCHD_TYPE_POSITION	FCHD_TYPE_COMBOVAL
FCHD_TYPE_FUNCTION	

Chart Data Series Complex Conversion Methods

The following table specifies available methods data series can use to convert complex output.

Constant Name	Constant Name
FCSCM_NONE	FCSC_ADD

Unpublished work. © 2025 Siemens

FCSCM_MATCHMODEL	FCSC_SUBTRACT
FCSCM_MATCHVIEW	FCSC_MULTIPLY
FCSCM_SYNC	FCSC_DIVIDE

Chart Styles

The following table lists constants to be used for Chart Style in the **Charting** pane.

Constant Name	Constant Name
FCH_STYLE_POINT	FCH_STYLE_LINESTEP
FCH_STYLE_LINE	FCH_STYLE_AREA
FCH_STYLE_LINEFAST	FCH_STYLE_AREASTEP

Chart Axis Styles

The following table lists constants to be used for Chart Axis Styles in the **Charting** pane.

Constant Name	Constant Name
FCH_AXISSTYLE_LINEAR	FCH_AXISSTYLE_LOG

Chart Palette

The following table lists constants to be used for Chart Palette in the **Charting** pane.

Constant Name	Constant Name
FCH_PALETTE_FEMAP	FCH_PALETTE_BLUE
FCH_PALETTE_GRAY	FCH_PALETTE_BROWN
FCH_PALETTE_OFFICE	FCH_PALETTE_GREEN
FCH_PALETTE_VIBRANT	FCH_PALETTE_RED

Unpublished work. © 2025 Siemens

Chart Legend Location

The following table lists constants to be used for Chart Legend Location in the **Charting** pane.

Constant Name	Constant Name
FCH_LEGEND_LOCATION_NEAROUTSIDE	FCH_LEGEND_LOCATION_FAR
FCH_LEGEND_LOCATION_NEAR	FCH_LEGEND_LOCATION_FAROUTSIDE
FCH_LEGEND_LOCATION_CENTER	

Chart Legend Direction

The following table lists constants to be used for Chart Legend Direction in the **Charting** pane.

Constant Name
FCH_LEGEND_DIRECTION_TOPTOBOTTOM
FCH_LEGEND_DIRECTION_LEFTTORIGHT

Chart Title Location

The following table lists constants to be used for Chart Title Location in the **Charting** pane.

Constant Name	Constant Name
FCH_TITLE_LOCATION_TOP	FCH_TITLE_LOCATION_RIGHT
FCH_TITLE_LOCATION_LEFT	FCH_TITLE_LOCATION_BOTTOM

Chart Text Justification

The following table lists constants to be used for Chart Text Justification in the **Charting** pane.

Constant Name	Constant Name
FCH_TEXT_JUSTIFICATION_LEFT	FCH_TEXT_JUSTIFICATION_RIGHT
FCH_TEXT_JUSTIFICATION_CENTER	

Unpublished work. © 2025 Siemens

Chart Number Format

The following table lists constants to be used for Chart Number Format in the **Charting** pane.

Constant Name	Constant Name
FCH_FORMAT_STANDARD	FCH_FORMAT_PERCENTAGE
FCH_FORMAT_SCIENTIFIC	

Chart Marker Style

The following table lists constants to be used for Chart Marker Style in the **Charting** pane.

Constant Name	Constant Name
FCH_MARKERSTYLE_CIRCLE	FCH_MARKERSTYLE_PENTAGON
FCH_MARKERSTYLE_SQUARE	FCH_MARKERSTYLE_HEXAGON
FCH_MARKERSTYLE_DIAMOND	FCH_MARKERSTYLE_STAR
FCH_MARKERSTYLE_TRIANGLE	

Chart Secondary Plot Location

The following table lists constants to be used for ComplexPlotLocation in the **Charting** pane.

Constant Name	Constant Name
FCCL_OFF	FCCL_BOTTOM
FCCL_TOP	

Results Exist State

The following table lists constants to be used to determine if data exists in the Results Browsing Object.

Constant Name	Constant Name
FEX_EXISTING	FEX_NONEXISTING

Unpublished work. © 2025 Siemens

Results Locate Options

The following table lists constants to be used to locate results in the Results Browsing Object.

Constant Name	Constant Name
FLO_AFTER	FLO_AFTER_EQUAL
FLO_BEFORE	FLO_BEFORE_EQUAL

Results Conversion Options

The following table lists constants used to specify data conversion in the Results Browsing Object.

Constant Name	Constant Name
FRC_NONE	FRC_AVG_SKIP_CORNER
FRC_AVG	FRC_MAX_SKIP_CORNER
FRC_MAX	FRC_MIN_SKIP_CORNER
FRC_MIN	

Attached Results Management

The following table lists constants to be used to manage attached results files.

Constant Name	Constant Name
FMRA_LOAD	FMRA_DETACH
FMRA_UNLOAD	FMRA_UPDATE_PATH

Attached Results Status

The following table lists constants to be used to determine status of attached results files.

Constant Name	Constant Name
FRA_UNLOADED	FRA_ERROR

Unpublished work. © 2025 Siemens

FRA_OPEN	FRA_UNLOAD_CHECK
FRA_OPEN_CHECK	

Results Location

The following table lists constants to be used to determine the location of results, which may be in the Femap database, in an attached results file, or in a virtual output set.

Constant Name	Constant Name
FRL_DB	FRL_CSV
FRL_OP2	FRL_VIRTUAL
FRL_FNO	FRL_ODB
FRL_XDB	

Beam Calculator Stress Component

The following table lists constants used for to specify which stress component to return when using the Beam Calculator.

Constant Name	Constant Name
FBMC_SC_ALL	FBMC_SC_VONMISES
FBMC_SC_MAXSHEAR	FBMC_SC_MAXPRIN
FBMC_SC_MINPRIN	FBMC_SC_AXIAL
FBMC_SC_YSHEAR	FBMC_SC_ZSHEAR
FBMC_SC_COMBSHEAR	

Set Object - AddConstrained Method Match Options

The following table lists constants used for to specify `enMatch` for the `AddConstrained` method of the Set Object.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FMTCH_ANY	FMTCH_ALL_TRUE
FMTCH_ALL	FMTCH_ANY_CONSTRAINED
FMTCH_ANY_TRUE	FMTCH_ALL_CONSTRAINED

Set Object - AddLoaded Method Range Options

The following table lists constants used for to specify `enRangeOpt` for the `AddLoaded` method of the Set Object.

Constant Name	Constant Name
FRNG_ANY	FRNG_BETWEEN
FRNG_ABOVE	FRNG_OUTSIDE
FRNG_BELOW	

Results Ranking to Data Table Options

The following table lists constants to be used to specify the Ranking Method when using the `feResultsRankingToDataTable` function.

Constant Name	Constant Name
FRKM_SETS	FRKM_ENTITY

The following table lists constants to be used to specify the Ranking Approach when using the `feResultsRankingToDataTable` function.

Constant Name	Constant Name
FRKA_ALL	FRKA_EACH

The following table lists constants to be used to specify the Ranking Type when using the `feResultsRankingToDataTable` function.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FRKT_MAX	FRKT_MAXABS
FRKT_MIN	

Model Data Contour and Discrete Value Plots

The constants used for plotting displays of model data and discrete values are described in **Model Contour Data Types**.

Nastran License Types

The following table lists constants used to specify license types for NX Nastran.

Constant Name	Constant Name
FSNAS_LIC_DESK	FSNAS_LIC_ENT

Orient Plate Output

The following table lists constants used to specify methods for orientation of plate element output.

Constant Name	Constant Name
FOOP_FIRSTEDGE	FOOP_MATL
FOOP_MIDSIDE	FOOP_BODY
FOOP_BISECT	

Orient Solid Output

The following table lists constants used to specify methods for orientation of solid element output.

Constant Name	Constant Name
FOOS_MATL	FOOS_ELEM
FOOS_GLOBALRECT	FOOS_BODY

Unpublished work. © 2025 Siemens

Rigid Method

The following table lists constants used to specify methods for Rigid Elements.

Constant Name	Constant Name
FRIG_METHOD_AUTO	FRIG_METHOD_LAGRAN
FRIG_METHOD_STIFF	FRIG_METHOD_LGELIM
FRIG_METHOD_LINEAR	

Analytic Surface Types

The following table lists types available for analytic surfaces.

Constant Name	Constant Name
FARS_CYLINDER	FARS_REVOLUTION
FARS_SEGMENTS	

Update Line Directions

The following table lists constants used to update line directions.

Constant Name	Constant Name
FULD_REVERSE	FULD_VECTOR
FULD_ELEM	

Copy Renumber Methods

The following table lists the renumbering capability available when copying geometry and/or mesh in the `CopyTool`.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FCPRNM_DEFAULT	FCPRNM_OFFSET
FCPRNM_BLOCK	

Copy Property Approach

The following table lists the property options available when copying geometry and/or mesh in the CopyTool.

Constant Name	Constant Name
FCPPR_KEEP	FCPPR_DUP_MATL
FCPPR_DUP_PROP	FCPPR_OVERRIDE

Optimization Goal

The following table lists constants to be used for Optimization Goal Method.

Constant Name	Constant Name
FOG_NONE (obsolete, use FOG_DRESP)	FOG_MINWEIGHT (obsolete, use FOG_WEIGHT)
FOG_DRESP	FOG_VOLUME
FOG_WEIGHT	FOG_COMPLIANCE

Optimization Variable / Topology Region Type

The following table lists types available for optimization variables or topology regions.

Constant Name	Constant Name
FOPRT_PROPERTY	FOPRT_TOPOLOGY
FOPRT_MATERIAL	FOPRT_ELEM

Unpublished work. © 2025 Siemens

Optimization Response Category

The following table lists constants used to specify type of optimization limits.

Constant Name	Constant Name
FOPRSP_CUSTOM	FOPRSP_EIGEN
FOPRSP_NDISP	FOPRSP_BUCKLE
FOPRSP_SPCFORCE	FOPRSP_COMPL
FOPRSP_ELEM	FOPRSP_VOL
FOPRSP_ESE	FOPRSP_WEIGHT
FOPRSP_FREQ	

Optimization Response Attribute Type

The following table lists attributes for optimization response.

Constant Name	Constant Name
FOPATT_BLANK	FOPATTB_INT
FOPATTB_CHAR	FOPATTB_REAL

Optimization Manufacturing Constraint Types

The following table lists types available when specifying manufacturing constraints.

Constant Name	Constant Name
FOPMC_ADDM	FOPMC_EXTR
FOPMC_CAST	FOPMC_MINS

Unpublished work. © 2025 Siemens

FOPMC_CHECK	FOPMC_MAXS
FOPMC_CSYM	FOPMC_PSYM

Optimization Variable Bounds Options

The following table lists the types available for optimization variable bounds.

Constant Name	Constant Name
FOPBT_VALUES	FOPBT_PCT
FOPBT_RELATIVE	

Optimization Constants

The following table lists constants to be used for Optimization Methods.

Constant Name	Constant Name
FOP_NONE	FOP_VARY
FOP_GOAL	FOP_LIMIT

The following table lists constants to be used for Optimization Goal Method.

Constant Name	Constant Name
FOG_NONE (obsolete, use FOG_DRESP)	FOG_MINWEIGHT (obsolete, use FOG_WEIGHT)
FOG_DRESP	FOG_VOLUME
FOG_WEIGHT	FOG_COMPLIANCE

The following table lists constants to be used for Optimization Vary Method.

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
FOV_NONE	FOV_BARI1
FOV_RODAREA	FOV_BARI2
FOV_RODTORSION	FOV_BARTORSION
FOV_BARAREA	FOV_PLATETHICKNESS

The following table lists constants to be used for Optimization Limit Method.

Constant Name	Constant Name
FOL_NONE	FOL_BARMINSTRAIN
FOL_NODXDISP	FOL_PLTXNORMALSTRESS
FOL_NODYDISP	FOL_PLTYNORMALSTRESS
FOL_NODZDISP	FOL_PLTXYSHEARSTRESS
FOL_NODXRDISP	FOL_PLTMAXPRINSTRESS
FOL_NODYRDISP	FOL_PLTMINPRINSTRESS
FOL_NODZRDISP	FOL_PLTVONMISESSTRESS
FOL_RODAXIALSTRESS	FOL_PLTXNORMALSTRAIN
FOL_RODTORSIONSTRESS	FOL_PLTYNORMALSTRAIN
FOL_RODAXIALSTRAIN	FOL_PLTXYSHEARSTRAIN
FOL_RODTORSIONSTRAIN	FOL_PLTMAXPRINSTRAIN
FOL_BARAXIALSTRESS	FOL_PLTMINPRINSTRAIN
FOL_BARMAXSTRESS	FOL_PLTVONMISESSTRAIN

Unpublished work. © 2025 Siemens

FOL_BARMINSTRESS	FOL_FREQUENCY
FOL_BARAXIALSTRAIN	FOL_EIGENVALUE
FOL_BARMAXSTRAIN	

Monitor Point Constants

The following table lists constants to be used for Monitor Point Type.

Constant Name	Constant Name
MPT_1_AEROPANEL	MPT_1_STRUCT
MPT_1_AEROMESH	MPT_3

The following table lists constants to be used for Monitor Point Components.

Constant Name	Constant Name
MPT_FX	MPT_MX
MPT_FY	MPT_MY
MPT_FZ	MPT_MZ

The following table lists constants to be used for Monitor Point Contributions.

Constant Name	Constant Name
MPT_SPC	MPT_APPLIED
MPT_MPC	MPT_DMIG

Direct Matrix Input Simulation Entity Constants

The following table lists constants to be used for Direct Matrix Input - Data Source type

Unpublished work. © 2025 Siemens

Constant Name	Constant Name
MAT_INT	MAT_COMB
MAT_EXT	

The following table lists constants to be used for Direct Matrix Input - File Format.

Constant Name	Constant Name
MAT_PCH	MAT_OP2

The following table lists constants to be used for Direct Matrix Input - Input Format.

Constant Name	Constant Name
MAT_SQUARE	MAT_RECT2
MAT_SYMMETRIC	MAT_RECT9

The following table lists constants to be used for Direct Matrix Input - Input/Output Type.

Constant Name	Constant Name
MAT_MACHPRES	MAT_CPLX_SINGLE
MAT_REAL_SINGLE	MAT_CPLX_DOUBLE
MAT_REAL_DOUBLE	

The following table lists constants to be used for Direct Matrix Input - Complex Format.

Constant Name	Constant Name
MAT_REAL_IMAG	MAT_AMP_PHASE

File Methods

The File Methods include the following sections:

Unpublished work. © 2025 Siemens

File Open/Exit

Unpublished work. © 2025 Siemens

feFileNew

feFileNew **(void)**	
Description:	
This function starts a new model. It works like the **File** → **New** command.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create a new file.
Remarks/Usage:	
In releases prior to v9, this API function required 2 arguments because multiple models were not supported and the previous model needed to be closed prior to opening the new model.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileOpen

feFileOpen **(useDlg, f\Name)**	
Description:	
This function opens an existing model, or starts a new model and loads the contents of a file in one of the supported analysis formats. It works like the **File → Open** command.	
Input:	
`BOOL useDlg`	If True, `fName` is ignored and the standard dialog boxes are displayed allowing the user to choose a model to open. If False, the model specified in `fName` is opened.
`STRING fName`	The filename of the file to be opened. In general, this name should be specified with a complete path. If no path is supplied, the file will be opened from the current Femap directory.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to open file.
`FE_NOT_EXIST`	Specified file does not exist.
Remarks/Usage:	
`feFileOpen( True, "wing.mod" )` - interactive use, even though you specified a filename, it will not be opened - the user will be asked to choose a model. `feFileNew( False, "C:\Models\Bracket.mod" )` - Opens the model *C:\Models\Bracket.mod* without asking for user input.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileClose

feFileClose **(saveChanges)**	
Description:	
This function closes the active model	
Input:	
`BOOL saveChanges`	If True, any changes in the model will be saved. If False, the model will simply close, discarding any changes.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This function will always display a dialog box asking for a filename if you are saving changes to a model that has never been saved previously. To prevent this, you must either discard the changes, or save the model prior to calling this method.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileExit

feFileExit **(void)**	
Description:	
This function causes Femap to exit.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	The exit operation succeeded.
`FE_FAIL`	The file exit operation failed. The lock_count has not been decremented to zero preventing Femap from exiting.
Remarks/Usage:	
This function will always display a dialog box asking for confirmation to save the current model unless it has not been modified since the last save. Therefore, to exit with no user interaction, you must either use one of the FileSave functions or `feFileNew` immediately prior to calling this function.	
Example:	
None	

File Save

Unpublished work. © 2025 Siemens

feFileSave

feFileSave **(useDlg)**	
Description:	
This function saves the current model to the current model name, like the **File** → **Save** command.	
Input:	
`BOOL useDlg`	If True, the function runs interactively. In this mode, if the model name has not been specified, the Save As dialog box will be displayed to allow you to choose the name. If a model filename has been specified, the model will simply be saved. If False, the model will be saved if the filename has been selected, or the function will fail if the model has never been saved before.
Output:	
None	
Return Code:	
`FE_FAIL`	The save operation failed. Either the filename specified cannot be written, or some other error occurred during the save operation.
`FE_NO_FILENAME`	`useDlg=False` and the model name has never been specified. Use `feFileSaveAs` instead to specify a name to be used to save the model.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileSaveAs

feFileSaveAs **(useDlg, fName)**	
Description:	
This function saves the current model, like the **File** → **Save As** command. The filename can either be specified or chosen interactively.	
Input:	
`BOOL useDlg`	If True, the filename to be used to save the current model will be chosen interactively by display the Save As dialog box. If False, operation is automatic and the fName parameter must be specified.
`STRING fName`	The filename to use to save the current model. This is only used if `useDlg=False`.
Output:	
None	
Return Code:	
`FE_FAIL`	The save operation failed. Either the filename specified cannot be written, or some other error occurred during the save operation.
`FE_CANCEL`	`useDlg=True` and the user cancelled the save operation.
`FE_INVALID`	The filename is not valid to save the model.
Remarks/Usage:	
`feFileSaveAs( True, " " )` - ask the user for a filename and save the current model to that file. `feFileSaveAs( False, "c:\models\car.modfem" )` - automatically save the current model to the specified file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileTimedSave

feFileTimedSave

(useDlg, saving, notify, interval, commands)

Description:

This function controls whether or not timed save is turned on, and sets the available parameters for timed save operation. It works like the **File → Timed Save** command.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays the dialog box displayed by the **File → Timed Save** command. In this case, all other parameters are ignored. If False, the other parameters automatically define the timed save options.
`BOOL saving`	Turns on/off timed save (True = on, False = off).
`BOOL notify`	If this is on, then the user will be notified prior to a timed save taking place.
`INT4 interval`	Time in minutes between timed saves (1..999).
`INT4 commands`	Number of commands between timed saves (0, 1..999). If you set this to 0, then timed saves will be done based on time only.

Output:

None	

Return Code:

`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	At least one of the arguments contains data that is invalid or out of the allowable range.

Remarks/Usage:

Calling this function automatically stops any previous timed save operations. When the function is complete, if timed save is turned on, it is reinitialized.

Unpublished work. © 2025 Siemens

Example:
None

Import Geometry

Unpublished work. © 2025 Siemens

feFileReadAcisOpt

feFileNewfeFileReadAcisOpt

```
( useDlg, fName, title, layer, color, increment_layer, increment_color,
assign_color, convert_to_splines,
convert_format, clean, heal, check, stringent, delete_invalid, faces,
scale )
```

Description:

This function loads geometry from an ACIS .SAT file. It works like **File** → **Import** → **Geometry**, when you pick an ACIS file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the ACIS read options.
`STRING fName`	The filename/path of the ACIS file to be read.
`STRING title`	The title that is assigned to created layers and the solid.
`INT4 layer`	The ID of the layer for new geometry. Set to -1 to use the active layer.
`INT4 color`	The color used for the geometry that is created. Set to -1 to use the active solid color. Only used if `assign_color=True`
`BOOL increment_layer`	If True and you read an assembly, geometry for each solid is placed on a different layer; otherwise all geometry is on the specified layer.
`BOOL increment_color`	If True and you read an assembly, geometry for each solid is colored differently.
`BOOL assign_color`	If True, the specified color value is used for geometry; otherwise the default colors are used.
`BOOL convert_to_splines`	If True, surfaces are all converted to B-Spline surfaces. If False, surfaces retain their original definition.
`BOOL convert_format`	Should always be set to True. ACIS geometry can no longer be read in native format - it is always converted to Parasolid.

Unpublished work. © 2025 Siemens

`BOOL clean`	Only used if `convert_format=True`. If True, this attempts to detect and remove sliver faces or other geometric problems.
`BOOL heal`	Only used if `convert_format=True`. If True, attempts to close gaps and correct geometric inaccuracies.
`BOOL check`	Only used if `convert_format=True`. If True, checks converted Parasolid geometry to find and report any problems.
`BOOL stringent`	Only used if `convert_format=True`. If True, checks for surface discontinuities or irregularities in the converted Parasolid geometry.
`BOOL delete_invalid`	Only used if `convert_format=True`. If True and geometry was checked, this deletes any geometry that failed the checking.
`BOOL faces`	Only used if `convert_format=True`. If True and it was not possible to convert a complete solid, this will allow unstitched, free surfaces to be retained.
`REAL8 scale`	Only used if `convert_format=True`. This is a scale factor that is applied to the geometry during conversion. It is used to ensure that the geometry lies inside the Parasolid modeling box (+/- 500 units).
None	
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage	
In general, you will not want to convert geometry to Parasolid format. In that case, all of the parameters after `convert_format` are ignored.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadAcis

feFileReadAcis

(fName)

Description:

Short form of `feFileReadAcisOpt( )`.

Input:

`STRING fName`	The filename/path of the ACIS file to be read.

Return Code:

`FE_FAIL`	No Femap model is open.

Remarks/Usage:

Using this function is equivalent to calling:

```
feFileReadAcisOpt( FALSE, fName, "", -1, -1, FALSE, TRUE, FALSE, FALSE,
FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, 1.0 )
```

For details, see **feFileReadAcisOpt**.

Example:

None

Unpublished work. © 2025 Siemens

feFileReadCatia

feFileReadCatia **(useDlg, fName, rdAnalytical, rdTopology, rdCurves, stitch, heal)**	
Description:	
This function loads geometry from a CATIA (.model, mdl, .exp) file. It works like **File → Import → Geometry**, when you pick a CATIA file. The file is loaded into the current model. Additional options are available by using the `feFileReadCatia2( )` method.	
Input:	
`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the CATIA read options.
`STRING fName`	The filename/path of the CATIA file to be read.
`BOOL rdAnalytical`	Set to True to read the analytical definition for surfaces defined as splines.
`BOOL rdTopology`	Set to True to read the CATIA topology to create a solid. Generally, it is better to set to False to read the surfaces as individual surfaces.
`BOOL rdCurves`	Set to True to read 3D trimming curves, or False to read 2D trimming curves.
`BOOL stitch`	Set to True to automatically stitch surfaces into a solid. If False, individual surfaces will be read.
`BOOL heal`	Set to True to attempt to repair tolerance and gap problems.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.

Unpublished work. © 2025 Siemens

`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadCatia2

feFileReadCatia2

(useDlg, fName, rdAnalytical, rdTopology, rdCurves, rdNoShow, rdNoPick, rdWorkplanes, stitch, heal, check)

Description:

This function loads geometry from a CATIA (.model, mdl, .exp) file. It works like **File → Import → Geometry**, when you pick a CATIA file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the CATIA read options.
`STRING fName`	The filename/path of the CATIA file to be read.
`BOOL rdAnalytical`	Set to True to read the analytical definition for surfaces defined as splines.
`BOOL rdTopology`	Set to True to read the CATIA topology to create a solid. Generally, it is better to set to False to read the surfaces as individual surfaces.
`BOOL rdCurves`	Set to True to read 3D trimming curves, or False to read 2D trimming curves.
`BOOL rdNoShow`	If True, entities that are marked as *No Show* in the Catia model will be read.
`BOOL rdNoPick`	If True, entities on the *No Pick* layer will be read.
`BOOL rdWorkplanes`	If True, workplane geometry will be read.
`BOOL stitch`	Set to True to automatically stitch surfaces into a solid. If False, individual surfaces will be read.
`BOOL heal`	Set to True to attempt to repair tolerance and gap problems.

Unpublished work. © 2025 Siemens

`BOOL check`	If True, the geometry will be checked for validity.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadCatia3

feFileReadCatia3

(useDlg, fName, rdAnalytical, rdTopology, rdCurves, rdNoShow, rdNoPick, rdWorkplanes, stitch, heal, check)

Description:

This function loads geometry from a CATIA (.model, mdl, .exp) file. It works like **File → Import → Geometry**, when you pick a CATIA file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the CATIA read options.
`STRING fName`	The filename/path of the CATIA file to be read.
`BOOL rdAnalytical`	Set to True to read the analytical definition for surfaces defined as splines.
`BOOL rdTopology`	Set to True to read the CATIA topology to create a solid. Generally, it is better to set to False to read the surfaces as individual surfaces.
`BOOL rdCurves`	Set to True to read 3D trimming curves, or False to read 2D trimming curves.
`BOOL rdNoShow`	If True, entities that are marked as *No Show* in the Catia model will be read.
`BOOL rdNoPick`	If True, entities on the *No Pick* layer will be read.
`BOOL rdWorkplanes`	If True, workplane geometry will be read.
`BOOL stitch`	Set to True to automatically stitch surfaces into a solid. If False, individual surfaces will be read.
`BOOL heal`	Set to True to attempt to repair tolerance and gap problems.

Unpublished work. © 2025 Siemens

`BOOL check`	If True, the geometry will be checked for validity.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadCatiaV5

feFileReadCatiaV5

(useDlg, fName, fPath, rdNoShow, rdInactiveLayers, rdPoints, rdCurves, rdSurfaces, rdBodies, simplify)

Description:

This function loads geometry from a CATIA V5 (.CATpart or .CATproduct) file. It works like **File** → **Import** → **Geometry**, when you pick a CATIA V5 file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. The fName and fPath options can not be set using this method, other options set the defaults for the dialog box. If False, the other parameters automatically define the CATIA V5 read options.
`STRING fName`	The filename/path of the CATIA V5 file to be read.
`STRING fPath`	The search path for other assembly parts. If all parts are in the same directory as the file you specify, you can pass this as a blank string. If you need to search multiple directories, you must separate the paths with semicolons in this string.
`BOOL rdNoShow`	If True, reads *No Show* geometry.
`BOOL rdInactiveLayers`	If True, reads geometry from layers marked as Inactive.
`BOOL rdPoints`	Set to True to read points.
`BOOL rdCurves`	Set to True to read curves.
`BOOL rdSurfaces`	Set to True to read surfaces.
`BOOL rdBodies`	Set to True to read bodies.
`BOOL simplify`	If True, simplifies the underlying geometry for curves and surfaces

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadDxf

feFileReadDxf

(useDlg, fName, rdText, rd3DFace, rdBlocks, rdPolyelem, rdTraceBoundary, rdTraceCenterline, rdLayer, firstLayer, mergePoints, mergeDist)

Description:

This function loads geometry (and other entities) from a DXF formatted file. It works like **File** → **Import** → **Geometry**, when you pick a DXF file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the DXF read options.
`STRING fName`	The filename/path of the DXF file to be read.
`BOOL rdText`	If True, reads text or notes. Skipped if False.
`BOOL rd3DFace`	If True, reads 3DFaces as elements. If False, lines are created to form the outline of the face.
`BOOL rdBlocks`	If True, entities inside block references will be read.
`BOOL rdPolyelem`	If True, reads polygon meshes as elements.
`BOOL rdTraceBoundary`	If True, creates lines to represent the edges of a traceline.
`BOOL rdTraceCenterline`	If True, creates lines to represent the center of a traceline.
`BOOL rdLayer`	If True, reads color and layer information and assigns it to the geometry.
`INT4 firstLayer`	If `rdLayer=True`, this is the ID of the first layer that will be created. A unique layer is created for each unique layer name in the DXF file.

Unpublished work. © 2025 Siemens

`BOOL mergePoints`	If True, coincident points will be merged after geometry is read.
`REAL8 mergeDist`	If `mergePoints=True`, this is the distance used to determine coincident points. Points that are closer than this distance from each other will be merged.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified an layer that was not in the supported range.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadIgesAdv

feFileReadIgesAdv

(useDlg, fName, read_curves, read_untrimmed_surfaces, read_trimmed_surfaces, read_solids, flavor, trim_curve_option)

Description:

This function loads geometry from a IGES (.IGS) file. It works like **File → Import → Geometry**, when you pick an IGES file, and choose the Alternate/Advanced interface. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the IGES read options.
`STRING fName`	The filename/path of the IGES file to be read.
`BOOL read_curves`	Set to True to read free curves (curves that are not connected to surfaces or solids).
`BOOL read_untrimmed_surfaces`	Set to True to read free untrimmed surfaces.
`BOOL read_trimmed_surfaces`	Set to True to read free trimmed surfaces.
`BOOL read_solids`	Set to True to read solid data.
`INT4 flavor`	=0 for No flavoring, =1 for AutoCAD customization (old versions of AutoCAD only).
`INT4 trim_curve_option`	=0 for default trim curves, =1 for 2D trim curves, =2 for 3D trim curves.

Output:

None	

Return Code:

`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.

Unpublished work. © 2025 Siemens

`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
If you are having trouble reading solid data, try turning off `read_solids`, and just read the trimmed surfaces. These surfaces can then be stitched into a solid.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadIges

feFileReadIges

(useDlg, fName, merge_points, read_layer, read_geom_only, read_surface_data, trim_preference, min_resolution)

Description:

This function loads geometry from a IGES (.IGS) file. It works like **File → Import → Geometry**, when you pick an IGES file, and you do not choose the Alternate/Advanced interface. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the IGES read options.
`STRING fName`	The filename/path of the IGES file to be read.
`BOOL merge_points`	Set to True to merge all coincident points that are created.
`BOOL read_layer`	Set to True to read layer information and assign geometry to the appropriate layers.
`BOOL read_geom_only`	Set to True to only read data marked as geometry.
`BOOL read_surface_data`	Set to True to read trimmed surface data. If you set this to True, you must have the appropriate license.
`INT4 trim_preference`	=0 for Default Trim curves per the IGES file specification. =1 for 2D trim curves, =2 for 3D trim curves.
`REAL8 min_resolution`	Set to the minimum resolution for coincident points in the file. Used for tolerances of surface data, and for merging coincident points.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadParasolidOpt

feFileReadParasolidOpt

(useDlg, fName, title, layer, color, increment_layer, increment_color, assign_color, convert_format, clean, heal, check, stringent, delete_invalid, faces, scale)

Description:

This function loads geometry from a Parasolid .X_T file. It works like **File** → **Import** → **Geometry**, when you pick a Parasolid file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the Parasolid read options.
`STRING fName`	The filename/path of the Parasolid file to be read.
`STRING title`	The title that is assigned to created layers and the solid.
`INT4 layer`	The ID of the layer for new geometry. Set to -1 to use the active layer.
`INT4 color`	The color used for the geometry that is created. Set to -1 to use the active solid color. Only used if `assign_color=True`.
`BOOL increment_layer`	If True and you read an assembly, geometry for each solid is placed on a different layer; otherwise, all geometry is on the specified layer.
`BOOL increment_color`	If True and you read an assembly, geometry for each solid is colored differently.
`BOOL assign_color`	If True, the specified color value is used for geometry; otherwise, the default colors are used.
`BOOL convert_format`	Should always be set to False. Parasolid geometry can no longer be converted to ACIS format.

Unpublished work. © 2025 Siemens

`BOOL clean`	Only used if `convert_format=True`. If True, this attempts to detect and remove sliver faces or other geometric problems.
`BOOL heal`	Only used if `convert_format=True`. If True, attempts to close gaps and correct geometric inaccuracies.
`BOOL check`	Only used if `convert_format=True`. If True, checks converted ACIS geometry to find and report any problems.
`BOOL stringent`	Only used if `convert_format=True`. If True, checks for surface discontinuities or irregularities in the converted ACIS geometry.
`BOOL delete_invalid`	Only used if `convert_format=True`. If True and geometry was checked, this deletes any geometry that failed the checking.
`BOOL faces`	Only used if `convert_format=True`. If True and it was not possible to convert a complete solid, this will allow unstitched, free surfaces to be retained.
`REAL8 scale`	Only used if `convert_format=True`. This is a scale factor that is applied to the geometry during conversion. It is used to ensure that the geometry lies inside the ACIS modeling box.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
In general, you will not want to convert geometry to ACIS format. In that case, all of the parameters after `convert_format` are ignored.	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feFileReadParasolid

feFileReadParasolid

(fName)

Description:

Short form of `feFileReadParasolidOpt( )`.

Input:

`STRING fName`	The filename/path of the Parasolid file to be read.

Return Code:

`FE_FAIL`	No Femap model is open.

Remarks/Usage:

Using this function is equivalent to calling:

```
feFileReadParasolidOpt( FALSE, fName, "", -1, -1, FALSE, TRUE, FALSE,
FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, 1.0 )
```

For more detailed documentation, see **feFileReadParasolidOpt**.

Example:

None

Unpublished work. © 2025 Siemens

feFileReadProe

feFileReadProe **(fName)**	
Description:	
This function loads geometry from a Pro/E (.PRT) file. It works like **File** → **Import** → **Geometry**, when you pick a Pro/E file. The file is loaded into the current model.	
Input:	
`STRING fName`	The filename/path of the Pro/E file to be read.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadSolidEdge

feFileReadSolidEdge **(fName)**	
Description:	
This function loads geometry from a Solid Edge (.PAR) file. It works like **File** → **Import** → **Geometry**, when you pick a Solid Edge file. The file is loaded into the current model.	
Input:	
`STRING fName`	The filename/path of the Solid Edge file to be read.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadStepOpt2

feFileReadStepOpt2

(useDlg, fName, layer, color, increment_layer, increment_color, assign_color, bodies, faceted, Surfaces, Curves, Points, optimize, simplify)

Description:

This function loads geometry from a STEP (.STP) file. It works like **File** → **Import** → **Geometry**, when you pick a STEP file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the fName option is used; all others are ignored. If False, the other parameters automatically define the STEP read options.
`STRING fName`	The filename/path of the STEP file to be read.
`INT4 layer`	The ID of the layer for new geometry. Set to -1 to use the active layer. GUI specifies the active layer
`INT4 color`	The color used for the geometry that is created. Set to -1 to use the active solid color. Only used if `assign_color=True`.
`BOOL increment_layer`	If True and you read an assembly, geometry for each solid is placed on a different layer; otherwise all geometry is on the specified layer. Default in GUI is False
`BOOL increment_color`	If True and you read an assembly, geometry for each solid is colored differently. Default in GUI is True
`BOOL assign_color`	If True, the specified color value is used for geometry; otherwise, the default colors are used. Default in GUI is True
`BOOL bodies`	Set to True if you want to read Bodies. Default in GUI is True.

Unpublished work. © 2025 Siemens

`BOOL faceted`	Set to True if you want to read Faceted Bodies. Default in GUI is True.
`BOOL Surfaces`	Set to True if you want to read Surfaces. Default in GUI is True.
`BOOL Curves`	Set to True if you want to read Curves. Default in GUI is True.
`BOOL Points`	Set to True if you want to read Points. Default in GUI is True.
`BOOL optimize`	Set to True if you want to Optimize Bodies. Default in GUI is True.
`BOOL simplify`	Set to True if you want to Simplify Geometry. Default in GUI is False.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

feFileReadStep

Unpublished work. © 2025 Siemens

feFileReadStep

(fName)

Description:

Short form of `feFileReadStepOpt2( )`.

Input:

`STRING fName`	The filename/path of the STEP file to be read.

Return Code:

`FE_FAIL`	No Femap model is open.

Remarks/Usage:

Using this function is equivalent to calling:

```
feFileReadStepOpt( FALSE, fName, "", -1, -1, FALSE, TRUE, FALSE, TRUE,
TRUE, TRUE, TRUE, TRUE, TRUE, FALSE, FALSE, TRUE, FALSE, FALSE, TRUE,
FALSE, FALSE, TRUE, FALSE )
```

For more detailed documentation, see **feFileReadStepOpt2**.

Example:

None

Unpublished work. © 2025 Siemens

feFileReadStepOpt - Obsolete

feFileReadStepOpt

**(useDlg, fName, title, layer, color, increment_layer, increment_color, assign_color, class_VI,
class_V, class_IV, class_III, class_II_wf, class_II_curv, class_II_rect, class_II_2d, smooth_g1, explode,
analytics, stitch, closed_orient, pro_e, check_input, check_output)**

Description:

This method is obsolete and should no longer be used. Use `feFileReadStepOpt2` instead. This function loads geometry from a STEP (.STP) file. It works like **File → Import → Geometry**, when you pick a STEP file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the STEP read options.
`STRING fName`	The filename/path of the STEP file to be read.
`STRING title`	The title that is assigned to created layers and the solid.
`INT4 layer`	The ID of the layer for new geometry. Set to -1 to use the active layer.
`INT4 color`	The color used for the geometry that is created. Set to -1 to use the active solid color. Only used if `assign_color=True`.
`BOOL increment_layer`	If True and you read an assembly, geometry for each solid is placed on a different layer; otherwise all geometry is on the specified layer.
`BOOL increment_color`	If True and you read an assembly, geometry for each solid is colored differently.
`BOOL assign_color`	If True, the specified color value is used for geometry; otherwise, the default colors are used.

Unpublished work. © 2025 Siemens

`BOOL class_VI`	Set to True if you want to read Class VI solids.
`BOOL class_V`	Set to True if you want to read Class V faceted solids.
`BOOL class_IV`	Set to True if you want to read Class IV surfaces.
`BOOL class_III`	Set to True if you want to read Class III wireframe - edge based and shell based wireframe geometry.
`BOOL class_II_wf`	Set to True if you want to read Class II wireframe - wireframe without topology or other underlying surfaces.
`BOOL class_II_curv`	Set to True if you want to read Class II curve bounded surfaces.
`BOOL class_II_rect`	Set to True if you want to read Class II rectangular trimmed surfaces. Normally this is False, because these surfaces are usually used for construction geometry, not part geometry.
`BOOL class_II_2d`	Set to True to read 2D trimming curves for surfaces. These are curves in the surface parametric coordinates. If you set this to False, 3D trimming curves will be read.
`BOOL smooth_g1`	If True, attempts to eliminate discontinuities in the STEP geometry which cannot be represented in Parasolid. Set this to True for the best conversion.
`BOOL explode`	If True, STEP geometry will be read as independent faces, then Parasolid will be used to restitch the solid. If False, the topology will be constructed directly from the data in the STEP file. Setting this to True will help to eliminate geometric tolerancing problems.
`BOOL analytics`	If True, geometry will be simplified to analytical surfaces whenever possible.
`BOOL stitch`	If True, surfaces will be stitched into a solid.
`BOOL closed_orient`	If True, extra checking is done to eliminate problems that can occur in closed analytical (sphere and torus) surfaces that produce complementary surfaces.
`BOOL pro_e`	If True, forces angle units to degrees to work around a problem in old Pro/E STEP files. Always

Unpublished work. © 2025 Siemens

	set to False unless you are having specific problems with Pro/E files.
`BOOL check_input`	If True, syntax checking of the STEP file is done prior to reading.
`BOOL check_output`	If True, the Parasolid geometry is checked for validity after the conversion.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadJT

feFileReadJT **(fName)**	
Description:	
This function loads geometry from a JT file. It works like **File → Import → Geometry**, when you pick a JT file. The file is loaded into the current model.	
Input:	
`STRING fName`	The filename/path of the JT file to be read.
Return Code:	
`FE_FAIL`	No Femap model is open.
`FE_NOT_AVAILABLE`	JT importer not installed/found.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadStl

feFileReadStl

(useDlg, fName, skip_short_edges, merge_nodes, close_gaps, PATRAN_file, min_distance)

Description:

This function loads geometry from a Stereolithography (.STL) file. It works like **File** → **Import** → **Geometry**, when you pick an STL file. The file is loaded into the current model.

Input:

`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the fName option is used; all others are ignored. If False, the other parameters automatically define the STL read options.
`STRING fName`	The filename/path of the STL file to be read.
`BOOL skip_short_edges`	Set to True to skip edges that are shorter than the min_distance setting.
`BOOL merge_nodes`	Set to True to merge all coincident nodes after the file is loaded.
`BOOL close_gaps`	Set to True to attempt to close gaps in the STL mesh.
`BOOL PATRAN_file`	Set to True to read a PATRAN generated STL file where negative coordinates can run together with no spaces between them.
`REAL8 min_distance`	The minimum distance of short edges to be skipped.

Output:

None	

Return Code:

`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.

Unpublished work. © 2025 Siemens

`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	No Femap model is open.
`FE_FAIL`	An error occurred while reading the file.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadUg

feFileReadUg **(useDlg, fName, rdBlanked, rdLayers)**	
Description:	
This function loads geometry from a Unigraphics (.PRT) file. It works like **File → Import → Geometry**, when you pick a Unigraphics file. The file is loaded into the current model.	
Input:	
`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. Only the `fName` option is used; all others are ignored. If False, the other parameters automatically define the Unigraphics read options.
`STRING fName`	The filename/path of the Unigraphics file to be read.
`BOOL rdBlanked`	Set to True to read parts that are blanked. If False, only unblanked parts will be read.
`BOOL rdLayers`	Set to True to read parts that are on invisible layers. If False, only parts on visible layers will be read.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

feFileReadSolidWorks

feFileReadSolidWorks **(fName)**	
Description:	
This function loads geometry from a SolidWorks part (.sldprt) or assembly (.sldasm) file. It works like **File → Import → Geometry**, when you pick a SolidWorks file. The file is loaded into the current model	
Input:	
`STRING fName`	The filename/path of the SolidWorks part or assembly file to be read.
Return Code:	
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Import Analysis Model

Unpublished work. © 2025 Siemens

feFileReadAbaqus

feFileReadAbaqus **(brand, fName)**	
Description:	
This function loads finite element model data from an existing ABAQUS file.	
Input:	
`INT4 brand`	Used to select the format of the ABAQUS file. 0=Abaqus Input File (.INP), 1=Abaqus Results File (.FIL)
`STRING fName`	The filename of the ABAQUS input file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadAnsys

feFileReadAnsys **(brand, fName)**	
Description:	
This function loads finite element model data from an existing ANSYS file.	
Input:	
`INT4 brand`	Used to select the format of the ANSYS input file. 4 = Version 4, 5 = Version 5+.
`STRING fName`	The filename of the ANSYS input file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadDyna

feFileReadDyna **(brand, fName)**	
Description:	
This function loads finite element model data from an existing LS-DYNA file.	
Input:	
`INT4 brand`	Used to select the format of the DYNA file. Currently this MUST be specified as 1=DYNA Results file.
`STRING fName`	The filename of the DYNA file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadMarc

feFileReadMarc **(brand, fName)**	
Description:	
This function loads finite element model data from an existing Marc file.	
Input:	
`INT4 brand`	Used to select the format of the Marc file. Currently this MUST be specified as 1=Marc Results file.
`STRING fName`	The filename of the Marc file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadNastran

feFileReadNastran **(brand, fName)**	
Description:	
This function loads finite element model data from an existing NASTRAN file.	
Input:	
`INT4 brand`	Used to select the format of the NASTRAN input file: 0=MSC, 1=NE, 2=UAI, 3=CSA, 4=SSS, 5=Cosmic, 6=ME, 7=VR, 8=NX.
`STRING fName`	The filename of the NASTRAN input file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadPatran

feFileReadPatran **(fName, bSaveZeroDispAsBC)**	
Description:	
This function loads finite element model data from an existing Patran file.	
Input:	
`STRING fName`	The filename of the Patran input file to read.
`BOOL bSaveZeroDispAsBC`	If True, Zero Nodal Displacements are stored as Constraints. If False, they are stored as loads.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	The file you specified does not exist.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadIdeas

feFileReadIdeas **(fName)**	
Description:	
This function loads finite element data from an existing I-DEAS Universal (.UNV) file. It works like **File** → **Import** → **Analysis Model**, when you pick I-DEAS file. The file is loaded into the current model.	
Input:	
`STRING fName`	The filename/path of the I-DEAS Universal file to be read.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
This call replaces an older version of `feFileReadIdeas`, which was used to import I-DEAS Geometry (.IDI) files. It is no longer possible to import *IDI files into Femap and another geometry format, such as STEP or IGES, should be used to transfer geometry from I-DEAS to Femap.	
Example:	
None	

Import Analysis Results

Unpublished work. © 2025 Siemens

feFileReadAbaqusResults

feFileReadAbaqusResults **(brand, fName)**	
Description:	
This function loads finite element results data from an existing ABAQUS file.	
Input:	
`INT4 brand`	Used to select the format of the ABAQUS file. Currently this MUST be specified as 1=Abaqus Results file.
`STRING fName`	The filename of the ABAQUS results file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadAnsysResults

feFileReadAnsysResults **(brand, fName)**	
Description:	
This function loads finite element results data from an existing ANSYS file.	
Input:	
`INT4 brand`	Used to select the format of the ANSYS results file. 4 = Version 4, 5 = Version 5..9, 10 = 10 and up.
`STRING fName`	The filename of the ANSYS results file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
This function can read results files from structural (.RST) and thermal analyses (.RTH).	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadDynaResults

feFileReadDynaResults **(brand, fName)**	
Description:	
This function loads finite element results data from an existing LS-DYNA file.	
Input:	
`INT4 brand`	Used to select the format of the DYNA file. Currently this MUST be specified as 1=DYNA Results file.
`STRING fName`	The filename of the DYNA results file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadMarcResults

feFileReadMarcResults **(brand, fName)**	
Description:	
This function loads finite element results data from an existing Marc file.	
Input:	
`INT4 brand`	Used to select the format of the Marc file. Currently this MUST be specified as 1=Marc Results file.
`STRING fName`	The filename of the Marc file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadNastranResults

feFileReadNastranResults **(brand, fName)**	
Description:	
This function loads finite element results data from an existing NASTRAN file.	
Input:	
`INT4 brand`	Used to select the format of the NASTRAN results file: 0=MSC, 1=NE, 2=UAI, 3=CSA, 4=SSS, 5=Cosmic, 6=ME, 7=VR, 8=NX.
`STRING fName`	The filename of the NASTRAN results file to read.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
This function can read both NASTRAN printed results file (.F06) or OUTPUT2 files (.OP2).	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadCSVResults

feFileReadCSVResults **(fName)**	
Description:	
This function loads finite element results data from an existing CSV (comma-separated values) file formatted for use with Femap.	
Input:	
`STRING fName`	The filename of the CSV results file to read.
Output:	
None	
Return Code:	
`FE_FAIL`	No Femap model is open.
Remarks/Usage:	
This call automatically determines if output is stored using the *Comma-Separated Table Format* or *Extended Comma-Separated Table Format*, then reads the results. See sections *8.I1.3 "The Comma-Separated Table Format"* or *8.11.4 "The Extended Comma-Separated Table Format"* in the Femap User Guide for more information.	
Example:	
None	

Attach To Results File

Unpublished work. © 2025 Siemens

feFileAttachResults

feFileAttachResults **(brand, fName, memory_mapped_file)**	
Description:	
This method is used to Attach a Results File to Femap for postprocessing.	
Input:	
`INT4 brand`	Analysis Program of Results File to Attach. 4 = MSC Nastran (FAP_MSC_NASTAN), 31 = NEi Nastran (FAP_NE_NASTRAN), 36 = NX Nastran (FAP_NX_NASTRAN), 16 = ABAQUS (FAP_ABAQUS), 12 = CSV File (FAP_TABLE), 1 = FNO File (FAP_FEMAP_GEN).
`STRING fName`	The filename and path of the Results File to Attach.
`BOOL memory_mapped_file`	If True Operating System attempts to load file into System RAM
Output:	
None	
Return Code:	
`FE_FAIL`	Error Attaching to Results File.
`FE_INVALID`	Solver is not supported.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileAttachManage

feFileAttachManage **(attSetID, opt, fName)**	
Description:	
This method is used to Manage Attached Results.	
Input:	
`INT4 attSetID`	ID if Femap Set that contains the Attached IDs. Alternatively, if you specify a negative value, this is simply the ID of a single Attached Results Set.

`INT4 opt`	Manage Options:
	
	`FMRA_LOAD`
	`FMRA_UNLOAD`
	`FMRA_DETACH`
	`FMRA_UPDATE_PATH`

`STRING fName`	The filename and path of the Results file.
Output:	
None	
Return Code:	
`FE_FAIL`	Failed to perform operation on Results.
Remarks/Usage:	
None	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim atts As femap.Set
```


Unpublished work. © 2025 Siemens

```
   Set atts =  App.feSet
   Dim fName As String

   rc = atts.Select(FT_RES_ATTACH , True, "Select Attach" )
   App.feFileAttachManage( atts.ID , FMRA_UNLOAD , fName )
End Sub
```


Unpublished work. © 2025 Siemens

feFileAttachInfo

feFileAttachInfo **(ID, status, ModelName, pOutSets)**	
Description:	
This method retrieves information for the selected Attached Results.	
Input:	
`INT4 ID`	ID of Attached Results to retrieve info.
Output:	

`INT4 status`	Status of the Results:
	
	`FRA_UNLOADED`
	`FRA_OPEN`
	`FRA_OPEN_CHECK`
	`FRA_ERROR`
	`FRA_UNLOAD_CHECK`

`STRING ModelName`	The filename of the selected Attached Results.
`OBJECT pOutputSets`	A Set object that contains the Femap Output Sets that are associated with the selected Results.
Return Code:	
`FE_FAIL`	Error retrieving info from Attached Results.
Remarks/Usage:	
For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is FE_OK.	
Example:	
`Sub Main`	

Unpublished work. © 2025 Siemens

```
Dim App As femap.model

Set App = feFemap()

Dim atts As femap.Set

Set atts = App.feSet

Dim fName As String

Dim ID As Long

Dim cset As femap.Set

Set cset = App.feSet

Dim status As Long


atts.Select(FT_RES_ATTACH, True, "Select Attached Results")


ID = atts.First()

While ID <> FE_FAIL

    rc = App.feFileAttachInfo( ID, status, fName, cset )

    App.feAppMessage (FCM_NORMAL, "Status " + Str( status ) )

    App.feAppMessage (FCM_NORMAL, fName )

    App.feAppMessage (FCM_NORMAL, Str( cset.Count() ) + "OutputSets from
this Attach")

    ID = atts.Next()

Wend

End Sub
```


Unpublished work. © 2025 Siemens

feFileAttachSaveV2

`feFileAttachSaveV2`

`(ID, outSetID, vecSetID, ndSetID, elSetID )`

Description:

This method will store Results from an Attached File to the Femap Database. The method replaces `feFileAttachSave`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 ID`	ID of Femap Set that contains the IDs of the Attached Results to Save to the Femap Database. Alternatively, if you specify a negative value, this is simply the ID of a single Attached Results Set.
`INT4 outSetID`	ID of Femap Set that contains Output Sets to Save. Alternatively, if you specify a zero all Output Sets in the Result File will be saved.
`INT4 outVecID`	ID of Femap Set that contains Output Vectors to Save. Alternatively, if you specify a zero all Output Vectors in the Result File will be saved.
`INT4 ndSetID`	ID of Femap Set that contains the nodes where output will be saved. Alternatively, if you specify a zero, results for all nodes in the Result File will be saved.
`INT4 elSetID`	ID of Femap Set that contains elements where output will be saved. Alternatively, if you specify a zero, results for all elements in the Result File will be saved.

Output:

None	

Return Code:

`FE_FAIL`	Error saving results from Attached File to Femap Database.

Remarks/Usage:

When saving results Femap will preserve the Femap Output Set and automatically detach the Results File when finished.

Unpublished work. © 2025 Siemens

Example:

```
Sub Main
Dim App As femap.model
Set App = feFemap()
Dim selID As Long
Dim outvec As femap.Set
Set outvec = App.feSet
Dim ndset As femap.Set
Set ndset = App.feSet
Dim atts As femap.Set
Set atts = App.feSet

atts.SelectID (FT_RES_ATTACH, "Select Attached File To Save", selID)
ndset.Select(FT_NODE, True, "Select Nodes")
outvec.Add(1)

App.feFileAttachSaveV2(-selID, 0, outvec.ID, ndset.ID, 0)
End Sub
```


Unpublished work. © 2025 Siemens

feFileAttachSave2V2

feFileAttachSave2V2

(ID, outSetID, vecSetID, ndSetID, elSetID, bDetach)

Description:

This method will store Results from an Attached File to the Femap Database. This method replace `feFileAttachSave2`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 ID`	ID of Femap Set that contains the IDs of the Attached Results to Save to the Femap Database. Alternatively, if you specify a negative value, this is simply the ID of a single Attached Results Set.
`INT4 outSetID`	ID of Femap Set that contains Output Sets to Save. If you specify a zero all Output Sets in the Result File will be saved. If you specify a negative value, this is simply the ID of a single Results Set.
`INT4 outVecID`	ID of Femap Set that contains Output Vectors to Save. If you specify a zero all Output Vectors in the Result File will be saved. If you specify a negative value, this is simply the ID of a single Output Vector.
`INT4 ndSetID`	ID of Femap Set that contains the nodes where output will be saved. Alternatively, if you specify a zero, results for all nodes in the Result File will be saved.
`INT4 elSetID`	ID of Femap Set that contains elements where output will be saved. Alternatively, if you specify a zero, results for all elements in the Result File will be saved.
`BOOL bDetach`	If True, result file will be detached after save operation.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

FE_FAIL	Error saving results from Attached File to Femap Database.

Remarks/Usage:

If bDetach is False then selected Output Sets will be saved to a new ids. If true then Femap will preserve the Femap Output Set and automatically detach the Results File when finished.

Example:

```
Sub Main

Dim App As femap.model

Set App = feFemap()


Dim selID As Long

Dim ndset As femap.Set

Set ndset = App.feSet

Dim atts As femap.Set

Set atts = App.feSet


atts.SelectID(FT_RES_ATTACH, "Select Attached File To Save", selID )

ndset.Select(FT_NODE, True, "Select Nodes" )


App.feFileAttachSave2V2(-selID, -1, -1, ndset.ID, 0, True)

End Sub
```


Unpublished work. © 2025 Siemens

feFileAttachByOutset

feFileAttachByOutset **(ID)**	
Description:	
This method returns the ID of the Attached Results. This is the same ID that is displayed in the Manage Results Files dialog box.	
Input:	
`INT4 ID`	ID of Output Set.
Output:	
None	
Return Code:	
`Attach ID`	ID of the Attached Results.
Remarks/Usage:	
None	
Example:	
None	

Import/Export Neutral File

Unpublished work. © 2025 Siemens

feFileReadNeutral

feFileReadNeutral

(brand, fName, read_geometry, read_model, read_output, status_messages, offsetID)

Description:

This method is the same as calling `feFileReadNeutral2` with `read_groups` and `read_views` equal to `read_model`. Refer to that method for more information.

Input:

None	None

Output:

None	None

Return Code:

`FE_FAIL`	No Femap model is open.
`FE_FILE_OPEN_FAILED`	The neutral file failed to open.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feFileReadNeutral2

feFileReadNeutral2

(brand, fName, read_geometry, read_model, read_output, read_groups, read_views, status_messages, offsetID)

Description:

This method is the same as calling `feFileReadNeutral3` with `renumber_layers` and `renumber_load_bc` set to True, and `create_new_output` set to False.

Input:

None	None

Output:

None	None

Return Code:

`FE_FAIL`	No Femap model is open.
`FE_FILE_OPEN_FAILED`	The neutral file failed to open.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feFileReadNeutral3

feFileReadNeutral3

**(brand, fName, read_geometry, read_model, read_output, read_groups, read_views, status_messages,
offsetID, renumber_layers, renumber_load_bc, create_new_output)**

Description:

This function reads a Femap Neutral file.

Input:

`INT4 brand`	Used to select the format of the Neutral file. Not currently used.
`STRING fName`	The filename of the Neutral file to read.
`BOOL read_geometry`	If True, geometry is read from the file.
`BOOL read_model`	If True, finite element model data is read from the file.
`BOOL read_output`	If True, analysis results will be read from the file.
`BOOL read_groups`	If True, groups will be read from the file.
`BOOL read_views`	If True, views will be read from the file.
`BOOL status_messages`	If True, status messages will be written to the Messages and Lists window to show how many entities are read.
`INT4 offsetID`	Normally set to 0. If you specify another value, all IDs that are read are offset by this value.
`BOOL renumber_layers`	If True, layers will be renumbered by the `offsetID` along with the rest of the model. If False, they are not renumbered.
`BOOL renumber_load_bc`	If True, Load and Constraint sets are renumbered by `offsetID` along with the rest of the model. If False, they are not renumbered.
`BOOL create_new_output`	If True, Output Sets are never overwritten - new ones are always created. The Output Set IDs in

Unpublished work. © 2025 Siemens

	the file are used to determine the order in which they are created - the ID is added to the previous maximum Output Set ID in the model. For example, if your model already contained Output Sets up to 10, then if you read Output Set 1 it would be stored as 11, Set 2 would become 12 and so on. If your model did not contain output, this option has no effect.
Output:	
None	
Return Code:	
`FE_FAIL`	No Femap model is open.
`FE_FILE_OPEN_FAILED`	The neutral file failed to open.
Remarks/Usage:	
`feFileWriteNeutral4` has an additional input `BOOL read_computed_output_defintion` for Computed Vectors write options. Use the `feFileWriteNeutral3` method if you need to write the file to read the Computed Vectors.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileReadNeutral4

feFileReadNeutral4

```
( bUseDlg, fName, read_geometry, read_model, read_output,
read_computed_output_defintion, read_groups,
 read_views, status_messages, offsetID, renumber_layers,
renumber_load_bc, create_new_output,  )
```

Description:

This function reads a Femap Neutral file.

Input:

`BOOL bUseDlg`	If True, the dialog box with import options will appear. If False the dialog box will not appear and all the argument options below will be considered.
`STRING fName`	The filename of the Neutral file to read.
`BOOL read_geometry`	If True, geometry is read from the file.
`BOOL read_model`	If True, finite element model data is read from the file.
`BOOL read_output`	If True, analysis results will be read from the file.
`BOOL read_computed_output_defintion`	If True, will read the Computed Vectors output definition.
`BOOL read_groups`	If True, groups will be read from the file.
`BOOL read_views`	If True, views will be read from the file.
`BOOL status_messages`	If True, status messages will be written to the Messages and Lists window to show how many entities are read.
`INT4 offsetID`	Normally set to 0. If you specify another value, all IDs that are read are offset by this value.
`BOOL renumber_layers`	If True, layers will be renumbered by the `offsetID` along with the rest of the model. If False, they are not renumbered.

Unpublished work. © 2025 Siemens

`BOOL renumber_load_bc`	If True, Load and Constraint sets are renumbered by `offsetID` along with the rest of the model. If False, they are not renumbered.
`BOOL create_new_output`	If True, Output Sets are never overwritten - new ones are always created. The Output Set IDs in the file are used to determine the order in which they are created - the ID is added to the previous maximum Output Set ID in the model. For example, if your model already contained Output Sets up to 10, then if you read Output Set 1 it would be stored as 11, Set 2 would become 12 and so on. If your model did not contain output, this option has no effect.
Output:	
None	
Return Code:	
`FE_FAIL`	No Femap model is open.
`FE_FILE_OPEN_FAILED`	The neutral file failed to open.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteNeutral

feFileWriteNeutral

(brand, fName, write_geometry_model, write_analysis_model, write_output, write_addl_info, expand_geometric, minimize_size, digits, version, groupID)

Description:

This method is the same as calling `feFileWriteNeutral2` with `write_selector` and `write_palette` equal to True, and `write_groups` and `write_views` equal to `write_analysis_model`. Refer to that method for more information.

Input:

`INT4 brand`	Used to select the format of the Neutral file: 0 = Femap Neutral (the default), 1=CAEFEM, 2=CDA/Sprint, 3=CFDesign, 4=SINDA. The formats other than 0 should not be used unless you are writing a file specifically to be used with that program.
`STRING fName`	The file name of the Neutral file to write.
`BOOL write_geometry_model`	If True, geometry is written to the file.
`BOOL write_analysis_model`	If True, finite element model data is written to the file.
`BOOL write_output`	If True, analysis results will be written to the file.
`BOOL write_addl_info`	If True, some additional data blocks will be written to the file. This should normally be False if the Neutral file is to be reloaded into Femap.
`BOOL expand_geometric`	If True, geometric loads and constraints will be expanded before being written.
`BOOL minimize_size`	If True, spaces are compressed out of the file to reduce its size.
`INT4 digits`	The number of digits of precision for all real numbers (must be between 2 and 20).
`REAL8 version`	The version of the neutral file to produce. Set to 0.0 to create the latest version, or specify a valid neutral version (reference the **File → Export**

Unpublished work. © 2025 Siemens

	→ **Neutral** command for a valid list of neutral versions). **Note** Some model data may be incompatible with previous versions of neutral files and writing to an older version could lead to a loss of model data.
`INT4 groupID`	The ID of the group to write to the file. Specify 0 to write the entire model.
Output:	
None	None
Return Code:	
`FE_INVALID`	You specified an invalid value for digits.
`FE_NOT_EXIST`	You chose a group that does not exist.
`FE_FAIL`	An error occurred processing the group.
Remarks/Usage:	
`feFileWriteNeutral2` and `feFileWriteNeutral3` have additional options for controlling specifically what data gets written to the Neutral file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteNeutral2

feFileWriteNeutral2

```
( brand, fName, write_geometry_model, write_analysis_model, write_output,
write_selector, write_palette, write_groups, write_views,
write_addl_info, expand_geometric,
minimize_size, digits, version, groupID )
```

Description:

This function writes a Femap Neutral file.

Input:

`INT4 brand`	Used to select the format of the Neutral file: 0=Femap Neutral (the default), 1=CAEFEM, 2=CDA/Sprint, 3=CFDesign, 4=SINDA. The formats other than 0 should not be used unless you are writing a file specifically to be used with that program.
`STRING fName`	The file name of the Neutral file to write.
`BOOL write_geometry_model`	If True, geometry is written to the file.
`BOOL write_analysis_model`	If True, finite element model data is written to the file.
`BOOL write_output`	If True, analysis results will be written to the file.
`BOOL write_selector`	If True, selector information will be written to the file
`BOOL write_palette`	If True, color palette information will be written to the file
`BOOL write_groups`	If True, groups will be written to the file
`BOOL write_views`	If True, views will be written to the file
`BOOL write_addl_info`	If True, some additional data blocks will be written to the file. This should normally be False if the Neutral file is to be reloaded into Femap.
`BOOL expand_geometric`	If True, geometric loads and constraints will be expanded before being written.

Unpublished work. © 2025 Siemens

`BOOL minimize_size`	If True, spaces are compressed out of the file to reduce its size.
`INT4 digits`	The number of digits of precision for all real numbers (must be 2 to 20).
`REAL8 version`	The version of the neutral file to produce. Set to 0.0 to create the latest version, or specify a valid neutral version (reference the **File → Export → Neutral** command for a valid list of neutral versions). **Note** Some model data may be incompatible with previous versions of neutral files and writing to an older version could lead to a loss of model data.
`INT4 groupID`	The ID of the group to write to the file. Specify 0 to write the entire model.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for digits.
`FE_NOT_EXIST`	You chose a group that does not exist.
`FE_FAIL`	An error occurred processing the group.
Remarks/Usage:	
`feFileWriteNeutral3` has an additional input `BOOL write_computed_output_definition` for Computed Vectors write options. Use the `feFileWriteNeutral3` method if you need to write the Computed Vectors to the file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteNeutral3

feFileWriteNeutral3

(brand, fName, write_geometry_model, write_analysis_model, write_output, write_selector, write_palette, write_groups, write_views, write_computed_output_definition, write_addl_info, expand_geometric, minimize_size, digits, version, groupID)

Description:

This function writes a Femap Neutral file.

Input:

`INT4 brand`	Used to select the format of the Neutral file: 0=Femap Neutral (the default), 1=CAEFEM, 2=CDA/Sprint, 3=CFDesign, 4=SINDA. The formats other than 0 should not be used unless you are writing a file specifically to be used with that program.
`STRING fName`	The file name of the Neutral file to write.
`BOOL write_geometry_model`	If True, geometry is written to the file.
`BOOL write_analysis_model`	If True, finite element model data is written to the file.
`BOOL write_output`	If True, analysis results will be written to the file.
`BOOL write_computed_output_definition`	If True, computed vector definitions will be written to the file.
`BOOL write_selector`	If True, selector information will be written to the file
`BOOL write_palette`	If True, color palette information will be written to the file
`BOOL write_groups`	If True, groups will be written to the file
`BOOL write_views`	If True, views will be written to the file

Unpublished work. © 2025 Siemens

`BOOL write_addl_info`	If True, some additional data blocks will be written to the file. This should normally be False if the Neutral file is to be reloaded into Femap.
`BOOL expand_geometric`	If True, geometric loads and constraints will be expanded before being written.
`BOOL minimize_size`	If True, spaces are compressed out of the file to reduce its size.
`INT4 digits`	The number of digits of precision for all real numbers (must be 2 to 20).
`REAL8 version`	The version of the neutral file to produce. Set to 0.0 to create the latest version, or specify a valid neutral version (reference the **File → Export → Neutral** command for a valid list of neutral versions). **Note** Some model data may be incompatible with previous versions of neutral files and writing to an older version could lead to a loss of model data.
`INT4 groupID`	The ID of the group to write to the file. Specify 0 to write the entire model.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for digits.
`FE_NOT_EXIST`	You chose a group that does not exist.
`FE_FAIL`	An error occurred processing the group.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteFNOV2

`feFileWriteFNOV2`

`( nSetSet, nVectorSet, nGroup, fName )`

Description:

This function writes a Femap Neutral Output File (.FNO). This method replaces `feFileWriteFNO`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nSetSet`	The ID of a Set Object that contains the IDs of the Output Sets to write to the FNO File. If you simply want to write all available Output Sets, specify 0.
`INT4 nVectorSet`	The ID of a Set Object that contains the IDs of the Output Vectors to write to the FNO File. If you simply want to write all available Output Vectors, specify 0.
`INT4 nGroup`	The ID of a Group that contains the nodes and/or elements whose output will be written. If you specify 0, output for all nodes/elements will be written. If the Group contains only elements and no nodes, and you choose to write nodal vectors, only output on the nodes connected to the selected elements will be written. If the Group contains nodes, then output on only those nodes will be written.
`STRING fName`	The filename of the FNO file to write.

Output:

None	

Return Code:

`FE_FAIL`	An error occurred writing the FNO file.

Remarks/Usage:

This method allows you to extract all or a subset of results from the Femap database or other attached results files and write it to a Femap Neutral Output File which can then be either read into the Femap database or attached.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feFileWriteFNO2V2

feFileWriteFNO2V2

(nSetSet, nVectorSet, nOuptutOnGroup, nElementConnectivityGroup, fName)

Description:

This function writes a Femap Neutral Output File (.FNO) with option to also include element connectivity records in the file. This method replaces `feFileWriteFNO2`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nSetSet`	The ID of a Set Object that contains the IDs of the Output Sets to write to the FNO File. If you simply want to write all available Output Sets, specify 0.
`INT4 nVectorSet`	The ID of a Set Object that contains the IDs of the Output Vectors to write to the FNO File. If you simply want to write all available Output Vectors, specify 0.
`INT4 nOutputOnGroup`	The ID of a Group that contains the nodes and/or elements whose output will be written. If you specify 0, output for all nodes/elements will be written. If the Group contains only elements and no nodes, and you choose to write nodal vectors, only output on the nodes connected to the selected elements will be written. If the Group contains nodes, then output on only those nodes will be written.

`INT4`
 	
`nElementConnectivityGroup`
	Does not affect output being sent to the FNO. The ID of a Group that contains plot only elements to be written. If you specify 0, no elements will be written to the FNO file.

`STRING fName`	The filename of the FNO file to write.

Output:

None	

Return Code:

`FE_FAIL`	An error occurred writing the FNO file.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method allows you to extract all or a subset of results from the Femap database or other attached results files and write it to a Femap Neutral Output File which can then be either read into the Femap database or attached. It also allows element connectivity records to be written to the FNO.
Example:
None

Unpublished work. © 2025 Siemens

feFileWriteFNO3V2

feFileWriteFNO3V2

(nSetSet, nVectorSet, nOuptutOnGroup, nElementConnectivityGroup, fName)

Description:

This function writes a Femap Neutral Output File (.FNO) with option to also include element connectivity records in the file. The file version must be specified for the map of output vector IDs. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nSetSet`	The ID of a Set Object that contains the IDs of the Output Sets to write to the FNO File. If you simply want to write all available Output Sets, specify 0.
`INT4 nVectorSet`	The ID of a Set Object that contains the IDs of the Output Vectors to write to the FNO File. If you simply want to write all available Output Vectors, specify 0.
`INT4 nOutputOnGroup`	The ID of a Group that contains the nodes and/or elements whose output will be written. If you specify 0, output for all nodes/elements will be written. If the Group contains only elements and no nodes, and you choose to write nodal vectors, only output on the nodes connected to the selected elements will be written. If the Group contains nodes, then output on only those nodes will be written.

`INT4`
 	
`nElementConnectivityGroup`
	Does not affect output being sent to the FNO. The ID of a Group that contains plot only elements to be written. If you specify 0, no elements will be written to the FNO file.

`INT4 nFileVersion`	The version of file to write.
`STRING fName`	The filename of the FNO file to write.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	An error occurred writing the FNO file.
Remarks/Usage:	
This method allows you to extract all or a subset of results from the Femap database or other attached results files and write it to a Femap Neutral Output File which can then be either read into the Femap database or attached. It also allows element connectivity records to be written to the FNO.	
Example:	
None	

Export Geometry

Unpublished work. © 2025 Siemens

feFileWriteAcis

feFileWriteAcis **(useDlg, fName, version, solidSET)**	
Description:	
This function writes solids from your model into a ACIS SAT file.	
Input:	
`BOOL useDlg`	If True, the function runs interactively. The dialog box that asks you to pick the solids to write will be displayed.
`STRING fName`	The filename of the ACIS SAT file to write.
`REAL8 version`	The ACIS version number of the file to be written. Set this to 0 to use the current version of ACIS. Available previous versions are the same numbers as displayed in the dialog box when you run the **File** → **Export** → **Geometry** command.
`INT4 solidSET`	The ID of a selection set that contains the IDs of the solids to write. Alternatively, if you specify a negative value, this is simply the ID of the single solid to write. Not used if `useDlg=True`.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteIges

feFileWriteIges **(fName, spline_curves, spline_surfaces, bounded_surfaces, customization, surface_options, export_as, solidSET)**	
Description:	
This function writes selected solids to an IGES file.	
Input:	
`STRING fName`	The filename of the IGES file to write.
`BOOL spline_curves`	If True, all curves are written as splines.
`BOOL spline_surfaces`	If True, surfaces are written as splines.
`BOOL bounded_surfaces`	If True, surfaces will be written as Type 143, bounded surfaces. Otherwise, Type 144 trimmed surfaces will be written.
`INT4 customization`	Controls the IGES customization options: 0=None, 1=AutoCAD, 2=JAMAIS, 3=I-DEAS.
`INT4 surface_options`	Controls how trimming curves will be written: 0=Both, 1=2D Curves, 2=3D Curves.
`INT4 export_as`	Controls how to write solids: 0=Solid MSBO, 1=Trimmed Surfaces, 2=Wireframe.
`INT4 solidSET`	The ID of a selection set that contains the IDs of the solids to write. Alternatively, if you specify a negative value, this is simply the ID of the single solid to write.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
Remarks/Usage:	
Only Parasolid solids can be written to an IGES file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteParasolid

`feFileWriteParasolid`
 	
`( useDlg, fName, version, solidSET )`	

Description:	
This function writes solids from your model into a Parasolid X_T file.	
Input:	
`BOOL useDlg`	If True, the function runs interactively. The dialog box that asks you to pick the solids to write will be displayed.
`STRING fName`	The filename of the Parasolid X_T file to write.
`REAL8 version`	The Parasolid version number of the file to be written. Set this to 0 to use the current version of Parasolid. Available previous versions are the same numbers as displayed in the dialog box when you run the **File** → **Export** → **Geometry** command.
`INT4 solidSET`	The ID of a selection set that contains the IDs of the solids to write. Alternatively, if you specify a negative value, this is simply the ID of the single solid to write. Not used if `useDlg=True`.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteStep2

feFileWriteStep2 **(fName, solidSET)**	
Description:	
This function writes selected solids to a STEP file.	
Input:	
`STRING fName`	The filename of the STEP file to write.
`INT4 solidSET`	The ID of a selection set that contains the IDs of the solids to write. Alternatively, if you specify a negative value, this is simply the ID of the single solid to write.
`INT4 format`	The format to use when exporting the STEP file, 0=AP203, 1=AP203 Edition 2, 2=AP214
Output:	
None	
Return Code:	
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteStep - Obsolete

feFileWriteStep **(fName, solidSET)**	
Description:	
This method is obsolete and should no longer be used. Use `feFileWriteStepOpt2` instead. This function writes selected solids to a STEP file.	
Input:	
`STRING fName`	The filename of the STEP file to write.
`INT4 solidSET`	The ID of a selection set that contains the IDs of the solids to write. Alternatively, if you specify a negative value, this is simply the ID of the single solid to write.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user aborted the operation by cancelling a dialog box.
`FE_INVALID`	You specified a parameter that was not in the supported range.
`FE_NOT_AVAILABLE`	The required module for this function is not loaded on this system.
`FE_SECURITY`	The required module for this function is not properly licensed.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteJT

feFileWriteJT **(fName, solidSET)**	
Description:	
This function writes selected solids to a JT file.	
Input:	
`STRING fName`	The filename of the STL file to write.
`INT4 solidSET`	The ID of a selection set that contains the IDs of the solids to write. Alternatively, if you specify a negative value, this is simply the ID of the single solid to write.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to Export.
`FE_INVALID`	Found a Solid that wasn't Parasolid.
`FE_NOT_EXIST`	No Parasolid Solids selected to output.
`FE_NOT_AVAILABLE`	JT exporter not installed/found.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteStl

feFileWriteStl

(fName, elemSET)

Description:

This function writes elements, or the faces of elements (solids) into a stereolithography (STL) file.

Input:

`STRING fName`	The filename of the STL file to write.
`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to write. Alternatively, if you specify a negative value, this is simply the ID of the single element to write.

Output:

None	

Return Code:

None	

Remarks/Usage:

All entities written to the STL file are in the form of triangles. Any quadrilateral faces that are selected are split into two triangles. Faces with midside nodes are split into multiple triangles. Planar elements are written *as-is*, while solid elements are combined and only their *free-faces* are written.

Example:

None

Unpublished work. © 2025 Siemens

feFileWriteTopologyStl

`feFileWriteTopologyStl`
 	
`( fName, solidSET )`	

Description:	
This function writes elements from criteria plot into a stereolithography (STL) file.	
Input:	
`STRING fName`	The filename of the STL file to write.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to Export.
Remarks/Usage:	
A criteria plot of the model must be shown in the active view to write an STL file. All entities written to the STL file are in the form of triangles. Any quadrilateral faces that are selected are split into two triangles. Faces with midside nodes are split into multiple triangles. Planar elements are written *as-is*, while solid elements are combined and only their *free-faces* are written.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteVrml

feFileWriteVrml

(fName, version, write_solid, single_color, write_deformed, write_contour, write_background, background_color, color, solidSET)

Description:

This function writes either elements or solids to a VRML file.

Input:

`STRING fName`	The filename of the VRML file to write.
`REAL8 version`	Either 1.0 or 2.0. Specifies the version of VRML to write.
`BOOL write_solid`	If True, the solids selected in solidSET will be written to the VRML file; otherwise, the elements displayed in the active view will be written.
`BOOL single_color`	If True, all entities are written in the color specified.
`BOOL write_deformed`	If True and elements are written and the current view style is deformed, then the geometry written is in its deformed location.
`BOOL write_contour`	If True and elements are written and the current view style is contour, then the geometry written will include contour information.
`BOOL write_background`	If True, a plane representing the background is written along with the model geometry.
`INT4 background_color`	If the background is written, this is its color.
`INT4 color`	If `single_color=True`, this is the color used for all entities.
`INT4 solidSET`	If `write_solid=True`, then this is the ID of a selection set that contains the solids to write. Alternatively, you can specify a negative number that will be the ID of a single solid to write.

Output:

None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_INVALID`	One of the color values is outside of its acceptable range.
Remarks/Usage:	
None	
Example:	
None	

Export Analysis Model

Unpublished work. © 2025 Siemens

feFileWriteAbaqus

feFileWriteAbaqus

(brand, fName)

Description:

This function writes an ABAQUS input file.

Input:

`INT4 brand`	Not currently used.
`STRING fName`	The filename of the ABAQUS input file to write.

Output:

None	

Return Code:

None	

Remarks/Usage:

If you have an active analysis set, preferably one that has ABAQUS selected as the solver, then no dialog boxes will be displayed. If not, this function will display dialog boxes to collect whatever data that it needs to complete the file using the legacy analysis translator. The global variable `Pref_AnalysisType` is used to determine the type of analysis that is to be performed by the file.

Example:

None

Unpublished work. © 2025 Siemens

feFileWriteAnsys

feFileWriteAnsys

(brand, fName)

Description:

This function writes an ANSYS input file.

Input:

`INT4 brand`	Not currently used.
`STRING fName`	The filename of the ANSYS input file to write.

Output:

None	

Return Code:

None	

Remarks/Usage:

If you have an active analysis set, preferably one that has ANSYS selected as the solver, then no dialog boxes will be displayed. If not, this function will display dialog boxes to collect whatever data that it needs to complete the file using the legacy analysis translator. The global variable `Pref_AnalysisType` is used to determine the type of analysis that is to be performed by the file.

Example:

None

Unpublished work. © 2025 Siemens

feFileWriteDyna

feFileWriteDyna **(brand, fName)**	
Description:	
This function writes an LS-DYNA input file.	
Input:	
`INT4 brand`	Not currently used.
`STRING fName`	The filename of the DYNA file to write.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If you have an active analysis set, preferably one that has LS-DYNA selected as the solver, then no dialog boxes will be displayed. If not, this function will display dialog boxes to collect whatever data that it needs to complete the file using the legacy analysis translator. The global variable `Pref_AnalysisType` is used to determine the type of analysis that is to be performed by the file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteIdeas

feFileWriteIdeas

(fName, PartName, WriteGroups, AbsoluteTempScale, UnitSystem, UserLengthFactor, UserForceOrMass, UserForceOrMassFactor, UserTempFactor, UserTempOffset)

Description:

This function writes an I-DEAS universal file.

Input:

`STRING fName`	The filename of the I-DEAS universal file to write.
`STRING PartName`	The *Part Name* specified in the universal file.
`BOOL WriteGroups`	When True, groups in the model will be written to the universal file. When False, no groups are written.
`BOOL AbsoluteTempScale`	When True, sets the *Temperature Mode* to Kelvin or Rankine (depends on option set in `UnitSystem`), while False sets it to either Celsius or Fahrenheit.
`INT4 UnitSystem`	Sets Unit Systems. 1=SI: Meter (newton), 2=BG: Foot (pound f), 3=MG: Meter (kilogram f), 4=BA: Foot (poundal), 5=MM: mm (milli-newton), 6=CM: cm (centi-newton), 7=IN: Inch (pound f), 8=GM: mm (kilogram f), 9=MN: mm (newton), 10=US: User Defined (*User* values below only used when set to 10)
`REAL8 UserLengthFactor`	*units/meter* scale factor
`BOOL UserForceOrMass`	When True, uses units/force. When False, uses units/mass

`REAL8`
 	
`UserForceOrMassFactor`
	units/force or *units/mass* scale factor

`REAL8 UserTempFactor`	*units/degree* scale factor (`UnitSystem = User Defined` only)

Unpublished work. © 2025 Siemens

`REAL8 UserTempOffset`	offset from temperature (`UnitSystem = User Defined` only)
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value.
`FE_BAD_DATA`	Your model is invalid or contains unsupported data
Remarks/Usage:	
This function matches using the **File** → **Export** → **Analysis Model** command and I-DEAS has been selected.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteMarc

feFileWriteMarc **(brand, fName)**	
Description:	
This function writes a Marc input file.	
Input:	
`INT4 brand`	Not currently used.
`STRING fName`	The filename of the Marc file to read.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If you have an active analysis set, preferably one that has MARC selected as the solver, then no dialog boxes will be displayed. If not, this function will display dialog boxes to collect whatever data that it needs to complete the file using the legacy analysis translator. The global variable `Pref_AnalysisType` is used to determine the type of analysis that is to be performed by the file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteNastran

feFileWriteNastran **(brand, fName)**	
Description:	
This function writes a NASTRAN input file.	
Input:	
`INT4 brand`	Used to select the format of the NASTRAN file. 0=MSC, 1=NE, 2=UAI, 3=CSA, 4=SSS, 5=Cosmic, 6=ME, 7=VR, 8=NX.
`STRING fName`	The filename of the NASTRAN input file to write.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value for the brand option.
Remarks/Usage:	
If you have an active analysis set, preferably one that has the appropriate *brand* of NASTRAN selected as the solver, then no dialog boxes will be displayed. If not, this function will display dialog boxes to collect whatever data that it needs to complete the file using the specified legacy analysis translator. The global variable `Pref_AnalysisType` is used to determine the type of analysis that is to be performed by the file.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileWriteSTAAD

feFileWriteSTAAD

```
( fName, AnalysisType, sStructType, sLengthUnits, sForceUnits,
sPropUnits,
dGravConstant, bModalCalc, dModalCutOffFreq, nModalCutOffMode,
bPrintModel, bPrintDisp, bPrintReactions, bPrintMemberForces,
bPrintMemberStresses, bPrintElemForces, bPrintJointForces,
bPostDisp, bPostSectionDisp, bPostMemberForces, bPostStresses)
```

Description:

This function writes a STAAD input file.

Input:

`STRING fName`	The filename of the STAAD input file to write.
`INT4 AnalysisType`	1=Static, 2=Modes
`STRING sStructType`	“PLANE”, “SPACE”, “TRUSS”, “FLOOR” or “DATA”
`STRING sLengthUnits`	“INCHES”, “FEET”, “CM”, “METER”, “MMS”, “DME” or “KM”
`STRING sForceUnits`	“KIP”, “POUND”, “KG”, “MTON”, “NEWTON”, “KNS”, “MNS” or “DNS”
`STRING sPropUnits`	“INCHES”, “FEET”, “CM”, “METER”, “MMS”, “DME” or “KM”
`REAL8 dGravConstant`	The gravity constant
`BOOL bModalCalc`	True=MODAL CALCULATION REQUESTED, False=CALCULATE NATURAL FREQUENCY. Only used if AnalysisType is Modes.
`REAL8 dModalCutOffFreq`	The maximum frequency requested. Only used if `AnalysisType` is Modes.
`INT4 nModalCutOffMode`	The maximum mode number requested. Only used if `AnalysisType` is Modes.

Unpublished work. © 2025 Siemens

`BOOL bPrintModel`
`BOOL bPrintDisp`
`BOOL bPrintReactions`
`BOOL bPrintMemberForces`
`BOOL bPrintMemberStresses`
`BOOL bPrintElemForces`
`BOOL bPrintJointForces`
	If True, the associated information/output is printed

`BOOL bPostDisp`
`BOOL bPostSectionDisp`
`BOOL bPostMemberForces`
`BOOL bPostStresses`
	If True, the associated results are written for postprocessing.

Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid value.
`FE_BAD_DATA`	Your model is invalid or contains unsupported data
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Notes

Unpublished work. © 2025 Siemens

feNotesGet

feNotesGet **(noteSET, notes)**	
Description:	
This function retrieves the currently defined model notes.	
Input:	
`INT4 noteSET`	1 = Model Notes, 2 = Translation Text
Output:	
`STRING notes`	The string containing the notes.
Return Code:	
`FE_FAIL`	An error occurred allocating the notes string.
`FE_NOT_EXIST`	There are no currently defined notes.
Remarks/Usage:	
This function automatically allocates/reallocates the string for the returned notes. Notes can contain multiple lines and will have carriage return (CR) and linefeed (LF) characters between the lines. It is the responsibility of the calling program to free the memory allocated to the returned string.	
Example:	
None	

Unpublished work. © 2025 Siemens

feNotesPut

feNotesPut

(noteSET, clear, notes)

Description:

This function updates the currently defined model notes.

Input:

`INT4 noteSET`	1 = Model Notes, 2 = Translation Text
`BOOL clear`	If True, any existing notes are deleted before these notes are added; otherwise, this function appends to the current notes.
`STRING notes`	The string containing the notes to add.

Output:

None	

Return Code:

`FE_FAIL`	An error occurred creating the notes.

Remarks/Usage:

If you want to create notes with multiple lines, you must add carriage return (CR) and linefeed (LF) characters between the lines. For example, from Basic, you could use:

```
s = "First Line" & Chr(13) & Chr(10) & "Second Line"
rc = femap.feNotesPut(1, True, s)
```

Example:

None

Print and Copy

Unpublished work. © 2025 Siemens

feFilePageSetupSave

feFilePageSetupSave

(void)

Description:

This function permanently saves the current page setup settings. It is the same as pressing the **Permanent** button in the **Page Setup** command.

Input:

None	

Output:

None	

Return Code:

None	

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feFilePageSetupReset

```
feFilePageSetupReset

( void )
```

Description:

This function sets all of the page setup settings back to their default values. It is the same as pressing the **Reset** button in the **Page Setup** command.

Input:

None	

Output:

None	

Return Code:

None	

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feFilePrint

feFilePrint **(useDlg, asBitmap, source, fromFile, toFile)**	
Description:	
This function is obsolete. You should use `feFilePrint2( )` instead This function invokes the printing function. It can be used to print data that is in the current session, or saved in a file.	
Input:	
`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. If False, the other parameters automatically define the printing options.
`BOOL asBitmap`	If True, printing is done at screen (bitmap) resolution. If False, printing is in printer resolution.
`INT4 source`	Selects the data to print. (Active View=0, Desktop=1, File=2, MultiView Layout=3, Messages=4, Program File=5, Entity Info=6, API Programming=7, Data Table=8)
`STRING fromFile`	Only used if source=2, print from file. In this case, this is the full pathname of the file to print.
`STRING toFile`	Complete path/filename of print destination. If blank, print will go to the current printer.
Output:	
None	
Return Code:	
`FE_CANCEL`	User cancelled the dialog box.
`FE_FAIL`	An error occurred while printing.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feFilePrint2

feFilePrint2 **(useDlg, source, toFile)**	
Description:	
This function invokes the printing function.	
Input:	
`BOOL useDlg`	If True, the function runs interactively and displays a dialog box to allow the user to define the options. If False, the other parameters automatically define the printing options.
`INT4 source`	Selects the data to print. (Active View=0, Desktop=1, File=2, MultiView Layout=3, Messages=4, Program File=5, Entity Info=6, API Programming=7, Data Table=8, Charting=9)
`STRING toFile`	Complete path/filename of print destination. If blank, print will go to the current printer.
Output:	
None	
Return Code:	
`FE_CANCEL`	User cancelled the dialog box.
`FE_FAIL`	An error occurred while printing.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFilePictureCopy

feFilePictureCopy	
(void)	
Description:	
This function copies the contents of the current Graphics window to the clipboard.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method is obsolete. For new applications you should use `feFilePictureCopy2( )` If you are copying a bitmap, then view that you are copying must be visible on the screen. If it is partially or fully obscured by other applications, then the picture that you copy will not be correct.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFilePictureCopy2

feFilePictureCopy2 **(saveRegion)**	
Description:	
This function copies portions of the screen to the clipboard.	
Input:	
`INT4 saveRegion`	0=Graphics Window, 1=Layout, 2=Entire Desktop
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If you are copying a bitmap, then view that you are copying must be visible on the screen. If it is partially or fully obscured by other applications, then the picture that you copy will not be correct.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFilePictureSave

feFilePictureSave **(useDlg, saveDesktop, format, fName)**	
Description:	
This function copies the contents of the current Graphics window (or the entire desktop) to a file.	
Input:	
`BOOL useDlg`	If True, this function runs interactively and displays a dialog box to ask the user to define the format and fName options.
`BOOL saveDesktop`	If True, the entire desktop (screen) is copied to the file. If False, only the contents of the active Graphics window are copied. If you are copying the desktop, the only valid formats are Bitmap, JPEG, PNG, GIF, and TIFF
`INT4 format`	The format of the file to create. 1=Bitmap, 2=Metafile, 3=Placeable Metafile, 4=JPEG, 5=Bitmap Series, 6=AVI, 7=JT, 8="Reserved for internal use", 9=GIF, 10=Animated GIF, 11=TIF, 12=PNG. Bitmap Series, Animated GIF, and AVI formats are only available for animating windows. Metafiles and Placeable Metafiles are only available for non-Render windows.
`STRING fName`	The name of the file to create.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid format.
`FE_FAIL`	An error occurred saving the file.
Remarks/Usage:	
This method is obsolete - you should use `feFilePictureSave2` for new applications.	

Unpublished work. © 2025 Siemens

If you are saving a bitmap, then view that you are saving must be visible on the screen. If it is partially or fully obscured by other applications, then the file that you create will not be correct.
Example:
None

Unpublished work. © 2025 Siemens

feFilePictureSave2

feFilePictureSave2

(useDlg, saveRegion, format, fName)

Description:

This function copies the contents of the current Graphics window (or the entire desktop) to a file.

Input:

`BOOL useDlg`	If True, this function runs interactively and displays a dialog box to ask the user to define the format and `fName` options.
`INT4 saveRegion`	0=Graphics Window, 1=Layout, 2=Entire Desktop
`INT4 format`	The format of the file to create. 1=Bitmap, 2=Metafile, 3=Placeable Metafile, 4=JPEG, 5=Bitmap Series, 6=AVI, 7=JT, 8="Reserved for internal use", 9=GIF, 10=Animated GIF, 11=TIF, 12=PNG. Bitmap Series, Animated GIF, and AVI formats are only available for animating windows. Metafiles and Placeable Metafiles are only available for non-Render windows.
`STRING fName`	The name of the file to create.

Output:

None	

Return Code:

`FE_INVALID`	You specified an invalid format.
`FE_FAIL`	An error occurred saving the file.

Remarks/Usage:

If you are saving a bitmap, then view that you are saving must be visible on the screen. If it is partially or fully obscured by other applications, then the file that you create will not be correct.

Example:

None

Unpublished work. © 2025 Siemens

feFilePictureReplay

feFilePictureReplay **(fName)**	
Description:	
This function replays an existing bitmap or metafile.	
Input:	
`STRING fName`	The name of the picture to replay - must be a bitmap or metafile.
Output:	
None	
Return Code:	
`FE_FAIL`	An error occurred replaying the picture - most likely the file does not exist.
Remarks/Usage:	
This function displays the picture using the Femap Replay program, which is launched in a separate window. There is no function available to automatically close this replay window. The Replay program can only replay bitmaps and metafiles. Other picture file formats can be saved, but cannot be replayed using this method.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFilePictureSaveJT

feFilePictureSaveJT

(UseDlg, fName, eJTFileVErsion, eJTFileUnits, JTHierarchy, eLoadSetOption, LoadSets, eBCSetOption, BCSets, LinesAsCylinders, dCylinderFactor)

Description:

This function saves a *Standard Output* JT visualization file using the specified options.

Input:

`BOOL UseDlg`	If True, this function runs interactively and displays a dialog box to ask the user to define `fName` and all other available options to create a JT visualization file.
`STRING fName`	The name of JT visualization file.
`enum eJTFileVersion`	JT Version - use `enum zJT_FileVersion`: `JT_V_8_0 = 8.0 (80), JT_V_8_1 = 8.1 (81), JT_V_8_2 = 8.2 (82), JT_V_9_0 = 9.0 (90), JT_V_9_1 = 9.1 (91), JT_V_9_2 = 9.2 (92), JT_V_9_3 = 9.3 (93), JT_V_9_4 = 9.4 (94), JT_V_9_5 = 9.5 (95), JT_V_10_0 = 10.0 (100), JT_V_10_1 = 10.1 (101), JT_V_10_2 = 10.2 (102), JT_V_10_3 = 10.3 (103), JT_V_10_4 = 10.4 (104), JT_V_10_5 = 10.5 (105), JT_V_10_6 = 10.6 (106), JT_V_10_7 = 10.7 (107), JT_V_10_8 = 10.8 (108), JT_V_10_9 = 10.8 (109)` **Note** JT Versions `JT_V_8_0 = 8.0 (80), JT_V_8_1 = 8.1 (81), JT_V_8_2 = 8.2`

Unpublished work. © 2025 Siemens

	`(82)` are deprecated, and will result in version `JT_V_10_3 = 10.3 (103)` being used as these versions are no longer supported by JT.

`enum eJTFileUnits`	JT Units - use `enum zJT_FileUnits`:
	
	`JT_UNIT_UNKNOWN = Unknown (0)`
	`JT_UNIT_MICRO = Micrometer (1)`
	`JT_UNIT_MILLI = Millimeter (2)`
	`JT_UNIT_CENTI = Centimeter (3)`
	`JT_UNIT_DECI = Decimeter (4)`
	`JT_UNIT_METER = Meter (5)`
	`JT_UNIT_KILO = Kilometer (6)`
	`JT_UNIT_INCH = Inch (7)`
	`JT_UNIT_FEET = Feet (8)`
	`JT_UNIT_YARD = Yard (9)`
	`JT_UNIT_MILE = Mile (10)`
	`JT_UNTI_MIL = Mil (11)`

`INT4 JTHeirarchy [0..6]`	Positions in the JT Hierarchy go from Top (0) to Bottom (6). All 7 items below must appear in the list but can only appear in the list once. The items are: Model = 1, Entity Type = 2, Entity Subtype = 3, Layer = 4, Property = 5, Material = 6, Curve & Surface ID = 7. Specifying a positive value for an item means item is enabled, while specifying a negative value for an item means item is deactivated.

`enum eLoadSetOption`	Load Set option - use enum zJT_SetOption:
	
	`JT_SETS_NONE = None (0),`
	`JT_SETS_VISIBLE = Visible (1),`
	`JT_SETS_ACTIVE = Active (2),`
	`JT_SETS_ALL = All (3), JT_SETS_SELECT = Select (4)`

`VARIANT LoadSets`	Only used when `eLoadSetOption` is set to `JT_SETS_SELECT` and contains the IDs of the Load Sets to include in the JT File

Unpublished work. © 2025 Siemens

`enum eBCSetOption`	Constraint Set option - use `enum zJT_SetOption` (see `eLoadSetOption`)
`VARIANT BCSets`	Only used when `eBCSetOption` is set to `JT_SETS_SELECT` and contains the IDs of the Load Sets to include in the JT File
`BOOL LinesAsCylinders`	When True, enables the Draw Lines as Cylinders option
`REAL8 dCylinderFactor`	Specifies value for Diameter
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFilePictureSaveJTMultiResults

feFilePictureSaveJTMultiResults

(UseDlg, fName, eJTFileVErsion, eJTFileUnits, OutputSets, OutputVectors, bRaw, bAverage, bTransform, bCorners)

Description:

This function saves a *Multi Result Output* JT visualization file using the specified options.

Input:

`BOOL UseDlg`	If True, this function runs interactively and displays a dialog box to ask the user to define fName and all other available options to create a JT visualization file.
`STRING fName`	The name of JT visualization file.
`enum eJTFileVersion`	JT Version - use `enum zJT_FileVersion`: `JT_V_8_0 = 8.0 (80), JT_V_8_1 = 8.1 (81), JT_V_8_2 = 8.2 (82), JT_V_9_0 = 9.0 (90), JT_V_9_1 = 9.1 (91), JT_V_9_2 = 9.2 (92), JT_V_9_3 = 9.3 (93), JT_V_9_4 = 9.4 (94), JT_V_9_5 = 9.5 (95), JT_V_10_0 = 10.0 (100), JT_V_10_1 = 10.1 (101), JT_V_10_2 = 10.2 (102), JT_V_10_3 = 10.3 (103), JT_V_10_4 = 10.4 (104), JT_V_10_5 = 10.5 (105), JT_V_10_6 = 10.6 (106), JT_V_10_7 = 10.7 (107), JT_V_10_8 = 10.8 (108), JT_V_10_9 = 10.8 (109)` **Note** JT Versions `JT_V_8_0 = 8.0 (80), JT_V_8_1 = 8.1 (81), JT_V_8_2 = 8.2 (82)` are deprecated, and will result in version

Unpublished work. © 2025 Siemens

	`JT_V_10_3 = 10.3 (103)` being used as these versions are no longer supported by JT.

`enum eJTFileUnits`	JT Units - use enum zJT_FileUnits:
	`JT_UNIT_UNKNOWN = Unknown (0)`
	`JT_UNIT_MICRO = Micrometer (1)`
	`JT_UNIT_MILLI = Millimeter (2)`
	`JT_UNIT_CENTI = Centimeter (3)`
	`JT_UNIT_DECI = Decimeter (4)`
	`JT_UNIT_METER = Meter (5)`
	`JT_UNIT_KILO = Kilometer (6)`
	`JT_UNIT_INCH = Inch (7)`
	`JT_UNIT_FEET = Feet (8)`
	`JT_UNIT_YARD = Yard (9)`
	`JT_UNIT_MILE = Mile (10)`
	`JT_UNTI_MIL = Mil (11)`

`VARIANT OutputSets`	IDs of the Output Sets to include in the JT visualization file. If the first entry of OutputSets is zero, all output sets are written and all other entries in the Variant are ignored. If the first entry of OutputSets is negative, then the value is the negative ID of a Femap Set. That set contains the Output Sets to write and all other entries in the Variant are ignored.
`VARIANT OutputVectors`	IDs of the Output Vectors to include in the JT visualization file. If the first entry of OutputVectors is zero, all output sets are written and all other entries in the Variant are ignored. If the first entry of OutputVectors is negative, then the value is the negative ID of a Femap Set. That set contains the Output Vectors to write and all other entries in the Variant are ignored.
`BOOL bRaw`	When True, the Raw Output Data option is specified. When False, Collated Vectors and Tensors is specified.

Unpublished work. © 2025 Siemens

`BOOL bAverage`	When True, enables the Use Averaging Options option. Only used when `bRaw = False`
`BOOL bTransform`	When True, enables the Use Transformation Options option. Only used when `bRaw = False`
`BOOL bCorners`	When True, enables the Use Corner Data option. Only used when `bRaw = False`
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Not all applications which allow visualization of JT data have the ability to display the CAE data created using the options in the Multi Result section. If a warning stating “The current service level doesn’t support displaying multiple CAE results.” or similar, then this functionality is not available.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileMessageCopy

feFileMessageCopy

(void)

Description:	
This function copies the selected messages from the Messages and Lists window to the clipboard.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If no messages are selected, then all messages will be copied; otherwise, only the selected ones are copied.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileMessageSave

feFileMessageSave **(useDlg, fName)**	
Description:	
This function copies the selected messages to a file.	
Input:	
`BOOL useDlg`	If True, this function runs interactively and displays a dialog box to ask the user to define the filename.
`STRING fName`	The name of the file to create.
Output:	
None	
Return Code:	
`FE_FAIL`	An error occurred saving the file.
Remarks/Usage:	
If no messages are selected, then all messages will be copied; otherwise, only the selected ones are copied.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileMessageSelect

feFileMessageSelect **(firstLine, lastLine)**	
Description:	
This function selects the specified lines in the Messages and Lists window.	
Input:	
`INT4 firstLine`	The line number of the first line of text to be selected.
`INT4 lastLine`	The line number of the last line of text to be selected.
Output:	
None	
Return Code:	
`FE_INVALID`	One of the line numbers that you selected was outside of the range of lines that are available.
Remarks/Usage:	
If either `firstLine` or `lastLine` are -1, the selection will be reset - no lines will be selected. Setting `firstLine = 0`, will automatically select starting at the first available line. Setting `lastLine = 0` will automatically select up to the last available line. You can use the `feFileMessageLineNumber( )` method to retrieve the current line number in the Messages and Lists Window. You can then save this value and later use it as a line number to select.	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileMessageLineNumber

feFileMessageLineNumber **(nLine)**	
Description:	
This function returns the line number of the last line currently in the message window	
Input:	
None	
Output:	
`INT4 nLine`	The line number of the last line in the message window
Return Code:	
`FE_INVALID`	Either the message window is closed or the last line can not be retrieved.
Remarks/Usage:	
You can use this function to retrieve the current line number in the Messages and Lists window. You can then save this value and later use it as a line number to call `feFileMessageSelect`.	
Example:	
None	

Rebuild and Preferences

Unpublished work. © 2025 Siemens

feFileRebuild

feFileRebuild **(useDlg, fullRebuild)**	
Description:	
This function is similar to the File,Rebuild command. It rebuilds the current Femap database and checks it for integrity.	
Input:	
`BOOL useDlg`	If True, this function runs interactively and displays a dialog box to ask whether to do a full or partial rebuild. An additional dialog box may also be displayed if solid geometry is found that cannot be loaded.
`BOOL fullRebuild`	Only used if `useDlg=False`. If True, a full rebuild is performed and all internal database pointers are reconstructed. If False, a partial rebuild is performed. This simply checks data references to make sure that all required entities still exist, and resets all usage counters.
Output:	
None	
Return Code:	
`FE_FAIL`	An error occurred during rebuild. Some data is missing or corrupted. (or the rebuild was cancelled if `useDlg=True`)
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFileRecoverDBData

feFileRecoverDBData **(useDlg, fName)**	
Description:	
This function is a low-level attempt to recover whatever data is still available in the scratch file of a corrupted model. It will not fully recover the model.	
Input:	
`BOOL useDlg`	If True, this function runs interactively and displays a dialog box to choose the file to recover
`STRING fName`	Only used if `useDlg=False`. The filename/path of the _DBData file to attempt to recover
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to recover from specified file.
`FE_NOT_EXIST`	The specified file does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feFilePreferencesSave

`feFilePreferencesSave`

`( void )`

Description:

This function permanently saves the current preference settings. It is the same as saving the preferences when you leave the **File → Preferences** command.

Input:

None	

Output:

None	

Return Code:

`FE_FAIL`	A problem occurred trying to save the settings.

Remarks/Usage:

This command does not allow you to change the preference settings. All preference settings can be accessed and changed using global variables. You should change the settings that you want to modify, then call this function to make those changes permanent.

Example:

None

Unpublished work. © 2025 Siemens

feFilePreferencesReset

```
feFilePreferencesReset

( void )
```

Description:

This function permanently resets the preference settings back to their default values. It is the same as pressing the Reset button in the **File** → **Preferences** command.

Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feRunIOTest

feRunIOTest **(sPath, nMethod)**	
Description:	
This function runs the *Read/Write Test* found in the **Database** tab of the **File** → **Preferences** command.	
Input:	
`STRING sPath`	Directory path to the current Scratch directory, where the test will take place.
Output:	
`INT4 nMethod`	Preferred result of test. 0=Windows I/O, 1=C I/O, 2=Windows 64K I/O, or 3=C 64K I/O.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Version Number

Unpublished work. © 2025 Siemens

feModelFileVersionV2

feModelFileVersionV2	
(void)	
Description:	
This function finds the Femap version number of a given file.	
Input:	
`STRING fName`	The file path and name.
Output:	
`REAL8`	The Femap version number.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Tool Methods

Tool Methods include:

Units

Unpublished work. © 2025 Siemens

feToolUnitConvertFactors

`feToolUnitConvertFactors`

`( baseMult, baseAdd, factorMult, factorAdd )`

Description:

This function computes the *computed* factors to use in a unit conversion of the current model.

Input:

`REAL8 baseMult[0..5]`	Base factors used to multiply dimensions.
`REAL8 baseAdd[0..5]`	Base factors added to converted dimensions.

Output:

`REAL8 factorMult[0..29]`	Computed factors used to multiply dimensions
`REAL8 factorAdd[0..29]`	Computed factors added to converted dimensions.

Return Code:

FE_TOO_SMALL	One of the arrays has dimensions that are too small to be used where you specified it.
`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).

Remarks/Usage:

For `baseMult[ ]` and `baseAdd[ ]` the array indices are:

0=Length, 1=Force, 2=Time, 3=Mass, 4=Temperature, 5=Energy

For `factorMult[ ]` and `factorAdd[ ]`, the array indices are:

0=Length, 1=Area, 2=Inertia, 3=Force, 4=Moment, 5=Spring, 6=SpringRot, 7=Pressure, 8=Acceleration, 9=AccelRot, 10=VelocityRot, 11=Damping, 12=DampingRot, 13=Mass, 14=Mass-Len, 15=Mass-PerLen, 16=MassPerArea, 17=Density, 18=MassInertia, 19=Temperature, 20=ThermalExpansion,

Unpublished work. © 2025 Siemens

21=ThermConductivity, 22=SpecificHeat, 23=HeatGenRate, 24=Velocity, 25=EnergyPerMass, 26=SpringRatePerArea, 27=DampingPerVolume, 28=VolumePerForce, 29=ThermalConductivity-LengthPerArea This method can be called prior to calling `feToolUnitConvert` to compute the factor arrays needed for that method.
Example:
None

Unpublished work. © 2025 Siemens

feToolUnitConvert

`feToolUnitConvert`
 	
`( baseMult, baseAdd, factorMult, factorAdd )`	

Description:	
This function converts the units of the current model.	
Input:	
`REAL8 baseMult[0..5]`	Base factors used to multiply dimensions.
`REAL8 baseAdd[0..5]`	Base factors added to converted dimensions.
`REAL8 factorMult[0..29]`	Computed factors used to multiply dimensions
`REAL8 factorAdd[0..29]`	Computed factors added to converted dimensions.
Output:	
None	
Return Code:	
`FE_TOO_SMALL`	One of the arrays has dimensions that are too small to be used where you specified it.
`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
For `baseMult[ ]` and `baseAdd[ ]` the array indices are: 0=Length, 1=Force, 2=Time, 3=Mass, 4=Temperature, 5=Energy For `factorMult[ ]` and `factorAdd[ ]`, the array indices are: 0=Length, 1=Area, 2=Inertia, 3=Force, 4=Moment, 5=Spring, 6=SpringRot, 7=Pressure, 8=Accelera-tion, 9=AccelRot, 10=VelocityRot, 11=Damping, 12=DampingRot, 13=Mass, 14=Mass-Len, 15=Mass-	

Unpublished work. © 2025 Siemens

PerLen, 16=MassPerArea, 17=Density, 18=MassInertia, 19=Temperature, 20=ThermalExpansion, 21=ThermConductivity, 22=SpecificHeat, 23=HeatGenRate, 24=Velocity, 25=EnergyPerMass,sion, 21=ThermConductivity, 22=SpecificHeat, 23=HeatGenRate, 24=Velocity, 25=EnergyPerMass, 26=SpringRatePerArea, 27=DampingPerVolume, 28=VolumePerForce, 29=ThermalConductivity-LengthPerArea You can use the `feToolUnitConvertFactors( )` method to automatically compute values for factorMult and factorAdd prior to calling this method.
Example:
None

Variables

Unpublished work. © 2025 Siemens

feVarPut

feVarPut

(varName, evalEqn, setConst, varValue, varEqn)

Description:

This function adds a variable/equation to the model.

Input:

`STRING varName`	Specifies the name of the variable to define. Must follow normal Femap variable naming conventions - start with a letter, unique to 5 characters.
`BOOL evalEqn`	If True, the `varEqn` equation is evaluated to compute `varValue`.
`BOOL setConst`	If True, the `varValue` value is converted to a text string and used as a constant equation in `varEqn`.
`REAL8 varValue`	Current evaluated value for the variable. Should be the current value of the equation specified.
`STRING varEqn`	The defining equation for the variable.

Output:

None	

Return Code:

`FE_BAD_DATA`	The equation that you specified in `varEqn` cannot be evaluated.
`FE_INVALID`	You specified an invalid variable name.

Remarks/Usage:

Normally you will not specify both `evalEqn=True` and `setConst=True`, but if you do, `evalEqn` is done first. Your original `varEqn` will be evaluated to find a constant which will then overwrite the original equation.

Example:

None

Unpublished work. © 2025 Siemens

feVarGet

feVarGet **(varName, varValue, varEqn)**	
Description:	
This function retrieves a variable including its current value and defining equation.	
Input:	
`STRING varName`	Specifies the name of the variable to retrieve. Must follow normal Femap variable naming conventions - start with a letter, unique to 5 characters.
Output:	
`REAL8 varValue`	Current evaluated value for the variable.
`STRING varEqn`	The defining equation for the variable.
Return Code:	
`FE_NOT_EXIST`	The variable name that you specified has not been defined.
`FE_INVALID`	You specified an invalid variable name.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feVarEval

feVarEval **(varEqn, varValue)**	
Description:	
This function evaluates an equation.	
Input:	
`STRING varEqn`	The equation to be evaluated. This string can follow the syntax of any equation that you can define in Femap, including the use of functions and other variables.
Output:	
`REAL8 varValue`	Evaluated value for the equation.
Return Code:	
`FE_BAD_DATA`	The equation cannot be evaluated.
Remarks/Usage:	
This function does not store its results or create any variables. It simply evaluates the expression that you supply.	
Example:	
None	

Layers

Unpublished work. © 2025 Siemens

feLayerPut

`feLayerPut` **`( layID, layColor, layTitle )`**	
Description:	
This function creates or updates a layer.	
Input:	
`INT4 layID`	The ID of the layer to create or update.
`INT4 layColor`	The color used for the layer that is created. Set to -1 to use the active layer color.
`STRING layTitle`	The title to be used for the layer.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLayerGet

feLayerGet **(layID, layColor, layTitle)**	
Description:	
This function retrieves information about a layer.	
Input:	
`INT4 layID`	The ID of the layer to create or update.
Output:	
`INT4 layColor`	The layer color.
`STRING layTitle`	The title of the layer.
Return Code:	
`FE_NOT_EXIST`	The layer that you requested has not been defined.
Remarks/Usage:	
None	
Example:	
None	

Text

Unpublished work. © 2025 Siemens

feTextPut

feTextPut **(textID, drawPointer, drawBorder, modelPos, horzJust, vertJust, visView, layer, fontID, backColor, bordColor, textColor, textLoc, pointerLoc, text)**	
Description:	
This function creates or updates a text annotation.	
Input:	
`INT4 textID`	The ID of the text entity to create or update.
`BOOL drawPointer`	If True, a pointer will be drawn from the text to a location that you specify in pointerLoc.
`BOOL drawBorder`	If True, a border will be drawn around the text.
`BOOL modelPos`	If True, both the textLoc and pointerLoc are in global Cartesian model coordinates. If False, they are in screen coordinates.
`INT4 horzJust`	The horizontal justification: 0=Center, 1=Left, 2=Right.
`INT4 vertJust`	The vertical justification: 0=Center, 1=Top, 2=Bottom.
`INT4 visView`	Specifies the views where the text will be visible: -1=All Views, 0=Currently Active View, >0=Specified View ID.
`INT4 layer`	The layer attribute for the text.
`INT4 fontID`	The ID of the font for the text. Same fonts as in the **Tools** → **Text** command.
`INT4 backColor`	The color of the background of the border (if `drawBorder=True`).
`INT4 bordColor`	The color of the edge of the border (if `drawBorder=True`).
`INT4 textColor`	The color of the text.

Unpublished work. © 2025 Siemens

`REAL8 textLoc[0..2]`	The location of the text. If in model coordinates, the X,Y,Z coordinates in global Cartesian. If in screen, the X and Y screen coordinates. In this case, the third coordinate should always be 0.0
`REAL8 pointerLoc[0..2]`	The location of the tip of the pointer.
`STRING text`	The text to create.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If you want to create text with multiple lines, you must add carriage return (CR) and linefeed (LF) characters between the lines. For example, from Basic, you could use: `s = "First Line" & Chr(13) & Chr(10) & "Second Line"`	
Example:	
None	

feTextPut2

`feTextPut2` **`( textID, drawPointer, drawBorder, pointerIs3D, modelPos, horzJust, vertJust, visView, layer, fontID, backColor, bordColor, textColor, textLoc, pointerLoc, text)`**
Description:
This function creates or updates a text annotation.
Input:

`INT4 textID`	The ID of the text entity to create or update.
`BOOL drawPointer`	If True, a pointer will be drawn from the text to a location that you specify in pointerLoc.

Unpublished work. © 2025 Siemens

`BOOL drawBorder`	If True, a border will be drawn around the text.
`BOOL pointerIs3D`	If True and modelPos is False, the pointer will be placed in Model space.
`BOOL modelPos`	If True, both the textLoc and pointerLoc are in global Cartesian model coordinates. If False, they are in screen coordinates.
`INT4 horzJust`	The horizontal justification: 0=Center, 1=Left, 2=Right.
`INT4 vertJust`	The vertical justification: 0=Center, 1=Top, 2=Bottom.
`INT4 visView`	Specifies the views where the text will be visible: -1=All Views, 0=Currently Active View, >0=Specified View ID.
`INT4 layer`	The layer attribute for the text.
`INT4 fontID`	The ID of the font for the text. Same fonts as in the **Tools** → **Text** command.
`INT4 backColor`	The color of the background of the border (if `drawBorder=True`).
`INT4 bordColor`	The color of the edge of the border (if `drawBorder=True`).
`INT4 textColor`	The color of the text.
`REAL8 textLoc[0..2]`	The location of the text. If in model coordinates, the X,Y,Z coordinates in global Cartesian. If in screen, the X and Y screen coordinates. In this case, the third coordinate should always be 0.0
`REAL8 pointerLoc[0..2]`	The location of the tip of the pointer.
`STRING text`	The text to create.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

If you want to create text with multiple lines, you must add carriage return (CR) and linefeed (LF) characters between the lines. For example, from Basic, you could use:

```
s = "First Line" & Chr(13) & Chr(10) & "Second Line"
```

Example:

None

Unpublished work. © 2025 Siemens

feTextMultiPut

feTextMultiPut

(drawPointer, drawBorder, modelPos, horzJust, vertJust, visView, layer, fontID, backColor, bordColor, textColor, nCount, textLoc, pointerLoc, text)

Description:

This function creates multiple text annotations.

Input:

`BOOL drawPointer`	If True, a pointer will be drawn from the text to a location that you specify in `pointerLoc`.
`BOOL drawBorder`	If True, a border will be drawn around the text.
`BOOL modelPos`	If True, both the textLoc and pointerLoc are in global Cartesian model coordinates. If False, they are in screen coordinates.
`INT4 horzJust`	The horizontal justification: 0=Center, 1=Left, 2=Right.
`INT4 vertJust`	The vertical justification: 0=Center, 1=Top, 2=Bottom.
`INT4 visView`	Specifies the views where the text will be visible: -1=All Views, 0=Currently Active View, >0=Specified View ID.
`INT4 layer`	The layer attribute for the text.
`INT4 fontID`	The ID of the font for the text. Same fonts as in the **Tools** → **Text** command.
`INT4 backColor`	The color of the background of the border (if `drawBorder=True`).
`INT4 bordColor`	The color of the edge of the border (if `drawBorder=True`).
`INT4 textColor`	The color of the text.

Unpublished work. © 2025 Siemens

`INT4 nCount`	The number of text items to create. This value specifies the number of entries in the remaining parameters
`REAL8 textLoc[0..3*nCount-1]`	The location of the text. If in model coordinates, the X,Y,Z coordinates in global Cartesian. If in screen, the X and Y screen coordinates. In this case, the third coordinate should always be 0.0. Entries in this array are stored [Text1X,Text1Y,Text1Z,Text2X,...TextNZ]
`REAL8 pointerLoc[0..3*nCount-1]`	The location of the tip of the pointer. Stored the same as `textLoc`.
`STRING text[0..nCount-1]`	The text items to create. Each string creates a unique Text entity.

Output:

None	

Return Code:

None	

Remarks/Usage:

If you want to create text with multiple lines, you must add carriage return (CR) and linefeed (LF) characters between the lines. For example, from Basic, you could use:

```
s = "First Line" & Chr(13) & Chr(10) & "Second Line"
```

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim xyz(8) As Double
   Dim t(2) As String

   Dim n As femap.Node
   Set n = App.feNode()

   n.Get(1)
   xyz(0) = n.x
   xyz(1) = n.y
   xyz(2) = n.z

   n.Get(2)
   xyz(3) = n.x
   xyz(4) = n.y
   xyz(5) = n.z
```


Unpublished work. © 2025 Siemens

```
   n.Get(3)
   xyz(6) = n.x
   xyz(7) = n.y
   xyz(8) = n.z

   t(0) = "AA"
   t(1) = "This is" & Chr(13) & Chr(10) & "Multi line"
   t(2) = "XYZDEF"

    App.feTextMultiPut ( False, False, True, 0, 0, -1, 1, 2,
              FCL_RED, FCL_BLUE, FCL_RED, 3, xyz, xyz, t )
End Sub
```

feTextMultiPut2

feTextMultiPut2

(drawPointer, pointerIs3D, drawBorder, modelPos, horzJust, vertJust, visView, layer, fontID, backColor, bordColor, textColor, nCount, textLoc, pointerLoc, text)

Description:

This function creates multiple text annotations.

Input:

`BOOL drawPointer`	If True, a pointer will be drawn from the text to a location that you specify in `pointerLoc`.
`BOOL pointerIs3D`	If True and modelPos is False, the pointer will be placed in Model space.
`BOOL drawBorder`	If True, a border will be drawn around the text.
`BOOL modelPos`	If True, both the textLoc and pointerLoc are in global Cartesian model coordinates. If False, they are in screen coordinates.
`INT4 horzJust`	The horizontal justification: 0=Center, 1=Left, 2=Right.
`INT4 vertJust`	The vertical justification: 0=Center, 1=Top, 2=Bottom.

Unpublished work. © 2025 Siemens

`INT4 visView`	Specifies the views where the text will be visible: -1=All Views, 0=Currently Active View, >0=Specified View ID.
`INT4 layer`	The layer attribute for the text.
`INT4 fontID`	The ID of the font for the text. Same fonts as in the **Tools** → **Text** command.
`INT4 backColor`	The color of the background of the border (if `drawBorder=True`).
`INT4 bordColor`	The color of the edge of the border (if `drawBorder=True`).
`INT4 textColor`	The color of the text.
`INT4 nCount`	The number of text items to create. This value specifies the number of entries in the remaining parameters
`REAL8 textLoc[0..3*nCount-1]`	The location of the text. If in model coordinates, the X,Y,Z coordinates in global Cartesian. If in screen, the X and Y screen coordinates. In this case, the third coordinate should always be 0.0. Entries in this array are stored [Text1X,Text1Y,Text1Z,Text2X,...TextNZ]
`REAL8 pointerLoc[0..3*nCount-1]`	The location of the tip of the pointer. Stored the same as `textLoc`.
`STRING text[0..nCount-1]`	The text items to create. Each string creates a unique Text entity.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

If you want to create text with multiple lines, you must add carriage return (CR) and linefeed (LF) characters between the lines. For example, from Basic, you could use:

```
s = "First Line" & Chr(13) & Chr(10) & "Second Line"
```


Unpublished work. © 2025 Siemens

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim xyz(8) As Double
   Dim t(2) As String

   Dim n As femap.Node
   Set n = App.feNode()

   n.Get(1)
   xyz(0) = n.x
   xyz(1) = n.y
   xyz(2) = n.z

   n.Get(2)
   xyz(3) = n.x
   xyz(4) = n.y
   xyz(5) = n.z

   n.Get(3)
   xyz(6) = n.x
   xyz(7) = n.y
   xyz(8) = n.z

   t(0) = "AA"
   t(1) = "This is" & Chr(13) & Chr(10) & "Multi line"
   t(2) = "XYZDEF"

    App.feTextMultiPut ( False, False, True, 0, 0, -1, 1, 2,
              FCL_RED, FCL_BLUE, FCL_RED, 3, xyz, xyz, t )
End Sub
```


Unpublished work. © 2025 Siemens

feTextGet

feTextGet

(textID, drawPointer, drawBorder, modelPos, horzJust, vertJust, visView, layer, fontID, backColor, bordColor, textColor, textLoc, pointerLoc, text)

Description:

This function retrieves a text entity.

Input:

`INT4 textID`	The ID of the text entity to create or update.

Output:

`BOOL drawPointer`	The status of the pointer (True=On).
`BOOL drawBorder`	The status of the border (True=On).
`BOOL modelPos`	If True, both the `textLoc` and `pointerLoc` are in global Cartesian model coordinates. If False, they are in screen coordinates.
`INT4 horzJust`	The horizontal justification: 0=Center, 1=Left, 2=Right.
`INT4 vertJust`	The vertical justification: 0=Center, 1=Top, 2=Bottom.
`INT4 visView`	Specifies the views where the text will be visible: -1=All Views, 0=Currently Active View, >0=Specified View ID.
`INT4 layer`	The layer attribute for the text.
`INT4 fontID`	The ID of the font for the text. Same fonts as in the **Tools** → **Text** command.
`INT4 backColor`	The color of the background of the border (if `drawBorder=True`).
`INT4 bordColor`	The color of the edge of the border (if `drawBorder=True`).
`INT4 textColor`	The color of the text.

Unpublished work. © 2025 Siemens

`REAL8 textLoc[0..2]`	The location of the text. If in model coordinates, the X,Y,Z coordinates in global Cartesian. If in screen, the X and Y screen coordinates. In this case the third coordinate should always be 0.0.
`REAL8 pointerLoc[0..2]`	The location of the tip of the pointer.
`STRING text`	The text. If there are multiple lines of text, there will be carriage return and linefeed characters in the string.
Return Code:	
`FE_NOT_EXIST`	The text entity that you requested does not exist.
Remarks/Usage:	
None	
Example:	
None	

feTextGet2

`feTextGet2`
`( textID, drawPointer, pointerIs3D, drawBorder, modelPos, horzJust, vertJust, visView, layer, fontID, backColor, bordColor, textColor, textLoc, pointerLoc, text)`
Description:
This function retrieves a text entity.
Input:

`INT4 textID`	The ID of the text entity to create or update.
Output:	
`BOOL drawPointer`	The status of the pointer (True=On).
`BOOL drawBorder`	The status of the border (True=On).

Unpublished work. © 2025 Siemens

`pointerIs3D`	If True pointer is in Model space. If False pointer is in View space.
`BOOL modelPos`	If True, both the `textLoc` and `pointerLoc` are in global Cartesian model coordinates. If False, they are in screen coordinates.
`INT4 horzJust`	The horizontal justification: 0=Center, 1=Left, 2=Right.
`INT4 vertJust`	The vertical justification: 0=Center, 1=Top, 2=Bottom.
`INT4 visView`	Specifies the views where the text will be visible: -1=All Views, 0=Currently Active View, >0=Specified View ID.
`INT4 layer`	The layer attribute for the text.
`INT4 fontID`	The ID of the font for the text. Same fonts as in the **Tools → Text** command.
`INT4 backColor`	The color of the background of the border (if `drawBorder=True`).
`INT4 bordColor`	The color of the edge of the border (if `drawBorder=True`).
`INT4 textColor`	The color of the text.
`REAL8 textLoc[0..2]`	The location of the text. If in model coordinates, the X,Y,Z coordinates in global Cartesian. If in screen, the X and Y screen coordinates. In this case the third coordinate should always be 0.0.
`REAL8 pointerLoc[0..2]`	The location of the tip of the pointer.
`STRING text`	The text. If there are multiple lines of text, there will be carriage return and linefeed characters in the string.
Return Code:	
`FE_NOT_EXIST`	The text entity that you requested does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Measure

Unpublished work. © 2025 Siemens

feMeasureDistance

feMeasureDistance **(p1, p2, dist)**	
Description:	
This function measures the distance between two coordinates.	
Input:	
`REAL8 p1[0..2]`	The first coordinate location.
`REAL8 p2[0..2]`	The second coordinate location.
Output:	
`REAL8 dist`	The distance between the coordinate locations.
Return Code:	
None	
Remarks/Usage:	
The coordinate locations must be in the same rectangular coordinate system for this function to work properly - preferably in global rectangular.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureDistanceToPlane

feMeasureDistanceToPlane

(p1, planePt, planeNorm, dist)

Description:

This function measures the distance between a coordinate location and a plane. The distance is the length of the perpendicular projection onto the plane.

Input:

`REAL8 p1[0..2]`	The coordinate location.
`REAL8 planePt[0..2]`	The coordinates of any point on the plane.
`REAL8 planeNorm[0..2]`	The components of the perpendicular vector to the plane.

Output:

`REAL8 dist`	The distance from the coordinate location to its projection onto the plane.

Return Code:

None	

Remarks/Usage:

The coordinate locations and components must be in the same rectangular coordinate system for this function to work properly - preferably in global rectangular.

Example:

None

Unpublished work. © 2025 Siemens

feMeasureDistanceBetweenGeometry

feMeasureDistanceBetweenGeometry

(enFromType, nFromSet, enToType, nToSet, bMinimum, nFromID, nToID, vFrom, vTo, dDistance)

Description:

This function measures the distance between two sets of geometric entities or a set of nodes and a set of geometric entities. The maximum or minimum distance is calculated using any combination of sets containing only points, curves, surfaces, or solids.

Input:

`INT4 enFromType`	The type of entities to measure “From”. For entity information, **Entity Types**. Can be FT_NODE (7), FT_POINT (3), FT_CURVE (4), FT_SURFACE (5), or FT_SOLID (39).
`INT4 nFromSet`	The ID of a set that contains a list of point, curve, surface, or solid IDs. Alternatively, if you specify a negative value, this is simply the ID of the single entity to measure *From*.
`INT4 enToType`	The type of entities to measure *To*. Can be FT_POINT (3), FT_CURVE (4), FT_SURFACE (5), or FT_SOLID (39).
`INT4 nToSet`	The ID of a set that contains a list of point, curve, surface, or solid IDs. Alternatively, if you specify a negative value, this is simply the ID of the single entity to measure *To*.
`INT4 bMinimum`	When True, calculates the minimum distance between geometric entities. When False, calculates the maximum distance.

Output:

`INT4 nFromID`	The ID of the geometric entity used as the *From* entity to determine minimum/maximum distance.
`INT4 nToID`	The ID of the geometric entity used as the *To* entity to determine minimum/maximum distance.
`REAL8 vFrom[0..2]`	The coordinates of the location on the `nFromID` entity used to determine minimum/maximum distance.

Unpublished work. © 2025 Siemens

`REAL8 vTo[0..2]`	The coordinates of the location on the `nToID` entity used to determine minimum/maximum distance.
`REAL8 dDistance`	The distance between the `vFrom` and `vTo` coordinates.
Return Code:	
`FE_NOT_EXIST`	One or more of the geometric entities does not exist.
Remarks/Usage:	
Offers similar functionality to the **Tools** → **Measure** → **Distance Between Geometry** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureDistanceBetweenNodesV2

feMeasureDistanceBetweenNodesV2

(nFrom, nTo, nOutSetID, nOutVecID, nCSysID, vecBase, vecDist, magDist)

Description:

This function measures the distance between two nodes which are optionally deformed using selected results. This function is superseded by `feMeasureDistanceBetweenNodes2`. This method replaces `feMeasureDistanceBetweenNodes`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nFrom`	The ID of the node to measure from.
`INT4 nTo`	The ID of the node to measure to.
`INT4 nOutSetID`	The ID of the output set which contains the results to use to deform the nodes. If 0, use the undeformed location of the nodes.
`INT4 nOutVecID`	The ID of the output vector in `nOutSetID` which contains the results to use to deform the nodes. If 0, use the undeformed locations.
`INT4 nCSysID`	The ID of the coordinate system in which the results are returned.

Output:

`REAL8 vecBase[0..2]`	The coordinates of the nFrom node (possibly deformed) in `CSys nCSysID`
`REAL8 vecDist[0..2]`	The components of the vector from nFrom to nTo in `CSys nCSysID`
`REAL8 magDist`	The distance between the nodes.

Return Code:

`FE_NOT_AVAILABLE`	Selected results Set/Vector do not exist.
`FE_INVALID`	No output available for one or more of the nodes in the selected results vector.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	One or more of the nodes does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureDistanceBetweenNodes2V2

feMeasureDistanceBetweenNodes2V2

(nFrom, nTo, nOutSetID, nOutVecID, nCSysID, vecBase, vecTip, vecDist, magDist)

Description:

This function measures the distance between two nodes which are optionally deformed using selected results.This function is supersedes by `feMeasureDistanceBetweenNodes`. This method replaces `feMeasureDistanceBetweenNodes2`. For more information, see**Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nFrom`	The ID of the node to measure from
`INT4 nTo`	The ID of the node to measure to
`INT4 nOutSetID`	The ID of the output set which contains the results to use to deform the nodes. If 0, use the undeformed location of the nodes.
`INT4 nOutVecID`	The ID of the output vector in `nOutSetID` which contains the results to use to deform the nodes. If 0, use the undeformed locations.
`INT4 nCSysID`	The ID of the coordinate system in which the results are returned.

Output:

`REAL8 vecBase[0..2]`	The coordinates of the `nFrom` node (possibly deformed) in `CSys nCSysID`
`REAL8 vecTip0..2]`	The coordinates of the `nTo` node (possibly deformed) in `CSys nCSysID`
`REAL8 vecDist[0..2]`	The components of the vector from `nFrom` to nTo in `CSys nCSysID`
`REAL8 magDist`	The distance between the nodes.

Return Code:

`FE_NOT_AVAILABLE`	Selected results Set/Vector do not exist.

Unpublished work. © 2025 Siemens

`FE_INVALID`	No output available for one or more of the nodes in the selected results vector.
`FE_NOT_EXIST`	One or more of the nodes does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureDistanceBetweenLines

feMeasureDistanceBetweenLines	
(pA1, pA2, pB1, pB2, dist)	
Description:	
This function measures the minimum distance between two line segments in space which are specified by the coordinates of their endpoints.	
Input:	
`REAL8 pA1[0..2]`	The coordinates at the first end of the first line.
`REAL8 pA2[0..2]`	The coordinates at the second end of the first line.
`REAL8 pB1[0..2]`	The coordinates at the first end of the second line.
`REAL8 pB2[0..2]`	The coordinates at the second end of the second line.
Output:	
`REAL8 dist`	The minimum distance between the two line segments.
Return Code:	
None	
Remarks/Usage:	
The coordinate locations must be in the same rectangular coordinate system for this function to work properly - preferably in global rectangular. The minimum distance will be the perpendicular distance if the closest point of approach lies within the lengths of the segments. If it does not, then the distance will be the distance from one of the endpoints to the other line.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureDistanceBetweenSolids

`feMeasureDistanceBetweenSolids` **`( solidID1, solidID2, dist )`**	
Description:	
This function measures the minimum distance between two solids in space.	
Input:	
`INT4 solidID1`	The IDs of the two solids to measure between.
`INT4 solidID2`	
Output:	
`REAL8 dist`	The minimum distance between the two solids.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureAngle

feMeasureAngle **(center, p1, p2, angle)**	
Description:	
This function measures the angle between three coordinate locations.	
Input:	
`REAL8 center[0..2]`	The coordinates of the center location.
`REAL8 p1[0..2]`	The first coordinate location.
`REAL8 p2[0..2]`	The second coordinate location.
Output:	
`REAL8 angle`	The angle between the coordinate locations (in degrees).
Return Code:	
None	
Remarks/Usage:	
The coordinate locations must be in the same rectangular coordinate system for this function to work properly - preferably in global rectangular. The center location is the point about which the angle is measured. The angle is the angle between the vector from the center to p1 and the vector from the center to p2. The angle returned is always less than 180 degrees. There is no indication of the direction swept by the angle.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureAngleBetweenNodesV2

feMeasureAngleBetweenNodesV2

(nVertex, nFrom, nTo, nOutSetID, nOutVecID, nCSysID, vecBase, vecNormal, magAngle)

Description:

This function measures the angle between three coordinate locations. This method replaces feMeasure-AngleBetweenNodes. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nVertex`	The ID of a node at the vertex of the angle
`INT4 nFrom`	The ID of a node to measure the angle from
`INT4 nTo`	The ID of a node to measure the angle to
`INT4 nOutSetID`	The ID of the output set which contains the results to use to deform the nodes. If 0, use the undeformed location of the nodes.
`INT4 nOutVecID`	The ID of the output vector in `nOutSetID` which contains the results to use to deform the nodes. If 0, use the undeformed locations.
`INT4 nCSysID`	The ID of the coordinate system in which the results are returned.

Output:

`REAL8 vecBase[0..2]`	The coordinates of the `nVertex` node (possibly deformed) in `CSys nCSysID`
`REAL8 vecNormal[0..2]`	The components of the vector that is normal to the plane formed by the three (possibly deformed) nodes, in `CSys nCSysID`
`REAL8 magAngle`	The angle between the nodes

Return Code:

None	

Remarks/Usage:

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feMeasureCurve

feMeasureCurve **(curveSET, totalLength)**	
Description:	
This function measures the length of one or more curves.	
Input:	
`INT4 curveSET`	The ID of a selection set that contains the IDs of the curves to measure. Alternatively, if you specify a negative value, this is simply the ID of the single curve to measure.
Output:	
`REAL8 totalLength`	The total length of all the curves.
Return Code:	
None	
Remarks/Usage:	
The total length of the curves is simply the sum of the lengths of the individual curves, it does not account for any overlap in the curves.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureSurfaceArea

feMeasureSurfaceArea

(surfaceSET, totalArea)

Description:

This function measures the area of one or more surfaces.

Input:

`INT4 surfaceSET`	The ID of a selection set that contains the IDs of the surfaces to measure. Alternatively, if you specify a negative value, this is simply the ID of the single surface to measure.

Output:

`REAL8 totalArea`	The total area of the selected surfaces.

Return Code:

`FE_NOT_EXIST`	One or more of the surfaces that you selected did not exist. `totalArea` is still the area of the existing surfaces.
`FE_BAD_TYPE`	One or more of the surfaces that you selected was not a solid surface.

Remarks/Usage:

This function only works for solid surfaces. Surfaces created with the standard geometry engine and boundary surfaces cannot be measured for area.

Example:

None

Unpublished work. © 2025 Siemens

feMeasureSolidMassProp

feMeasureSolidMassProp

(solidID, area, volume, cg, inertia)

Description:

This function computes the mass properties of a solid.

Input:

`INT4 solidID`	The ID of the solid to compute the mass properties.

Output:

`REAL8 area`	The total surface area of the solid.
`REAL8 volume`	The volume of the solid (if closed).
`REAL8 cg[0..2]`	The location of the center of gravity of the solid.
`REAL8 inertia[0..9]`	The 3x3 inertia matrix stored in row order [0][0], [0][1], [0][2], [1][0]...[2][2].

Return Code:

`FE_NOT_EXIST`	The solid that you have selected does not exist.

Remarks/Usage:

The cg and inertia terms are in global rectangular coordinates.

Example:

None

Unpublished work. © 2025 Siemens

feMeasureMeshMassProp

feMeasureMeshMassProp

**(elemSET, csysID, printResults, printDetails, len, area, volume, structMass, nonstructMass,
totalMass, structCG, nonstructCG, totalCG, inertia, inertiaCG)**

Description:

This function computes the mass properties of a mesh.

Input:

`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to measure. Alternatively, if you specify a negative value, this is simply the ID of the single element to measure.
`INT4 csysID`	The coordinate system used to compute the mass properties.
`BOOL printResults`	If True, the mass properties are also listed to the print destination.
`BOOL printDetails`	If True, and `printResults=True`, the detailed properties for each element are listed.

Output:

`REAL8 len`	The total length of all selected line elements.
`REAL8 area`	The total area of all selected planar elements.
`REAL8 volume`	The total volume of all selected elements.
`REAL8 structMass`	The structural mass of all selected elements.
`REAL8 nonstructMass`	The nonstructural mass of all selected elements.
`REAL8 totalMass`	The total (structural+nonstructural) mass of all selected elements.
`REAL8 structCG[0..2]`	The center of gravity of the structural mass.
`REAL8 nonstructCG[0..2]`	The center of gravity of the nonstructural mass.

Unpublished work. © 2025 Siemens

`REAL8 totalCG[0..2]`	The center of gravity of the total mass.
`REAL8 inertia[0..5]`	The inertias of the selected elements (0=I11, 1=I21, 2=I22, 3=I31, 4=I32, 5=I33) about the origin of the selected coordinate system.
`REAL8 inertiaCG[0..5]`	The inertias of the selected elements (0=I11, 1=I21, 2=I22, 3=I31, 4=I32, 5=I33) about the center of gravity.
Return Code:	
`FE_NOT_EXIST`	The solid that you have selected does not exist.
`FE_NEGATIVE_MASS_VOLUME`	Some of the elements that you have selected have either negative mass or volume. Returned values may be less than the total absolute mass or volume.
Remarks/Usage:	
Only total mass properties are returned. If you want to get individual properties, you must call this function and just select a single element.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureSurfSectionProp

feMeasureSurfSectionProp

(surfID, vecBase, vecDir, sectProp)

Description:

This function computes the section properties for a planar surface.

Input:

INT4 surfID	The ID of the surface to compute the properties.
REAL8 vecBase[0..2]	The coordinates of the origin of the orientation vector. Section properties are reported about this point.
REAL8 vecDir[0..2]	A vector in the direction of the Y axis of the section. Section properties are reported about this axis.

Output:

REAL8 sectProp[0..24]	The computed properties. 0= Area, 1= Y centroid, 2= Z centroid, 3= Y moment of inertia, 4= Z moment of inertia, 5= YZ product of inertia, 6= Angle to principal axes, 7= Y radius of gyration, 8= Z radius of gyration, 9= Polar moment of inertia, 10= Max Principal moment of inertia, 11= Min Principal moment of inertia, 12= Y Principal moment of inertia, 13= Z Principal moment of inertia, 14= YZ Principal moment of inertia, 15= Y Shear Center - from origin, 16= Z Shear Center - from origin, 17= Y Shear Center - from centroid, 18= Z Shear Center - from centroid, 19= Y Shear Area, 20= Z Shear Area, 21= YZ Shear Area, 22= Warping Constant, 23= Torsional Constant, 24= Not Used.

Return Code:

FE_TOO_SMALL	One of the arrays has dimensions which are too small to be used where you specified it.
FE_NOT_AVAILABLE	One of the arrays does not contain the required data.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeasureMeshSectionProp

feMeasureMeshSectionProp

(elemSET, vecBase, vecDir, sectProp)

Description:

This function computes the section properties for a selected set of elements.

Input:

`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to measure. Alternatively, if you specify a negative value, this is simply the ID of the single element to measure.
`REAL8 vecBase[0..2]`	The coordinates of the origin of the orientation vector. Section properties are reported about this point.
`REAL8 vecDir[0..2]`	A vector in the direction of the Y axis of the section. Section properties are reported about this axis.

Output:

`REAL8 sectProp[0..24]`	The computed properties: 0= Area, 1= Y centroid, 2= Z centroid, 3= Y moment of inertia, 4= Z moment of inertia, 5= YZ product of inertia, 6= Angle to principal axes, 7= Y radius of gyration, 8= Z radius of gyration, 9= Polar moment of inertia, 10= Max Principal moment of inertia, 11= Min Principal moment of inertia, 12= Y Principal moment of inertia, 13= Z Principal moment of inertia, 14= YZ Principal moment of inertia, 15= Y Shear Center - from origin, 16= Z Shear Center - from origin, 17= Y Shear Center - from centroid, 18= Z Shear Center - from centroid, 19= Y Shear Area, 20= Z Shear Area, 21= YZ Shear Area, 22= Warping Constant, 23= Torsional Constant, 24= Not Used.

Return Code:

`FE_TOO_SMALL`	One of the arrays has dimensions which are too small to be used where you specified it.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
The elements that you select should all be planar elements.	
Example:	
None	

Checks

Unpublished work. © 2025 Siemens

feCheckCoincidentNode

feCheckCoincidentNode

(keepSET, mergeSET, tolerance, merge, messages, coinCount, coinNodes)

Description:

This function checks, and optionally merges, coincident nodes. This method is obsolete and should be replaced by `feCheckCoincidentNode3`.

Input:

`INT4 keepSET`	The ID of a selection set that contains the IDs of the nodes to check for coincidence. Alternatively, if you specify a negative value, this is simply the ID of the single node to check. The nodes in this set are retained if they are merged with a node in the `mergeSET`.
`INT4 mergeSET`	The ID of a selection set that contains the IDs of the nodes to check for coincidence. Alternatively, if you specify a negative value, this is simply the ID of the single node to check. The nodes in this set are removed if they are merged with a node in the `keepSET`. You can specify the same value for both the `keepSET` and the `mergeSET`.
`REAL8 tolerance`	The distance below which nodes will be considered to be coincident.
`BOOL merge`	If True, coincident nodes will be merged.
`BOOL messages`	If True, messages will be printed listing all coincident nodes that are found.

Output:

`INT4 coinCount`	The number of pairs of coincident nodes in `coinNodes`.
`INT4 coinNodes[0..N]`	The IDs of the nodes that are found to be coincident. The entries in this array come in pairs. That is, `coinNodes[0]` and `coinNodes[1]` are coincident with each other, as are `coinNodes[2]` and `coinNodes[3]`. The entries in `coinNodes[0]`, `coinNodes[2]`, `coinNodes[4]`... come from the `keepSET`,

Unpublished work. © 2025 Siemens

	`coinNodes[1]`, `coinNodes[3]`... come from the `mergeSET`. There are `coinCount` pairs.
Return Code:	
`FE_TOO_SMALL`	One of the arrays has dimensions which are too small to be used where you specified it.
`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckCoincidentNode2

feCheckCoincidentNode2

(mergeSET, tolerance, merge, mergeMode, mergeLoc, bMergeAcrossConnections, nMessageMode, bSaveGroups)

Description:

This function checks, and optionally merges, coincident nodes. This method is obsolete and should be replaced by `feCheckCoincidentNode3`.

Input:

`INT4 mergeSET`	The ID of a selection set that contains the IDs of the nodes to check for coincidence.
`REAL8 tolerance`	The distance below which nodes will be considered to be coincident.
`BOOL merge`	If True, coincident nodes will be merged, otherwise just checked for coincidence.
`INT4 mergeMode`	Select Nodes to Keep 0=Automatic, 1=LowerID, 2=HigherID
`INT4 mergeLoc`	Location of merged nodes 0=Original, 1=LowerID, 2=HigherID, 3=Middle
`BOOL bMergeAcrossConnections`	If True, nodes can be merged even though they are on opposite sides of a connection. Otherwise they are not merged.
`INT4 nMessageMode`	Message Mode 0=Off, 1=On, 2=Detailed
`BOOL bSaveGroups`	If True, groups are created of the nodes to be kept, and if `merge=False` of the nodes to be merged.

Output:

None	

Return Code:

`FE_FAIL`	Unable to check for coincident nodes.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feCheckCoincidentNode3

feCheckCoincidentNode3

(mergeSET, tolerance, merge, mergeMode, mergeLoc, bMergeAcrossConnections, bMergeAcrossOutputCSys, nMergeAcrossCSysID, nMessageMode, bSaveGroups)

Description:

This function checks, and optionally merges, coincident nodes. This method supersedes both `feCheck-CoincidentNode` and `feCheckCoincidentNode2`.

Input:

`INT4 mergeSET`	The ID of a selection set that contains the IDs of the nodes to check for coincidence.
`REAL8 tolerance`	The distance below which nodes will be considered to be coincident.
`BOOL merge`	If True, coincident nodes will be merged, otherwise just checked for coincidence.
`INT4 mergeMode`	Select Nodes to Keep 0=Automatic, 1=LowerID, 2=HigherID
`INT4 mergeLoc`	Location of merged nodes 0=Original, 1=LowerID, 2=HigherID, 3=Middle
`BOOL bMergeAcrossConnections`	If True, nodes can be merged even though they are on opposite sides of a connection. Otherwise they are not merged.
`BOOL bMergeAcrossOutputCSys`	If True, nodes can be merged even though they have different output coordinate systems.
`INT4 nMergeAcrossCSysID`	ID of coordinate system to use for nodes merged which had different output coordinate systems before merging. Set to -1 to use the output coordinate system of the node that was *kept* after the merge.
`INT4 nMessageMode`	Message Mode 0=Off, 1=On, 2=Detailed
`BOOL bSaveGroups`	If True, groups are created of the nodes to be kept, and if merge=False of the nodes to be merged.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Unable to check for coincident nodes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckCoincidentNode4

feCheckCoincidentNode4

(mergeSET, bMergeIntoAll, tolerance, merge, mergeMode, mergeLoc, bMergeAcrossConnections, bMergeAcrossOutputCSys, nMergeAcrossCSysID, nMessageMode, bSaveGroups)

Description:

This function checks, and optionally merges, coincident nodes. This method supersedes both `feCheck-CoincidentNode`, `feCheckCoincidentNode2` and `feCheckCoincidentNode3`.

Input:

`INT4 mergeSET`	The ID of a selection set that contains the IDs of the nodes to check for coincidence.
`BOOL bMergeIntoAll`	If True, all nodes in mergeSet will be checked against all other coincident nodes whether also in `mergeSet` or not. If False, only nodes within `mergeSet` will be checked for coincidence against other nodes in `mergeSet`.
`REAL8 tolerance`	The distance below which nodes will be considered to be coincident.
`BOOL merge`	If True, coincident nodes will be merged, otherwise just checked for coincidence
`INT4 mergeMode`	Select Nodes to Keep 0=Automatic, 1=LowerID, 2=HigherID
`INT4 mergeLoc`	Location of merged nodes 0=Original, 1=LowerID, 2=HigherID, 3=Middle
`BOOL bMergeAcrossConnections`	If True, nodes can be merged even though they are on opposite sides of a connection. Otherwise they are not merged.
`BOOL bMergeAcrossOutputCSys`	If True, nodes can be merged even though they have different output coordinate systems.
`INT4 nMergeAcrossCSysID`	ID of coordinate system to use for nodes merged which had different output coordinate systems before merging. Set to -1 to use the output coordinate system of the node that was *kept* after the merge.

Unpublished work. © 2025 Siemens

`INT4 nMessageMode`	Message Mode 0=Off, 1=On, 2=Detailed
`BOOL bSaveGroups`	If True, groups are created of the nodes to be kept, and if `merge=False` of the nodes to be merged.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to check for coincident nodes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckCoincidentNode5

feCheckCoincidentNode5

(mergeSET, bMergeIntoAll, tolerance, max_midside_angle_distortion, merge, mergeMode, mergeLoc, bMergeAcrossConnections, bMergeAcrossOutputCSys, nMergeAcrossCSysID, nMessageMode, bSaveGroups)

Description:

This function checks, and optionally merges, coincident nodes. This method supersedes both `feCheckCoincidentNode`, `feCheckCoincidentNode2` and `feCheckCoincidentNode3`.

Input:

`INT4 mergeSET`	The ID of a selection set that contains the IDs of the nodes to check for coincidence.
`BOOL bMergeIntoAll`	If True, all nodes in `mergeSet` will be checked against all other coincident nodes whether also in `mergeSet` or not. If False, only nodes within `mergeSet` will be checked for coincidence against other nodes in `mergeSet`.
`REAL8 tolerance`	The distance below which nodes will be considered to be coincident.
`REAL8 max_midside_angle_distortion`	The distance below which nodes will be considered to be coincident.
`BOOL merge`	If True, coincident nodes will be merged, otherwise just checked for coincidence
`INT4 mergeMode`	Select Nodes to Keep 0=Automatic, 1=LowerID, 2=HigherID
`INT4 mergeLoc`	Location of merged nodes 0=Original, 1=LowerID, 2=HigherID, 3=Middle
`BOOL bMergeAcrossConnections`	If True, nodes can be merged even though they are on opposite sides of a connection. Otherwise they are not merged.
`BOOL bMergeAcrossOutputCSys`	If True, nodes can be merged even though they have different output coordinate systems.
`INT4 nMergeAcrossCSysID`	ID of coordinate system to use for nodes merged which had different output coordinate systems before merging. Set to -1 to use the output

Unpublished work. © 2025 Siemens

	coordinate system of the node that was *kept* after the merge.
`INT4 nMessageMode`	Message Mode 0=Off, 1=On, 2=Detailed
`BOOL bSaveGroups`	If True, groups are created of the nodes to be kept, and if `merge=False` of the nodes to be merged.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to check for coincident nodes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckCoincidentPoint

feCheckCoincidentPoint **(keepSET, mergeSET, tolerance, merge, messages, coinCount, coinPoints)**	
Description:	
This function checks, and optionally merges, coincident points.	
Input:	
`INT4 keepSET`	The ID of a selection set that contains the IDs of the points to check for coincidence. Alternatively, if you specify a negative value, this is simply the ID of the single point to check. The points in this set are retained if they are merged with a point in the `mergeSET`.
`INT4 mergeSET`	The ID of a selection set that contains the IDs of the points to check for coincidence. Alternatively, if you specify a negative value, this is simply the ID of the single point to check. The points in this set are removed if they are merged with a point in the `keepSET`. You can specify the same value for both the `keepSET` and the `mergeSET`.
`REAL8 tolerance`	The distance below which points will be considered to be coincident.
`BOOL merge`	If True, coincident points will be merged.
`BOOL messages`	If True, messages will be printed listing all coincident points that are found.
Output:	
`INT4 coinCount`	The number of pairs of coincident points in `coinPoints`.
`INT4 coinPoints[0..N]`	The IDs of the points that are found to be coincident. The entries in this array come in pairs. That is, `coinPoints[0]` and `coinPoints[1]` are coincident with each other, as are `coinPoints[2]` and `coinPoints[3]`. The entries in `coinPoints[0]`, `coinPoints[2]`, `coinPoints[4]`... come from the `keepSET`,

Unpublished work. © 2025 Siemens

	`coinPoints[1]`, `coinPoints[3]`... come from the `mergeSET`. There are `coinCount` pairs.
Return Code:	
`FE_TOO_SMALL`	One of the arrays has dimensions which are too small to be used where you specified it.
`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckCoincidentCurve

feCheckCoincidentCurve **(curveSET, tolerance)**	
Description:	
This function checks and merges coincident curves.	
Input:	
`INT4 curveSET`	The ID of a selection set that contains the IDs of the curves to check for coincidence. Alternatively, if you specify a negative value, this is simply the ID of the single curve to check.
`REAL8 tolerance`	The distance below which curves (and their endpoints) will be considered to be coincident.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckPlanarNodes

feCheckPlanarNodes **(nodeSET, messages, project, tolerance, askPlane, planeBase, planeNormal)**	
Description:	
This function checks a set of nodes to see if they are planar. The nodes can also be projected onto the plane.	
Input:	
`INT4 nodeSET`	The ID of a selection set that contains the IDs of the nodes to check. Alternatively, if you specify a negative value, this is simply the ID of the single node to check.
`BOOL messages`	If True, status messages are written showing the nodes that are not on the plane.
`BOOL project`	If True, nodes that are not in the plane but are less than the tolerance distance out of the plane will be projected onto the plane. Nodes that are greater than the tolerance distance away from the plane will remain in their current location.
`REAL8 tolerance`	The maximum distance from the plane for nodes to be moved onto the plane. Only used if `project=True`.
`BOOL askPlane`	If True, a dialog box will be displayed asking the user to pick a plane. If False, the `planeBase` and `planeNormal` will be used to specify the plane location.
`REAL8 planeBase[0..2]`	A location (in global rectangular coordinates) on the plane.
`REAL8 planeNormal[0..2]`	A vector that is normal to the plane.
Output:	
None	
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

feCheckCoincidentElem

feCheckCoincidentElem

(elemSET, chkTYPE, chkSHAPE, chkMASS, doLIST, makeGROUP1, makeGROUP2)

Description:

This function checks a set of elements to see if any of them are coincident (share the same nodes).

Input:

`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check.
`BOOL chkTYPE`	If True, elements of different type can still be considered coincident.
`BOOL chkSHAPE`	If True, elements of different shape can be considered coincident if all of the nodes of one of the elements are on the other element.
`BOOL chkMASS`	If True, mass elements are included in the check.
`BOOL doLIST`	If True, coincident elements are listed in the Messages and Lists window.
`BOOL makeGROUP1`	If True, a group is made of the first element in the coincident pair.
`BOOL makeGROUP2`	If True, a group is made of the second element in the coincident pair.

Output:

None	

Return Code:

None	

Remarks/Usage:

This function only finds coincident elements that share the same nodes - not ones that may be coincident in physical location, but still have different nodes.

Example:

None

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feCheckCoincidentElemQuick

feCheckCoincidentElemQuick

(elemSET, chkTYPE, chkSHAPE, chkMASS, doLIST, makeGROUP1, makeGROUP2)

Description:

Mimics the API function **feCheckCoincidentElem**. This *Quick* version (`feCheckCoincidentElemQuick`) corresponds to the user interface option for **Quick** being turned on (`feCheckCoincidentElem` goes through the full check).

Input:

`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check.
`BOOL chkTYPE`	If True, elements of different type can still be considered coincident.
`BOOL chkSHAPE`	If True, elements of different shape can be considered coincident if all of the nodes of one of the elements are on the other element.
`BOOL chkMASS`	If True, mass elements are included in the check.
`BOOL doLIST`	If True, coincident elements are listed in the Messages and Lists window.
`BOOL makeGROUP1`	If True, a group is made of the first element in the coincident pair.
`BOOL makeGROUP2`	If True, a group is made of the second element in the coincident pair.

Output:

None

Return Code:

None

Remarks/Usage:

This function only finds coincident elements that share the same nodes - not ones that may be coincident in physical location, but still have different nodes.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feCheckElemFixup

feCheckElemFixup **(elemSET)**	
Description:	
This function checks and attempts to fix a set of elements to see if any of them are invalid.	
Input:	
`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This function is just like the fixup option in the **Tools** → **Check** → **Distortion** command. Checks include zero length, twisted faces, inverted solids, and element/property mismatch.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckElemDistortion

feCheckElemDistortion

**(elemSET, doLIST, makeGROUP, chkASPECT, chkTAPER, chkANGLE, chkWARP, chkALTTAPER, chkTET, chkJDet, chkComb,
maxASPECT, maxTAPER, maxANGLE, maxWARP, maxALTTAPER, maxTET, maxJDet, maxComb)**

Description:

This function checks a set of elements to see if any of them are distorted.

Input:

`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check.
`BOOL doLIST`	If True, distorted elements are listed in the Messages and Lists window.
`BOOL makeGROUP`	If True, a group is made of the distorted elements.
`BOOL chkASPECT`	If True, the aspect ratio check is performed.
`BOOL chkTAPER`	If True, element taper is checked.
`BOOL chkANGLE`	If True, the element internal angle check is performed.
`BOOL chkWARP`	If True, the element warping check is performed.
`BOOL chkALTTAPER`	If True, the alternate element taper check is performed.
`BOOL chkTET`	If True, tetrahedral elements are checked for collapse.
`BOOL chkJDet`	If True, checks element Jacobian.
`BOOL chkComb`	If True, checks for combined element distortion factor.
`REAL8 maxASPECT`	Limit for aspect ratio check.

Unpublished work. © 2025 Siemens

`REAL8 maxTAPER`	Limit for taper check.
`REAL8 maxANGLE`	Limit for internal angle check.
`REAL8 maxWARP`	Limit for warping check.
`REAL8 maxALTTAPER`	Limit for alternate element taper check.
`REAL8 maxTET`	Limit for tet collapse check.
`REAL8 maxJDet`	Limit for Jacobian check.
`REAL8 maxComb`	Limit for Combined Element Distortion check.
Output:	
None	
Return Code:	
`FE_INVALID`	Data in the limit values is outside the acceptable bounds.
Remarks/Usage:	
This function is just like the **Tools** → **Check** → **Distortion** command. The checks performed and the limit values are identical to that command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckElemDistortion2

feCheckElemDistortion2

**(elemSET, doLIST, makeGROUP, chkASPECT, chkTAPER, chkANGLE, chkWARP, chkALTTAPER, chkTET,
chkJDet, chkComb, chkExpTime, maxASPECT, maxTAPER, maxANGLE, maxWARP, maxALTTAPER, maxTET, maxJDet, maxComb, maxExpTime)**

Description:

This function checks a set of elements to see if any of them are distorted.

Input:

INT4 elemSET	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check.
BOOL doLIST	If True, distorted elements are listed in the Messages and Lists window.
BOOL makeGROUP	If True, a group is made of the distorted elements.
BOOL chkASPECT	If True, the aspect ratio check is performed.
BOOL chkTAPER	If True, element taper is checked.
BOOL chkANGLE	If True, the element internal angle check is performed.
BOOL chkWARP	If True, the element warping check is performed.
BOOL chkALTTAPER	If True, the alternate element taper check is performed.
BOOL chkTET	If True, tetrahedral elements are checked for collapse.
BOOL chkJDet	If True, checks element Jacobian.
BOOL chkComb	If True, checks for combined element distortion factor.
BOOL chkExpTime	If True, checks for Explicit Time Step factors.

Unpublished work. © 2025 Siemens

`REAL8 maxASPECT`	Limit for aspect ratio check.
`REAL8 maxTAPER`	Limit for taper check.
`REAL8 maxANGLE`	Limit for internal angle check.
`REAL8 maxWARP`	Limit for warping check.
`REAL8 maxALTTAPER`	Limit for alternate element taper check.
`REAL8 maxTET`	Limit for tet collapse check.
`REAL8 maxJDet`	Limit for Jacobian check.
`REAL8 maxComb`	Limit for Combined Element Distortion check.
`REAL8 maxExpTime`	Limit for Explicit Time Step check.
Output:	
None	
Return Code:	
`FE_INVALID`	Data in the limit values is outside the acceptable bounds.
Remarks/Usage:	
This function is just like the **Tools → Check → Distortion** command. The checks performed and the limit values are identical to that command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGetElemDistortion

feGetElemDistortion

(elemID, valASPECT, valTAPER, valANGLE, valWARP, valNasWARP, valALTTAPER, valTET, valJDet, valComb)

Description:

This function checks an element and returns the distortion values.

Input:

`INT4 elemID`	The ID of the element to check for distortion.

Output:

`REAL8 valASPECT`	Result of aspect ratio check.
`REAL8 valTAPER`	Result of taper check.
`REAL8 valANGLE`	Result of internal angle check.
`REAL8 valWARP`	Result of warping check.
`REAL8 valNasWARP`	Result of Nastran warping check.
`REAL8 valALTTAPER`	Result of alternate element taper check.
`REAL8 valTET`	Result of tet collapse check.
`REAL8 valJDet`	Result of Jacobian check.
`REAL8 valComb`	Results of Combined Quality check.

Return Code:

None	

Remarks/Usage:

This function is just like the **Tools** → **Check** → **Distortion** command. The checks performed and the returned values are identical to that command.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feGetElemDistortion2

feGetElemDistortion2

(elemID, valASPECT, valTAPER, valANGLE, valWARP, valNasWARP, valALTTAPER, valTET, valJDet, valComb, valExpTime)

Description:

This function checks an element and returns the distortion values.

Input:

`INT4 elemID`	The ID of the element to check for distortion.

Output:

`REAL8 valASPECT`	Result of aspect ratio check.
`REAL8 valTAPER`	Result of taper check.
`REAL8 valANGLE`	Result of internal angle check.
`REAL8 valWARP`	Result of warping check.
`REAL8 valNasWARP`	Result of Nastran warping check.
`REAL8 valALTTAPER`	Result of alternate element taper check.
`REAL8 valTET`	Result of tet collapse check.
`REAL8 valJDet`	Result of Jacobian check.
`REAL8 valComb`	Results of Combined Quality check.
`REAL8 valExpTime`	Results of Explicit Time Step check.

Return Code:

None	

Remarks/Usage:

Unpublished work. © 2025 Siemens

This function is just like the **Tools** → **Check** → **Distortion** command. The checks performed and the returned values are identical to that command.
Example:
None

Unpublished work. © 2025 Siemens

feCheckElemNormal

feCheckElemNormal

(elemSET, csysID, doLIST, swapNORMAL, autoNORMAL, manNORMAL)

Description:

This function checks and optionally reverses element normals.

Input:

`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check.
`INT4 csysID`	The coordinate system in which the check is done. If you are specifying `manNORMAL`, it must be in this coordinate system.
`BOOL doLIST`	If True, swapped elements are listed in the Messages and Lists window.
`BOOL swapNORMAL`	If True, elements with normals that are opposite to the desired normal are reversed.
`BOOL autoNORMAL`	If True, the desired normal direction is automatically determined from the first selected element. If False, the normal must be specified in `manNORMAL`.
`REAL8 manNORMAL[0..2]`	A unit vector, with coordinates in CSys *csysID*, in the direction of the desired normal. Only used if `autoNORMAL=False`.

Output:

None	

Return Code:

`FE_TOO_SMALL`	One of the arrays has dimensions which are too small to be used where you specified it.
`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.

Unpublished work. © 2025 Siemens

FE_BAD_TYPE	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
Only works with planar elements.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckConstraints

feCheckConstraints **(doPERM, doLIST, bcSUM, bcSEP)**	
Description:	
This function checks constraints in the active constraint set.	
Input:	
`BOOL doPERM`	If True, permanent nodal constraints are included in the check.
`BOOL doLIST`	If True, a summary report is listed.
Output:	
`REAL8 bcSUM[0..5]`	The sum of the constraint values in each of the six degrees of freedom.
`REAL8 bcSUM[0..8]`	The max separation of constraints. [0],[1],[2] are the max XYZ separation of the X constraints. [3],[4],[5] are the XYZ separation of the Y constraints, [6],[7],[8] are the XYZ separation of the Z constraints.
Return Code:	
`FE_TOO_SMALL`	One of the arrays has dimensions which are too small to be used where you specified it.
`FE_NOT_AVAILABLE`	One of the arrays does not contain the required data.
`FE_INVALID`	Data in the input arrays is outside the acceptable bounds.
`FE_BAD_TYPE`	Data in the input arrays is not of the correct type (e.g. REAL8 or INT4).
Remarks/Usage:	
Automatically uses whichever constraint set is active.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckCoincidentLoads

feCheckCoincidentLoads

(loadSET, mergeLOADS)

Description:

This function checks loads from a group of load sets to see if similar loads are applied to the same geometry or mesh.

Input:

`INT4 loadSET`	The ID of a selection set that contains the IDs of the load sets to check. Alternatively, if you specify a negative value, this is simply the ID of the single load set to check.
`BOOL mergeLOADS`	If True and coincident loads are found, an attempt is made to merge/combine them.

Output:

None	

Return Code:

None	

Remarks/Usage:

Unlike the **Tools** → **Check** → **Coincident Loads** command, this command does not give you the opportunity to selectively merge individual types of loads - either all types, or no types, are merged.

Example:

None

Unpublished work. © 2025 Siemens

feCheckSumForces

feCheckSumForces

(expandGEOM, doLIST, useSETS, nodeSET, elemSET, basePOINT, csysID, summed FORCES)

Description:

This function checks loads from the active load set and sums the forces around a specific location. This function is obsolete and has been superseded by `feCheckSumForces2`.

Input:

`BOOL expandGEOM`	If True, geometric loads are expanded before the check. If not, nonexpanded geometric loads will be ignored.
`BOOL doLIST`	If True, a summary report is written.
`BOOL useSETS`	If True, `nodeSET` and `elemSET` must specify the nodes and elements to consider in the summation. If False, the full model is used.
`INT4 nodeSET`	The ID of a selection set that contains the IDs of the nodes to check. Alternatively, if you specify a negative value, this is simply the ID of the single node to check. Only used if `useSETS=True`.
`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check. Only used if `useSETS=True`.
`REAL8 basePOINT[0..2]`	The coordinates of the location to sum about. Must be in global rectangular coordinates.
`INT4 csysID`	The coordinate system used to report and returned the summed loads.

Output:

`REAL8 summedFORCES[0..26]`	The results of the force summation. Entries [0..5] are the nodal force summations, [6..8] contain the elemental force summation, [9..14] contain the total summation, [15-17] contain body translational acceleration, [18-20] contain body varying translational acceleration, [21-23] contain

Unpublished work. © 2025 Siemens

	body rotational acceleration, [24-26] contain body rotational velocity.
Return Code:	
None	
Remarks/Usage:	
Uses the loads in the active load set.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckSumForces2

feCheckSumForces2

(useBodyLoad, expandGEOM, doLIST, useSETS, nodeSET, elemSET, loaddefSET, basePOINT, csysID, summedFORCES)

Description:

This function checks loads from the active load set and sums the forces around a specific location.

Input:

`BOOL useBodyLoad`	If True, body loads will be included in the calculation. If not, body loads will be ignored
`BOOL expandGEOM`	If True, geometric loads are expanded before the check. If not, nonexpanded geometric loads will be ignored.
`BOOL doLIST`	If True, a summary report is written.
`BOOL useSETS`	If True, `nodeSET` and `elemSET` OR `loaddefSET` must specify the nodes and elements OR load definitions to consider in the summation. To use `nodeSET` and `elemSET`, set `loaddefSET = 0`, otherwise, `loaddefSET` will be used. If False, the full model is used.
`INT4 nodeSET`	The ID of a selection set that contains the IDs of the nodes to check. Alternatively, if you specify a negative value, this is simply the ID of the single node to check. Only used if `useSETS=True` and `loaddefSET = 0`
`INT4 elemSET`	The ID of a selection set that contains the IDs of the elements to check. Alternatively, if you specify a negative value, this is simply the ID of the single element to check. Only used if `useSETS=True` and `loaddefSET = 0`.
`INT4 loaddefSET`	The ID of a selection set that contains the IDs of the load definitions to check. Alternatively, if you specify a negative value, this is simply the ID of the single load definition. Only used if `useSETS=True`.
`REAL8 basePOINT[0..2]`	The coordinates of the location to sum about. Must be in global rectangular coordinates.

Unpublished work. © 2025 Siemens

`INT4 csysID`	The coordinate system used to report and returned the summed loads.
Output:	
`REAL8 summedFORCES[0..26]`	The results of the force summation. Entries [0..5] are the nodal force summations. [6..8] contain the elemental force summation. [9..14] contain the total summation. [15-17] contain body translational acceleration. [18-20] contain body varying translational acceleration. [21-23] contain body rotational acceleration. [24-26] contain body rotational velocity.
Return Code:	
None	
Remarks/Usage:	
Uses the loads in the active load set.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckElemInterference

feCheckElemInterference **(nSetID, bFreeFaceSolids, bMakeGroup)**	
Description:	
This function checks the interferences of a give set of elements.	
Input:	
`INT4 nSetID`	The ID of the set of elements to check.
`BOOL bFreeFaceSolids`	If True, limits to free faces of solid elements. If False, all faces are checked.
`BOOL bMakeGroup`	If True, a group will be made with the elements that interfere.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCheckElemInterference2

feCheckElemInterference2 **(elSetID, bFreeFaceSolids, bMakeGroup, bIntFactor, dTol)**	
Description:	
This function checks the interferences of a give set of elements.	
Input:	
`INT4 elSetID`	The ID of the set of elements to check.
`BOOL bFreeFaceSolids`	If True, limits to free faces of solid elements. If False, all faces are checked.
`BOOL bMakeGroup`	If True, a group will be made with the elements that interfere.
`BOOL bIntFactor`	If True, use Interference Factor. If False, use Proximity Factor.
`REAL8 dTol`	Specifies value for Interference Factor/Proximity Factor.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Charting Pane

Unpublished work. © 2025 Siemens

feChartPaneGetDisplayed

feChartPaneGetDisplayed **(void)**	
Description:	
This function gets the ID of the chart object displayed in the charting pane.	
Input:	
None	
Output:	
None	
Return Code:	
`INT4`	Returns a integer value corresponding to the displayed chart >= 1: Chart ID 0: No active chart -1: Chart pane is not visible
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feChartPaneSetDisplayed

feChartPaneSetDisplayed **(nChartID)**	
Description:	
This function sets the displayed chart in the charting pane.	
Input:	
`INT4 nChartID`	ID of the chart object to display in the charting pane.
Output:	
None	
Return Code:	
`FE_FAIL`	Charting pane is not displayed.
`FE_NOT_EXIST`	Provided chart ID does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feChartPaneCopy

feChartPaneCopy **(bCopyPicture , bCopyText)**	
Description:	
This function will send a copy of the image currently shown in the charting pane and/or text of the data currently being displayed in the charting pane to the clipboard.	
Input:	
`BOOL bCopyPicture`	If True, a copy of the image currently displayed in the **Charting Pane** will be sent to the clipboard.
`BOOL bCopyText`	If True, text representing the data currently being displayed in the **Charting Pane** will be sent to the clipboard.
Output:	
None	
Return Code:	
`FE_FAIL`	**Charting Pane** is not displayed.
Remarks/Usage:	
If both the image and the text are sent to the clipboard at the same time with this call, only the text will be available to paste into some applications, while other applications will give the user a choice to paste either the image or the text.	
Example:	
None	

Geometry Methods

Lines

Unpublished work. © 2025 Siemens

feLinePoints

feLinePoints **(ontoWorkplane, p1, p2, messages)**	
Description:	
Creates a line between two coordinate locations.	
Input:	
`BOOL ontoWorkplane`	If True, the coordinates specified are projected onto the workplane before the line is created.
`REAL8 p1[0..2]`	Coordinates at the first end of the line.
`REAL8 p2[0..2]`	Coordinates at the second end of the line.
`BOOL messages`	If True, messages are written to the Message and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLineHorzVert

feLineHorzVert **(horizontal, p1, length, messages)**	
Description:	
Creates a horizontal or vertical line in the workplane.	
Input:	
`BOOL horizontal`	If True, the line will be horizontal (workplane X axis). If False, the line will be vertical (workplane Y axis).
`REAL8 p1[0..2]`	Coordinates that will be projected onto the workplane at the center of the line.
`REAL8 length`	The overall length of the line.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLineAtAngle

feLineAtAngle **(angle, curveID, origin, side, messages)**	
Description:	
Creates a line at an angle to the workplane, or another curve.	
Input:	
`REAL8 angle`	The angle from the workplane X axis, or the selected curve.
`INT4 curveID`	The ID of the curve used to measure the angle. Set to 0 if you want to specify the angle from the workplane X axis (toward the Y axis).
`REAL8 origin[0..2]`	The coordinates at the start of the line. These are projected onto the curve to find location where the tangent vector is determined and at which the angle is measured.
`REAL8 side[0..2]`	A location on the side of the curve where the line will be drawn toward. Not used if you are measuring from the workplane X axis. The line does not necessarily pass through this point; it is just used to determine which direction to go from the origin location.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line - probably the curve does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLineParallel

feLineParallel **(offset, curveID, side, messages)**	
Description:	
Creates a line parallel to another in the workplane.	
Input:	
`REAL8 offset`	The perpendicular distance from the selected curve used to locate the new curve.
`INT4 curveID`	The ID of the reference curve.
`REAL8 side[0..2]`	A location on the side of the curve where the line will be drawn. The line does not necessarily pass through this point; it is just used to determine which direction to go from the reference curve location.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line - either the curve does not exist or it is not of the correct type.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLineMidline

feLineMidline **(curve1, curve2, messages)**	
Description:	
Creates a line that is midway between two other lines.	
Input:	
`INT4 curve1`	The two reference curves used to determine the midline location.
`INT4 curve2`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line - either one of the curves does not exist or is not of the correct type.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLinePointTangent

feLinePointTangent **(curveID, origin, side, messages)**	
Description:	
Creates a line through an origin that is tangent to a selected curve.	
Input:	
`INT4 curveID`	The ID of the curve that the line will be tangent to.
`REAL8 origin[0..2]`	The location at the start of the line. The line will pass through this location and be tangent to the curve.
`REAL8 side[0..2]`	A location on the side of the curve where the line will be tangent. The line does not normally pass through this point; it is just used to determine which tangent to select.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line - either the curve does not exist or there is no tangent that could be computed.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLine2Tangent

This is a para in the topic template.

feLine2Tangent **(curve1, curve2, side, messages)**	
Description:	
Creates a line that is tangent to two selected curves.	
Input:	
`INT4 curve1`	The IDs of the curves that the line will be tangent to.
`INT4 curve2`	
`REAL8 side[0..2]`	A location on the side of the first curve where the line will be tangent. The line does not normally pass through this point; it is just used to determine which tangent to select.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the line - either one of the curves does not exist or there is no tangent that could be computed.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLineRectangle

feLineRectangle **(p1, p2, messages)**	
Description:	
Creates four lines that form a rectangle in the workplane between two diagonal locations.	
Input:	
`REAL8 p1[0..2]`	The diagonal locations that define the corners of the rectangle. These locations are projected onto the workplane.
`REAL8 p2[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feLinePointIDs

feLinePointIDs **(p1, p2, messages)**	
Description:	
Creates a line that joins two points.	
Input:	
`INT4 p1`	The ID of the point at the start of the line.
`INT4 p2`	The ID of the point at the end of the line.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the line - probably one of the points does not exist.
Remarks/Usage:	
None	
Example:	
None	

Arcs

Unpublished work. © 2025 Siemens

feArcCenterStartEnd

feArcCenterStartEnd **(center, start, end, messages)**	
Description:	
Creates an arc in the workplane by specifying the center of the arc, a starting location, and an ending location.	
Input:	
`REAL8 center[0..2]`	The locations used to define the arc. The center is at the center of the arc. start is a point on the perimeter at the beginning of the arc, and defines the radius. end is not necessarily on the arc, but specifies the ending location.
`REAL8 start[0..2]`	
`REAL8 end[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcRadiusStartEnd

feArcRadiusStartEnd **(radius, start, end, messages)**	
Description:	
Creates an arc in the workplane by specifying the radius of the arc, a starting location and an ending location.	
Input:	
`REAL8 radius`	The radius of the arc.
`REAL8 start[0..2]`	The locations used to define the arc. start is a point on the perimeter at the beginning of the arc, end is on the perimeter at the end of the arc.
`REAL8 end[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc - radius could be too small for locations specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcAngleStartEnd

feArcAngleStartEnd **(angle, start, end, messages)**	
Description:	
Creates an arc in the workplane by specifying the included angle of the arc, a starting location, and an ending location.	
Input:	
`REAL8 angle`	The included angle swept out by the arc.
`REAL8 start[0..2]`	The locations used to define the arc. start is a point on the perimeter at the beginning of the arc, end is on the perimeter at the end of the arc.
`REAL8 end[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc - angle may not be valid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcAngleCenterStart

feArcAngleCenterStart **(angle, center, start, messages)**	
Description:	
Creates an arc in the workplane by specifying the included angle of the arc, a location at the center, and a location on the perimeter.	
Input:	
`REAL8 angle`	The included angle swept out by the arc.
`REAL8 center[0..2]`	The locations used to define the arc. center is at the center of the arc. start is a point on the perimeter and is used with center to determine the radius.
`REAL8 start[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc - angle may not be valid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcChordCenterStart

feArcChordCenterStart **(length, center, start, messages)**	
Description:	
Creates an arc in the workplane by specifying the chord length of the arc, a location at the center, and a location on the perimeter.	
Input:	
`REAL8 length`	The chord length of the arc, used to determine the included angle.
`REAL8 center[0..2]`	The locations used to define the arc. center is at the center of the arc. start is a point on the perimeter and is used with center to determine the radius.
`REAL8 start[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcPoints

feArcPoints **(start, mid, end, messages)**	
Description:	
Creates a 3D arc in space, through the three points.	
Input:	
`REAL8 start[0..2]`	The three points on the arc. The arc goes from start, toward mid and stops at end.
`REAL8 mid[0..2]`	
`REAL8 end[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc - points may be colinear.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcCenterPoints

feArcCenterPoints **(center, start, mid, end, messages)**	
Description:	
Creates a 3D arc in space, about a center and defined by three additional points.	
Input:	
`REAL8 center[0..2]`	The location at the center of the arc.
`REAL8 start[0..2]`	The location at the start of the arc. This, along with center, determine the radius of the arc.
`REAL8 mid[0..2]`	A location used to determine the direction of the arc from start - it does not necessarily lie on the arc.
`REAL8 end[0..2]`	A location used to determine the ending angle of the arc - it does not necessarily lie on the arc.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc - points may be colinear or invalid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feArcTangentStartEnd

feArcTangentStartEnd **(tangent, start, end, messages)**	
Description:	
Creates a 3D arc in space, from a point with a specified starting tangent.	
Input:	
`REAL8 tangent[0..2]`	A vector in the direction of the starting tangent of the arc.
`REAL8 start[0..2]`	The location at the start of the arc.
`REAL8 end[0..2]`	The location at the end of the arc.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the arc.
Remarks/Usage:	
None	
Example:	
None	

Circles

Unpublished work. © 2025 Siemens

feCircleRadius

feCircleRadius **(center, start, messages)**	
Description:	
Creates a circle in the workplane defined by a location at the center and one on the perimeter.	
Input:	
`REAL8 center[0..2]`	The location at the center of the circle.
`REAL8 start[0..2]`	A location on the perimeter of the circle.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircleDiameter

feCircleDiameter **(dia1, dia2, messages)**	
Description:	
Creates a circle in the workplane defined by two locations at the opposite ends of a diameter.	
Input:	
`REAL8 dia1[0..2]`	The locations at the opposite ends of the diameter of the circle.
`REAL8 dia2[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircleCenter

feCircleCenter **(radius, center, messages)**	
Description:	
Creates a circle in the workplane defined a center location and the radius.	
Input:	
`REAL8 radius`	The radius of the circle.s
`REAL8 center[0..2]`	The location of the center of the circle.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircle2Point

feCircle2Point **(radius, pt1, pt2, messages)**	
Description:	
Creates a circle in the workplane defined by two points on the circle, and the radius.	
Input:	
`REAL8 radius`	The radius of the circle. Specify a positive number to go in a counterclockwise direction from `pt1` to `pt2`. Specify a negative number to go in a clockwise direction. This is used to pick which of the two possible circles through the two points that you are trying to define.
`REAL8 pt1[0..2]`	The locations on the circle.
`REAL8 pt2[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircleConcentric

feCircleConcentric **(radius, curveID, messages)**	
Description:	
Creates a circle in the workplane, with a specified radius, that is concentric with another arc or circle.	
Input:	
`REAL8 radius`	The radius of the circle.
`INT4 curveID`	The ID of another arc or circle - used to define the center.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircle3Point

feCircle3Point **(pt1, pt2, pt3, messages)**	
Description:	
Creates a circle, in 3D space, through three points.	
Input:	
`REAL8 pt1[0..2]`	The locations on the circle.
`REAL8 pt2[0..2]`	
`REAL8 pt3[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircleCenterPoints

feCircleCenterPoints **(center, pt1, pt2, messages)**	
Description:	
Creates a circle, in 3D space, using a center location, and specified by two other points.	
Input:	
`REAL8 center[0..2]`	The center of the circle.
`REAL8 pt1[0..2]`	Two additional points on the circle. pt1 is used to define the radius of the circle. Both points (and the center) are used to define the plane of the circle.
`REAL8 pt2[0..2]`	
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircleCenterTangent

feCircleCenterTangent **(curveID, center, messages)**	
Description:	
Creates a circle in the workplane that is tangent to a selected curve and uses the specified center.	
Input:	
`INT4 curveID`	ID of the curve that this circle will be tangent to.
`REAL8 center[0..2]`	The center of the circle.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCircleTangentTangent

feCircleTangentTangent **(curve1, curve2, radius, nearloc, messages)**	
Description:	
Creates a circle in the workplane that is tangent to two other curves, with a specified radius.	
Input:	
`INT4 curve1`	IDs of the curves that this circle will be tangent to.
`INT4 curve2`	
`REAL8 radius`	The radius of the circle being created.
`REAL8 nearloc[0..2]`	A location near the tangency. This is used to select which tangency to use when there are several possible circles that meet the tangency and radius criteria.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the circle.
Remarks/Usage:	
None	
Example:	
None	

Splines

Unpublished work. © 2025 Siemens

feSplinePoints

feSplinePoints

(project, controlpoints, count, xyz, messages)

Description:

Creates a spline using the specified points.

Input:

`BOOL project`	Set to True if you want to project the xyz coordinates onto the workplane before creating the spline.
`BOOL controlpoints`	Set to True if xys contains coordinates of the spline control points. Set to False if it contains coordinates of points that actually lie on the spline.
`INT4 count`	The number of coordinates specified in xyz. Count must be at least 4, and can be no larger than 109.
`REAL8 xyz[0..3*count-1]`	The coordinates of the locations used to define the spline. This array contains 3 * count entries stored as: x1,y1,z1,x2,y2,z2...xN,yN,zN.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.

Output:

None	

Return Code:

`FE_FAIL`	Unable to create the spline.

Remarks/Usage:

None

Example:

This is a sample program to create a spline through 6 points.

```
Sub Main

Dim App As femap.model
Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
Dim x(18) As Double
Dim v As Variant
Dim rc As Integer

x(0) = 0#
x(1) = 0#
x(2) = 0#
x(3) = 1#
x(4) = 0#
x(5) = 0
x(6) = 1
x(7) = 1
x(8) = 0
x(9) = 2
x(10) = 1
x(11) = 0
x(12) = 2
x(13) = 0
x(14) = 0
x(15) = 2.5
x(16) = 1
x(17) = 1
v = x

rc = App.feSplinePoints(False, False, 6, v, True)
App.feViewRegenerate(0)


End Sub
```


Unpublished work. © 2025 Siemens

feSplineTangent

feSplineTangent **(x1, v1, x2, v2, messages)**	
Description:	
Creates a spline using the tangent vectors at each end.	
Input:	
`REAL8 x1[0..2]`	The coordinates of the start of the spline.
`REAL8 v1[0..2]`	The components of the starting tangent of the spline.
`REAL8 x2[0..2]`	The coordinates of the ending location of the spline.
`REAL8 v2[0..2]`	The components of the ending tangent of the spline.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the spline.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSplineEllipse

feSplineEllipse **(center, vector, a, b, messages)**	
Description:	
Creates splines in the workplane that form an ellipse.	
Input:	
`REAL8 center[0..2]`	The coordinates at the center of the ellipse.
`REAL8 vector[0..2]`	The components of the vector along one axis of the ellipse.
`REAL8 a`	The radius of the ellipse along the vector.
`REAL8 b`	The radius of the ellipse perpendicular to the vector.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the spline.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSplineParabola

feSplineParabola **(center, focus, trim, messages)**	
Description:	
Creates a spline in the workplane in the shape of a parabola.	
Input:	
`REAL8 center[0..2]`	The coordinates at the center of the parabola.
`REAL8 focus[0..2]`	The coordinates of the focus of the parabola.
`REAL8 trim[0..2]`	The coordinates used to trim the outer end of the parabola.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the spline.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSplineHyperbola

feSplineHyperbola **(center, towardfocus, angle, height, trim, messages)**	
Description:	
Creates a spline in the workplane in the shape of a hyperbola.	
Input:	
`REAL8 center[0..2]`	The coordinates at the center of the hyperbola.
`REAL8 towardfocus[0..2]`	The components of a vector in the direction pointing toward the focus from the center.
`REAL8 angle`	The asymptote angle.
`REAL8 height`	The vertex height.
`REAL8 trim[0..2]`	The coordinates used to trim the outer end of the hyperbola.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
FE_FAIL	Unable to create the spline.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSplineBlend

feSplineBlend **(curve1, near1, curve2, near2, factor, messages)**	
Description:	
Creates a spline in that blends the ends of two curves.	
Input:	
`INT4 curve1`	The ID of the first curve.
`REAL8 near1[0..2]`	Coordinates near the desired end of curve1.
`INT4 curve2`	The ID of the second curve.
`REAL8 near2[0..2]`	Coordinates near the desired end of curve2.
`REAL8 factor`	The blend factor - adjusts the curvature of the spline between the curves.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the spline.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSplineMultipleCurves

feSplineMultipleCurve **(cuSetID)**	
Description:	
This method creates a spline from a set of curves.	
Input:	
`INT4 cuSetID`	The ID of the set of curves.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the spline.
Remarks/Usage:	
None	
Example:	
None	

Curves from Surfaces Methods

Unpublished work. © 2025 Siemens

feCurveUpdateSurfaces

feCurveUpdateSurfaces **(update, messages)**	
Description:	
This method sets the flag that decides whether the *Curves from Surfaces* methods will update their underlying surface.	
Input:	
`BOOL update`	Set to True to update/split the underlying surfaces as curves are projected or created on the surfaces. If False, then the curves will be created at the same locations, but the surfaces will remain unchanged.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveIntersectSolids

feCurveIntersectSolids

(solid1, solid2)

Description:

This method creates curves along the intersections of two solids. It can also be used to intersect two solid surfaces.

Input:

`INT4 solid1`	The IDs of the two solids to intersect. These solids can be regular solids, or sheet solids (solids with one or more surfaces, but no enclosed volume).
`INT4 solid2`	

Output:

None	

Return Code:

`FE_FAIL`	Unable to create the curves.

Remarks/Usage:

If `feCurveUpdateSurfaces` is True, the surfaces of the solids will be split at the intersection curves.

Example:

None

Unpublished work. © 2025 Siemens

feCurveProjectOntoSolid

feCurveProjectOntoSolid **(projectNormal, along, solidID, curveSET, messages)**	
Description:	
Creates curves by projecting one or more curves onto a solid.	
Input:	
`BOOL projectNormal`	If True, the projection is done to the closest location on the solid. If False, the projection is along the *along* vector.
`REAL8 along[0..2]`	The components of the vector to project along. Ignored unless `projectNormal` is False.
`INT4 solidID`	The ID of the solid to project onto.
`INT4 curveSET`	The ID of the set of existing curves to project onto the solid. Specify as a negative value to project a single curve.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves.
Remarks/Usage:	
If `feCurveUpdateSurfaces` is True, the surfaces of the solids will be split at the projected curves.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveProjectOntoSurfaces

feCurveProjectOntoSurfaces **(projectNormal, along, surfSET, curveSET, messages)**	
Description:	
Creates curves by projecting one or more curves onto one or more surfaces.	
Input:	
`BOOL projectNormal`	If True, the projection is done to the closest location on the surface. If False, the projection is along the *along* vector.
`REAL8 along[0..2]`	The components of the vector to project along. Ignored unless `projectNormal` is False.
`INT4 surfSET`	The ID of the set of existing surfaces to project onto. Specify as a negative value to project onto a single surface.
`INT4 curveSET`	The ID of the set of existing curves to project onto the surfaces. Specify as a negative value to project a single curve.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves.
Remarks/Usage:	
If `feCurveUpdateSurfaces` is True, the surfaces will be split at the projected curves.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveParametricOnSurface

feCurveParametricOnSurface **(surfID, uDir, location, messages)**	
Description:	
Creates a curve on a surface along one of the parametric directions of the surface.	
Input:	
`INT4 surfID`	The ID of the surface used to define the curve.
`BOOL uDir`	If True, the curve will be along the surface U parametric direction. If False, the V parametric direction will be used.
`REAL8 location[0..2]`	A location that the curve will pass through. This location is projected onto the surface and used to find the parametric value used to define the curve.
`BOOL messages`	If True, messages are written to the Messages and Lists window as the curve is created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves.
Remarks/Usage:	
If `feCurveUpdateSurfaces` is True, the surface will be split at the parametric curve.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveSlice

feCurveSlice **(solidID, planeBASE, planeNORMAL, planeX)**	
Description:	
Creates curves by slicing a solid with a plane.	
Input:	
`INT4 solidID`	The ID of the solid being sliced.
`REAL8 planeBASE[0..2]`	Any location that lies on the plane used to slice the solid.
`REAL8 planeNORMAL[0..2]`	The components of a vector that is normal to the plane.
`REAL8 planeX[0..2]`	The components of a reference vector that lies in the plane - must be perpendicular to `planeNORMAL`.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveOffsetCurveWasher

`feCurveOffsetCurveWasher`

`( curveSET, faceID, nMode, bAutoSelect, bSaveSplit, bExtend, dOffset )`

Description:

This method is obsolete and should no longer be used. It has been replaced by `feCurveOffsetCurveWasher2`. Creates curves by offsetting selected curves on a surface.

Input:

`INT4 curveSET`	The ID of the set of existing curves to create a washer or offset curve.
`INT4 faceID`	The ID of the surface to offset the curves on.
`INT4 nMode`	Offset Type: 0 = Curve, 1 = Washer
`BOOL bAutoSelect`	Offset Only: Flag to automatically offset the selected curves to all of the surfaces connected to those curves
`BOOL bSaveSplit`	Flag to keep split lines used when creating washer or offset curves.
`BOOL bExtend`	Offset Only: Flag to extend offset curves to closest edge.
`REAL8 dOffset`	Distance to offset curves.

Output:

None	

Return Code:

`FE_FAIL`	Unable to create the curves.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feCurveOffsetCurveWasher2

feCurveOffsetCurveWasher2

(curveSET, faceID, nMode, bAutoSelect, bSaveSplit, bExtend, dValue, nOffMode)

Description:

Creates curves by offsetting selected curves on a surface.

Input:

`INT4 curveSET`	The ID of the set of existing curves to create a washer or offset curve.
`INT4 faceID`	The ID of the surface to offset the curves on.
`INT4 nMode`	Offset Type: 0 = Curve, 1 = Washer
`BOOL bAutoSelect`	Offset Only: Flag to automatically offset the selected curves to all of the surfaces connected to those curves.
`BOOL bSaveSplit`	Flag to keep split lines used when creating washer or offset curves.
`BOOL bExtend`	Offset Only: Flag to extend offset curves to closest edge.
`REAL8 dValue`	Distance to offset curves or washer factor
`INT4 nOffMode`	nMode=0 - nOffMode: 1 = Offset Distance nMode=1 - nOffMode: 0 = Factor, 1 = Offset Distance

Output:

None	

Return Code:

`FE_FAIL`	Unable to create the curves.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feCurveSplitPointToPoint

feCurveSplitPointToPoint **(nPtA, nPtB)**	
Description:	
Splits a surface between two selected points.	
Input:	
`INT4 nPtA`	The two points to split the surface between.
`INT4 nPtB`	
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves / split the surface.
Remarks/Usage:	
Both points must be on the same surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

CurveSplitPointToPoint2

CurveSplitPointToPoint2 **(nPtA, nPtB, nSurfMethod, nSurfID)**	
Description:	
Splits a surface between two selected points.	
Input:	
`INT4 nPtA`	The two points to split the surface between.
`INT4 nPtB`	
`nSurfMethod`	The entity selection method.
`nSurfID`	The IDs of the points to split.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves / split the surface.
Remarks/Usage:	
Both points must be on the same surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveSplitPointToEdge

feCurveSplitPointToEdge **(nPtA, nCuID)**	
Description:	
Splits a surface between a point and an edge.	
Input:	
`INT4 nPtA`	The points to split the surface from.
`INT4 nCuID`	The ID of the curve to split to.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves / split the surface.
Remarks/Usage:	
The surface is split from the point to the closest location on the curve. Both the point and the edge must lie on the same surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveSplitPointToEdge2

feCurveSplitPointToEdge2 **(nPtA, nCuID, nSurfMethod, nSurfID)**	
Description:	
Splits a surface between a point and an edge.	
Input:	
`INT4 nPtA`	The points to split the surface from.
`INT4 nCuID`	The ID of the curve to split to.
`nSurfMethod`	The entity selection method.
`nSurfID`	The IDs of the points/edges to split.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves / split the surface.
Remarks/Usage:	
The surface is split from the point to the closest location on the curve. Both the point and the edge must lie on the same surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveSplitEdgeToEdge

feCurveSplitEdgeToEdge **(nLongEdge, nShortEdgeSet)**	
Description:	
Splits a surface between a long edge and the endpoints of one or more selected other edges.	
Input:	
`INT4 nLongEdge`	The ID of the edge to be split
`INT4 nShortEdgeSet`	The ID of a Set that contains the IDs of the edges whose endpoints will be used to split the long edge and associated surface. Alternatively, specify a negative value to choose the ID of a single short edge.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves / split the surface.
Remarks/Usage:	
The surface is split from the endpoints of the selected edges to point to the closest location on the long edge. All edges must lie on the same surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

CurveSplitEdgeToEdge2

CurveSplitEdgeToEdge2 **(nLongEdge, nShortEdgeSet)**	
Description:	
Splits a surface between a long edge and the endpoints of one or more selected other edges.	
Input:	
`INT4 nLongEdge`	The ID of the edge to be split
`INT4 nShortEdgeSet`	The ID of a Set that contains the IDs of the edges whose endpoints will be used to split the long edge and associated surface. Alternatively, specify a negative value to choose the ID of a single short edge.
`nSurfMethod`	The entity selection method.
`nSurfID`	The IDs of the edges to split.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the curves / split the surface.
Remarks/Usage:	
The surface is split from the endpoints of the selected edges to point to the closest location on the long edge. All edges must lie on the same surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurvePad

feCurvePad

(curveSET, dValue, enPadOffsetType, enPadAlignment, bPadAddWasher, nPadAlignCurve, bSetupMappedMeshing, alignVEC)

Description:

Creates a rectangular Pad around a circular hole.

Input:

`INT4 curveSET`	The ID of the set of existing curves to create a pad.
`REAL8 dValue`	Factor or Offset distance for Pad creation.
`enum zPadOffsetType enPadOffsetType`	Type of offset value: `FPAD_SCALE`, `FPAD_DIST`.
`enum zPadAlignment enPadAlignment`	Alignment type: `FPAD_AUTO`, `FPAD_VECTOR`, `FPAD_CURVE`.
`BOOL bPadAddWasher`	Flag to add washer around circular hole.
`INT4 nPadAlignCurve`	ID of curve to align pad to.
`BOOL bSetupMappedMeshing`	Flag to set up mapped meshing approaches and sizing.
`REAL8 alignVEC[0..2]`	The components of the vector to align pad. Ignored unless `enPadAlignment = FPAD_VECTOR`.

Output:

None	

Return Code:

`FE_FAIL`	Unable to create the pad.

Remarks/Usage:

None

Example:

```
Sub Main
```


Unpublished work. © 2025 Siemens

```
   Dim App As femap.model
   Set App = feFemap()

   Dim cSet As femap.Set
   Set cSet = App.feSet
   cSet.Select ( FT_CURVE , True, "Select Curves For Pad")

   Dim dFact As Double
   Dim aCuID As Long
   Dim bMap As Boolean
   Dim bWash As Boolean
   Dim valign As Variant
   Dim align(3) As Double

   dFact = 1

   align(0) = 1.0
   align(1) = 0.0
   align(2) = 1.0
   valign = align
    'aCuID = 1204

   bMap = True
   bWash = True

    App.feCurvePad( cSet.ID, dFact, FPAD_SCALE, FPAD_AUTO, bWash, aCuID,
bMap, valign )
```


Unpublished work. © 2025 Siemens

```
End Sub
```

Curve Modification Methods

Unpublished work. © 2025 Siemens

feCurveTrim

feCurveTrim **(curveID, trimID, nearLoc, extend)**	
Description:	
Trims a curve to its intersection with another curve.	
Input:	
`INT4 curveID`	The ID of the curve to be trimmed.
`INT4 trimID`	The ID of the curve to use as the trimming boundary.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location near which the curve will be trimmed. This helps to resolve ambiguities of which end of the curve to trim, and to choose between multiple intersections of the curves.
`BOOL extend`	If True, the trim curve is considered infinite and intersections are found anywhere along its length. If False, intersections must be between the curve endpoints.
Output:	
None	
Return Code:	
`FE_FAIL`	Did not trim the curve - possibly there was no intersection.
Remarks/Usage:	
This call mimics the behavior of the **Modify** → **Trim** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveExtend

feCurveExtend **(curveID, nearLoc)**	
Description:	
Extends a curve to a specified location.	
Input:	
`INT4 curveID`	The ID of the curve to be extended.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location to which the curve will be extended. The location does not have to lie on the extended curve; it is automatically projected as the curve is extended.
Output:	
None	
Return Code:	
`FE_FAIL`	Did not extend the curve.
Remarks/Usage:	
This call mimics the behavior of the **Modify** → **Extend** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveBreak

feCurveBreak

(curveID, nearLoc)

Description:

Breaks a curve at a specified location.

Input:

`INT4 curveID`	The ID of the curve to be broken.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location at which the curve will be broken. The location does not have to lie on the curve; it is automatically projected to the break location.

Output:

None	

Return Code:

FE_FAIL	Did not break the curve.

Remarks/Usage:

None

This call mimics the behavior of the **Modify** → **Break** → **At Location** command.

None

Unpublished work. © 2025 Siemens

feCurveBreak2

feCurveBreak2 **(nSetID, nearLoc)**	
Description:	
Breaks a curve at a specified location.	
Input:	
`INT4 nSetID`	The ID of the curve set to be broken.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location at which the curve will be broken. The location does not have to lie on the curve; it is automatically projected to the break location.
Output:	
None	
Return Code:	
FE_FAIL	Did not break the curve.
Remarks/Usage:	
None	
This call mimics the behavior of the **Modify** → **Break** → **At Location** command.	
None	

Unpublished work. © 2025 Siemens

feCurvesBreakAtIntersections

feCurvesBreakAtIntersections **(curveSET)**	
Description:	
Breaks all curves in a given set at all intersections between the curves in the set.	
Input:	
`INT4 curveSET`	The ID of a set that contains a list of Curve IDs used to break curves at all intersections between the curves in the set.
Output:	
None	
Return Code:	
`FE_FAIL`	No intersections were found between the curves in the set.
Remarks/Usage:	
This call mimics the behavior of the **Modify** → **Break** → **At All Intersections** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveJoin

feCurveJoin **(curve1, curve2, nearLoc, update1, update2)**	
Description:	
Extends or trims two curves to their intersection location.	
Input:	
`INT4 curve1`	The ID of the first curve to be joined.
`INT4 curve2`	The ID of the second curve to be joined.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location near which the curves will be joined. This helps to resolve ambiguities of which end of the curves to trim/extend, and to choose between multiple intersections of the curves.
`BOOL update1`	If True, curve1 is extended or trimmed to the join location. If False, curve1 is left unchanged.
`BOOL update2`	If True, curve2 is extended or trimmed to the join location. If False, curve2 is left unchanged.
Output:	
None	
Return Code:	
`FE_FAIL`	Did not join the curves - possibly there was no intersection.
Remarks/Usage:	
This call mimics the behavior of the **Modify → Join** command. Also, do not set `update1=update2=False`. In this case, nothing will be changed.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveFillet

feCurveFillet **(radius, curve1, curve2, nearLoc, update1, update2)**	
Description:	
Fillets the intersection of two curves.	
Input:	
`REAL8 radius`	The fillet radius.
`INT4 curve1`	The ID of the first curve to be filleted.
`INT4 curve2`	The ID of the second curve to be filleted.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location near which the curves will be filleted. Specification of this location is very important. It chooses the quadrant of the intersection that is going to be filleted.
`BOOL update1`	If True, curve1 is extended or trimmed to the fillet location. If False, curve1 is left unchanged.
`BOOL update2`	If True, curve2 is extended or trimmed to the fillet location. If False, curve2 is left unchanged.
Output:	
None	
Return Code:	
`FE_FAIL`	Did not fillet the curves - possibly there was no intersection.
Remarks/Usage:	
This call mimics the behavior of the **Modify** → **Fillet** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feCurveChamfer

feCurveChamfer

(len1, len2, curve1, curve2, nearLoc, update1, update2)

Description:

Chamfers the intersection of two curves.

Input:

`REAL8 len1`	The chamfer length along curve1.
`REAL8 len2`	The chamfer length along curve2.
`INT4 curve1`	The ID of the first curve to be chamfered.
`INT4 curve2`	The ID of the second curve to be chamfered.
`REAL8 nearLoc[0..2]`	The coordinates, in global rectangular, of the location near which the curves will be chamfered. Specification of this location is very important. It chooses the quadrant of the intersection that is going to be chamfered.
`BOOL update1`	If True, curve1 is extended or trimmed to the fillet location. If False, curve1 is left unchanged.
`BOOL update2`	If True, curve2 is extended or trimmed to the fillet location. If False, curve2 is left unchanged.

Output:

None	

Return Code:

`FE_FAIL`	Did not chamfer the curves - possibly there was no intersection.

Remarks/Usage:

This call mimics the behavior of the **Modify** → **Chamfer** command.

Example:

None

Unpublished work. © 2025 Siemens

Boundaries

Unpublished work. © 2025 Siemens

feBoundaryFromPoints

feBoundaryFromPoints

(nCSys, nPoints, pXYZ, nBoundaryID)

Description:

Creates a boundary surface from a group of locations specified in order around the surface.

Input:

`INT4 nCSys`	The ID of a Coordinate System that the pXYZ coordinates are defined in. 0=Global Rectangular
`INT4 nPoints`	The number of locations specified in the pXYZ array.
`REAL8 pXYZ[0..3*nPoints-1]`	The coordinates of the points used to define the boundary, specified in the selected Coordinate System. The array must be stored as [x0,y0,z0,x1,y1,z1,...xN,yN,zN]

Output:

`INT4 nBoundaryID`	The ID of the boundary that is created.

Return Code:

`FE_FAIL`	Unable to create the boundary - either points and curves failed to be created or the loop is not connected.

Remarks/Usage:

This method creates points at the locations specified, and lines between them to form a loop. The points must define a single loop. If you need to create a boundary with multiple loops (holes), then you must use `feBoundaryFromCurves`. The ID of the boundary created will use the next available surface ID.

Example:

None

Unpublished work. © 2025 Siemens

feBoundaryFromCurves

feBoundaryFromCurves

(setID)

Description:

Creates a boundary surface from a group of connected curves.

Input:

INT4 setID	The ID of a set that contains a list of Curve IDs used to define the boundary. Alternatively, if you specify a negative value, this is simply the ID of the single curve to write (which must be closed).

Output:

None	

Return Code:

FE_FAIL	Unable to create the boundary - either curves do not exist, or are not connected.

Remarks/Usage:

The ID of the boundary created will use the next available surface ID. The curves selected must form one or more closed loops. If there are multiple loops, there must be one outer loop, and one or more internal loops which are used as holes. Loops may not be nested inside holes.

Example:

This function creates a boundary from four curves.

```
Sub Main
Dim App As femap.model
Set App = feFemap()

Dim s As Object
Set s = femap.feSet()
Dim rc As Integer
startID = 1
stopID = 4
IDincrement = 1

rc = s.AddRange(startID, stopID, IDincrement)
rc = femap.feBoundaryFromCurves(s.Id)

App.feViewRegenerate(0)
```


Unpublished work. © 2025 Siemens

```
End Sub
```


Unpublished work. © 2025 Siemens

feBoundaryFromSurfaces

feBoundaryFromSurfaces

(surfaceSET)

Description:

Creates a boundary surface from a group of adjacent surfaces on a solid.

Input:

INT4 surfaceSET	The ID of a set that contains a list of Surface IDs used to define the boundary. Alternatively, if you specify a negative value, this is simply the ID of the single surface to use.

Output:

None	

Return Code:

FE_FAIL	Unable to create the boundary - either surfaces do not exist, or are not adjacent.

Remarks/Usage:

The ID of the boundary created will use the next available surface ID.

Example:

This function creates a boundary from two surfaces.

```
Sub Main
Dim femap As femap.model
Set App = feFemap()

Dim s As Object
Set s = femap.feSet()
Dim rc As Integer
surf1 = 14
surf2 = 17

rc = s.Add(surf1)
rc = s.Add(surf2)
rc = femap.feBoundaryFromSurfaces(s.ID)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feBoundaryAddSurfaces

feBoundaryAddSurfaces

(nBoundaryID, surfaceSET, bAddToExisting)

Description:

Adds surfaces to an already defined multi-surface boundary.

Input:

`INT4 nBoundaryID`	The ID of the boundary to update
`INT4 surfaceSET`	The ID of a set that contains a list of Surface IDs used to define the boundary. Alternatively, if you specify a negative value, this is simply the ID of the single surface to use.
`BOOL bAddToExisting`	Set to True, to add the new surfaces into the existing boundary.

Output:

None	

Return Code:

`FE_FAIL`	Unable to create the boundary - either surfaces do not exist, or are not adjacent.

Remarks/Usage:

The ID of the boundary created will use the next available surface ID.

Example:

This function creates a boundary from two surfaces.

```
Sub Main
Dim femap As femap.model
Set App = feFemap()

Dim s As Object
Set s = femap.feSet()
Dim rc As Integer
surf1 = 14
surf2 = 17

rc = s.Add(surf1)
rc = s.Add(surf2)
```


Unpublished work. © 2025 Siemens

```
rc = femap.feBoundaryFromSurfaces(s.ID)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feBoundaryUpdateSurfaces

feBoundaryUpdateSurfaces **(boundaryID)**	
Description:	
Updates a boundary that was defined from surfaces, when the underlying surfaces have changed.	
Input:	
`INT4 boundaryID`	The ID of the boundary surface to update.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to update the boundary - possibly the original definition is no longer valid.
Remarks/Usage:	
None	
Example:	
None	

Surfaces

Unpublished work. © 2025 Siemens

feSurfaceCorners

feSurfaceCorners **(fourCorners, c1, c2, c3, c4)**	
Description:	
Creates a surface defined by three or four corner locations.	
Input:	
`BOOL fourCorners`	Set to True to define a surface from four corners, or False to define a three-corner surface.
`REAL8 c1[0..2]`	The corner locations. These locations should be specified in global coordinates, and must be specified in a continuous loop around the surface. When `fourCorners` is set to False, values must be specified in the c4 array, even though these values will be ignored
`REAL8 c2[0..2]`	
`REAL8 c3[0..2]`	
`REAL8 c4[0..2]`	
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface. Points may be colinear.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceCornersMultiple

feSurfaceCornersMultiple

(NumberOfSurfaces, fourCorners, c1, c2, c3, c4)

Description:

Creates multiple surfaces, each defined by three or four corner locations.

Input:

`INT4 NumberOfSurfaces`	The number of surfaces to create.
`BOOL fourCorners[0..(Num▶ berOfSurfaces -1)]`	Set to True to define a surface from four corners, or False to define a three-corner surface.
`REAL8 c1[0..(3*NumberOfSurfaces)-1]`	The corner locations. These locations should be specified in global coordinates, and must be specified in a continuous loop around each surface. When `fourCorners` is set to False, values must be specified in the c4 array for every surface, even though the values will be ignored.
`REAL8 c2[0..(3*NumberOfSurfaces)-1]`	
`REAL8 c3[0..(3*NumberOfSurfaces)-1]`	
`REAL8 c4[0..(3*NumberOfSurfaces)-1]`	

Output:

None	

Return Code:

`FE_FAIL`	Unable to create surface. Points may be colinear.

Remarks/Usage:

None

Example:

The following example creates 8 1.0 x 1.0 four corner surfaces and 2 three corner surfaces

```
Sub Main

Dim App As femap.model

Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
'Dimension Array for fourCorners option for 10 surfaces
Dim FourC(9) As Boolean
'Dimension Array for Corner 1 Coordinates for 10 surfaces
Dim corn1(29) As Double
'Dimension Array for Corner 2 Coordinates for 10 surfaces
Dim corn2(29) As Double
'Dimension Array for Corner 3 Coordinates for 10 surfaces
Dim corn3(29) As Double
'Dimension Array for Corner 4 Coordinates for 10 surfaces
Dim corn4(29) As Double

For i = 0 To 9
Select Case i
Case 0,9
FourC(i) = False
Case Else
FourC(i) = True
End Select

corn1(i*3) = i
corn1(i*3+1) = 0.0
corn1(i*3+2) = 0.0

corn2(i*3) = i+1
corn2(i*3+1) = 0.0
corn2(i*3+2) = 0.0
```


Unpublished work. © 2025 Siemens

```
corn3(i*3) = i+1
corn3(i*3+1) = 1.0
corn3(i*3+2) = 0.0

corn4(i*3) = i
corn4(i*3+1) = 1.0
corn4(i*3+2) = 0.0
Next

App.feSurfaceCornersMultiple (10, FourC, corn1, corn2, corn3, corn4)

End Sub
```


Unpublished work. © 2025 Siemens

feSurfaceEdgeCurves

feSurfaceEdgeCurves **(curve1, curve2, curve3, curve4)**	
Description:	
Creates a surface defined by three or four edge curves.	
Input:	
`INT4 curve1`	ID of the first curve to select as a surface edge.
`INT4 curve2`	ID of the second curve.
`INT4 curve3`	ID of the third curve.
`INT4 curve4`	ID of the fourth curve (If curve4=0, create a 3 sided surface).
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface. Curves may not be connected, or may not form a closed loop to define the surface.
Remarks/Usage:	
The curve IDs must be specified in order, forming a closed loop around the surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceAlignedCurves

feSurfaceAlignedCurves

(curveCOUNT, curveID)

Description:

Creates a surface defined by multiple aligned curves.

Input:

`INT4 curveCOUNT`	The number of curve IDs that are specified in `curveID`.
`INT4 curveID[0..curveCOUNT]`	The IDs of the curves that will be used to create the surface. This array must contain as many `curveIDs` as are specified by `curveCOUNT`. Furthermore, the curves must be specified in the order that they will be used to create the lofted surface.

Output:

None	

Return Code:

`FE_FAIL`	Unable to create surface. Curves may not be defined properly to create aligned surface.

Remarks/Usage:

None

Example:

The following shows an example of connecting to a model and creating a surface through six curves. This assumes that the curves already existed.

```
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim curveID(10) As Long
Dim vcurveID As Variant
Dim curveCOUNT As Long
Dim rc As Long

curveCOUNT = 6
```


Unpublished work. © 2025 Siemens

```
curveID(0) = 5
curveID(1) = 4
curveID(2) = 3
curveID(3) = 2
curveID(4) = 6
curveID(5) = 1
vcurveID = curveID

rc = App.feSurfaceAlignedCurves(curveCOUNT, vcurveID)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feSurfaceRuled

feSurfaceRuled **(c1, c2)**	
Description:	
Creates a ruled surface between two curves.	
Input:	
`INT4 c1`	The ID of the first curve used to define the surface.
`INT4 c2`	The ID of the second curve used to define the surface.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface. Curves may not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceExtrude

feSurfaceExtrude **(curveSET, axisLEN, axisVEC)**	
Description:	
Creates one or more surfaces by extruding a set of curves.	
Input:	
`INT4 curveSET`	ID of set containing the IDs of the curves to be extruded. Alternatively, if you specify a negative value, this is simply the ID of the single curve to extrude.
`REAL8 axisLEN`	The length of the extrusion.
`REAL8 axisVEC[0..2]`	The components of the extrusion vector. This vector is normalized and multiplied by `axisLEN` to find the total extrusion vector.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface. Curves may not exist.
Remarks/Usage:	
Curves are extruded from their current location along the specified vector.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceRevolve

feSurfaceRevolve **(curveSET, axisANGLE, axisBASE, axisVEC)**	
Description:	
Creates one or more surfaces by revolving a set of curves.	
Input:	
`INT4 curveSET`	ID of set containing the IDs of the curves to be revolved. Alternatively, if you specify a negative value, this is simply the ID of the single curve to revolve.
`REAL8 axisANGLE`	The angle of revolution, in degrees.
`REAL8 axisBASE[0..2]`	A location, specified in global coordinates, that defines the base of the axis of revolution.
`REAL8 axisVEC[0..2]`	The components of the axis of revolution. This vector will be located at the `axisBASE` location, and the curves will be revolved around it.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface. Curves may not exist.
Remarks/Usage:	
Curves are revolved from their current location around the specified vector.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceSweep

feSurfaceSweep **(curveSET, pathSET)**	
Description:	
Creates one or more surfaces by sweeping a set of curves along a path.	
Input:	
`INT4 curveSET`	ID of set containing the IDs of the curves to be swept. Alternatively, if you specify a negative value, this is simply the ID of the single curve to sweep.
`INT4 pathSET`	ID of a set containing the IDs of the curves to be used as a path. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use as the path. If you specify a set, the curves must form a single connected path. If you are not using the standard Femap geometry engine, then this set must contain a single curve. Only the standard engine can sweep along a multicurve path.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface. Curves may not exist, or path may not be connected.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceNormalDeviation

feSurfaceNormalDeviation

(surfaceSetID, dAvgNormal, dMaxAngle)

Description:

Finds the average normal of a set of surfaces and the maximum deviation of the normal from that average.

Input:

`INT4 surfaceSetID`	The ID of a Set containing the IDs of one or more Surfaces. If you want to test just one surface, simply specify `surfaceSetID` equal to the negative of the Surface ID.

Output:

`REAL8 dAvgNormal[0..2]`	The vector components of the average normal to the entire set of selected surfaces.
`REAL8 dMaxAngle`	The maximum angle, in degrees, that any surface normal deviates from the vector returned in `dAvgNormal`.

Return Code:

`FE_FAIL`	Unable to determine average normal and deviation.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceConvert

feSurfaceConvert **(surfaceSET, bDelete)**	
Description:	
Converts Femap boundary surfaces into Solid (Parasolid) surfaces.	
Input:	
`INT4 surfaceSetID`	The ID of a Set containing the IDs of one or more boundary surfaces to convert. If you want to convert just one boundary surface, simply specify `surfaceSET` equal to the negative of the Boundary Surface ID.
`BOOL bDelete`	If True, the original Boundary Surfaces are deleted after being converted.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to convert one or more of the boundary surfaces
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceNonManifoldAdd

feSurfaceNonManifoldAdd

(solidSET)

Description:

Adds solids and sheet solids (surfaces) together to create a *general body* in parasolid using the default Non-Manifold Add Tolerance.

Input:

`INT4 solidSET`	The ID of a set that contains a list of Solid IDs that will be added to create a *general body*.

Output:

None	

Return Code:

`FE_FAIL`	Could not add selected solids and sheet solids into a *general body*.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceNonManifoldAdd2

feSurfaceNonManifoldAdd2 **(solidSET, dTolerance**	
Description:	
Adds solids and sheet solids (surfaces) together to create a *general body* in Parasolid using the specified Non-Manifold Add Tolerance.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of Solid IDs that will be added to create a *general body*.
`REAL8 dTolerance`	The tolerance at which to attempt to add the solids together.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not add selected solids and sheet solids into a *general body*.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceNonManifoldAdd3

`feSurfaceNonManifoldAdd3`

`( solidSET,  dTolerance, bIntermediateCheck)`

Description:

Adds solids and sheet solids (surfaces) together to create a *general body* in parasolid using a user-specified Non-Manifold Add Tolerance value. Also has an option to activate or deactivate *Incremental Checking* when more than 2 solids and/or sheet solids are being Non-Manifold added together.

Input:

`INT4 solidSET`	The ID of a set that contains a list of Solid IDs that will be added to create a *general body*.
`REAL8 dTolerance`	The Non-Manifold Add Tolerance value.
`BOOL bIntermediateCheck`	If True, *Incremental Checking* will be activated, meaning that the *general body* is checked after each solid or sheet solid is added to the *general body* to make sure it passes body checking. If the addition of a solid or sheet solid causes the *general body* to fail body checking, that addition will be rolled back and that solid or sheet solid will not be added to the *general body*. If False, *Incremental Checking* will not be done.

Output:

None	

Return Code:

`FE_FAIL`	Could not add selected solids and sheet solids into a*general body*.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceNonManifoldAddExpress

feSurfaceNonManifoldAddExpress **(solidSETID, dTolerance)**	
Description:	
Adds solids and sheet solids (surfaces) together to create a *general body* using a single call to the Parasolid engine and also allows a user-specified Non-Manifold Add Tolerance value.	
Input:	
`INT4 solidSETID`	The ID of a set that contains a list of Solid IDs that will be added to create a *general body*.
`REAL8 dTolerance`	The Non-Manifold Add Tolerance value.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not add selected solids and sheet solids into a *general body*.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceRecoverManifold

feSurfaceRecoverManifold **(solidSET**	
Description:	
Recovers *manifold* solids and sheet solids from *General Bodies* created using the **NonManifold Add** command.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of Solid IDs (General Body IDs) to recover manifold geometry from. Alternatively, if you specify a negative value, this is simply the ID of the single *General Body*.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not recover manifold geometry from *General Body*.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceExtract

feSurfaceExtract **(suSetID)**	
Description:	
Extracts surfaces from existing Parasolid bodies (Solids, Sheet Solids, *General Bodies*) and creates appropriate sheet solids and/or general bodies, as needed.	
Input:	
`INT4 suSetID`	The ID of a Set containing the IDs of one or more Surfaces to extract from an existing Parasolid body. Alternatively, if you want to extract just one surface from a Parasolid body, simply specify `suSetID` equal to the negative of the Surface ID.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to extract one or more surfaces.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceRemoveHole

feSurfaceRemoveHole **(curveSET)**	
Description:	
Removes holes from a surface by curve.	
Input:	
`INT4 curveSET`	The ID of a set that contains a list of Curve IDs that are part of a one or more holes to remove from a surface. Alternatively, if you specify a negative value, this is simply the ID of the single curve.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not remove hole from body.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceFromMesh

feSurfaceFromMesh

(elemSetID, bDeleteMesh, bWireframeOnly, tolerance, bLengthCutoff, lengthCutVal)

Description:

This method is obsolete and should no longer be used. It has been replaced by `feSurfaceFromMesh2`. Create a surface that approximates a shell element mesh.

Input:

`INT4 elemSetID`	The ID of a set that contains a list of shell element IDs that are used to create an approximate surface.
`BOOL bDeleteMesh`	If True, the specified elements will be deleted when the surface is created.
`BOOL bWireframeOnly`	Create only the edges/curves that bound the mesh, not the surface.
`REAL8 tolerance`	The tolerance that is used to calculate an approximate surface fit thru the mesh.
`BOOL bLengthCutoff`	If True, the `lengthCutVal` is used so that any edge greater than this value will create a single line. The edge will not be included in any spline or curved edge of the surface.
`REAL8 lengthCutVal`	The length used if `bLengthCutoff` is True

Output:

None	

Return Code:

`FE_FAIL`	Unable to create surface

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceFromMesh2

This is a para in the topic template.

feSurfaceFromMesh **(elemSetID, bDeleteMesh, bWireframeOnly, tolerance, bLengthCutoff, lengthCutVal, ang_tol)**	
Description:	
Create a surface that approximates a shell element mesh. This method supersedes `feSurfaceFromMesh`.	
Input:	
`INT4 elemSetID`	The ID of a set that contains a list of shell element IDs that are used to create an approximate surface.
`BOOL bDeleteMesh`	If True, the specified elements will be deleted when the surface is created.
`BOOL bWireframeOnly`	Create only the edges/curves that bound the mesh, not the surface.
`REAL8 tolerance`	The tolerance that is used to calculate an approximate surface fit thru the mesh.
`BOOL bLengthCutoff`	If True, the `lengthCutVal` is used so that any edge greater than this value will create a single line. The edge will not be included in any spline or curved edge of the surface.
`REAL8 lengthCutVal`	The length used if `bLengthCutoff` is True
`REAL8 ang_tol`	The tolerance controls how close the resulting surface will match the average mesh normal at each node.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create surface
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feSurfaceRayFire

This is a para in the topic template.

feSurfaceRayFire **(suSetID, nNumRay, vRayLoc, vRayDir, nHitSurface, vHitLoc)**	
Description:	
This method finds the locations on surfaces *hit* by *rays* specified by the user.	
Input:	
`INT4 soSetID`	ID of a Set object containing the IDs of the surfaces that you want to *hit*.
`INT4 nNumRay`	The number of *rays* that are contained in `vRayLoc`, `vRayDir`
`REAL8 vRayLoc[0..3*nNumRay-1]`	The location of the base of the ray/vector in Global Rectangular coordinates.
`REAL8 vRayDir[0..3*nNumRay-1]`	The components of the direction of the ray.
Output:	
`INT4 nHitSurface[0..nNumRay-1]`	The ID of the surface that each ray hits (0 if it misses all surfaces).
`REAL8 vHitLoc[0..3*nNumRay-1]`	The Global Rectangular XYZ coordinates where the ray hit the surfaces.
Return Code:	
`FE_BAD_DATA`	If any of the surfaces have a different scale factor than the others.
`FE_NOT_AVAILABLE`	If none of the Surface IDs you specified exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feImprintLocationInSurface

This is a para in the topic template.

feImprintLocationInSurface **(vXYZ, nSurfaceID)**	
Description:	
This method imprints an internal point on a parasolid surface.	
Input:	
`REAL8 vXYZ[0..2]`	A variant with the location the point to imprint.
`INT4 nSurfaceID`	The ID of the surface to imprint a point in.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	If the Surface ID specified does not exist.
Remarks/Usage:	
None	
Example:	
None	

Midsurfaces

Unpublished work. © 2025 Siemens

feSurfaceMidSingle

feSurfaceMidSingle **(trim, surfID1, surfID2)**	
Description:	
Creates a midsurface between two existing surfaces.	
Input:	
`BOOL trim`	If True, the new surface is trimmed to lie within the bounds of the solid containing the original surfaces.
`INT4 surfID1`	The first surface to use to generate the midsurface.
`INT4 surfID2`	The other surface used to generate the midsurface.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the midsurface.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceMidAuto

feSurfaceMidAuto **(surfSET, thick)**	
Description:	
Creates midsurfaces between a set of selected surfaces using the Pre-V11.1 midsurfacing capabilities (i.e., does not use Parasolid face pairing algorithm).	
Input:	
`INT4 surfSET`	ID of set containing the IDs of the surfaces to be used to generate midsurfaces.
`REAL8 thick`	The target thickness to be used to find midsurfaces.
Output:	
None	
`FE_FAIL`	Could not create the midsurfaces.
None	
Remarks/Usage:	
Similar to the **Geometry** → **Midsurface** → **Automatic** command when using the pre-V11.1 Midsurface Method.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceMidAuto2

feSurfaceMidAuto2

(suSETID, dTargetThickness, bCombineTangent, dAngleTolerance, bAutoAdd, bReversePairs)

Description:

Creates midsurfaces between a set of selected surfaces using Parasolid face pairing algorithm. This method is obsolete ans should no longer be used. It has been replaced by `feSurfaceMidAuto3`.

Input:

`INT4 suSETID`	ID of set containing the IDs of the surfaces to be used to generate midsurfaces.
`REAL8 dTargetThickness`	The target thickness to be used to find midsurfaces.
`BOOL bCombineTangent`	When True, uses Combine Tangent Surfaces option.
`REAL8 dAngleTolerance`	The Angle Tolerance used to determine surface tangency. Only used when `bCombineTangent = True`.
`BOOL bAutoAdd`	When True, uses Combine Midsurfaces option.
`BOOL bReversePairs`	When True, uses Reverse Pairs option.

Output:

None	

Return Code:

`FE_FAIL`	Could not create the midsurfaces.

Remarks/Usage:

Similar to the **Geometry** → **Midsurface** → **Automatic** command.

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceMidAuto3

feSurfaceMidAuto3

(suSETID, dTargetThickness, bCombineTangent, dAngleTolerance, bAutoAdd, bReversePairs, bVariableThick)

Description:

Creates midsurfaces between a set of selected surfaces using Parasolid face pairing algorithm.

Input:

`INT4 suSETID`	ID of set containing the IDs of the surfaces to be used to generate midsurfaces.
`REAL8 dTargetThickness`	The target thickness to be used to find midsurfaces.
`BOOL bCombineTangent`	When True, uses Combine Tangent Surfaces option.
`REAL8 dAngleTolerance`	The Angle Tolerance used to determine surface tangency. Only used when `bCombineTangent = True`.
`BOOL bAutoAdd`	When True, uses Combine Midsurfaces option.
`BOOL bReversePairs`	When True, uses Reverse Pairs option.
`BOOL bVariableThick`	When True, uses Variable Thickness Processing option.

Output:

None	

Return Code:

`FE_FAIL`	Could not create the midsurfaces.

Remarks/Usage:

Similar to the **Geometry** → **Midsurface** → **Automatic** command.

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceMidAuto4

feSurfaceMidAuto4

(suSETID, dTargetThickness, bCombineTangent, dAngleTolerance, bAutoAdd, bReversePairs, bVariableThick, bOverlapRepair)

Description:

Creates midsurfaces between a set of selected surfaces using Parasolid face pairing algorithm

Input:

`INT4 suSETID`	ID of set containing the IDs of the surfaces to be used to generate midsurfaces.
`REAL8 dTargetThickness`	The target thickness to be used to find midsurfaces.
`BOOL bCombineTangent`	When True, uses Combine Tangent Surfaces option.
`REAL8 dAngleTolerance`	The Angle Tolerance used to determine surface tangency. Only used when `bCombineTangent = True`.
`BOOL bAutoAdd`	When True, uses Combine Midsurfaces option.
`BOOL bReversePairs`	When True, uses Reverse Pairs option.
`BOOL bVariableThick`	When True, uses Variable Thickness Processing option.
`BOOL bOverlapRepair`	When True, uses Overlap Repair option.

Output:

None	
`FE_FAIL`	Could not create the midsurfaces.
None	

Remarks/Usage:

Similar to the **Geometry** → **Midsurface** → **Automatic** command.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feSurfaceMidAuto5

feSurfaceMidAuto5

(suSETID, dTargetThickness, bCombineTangent, dAngleTolerance, bAutoAdd, bReversePairs, bVariableThick, bOverlapRepair, bRemoveHoles, bKeepUntrimmed)

Description:

Creates midsurfaces between a set of selected surfaces using Parasolid face pairing algorithm

Input:

`INT4 suSETID`	ID of set containing the IDs of the surfaces to be used to generate midsurfaces.
`REAL8 dTargetThickness`	The target thickness to be used to find midsurfaces.
`BOOL bCombineTangent`	When True, uses Combine Tangent Surfaces option.
`REAL8 dAngleTolerance`	The Angle Tolerance used to determine surface tangency. Only used when `bCombineTangent = True`.
`BOOL bAutoAdd`	When True, uses Combine Midsurfaces option.
`BOOL bReversePairs`	When True, uses Reverse Pairs option.
`BOOL bVariableThick`	When True, uses Variable Thickness Processing option.
`BOOL bOverlapRepair`	When True, uses Overlap Repair option.
`BOOL bRemoveHoles`	When True, uses Remove Holes option.
`BOOL bKeepUntrimmed`	When True, uses Keep Untrimmed Midsurfaces option.

Output:

`FE_FAIL`	Could not create the midsurfaces.

Remarks/Usage:

Similar to the **Geometry → Midsurface → Automatic** command.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feSurfaceOffsetTangent

`feSurfaceOffsetTangent`

`( nSurfaceID, dTolerance, bAutoOffset, dOffset, bDeleteOriginal, bCreateGroup )`

Description:

Creates midsurfaces by offsetting a set of surfaces tangent to specified seed surface.

Input:

`INT4 nSurfaceID`	Seed surface ID to find tangent surfaces.
`REAL8 dTolerance`	Tangency tolerance (default in GUI is 0.1). Value between 0.0 and 360.0.
`BOOL bAutoOffset`	Automatically attempt to offset to the solid mid-thickness
`REAL8 dOffset`	Offset value, ignored if `bAutoOffset` is TRUE
`BOOL bDeleteOriginal`	Delete the original solid.
`BOOL bCreateGroup`	Create a group of new surfaces.

Output:

None	

Return Code:

`FE_FAIL`	Unable to offset surface.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceOffset

feSurfaceOffset

(surfID, dOffset)

Description:

Creates a new surface offset from the original surface by the specified amount. This method is obsolete and should no longer be used. It has been replaced by `feSurfaceOffset2`.

Input:

`INT4 surfID`	ID of surface to offset
`REAL8 dOffset`	Offset value. Positive value = offset direction matches surface normal, negative value = offset direction opposite surface normal.

Output:

None	

Return Code:

`FE_FAIL`	Unable to offset surface.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceOffset2

feSurfaceOffset **(surfaceSet, dOffset)**	
Description:	
Creates a new surface offset from the original surface(s) by the specified amount.	
Input:	
`INT4 surfaceSet`	ID of the set object containing the surfaces to offset. Alternatively use a negative value to identify a single surface.
`REAL8 dOffset`	Offset value. Positive value = offset direction matches surface normal, negative value = offset direction opposite surface normal.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to offset surface(s).
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceMidAttrib

feSurfaceMidAttrib

(surfSET, matlID)

Description:

Assigns meshing attributes, including thicknesses, to surfaces that were created as midsurfaces.

Input:

`INT4 surfSET`	ID of set containing the IDs of the surfaces to have attributes assigned. Alternatively, if you specify a negative value, this is simply the ID of the single surface to use.
`INT4 matlID`	The ID of the material to assign as a meshing attribute.

Output:

None	
`FE_FAIL`	Could not assign attributes.
None	

Remarks/Usage:

The surface set that you specify can include surfaces that were not generated as midsurfaces, but only those surfaces that were midsurfaces will have meshing attributes assigned. This means that you can simply include all surfaces in the model and it will automatically find all the midsurfaces and assign attributes.

Example:

None

Unpublished work. © 2025 Siemens

feSurfaceTrimToSolid

feSurfaceTrimToSolid **(sheetID, solidID)**	
Description:	
Trims an open, sheet solid to the boundaries of a solid that contains it.	
Input:	
`INT4 sheetID`	The ID of the solid that is defined by the surfaces that are to be trimmed.
`INT4 solidID`	The ID of the bounding solid used to trim the *sheet*.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not trim to the selected surface.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceTrimWithCurves

feSurfaceTrimWithCurves	
(surfID, curveSET)	
Description:	
Trims a surface with one or more curves.	
Input:	
`INT4 surfID`	The ID of the surface to be trimmed.
`INT4 curveSET`	ID of set containing the IDs of the curves to be used for trimming. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not trim the selected surface.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceExtend

feSurfaceExtend **(curveSET, Shape, ExtendTo, ExtendToSolidID, ExtendToLoc, ExtendDist)**	
Description:	
Extends a surface edge using one of several methods.	
Input:	
`INT4 curveSET`	ID of set containing the IDs of the curves to be used for trimming. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`INT4 Shape`	The Shape of the extended Surface 0=Linear, 1=Continuous Curvature, 2=Reflective
`INT4 ExtendTo`	The Location to Extend the surface to 0=ID of Solid, 1=To Specified Location, 2=Specified Distance
`INT4 ExtendToSolidID`	Only used if ExtendTo=0. The ID of the Solid to extend surface until it hits this solid.
`REAL8 ExtendToLoc[0..2]`	Only used if `ExtendTo=1`. The location in Global Rectangular coordinates of the location to extend to.
`REAL8 ExtendDist`	Only used if `ExtendTo=2`. The distance to extend the surface.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not extend the selected surface.
Remarks/Usage:	
Although only one of the `ExtendToSolidID`, `ExtendToLoc` and `ExtendDist` are used depending on the value of `ExtendTo`, all must be provided. The values of the unused ones do not matter.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSurfaceGetUnderlying

feSurfaceGetUnderlying **(nSurfaceID, bAutoOffset, dOffset, dSizeFactor)**	
Description:	
This function extracts the complete surface underlying a Parasolid face, i.e. without any of its current edge curves, it then make a parametrically rectangular patch, matching the rough size of the surface.	
Input:	
`INT4 nSurfaceID`	ID of the original Parasolid surface
`BOOL bAutoOffset`	When True, the new surface will be offset an amount determined by the program. When False, the new surface will be offset using the value specified in `dOffset`.
`REAL8 dOffset`	Distance to offset new surface from original surface when `bAutoOffset` is set to False.
`REAL8 dSizeFactor`	Scale Factor applied to new surface. Setting this value above 1.0 will create a larger rectangular surface, while setting the value below 1.0 will create a smaller rectangular surface.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not extend the selected surface.
Remarks/Usage:	
None	
Example:	
None	

Solids

Unpublished work. © 2025 Siemens

feSolidPrimitive

feSolidPrimitive	
(operation, shape, positive, origin, size, title)	
Description:	
Creates a solid primitive.	
Input:	
`INT4 operation`	The type of operation to perform when creating the solid: 0=Create New Solid, 1=Add to Active Solid, 2=Remove from Active Solid, 3=Common with Active Solid.
`INT4 shape`	The type of primitive to create: 0=Block (Center), 1=Block (Corner), 2=Cylinder, 3=Cone, 4=Sphere, 5=Sphere (Alt)
`BOOL positive`	Set to True if the Z direction should be in the positive workplane Z direction, or False to go in the negative workplane Z direction. Dimensions should all be positive.
`REAL8 origin[0..1]`	The location, in the workplane, of the origin of the solid.
`REAL8 size[0..2]`	The size of the primitive to create. The exact meaning of these dimensions varies depending on the value of shape. Refer to the table at the end of this command for details.
`STRING title`	The title to assign to the solid - only used if creating a new solid.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the primitive.
Remarks/Usage:	
The Add, Remove and Common operations manipulate the active solid. You can select the active solid by setting `Femap.Info_ActiveID(39)` to the ID of the solid that you want to use.	

Unpublished work. © 2025 Siemens

Example:

Primitive/Shape	Size
Block (Center or Corner)	size(0) = X length (in workplane)
	size(1) = Y length (in workplane)
	size(2) = Z length (in workplane)
Cylinder	size(0) = Radius
	size(1) = not used
	size(2) = Height
Cone	size(0) = Bottom Radius
	size(1) = Top Radius
	size(2) = Height
Sphere	size(0) = Radius
	size(1) = not used
	size(2) = not used

Unpublished work. © 2025 Siemens

feSolidExtrude

feSolidExtrude **(operation, surfID, dirMode, depthMode, dirVec, depthVec)**	
Description:	
Extrudes a surface into a solid, or modifies an existing solid by extruding.	
Input:	
`INT4 operation`	The type of operation to perform when creating the solid: 0=Create New Solid, 1=Add to Active Solid, 2=Remove from Active Solid.
`INT4 surfID`	The surface to extrude. This surface can either be an existing solid surface, or a planar boundary.
`INT4 dirMode`	Specifies the approach you will use to specify the extrusion direction: 0=Positive Surface Normal, 1=Negative Surface Normal, 2=Surface Normal, Both Positive and Negative, 3=Along the Vector specified in dirVec, 4=Opposite the Vector specified in `dirVec`, 5=Along and Opposite the Vector specified in `dirVec`.
`INT4 depthMode`	Specifies the approach you will use to specify the extrusion length/depth: 0=Specified Depth - the depth is defined in `depthVec[0]`, 1=To Location - the location to extrude to is specified in `depthVec[0..2]`, in global coordinates, 2=Thru All - Only used for the Remove operation to cut completely through the active solid.
`REAL8 dirVec[0..2]`	The components, in global rectangular coordinates, of the extrusion vector. Only used if `dirMode` = 3, 4 or 5.
`REAL8 depthVec[0..2]`	The extrusion length (`depthMode=0`) or the location to extrude to, in global rectangular coordinates (`depthMode=1`). Not used for Thru All extrusions.
Output:	
None	
`FE_FAIL`	Could not create the extrusion.

Unpublished work. © 2025 Siemens

None	
The Add and Remove operations manipulate the active solid. You can select the active solid by setting `femap.Info_ActiveID(39)` to the ID of the solid that you want to use.	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidRevolve

feSolidRevolve **(operation, surfID, dirMode, angleMode, axisBase, axisVec, angleVec)**	
Description:	
Revolves a surface into a solid, or modifies an existing solid by revolving.	
Input:	
`INT4 operation`	The type of operation to perform when creating the solid: 0=Create New Solid, 1=Add to Active Solid, 2=Remove from Active Solid.
`INT4 surfID`	The surface to revolve. This surface can either be an existing solid surface, or a planar boundary.
`INT4 dirMode`	Specifies the approach you will use to specify the extrusion direction: 0=Positive Around Axis, 1=Negative Around Axis, 2=Both Positive and Negative Around Axis.
`INT4 angleMode`	Specifies the approach you will use to specify the angle of revolution: 0=Specified Angle - the angle is defined in `angleVec[0]`, 1=To Location - the location to revolve to is specified in `angleVec[0..2]`, 2=Full 360 degree revolution. All angles are specified in degrees.
`REAL8 axisBase[0..2]`	The location of the base of the axis of revolution, specified in global rectangular coordinates.
`REAL8 axisVec[0..2]`	The components of the axis of revolution, specified in global rectangular coordinates.
`REAL8 angleVec[0..2]`	The angle of revolution (`angleMode=0`) or the location to revolve to, in global rectangular coordinates (`angleMode=1`). Not used for Full 360 revolutions.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not create the revolution.

Unpublished work. © 2025 Siemens

Remarks/Usage:
The Add and Remove operations manipulate the active solid. You can select the active solid by setting `femap.Info_ActiveID(39)` to the ID of the solid that you want to use.
Example:
None

Unpublished work. © 2025 Siemens

feSolidExplode

feSolidExplode **(solidSET)**	
Description:	
Explodes selected solids into individual surfaces.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of Solid IDs that will be exploded. Alternatively, if you specify a negative value, this is simply the ID of the single solid to explode.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not explode selected solids.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidStitch

feSolidStitch **(surfaceSET, tolerance)**	
Description:	
Stitches selected surfaces into solids. A solid cleanup routine is run at the end of the command to remove any redundant geometry and simplify the geometry.	
Input:	
`INT4 surfaceSET`	The ID of a set that contains a list of Surface IDs that will be stitched. Alternatively, if you specify a negative value, this is simply the ID of the single surface to stitch.
`REAL8 tolerance`	The tolerance to use when stitching the surfaces (Parasolid only). Specify 0.0 to use the default stitching tolerance.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not stitch selected surfaces.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidStitchNoCleanup

feSolidStitchNoCleanup

(surfaceSET, tolerance)

Description:

Stitches selected surfaces into solids. A solid cleanup routine is NOT run at the end of the command to remove any redundant geometry, therefore all internal curves and individual surfaces remain intact.

Input:

`INT4 surfaceSET`	The ID of a set that contains a list of Surface IDs that will be stitched. Alternatively, if you specify a negative value, this is simply the ID of the single surface to stitch.
`REAL8 tolerance`	The tolerance to use when stitching the surfaces (Parasolid only). Specify 0.0 to use the default stitching tolerance.

Output:

None	

Return Code:

`FE_FAIL`	Could not stitch selected surfaces.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSolidSweep

feSolidSweep **(surfaceSET, curveSET)**	
Description:	
Creates solid(s) by selecting any number of surfaces to be swept along any number of curves along a connected path.	
Input:	
`INT4 surfaceSET`	The ID of a set that contains a list of Surface IDs to sweep. Alternatively, if you specify a negative value, this is simply the ID of a single surface to sweep.
`INT4 curveSET`	The ID of a set that contains a list of Curve IDs representing a sweep path. Alternatively, if you specify a negative value, this is simply the ID of a single curve to use as the sweep path.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create solid. Curves may not exist, or path may not be connected.
Remarks/Usage:	
Similar functionality to the Geometry, Solid, Sweep command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidSweepBetween

feSolidSweepBetween

(faceID1, faceID2, ptID1, ptID2, blendFactor)

Description:

Creates a swept solid between from one surface to another surface by matching a point specified on one surface to a point specified on the other surface. Additionally, a blend factor may be specified to potentially change the curvature of the swept solid.

Input:

`INT4 faceID1`	The ID of a surface to use as the *From* surface
`INT4 faceID2`	The ID of a surface to use as the *To* surface.
`INT4 ptID1`	The ID of a point to use as the *From* point
`INT4 ptID2`	The ID of a point to use as the *To* point.
`REAL8 blendFactor`	Adjusts the curvature of the swept solid (default = 1.0).

Output:

None	

Return Code:

`FE_FAIL`	Could not sweep solid between selected surfaces and points.

Remarks/Usage:

Similar functionality to the **Geometry → Solid → Sweep Between** command.

The normal direction of the two surfaces should typically face one another for best results.

The `blendFactor` refers to how long the swept solid follows tangency between the normal vector calculated at the centroid of each surface. By specifying a larger value, the swept body will closely follow the tangents for a longer distance, typically causing more curvature near the center of the swept solid. Smaller values make the tangency weaker, therefore, most of the curvature will be near the ends of the swept solid.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feSolidFillet

feSolidFillet **(curveSET, radius)**	
Description:	
Fillets one or more edges of a solid.	
Input:	
`INT4 curveSET`	The ID of a set that contains a list of Curve IDs that will be filleted. Alternatively, if you specify a negative value, this is simply the ID of the single curve to fillet.
`REAL8 radius`	The fillet radius.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not fillet selected curves.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidChamfer

feSolidChamfer **(curveSET, length)**	
Description:	
Chamfers one or more edges of a solid.	
Input:	
`INT4 curveSET`	The ID of a set that contains a list of Curve IDs that will be chamfered. Alternatively, if you specify a negative value, this is simply the ID of the single curve to chamfer.
`REAL8 chamfer`	The chamfer length.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not chamfer selected curves.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidAdd

feSolidAdd **(solidSET, redraw)**	
Description:	
Adds two or more solids to form a new solid.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of Solid IDs that will be added. This list must contain at least two solids to be added.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not add the selected solids.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidRemove

feSolidRemove **(baseID, solidSET, redraw)**	
Description:	
Removes (subtracts) one or more solids from a base solid.	
Input:	
`INT4 baseID`	The ID of the base solid, from which the solids in `solidSET` will be removed.
`INT4 solidSET`	The ID of a set that contains a list of Solid IDs that will be removed. Alternatively, if you specify a negative value, this is simply the ID of the single solid to remove.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not remove the selected solids.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidCommon

feSolidCommon **(baseID, toolID, redraw)**	
Description:	
Creates a new solid that represents the common volume between two original solids.	
Input:	
`INT4 baseID`	The ID of the base solid.
`INT4 toolID`	The ID of the tool solid.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not find the common volume between the selected solids.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidEmbed

feSolidEmbed **(baseID, toolID, redraw)**	
Description:	
Embeds one solid (the tool) inside another (the base).	
Input:	
`INT4 baseID`	The ID of the base solid.
`INT4 toolID`	The ID of the tool solid.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not embed the selected solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidEmbedMultiple

feSolidEmbedMultiple **(baseID, toolSetID, redraw)**	
Description:	
Embeds any number of solids (the tools) inside another solid (the base).	
Input:	
`INT4 baseID`	The ID of the base solid.
`INT4 toolSetID`	The ID of a set that contains a list of solid IDs to embed into the base solid. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not embed the selected solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidIntersect

feSolidIntersect **(solidSET, redraw)**	
Description:	
Breaks all of the surfaces of the selected solids at their intersections with other surfaces.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be intersected. This list must contain at least two solids.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not intersect the selected solids.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidShell

feSolidShell **(baseID, surfaceSET, thickness, redraw)**	
Description:	
Converts a solid into a thin-walled solid by offsetting outer surfaces.	
Input:	
`INT4 baseID`	The ID of the base solid to be modified.
`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will be *pierced* by the shelling operation: that is, the list of surfaces that are really on the *thru-the-thickness* edges of the shelled solid. Alternatively, if you specify a negative value, this is simply the ID of the single surface to pierce.
`REAL8 thickness`	The thickness of the resulting shelled solid. This is the distance that the outer surfaces are offset to form the inner surfaces.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not shell the selected solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidSlice

feSolidSlice **(solidSET, planeBASE, planeNORMAL, redraw)**	
Description:	
Slices one or more solids with a plane.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be sliced. Alternatively, if you specify a negative value, this is simply the ID of the single solid to slice.
`REAL8 planeBASE[0..2]`	Any location that lies on the plane used to slice the solid.
`REAL8 planeNORMAL[0..2]`	The components of a vector that is normal to the plane.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not slice the selected solids.
Remarks/Usage:	
This method corresponds to the **Geometry** → **Solid** → **Slice** command found in Femap prior to version 11.2 To use features of the **Geometry** → **Solid** → **Slice** command implemented in Femap 11.2, use `feSolidSlice2`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidSliceMatch

feSolidSliceMatch

(solidSET, planeBASE, planeNORMAL, redraw)

Description:

Slices one or more solids with a plane and leaves matching surfaces on both new solids.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be sliced. Alternatively, if you specify a negative value, this is simply the ID of the single solid to slice.
`REAL8 planeBASE[0..2]`	Any location that lies on the plane used to slice the solid.
`REAL8 planeNORMAL[0..2]`	The components of a vector that is normal to the plane.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.

Output:

None	

Return Code:

`FE_FAIL`	Could not slice the selected solids.

Remarks/Usage:

This method corresponds to the **Geometry → Solid → Slice Match** command found in Femap prior to version 11.2. The functionality of **Geometry → Solid → Slice Match** was incorporated into the **Geometry → Solid → Slice** command implemented in Femap 11.2, so use `feSolidSlice2` with `bMatchAcrossSlice=True` to duplicate the functionality of this method.

Example:

None

Unpublished work. © 2025 Siemens

feSolidSlice2

feSolidSlice2

**(solidSET, planeBASE, planeNORMAL, bSliceOrSection, bMatchAcrossSlice, nNumber of Planes,
dMultiPlaneOffset, nSliceColor, redraw)**

Description:

Slices one or more solids with a single plane or any number of planes which are parallel to, and offset from, the specified plane. Also, offers an option to use each slice plane to create a surface representing the cross-section of each solid at the slice location, instead of slicing each solid into multiple solids.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be sliced. Alternatively, if you specify a negative value, this is simply the ID of the single solid to slice.
`REAL8 planeBASE[0..2]`	Any location that lies on the plane used to slice the solid.
`REAL8 planeNORMAL[0..2]`	The components of a vector that is normal to the plane.
`BOOL bSliceOrSection`	If True, each solid will be sliced into multiple solids. If False, cross-sectional surface(s) will be created for each solid.
`BOOL bMatchAcrossSlice`	If True, matching surfaces will be created on both new solids. Only used when `bSliceOrSection` is True.
`INT4 nNumberOfPlanes`	If value > 1, multiple slice planes will be used
`REAL8 dMultiPlaneOffset`	Offset distance between each parallel plane. Only used when `nNumberOfPlanes` is > 1.
`INT4 nSliceColor`	Specifies the color of the surfaces at the slice locations. To use the Active Color for Solids, specify a -1
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.

Output:

None	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not slice the selected solids.
None	
Remarks/Usage:	
This method provides access to features of the **Geometry → Solid → Slice** command implemented in Femap 11.2, when **Method** is set to **With Plane**. Also, combines functionality of `feSolidSlice` and `feSolidSliceMatch`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidSliceAlongFace

feSolidSliceAlongFace **(surfaceID)**	
Description:	
Separates a solid by slicing it with one of its surfaces.	
Input:	
`INT4 surfaceID`	The ID of the surface used to slice the solid. This surface is also used to determine the solid to slice, since only the solid that contains the surface can be sliced.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not slice the solid with the selected surface.
Remarks/Usage:	
This method corresponds to the **Geometry** → **Solid** → **Slice Along Face** command found in Femap prior to version 11.2. To use features of the **Geometry** → **Solid** → **Slice** command implemented in Femap 11.2, use `feSolidSliceAlongFace2`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidSliceAlongFace2

feSolidSliceAlongFace2

(surfaceID, bSliceOrSection, bMatchAcrossSlice, nSliceColor, redraw)

Description:

Separates a solid by slicing it with one of its surfaces. Essentially, performs the same operation as `feSolidSliceAlongFace`, while offering some additional options.

Input:

`INT4 surfaceID`	The ID of the surface used to slice the solid. This surface is also used to determine the solid to slice, since only the solid that contains the surface can be sliced. To prompt the user to select a surface via a dialog box, enter a value of 0.
`BOOL bSliceOrSection`	If True, each solid will be sliced into multiple solids. If False, cross-sectional surface(s) will be created for each solid.
`BOOL bMatchAcrossSlice`	If True, matching surfaces will be created on both new solids. Only used when `bSliceOrSection` is True.
`INT4 nSliceColor`	Specifies the color of the surfaces at the slice locations. To use the Active Color for Solids, specify a -1
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.

Output:

None	

Return Code:

`FE_FAIL`	Could not slice the solid with the selected surface.

Remarks/Usage:

This method provides access to features of the **Geometry** → **Solid** → **Slice** command implemented in Femap 11.2, when **Method** is set to **Along Face**.

Example:

None

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feSolidSliceWithSheet2

feSolidSliceWithSheet2

(solidSET, sheetID, bSliceOrSection, bMatchAcrossSlice, nSliceColor, deleteTool, redraw)

Description:

Slices one or more solids with any number of *sheet solids* (i.e., any number of surfaces stitched together into a single body or a single surface). Also, offers an option to use each *sheet solid* to create a surface representing the cross-section of each solid at the slice location, instead of slicing each solid into multiple solids.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be sliced. Alternatively, if you specify a negative value, this is simply the ID of the single solid to slice.
`INT4 sheetSET`	The ID of a set that contains a list of sheet solid IDs that will be used as *slicing tools*. Alternatively, if you specify a negative value, this is simply the ID of the single sheet solid to use as a *slicing tool*.
`BOOL bSliceOrSection`	If True, each solid will be sliced into multiple solids. If False, cross-sectional surface(s) will be created for each solid.
`BOOL bMatchAcrossSlice`	If True, matching surfaces will be created on both new solids. Only used when `bSliceOrSection` is True.
`INT4 nSliceColor`	Specifies the color of the surfaces at the slice locations. To use the Active Color for Solids, specify a -1
`BOOL deleteTool`	If True, the *sheet solids* used as *slicing tools* will be deleted.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not slice a solid with the selected sheet solid.
Remarks/Usage:	
This method provides access to features of the **Geometry → Solid → Slice** command implemented in Femap 11.2, when **Method** is set to **With Sheet Solid**. Also, there was never a `feSolidSliceWithSheet` method, as the *2* simply indicates this method was implemented at the same time as `feSolidSlice2`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidSliceWithCurve2

feSolidSliceWithCurve2

(solidSET, curveID, bSliceOrSection, bMatchAcrossSlice, nSliceColor, , bAlongCurveNormal, bVectorDirectionOnly, vectorBase, vectorDirection, deleteTool, redraw)

Description:

Slices one or more solids with any number of curves, which has been *extruded* along a vector normal to the curve normal or along a vector specified by the user. Also, offers an option to use the *extruded curve* to create a surface representing the cross-section of each solid at the slice location, instead of slicing each solid into multiple solids.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be sliced. Alternatively, if you specify a negative value, this is simply the ID of the single solid to slice.
`INT4 curveID`	The ID of a set that contains a list of curve IDs that will be used as *slicing tools*. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use as a *slicing tool*.
`BOOL bSliceOrSection`	If True, each solid will be sliced into multiple solids. If False, cross-sectional surface(s) will be created for each solid.
`BOOL bMatchAcrossSlice`	If True, matching surfaces will be created on both new solids. Only used when `bSliceOrSection` is True.
`INT4 nSliceColor`	Specifies the color of the surfaces at the slice locations. To use the Active Color for Solids, specify a -1
`BOOL bAlongCurveNormal`	If True, *extrudes* the curve along the curve normal, which can only be inferred for planar curves (i.e., arcs, circles, but NOT straight lines). If False, *extrudes* the curve along the vector specified with the `vectorBase` and `vectorDirection` arguments.
`BOOL bVectorDirectionOnly`	If True, *extrudes*, thus *slices* in the vector direction only. If False, *extrudes*, thus *slices* in both the

Unpublished work. © 2025 Siemens

	positive and negative vector directions. Only used when `bAlongCurveNormal = False`.
`REAL8 vectorBase[0..2]`	Location at base of vector to slice along.
`REAL8 vectorDirection[0..2]`	Direction of slice.
`BOOL deleteTool`	If True, the curves used as *slicing tools* will be deleted.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not slice a solid with the selected curve.
Remarks/Usage:	
This method provides access to features of the **Geometry → Solid → Slice** command implemented in Femap 11.2, when **Method** is set to **With Curve**. Also, there was never a `feSolidSliceWithCurve` method, as the "2" simply indicates this method was implemented at the same time as `feSolidSlice2`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidEmbedFace

feSolidEmbedFace **(surfaceID, dirMode, curveOpt, direction, offset, redraw)**	
Description:	
Uses a surface of a solid to divide it.	
Input:	
`INT4 surfaceID`	The ID of the surface used to slice the solid. This surface is also used to determine the solid to slice, since only the solid that contains the surface can be sliced.
`INT4 dirMode`	The option used to determine which direction the surface will be projected as it is embedded into the solid: 0=Automatic direction and offset selection, 1=Specified direction, Automatic offset, 2= Automatic Direction, Specified offset.
`INT4 curveOpt`	The option used to determine which curves of the surface will be embedded: 0=Outline only (skip interior curves/holes), 1=All Curves.
`REAL8 direction[0..2]`	The vector used to specify the direction to embed the face - only used if `dirMode=1`.
`REAL8 offset`	The distance that the face will be embedded into the solid. Only used if `dirMode=1` or `dirMode=2`.
`BOOL redraw`	If True, the graphics will be redrawn to reflect the updated solids following this operation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not embed the surface into the solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feSolidCleanup

feSolidCleanup **(solidSET, redundant, sliver, check)**	
Description:	
Performs various cleanup and checking on selected solids.	
Input:	
`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be checked. Alternatively, if you specify a negative value, this is simply the ID of the single solid to check.
`BOOL redundant`	If True, redundant geometry that is not required to define the solid will be removed. For example, extra lines or surfaces that were *cut* into the underlying surfaces will be removed.
`BOOL sliver`	If True, an attempt will be made to remove any and all small sliver surfaces. This option is only available if the solid is a Parasolid solid.
`BOOL check`	If True, the solids will be checked to ensure that they are still valid. This is especially important if you have attempted to remove sliver surfaces.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not cleanup the selected solids.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidCleanupAdvanced

feSolidCleanupAdvanced

```
( solidSET, RepairEdges, RepairEdgesVal, CleanSmoothDiscont,
CleanSmoothDiscontVal, RemoveSelfInt, AllowSurfMod, RemoveSpikes,
RemoveSpikesVal,
RemoveSmallEdges, RemoveSmallEdgesVal, RemoveSmallFaces,
RemoveSmallFacesVal, RemoveSliverFaces, RemoveSliverFacesVal,
ConvertToAnalytic, ConvertToAnalyticVal,
RecalcEdges, RecalcEdgesVal, MergeEdges, HealSurfaces, HealSurfacesVal,
HealSmoothDiscont, HealSmoothDiscontVal, ReplaceMissing )
```

Description:

Performs more advanced cleanup and checking on selected solids.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be checked. Alternatively, if you specify a negative value, this is simply the ID of the single solid to check.
`BOOL RepairEdges`	If True, turn on Cleaning Repair Edges.
`REAL8 RepairEdgesVal`	Size for repairing edges.
`BOOL CleanSmoothDiscont`	If True, turn on Cleaning Smooth or Split Discontinuities.
`REAL8 CleanSmoothDiscontVal`	Angle for Smooth or Split Discontinuities.
`BOOL RemoveSelfInt`	If True, turn on Cleaning Remove Surface Self-Intersections.
`BOOL AllowSurfMod`	If True, turn on Cleaning Allow Surface Modifications.
`BOOL RemoveSpikes`	If True, turn on Small Feature Remove Spikes.
`REAL8 RemoveSpikesVal`	Size for spike removal.
`BOOL RemoveSmallEdges`	If True, turn on Small Feature Remove Small Edges.
`REAL8 RemoveSmallEdgesVal`	Size for small edge removal.

Unpublished work. © 2025 Siemens

`BOOL RemoveSmallFaces`	If True, turn on Small Feature Remove Small Faces.
`REAL8 RemoveSmallFacesVal`	Size for small face removal.
`BOOL RemoveSliverFaces`	If True, turn on Small Feature Remove Sliver Faces.
`REAL8 RemoveSliverFacesVal`	Size for sliver face removal.
`BOOL ConvertToAnalytic`	If True, turn on Geometry Simplification Convert to Analytic Geometry.
`REAL8 ConvertToAnalyticVal`	Value for Analytic Geometry conversion.
`BOOL RecalcEdges`	If True, turn on Edge Heal Recalculate Edge Geometry.
`REAL8 RecalcEdgesVal`	Value for recalculating edge geometry.
`BOOL MergeEdges`	If True, turn on Edge Heal Merge Edges.
`BOOL HealSurfaces`	If True, turn on Surface Heal and Stitch Heal Surfaces.
`REAL8 HealSurfacesVal`	Value for healing surfaces.
`BOOL HealSmoothDiscont`	If True, turn on Surface Heal and Stitch Smooth or Split Discontinuities.
`REAL8 HealSmoothDiscontVal`	Value for Smoothing/Splitting Discontinuities during Surface Healing.
`BOOL ReplaceMissing`	If True, turn on Surface Heal and Stitch Replace Missing Geometry.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not cleanup the selected solids.
Remarks/Usage:	
For all numeric values used by this method, specifying a value of 0.0 will automatically use the default value that is shown in the dialog box when you access this functionality from the user interface.	

Unpublished work. © 2025 Siemens

Example:

This example calls Advanced Cleanup on Solid 1, with all the default options that would be used had the functionality been accessed from the user interface.

```
Sub Main
Dim App As femap.model
Set App = feFemap()

App.feSolidCleanupAdvanced( -1, True, 0.0, True, 0.0, True, _    True,
True, 0.0, True, 0.0, True, 0.0, True, 0.0, True, 0.0, _ True, 0.0, True,
False, 0.0, False, 0.0, False )

End Sub
```


Unpublished work. © 2025 Siemens

feSolidRemoveFace

feSolidRemoveFace **(surfaceSET, aggressive, redraw)**	
Description:	
Removes face(s) from a solid and then heals the gap.	
Input:	
`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will be removed. Alternatively, if you specify a negative value, this is simply the ID of the single surface to remove.
`BOOL aggressive`	Attempts aggressive face removal.
`BOOL redraw`	Redraw the model after command.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not remove face.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidRemoveCurve

feSolidRemoveCurve **(cuSET, nRemoveOption)**	
Description:	
Removes curve(s) from a solid or sheet solid.	
Input:	
`INT4 cuSET`	The ID of a set that contains a list of curve IDs that will be removed. Alternatively, if you specify a negative value, this is simply the ID of the single curve to remove.
`INT4 nRemoveOption`	Allows you to choose a curves removal strategy: 0=Basic (`FCRO_NORMAL`) 1=Aggressive (`FCRO_AGGRESSIVE`) 2=Combine Surfaces (`FCRO_COMBINE`)
Output:	
None	
Return Code:	
`FE_FAIL`	Could not remove curve.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidExtendToSurface

feSolidExtendToSurface **(suID1, suID2)**	
Description:	
Attempts to extend a selected surface on a solid to a different surface, which may or may not be part of a solid. If successful, the original solid will be modified.	
Input:	
`INT4 suID1`	The ID of a set that contains a list of surface IDs that will be extended. Alternatively, if you specify a negative value, this is simply the ID of the single surface on a solid to extend.
`INT4 suID2`	ID of a single surface or surface on a solid to extend the solid containing the surface specified in suID1 to.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not extend face(s) of solid(s) to specified surface.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidFillCavity

feSolidFillCavity

(curveID)

Description:

Fills a cavity or hole in a solid by selecting a single seed curve, usually part of a loop.

Input:

`INT4 curveID`	The ID of a single curve to use as the seed curve, usually part of a loop.

Output:

None	

Return Code:

`FE_FAIL`	Could not remove face.

Remarks/Usage:

Using this API call is essentially the same as using the **Geometry** → **Solid** → **Fill Hole** command in the user interface.

Example:

None

Unpublished work. © 2025 Siemens

feSolidRemoveBlendsBelowRadius

feSolidRemoveBlendsBelowRadius

(soSetID, max_rad)

Description:

This method attempts to remove all blends (fillet surfaces) from a solid, with an option to limit blend removal to the blends which have a radius less than or equal to the specified value.

Input:

`INT4 solSetID`	ID of a Set object containing existing solids for blend removal. Alternatively, specify as a negative value try and remove blends from a single solid.
`REAL8 max_rad`	Value of maximum blend radius to remove. An attempt will be made to remove all blends with a radius less than or equal to this value. Enter a value of 0.0 to attempt to remove all blends, regardless of radius.

Output:

None	

Return Code:

`FE_FAIL`	Could not remove face.

Remarks/Usage:

Using this call is essentially the same as using the **Feature Removal** Tool in the **Meshing** Toolbox with **Feature Type** set to **Blends** and `max_rad` being > 0.0 activates Limit Size and sets the value.

Example:

None

Unpublished work. © 2025 Siemens

feSolidRemoveRedundantPoint

feSolidRemoveRedundantPoint **(ptID)**	
Description:	
This method attempts to remove redundant vertices from a solid. Internally, this command will take a vertex on a Parasolid body, and if the attached curves are tangent, it attempts to replace the curves with a single one, and remove the vertex.	
Input:	
`INT4 ptID`	ID of point to attempt to remove.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not remove point.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidMidSurfaceManualInput

feSolidMidSurfaceManualInput **(suSet1_ID, suSet2_ID)**	
Description:	
This method attempts to create a midsurface using the Variable Thickness Processing option between two sets of specified surfaces.	
Input:	
`INT4 suSet1_ID`	ID of a Set object containing existing surfaces on *side 1* of a solid. Alternatively, specify as a negative value to select a single surface.
`INT4 suSet2_ID`	ID of a Set object containing existing surfaces on *side 2* of a solid. Alternatively, specify as a negative value to select a single surface.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidRayFire

feSolidRayFire

(soSetID, nNumRay, vRayLoc, vRayDir, nHitSurface, vHitLoc)

Description:

This method finds the locations on solids *hit* by *rays* specified by the user.

Input:

`INT4 soSetID`	ID of a Set object containing the IDs of the solids that you want to *hit*.
`INT4 nNumRay`	The number of *rays* that are contained in `vRayLoc, vRayDir`.
`REAL8 vRayLoc[0..3*nNumRay-1]`	The location of the base of the ray/vector in Global Rectangular coordinates.
`REAL8 vRayDir[0..3*nNumRay-1]`	The components of the direction of the ray.

Output:

`INT4 nHitSurface[0..nNumRay-1]`	The ID of the surface that each ray hits (0 if it misses all surfaces).
`REAL8 vHitLoc[0..3*nNumRay-1]`	The Global Rectangular XYZ coordinates where the ray hit the surfaces.

Return Code:

`FE_BAD_DATA`	If any of the solids have a different scale factor than the others.
`FE_NOT_AVAILABLE`	If none of the Solid IDs you specified exist.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feSolidAlignSplits

feSolidAlignSplits

(entType, setID, vecMode, alignVec)

Description:

The method attempts to reorient the split lines of *periodic faces* (i.e., holes, cylinders, cones, etc), either automatically or using a alignment vector specified by the user.

Input:

`INT4 entType`	Type of entities which will be in the input set, setID, will contain, surfaces (5=FT_SURFACE) or solids (39=FT_SOLID). If you specify just surfaces, only those specified will be reoriented (and their matching 180 to 360 surface). If you specify solids, all faces on the solid that can be reoriented, will be reoriented.
`INT4 setID`	ID of a Set object containing existing surfaces or solids of a solid. Alternatively, specify as a negative value to select a single surface or solid.
`INT4 vecMode`	Vector Mode. `0 = FSAV_VECAUTO` - Automatically picks a vector perpendicular to the axis of spherical, conical, toroidal, or cylindrical surfaces. Any co-aligned surface of these types will always get the same perpendicular vector, therefore exactly aligning their splits. `1 = FSAV_VECINPUT` - Use the vector specified in `alignVec` to reorient the surfaces. `2 = FSAV_VECASK` - Prompts user to choose a vector.
`REAL8 alignVec[0..2]`	Alignment Vector (only `vecMode = FSAV_VECINPUT`).

Output:

None	

Return Code:

None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

feSolidThicken

feSolidThicken **(nSurfaceSet, dThickIn, dThickOut, nBooleanMode, bIdividual, bDeleteOriginalSurface)**	
Description:	
The method attempts to thicken a surface of a solid.	
Input:	
`INT4 nSurfaceSet`	ID of a Set object containing existing surfaces of a solid. Alternatively, specify as a negative value to select a single surface.
`REAL8 dThickIn`	The value of the distance to thicken the surface inward.
`REAL8 dThickOut`	The value of the distance to thicken the surface outward.
`INT4 nBooleanMode`	Number of the mode to use. Use 0 for None, 1 for Add, 2 for Subtract, or 3 for Embed.
`BOOL bClean`	If True, an attempt will be made to clean the geometry. This option is only available if the Add, Subtract, or Embed is chosen.
`BOOL bIndividual`	If True, each surface selected to thicken will create and individual solid.
`BOOL bDeleteOriginalSurface`	If True, an attempt will be made to delete the selected surface(s).
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidInsideMulti

feSolidInsideMulti **(soSetID, nNumPoints, vPtLocs, nInside)**	
Description:	
The method determines if a location is *inside*, *outside*, or *on* a point, curve, or surface of all the selected solids considered as a *single combined solid*. To determine if a location is *inside*, *outside*, or *on* a point, curve, or surface of a particular solid, then only the ID of that single solid should be included in the Set object.	
Input:	
`INT4 soSetID`	ID of a Set object containing existing solids to be considered as a*single combined solid*. Alternatively, specify as a negative value to select a single solid.
`INT4 nNumPts`	Number of locations to test for *inside*, *outside*, or *on* a point, curve, or surface of the *single combined solid*.
`REAL8 vPtLocs[0..3*nNumPts-1]`	The Global Rectangular coordinates of the location(s) to be tested.
Output:	
`INT4 nInside`	0=Location is outside the single combined solid, 1=Location is inside the single combined solid, 2=Location is on a point, curve, or surface of the single combined solid.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Geometry and Mesh Generation Methods

In Femap version 12 and above, a CopyTool Object was created to grant users access the enhanced **Copy**, **Rotate**, and **Reflect** commands on both the **Geometry** and **Mesh** menus. For more information, see **CopyTool Objects**.

The commands on the **Copy...** (**feGenerateCopy2**), **Rotate...** (**feGenerateRotate2**), and **Reflect...** (**feGenerateReflect2**) menus which existed in Femap prior to version 12, along with the commands on the current **Radial Copy...** () and **Scale...** **feGenerateScale2**) menus for both Geometry and Mesh, may be accessed using the methods described in this section. The appropriate entities that can be used with each method are listed in the method tables.

Unpublished work. © 2025 Siemens

feGenerateCopy

feGenerateCopy **(entityTYPE, entitySET, alongVEC, nodeINC, match)**	
Description:	
Creates new entities by copying existing entities along a vector. This call behaves identically to `feGenerateCopy2` when `matchColorLayer` argument is set to False.	
Input:	
`INT4 entityTYPE`	The entity type to copy: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be copied. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy.
`REAL8 alongVEC[0..2]`	The components of the vector used to copy the entities. Components must be specified in global rectangular coordinates. This is not a unit vector; the entities are copied a distance equal to the length of the vector.
`INT4 nodeINC`	Only used if `entityTYPE=Element`. When you are copying elements, there are two possible approaches that you can take. You can either copy the elements and all of the nodes that they are used to define them, or you can just copy the element connectivity onto another existing set of nodes. If you want to copy the nodes and elements, set `nodeINC=0`. If you want to copy the elements onto existing nodes, you should set `nodeINC` equal to the difference in `nodeIDs` from the nodes on the existing elements to the nodes on the new elements.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Could not copy the entities.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feGenerateCopy2

feGenerateCopy2

(entityTYPE, entitySET, alongVEC, nodeINC, match, matchColorLayer)

Description:

Creates new entities by copying existing entities along a vector. When `matchColorLayer` argument is set to False, behaves identical to `feGenerateCopy`.

Input:

`INT4 entityTYPE`	The entity type to copy: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be copied. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy.
`REAL8 alongVEC[0..2]`	The components of the vector used to copy the entities. Components must be specified in global rectangular coordinates. This is not a unit vector; the entities are copied a distance equal to the length of the vector.
`INT4 nodeINC`	Only used if `entityTYPE=Element`. When you are copying elements, there are two possible approaches that you can take. You can either copy the elements and all of the nodes that they are used to define them, or you can just copy the element connectivity onto another existing set of nodes. If you want to copy the nodes and elements, set `nodeINC=0`. If you want to copy the elements onto existing nodes, you should set `nodeINC` equal to the difference in `nodeIDs` from the nodes on the existing elements to the nodes on the new elements.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.
`BOOL matchColorLayer`	If True, then color, layer, and some additional parameters of original entities are transferred to the newly created entities. If False, active

Unpublished work. © 2025 Siemens

	parameters for the entity type will be used for the new entities.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not copy the entities.
Remarks/Usage:	
Additional parameters included when `matchColorLayer` is True for Nodes include definition coordinate system, output coordinate system, permanent constraints, and Superelement ID, while additional parameters for Elements include formulation and material orientation angle.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGenerateRadialCopy

feGenerateRadialCopy

(entityTYPE, entitySET, origVEC, length, Match)

Description:

Creates new entities by radially copying existing entities about an origin. This call behaves identically to `feGenerateRadialCopy2` when `matchColorLayer` and `bScaleAroundVec` arguments are set to False.

Input:

`INT4 entityTYPE`	The entity type to copy: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element. Solids cannot be radially copied.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be copied. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy.
`REAL8 origVEC[0..2]`	The coordinates of the origin around which the radial copy will be performed. The coordinates must be in global rectangular.
`REAL8 length`	The radial distance to copy the entities. A positive value increases the distance from the origin. A negative value decreases the distance.
`BOOL Match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.

Output:

None	

Return Code:

`FE_FAIL`	Could not copy the entities.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feGenerateRadialCopy2

feGenerateRadialCopy2

(entityTYPE, entitySET, origXYZ, aroundVec, length, bScaleAroundVec, Match, matchColorLayer)

Description:

Creates new entities by radially copying existing entities about an origin or about a vector. When both `matchColorLayer` and `bScaleAroundVec` arguments are set to False, behaves identical to `feGenerateRadialCopy`.

Input:

`INT4 entityTYPE`	The entity type to copy: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element. Solids cannot be radially copied.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be copied. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy.
`REAL8 origXYZ[0..2]`	When `bScaleAroundVec=True`, coordinates of the origin of the axis around which the radial copy will be performed. When `bScale-ArouncVec=False`, coordinates of the origin around which the radial copy will be performed. In both cases, the coordinates must be defined in global rectangular.
`REAL8 aroundVec[0..2]`	When `bScaleAroundVec=True`, components of the axis of radial copy, in global rectangular coordinates. When `bScaleAroundVec=False`, argument is ignored.
`REAL8 length`	The radial distance to copy the entities. A positive value increases the distance from the origin. A negative value decreases the distance.
`BOOL bScaleAroundVec`	If True, radial copy is performed around vector defined by both `origXYZ` and `aroundVec`. If False, radial copy is performed around location specified in `origXYZ` only.

Unpublished work. © 2025 Siemens

`BOOL Match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.
`BOOL matchColorLayer`	If True, then color, layer, and some additional parameters of original entities are transferred to the newly created entities. If False, active parameters for the entity type will be used for the new entities.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not copy the entities.
Remarks/Usage:	
Additional parameters included when `matchColorLayer` is True for Nodes include definition coordinate system, output coordinate system, permanent constraints, and Superelement ID, while additional parameters for Elements include formulation and material orientation angle.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGenerateScale

feGenerateScale

(entityTYPE, entitySET, origVEC, scaleVEC, csysID, match)

Description:

Creates new entities by scaling entities about a location. This call behaves identically to `feGenerateScale2` when `matchColorLayer` argument is set to False.

Input:

`INT4 entityTYPE`	The entity type to scale: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be scaled. Alternatively, if you specify a negative value, this is simply the ID of the single entity to scale.
`REAL8 origVEC[0..2]`	The coordinates of the origin around which the scale will be performed. The coordinates must be in the coordinate system specified in `csysID` - not necessarily global rectangular.
`REAL8 scaleVEC[0..2]`	The scaling values. The entries in this vector are the scale factors in the X,Y and Z coordinate directions of the `csysID` coordinate system. If you want a uniform scale, you must still set all three values to the same scale factor.
`INT4 csysID`	The ID of a coordinate system that will be used to perform the scaling. Usually this will be 0=Global rectangular, unless you want to scale about some rotated set of axes.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not scale the entities.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feGenerateScale2

feGenerateScale2

(entityTYPE, entitySET, origVEC, scaleVEC, csysID, match, matchColorLayer)

Description:

Creates new entities by scaling entities about a location. When `matchColorLayer` argument is set to False, behaves identical to `feGenerateScale`.

Input:

`INT4 entityTYPE`	The entity type to scale: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be scaled. Alternatively, if you specify a negative value, this is simply the ID of the single entity to scale.
`REAL8 origVEC[0..2]`	The coordinates of the origin around which the scale will be performed. The coordinates must be in the coordinate system specified in `csysID` - not necessarily global rectangular.
`REAL8 scaleVEC[0..2]`	The scaling values. The entries in this vector are the scale factors in the X,Y and Z coordinate directions of the `csysID` coordinate system. If you want a uniform scale, you must still set all three values to the same scale factor.
`INT4 csysID`	The ID of a coordinate system that will be used to perform the scaling. Usually this will be 0=Global rectangular, unless you want to scale about some rotated set of axes.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated. (only available in v8.2 and later).
`BOOL matchColorLayer`	If True, then color, layer, and some additional parameters of original entities are transferred to the newly created entities. If False, active

Unpublished work. © 2025 Siemens

	parameters for the entity type will be used for the new entities.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not scale the entities.
Remarks/Usage:	
Additional parameters included when `matchColorLayer` is True for Nodes include definition coordinate system, output coordinate system, permanent constraints, and Superelement ID, while additional parameters for Elements include formulation and material orientation angle.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGenerateRotate

feGenerateRotate

(entityTYPE, entitySET, origVEC, axisVEC, rotANGLE, transDIST, match)

Description:

Creates new entities by rotating entities about an axis. This call behaves identically to `feGenerateRotate2` when `matchColorLayer` argument is set to False.

Input:

`INT4 entityTYPE`	The entity type to rotate: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be rotated. Alternatively, if you specify a negative value, this is simply the ID of the single entity to rotate.
`REAL8 origVEC[0..2]`	The coordinates of the origin of the axis of rotation. Must be specified in global rectangular coordinates.
`REAL8 axisVEC[0..2]`	The components of the axis of rotation, in global rectangular coordinates. The rotation will be in positive, right-hand rule sense about this axis.
`REAL8 rotANGLE`	The angle of rotation about the axis vector, in degrees.
`REAL8 transDIST`	The translation distance, through which the entities are translated along the rotation axis, as they are rotated. Usually this is zero.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.

Output:

None	

Return Code:

`FE_FAIL`	Could not rotate the entities.

Remarks/Usage:

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feGenerateRotate2

feGenerateRotate2

(entityTYPE, entitySET, origVEC, axisVEC, rotANGLE, transDIST, match, matchColorLayer)

Description:

Creates new entities by rotating entities about an axis. When `matchColorLayer` argument is set to False, behaves identical to `feGenerateRotate`.

Input:

`INT4 entityTYPE`	The entity type to rotate: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be rotated. Alternatively, if you specify a negative value, this is simply the ID of the single entity to rotate.
`REAL8 origVEC[0..2]`	The coordinates of the origin of the axis of rotation. Must be specified in global rectangular coordinates.
`REAL8 axisVEC[0..2]`	The components of the axis of rotation, in global rectangular coordinates. The rotation will be in positive, right-hand rule sense about this axis.
`REAL8 rotANGLE`	The angle of rotation about the axis vector, in degrees.
`REAL8 transDIST`	The translation distance, through which the entities are translated along the rotation axis, as they are rotated. Usually this is zero.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.
`BOOL matchColorLayer`	If True, then color, layer, and some additional parameters of original entities are transferred to the newly created entities. If False, active parameters for the entity type will be used for the new entities.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Could not rotate the entities.
Remarks/Usage:	
Additional parameters included when `matchColorLayer` is True for Nodes include definition coordinate system, output coordinate system, permanent constraints, and Superelement ID, while additional parameters for Elements include formulation and material orientation angle.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGenerateReflect

feGenerateReflect **(entityTYPE, entitySET, origVEC, normVEC, trapWIDTH, match)**	
Description:	
Creates new entities by reflecting entities about a plane. This call behaves identically to `feGenerateReflect2` when `matchColorLayer` argument is set to False.	
Input:	
`INT4 entityTYPE`	The entity type to reflect: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be reflected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to reflect.
`REAL8 origVEC[0..2]`	The coordinates of the origin of the reflection plane. Must be specified in global rectangular coordinates.
`REAL8 normVEC[0..2]`	The components of the normal to the reflection plane, in global rectangular coordinates. It is not necessary for this axis to point toward (or away from) the existing entities - either vector will yield the same result.
`REAL8 trapWIDTH`	The `trapWIDTH` of the reflection plane. Entities that fall within this distance from the reflection plane will not be reflected.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not reflect the entities.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

feGenerateReflect2

feGenerateReflect2

(entityTYPE, entitySET, origVEC, normVEC, trapWIDTH, match, matchColorLayer)

Description:

Creates new entities by reflecting entities about a plane. When `matchColorLayer` argument is set to False, behaves identical to `feGenerateReflect`.

Input:

`INT4 entityTYPE`	The entity type to reflect: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 39=Solid.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be reflected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to reflect.
`REAL8 origVEC[0..2]`	The coordinates of the origin of the reflection plane. Must be specified in global rectangular coordinates.
`REAL8 normVEC[0..2]`	The components of the normal to the reflection plane, in global rectangular coordinates. It is not necessary for this axis to point toward (or away from) the existing entities - either vector will yield the same result.
`REAL8 trapWIDTH`	The `trapWIDTH` of the reflection plane. Entities that fall within this distance from the reflection plane will not be reflected.
`BOOL match`	If True, then loads and constraints on the original entities are copied to the newly created entities. If False, only the selected entities are duplicated.
`BOOL matchColorLayer`	If True, then color, layer, and some additional parameters of original entities are transferred to the newly created entities. If False, active parameters for the entity type will be used for the new entities.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Could not reflect the entities.
Remarks/Usage:	
Additional parameters included when `matchColorLayer` is True for Nodes include definition coordinate system, output coordinate system, permanent constraints, and Superelement ID, while additional parameters for Elements include formulation and material orientation angle.	
Example:	
None	

Meshing Toolbox - Geometry Editing Tool Methods

The methods described in this section provide access to certain operations found in the **Meshing** Toolbox's **Geometry Editing** Tool.

Unpublished work. © 2025 Siemens

feCurveProjectCurvesOntoSurfaces

feCurveProjectCurvesOntoSurfaces **(cuSetID, suSetID, extendMode, attrib_trans)**	
Description:	
This method mimics the functionality of the **Meshing** Toolbox's **Geometry Editing** Tool - Project Curve Operation, which projects any number of selected curves onto any number of selected surfaces.	
Input:	
`INT4 cuSetID`	The ID of the set of existing curves to project. Alternatively, specify as a negative value to project a single curve.
`INT4 suSetID`	The ID of the set of existing surfaces to project onto. Alternatively, specify as a negative value to project onto a single surface.
`INT4 extendMode`	Specifies the Imprint Mode, 0=Impint (no extension of curves), 1=Imprint, Extend, 2=Imprint, Extend, Clean
`BOOL attrib_trans`	If True, will assign any meshing attributes on the curves in `cuSetID` and assign them to the new curves created by the projection.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feSolidExtendEdgesToSurfaces

`feSolidExtendEdgesToSurfaces`

`( cuSetID, suSetID, extendShape, bEdgeAuto )`

Description:

This method mimics the functionality of the **Meshing** Toolbox's **Geometry Editing** Tool - Extend Operation, which extends any number of selected surface edge curves to the selected surfaces.

Input:

`INT4 cuSetID`	ID of a Set object containing existing surface edge curves to extend to a surface. Alternatively, specify as a negative value to extend a single surface edge curve.
`INT4 suSetID`	ID of a Set object containing existing surfaces to extend to. Alternatively, specify as a negative value to extend to a single surface.
`INT4 extendShape`	Specifies the Extend Shape, 0=Linear, 1=Continuous Curvature, 2=Reflective
`BOOL bAutoEdge`	If True, will attempt to automatically select additional curves needed to extend an entire body at once (i.e., if one curve at the end of a pipe is selected, both curves on the end of a pipe would be extended when this option is set to True). When False, only selected curves are extended.

Output:

None	

Return Code:

None	

Remarks/Usage:

Using this call with `bAutoEdge` set to False mimics behavior of the Extend Operation when using Extend To set to Surface, while setting it to True mimics behavior of Extend To set to Surface Auto Curve.

Example:

None

Unpublished work. © 2025 Siemens

feSolidMovePointOntoGeometry

feSolidMovePointOntoGeometry **(ptSetID, geomID, extendShape, bAlongEdge)**	
Description:	
This method mimics the functionality of the **Meshing** Toolbox's **Geometry Editing** Tool - Project/Move Point Operation, which moves points to a specified point or projects points onto the specified curve, surface, or solid. This only works for points on solid or surface geometry and the underlying geometry will be modified after the points are moved/projected.	
Input:	
`INT4 ptSetID`	ID of a Set object containing existing points on solids or surfaces to move onto the type of geometry selected with geomType. Alternatively, specify as a negative value to move a single point.
`INT4 geomID`	ID of geometric entity to move/project the points. Type of entity defined using `geomType`
`INT4 geomType`	Type of geometric entity to move to/project onto. Use `enum zDataType` to select from available entity types.
`BOOL bAlongEdge`	If True, will project using a vector matching an edge each point is connected to instead of doing a normal projection based on proximity to the curve, surface, or solid.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Creating FEMAP geometry from Parasolid Geometry

Unpublished work. © 2025 Siemens

feCurveSolidToFEMAP

feCurveSolidToFEMAP

(cuSetID)

Description:

Creates new Femap curves with Femap points matching the selected Parasolid curves.

Input:

`INT4 cuSetID`	ID of set containing Parasolid curves. Alternatively, specify a negative value to choose the ID of a single Parasolid Curve.

Output:

None	

Return Code:

`FE_FAIL`	Could not create Femap curves. This can occur if a single Femap curve is selected instead of Parasolid curve.

Remarks/Usage:

If both Parasolid curves and Femap curves are selected, the Femap curves will be ignored.

Example:

None

Unpublished work. © 2025 Siemens

feSolidExtractCenterlines

feSolidExtractCenterlines

(solidSetID, inMatID, tubular_only)

Description:

Extracts lines and arcs at the centerline of the input solid geometry. Straight and curved tubes are extracted first, properties are created as Femap Circular Bar or Circular Tube. A second method is used for all other solids, the longest line or arc on the surface is found, a cross section is then found perpendicular to this curve. A new centerline line or arc is then created through the neutral axis of that section and added to the model.

Input:

`INT4 solidSetID`	ID of a Set object containing existing solids to be processed. Alternatively, specify as a negative value to process a single solid.
`INT4 inMatID`	The input material ID. If -1 is specified, Femap will use the material associated with the solid if defined for the beam property. If no material is defined for a solid, Femap will prompt the user to create one. That material will then be used for all solids that do not have a property/material assigned. If 0 is specified, Femap will prompt the user for a material, and then use that material for all beam properties created. Any value above 0 is an override, this material ID will be assigned to all beam properties whether the material exists or not.
`BOOL tubular_only`	If True, command will only attempt to find circular bar and tube cross sections. If False, after all circular bar and tube cross sections are processed, command moves on to other selected solids using the longest line/arc algorithm described above.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	
Remarks/Usage:	
None	
Example:	
None	

Nodal Area on Geometry

Unpublished work. © 2025 Siemens

feGetNodalAreaOrLength

feGetNodalAreaOrLength

(nNumElem, nElems, nFaces, bAdjustMideside, bUnitArea, pnNumNodes, pvNodes, pvAreaOrLength)

Description:

Extracts the area, unit area, or length for the nodes of a given set of elements and face ID.

Input:

INT4 nNumElem	The number of elements to get.
INT4 nElems[0..nNumElems-1]	An array with the element IDs.
INT4 nFaces[0..nNumElems-1]	An array with the corresponds face ID of the element.
BOOL bAdjustMidside	If True, accounts for midside nodes.
BOOL bUnitArea	If True, unit area is found. If False, the portion of the entire area defined by the given elements is found.

Output:

INT4 pnNumNodes	The number of nodes found on the given elements.
INT4 pvNodes[0..nNumNodes-1]	An array with the node IDs.
REAL8 pvAreaOr► Length[0..nNumNodes-1]	An array with the corresponding values of area or length

Return Code:

None

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feGetNodalAreaOnSurface

feGetNodalAreaOnSurface **(nSurface, bAdjustMideside, bUnitArea, pnNumNodes, pvNodes, pvAreas)**	
Description:	
Extracts the area or unit area for the nodes of associated elements of a given surface.	
Input:	
`INT4 nNumElem`	The ID of the surface to find get associated nodes from.
`BOOL bAdjustMidside`	If True, accounts for midside nodes.
`BOOL bUnitArea`	If True, unit area is found. If False, portion of the surface area is found.
Output:	
`INT4 pnNumNodes`	The number of nodes found on the given surface.
`INT4 pvNodes[0..nNumNodes-1]`	An array with the node IDs.
`REAL8 pvAreas[0..nNumNodes-1]`	An array with the corresponding values of area or length.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Connection Methods

Unpublished work. © 2025 Siemens

Generating Connections

Unpublished work. © 2025 Siemens

feConnectAuto

feConnectAuto

(solidSET, linearTolerance, angleTolerance, strategy, bSelfConnect, bCombineAll, nConnectProp, bGlued)

Description:

This function is obsolete. It is the same as calling `feConnectAuto2( )` with `connectType = 1`. Refer to the documentation for that function for more information.

Input:

None	

Output:

None	

Return Code:

None	

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feConnectAuto2

feConnectAuto2

**(solidSET, connectType, linearTolerance, angleTolerance, strategy, bSelfConnect, bCombineAll,
 nConnectProp, bGlued)**

Description:

Automatically generates connections between solids.

Input:

`INT4 solidSET`	The ID of a Set object that contains the IDs of solids to connect. Alternatively, if `solidSET` is negative, it is the ID of a single solid to use (although this will not do anything unless `bSelfConnect` is also True)
`INT4 connectType`	The type of connections to search for 0=All, 1=Face to Face, 2=Edge to Face
`REAL8 linearTolerance`	The maximum distance between adjacent faces to find connections. If 0.0, then use default.
`REAL8 angeTolerance`	The maximum angle deviation between adjacent planar faces to still consider them for connection. If 0.0, then use default.
`INT4 strategy`	A value between 1 and 5 that indicates the search strategy. You can also specify strategy = 0 to use the default strategy.
`BOOL bSelfConnect`	If True, allow connections between surfaces of the same solid.
`BOOL bCombineAll`	If True, connections between faces of the same two solids will be combined into a single connection.
`INT4 nConnectProp`	The ID of a Connection Property to be used for the connections (if nonzero, this Connection Property must exist). If specified as 0, a new default property will be created.
`BOOL bGlued`	Only used if `nConnectProp` is 0. If True, the default property will be a Glued type, otherwise a Contact type.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Creating connections failed.
`FE_INVALID`	The search strategy is invalid.
`FE_NOT_EXIST`	The Connection Property does not exist.
Remarks/Usage:	
None	
Example:	

The following routine automatically connects all solids with a new Glued connection...

```
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim s As femap.Set
Set s = App.feSet

s.AddAll( FT_SOLID )
App.feConnectAuto2(s.ID, 1, 0.001, 1.0, 0, False, True, 0, True)


End Sub
```

Model Methods

In general, the API does not contain methods that duplicate the functionality that is found on the **Model** menu. Most of these commands simply create single finite element entities in your model, and those operations are supported through the Entity objects, described in **Femap Entity Objects**.

In addition to those basic creation capabilities however, there are several output processing commands that are located on the **Model** menu. Those are supported directly, and are described in the following sections.

Model Load Methods

Unpublished work. © 2025 Siemens

feLoadCombine

feLoadCombine **(fromSET, toSET, scale)**	
Description:	
Combines a Load set into another set.	
Input:	
`INT4 fromSET`	The ID of the Load Set to be combined.
`INT4 toSET`	The ID of the Load Set which will contain the combined loads.
`REAL8 scale`	A scale factor to apply to the loads being combined. Normally set to 1.0 to simply combine loads.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not combine the load set, most likely it does not exist.
Remarks/Usage:	
The functionality of this method is identical to the **Model → Load → Combine** command. All restrictions on combinations and scale factors that apply to that command apply equally to this method.	
Example:	
None	

Unpublished work. © 2025 Siemens

feLoadFromFreebody

feLoadFromFreebody

(freebodyID, nodeSetID, outSetID, loadSetID, doSummation, doNodes, overrideNodeCheck)

Description:

Creates applied loads from freebody loads.

Input:

INT4 freebodyID	ID of freebody to use to determine load values.
INT4 nodeSetID	Set ID containing node IDs to load; note that given nodes must exist in the specified freebody.
INT4 outSetID	Output set ID from which to retrieve load values.
INT4 loadSetID	Load set ID to store load definitions.
BOOL doSummation	Create loads for total summation.
BOOL doNodes	Create loads for nodal summations.
BOOL overrideNodeCheck	When set to False, then if you specify a node that is not in the freebody and potentially may not have any nodes, the routine returns FE_FAIL or FE_NOT_EXIST. If True, the loads will be created anyway (has potential to create Zero loads).

Output:

None	

Return Code:

FE_FAIL	Invalid freebody, node(s), output set, or load set ID specified.
FE_NOT_EXIST	Freebody, node(s), output set, or load set specified does not exist in the database.
FE_INVALID	One or more of the nodes specified are not included in the freebody object or doSummation

Unpublished work. © 2025 Siemens

	is TRUE when specified freebody is not in interface mode.
Remarks/Usage:	
`feLoadFromFreebody` creates applied loads from freebody loads. The load components and contributions to be included in the created loads are obtained from the specified freebody. Up four load definitions will be created in the specified load set based on the options specified in this command and the attributes set in the freebody: total resultant forces, total resultant moments, nodal forces, and nodal moments.	
Example:	
None	

Model Constraint Methods

Unpublished work. © 2025 Siemens

feBCCombine

feBCCombine **(fromSET, toSET, overwrite)**	
Description:	
Combines a Constraint set into another set.	
Input:	
`INT4 fromSET`	The ID of the Constraint Set to be combined.
`INT4 toSET`	The ID of the Constraint Set which will contain the combined constraints.
`BOOL overwrite`	If True, constraints in the *toSET* will be overwritten with any duplicate constraints on the same node. If False, degrees of freedom on the common nodes will be combined to form a new constraint. Does not apply to constraint equations.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not combine the constraint set, most likely it does not exist.
Remarks/Usage:	
The functionality of this method is identical to the **Model** → **Constraint** → **Combine** command. All restrictions on combinations that apply to that command apply equally to this method.	
Example:	
None	

Model Output Methods

Unpublished work. © 2025 Siemens

feOutputProcessV2

Note

This method is obsolete. There are many combinations of options that could be passed to this method that are invalid and will not work. Please switch to using one of the other *OutputProcess* methods that provide specific access to each of the processing methods.

feOutputProcessV2

(count, full_set, processing_method, approach, from_setID, from_vectorID, to_setID, to_vectorID, groupID, factor, overwrite, ply_summary, set_info)

Description:

Provides the functionality that is available through the **Model** → **Output** → **Process** command to copy, combine, envelope, or convert output. This method replaces `feOutputProcess`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 count`	The number of processing operations; that is, the number of entries in the other arrays.
`BOOL full_set`	Set to True if the operations will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_vectorID` are ignored.
`INT4 processing_method`	The type of processing to be done: 0=Copy, 1=Combine, 2=Linear, 3=RSS, 4=Convert, 5=Envelope, 6=Error Estimate.
`INT4 approach[0..count-1]`	Additional approach required by certain methods. If `processing_method=5,Envelope` then (0=Max, 1=Min, 2=Max Abs). If `processing_method=6, Error Estimate` then (0=Max Diff, 1=Diff to Avg, 2=%Max Diff, 3=%Diff to Avg, 4=Norm% Max Diff, 5=Norm% Diff to Avg). If `processing_method=4, Convert` then (0=Avg, 1=Max).
`INT4 from_setID[0..count-1]`	The ID of the output set to process.
`INT4 from _vectorID[0..count-1]`	The ID of the output vector to process (not used if `full_set=True`).

Unpublished work. © 2025 Siemens

`INT4 to_setID[0..count-1]`	The ID of the output set where processed data will be created.
`INT4 to_vectorID[0..count-1]`	The ID of the output vector where processed data will be created (not used if `full_set=True`).
`INT4 groupID[0..count-1]`	The ID of the group used to restrict output processing. If 0, all available output, on the full model is processed.
`REAL8 factor[0..count-1]`	The scale factor for linear and RSS combinations. The Skip Below value for Error Estimates.
`BOOL overwrite[0..count-1]`	Flag to overwrite existing output if you are combining output.
`BOOL ply_summary[0..count-1]`	For Envelope, if True, this will compute the ply summary information for laminate output.
`BOOL set_info[0..count-1]`	For Envelope, if True and doing full sets, this will compute the additional set summary information for the enveloped data.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not process the requested information.
Remarks/Usage:	
Only combinations of options that are available in the **Model** → **Output** → **Process** command in Femap will correctly process information through the API.	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputProcessCopyV2

feOutputProcessCopyV2 **(bFullSet, from_setID, from_vectorID, to_setID)**	
Description:	
Allows you to Copy a single output set or vector. This method replaces `feOutputProcessCopy`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`BOOL bFullSet`	Set to True if the operation will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_setID` are ignored.
`INT4 from_setID`	The ID of the output set to process.
`INT4 from _vectorID`	The ID of the output vector to process.
`INT4 to_setID`	The ID of the Set to contain the resulting copy. If 0, a new Output Set will be created.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not process the requested information.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputProcessMergeV2

feOutputProcessMergeV2

(bFullSet, bOverwrite, nCount, from_setID, from_vectorID, to_setID)

Description:

Allows you to Combine/Merge Output in Sets or Vectors. This method replaces `feOutputProcessMerge`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bFullSet`	Set to True if the operation will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_setID` are ignored.
`BOOL bOverwrite`	If True, existing Output will be overwritten. If False, output will only be merged onto nodes/elements that have no output.
`INT4 nCount`	The number of entries in `from_setID` and `from_vectorID`.
`INT4 from_setID[0..nCount-1]`	The IDs of the output sets to process.
`INT4 from _vectorID[0..nCount-1]`	The IDs of the output vectors to process.
`INT4 to_setID`	The ID of the Set to contain the resulting copy. If 0, a new Output Set will be created.

Output:

None	

Return Code:

`FE_FAIL`	Could not process the requested information.

Remarks/Usage:

Each entry in `from_setID` and `from_vectorID` is used as a pair. For example `from_setID[0]` and `from_vectorID[0]` mean output in the vector `from_vectorID[0]` in `from_setID[0]`.

Example:

None

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feOutputProcessLinearCombinationV2

feOutputProcessLinearCombinationV2

(bFullSet, nApproach, nCount, dScaleFactor, from_setID, from_vectorID, to_setID)

Description:

Performs a linear combination, with a scale factor, of Output Sets or Vectors. This method replaces `feOutputProcessLinearCombination`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bFullSet`	Set to True if the operation will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_setID` are ignored.
`INT4 nApproach`	0=Combine All Vectors in All Sets, 1=Combine All Vectors in Each Set, 2=Combine Each Vector across All Sets. Ignored and always =2 if `bFullSet` is True.
`INT4 nCount`	The number of entries in `dScaleFactor`, `from_setID` and `from_vectorID`.
`REAL8 dScaleFactor[0..nCount-1]`	The scale factor to apply to each set or vector.
`INT4 from_setID[0..nCount-1]`	The IDs of the output sets to process.
`INT4 from _vectorID[0..nCount-1]`	The IDs of the output vectors to process.
`INT4 to_setID`	The ID of the Set to contain the resulting combination. If 0, a new Output Set will be created.

Output:

None	

Return Code:

`FE_FAIL`	Could not process the requested information.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feOutputProcessRSSCombinationV2

feOutputProcessRSSCombinationV2

(bFullSet, nApproach, nCount, dScaleFactor, from_setID, from_vectorID, to_setID)

Description:

Performs an RSS (root sum squared) combination, with a scale factor, of Output Sets or Vectors. This method replaces feOutputProcessRSSCombination. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bFullSet`	Set to True if the operation will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_setID` are ignored.
`INT4 nApproach`	0=Combine All Vectors in All Sets, 1=Combine All Vectors in Each Set, 2=Combine Each Vector across All Sets. Ignored and always =2 if `bFullSet` is True.
`INT4 nCount`	The number of entries in `dScaleFactor`, `from_setID` and `from_vectorID`.
`REAL8 dScaleFactor[0..nCount-1]`	The scale factor to apply to each set or vector.
`INT4 from_setID[0..nCount-1]`	The IDs of the output sets to process.
`INT4 from _vectorID[0..nCount-1]`	The IDs of the output vectors to process.
`INT4 to_setID`	The ID of the Set to contain the resulting combination. If 0, a new Output Set will be created.

Output:

None	

Return Code:

`FE_FAIL`	Could not process the requested information.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feOutputProcessConvertV2

This is a para in the topic template.

feOutputProcessConvertV2 **(approach, from_setID, from_vectorID, to_setID, to_vectorID, groupID)**	
Description:	
Allows you to “Convert” a single output vector between nodal and elemental locations. This method replaces `feOutputProcessConvert`. For more information see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 approach`	0=Avg, 1=Max
`INT4 from_setID`	The ID of the output set to process.
`INT4 from _vectorID`	The ID of the output vector to process.
`INT4 to_setID`	Not Used.
`INT4 to_vectorID`	Not Used.
`INT4 groupID`	The ID of the group used to restrict output processing. If 0, all available output, on the full model is processed.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not process the requested information.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputProcessEnvelopeV2

feOutputProcessEnvelopeV2

(bFullSet, nType, nApproach, bEnvelopeInSets, bEnvelopeAcrossSets, bSetInfo, nCount, from_setID, from_vectorID, to_setID)

Description:

Envelopes results in one or more Output Sets or Vectors to find worst-case scenarios. This method replaces feOutputProcessEnvelope. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bFullSet`	Set to True if the operation will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_setID` are ignored.
`INT4 nType`	The Type of envelope to create 0=Max, 1=Min, 2=Max Absolute Value.
`INT4 nApproach`	0=Envelope All Vectors in All Sets, 1=Envelope All Locations (like all Plys, all Stress Recovery Locations, ...), 2=Envelope Each Vector across All Sets. Ignored and always =2 if `bFullSet` is True.
`BOOL bEnvelopeInSets`	If True, an Envelope is created and stored in the same set as the original results Ignored and always False if `bFullSet`=True or `nApproach`=2
`BOOL bEnvelopeAcrossSets`	If True, results for similar vectors are enveloped across all of the selected sets. Ignored and always True if `bFullSet`=True or `nApproach`=2
`BOOL bSetInfo`	If True extra output vectors are created which contain the ID of the set and possibly the location where the envelope value came from.
`INT4 nCount`	The number of entries in `from_setID` and `from_vectorID`.
`INT4 from_setID[0..nCount-1]`	The IDs of the output sets to process.
`INT4 from _vectorID[0..nCount-1]`	The IDs of the output vectors to process. Ignored if `bFullSet`=True

Unpublished work. © 2025 Siemens

`INT4 to_setID`	The ID of the Set to contain the resulting combination. If 0, a new Output Set will be created. Ignored and always 0 if `bFullSet`=True or `nApproach`=2
Output:	
None	
Return Code:	
`FE_FAIL`	Could not process the requested information.
Remarks/Usage:	
Provides more flexibility than `feOutputProcessEnvelopeFromSets` as different output vectors may be specified for each output set.	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputProcessEnvelopeFromSetsV2

`feOutputProcessEnvelopeFromSetsV2`

`( bFullSet, nType, nApproach, bEnvelopeInSets, bEnvelopeAcrossSets, bSetInfo, setSET, vectorSET, to_setID )`

Description:

Envelopes results in one or more Output Sets or Vectors to find worst-case scenarios specified via a set of Output Sets and optionally a set of Output Vectors. This method replaces `feOutputProcessEnvelopeFromSets`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bFullSet`	Set to True if the operation will process entire sets of output, or False if individual vectors are to be processed. If True, `from_vectorID` and `to_setID` are ignored.
`INT4 nType`	The Type of envelope to create 0=Max, 1=Min, 2=Max Absolute Value.
`INT4 nApproach`	0=Envelope All Vectors in All Sets, 1=Envelope All Locations (like all Plys, all Stress Recovery Locations, ...), 2=Envelope Each Vector across All Sets. Ignored and always =2 if `bFullSet` is True.
`BOOL bEnvelopeInSets`	If True, an Envelope is created and stored in the same set as the original results Ignored and always False if `bFullSet`=True or `nApproach`=2
`BOOL bEnvelopeAcrossSets`	If True, results for similar vectors are enveloped across all of the selected sets. Ignored and always True if `bFullSet`=True or `nApproach`=2.
`BOOL bSetInfo`	If True extra output vectors are created which contain the ID of the set and possibly the location where the envelope value came from.
`INT4 setSET`	The ID of a set that contains a list of Output Set IDs to use in the envelope. Alternatively, if you specify a negative value, this is simply the ID of the single Output Set to use.
`INT4 vectorSET`	The ID of a set that contains a list of Output Vector IDs to use in the envelope. Alternatively, if you

Unpublished work. © 2025 Siemens

	specify a negative value, this is simply the ID of the single Output Vector to use.
`INT4 to_setID`	The ID of the Set to contain the resulting combination. If 0, a new Output Set will be created. Ignored and always 0 if `bFullSet`=True or `nApproach`=2.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not process the requested information.
Remarks/Usage:	
Provides a streamlined method of entering multiple output sets and/or output vectors by using two Set objects. Especially useful when using the `feSelectOutputV2`/`feSelectOutput2V2` (**feSelectOutputV2**/**feSelectOutput2V2**) or `feOutputSelectSets`/`feOutputSelectSets2` (**feSelectOutputSets**/**feSelectOutputSets2**) entity selection methods for selecting the Output Set(s) and Output Vector(s).	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputProcessErrorEstimateV2

feOutputProcessErrorEstimateV2 **(nMethod, from_setID, from_vectorID, groupID)**	
Description:	
Allows you to compute an Error Estimate of a single output vector. This method replaces `feOutputProcessErrorEstimate`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nMethod`	0=Max Diff, 1=Diff to Avg, 2=%Max Diff, 3=%Diff to Avg, 4=Norm% Max Diff, 5=Norm% Diff to Avg
`INT4 from_setID`	The ID of the output set to process.
`INT4 from _vectorID`	The ID of the output vector to process.
`INT4 groupID`	The ID of the group used to restrict output processing. If 0, all available output, on the full model is processed.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not process the requested information.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputFromLoad

feOutputFromLoad **(setID, loadType)**	
Description:	
Creates output from existing load information.	
Input:	
`INT4 setID`	The ID of a set that contains a list of node or elements IDs that will be checked for loads to create the output. Alternatively, if you specify 0, all nodes/elements will be checked.
`INT4 loadType`	The type of load to check to create output: Force=16, Displacement=32, Velocity=2048, Acceleration=64, Nodal Temperature=4096, Nodal HeatGen=2, Nodal Heat Flux=4, Elem Distributed Load=128, Pressure=256, Elem Temperature=8192, Elem Heat Gen 512, Elem Heat Flux=1024, Elem Convection=16384, Elem Radiation=32768, Fluid Pressure=131072, Fluid Tracking=262144, Fluid Fan Curve=2097152.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not compute output.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputTransform

`feOutputTransform`	
(REMOVED - CAN NO LONGER BE USED)	
Description:	
*This method is obsolete and has been removed. It will not work in version 11.1 and above. Use `feOutputTransform2` instead.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputTransform2V2

feOutputTransform2V2

(setSET, vecSET, nodeSET, elemSET, nodalMode, nodalCSysID, nodalGlobalComp, plateMode, plateCSysID, plateDOF, plateVec, plateAngle, solidMode, solidCSysID, engrShearStrain)

Description:

Transforms existing output to another orientation. This function replaces `feOutputTransform`, which is obsolete and should no longer be used. This method replaces `feOutputTransform`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 setSET`	ID of set containing the IDs of output sets containing output vectors to transform. Alternatively, if you specify a negative value, this is simply the ID of the single output set.
`INT4 vecSET`	ID of set containing the IDs of output vectors to transform. Alternatively, if you specify a negative value, this is simply the ID of the single output vector.
`INT4 nodeSET`	ID of set containing the IDs of nodes to transform output. Alternatively, if you specify a negative value, this is simply the ID of the single node.
`INT4 elemSET`	ID of set containing the IDs of elements to transform output. Alternatively, if you specify a negative value, this is simply the ID of the single element.
`enum nodalMode`	Sets to transformation method for Nodal Output Vectors. This can only be set to `FOD_NONE` (no transformation of Nodal Output Vectors), `FOD_VECTOR_TO_CSYS` (Into CSys transformation method), or `FOD_VECTOR_TO_NODE_OUTPUT_CSYS` (Into Node Output CSys transformation method).
`INT4 nodalCSysID`	ID of coordinate system used to transform nodal output vectors when `nodalMode` is set to `FOD_VECTOR_TO_CSYS`.

Unpublished work. © 2025 Siemens

`BOOL nodalGlobalComp`	When True, additional output vectors will be created representing the components used to generate the transformed output vector(s).
`enum plateMode`	Sets to transformation method for Plate Forces, Stresses and Strains. This can only be set to `FOD_NONE` (no transformation of Plate Forces, Stresses and Strains), `FOD_PLATE_TO_CSYS` (Into CSys transformation method), `FOD_PLATE_TO_MATL` (Into Matl Direction transformation method), or `FOD_PLATE_TO_VECTOR` (Along Vector transformation method).
`INT4 plateCSysID`	ID of coordinate system used to transform plate force, stress, and/or strain vectors when `plateMode` is set to `FOD_PLATE_TO_CSYS`.
`INT4 plateDOF`	Indicates which axis of the coordinate system specified in `plateCSysID` will be used for transformation. 1=X axis, 2=Y axis, and 3=Z axis
`REAL8 plateVec[0..2]`	The XYZ components of the vector to use for transformation when `plateMode` is set to `FOD_PLATE_TO_VECTOR`.
`REAL8 plateAngle`	Specifies value for Tolerance Angle.
`enum solidMode`	Sets to transformation method for Solid Stresses and Strains. This can only be set to `FOD_NONE` (no transformation of Solid Stresses and Strains), `FOD_SOLID_TO_CSYS` (Into CSys transformation method), or `FOD_SOLID_TO_MATL` (Into Matl Direction transformation method).
`INT4 solidCSysID`	ID of coordinate system used to transform solid stress and/or strain vectors when `solidMode` is set to `FOD_SOLID_TO_CSYS`.
`BOOL engrShearStrain`	When True, transformation assumes Engineering Shear Strain.
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	Variant does not contain real numbers.
`FE_NOT_AVAILABLE`	Problem retrieving data from the variant.

Unpublished work. © 2025 Siemens

`FE_TOO_SMALL`	Not enough entries in array.
`FE_INVALID`	Invalid value for `nodalMode`, `plateMode`, `plateDOF`, or `solidMode`.
`FE_NOT_EXIST`	Coordinate System does not exist.
`FE_FAIL`	Could not transform output.

Remarks/Usage:

The Output Orientation can be set using the various `Pref_Orient` options. See **File → Preferences - Geometry/Model tab** for a complete list of available output orientation options.

For example, `Pref_OrientQuad4ForceOutput` sets the output orientation for Force Output on 4-noded quadrilateral elements.

Example:

None

Unpublished work. © 2025 Siemens

feOutputCalculate

feOutputCalculate **(outSET, ndelem, elemSET, title, IDvar, CASEvar, formula)**	
Description:	
Computes output values based on an equation that you provide.	
Input:	
`INT4 outSET`	The ID of the set that contains the output set IDs where the calculations will be performed.
`INT4 ndelem`	Specifies whether the calculation will create nodal or elemental data. (7=Nodal, 8=Elemental).
`INT4 elemSET`	The ID of the set that contains the nodes or elements where the output will be computed.
`STRING title`	The title of the output vectors to be created.
`STRING IDvar`	The name of the variable used as the Node/Element ID in the formula that you specify.
`STRING CASEvar`	The name of the variable used as the Output Set ID in the formula that you specify.
`STRING formula`	The equation used to calculate the results. This must be specified in standard Femap form, using the two variables defined in `IDvar` and `CASEvar`.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not compute output.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputConvertV92VectorIDV2

feOutputConvertV92VectorIDV2 **(oldVecID)**	
Description:	
Some Output Vector IDs changed between Femap version 9.2 and version 9.3. This method takes a Femap version 9.2 (and before) vector ID and returns the equivalent Femap version 9.3 vector ID. This method replaces `feOutputConvertV92VectorIDV2`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 oldVecID`	The v9.2 (and before) vector ID that you want to convert.
Output:	
None	
Return Code:	
`newVecID`	The v9.3 (and later) vector ID.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputConvertV2019VectorIDToV2020V2

feOutputConvertV2019VectorIDToV2020V2

(v2019VecID)

Description:

Some Output Vector IDs changed between Femap version 2019.1 and version 2020.1. This method takes a vector ID from versions prior to Femap version 2020.1and returns the equivalent vector ID in Femap version 2020.1. For more information see, **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 v2019VecID`	The vector ID from versions prior to 2020.1 to be converted.

Output:

None	

Return Code:

`newVecID`	The version 2020.1 (and later) vector ID.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feOutputConvertV2020VectorIDToV2019V2

feOutputConvertV2020VectorIDToV2019V2 **(v2020VecID)**	
Description:	
Some Output Vector IDs changed between Femap version 2019.1 and version 2020.1. This method takes a vector ID from Femap versions 2020.1 and later and returns the equivalent vector ID in Femap version 2019.1 and prior. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 v2020VecID`	The vector ID from versions 2020.1 and later to be converted.
Output:	
None	
Return Code:	
`newVecID`	The version 2019.1 (and prior) vector ID.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputForceBalance

feOutputForceBalance

(outSETS, boundarynodeSET, boundaryelemSET, csysID, aroundXYZ, numSETS, resultant)

Description:

This method is obsolete. For new applications, use freebody objects and the `CalculateSummation2` method.

Returns the resulting force/moment information on a model interface defined by selected nodes/elements across selected output sets.

Input:

`INT4 outSETS`	The ID of a Set object that contains the output sets to process. Alternatively, if you specify a negative value, this is simply the ID of the single output set to use.
`INT4 boundarynodeSET`	The ID of a Set object that contains the nodes on the model interface where the force balance will be computed. Alternatively, if you specify a negative value, this is simply the ID of a single node.
`INT4 boundaryelemSET`	The ID of a Set object that contains the elements on the model interface where the force balance will be computed. Alternatively, if you specify a negative value, this is simply the ID of a single element.
`INT4 csysID`	The ID of the coordinate system to use. The coordinates in *aroundXYZ* must be specified in this coordinate system, and all results are returned in this coordinate system.
`REAL8 aroundXYZ[0..2]`	The coordinates of the location where the resulting force/moment are computed.

Output:

`INT4 numSETS`	The numbef of output set results contained in *resultant*.
`REAL8 resultant[0..6*numSETS-1]`	The resulting force/moment resultant computed from the loads on the selected interface. For each output set selected in *outSETS*, there will be 6 consecutive entries in this array [Fx1, Fy1, Fz1,

Unpublished work. © 2025 Siemens

	Mx1, My1, Mz1, Fx2, Fy2, ...]. All values are relative to the coordinate system selected by csysID.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to compute and return all Resultants.
Remarks/Usage:	
***This method will NOT work with *attached* output. Please use `feOutputForceBalance2` instead. Also, this method requires that all selected output sets contain Nastran Grid Point Force Balance results.	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputForceBalance2

`feOutputForceBalance2`

`( freebodyID, outSetID, toDataTable, printSummation, printDetails )`

Description:

Outputs freebody data to the message window or the data table. To retrieve the calculated total summation load programmatically, see the `CalculateSummation2()` method of `feFreebody`.

Input:

`INT4 freebodyID`	Freebody ID.
`INT4 outSetID`	Output Set ID.
`BOOL toDataTable`	If TRUE, output is sent to the data table; otherwise it is sent to the message window.
`BOOL printSummation`	When TRUE, data related to the freebody summation is printed, otherwise raw grid point force balance data for all nodes in the freebody will be output.
`BOOL printDetails`	If `printSummation` is TRUE, details regarding the force balance calculations will be output; ignored when printSummation is FALSE.

Output:

None	

Return Code:

`FE_FAIL`	Freebody summation has been calculated with potential errors (ie missing contributions of load, missing elements, etc.) The values need to be verified.
`FE_NOT_EXIST`	Specified freebody or output set does not exist.
`FE_INVALID`	Freebody is not set to interface mode (this error may occur if `printSummation` is TRUE).
`FE_SECURITY`	Data is to be output to the data table, however it is locked or not visible.

Remarks/Usage:

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feOutputCompareV2

feOutputCompareV2 **(bList, bFullCase, fromSET, toSETS, ifromVEC, itoVEC, dMaxDiff)**	
Description:	
This method is used to compare output from individual vectors or entire output sets. This method replaces `feOutputCompare`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`BOOL bList`	Flag to list the output at individual nodes or elements which differ.
`BOOL bFullCase`	Flag to compare all vectors in the output case.
`INT4 fromSET`	The ID of a Output Set you with to compare.
`INT4 toSETS`	The ID of Femap Set object containing the output sets you with to compare to. Alternatively, if you specify a negative value, this is simply the ID of a single output case.
`INT4 ifromVEC`	If not comparing the full output case then set `ifromVEC` to compare a specific output vector id in the `fromSET`.
`INT4 itoVEC`	If not comparing the full output case then set `itoVEC` to compare a specific output vector id in the `toSETS`.
`REAL8 dMaxDiff`	Value specifying the maximum value used to consider the output different.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feCompute3DInvariants

feCompute3DInvariants **(bStress, dTensor, dInvariant)**	
Description:	
This method is used to compute Stress or Strain Invariant values for a 3D element using 6 Tensor Stress or Strain Values (Solid X Normal, Solid Y Normal, Solid Z Normal, Solid XY Shear, Solid YZ Shear, and Solid ZX Shear).	
Input:	
`BOOL bStress`	Flag to indicate which invariant values to compute. True = Stress, False = Strain
`REAL8 dTensor[0..5]`	Array of Stress or Strain Tensor values: 0 = Solid X Normal 1 = Solid Y Normal 2 = Solid Z Normal 3 = Solid XY Shear 4 = Solid YZ Shear 5 = Solid ZX Shear
Output:	
`VARIANT dInvariant`	Returns Stress of Strain invariant values in a variant: 0 = Solid Maximum Principal 1 = Solid Intermediate Principal 2 = Solid Minimum Principal 3 = Solid Mean 4 = Solid Max Shear 5 = Solid Von Mises
Return Code:	
`FE_INVALID`	Values in dTensor must be REAL8, not INT4.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCompute2DInvariants

feCompute2DInvariants

(bStress, dTensor, dInvariant)

Description:

This method is used to compute Stress or Strain Invariant values for a 2D element using 3 Stress or Strain Tensor Values (Plate Top/Bottom X Normal, Plate Top/Bottom Y Normal, Plate Top/Bottom XY Shear).

Input:

`BOOL bStress`	Flag to indicate which invariant values to compute. True = Stress, False = Strain
`REAL8 dTensor[0..2]`	Array of Stress or Strain Tensor values: 0 = Plate Top/Bottom X Normal 1 = Plate Top/Bottom Y Normal 2 = Plate Top/Bottom XY Shear

Output:

`VARIANT dInvariant`	Returns Stress of Strain invariant values in a variant: 0 = Plate Top/Bottom Major Principal 1 = Plate Top/Bottom Minor Principal 2 = Plate Top/Bottom Mean Stress 3 = Plate Top/Bottom Max Shear 4 = Plate Top/Bottom Von Mises

Return Code:

`FE_INVALID`	Values in `dTensor` must be REAL8, not INT4.

Remarks/Usage:

None

Example:

```
Sub Main

   Dim App As femap.model

   Set App = feFemap()

   Dim Tensors(2) As Double
```


Unpublished work. © 2025 Siemens

```
  Tensors(0) = 1447.109
  Tensors(1) = 61.86481
  Tensors(2) = -123.9129
   Dim inVars As Variant
   rc = App.feCompute2DInvariants (True, Tensors, inVars)
End Sub
```


Unpublished work. © 2025 Siemens

feResultsToDataTableV2

feResultsToDataTableV2

**(bOverrideLock, nCSysID, bSetInColumns, bVecInColumns, bEntInColumns, bAltColumnOrder,
nResultsSetSetID, nResultsVectorSetID, nEntitySetID)**

Description:

This function is used to list output on selected nodes or elements, for selected Output Vectors, in selected Output Sets to the Data Table. This method is obsolete and should no longer be used. It has been replaced with `feResultsToDataTable2`. This method replaces `feResultsToDataTable`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bOverrideLock`	Flag to unlock Data Table, if currently locked. True will unlock, False will keep the Data Table locked and no data will be sent. Ignored if Data Table is currently unlocked.
`INT4 nCSysID`	ID of coordinate system used to list Nodal Output. If set to -1, will use each node's Nodal Output Coordinate System.
`BOOL bSetInColumns`	When True, Output Set information will be listed in Columns. When False, Output Set information will be listed in Rows.
`BOOL bVecInColumns`	When True, Output Vector information will be listed in Columns. When False, Output Vector information will be listed in Rows.
`BOOL bEntInColumns`	When True, Node/Element information will be listed in Columns. When False, Node/Element information will be listed in Rows.
`BOOL bAltColumnOrder`	When True, will list columns in alternating order. When False, will list columns in standard order
`INT4 nResultsSetSetID`	Set ID of set containing IDs of Output Sets to list to Data Table. Alternatively, if you specify a negative value, this is simply the ID of a single Output Set to list to Data Table.
`INT4 nResultsVectorSetID`	Set ID of set containing IDs of Output Vectors to list to Data Table. Alternatively, if you specify a

Unpublished work. © 2025 Siemens

	negative value, this is simply the ID of a single output vector to list to Data Table.
`INT4 nEntitySetID`	Set ID of set containing Node or Element IDs to list to Data Table Alternatively, if you specify a negative value, this is simply the ID of a single node or element to list to Data Table.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	Data table is not open, or is locked and you did not specify `bOverrideLock=TRUE`.
`FE_NOT_EXIST`	The coordinate system specified for `nCSysID` does not exist or the set object specified for `nResultsSetSetID`, `nResultsVectorSetID`, or `nEntitySetID` is empty.
`FE_INVALID`	The values for `bSetInColumns`, `bVecInColumns`, and `bEntInColumns` are all the same. Only two of these Booleans may be set to True or set to False at one time.
`FE_BAD_DATA`	Only Nodal or Elemental Output Vectors may be sent to the Data Table when using this function, not both.
Remarks/Usage:	
This function matches the functionality of the **List → Output → Results to Data Table** command. It may be helpful to use the `AddComponentOutputVectors` (see AddComponentOutputVectorsV2), `AddSimilarOutputVectors` (see AddSimilarOutputVectorsV2), and/or `AddComplexOutputVectors` (see AddComplexOutputVectorsV2) methods on the Set Object to select all desired output vectors.	
Example:	
None	

Unpublished work. © 2025 Siemens

feResultsToDataTable2V2

feResultsToDataTable2V2

**(bOverrideLock, nViewForTransform, bSetInColumns, bVecInColumns, bEntInColumns,
bAltColumnOrder, bSummaryTable nResultsSetSetID, nResultsVectorSetID, nEntitySetID,)**

Description:

This function is used to list output on selected nodes or elements, from a selected view, for selected Output Vectors, in selected Output Sets to the Data Table. This method replaces `feResultsToDataTable2`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bOverrideLock`	Flag to unlock Data Table, if currently locked. True will unlock, False will keep the Data Table locked and no data will be sent. Ignored if Data Table is currently unlocked.
`INT4 nViewForTransform`	ID of the view to list transformed data from.
`BOOL bSetInColumns`	When True, Output Set information will be listed in Columns. When False, Output Set information will be listed in Rows.
`BOOL bVecInColumns`	When True, Output Vector information will be listed in Columns. When False, Output Vector information will be listed in Rows.
`BOOL bEntInColumns`	When True, Node/Element information will be listed in Columns. When False, Node/Element information will be listed in Rows.
`BOOL bAltColumnOrder`	When True, will list columns in alternating order. When False, will list columns in standard order.
`BOOL bSummaryTable`	When True, will list the summary table of the data selected.
`INT4 nResultsSetSetID`	Set ID of set containing IDs of Output Sets to list to Data Table. Alternatively, if you specify a negative value, this is simply the ID of a single Output Set to list to Data Table.

Unpublished work. © 2025 Siemens

`INT4 nResultsVectorSetID`	Set ID of set containing IDs of Output Vectors to list to Data Table. Alternatively, if you specify a negative value, this is simply the ID of a single output vector to list to Data Table.
`INT4 nEntitySetID`	Set ID of set containing Node or Element IDs to list to Data Table Alternatively, if you specify a negative value, this is simply the ID of a single node or element to list to Data Table.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	Data table is not open, or is locked and you did not specify `bOverrideLock=TRUE`.
`FE_NOT_EXIST`	The view specified for nViewForTransform does not exist or the set object specified for `nResultsSetSetID`, `nResultsVectorSetID`, or `nEntitySetID` is empty.
`FE_INVALID`	The values for `bSetInColumns`, `bVecInColumns`, and `bEntInColumns` are all the same. Only two of these Booleans may be set to True or set to False at one time.
`FE_BAD_DATA`	Only Nodal or Elemental Output Vectors may be sent to the Data Table when using this function, not both.
Remarks/Usage:	
This function matches the functionality of the **List → Output → Results to Data Table** command. It may be helpful to use the `AddComponentOutputVectors` (AddComponentOutputVectorsV2), `AddSimilarOutputVectors` (AddSimilarOutputVectorsV2), and/or `AddComplexOutputVectors` (AddComplexOutputVectorsV2) methods on the Set Object to select all desired output vectors.	
Example:	
None	

Unpublished work. © 2025 Siemens

feResultsRankingToDataTableV2

feResultsRankingToDataTableV2

(bOverrideLock, nRankByEntity, nMethod, nApproach, nType, nNumberToRank, nEntitySetID, nResultsSetSetID, nResultsVectorSetID)

Description:

This function is used to send a list of entities (nodes, elements, elements referencing selected materials, or elements referencing selected properties) to the Data Table, ranked using selected output values in selected output sets. This method replaces feResultsRankingToDataTable. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`BOOL bOverrideLock`	Flag to unlock Data Table, if currently locked. True will unlock, False will keep the Data Table locked and no data will be sent. Ignored if Data Table is currently unlocked.
`INT4 nRankByEntity`	The type of entity to Rank By. For entity information, see **Entity Types**. Can be FT_NODE, FT_ELEM, FT_MATL, or FT_PROP.
`INT4 nMethod`	Ranking Method. 0=Sets for Each Entity (FRKM_SETS), 1=Entities for Each Set (FRKM_ENTITY)
`INT4 nApproach`	Ranking Approach. 0=All Vectors Together (FRKA_ALL), 1=Each Vector Independently (FRKA_EACH).
`INT4 nType`	Ranking Type. 0=Max Value (FRKT_MAX), 1=Min Value (FRKT_MIN), 2=Max Absolute Value (FRKT_MAXABS).
`INT4 nNumberToRank`	Number to Rank. Must be between 1 and 99,999,999
`INT4 nEntitySetID`	Set ID of set containing node IDs when nRankByEntity is set to FT_NODE or Set ID of set containing element IDs when nRankByEntity is set to FT_ELEM, FT_PROP, or FT_MATL. Alternatively, if you specify a negative value, this is simply the ID of a single node or single element to rank.

Unpublished work. © 2025 Siemens

`INT4 nResultsSetSetID`	Set ID of set containing IDs of Output Sets to consider. Alternatively, if you specify a negative value, this is simply the ID of a single Output Set to consider.
`INT4 nResultsVectorSetID`	Set ID of set containing IDs of Output Vectors used for ranking. Alternatively, if you specify a negative value, this is simply the ID of a single output vector used for ranking.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to Rank
`FE_NOT_AVAILABLE`	Data table is not open, or is locked and you did not specify `bOverrideLock=TRUE`.
`FE_NOT_EXIST`	One or more of `nEntitySetID`, `nResultsSetSetID`, or `nResultsVectorSetID` are empty.
`FE_BAD_DATA`	`nMethod`, `nApproach` or `nType` have an invalid value.
`FE_BAD_TYPE`	`nRankByEntity` is not `FT_NODE`, `FT_ELEM`, `FT_MATL`, nor `FT_PROP`.
Remarks/Usage:	
This function matches the functionality of the **List → Output → Results to Data Table** command. It may be helpful to use the `AddComponentOutputVectors` (see AddComponentOutputVectorsV2), `AddSimilarOutputVectors` (see AddSimilarOutputVectorsV2), and/or `AddComplexOutputVectors` (see AddComplexOutputVectorsV2) methods on the Set Object to select all desired output vectors.	
Example:	
None	

Unpublished work. © 2025 Siemens

feOutputGlobalPlyV2

feOutputGlobalPlyV2 **(nSetSET, nVectorSET, nGlobalPlySET, bTop, bBottom, bGlobalPly)**	
Description:	
This method is used to create Top, Bottom, and/or Global Ply results in the selected output vectors in the specified output set. This method replaces feOutputGlobalPly. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetSET`	The ID of a Set object that contains the Output Sets to process. Alternatively, if you specify a negative number it is the ID of a single Output Set to process. You can also specify this value as zero to process all available Output Sets.
`INT4 nVectorSET`	The ID of a Set object that contains the Output Vectors to process. Alternatively, if you specify a negative number it is the ID of a single Output Vector to process. You can also specify this value as zero to process all available Output Vectors. The IDs specified here should be the IDs of existing Laminate results. You should only specify the ID from one ply for each output type.
`INT4 nGlobalPlySET`	The ID of a Set object that contains the Global Plies to process. Alternatively, if you specify a negative number it is the ID of a single Global Ply to process. You can also specify this value as zero to process all available Global Plies.
`BOOL bTop`	If True, results for the Top ply across all laminates are computed and stored.
`BOOL bBottom`	If True, results for the Bottom ply across all laminates are computed and stored.
`BOOL bGlobalPly`	If True, results for the selected Global Plys across all laminates are computed and stored.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to compute Global Ply results.
Remarks/Usage:	
Performs the same operation as running the **Model** → **Output** → **Global Ply** command.	
Example:	
None	

Model Information Methods

Unpublished work. © 2025 Siemens

feElemCountOnProp

feElemCountOnProp **(nPropSetID, vPropID, vElemCount)**	
Description:	
Returns the element count for each property in the specified set.	
Input:	
`INT4 fromSET`	The ID of set containing any number of property IDs.
Output:	
`VARIANT vPropID`	This Variant contains an array with the property IDs.
`VARIANT vElemCount`	This Variant contains an array with the element count for each property ID.
Return Code:	
`FE_FAIL`	No properties exist in the model.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Meshing Methods

Mesh Control

Unpublished work. © 2025 Siemens

feMeshSizePoint

feMeshSizePoint **(pointSET, meshSIZE)**	
Description:	
Sets mesh sizes on a selected set of points.	
Input:	
`INT4 pointSET`	The ID of a set that contains a list of point IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single point to use.
REAL8 meshSIZE	Set this to the mesh size that you want to define at the points. The size should be in model units. Specify 0.0 to remove mesh sizes from the points.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh sizes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSizeCurve

feMeshSizeCurve

**(curveSET, numElem, meshSize, minLine, minClosed, minOther, spacing, biasMethod, bias,
biasLoc, customSize)**

Description:

Sets mesh sizes on a selected set of curves.

Input:

`INT4 curveSET`	The ID of a set that contains a list of curve IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`INT4 numElem`	The number of elements to mesh along the selected curves. If 0, then `meshSize`, `minLine`, `minClosed` and `minOther` determine the number of elements. If -1, then the mesh sizes are removed from the selected curves.
`REAL8 meshSIZE`	The default mesh spacing used to compute the number of elements that will be meshed along the curve. Only used if `numElem=0`.
`INT4 minLine`	The minimum number of elements to mesh along any curve that is a straight line. If you do not want to use this feature, simply set the value to 0 (or 1). Only used if `numElem=0`.
`INT4 minClosed`	The minimum number of elements to mesh along any curve that is closed - like a circle. If you do not want to use this feature, simply set the value to 0 (or 1). Only used if `numElem=0`.
`INT4 minOther`	The minimum number of elements to mesh along any curve that is not a straight line, and is not closed - like arcs and splines. If you do not want to use this feature, simply set the value to 0 (or 1). Only used if `numElem=0`.
`INT4 spacing`	If 0, then parametric spacing is used; otherwise ,constant length spacing is used.

Unpublished work. © 2025 Siemens

`INT4 biasMethod`	0=Equal length spacing, 1=Linear Bias, 2=Geometric Bias.
`REAL8 bias`	The bias value - greater than or equal to 1.0. A value of 1.0 is not biased. A value of 2.0 means that the largest element is approximately 2.0 times the length of the smallest.
`INT4 biasLoc`	0=Small elements at start of curve, 1=Small elements at end of curve, 2=Small elements at center of curve, 3=Small elements at both ends of curve.
`BOOL customSize`	For constant length spacing, mesh sizes will be saved as a custom mesh size if this is True. Not used for parametric spacing.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh sizes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSizeCurve2

feMeshSizeCurve2

(curveSET, numElem, meshSize, minLine, minClosed, minOther, spacing, biasMethod, bias, biasLoc, customSize, bPropogateMeshSize)

Description:

Sets mesh sizes on a selected set of curves.

Input:

`INT4 curveSET`	The ID of a set that contains a list of curve IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`INT4 numElem`	The number of elements to mesh along the selected curves. If 0, then `meshSize`, `minLine`, `minClosed` and `minOther` determine the number of elements. If -1, then the mesh sizes are removed from the selected curves.
`REAL8 meshSIZE`	The default mesh spacing used to compute the number of elements that will be meshed along the curve. Only used if `numElem=0`.
`INT4 minLine`	The minimum number of elements to mesh along any curve that is a straight line. If you do not want to use this feature, simply set the value to 0 (or 1). Only used if `numElem=0`.
`INT4 minClosed`	The minimum number of elements to mesh along any curve that is closed - like a circle. If you do not want to use this feature, simply set the value to 0 (or 1). Only used if `numElem=0`.
`INT4 minOther`	The minimum number of elements to mesh along any curve that is not a straight line, and is not closed - like arcs and splines. If you do not want to use this feature, simply set the value to 0 (or 1). Only used if `numElem=0`.
`INT4 spacing`	If 0, then parametric spacing is used; otherwise ,constant length spacing is used.

Unpublished work. © 2025 Siemens

`INT4 biasMethod`	0=Equal length spacing, 1=Linear Bias, 2=Geometric Bias.
`REAL8 bias`	The bias value - greater than or equal to 1.0. A value of 1.0 is not biased. A value of 2.0 means that the largest element is approximately 2.0 times the length of the smallest.
`INT4 biasLoc`	0=Small elements at start of curve, 1=Small elements at end of curve, 2=Small elements at center of curve, 3=Small elements at both ends of curve.
`BOOL customSize`	For constant length spacing, mesh sizes will be saved as a custom mesh size if this is True. Not used for parametric spacing.
`BOOL bPropagateMeshSize`	If True, mesh sizing will be automatically propagated.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh sizes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSizeCurveMatchNodes

feMeshSizeCurveMatchNodes

(curveSetID, nodeSetID, dTolerance)

Description:

Sets custom mesh size locations on a selected set of curves using the locations of a selected set of nodes.

Input:

`INT4 curveSetID`	The ID of a set that contains a list of curve IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`INT4 nodeSetID`	The ID of a set that contains a list of node IDs. The location of each node in the set will be projected based on the Tolerance then used to specify custom mesh size locations on the selected curve(s). Alternatively, if you specify a negative value, this is simply the ID of a single node to use.
`REAL8 dTolerance`	Allows specification of a maximum distance to project the locations of the selected nodes onto the selected curves. If set to 0.0, which is the default, then the nodal locations of all selected nodes will be projected onto all of the selected curves.

Output:

None	

Return Code:

`FE_FAIL`	Could not define mesh sizes.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feMeshSizeCurveMatchXYZ

feMeshSizeCurveMatchXYZ **(curveSetID, nNumXYZ, vXYZ)**	
Description:	
Sets custom mesh size locations on a selected set of curves using specified XYZ locations.	
Input:	
`INT4 curveSetID`	The ID of a set that contains a list of curve IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`INT4 nNumXYZ`	The number of sets of XYZ coordinates to be used for custom mesh sizing locations (i.e. the number of entries in the arrays)
`REAL8 xyz[0..(3*nNumXYZ)-1]`	The coordinates of each specified location, which are projected a maximum distance based on the Tolerance, to use for custom mesh sizing locations on the selected curves. There are 3 entries, X, Y, Z for each location.
`REAL8 dTolerance`	Allows specification of a maximum distance to project the specified locations onto the selected curves. If set to 0.0, which is the default, then all of the specified locations will be projected onto all of the selected curves.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh sizes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSizePropogate

feMeshSizePropogate **(CurvesWithSizesToPropagateSET)**	
Description:	
None	
Input:	
`INT4 CurvesWithSizesToPropagateSET`	The ID of a set that contains a list of curve IDs with mesh sizes on them to then propagate to the rest of the mesh, such as connected mapped meshes.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not propagate mesh size.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSizeSurface

feMeshSizeSurface

(surfaceSET, replaceAll, meshSize, minEdge, maxAngle, maxOnSmall, smallSize, vertexAspect, mappedRefinement, growthFactor, refineRatio, refineSurf)

Description:

Sets mesh sizes on a selected set of surfaces.

Input:

`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single surface to use.
`BOOL replaceAll`	If True, then mesh sizes will be replaced on all curves of all selected surfaces. If False, then only curves with no sizes defined will be sized.
`REAL8 meshSIZE`	The default mesh spacing used to compute the number of elements that will be meshed along each curve. If you set the `meshSIZE=0.0`, then all of the other option settings except `replaceAll` are ignored. Femap automatically determines the default values, including the default mesh size. If you set `meshSIZE` to a negative value, then Femap uses the absolute value of that number as a scale factor on the default mesh size it calculates. For example `meshSIZE=-0.5` means elements that are half the size of the default, `meshSIZE=-2.0` means elements that are twice as big as the default.
`INT4 minEdge`	The minimum number of elements to mesh along any edge. Set to 0 if you do not want to specify a minimum.
`REAL8 maxAngle`	The maximum angle tolerance between the geometry tangent vector and the vector connecting consecutive mesh locations along the curves. Additional mesh locations are added along the curves to make sure this criterion is satisfied. Set to 0.0 if you do not want to use this type of mesh spacing adjustment.

Unpublished work. © 2025 Siemens

`INT4 maxOnSmall`	The maximum number of elements on any feature that is identified as small (depends on smallSize). Set to 0 if you do not want to adjust mesh sizes on small features.
`REAL8 smallSize`	The maximum size of a small feature. Set to 0.0 if you do not want to adjust mesh sizes on small features.
`REAL8 vertexAspect`	Used to adjust the mesh sizes at vertices that connect both short and long edges. Set to 0.0 if you do not want to do vertex aspect ratio adjustments. If used, mesh sizes at vertices are adjusted so that the ratio of the length of the longest element to the shortest element at a vertex is within the specified limit.
`BOOL mappedRefinement`	If True, additional modifications are done to the mesh sizes on a surface that favor mapped meshing. This does not guarantee that mapped meshing sizes will be created. If you must have a mapped mesh, you need to set that approach on the surface before you size.
`REAL8 growthFactor`	The factor that is multiplied by the average curve element size to adjust the mesh size on the interior of a surface. Set to 0.0 to skip setting interior growth factors.
`REAL8 refineRatio`	The curvature-based mesh refinement ratio. Adjusts the mesh sizes along curves in areas of high curvature. Set to 0.0 to skip curvature-based sizing.
`BOOL refineSurf`	Set to True to also consider curvature requirements when sizing the interior of the mesh. You must specify a nonzero `refineRatio`, or this option will be ignored.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh sizes.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feMeshSizeSolid

feMeshSizeSolid

(solidSET, replaceAll, meshSize, minEdge, maxAngle, maxOnSmall, smallSize, vertexAspect, mappedRefinement, growthFactor, refineRatio, refineSurf, matchAdjacent, hexMesh, adjustColor, removeSlaving)

Description:

Sets mesh sizes on a selected set of solids.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will have mesh sizes defined. Alternatively, if you specify a negative value, this is simply the ID of the single solid to use.
`BOOL replaceAll`	If True, then mesh sizes will be replaced on all curves of all selected solids. If False, then only curves with no sizes defined will be sized.
`REAL8 meshSIZE`	The default mesh spacing used to compute the number of elements that will be meshed along each curve. If you set the **meshSIZE=0.0** (or negative), then all of the other option settings except `replaceAll`, `matchAdjacent`, `hexMesh`, `adjustColor` and `removeSlaving` are ignored. Femap automatically determines the default values, including the default mesh size. If you set `meshSIZE` to a negative value, then Femap uses the absolute value of that number as a scale factor on the default mesh size it calculates. For example `meshSIZE=-0.5` means elements that are half the size of the default, `meshSIZE=-2.0` means elements that are twice as big as the default.
`INT4 minEdge`	The minimum number of elements to mesh along any edge. Set to 0 if you do not want to specify a minimum.
`REAL8 maxAngle`	The maximum angle tolerance between the geometry tangent vector and the vector connecting consecutive mesh locations along the curves. Additional mesh locations are added along the curves to make sure this criterion is satisfied.

Unpublished work. © 2025 Siemens

	Set to 0.0 if you do not want to use this type of mesh spacing adjustment.
`INT4 maxOnSmall`	The maximum number of elements on any feature that is identified as small (depends on smallSize). Set to 0 if you do not want to adjust mesh sizes on small features.
`REAL8 smallSize`	The maximum size of a small feature. Set to 0.0 if you do not want to adjust mesh sizes on small features.
`REAL8 vertexAspect`	Used to adjust the mesh sizes at vertices that connect both short and long edges. Set to 0.0 if you do not want to do vertex aspect ratio adjustments. If used, mesh sizes at vertices are adjusted so that the ratio of the length of the longest element to the shortest element at a vertex is within the specified limit.
`BOOL mappedRefinement`	If True, additional modifications are done to the mesh sizes on a surface that favor mapped meshing. This does not guarantee that mapped meshing sizes will be created. If you must have a mapped mesh, you need to set that approach on the surface before you size.
`REAL8 growthFactor`	The factor that is multiplied by the average curve element size to adjust the mesh size on the interior of a surface. Set to 0.0 to skip setting interior growth factors.
`REAL8 refineRatio`	The curvature-based mesh refinement ratio. Adjusts the mesh sizes along curves in areas of high curvature. Set to 0.0 to skip curvature-based sizing.
`BOOL refineSurf`	Set to True to also consider curvature requirements when sizing the interior of the mesh. You must specify a nonzero refineRatio or this option will be ignored.
`BOOL matchAdjacent`	Set to True to automatically find adjacent surfaces between multiple solids that you are sizing. The matched surfaces will automatically have a *Linked* approach defined.
`BOOL hexMesh`	Set to True if you want to size for hex meshing. Set to False to size for tetrahedral meshing.
`BOOL adjustColor`	Set to True if you want to change the surface colors to indicate the status and success of the mesh

Unpublished work. © 2025 Siemens

	sizing. This is especially important when sizing for hex meshing.
`BOOL removeSlaving`	Set to True to remove any previous *Linked* approaches on the surfaces of the solids that you are sizing.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh sizes.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshAttrPoint

feMeshAttrPoint **(pointSET, propID)**	
Description:	
Sets mesh attributes on a selected set of points.	
Input:	
`INT4 pointSET`	The ID of a set that contains a list of point IDs that will have mesh attributes defined. Alternatively, if you specify a negative value, this is simply the ID of the single point to use.
`INT4 propID`	Specifies the ID of the property to use as a meshing attribute at the selected points. Set `propID=0` to remove meshing attributes from the points. If nonzero, this must be the ID of a property of an element type that is compatible with point meshing - a mass element, for example.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh attribute.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshAttrCurve

feMeshAttrCurve

(curveSET, propID, Orient[0..2], Offset1[0..2], OrientType, OffsetType, Release1[0..5])

Description:

Sets mesh attributes on a selected set of curves.

Input:

`INT4 curveSET`	The ID of a set contains a list of curve IDs that will have mesh attributes defined. Alternatively, if you specify a negative value, this is simply the ID of the single curve to use.
`INT4 propID`	Specifies the ID of the property to use as a meshing attribute at the selected curves. Set `propID=0` to remove meshing attributes from the curves. If nonzero, this must be the ID of a property of an element type that is compatible with curve meshing - a beam element, for example.
`REAL8 Orient[0..2]`	The orientation vector meshing attribute. Either a location or vector (based on offset type), specified in global rectangular coordinates.
`REAL8 Offset1[0..2]`	The meshing attribute defining the offset to be applied to elements. `Offset1[0..2]` is at the beginning of the curve, `Offset2[0..2]` is at the end. Specified in global rectangular coordinates. If doing radial offsets, `Offset1[0..2]` contains the radial origin; `Offset2[0]` contains the radial offset distance.
`INT4 OrientType`	Type of orientation for property attribute: 0=Orient By Vector, 1=Orient By Location, 2=Orient By Vector, Reversed Element Direction, 3=Orient By Location, Reversed Element Direction.
`INT4 OffsetType`	Type of offset for property attribute: 0=Offset by Vector, 1=Offset Radial, 2=Offset by Location.
`INT4 Release1[0..5]`	The release flags for the start and end of the curve. Nonzero if the associated degree of freedom is released. `Release1[0..5]` is at the beginning of the curve; `Release2[0..5]` is at the end.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh attribute.
Remarks/Usage:	
Only element/property types with the appropriate orient/offset/release options available can have those options defined using this method. For example, you cannot add offsets to rod elements, because they are not available.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshAttrSurface

feMeshAttrSurface

(surfaceSET, propID, offset)

Description:

Sets mesh attributes on a selected set of surfaces.

Input:

`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will have mesh attributes defined. Alternatively, if you specify a negative value, this is simply the ID of the single surface to use.
`INT4 propID`	Specifies the ID of the property to use as a meshing attribute at the selected surfaces. Set **propID=0** to remove meshing attributes from the surfaces. If nonzero, this must be the ID of a property of an element type that is compatible with surface meshing - a planar element, for example.
`REAL8 offset`	The offset to be applied to elements meshed on this surface. Normally this is specified as 0 - the elements are meshed directly on the surface. Offsets can only be applied to certain planar elements such as plates.

Output:

None	

Return Code:

`FE_FAIL`	Could not define mesh attribute.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feMeshAttrSolid

feMeshAttrSolid

(solidSET, propID)

Description:

Sets mesh attributes on a selected set of solids or volumes.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid (or volume) IDs that will have mesh attributes defined. Alternatively, if you specify a negative value, this is simply the ID of the single solid to use.
`INT4 propID`	Specifies the ID of the property to use as a meshing attribute in the selected solids or volumes. Set `propID=0` to remove meshing attributes from the solids. If nonzero, this must be the ID of a property of an element type that is compatible with solid meshing - a solid element, for example.

Output:

None	

Return Code:

`FE_FAIL`	Could not define mesh attribute.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feMeshApproachSurface

feMeshApproachSurface **(surfaceSET, approach, approachData)**	
Description:	
Sets the meshing approach on a selected set of surfaces.	
Input:	
`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will have mesh approaches defined. Alternatively, if you specify a negative value, this is simply the ID of the single surface to use.
`INT4 approach`	The meshing approach for the surface: 0=Not Specified, 1=Free-Parametric, 2=Free-Planar, 3=Mapped-Four Corner, 4=Mapped-Three Corner, 5=Mapped-Three Corner Fan, 6=Link to Surface, 7=Fast Tri-Parametric, 8=Fast Tri-Planar.
`INT4 approachData[0..3]`	The data for various approaches. For the mapped approaches, these are the point IDs defining the *corners* of the mapped mesh. For the Link to Surface approach, `approachData[0]` is the ID of the surface that this surface is linked to.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define mesh approach.
Remarks/Usage:	
None	
Example:	
None	

Meshing Geometry

Unpublished work. © 2025 Siemens

feMeshPoint

feMeshPoint

(pointSET, meshElem, propID)

Description:

Meshes a selected set of points. This method is obsolete an should no longer be used. It has been replaced with `feMeshPoint2`.

Input:

`INT4 pointSET`	The ID of a set that contains a list of point IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single point to mesh.
`BOOL meshElem`	If True, then nodes and elements will be generated on the selected points. If False, only nodes will be generated.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected points must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with point meshing.

Output:

None	

Return Code:

`FE_FAIL`	Could not mesh points.

Remarks/Usage:

None

Example:

The following example meshes each point in a model with a mass element. The total mass added to all points is 25 pounds.

```
Sub Main
Dim App As femap.model
Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
Dim rc As Long

Dim pr as femap.Prop

Set pr = App.feProp


Dim pt As femap.Set
Set pt = App.feSet()

pt.AddAll (3)   'Pick all points
pr.Type = 27     ' Mass Type
pr.Title = "25 Lb. Mass"
pr.pval(7) = 25 / pt.Count()   'Total=25 pound/Number of Points
pr.Put (pr.NextEmptyID)         ' Create new property
rc = App.feMeshPoint(pt.ID, True, pr.ID)  'Mesh the points

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshPoint2

feMeshPoint2

(pointSET, meshElem, propID, merge_nodes)

Description:

Meshes a selected set of points.

Input:

`INT4 pointSET`	The ID of a set that contains a list of point IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single point to mesh.
`BOOL meshElem`	If True, then nodes and elements will be generated on the selected points. If False, only nodes will be generated.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected points must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with point meshing.
`BOOL merge_nodes`	If True, coincident nodes on point will be merged.

Output:

None	

Return Code:

`FE_FAIL`	Could not mesh points.

Remarks/Usage:

None

Example:

The following example meshes each point in a model with a mass element. The total mass added to all points is 25 pounds.

```
Sub Main
Dim App As femap.model
```


Unpublished work. © 2025 Siemens

```
Set App = feFemap()


Dim rc As Long

Dim pr as femap.Prop

Set pr = App.feProp


Dim pt As femap.Set
Set pt = App.feSet()

pt.AddAll (3)   'Pick all points
pr.Type = 27     ' Mass Type
pr.Title = "25 Lb. Mass"
pr.pval(7) = 25 / pt.Count()   'Total=25 pound/Number of Points
pr.Put (pr.NextEmptyID)         ' Create new property
rc = App.feMeshPoint(pt.ID, True, pr.ID, TRUE)  'Mesh the points

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshCurve

feMeshCurve **(curveSET, meshElem, propID, orient)**	
Description:	
Meshes a selected set of curves. This method is obsolete and should no longer be used. It has been replaced by `feMeshCurve2`.	
Input:	
`INT4 curveSET`	The ID of a set that contains a list of curve IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single curve to mesh.
`BOOL meshElem`	If True, then nodes and elements will be generated on the selected curves. If False, only nodes will be generated.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected curves must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with curve meshing. Set `propID=-1` to automatically mesh with plot-only elements, which do not require other property information.
`REAL8 orient[0..2]`	The components of the element orientation vector, in global rectangular coordinates. This is only used for those element types that require orientation - like bars and beams.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh curves.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

The following example meshes each curve in a model with bar elements. It establishes a mesh size of 0.1 along each curve, creates a property, and meshes the curves.

```
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim rc As Long


Dim pr As femap.Prop
Set pr = App.feProp()


Dim pt As femap.Set
Set pt = App.feSet()


Dim orient(2) As Double
Dim vOrient As Variant

orient(0) = 1
orient(1) = 1
orient(2) = 0.5
vOrient = orient

pt.AddAll (4)
pr.Type = 2
pr.Title = "Bar"
pr.pval(0) = 0.125
pr.pval(1) = 10.3
pr.pval(2) = 0.5
pr.Put (pr.NextEmptyID)

rc = App.feMeshSizeCurve(pt.ID,0,0.1,0,0,0,0,0,1#,0,False)
rc = App.feMeshCurve(pt.ID, True, pr.ID, vOrient)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshCurve2

feMeshCurve2 **(curveSET, meshElem, propID, merge_nodes, offsetToRefPt, orient)**	
Description:	
Meshes a selected set of curves.	
Input:	
`INT4 curveSET`	The ID of a set that contains a list of curve IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single curve to mesh.
`BOOL meshElem`	If True, then nodes and elements will be generated on the selected curves. If False, only nodes will be generated.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected curves must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with curve meshing. Set `propID=-1` to automatically mesh with plot-only elements, which do not require other property information.
`BOOL merge_nodes`	If True, coincident nodes on curve will be merged.
`BOOL offsetToRefPt`	If True, elements are offset to beam property reference point.
`REAL8 orient[0..2]`	The components of the element orientation vector, in global rectangular coordinates. This is only used for those element types that require orientation - like bars and beams.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh curves.

Unpublished work. © 2025 Siemens

Remarks/Usage:

None

Example:

The following example meshes each curve in a model with bar elements. It establishes a mesh size of 0.1 along each curve, creates a property, and meshes the curves.

```
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim rc As Long


Dim pr As femap.Prop
Set pr = App.feProp()


Dim pt As femap.Set
Set pt = App.feSet()


Dim orient(2) As Double
Dim vOrient As Variant

orient(0) = 1
orient(1) = 1
orient(2) = 0.5
vOrient = orient

pt.AddAll (4)
pr.Type = 2
pr.Title = "Bar"
pr.pval(0) = 0.125
pr.pval(1) = 10.3
pr.pval(2) = 0.5
pr.Put (pr.NextEmptyID)

rc = App.feMeshSizeCurve(pt.ID,0,0.1,0,0,0,0,0,1#,0,False)
rc = App.feMeshCurve(pt.ID, True, pr.ID, TRUE, TRUE, vOrient)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshSurface2

feMeshSurface2 **(surfaceSET, propID, elemShape, bAllowMapped, beDefaults)**	
Description:	
Meshes a selected set of surfaces. This function replaces `feMeshSurface( )`.	
Input:	
`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single surface to mesh.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use predefined meshing attributes, set `propID=0`. In this case, all selected surfaces must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with surface meshing.
`INT4 elemShape`	The shape of the elements to mesh. Must be 2=Tria3, 3=Tria6, 4=Quad4, 5=Quad8 or set to 0 to use the shape that is predefined on the meshing attributes.
`BOOL bAllowMapped`	If True, mapped meshing of the surfaces is allowed, otherwise it is not. This argument is ignored if `elemShape=0`
`BOOL bResetDefaults`	If True, all meshing attributes are reset to their defaults prior to applying the `propID`, `elemShape` and `bAllowMapped` settings. Normally this argument should be False to use the existing attributes.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh surfaces.

Unpublished work. © 2025 Siemens

FE_INVALID	The value specified for elemShape is invalid
FE_NOT_EXIST	The property specified does not exist.

Remarks/Usage:

None

Example:

The following example meshes each surface on Solid 1 in a model. It requires the mesh size to be already established, but it creates a plate property with a 0.1 thickness and meshes the surfaces with quads.

```
Sub Main
Const FGD_Surface_onSolid = 78

Dim App As femap.model
Set App = feFemap()


Dim rc As Long
Dim pr As femap.Prop
Set pr = App.feProp()


Dim pt As femap.Set
Set pt = App.feSet()

rc = pt.AddRule(1, FGD_Surface_onSolid)
pr.Type = 17
pr.Title = "0.1 Plate"
pr.pval(0) = 0.1
pr.Put (pr.NextEmptyID)

rc=App.feMeshSurface2(pt.ID,pr.ID,FTO_QUAD4,True,False)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshSurface - obsolete

feMeshSurface

(surfaceSET, propID, minBetween, maxAspect, maxQuickNodes, meshApproach, quadAngle, midsideGeom, midsideAngle, smoothLaplacian, smoothIter, smoothTol)

Description:

Meshes a selected set of surfaces.

Input:

`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single surface to mesh.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected surfaces must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with surface meshing. Set `propID=-1` to automatically mesh with plot-only elements, which do not require other property information.
`INT4 minBetween`	The minimum number of elements between boundaries during mesh generation. Not used for fast-tri mesher.
`REAL8 maxAspect`	The maximum allowable element aspect ratio. Not used for fast-tri mesher.
`INT4 maxQuickNodes`	Boundaries/surfaces with more than this number of nodes on the boundary are *quick-cut*. Not used for fast-tri mesher.
`INT4 meshApproach`	The mesher to use when meshing the surfaces. (0=Triangle, 1=Quad, 2=Fast-Triangle).
`REAL8 quadAngle`	The maximum angular deviation from 90 degrees allowed in a quad before it is split into triangles. Only applies to quad mesher.

Unpublished work. © 2025 Siemens

`BOOL midsideGeom`	If True and you are meshing with parabolic elements, midside nodes will be projected toward the geometry.
`REAL8 midsideAngle`	The maximum angle formed by the edge of the element as the midside nodes are projected onto the geometry. If `midsideAngle=0.0` and `midsideGeom=True`, then midside nodes will be projected completely onto the geometry regardless of the distortion introduced in the elements.
`BOOL smoothLaplacian`	If True, Laplacian smoothing is performed. If False, centroidal smoothing is used.
`INT4 smoothIter`	The number of smoothing iterations to perform. Set to 0 to skip smoothing.
`INT4 smoothTcol`	The smoothing tolerance. Smoothing will stop if node movement during a smoothing pass falls below this tolerance.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh surfaces.
Remarks/Usage:	

This function has been replaced by `feMeshSurface2( )`

Nodes along common edges between surfaces meshed in a single call to this method will be merged. If you call this method multiple times, nodes along common edges of the surfaces meshed in different calls will not be merged - you must do that manually by calling `feCheckCoincidentNode`.

Example:

The following example meshes each surface on Solid 1 in a model. It requires the mesh size to be already established, but it creates a plate property with a 0.1 thickness and meshes the surfaces with quads. The software performs 20 Laplacian smoothing passes, or continues until the smoothing converges to a 0.01 tolerance.

```
Sub Main
Const FGD_Surface_onSolid = 78

Dim App As femap.model
Set App = feFemap()
Dim rc As Long
Dim pr As femap.Prop
Set pr = App.feProp()
Dim pt As femap.Set
```


Unpublished work. © 2025 Siemens

```
Set pt = App.feSet()

rc = pt.AddRule(1, FGD_Surface_onSolid)
pr.Type = 17
pr.Title = "0.1 Plate"
pr.pval(0) = 0.1
pr.Put (pr.NextEmptyID)

rc=App.feMeshSurface(pt.ID,pr.ID,1,2,30,1,45,0,0,True,20,.01)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshSurfaceByAttributes

feMeshSurfaceByAttributes **(surfaceSET)**	
Description:	
Meshes a selected set of surfaces using its stored attributes.	
Input:	
`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that will be meshed using their saved attributes. Alternatively, if you specify a negative value, this is simply the ID of the single surface to mesh.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh surfaces.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshTetSolid3

feMeshTetSolid3

**(entityTYPE, entitySET, propID, meshingApproach, makeMidside, tetGrowth, checkSurfElem, deleteSurfElem, midsideGeom, midsideAngle, multiThruThickness,
sliverRemoval, tetOptimization, pyramidSurfaceSET,
pyramidMatchAdjacentParabolic)**

Description:

Meshes a selected set of solids (or surfaces or planar elements) with tetrahedral and, optionally, pyramid elements. This method supersedes `feMeshTetSolid2` and `feMeshTetSolid`.

Input:

`INT4 entityTYPE`	The type of entities being meshed that are contained in `entitySET`. For entity information, see **Entity Types**. Can be `FT_SOLID`, `FT_SURFACE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of solid/surface/element IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single entity to mesh - probably only for solids. In any case, the list of entities must form one or more closed solid volumes.
`INT4 propID`	Specifies the ID of the property to use for tetrahedral elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected solids must have meshing attributes defined or they will not be meshed. This only works if `entityTYPE=FT_SOLID`. The `propID` property must be compatible with solid meshing.
`INT4 meshingApproach`	Set to one of three values: 0=Surface Mesh Only, 1=Mesh Volume with Tets Only, 2=Mesh Volume with Tets and Pyramids. When using option 2, a layer of pyramids will be created anywhere the volume is bounded by quadrilateral surface elements and/or on surface IDs found in the `pyramidSurfaceSET`.

Unpublished work. © 2025 Siemens

`BOOL makeMidside`	If True, the solids will have midside nodes created. Otherwise, linear elements are created.
`REAL8 tetGrowth`	The tetrahedral growth factor. Numbers larger than 1.0 mean that the elements on the inside of the solid will be larger than those on the outside. It is usually best to use a factor slightly larger than 1.0. Default value is 1.1 - if you specify `tetGrowth=0.0`.
`BOOL checkSurfElem`	Checks the surface elements for validity prior to tet meshing if True. Not usually required if meshing solids, but should be done for surface or element meshing.
`BOOL deleteSurfElem`	Set to True if you want to delete the surface elements that are being generated or meshed.
`BOOL midsideGeom`	If True, and you are meshing with parabolic elements, midside nodes will be projected toward the geometry.
`REAL8 midsideAngle`	The maximum angle formed by the edge of the element as the midside nodes are projected onto the geometry. If `midsideAngle=0.0` and `midsideGeom=True`, then midside nodes will be projected completely onto the geometry regardless of the distortion introduced in the elements.
`BOOL multiThruThickness`	If True, mesher will place multiple tet elements through the thickness of the solid. Default should be set to False.
`BOOL sliverRemoval`	If True, processing will be done to remove sliver elements. Should always be set to True unless the mesher encounter issues.
`INT4 tetOptimization`	Sets the Tet Optimization level. Available values are 0-5, which 0=off, 1=less optimization, 5=more optimization, 3=default.
`INT4 pyramidSurfaceSET`	Only used if `meshingApproach=2` and `entityType` is not `FT_ELEM`. The ID of a Set object that contains the IDs of surfaces to mesh with quadrilateral elements used to then create pyramid elements. Any surfaces that were already meshed prior to this call do not need to be included. Also, if all desired surfaces are adjacent to or linked to quad-meshed surfaces, or have quad meshing attributes, then specify `pyramidSurfaceSET=0`. If you specify any other

Unpublished work. © 2025 Siemens

	ID, the Set object must contain all unmeshed surfaces where you want pyramids.
`BOOL pyramidMatchAdjacentParabolic`	If True, the bases of the Pyramids will reference midside nodes. If False, bases of the Pyramids will not have midside nodes.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshTetSolid2

feMeshTetSolid2

(entityTYPE, entitySET, propID, surfaceMeshOnly, makeMidside, tetGrowth, checkSurfElem, deleteSurfElem, midsideGeom, midsideAngle, multiThruThickness, sliverRemoval, tetOptimization)

Description:

Meshes a selected set of solids (or surfaces or planar elements) with tetrahedral elements. This method supersedes `feMeshTetSolid`, but has also been superseded by `feMeshTetSolid3`.

Input:

`INT4 entityTYPE`	The type of entities being meshed that are contained in `entitySET`. For entity information, see **Entity Types**. Can be `FT_SOLID`, `FT_SURFACE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of solid/surface/element IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single entity to mesh - probably only for solids. In any case, the list of entities must form one or more closed solid volumes.
`INT4 propID`	Specifies the ID of the property to use for tetrahedral elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected solids must have meshing attributes defined or they will not be meshed. This only works if `entityTYPE=FT_SOLID`. The `propID` property must be compatible with solid meshing.
`BOOL surfaceMeshOnly`	If True, only surface elements are generated, and the meshing process stops before generating tetrahedra.
`BOOL makeMidside`	If True, the solids will have midside nodes created. Otherwise, linear elements are created.
`REAL8 tetGrowth`	The tetrahedral growth factor. Numbers larger than 1.0 mean that the elements on the inside of the solid will be larger than those on the outside. It is usually best to use a factor slightly larger

Unpublished work. © 2025 Siemens

	than 1.0. Default value is 1.5 - if you specify `tetGrowth=0.0`.
`BOOL checkSurfElem`	Checks the surface elements for validity prior to tet meshing if True. Not usually required if meshing solids, but should be done for surface or element meshing.
`BOOL deleteSurfElem`	Set to True if you want to delete the surface elements that are being generated or meshed.
`BOOL midsideGeom`	If True, and you are meshing with parabolic elements, midside nodes will be projected toward the geometry.
`REAL8 midsideAngle`	The maximum angle formed by the edge of the element as the midside nodes are projected onto the geometry. If `midsideAngle=0.0` and `midsideGeom=True`, then midside nodes will be projected completely onto the geometry regardless of the distortion introduced in the elements.
`BOOL multiThruThickness`	If True, mesher will place multiple tet elements through the thickness of the solid. Default should be set to False.
`BOOL sliverRemoval`	If True, processing will be done to remove sliver elements. Should always be set to True unless the mesher encounter issues.
`INT4 tetOptimization`	Sets the Tet Optimization level. Available values are 0-5, which 0=off, 1=less optimization, 5=more optimization, 3=default.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshTetSolid

feMeshTetSolid

**(entityTYPE, entitySET, propID, surfaceMeshOnly, makeMidside, tetGrowth, checkSurfElem,
deleteSurfElem, midsideGeom, midsideAngle, allocateElem)**

Description:

Meshes a selected set of solids (or surfaces or planar elements) with tetrahedral elements. This method has been superseded by `feMeshTetSolid2` and `feMeshTetSolid3`.

Input:

`INT4 entityTYPE`	The type of entities being meshed that are contained in `entitySET`. For entity information, see **Entity Types**. Can be `FT_SOLID`, `FT_SURFACE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of solid/surface/element IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single entity to mesh - probably only for solids. In any case, the list of entities must form one or more closed solid volumes.
`INT4 propID`	Specifies the ID of the property to use for tetrahedral elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected solids must have meshing attributes defined or they will not be meshed. This only works if `entityTYPE=FT_SOLID`. The `propID` property must be compatible with solid meshing.
`BOOL surfaceMeshOnly`	If True, only surface elements are generated, and the meshing process stops before generating tetrahedra.
`BOOL makeMidside`	If True, the solids will have midside nodes created. Otherwise, linear elements are created.
`REAL8 tetGrowth`	The tetrahedral growth factor. Numbers larger than 1.0 mean that the elements on the inside of the solid will be larger than those on the outside. It is usually best to use a factor slightly larger than 1.0. Default value is 1.5 - if you specify `tetGrowth=0.0`.

Unpublished work. © 2025 Siemens

`BOOL checkSurfElem`	Checks the surface elements for validity prior to tet meshing if True. Not usually required if meshing solids, but should be done for surface or element meshing.
`BOOL deleteSurfElem`	Set to True if you want to delete the surface elements that are being generated or meshed.
`BOOL midsideGeom`	If True, and you are meshing with parabolic elements, midside nodes will be projected toward the geometry.
`REAL8 midsideAngle`	The maximum angle formed by the edge of the element as the midside nodes are projected onto the geometry. If `midsideAngle=0.0` and `midsideGeom=True`, then midside nodes will be projected completely onto the geometry regardless of the distortion introduced in the elements.
`INT4 allocateElem`	Estimated maximum number of tetrahedral elements to be generated. This option simply controls memory allocation for the mesher - more elements may be generated. You should usually set this value to 0. This will cause the mesher to automatically estimate the memory requirements from the surface mesh.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh solid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshHexSolid

feMeshHexSolid

(solidSET, propID, surfaceMeshOnly, makeMidside, checkSurfElem, deleteSurfElem, midsideGeom, midsideAngle)

Description:

Meshes a selected set of solids with hexahedral elements.

Input:

`INT4 solidSET`	The ID of a set that contains a list of solid IDs that will be meshed. Alternatively, if you specify a negative value, this is simply the ID of the single entity to mesh.
`INT4 propID`	Specifies the ID of the property to use for hexahedral elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected solids must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with solid meshing.
`BOOL surfaceMeshOnly`	If True, only surface elements are generated, and the meshing process stops before generating hex elements.
`BOOL makeMidside`	If True, the solids will have midside nodes created. Otherwise, linear elements are created.
`BOOL checkSurfElem`	Checks the surface elements for validity prior to tet meshing if True. Not usually required if meshing solids, but should be done for surface or element meshing.
`BOOL deleteSurfElem`	Set to True if you want to delete the surface elements that are being generated or meshed.
`BOOL midsideGeom`	If True and you are meshing with parabolic elements, midside nodes will be projected toward the geometry.
`REAL8 midsideAngle`	The maximum angle formed by the edge of the element as the midside nodes are projected onto the geometry. If `midsideAngle=0.0` and `midsideGeom=True`, then midside nodes will be

Unpublished work. © 2025 Siemens

	projected completely onto the geometry regardless of the distortion introduced in the elements.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh solid.
Remarks/Usage:	
For this function to work, the solids must be compatible with, and properly sized for hex meshing.	
Example:	

This example hex meshes all of the solids in a model with parabolic hex elements and projects the midside nodes onto the surfaces.

```
Sub Main

Dim App As femap.model
Set App = feFemap()


Dim so As femap.Set
Set so = App.feSet()

so.AddAll (39)


Dim pr as femap.Prop
Set pr = App.feProp()


pr.Type = 25
pr.Title = "Bricks"
pr.Put (pr.NextEmptyID)

rc=femap.feMeshHexSolid(so.ID,pr.ID, _
                        False,True,False,True,True,0#)

App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshHexSolidFromElement

feMeshHexSolidFromElement

(baseSET, topSET, propID, numLayers, match1, match2, makeMidside, checkSurfElem, deleteSurfElem)

Description:

Meshes selected sets of surface elements with hexahedral elements.

Input:

`INT4 baseSET`	The ID of a set that contains a list of surface element IDs that will be used as the *base* of the hex mesh. The *base* of the hex mesh is the set of elements that form the pattern that will be extruded into the hex elements.
`INT4 topSET`	The ID of a set that contains a second list of surface element IDs that will be used as the *top* of the hex mesh. The *top* set can be selected in two forms. It can either contain a set of elements that match the base but is located at the opposite end of the extrusion, or it can contain all of the elements (other than those in the base) that enclose a complete, closed volume. If you use the true *top* option (not all the enclosing elements), then the base may not contain any holes.
`INT4 propID`	Specifies the ID of the property to use for hexahedral elements as they are created. If you want to use meshing attributes, set `propID=0`. In this case, all selected solids must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with solid meshing.
`INT4 numLayers`	Only used if `topSET` only includes the elements that match `baseSET` at the other end of the extrusion. This is the number of layers of hexahedral elements that will be generated.
`INT4 match1` `INT4 match2`	Only used if `topSET` only includes the elements that match `baseSET` at the other end of the extrusion. These should be both set to 0 if you want to automatically match the element patterns in the base and top meshes. If you have a symmetric or rotated mesh, it may be necessary

Unpublished work. © 2025 Siemens

	to set these values to node IDs that will match the base mesh to the top. If not 0, match1 must be a node ID from the `baseSET` that matches the `match2 nodeID` from the `topSET`.
`BOOL makeMidside`	If True, the solids will have midside nodes created. Otherwise, linear elements are created.
`BOOL checkSurfElem`	Checks the surface elements for validity prior to tet meshing if True. Not usually required if meshing solids, but should be done for surface or element meshing.
`BOOL deleteSurfElem`	Set to True if you want to delete the surface elements that are being generated or meshed.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh solid.
Remarks/Usage:	
None	
Example:	

This example hex meshes between the surface mesh that is on Surface 1 and the mesh on Surface 5. It creates 17 layers of hex elements between the surfaces.

```
Sub Main
Const FGD_Elem_atSurface = 44

Dim App As femap.model
Set App = feFemap()


Dim b As femap.Set
Dim t As femap.Set
Set b = App.feSet()
Set t = App.feSet()

iSurf1 = 1
iSurf2 = 5
nLayers = 17

rc = b.AddRule(iSurf1, FGD_ELEM_ATSURFACE)
rc = t.AddRule(iSurf2, FGD_ELEM_ATSURFACE)


Dim pr as femap.Prop
```


Unpublished work. © 2025 Siemens

```
Set pr = App.feProp()


pr.type = 25
pr.title = "Bricks"
pr.Put (pr.NextEmptyID)
rc=App.feMeshHexSolidFromElement(b.ID,t.ID,pr.ID, _
       nLayers,0,0,True,False,True)App.feViewRegenerate(0)

End Sub
```

Mesh Editing

Unpublished work. © 2025 Siemens

feMeshEdgeSplit

feMeshEdgeSplit

(nNode1, nNode2, nNodeSetorZero)

Description:

Splits a mesh by choosing an element edge.

Input:

`INT4 nNode1`	The IDs of two nodes that lie on an element edge that is to be split. Only line, planar and solid (hex and wedge) elements can be split.
`INT4 nNode2`	
`INT4 nNodeSetorZero`	The ID of a set containing node IDs that limit the split to a specified region of the model. If you want the split to go thru the entire model simply specify 0.

Output:

None	

Return Code:

`FE_FAIL`	Split failed.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feMeshEdgeSplit2

feMeshEdgeSplit2

(nNode1, nNode2, nNumSplits, nBiasType, nBiasLocation, dBias, nNodeSetorZero)

Description:

Splits a mesh by choosing an element edge and allows you to enter a number of splits, bias type, bias location, and bias value.

Input:

INT4 nNode1	The IDs of two nodes that lie on an element edge that is to be split. Only line, planar and solid (hex and wedge) elements can be split.
INT4 nNode2	
INT4 nNumSplits	Specifies the number of splits between the two nodes (i.e., a value of 2 will split each element into three elements).
INT4 nBiasType	Sets the option for Bias Type, 0=Equal (i.e., no biasing), 1=Biased, 2=Geometric Bias.
INT4 nBiasLocation	Sets the option for Bias Location. Only used when nBiasType is set to 1 or 2, 0=At Start (smaller elements closer to nNode1), 1=At Center, 2=At Ends.
REAL8 dBias	Specifies the Bias Factor. Only used when nBiasType is set to 1 or 2, value must be between 1.0 and 65535.0
INT4 nNodeSetorZero	The ID of a set containing node IDs that limit the split to a specified region of the model. If you want the split to go thru the entire model simply specify 0.

Output:

None	

Return Code:

FE_FAIL	Split failed.

Remarks/Usage:

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feMeshEditingSplit

feMeshEditingSplit

(INT4 nElemSet, INT4 nSplitMode, REAL8 dWarping, BOOL bMergeNodes, BOOL bOntoGeometry, REAL8 dMaxAngle)

Description:

Splits selected linear or parabolic quad or triangular elements into smaller quads or triangles based on a defined split pattern.

Input:

`INT4 nElemSet`	The ID of a Set that contains the IDs of elements to Split. Can be a negative number to split a single element.
`INT4 nSplitMode`	0=QUAD->2QUAD, 1 = QUAD->4QUAD, 2 = QUAD->2TRI, 3 = QUAD->3TRI, 4 = TRI->4TRI, 5=TRI->3QUAD
`REAL8 dWarping`	Only used with QUAD->2TRI. Only warped quads that are warped greater than the specified amount will be split.
`BOOL bMergeNodes`	If True, new nodes will be merged with the rest of the model.
`BOOL bOntoGeometry`	If True, new nodes will be projected onto any associated geometry.
`REAL8 dMaxAngle`	If `bOntoGeometry` is True, this limits the distance that nodes can be projected toward the geometry based on the angle of warping the element edges.

Output:

None	None

Return Code:

`FE_INVALID`	invalid Split Mode specified.
`FE_NOT_AVAILABLE`	No elements specified to split.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Other Meshing

Unpublished work. © 2025 Siemens

feMeshBetween

feMeshBetween

**(propID, genMode, shape, rightSplit, alternate, allQuad, logSpacing, numCorners, numNodes,
bias, nodeInc, cornerXYZ)**

Description:

Meshes between selected coordinate locations. No geometry is required.

Input:

INT4 propID	Specifies the ID of the property to use for elements as they are created. The propID property must be compatible with the type of elements that you will be creating, and the *shape* value.
INT4 genMode	The mesh generation mode: 0=Nodes Only, 1=Elements only on existing Nodes, 2=Both Nodes and Elements.
INT4 shape	The shape of the elements to generate: 0=Line, 2=Tria, 4=Quad, 6=Tetra, 7=Wedge, 8=Brick. Not used if only generating nodes.
BOOL rightSplit	Only used for Tria,Tetra and Wedge shapes. If True, indicates that the first elements should be split along a right-hand diagonal.
BOOL alternate	Only used for Tria,Tetra and Wedge shapes. If True, indicates that the element split direction should alternate with each group of elements, eliminating any biasing in the mesh patterns.
BOOL allQuad	Indicates that you want to use the alternate all-quad meshing scheme on a mesh that would otherwise generate triangular or wedge elements. This also requires compatible numbers of nodes along all edges.
BOOL logSpacing	Only used if you are generating nodes (or nodes and elements), and using a bias factor other than 1.0. If True, indicates that you want the mesh spacing graded in a logarithmic manner.
INT4 numCorners	The number of corners in the meshing region. Available values are: 2 (for line elements), 3 or

Unpublished work. © 2025 Siemens

	4 (for planar elements), 4,5,6, or 8 for (solid elements).
`INT4 numNodes[0..2]`	The number of nodes in each of the meshing directions. Only the entries required for the number of corners are used. For example, if you specify 2 corners, only `numNodes[0]` is used. If you specify 3 corners, `numNodes[0]` and `numNodes[1]` will be used.
`REAL8 bias[0..2]`	The biasing factor in each of the meshing directions. Set to 1.0 for no biasing.
`INT4 nodeINC[0..2]`	Only used if `genMode=1` (generate elements only). In this case, `nodeINC[0]` must be the ID of the first node in the pattern to be meshed with elements. `nodeINC[1]` and `nodeINC[2]` are the additional node increments in second and third (if required) meshing directions. These will be 0, if the nodes are numbered consecutively.
`REAL8 cornerXYZ[0..23]`	The coordinates of the corner locations, in global rectangular coordinates. Each corner is specified in consecutive locations in this array. For example, the first corner is in `cornerXYZ[0..2]`, the second is in `cornerXYZ[3..5]`, the third in cornerXYZ[6..8] and so on.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh between corners.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshClosestLink

feMeshClosestLink **(fromSET, toSET, connectWith, orient, dof, reverse)**	
Description:	
Meshes connections based on the shortest distance between two sets of nodes. This method provides programmatic access to the functionality of the Mesh, Connect, Closest Link command.	
Input:	
`INT4 fromSET`	A set containing the node IDs of the nodes to mesh from. Each node in this set will have one element connecting from it to the closest node in the `toSET`. Alternatively, if you specify a negative value, this is simply the ID of the single node to mesh.
`INT4 toSET`	A set containing the node IDs to mesh *to*. The closest node from this set will be matched with the node from the `fromSET`. Alternatively, if you specify a negative value, this is simply the ID of the single node to mesh.
`INT4 connectWith`	The type of element to use for the connection: 1=Constraint Equation, 2=Rigid, 3=Line Element. If you choose constraint equations, you must have an active constraint set for this method to work.
`REAL8 orient[0..2]`	The orientation vector used for elements created with this method (if required by `connectWith`).
`INT4 dof[0..5]`	The degrees of freedom (DOF) to connect. Set values to 1 for DOF to connect; 0 for DOF to skip.
`BOOL reverse`	Reverses the connection order: connections are still made from the `fromSET` to the `toSET`, but the element nodal order is reversed.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh.

Unpublished work. © 2025 Siemens

Remarks/Usage:
When `connectWith` is set to 3 (Line Elements) the Line Orientation option is always set to Vector. If you want to simulate the same capability found in the Multiple Link command, simply call this function multiple times, using negative `nodeIDs` in place of the `fromSET` and `toSET`.
Example:
None

Unpublished work. © 2025 Siemens

feMeshCoincidentLink

`feMeshCoincidentLink`

`( fromSET, toSET, connectWith, orient, dof, tolerance )`

Description:

Meshes connections between coincident nodes. This method provides programmatic access to the functionality of the **Mesh** → **Connect** → **Coincident Link** command.

Input:

`INT4 fromSET`	A set containing the node IDs of the nodes to mesh. Only nodes in this set that are coincident (within the specified tolerance) with another node in the set are meshed.
`INT4 connectWith`	The type of element to use for the connection: 1=Constraint Equation, 2=Rigid, 3=Line Element, 4=Gap Element. If you choose constraint equations, you must have an active constraint set for this method to work.
`REAL8 orient[0..2]`	The orientation vector used for elements created with this method (if required by `connectWith`).
`INT4 dof[0..5]`	The degrees of freedom (DOF) to connect. Set values to 1 for DOF to connect; 0 for DOF to skip.
`REAL8 tolerance`	The coincident node tolerance. Only nodes within this tolerance of each other are meshed.

Output:

None	

Return Code:

`FE_FAIL`	Could not mesh.

Remarks/Usage:

When `connectWith` is set to 3 (Line Elements) or 4 (Gap Elements) the Line Orientation option is always set to Vector.

Example:

None

Unpublished work. © 2025 Siemens

feMeshUnzip

feMeshUnzip

(fromSET, toSET, connectWith, orient, dof)

Description:

Unzips an existing mesh and places connecting elements at the new, unzipped coincident nodes. This call is obsolete, use `feMeshUnzip2` instead.

Input:

`INT4 fromSET`	A set containing the element IDs on the primary side of the interface. Alternatively, if you specify a negative value, this is simply the ID of the single element to use.
`INT4 toSET`	A set containing the node IDs to unzip. Each of these nodes will be duplicated and nodes on the elements that were not selected as the “primary side” will be replaced with the copies. Connecting elements will be created between the original and new nodes.
`INT4 connectWith`	The type of element to use for the connection: 0=None, 1=Constraint Equation, 2=Rigid, 3=DOF Spring, 4=Gap Element, 5=Spring/Damper. If you choose constraint equations, you must have an active constraint set for this method to work. If you choose None, the mesh will be unzipped, but no connection elements will be created.
`REAL8 orient[0..2]`	The orientation vector used for elements created with this method (if required by connectWith).
`INT4 dof[0..5]`	The degrees of freedom (DOF) to connect. Set values to 1 for DOF to connect; 0 for DOF to skip.

Output:

None	

Return Code:

`FE_FAIL`	Could not mesh.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feMeshUnzip2

feMeshUnzip2

(fromSET, toSET, connectWith, orientMode, orientID, orientVec, dof, rigidReverse)

Description:

Unzips an existing mesh and places connecting elements at the new, unzipped coincident nodes. This call supersedes `feMeshUnzip` and provides programmatic access to the functionality of the **Mesh** → **Connect** → **Unzip** command.

Input:

`INT4 fromSET`	A set containing the element IDs on the primary side of the interface. Alternatively, if you specify a negative value, this is simply the ID of the single element to use.
`INT4 toSET`	A set containing the node IDs to unzip. Each of these nodes will be duplicated and nodes on the elements that were not selected as the *primary side* will be replaced with the copies. Connecting elements will be created between the original and new nodes.
`INT4 connectWith`	The type of element to use for the connection: 0=None, 1=Constraint Equation, 2=Rigid, 3=DOF Spring, 4=Gap Element, 5=Spring/Damper. If you choose constraint equations, you must have an active constraint set for this method to work. If you choose None, the mesh will be unzipped, but no connection elements will be created.
`INT4 orientMode`	The type of orientation to be used with Line Elements. 0=Node, 1=Vector, 2=CSys. If you specify either Node or Csys you must also specify the corresponding ID in orientID. If you specify Vector it must be specified in `orientVec`.
`INT4 orientID`	ID of the Node when orientMode = 0 or ID of CSys when `orientMode = 2`, used to orient Line Elements (if required by `connectWith`)
`REAL8 orientVec[0..2]`	The orientation vector used for Line Elements created when `orientMode = 1` (if required by `connectWith`).

Unpublished work. © 2025 Siemens

`INT4 dof[0..5]`	The degrees of freedom (DOF) to connect. Set values to 1 for DOF to connect; 0 for DOF to skip
`BOOL rigidReverse`	When True, reverses the direction of rigid connections so they are created from the copied nodes back to the *primary side* nodes
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshConnectRigid

feMeshConnectRigid

(NSourceNodeSetID, NTargetNodeSetID, bIsRbe3, bbSourceDoF, bbTargetDoF,bMaxDistance, dMaxDistance, bMultiMap, nMinTargetNum, nMaxTargetNum, nBadSourceSet, nBadTargetSet, pnCreated)

Description:

This method can be used to automatically create RBE2 or RBE3 elements using a set of *Source Nodes* as the Independent (RBE2) or Dependent (RBE3) node for each element, which then connect to any number of *Target Nodes* in the model, based on the specified parameters. This method provides programmatic access to the functionality of the **Mesh** → **Connect** → **Rigid** command.

Input:

`INT4 nSourceNodeSetID`	ID of Set object containing the *Source Nodes*.
`INT4 nTargetNodeSetID`	ID of Set object containing the *Target Nodes* to consider. If providing a Set ID, then using *Maximum Target Distance* (`dMaxDistance`) and/or *Maximum Number of Target Nodes* (`nMaxTargetNum`) is optional. Alternatively, a value of 0 can be used, along with `dMaxDistance` and `nMaxTargetNum`, to automatically determine the target nodes based on those required parameters.
`BOOL bIsRbe3`	When True, attempts to create RBE3 elements. When False, attempts to create RBE2 elements.
`BOOL bbSourceDoF [0..5]`	Array of 6 indices which correspond to the degrees of freedom for *Source Nodes*. True = DOF enabled, False = disabled. True = DOF enabled, False = disabled. Only used when `bIsRbe3` is set to True. [0] = FX, [1] = FY, [2] = FZ, [3] = MX, [4] = MY, [5] = MZ.
`BOOL bbTargetDoF [0..5]`	Array of 6 indices which correspond to the degrees of freedom *Target Nodes*. True = DOF enabled, False = disabled. [0] = FX/TX, [1] = FY/TY, [2] = FZ/TZ, [3] = MX/RX, [4] = MY/RY, [5] = MZ/RZ
`BOOL bMaxDistance`	When True, will use the value specified for *Maximum Target Distance* (`dMaxDistance`) to

Unpublished work. © 2025 Siemens

	determine which nodes to consider. When False, any node in the Target Set can be considered. When using a value of 0 for `nTargetNodeSetID`, must be set to True.
`REAL8 dMaxDistance`	Value for *Maximum Target Distance*, which is used to determine which nodes to consider in the Target Set. Only used when `bMaxDistance` is set to True.
`BOOL bMultiMap`	When True, any Target Node can be mapped to multiple Source Nodes. When False, each Target Node can only be mapped to the closest Source Node.
`INT4 nMinTargetNum`	Minimum number of Target Node(s) per Source Node. Enter a value of 0 to not specify a minimum number of Target Nodes.
`INT4 nMaxTargetNum`	Maximum number of Target Node(s) per Source Node. Enter a value of 0 to not specify a maximum number of Target Nodes. When using a value of 0 for `nTargetNodeSetID`, must be set to a value > 0.
`INT4 nBadSourceSet`	ID of Set object to be filled any Source Node(s) which were not paired with any Target Nodes. Alternatively, if this information is not desired, enter a value of 0.
`INT4 nBadTargetSet`	ID of Set object to be filled any Target Node(s) which were not paired with any Source Nodes. Alternatively, if this information is not desired, enter a value of 0.
Output:	
`INT4 pnCreated`	Number of RBE2 or RBE3 elements created by this method.
Return Code:	
`FE_FAIL`	Could not create any elements based on the specified parameters.
`FE_INVALID`	At least one argument is not specified properly.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim ndset As femap.Set
   Set ndset = App.feSet
   Dim source(5) As Boolean
   Dim target(5) As Boolean
   Dim rigidcreate As Long
   For i= 0 To 5
   source(i) = True
   target(i) = True
   Next
ndset.Select (FT_NODE, True, "Select Source Nodes")
    rc = App.feMeshConnectRigid (ndset.ID, 0, True, source,_
target, True, 2.0, False, 0, 5, 0, 0, rigidcreate)
End Sub
```


Unpublished work. © 2025 Siemens

feMeshTransition

feMeshTransition	
(useNodes, entCount, entID, edgeNum, edgeBias, edgeSize, propID, minBetween, maxAspect, maxQuickNodes, meshApproach, quadAngle, smoothLaplacian, smoothIter, smoothTol)	
Description:	
Meshes between nodes or line elements that form a closed boundary.	
Input:	
`BOOL useNodes`	If True, then entID contains a list of nodes used to define the boundary; otherwise, `entID` contains a list of line elements.
`INT4 entCount`	The number of items contained in `entID`, `edgeNum`, `edgeBias`, and `edgeSize`.
`INT4 entID[0..entCount-1]`	The IDs of the node or elements used to define the boundary being meshed.
`INT4 edgeNum[0..entCount-1]`	The number of elements to create between the associated nodes. `edgeNum[0]` applies to the space between `entID[0]` and `entID[1]`. `edgeNum[entCount-1]` applies to the space between `entID[entCount-1]` and `entID[0]`; that is, the last segment closing the boundary. Specify `edgeNum[i] = 0` to use `edgeSize[i]` to determine the number of elements. This option is not used if `useNodes=False`.
`INT4 edgeBias[0..entCount-1]`	The bias value to use with the elements being generated between successive nodes. Only used if `edgeNum[i]` is not zero, and `useNodes` is True. Use of indexes is the same as for `edgeNum`.
`REAL8 edgeSize[0..entCount-1]`	Similar to `edgeNum` and `edgeBias`; this is the element size to use for each segment. The distance between nodes is simply divided by this value to find the number of elements in the segment. Usage is the same as `edgeNum`. Only used if `edgeNum[i] = 0`.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. The `propID` property must be

Unpublished work. © 2025 Siemens

	compatible with surface meshing. Set `propID=0` to mesh with whatever property and element type are currently active. Set `propID=-1` to automatically mesh with plot-only elements, which do not require other property information.
`INT4 minBetween`	The minimum number of elements between boundaries during mesh generation. Not used for fast-tri mesher.
`REAL8 maxAspect`	The maximum allowable element aspect ratio. Not used for fast-tri mesher.
`INT4 maxQuickNodes`	Boundaries/surfaces with more than this number of nodes on the boundary are *quick-cut*. Not used for fast-tri mesher.
`INT4 meshApproach`	The mesher to use when meshing the surfaces: 0=Triangle, 1=Quad, 2=Fast-Triangle.
`REAL8 quadAngle`	The maximum angular deviation from 90 degrees allowed in a quad before it is split into triangles. Only applies to quad mesher.
`BOOL smoothLaplacian`	If True, Laplacian smoothing is performed. If False, centroidal smoothing is used.
`INT4 smoothIter`	The number of smoothing iterations to perform. Set to 0 to skip smoothing.
`INT4 smoothTol`	The smoothing tolerance. Smoothing will stop if node movement during a smoothing pass falls below this tolerance.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh. Boundary may not be valid - it may be self intersecting.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

The following example meshes the region bounded by nodes 1 through 7. Between each node, five new elements are created. The fast triangle mesher is being used. The software performs 17 Laplacian smoothing passes, or continues until the smoothing converges to a 0.001 tolerance.

```
Sub Main
Dim App As femap.model
Set App = feFemap()

Dim nodeCOUNT As Long
Dim nodes(20) As Long
Dim edgeNUM(20) As Long
Dim edgeBIAS(20) As Double
Dim edgeSIZE(20) As Double
Dim vnodes As Variant
Dim vedgeNUM As Variant
Dim vedgeSIZE As Variant
Dim vedgeBiAS As Variant
Dim i As Long
Dim rc As Long

For i = 0 To 6
 nodes(i) = i + 1
 edgeNUM(i) = 5
 edgeBIAS(i) = 1#
 edgeSIZE(i) = 0#
Next i

 vnodes = nodes
 vedgeNUM = edgeNUM
 vedgeSIZE = edgeSIZE
 vedgeBiAS = edgeBIAS

rc = App.feMeshTransition(True, 7, vnodes, vedgeNUM, _
            vedgeBiAS, vedgeSIZE, -1, 2, 2#, 30, 2, 60, True, _
            10, 0.001)
App.feViewRegenerate(0)

End Sub
```


Unpublished work. © 2025 Siemens

feMeshRemesh

feMeshRemesh

```
( elemSET, nodeSET, remeshMode, refineRatio, breakAngle, deleteOrig,
propID, minBetween,
 maxAspect, maxQuickNodes, meshApproach, quadAngle, smoothLaplacian,
smoothIter, smoothTol )
```

Description:

Remeshes a portion of an existing mesh.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be remeshed. Alternatively, if you specify a negative value, this is simply the ID of the single element to remesh. The elements contained in this set can be either surface (planar) elements or line elements. If you select line elements, then the only other option that is used is `refineRatio`.
`INT4 nodeSET`	The ID of a set that contains the node IDs where the refinement will be applied. Not used if you set `remeshMode=1`.
`INT4 remeshMode`	The remeshing mode: 0=Refine, 1=Remesh, 2=Unrefine.
`INT4 refineRatio`	The ratio between the number of elements along each edge currently and the number after remeshing. This only applies to edges selected by the nodes in `nodeSET`. For example, if `remeshMode=0`, refine, and `refineRatio=3`, then three elements will be generated along each refined edge. If `remeshMode=2`, unrefine, and `refineRatio=2`, then half the elements along the *refined* edge will be removed.
`REAL8 breakAngle`	The break angle along the refinement edges. If two adjacent edges intersect at an angle greater than this value, then they will not be removed, no matter what the `refineRatio` value is. This helps to preserve *corners* in the mesh. Only used when `remeshMode=2`, unrefine.

Unpublished work. © 2025 Siemens

`BOOL deleteOrig`	If True, the original nodes and elements that are being remeshed (those elements selected in `elemSET` and their nodes) will be deleted.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. Set `propID=0` to mesh with whatever property and element type are currently active. Set `propID=-1` to automatically mesh with plot-only elements, which do not require other property information.
`INT4 minBetween`	The minimum number of elements between boundaries during mesh generation. Not used for fast-tri mesher.
`REAL8 maxAspect`	The maximum allowable element aspect ratio. Not used for fast-tri mesher.
`INT4 maxQuickNodes`	Boundaries/surfaces with more than this number of nodes on the boundary are *quick-cut*. Not used for fast-tri mesher.
`INT4 meshApproach`	The mesher to use when meshing the surfaces: 0=Triangle, 1=Quad, 2=Fast-Triangle.
`REAL8 quadAngle`	The maximum angular deviation from 90 degrees allowed in a quad before it is split into triangles. Only applies to quad mesher.
`BOOL smoothLaplacian`	If True, Laplacian smoothing is performed. If False, centroidal smoothing is used.
`INT4 smoothIter`	The number of smoothing iterations to perform. Set to 0 to skip smoothing.
`INT4 smoothTol`	The smoothing tolerance. Smoothing will stop if node movement during a smoothing pass falls below this tolerance.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not mesh. Boundary may not be valid - it may be self intersecting.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feMeshConvertFacets

```
feMeshConvertFacets

( solidID )
```

Description:

Converts the current graphical facets of a solid into nodes and plot-only elements.

Input:

`INT4 solidID`	The ID of the solid to use to create the nodes and elements.

Output:

None	

Return Code:

None	

Remarks/Usage:

The current facetting of the solid is used to generate the elements. You can update the facetting tolerance prior to using this method if you want more (or fewer) elements.

Example:

None

Femap API Guide
Unpublished work. © 2025 Siemens

feMeshCleanupSlivers

feMeshCleanupSlivers **(elemSET, minAngle, minLength)**	
Description:	
Automatically removes *slivers* from a selected set of elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be checked. Alternatively, if you specify a negative value, this is simply the ID of the single element to check. The elements contained in this set must be surface (planar) elements.
`REAL8 minAngle`	Identifies slivers to be removed based on a minimum angle. Set to 0.0 to skip angle checking.
`REAL8 minLength`	Identifies slivers to be removed based on a minimum edge length. Set to 0.0 to skip length checking.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remove slivers.
Remarks/Usage:	
You must not include missing elements in the `elemSET`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshEdgeRemoval

feMeshEdgeRemoval **(elemSET, nd1, nd2)**	
Description:	
Removes an edge from a surface mesh by collapsing the connected elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be checked. The elements contained in this set must be surface (planar) elements.
`INT4 nd1`	The ID of the first node along the element edge.
`INT4 nd2`	The ID of the second node along the element edge.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remove edge.
Remarks/Usage:	
The elements selected must form a single, manifold hull - there can only be two elements that share each edge.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshBuildRemeshRegions

feMeshBuildRemeshRegions

(elemSET, breakAngle)

Description:

Builds remeshing regions from a set of planar elements.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be checked. The elements contained in this set must be surface (planar) elements.
`REAL8 breakAngle`	The maximum angle between adjacent elements for those elements to be considered in the same region.

Output:

None	

Return Code:

`FE_FAIL`	Unable to build regions.

Remarks/Usage:

The elements selected must form a single, manifold hull - there can only be two elements that share each edge. Any quadrilateral elements selected will be converted to triangles.

Example:

None

Unpublished work. © 2025 Siemens

feMeshEditRemeshRegions

feMeshEditRemeshRegions **(el1, el2)**	
Description:	
Moves an element from one remeshing region to an adjacent region.	
Input:	
`INT4 el1`	The ID of the element to be moved to the other region.
`INT4 el2`	The ID of the element used to identify the region where the first element will be placed.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to update regions.
Remarks/Usage:	
The element to be moved, `el1`, must be adjacent, on at least one edge to the region containing element `el2`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshRemeshRegions

feMeshRemeshRegions	
(meshSize, propID, minBetween, maxAspect, maxQuickNodes, meshApproach, quadAngle, smoothLaplacian, smoothIter, smoothTol)	
Description:	
Moves an element from one remeshing region to an adjacent region.	
Input:	
`REAL8 meshSize`	The default mesh size, used to determine the size of elements that will be generated.
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. Set `propID=0` to mesh with whatever property and element type are currently active. Set `propID=-1` to automatically mesh with plot-only elements, which do not require other property information.
`INT4 minBetween`	The minimum number of elements between boundaries during mesh generation. Not used for fast-tri mesher.
`REAL8 maxAspect`	The maximum allowable element aspect ratio. Not used for fast-tri mesher.
`INT4 maxQuickNodes`	Boundaries/surfaces with more than this number of nodes on the boundary are *quick-cut*. Not used for fast-tri mesher.
`INT4 meshApproach`	The mesher to use when meshing the surfaces (0=Triangle, 1=Quad, 2=Fast-Triangle).
`REAL8 quadAngle`	The maximum angular deviation from 90 degrees allowed in a quad before it is split into triangles. Only applies to quad mesher.
`BOOL smoothLaplacian`	If True, Laplacian smoothing is performed. If False, centroidal smoothing is used.
`INT4 smoothIter`	The number of smoothing iterations to perform. Set to 0 to skip smoothing.

Unpublished work. © 2025 Siemens

`INT4 smoothTol`	The smoothing tolerance. Smoothing will stop if node movement during a smoothing pass falls below this tolerance.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remesh.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshEdgeMembers

feMeshEdgeMembers

(elemSET, nodeSET, propID, orient)

Description:

Generates elements along the free edges or free faces of selected elements. This method is obsolete and should no longer be used. It has been replaced by `feMeshEdgeMembers2`.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be checked. The elements contained in this set must be surface (planar) elements or solid elements.
`INT4 nodeSET`	The ID of a set that contains the IDs of the nodes which define the portions of the free edge/free face where you want to define edge members. An edge member is created for every edge/face that has at least one node in this set.
`INT4 propID`	The ID of an existing property used to define the edge member elements. If planar elements are selected in the `elemSET`, then this must be the ID of a line element property. If solid elements are selected in `elemSET`, then this must be a planar element property.
`REAL8 orient[0..2]`	The element orientation vector for edge members. This is only used if `elemSET` contains planar elements, and `propID` specifies a line element type that requires orientation.

Output:

None	

Return Code:

`FE_FAIL`	Unable to create edge members. Either you selected elements that did not exist, or they are of the wrong type.

Remarks/Usage:

Only planar or solid elements can be selected in `elemSET`, not both at the same time.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feMeshEdgeMembers2

feMeshEdgeMembers2

(bOnFreeEdges, elemSET, nodeSET, propID, orient)

Description:

Generates elements along the free edges or free faces of selected elements.

Input:

`BOOL bOnFreeEdges`	If true, elements will be generated on only free edges of the selected elements. If false, the specified orientation vector definition is not generated.
`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be checked. The elements contained in this set must be surface (planar) elements or solid elements.
`INT4 nodeSET`	The ID of a set that contains the IDs of the nodes which define the portions of the free edge/free face where you want to define edge members. An edge member is created for every edge/face that has at least one node in this set.
`INT4 propID`	The ID of an existing property used to define the edge member elements. If planar elements are selected in the `elemSET`, then this must be the ID of a line element property. If solid elements are selected in `elemSET`, then this must be a planar element property. **Note** If you select a solid element for `elemSET` and select the properties of a beam element, an error will not occur and a beam element will be created.
`REAL8 orient[0..2]`	The element orientation vector for edge members. This is only used if `elemSET` contains planar elements, and `propID` specifies a line element type that requires orientation.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Unable to create edge members. Either you selected elements that did not exist, or they are of the wrong type.
Remarks/Usage:	
Only planar or solid elements can be selected in `elemSET`, not both at the same time.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSmooth

feMeshSmooth **(elemSET, laplaceSmooth, iter, tolerance)**	
Description:	
Smooths a selected set of elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be smoothed. The elements contained in this set must be surface (planar) elements or solid elements.
`BOOL laplaceSmooth`	If True, Laplacian smoothing is performed; otherwise, centroidal smoothing is used.
`INT4 iter`	The maximum number of smoothing passes that will be performed. Smoothing will terminate sooner if the tolerance is reached.
`REAL8 tolerance`	The nodal movement tolerance. If no nodes move farther than this distance during a smoothing pass, smoothing will be terminated.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to smooth. Your `elemSET` may contain multiple element types, or element types that are not compatible with this operation.
Remarks/Usage:	
Only planar or solid elements can be selected in `elemSET`, not both at the same time.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshSmooth2

feMeshSmooth2

(elemSET, fixednodeSET, laplaceSmooth, iter, tolerance)

Description:

Smooths a selected set of elements while allowing any number of specified nodes to remain in a fixed location.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be smoothed. The elements contained in this set must be surface (planar) elements or solid elements.
`INT4 fixednodeSET`	The ID of a set that contains the IDs of the nodes to remain in a fixed location during the smoothing operation. Specifying a value of 0 will create the same results as using `feMeshSmooth`.
`BOOL laplaceSmooth`	If True, Laplacian smoothing is performed; otherwise, centroidal smoothing is used.
`INT4 iter`	The maximum number of smoothing passes that will be performed. Smoothing will terminate sooner if the tolerance is reached.
`REAL8 tolerance`	The nodal movement tolerance. If no nodes move farther than this distance during a smoothing pass, smoothing will be terminated.

Output:

None	

Return Code:

`FE_FAIL`	Unable to smooth. Your `elemSET` may contain multiple element types, or element types that are not compatible with this operation.

Remarks/Usage:

Only planar or solid elements can be selected in `elemSET`, not both at the same time.

Example:

None

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

feMeshExtrudeElem

feMeshExtrudeElem

(elemSET, axisLen, axisVec, propID, numElem, method, matchCurrent, deleteOrig)

Description:

Extrudes existing elements into new elements.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be extruded. The elements contained in this set must be either line or surface (planar) elements, but not both. Line elements are extruded into planar elements; planar elements are extruded into solid elements.
`REAL8 axisLen`	The extrusion length (See *Remarks*).
`REAL8 axisVec[0..2]`	The extrusion direction vector (See *Remarks*).
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
`INT4 numElem`	The number of elements generated by each original element along the extrusion direction.
`INT4 method`	The method of determining the extrusion direction. 0=Along axisVec, 1=along the element normal, 2=along the element normal with thickness corrections.
`BOOL matchCurrent`	If True, the attributes of the elements that are created will match the active settings. If False, they will match the generating elements.
`BOOL deleteOrig`	If True, the elements in `elemSET` will be deleted, leaving only the new elements. If False, the elements in `elemSet` will remain in the model.

Output:

None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to extrude. Check the extrusion direction, and the element and property types.
Remarks/Usage:	
The extrusion length (with method=0) is really the length of the `axisVec` multiplied by `axisLen`. Normally you will either want to specify `axisLen=1.0`, and set the `axisVec` to the true extrusion vector, or specify `axisVec` as a unit vector with the proper components, and set `axisLen` to the true extrusion length.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshExtrudeElem2

feMeshExtrudeElem2

**(elemSET, axisLen, axisOrig, axisVec, propID, numElem, method, radialspherical,
ontoSurfSetID, matchCurrent, deleteOrig)**

Description:

Extrudes existing elements into new elements allowing use of all 5 methods in the **Mesh** → **Extrude** → **Elements** command (Vector, Normals, Normals with Thickness Correction, Radial, and Along Vector to Surfaces)

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be extruded. The elements contained in this set must be either line or surface (planar) elements, but not both. Line elements are extruded into planar elements; planar elements are extruded into solid elements.
`REAL8 axisLen`	The extrusion length (See *Remarks*).
`REAL8 axisOrig[0..2]`	The extrusion direction origin (See *Remarks*).
`REAL8 axisVec[0..2]`	The extrusion direction vector (See *Remarks*).
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
`INT4 numElem`	The number of elements generated by each original element along the extrusion direction.
`INT4 method`	The method of determining the extrusion direction. 0=Along axisVec, 1=along the element normal, 2=along the element normal with thickness corrections, 3=radial, 4=along vector to surfaces
`BOOL radialspherical`	Only used when method is set to 3 (radial). If True, the mesh will be extruded radially around a specified coordinate location in `axisOrig` (spherical extrusion) to a length specified with `axisLen`. If False, the mesh will be extruded

Unpublished work. © 2025 Siemens

	radially about a vector specified with `axisOrig` and `axisVec` (cylindrical extrusion) to a length specified with `axisLen`
`INT4 ontoSurfSETID`	Only used when method is set to 4 (along vector to surface). The ID of a set that contains the IDs of the surfaces to be extruded to along a vector specified with `axisOrig` and `axisVec`.
`BOOL matchCurrent`	If True, the attributes of the elements that are created will match the active settings. If False, they will match the generating elements.
`BOOL deleteOrig`	If True, the elements in `elemSET` will be deleted, leaving only the new elements. If False, the elements in `elemSet` will remain in the model.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to extrude. Check the extrusion direction, and the element and property types.
Remarks/Usage:	
The extrusion length (with method=0) is really the length of the `axisVec` multiplied by `axisLen`. Normally you will either want to specify `axisLen=1.0`, and set the `axisVec` to the true extrusion vector, or specify `axisVec` as a unit vector with the proper components, and set `axisLen` to the true extrusion length.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshExtrudeElem3

feMeshExtrudeElem3

**(elemSET, axisLen, axisOrig, axisVec, dBias, propID, numElem, method, radialspherical,
ontoSurfSetID, matchCurrent, deleteOrig)**

Description:

Extrudes existing elements into new elements allowing use of all 5 methods in the **Mesh → Extrude → Elements** command (Vector, Normals, Normals with Thickness Correction, Radial, and Along Vector to Surfaces)

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be extruded. The elements contained in this set must be either line or surface (planar) elements, but not both. Line elements are extruded into planar elements; planar elements are extruded into solid elements.
`REAL8 axisLen`	The extrusion length (See *Remarks*).
`REAL8 axisOrig[0..2]`	The extrusion direction origin (See *Remarks*).
`REAL8 axisVec[0..2]`	The extrusion direction vector (See *Remarks*).
`REAL8 dBias`	Bias value – greater than 0.0. A value greater than 1.0 means that the larger elements will be at the end of the extrusion/revolve. A value less than 1.0 means that the larger elements will be at the beginning. Specifying either 0.0 or 1.0 means no biasing.
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
`INT4 numElem`	The number of elements generated by each original element along the extrusion direction.
`INT4 method`	The method of determining the extrusion direction. 0=Along axisVec, 1=along the element normal,

Unpublished work. © 2025 Siemens

	2=along the element normal with thickness corrections, 3=radial, 4=along vector to surfaces
`BOOL radialspherical`	Only used when method is set to 3 (radial). If True, the mesh will be extruded radially around a specified coordinate location in `axisOrig` (spherical extrusion) to a length specified with `axisLen`. If False, the mesh will be extruded radially about a vector specified with `axisOrig` and `axisVec` (cylindrical extrusion) to a length specified with `axisLen`
`INT4 ontoSurfSETID`	Only used when method is set to 4 (along vector to surface). The ID of a set that contains the IDs of the surfaces to be extruded to along a vector specified with `axisOrig` and `axisVec`.
`BOOL matchCurrent`	If True, the attributes of the elements that are created will match the active settings. If False, they will match the generating elements.
`BOOL deleteOrig`	If True, the elements in `elemSET` will be deleted, leaving only the new elements. If False, the elements in `elemSet` will remain in the model.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to extrude. Check the extrusion direction, and the element and property types.
Remarks/Usage:	
The extrusion length (with method=0) is really the length of the `axisVec` multiplied by `axisLen`. Normally you will either want to specify `axisLen=1.0`, and set the `axisVec` to the true extrusion vector, or specify `axisVec` as a unit vector with the proper components, and set `axisLen` to the true extrusion length.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshExtrudeCurve

feMeshExtrudeCurve **(curveSET, axisLen, axisVec, propID, numElem)**	
Description:	
Extrudes existing curves into new elements.	
Input:	
`INT4 curveSET`	The ID of a set that contains the IDs of the curves to be extruded. The curves are extruded into planar elements. The number and spacing of elements is dependent on the mesh size established on the curves.
`REAL8 axisLen`	The extrusion length (See *Remarks*).
`REAL8 axisVec[0..2]`	The extrusion direction vector (See *Remarks*).
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be a planar type.
`INT4 numElem`	The number of elements generated along the extrusion direction.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to extrude. Check the extrusion direction, and the element and property types.
Remarks/Usage:	
The extrusion length is really the length of the `axisVec` multiplied by `axisLen`. Normally you will either want to specify `axisLen=1.0`, and set the `axisVec` to the true extrusion vector, or specify `axisVec` as a unit vector with the proper components, and set `axisLen` to the true extrusion length.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshExtrudeCurve2

feMeshExtrudeCurve2

(curveSET, axisLen, axisVec, dBias, propID, numElem)

Description:

Extrudes existing curves into new elements.

Input:

`INT4 curveSET`	The ID of a set that contains the IDs of the curves to be extruded. The curves are extruded into planar elements. The number and spacing of elements is dependent on the mesh size established on the curves.
`REAL8 axisLen`	The extrusion length (See *Remarks*).
`REAL8 axisVec[0..2]`	The extrusion direction vector (See *Remarks*).
`REAL8 dBias`	Bias value – greater than 0.0. A value greater than 1.0 means that the larger elements will be at the end of the extrusion/revolve. A value less than 1.0 means that the larger elements will be at the beginning. Specifying either 0.0 or 1.0 means no biasing.
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be a planar type.
`INT4 numElem`	The number of elements generated along the extrusion direction.

Output:

None	

Return Code:

`FE_FAIL`	Unable to extrude. Check the extrusion direction, and the element and property types.

Remarks/Usage:

The extrusion length is really the length of the `axisVec` multiplied by `axisLen`. Normally you will either want to specify `axisLen=1.0`, and set the `axisVec` to the true extrusion vector, or specify `axisVec` as a unit vector with the proper components, and set `axisLen` to the true extrusion length.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feMeshRevolveElem

feMeshRevolveElem

(elemSET, rotAngle, rotThrust, axisOrig, axisVec, propID, numElem, matchCurrent, deleteOrig)

Description:

Revolves existing elements into new elements.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be revolved. The elements contained in this set must be either line or surface (planar) elements, but not both. Line elements are revolved into planar elements; planar elements are revolved into solid elements.
`REAL8 rotAngle`	The angle of rotation, in degrees.
`REAL8 rotThrust`	The total thrust distance (along the axis of rotation), typically zero.
`REAL8 axisOrig[0..2]`	The origin of the axis of revolution.
`REAL8 axisVec[0..2]`	The direction of the axis of revolution.
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
`INT4 numElem`	The number of elements generated by each original element along the revolution direction.
`BOOL matchCurrent`	If True, the attributes of the elements that are created will match the active settings. If False, they will match the generating elements.
`BOOL deleteOrig`	If True, the elements in `elemSET` will be deleted, leaving only the new elements.

Output:

None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to revolve. Check the axis of revolution, and the element and property types.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshRevolveElem2

feMeshRevolveElem2

**(elemSET, rotAngle, rotThrust, axisOrig, axisVec, dBias, propID, numElem,
matchCurrent, deleteOrig)**

Description:

Revolves existing elements into new elements.

Input:

INT4 elemSET	The ID of a set that contains the IDs of the elements to be revolved. The elements contained in this set must be either line or surface (planar) elements, but not both. Line elements are revolved into planar elements; planar elements are revolved into solid elements.
REAL8 rotAngle	The angle of rotation, in degrees.
REAL8 rotThrust	The total thrust distance (along the axis of rotation), typically zero.
REAL8 axisOrig[0..2]	The origin of the axis of revolution.
REAL8 axisVec[0..2]	The direction of the axis of revolution.
REAL8 dBias	Bias value – greater than 0.0. A value greater than 1.0 means that the larger elements will be at the end of the extrusion/revolve. A value less than 1.0 means that the larger elements will be at the beginning. Specifying either 0.0 or 1.0 means no biasing.
INT4 propID	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
INT4 numElem	The number of elements generated by each original element along the revolution direction.

Unpublished work. © 2025 Siemens

`BOOL matchCurrent`	If True, the attributes of the elements that are created will match the active settings. If False, they will match the generating elements.
`BOOL deleteOrig`	If True, the elements in `elemSET` will be deleted, leaving only the new elements.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to revolve. Check the axis of revolution, and the element and property types.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshRevolveCurve

feMeshRevolveCurve	
(curveSET, rotAngle, rotThrust, axisOrig, axisVec, propID, numElem)	
Description:	
Revolves existing curves into new elements.	
Input:	
`INT4 curveSET`	The ID of a set that contains the IDs of the curves to be revolved. The curves are revolved into planar elements. The number and spacing of elements is dependent on the mesh size established on the curves.
`REAL8 rotAngle`	The angle of rotation, in degrees.
`REAL8 rotThrust`	The total thrust distance (along the axis of rotation), typically zero.
`REAL8 axisOrig[0..2]`	The origin of the axis of revolution.
`REAL8 axisVec[0..2]`	The direction of the axis of revolution.
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
`INT4 numElem`	The number of elements generated by each original element along the revolution direction.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to revolve. Check the axis of revolution, and the element and property types.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feMeshRevolveCurve2

feMeshRevolveCurve2

(curveSET, rotAngle, rotThrust, axisOrig, axisVec, dBias, propID, numElem)

Description:

Revolves existing curves into new elements.

Input:

`INT4 curveSET`	The ID of a set that contains the IDs of the curves to be revolved. The curves are revolved into planar elements. The number and spacing of elements is dependent on the mesh size established on the curves.
`REAL8 rotAngle`	The angle of rotation, in degrees.
`REAL8 rotThrust`	The total thrust distance (along the axis of rotation), typically zero.
`REAL8 axisOrig[0..2]`	The origin of the axis of revolution.
`REAL8 axisVec[0..2]`	The direction of the axis of revolution.
`REAL8 dBias`	Bias value – greater than 0.0. A value greater than 1.0 means that the larger elements will be at the end of the extrusion/revolve. A value less than 1.0 means that the larger elements will be at the beginning. Specifying either 0.0 or 1.0 means no biasing.
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid), depending on the type of elements selected in `elemSET`.
`INT4 numElem`	The number of elements generated by each original element along the revolution direction.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to revolve. Check the axis of revolution, and the element and property types.
Remarks/Usage:	
None	
Example:	
None	

Femap API Guide
Unpublished work. © 2025 Siemens

feMeshSweepElem

feMeshSweepElem	
(elemSET, numPath, curvePath, numAlign, curveAlign, alignLoc, propID)	
Description:	
Sweeps existing elements along one or more curves into new elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains the IDs of the elements to be swept. The curves are swept into planar elements. The number and spacing of elements is dependent on the mesh size established on the curves.
`INT4 numPath`	The number of curves in the `curvePath` array.
`INT4 curvePath[0..numPath-1]`	The IDs of the curves along the sweep path. These IDs should be specified in order, starting at the beginning of the path to sweep along.
`INT4 numAlign`	The number of curves in the `curveAlign` array (usually 0, unless you are using an alignment path to orient the elements along sweep path).
`INT4 curveAlign[0..numAlign-1]`	The IDs of curves along the alignment path.
`REAL8 alignLoc[0..2]`	The alignment position. Only used if the sweep path is nonplanar, and `numPath=0` (not using an alignment path).
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be a planar type.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to sweep. Check the order of the curves for the path and alignment, and the element and property types.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feMeshSweepCurve

feMeshSweepCurve

(curveSET, numPath, curvePath, numAlign, curveAlign, alignLoc, propID)

Description:

Sweeps curves along one or more curves into new elements.

Input:

`INT4 elemSET`	The ID of a set that contains the IDs of the curves to be swept. The elements contained in this set must be either line or surface (planar) elements, but not both. Line elements are swept into planar elements; planar elements are swept into solid elements.
`INT4 numPath`	The number of curves in the `curvePath` array.
`INT4 curvePath[0..numPath-1]`	The IDs of the curves along the sweep path. These IDs should be specified in order, starting at the beginning of the path to sweep along.
`INT4 numAlign`	The number of curves in the `curveAlign` array (usually 0, unless you are using an alignment path to orient the elements along sweep path).
`INT4 curveAlign[0..numAlign-1]`	The IDs of curves along the alignment path.
`REAL8 alignLoc[0..2]`	The alignment position. Only used if the sweep path is nonplanar, and `numPath=0` (not using an alignment path).
`INT4 propID`	The ID of the property that will be used for the new elements. This property must be of the correct type (planar or solid) depending on the type of elements selected in `elemSET`.
`BOOL matchCurrent`	If True, the attributes of the elements that are created will match the active settings. If False, they will match the generating elements.
`BOOL deleteOrig`	If True, the elements in `elemSET` will be deleted, leaving only the new elements.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Unable to sweep. Check the order of the curves for the path and alignment, and the element and property types.
Remarks/Usage:	
None	
Example:	
None	

feMeshPlatePatternRefine

`feMeshPlatePatternRefine`
`( INT4 nSetID, INT4 nSplitPattern )`
Description:
This function provides API access to the **Mesh** → **Editing** → **Element Refine** command. Input is the ID of a Femap Set containing the shell elements to be refined, and definition of the Split Pattern to be used.
Input:

`INT4 nSetID`	The ID of a set containing the elements to be refined.
`INT4 nSplitPattern`	Matching the dialog box from the GUI command, 0 for the first pattern, 1 for the second.
Output:	
None	None
Return Code:	
`FE_OK`	The mesh refine succeeded.
`FE_FAIL`	The mesh refine failed.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Dim elSet As femap.Set
    Set elSet = App.feSet

    Dim ndSet As femap.Set
    Set ndSet = App.feSet

    ndSet.AddRule( 30, FGD_NODE_ATCURVE  ) ' Get the nodes attached to
Curve 30 in test model
    elSet.AddSetRule( ndSet.ID, FGD_ELEM_BYNODE ) 'add the elements
connected to these nodes

    App.feMeshPlatePatternRefine( elSet.ID, 1 ) ' Refine Once

    'Clear the sets and do it a second time
    ndSet.clear
    elSet.clear
    ndSet.AddRule( 30, FGD_NODE_ATCURVE  )
    elSet.AddSetRule( ndSet.ID, FGD_ELEM_BYNODE )

    App.feMeshPlatePatternRefine( elSet.ID, 1 )

End Sub
```

Meshing Utilities

These utilities can be useful when trying to query an existing mesh to determine a *path* of nodes across a mesh or a *row* of solid elements.

Unpublished work. © 2025 Siemens

feNodesBetweenNodes

feNodesBetweenNodes

(ndID1, ndID2, elSetID, nNodes, nNodeList)

Description:

Returns a list of nodes, in order, between two specified nodes (i.e., a *node path*). Also, the path from Node 1 to Node 2 can be limited to only consider the edges of a subset of elements specified by the user.

Input:

`INT4 ndID1`	ID of node at start of the node path.
`INT4 ndID2`	ID of node at end of the node path.
`INT4 elSetID`	ID of Set Object containing element IDs to potentially limit the node path.

Output:

`INT4 nNodes`	The number of nodes in the node path, including the nodes specified for `ndID1` and `ndID2` (First and last values in `nNodeList`).
`VARIANT vNodeList (0..nNodes-1)`	This variant contains an array with the node IDs in the node path, in order, from `ndID1` to `ndID2`.

Return Code:

`FE_FAIL`	A continuous node path, using only the edges of the elements specified in `elSetID`, cannot be determined.

Remarks/Usage:

Currently, only 2D and 3D elements are considered when attempting to determine a node path.

Example:

```
Sub Main

   Dim App As femap.model

   Set App = feFemap()

   Dim elset As femap.Set

   Set elset = App.feSet
```


Unpublished work. © 2025 Siemens

```
   Dim ncount As Long
   Dim nds As Variant
   elset.AddAll (FT_ELEM)
   rc = App.feNodesBetweenNodes (1, 91, elset.ID, ncount, nds)
   For i = 0 To (ncount-1)
       App.feAppMessage (FCM_NORMAL, "Node "+Str$(nds(i)))
   Next
End Sub
```


Unpublished work. © 2025 Siemens

feSolidElementsInARow

feSolidElementsInARow

(numElems, nElemIDs, nFaceIDs, nMaxFind, bVisibleOnly, nCount, nElemList)

Description:

Returns a list of solid hex and wedge elements found in a *row*, starting with a specified element face and continuing through a mesh until no additional hex or wedge elements can be found for a particular *row*. If a *row* of hex elements comes to a quadrilateral face of a wedge element, the *row* ends at the wedge. In addition, there is an option to consider all hex and wedge elements or only hex and wedge elements visible in the active view.

Input:

`INT4 numElems`	The number of hex and wedge elements and corresponding element faces to be used to search for *rows* of elements
`INT4 nElemIDs (0..numElems-1)`	This array contains a list of hex and wedge element IDs to consider, along with their corresponding element faces found in `nFaceIDs`, when searching for *rows*.
`INT4 nFaceIDs (0..numElems-1)`	This array contains a list of solid element face IDs, corresponding to the hex and wedge elements in `nElemIDs`, to consider when searching for *rows* (Only triangular faces of wedge elements are valid).
`INT4 nMaxFind`	Maximum number of hex or wedge elements which can be found in each *rows* of solid elements.
`BOOL bVisibleOnly`	When True, only hex and wedge elements which are visible in the active view will be considered, otherwise all hex and wedge elements are considered.

Output:

`INT4 nCount`	Total number of hex and wedge elements for all *found rows*.
`VARIANT vElemList (0..nNodes-1)`	This variant contains an array of element IDs, stored *found row-by-found row*, using the order of each *seed* element and corresponding face in `nElemIDs` and `nFaceIDs`, respectively.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	A *row* of hex or wedge elements could not be found for at least one of the Element ID/Face ID pairs.

Remarks/Usage:

To determine when one *found row* ends and another *found row* begins, it will often be helpful to compare the element IDs in `vElemList` to the ones found in `nElemIDs`. Also, it maybe helpful to use this call with `feGetElementFaces` (see **feGetElementFaces**), `feGetElementFacesFromSet` (see **feGetElementFacesFromSet**) or `feGetElementAdjacentFaces` (see **feStartSelectionTool**).

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim origElem(0) As Long
   Dim origFace(0) As Long
   Dim fcount As Long
   Dim vElIDs As Variant
   Dim vElFaces As Variant
   Dim rowcount As Long
   Dim vElRows As Variant

   If App.feGetElementFaces ("Select Element Faces", 0,
   origElem, origFace, fcount, vElIDs, vElFaces) = 2 Then
          Exit Sub
   End If

rc = App.feSolidElementsInARow (fcount, vElIDs, vElFaces,
   1000, False, rowcount, vElRows)
```


Unpublished work. © 2025 Siemens

```
For i = 0 To rowcount-1

App.feAppMessage (FCM_NORMAL, "Element"+Str$(vElRows(i)))

Next


End Sub
```

Mesh Copy/Radial Copy/Scale/Rotate/Reflect commands

In Femap v12 and above, a CopyTool Object was created to grant users access the enhanced **Copy**, **Rotate**, and **Reflect** commands on both the **Geometry** and **Mesh** menus. For more information, see CopyTool Objects.

The commands on the **Mesh** → **Copy...**; **Mesh** → **Rotate...**; and **Mesh** → **Reflect...** menus which existed in Femap prior to version 12, along with the commands on the current **Mesh** → **Radial Copy...** and **Mesh** → **Scale...** menus may be accessed via the API using the `feGenerate...` methods found in Geometry and Mesh Generation Methods.

Modification Methods

Projection

Unpublished work. © 2025 Siemens

feProjectOnto

feProjectOnto

(entityTYPE, entitySET, projOnto, projAlong, ontoSET, OntoLocation, OntoVector, AlongVector, ExtendCurves, VectorBothDirections)

Description:

Projects nodes or points onto a curve, surface, vector, or plane using either a closest/normal direction or along a specified vector. Additional options to **Extend Curves** and **Project Both Directions** are also available when using certain combinations of the available options.

Input:

`INT4 entityTYPE`	The type of entities to be projected. For entity information, see **Entity Types**. Can be `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of point or node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`INT4 projOnto`	Used to set the **Project Onto** option. 0=onto Curve (`FPO_CURVE`), 1=onto Surface (`FPO_SURFACE`), 2=onto Vector (`FPO_VECTOR`), or 3=onto Plane (`FPO_PLANE`).
`INT4 projAlong`	Used to set the **Project Along** option. 0=use Closest/Normal (`FPA_CLOSEST`) or 1=along Vector (`FPA_VECTOR`).
`INT4 ontoSET`	The ID of a set that contains a list of curve or surface IDs to project onto.
`REAL8 ontoLocation[0..2]`	The XYZ location of the base of the vector to project onto. Must be specified in Global Rectangular coordinates.
`REAL8 ontoVector[0..2]`	The XYZ components of the vector to project onto. Must be specified in Global Rectangular coordinates.
`REAL8 alongVector [0..2]`	The XYZ components of the vector to project along. Must be specified in Global Rectangular coordinates.

Unpublished work. © 2025 Siemens

`BOOL ExtendCurves`	Only used when `projOnto = 0 (FPO_CURVE)` and `projAlong = 0 (FPA_CLOSEST)`. When True, the curve to project onto will be extended and the point or node will be projected onto the location closest to the *Extended Curve* (i.e., not actually on the curve itself). When False, the point or node will be projected onto the curve, many projecting onto one of the curve's end points.
`BOOL VectorBothDirections`	Only used when `projOnto = 1 (FPO_SURFACE)` and `projAlong = 1 (FPA_VECTOR)`. When True, the point(s) or node(s) will be projected in both directions along the specified vector. This allows points on both sides of a surface to be projected using a single specified vector. When False, the point(s) and node(s) will only be projected in the direction of the specified vector.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify either points or nodes.
`FE_FAIL`	Could not project.
Remarks/Usage:	
This call is similar to using the **Modify** → **Project** → **Point** or **Modify** → **Project** → **Node** commands.	
Example:	
None	

Unpublished work. © 2025 Siemens

feProjectOntoCurve

feProjectOntoCurve **(entityTYPE, entitySET, curveID)**	
Description:	
Projects nodes or points onto a curve.	
Input:	
`INT4 entityTYPE`	The type of entities to be projected. For entity information, see **Entity Types**. Can be `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of point or node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`INT4 curveID`	The ID of the curve to project onto.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify either points or nodes.
`FE_FAIL`	Could not project.
Remarks/Usage:	
Points/nodes are projected to the closest location on the curve, not along any specified vector.	
Example:	
None	

Unpublished work. © 2025 Siemens

feProjectOntoSurface

feProjectOntoSurface **(entityTYPE, entitySET, surfaceID)**	
Description:	
Projects nodes or points onto a surface.	
Input:	
`INT4 entityTYPE`	The type of entities to be projected. For entity information, see **Entity Types**. Can be `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of point or node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`INT4 surfaceID`	The ID of the surface to project onto.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify either points or nodes.
`FE_FAIL`	Could not project.
Remarks/Usage:	
Points/nodes are projected to the closest location on the surface, not along any specified vector.	
Example:	
None	

Unpublished work. © 2025 Siemens

feProjectOntoVector

feProjectOntoVector **(entityTYPE, entitySET, vecLoc, vecDir)**	
Description:	
Projects points or nodes onto a vector.	
Input:	
`INT4 entityTYPE`	The type of entities to be projected. For entity information, see **Entity Types**. Can be `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of point or node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`REAL8 vecLoc[0..2]`	The XYZ location of the base of the vector to project onto. Must be specified in Global Rectangular coordinates.
`REAL8 vecDir[0..2]`	The XYZ components of the vector to project onto. Must be specified in Global Rectangular coordinates.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not project.
`FE_INVALID`	`entityTYPE` must be either `FT_POINT` or `FT_NODE`.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feProjectAlongVector

feProjectAlongVector **(entityTYPE, entitySET, surfaceSET, vecAlong)**	
Description:	
Projects points or nodes along a vector onto selected surfaces.	
Input:	
`INT4 entityTYPE`	The type of entities to be projected. For entity information, see **Entity Types**. Can be `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of point or node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`INT4 surfaceSET`	The ID of a set that contains a list of surface IDs that the entities will be projected onto. Alternatively, if you specify a negative value, this is the ID of the single surface to project onto.
`REAL8 vecAlong[0..2]`	The XYZ components of the vector to project along. Must be specified in Global Rectangular coordinates.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not project.
`FE_INVALID`	`entityTYPE` must be either `FT_POINT` or `FT_NODE`.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feProjectOntoPlane

feProjectOntoPlane **(entityTYPE, entitySET, planeLoc, planeNormal)**	
Description:	
Projects points or nodes onto a plane.	
Input:	
`INT4 entityTYPE`	The type of entities to be projected. For entity information, see **Entity Types**. Can be `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of point or node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`REAL8 planeLoc[0..2]`	The XYZ location of the base of the plane to project onto. Must be specified in Global Rectangular coordinates.
`REAL8 planeNormal[0..2]`	The XYZ components of the plane normal to project onto. Must be specified in Global Rectangular coordinates.
Output:	
None	
`FE_FAIL`	Could not project.
`FE_INVALID`	`entityTYPE` must be either `FT_POINT` or `FT_NODE`.
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feProjectOntoSolid

feProjectOntoSolid

(nodeSET, elemSET, solidID, surfCount, surfID)

Description:

Projects a mesh onto a solid.

Input:

`INT4 nodeSET`	The ID of a set that contains a list of node IDs that will be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project.
`INT4 elemSET`	The ID of a set that contains a list of element IDs that reference the nodes to be projected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to project. These elements are used during the smoothing process as the nodes are projected.
`INT4 solidID`	The ID of the solid to project onto. Must be 0 if you are going to specify surfaces.
`INT4 surfCount`	The number of surfaces specified in `surfID` - only used if `solidID=0`.
`INT4 surfID[0..surfCount-1]`	The IDs of the surfaces to project onto - only used if `solidID=0` and `surfCount` is not 0.

Output:

None	

Return Code:

`FE_FAIL`	Could not project.

Remarks/Usage:

A mesh can either be projected to a solid by specifying `solidID`, or to surfaces by specifying `surfCount` and `surfID`.

Example:

None

Unpublished work. © 2025 Siemens

Move, Rotate and Scale

Unpublished work. © 2025 Siemens

feMoveTo

feMoveTo **(entityTYPE, entitySET, x, y, z, doX, doY, doZ, inCsys, moveRef)**	
Description:	
Moves nodes, points, or coordinate systems to a new location.	
Input:	
`INT4 entityTYPE`	The type of entities to be moved. For entity information, see **Entity Types**. Can be `FT_CSYS`, `FT_POINT` or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of coordinate system, point, or node IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`REAL8 x`	The coordinates that you want to move to. Specified in the `inCsys` coordinate system.
`REAL8 y`	
`REAL8 z`	
`BOOL doX`	If True, the corresponding coordinate direction (in `inCsys`) will be updated to the specified value. Only directions specified as True will be moved.
`BOOL doY`	
`BOOL doZ`	
`INT4 inCsys`	The ID of the coordinate system where the update will take place.
`BOOL moveRef`	Only used if `entityTYPE=FT_CSYS`. If True, then all entities that reference the coordinate systems being moved will also be updated as a rigid body.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_INVALID`	`EntityTYPE` does not specify coordinate systems, points, or nodes.
`FE_FAIL`	Could not move.
Remarks/Usage:	
You cannot move the global coordinate systems. You can move *solid points* on a Parasolid surface or solid, general body, or sheet solid. When a solid point is moved, any surface which references that point will also be modified.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMoveBy

feMoveBy

(entityTYPE, entitySET, radial, length, vecLoc)

Description:

Moves coordinate systems, points, curves, surfaces, solids, volumes, nodes, or elements along the specified vector, by the specified length. In addition, nodes and elements can optionally be moved radially.

Input:

`INT4 entityTYPE`	The type of entities to be moved. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_CSYS`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`BOOL radial`	If True, entities will be moved radially from a central location. If False, they will be moved along a vector. Only nodes and elements can be moved radially.
`REAL8 length`	The distance to move. If `radial=False`, this is the length the entities are moved along the specified vector.
`REAL8 vecLoc[0..2]`	If `radial=False`, contains the components of the vector to move along, specified in global rectangular coordinates. If `radial=True`, specifies the coordinates of the center that you are moving around, also specified in global rectangular coordinates.

Output:

None	

Return Code:

`FE_INVALID`	Invalid entity type. Must specify one of the types listed in `entityTYPE`. Can only specify nodes or elements if `radial=True`.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not move.
Remarks/Usage:	
You cannot move the global coordinate systems. You can move *solid points or curves* on a Parasolid surface or solid, general body, or sheet solid. When a solid point or curve is moved, any surface which references that point/curve will also be modified.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMoveOffset

feMoveOffset **(elemSET, method, length, alongVec)**	
Description:	
Moves elements by offsetting them from their current location (normally used with planar elements).	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be offset. Alternatively, if you specify a negative value, this is simply the ID of the single entity to offset.
`INT4 method`	The method of determining the offset direction: 0=Along alongVec, 1=along the element normal, 2=along the element normal with thickness corrections.
`REAL8 length`	The distance to offset. If method=0, this length is multiplied by the length of the vector to get the total distance.
`REAL8 alongVec[0..2]`	Only used if method=0. Contains the components of the vector to move along, specified in global rectangular coordinates.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not offset.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feRotateTo

feRotateTo **(entityTYPE, entitySET, axisLoc, axisVec, fromLoc, toLoc)**	
Description:	
Rotates entities around an axis from one location to another.	
Input:	
`INT4 entityTYPE`	The type of entities to be moved. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_CSYS`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be rotated. Alternatively, if you specify a negative value, this is simply the ID of the single entity to rotate.
`REAL8 axisLoc[0..2]`	The coordinates of the origin of the axis of rotation, in global rectangular coordinates.
`REAL8 axisVec[0..2]`	The components of the axis of rotation, in global rectangular coordinates.
`REAL8 fromLoc[0..2]`	The coordinates of a point that you will rotate from, in global rectangular coordinates.
`REAL8 toLoc[0..2]`	The coordinates of a point that you will rotate to, in global rectangular coordinates.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not rotate. Check to make sure that the locations and vectors that you defined are not colinear or otherwise invalid.
Remarks/Usage:	
The axis of rotation is combined with the `fromLoc` and `toLoc` to compute an angle of rotation, which is then applied around the axis of rotation to every entity that you select.	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feRotateBy

feRotateBy **(entityTYPE, entitySET, axisLoc, axisVec, rotAngle, rotThrust)**	
Description:	
Rotates entities around an axis through a specified angle.	
Input:	
`INT4 entityTYPE`	The type of entities to be moved. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_CSYS`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be rotated. Alternatively, if you specify a negative value, this is simply the ID of the single entity to rotate.
`REAL8 axisLoc[0..2]`	The coordinates of the origin of the axis of rotation, in global rectangular coordinates.
`REAL8 axisVec[0..2]`	The components of the axis of rotation, in global rectangular coordinates.
`REAL8 rotAngle`	The angle of rotation, in degrees.
`REAL8 rotThrust`	The total thrust distance (along the axis of rotation), typically zero.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not rotate. Check to make sure that the locations and vectors that you defined are not colinear or otherwise invalid.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feAlignTo

feAlignTo **(entityTYPE, entitySET, fromLoc, fromVec, toLoc, toVec)**	
Description:	
Aligns entities from one vector to another.	
Input:	
`INT4 entityTYPE`	The type of entities to be aligned. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_CSYS`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_NODE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be aligned. Alternatively, if you specify a negative value, this is simply the ID of the single entity to align.
`REAL8 fromLoc[0..2]`	The coordinates of the origin of the original vector, in global rectangular coordinates.
`REAL8 fromVec[0..2]`	The components of the original vector, in global rectangular coordinates.
`REAL8 toLoc[0..2]`	The coordinates of the origin of the destination vector, in global rectangular coordinates.
`REAL8 toVec[0..2]`	The components of the destination vector, in global rectangular coordinates.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not align. Check to make sure that the locations and vectors that you defined are not colinear or otherwise invalid.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feAlignBetweenCsys

feAlignBetweenCsys

(entityTYPE, entitySET, nFromCSysID, nToCSysID)

Description:

Aligns entities from one coordinate system to another.

Input:

`INT4 entityTYPE`	The type of entities to be aligned. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_CSYS`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_NODE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be aligned. Alternatively, if you specify a negative value, this is simply the ID of the single entity to align.
`INT4 nFromCSysID`	The ID of the Coordinate System to use as the *From* Coordinate System
`INT4 nToCSysID`	The ID of the Coordinate System to use as the *To* Coordinate System

Output:

None	

Return Code:

`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not align. Check to make sure that the locations and coordinate systems that you selected are not colinear or otherwise invalid.

Remarks/Usage:

Using this API call is essentially the same as using the **Modify → Align...** commands in the user interface when the **Between Coordinate Systems** method is selected.

Example:

None

Unpublished work. © 2025 Siemens

feScale

feScale **(entityTYPE, entitySET, fromLoc, csysID, xScale, yScale, zScale)**	
Description:	
Scales entities around an origin.	
Input:	
`INT4 entityTYPE`	The type of entities to be aligned. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_CSYS`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_NODE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be scaled. Alternatively, if you specify a negative value, this is simply the ID of the single entity to scale.
`REAL8 fromLoc[0..2]`	The coordinates of the origin around which the entities will be scaled, in global rectangular coordinates.
`INT4 csysID`	The ID of the coordinate system in which scaling will take place.
`REAL8 xScale`	The scale factors in the axis directions of coordinate system `csysID`.
`REAL8 yScale`	
`REAL8 zScale`	
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not scale.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Edit

Unpublished work. © 2025 Siemens

feModifyEdit

feModifyEdit **(entityTYPE, nEntityID)**	
Description:	
Modifies an entity using the dialog box for the entity type available when editing via the User Interface.	
Input:	
`INT4 entityTYPE`	The type of entities to be modified via entity specific dialog box. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_BOUNDARY`, `FT_CONNECTION_PROP`, `FT_CONTACT`, `FT_CONNECTION`, `FT_CSYS`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_LAYUP`, `FT_FUNCTION`, `FT_TEXT`, `FT_AERO_PANEL`, `FT_AERO_PROP`, `FT_AERO_SPLINE`, or `FT_AERO_SURF`.
`INT4 nEntityID`	The ID of the entity to edit via the dialog box for that entity type available in User Interface.
Output:	
None	
Return Code:	
`FE_OK`	User pressed OK in the dialog box.
`FE_CANCEL`	User pressed Cancel in the dialog box.
`FE_FAIL`	Could not modify specified entity.
`FE_INVALID`	`nEntityID` is a point on a Parasolid body, which can not be edited with this command.
`FE_NOT_EXIST`	`nEntityID` does not exist in the model.
`FE_BAD_DATA`	`EntityTYPE` does not specify a valid type.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Color and Layer

Unpublished work. © 2025 Siemens

feModifyColor

feModifyColor **(entityTYPE, entitySET, color, randomize)**	
Description:	
Modifies the color of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_CONNECTION_PROP`, `FT_CONTACT`, `FT_CONNECTION`, `FT_CSYS`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_TEXT`, `FT_AERO_PANEL`, `FT_AERO_PROP`, `FT_AERO_SPLINE`, `FT_AERO_SURF`, or `FT_VOLUME`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`INT4 color`	The new color that you want to assign to the selected entities.
`BOOL randomize`	Only valid for `FT_PROP`, `FT_MATL`, and `FT_SURFACE`. If True, then colors are randomized for the selected entities. The specified color is simply used as initialization for the randomization process.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not modify.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feModifyColorMultiple

feModifyColorMultiple **(entityTYPE, entitySET, color, method, matchRange, limitToGroupSet)**	
Description:	
Modifies the color of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, or `FT_PROP`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`INT4 color`	The first color used when assigning colors. Other colors are assigned *randomly* based on this color.
`INT4 method`	0= Single Color for all entities, 1=Assign color based on entity ID, 2=Assign color based on entity Type, 3=Assign color based on inclusion in Groups
`INT4 matchRange`	Only used when method=1. IDs in bands of this range receive the same color. For example if `matchRange=1000`, then IDs 1000-1999 would all be the same color, as would 2000-2999, 3000-3999, ... Set to 1 for fully random colors.
`INT4 limitToGroupSet`	Only used when method=3. The ID of a Set that contains the IDs of the Groups to consider when looking for entity inclusion. Set to 0 to consider all Groups.
Output:	
None	
Return Code:	
`FE_FAIL`	EntityTYPE does not specify a valid type.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Unpublished work. © 2025 Siemens

feModifyTransparency

feModifyTransparency **(entityTYPE, entitySET, transparency)**	
Description:	
Modifies the transparency of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_SURFACE`, `FT_ELEM`, `FT_PROP`, or `FT_MATL`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`REAL8 transparency`	The transparency value to assign. Must be between 0 (opaque) and 100 (fully transparent).
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not modify.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyLayer

feModifyLayer **(entityTYPE, entitySET, layer)**	
Description:	
Modifies the layer of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_CONNECTION_PROP`, `FT_CONTACT`, `FT_CONNECTION`, `FT_CSYS`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_TEXT`, `FT_AERO_PANEL`, `FT_AERO_PROP`, `FT_AERO_SPLINE`, `FT_AERO_SURF`, or `FT_VOLUME`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`INT4 layer`	The new layer that you want to assign to the selected entities.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not modify.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Renumbering

Unpublished work. © 2025 Siemens

feRenumber

feRenumber	
(entityTYPE, entitySET, startID)	
Description:	
Renumbers the IDs of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_CSYS`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_LAYUP`, `FT_OUT_CASE`, `FT_GROUP`, `FT_LOAD_DIR`, `FT_BC_DIR`, `FT_AMGR_DIR`, `FT_LAYER`, `FT_CONNECTION_PROP`, `FT_CONTACT`, `FT_CONNECTION`, and `FT_FUNCTION_DIR`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`INT4 startID`	The new ID of the first entity to be renumbered. All other entities will be renumbered to available IDs that are higher than this number.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not renumber.
Remarks/Usage:	
For more renumbering options, use `feRenumberOpt( )`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feRenumberOpt

feRenumberOpt

(entityTYPE, entitySET, startID, renumMethod, renumCSysID, renumAbs, renumDescend, renumConstOff)

Description:

Renumbers the IDs of selected entities. This method has been superseded by `feRenumberOpt2`.

Input:

`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_CSYS`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_LAYUP`, `FT_OUT_CASE`, `FT_GROUP`, `FT_LOAD_DIR`, `FT_BC_DIR`, `FT_AMGR_DIR`, `FT_LAYER`, `FT_CONNECTION_PROP`, `FT_CONTACT`, `FT_CONNECTION`, and `FT_FUNCTION_DIR`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`INT4 startID`	The new ID of the first entity to be renumbered. All other entities will be renumbered to available IDs that are higher than this number.
`INT4 renumMethod`	The method used for sorting the entities to be renumbered. 0=ID, 1=Not Available, 2=Color, 3=Layer, 4=X Coord, 5=Y Coord, 6=Z Coord, 7=Reference Entity ID (Material ID if renumbering properties, Property ID if renumbering Elements), 8=Entity Type, 9=Minimum Node ID
`INT4 renumCSysID`	The coordinate system ID in which coordinate renumbering is done. This is only used for the `entityTYPEs` where X,Y,Z sorting is available, and when `renumMethod` is set to one of those options
`BOOL renumAbs`	If True, the absolute value of coordinate values are used when sorting by coordinates.
`BOOL renumDescend`	If True, the sort is done in Descending order, otherwise in Ascending order.

Unpublished work. © 2025 Siemens

`BOOL renumConstOff`	If True, all entities are renumbered with a constant ID offset.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not renumber.
Remarks/Usage:	
This works just like `feRenumber`, but allows you to specify additional renumbering options. Use `feRenumber` for a simpler interface if you are simply renumbering by ID. Not all combinations of `renumMethod` and `entityTYPE` are available - only those combinations available in the interactive renumbering commands will work properly.	
Example:	
None	

Unpublished work. © 2025 Siemens

feRenumberOpt2

feRenumberOpt2

(entityTYPE, entitySET, startID, renumMethod, renumCSysID, renumAbs, renumDescend, renumConstOff, xyzSortOrder)

Description:

Renumbers the IDs of selected entities. Supersedes `feRenumberOpt`.

Input:

`INT4 entityTYPE`	The type of entities to be modified. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_VOLUME`, `FT_SOLID`, `FT_CONNECTION_PROP`, `FT_CONTACT`, `FT_CONNECTION`, `FT_CSYS`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_LAYUP`, `FT_LOAD_DIR`, `FT_LOAD_DEFINITION`, `FT_BC_DIR`, `FT_BC_DEFINITION`, `FT_AMGR_DIR`, `FT_FUNCTION_DIR`, `FT_OUT_CASE`, `FT_GROUP`, `FT_LAYER`, `FT_AERO_PANEL`, `FT_AERO_PROP`, `FT_AERO_SPLINE`, and `FT_AERO_SURF`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be modified. Alternatively, if you specify a negative value, this is simply the ID of the single entity to modify.
`INT4 startID`	The new ID of the first entity to be renumbered. All other entities will be renumbered to available IDs that are higher than this number.
`INT4 renumMethod`	The method used for sorting the entities to be renumbered. 0=ID, 1=Not Available, 2=Color, 3=Layer, 4=X Coord, 5=Y Coord, 6=Z Coord, 7=Reference Entity ID (Material ID if renumbering properties, Property ID if renumbering Elements), 8=Entity Type, 9=Minimum Node ID, 10=Use XYZ Sort Order (`zyzSortOrder`).
`INT4 renumCSysID`	The coordinate system ID in which coordinate renumbering is done. This is only used for the `entityTYPEs` where X,Y,Z sorting is available, and when `renumMethod` is set to one of those options.

Unpublished work. © 2025 Siemens

`BOOL renumAbs`	If True, the absolute value of coordinate values are used when sorting by coordinates.
`BOOL renumDescend`	If True, the sort is done in Descending order, otherwise in Ascending order.
`BOOL renumConstOff`	If True, all entities are renumbered with a constant ID offset.
`INT4 xyzSortOrder[0..2]`	Specifies the XYZ sort order using 3 unique values. 0=None, 1=+X, 2=-X, 3=+Y, 4=-Y, 5=+Z, 6=-Z.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not renumber.
Remarks/Usage:	
This works just like `feRenumber`, but allows you to specify additional renumbering options. Use `feRenumber` for a simpler interface if you are simply renumbering by ID. Not all combinations of renumMethod and `entityTYPE` are available - only those combinations available in the interactive renumbering commands will work properly.	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim elset As femap.Set
   Set elset = App.feSet

   elset.AddAll (FT_ELEM)

  Dim xyzOrder(2) As Long
  'Use +X, +Y, and +Z as the Sort Order
```


Unpublished work. © 2025 Siemens

```
  xyzOrder(0) = 1

  xyzOrder(1) = 3

  xyzOrder(2) = 5


rc = App.feRenumberOpt2 (FT_ELEM, elset.ID, 10000, 10, 0, False, False,
False, xyzOrder)

End Sub
```

Associativity

Unpublished work. © 2025 Siemens

feAutoMeshAssociativity

feAutoMeshAssociativity

**(elemSetID, solidSetID, dTolerance, bClearPrevious, bCheckContainment, bAttachMidside,
 bGroupExtra, bMessages, nUnattachedGroup)**

Description:

Automatically associates both nodes and elements with geometry that is coincident in your model.

Input:

`INT4 elemSetID`	The ID of a set that contains a list of element IDs that will be attached. Alternatively, if you specify a negative value, this is simply the ID of the single entity to attach. Nodes are automatically determined for attachment based on the selected elements.
`INT4 solidSetID`	The ID of a set that contains a list of solid IDs to which the mesh will be attached.
`REAL8 dTolerance`	The tolerance value. Nodes must be within this tolerance distance of the geometry that they will be associated too.
`BOOL bClearPrevious`	If True, removes and previous atttachments from the nodes/elements prior to completing this command.
`BOOL bCheckContainment`	If True and you are associating to multiple solids this will first find the solid that you an element is in by checking the containment of the location of its centroid.
`BOOL bAttachMidside`	If True, midside nodes will be associated as well as corner nodes. Otherwise only corners will be associated.
`BOOL bGroupExtra`	If True, a group is created with all selected nodes and elements which were NOT attached to any geometry during the command.
`BOOL bMessages`	If True, status and warning messages will be written during the processing.

Output:

Unpublished work. © 2025 Siemens

`INT4 nUnattachedGroup`	The ID of a group that will receive any geometry that could not be attached fully.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshAttach

feMeshAttach

(entityTYPE, entitySET, attachTYPE, attachID, doNodes)

Description:

Associates nodes or elements with specified geometry.

Input:

`INT4 entityTYPE`	The type of entities to be attached. For entity information, see **Entity Types**. Can be `FT_NODE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be attached. Alternatively, if you specify a negative value, this is simply the ID of the single entity to attach.
`INT4 attachTYPE`	The type of geometry to attach the node/element to. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, or `FT_SOLID`.
`INT4 attachID`	The ID of the geometry to attach to.
`BOOL doNodes`	Only used if `entityTYPE=FT_ELEM`. In this case, if True, then the nodes on the element will also be associated with the same geometry as the element.

Output:

None	

Return Code:

`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not attach.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feMeshAttachNodes

feMeshAttachNodes **(nNodes, nodeID, attachTYPE, attachID)**	
Description:	
Associates nodes with specified geometry.	
Input:	
`INT4 nNodes`	The number of nodes specified in the other arrays
`INT4 nodeID[0..nNodes-1]`	The IDs of the nodes being attached to geometry.
`INT4 attachTYPE[0..nNodes-1]`	The types of geometry to attach the node/element to. Entries can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, or `FT_SOLID`.
`INT4 attachID[0..nNodes-1]`	The IDs of the geometry to attach each node to.
Output:	
None	
Return Code:	
`FE_INVALID`	One or more `attachTYPEs` do not specify a valid type.
`FE_FAIL`	Could not attach one or more nodes.
Remarks/Usage:	
For each node you want to attach to geometry, there must be corresponding entries in the three arrays.	
Example:	
None	

Unpublished work. © 2025 Siemens

feMeshDetach

feMeshDetach **(entityTYPE, entitySET, attachTYPE, attachID, interior)**	
Description:	
Removes association between nodes or elements and specified geometry.	
Input:	
`INT4 entityTYPE`	The type of entities to be detached. For entity information, see **Entity Types**. Can be `FT_NODE` or `FT_ELEM`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be detached. Alternatively, if you specify a negative value, this is simply the ID of the single entity to detach.
`INT4 attachTYPE`	The type of geometry to attach the node/element to. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, or `FT_SOLID`. You can also specify `attachTYPE=0`, in which case `attachID` is not used, and the node/element is detached from all geometry.
`INT4 attachID`	The ID of the geometry to detach. Not used if `attachTYPE=0`.
`BOOL interior`	Only used if `entityTYPE=FT_NODE`, and `attachTYPE=FT_CURVE`, `FT_SURFACE` or `FT_SOLID`. If True, the node is detached only if it is an interior node.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not detach.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Other Updates

feModifyDefCSys

feModifyDefCSys **(entityTYPE, entitySET, csysID)**	
Description:	
Updates the definition coordinate system of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be updated. For entity information, see **Entity Types**. Can be `FT_CSYS`, `FT_POINT`, or `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single entity to update.
`INT4 csysID`	The ID of the coordinate system to use as the definition coordinate system.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not update. Check to see that coordinate system exists.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

feModifyOutCSys

feModifyOutCSys **(entityTYPE, entitySET, csysID)**	
Description:	
Updates the output coordinate system of selected entities.	
Input:	
`INT4 entityTYPE`	The type of entities to be updated. For entity information, see **Entity Types**. Must be `FT_NODE`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single entity to update.
`INT4 csysID`	The ID of the coordinate system to use as the output coordinate system.
Output:	
None	
Return Code:	
`FE_INVALID`	`entityTYPE` does not specify a valid type.
`FE_FAIL`	Could not update. Check to see that coordinate system exists.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyPermBC

feModifyPermBC **(entitySET, t1, t2, t3, r1, r2, r3)**	
Description:	
Updates the permanent constraints on nodes.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of node IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single node to update.
`BOOL t1`	Corresponds to the six degrees of freedom of each node. If True, the associated degree of freedom is constrained.
`BOOL t2`	
`BOOL t3`	
`BOOL r1`	
`BOOL r2`	
`BOOL r3c`	
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifySplineOrder

feModifySplineOrder **(entitySET, order)**	
Description:	
Updates the order of b-spline curves.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of curve IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single curve to update.
`INT4 order`	The new order of the b-spline curve. Must be a value between 3 and 10.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
Only used with standard geometry engine b-splines - not solid engine curves.	
Example:	
None	

feModifySplineKnots

feModifySplineKnots **(entitySET, knotLoc)**
Description:
Insert knots into b-spline curves.

Unpublished work. © 2025 Siemens

Input:	
`INT4 entitySET`	The ID of a set that contains a list of curve IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single curve to update.
`REAL8 knotLoc[0..2]`	The location, in global rectangular coordinates, of the knot to insert.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
Only used with standard geometry engine b-splines - not solid engine curves.	
Example:	
None	

feModifyReverseCurve

`feModifyReverseCurve` **`(  entitySET )`**	
Description:	
Reverses the direction of selected curves.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of curve IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single curve to update.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not update.
Remarks/Usage:	
Only used with standard geometry engine curves - not solid engine curves.	
Example:	
None	

feModifyNoMergeCurve

`feModifyNoMergeCurve` **`(  entitySET, bNoMerge )`**	
Description:	
Marks selected curves as mergeable or nonmergeable.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of curve IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single curve to update.
`BOOL bNoMerge`	If True, curves are made nonmergeable. If False, they are mergeable.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
Only used with solid engine curves - not standard geometry engine curves.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifySurfaceDivisions

feModifySurfaceDivisions **(entitySET, sDiv, tDiv, tolerance)**	
Description:	
Updates the graphical divisions and facetting used to display surfaces.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of surface IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single surface to update.
`INT4 sDiv`	The number of divisions displayed along the first parametric direction of the surface.
`INT4 tDiv`	The number of divisions displayed along the second parametric direction of the surface.
`REAL8 tolerance`	The accuracy with which the graphical facetting of solid geometry engine surfaces is created. Decreasing this number results in more facets and slower draw times, but a closer approximation to the actual geometry. Increasing the number speeds up draw times, but can lead to coarse approximations of the true geometry.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifySurfaceNormal

feModifySurfaceNormal **(entityTYPE, entitySET)**	
Description:	
Reverses the normal direction of surfaces of Parasolid Sheet Solids. Only surfaces which are part of a Parasolid Sheet Solid have their normal direction reversed. Also, if only a subset of surfaces comprising a sheet solid are selected, ALL of the surfaces will be reversed, not just the selected surfaces.	
Input:	
`INT4 entityTYPE`	The type of entities to be updated. For entity information, see **Entity Types**. Can be `FT_SURFACE` or `FT_SOLID`.
`INT4 entitySET`	The ID of a set that contains a list of surface or solid (sheet solids only) IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single surface or sheet solid to update.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
It is important that no surfaces which are on a Parasolid Solid are included in the set of surfaces, otherwise the normal direction will not be reversed on any surfaces, not just the surfaces on the Solid(s).	
Example:	
None	

feModifySolidFacetting

Unpublished work. © 2025 Siemens

feModifySolidFacetting **(entitySET, sDiv, tDiv, dAngleTol, dChordTol, dCurveFactor)**	
Description:	
Updates the graphical divisions and facetting used to display solids.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of solid IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single solid to update.
`INT4 sDiv`	The number of divisions displayed along the first parametric direction of the surfaces of the solid.
`INT4 tDiv`	The number of divisions displayed along the second parametric direction of the surfaces of the solid.
`REAL8 dAngleTol`	The percent of angle error allowed between the solid and facets
`REAL8 dChordTol`	The percent of chord error allowed between the solid and facets
`REAL8 dCurveFactor`	The refinement factor for curve facets compared to the default solid facetting.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
None	
Example:	
None	

feModifySuperelementID

Unpublished work. © 2025 Siemens

feModifySuperelementID **(entitySET, nSEID)**	
Description:	
Updates the superelementID of a set of nodes.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of node IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single node to update.
`INT4 nSEID`	The superelementID to assign to the nodes.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
None	
Example:	
None	

feModifyLoadPhase

feModifyLoadPhase **(entityTYPE, entitySET, phase)**
Description:
Updates the phase of selected loads.
Input:

Unpublished work. © 2025 Siemens

`INT4 entityTYPE`	The type of entities where the loads to be modified are applied. For entity information, see **Entity Types**. Can be `FT_NODE`, `FT_ELEM`, `FT_POINT`, `FT_CURVE`, or `FT_SURFACE`. This is the type of entity that is selected in `entitySET`.
`INT4 entitySET`	The ID of a set that contains a list of entity IDs where the loads are applied. Alternatively, if you specify a negative value, this is simply the ID of the single entity with a load applied.
`REAL8 phase`	The new phase value.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not update.
Remarks/Usage:	
Only loads in the active load set will be updated. The selection/specification of entities refers to the entities where the loads are applied - not the loads themselves.	
Example:	
None	

Element Updates

Unpublished work. © 2025 Siemens

feModifyElemPropID

feModifyElemPropID **(entitySET, propID)**	
Description:	
Updates the property ID of selected elements.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 propID`	The ID of the new property to assign to the elements. This property must be compatible with the element types chosen.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not update. Check to see that property exists and is of the correct type.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyPropMatlID

feModifyPropMatlID **(entitySET, matlID**	
Description:	
Updates the material ID of selected properties.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of property IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single property to update.
`INT4 matlID`	The ID of the new material to assign to the properties.
Output:	
None	
Return Code:	
`FE_INVALID`	`EntityTYPE` does not specify a valid type.
`FE_FAIL`	Could not update. Check to see that material exists.
Remarks/Usage:	
This method only updates the primary material identifier. Laminate properties cannot be updated using this method.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyMaterialAngle

feModifyMaterialAngle **(entitySET, method, vecDir, coordDir, coordCSys, angle)**	
Description:	
Updates the material angle of selected entities.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 method`	The method for specifying material angles: 0=None/Off, 1=Vector, 2=CoordSys, 3=Angle.
`REAL8 vecDir[0..2]`	The vector to be used for definition of material angles if method=1.
`INT4 coordDir`	Sets the direction (0=X,1=Y,2=Z) in the specified coordinate system to be used for material angles. Only used if method=2.
`INT4 coordCSys`	Sets the coordinate system ID to be used if method=2.
`REAL8 angle`	Sets the angle to be used for the definition of material angles when method=3.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not define material angle.
Remarks/Usage:	
To turn material angles off for the selected elements, simply specify method=0.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyElemReverse2

feModifyElemReverse2

(entitySET, method, vecDir, csysID, csysDir, bPreserveMatlDir)

Description:

Reverses the normal of planar elements, or optionally spins them to orient the first edge along a vector.

Input:

`INT4 entitySET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 method`	The method for updating the elements: 0=Reverse, 1=Normals Outward, 2=Normals Inward, 3=Align to Vector, 4=Align to CSys Direction.
`REAL8 vecDir[0..2]`	The vector to be used for aligning the first edge of the elements if method=3.
`INT4 csysID`	ID of Coordinate System to use if method=4.
`INT4 csysDir`	Coordinate system axis to use to align first edge if method=4. This must have values 1=X (or R), 2=Y (or theta), 3=Z (or phi). You can also specify -1,-2, or -3 to choose the negative coordinate system direction.
`BOOL bPreserveMatlDir`	Set to True to preserve the material direction of the elements being reversed or reoriented. Set to False to update the material direction of elements being reversed or reoriented.

Output:

None	

Return Code:

`FE_FAIL`	Could not update.

Remarks/Usage:

This method should only be used with planar elements.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feModifyElemReverse

feModifyElemReverse

(entitySET, method, vecDir)

Description:

Reverses the normal of planar elements, or optionally spins them to orient the first edge along a vector. This method is obsolete - `feModifyElemReverse2( )` provides additional functionality.

Input:

`INT4 entitySET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 method`	The method for updating the elements: 0=Reverse, 1=Normals Outward, 2=Normals Inward, 3=Align to Vector.
`REAL8 vecDir[0..2]`	The vector to be used for aligning the first edge of the elements if method=3.

Output:

None	

Return Code:

`FE_FAIL`	Could not update.

Remarks/Usage:

This method should only be used with planar elements.

Example:

None

Unpublished work. © 2025 Siemens

feModifyElemRelease

feModifyElemRelease **(entitySET, rel1, rel2)**	
Description:	
Updates the releases at the ends at the ends of beam (and other) elements.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`BOOL rel1[0..5]`	The release flags for End 1 of the element. Specify True to release a corresponding elemental degree of freedom.
`BOOL rel2[0..5]`	The release flags for End 2 of the element. Specify True to release a corresponding elemental degree of freedom.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method should only be used with beam or bar elements.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyElemOrder

feModifyElemOrder **(entitySET, makeParabolic, ontoGeometry, maxAngle)**	
Description:	
Updates the order of elements between linear and parabolic.	
Input:	
`INT4 entitySET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`BOOL makeParabolic`	Set to True to convert elements to parabolic. Set to False to convert to linear elements. Midside nodes will be created when converting to parabolic.
`BOOL ontoGeometry`	If you are converting to parabolic, set to True to project the midside nodes onto any referenced geometry. Set to False to leave the nodes at the midside.
`REAL8 maxAngle`	If you are projecting midside nodes onto geometry, this is the maximum angle that any edge can be distorted by the movement.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method should only be used with elements that have both linear and parabolic types.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyMidside

feModifyMidside **(elemSET, nodeSET)**	
Description:	
Updates the midside nodes of selected elements to be at the middle of the element edge.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 nodeSET`	The ID of a set that contains a list of node IDs that will not be updated. All midside nodes on the elements in `elemSET` are normally updated. If you want to skip some nodes, include them in this set. Specify `nodeSET=0` if you want to update all midside nodes.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyFormulation

feModifyFormulation **(elemSET, form1, form2)**	
Description:	
Updates the element formulation for selected elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 form1`	The formulation values.
`INT4 form2`	
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyOffsets

feModifyOffsets **(elemSET, endA, endB, radial, offA, offB)**	
Description:	
Modifies the offsets of beam and bar elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`BOOL endA`	If True, the offset at the first end of the element will be updated.
`BOOL endB`	If True, the offset at the second end of the element will be updated.
`BOOL radial`	If True, the offsets are radial, around a location, instead of being vector based.
`REAL8 offA[0..2]`	If radial=True, these are the coordinates of the center location. If radial=False, then these are components of the vector offset at the first end of the element. In either case, the values are specified in global rectangular coordinates.
`REAL8 offB[0..2]`	If radial=True, then `offB[0]` is the radial offset length, and the other `offB` values are not used. If radial=False, then `offB` contains the components of the vector offset at the second end of the element.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method can only be used with elements that have offsets: beams and bars.	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feModifyOffsetsToRefPt

feModifyOffsetsToRefPt

(elemSET)

Description:

Modifies the offsets of beam and bar elements so that the shape reference point is at the nodes.

Input:

`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.

Output:

None	

Return Code:

`FE_FAIL`	Could not update.

Remarks/Usage:

This method can only be used with elements that have offsets: beams and bars. Likewise, it can only be used if the reference point on the property associated with the elements has been properly defined. Offsets for each element are redefined so that the reference point is moved to the nodal location.

Example:

None

Unpublished work. © 2025 Siemens

feModifyOffsetsToRefPt2

`feModifyOffsetsToRefPt2`
 	
`( elemSET, dYOff, dZOff)`	

Description:	
Modifies the offsets of beam and bar elements so that the shape reference point is at the nodes by a specified distance.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`REAL8 dYOff`	The distance that the reference point is offset from the node in the Elemental Y direction
`REAL8 dZOff`	The distance that the reference point is offset from the node in the Elemental Z direction
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method can only be used with elements that have offsets: beams and bars. Likewise, it can only be used if the reference point on the property associated with the elements has been properly defined. Offsets for each element are redefined so that the reference point is moved to the nodal location by the specified amount.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyRadialOffsets

feModifyRadialOffsets **(elemSET, bAroundVector, dRadialOffset, vOrig, vAxis)**	
Description:	
Modifies the offsets of beam and bar elements in a radial direction.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`BOOL bAroundVector`	If True, the offsets will be radial around the specified vector. If False, they will be radial around the location specified by `vOrig`.
`REAL8 dRadialOffset`	The distance to offset radially around the vector or specified point.
`REAL8 vOrig[0..2]`	The coordinates of either the origin of the vector or the single location to offset around. Specified in Global Rectangular coordinates.
`REAL8 vAxis[0..2]`	The components of the Vector to offset around if `bAroundVector` is set to True.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to offset the selected elements.
Remarks/Usage:	
This method can only be used with line elements that have offsets, such as beams and bars.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyRadialOffsets2

feModifyRadialOffsets2

(elemSET, bAroundVector, dRadialOffset, dOffsetAlongVector, vOrig, vAxis)

Description:

Modifies the offsets of beam and bar elements in a radial direction.

Input:

`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`BOOL bAroundVector`	If True, the offsets will be radial around the specified vector. If False, they will be radial around the location specified by `vOrig`.
`REAL8 dRadialOffset`	The distance to offset radially around the vector or specified point.
`REAL8 dOffsetAlongVector`	The distance to offset along the specified vector if `bAroundVector` is set to True.
`REAL8 vOrig[0..2]`	The coordinates of either the origin of the vector or the single location to offset around. Specified in Global Rectangular coordinates.
`REAL8 vAxis[0..2]`	The components of the Vector to offset around if `bAroundVector` is set to True.

Output:

None	

Return Code:

`FE_FAIL`	Unable to offset the selected elements.

Remarks/Usage:

This method can only be used with line elements that have offsets, such as beams and bars.

Example:

None

Unpublished work. © 2025 Siemens

feModifyOrient

feModifyOrient **(elemSET, method, nNodeOrCSysID, orientVec)**	
Description:	
Modifies the orientation of line elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 method`	Defines the method used for updating the orientation: 0=Orient with third node, 1=Orient with Vector defined in Global Rectangular Coordinates, -1= Orient with Vector defined in Coordinate System specified for `nNodeOrCSysID`, 2=Convert existing orientation to equivalent vectors, 3=Make existing orientation perpendicular to elements, 4=Orient Radially with regard to Coordinate System specified for `nNodeOrCSysID`, 5=Orient by applying Orientation Coordinate System specified for `nNodeOrCSysID`.
`INT4 nNodeOrCSysID`	If method=0, ID of the node to use as a third node for orientation. If method=-1, this contains the ID of the coordinate system in which the `orientVec` is specified. If method=5, this contains the ID of Coordinate System to use as an Orientation Coordinate System.
`REAL8 orientVEC[0..2]`	The components, in global rectangular coordinates, of the orientation vector to use if method=1. The components of the orientation vector, in coordinate system specified for `nNodeOrCSysID` to use if method=-1. If method=4, values are ignored.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method can only be used with elements that have orientations. Method=4 can be used to orient elements in a radial direction by specifying any type of coordinate system. Method=5 should only be used to specify an Orientation Coordinate System for Spring/Damper elements referencing a CBUSH Property or Spring/Damper to Ground elements.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyElemType

feModifyElemType **(elemSET, propID, method, nodeID, orientVec)**	
Description:	
Modifies the element type of elements.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 propID`	The ID of the new property to assign to the elements. The element type is changed to match the type of the property.
`INT4 method`	Defines the method used for updating the orientation: 0=Orient with third node, 1=Orient with Vector. This parameter is only used if you are modifying line elements with no orientation to a type that requires orientation.
`INT4 nodeID`	The ID of the node to use as a third node for orientation - only used if method=0. This parameter is only used if you are modifying line elements with no orientation to a type that requires orientation.
`REAL8 orientVEC[0..2]`	The components, in global rectangular coordinates, of the orientation vector to use if method=1. This parameter is only used if you are modifying line elements with no orientation to a type that requires orientation.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method will only work to change elements from one type to another if the element shapes are compatible.
Example:
None

Unpublished work. © 2025 Siemens

feModifyBeamWarping

feModifyBeamWarping **(elemSET, method, nodeType,nodeID_endA, nodeID_endB)**	
Description:	
Modifies/Creates warping NODES or SPOINTS for Beam Warping.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`INT4 method`	Defines the method for updating the warping node. 0 = None(Remove), 1 = Continuous, 2 = All Continuous, 3 = Discontinuous.
`INT4 nodeType`	The type of node to create 0 = Node, 1 = SPOINT.
`INT4 nodeID_endA, nodeID_endB`	If the `elemSET` contains a single element then the warping node for `endA` and `endB` will be set to `nodeID_endA`, `nodeID_endB` respectively.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method can only be used with Beam elements.	
Example:	
None	

Unpublished work. © 2025 Siemens

feModifyLineElementDirection

feModifyLineElementDirection **(elemSET, reverseMode, nAlignToElemID, AlignToVector)**	
Description:	
Modifies line element directions.	
Input:	
`INT4 elemSET`	The ID of a set that contains a list of element IDs that will be updated. Alternatively, if you specify a negative value, this is simply the ID of the single element to update.
`ENUM reverseMode`	Specifies whether to reverse direction, align to element, or align to vector.
`INT4 nAlignToElemID`	The ID of the element to align direction to.
`REAL8 AlignToVector[0..2]`	A variant with the vector direction from original to set the element directions to.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
This method can only be used with line elements.	
Example:	
None	

Unpublished work. © 2025 Siemens

feConnectLinearParabolic

feConnectLinearParabolic

(enType, nSetID, bAddMidside, bMerge, bGroupBadMidnodes, bGroupUpdatedElements)

Description:

The method connects linear to parabolic elements.

Input:

INT4 enType	The entity type for connecting selection. 5 = FT_SURFACE, 7 = FT_NODE, 8 = FT_ELEM, 39 = FT_SOLID.
INT4 nSetID	The ID of the set of entities to use for connecting.
BOOL bAddMidside	If True, the midside nodes are added to the linear elements. If False, the midside nodes are removed from the parabolic elements.
BOOL bMerge	If True, nodes will be merged before updating.
BOOL bGroupBadMidnodes	If True, disconnected nodes will be added to a group.
BOOL bGroupUpdateElements	If True, the updated elements will be added to a group.

Output:

None	

Return Code:

FE_BAD_TYPE	Wrong entity type was used.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

feModifyPlateThickOffsetBetweenNodes

feModifyPlateThickOffsetBetweenNodes

(nSetID, nUpdating, nFromNode, dFromValue, nToNode, dToValue, bAverageForElement, dTolerance, dMinValue, dMaxValue)

Description:

Modifies the **Element Thickness**, **Property Thickness**, or **Element Offset** of the specified elements by varying the thickness or offset from the value specified at one node (**From Node**) to the value specified at another node (**To Node**). This method provides programmatic access to functionality of the **Modify → Update Elements → Adjust Plate Thickness/Offset** command when **Method** is set to **Vary Between Nodes**.

Input:

`INT4 nSetID`	The ID of a set that contains a list of element IDs that will have the thickness or offset modified

`ENUM nUpdating`	Specifies what is being modified.
	`FPTO_ELEM_THICKNESS = Element Thickness (0)`
	`FPTO_PROP_THICKNESS = Property Thickness (1)`
	`FPTO_ELEM_OFFSET = Element Offset (2)`

`INT4 nFromNode`	The ID of the node to start modification.
`REAL8 dFromValue`	Staring value for modification at `nFromNode`.
`INT4 nToNode`	The ID of the node to end modification.
`REAL8 dToValue`	Ending value for modification at `nToNode`.
`BOOL bAverageForElement`	Only used when `nUpdating` is set to `FPTO_ELEM_THICKNESS` or `FPTO_PROP_THICKNESS`. When True, applies a single thickness value to an element, based on the average of the thickness values at the element's

Unpublished work. © 2025 Siemens

	corners. When False, the corner values are used directly, creating an element with a variable thickness. See Remarks.
`REAL8 dTolerance`	Tolerance used to aggregate or round computed values.
`REAL8 dMinValue`	Specifies the minimum value. Anything lower will be assigned this value. Set to 0.0 for no limit.
`REAL8 dMaxValue`	Specifies the maximum value. Anything higher will be assigned this value. Set to 0.0 for no limit.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
When nUpdating is set to `FPTO_PROP_THICKNESS`, additional properties will be created, as that is the only way new thickness values can be assigned to the elements by property. Also, `FPTO_ELEM_THICKNESS` should only be used when creating a model for Nastran or another solver which supports thickness overrides of property values at element corners.	
Example:	

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim elset As femap.Set
   Set elset = App.feSet

elset.Select (FT_ELEM, True, "Select Elements")

    rc = App.feModifyPlateThickOffsetBetweenNodes (elset.ID,
FPTO_ELEM_THICKNESS, 121, 0.1, 111, 0.2, False, 1.E-8, 0, 0)

App.feWindowRegenerate (0)
```


Unpublished work. © 2025 Siemens

```
End Sub
```


Unpublished work. © 2025 Siemens

feModifyPlateThickOffsetValue

feModifyPlateThickOffsetValue

(nSetID, nUpdating, nMethod, dValue, bAverageForElement, dTolerance, dMinValue, dMaxValue)

Description:

Modifies the **Element Thickness**, **Property Thickness**, or **Element Offset** of the specified elements by varying the thickness or offset using a specified method. This method provides programmatic access to functionality of the **Modify** → **Update Elements** → **Adjust Plate Thickness/Offset** command when **Method** is set to **Equation** or **Constant**, but is specifically designed to work with *constants*. To use the *Equation* capabilities, see **feModifyPlateThickOffsetEquation**.

Input:

`INT4 nSetID`	The ID of a set that contains a list of element IDs that will have the thickness or offset modified.

`ENUM nUpdating`	Specifies what is being modified.
	
	`FPTO_ELEM_THICKNESS = Element Thickness (0)`
	
	`FPTO_PROP_THICKNESS = Property Thickness (1)`
	
	`FPTO_ELEM_OFFSET = Element Offset (2)`

`ENUM nMethod`	Specifies the modification method. Available for all `nUpdating` options: `FPTM_SETTOVALUE = Value (0)`. Simply specifies a single value for all elements. `FPTM_INCREMENT = Increment (1)`. Value can be positive or negative and will simply be added to or subtracted from the existing thickness/offsets. `FPTM_ADJUSTPCT = Percent (2)`. Value can be positive or negative, must be entered as a percentage, and is used to modify the thickness/offsets by multiplying the original thickness/

Unpublished work. © 2025 Siemens

	offsets by 1/100 of entered value, then adding the value to or subtracting the value from the original thickness/offsets. Only available when nUpdating is set to `FPTO_ELEM_OFFSET`: `FPTM_TOPTONODE = Top At Nodes (3)`. Creates the appropriate offsets in order to have nodes of each element coincide with the *top* of each element, based on thickness `FPTM_BOTTOMTONODE = Bottom At Nodes (4)`. Creates the appropriate offsets in order to have nodes of each element coincide with the *bottom* of each element, based on thickness.
`REAL8 dValue`	Value used when `nMethod` is set to `FPTM_SETTOVALUE`, `FPTM_INCREMENT`, or `FPTM_ADJUSTPCT`.
`BOOL bAverageForElement`	Only used when `nUpdating` is set to `FPTO_ELEM_THICKNESS` or `FPTO_PROP_THICKNESS`. When True, applies a single thickness value to an element, based on the average of the thickness values at the element's corners. When False, the corner values are used directly, creating an element with a variable thickness. See Remarks.
`REAL8 dTolerance`	Tolerance used to aggregate or round computed values.
`REAL8 dMinValue`	Specifies the minimum value. Anything lower will be assigned this value. Set to 0.0 for no limit.
`REAL8 dMaxValue`	Specifies the maximum value. Anything higher will be assigned this value. Set to 0.0 for no limit.
Output:	
None	
Return Code:	
`FE_FAIL`	Could not update.
Remarks/Usage:	
When `nUpdating` is set to `FPTO_PROP_THICKNESS`, additional properties will be created, as that is the only way new thickness values can be assigned to the elements by property. Also, `FPTO_ELEM_THICKNESS` should only be used when creating a model for Nastran or another solver which supports thickness overrides of property values at element corners.	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feModifyPlateThickOffsetEquation

feModifyPlateThickOffsetEquation

(nSetID, nUpdating, nMethod, sNodeElemVariable, sEquation, bAverageForElement, dTolerance, dMinValue, dMaxValue)

Description:

Modifies the **Element Thickness**, **Property Thickness**, or **Element Offset** of the specified elements by varying the thickness or offset using a specified method. This method provides programmatic access to functionality of the **Modify** → **Update Elements** → **Adjust Plate Thickness/Offset** command when **Method** is set to **Equation** or **Constant**, but is specifically designed to work with *equations*. To use the *Equation* capabilities, see **feModifyPlateThickOffsetValue**.

Input:

`INT4 nSetID`	The ID of a set that contains a list of element IDs that will have the thickness or offset modified

`ENUM nUpdating`	Specifies what is being modified.
	
	`FPTO_ELEM_THICKNESS = Element Thickness (0)`
	
	`FPTO_PROP_THICKNESS = Property Thickness (1)`
	
	`FPTO_ELEM_OFFSET = Element Offset (2)`

`ENUM nMethod`	Specifies the modification method which evaluates the equation. Available for all `nUpdating` options: `FPTM_SETTOVALUE = Value (0)`. Simply specifies uses the single value for all elements. `FPTM_INCREMENT = Increment (1)`. Value can be positive or negative and will simply be added to or subtracted from the existing thickness/offsets. `FPTM_ADJUSTPCT = Percent (2)`. Value can be positive or negative, must be entered as a

Unpublished work. © 2025 Siemens

	percentage, and is used to modify the thickness/offsets by multiplying the original thickness/offsets by 1/100 of entered value, then adding the value to or subtracting the value from the original thickness/offsets. Does NOT evaluate the entered equation. Only used when `nUpdating` is set to `FPTO_ELEM_OFFSET:`. `FPTM_TOPTONODE = Top At Nodes (3)`. Creates the appropriate offsets in order to have nodes of each element coincide with the *top* of each element, based on thickness. `FPTM_BOTTOMTONODE = Bottom At Nodes (4)`. Creates the appropriate offsets in order to have nodes of each element coincide with the *bottom* of each element, based on thickness.
`STRING sNodeElemVariable`	String that contains the name of the variable used for Nodes if updating thickness or Elements if updating offsets. Typically set to *i*, as it is in the dialog box of the command in the user interface.
`STRING sEquation`	String that contains the equation, which typically would contain references to node or element locations and use the value specified for `sNodeElemVariable`. See Remarks.
`BOOL bAverageForElement`	Only used when `nUpdating` is set to `FPTO_ELEM_THICKNESS` or `FPTO_PROP_THICKNESS`. When True, applies a single thickness value to an element, based on the average of the thickness values at the element's corners. When False, the corner values are used directly, creating an element with a variable thickness. See Remarks.
`REAL8 dTolerance`	Tolerance used to aggregate or round computed values.
`REAL8 dMinValue`	Specifies the minimum value. Anything lower will be assigned this value. Set to 0.0 for no limit.
`REAL8 dMaxValue`	Specifies the maximum value. Anything higher will be assigned this value. Set to 0.0 for no limit.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

FE_FAIL	Could not update.

Remarks/Usage:

Typically, the Femap *Functions* used for equations this command are:

Node = XND(!i), YND(!i), ZND(!i).
Element Centroid = XEL(!i), YEL(!i), ZEL(!i).
Element Face = XEF(!i;faceID), YEF(!i;faceID), ZEF(!i;faceID), where face ID is a value from 1 to 6 specified by the user.

For a full list of available Femap *Functions*, see *Appendix C - Function Reference* of the Femap User Guide.

When nUpdating is set to FPTO_PROP_THICKNESS, additional properties will be created, as that is the only way new thickness values can be assigned to the elements by property. Also, FPTO_ELEM_THICKNESS should only be used when creating a model for Nastran or another solver which supports thickness overrides of property values at element corners.

Example:

None

Unpublished work. © 2025 Siemens

feModifyPlateThickOffsetReset

feModifyPlateThickOffsetReset **(nSetID, bResetElementThickness, bResetElementOffset)**	
Description:	
Resets the **Element Thickness** and/or **Element Offset** of the specified elements. This method provides programmatic access to functionality of the **Reset Element Thickness** and **Reset Element Offset** buttons of the **Modify** → **Update Elements** → **Adjust Plate Thickness/Offset** command.	
Input:	
`INT4 nSetID`	The ID of a set that contains a list of element IDs that will have the element thickness and/or offset reset.
`BOOL bResetElementThickness`	When True, resets the element thickness property overrides for the specified elements. When False, does nothing.
`BOOL bResetElementOffset`	When True, resets the element offsets for the specified elements. When False, does nothing.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This command cannot remove any thickness values for plate elements specified on Properties. It is only removing the thickness values specified for elements which override the property thickness values.	
Example:	
None	

Delete Methods

Unpublished work. © 2025 Siemens

feDeleteAll

feDeleteAll **(geometry, mesh, output, confirm)**	
Description:	
This method deletes entire portions of your model - either geometry or the finite element mesh.	
Input:	
BOOL geometry	If True, all geometry in the model will be deleted - points, curves, surfaces, solids, volumes, etc.
BOOL mesh	If True, the entire finite element mesh, including all loads, constraints, properties, all results, etc. will be deleted.
BOOL output	If True, all results will be deleted. If you set mesh=True, then this is ignored and results will always be deleted.
BOOL confirm	If False, the user will be asked to confirm the delete operation. If True, the selected portions of the model will be deleted without warning.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Coordinate systems will only be deleted if both geometry and the mesh are deleted.	
Example:	
None	

Unpublished work. © 2025 Siemens

feDelete

feDelete **(entityTYPE, entitySET)**	
Description:	
This method deletes selected entities from your model.	
Input:	
`INT4 entityTYPE`	The entity type to delete: 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 9=Coordinate System, 10=Material, 11=Property, 12=Load Set, 17=Constraint Set, 21=Text, 22=View, 24=Group, 28=Output Set, 31=Report, 33=Layer, 35=Function, 39=Solid, 58=Connection Region (Contact Segment), 60=Analysis Set (in Analysis Set Manager), 71=Connector, 72=Connection Property, 81=Layup, 95=Aero Panel/Body, 96=Aero Property, 98=Aero Spline, 99=Aero Control Surface, 102=Freebody, 106=Chart, 107=Data Series.
`INT4 entitySET`	The ID of the set containing the IDs of the entities to delete.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid `entityTYPE` - the `entityTYPE` must be one of the values listed above.
Remarks/Usage:	
Entities will only be deleted if they are not referenced by other entities. You should always delete entities in the order of their references to each other in the model. For example, delete elements first, properties next, and materials last.	
Example:	
None	

Unpublished work. © 2025 Siemens

feDeleteMesh

feDeleteMesh **(entityTYPE, entitySET, bDeleteUnused)**	
Description:	
This method deletes entire meshes from your model.	
Input:	
`INT4 entityTYPE`	Must be either 7=Node or 8=Element.
`INT4 entitySET`	The ID of the set containing the IDs of the nodes/elements which define the mesh to be deleted. If `entitySET<0`, it is the ID of the single entity where the mesh will be deleted.
`BOOL bDeleteUnused`	If True, then any unused Properties and Materials will also be deleted.
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	`entityTYPE` must be either Node or Element.
Remarks/Usage:	
If you specify elements, those will be deleted. If you specify nodes, any element referencing any of the specified nodes will be deleted.	
Example:	
None	

Unpublished work. © 2025 Siemens

feDeleteLoads

feDeleteLoads **(loadSetID, entityTYPE, entitySET, loadTYPE)**	
Description:	
This method deletes selected loads from your model.	
Input:	
`INT4 loadSetID`	The ID of the Load Set containing the loads to be deleted.
`INT4 entityTYPE`	The entity type of the entities specified in `entitySET`. Loads of *loadTYPE* will be deleted from these entities. Valid values are: 3=Point, 4=Curve, 5=Surface, 7=Node, 8=Element.
`INT4 entitySET`	The ID of the set containing the IDs of the entities where loads will be deleted. If `entitySET<0`, it is the ID of the single entity where loads will be deleted.
`INT4 loadTYPE`	The type of loads to delete. Any=-1, Nodal Heat Gen=2, Nodal Heat Flux=4, Force=16, Displacement=32, Acceleration=64, LineLoad=128, Pressure=256, Elem Heat Gen=512, Elem Heat Flux=1024, Velocity=2048, Nodal Temp=4096, Elem Temp=8192, Convection=16384, Radiation=32768, Transient=65536, Fluid Pressure=131072, Fluid Tracking = 262144, Fluid Unknown=524288, Fluid Slip=1048576, Fluid Fan=2097152, Fluid Periodic=4194304
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid `entityTYPE` or `loadTYPE` - they must be one of the values listed above.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

feDeleteConstraints

feDeleteConstraints **(bcSetID, entityTYPE, entitySET, deleteBC, deleteBE)**	
Description:	
This method deletes selected constraints from your model.	
Input:	
`INT4 bcSetID`	The ID of the Constraint Set containing the constraints to be deleted.
`INT4 entityTYPE`	The entity type of the entities specified in `entitySET`. Constraints will be deleted from these entities. Valid values are : 3=Point, 4=Curve, 5=Surface, 7=Node
`INT4 entitySET`	The ID of the set containing the IDs of the entities where constraints will be deleted. If `entitySET<0`, it is the ID of the single entity where constraints will be deleted.
`BOOL deleteBC`	If True, normal constraints will be deleted.
`BOOL deleteBE`	If True, constraint equations will be deleted.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified an invalid `entityTYPE` - it must be one of the values listed above.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feDeleteOutputV2

feDeleteOutputV2 **(outputSetID, vectorSET)**	
Description:	
This method deletes selected output vectors from a single output set. To delete output vectors from any number of selected output sets, use `feDeleteOutput2`. This method replaces `feDeleteOutput`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 outputSetID`	The ID of the Output Set from which to delete the vectors.
`INT4 vectorSET`	The ID of the set containing the IDs of the output vectors to delete. Alternatively, if you specify a negative value, this is simply the ID of a single output vector to delete.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
It may be helpful to use the `AddComponentOutputVectors` (**AddComponentOutputVectorsV2**), `AddSimilarOutputVectors` (**AddSimilarOutputVectorsV2**), and/or `AddComplexOutputVectors` (**AddComplexOutputVectorsV2**) methods on the Set Object to select all desired output vectors.	
Example:	
None	

Unpublished work. © 2025 Siemens

feDeleteOutput2V2

feDeleteOutput2V2 **(outputSetSET, vectorSET)**	
Description:	
This method deletes selected output vectors from selected output sets. This method replaces `feDeleteOutput2`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 outputSetSET`	The ID of the set containing the IDs of the output sets to delete output vectors from. Alternatively, if you specify a negative value, this is simply the ID of a single output set to delete output vectors from.
`INT4 vectorSET`	The ID of the set containing the IDs of the output vectors to delete. Alternatively, if you specify a negative value, this is simply the ID of a single output vector to delete.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This function offers the same capabilities as the **Delete → Output → Vector** command in the User Interface. It may be helpful to use the `AddComponentOutputVectors` (**AddComponentOutputVectorsV2**), `AddSimilarOutputVectors` (**AddSimilarOutputVectorsV2**), and/or `AddComplexOutputVectors` (**AddComplexOutputVectorsV2**) methods on the Set Object to select all desired output vectors.	
Example:	
None	

Unpublished work. © 2025 Siemens

feDeleteOutputEntryV2

feDeleteOutputEntryV2

(setSET, vectorSET, nodeSET, elemSET)

Description:

This method deletes selected output on individual nodes and/or elements from selected output vectors in selected output sets. This method replaces `feDeleteOutputEntry`. For more information, see **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 setSet`	The ID of the set containing the IDs of the output sets. Alternatively, if you specify a negative value, this is simply the ID of a single output set.
`INT4 vectorSET`	The ID of the set containing the IDs of the output vectors. Alternatively, if you specify a negative value, this is simply the ID of a single output vector.
`INT4 nodeSet`	The ID of the set containing the IDs of the nodes to delete output from. Alternatively, if you specify a negative value, this is simply the ID of a single node to delete output from.
`INT4 elemSET`	The ID of the set containing the IDs of the elements to delete output from. Alternatively, if you specify a negative value, this is simply the ID of a single element to delete output from.

Output:

None

Return Code:

None

Remarks/Usage:

This function offers the same capabilities as the **Delete → Output → Entry** command in the User Interface. It may be helpful to use the `AddComponentOutputVectors` (see **AddComponentOutputVectorsV2**), `AddSimilarOutputVectors` (see **AddSimilarOutputVectorsV2**), and/or `AddComplexOutputVectors` (see **AddComplexOutputVectorsV2**) methods on the Set Object to select all desired output vectors.

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

View, Window and Visibility Methods

feViewVisible (or feWindowVisible)

feViewVisible **(viewID, visible)**	
Description:	
This method makes views visible or removes them from the screen.	
Input:	
`INT4 viewID`	The ID of the view to manipulate.
`BOOL visible`	Set to True to activate a view; False to remove it from the screen.
Output:	
None	
Return Code:	
`FE_FAIL`	The operation failed. Either you asked to make a view visible that was already visible, or to remove a view that was not visible.
Remarks/Usage:	
This is similar to the **Window → New Window** command. `feViewVisible` and `feWindowVisible` perform identical functions.	
Example:	
None	

feViewRedraw (or feWindowRedraw)

Unpublished work. © 2025 Siemens

feViewRedraw	
(viewID)	
Description:	
This method redraws the selected view.	
Input:	
`INT4 viewID`	The ID of the view to redraw. Use `viewID=0` to redraw the active view.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The selected view must be visible. Same as the **Window** → **Redraw** command. `feViewRedraw` and `feWindowRedraw` are identical.	
Example:	
None	

feViewRegenerate (or feWindowRegenerate)

feViewRegenerate	
(viewID)	
Description:	
This method regenerates the selected view.	
Input:	
`INT4 viewID`	The ID of the view to regenerate. Use `viewID=0` to regenerate the active view.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
None	
Remarks/Usage:	
The selected view must be visible. Same as the **Window** → **Regenerate** command. `feViewRegenerate` and `feWindowRegenerate` are identical.	
Example:	
None	

feViewAutoscaleAll

`feViewAutoscaleAll` **`( viewID, redoBox)`**	
Description:	
This method performs an autoscale on the selected view using all of the entities in the model, visible or hidden.	
Input:	
`INT4 viewID`	The ID of the view to Autoscale. Use `viewID=0` to autoscale the active view.
`BOOL redoBox`	When set to True, forces a recalculation of the *box* containing all of the entities in the model.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The selected view does not have to be visible. This command must be used in combination with `feWindowRegenerate` or `feWindowRedraw` in order to see the view update.	
Example:	
None	

Unpublished work. © 2025 Siemens

feViewAutoscaleVisible

feViewAutoscaleVisible

(viewID, redoBox)

Description:

This method performs an autoscale on the selected view while only considering entities currently visible in the selected view.

Input:

`INT4 viewID`	The ID of the view to Autoscale. Use `viewID=0` to autoscale the active view.
`BOOL redoBox`	When set to True, forces a recalculation of the *box* containing all of the visible entities in the selected view.

Output:

None	

Return Code:

None	

Remarks/Usage:

The selected view does not have to be visible. This command must be used in combination with `feWindowRegenerate` or `feWindowRedraw` in order to see the view update.

Example:

None

feViewAllViews

feViewAllViews

(on_off)

Description:

Unpublished work. © 2025 Siemens

This function turns on and off the All Views functionality.	
Input:	
`BOOL on_off`	Set to True to enable All Views; False to only update a single view.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **View → All Views** command.	
Example:	
None	

feViewShow (or feWindowShow)

`feViewShow` **`( entityTYPE, showSET )`**	
Description:	
This function shows a selected group of entities.	
Input:	
`INT4 entityTYPE`	The entity type to show: 3=Point, 4=Curve, 5=Surface, 7=Node, 8=Element, 9=Coordinate System, 10=Material (shows Elements referencing Material), 11=Property (shows Elements referencing Property), 53=Solid/Volume, 58=Connection Region/Region, 71=Connector, 81=Layup (shows Elements referencing Layup), 83=Global Ply (shows Elements referencing Layups which use Global Ply), 95=Areo Panel, 98=Aero Spline, and 99=Aero Control Surface.
`INT4 showSET`	The ID of a selection set that contains the IDs of the entities to show. Alternatively, if you specify a

Unpublished work. © 2025 Siemens

	negative value, this is simply the ID of the single entity to show.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Like the **Window → Show Entities** command. `feViewShow` and `feWindowShow` are identical.	
Example:	
None	

feViewShow2 (or feWindowShow2)

`feViewShow2` **`( entityTYPE, showSET, qAutoscale )`**	
Description:	
This function shows a selected group of entities and has the option to *Autoscale* around the entities being shown.	
Input:	
`INT4 entityTYPE`	The entity type to show: 3=Point, 4=Curve, 5=Surface, 7=Node, 8=Element, 9=Coordinate System, 10=Material (shows Elements referencing Material), 11=Property (shows Elements referencing Property), 53=Solid/Volume, 58=Connection Region/Region, 71=Connector, 81=Layup (shows Elements referencing Layup), 83=Global Ply (shows Elements referencing Layups which use Global Ply), 95=Areo Panel, 98=Aero Spline, and 99=Aero Control Surface.
`INT4 showSET`	The ID of a selection set that contains the IDs of the entities to show. Alternatively, if you specify a negative value, this is simply the ID of the single entity to show.

Unpublished work. © 2025 Siemens

`BOOL qAutoscale`	If True, the view will autoscale around the entities being shown. If False, only the entities will be shown and the scaling of the view will remain the same.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Like the **Window → Show Entities** command. `feViewShow2` and `feWindowShow2` are identical.	
Example:	
None	

feViewTile (or feWindowTile)

`feViewTile`
`( void )`
Description:
This function tiles all of the active views vertically.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
Like the **Window → Tile Vertical** command. `feViewTile` and `feWindowTile` are identical.

Unpublished work. © 2025 Siemens

Example:
None

feWindowTileHorz

`feWindowTileHorz`	
`( void )`	
Description:	
This function tiles all of the active views horizontally.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Like the **Window → Tile Horizontal** command.	
Example:	
None	

feViewCascade (or feWindowCascade)

`feViewCascade`
`( void )`
Description:

Unpublished work. © 2025 Siemens

This function cascades all of the active views.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Like the **Window → Cascade** command. `feViewCascade` and `feWindowCascade` are identical.	
Example:	
None	

feWindowMinimize

`feWindowMinimize` **`( vuID )`**	
Description:	
This function minimizes the window that contains the specified view.	
Input:	
`INT4 vuID`	The ID of the view in the window to be modified. Specify `vuID=0` to modify the active view.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

feWindowMaximize

feWindowMaximize **(vuID)**	
Description:	
This function maximizes the window that contains the specified view.	
Input:	
`INT4 vuID`	The ID of the view in the window to be modified. Specify `vuID=0` to modify the active view.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feWindowRestore

feWindowRestore **(vuID)**
Description:

Unpublished work. © 2025 Siemens

This function restores the window (from either a maximized or minimized state) that contains the specified view.	
Input:	
`INT4 vuID`	The ID of the view in the window to be modified. Specify `vuID=0` to modify the active view.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feWindowSetRect

`feWindowSetRect` **`( vuID, pctLeft, pctTop, pctRight, pctBottom )`**	
Description:	
This function sizes the window using percentages of graphics area from the Left, Top, Right, and/or Bottom of the graphics area.	
Input:	
`INT4 vuID`	The ID of the view in the window to be modified. Specify `vuID=0` to modify the active view.
`REAL8 pctLeft`	Location of view window in percent of graphics area. Numbers must be between 0.0 and 100.0.
`REAL8 pctTop`	
`REAL8 pctRight`	

Unpublished work. © 2025 Siemens

`REAL8 pctBottom`	
Output:	
None	
Return Code:	
`FE_FAIL`	The operation failed. View is invalid or does not exist.
`FE_BAD_DATA`	Values for `pctLeft`, `pctTop`, `pctRight`, or `pctBottom` are out of 0.0 to 100.0 range.
Remarks/Usage:	
None	
Example:	
None	

feWindowSetSize

`feWindowSetSize`
`( vuID, left, top, horz, vert )`
Description:
This function sizes the window by specifying the location of the *top left* corner of the view in number of pixels from the *top left* corner of the graphics area. The overall size of the view is then specified using number of pixels from the *top left* corner in the horizontal and vertical directions.
Input:

`INT4 vuID`	The ID of the view in the window to be modified. Specify `vuID=0` to modify the active view.
`INT4 left`	Number of pixels from left edge of graphics area to left edge of view window.
`INT4 top`	Number of pixels from top edge of graphics area to top edge of view window.
`INT4 horz`	Number of pixels from left of view window to right of view window.

Unpublished work. © 2025 Siemens

`INT4 vert`	Number of pixels from top of view window to bottom of view window.
Output:	
None	
Return Code:	
`FE_FAIL`	The operation failed. View is invalid or does not exist.
Remarks/Usage:	
None	
Example:	
None	

feWindowTitle

`feWindowTitle` **`( vuID, sTitle )`**	
Description:	
This function sets the View Title and the Tab Title at once for an existing view.	
Input:	
`INT4 vuID`	The ID of the view in the window to be modified. Specify `vuID=0` to modify the active view.
`STRING sTitle`	View and Tab Title (maximum 79 characters).
Output:	
None	
Return Code:	
`FE_FAIL`	The operation failed. View is invalid or does not exist.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

feEntitySetVisibility

feEntitySetVisibility **(visType, visID, bIsVisible, bRedraw)**	
Description:	
This function sets entities to be visible or blanked. It is not view dependent but always applies to all views. This call is identical to `feEntityVisiblity`, which should no longer be used.	
Input:	
`INT4 visType`	The type of entity to blank - can be `FVIS_POINT`, `FVIS_CURVE`, `FVIS_SURFACE`, `FVIS_SOLID`, `FVIS_CSYS`, `FVIS_CONNECTION_REGION`, `FVIS_CONNECTION`, `FVIS_PROP`, `FVIS_MATL`, `FVIS_ELEM`, `FVIS_ELEM_TYPE`, `FVIS_ELEM_SHAPE`, `FVIS_FREEBODY`, `FVIS_AERO_PANE`, `FVIS_AERO_SPLINE`, `FVIS_AERO_SURFACE`.
`INT4 visID`	The ID of a set which contains the IDs/types/shapes of the entity to show or blank. For the element `visTypes` the set must contain element types or shapes, not element IDs. Alternatively, specify a negative value to select a single ID/type/shape.
`BOOL bIsVisible`	Set to True to make entity visible, False to hide it.
`BOOL bRedraw`	If True, the model is redrawn after the visibility change is made.
Output:	
None	
Return Code:	
`FE_INVALID`	Invalid `visType`.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

The visibility of any points, curves, or surfaces associated with a solid will have their visibility controlled by the visibility setting of the solid.
Example:
None

feEntitySetVisibility2

feEntitySetVisibility2 **(visType, nCount, visEnt, entVisible, bRedraw)**	
Description:	
This function sets entities to be visible or blanked using arrays. It is not view dependent but always applies to all views.	
Input:	
`INT4 visType`	The type of entity to blank - can be `FVIS_POINT`, `FVIS_CURVE`, `FVIS_SURFACE`, `FVIS_SOLID`, `FVIS_CSYS`, `FVIS_CONNECTION_REGION`, `FVIS_CONNECTION`, `FVIS_PROP`, `FVIS_MATL`, `FVIS_ELEM`, `FVIS_ELEM_TYPE`, `FVIS_ELEM_SHAPE`, `FVIS_FREEBODY`, `FVIS_AERO_PANE`, `FVIS_AERO_SPLINE`, `FVIS_AERO_SURFACE`.
`INT4 nCount`	Number of different visibility items to set using `visEnt` and `entVisible` arrays.
`INT4 visEnt[0..nCount-1]`	Array containing different types of entities of the specified `visType`. For instance, when `visType` is set to `FVIS_ELEM_SHAPE`, the `visEnt` can be filled with different element topologies, while when it is set to `FVIS_PROP`, it would contain different property types.
`BOOL entVisible[0..nCount-1]`	Array of visibility flags corresponding to each type specified in the `visEnt` array. Set to True to make an entity visible, False to hide.
`BOOL bRedraw`	If True, the model is redrawn after the visibility change is made.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_INVALID`	Invalid `visType`.
Remarks/Usage:	
The visibility of any points, curves, or surfaces associated with a solid will have their visibility controlled by the visibility setting of the solid.	
Example:	
None	

feEntityGetVisibility

`feEntityGetVisibility` **`( visType, visID, bIsVisible )`**	
Description:	
This function populates a set object with the IDs of entities which are visible or blanked. It is not view dependent but always applies to all views.	
Input:	
`INT4 visType`	The type of entity - can be `FVIS_POINT`, `FVIS_CURVE`, `FVIS_SURFACE`, `FVIS_SOLID`, `FVIS_CSYS`, `FVIS_CONNECTION_REGION`, `FVIS_CONNECTION`, `FVIS_PROP`, `FVIS_MATL`, `FVIS_ELEM`, `FVIS_ELEM_TYPE`, `FVIS_ELEM_SHAPE`, `FVIS_FREEBODY`, `FVIS_AERO_PANE`, `FVIS_AERO_SPLINE`, `FVIS_AERO_SURFACE`.
`INT4 visSetID`	The ID of set object to populate with visible or blanked entities.
`BOOL bIsVisible`	Set to True to get visible entities, False to get hidden entities.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

FE_INVALID	Invalid visType.

Remarks/Usage:

When using FVIS_ELEM_TYPE, the set object will be filled with the IDs of the Element Types which are currently visible or blanked (1-42). For instance, if only linear plate elements are blanked, and bIsVisible is set to False, then only the Type ID for linear plate elements (17) would be in the set object. If bIsVisible is set to True, then all IDs from 1-42 except 17 would be in the set object.

When using FVIS_ELEM_SHAPE, the set object will be filled with the IDs of the Element Shapes which are visible or blanked (1-20, 17 is a *Contact Element*, which is no longer used, but may appear in the list).

See **Element Object Properties** for more information on the element properties for Type (type) and Shape (topology).

Example:

> **Note**
>
> For FVIS_ELEM_SHAPE the FTO_LINE2 topology needs special handling. The Femap Set object cannot iterate and return the FTO_LINE2 topology since it is defined as zero. In this case you will need to check for existence of the zero item directly with the IsAdded(0) method.

```
Sub Main

   Dim App As femap.model

   Set App = feFemap()


   Dim fs As femap.Set

   Set fs = App.feSet


   App.feEntityGetVisibility( FVIS_ELEM_SHAPE, fs.ID, True )


   fs.Next()

   While fs.Next() <> FE_FAIL

        App.feAppMessage( FCM_NORMAL, "Element Topology" +
Str( fs.CurrentID ) + " is visible " )

   Wend
```


Unpublished work. © 2025 Siemens

```
   If fs.IsAdded( FTO_LINE2 ) Then

        App.feAppMessage( FCM_NORMAL, "So is Element Topology" +
Str( FTO_LINE2 ) )

   End If


End Sub
```

feAppUpdateModelBox

feAppUpdateModelBox

(bForceUpdate)

Description:	
This function recomputes the size of the box around the model so scaling for graphics are correct. It is not normally necessary to call this function.	
Input:	
`BOOL bForceUpdate`	If True, the box is always recomputed. If False, only if necessary.
Output:	
None	
Return Code:	
`FE_FAIL`	No model is active.
Remarks/Usage:	
None	
Example:	
None	

feViewShowMultiple

Unpublished work. © 2025 Siemens

feViewShowMultiple

(nSets, piTypes, piSetIDs, qAutoscale)

Description:

This function shows multiple sets of entities.

Input:

`INT4 nSets`	Number of different sets of entities to show.
`INT4 piTypes [0..nSets-1]`	Array containing the entity type of each set to show. Allowable entity types are: 3=Point, 4=Curve, 5=Surface, 7=Node, 8=Element, 9=Coordinate System, 10=Material (shows Elements referencing Material), 11=Property (shows Elements referencing Property), 53=Solid/Volume, 58=Connection Region/Region, 71=Connector, 81=Layup (shows Elements referencing Layup), 83=Global Ply (shows Elements referencing Layups which use Global Ply), 95=Areo Panel, 98=Aero Spline, and 99=Aero Control Surface.
`INT4 piSetIDs [0..nSets-1]`	Array of Set Object IDs which contain the entity IDs of the type specified in the corresponding index in `piTypes`.
`BOOL qAutoscale`	If True, the view will autoscale around the entities being shown. If False, only the entities will be shown and the scaling of the view will remain the same.

Output:

None	

Return Code:

None	

Remarks/Usage:

The sets can be of different types of entities. If they are of the same type, duplicates are not removed.

Example:

None

Unpublished work. © 2025 Siemens

Group Methods

feGroupEvaluate

`feGroupEvaluate`
 	
`( grSet, forceEval )`	

Description:	
This method evaluates all of the selection rules, clipping and other operations defined in one or more groups.	
Input:	
`INT4 grSET`	The ID of a set that contains a list of Group IDs that you want to Evaluate. Alternatively, if you specify a negative value, this is simply the ID of the single group to evaluate.
`BOOLEAN forceEval`	If True, the group will always be evaluated. If False, the group will only be evaluated if it is marked as needing evaluation.
Output:	
None	
Return Code:	
`FE_FAIL`	One or more of the groups you chose was unable to be evaluated.
Remarks/Usage:	
This method accomplishes the same tasks as the **Group** → **Operations** → **Evaluate** command.	
Example:	
None	

feGroupBoolean

Unpublished work. © 2025 Siemens

feGroupBoolean

(operation, grID1, grID2, grCreate, grTitle)

Description:

This method is obsolete - use `feGroupBoolean2()` instead.

This method takes one or two existing groups and creates a new group based on a specified operation.

Input:

`INT4 operation`	The operation to be done: 0=And, 1=Or, 2=Exclusive Or, 3=Invert.
`INT4 grID1`	The first group to use in the operation.
`INT4 grID2`	The second group to use in the operation. (This should be 0 for the Invert operation, because only the first group is used.)
`INT4 grCreate`	The ID of the group to create.
`STRING grTitle`	The title of the group that is created.

Output:

None	

Return Code:

`FE_INVALID`	The ID of the group that you specified to create is invalid. Either it is not a valid ID, or that group already exists.
`FE_FAIL`	Unable to manipulate the groups that you specified. Check to see if they exist.

Remarks/Usage:

This method accomplishes the same tasks as the **Group → Operations → And**, **Group → Operations → Or**, **Group → Operations → Exclusive Or**, and **Group → Operation → Not** commands.

And means an entity must be in both groups to be in the new group, **Or** means all the entities in either group, and **Exclusive Or** means all the entities in one, but not both of the initial groups. Invert simply means all the entities not in the first group.

Example:

None

Unpublished work. © 2025 Siemens

feGroupBoolean2

feGroupBoolean2	
(GroupsToProcessSetID, GroupToSubtractFromID, GroupOp, GroupToCreateID, GroupTitle)	
Description:	
This method takes a set of Groups and creates a new group based on a specified operation.	
Input:	
`INT4 GroupsToProcessSetID`	The ID of a Set that contains the IDs of the Groups to Process.
`INT4 GroupToSubtractFromID`	The ID of a Group to Subtract the other Groups From. This value is ignored, but must be specified, unless `GroupOp=1`.
`INT4 GroupOp`	The Operation to perform 0=Add, 1=Subtract, 2=InAll, 3=OnlyInOne, 4=NotInAny, 5=NotInAll
`INT4 GroupToCreateID`	The ID of the group to create. If this is specified as 0, or the ID of an existing group, a new group will be created with an arbitrary ID.
`STRING GroupTitle`	The title of the group that is created. If you specify this title a blank a default title will be created.
Output:	
None	
Return Code:	
`FE_INVALID`	Invalid group to subtract from.
`FE_FAIL`	Unable to manipulate the groups that you specified. Check to see if they exist.
Remarks/Usage:	
This method accomplishes the same tasks as the **Group → Operations → Booleans** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGroupCombine

feGroupCombine **(groupSET, groupID)**	
Description:	
This method takes one or more existing groups and combines them to create a new group.	
Input:	
`INT4 groupSET`	The ID of a Set Object that contains the IDs of the groups to combine.
Output:	
`INT4 groupID`	The ID of the group that is created.
Return Code:	
`FE_INVALID`	Unable to combine groups.
Remarks/Usage:	
None	
Example:	
None	

feGroupGenerate

feGroupGenerate **(elemSET, planar, brkPropMatl, brkLayer, brkColor, brkAngle, addLayers)**
Description:
This method automatically makes one or more groups by subdividing the elements you select based on one or more criteria.
Input:

Unpublished work. © 2025 Siemens

`INT4 elemSET`	The ID of the set of elements to be used in defining the groups.
`BOOL planar`	If true, only planar elements will be used. If False, only solid elements will be used.
`INT4 brkPropMatl`	Set to 0 to ignore property or material discontinuities. Set to 1 to start a new group when property IDs change. Set to 2 to start a new group when material IDs change.
`INT4 brkLayer`	Set to nonzero to start a new group when layers change.
`INT4 brkColor`	Set to nonzero to start a new group when element colors change.
`REAL8 brkAngle`	Set to the maximum angle between planar elements for them to still be in the same group. Ignored for solid elements. Specified in degrees.
`BOOL addLayers`	Set to True if you not only want to create groups, but move the elements in each group to a separate layer.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group** → **Operations** → **Generate** command for Femap versions prior to version 11.1. For full access to all the options now available, use `feGroupGenerate2`.	
Example:	
None	

feGroupGenerate2

Unpublished work. © 2025 Siemens

feGroupGenerate2

```
( elemSET, planar, brkPropMatl, brkLayer, brkColor, brkAngle,
brkMatchNormal,
brkNonManifold, addLayers, nExpandGroup)
```

Description:

This method automatically makes one or more groups by subdividing the elements you select based on one or more criteria.

Input:

`INT4 elemSET`	The ID of the set of elements to be used in defining the groups.
`BOOL planar`	If true, only planar elements will be used. If False, only solid elements will be used.
`INT4 brkPropMatl`	Set to 0 to ignore property or material discontinuities. Set to 1 to start a new group when property IDs change. Set to 2 to start a new group when material IDs change.
`INT4 brkLayer`	Set to nonzero to start a new group when layers change.
`INT4 brkColor`	Set to nonzero to start a new group when element colors change.
`REAL8 brkAngle`	Set to the maximum angle between planar elements for them to still be in the same group. Ignored for solid elements. Specified in degrees. Setting this value to 0.0 turns off all geometry breaks unless `brkMatchNormal` is set to True.
`BOOL brkMatchNormal`	If True, the normal direction of planar elements will be considered when determining how many groups to make. If False, the normal direction of planar elements will not be considered.
`BOOL brkNonManifold`	If True, any non-manifold edges found between planar elements will be used to create additional groups and is independent of the `brkAngle` value. If False, non-manifold edges between planar elements will not be considered.
`BOOL addLayers`	Set to True if you not only want to create groups, but move the elements in each group to a separate layer.

Unpublished work. © 2025 Siemens

`INT4 nExpandGroup`	Specifies the **Group Expansion** option. 0=Elements Only, 1=Elements and Nodes, 2=All (Elements, Nodes, Materials, Properties, Loads on Nodes, Loads on Elements, Constraints referencing Nodes, and Constraint Equations referencing Nodes).
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group** → **Operations** → **Generate** command in Femap 11.1 and above.	
Example:	
None	

feGroupGenSolid

`feGroupGenSolid`	
`( solidSET )`	
Description:	
This method automatically makes one or more groups by subdividing geometry based on the solids that it defines.	
Input:	
`INT4 solidSET`	The ID of the set of solids to be used in defining the groups.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group** → **Operations** → **Generate Solid** command. A group is created for each solid that contains the solid, its surfaces, curves and points.	

Unpublished work. © 2025 Siemens

Example:
None

feGroupGenProp

feGroupGenProp	
(propSET)	
Description:	
This method automatically makes one or more groups by subdividing a mesh based on property IDs.	
Input:	
`INT4 propSET`	The ID of the set of properties to be used in defining the groups.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group** → **Operations** → **Generate Property** command. A group of elements is created that use each selected property.	
Example:	
None	

feGroupGenMatl

feGroupGenMatl
(matlSET)
Description:

Unpublished work. © 2025 Siemens

This method automatically makes one or more groups by subdividing a mesh based on material IDs.	
Input:	
`INT4 matlSET`	The ID of the set of materials to be used in defining the groups.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group** → **Operations** → **Generate Material** command. A group of elements is created that use each selected material.	
Example:	
None	

feGroupGenElemType

`feGroupGenElemType` **`( elemSET )`**	
Description:	
This method automatically makes one or more groups by subdividing a mesh based on element type.	
Input:	
`INT4 elemSET`	The ID of the set of elements to be used in defining the groups.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Same as the **Group → Operations → Generate Elem Type** command. A group is created for each element type used by the selected elements.
Example:
None

feGroupGenElemShape

`feGroupGenElemShape` **`( elemSET )`**	
Description:	
This method automatically makes one or more groups by subdividing a mesh based on element shape.	
Input:	
`INT4 elemSET`	The ID of the set of elements to be used in defining the groups.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group → Operations → Generate Elem Shape** command. A group is created for each element shape used by the selected elements.	
Example:	
None	

feGroupPeel

Unpublished work. © 2025 Siemens

feGroupPeel	
(elemSET, peelLayers, keepPeel, keepCore)	
Description:	
This method automatically makes one or more groups by *peeling* layers of solid elements from the outside of a mesh.	
Input:	
`INT4 elemSET`	The ID of the set of elements to be used in defining the groups.
`INT4 peelLayers`	The number of layers of elements to remove from the mesh.
`BOOL keepPeel`	If True, each layer that is removed will be placed into a group.
`BOOL keepCore`	If True, the remaining elements, after the number of layers have been removed, will be placed into a group.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Same as the **Group** → **Operations** → **Peel** command.	
Example:	
None	

feGroupMoveToLayer

feGroupMoveToLayer
(nGroupID, nLayerID)
Description:

Unpublished work. © 2025 Siemens

Assigns all entities in the selected group to the specified layer.	
Input:	
`INT4 nGroupID`	The ID of the Group to use to place entities onto a layer.
`INT4 nLayerID`	The ID of the Layer to assign entities to from the specified group.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feGroupsContaining

`feGroupsContaining` **`( enDataType, nEntityID, GroupSetID )`**
Description:
This method does not mimic a command in the **Group** menu, but instead populates an existing set object with the IDs of every group in which the specified entities (or single entity) currently reside.
Input:
`INT4 enDataType` — The type of entity to be checked for group inclusion. For entity information, see **Entity Types**.
`INT4 nEntityID` — The ID of a set that contains a list of entity IDs to determine in which groups the entities reside. Alternatively, if you specify a negative value, this is simply the ID of the single entity to determine in which groups that entity resides.
Output:

Unpublished work. © 2025 Siemens

`INT4 nGroupSetID`	The ID of set to be populated with group IDs. The set will always be cleared and then repopulated.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feGroupGenVisible

`feGroupGenVisible`
`( nGroupID, bIncludePropAndMatl, title )`
Description:
This method automatically makes a group containing only the entities currently visible in the active view.
Input:

`INT4 nGroupID`	ID of group to create with visible entities. Setting this value to -1 will use the next available group ID, while setting this value to 0 will display the dialog boxes associated with the **Group** → **Operations** → **Generate Visible** command.
`BOOL bIncludePropAndMatl`	When True, Properties and Materials associated with any visible elements will be added to the new group. When False, the Properties and Materials associated to visible elements will not be placed into the new group.
`STRING title`	Title of created group

Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	Selected group ID already exists.
`FE_INVALID`	Invalid group ID specified.
Remarks/Usage:	
Same as the **Group → Operations → Generate Visible** command.	
Example:	
None	

feGroupFromCriteria

`feGroupFromCriteria` **`( vuID, bVisibleOnly, nGroupID )`**	
Description:	
This method automatically makes a group containing only the elements which pass the current criteria when displaying a Criteria plot.	
Input:	
`INT4 vuID`	ID of view, which must be currently displaying a Criteria plot, to use to automatically create group of elements which pass the criteria specified for that view. Enter a value of 0 to select the active view.
`BOOL bVisibleOnly`	When True, only elements which are visible in the view specified for `vuID` and pass the criteria specified for the view will be included in the automatically created group. When False, all elements in the model which pass the criteria specified for the view specified for vuID will be included in the automatically created group.
Output:	
`INT4 nGroupID`	ID of the automatically created group.
Return Code:	
`FE_INVALID`	Can not access the specified View.

Unpublished work. © 2025 Siemens

`FE_BAD_TYPE`	Selected View does not have a Criteria plot displayed.
`FE_NOT_AVAILABLE`	Criteria results are not available for the selected view. This will occur if you select a view that is not currently on-screen.
`FE_NOT_EXIST`	No elements were found passing the specified criteria.
Remarks/Usage:	
Same as the **Group → Operations → Generate From Criteria** command.	
Example:	
None	

feGroupCondense

`feGroupCondense`	
`( groupSET )`	
Description:	
This function takes a group and condenses all clipping and related rules into ID rules in a new group.	
Input:	
`INT4 groupSET`	ID of set of groups to condense.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feGroupReduceToExisting

feGroupReduceToExisting **(groupSET)**	
Description:	
This function takes a group and updates each rule type to contain its existing entities.	
Input:	
`INT4 groupSET`	ID of set of groups to reduce to existing.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Does not condense the rules down to ID rules so you retain the same functionality of the Group.	
Example:	
None	

Standard Dialog Boxes

Coordinate Methods

feCoordPick

Unpublished work. © 2025 Siemens

feCoordPick **(dlgTITLE, xyz)**	
Description:	
This function displays a dialog box to allow a user to pick or enter coordinates.	
Input:	
`STRING dlgTITLE`	A string that specifies a title to be added to the dialog box title.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Coordinates are not valid.
Remarks/Usage:	
The default coordinate values that are displayed when the dialog is displayed can be set using the global parameters `Info_DefaultX`, `Info_DefaultY`, and `Info_DefaultZ` before calling this function.	
Example:	
None	

feCoordPickByMethod

feCoordPickByMethod **(method, dlgTITLE, xyz)**	
Description:	
This function displays a dialog box to allow a user to pick or enter coordinates. Same as `feCoordPick`, but lets you specify the initial dialog box method.	
Input:	
`INT4 method`	The initial dialog box method. Refer to *Coordinate Dialog Methods* for possible values.

Unpublished work. © 2025 Siemens

`STRING dlgTITLE`	A string that specifies a title to be added to the dialog box title.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Coordinates are not valid.
Remarks/Usage:	
The default coordinate values that are displayed when the dialog is displayed can be set using the global parameters `Info_DefaultX`, `Info_DefaultY`, and `Info_DefaultZ` before calling this function.	
Example:	
None	

feCoordFromWorkplane

`feCoordFromWorkplane` **`( wpXYZ, xyz )`**	
Description:	
This function converts coordinates from workplane coordinates to global rectangular coordinates.	
Input:	
`REAL8 wpXYZ[0..2]`	The original workplane coordinates. Although three coordinates can be specified, normally `wpXYZ[2]` is 0.0 if the location is actually on the workplane.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

feCoordToWorkplane

`feCoordToWorkplane`
 	
`( xyz, wpXYZ )`	

Description:	
This function converts coordinates from Global Rectangular coordinates to Workplane Coordinates.	
Input:	
`REAL8 xyz[0..2]`	The original coordinates. Must be specified in global rectangular coordinates.
Output:	
`REAL8 wpXYZ[0..2]`	The workplane coordinates. Three coordinates are returned. `wpXYZ[2]` is the elevation off of the workplane. Ignore this value to get the coordinates in the workplane.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feCoordIntersectCurves

Unpublished work. © 2025 Siemens

feCoordIntersectCurves	
(cuID1, cuID2, nearXYZ, xyz)	
Description:	
This function finds the coordinates at the intersection of two curves.	
Input:	
`INT4 cuID1`	The ID of the first curve to intersect.
`INT4 cuID2`	The ID of the second curve to intersect.
`REAL8 nearXYZ[0..2]`	A location near the desired intersection. This is not used unless there are multiple intersections detected.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_INVALID`	The curve IDs may not specify the same curve.
Remarks/Usage:	
None	
Example:	
None	

feCoordIntersectLines

feCoordIntersectLines
(infLine, pA1, pA2, pB1, pB2, pIntersect)
Description:
This function finds the coordinates at the intersection of two lines which are specified by their endpoint coordinates.

Unpublished work. © 2025 Siemens

Input:	
`BOOL infLine`	Set to True if you want the intersection calculation to consider the lines as infinite. Set to False if you want to only consider the line segments between the endpoints that you provided.
`REAL8 pA1[0..2]`	The coordinates of the first point on the first line.
`REAL8 pA2[0..2]`	The coordinates of the second point on the first line.
`REAL8 pA1[0..2]`	The coordinates of the first point on the second line.
`REAL8 pA1[0..2]`	The coordinates of the second point on the second line.
Output:	
`REAL8 pIntersect[0..2]`	The coordinates of the intersection point.
Return Code:	
`FE_FAIL`	No intersection was found. This could be because the lines were parallel, or you were working with line segments and the intersection was outside of the segments you specified.
Remarks/Usage:	
None	
Example:	
None	

feCoordOntoCurve

`feCoordOntoCurve`
`( cuID, fromXYZ, xyz )`
Description:
This function projects coordinates onto a curve.

Unpublished work. © 2025 Siemens

Input:	
`INT4 cuID`	The ID of the curve to project onto.
`REAL8 fromXYZ[0..2]`	The original location that is to be projected onto the curve.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected curve does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordAlongCurve

`feCoordAlongCurve`
`( cuID, fromSTART, cuLENGTH, xyz )`
Description:
This function finds the coordinates that are located at specified distance along a curve.
Input:

`INT4 cuID`	The ID of the curve to measure along.
`BOOL fromSTART`	If True, then the length is measured from the start of the curve; otherwise, it is measured from the end of the curve.
`REAL8 cuLENGTH`	The length, along the curve, from the selected endpoint to the coordinates.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected curve does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordCurveMidpoint

`feCoordCurveMidpoint`	
`( cuID, xyz )`	
Description:	
This function finds the coordinates that are located at the midpoint of a curve.	
Input:	
`INT4 cuID`	The ID of the curve to measure along. The midpoint is halfway (along the curve) between the start and end of the curve.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected curve does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCoordCurveCenter

feCoordCurveCenter

(cuID, xyz)

Description:

This function finds the coordinates that are located at the center of an arc, circle, or solid curve that forms an arc or circle.

Input:

`INT4 cuID`	The ID of the curve to find the center.

Output:

`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.

Return Code:

`FE_NOT_EXIST`	The selected curve does not exist.

Remarks/Usage:

This function will return `FE_FAIL` if you give it any other curve type except an arc, circle or solid. If you choose a solid curve, this function assumes it is an arc/circle and returns the appropriate radius. No checking is done to ensure that it is an arc or circle.

Example:

None

feCoordOntoSurface

feCoordOntoSurface

(suID, fromXYZ, xyz)

Description:

This function projects coordinates onto a selected surface.

Unpublished work. © 2025 Siemens

Input:	
`INT4 suID`	The ID of the surface to project onto.
`REAL8 fromXYZ[0..2]`	The original coordinate which will be projected.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected surface does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordInSurface

`feCoordInSurface` **`( suID, suXY, xyz )`**	
Description:	
This function returns coordinates located at the parametric coordinates of a surface.	
Input:	
`INT4 suID`	The ID of the surface to use.
`REAL8 suXY[0..1]`	The parametric coordinates on the surface.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	The selected surface does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordSurfaceIntersect

`feCoordSurfaceIntersect` **`( cuID, suID, nearXYZ, xyz )`**	
Description:	
This function returns coordinates located at the intersection of a curve and surface.	
Input:	
`INT4 cuID`	The ID of the curve to use.
`INT4 suID`	The ID of the surface to use.
`REAL8 nearXY[0..2]`	Coordinates near the intersection location. This is only used if multiple intersections are found. In this case, the closest one to this location is returned. Specify in global rectangular coordinates.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected curve or surface does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCoordOnPoint

feCoordOnPoint	
(ptID, xyz)	
Description:	
This function returns the coordinates of a point.	
Input:	
`INT4 ptID`	The ID of the point to locate.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected point does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordOnNode

feCoordOnNode
(ndID, xyz)
Description:
This function returns the coordinates of a node.
Input:

Unpublished work. © 2025 Siemens

`INT4 ndID`	The ID of the node to locate.
Output:	
`REAL8 xyz[0..2]`	The coordinates selected. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_NOT_EXIST`	The selected node does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordTransform

`feCoordTransform`	
`( fromCSYS, fromXYZ, toCSYS, xyz )`	
Description:	
This function transforms coordinates from one coordinate system to another.	
Input:	
`INT4 fromCSYS`	The ID of the coordinate system in which the fromXYZ coordinates are currently specified. Specify 0 for global rectangular.
`REAL8 fromXYZ[0..2]`	The coordinates to transform, specified in `fromCSYS`.
`INT4 toCSYS`	The ID of the coordinate system to transform into.
Output:	
`REAL8 xyz[0..2]`	The transformed coordinates, returned in the `toCSYS` coordinate system. The two sets of coordinates MAY NOT be specified as the same variable.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_EXIST`	One of the coordinate systems does not exist.
Remarks/Usage:	
None	
Example:	
None	

feCoordArrayTransform

`feCoordArrayTransform` **`( count, fromCSYS, toCSYS, xyz )`**	
Description:	
This function transforms an array of coordinates from one coordinate system to another.	
Input:	
`INT4 count`	The number of sets of coordinates in xyz. There must be 3*count entries in xyz.
`INT4 fromCSYS`	The ID of the coordinate system in which the coordinates are currently specified. Specify 0 for global rectangular.
`INT4 toCSYS`	The ID of the coordinate system to transform into.
Output:	
`REAL8 xyz[0..3*count-1]`	On input, this array contains the original coordinates. On output, it contains the transformed coordinates, returned in the `toCSYS` coordinate system. Coordinates in this array must be in the order X0,Y0,Z0,X1,Y1,Z1,...,Xcount,Ycount,Zcount.
Return Code:	
`FE_INVALID`	The count must be a positive number.
`FE_NOT_EXIST`	One of the coordinate systems does not exist.

Unpublished work. © 2025 Siemens

Remarks/Usage:
Note This method will not work with .NET because the *xyz* argument can not be used for both input and output. Use the `feCoordArrayTransform2( )` method instead.
Example:
None

feCoordArrayTransform2

feCoordArrayTransform2 **(count, fromCSYS, toCSYS, inXYZ, xyz)**	
Description:	
This function transforms an array of coordinates from one coordinate system to another.	
Input:	
`INT4 count`	The number of sets of coordinates in xyz. There must be 3*count entries in xyz.
`INT4 fromCSYS`	The ID of the coordinate system in which the coordinates are currently specified. Specify 0 for global rectangular.
`INT4 toCSYS`	The ID of the coordinate system to transform into.
`REAL8 inXYZ[0..3*count-1]`	This array contains the original coordinates that will be transformed. Coordinates in this array must be in the order X0,Y0,Z0,X1,Y1,Z1,...,Xcount,Ycount,Zcount.
Output:	
`REAL8 xyz[0..3*count-1]`	Contains the transformed coordinates, returned in the `toCSYS` coordinate system. Coordinates in this array are in the same order as in `inXYZ`.
Return Code:	
`FE_INVALID`	The count must be a positive number.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	One of the coordinate systems does not exist.
Remarks/Usage:	
This method replaces the `feCoordArrayTransform( )` method which will not work with .NET	
Example:	
None	

feCoordColinear

`feCoordColinear` **`( xyz1, xyz2, xyz3, isColinear )`**	
Description:	
This function checks three sets of coordinates and determines if they are colinear.	
Input:	
`REAL8 xyz1[0..2]`	The coordinates of the first point. This, and the other coordinates, must be stored in rectangular coordinates.
`REAL8 xyz2[0..2]`	The coordinates of the second point.
`REAL8 xyz3[0..2]`	The coordinates of the third point.
Output:	
`BOOL isColinear`	True if the points are colinear (within the tolerance specified internally); False otherwise.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feCoordToDirCos

feCoordToDirCos **(origin, xAxis, xyPlane, dircos)**	
Description:	
This function converts three coordinates into a 3x3 direction cosine matrix.	
Input:	
`REAL8 origin[0..2]`	The location at the origin.
`REAL8 xAxis[0..2]`	A location along the positive X axis.
`REAL8 xyPlane[0..2]`	A location in the positive quadrant of the XY plane.
Output:	
`REAL8 dircos[0..8]`	The 3x3 direction cosine matrix. Data is stored in row-order. [0,0]=[0], [0,1]=[1], [0,2]=[2], [1,0]=3 ... [2,2]=[8]
Return Code:	
None	
Remarks/Usage:	
All coordinates must be specified in rectangular coordinates. The direction cosine matrix is relative to whatever coordinate system the original points were specified in.	
Example:	
None	

feCoordRotate

Unpublished work. © 2025 Siemens

feCoordRotate	
(xyz, angle, origin, axis, xyzROT)	
Description:	
This function takes a coordinate location and rotates it through a prescribed angle about a vector.	
Input:	
`REAL8 xyz[0..2]`	The original unrotated coordinates - must be specified in rectangular coordinates.
`REAL8 angle`	The angle of rotation.
`REAL8 origin[0..2]`	The coordinates at the origin/base of the rotation axis vector.
`REAL8 axis[0..2]`	The components of the rotation axis vector.
Output:	
`REAL8 xyzROT[0..2]`	The rotated coordinates.
Return Code:	
None	
Remarks/Usage:	
All coordinates and vector components must be specified in the same rectangular coordinate system - normally global rectangular.	
Example:	
None	

feCoordOntoPlane

feCoordOntoPlane
(xyz, projDir, planePt, planeNorm, xyzPlane)
Description:

Unpublished work. © 2025 Siemens

This function takes a coordinate location and projects it along a vector onto a plane.	
Input:	
`REAL8 xyz[0..2]`	The original unprojected coordinates - must be specified in rectangular coordinates.
`REAL8 projDir[0..2]`	The components of the vector to project along.
`REAL8 planePt[0..2]`	The coordinates of any point on the plane.
`REAL8 planeNorm[0..2]`	The components of the normal vector of the plane.
Output:	
`REAL8 xyzPlane[0..2]`	The projected coordinates.
Return Code:	
`FE_FAIL`	The projection failed. Probably the `projDir` vector is parallel to the plane (perpendicular to `planeNorm`).
Remarks/Usage:	
All coordinates and vector components must be specified in the same rectangular coordinate system - normally global rectangular.	
Example:	
None	

feCoordOntoLine

`feCoordOntoLine` **`( xyz, p1, p2, xyzLine )`**	
Description:	
This function takes a coordinate location and projects it onto a line specified by two endpoint locations.	
Input:	
`REAL8 xyz[0..2]`	The original unprojected coordinates - must be specified in rectangular coordinates.

Unpublished work. © 2025 Siemens

`REAL8 p1[0..2]`	The first set of coordinates along the line.
`REAL8 p2[0..2]`	The second set of coordinates along the line.
Output:	
`REAL8 xyzLine[0..2]`	The projected coordinates.
Return Code:	
None	
Remarks/Usage:	
All coordinates and vector components must be specified in the same rectangular coordinate system - normally global rectangular. The line which the coordinates are projected onto is considered to be infinite.	
Example:	
None	

feCoordPlaneIntersect

`feCoordPlaneIntersect` **`( baseA, normA, baseB, normB, baseC, normC, ptV )`**	
Description:	
This function finds the coordinate location at the intersection of three planes.	
Input:	
`REAL8 baseA[0..2]`	The coordinates of a point on the first plane.
`REAL8 normA[0..2]`	The components of the normal of the first plane.
`REAL8 baseB[0..2]`	The coordinates of a point on the second plane.
`REAL8 normB[0..2]`	The components of the normal of the second plane.
`REAL8 baseC[0..2]`	The coordinates of a point on the third plane.

Unpublished work. © 2025 Siemens

`REAL8 normC[0..2]`	The components of the normal of the third plane.
Output:	
`REAL8 ptV[0..2]`	The coordinates at the intersection location.
Return Code:	
`FE_FAIL`	No intersection could be found.
Remarks/Usage:	
All coordinates and vector components must be specified in the same rectangular coordinate system - normally global rectangular.	
Example:	
None	

feCoordVectorPlaneIntersect

`feCoordVectorPlaneIntersect`	
`( ptVEC, normVEC, ptPLANE, normPLANE, ptV )`	
Description:	
This function finds the coordinate location at the intersection of a vector and a plane.	
Input:	
`REAL8 ptVEC[0..2]`	The coordinates of a point on the vector
`REAL8 normVEC[0..2]`	The components of direction along the vector
`REAL8 ptPLANE[0..2]`	The coordinates of a point on the plane.
`REAL8 normPLANE[0..2]`	The components of the normal of the plane.
Output:	
`REAL8 ptV[0..2]`	The coordinates at the intersection location.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	No intersection could be found.
Remarks/Usage:	
All coordinates and vector components must be specified in the same rectangular coordinate system - normally global rectangular.	
Example:	
None	

feRotationTransform

`feRotationTransform` **`( fromCSYS, fromROT, toCSYS, rot )`**	
Description:	
This function transforms a triple rotation from one coordinate system to another.	
Input:	
`INT4 fromCSYS`	The ID of the coordinate system in which the `fromROT` rotations are currently specified. Specify 0 for global rectangular.
`REAL8 fromROT[0..2]`	The rotations to transform, specified in `fromCSYS`. The rotations are triple rotations, about X, the rotated Y and the doubly-rotated Z.
`INT4 toCSYS`	The ID of the coordinate system to transform into.
Output:	
`REAL8 rot[0..2]`	The transformed rotations, returned in the `toCSYS` coordinate system. The two sets of rotations MAY NOT be specified as the same variable.
Return Code:	
`FE_NOT_EXIST`	One of the coordinate systems does not exist.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

feRotationToDirCos

feRotationToDirCos	
(rot, dircos)	
Description:	
This function converts a triple-rotation, about X, then Y, then Z, into a 3x3 direction cosine matrix.	
Input:	
`REAL8 rot[0..2]`	The rotations, first about X, then the rotated Y, then the doubly-rotated Z.
Output:	
`REAL8 dircos[0..8]`	The 3x3 direction cosine matrix. Data is stored in row-order. [0,0]=[0], [0,1]=[1], [0,2]=[2], [1,0]=3 ... [2,2]=[8]
Return Code:	
None	
Remarks/Usage:	
Throughout this description, M is the dircos matrix. The direction cosine matrix is of the following form: `X' = { M } X` The rows of the matrix represent a vector in the primed direction relative to the unprimed coordinates. For example, `{M[1][0],M[1][1],M[1][2]}` is a vector in the Y' direction relative to X,Y,Z. Similarly, since this is a direction cosine matrix the columns of the matrix represent vectors in the unprimed direction relative to primed coordinates. For example: `{M[0][1],M[1][1],M[2][1]}` is a vector in the Y direction relative to X',Y',Z'.	
Example:	
None	

Unpublished work. © 2025 Siemens

feRotationFromDirCos

feRotationFromDirCos	
(dircos, rot)	
Description:	
This function converts a 3x3 direction cosine matrix into an equivalent triple-rotation, about X, then Y, then Z.	
Input:	
`REAL8 dircos[0..8]`	The 3x3 direction cosine matrix. Data is stored in row-order. [0,0]=[0], [0,1]=[1], [0,2]=[2], [1,0]=3 ... [2,2]=[8].
Output:	
`REAL8 rot[0..2]`	The rotations, first about X, then the rotated Y, then the doubly-rotated Z.
Return Code:	
None	
Remarks/Usage:	
There can be multiple possible solutions to the same direction cosine matrix. This routine simply returns one of the possible solutions.	
Example:	
None	

Vector Methods

feVectorTransform

Unpublished work. © 2025 Siemens

feVectorTransform	
(fromCSYS, baseXYZ, fromXYZ, toCSYS, xyz)	
Description:	
This function transforms vectors from one coordinate system to another.	
Input:	
`INT4 fromCSYS`	The ID of the coordinate system in which the `fromXYZ` coordinates are currently specified. Specify 0 for global rectangular.
`REAL8 baseXYZ[0..2]`	The coordinates at the base of the vector to be transformed. Must be specified in the `fromCSYS` coordinate system.
`REAL8 fromXYZ[0..2]`	The components of the vector to be transformed. Must be specified in the `fromCSYS` coordinate system.
`INT4 toCSYS`	The ID of the coordinate system to transform into.
Output:	
`REAL8 xyz[0..2]`	The transformed vector components, returned in the `toCSYS` coordinate system.
Return Code:	
`FE_NOT_EXIST`	One of the coordinate systems does not exist.
Remarks/Usage:	
None	
Example:	
None	

feVectorArrayTransform

Unpublished work. © 2025 Siemens

feVectorArrayTransform

(count, fromCSYS, toCSSYS, baseXYZ, fromXYZ, xyz)

Description:

This function transforms an array of vectors from one coordinate system to another.

Input:

`INT4 count`	The number of vectors to be transformed, not the total size of the `baseXYZ` and `fromXYZ` arrays.
`INT4 fromCSYS`	The ID of the coordinate system in which the `fromXYZ` coordinates are currently specified. Specify 0 for global rectangular.
`INT4 toCSYS`	The ID of the coordinate system to transform into.
`REAL8 baseXYZ[0..3*count-1]`	The coordinates at the base of the vector to be transformed. Must be specified in the `fromCSYS` coordinate system.
`REAL8 fromXYZ[0..3*count-1]`	The components of the vector to be transformed. Must be specified in the `fromCSYS` coordinate system.

Output:

`REAL8 xyz[0..3*count-1]`	The transformed vector components for each original vector, returned in the `toCSYS` coordinate system.

Return Code:

`FE_NOT_EXIST`	One of the coordinate systems does not exist.
`FE_INVALID`	The value for count is invalid.
`FE_TOO_SMALL`	The `baseXYZ` array or `fromXYZ` array does not have enough entries.

Remarks/Usage:

None

Example:

```
Sub Main

   Dim App As femap.model
```


Unpublished work. © 2025 Siemens

```
   Set App = feFemap()
   Dim vecbase(5) As Double
   Dim vecfrom(5) As Double
   Dim vecto As Variant
   vecbase(0) = 0.0
   vecbase(1) = 0.0
   vecbase(2) = 0.0
   vecbase(3) = 0.0
   vecbase(4) = 0.0
   vecbase(5) = 0.0
   vecfrom(0) = 1.0
   vecfrom(1) = 0.0
   vecfrom(2) = 0.0
   vecfrom(3) = 0.0
   vecfrom(4) = -1.0
   vecfrom(5) = 0.0
   rc = App.feVectorArrayTransform _
   (1, 0, 3, vecbase, vecfrom, vecto)
End Sub
```

feVectorFromBeam

feVectorFromBeam

(elemID, bmXYZ, xyz)

Description:

This function transforms a vector from beam/bar elemental coordinates to global rectangular.

Unpublished work. © 2025 Siemens

Input:	
`INT4 elemID`	The ID of the element that defines the coordinate system where the `bmXYZ` components are defined.
`REAL8 bmXYZ[0..2]`	The components of the vector to be transformed. Must be specified in the elemental coordinate system of element `elemID`.
Output:	
`REAL8 xyz[0..2]`	The transformed vector components, returned in the global rectangular coordinate system.
Return Code:	
None	
Remarks/Usage:	
The element specified must exist, and must be an appropriate line element (beam, bar...) type.	
Example:	
None	

feVectorToBeam

`feVectorToBeam`
`( elemID, xyz, bmXYZ )`
Description:
This function transforms a vector from global rectangular coordinates into beam/bar elemental coordinates.
Input:

`INT4 elemID`	The ID of the element that defines the coordinate system where the `bmXYZ` components are defined.
`REAL8 xyz[0..2]`	The components of the vector to be transformed. Must be specified in global rectangular coordinates.
Output:	
`REAL8 bmXYZ[0..2]`	The transformed vector components, returned in the elemental coordinates of `elemID`.

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
The element specified must exist, and must be an appropriate line element (beam, bar...) type.	
Example:	
None	

feVectorLength

`feVectorLength`
 	
`( xyz, vecLength )`	

Description:	
This function computes the length of a vector.	
Input:	
`REAL8 xyz[0..2]`	The components of the vector to be measure. Must be specified in rectangular coordinates.
Output:	
`REAL8 vecLength`	The length of the vector.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feVectorCrossPoduct

Unpublished work. © 2025 Siemens

feVectorCrossPoduct	
(vecA, vecB, vecAxB)	
Description:	
This function computes the cross product of two vectors.	
Input:	
`REAL8 vecA[0..2]`	The components of the first vector.
`REAL8 vecB[0..2]`	The components of the second vector.
Output:	
`REAL8 vecAxB[0..2]`	The components of the cross product of the two vectors.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feVectorDotProduct

feVectorDotProduct	
(vecA, vecB, dot)	
Description:	
This function computes the dot (scalar) product of two vectors.	
Input:	
`REAL8 vecA[0..2]`	The components of the first vector.

Unpublished work. © 2025 Siemens

`REAL8 vecB[0..2]`	The components of the second vector.
Output:	
`REAL8 dot`	The scalar product of the two vectors.
Return Code:	
None	
Remarks/Usage:	
In Femap versions prior to v10.3, this method normalized the supplied vectors prior to computing the dot product. It now computes a true dot product and a new method `feVectorNormalizedDotProduct( )` was added to provide the previous functionality.	
Example:	
None	

feVectorNormalizedDotProduct

`feVectorNormalizedDotProduct`
`( vecA, vecB, dot )`
Description:
This function normalizes the supplied vectors and computes their dot (scalar) product.
Input:

`REAL8 vecA[0..2]`	The components of the first vector.
`REAL8 vecB[0..2]`	The components of the second vector.
Output:	
`REAL8 dot`	The scalar product of the two vectors.
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

The result of this method is the cosine of the angle between the vectors.
Example:
None

feVectorPerpendicular

feVectorPerpendicular **(vecIN, vecOUT)**	
Description:	
This function computes an arbitrary vector that is perpendicular to the specified vector.	
Input:	
`REAL8 vecIN[0..2]`	The components of the first vector.
Output:	
`REAL8 vecOUT[0..2]`	The perpendicular vector.
Return Code:	
None	
Remarks/Usage:	
If the vector specified is aligned with a global axis, you will always get a different global axis returned. If not, a cross product with either the global X (or global Z if the original vector is too close to the global X axis) is used to determine the perpendicular.	
Example:	
None	

feVectorAngle

Unpublished work. © 2025 Siemens

feVectorAngle	
(vecA, vecB, angle)	
Description:	
This function computes the angle between two vectors.	
Input:	
`REAL8 vecA[0..2]`	The components of the first vector.
`REAL8 vecB[0..2]`	The components of the second vector.
Output:	
`REAL8 angle`	The included angle between the two vectors.
Return Code:	
None	
Remarks/Usage:	
The angle is always between 0 and 180 degrees. There is no indication or consideration of the direction of the included angle between the vectors.	
Example:	
None	

feVectorAngleAxis

feVectorAngleAxis	
(vecA, vecB, axis, angle)	
Description:	
This function computes the angle between two vectors rotating around an axis.	
Input:	
`REAL8 vecA[0..2]`	The components of the first vector.

Unpublished work. © 2025 Siemens

`REAL8 vecB[0..2]`	The components of the second vector.
`REAL8 axis[0..2]`	The components of the axis of rotation.
Output:	
`REAL8 angle`	The included angle between the two vectors.
Return Code:	
None	
Remarks/Usage:	
The angle is always between -180 and 180 degrees. If the angle subtended when going from vecA to vecB is in a positive direction (right-hand rule sense) around the axis of rotation, then the returned angle is positive; otherwise a negative angle is returned.	
Example:	
None	

feVectorPlaneIntersect

`feVectorPlaneIntersect`
`( baseA, normA, baseB, normB, ptV, dirV )`
Description:
This function computes the vector at the intersection of two planes.
Input:

`REAL8 baseA[0..2]`	The coordinates of a location on the first plane.
`REAL8 normA[0..2]`	The components of the normal vector of the first plane.
`REAL8 baseB[0..2]`	The coordinates of a location on the second plane.
`REAL8 normB[0..2]`	The components of the normal vector of the second plane.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 ptV[0..2]`	The coordinates of a location on the intersection vector.
`REAL8 dirV[0..2]`	The components of the vector along the line of intersection.
Return Code:	
None	
Remarks/Usage:	
All coordinates and components must be specified in the same rectangular coordinate system - normally global rectangular.	
Example:	
None	

feVectorUnit

`feVectorUnit` **`( vecIN, vecNORM )`**	
Description:	
This function normalizes a vector to make it into an equivalent unit vector.	
Input:	
`REAL8 vecIN[0..2]`	The componets of the vector to be normalized
Output:	
`REAL8 vecNORM[0..2]`	The normalized vector.
Return Code:	
None	
Remarks/Usage:	
In addition to simply dividing by the length of the vector, this function also completely eliminates components of the vector that are very small (less than 1E-8 times the length) compared to the other components. This helps to eliminate numerical errors which can creep in during transformations or other vector operations.	

Unpublished work. © 2025 Siemens

Example:
None

feVectorPick

feVectorPick **(dlgTitle, unitVector, vecLength, vecBase, vecDir)**	
Description:	
This function displays a dialog box to allow a user to pick or enter vectors.	
Input:	
`STRING dlgTitle`	A string that specifies a title to be added to the dialog box title.
`BOOL unitVector`	Set to False if you want the user to be able to enter a vector length.
Output:	
`REAL8 vecLength`	The length of the vector. This will be 1.0 if `unitVector=True`.
`REAL8 vecBase[0..2]`	The coordinates at the base of the vector.
`REAL8 vecDir[0..2]`	The components of the vector.
Return Code:	
`FE_CANCEL`	The dialog box was cancelled.
Remarks/Usage:	
The components in `vecDir` will always be those of a unit vector. The vector length is always returned in `vecLength`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feVectorPickByMethod

feVectorPickByMethod **(method, dlgTitle, unitVector, vecLength, vecBase, vecDir)**	
Description:	
This function displays a dialog box to allow a user to pick or enter vectors. Same as `feVectorPick` except that you can specify the initial dialog box method.	
Input:	
`INT4 method`	The initial dialog box method. Refer to **Vector Dialog Methods** for possible values.
`STRING dlgTitle`	A string that specifies a title to be added to the dialog box title.
`BOOL unitVector`	Set to False if you want the user to be able to enter a vector length.
Output:	
`REAL8 vecLength`	The length of the vector. This will be 1.0 if `unitVector=True`.
`REAL8 vecBase[0..2]`	The coordinates at the base of the vector.
`REAL8 vecDir[0..2]`	The components of the vector.
Return Code:	
`FE_CANCEL`	The dialog box was cancelled.
Remarks/Usage:	
The components in `vecDir` will always be those of a unit vector. The vector length is always returned in `vecLength`.	
Example:	
None	

feVectorAxis

Unpublished work. © 2025 Siemens

feVectorAxis

(csysID, axis, positive, vecDir)

Description:

This function creates a unit vector in the direction of one of the axes of a coordinate system.

Input:

`INT4 csysID`	The ID of the coordinate system to use for the vector definition.
`INT4 axis`	Set to 0=X axis, 1=Y axis, 2=Z axis.
`BOOL positive`	True if you want the positive axis; False for the negative axis.

Output:

`REAL8 vecDir[0..2]`	The components of the vector.

Return Code:

`FE_NOT_EXIST`	The coordinate system that you requested does not exist.

Remarks/Usage:

The components in `vecDir` will always be those of a unit vector.

Example:

None

feVectorTangentToCurve

feVectorTangentToCurve

(cuID, baseXYZ, vecDir)

Description:

This function returns a unit vector that is the tangent to a curve at a specified location.

Input:

Unpublished work. © 2025 Siemens

`INT4 cuID`	The ID of the curve used to compute the tangent.
`REAL8 baseXYZ[0..2]`	The coordinates of the location used to compute the tangent.
Output:	
`REAL8 vecDir[0..2]`	The components of the tangent vector at `baseXYZ`.
Return Code:	
`FE_NOT_EXIST`	The specified curve does not exist.
Remarks/Usage:	
The components in `vecDir` will always be those of a unit vector. If the location specified in `baseXYZ` is not on the curve, it will be projected onto the curve and the tangent will be at the projected location.	
Example:	
None	

feVectorNormalToSurface

`feVectorNormalToSurface`	
`( suID, baseXYZ, vecDir )`	
Description:	
This function returns a unit vector that is the normal to a surface at a specified location.	
Input:	
`INT4 suID`	The ID of the surface used to compute the normal.
`REAL8 baseXYZ[0..2]`	The coordinates of the location used to compute the normal.
Output:	
`REAL8 vecDir[0..2]`	The components of the normal vector at baseXYZ.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	The specified surface does not exist.
Remarks/Usage:	
The components in `vecDir` will always be those of a unit vector. If the location specified in `baseXYZ` is not on the surface, it will be projected onto the surface and the normal will be at the projected location.	
Example:	
None	

feVectorNormalToView

`feVectorNormalToView` **`( suID, vecDir )`**	
Description:	
This function returns a unit vector that is the normal to a view.	
Input:	
`INT4 vuID`	The ID of the view used to compute the normal. Set to -1 to compute the normal of the active view.
Output:	
`REAL8 vecDir[0..2]`	The components of the normal vector.
Return Code:	
`FE_NOT_EXIST`	The specified view does not exist.
Remarks/Usage:	
The components in `vecDir` will always be those of a unit vector. The normal vector always points out of the screen.	
Example:	
None	

feVectorAxisOfSurface

Unpublished work. © 2025 Siemens

feVectorAxisOfSurface	
(suID, baseXYZ, vecDir)	
Description:	
This function returns the axis of revolution for a surface.	
Input:	
`INT4 suID`	The ID of the surface to find revolution axis.
Output:	
`REAL8 baseXYZ[0..2]`	The coordinates at the base of the revolution vector.
`REAL8 vecDir[0..2]`	The components of the revolution vector.
Return Code:	
`FE_NOT_EXIST`	Surface does not exist.
`FE_CANCEL`	Unable to find surface axis or surface is not a revolute type.
Remarks/Usage:	
None	
Example:	
None	

feVectorRotate

feVectorRotate	
(baseXYZ, vecXYZ, angle, origin, axis, baseROT, vecROT)	
Description:	
This function returns a vector computed from rotating the original vector around a given axis using the specified angle, in degrees.	

Unpublished work. © 2025 Siemens

Input:	
`REAL8 baseXYZ[0..2]`	The coordinates at the base of the original vector.
`REAL8 vecXYZ[0..2]`	The components of the original vector.
`REAL8 angle`	Rotation angle in degrees
`REAL8 origin[0..2]`	The coordinates at the base of the vector to rotate about.
`REAL8 axis[0..2]`	The components of the vector to rotate about.
Output:	
`REAL8 baseROT[0..2]`	The coordinates at the base of the rotated vector.
`REAL8 vecROT[0..2]`	The components of the rotated vector.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Plane Methods

fePlanePick

`fePlanePick`
`( dlgTITLE, plBase, plNormal, plAxis )`
Description:
This function displays a dialog box to allow a user to define a plane.

Unpublished work. © 2025 Siemens

Input:	
`STRING dlgTitle`	A string that specifies a title to be added to the dialog box title.
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_CANCEL`	The dialog box was cancelled.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

fePlanePickByMethod

`fePlanePickByMethod`
`( method, dlgTITLE, plBase, plNormal, plAxis )`
Description:
This function displays a dialog box to allow a user to define a plane. Same as `fePlanePick` except you can specify the initial dialog box method.
Input:
`INT4 method` — The initial dialog box method. Refer to **Vector Dialog Methods** for possible values.
`STRING dlgTitle` — A string that specifies a title to be added to the dialog box title.
Output:

Unpublished work. © 2025 Siemens

`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_CANCEL`	The dialog box was cancelled.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

fePlaneLocation

`fePlaneLocation` **`( p1, p2, p3, planeBase, planeNormal, planeX )`**	
Description:	
Defines a plane by specifying the coordinates of three points on the plane.	
Input:	
`REAL8 p1[0..2]`	The coordinates of the points on the plane. The first point is the origin. The vector from the first to the second point is the X axis.
`REAL8 p2[0..2]`	
`REAL8 p3[0..2]`	
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.

Unpublished work. © 2025 Siemens

`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The three locations were colinear.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the Normal and the X axis.	
Example:	
None	

fePlaneNodes

`fePlaneNodes` **`( n1, n2, n3, planeBase, planeNormal, planeX )`**	
Description:	
Defines a plane by specifying the IDs of three nodes on the plane.	
Input:	
`INT4 n1`	The IDs of the three nodes on the plane. The first point is the origin. The vector from the first to the second point is the X axis.
`INT4 n2`	
`INT4 n3`	
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	The three locations were colinear.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

fePlanePoints

`fePlanePoints` **`( p1, p2, p3, planeBase, planeNormal, planeX )`**	
Description:	
Defines a plane by specifying the IDs of three points on the plane.	
Input:	
`INT4 p1`	The IDs of the three points on the plane. The first point is the origin. The vector from the first to the second point is the X axis.
`INT4 p2`	
`INT4 p3`	
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The three locations were colinear.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	

Unpublished work. © 2025 Siemens

Example:
None

fePlaneBisect

fePlaneBisect **(p1, p2, p3, planeBase, planeNormal, planeX)**	
Description:	
Defines a plane that bisects the angle formed between three points.	
Input:	
`REAL8 p1[0..2]`	The location at the origin of the two vectors to be bisected.
`REAL8 p2[0..2]`	The end of the first vector (from p1) that will be used for the bisection.
`REAL8 p3[0..2]`	The end of the second vector (from p1) that will be used for the bisection.
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The three locations were colinear.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

Unpublished work. © 2025 Siemens

fePlaneCSys

fePlaneCSys **(p1, csysID, planeID, planeBase, planeNormal, planeX)**	
Description:	
Defines a plane that is one of the principal planes of a coordinate system.	
Input:	
`REAL8 p1[0..2]`	A location that will be the origin of the plane - specified in global coordinates.
`INT4 csysID`	The ID of the coordinate system to use to define the plane. Specify as 0 for global coordinates.
`INT4 planeID`	A number that selects the plane of the coordinate system to use: 0=Positive XY, 1=Positive, YZ, 2=Positive ZX, 3=Negative XY, 4=Negative, YZ, 5=Negative ZX.
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The coordinate system was invalid or does not exist.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

fePlaneSurfaceNormal

Unpublished work. © 2025 Siemens

fePlaneSurfaceNormal

(surfID, ptID1, ptID2, reversed, align, offset, planeBase, planeNormal, planeX)

Description:	
Defines a plane that is normal to a surface.	
Input:	
`INT4 surfID`	The ID of the surface used to define the normal.
`INT4 ptID1`	The ID of a point that will define the origin of the plane.
`INT4 ptID2`	The ID of a point that will be used to define the X axis of the plane.
`BOOL reversed`	Set to True if you want the normal that is reversed from the surface normal.
`BOOL align`	If True, the axis of the plane will be aligned to the surface axes.
`REAL8 offset`	The distance that the plane will be offset, along its normal vector, from the first point (origin).
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The surface was invalid or does not exist.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

Unpublished work. © 2025 Siemens

fePlaneCurveNormal

fePlaneCurveNormal **(cuID, planeBase, planeNormal, planeX)**	
Description:	
Defines a plane that is normal to a curve.	
Input:	
`INT4 cuID`	The ID of the curve used to determine the normal. The curve must be planar, and cannot be a line. This is usually used with arcs and circles to determine their plane.
Output:	
`REAL8 plBase[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The curve was invalid or does not exist.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

fePlaneViewAlign

Unpublished work. © 2025 Siemens

fePlaneViewAlign	
(viewID, intoView, planeNormal, planeX)	
Description:	
Defines a plane that is normal to a view.	
Input:	
`INT4 viewID`	The ID of a view.
`BOOL intoView`	Determines whether the normal will be into or out of the screen.
Output:	
`REAL8 plNormal[0..2]`	The components of the vector that is normal to the plane.
`REAL8 plAxis[0..2]`	The components of a vector along the plane X axis.
Return Code:	
`FE_FAIL`	The view was invalid or does not exist.
Remarks/Usage:	
The plane Y axis can be computed from a cross product of the normal and the X axis.	
Example:	
None	

fePlaneRotate

fePlaneRotate
(baseXYZ, vecXXYZ, vecZXYZ, angle, origin, axis, baseROT, vecXROT, vecYROT, vecZROT)
Description:

Unpublished work. © 2025 Siemens

This function returns three vectors computed from rotating the original plane around a given axis using the specified angle, in degrees.	
Input:	
`REAL8 baseXYZ[0..2]`	The coordinates of a location on the plane, in global rectangular coordinates.
`REAL8 vecXXYZ[0..2]`	The components of a vector along the plane X axis.
`REAL8 vecZXYZ[0..2]`	The components of the vector that is normal to the plane.
`REAL8 angle`	Rotation angle in degrees
`REAL8 origin[0..2]`	The coordinates at the base of the vector to rotate about.
`REAL8 axis[0..2]`	The components of the vector to rotate about.
Output:	
`REAL8 baseROT[0..2]`	The coordinates at the base of the rotated vector.
`REAL8 vecXROT[0..2]`	The components of the rotated vecXXYZ vector.
`REAL8 vecYROT[0..2]`	The rotated plane Y.
`REAL8 vecZROT[0..2]`	The components of the rotated vecZXYZ vector.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Entity Selection Methods

Unpublished work. © 2025 Siemens

Selection Dialog Boxes

Entity selection dialog boxes available in the Femap interface are available through the API using certain Set Object Methods. These Methods require you to create a Femap Set Object before they can be used. More information on these Set Object Methods can be found in:

Select - standard Femap entity selection dialog box
SelectID - standard Single entity selection dialog box in Femap.
SelectMultiIDV2 - standard Multiple entity selection dialog box in Femap.
SelectOutputVectorIDV2 - select a single Output Vector ID from a list of Output Vectors in a Femap dialog box.

In addition to these methods that select entire entities using Sets, you can select output or element faces using the following methods:

feSelectOutputV2

feSelectOutputV2

**(dlgTITLE, nBaseOutputSetID, limitOutputType, limitComplex, limitToEntity,
includeCorner, pOutputSets, pOutputVecs)**

Description:

This function displays a dialog box to allow a user to select Output Sets and Vectors. This method is obsolete and should no longer be used. It has been replaced by `feSelectOutput2`. This method replaces `feSelectOutput`. For more information, see **Working with Output in Femap v2020.1 and above**.

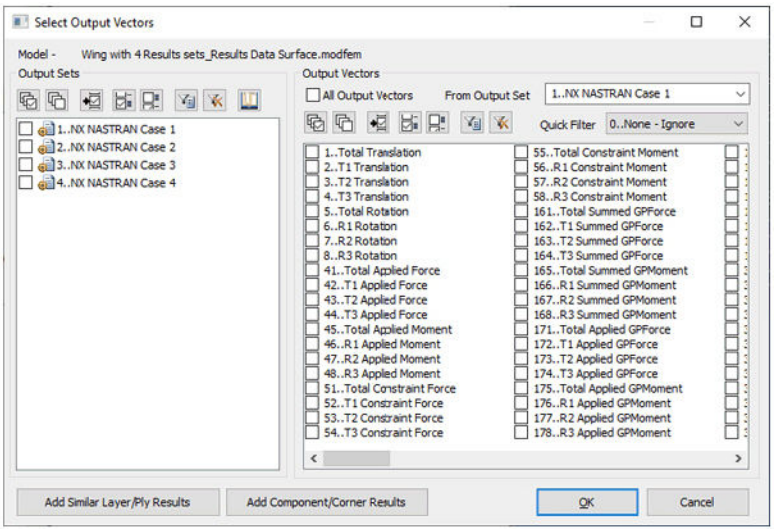
Input:

`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
`INT4 nBaseOutputSetID`	The ID of an output set to use to fill the Output Vector selection control. If 0, then the active Output Set is used, or if none is active, the first available one is used. If specified as a negative number, then the absolute value of that number is used, but the Output Set selection control only contains that Output Set, not all the sets available.
`INT4 limitOutputType`	Reduces list of output vectors shown to the specified type (0=All, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp).

Unpublished work. © 2025 Siemens

`INT4 limitComplex`	Reduces list of output vectors shown based on type of real or complex data they contain (0=Magnitude, 1=Phase, 2=Real, 3=Imaginary, 4=Any).
`INT4 limitToEntity`	Reduces list of output vectors to those containing data on the specified entity type.
`BOOL includeCorner`	If True, output vectors at element corners will also be included.
Output:	
`OBJECT pOutputSets`	A Set object that contains the Output Sets that were selected.
`OBJECT pOutputVectors`	A Set object that contains the Output Vectors that were selected.
Return Code:	
`FE_FAIL`	The user cancelled the dialog box.
`FE_NOT_EXIST`	No output sets/vectors exist.
Remarks/Usage:	

The following dialog box is used for the set/vector selection...



To have this dialog box be displayed showing Output Sets organized under already existing Analysis Studies, set `SelectOutputSetsFromStudies` to True (see **Global Boolean Properties**) before using this call.

Pressing **Add Similar Layer/Ply Results** will add Output Vectors that are of the same type but at a different location in the element. For example if **Top Von Mises Stress** is selected, pressing this will

Unpublished work. © 2025 Siemens

automatically select **Bottom Von Mises Stress**. For laminates it will select output on all plys. Pressing **Add Component/Corner Results** will select output on element corners or nodal vector component output of the same type as those vectors already selected.

For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is FE_OK.

Example:

The following routine allows you to select output vectors and list the max/min values for selected elements...

```
Sub Main

Dim App As femap.model

Set App = feFemap

Dim s As femap.Set

Dim v As femap.Set

Dim e As femap.Set

Set e = App.feSet

Dim RBO As femap.Results

Set RBO = App.feResults

Dim LCol As Long

Dim vCol As Variant

Dim minID As Long

Dim maxID As Long

Dim minVAL As Double

Dim maxVAL As Double

Dim rc As Long

If App.feSelectOutput2V2( "Select Output Vectors", 0, 0, 0, FOT_ANY,
FOC_ANY, FT_ELEM, False, s, v ) <> FE_OK Then End

 If e.Select( FT_ELEM, True, "Select Elements" ) <> FE_OK Then End

    App.feAppMessage( FCM_NORMAL, "Results ID, Vector ID, Minimum ID,
Minimum Value, Maximum ID, Maximum Value" )
```


Unpublished work. © 2025 Siemens

```
   While s.Next()

     v.Reset()

     While v.Next()

       rc = RBO.Clear

        rc = RBO.AddColumnV2( s.CurrentID, v.CurrentID, False, LCol,
vCol )

        rc = RBO.GetColumnMinMax( 0, e.ID, minID, maxID, minVAL, maxVAL )

         App.feAppMessage( FCM_NORMAL, Str$(s.CurrentID)+"          "+Str$
(v.CurrentID)+" "+Str$(minID)+"="+Str$(minVAL)+"         "+Str$(maxID)
+"="+Str$(maxVAL) )

     Wend

   Wend

End Sub
```

feSelectOutput2V2

feSelectOutput2V2

**(title, nBaseOutputSetID, nPreCheckedSetSetID, nPreCheckedVectorSetID, limitOutputType,
limitComplex, limitToEntity, includeCorner, pOutputSets, pOutputVecs)**

Description:

This function displays a dialog box to allow a user to select Output Sets and Vectors along with already selected Output Sets and/or Vectors. This method replaces `feSelectOutput2`. For more information, see **Working with Output in Femap v2020.1 and above**.

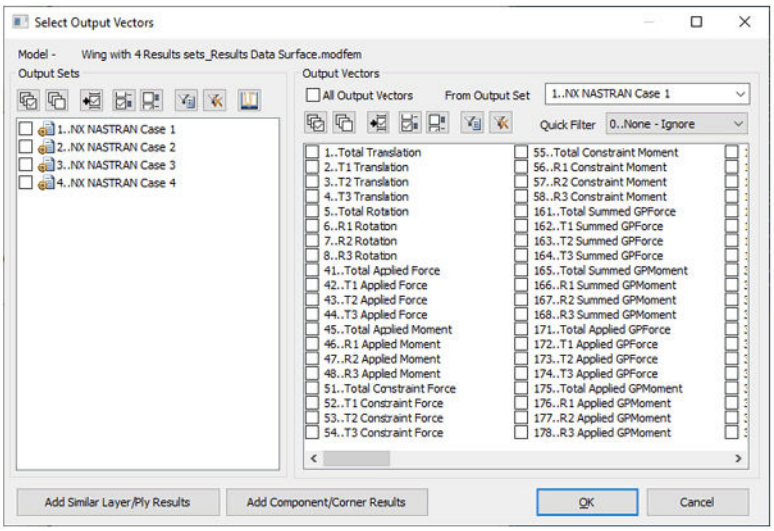
Input:

`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
`INT4 nBaseOutputSetID`	The ID of an output set to use to fill the Output Vector selection control. If 0, then the active Output Set is used, or if none is active, the first available one is used. If specified as a negative number, then the absolute value of that number

Unpublished work. © 2025 Siemens

	is used, but the Output Set selection control only contains that Output Set, not all the sets available.
`INT4 nPreCheckedSetSetID`	The ID of the set object containing Output Set IDs to be pre-checked.
`INT4 nPreCheckedVectorSetID`	The ID of the set object containing Output Vector IDs to be prechecked.
`INT4 limitOutputType`	Reduces list of output vectors shown to the specified type (0=All, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp).
`INT4 limitComplex`	Reduces list of output vectors shown based on type of real or complex data they contain (0=Magnitude, 1=Phase, 2=Real, 3=Imaginary, 4=Any).
`INT4 limitToEntity`	Reduces list of output vectors to those containing data on the specified entity type.
`BOOL includeCorner`	If True, output vectors at element corners will also be included.
Output:	
`OBJECT pOutputSets`	A Set object that contains the Output Sets that were selected.
`OBJECT pOutputVectors`	A Set object that contains the Output Vectors that were selected.
Return Code:	
`FE_FAIL`	The user cancelled the dialog box.
`FE_NOT_EXIST`	No output sets/vectors exist.
Remarks/Usage:	
The following dialog box is used for the set/vector selection...	

Unpublished work. © 2025 Siemens



To have this dialog box be displayed showing Output Sets organized under already existing Analysis Studies, set `SelectOutputSetsFromStudies` to True (see **Global Boolean Properties**) before using this call.

Pressing **Add Similar Layer/Ply Results** will add Output Vectors that are of the same type but at a different location in the element. For example if **Top Von Mises Stress** is selected, pressing this will automatically select **Bottom Von Mises Stress**. For laminates it will select output on all plys. Pressing **Add Component/Corner Results** will select output on element corners or nodal vector component output of the same type as those vectors already selected.

For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.

Example:

None

feSelectOutputSets

`feSelectOutputSets`

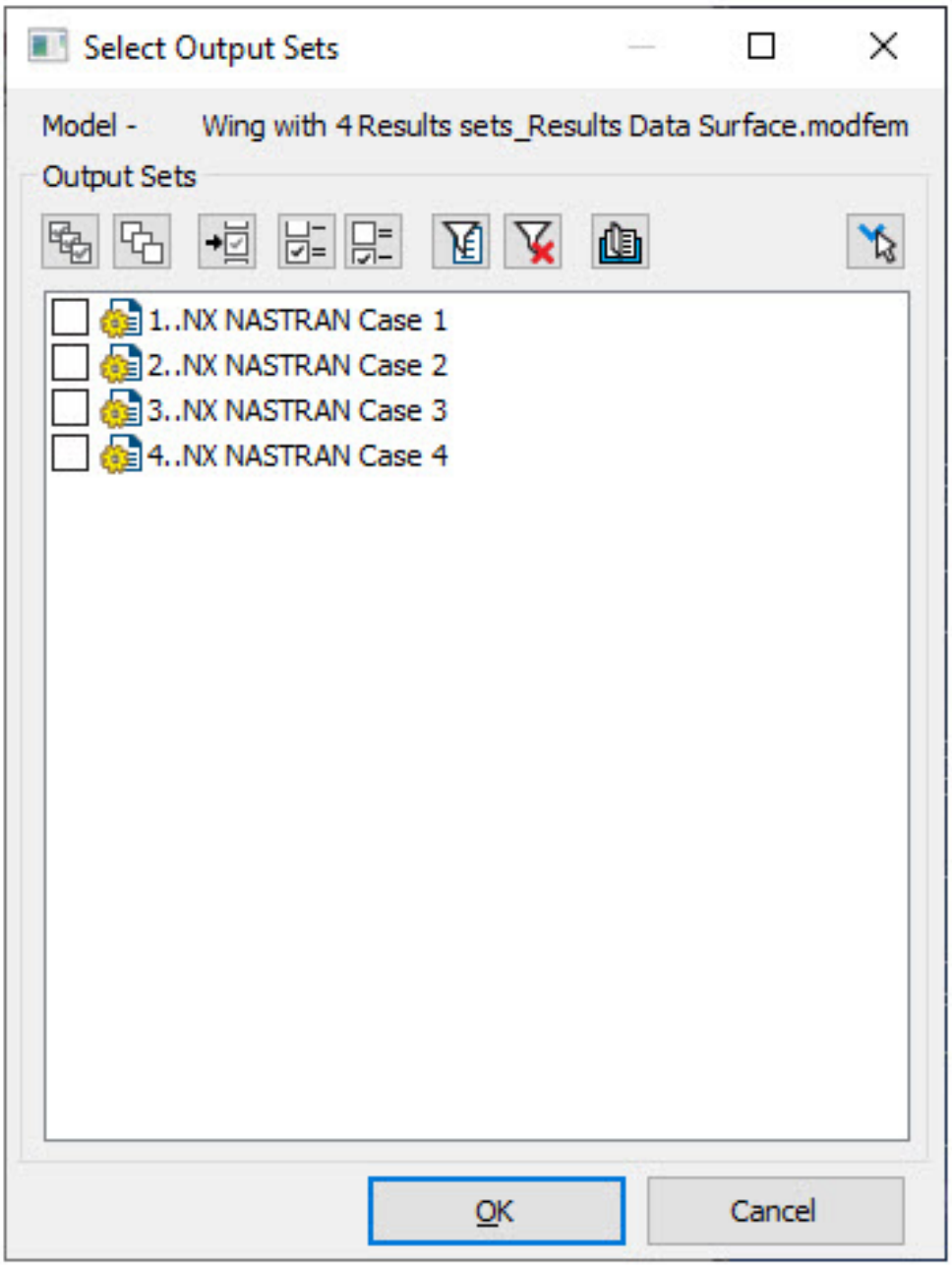
`( dlgTITLE, pOutputSets )`

Description:

Unpublished work. © 2025 Siemens

This function displays a dialog box to allow a user to select Output Sets. This method is obsolete and should no longer be used. It has been replaced by `feSelectOutputSets2`.	
Input:	
`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
Output:	
`OBJECT pOutputSets`	A Set object that contains the Output Sets that were selected.
Return Code:	
`FE_FAIL`	The user cancelled the dialog box.
`FE_NOT_EXIST`	No output sets exist.
Remarks/Usage:	
The following dialog box is used for the set selection:	

Unpublished work. © 2025 Siemens



To have this dialog box be displayed showing Output Sets organized under already existing Analysis Studies, set `SelectOutputSetsFromStudies` to True (see **Global Boolean Properties**) before using this call.

For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.

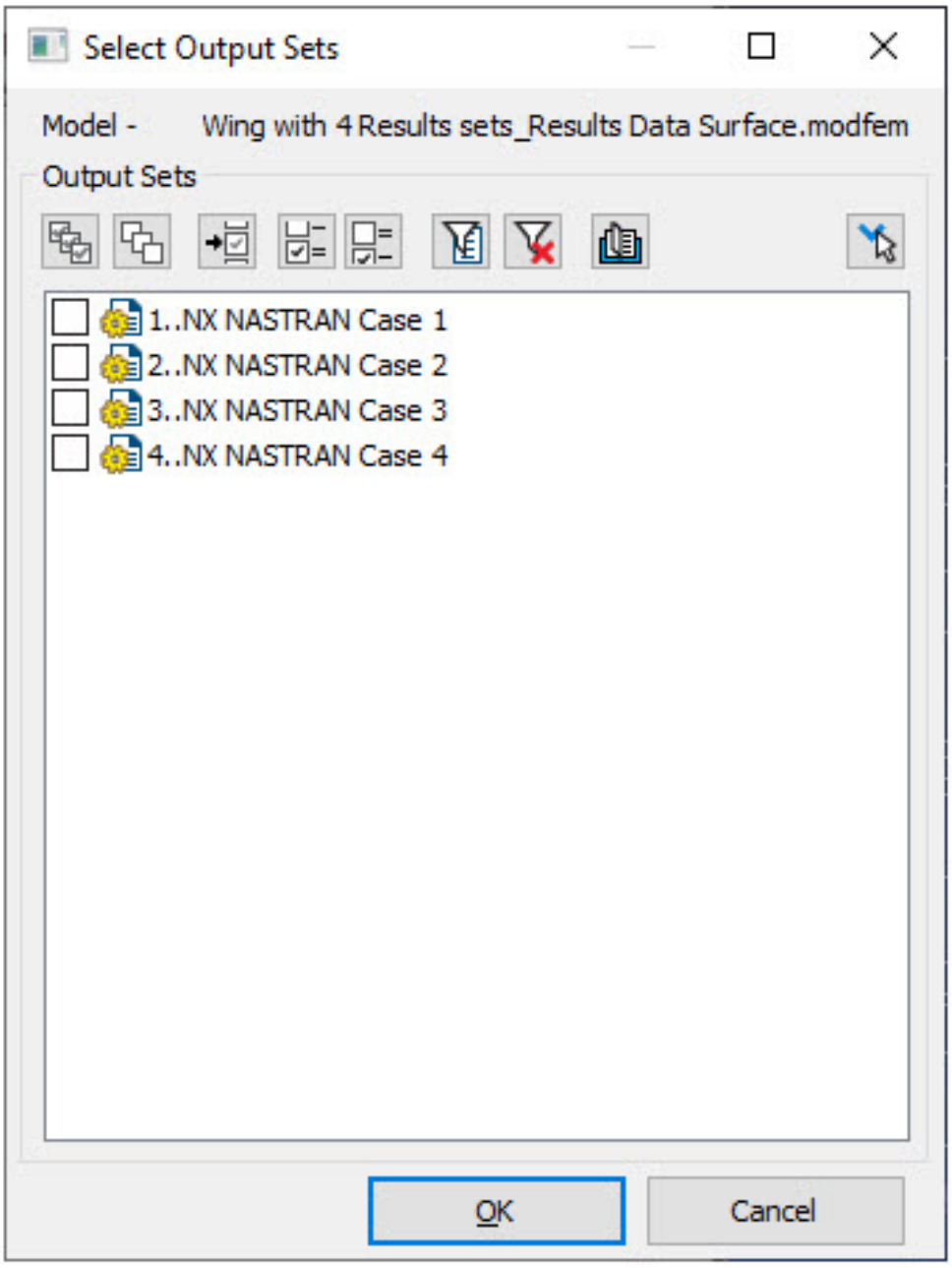
Unpublished work. © 2025 Siemens

Example:
None

feSelectOutputSets2

feSelectOutputSets2 **(title, nPreCheckedSetSetID, pOutputSets)**	
Description:	
This function displays a dialog box to allow a user to select Output Sets with pre-checked Output Sets.	
Input:	
`STRING title`	A string that specifies a title to be used as the dialog box title.
`INT4 nPreCheckedSetSetID`	The ID of the set object containing Output Set IDs to be pre-checked.
Output:	
`OBJECT pOutputSets`	A Set object that contains the Output Sets that were selected.
Return Code:	
`FE_FAIL`	The user cancelled the dialog box.
`FE_NOT_EXIST`	No output sets exist.
Remarks/Usage:	
The following dialog box is used for the set selection:	

Unpublished work. © 2025 Siemens



To have this dialog box be displayed showing Output Sets organized under already existing Analysis Studies, set `SelectOutputSetsFromStudies` to True (see **Global Boolean Properties**) before using this call.

For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.

Unpublished work. © 2025 Siemens

Example:
None

feGetElementFaces

feGetElementFaces **(dlgTITLE, orig_nCount, orig_elemID, orig_faceID, nCount, elemID, faceID)**	
Description:	
This function displays a dialog box to allow a user to select element faces.	
Input:	
`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
`INT4 orig_nCount`	The number of elements/faces that are contained in `orig_elemID` and `orig_faceID`. These element faces are displayed in the dialog box when it is opened. Specify 0 to display an empty dialog box.
`INT4 orig_elemID[0..orig_nCount-1]`	The element IDs to be loaded when the dialog is opened.
`INT4 orig_faceID[0..orig_nCount-1]`	The face IDs to be loaded when the dialog is opened.
Output:	
`INT4 nCount`	The number of elements/faces that have been selected. Unless some or all of the *orig* faces were deleted, they will also be contained in this list.
`INT4 elemID[0..nCount-1]`	The element IDs that were selected.
`INT4 faceID[0..nCount-1]`	The face IDs that were selected.
Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Value is not valid.

Unpublished work. © 2025 Siemens

Remarks/Usage:
Element faces are paired in the `elemID/faceID` arrays. The face ID corresponds to the element ID at the same index in the array.
Example:
None

feGetElementEdges

feGetElementEdges **(dlgTITLE, orig_nCount, orig_elemID, orig_edgeID, nCount, elemID, edgeID)**	
Description:	
This function displays a dialog box to allow a user to select element edges.	
Input:	
`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
`INT4 orig_nCount`	The number of elements/edges that are contained in `orig_elemID` and `orig_faceID`. These element faces are displayed in the dialog box when it is opened. Specify 0 to display an empty dialog box.
`INT4 orig_elemID[0..orig_nCount-1]`	The element IDs to be loaded when the dialog is opened.
`INT4 orig_edgeID[0..orig_nCount-1]`	The `edgeIDs` to be loaded when the dialog is opened.
Output:	
`INT4 nCount`	The number of elements/edges that have been selected. Unless some or all of the *orig* edges were deleted, they will also be contained in this list.
`INT4 elemID[0..nCount-1]`	The element IDs that were selected.
`INT4 edgeID[0..nCount-1]`	The edge IDs that were selected.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Value is not valid.
Remarks/Usage:	
Element edges are paired in the `elemID/edgeID` arrays. The edge ID corresponds to the element ID at the same index in the array. The `edgeID` is the same as the `faceID` on the edge of planar elements. This method simply limits selection to those edge faces.	
Example:	
None	

feGetElementFacesFromSet

feGetElementFacesFromSet **(title, nElemSetID, nCount, elemID, faceID)**	
Description:	
This function displays a dialog box to allow a user to select element faces with the added option to limit the face selection using a set object loaded with element IDs.	
Input:	
`STRING title`	A string that specifies a title to be used as the Face Selection dialog box title.
`INT4 nElemSetID`	The ID of a Set object containing element IDs which will be used to limit face selection. Alternatively, specify 0 to display a standard entity selection dialog box for Element selection.
Output:	
`INT4 nCount`	The number of elements/faces that have been selected.
`INT4 elemID[0..nCount-1]`	The element IDs that were selected.
`INT4 faceID[0..nCount-1]`	The face IDs that were selected. These are the original seed “faces”, and can be the planar faces of solid 3D elements or 2D elements, or the linear edges of 2D elements.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_CANCEL`	The user canceled the dialog box. Value is not valid.
Remarks/Usage:	
Element faces are paired in the `elemID/faceID` arrays. The face ID corresponds to the element ID at the same index in the array.	
Example:	
None	

feGetElementAdjacentFaces

`feGetElementAdjacentFaces`

`( nElemID, nFaceID, nFromElemSetID, dTolerance, bMatchNormals, nNumFaces, nElems, nFaces)`

Description:	
This function will get all adjacent faces within a specified tolerance from a given element and face ID.	
Input:	
`INT4 nElemID`	The ID of the element.
`INT4 nFaceID`	The ID of the face from the element.
`INT4 nFromElemSetID`	Selection from specific elements. **Note** If set to less than or 0, all elements will be added to the *from* set.
`REAL8 dTolerance`	The angle tolerance of which to select adjacent faces.
`BOOL bMatchNormals`	If True, only faces with matching normals will be selected.
Output:	

Unpublished work. © 2025 Siemens

`INT4 nNumFaces`	The number of faces selected.
`INT4 nElems[0..nNumFaces-1]`	An array of the IDs of the elements selected.
`INT4 nFaces[0..nNumFaces-1]`	An array of the IDs of the faces from the elements selected.
Return Code:	
`FE_FAIL`	
Remarks/Usage:	
This mimics the **Adjacent** option in the Face Selection dialog.	
Example:	
None	

Other Methods

feGetReal

`feGetReal` **`( dlgTITLE, lowerlimit, upperlimit, value )`**	
Description:	
This function displays a dialog box to allow a user to enter a real number.	
Input:	
`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
`REAL8 lowerlimit`	The minimum acceptable value for the number.
`REAL8 upperlimit`	The maximum acceptable value for the number.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 value`	The number entered by the user. On input, value is displayed as the default.
Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Value is not valid.
Remarks/Usage:	
None	
Example:	
None	

feGetRealLength

`feGetRealLength` **`( title, value )`**	
Description:	
This function displays a dialog box to allow a user to enter a real number or use an icon button to measure a distance using the **Tools → Measure Distance** functionality.	
Input:	
`STRING title`	A string that specifies a title to be used as the dialog box title.
Output:	
`REAL8 value`	The value entered by the user or distance returned when using the icon button. On input, value is displayed as the default.
Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Value is not valid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feGetInt

feGetInt **(dlgTITLE, lowerlimit, upperlimit, value)**	
Description:	
This function displays a dialog box to allow a user to enter an integer number.	
Input:	
`STRING dlgTITLE`	A string that specifies a title to be used as the dialog box title.
`INT4 lowerlimit`	The minimum acceptable value for the number.
`INT4 upperlimit`	The maximum acceptable value for the number.
Output:	
`INT4 value`	The number entered by the user. On input, value is displayed as the default.
Return Code:	
`FE_CANCEL`	The user cancelled the dialog box. Value is not valid.
Remarks/Usage:	
None	
Example:	
None	

feAppMessageBox

Unpublished work. © 2025 Siemens

feAppMessageBox	
(msgmode, message)	
Description:	
Displays a message box, asking the user to press a button to continue.	
Input:	
`INT4 msgmode`	Determines the buttons that will be available in the box. 0=OK, 1=OK and Cancel, 2 = Yes and No, 3 = Yes, No and Cancel.
`STRING message`	The text of the message to write. Must be less than 160 characters.
Output:	
None	
Return Code:	
`FE_OK`	User pressed either the OK or Yes button.
`FE_FAIL`	User pressed No button.
`FE_CANCEL`	User pressed Cancel button.
Remarks/Usage:	
This method is useful if your application is windowless since it ensure the Message Box will be displayed on top of all Femap windows.	
Example:	
None	

feAppColorPalette

feAppColorPalette
(nDefault, nColor)
Description:

Unpublished work. © 2025 Siemens

Displays the color palette dialog box allowing a user to pick a color.	
Input:	
`INT4 nDefault`	The default color, used to initialize the dialog box. This is a combination of the desired color, pattern and linestyle.
Output:	
`INT4 nColor`	The color/pattern/linestyle that the user selected. If the user presses Cancel, this will be the same as `nDefault`.
Return Code:	
`FE_OK`	Dialog box was displayed, then OK is pressed.
`FE_CANCEL`	Dialog box was displayed, then Cancel is pressed.
Remarks/Usage:	
None	
Example:	
None	

feCreateFunction

`feCreateFunction` **`( nNumberAvailableTypes, AvailableTypes, nID )`**	
Description:	
Displays the standard Create Function dialog box and has an option to restrict which function types.	
Input:	
`INT4 nNumberAvailableTypes`	The number of different function types that will be shown in the **Type** drop-down list. Use a value of 0.
`INT4 AvailableTypes[0..nNumberAvaliableTypes-1]`	Array of IDs representing different function types.

Unpublished work. © 2025 Siemens

Output:	
`INT4 nID`	ID of newly created function.
Return Code:	
`FE_OK`	Dialog box was displayed, then OK is pressed.
`FE_CANCEL`	Dialog box was displayed, then Cancel is pressed.
Remarks/Usage:	
None	
Example:	
None	

feAppColorCreate

`feAppColorCreate` **`( enColor, enPattern, enLinestyle )`**	
Description:	
Returns a Femap color containing a base color, pattern and linestyle mask.	
Input:	
`enum zColor enColor`	Base color - use enum `zColor.FCL_*`.
`enum zColor enPattern`	Pattern mask - use `enum zColor.FPF_*`. `FPF_SOLID` or 0 for none.
`enum zColor enLinestyle`	Linestyle mask - use `enum zColor.FPL_*`. `FPL_SOLID` or 0 for none.
Output:	
None	
Return Code:	
`INT4 color`	Femap color ID containing specified base color and pattern / linestyle masks.

Unpublished work. © 2025 Siemens

Remarks/Usage:
Use the enumerations provided in `zColor`. Numbers outside of this range may have unintended results.
Example:
None

feAppColorGetBaseColor

feAppColorGetBaseColor **(nColor, penColor)**	
Description:	
Extracts the base color from a Femap color ID.	
Input:	
`INT4 nColor`	Femap color ID.
Output:	
`enum zColor penColor`	Base color. Will be a value `zColor.FCL_*`.
Return Code:	
None	
Remarks/Usage:	
Color IDs are a combination of colors and pattern / linestyle masks from the `zColor` enum. They can be created manually or obtained from `feAppColorPalette` / `feAppColorCreate`.	
Example:	
None	

feAppColorGetPattern

Unpublished work. © 2025 Siemens

feAppColorGetPattern	
(infLine, pA1, pA2, pB1, pB2, pIntersect)	
Description:	
Extracts the pattern mask from a Femap color ID.	
Input:	
`INT4 nColor`	Femap color ID.
Output:	
`enum zColor penPattern`	Pattern Mask. Will be a value `zColor.FPF_*`.
Return Code:	
None	
Remarks/Usage:	
Color IDs are a combination of colors and pattern / linestyle masks from the `zColor` enum. They can be created manually or obtained from `feAppColorPalette / feAppColorCreate`.	
Example:	
None	

feAppColorGetLinestyle

feAppColorGetLinestyle	
(nColor, penLinestyle)	
Description:	
Extracts the pattern mask from a Femap color ID.	
Input:	
`INT4 nColor`	Femap color ID.
Output:	

Unpublished work. © 2025 Siemens

`enum zColor penLinestyle`	Linestyle mask. Will be a value `zColor.FPL_*`.
Return Code:	
None	
Remarks/Usage:	
Color IDs are a combination of colors and pattern / linestyle masks from the `zColor` enum. They can be created manually or obtained from `feAppColorPalette / feAppColorCreate`.	
Example:	
None	

feAppColorGetRGB

`feAppColorGetRGB` **`( nColor, pnRed, pnGreen, pnBlue )`**	
Description:	
Returns the red, green and blue components from a Femap color ID	
Input:	
`INT4 nColor`	Femap color ID.
Output:	
`INT4 pnRed`	Red component. Integer value 0-255.
`INT4 pnGreen`	Green component. Integer value 0-255.
`INT4 pnBlue`	Blue component. Integer value 0-255.
Return Code:	
None	
Remarks/Usage:	
Color IDs are a combination of colors and pattern / linestyle masks from the `zColor` enum. They can be created manually or obtained from `feAppColorPalette / feAppColorCreate`.	

Unpublished work. © 2025 Siemens

Pattern and linestyle masks have no effect on the returned RGB values.
Example:
None

feAppLoadColorPalette

feAppLoadColorPalette **(sPaletteFileName)**	
Description:	
Loads a color palette.	
Input:	
`STRING sPaletteFileName`	Full path to a Femap color palette file.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	Can not find or open the Palette File.
`FE_BAD_TYPE`	File is not a valid Palette File.
`FE_FAIL`	No color definitions found in the Palette File.
Remarks/Usage:	
None	
Example:	
None	

feAppLoadContourPalette

Unpublished work. © 2025 Siemens

feAppLoadContourPalette	
(bUpdateViewPalette, sPaletteFileName)	
Description:	
Loads a contour color palette.	
Input:	
`BOOL bUpdateViewPalette`	If True, then the Contour Palette will be loaded for any new View.
`STRING sPaletteFileName`	Full path to a Femap Contour Palette file.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	Can not find or open the Contour Palette File.
`FE_BAD_TYPE`	File is not a valid Contour Palette File.
`FE_FAIL`	No color definitions found in the Contour Palette File.
Remarks/Usage:	
The selected palette will be loaded and will be used for any new View that is created. It will not however update any existing Views. If `bUpdateViewPalette` is True, the User Contour Palette in the Active View will also be updated and if **All Views** is activated, all active views will be updated.	
Example:	
None	

feScreenPctPick

feScreenPctPick
(x, y)
Description:

Unpublished work. © 2025 Siemens

Displays a dialog box that allows picking a location in a view.	
Input:	
`REAL8 x`	The x (horizontal) and y (vertical) locations in the view. These values are returned as a percentage of the overall view size - i.e. 0.8 means 80% of the way from the top/left to the bottom/right of the view.
`REAL8 y`	
Output:	
None	
Return Code:	
`FE_CANCEL`	Dialog box was cancelled.
Remarks/Usage:	
None	
Example:	
None	

feOutputValueGetRGB

`feOutputValueGetRGB`
`( nViewID, dValue, pdRed, pdGreen, pdBlue )`
Description:
Returns the red, green and blue components from a Femap color ID of specific output value on a contour legend.
Input:

`INT4 nViewID`	The ID of view. If set to 0 the active view will be used.
`REAL8 dValue`	The value of the output desired to get color level.
Output:	
`INT4 pdRed`	Red component. Integer value 0-255.

Unpublished work. © 2025 Siemens

`INT4 pdGreen`	Green component. Integer value 0-255.
`INT4 pdBlue`	Blue component. Integer value 0-255.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

User Interface Methods

Free Edge and Free Face Methods

feElementFreeEdge

`feElementFreeEdge` **`( elemSetID, bParabolicEdges, bLineElem, nFreeCount, nFreeData )`**	
Description:	
Allows you to find the Free Edges in a set of elements.	
Input:	
`INT4 elemSetID`	The ID of a Set Object that contains the IDs of the elements to be used to compute the Free Edges. If 0, all elements in the model are used.
`BOOL bParabolicEdges`	If True, the computation of Free Edges includes checking the midside nodes (if any). If False, only corner nodes are considered.

Unpublished work. © 2025 Siemens

`BOOL bLineElem`	If True, then any line elements that lie along an edge are considered when deciding if the edge is *free*.
Output:	
`INT4 nFreeCount`	The number of Free Edges returned.
`INT4 nFreeData[0..3*nFreeCount-1]`	Data that defines the Free Edges. This is an array, stored as follows...`[element1, node1A, node1B, element2, node2A, node2B...]`. Each free edge is defined by 3 entries in the array - the ID of the element containing the free edge, followed by the two corner nodes on the element of the free edge.
Return Code:	
`FE_NO_MEMORY`	Could not allocate memory to return the Free Edge Data.
`FE_FAIL`	Could not compute the Free Edges for your selected elements.
Remarks/Usage:	
None	
Example:	
None	

feElementFreeFace

`feElementFreeFace`
`( elemSetID, bParabolicEdges, bPlaneElem, nFreeCount, nFreeData )`
Description:
Allows you to find the Free Faces in a set of elements.
Input:

`INT4 elemSetID`	The ID of a Set Object that contains the IDs of the elements to be used to compute the Free Faces. If 0, all elements in the model are used.

Unpublished work. © 2025 Siemens

`BOOL bParabolicEdges`	If True, the computation of Free Faces includes checking the midside nodes (if any). If False, only corner nodes are considered.
`BOOL bPlaneElem`	If True, then any selected plane elements are considered when deciding if the face is *free*.
Output:	
`INT4 nFreeCount`	The number of Free Faces returned.
`INT4 nFreeData[0..2*nFreeCount-1]`	Data that defines the Free Faces. This is an array, stored as follows... `[element1, face1, element2, face2, element3...]`. Each free face is defined by 2 entries in the array - the ID of the element containing the free face, followed by the face ID.
Return Code:	
`FE_NO_MEMORY`	Could not allocate memory to return the Free Face Data.
`FE_FAIL`	Could not compute the Free Faces for your selected elements.
Remarks/Usage:	
None	
Example:	
None	

feEdgesOfFreeFaces

`feEdgesOfFreeFaces` **`( elemSetID, bParabolicEdges, bPlaneElem, nFreeCount, nFreeData )`**
Description:
Allows you to find the Edges of Free Faces in a set of elements.
Input:

Unpublished work. © 2025 Siemens

`INT4 elemSetID`	The ID of a Set Object that contains the IDs of the elements to be used to compute the Free Faces. If 0, all elements in the model are used.
`BOOL bParabolicEdges`	If True, the computation of Free Faces includes checking the midside nodes (if any). If False, only corner nodes are considered.
`BOOL bPlaneElem`	If True, then any selected plane elements are considered when deciding if the face is “free”.
Output:	
`INT4 nFreeCount`	The number of Edges of Free Faces returned.
`INT4 nFreeData[0..2*nFreeCount-1]`	Data that defines the Edges of Free Faces. This is an array, stored as follows... `[nodeA_of_edge1, nodeB_of_edge1, nodeA_of_edge2, nodeB_of_edge2, nodeA_of_edge3, nodeB_of_edge3,...]`. Each edge of a free face is defined by 2 entries in the array - two corner nodes on the edge of each free face.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Could not allocate memory to return the Free Face Data.
`FE_FAIL`	Could not compute the Free Faces for your selected elements.
`FE_NOT_AVAILABLE`	No Free Faces found.
Remarks/Usage:	
None	
Example:	
None	

Model Activation and Management

When working with multiple models in Femap, it is important to carefully manage which model is active. All other API and program functionality works on the concept of a single active model. By switching the

Unpublished work. © 2025 Siemens

active model, you can work with multiple models simultaneously, however each time you call an API function that deals with model data, it will apply to the active model only. Using the methods in this section, you can find and choose this model.

feAppGetModel

feAppGetModel	
(model)	
Description:	
Retrieves the internal ID of the active model.	
Input:	
None	
Output:	
`INT8 model`	The ID of the active model.
Return Code:	
`FE_FAIL`	There is no active model.
Remarks/Usage:	
You can call this function after creating a New model, or Opening a model to retrieve its internal ID, so if you activate other models, you can reactivate the original by calling `feAppSetModel`.	
Example:	
None	

feAppGetModel32

feAppGetModel32
(nHiModelID, nLoModelID)
Description:

Unpublished work. © 2025 Siemens

Retrieves the internal ID of the active model, which is a 64-bit integer, as a pair of 32-bit integers.	
Input:	
None	
Output:	
`INT4 nHiModelID`	The first 4-bytes of the 64-bit integer which is the ID of the active model.
`INT4 nLoModelID`	The second 4-bytes of the 64-bit integer which is the ID of the active model.
Return Code:	
`FE_FAIL`	There is no active model.
Remarks/Usage:	
You can call this function after creating a New model, or Opening a model to retrieve its internal ID as a pair of 32-bit integers, so if you activate other models, you can reactivate the original by calling `feAppSetModel32` and using that pair of 32-bit integers.	
Example:	
None	

feAppGetModelName

`feAppGetModelName` **`( model, ModelName )`**	
Description:	
Retrieves the name of the selected model.	
Input:	
`INT8 model`	The ID of the model to retrieve the name.
Output:	
`STRING ModelName`	The name of the specified model.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	The specified model is not currently open.
Remarks/Usage:	
If the model has not yet been saved, it will not have a name, and the `ModelName` will be returned as *Untitled*. You can use this method in conjunction with `feAppGetAllModels( )` to retrieve the names of all open models.	
Example:	
None	

feAppGetModelByName

`feAppGetModelByName` **`( fName, nModelID )`**	
Description:	
Retrieves the ID of an open model using the model name or a portion of the model name with enough characters to identify only a single open model.	
Input:	
`STRING fName`	The name of the model to retrieve the ID.
Output:	
`INT8 model`	The ID of the open model retrieved by the model name.
Return Code:	
`FE_FAIL`	A single model ID to retrieve cannot be identified by the given model name or portion of model name.
Remarks/Usage:	
While it is possible to use only a portion of the model name to retrieve the model ID, enough of the title must be used to identify one and only model, otherwise the call will return `FE_FAIL`.	
Example:	
None	

Unpublished work. © 2025 Siemens

feAppSetModel

feAppSetModel	
(model)	
Description:	
Activates a model chosen by its internal ID.	
Input:	
`INT8 model`	The ID of the model that you want to activate.
Output:	
None	
Return Code:	
`FE_FAIL`	The model ID that you specified is not a valid, open model.
Remarks/Usage:	
You can find the ID of the active model using `feAppGetModel`, and that ID can then be passed to this method at a later time to reactivate the model.	
Example:	
None	

feAppSetModel32

feAppSetModel32
(nHiModelID, nLoModelID)
Description:
Activates a model chosen by a pair of 32-bit integers representing its internal ID, which is a 64-bit integer.
Input:

Unpublished work. © 2025 Siemens

`INT4 nHiModelID`	The first 4-bytes of the 64-bit integer which is the ID of the model that you want to activate.
`INT4 nLoModelID`	The second 4-bytes of the 64-bit integer which is the ID of the model that you want to activate.
Output:	
None	
Return Code:	
`FE_FAIL`	The model ID pair that you specified does not represent a valid, open model.
Remarks/Usage:	
You will need to find the `nHiModelID` and `nLoModelID` of the active model using `feAppGetModel32`, and those IDs can then be passed to this method at a later time to reactivate the model.	
Example:	
None	

feAppSetModelByName

`feAppSetModelByName`
`( fName )`
Description:
Activates a model chosen by specifying its filename.
Input:

`STRING fName`	The complete filename of the model that you want to activate.
Output:	
None	
Return Code:	
`FE_FAIL`	The model name that you specified is not a valid, open model.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

To use this method successfully, you must specify a complete filename, including the directory/path, and the *.MOD* extension. You can retrieve the full name of the active model from the `ModelName` parameter. This method can normally only be used for models that have been saved or opened from a saved model. When a new, unsaved model is created, the model name is not defined. You can activate an unsaved model by setting `fName` to an empty/blank (“”) string, however if you have multiple unsaved models, you will always activate the first one opened, and the others can not be accessed with this method.
Example:

To activate a saved model:
 	
`rc = femap.feFileSetActiveName("c:\models\xyz.mod")`
 	
To activate a new, unsaved model:
 	
`rc = femap.feFileSetActiveName( "" )`

feAppCountModels

`feAppCountModels` **`( numModels )`**	
Description:	
Finds the number of models that are currently opened.	
Input:	
None	
Output:	
`INT4 numModels`	The number of currently opened models. Could be 0 if no models are open.
Return Code:	
`FE_FAIL`	Unable to retrieve the number of active models.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

feAppGetAllModels

feAppGetAllModels **(numModels, model)**	
Description:	
Finds the number of models that are currently opened.	
Input:	
None	
Output:	
`INT4 numModels`	The number of currently opened models.
`INT8 model[0..numModels-1]`	The IDs of all of the currently opened models.
Return Code:	
`FE_FAIL`	There are either no models opened, or unable to retrieve the model IDs.
Remarks/Usage:	
This function will return the IDs of all currently opened models, which can then be used with `feAppSetModel` to activate any of those desired models.	
Example:	
None	

feAppModelContents

Unpublished work. © 2025 Siemens

feAppModelContents

(bList, nEmptyBlocks, nEntityBlocks)

Description:

Finds, and alternatively lists, the number of blocks of each entity type that are currently in the model.

Input:

`BOOL bList`	If True, a report of the contents is written to the Message Window.

Output:

`INT4 nEmptyBlocks`	The number of blocks which are allocated but unused because data has been deleted. These blocks will be reused as new data is added to the database.
`INT4 nEntityBlocks[0..max]`	An array of the number of blocks being used for each entity type. The index in this array is the Entity Type being reported. Common Entity types are define in **Entity Types**.

Return Code:

`FE_FAIL`	There are either no models opened, or unable to retrieve the model information.

Remarks/Usage:

The number of bytes in a Database block can change from version to version, however currently one block is 4096 bytes.

Example:

None

feAppModelDefragment

feAppModelDefragment

(void)

Description:

Unpublished work. © 2025 Siemens

Rearranges data in the model so that it is organized with similar data grouped together.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to defragment.
Remarks/Usage:	
This method will rearrange data in your model. Defragmenting should improve performance. Another way to accomplish this is to write a Neutral File and read it into a new model.	
Example:	
None	

Window Management

When you are running Femap via the API, especially if you are embedding it in your own applications, you may want to control which portions of the user interface are available. The methods in this section allow you to selectively turn various windows on and off.

feAppManageToolbars

`feAppManageToolbars` **`( barName, visible )`**	
Description:	
Makes one or more toolbars visible (or hidden).	
Input:	
`STRING barName`	The name of the toolbar to manage.
`BOOL visible`	If True, the toolbar is made visible, otherwise it is hidden.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the selected toolbar.
Remarks/Usage:	
The names of the toolbars can be found by running Femap manually and looking on the **Tools** → **Toolbars** menu. In addition to the toolbars listed there, you can also control the visibility of the Menu using this method - simply use the `barName` of "Menu Bar". If you want to set all possible toolbars in a single call, set barName to an empty string (""). **Note** This method is identical to the `feSetToolbarVisible` method.	
Example:	
None	

feAppManagePanes

`feAppManagePanes` **`( paneName, action )`**	
Description:	
Changes the state of one or more of the docking panes.	
Input:	
`STRING paneName`	The name of the pane to manage.
`INT4 action`	0=Hide, 1=Show/Visible, 2=Dock as a Tab, 3=Toggle Docking
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to find the selected pane.
`FE_INVALID`	Invalid action specified.
Remarks/Usage:	
The names of the panes can be found in the title bars of the panes. Currently they are **Entity Editor**, **Messages**, **Data Table**, **Model Info**, **Entity Info**, **Data Surface Editor**, **Meshing** Toolbox, **PostProcessing** Toolbox, **Charting**, **API Programming**, and **Program File**. If you have added your own application into Femap using `feAppRegisterAddInPane`, the title of your window will also be available. If you want to set all possible panes in a single call, set `paneName` to an empty string (“”). If you dock a pane as a tab, you can undock/float it by calling this method with **Toggle Docking**. If you then call this method again with **Toggle Docking**, the pane will be docked in its open state (not as a tab).	
Example:	
None	

feAppUpdatePanes

`feAppUpdatePanes` **`( fullRefresh )`**	
Description:	
Updates the contents of the docking panes.	
Input:	
`BOOL fullRefresh`	If True, the **Model Info** pane is fully rebuilt, otherwise incremental changes are added to all panes.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

If called with `fullRefresh=False`, then it must be called after each command, before redraws occur, otherwise some changes may be lost.
Example:
None

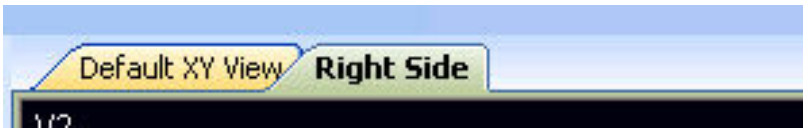
feAppManageStatusBar

`feAppManageStatusBar`	
`( visible )`	
Description:	
Sets the visibility of the status bar.	
Input:	
`BOOL visible`	If True, show the status bar, otherwise hide it.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the status bar.
Remarks/Usage:	
None	
Example:	
None	

feAppManageGraphicsTabs

Unpublished work. © 2025 Siemens

`feAppManageGraphicsTabs`
 	
`( visible )`	

Description:	
Sets the visibility of the tabs on the top of the graphics windows.	
Input:	
`BOOL visible`	If True, show the tabs, otherwise hide them.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the tabs.
Remarks/Usage:	
The tabs controlled by this method are shown here: 	
Example:	
None	

View Management

Sometimes you will need to know and control which views are available on the screen. The methods in this section allow you to manage this information.

feAppGetActiveView

Unpublished work. © 2025 Siemens

feAppGetActiveView	
(viewID)	
Description:	
Retrieves the ID of the active view in the active model.	
Input:	
None	
Output:	
`INT4 viewID`	The ID of the active view.
Return Code:	
`FE_FAIL`	There is no active view.
Remarks/Usage:	
This method allows you to retrieve the active view from the active model.	
Example:	
None	

feAppSetActiveView

feAppSetActiveView	
(viewID)	
Description:	
Activates a view in the active model chosen by its ID.	
Input:	
`INT4 viewID`	The ID of the view that you want to activate.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
This method allows you to activate any existing view in the active model. Use `feAppGetActiveView` and `feAppGetAllViews` to retrieve the ID's of views that can be activated.	
Example:	
None	

feAppGetAllViews

`feAppGetAllViews`	
`( numViews, viewID )`	
Description:	
Retrieves the ID's of all the visible views in the active model as well as the number of views that are visible.	
Input:	
None	
Output:	
`INT4 numViews`	The number of views visible in the active model.
`INT4 viewID[0..numViews-1]`	The IDs of all of the views visible in the active model.
Return Code:	
`FE_FAIL`	No views exist.
Remarks/Usage:	
This method allows you to return the ID's of all the views that are currently visible, which can then be activated with `feAppSetActiveView`.	
Example:	
None	

Unpublished work. © 2025 Siemens

Toolbar and Menu Customization

The methods in this section allow you to update the Femap menus and toolbars to include new commands that you define, to change the visiblity of commands that are already there, or to change title and icons associated with commands.

Toolbar and Menu Customization

The methods in this section allow you to update the Femap menus and toolbars to include new commands that you define, to change the visiblity of commands that are already there, or to change title and icons associated with commands.

feGetUserCommands

feGetUserCommands **(numCmd, sTitle, sCmd, sArg, sStartDir)**	
Description:	
Retrieves all user commands that have been defined.	
Input:	
None	
Output:	
`INT4 numCmd`	The number of user commands that are current defined, and the number of entries in the other arrays.
`STRING sTitle[0..numCmd-1]`	The title assigned to each command.
`STRING sCmd[0..numCmd-1]`	The command line used to execute the command.
`STRING sArg[0..numCmd-1]`	Any command line arguments assigned to the command.
`STRING sStartDir[0..numCmd-1]`	The starting directory, if any, for the command.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to retrieve the user commands.
`FE_NOT_EXIST`	No user commands are defined.
Remarks/Usage:	
None	
Example:	
None	

feAddUserCommand

`feAddUserCommand` **`( sTitle, sCmd, sArg, sStartDir )`**	
Description:	
Defines a user command that can then be used in the menu or toolbar.	
Input:	
`STRING sTitle`	The title assigned to the command. This title is used later to assign the command to the menu or toolbar. It must be unique when compared to all other user commands.
`STRING sCmd`	The command line used to execute the command.
`STRING sArg`	Any command line arguments assigned to the command.
`STRING sStartDir`	The starting directory, if any, for the command.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add the user commands.

Unpublished work. © 2025 Siemens

`FE_INVALID`	A user command with the same title has already been defined. You must specify a unique title.
Remarks/Usage:	
User commands defined with this method can be assigned to a toolbar using the `feAddToolbarUserCommand` method.	
Example:	
None	

AddUserCommand2

`AddUserCommand2` **`( strTitle, strCmd, strArg, bPIDMID, sStartDir )`**	
Description:	
Defines a user command that can then be used in the menu or toolbar.	
Input:	
`STRING strTitle`	The title assigned to the command. This title is used later to assign the command to the menu or toolbar. It must be unique when compared to all other user commands.
`STRING strCmd`	The command line used to execute the command.
`STRING strArg`	Any command line arguments assigned to the command.
`BOOL bPIDMID`	If True, the Add Process and Model ID Arguments option is turned on.
`STRING sStartDir`	The starting directory, if any, for the command.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add the user commands.

Unpublished work. © 2025 Siemens

`FE_INVALID`	A user command with the same title has already been defined. You must specify a unique title.
Remarks/Usage:	
User commands defined with this method can be assigned to a toolbar using the `feAddToolbarUserCommand` method.	
Example:	
None	

feDeleteUserCommand

`feDeleteUserCommand`	
`( sTitle )`	
Description:	
Deletes a user command.	
Input:	
`STRING sTitle`	The title of the command to be deleted.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to delete the user commands.
Remarks/Usage:	
None	
Example:	
None	

feGetToolbars

Unpublished work. © 2025 Siemens

feGetToolbars	
(numBars, barID, barVisible, barTitle)	
Description:	
Retrieves all toolbars that have been defined.	
Input:	
None	
Output:	
`INT4 numBars`	The number of toolbars that are current defined, and the number of entries in the other arrays.
`INT4 barID[0..numBars-1]`	The ID of each toolbar.
`BOOLEAN barVisible[0..numBars-1]`	A flag indicating whether the toolbar is currently visible. True=Visible.
`STRING barTitle[0..numBars-1]`	The title of the toolbar.
Return Code:	
`FE_FAIL`	Unable to retrieve the toolbar information.
Remarks/Usage:	
Toolbars do not need to be visible on the screen to be accessed by this method.	
Example:	
None	

feAddToolbar

feAddToolbar
(barName, barLocation)
Description:
Creates a new toolbar.

Unpublished work. © 2025 Siemens

Input:	
`STRING barName`	The title assigned to the toolbar. This title must be unique, and will be used in other methods to access the toolbar.
`INT4 barLocation`	The location where the bar will be docked. 0=Top, 1=Left, 2=Right, 3=Top, 4=Bottom
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add the toolbar.
`FE_INVALID`	A toolbar with the same title has already been defined. You must specify a unique title.
Remarks/Usage:	
After creating a toolbar with this method, you can add commands to it using either `feAddToolbarCommand` or `feAddToolbarUserCommand`.	
Example:	
None	

feAddToolbarSubmenu

`feAddToolbarSubmenu` **`( barName, cmdIndex, menuName, menuID )`**	
Description:	
Creates a new submenu in a toolbar.	
Input:	
`STRING barName`	The title assigned to the toolbar. This title must be unique, and will be used in other methods to access the toolbar.
`INT4 cmdIndex`	Location in `barName` to place the new menu. 0=Before first item, -1=After last item, N=After Nth item.

Unpublished work. © 2025 Siemens

`STRING menuName`	Title to be used for the menu drop-down.
Output:	
`INT4 menuID`	ID of the menu that was inserted - needed for `feAddToolbarSubmenuSubmenu`, `feAddToolbarSubmenuCommand` or `feAddToolbarSubmenuUserCommand` to add commands or submenus to this menu.
Return Code:	
`FE_FAIL`	Unable to add the menu.
`FE_NOT_EXIST`	The toolbar `barName` does not exist.
Remarks/Usage:	
After creating a toolbar with this method, you can add commands to it using either `feAddToolbarSubmenuCommand` or `feAddToolbarSubmenuUserCommand`, or submenus using `feAddToolbarSubmenuSubmenu`.	
Example:	
None	

feAddToolbarSubmenuSubmenu

`feAddToolbarSubmenuSubmenu`
`( menuID, cmdIndex, menuName, submenuID )`
Description:
Adds a new submenu to a submenu on a toolbar.
Input:

`INT4 menuID`	The ID of the menu where you want to place the command. This is returned by `feAddToolbarSubmenu`.
`INT4 cmdIndex`	The location in the menu for the submenu. `0=At` the front of the menu before any other buttons. `-1=At` the end of the menu, after all other

Unpublished work. © 2025 Siemens

	buttons. Any other positive value means after that button.
`STRING menuName`	Title to be used for the menu drop-down.
Output:	
`INT4 submenuID`	ID of the menu that was inserted - needed for `feAddToolbarSubmenuCommand` or `feAddToolbarSubmenuUserCommand` to add commands to this menu.
Return Code:	
`FE_FAIL`	Unable to add the submenu.
`FE_NOT_EXIST`	The menu does not exist.
Remarks/Usage:	
None	
Example:	
None	

feDeleteToolbar

`feDeleteToolbar`
`( barName )`
Description:
Deletes a toolbar.
Input:

`STRING barName`	The title of the toolbar to be deleted.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to delete the toolbar.
Remarks/Usage:	
None	
Example:	
None	

feResetToolbar

`feResetToolbar` **`( barName )`**	
Description:	
Resets a toolbar back to its original state.	
Input:	
`STRING barName`	The title of the toolbar to be reset. If this is an empty string (“”) then all toolbars will be reset.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to reset the toolbar.
Remarks/Usage:	
You can call this method to reset any changes you have made in the predefined Femap toolbars and menu back to their default configuration. This method will always display a dialog box asking the user to confirm resetting the toolbar.	
Example:	
None	

feSetToolbarVisible

Unpublished work. © 2025 Siemens

feSetToolbarVisible	
(barName, bVisible)	
Description:	
Makes one or more toolbars visible (or hidden).	
Input:	
`STRING barName`	The title of the toolbar to be updated. If this is an empty string (“”) then all toolbars will be reset.
`BOOLEAN bVisible`	If True, the toolbar will be made visible. If False, it will be hidden.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to reset the toolbar.
Remarks/Usage:	
The names of the toolbars can be found by running Femap manually and looking on the **Tools** → **Toolbars** menu. In addition to the toolbars listed there, you can also control the visibility of the Menu using this method - simply use the `barName` of Menu Bar. This method is identical to the `feAppManageToolbars` method.	
Example:	
None	

feGetMenuCommands

feGetMenuCommands
(menuName1, menuName2, numCmds, cmdID, cmdTitle)
Description:
Retrieves information about all commands on a selected menu.

Unpublished work. © 2025 Siemens

Input:	
`STRING menuName1`	The menu name you see at the top level (for example, “File”). Specify an empty string (“”) here to retrieve information about the commands at the top level (**File** → **Tools**, ...)
`STRING menuName2`	The menu name you see inside the top level menu (for example, “Import” if `menuName1`=”File”). Specify an empty string here to retrieve the commands on the drop down from the top level.
Output:	
`INT4 numCmds`	The number of commands on this menu, and the number of entries in the other arrays.
`INT4 cmdID[0..numCmds-1]`	The IDs of the commands associated with each menu item. The `cmdID` of any item on the menu that is a drop-down (rather than a command) will be -1.
`STRING cmdTitle[0..numCmds-1]`	The titles of the commands associated with each menu item.
Return Code:	
`FE_FAIL`	Unable to retrieve information about the specified menu - make sure you are specifying the name of a drop-down menu, and not an individual command.
Remarks/Usage:	
When specifying the menu names, you should always leave off the “...” characters and shortcut key specifications. Also do not include any special characters to represent the underlined letters - just specify the title as if nothing was underlined.	
Example:	

Retrieve the commands on the **View** → **Advanced Post** menu.
`rc = femap.feGetMenuCommands("View", "Advanced Post", n, i, t)`
Retrieve the commands on the **Tools** menu:
`rc = femap.feGetMenuCommands("Tools", "", n, i, t)`	

feGetMenuCommand

Unpublished work. © 2025 Siemens

feGetMenuCommand	
(menuName1, menuName2, menuName3, cmdID)	
Description:	
Retrieves the ID of a specified command.	
Input:	
`STRING menuName1`	The menu name you see at the top level (for example, “File”).
`STRING menuName2`	The menu name you see inside the top level menu (for example, “Save As” if `menuName1`=”File”). Specify an empty string here to retrieve a command on the top level of the menu.
`STRING menuName3`	The menu name you see inside the first level drop-down menu (for example, “Analysis Results” if `menuName1`=”File”, and `menuName2`=”Import”). Specify an empty string here to retrieve a command on the first level drop-down menu.
Output:	
`INT4 cmdID`	The ID of the command associated with the menu item. The `cmdID` of any item on the menu that is a drop-down (rather than a command) will be -1.
Return Code:	
`FE_FAIL`	Unable to retrieve information about the specified menu.
Remarks/Usage:	
When specifying the menu names, you should always leave off the “...” characters and shortcut key specifications. Also do not include any special characters to represent the underlined letters - just specify the title as if nothing was underlined.	
Example:	
None	

feGetToolbarCommands

Unpublished work. © 2025 Siemens

feGetToolbarCommands

(barName, numCmds, cmdID, cmdVisible, cmdSubmenu, cmdTitle)

Description:

Retrieves information about all commands on a selected toolbar.

Input:

`STRING barName`	The title of the toolbar containing the commands to be retrieved.

Output:

`INT4 numCmds`	The number of commands on this toolbar, and the number of entries in the other arrays.
`INT4 cmdID[0..numCmds-1]`	The IDs of the commands associated with each toolbar item.
`BOOLEAN cmdVisible[0..numCmds-1]`	A flag indicating whether or not the item is currently visible on the toolbar. True=Visible.
`BOOLEAN cmdSubmenu[0..numCmds-1]`	A flag indicating whether the item is a command button, or a button that drops down another menu. True=Drop-Down a Submenu.
`STRING cmdTitle[0..numCmds-1]`	The titles of the commands associated with each toolbar item.

Return Code:

`FE_FAIL`	Unable to retrieve information about the specified toolbar commands.
`FE_NOT_AVAILABLE`	No commands are available on the selected toolbar.
`FE_INVALID`	Unable to find the specified toolbar.

Remarks/Usage:

The value of `numCmds` will always be the number of all commands added to a toolbar, visible or hidden, + 1. The *extra* command is the **Toolbar Options** command which is available on every toolbar and will always be the final command in the returned arrays or variants. It always has a `cmdID` value of 9148 and a `cmdTitle` value of **Toolbar Options**.

Example:

None

Unpublished work. © 2025 Siemens

feGetToolbarSubCommands

feGetToolbarSubCommands **(cmdID, numCmds, cmdID, cmdVisible, cmdTitle)**	
Description:	
Retrieves information about all commands on a menu that drops-down from a toolbar button.	
Input:	
`INT4 cmdID`	The ID of the toolbar button that drops down the menu.
Output:	
`INT4 numCmds`	The number of commands on this menu, and the number of entries in the other arrays.
`INT4 cmdID[0..numCmds-1]`	The IDs of the commands associated with each menu item.
`BOOLEAN cmdVisible[0..numCmds-1]`	A flag indicating whether or not the item is currently visible on the menu. True=Visible.
`STRING cmdTitle[0..numCmds-1]`	The titles of the commands associated with each menu item.
Return Code:	
`FE_FAIL`	Unable to retrieve information about the specified menu commands.
`FE_NOT_AVAILABLE`	No commands are available on the selected menu.
`FE_INVALID`	Unable to find the specified toolbar.
Remarks/Usage:	
None	
Example:	
None	

feGetToolbarCommand

Unpublished work. © 2025 Siemens

feGetToolbarCommand	
(barName, cmdIndex, cmdID)	
Description:	
Finds the ID of the command associated with a toolbar button.	
Input:	
`STRING barName`	The title of the toolbar to be searched.
`INT4 cmdIndex`	The index of the button to retrieve. The first button on the toolbar is button 0, the second is 1, and so on.
Output:	
`INT4 cmdID`	The command ID associated with the button.
Return Code:	
`FE_FAIL`	Unable to find the toolbar.
`FE_NOT_EXIST`	The specified button does not exist on the toolbar.
Remarks/Usage:	
Setting the value of `cmdIndex` to the number of commands on the toolbar, visible or hidden, will always return **Toolbar Options** command which is available on every toolbar. It will always have a cmdID value of 9148. Therefore, if there are a total of 3 commands on the toolbar, setting cmdIndex to 3 will return the command ID of 9148 (**Toolbar Options**).	
Example:	
None	

feAddToolbarCommand

feAddToolbarCommand
(barName, cmdIndex, cmdID)
Description:

Unpublished work. © 2025 Siemens

Adds a new button to a toolbar and associates it with a standard command.	
Input:	
`STRING barName`	The title of the toolbar where the command button will be placed.
`INT4 cmdIndex`	The location on the toolbar for the button. `0=At` the front of the toolbar before any other buttons. `-1=At` the end of the toolbar, after all other buttons. Any other positive value means after that button.
`INT4 cmdID`	The command ID associated with the button.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add the command.
`FE_NOT_EXIST`	The toolbar does not exist.
Remarks/Usage:	
None	
Example:	
None	

feAddToolbarSubmenuCommand

`feAddToolbarSubmenuCommand`
`( menuID, cmdIndex, cmdID )`
Description:
Adds a new button to a submenu on a toolbar and associates it with a standard command.
Input:

Unpublished work. © 2025 Siemens

`INT4 menuID`	The ID of the menu where you want to place the command. This is returned by `feAddToolbarSubmenu`.
`INT4 cmdIndex`	The location in the menu for the button. 0=At the front of the menu before any other buttons. -1=At the end of the menu, after all other buttons. Any other positive value means after that button.
`INT4 cmdID`	The command ID associated with the button
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add the command.
`FE_NOT_EXIST`	The menu does not exist.
Remarks/Usage:	
None	
Example:	
None	

feAddToolbarUserCommand

`feAddToolbarUserCommand`
`( barName, cmdIndex, cmdTitle, cmdBitmap, cmdID )`
Description:
Adds a new button to a toolbar and associates it with a user command.
Input:

`STRING barName`	The title of the toolbar where the command button will be placed.
`INT4 cmdIndex`	The location on the toolbar for the button. `0=At` the front of the toolbar before any other buttons. `-1=At` the end of the toolbar, after all other

Unpublished work. © 2025 Siemens

	buttons. Any other positive value means after that button.
`STRING cmdTitle`	The title of the user command to associate with the button. This is the title given to the command in `feAddUserCommand`.
`STRING cmdBitmap`	The full pathname of a Windows Bitmap file that will be the icon associated with the button. Specify an empty string (“”) if you do not want the button to display as an icon - it will then display as text. Refer to `feSetToolbarCommandBitmap` for more information about defining the bitmap to be used for the icon.
Output:	
INT4 cmdID	The command ID assigned to the button
Return Code:	
`FE_FAIL`	Unable to add the command.
`FE_NOT_EXIST`	The toolbar does not exist.
`FE_NOT_AVAILABLE`	The specified user command has not been defined.
Remarks/Usage:	
None	
Example:	
None	

feAddToolbarSubmenuUserCommand

`feAddToolbarSubmenuUserCommand`
`( menuID, cmdIndex, cmdTitle, cmdBitmap, cmdID )`
Description:
Adds a new button to a submenu and associates it with a user command.
Input:

Unpublished work. © 2025 Siemens

`INT4 menuID`	The ID of the menu where the command button will be placed.
`INT4 cmdIndex`	The location on the menu for the button. `0=At` the front of the menu before any other buttons. `-1=At` the end of the menu, after all other buttons. Any other positive value means after that button.
`STRING cmdTitle`	The title of the user command to associate with the button. This is the title given to the command in `feAddUserCommand`.
`STRING cmdBitmap`	The full pathname of a Windows Bitmap file that will be the icon associated with the button. Specify an empty string (“”) if you do not want the button to display as an icon - it will then display as text. Refer to `feSetToolbarCommandBitmap` for more information about defining the bitmap to be used for the icon.
Output:	
`INT4 cmdID`	The command ID assigned to the button.
Return Code:	
`FE_FAIL`	Unable to add the command.
`FE_NOT_EXIST`	The menu does not exist.
`FE_NOT_AVAILABLE`	The specified user command has not been defined.
Remarks/Usage:	
None	
Example:	
None	

feSetToolbarCommandVisible

Unpublished work. © 2025 Siemens

feSetToolbarCommandVisible	
(barName, cmdID, bVisible)	
Description:	
Shows (or hides) a specified command on a toolbar or menu.	
Input:	
`STRING barName`	The title of the toolbar where the command button is found. Specify an empty string (“”) to show or hide the command on all toolbars and the menu.
`INT4 cmdID`	The ID of the command to show or hide.
`BOOLEAN bVisible`	True=Show the command, False=Hide.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the specified command.
Remarks/Usage:	
None	
Example:	
None	

feSetToolbarSeparator

feSetToolbarSeparator
(barName, cmdID, bAddSep)
Description:
Adds or removes a separator from before a command or menu on a toolbar.
Input:

Unpublished work. © 2025 Siemens

`STRING barName`	The title of the toolbar where the command button is found. Specify an empty string (“”) to show or hide the command on all toolbars and the menu.
`INT4 cmdID`	The ID of the command or menu to modify the separator.
`BOOLEAN bAddSep`	True=Add a separator before the command/menu, False=Remove separator.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the specified command.
Remarks/Usage:	
None	
Example:	
None	

feSetToolbarCommandTitle

`feSetToolbarCommandTitle`
`( barName, cmdID, cmdTitle, cmdToolTip, cmdDescription )`
Description:
Sets the title, tooltip and description of a command on the menu or toolbars.
Input:

`STRING barName`	The title of the toolbar where the command button is found. Specify an empty string (“”) to set the titles for the command on all toolbars and the menu.
`INT4 cmdID`	The ID of the command to update.
`STRING cmdTitle`	Sets the title of the command. The title is the text shown on the toolbar or menu. Specify an

Unpublished work. © 2025 Siemens

	empty string (“”) if you do not want to update the command title.
`STRING cmdToolTip`	Sets the tooltip for the command. The tooltip is the text shown in the popup when you leave your mouse pointing at the command. Specify an empty string (“”) if you do not want to update the command tooltip.
`STRING cmdDescription`	Sets the description for the command. The description is the one-line help text shown in the status bar when you point your mouse at the command. Specify an empty string (“”) if you do not want to update the command description.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the specified command.
Remarks/Usage:	
None	
Example:	
None	

feSetToolbarCommandBitmap

`feSetToolbarCommandBitmap` **`( cmdID, cmdBitmap )`**	
Description:	
Sets the icon for a command on the menu or toolbar.	
Input:	
`INT4 cmdID`	The ID of the command to update.
`STRING cmdBitmap`	The full pathname to a Windows Bitmap file that is used for the command icon.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	Unable to find the specified command.
`FE_NOT_AVAILABLE`	Unable to access the bitmap file specified.
Remarks/Usage:	
The Bitmap file that you specify must be created following very specific guidelines, or this method will not work properly - possibly resulting in Femap crashing. The bitmap file must contain a single 16x16 pixel image. Other sizes are not supported. If you want part of the bitmap to be transparent, set those pixels to RGB=(192,192,192) - light gray. This means that you can not use this color in your image - it will be replaced by the appropriate background color.	
Example:	
None	

feSaveToolbarLayout

`feSaveToolbarLayout` **`( fName )`**	
Description:	
Saves the current toolbar and menu definition and layout to a file for later use.	
Input:	
`STRING fName`	The full pathname of the file to create (normally with a *.tbr* extension)
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to save the layout.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

feLoadToolbarLayout

`feLoadToolbarLayout` **`( fName )`**	
Description:	
Loads a toolbar and menu definition and layout from a file.	
Input:	
`STRING fName`	The full pathname of the file to load, that was either created with `feSaveToolbarLayout` or through the **File** → **Preferences** command.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to load the layout.
Remarks/Usage:	
None	
Example:	
None	

Embedding Applications

The methods in this section allow you to embed your application in the Femap user interface as an Add-In, or to embed Femap into the user interface of your application.

Unpublished work. © 2025 Siemens

feAppRegisterAddIn

`feAppRegisterAddIn`

`( register, windowID, messageID, tile, location )`

Description:

Registers an add-in application with Femap.

Input:

`BOOL register`	Set to True to register a new application, or False to unregister an add-in that is about to close.
`INT4 windowID`	The handle to the add-in application main window/ form.
`INT4 messageID`	The handle to the add-in application window that is to receive event messages. This must be a window that has a message loop.
`BOOL tile`	Not used
`INT4 location`	Not used

Output:

None

Return Code:

None

Remarks/Usage:

Usually you will want to specify the same handle for `windowID` and `messageID` - unless you want some other window than your main window to receive Femap event messages.

This method is obsolete, and is included only for compatibility with existing applications. Any new applications should use `feAppRegisterAddInPane`, and old applications should be converted as soon as possible.

Example:

None

Unpublished work. © 2025 Siemens

feAppRegisterAddInPane

feAppRegisterAddInPane

(register, windowID, messageID, bPreventClose, bPreventHide, location, neighborID)

Description:

Registers an add-in application with Femap.

Input:

`BOOL register`	Set to True to register a new application, or False to unregister an add-in that is about to close.
`INT4 windowID`	The handle to the add-in application main window/ form.
`INT4 messageID`	The handle to the add-in application window that is to receive event messages. This must be a window that has a message loop.
`BOOL bPreventClose`	If True, the pane will not include the “X” in the title bar to allow users to close the addin. If you set this option, you must provide some other method to allow users to close your application.
`BOOL bPreventHide`	If True, the pane will not include the “pin” in the title bar to allow the window to be collapsed to a tab.
`INT4 location`	0 - floating window, 1 - dock left, 2 - dock right, 3 - dock top, 4 - dock bottom
`INT4 neighborID`	0-femap main window, 1-Message Pane, 2-Model Info Tree Pane, 3-Entity Editor Pane, 4-Data Table Pane, otherwise the handle to an existing add-in window.

Output:

None

Return Code:

None

Remarks/Usage:

Unpublished work. © 2025 Siemens

Usually you will want to specify the same handle for `windowID` and `messageID` - unless you want some other window than your main window to receive Femap event messages. The window identified by `neighborID` is used in conjunction with the location you specify to determine where the window is docked and how it is sized. For example if you specify `location=3` and `neighborID=1`, then your window will be placed at the top of the message pane. If your add-in contains a menu, you may need to use `feAppRegisterAddInPaneWithFrame` to get the menu to appear.
Example:
None

feAppRegisterAddInPaneWithFrame

`feAppRegisterAddInPaneWithFrame`

`( register, windowID, messageID, bPreventClose, bPreventHide, location, neighborID )`

Description:	
Registers an add-in application with Femap.	
Input:	
`BOOL register`	Set to True to register a new application, or False to unregister an add-in that is about to close.
`INT4 windowID`	The handle to the add-in application main window/ form.
`INT4 messageID`	The handle to the add-in application window that is to receive event messages. This must be a window that has a message loop.
`BOOL bPreventClose`	If True, the pane will not include the “X” in the title bar to allow users to close the addin. If you set this option, you must provide some other method to allow users to close your application.
`BOOL bPreventHide`	If True, the pane will not include the “pin” in the title bar to allow the window to be collapsed to a tab.

Unpublished work. © 2025 Siemens

`INT4 location`	0 - floating window, 1 - dock left, 2 - dock right, 3 - dock top, 4 - dock bottom
`INT4 neighborID`	0-femap main window, 1-Message Pane, 2-Model Info Tree Pane, 3-Entity Editor Pane, 4-Data Table Pane, otherwise the handle to an existing add-in window.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method is essentially the same as `feAppRegisterAddInPane`, however it does not modify the window style of the window being added. This has both positive and negative effects. On the positive side, it allows some programs to be added that contain menus that disappear if you used `feAppRegisterAddInPane`. However, it also leaves window borders and forces you to do more in your add-in application to make sure sizes do not change and that your windows do not accidentally move or close. If you allow this and do not fully manage it, you can end up with a pane that contains only part of your application, or nothing. So, unless you absolutely need to keep menus or frames, use `feAppRegisterAddInPane` instead.	
Example:	
None	

feAppEmbed

`feAppEmbed` **`( windowID, x, y, cx, cy )`**	
Description:	
Embeds the Femap window(s) into another application.	
Input:	
`INT4 windowID`	The handle to your application main window/form, or any other window where you want the Femap window displayed. Inputting 0 will unembed the Femap application.

Unpublished work. © 2025 Siemens

`INT4 x`	The coordinates of the upper left corner of the embedded window(s). These are specified in the client coordinates of the parent window specified by `windowID`.
`INT4 y`	
`INT4 cx`	The horizontal and vertical size of the embedded window(s).
`INT4 cy`	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method is normally used with Femap in a non-visible state. In that case, Femap becomes visible, and is embedded (parented) into the window you specify - normally the main window of your application. You may call this method multiple times to update the window where the windows are embedded, or to change the embedded region. Refer to **Embedding Femap in your application** for more information.	
Example:	
None	

feAppBringToTop

`feAppBringToTop`	
`( toTop, underWindow )`	
Description:	
Allows you to position the Femap window on top of, or under other windows.	
Input:	
`BOOL toTop`	If True, the window is moved forward in the window list. If False, it is moved to the back of the window list, under all other windows.
`INT4 underWindow`	Only used if `toTop=True`. If you want to move the window on top of all other windows, set this to 0. If you want to move it forward, but leave it behind

Unpublished work. © 2025 Siemens

	another window, specify that window handle in this field.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feAppPosition

feAppPosition **(move, size, x, y, cx, cy)**	
Description:	
Allows you to move and size the Femap window.	
Input:	
`BOOL move`	If True, the upper left corner of the window is moved to the new location specified by `x,y`.
`BOOL size`	If True, the window is resized to the size specified by `cx,cy`.
`REAL8 x`	The horizontal location of the upper left corner of the window. Specified as a percent of the total screen size.
`REAL8 y`	The vertical location of the upper left corner of the window. Specified as a percent of the total screen size.
`REAL8 cx`	The horizontal size of the window. Specified as a percent of the total screen size.

Unpublished work. © 2025 Siemens

`REAL8 cy`	The vertical size of the window. Specified as a percent of the total screen size.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feAppShowWindow

`feAppShowWindow`
`( maximize )`
Description:
Allows you to minimize and maximize the Femap window.
Input:

`BOOL maximize`	If True, the window will be maximized. If False, it will be minimized to an icon.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

feAppVisible

feAppVisible	
(visible)	
Description:	
Makes a Femap session that was started with `CreateObject` visible (or invisible).	
Input:	
`BOOL visible`	If True, the session becomes visible, if False, invisible.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method is required to make Femap visible if you started a new session with `CreateObject`. You can also make an existing session invisible with the appropriate call. You should not call this method with a session where you have embedded graphics or messages windows into your application.	
Example:	
None	

feAppRegisterMessageHandler

feAppRegisterMessageHandler
(register, windowID)
Description:
Registers your application to receive all of the text that is sent to the Femap Messages window.
Input:

Unpublished work. © 2025 Siemens

`BOOL register`	If True, your application will start receiving messages as they are written to the window. If False, messages will no longer be sent.
`INT4 windowID`	The handle your application's main window/form, or any other window that you want to receive the message text.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method allows you to capture the Femap messages and process them in any way you would like. You can display or modify them in any manner you would like. Refer to **Capturing the Femap Messages** for more information.	
Example:	
None	

feAppEventCallback

`feAppEventCallback` **`( eventID, callback )`**	
Description:	
Registers a program to be called when a particular event occurs.	
Input:	
`INT4 eventID`	The ID of the event to watch. Refer to **The Femap Events** for possible event values.
`STRING callback`	A full path to a Femap Program file, Basic program, or any executable that can be run from the operating system command line. Specify callback as a blank (zero length) string to *unregister* your callback.

Unpublished work. © 2025 Siemens

`STRING ParentFileName[260]`	A full directory path and file name of parent Geometry, Analysis Model, or Analysis Results file (Maximum 260 Characters).
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	You tried to unregister the callback for an event which was not registered.
`FE_NO_MEMORY`	Unable to register callback.
Remarks/Usage:	
This method allows you to execute a particular task when an event occurs without having to implement complicated event handling code. From the time you register the callback, until you unregister it by calling with a blank string, your code will be called every time the event occurs. Only one callback can be registered per event. You can simply call this method with a different file for the same event to switch callbacks. If the event that your callback processes uses the `lParam` value (like `FEVENT_COMMAND`), you can access it from your callback routine by using `EventLParam` described in **Global Integer Properties**	
Example:	
None	

feAppRunningApplicationInfo

`feAppRunningApplicationInfo` **`( appName, processIDs, activeWindows, nint )`**	
Description:	
Returns information about running applications to use for API attachment.	
Input:	
`STRING appName`	Set this equal to “Femap”.
Output:	

Unpublished work. © 2025 Siemens

`INT4 processIDs[0..n-1]`	The Process IDs of each running application specified by appName. Where n is the total number of active windows. There may be duplicates in the array if an application has more than 1 active window. See the description for `activeWindows`.
`INT64 activeWindows[0..n-1]`	Returns an array of the open Model IDs. (i.e. 2077739520, 30213888, etc.). For example, If there are 2 running Femaps, each with 3 open models, n will equal 6.
`INT4 nInt[0..n-1]`	Returns an array of the version number of the running windows.
Return Code:	
`FE_FAIL`	No application found from the string in `appName`.
Remarks/Usage:	
This method is intended return information of the running application to be used by `feAppGetRunningApplication`. These methods overcome a pitfall of the Microsoft function `GetObject` that limits an external API program to only attach to the 1st running instance of an application. Reference Microsoft documentation about the Running Object Table for more information.	
Example:	

```
Sub Main

   Dim App as femap.model

   Set App = feFemap()


   App.feAppMessage(FCM_NORMAL, "First Femap")


   Dim vProcessIDs As Variant

   Dim vModelIDs As Variant

   Dim vVersion As Variant


   App.feAppRunningApplicationInfo("Femap", vProcessIDs, vModelIDs,
vVersion )
```


Unpublished work. © 2025 Siemens

```
   App.feAppGetRunningApplication("Femap", vProcessIDs( 1 ),
vModelIDs( 1 ), appObj )


   Dim App2 As femap.model

   Set App2 = appObj


   App2.feAppMessage(FCM_NORMAL,"Second Femap" )


End Sub
```

feAppGetRunningApplication

feAppGetRunningApplication

(appName, processID, activeWindow, appObj)

Description:

Returns the top-level COM Application Object.

Input:

`STRING appName`	Set this equal to “Femap”.
`INT4 processID`	The Process ID of the running Application.
`VARIANT activeWindow`	The open Model ID. Can also be an empty Variant.

Output:

`VARIANT appObject`	The top-level Application Object. Also known as the `IDispatch` pointer.

Return Code:

`FE_FAIL`	No application found from the string in `appName`.

Remarks/Usage:

Unpublished work. © 2025 Siemens

This method pairs with `feAppRunningApplicationInfo`.
Example:
See the example code for `feAppRunningApplicationInfo`.

Locking Femap

The methods in this section allow you to prevent certain actions in Femap when they are not appropriate due to the state of your application.

feAppLock

`feAppLock`	
`( void )`	
Description:	
Locks the Femap user interface.	
Input:	
None	
Output:	
None	
Return Code:	
`INT4 lock_count`	The lock count after this call. Will be nonzero if the interface was locked.
Remarks/Usage:	
This method locks the user interface so that the user cannot pick any commands from the Femap menus or toolbars. Some aspects of the Femap user interface (such as docking panes) will still remain active - there is no way to deactivate those, however they can be hidden. Add-in applications are active. When Femap is locked, the cursor is changed to be an hourglass with a lock. You must call `feAppUnlock` before Femap will again be active.	
Example:	
None	

Unpublished work. © 2025 Siemens

feAppUnlock

feAppUnlock	
(void)	
Description:	
Unlocks the Femap user interface.	
Input:	
None	
Output:	
None	
Return Code:	
`INT4 lock_count`	The number of locks remaining after this call. Will be 0 if the user interface was actually unlocked.
Remarks/Usage:	
This method unlocks the user interface, after a call to `feAppLock`, so that the user can again use the Femap commands. It changes the cursor back to the standard Femap cursors. Actually, `feAppLock` increases a lock count, so if it has been called multiple times, then `feAppUnlock` must be called an equal number of times before the user interface will be unlocked. There is no harm in calling `feAppUnlock` extra times, or even if `feAppLock` was never called.	
Example:	
None	

feAppLockExit

feAppLockExit
(void)
Description:
Locks the Femap session so that a user can not exit.

Femap API Guide
Unpublished work. © 2025 Siemens

Input:	
None	
Output:	
None	
Return Code:	
`INT4 lock_count`	The lock count after this call. Will be nonzero if the exit was locked.
Remarks/Usage:	
This method locks the session so that the user cannot exit using the **File → Exit** command, or any of the normal system application close methods. This does not prevent the Femap process from being accidentally or intentionally terminated using Task Manager or other approaches.	
Example:	
None	

feAppUnlockExit

`feAppUnlockExit`
`( void )`
Description:
Unlocks the Femap session so that the user can exit.
Input:
None
Output:
None
Return Code:
`INT4 lock_count` — The number of locks remaining after this call. Will be 0 if the session was actually unlocked.
Remarks/Usage:
This method unlocks the session, after a call to `feAppLockExit`, so that the user can again exit Femap. Actually, `feAppLockExit` increases a lock count, so if it has been called multiple times, then

Unpublished work. © 2025 Siemens

`feAppUnlockExit` must be called an equal number of times before exit will be unlocked. There is no harm in calling `feAppUnlockExit` extra times, or even if `feAppLockExit` was never called.
Example:
None

feAppLockModel

`feAppLockModel`	
`( void )`	
Description:	
Locks the Femap session so that a user can not start or open a different model.	
Input:	
None	
Output:	
None	
Return Code:	
`INT4 lock_count`	The lock count after this call. Will be nonzero if the exit was locked.
Remarks/Usage:	
This method locks the session so that the user cannot change to a new model using either the **File** → **New** or **File** → **Open** commands.	
Example:	
None	

feAppUnlockModel

Femap API Guide
Unpublished work. © 2025 Siemens

feAppUnlockModel

(void)

Description:

Unlocks the Femap session so that the user can start or open a new model.

Input:

None	

Output:

None	

Return Code:

`INT4 lock_count`	The number of locks remaining after this call. Will be 0 if the session was actually unlocked.

Remarks/Usage:

This method unlocks the session, after a call to `feAppLockModel`, so that the user can again start or open a different model. Actually, `feAppLockModel` increases a lock count, so if it has been called multiple times, then `feAppUnlockModel` must be called an equal number of times before the model will be unlocked. There is no harm in calling `feAppUnlockModel` extra times, or even if `feAppLockModel` was never called.

Example:

None

feAppLockDraw

feAppLockDraw

(void)

Description:

Locks the Femap session so that redraws of the graphics windows do not occur.

Input:

None	

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`INT4 lock_count`	The lock count after this call. Will be nonzero if the exit was locked.
Remarks/Usage:	
This method locks the session so that redraws will not occur - even if you explicitly call API functions that cause redraws. Calling this function can improve the performance of other API functions since the on-screen graphics do not need to be updated, however when you are done with your processing, you must call `feAppUnlockDraw`, and then call `feViewRedraw` or `feViewRegenerate`.	
Example:	
None	

feAppUnlockDraw

`feAppUnlockDraw` **`( void )`**	
Description:	
Unlocks the Femap session so that redraws can occur in the graphics window.	
Input:	
None	
Output:	
None	
Return Code:	
`INT4 lock_count`	The number of locks remaining after this call. Will be 0 if the session was actually unlocked.
Remarks/Usage:	
This method unlocks drawing, after a call to `feAppLockDraw`, so that redraws can again be done. Actually, `feAppLockDraw` increases a lock count, so if it has been called multiple times, then `feAppUnlockDraw` must be called an equal number of times before drawing will be unlocked. There is no harm in calling `feAppUnlockDraw` extra times, or even if `feAppLockDraw` was never called.	

Unpublished work. © 2025 Siemens

Example:
None

Status Bar

The methods described here allow you to control the functionality of the Status Bar. In addition to these functions, you can set the visibility of the status bar using **feAppManageStatusBar**.

It's important to use the Status Bar sparingly. While frequent updates may provide the user more feedback with regards to the status of your process, too frequent updates may have an adverse impact on the performance of your API program.

feAppStatusRedraw

```
feAppStatusRedraw

( void )
```

Description:	
Redraws the Status bar.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Simply redraws the Status bar to reflect any changes that have taken place - for example creation or deletion of nodes or elements.	
Example:	
None	

Unpublished work. © 2025 Siemens

feAppStatusShow

feAppStatusShow **(enable, max_status)**	
Description:	
Enables or disables the status tracker inside the Status bar.	
Input:	
`BOOL enable`	Set to True to enable/show the status tracker. Set to False to remove it.
`INT4 max_status`	The value that you will use as 100% complete when sending `feAppStatusUpdate` commands.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
You must call `feAppStatusShow( False, ...)` after you are done with the status tracker to remove it from the display.	
Example:	
None	

feAppStatusUpdate

feAppStatusUpdate **(current_status)**
Description:
Updates the status tracker to the specified level of completeness.
Input:

Unpublished work. © 2025 Siemens

`INT4 current_status`	The current completion value. This should be a number that is between 0, and the value you specified as *max_status* in the `feAppStatusShow` method. When you specify `current_status=max_status`, the status tracker will be drawn as fully complete.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Although it is normal to send increasing values of `current_status`, starting at 0, and proceeding toward `max_status`, there is no requirement to do so. `current_status` values can be sent in any order.	
Example:	
None	

Message Window

The methods described here allow you to access functionality of the Message Window. In addition to these methods, you can control the visibility of the Message window using feAppManagePanes.

feAppMessage

`feAppMessage`	
`( color, message )`	
Description:	
Writes a message to the Messages and Lists window.	
Input:	
`INT4 color`	The color/status of the message to write: 0=Normal, 1=Highlight, 2=Warning, 3=Error.

Unpublished work. © 2025 Siemens

`STRING message`	The text of the message to write. Must be less than 160 characters.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This message is only written to the Message Window, regardless of the location (or locations) that is currently specified in the listing destination.	
Example:	
None	

feAppMessageStartListing

`feAppMessageStartListing`	
`( )`	
Description:	
Begins a listing to the active listing destinations.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to begin listing - no need to call `feAppMessageEndListing()`.
Remarks/Usage:	
Calling this function initializes **Listing Mode** in the Message window. This means that text will appear in the Listing font - typically fixed pitch. It also means that the text will be written to whatever destinations are selected - Message Window, File and/or Printer. Finally it means that text written to the Message Window will not appear until you call `feAppMessageEndListing()`. Between these two calls redrawing of the Message Window is disabled to improve the speed of writing large blocks of text.	

Unpublished work. © 2025 Siemens

Note

You MUST call `feAppMessageEndListing()` after your last call to `feAppMessage()`, so that the Message Window display is turned back on. If you are using the Printer as a destination, all text that is written between your call to `feAppMessageStartListing()` and `feAppMessageEndListing()` is sent to the printer as a single job when you call `feAppMessageEndListing()`.

Example:

This example writes all nodal coordinates in a listing.

```
Sub Main
  Dim App As femap.model
  Set App = feFemap()
  Dim n As femap.Node
  Set n = App.feNode

  App.feAppMessageStartListing()
  App.feAppMessage( FCM_NORMAL, "Nodal Coordinates" )
  While n.Next()
     App.feAppMessage( FCM_NORMAL, Str$(n.ID) + "  xyz=" + _
         Str$(n.x) + "  " + Str$(n.y) + "  " + Str$(n.z) )
   Wend

   App.feAppMessageEndListing()
End Sub
```

feAppMessageEndListing

feAppMessageEndListing

( )

Description:

Ends a listing that was started by `feAppMessageStartListing`.

Input:

None	

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

None	

Remarks/Usage:

> **Note**
>
> You MUST call `feAppMessageEndListing()` after your last call to `feAppMessage()`, so that the Message Window display is turned back on. If you are using the Printer as a destination, all text that is written between your call to `feAppMessageStartListing()` and `feAppMessageEndListing()` is sent to the printer as a single job when you call `feAppMessageEndListing()`.

Example:

None

feAppMessageClear

`feAppMessageClear`

`( void )`

Description:

Removes all items currently in the Messages and Lists window.

Input:

None	

Output:

None	

Return Code:

None	

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

Model Info Tree

The methods described here allow you to access functionality of the Model Info Tree. In addition to these methods, you can control the visibility of the Model Info Tree using **feAppManagePanes**

feAppModelInfoShow

`feAppModelInfoShow`
 	
`( void )`	

Description:	
Returns the current state of the **Show When Selected** menu in the Model Info Tree dockable pane.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Show When Selected is currently set to Highlight, Transparent Highlight, or Show Selected Only option.
`FE_FAIL`	Show When Selected is currently set to Off.
Remarks/Usage:	
None	
Example:	
None	

feAppSetModelInfoShow

Unpublished work. © 2025 Siemens

feAppSetModelInfoShow	
(bEnable)	
Description:	
Sets the **Show Mode** of the **Show When Selected** menu in the Model Info Tree dockable pane.	
Input:	
`BOOL bEnable`	IIf True, sets **Show When Selected** menu to Highlight, Transparent Highlight, or Show Selected Only, based on the setting in the current model for **Window → Show Entities**. If False, sets **Show When Selected** menu to Off.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
To ensure a particular **Show Mode** will be used, be sure to set some combination of the available `Info_ViewShow...` options before calling this method.	
Example:	
None	

feElementShapeCount

feElementShapeCount	
(enShape, nCount)	
Description:	
Returns the number of elements of the specified topology currently in the model.	
Input:	
`INT4 or enum enShape`	Element Topology (Shape). Use enum `zTopologyType`. The shape of the element:

Unpublished work. © 2025 Siemens

	0=Line2 (FTO_LINE2) 1=Line3 (FTO_LINE3) 2=Tri3 (FTO_TRIA3) 3=Tri6 (FTO_TRIA6) 4=Quad4 (FTO_QUAD4) 5=Quad8 (FTO_QUAD8) 6=Tetra4 (FTO_TETRA4)(7=Wedge6 (FTO_WEDGE6) 8=Brick8 (FTO_BRICK8) 9=Point (FTO_POINT1) 10=Tetra10 (FTO_TETRA10) 11=Wedge15 (FTO_WEDGE15) 12=Brick20 (FTO_BRICK20) 13=Rigid (FTO_RIGIDLIST) 14= Pyramid5 (FTO_PYRAMID5) 15=MultiList (FTO_MULTILIST2) 16=Contact (FTO_CONTACT2) - Do not use 17=Weld (FTO_WELD2) 18=Rigid (FTO_RIGIDLIST2) - RBE1 elements only 19=Pyramid13 (FTO_PYRAMID13)
Output:	
`INT4 nCount`	Number of elements matching the specified topology.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feElementTypeCount

`feElementTypeCount`
`( enType, nCount )`
Description:

Unpublished work. © 2025 Siemens

Returns the number of elements of the specified type currently in the model.

Input:

`INT4 or enum enType`	Element Type. Use enum `zElementType`. The type of the element: 0=None (FET_NONE), 1=Rod (FET_L_ROD), 2=Bar (FET_L_BAR), 3=Tube (FET_L_TUBE), 4=Link (FET_L_LINK) 5= Linear Beam (FET_L_BEAM) 6=Spring/Damper (FET_L_SPRING) 7=DOF Spring (FET_L_DOF_SPRING) 8=Curved Beam (FET_L_CURVED_BEAM) 9=Gap (FET_L_GAP), 10=Plot Only Line (FET_L_PLOT) 11=Linear Shear Panel (FET_L_SHEAR) 12=Parabolic Shear Panel (FET_P_SHEAR) 13=Linear Membrane (FET_L_MEMBRANE) 14=Parabolic Membrane (FET_P_MEMBRANE) 15=Linear Bending Only (FET_L_BENDING) 16=Parabolic Bending Only (FET_P_BENDING) 17=Linear Plate (FET_L_PLATE), 18=Parabolic Plate (FET_P_PLATE) 19=Linear Plane Strain (FET_L_PLANE_STRAIN) 20=Parabolic Plane Strain (FET_P_PLANE_STRAIN) 21=Linear Laminate (FET_L_LAMINATE_PLATE) 22=Parabolic Laminate (FET_P_LAMINATE_PLATE) 23=Linear Axisymmetric (FET_L_AXISYM) 24=Parabolic Axisymmetric (FET_P_AXISYM) 25=Linear Solid (FET_L_SOLID), 26=Parabolic Solid (FET_P_SOLID) 27=Mass (FET_L_MASS), 28=Mass Matrix (FET_L_MASS_MATRIX) 29=Rigid (FET_L_RIGID) 30=General Matrix (FET_L_GENERAL_MATRIX) 31=Curved Tube (FET_L_CURVED_TUBE) 32=Plot Only Plate (FET_L_PLOT_PLATE) 33=SlideLine (FET_L_SLIDE_LINE), 34=Contact (do not use) 35=Linear Axisymmetric Shell (FET_L_AXISYM_SHELL) 36=Parabolic Axisymmetric Shell (FET_P_AXISYM_SHELL)

Unpublished work. © 2025 Siemens

	37=Parabolic Beam (FET_P_BEAM) 38=Weld/ Fastener (FET_L_WELD) 39=Linear Solid Laminate (FET_L_LAMINATE_SOLID) 40=Parabolic Solid Laminate (FET_P_LAMINATE_SOLID) 41=Spring/Damper to Ground (FET_L_SPRING_TO_GROUND) 42=DOF Spring to Ground (FET_L_DOF_SPRING_TO_GROUND) 43=Linear Cohesive (FET_L_COHESIVE_SOLID) 44=Parabolic Cohesive (FET_P_COHESIVE_SOLID) 45=Nastran General Matrix (FET_L_NASTRAN_MATRIX)
Output:	
`INT4 nCount`	Number of elements matching the specified type.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Real Number Text Formatting

The methods described here allow you to access functionality used in Femap to format real numbers as text.

feFormatReal

Unpublished work. © 2025 Siemens

feFormatReal	
(dValue, nWidth, nSigFig, nFormat, strValue)	
Description:	
Formats a real number into a text string using the options used internally by Femap.	
Input:	
`REAL8 dValue`	The real value to format.
`INT4 nWidth`	The overall width of the text string to produce.
`INT4 nSigFig`	The number of significant figures to attempt to format. Must be able to fit within the specified width.
`INT4 nFormat`	The formatting method to use 0=Normal, 1=Force Exponential, 2=Nastran.
Output:	
`CHAR strValue`	The text string containing the formatted real number.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feTruncateReal

feTruncateReal
(dValue, nWidth, nSigFig, nFormat, truncValue)
Description:

Unpublished work. © 2025 Siemens

Truncates a real number to the precision of a formatted text string.	
Input:	
`REAL8 dValue`	The real value to format.
`INT4 nWidth`	The overall width of the text string to produce.
`INT4 nSigFig`	The number of significant figures to attempt to format. Must be able to fit within the specified width.
`INT4 nFormat`	The formatting method to use 0=Normal, 1=Force Exponential, 2=Nastran.
Output:	
`REAL8 truncValue`	The precision truncated version of the formatted real number.
Return Code:	
None	
Remarks/Usage:	
This is the same as calling `feFormatReal()` and then parsing the resulting text string into a new real variable.	
Example:	
None	

Library Management

The methods described here allow you manage libraries, which are available for some entity types.

feGetCurrentLibraryName

`feGetCurrentLibraryName`
`( entityTYPE, libraryTYPE, libraryNAME)`
Description:

Unpublished work. © 2025 Siemens

Retrieves the name of the current library file (*.*esp* file) being used for a particular entity type.	
Input:	
`INT4 or enum entityTYPE`	The type of entity to retrieve library name. Currently, there are libraries for `FT_VIEW`, `FT_MATL`, `FT_PROP`, `FT_LAYUP`, `FT_CONNECTION_PROP`, `FT_FUNCTION_DIR`, `FT_AMGR_DIR`, `FT_REPORT`, and `FT_CHART`. For entity information, see **Entity Types**
`INT4 or enum libraryTYPE`	Library to consider (i.e., tab on Library Management dialog box). 0=Display dialog box (`FLIB_DIALOG`) - not valid for this command 1=Personal tab (`FLIB_PERSONAL`) 2=Shared tab (`FLIB_SHARED`) 3=Femap Standard Libraries (`FLIB_SYSTEM`)
Output:	
`STRING libraryNAME`	Name of current library being used by specified entity type.
Return Code:	
`FE_BAD_TYPE`	Entity type specified is not a valid library type.
`FE_INVALID`	library type specified was not one of the three possible choices `FLIB_PERSONAL`, `FLIB_SHARED` or `FLIB_SYSTEM`.
Remarks/Usage:	
The full directory path to the library file is included in `libraryNAME`.	
Example:	
None	

feSetCurrentLibraryName

Unpublished work. © 2025 Siemens

feSetCurrentLibraryName

(entityTYPE, libraryTYPE, libraryNAME)

Description:

Sets or changes the current library file (*.*esp* file) being used for a particular entity type.

Input:

INT4 or enum entityTYPE	The type of entity to set or change the current library. Currently, there are libraries for FT_VIEW, FT_MATL, FT_PROP, FT_LAYUP, FT_CONNECTION_PROP, FT_FUNCTION_DIR, FT_AMGR_DIR, FT_REPORT, and FT_CHART. For entity information, see **Entity Types**.
INT4 or enum libraryTYPE	Library to specify (i.e., tab on Library Management dialog box): 0=Display dialog box (FLIB_DIALOG) - not valid for this command 1=Personal tab (FLIB_PERSONAL) 2=Shared tab (FLIB_SHARED) 3=Femap Standard Libraries (FLIB_SYSTEM)
STRING libraryNAME	Name of current library to use for the specified entity type.

Output:

None	

Return Code:

FE_BAD_TYPE	Entity type specified is not a valid library type.
FE_INVALID	Library type specified was not one of the three possible choices FLIB_PERSONAL, FLIB_SHARED or FLIB_SYSTEM.
FE_NOT_EXIST	The specified filename in libraryNAME does not exist.

Remarks/Usage:

The full directory path to the library file should be included in libraryNAME.

Unpublished work. © 2025 Siemens

Example:
None

API Selection Tools

The methods described here allow you to create commands which can be called from the **API Selection Tools** menu found on the **Pick^** menu of the standard entity selection dialog box.

There are some restrictions on API scripts used for Selection and care must be taken to follow these guidelines:

- Every selection tool API must call `feStartSelectionTool` and should always check the entity type being selected. It should always abort if it does not support the entity type being selected. This call should be one of the first API calls that you make. It must come before any calls that would display another selection dialog.
- A Selection API may display dialog boxes, including other selection dialogs. If it does display another selection dialog, the **Selection API** menu will not be available as only one API can be run at a time.
- You should not add API scripts to the Selection API directory that are not intended simply to update the selection process.
- Care should be taken not to update the model database during a Selection API. While all API functionality is available, results will be unpredictable and potentially damaging if you attempt to update the model while already in the selection process.
- Either `feEndSelectionTool` or `feAbortSelectionTool` should be called at the end of the API ... and they may only be called once after calling `feStartSelectionTool`.

feStartSelectionTool

feStartSelectionTool **(nSelectedSET, nEntityType)**	
Description:	
Initial call to start a selection tool to determine type of entities being selected.	
Input:	
`INT4 nSelectedSET`	The ID of Set object that will contain the entity IDs that have already been selected in the dialog, if

Unpublished work. © 2025 Siemens

	any. If your API does not require this information you may set `nSelectedSET = 0`.
`INT4 nEntityType`	The type of entities currently being selected in the dialog box. For entity information, see **Entity Types**.
Output:	
None	
Return Code:	
`FE_FAIL`	This API call has been made outside of the standard selection dialog.
Remarks/Usage:	
This method must be called at the beginning of all Selection API Tools and you must check `nEntityType` to make sure that the API will provide IDs of the desired type. If not, the API should abort.	
Example:	

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Dim enType As femap.zDataType
    Dim s As femap.Set
    Set s = App.feSet

    App.feStartSelectionTool( 0, enType )
    If enType = FT_NODE Then
    s.AddRange( 1, 10, 2 )
    App.feEndSelectionTool( s.ID, 0 )
    Else
    App.feAbortSelectionTool()
    End If
End Sub
```

feEndSelectionTool

`feEndSelectionTool`
`( nAddSet, nRemoveSet )`
Description:

Unpublished work. © 2025 Siemens

Final call in a selection tool to update the selected entities in the dialog.	
Input:	
`INT4 nAddSet`	The ID of Set object that should contain any entity IDs that are to be added to the selection. If you are not adding entities, you may set `nAddSet = 0`.
`INT4 nRemoveSet`	The ID of Set object that should contain any entity IDs that are to be removed from the selection. If you are not removing entities, you may set `nRemoveSet = 0`.
Output:	
None	
Return Code:	
`FE_FAIL`	Either `feStartSelectionTool` was never called, the selection dialog box has been closed already, or the type of entities being selected in the dialog has changed prior to this update.
Remarks/Usage:	
This method must be called if you want to update the selected entities.	
Example:	
None	

feAbortSelectionTool

`feAbortSelectionTool` **`( void )`**	
Description:	
Final call in a selection tool to update the selected entities in the dialog.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
This method should be called to terminate the selection process if the type of entities being selected does not match what the API expects.	
Example:	
None	

Other Utilities

The methods described here allow you to access other functionality.

feLicenseExpiration

`feLicenseExpiration` **`( nDays )`**	
Description:	
Returns the number of days until your current license expires.	
Input:	
None	
Output:	
`INT4 nDays`	The number of days until the license expires.
Return Code:	
None	
Remarks/Usage:	
`nDays` is returned as "`-1`" for perpetual licenses.	
Example:	
None	

Unpublished work. © 2025 Siemens

feGFXEnableView

feGFXEnableView

(bEnableAll, nVuID)

Description:

Activates display of User Graphics in one or more selected views.

Input:

`BOOL bEnableAll`	If True, then display of User Graphics will be activated for all active views. If False, only the view specified by `nVuID` will be activated.
`INT4 nVuID`	The ID of the View to activate. Only used if `bEnableAll=False`.

Output:

None	

Return Code:

None	

Remarks/Usage:

This method is only needed if you only want to enable display of User Graphics in certain views. To display in every view, simply specify `bEnableAll=True` when you call feGFXSelect to select the data to display.

Example:

None

feLicenseMethod

feLicenseMethod

(nLicenseType)

Description:

Returns the method of licensing currently being used.

Unpublished work. © 2025 Siemens

Input:	
None	
Output:	
`INT4 nLicenseType`	The type of licensing currently being used: 0 = None, this means that licensing has somehow failed 1 = Dongle 2 = Node/Curve limited Demo 4 = FLEXlm-based Network Licensing 5 = API-only Licensing
Return Code:	
`FE_FAIL`	Licensing information is not valid.
`FE_SECURITY`	Licensing method is valid, but currently exceeding some limitation on the license (like a node limit).
Remarks/Usage:	
None	
Example:	
None	

feLicenseIsAvailable

`feLicenseIsAvailable` **`( nFeature )`**
Description:
Indicates whether a specific feature is licensed with this copy of Femap.
Input:

`INT4 nFeature`	The feature to check licensing status. 0 = NX Nastran Basic 1 = NX Nastran Nonlinear 2 = NX Nastran Dynamics 3 = NX Nastran DMAP

Unpublished work. © 2025 Siemens

	4 = NX Nastran Superelements 5 = NX Nastran Aero Basic 7 = NX Nastran Optimization 8 = NX Nastran Advanced Nonlinear 10 = NX Nastran RotorDynamics 11 = NX Nastran Advanced Nonlinear Explicit 15 = Femap Thermal 16 = Femap Advanced Thermal 17 = Femap Flow 18 = Femap Structural Analysis Toolkit
Output:	
None	
Return Code:	
`FE_FAIL`	Feature is not licensed.
Remarks/Usage:	
None	
Example:	
None	

feLicenseGetInfo

`feLicenseGetInfo` **`( nFeature, bUser, pnLic, pFeatures, pLicAvail, pLicInUse, pUsers )`**	
Description:	
Indicates whether a specific feature is licensed with this copy of Femap.	
Input:	
`INT4 nFeature`	The feature to check licensing status. -1 = Will return everything. 0 = NX Nastran Basic 1 = NX Nastran Nonlinear 2 = NX Nastran Dynamics 3 = NX Nastran DMAP 4 = NX Nastran Superelements

Unpublished work. © 2025 Siemens

	5 = NX Nastran Aero Basic 7 = NX Nastran Optimization 8 = NX Nastran Advanced Nonlinear 10 = NX Nastran RotorDynamics 11 = NX Nastran Advanced Nonlinear Explicit 15 = Femap Thermal 16 = Femap Advanced Thermal 17 = Femap Flow 18 = Femap Structural Analysis Toolkit
`BOOL bUser`	If True, the user will be returned. If False, just the license types will be returned.
Output:	
`INT4 pnLic`	The number of license returned.
`INT4 pFeatures[0..N]`	An array of the features.
`INT4 pLicAvail[0..N]`	Total number of licenses for each feature returned.
`INT4 pLicInUse[0..N]`	Number of used license for each feature returned.
`STRING pUsers[0..Sum(LicInUse)]`	An array of users corresponding to the licenses in use.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feLicensePrintInfo

`feLicensePrintInfo`
`( nFeature )`
Description:

Unpublished work. © 2025 Siemens

Print the features and licenses in use to the messages window.	
Input:	
`INT4 nFeature`	The feature to check licensing status. -1 = Will return everything. 0 = NX Nastran Basic 1 = NX Nastran Nonlinear 2 = NX Nastran Dynamics 3 = NX Nastran DMAP 4 = NX Nastran Superelements 5 = NX Nastran Aero Basic 7 = NX Nastran Optimization 8 = NX Nastran Advanced Nonlinear 10 = NX Nastran RotorDynamics 11 = NX Nastran Advanced Nonlinear Explicit 15 = Femap Thermal 16 = Femap Advanced Thermal 17 = Femap Flow 18 = Femap Structural Analysis Toolkit
`BOOL bUser`	If True, the user will be returned. If False, just the license types will be returned.
Output:	
None	
Return Code:	
`FE_FAIL`	Feature is not licensed,
Remarks/Usage:	
None	
Example:	
None	

feRunCommand

Unpublished work. © 2025 Siemens

feRunCommand	
(nCmdID, bWaitForCompletion)	
Description:	
Runs a Femap command.	
Input:	
`INT4 nCmdID`	The ID of the command to run.
`BOOL bWaitForCompletion`	If True, this method only returns when the command is finished. If False, it returns immediately.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Command IDs can be found by calling various functions defined earlier in this section, like `feGetMenuCommand( )`. This method provides no means for automatically answering dialog boxes that are displayed by the selected command - the command is run interactively.	
Example:	
None	

feAppVersion

feAppVersion	
(void)	
Description:	
Returns the version of Femap.	
Input:	
None	

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`INT4 version`	The version of Femap specified as an integer. This is 100 times the version number. For Example, v9.3.1 will return 931, v10.0 will return 1000.
Remarks/Usage:	
None	
Example:	
None	

feAppUndoCheckpoint

`feAppUndoCheckpoint` **`( undoTitle )`**	
Description:	
Establishes a point that the **Undo** command can roll back to.	
Input:	
`STRING undoTitle`	Once an *Undo checkpoint* is set, the specified String will appear between *Undoing* and *Command* in the Messages window when the **Tools** → **Undo** command is used.
Output:	
None	
Return Code:	
`FE_FAIL`	No model is open.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

feAppUndoClear

feAppUndoClear	
(void)	
Description:	
Clears the Undo Stack.	
Input:	
None	
Output:	
None	
Return Code:	
FE_FAIL	No model is open.
Remarks/Usage:	
None	
Example:	
None	

feCrossSectionReport

feCrossSectionReport
(nPropID, bDoEndA, bCopyToClipboard, fName, nFileFormat)
Description:

Unpublished work. © 2025 Siemens

Copies a picture of the cross section and the section property information from the Cross Section Definition dialog box to the clipboard for a specified bar, beam, or curved beam property. Alternatively, the picture and section information can be saved to a file.	
Input:	
`INT4 nPropID`	ID of bar, beam, or curved beam property.
`BOOL bDoEndA`	When True, copies or saves the picture and section properties for End A of the selected property. When False, copies or saves the picture and section properties of End B.
`BOOL bCopyToClipboard`	When True, copies picture and information to the clipboard. When False, saves picture and information to a file using the format selected in `nFileFormat`.
`STRING fName`	The complete filename of the picture file to save the picture and section information.
`INT4 nFileFormat`	Format of save picture file: 0=Bitmap (`FPM2_BMP`), 1=JPEG (`FPM2_JPEG`), 2=GIF (`FPM2_GIF`), 3=TIFF (`FPM2_TIF`), 4=PNG (`FPM2_PNG`).
Output:	
None	
Return Code:	
`FE_INVALID`	Invalid file format.
`FE_NOT_EXIST`	Selected property does not exist.
`FE_NOT_AVAILABLE`	selected property is not a type with a cross section.
Remarks/Usage:	
None	
Example:	
None	

feAppResetLocale

Unpublished work. © 2025 Siemens

`feAppResetLocale`
 	
`( void )`	

Description:	
Resets Locale.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No model is open.
Remarks/Usage:	
None	
Example:	
None	

User Graphics Methods

Managing User Graphics Data

User Graphics data allows you to create additional points, lines, arrows, and polygons (triangles and quads) that can be displayed with your model. User Graphics data has no interaction with your other model data, it is simply used for display.

Just like many other types of data in Femap (loads, constraints...), User Graphics data is stored in sets. You can create as many different sets of data in you model as you want (with the normal ID limits). Simply creating the data however does not make it visible in your graphics windows. Graphics Data only becomes visible when its set is selected, and only one set can be selected at a time. This gives you the flexibility of creating many different graphical "*objects*", each in its own set, and then quickly switching between them by selecting the appropriate set.

Unpublished work. © 2025 Siemens

User Graphics data can be managed either through these methods, or using the various User Graphics Entity objects. These methods provide a bit simpler interface, the entity objects provide more overall capability.

The following methods allow you to define and manage your graphics data. Selecting the data for display is described in the next section.

feGFXLoad

feGFXLoad	
(nSetID, nxyz, xyz, npoly, poly, color, edgecolor, symbol)	
Description:	
Creates User Graphics Data containing points, lines, triangles and/or quads.	
Input:	
`INT4 nSetID`	The User Graphics Set to create.
`INT4 nxyz`	The number of locations specified in the coordinate table (xyz).
`REAL8 xyz[0..(3*nxyz)-1]`	The coordinates used to define the endpoints/ corners of the graphics data. This data is referenced by the entries in *poly*. All values must be in global rectangular coordinates.
`INT4 npoly`	The number of entries in the *poly* array.
`INT4 poly[0..(4*npoly)-1]`	This array contains 4 entries for each graphics *entity* to be created. The values must be the index in the *xyz* array of the coordinates for that endpoint/corner. For points, only the first of the four entries is used. For lines, the first two are used. For triangles, the first three are used. **ALL UNUSED ENTRIES MUST BE SET TO -1** (that is what indicates which shape you are describing). The array must contain data in the following order (`Entity1Corner1`, `Entity1Corner2`, `Entity1Corner3`, `Entity1Corner4`, `Entity2Corner1`, `Entity2Corner2...`). Note that the first coordinate in the xyz array is index 0, not 1.

Unpublished work. © 2025 Siemens

`INT4 color`	The color of the entities created. For triangles and quads this is the fillcolor
`INT4 edgecolor`	Only used for triangles and quads. The color of the edges.
`INT4 symbol`	Only used for points. The symbol to be displayed for points.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to allocate memory or access the data provided.
`FE_BAD_DATA`	One of the indices in *poly* is out of the range provided in *xyz*.
Remarks/Usage:	
None	
Example:	

```
Sub Main
Dim App As femap.model
Set App = feFemap()
Dim xyz(2,10000) As Double
Dim poly(3,10000) As Long

Dim n As femap.Node
Set n = App.feNode
Dim e As femap.Elem
Set e = App.feElem

' Grab the nodal coordinates, but offset by 1 in the Z direction
While (n.Next())
 xyz(0,n.ID) = n.x
 xyz(1,n.ID) = n.y
 xyz(2,n.ID) = n.z + 1.0
Wend

' Build a line/tri/quad for each element we find
nPoly = 0
While (e.Next())
  Select Case e.topology
     Case FTO_LINE2
        poly(0,nPoly) = e.Node(0)
        poly(1,nPoly) = e.Node(1)
        poly(2,nPoly) = -1
        poly(3,nPoly) = -1
     Case FTO_TRIA3
        poly(0,nPoly) = e.Node(0)
```


Unpublished work. © 2025 Siemens

```
        poly(1,nPoly) = e.Node(1)
        poly(2,nPoly) = e.Node(2)
        poly(3,nPoly) = -1
     Case FTO_QUAD4
        poly(0,nPoly) = e.Node(0)
        poly(1,nPoly) = e.Node(1)
        poly(2,nPoly) = e.Node(2)
        poly(3,nPoly) = e.Node(3)
  End Select
  nPoly = nPoly + 1
Wend
' Load the graphics data
rc = App.feGFXLoad( 1, 10000, xyz, nPoly, poly, FCL_RED, FCL_ORANGE,
GPS_DIAMOND )
' Select and draw the graphics data
rc = App.feGFXSelect( 1, True, True )
End Sub
```

feGFXDelete

feGFXDelete **(bAllSets, nSetID)**	
Description:	
Deletes all User Graphics entities (points, lines, arrows, triangles and quads) in one or more sets.	
Input:	
`BOOL bAllSets`	If True, all entities, in all sets are deleted. If False, then all entities in Set `nSetID` are deleted.
`INT4 nSetID`	The `setID` to delete if `bAllSets=False`.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to delete entities.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Calling this method deletes the User Graphics data from your model, however if User Graphics were selected for display, they will continue to be displayed even after calling this method (even though the underlying data has been deleted). You must call **feGFXReset** to remove the graphics from the display.
Example:
None

Displaying User Graphics Data

Once User Graphics data has been created, either using **feGFXLoad** or using the User Graphics Entity objects, it can be selected for display.

feGFXSelect

feGFXSelect **(nSetID, bEnableAll, bRedraw)**	
Description:	
Selects a predefined set of User Graphics entities for display.	
Input:	
`INT4 nSetID`	The setID to be selected for display
`BOOL bEnableAll`	If True, then display of User Graphics will be enabled for all active views. If False, you must subsequently call **feGFXEnableView** to enable display for the views you want.
`BOOL bRedraw`	Once selected User Graphics will only be displayed after a Redraw. If True, this forces an immediate redraw. Otherwise you will have to redraw later to see your graphics.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to load selected set.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

feGFXEnableView

feGFXEnableView **(bEnableAll, nVuID)**	
Description:	
Activates display of User Graphics in one or more selected views.	
Input:	
`BOOL bEnableAll`	If True, then display of User Graphics will be activated for all active views. If False, only the view specified by `nVuID` will be activated.
`INT4 nVuID`	The ID of the View to activate. Only used if `bEnableAll=False`.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method is only needed if you only want to enable display of User Graphics in certain views. To display in every view, simply specify `bEnableAll=True` when you call feGFXSelect to select the data to display.	
Example:	
None	

feGFXReset

Unpublished work. © 2025 Siemens

feGFXReset	
(void)	
Description:	
Disables display of User Graphics in all views.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	User Graphics not reset.
Remarks/Usage:	
This method resets the display of User Graphics. It does not delete the data from your model. You must call **feGFXDelete** or one of the User Graphics Entity delete methods to delete the data. After calling this method, you can redisplay the data by calling **feGFXSelect**.	
Example:	
None	

File Utility Methods

feFileGetName

feFileGetName
(title, filterName, filterStr, openForRead, fName)
Description:
Allows you to display a dialog box to retrieve a filename.
Input:

Unpublished work. © 2025 Siemens

`STRING title`	A text string to be displayed as the title of the dialog box.
`STRING filterName`	A text string describing the type of files that you will be looking for.
`STRING filterStr`	A text string used to limit the types of files displayed. This should be of the form "*.XXX" to display all files with an XXX file extension. Use "*.*" to display all files.
`BOOL openForRead`	If True, the name of an existing file must be selected.
Output:	
`STRING fName`	The name of the file chosen by the user.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	

```
Sub Main
Dim App As femap.model
Set App = feFemap()

Dim rc As Long
Dim fName As String

rc = App.feFileGetName("Open Neutral File", "Neutral Files", _
 "*.NEU", True, fName)

MsgBox(fName) ' Show filename in dialog box

End Sub
```

feFileExecute

Unpublished work. © 2025 Siemens

feFileExecute **(cmdLine, waitForCompletion)**	
Description:	
Allows you to run a program or other object as a separate process.	
Input:	
`STRING cmdLine`	The full command line used to launch the program.
`BOOL waitForCompletion`	If True, this method will wait until the other process completes before returning. If False, it will return as soon as the process starts.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feFileProgramRun

feFileProgramRun **(useDlg, init, isFile, buff)**	
Description:	
This function selects and runs a program file.	
Input:	
`BOOL useDlg`	If True, this function runs interactively and displays a dialog box to ask the user to define the filename of the program file to run.

Unpublished work. © 2025 Siemens

`BOOL init`	If True, any previously running program file is terminated and other variables are reinitialized. In most cases, this should be True.
`BOOL isFile`	If True, the buff variable must contain the full name of the program file to run. If False, buff contains the actual text of the program file. Ignored if `useDlg=True`.
`STRING buff`	Either the name or the contents of the program file to run (depends on the `isFile` parameter).
Output:	
None	
Return Code:	
`FE_FAIL`	The program file could not be run.
Remarks/Usage:	
Refer to the Section later in this document on Program Files for more information on the syntax required. When you call this function, the program file is started, however the function returns immediately, it can not wait for the program file to complete. Refer to the `feFileProgramRunning( )` method, and the chapter on events for possible methods for determining when a program file finishes.	
Example:	
None	

feFileProgramRunning

`feFileProgramRunning` **`( void )`**	
Description:	
This function can be called to see if a program file is still running.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_OK`	Program file is still running.
`FE_FAIL`	No program file is running.
Remarks/Usage:	
Refer to the Section later in this document on Program Files for more information on the syntax required. You can call this method, see if the program file is running, and if it is, wait for some period and then check again, until the program file finishes.	
Example:	
None	

feFileCurrentDirectory

`feFileCurrentDirectory` **`( sCurrentDir )`**	
Description:	
This function sets the current directory for a model.	
Input:	
`STRING sCurrentDir`	The directory to set as current.
Output:	
None	
Return Code:	
`FE_FAIL`	The directory specified is not valid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Set Utility Methods

feSetFree

feSetFree **(setID)**	
Description:	
Deletes a saved Femap Set.	
Input:	
`INT4 setID`	ID of saved Set you wish to delete.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

feSetFreeNotInSet

feSetFreeNotInSet **(setID)**
Description:
Deletes all saved sets not in the input set.

Unpublished work. © 2025 Siemens

Input:	
`INT4 setID`	ID of Set that contains IDS to not delete. Set to 0 to delete all saved Sets.
Output:	
None	
Return Code:	
`FE_OK`	Sets were deleted.
Remarks/Usage:	
None	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim cs As femap.Set
   Set cs = App.feSet

    ' Delete all saved sets except 1
   cs.Add( 1 )

   rc = App.feSetFreeNotInSet( cs.ID )
End Sub
```

Data Management

feLoginToTeamcenter

Unpublished work. © 2025 Siemens

feLoginToTeamcenter **(sUserID, sPassWord, sGroup, sRole, sServerUrl)**	
Description:	
Login information for logging into Teamcenter from inside Femap.	
Input:	
`STRING sUserID`	User name.
`STRING sPassWord`	User password.
`STRING sServerUrl`	Valid Teamcenter Client Communication System (TCCS) address.
Output:	
`STRING sGroup`	Group name.
`STRING sRole`	Role in Teamcenter.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

feIsUserLoggedInToTeamcenter

feIsUserLoggedInToTeamcenter **(sUserID)**
Description:
Checks if logged into Teamcenter from inside Femap.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
`STRING sUserID`	User name.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

feOpenModelFromTeamcenter

`feOpenModelFromTeamcenter` **`( sItemID, sRevision sDatasetName, sDownloadDir, bCheckOut )`**	
Description:	
Information for checking out an Item from Teamcenter to Femap.	
Input:	
`STRING sItemID`	Teamcenter Item ID of file being checked out.
`STRING sRevision`	Revision of the Item in Teamcenter.
`STRING sDatasetName`	Dataset name supporting the Item being cecked out from Teamcenter.
`STRING sDownloadDir`	Local directory for downloading the Item being cecked out from Teamcenter.
`BOOL bCheckOut`	If True, the Check Out option is activated.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

feSaveModelToTeamcenter

feSaveModelToTeamcenter **(sItemID, sRevision sDatasetName, sFilePath, bOverWrite, bCheckIn)**	
Description:	
Information for checking in a model from Femap into the Teamcenter Item.	
Input:	
`STRING sItemID`	Teamcenter Item ID of file being checked into.
`STRING sRevision`	Revision of the Item in Teamcenter.
`STRING sDatasetName`	Dataset name supporting the model being cecked into Teamcenter.
`STRING sFilePath`	Source directory for uploading the model being cecked into Teamcenter.
`BOOL bOverWrite`	When True, the file being checked in overwrites the existing file saved in the Teamcenter Item. Otherwise a prompt to overwrite appears.
`BOOL bCheckOut`	If True, the Check In option is activated.
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

feGetCurrentModelPDMInfo

`feGetCurrentModelPDMInfo`
 	
`( pPDMInfo )`	

Description:	
Information for checking information about the currently active model.	
Input:	
None	None
Output:	
`STRING pPDMInfo`	Information about the current model.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

feGetTeamcenterDatasetInfo

Unpublished work. © 2025 Siemens

feGetTeamcenterDatasetInfo	
(sItemID, sRevision sDatasetName, sDatasetinfo)	
Description:	
Information for checking information about the currently active model.	
Input:	
`STRING sItemID`	Teamcenter Item ID.
`STRING sRevision`	Revision of the Item in Teamcenter.
`STRING sDatasetName`	Dataset name supporting the model in Teamcenter.
Output:	
`STRING sDatasetinfo`	Information about the dataset.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

4. Femap Tool Objects

In addition to the various methods and properties that are available directly from the main Femap application object, and the entity objects described later, Femap also provides other objects that provide access to other functionality and tools. Some of these objects allow access to the various windows available in the user interface, others provide entity selection, and general data and file functionality that might not be available from the environment where you are programming.

Creating an Object

To link your application to Femap, you had to declare an object and then connect it to Femap. In Basic, this is done via the `GetObject` or `CreateObject` functions. Accessing these additional Femap objects is similar. First you must declare an object. Then, however, instead of using the standard system level functions, you use one of the following Femap functions to actually create the object. Once the object has been created, all of the properties and methods for that type of object are available for use.

All of the following methods are available from the Femap application object, and return the object that they create.

DataTable Objects

Method	Description	Other Methods
`feDataTable` (DataTable Object)	Creates a DataTable Object	

Element Quality Objects

Method	Description	Other Methods
`feElementQuality` (Element Quality Objects)	Creates an Element Quality Object	

Geometry Preparation and Meshing Objects

Method	Description	Other Methods
`feMesher` (Geometry Preparation and Meshing Objects)	Creates a Geometry Preparation and Meshing Object	

Unpublished work. © 2025 Siemens

Geometry Interface Objects

Method	Description	Other Methods
`feGeometryInterface` (Geometry Interface Objects)	Creates a GeometryInterface Object	

ReadFile Objects

Method	Description	Other Methods
`feRead` (ReadFile Objects)	Creates a ReadFile Object	

Selector Objects

Method	Description	Other Methods
`feSelector` (Selector Objects)	Creates a Selector Object	

Set Objects

Method	Description	Other Methods
`feSet` (Set Objects)	Creates a Set Object	Group.List

Sort Objects

Method	Description	Other Methods
`feSort` (Sort Objects)	Creates a Sort Object	

UserData Objects

Method	Description	Other Methods
`feUserData`)UserData Objects)	Creates a UserData Object	

Unpublished work. © 2025 Siemens

Results Browsing Objects

Method	Description	Other Methods
`feResults ()`	Creates a Results Browsing Object	

MapData Objects

Method	Description	Other Methods
`feMapData` (MapData Objects)	Creates a MapData Object	

Entity Tracking Objects

Method	Description	Other Methods
`feTrackData` (Entity Tracking Objects)	Creates a Entity Tracking Object	

Beam Calculator Objects

Method	Description	Other Methods
`feBeamCalculator` (Beam Calculator Objects)	Creates a Beam Calculator Object	

Draw/Erase Objects

Method	Description	Other Methods
`feDrawErase` (Draw/Erase Objects)	Creates a Draw/Erase Object	

User Defined Graphics Objects

Method	Description	Other Methods
`feUserDefinedGraphics` (User Defined Graphics Objects)	Creates a User Defined Graphics Object	

Unpublished work. © 2025 Siemens

CopyTool Objects

Method	Description	Other Methods
`feCopyTool` (CopyTool Objects)	Creates a CopyTool Object	

MoveTool Objects

Method	Description	Other Methods
`feMoveTool` (MoveTool Objects)	Creates a MoveTool Object	

MergeTool Objects

Method	Description	Other Methods
`feMergeTool` (MergeTool Objects)	Creates a MergeTool Object	

Results Vector ID Objects

Method	Description	Other Methods
`feResultsIDQuery` (Results Vector ID Query Objects)	Creates a Results Vector ID Query Object	

Stress Linearization (StressLinear) Objects

Method	Description	Other Methods
`feStressLinear` (Stress Linearization (StressLinear) Objects)	Creates a Stress Linearization Object	

PublishTool Objects

Method	Description	Other Methods
`fePublishTool` (PublishTool Objects)	Creates a PublishTool Object	

Unpublished work. © 2025 Siemens

PublishTable Objects

Method	Description	Other Methods
`fePublishTable` (PublishTable Objects)	Creates a PublishTable Object	

BodyMesher Objects

Method	Description	Other Methods
`feBodyMesher` (BodyMesher Objects)	Creates a BodyMesher Object	

SolidCleanupTool Objects

Method	Description	Other Methods
`feSolidCleanupTool` (SolidCleanupTool Objects	Creates a SolidCleanupTool Object	

DataTable Object

The DataTable object provides access to the functionality of the Data Table in your model. It is not a model entity.

Use the feDataTable method of the Femap Application object to access the DataTable object.

DataTable Object Properties

Property	Description
`INT4 NumberOfRows`	The number of data rows in the Data Table (Read Only)
`INT4 NumberOfColumns`	The number of columns in the Data Table (Read Only)
`INT4 Type`	The type of entities being shown in the Data Table. For more information on Entity Types refer to the table in Entity Types.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL VisibleRowsOnly`	If True, then row indexes reference the rows as currently displayed in the table. If False, they reference the rows as they were added to the table, regardless of which rows are displayer or their order.
`BOOL VisibleColsOnly`	If True, then column indexes reference the columns as currently displayed in the table. If False, they reference the columns as they were added to the table, regardless of which columns are displayer or their order.
`BOOL Visible`	Data table visibility (Read Only). If True, then the Data Table is currently visible in the Femap GUI. If False, then the Data Table is not currently visible. Use `feAppManagePanes` application method to change visibility of the Data Table pane
`BOOL Locked`	Data table lock status (Read Only). If True, then the Data Table is currently locked in the Femap GUI. If False, then the Data Table is not currently locked. Always returns TRUE if the Data Table is not visible. Use the Data Table Lock method to lock and unlock the Data Table.

Specifying Row and Column Indexes

Most methods in the DataTable object require you to specify row and column indexes to locate the data that you want to manipulate. To make access to that DataTable as flexible as possible for different applications, there are two separate ways to specify those indexes. The choice of these methods are controlled by the `VisibleRowsOnly` and `VisibleColsOnly` properties.

The default for both of these properties is False - which means that both row and column indexes refer to the order in which the rows and columns were added to the table. For most applications, this is the way in which you will want to work, since these numbers do not normally change (unless you delete rows) - they are not affected by changing sort order, hiding rows or columns, grouping or any other manipulation of how the table is displayed.

In `VisibleRowsOnly`/`VisibleColsOnly` mode (set the property to True), the indexes refer to the visible row or column in the table. If you hide rows or columns, those columns are no longer accessible in this mode. If you change column order, or change the sort order of the rows, they are still available, but the indexes change, since the positions in the display have changed.

In either mode, row and column indexes both start at 0, which is either the first row/column created, or the top row/left column in the table.

Unpublished work. © 2025 Siemens

DataTable Object Row Methods

The following methods are available to access and manipulate Data Table rows.

FindRow

FindRow **(ModelID, SetID, ID)**	
Description:	
This method returns the index of the row that matches your search criteria.	
Input:	
`INT8 ModelID`	The ID of the Model to match. (0 = Any Model). This is only necessary if you want to load data from multiple models in the same table.
`INT4 SetID`	The `SetID` of the entity to match (0 = Any `SetID`). This is not necessary for most model entities (like nodes, elements, properties...) which all have `SetID = 1`.
`INT4 ID`	The ID of the entity to match (0 = Any ID).
Output:	
None	
Return Code:	
`rowIndex`	The index of the row in the Data Table that matches your criteria. If no row matches rowIndex will be negative.
Remarks/Usage:	
This function can be used if you do not want to keep track of row indices. Because it returns the row index as the Return Code, this method can be used directly in other calls as shown in the example. It will always return the appropriate matching index, regardless of the setting of `VisibleRowsOnly`.	

Unpublished work. © 2025 Siemens

Example:

This example hides the row that contains the entity with ID = 3
 	
`Sub Main`
`Dim App As femap.model`
`Set App = feFemap`
 	
`Dim r As femap.DataTable`
`Set r = App.feDataTable`
 	
`r.SetRowVisible( r.FindRow(0,0,3), False )`
 	
`End Sub`

GetRowInfo

`GetRowInfo` **`( numRows, ModelID, SetID, ID )`**	
Description:	
Returns information about all the rows in the table.	
Input:	
None	
Output:	
`INT4 numRows`	The number of entries in the `ModelID`, `SetID` and ID arrays.
`INT8 ModelID[0..numRows-1]`	The ID of the Model associated with the entity in each row.
`INT4 SetID[0..numRows-1]`	The `SetID` of the entity in the row.
`INT4 ID[0..numRows-1]`	The ID of the entity in the row.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	The Data Table does not contain any data.
`FE_NO_MEMORY`	Can not allocate memory to retrieve info from the Data Table.
Remarks/Usage:	
This method can be used to retrieve arrays that indicate all of the entities in the Data Table. If `VisibleRowsOnly` is True, only information about visible rows is returned.	
Example:	
None	

GetRowVisible

`GetRowVisible`	
`( nRow, bVisible )`	
Description:	
Returns the visibility of a row in the table.	
Input:	
`INT4 nRow`	The index of the row to query.
Output:	
`BOOL bVisible`	True if the row is visible, False if hidden.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified row does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetRowVisible

SetRowVisible **(nRowOrAll, bVisible)**	
Description:	
Sets one or all rows visible or hidden.	
Input:	
`INT4 nRowOrAll`	The index of the row to update (-1=All Rows).
`BOOL bVisible`	True to make the row(s) visible, or False to hide them.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified row does not exist.
Remarks/Usage:	
None	
Example:	
None	

DataTable Object Column Methods

There following methods are available to access and manipulate Data Table columns.

FindColumn

Unpublished work. © 2025 Siemens

FindColumn

(colTitle)

Description:

This method returns the index of the column that matches your search criteria.

Input:

`STRING colTitle`	The title of the column to locate.

Output:

None	

Return Code:

`colIndex`	The index of the column in the Data Table that matches your criteria. If no column matches `colIndex` will be negative.

Remarks/Usage:

This function can be used if you do not want to keep track of column indices. Because it returns the column index as the Return Code, this method can be used directly in other calls as shown in the example. It will always return the appropriate matching index, regardless of the setting of `VisibleColsOnly`.

Example:

This example hides the column that contains the title "*Topology*"

```
Sub Main
Dim App As femap.model
Set App = feFemap

Dim r As femap.DataTable
Set r = App.feDataTable

r.SetColumnVisible( r.FindColumn("Topology", False )

End Sub
```

GetColumnInfo

Unpublished work. © 2025 Siemens

GetColumnInfo **(numColumns, colType, colTitle)**	
Description:	
This method is obsolete and should no longer be used. Use `GetColumnInfo2` instead. Returns information about all the columns in the table.	
Input:	
None	
Output:	
`INT4 numColumns`	The number of entries in the `colType` and `colTitle` arrays.
`INT4 colType[0..numColumns-1]`	A value indicating the type of data in the column.
`STRING colTitle[0..numColumns-1]`	The titles of the columns.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The Data Table does not contain any data.
`FE_NO_MEMORY`	Can not allocate memory to retrieve info from the Data Table.
Remarks/Usage:	
This method can be used to retrieve arrays that indicate all of the entities in the Data Table. If `VisibleColsOnly` is True, only information about visible columns is returned.	
Example:	
None	

GetColumnInfo2

Unpublished work. © 2025 Siemens

GetColumnInfo2

(numColumns, colType, colTitle)

Description:

Returns information about all the columns in the table. Returns column types that match those needed for `AddColumn()` and `UpdateColumn()`.

Input:

None	

Output:

`INT4 numColumns`	The number of entries in the `colType` and `colTitle` arrays.
`INT4 colType[0..numColumns-1]`	A value indicating the type of data in the column.
`STRING colTitle[0..numColumns-1]`	The titles of the columns.

Return Code:

`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The Data Table does not contain any data.
`FE_NO_MEMORY`	Can not allocate memory to retrieve info from the Data Table.

Remarks/Usage:

This method can be used to retrieve arrays that indicate all of the entities in the Data Table. If `VisibleColsOnly` is True, only information about visible columns is returned.

Example:

```
' === First, use FEMAP to load Data Table with Nodal Output Vectors ===

' =====Example: List, Output, Results to Data Table, pick Translation
Vectors… ===


  Sub Main

   Dim App As femap.model

   Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
   Dim d As femap.DataTable
   Set d = App.feDataTable

    Dim numRows As Long, ModelID As Variant, setID As Variant, ID As
Variant
   d.GetRowInfo( numRows, ModelID, setID, ID )

   If numRows < 1 Then End

   Dim values() As Variant
   ReDim values( numRows-1 )

    ' === Filler values ===
   For i = 0 To numRows-1
       values(i) = i+1 + 1000
   Next

    '=== GetColumnInfo should provide immediate transition to "AddColumn"
method ===
   Dim numCols&, colType As Variant, colTitle As Variant
   rc = d.GetColumnInfo2( numCols, colType, colTitle )

   Dim newID As Long
    ' === using colTitle AND colType here that we populated from
"GetColumnInfo2" ====
    rc = d.AddColumn( False, False, FT_NODE, 0, "API - " &
colTitle(numCols-1), colType(numCols-1), numRows, ID, values, newID )
```


Unpublished work. © 2025 Siemens

```
End Sub
```

GetColumnVisible

GetColumnVisible **(nColumn, bVisible)**	
Description:	
Returns the visibility of a column in the table.	
Input:	
`INT4 nColumn`	The index of the column to query.
Output:	
`BOOL bVisible`	True if the column is visible, False if hidden.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetColumnVisible

Unpublished work. © 2025 Siemens

SetColumnVisible	
(nColumnOrAll, bVisible)	
Description:	
Sets one or all columns visible or hidden.	
Input:	
`INT4 nColumnOrAll`	The index of the column to update (`-1=All Columns`).
`BOOL bVisible`	True to make the column(s) visible, or False to hide them.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

GetColumnWidth

GetColumnWidth
(nColumn, nWidth)
Description:
Returns the width of a column in the table.
Input:

Unpublished work. © 2025 Siemens

`INT4 nColumn`	The index of the column to query.
Output:	
`INT4 nWidth`	The width of the column in pixels.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetColumnWidth

`SetColumnWidth` **`( nColumnOrAll, nWidth )`**	
Description:	
Sets the width of one or all columns.	
Input:	
`INT4 nColumnOrAll`	The index of the column to update (`-1=All Columns`).
`INT4 nWidth`	The width of the columns in pixels.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

GetColumnTitle

`GetColumnTitle` **`( nColumn, colTitle )`**	
Description:	
Returns the title of a column in the table.	
Input:	
`INT4 nColumn`	The index of the column to query.
Output:	
`STRING colTitle`	The title of the column.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetColumnTitle

Unpublished work. © 2025 Siemens

SetColumnTitle	
(nColumn, colTitle)	
Description:	
Sets the title for a column.	
Input:	
`INT4 nColumn`	The index of the column to update.
`STRING colTitle`	The title of the column.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetColumnPosition

SetColumnPosition	
(nColumn, nNewIndex, bAfter)	
Description:	
Moves the display location of a column in the Data Table.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 nColumn`	The index of the column to update.
`INT4 nNewIndex`	The index of the new location in the table. The column is moved to the left of this location.
`BOOL bAfter`	If True, the `nColumn` is placed after the `nNewIndex` column, otherwise it is placed before that column.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
`FE_FAIL`	Not able to move column to the requested location.
Remarks/Usage:	
This method will automatically make the specified column that you are moving visible - even if it is not currently.	
Example:	
None	

SetColumnAlignment

`SetColumnAlignment`
`( nColumn, nAlign )`
Description:
Updates the alignment of data in the selected column(s).
Input:

`INT4 nColumnOrAll`	The index of the column to update (-1 = All Columns).

Unpublished work. © 2025 Siemens

`INT4 nAlign`	The data alignment (0=Left, 1=Center, 2=Right).
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The specified column does not exist.
Remarks/Usage:	
None	
Example:	
None	

DataTable Object Data Methods

The following methods are available to access and manipulate Data Table data.

GetColumnValue

`GetColumnValue` **`( nColumn, numRowsOrAll, Rows, numData, ColumnData )`**	
Description:	
This method returns data from selected rows in a column.	
Input:	
`INT4 nColumn`	The index of a column to query.
`INT4 numRowsOrAll`	The number of entries in the Rows array, or -1 to retrieve data from all rows.
`INT4 Rows[0..numRowsOrAll-1]`	The indices of the rows to query.

Unpublished work. © 2025 Siemens

Output:	
`INT4 numData`	The number of items returned in `ColumnData`.
`VARIANT ColumnData`	The data recovered from Column `nColumn` and the selected rows. Depending on the type of data contained in the column, this could be an array of booleans, integers, doubles or strings.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The Data Table does not contain any data.
`FE_NO_MEMORY`	Can not allocate memory to retrieve info from the Data Table.
`FE_BAD_TYPE`	The selected column contains a type of data that is not supported by this method.
Remarks/Usage:	
Typically `numData` will be the same as `numRowsOrAll` (if `numRowsOrAll` is not -1), unless some of the rows you request are invalid. The `GetColumnText` method is similar, but always returns the data as text strings.	
Example:	
This example retrieves the Property ID from Rows 4, 1, and 0 of the Data table.	

```
Sub Main
Dim App As femap.model
Set App = feFemap

Dim r As femap.DataTable
Set r = App.feDataTable

Dim ro(3) As Long
ro(0) = 4
ro(1) = 1
ro(2) = 0

Dim n As Long
Dim c As Variant

r.GetColumnValue( r.FindColumn("Prop ID"), 3, ro, n, c )
End Sub
```


Unpublished work. © 2025 Siemens

GetColumnText

GetColumnText **(nColumn, numRowsOrAll, Rows, numData, ColumnText)**	
Description:	
This method returns data from selected rows in a column.	
Input:	
`INT4 nColumn`	The index of a column to query.
`INT4 numRowsOrAll`	The number of entries in the Rows array, or -1 to retrieve data from all rows.
`INT4 Rows[0..numRowsOrAll-1]`	The indices of the rows to query.
Output:	
`INT4 numData`	The number of items returned in `ColumnData`.
`STRING ColumnText[0..numData-1]`	The data recovered from Column `nColumn` and the selected rows. This method always retrieves the text that is displayed, rather than numeric values. For columns displaying check boxes, the string returned is “1” if the box is on, and “0” if it is off.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The Data Table does not contain any data.
`FE_NO_MEMORY`	Can not allocate memory to retrieve info from the Data Table.
`FE_BAD_TYPE`	The selected column contains a type of data that is not supported by this method.
Remarks/Usage:	
This method is identical to `GetColumnValue`, except that it always returns the data as a text string.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetRowValues

GetRowValues **(numRowsOrAll, Rows, numData, RowData)**	
Description:	
This method returns data from all columns for selected rows	
Input:	
`INT4 numRowsOrAll`	The number of entries in the Rows array, or -1 to retrieve data from all rows.
`INT4 Rows[0..numRowsOrAll-1]`	The indices of the rows to query.
Output:	
`INT4 numData`	The number of Columns returned in `RowData`.
`VARIANT RowData`	The data recovered from the selected rows for all columns. The data is returned as an array of Variants, where each variant contains an array of data for the corresponding column. Depending on the type of data contained in the column, this could be an array of booleans, integers, doubles or strings.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The Data Table does not contain any data.
`FE_NO_MEMORY`	Can not allocate memory to retrieve info from the Data Table.
`FE_BAD_TYPE`	The selected column contains a type of data that is not supported by this method.
Remarks/Usage:	
The `RowData` variant that is returned can be accessed as a two-dimensional array, where the first index represents the column, and the second the row ... `RowData(column)(row)`.	

Unpublished work. © 2025 Siemens

Example:

This example retrieves all the data in the table and echoes it to the Message Window.

```
Sub Main
Dim App As femap.model
Set App = feFemap

Dim r As femap.DataTable
Set r = App.feDataTable

Dim nd As Long
Dim rd As Variant

r.GetRowValues( -1, rr, nd, rd )

For i=0 To r.NumberOfRows-1
  For c = 0 To nd-1
         App.feAppMessage( FCM_NORMAL, "Row "+Str$(i)+" Column
"+Str$(c)+" = "+Str$(rd(c)(i)) )
  Next c
Next i
End Sub
```

AddColumn

AddColumn

(bClear, bHidden, entityType, entitySetID, colTitle, colType, nRows, ID, Value, nNewColumnID)

Description:

This method creates a new column in the Data Table and adds data to it.

Input:

`BOOL bClear`	If True, all previously existing data in the Data Table is deleted.
`BOOL bHidden`	If True, the column is created hidden, if False, the column is visible.
`INT4 entityType`	The type of entities being shown in the Data Table. This needs to be the same for all columns. For more information on Entity Types refer to the table in **Entity Types**.

Unpublished work. © 2025 Siemens

`INT4 entitySetID`	The Set ID associated with these values being added. Can be 0 to match any Set ID or use the default for new rows.
`STRING colTitle`	The title for the column - must be unique among all existing columns.
`INT4 colType`	The type of column to create (0=Bool/Check Box, 1=Integer, 2=Real, 3=String). Boolean arrays will create a column that is displayed as Check Boxes (checked on if True).
`INT4 nRows`	The number of entries in the ID and Value arrays.
`INT4 ID[0..nRows-1]`	The entity IDs associated with the data in the corresponding location in *Value*. If a row with the same ID already exists in the Data Table, the associated *Value* is assigned to that row. If no row with that ID exists, a new row is created with the current Model ID, the specified `entitySetID`, and this ID, and the new row is assigned the value.
`VARIANT Value`	This Variant contains an array with the values to be assigned to the rows selected by the corresponding entity IDs from `ID`. Depending on the option you specify in `colType`, the variant must contain either a Boolean, Integer, Double or String array.
Output:	
`INT4 nNewColumnID`	The index of the column that was created. This index is always based on the order the column was created, not the visible location, no matter how `VisibleColsOnly` is set. If `VisibleColsOnly`=False, this number can be used to access the column.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_INVALID`	The colTitle is invalid. It is either zero length or matches an existing column.
`FE_FAIL`	Unable to add column.
`FE_BAD_TYPE`	The Value Array has an invalid type.

Unpublished work. © 2025 Siemens

Remarks/Usage:

This method will automatically create **Model ID**, **Set ID** and **ID** columns in the Data Table if they do not exist - you should never attempt to create these columns yourself.

Example:

```
Sub Main
Dim App As femap.model
Set App = feFemap

Dim r As femap.DataTable
Set r = App.feDataTable

id = Array( 1, 7, 3, 17 )
i = Array( 45, 833, -23, 124 )
x = Array( 1.2, 4.56, 3.987E-6, -0.0023 )
c  = Array( True, False, False, True )
t = Array( "Abc", "Title 1", "++ XX ++", "" )

Dim newi As Long
Dim newc As Long
Dim newx As Long
Dim newt As Long

r.AddColumn( True, False, FT_ELEM, 0, "IntCol", FCT_INT, 4, id, i, newi )
r.AddColumn( False,  False, FT_ELEM, 0,"Check", FCT_BOOL, 4, id, c, newc )
r.AddColumn( False,  False, FT_ELEM, 0,"Column Title ", FCT_DOUBLE, 4,
id, x, newx )
r.AddColumn( False,  False, FT_ELEM, 0, "Names", FCT_STRING, 4, id, t,
newt )
End Sub
```

ID	IntCol	Check	Column Title	Names
1	45	☑	1.2	Abc
7	833	☐	4.56	Title 1
3	-23	☐	3.987e-006	++ XX ++
17	124	☑	-0.0023	

AddDataColumn

Unpublished work. © 2025 Siemens

AddDataColumn

(bHidden, colTitle, ColType, nRows, Rows, value, nNewColumnID)

Description:

This method creates a new column and updates values in it by row index.

Input:

`BOOL bHidden`	If True, the column is created hidden. If False, the column is visible.
`STRING colTitle`	The title for the column - must be unique for every column.
`INT4 ColType`	The type of column to create (0=BOOL/Check Box, 1=Integer, 2=Real, 3=String). Boolean arrays will create a column that is displayed as Check Boxes (checked on if True).
`INT4 nRows`	The number of entries in the ID and Value arrays.
`INT4 Rows[0..nRows-1]`	The row indices of the rows to update.
`VARIANT Value`	This Variant contains an array with the values to be assigned to the rows selected by the corresponding indices from Rows. Depending on the option you specify in `ColType`, the variant must contain either a Boolean, Integer, Double or String array.

Output:

`INT4 nNewColumnID`	The index of the column that was created. This index is always based on the order the column was created, not the visible location, no matter how the property `VisibleColsOnly` is set. If `VisibleColsOnly=False`, this number can be used to access the column.

Return Code:

`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_INVALID`	The `colTitle` is invalid. It is either zero length or matches an existing column.

Unpublished work. © 2025 Siemens

FE_FAIL	Unable to add column.
FE_BAD_TYPE	The Value Array has an invalid type.
FE_NOT_EXIST	Data Table is currently empty.

Remarks/Usage:

None

Example:

```
' === First, use FEMAP to load Data Table with Nodal Output Vectors ===

Sub Main

   Dim App As femap.model

   Set App = feFemap()

   Dim d As femap.DataTable

   Set d = App.feDataTable

    Dim numRows As Long, ModelID As Variant, setID As Variant, ID As
Variant

   d.GetRowInfo( numRows, ModelID, setID, ID )

   If numRows < 1 Then End

   Dim Rows() As Long

       ReDim Rows( numRows-1 )

   Dim values() As Variant

       ReDim values( numRows-1 )

   Dim nCol As Long

   For i = 0 To numRows-1

       values(i) = i+1 + 1000

       Rows(i) = i

   Next

    ' === Data Table must be populated beforehand ===

    ' === Need unique title ===
```


Unpublished work. © 2025 Siemens

```
    rc= d.AddDataColumn( False, "API Column",FCT_INT,
numRows,Rows, values, nCol )

End Sub
```

UpdateColumnByID

UpdateColumnByID **(nColumn, entitySetID, colType, nRows, ID, Value)**	
Description:	
This method updates a column in the Data Table and adds data to it if selected IDs do not exist. Rows to update are selected by IDs.	
Input:	
`INT4 nColumn`	Index of the column to update.
`INT4 entitySetID`	The Set ID associated with these values being updated. Can be 0 to match any Set ID or use the default for new rows.
`INT4 colType`	The type of data being updated (0=Bool/Check Box, 1=Integer, 2=Real, 3=String).
`INT4 nRows`	The number of entries in the ID and Value arrays.
`INT4 ID[0..nRows-1]`	The entity IDs associated with the data in the corresponding location in *Value*. If a row with the same ID already exists in the Data Table, the associated *Value* is assigned to that row. If no row with that ID exists, a new row is created with the current Model ID, the specified `entitySetID`, and this ID, and the new row is assigned the value.
`VARIANT Value`	This Variant contains an array with the values to be assigned to the rows selected by the corresponding entity IDs from `ID`. Depending on the option you specify in `colType`, the variant must contain either a Boolean, Integer, Double or String array.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_FAIL`	Unable to update column.
`FE_BAD_TYPE`	The Value Array has an invalid type.
Remarks/Usage:	
None	
Example:	
None	

UpdateColumn

`UpdateColumn` **`( nColumn, colType, nRows, Rows, Value )`**	
Description:	
This method updates a column in the Data Table. Rows are specified by row indices.	
Input:	
`INT4 nColumn`	Index of the column to update.
`INT4 colType`	The type of data being updated (0=Bool/Check Box, 1=Integer, 2=Real, 3=String).
`INT4 nRows`	The number of entries in the ID and Value arrays.
`INT4 Rows[0..nRows-1]`	The row indices of the rows to update.
`VARIANT Value`	This Variant contains an array with the values to be assigned to the rows selected by the corresponding indices from Rows. Depending on the option you specify in `colType`, the variant must contain either a Boolean, Integer, Double or String array.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_FAIL`	Unable to update column.
`FE_BAD_TYPE`	The Value Array has an invalid type.
Remarks/Usage:	
This method is similar to `UpdateColumnByID` except that it cannot add additional rows to the table.	
Example:	
None	

AddEntityData

`AddEntityData` **`( bClear, entityType, entitySetID, entityIDSet )`**	
Description:	
This method adds entity data to the Data Table.	
Input:	
`BOOL bClear`	If True, all previously existing data in the Data Table is deleted.
`INT4 entityType`	The type of entities being added to the Data Table. For more information on Entity Types refer to the table in Entity Types.
`INT4 entitySetID`	The Set ID associated with these values being added. Can be 0 to use the default Set ID.
`INT4 entityIDSet`	The ID of a Set object that contains the IDs of the entities to add. If `entityIDSet` is negative,

Unpublished work. © 2025 Siemens

	then just a single entity is added whose id is the absolute value of `entityIDSet`.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_FAIL`	Unable to add entities.
Remarks/Usage:	
None	
Example:	

The following routine adds all Plate elements to the Data Table...

```
Sub Main
Dim App As femap.model
Set App = feFemap
Dim r As femap.DataTable
Set r = App.feDataTable
Dim s As femap.Set
Set s = App.feSet
s.AddRule( FET_L_PLATE, FGD_ELEM_BYTYPE )
r.AddEntityData( True, FT_ELEM, 0, s.ID )
End Sub
```

AddOutputV2

AddOutputV2

(nOutputSet, nOutputVector, nNewColumnID)

Description:
This method adds data from an Output Vector to a column in the Data Table. This method replaces `AddOutput`. For more information see, **Working with Output in Femap v2020.1 and above**.
Input:

Unpublished work. © 2025 Siemens

`INT4 nOutputSet`	The Output Set that contains the data to add.
`INT4 nOutputVector`	The ID of the Output Vector that contains the data to add.
Output:	
`INT4 nNewColumnID`	The column ID of the column that is created.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed, or does not contain rows with Nodes/Elements.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_NOT_EXIST`	The selected Output Set/Output Vector does not exist.
`FE_FAIL`	Unable to add entities.
Remarks/Usage:	
This method provides a simple way to add analysis results to the data table, however the table must be preloaded with the nodes/elements where you want to retrieve the results. Only the specified Output Vector is loaded. There are no options to also include results for other components, plys, corners, etc... The Data Table may contain results from different/multiple models, however it will be loaded with results from the same Output Set/Output Vector in each respective model - there is no way using this method to choose different Output Sets for the different models.	
Example:	
None	

AddMassProperties

`AddMassProperties`
`( void )`
Description:

Unpublished work. © 2025 Siemens

Adds mass properties to the existing entities in the table.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_BAD_TYPE`	Invalid entity type or no data in the Data Table.
Remarks/Usage:	
This method simply calls the **Add Mass Properties** command from the Data Table toolbar. The Data Table must contain one or more Curves, Surfaces, Solids, Elements, Properties or Materials.	
Example:	
None	

AddElementChecks

`AddElementChecks`
`( void )`
Description:
Adds Femap element checks to the existing entities in the table.
Input:
None
Output:
None
Return Code:

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_BAD_TYPE`	Invalid entity type or no data in the Data Table.
Remarks/Usage:	
This method simply calls the **Add Femap Element Checks** command from the **Data Table** toolbar. The Data Table must contain one or more Elements.	
Example:	
None	

AddNastranElementChecks

`AddNastranElementChecks`
`( void )`
Description:
Adds NX Nastran element checks to the existing entities in the table.
Input:
None
Output:
None
Return Code:
`FE_NOT_AVAILABLE` — The Data Table is not currently displayed.
`FE_SECURITY` — The Data Table is locked and can not be updated.
`FE_BAD_TYPE` — Invalid entity type or no data in the Data Table.
Remarks/Usage:
This method simply calls the **Add Nastran Element Checks** command from the **Data Table** toolbar. The Data Table must contain one or more Elements.

Unpublished work. © 2025 Siemens

Example:
None

AddMeshAssociativity

`AddMeshAssociativity`	
`( void )`	
Description:	
Adds mesh associativity data to the existing entities in the table.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
`FE_BAD_TYPE`	Invalid entity type or no data in the Data Table.
Remarks/Usage:	
This method simply calls the **Add Mesh Associativity** command from the **Data Table** toolbar. The Data Table must contain one or more Points, Curves, Surfaces or Solids.	
Example:	
None	

SetTextColor

Unpublished work. © 2025 Siemens

SetTextColor	
(nColumn, numRows, Rows, nRed, nGreen, nBlue)	
Description:	
Changes the color of the text displayed in one or more rows in a selected column.	
Input:	
`INT4 nColumn`	The index of the column to update.
`INT4 numRows`	The number of entries in th Rows array.
`INT4 Rows[0..numRows-1]`	The indices of the rows to update.
`INT4 nRed`	The Red level (0-255).
`INT4 nGreen`	The Green level (0-255).
`INT4 nBlue`	The Blue level (0-255).
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_FAIL`	Unable to update Data Table.
Remarks/Usage:	
None	
Example:	
None	

SetBackgroundColor

Unpublished work. © 2025 Siemens

SetBackgroundColor **(nColumn, numRows, Rows, nRed, nGreen, nBlue)**	
Description:	
Changes the color of the background displayed in one or more rows in a selected column.	
Input:	
`INT4 nColumn`	The index of the column to update.
`INT4 numRows`	The number of entries in th Rows array.
`INT4 Rows[0..numRows-1]`	The indices of the rows to update.
`INT4 nRed`	The Red level (0-255).
`INT4 nGreen`	The Green level (0-255).
`INT4 nBlue`	The Blue level (0-255).
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_FAIL`	Unable to update Data Table.
Remarks/Usage:	
None	
Example:	
None	

DataTable Object Grouping Methods

The following methods are available to access and manipulate Data Table grouping.

Unpublished work. © 2025 Siemens

AddGroup

AddGroup **(nColumn, bShowInGroupOnly, bSortAscending)**	
Description:	
This method updates the Data Table to group by a selected column.	
Input:	
`INT4 nColumn`	The index of a column to add to Grouping.
`BOOL bShowInGroupOnly`	If True, the column is removed from being visible in the table, and is only visible in grouping and the group headers. If False, the column is still used for grouping, but is also visible in the table.
`BOOL bSortAscending`	True=Sort in Ascending order, False=Descending order.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The selected column does not exist.
Remarks/Usage:	
This method can be used multiple times to add additional columns to the grouping order. If a column was already in the grouping order it is moved to the lowest level. Calling this method will automatically show the `GroupBy` header. You can call `ShowGroup` to hide it, if necessary.	
Example:	
None	

ClearGroup

Unpublished work. © 2025 Siemens

ClearGroup	
(void)	
Description:	
This method removes any grouping order.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
None	
Example:	
None	

GetGroup

GetGroup	
(numColumns, Columns)	
Description:	
This method retrieves the indices of all columns used for grouping.	
Input:	
None	
Output:	
`INT4 numColumns`	The number of columns in the grouping order.

Unpublished work. © 2025 Siemens

`INT4 Columns[0..numColumns-1]`	The indices of the grouping columns.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	No grouping columns exist.
`FE_BAD_TYPE`	`VisibleColsOnly` is True. This method is only available when `VisibleColsOnly` is False because grouped/sorted columns do not need to be visible.
Remarks/Usage:	
None	
Example:	
None	

ShowGroup

`ShowGroup` **`( bVisible )`**	
Description:	
This method shows and hides the **Group By** header.	
Input:	
`BOOL bVisible`	True = Show Header, False = Hide Header.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

DataTable Object Sorting Methods

There following methods are available to access and manipulate Data Table sorting.

AddSort

AddSort **(nColumn, bSortAscending)**	
Description:	
This method updates the Data Table to sort by a selected column.	
Input:	
`INT4 nColumn`	The index of a column to sort by.
`BOOL bSortAscending`	True=Sort in Ascending order, False=Descending order.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	The selected column does not exist.
Remarks/Usage:	
This method can be used multiple times to add additional columns to the sorting order. If a column was already in the sorting order it is moved to the lowest level.	
Example:	
None	

Unpublished work. © 2025 Siemens

ClearSort

ClearSort	
(void)	
Description:	
This method removes any sorting order.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
None	
Example:	
None	

GetSort

GetSort	
(numColumns, Columns)	
Description:	
This method retrieves the indices of all columns used for sorting.	
Input:	
`INT4 numColumns`	The number of columns in the sorting order.

Unpublished work. © 2025 Siemens

`INT4 Columns[0..numColumns-1]`	The indices of the sorting columns.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	No sorting columns exist.
`FE_BAD_TYPE`	`VisibleColsOnly` is True. This method is only available when `VisibleColsOnly` is False because grouped/sorted columns do not need to be visible.
Remarks/Usage:	
None	
Example:	
None	

DataTable Object Selection Methods

There following methods are available to access and manipulate Data Table selection.

AddSelection

`AddSelection`
`( numRows, Rows, bSelect )`
Description:
This method updates the Data Table to select/deselect one or more rows.
Input:

`INT4 numRows`	The number of entries in the Rows array.

Unpublished work. © 2025 Siemens

`INT4 Rows[0..numRows-1]`	The indices of the rows to select/deselect.
`BOOL bSelect`	True = Select, False = Deselect Rows.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
This method can be used multiple times to add additional rows to the selection.	
Example:	
None	

ClearSelection

`ClearSelection`
`( void )`
Description:
This method removes selection from all rows.
Input:
None
Output:
None
Return Code:
`FE_NOT_AVAILABLE` — The Data Table is not currently displayed.
Remarks/Usage:
None

Unpublished work. © 2025 Siemens

Example:
None

GetSelection

`GetSelection` **`( numRows, Rows )`**	
Description:	
This method retrieves the indices of all rows that are selected.	
Input:	
None	
Output:	
`INT4 numRows`	The number of rows in the selection.
`INT4 Rows[0..numRows-1]`	The indices of the selected rows.
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
None	
Example:	
None	

DataTable Object Other Methods

There following methods are available to provide access to other functionality in the Data Table.

Lock

Unpublished work. © 2025 Siemens

Lock	
(bLock)	
Description:	
This method Locks or Unlocks the Data Table to allow it to be updated.	
Input:	
`BOOL bLock`	True = Lock, False = Unlock the Data Table.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
None	
Example:	
None	

Copy

Copy	
(bCopyAll)	
Description:	
This method copies the contents of the Data Table to the clipboard.	
Input:	
`BOOL bCopyAll`	True = Copy entire Data Table, False = Copy only selected rows (and header).
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Save

Save **(bCopyAll, fName, nFormat)**	
Description:	
This method copies the contents of the Data Table to a file.	
Input:	
`BOOL bCopyAll`	True = Copy entire Data Table, False = Copy only selected rows (and header).
`STRING fName`	The name of the file that you want to create.
`INT4 nFormat`	The format that you want to use when creating the file (0=Tab Delimitted Text, 1=Comma-Separated Text, 2=Rich Text Format - RTF).
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Clear

Clear	
(void)	
Description:	
This method clears the contents of the Data Table.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
Remarks/Usage:	
None	
Example:	
None	

DeleteRows

DeleteRows	
(numRows, Rows)	
Description:	
This method deletes specified rows from the Data Table.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 numRows`	The number of entries in the Rows array.
`INT4 Rows[0..numRows-1]`	The indices of the rows to delete.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_SECURITY`	The Data Table is locked and can not be updated.
Remarks/Usage:	
None	
Example:	
None	

Show

Show **(bShowAll)**	
Description:	
This method shows (draws) the specified contents of the Data Table.	
Input:	
`BOOL bShowAll`	True = Show all visible Rows, False = Show only selected rows.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Print

Print **(void)**	
Description:	
This method prints the visible contents of the Data Table.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	The Data Table is not currently displayed.
`FE_NOT_EXIST`	There are no rows in the Data Table.
Remarks/Usage:	
None	
Example:	
None	

Element Quality Objects

The Element Quality object provides access to the Element Quality checks available in Femap. It is not a model entity.

Use the `feElementQuality` method of the Femap Application object to access the Element Quality object.

Unpublished work. © 2025 Siemens

Element Quality Object Properties

Property	Description
Femap Element Checks	
`BOOL AspectRatioOn`	If True, the aspect ratio check is performed.
`REAL8 AspectRatioLimit`	Limit for aspect ratio check.
`BOOL TaperOn`	If True, element taper is checked.
`REAL8 TaperLimit`	Limit for taper check.
`BOOL AlternateTaperOn`	If True, the alternate element taper check is performed.
`REAL8 AlternateTaperLimit`	Limit for alternate taper check.
`BOOL InternalAngleOn`	If True, the element internal angle check is performed.
`REAL8 InternalAngleLimit`	Limit for internal angle check.
`BOOL SkewOn`	If True, the element skew check is performed.
`REAL8 SkewLimit`	Limit for skew check.
`BOOL WarpingOn`	If True, the element warping check is performed.
`REAL8 WarpingLimit`	Limit for warping check.
`BOOL NastranWarpingOn`	If True, the Nastran warping check is performed.
`REAL8 NastranWarpingLimit`	Limit for nastran warping check.
`BOOL TetCollapseOn`	If True, tetrahedral elements are checked for collapse.
`REAL8 TetCollapseLimit`	Limit for tet collapse check.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL JacobianOn`	If True, checks element Jacobian.
`REAL8 JacobianLimit`	Limit for Jacobian check.
`BOOL CombinedOn`	If True, checks for combined element distortion factor.
`REAL8 CombinedLimit`	Limit for combined check.
`BOOL ExplicitTimeStepOn`	If True, checks for Explicit Time Step factors.
`REAL8 ExplicitTimeStepLimit`	Limit for explicit timestep check.
NX Nastran Element Checks	
`BOOL NastranQuadSkewOn`	If True, checks element `QUAD SKEW`.
`REAL8 NastranQuadSkewLimit`	Limit for `QUAD SKEW` check.
`BOOL NastranQuadTaperOn`	If True, checks element `QUAD TAPER`.
`REAL8 NastranQuadTaperLimit`	Limit for `QUAD TAPER` check.
`BOOL NastranQuadWarpOn`	If True, checks element `QUAD WARP`.
`REAL8 NastranQuadWarpLimit`	Limit for `QUAD WARP` check.
`BOOL NastranQuadIAMinOn`	If True, checks element `QUAD IAMIN`.
`REAL8 NastranQuadIAMinLimit`	Limit for `QUAD IAMIN` check.
`BOOL NastranQuadAROn`	If True, checks element `QUAD AR` (Aspect Ratio).
`REAL8 NastranQuadARLimit`	Limit for `QUAD AR` (Aspect Ration) check.
`BOOL NastranQuadIAMaxOn`	If True, checks element `QUAD IAMAX`.
`REAL8 NastranQuadIAMaxLimit`	Limit for `QUAD IAMAX` check.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NastranTriaSkewOn`	If True, checks element `TRIA SKEW`.
`REAL8 NastranTriaSkewLimit`	Limit for `TRIA SKEW` check.
`BOOL NastranTriaIAMaxOn`	If True, checks element `TRIA IAMAX`.
`REAL8 NastranTriaIAMaxLimit`	Limit for `TRIA IAMAX` check.
`BOOL NastranTriaAROn`	If True, checks element `TRIA AR` (Aspect Ratio).
`REAL8 NastranTriaARLimit`	Limit for `TRIA AR` (Aspect Ratio) check.
`BOOL NastranTriaEPLROn`	If True, checks element `TRIA EPLR`.
`REAL8 NastranTriaEPLRLimit`	Limit for `TRIA EPLR` check.
`BOOL NastranTetraAROn`	If True, checks element `TETRA AR`.
`REAL8 NastranTetraARLimit`	Limit for `TETRA AR` check.
`BOOL NastranTetraEPLROn`	If True, checks element `TETRA EPLR`.
`REAL8 NastranTetraEPLRLimit`	Limit for `TETRA EPLR` check.
`BOOL NastranTetraDetJOn`	If True, checks element `TETRA DETJ`.
`REAL8 NastranTetraDetJLimit`	Limit for `TETRA DETJ` check.
`BOOL NastranTetraEPIAOn`	If True, checks element `TETRA EPIA`.
`REAL8 NastranTetraEPIALimit`	Limit for `TETRA EPIA` check.
`BOOL NastranHexAROn`	If True, checks element `HEX AR`.

Unpublished work. © 2025 Siemens

Property	Description
REAL8 NastranHexARLimit	Limit for HEX AR check.
BOOL NastranHexEPLROn	If True, checks element HEX EPLR.
REAL8 NastranHexEPLRLimit	Limit for HEX EPLR check.
BOOL NastranHexDetJOn	If True, checks element HEX DETJ.
REAL8 NastranHexDetJLimit	Limit for HEX DETJ check.
BOOL NastranHexEPIAOn	If True, checks element HEX EPIA.
REAL8 NastranHexEPIALimit	Limit for HEX EPIA check.
BOOL NastranHexWarpOn	If True, checks element HEX WARP.
REAL8 NastranHexWarpLimit	Limit for HEX WARP check.
BOOL NastranPenAROn	If True, checks element PENTA AR.
REAL8 NastranPenARLimit	Limit for PENTA AR check.
BOOL NastranPenEPLROn	If True, checks element PENTA EPLR.
REAL8 NastranPenEPLRLimit	Limit for PENTA EPLR check.
BOOL NastranPenDetJOn	If True, checks element PENTA DETJ.
REAL8 NastranPenDetJLimit	Limit for PENTA DETJ check.
BOOL NastranPenEPIAOn	If True, checks element PENTA EPIA.
REAL8 NastranPenEPIALimit	Limit for PENTA EPIA check.
BOOL NastranPenWarpOn	If True, checks element PENTA WARP.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NastranPenWarpLimit`	Limit for `PENTA WARP` check.
`BOOL NastranPyrAROn`	If True, checks element `PYR AR`.
`REAL8 NastranPyrARLimit`	Limit for `PYR AR` check.
`BOOL NastranPyrEPLROn`	If True, checks element `PYR EPLR`.
`REAL8 NastranPyrEPLRLimit`	Limit for `PYR EPLR` check.
`BOOL NastranPyrDetJOn`	If True, checks element `PYR DETJ`.
`REAL8 NastranPyrDetJLimit`	Limit for `PYR DETJ` check.
`BOOL NastranPyrEPIAOn`	If True, checks element `PYR EPIA`.
`REAL8 NastranPyrEPIALimit`	Limit for `PYR EPIA` check.
`BOOL NastranPyrWarpOn`	If True, checks element `PYR WARP`.
`REAL8 NastranPyrWarpLimit`	Limit for `PYR WARP` check.

Element Quality Methods

The following methods are available.

Clear

Unpublished work. © 2025 Siemens

Clear	
(void)	
Description:	
This method resets all check flags and clears all quality limit values.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SetModelDefaults

SetModelDefaults	
(void)	
Description:	
This method update the quality flags and limit values of the Model with the values stored in the current Quality Object properties.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

None	
Remarks/Usage:	
None	
Example:	
None	

GetModelDefaults

`GetModelDefaults`
`( void )`
Description:
This method loads the quality flags and limit values from the Femap model into the current Quality Object.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
None
Example:
None

CheckQuality

Unpublished work. © 2025 Siemens

CheckQuality

(nElemSet, bList, bGroup)

Description:

This function checks a set of elements to see if any of them are distorted based on the quality flags and limit values defined in the Quality Object properties for the Femap Element Checks. A Femap Set object is returned filled with the element IDs that failed the selected quality checks.

Input:

`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`BOOL bList`	If True, distorted elements are listed in the Messages and Lists window.
`BOOL bGroup`	If True, a group is made of the distorted elements.

Output:

None	

Return Code:

The Set object is returned in place of the return code.	

Remarks/Usage:

This function is just like the **Tools** → **Check** → **Distortion** command when using the Femap tab. The checks performed and the limit values are identical to that command.

Example:

None

CheckNastranQuality

CheckNastranQuality

(nElemSet, bList, bGroup)

Description:

Unpublished work. © 2025 Siemens

This function checks a set of elements to see if any of them are distorted based on the quality flags and limit values defined in the Quality Object properties for the NX Nastran Element Checks. A Femap Set object is returned filled with the element IDs that failed the selected quality checks.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`BOOL bList`	If True, distorted elements are listed in the Messages and Lists window.
`BOOL bGroup`	If True, a group is made of the distorted elements.
Output:	
None	
Return Code:	
The Set object is returned in place of the return code.	
Remarks/Usage:	
This function is just like the **Tools → Check → Distortion** command when using the NX Nastran tab. The checks performed and the limit values are identical to that command.	
Example:	
None	

GetAspectRatio

`GetAspectRatio`
`( nElemID, pdValue )`
Description:
This method calculates and returns the quality value for the desired element.
Input:

`INT4 nElemID`	ID of element to check.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

AspectRatio

`AspectRatio` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Once you use this method you can use the returned Sort object to sort the elements by quality and find the worst ones.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetTaper

GetTaper	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Taper

Taper	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetAlternateTaper

`GetAlternateTaper` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

AlternateTaper

`AlternateTaper` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetInternalAngle

Unpublished work. © 2025 Siemens

GetInternalAngle	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

InternalAngle

InternalAngle	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.

Unpublished work. © 2025 Siemens

`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetSkew

`GetSkew`
`( nElemID, pdValue )`
Description:
This method calculates and returns the quality value for the desired element.
Input:
`INT4 nElemID` — ID of element to check.
Output:
`REAL8 pdValue` — Quality value.
Return Code:
None
Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

Skew

Skew	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetWarping

GetWarping
(nElemID, pdValue)
Description:
This method calculates and returns the quality value for the desired element.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Warping

`Warping` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetNastranWarping

`GetNastranWarping` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranWarping

Unpublished work. © 2025 Siemens

NastranWarping **(nElemSet, dLimitVal)**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetTetCollapse

GetTetCollapse **(nElemID, pdValue)**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.

Unpublished work. © 2025 Siemens

Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

TetCollapse

`TetCollapse` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetJacobian

GetJacobian **(nElemID, pdValue)**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Jacobian

Jacobian **(nElemSet, dLimitVal)**
Description:

Unpublished work. © 2025 Siemens

This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetCombined

`GetCombined` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Combined

`Combined` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetExplicitTimeStep

Unpublished work. © 2025 Siemens

GetExplicitTimeStep	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

ExplicitTimeStep

ExplicitTimeStep	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.

Unpublished work. © 2025 Siemens

`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranQuadSkew

`GetNastranQuadSkew`
`( nElemID, pdValue )`
Description:
This method calculates and returns the quality value for the desired element.
Input:

`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

NastranQuadSkew

NastranQuadSkew **(nElemSet, dLimitVal)**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranQuadTaper

GetNastranQuadTaper **(nElemID, pdValue)**
Description:
This method calculates and returns the quality value for the desired element.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranQuadTaper

`NastranQuadTaper`
`( nElemSet, dLimitVal )`
Description:
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.
Input:

`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetNastranQuadWarp

`GetNastranQuadWarp` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranQuadWarp

Unpublished work. © 2025 Siemens

NastranQuadWarp	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranQuadIAMin

GetNastranQuadIAMin	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.

Unpublished work. © 2025 Siemens

Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranQuadIAMin

`NastranQuadIAMin` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetNastranQuadAR

GetNastranQuadAR	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranQuadAR

NastranQuadAR
(nElemSet, dLimitVal)
Description:

Unpublished work. © 2025 Siemens

This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranQuadIAMax

`GetNastranQuadIAMax` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

NastranQuadIAMax

`NastranQuadIAMax` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranTriaSkew

Unpublished work. © 2025 Siemens

GetNastranTriaSkew	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranTriaSkew

NastranTriaSkew	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.

Unpublished work. © 2025 Siemens

`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranTriaIAMax

`GetNastranTriaIAMax`
`( nElemID, pdValue )`
Description:
This method calculates and returns the quality value for the desired element.
Input:

`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

NastranTriaIAMax

NastranTriaIAMax **(nElemSet, dLimitVal)**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranTriaAR

GetNastranTriaAR **(nElemID, pdValue)**
Description:
This method calculates and returns the quality value for the desired element.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranTriaAR

`NastranTriaAR` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetNastranTriaEPLR

`GetNastranTriaEPLR` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranTriaEPLR

Unpublished work. © 2025 Siemens

NastranTriaEPLR	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranTetraAR

GetNastranTetraAR	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.

Unpublished work. © 2025 Siemens

Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranTetraAR

`NastranTetraAR` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetNastranTetraEPLR

GetNastranTetraEPLR **(nElemID, pdValue)**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranTetraEPLR

NastranTetraEPLR **(nElemSet, dLimitVal)**
Description:

Unpublished work. © 2025 Siemens

This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranTetraDetJ

`GetNastranTetraDetJ` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

NastranTetraDetJ

`NastranTetraDetJ` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranTetraEPIA

Unpublished work. © 2025 Siemens

GetNastranTetraEPIA	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranTetraEPIA

NastranTetraEPIA	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.

Unpublished work. © 2025 Siemens

`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranHexAR

`GetNastranHexAR` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

NastranHexAR

NastranHexAR **(nElemSet, dLimitVal)**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranHexEPLR

GetNastranHexEPLR **(nElemID, pdValue)**
Description:
This method calculates and returns the quality value for the desired element.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranHexEPLR

`NastranHexEPLR`
`( nElemSet, dLimitVal )`
Description:
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.
Input:

`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetNastranHexDetJ

`GetNastranHexDetJ`	
`( nElemID, pdValue )`	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranHexDetJ

Unpublished work. © 2025 Siemens

NastranHexDetJ	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranHexEPIA

GetNastranHexEPIA	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.

Unpublished work. © 2025 Siemens

Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranHexEPIA

`NastranHexEPIA` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetNastranHexWarp

GetNastranHexWarp **(nElemID, pdValue)**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranHexWarp

NastranHexWarp **(nElemSet, dLimitVal)**
Description:

Unpublished work. © 2025 Siemens

This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPenAR

`GetNastranPenAR` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

NastranPenAR

`NastranPenAR` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPenEPLR

Unpublished work. © 2025 Siemens

GetNastranPenEPLR	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranPenEPLR

NastranPenEPLR	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.

Unpublished work. © 2025 Siemens

`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPenDetJ

`GetNastranPenDetJ` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

NastranPenDetJ

NastranPenDetJ	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPenEPIA

GetNastranPenEPIA
(nElemID, pdValue)
Description:
This method calculates and returns the quality value for the desired element.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranPenEPIA

`NastranPenEPIA`
`( nElemSet, dLimitVal )`
Description:
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.
Input:

`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetNastranPenWarp

`GetNastranPenWarp` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranPenWarp

Unpublished work. © 2025 Siemens

NastranPenWarp **(nElemSet, dLimitVal)**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPyrAR

GetNastranPyrAR **(nElemID, pdValue)**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.

Unpublished work. © 2025 Siemens

Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranPyrAR

`NastranPyrAR` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetNastranPyrEPLR

GetNastranPyrEPLR **(nElemID, pdValue)**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranPyrEPLR

NastranPyrEPLR **(nElemSet, dLimitVal)**
Description:

Unpublished work. © 2025 Siemens

This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPyrDetJ

`GetNastranPyrDetJ` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

NastranPyrDetJ

`NastranPyrDetJ` **`( nElemSet, dLimitVal )`**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPyrEPIA

Unpublished work. © 2025 Siemens

GetNastranPyrEPIA	
(nElemID, pdValue)	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NastranPyrEPIA

NastranPyrEPIA	
(nElemSet, dLimitVal)	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.

Unpublished work. © 2025 Siemens

`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetNastranPyrWarp

`GetNastranPyrWarp` **`( nElemID, pdValue )`**	
Description:	
This method calculates and returns the quality value for the desired element.	
Input:	
`INT4 nElemID`	ID of element to check.
Output:	
`REAL8 pdValue`	Quality value.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

NastranPyrWarp

NastranPyrWarp **(nElemSet, dLimitVal)**	
Description:	
This method takes a element or set of elements and checks their quality against a limit value. A Femap Sort object is returned that contains the elements that have failed the check and their quality.	
Input:	
`INT4 nElemSet`	The ID of a set that contains the IDs of elements to check. Specify a negative number to check a single element.
`REAL8 dLimitVal`	Limit value for check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Geometry Preparation and Meshing Objects

The Geometry Preparation and Meshing object provides the functionality of the **Geometry Preparation** command. Currently this is limited to preparing geometry and mesh sizing.

Use the `feMesher` method of the Femap Application object to access the Geometry Preparation and Meshing object.

Unpublished work. © 2025 Siemens

Note

All properties that are shown in gray are not yet implemented. These properties should not be used. They may have unpredictable results and may be changed prior to their final implementation.

Geometry Preparation and Meshing Object Properties

Property	Description
`BOOL PrepareGeometry`	Turn Geometry Preparation On/Off for the `Apply()` and `UI()` methods.
`BOOL MeshSizing`	Turn Mesh Sizing On/Off for the `Apply()` and `UI()` methods.
`BOOL Meshing`	Turn Meshing On/Off for the `Apply()` and `UI()` methods.
`BOOL RemovePreviousOnly`	If True, the `Apply()` method will simply remove previous Geometry Preparation regardless of other settings.
`BOOL Silent`	If Geometry Preparation is attempted on an already meshed solid, if this property is True the existing mesh will be deleted automatically, otherwise the user will be asked to confirm deletion of the mesh.
`BOOL SyncSizes`	Only used for the `UI()` method. If True, the geometry preparation size and the mesh size will always be the same.
Geometry Preparation Options	
`REAL8 PrepareSize`	The size used to prepare the geometry.
`REAL8 NarrowSizePct`	The percentage of the `PrepareSize` which is used to detect narrow regions of surfaces. (Default=10%)
`REAL8 CurveSuppressPct`	The percentage of the `PrepareSize` below which curves are considered to be very short and are suppressed. (Default=5%)
`REAL8 NarrowAngle`	The angle in degrees below which the corner of a surface is considered to be narrow and a candidate for removal.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 FeatureEdgeAngle`	The angle in degrees between the normals of two adjacent surfaces at a given curve. Angles above the specified value are considered to be important *features* and attempts are made to preserve these edges. In some cases however surfaces may still be combined across edges that exceed this angle.
`REAL8 CombinedCurveAngle`	The angle in degrees between the ending tangents of two curves that meet at a common point. If the curves meet smoothly (tangent to each other), they would have a 0.0 value. All angles below the specified value are candidates for creating combined curves during Geometry Preparation.
`BOOL GeometryCleanup`	If True, initial processing is done during Geometry Preparation to remove self-intersections, discontinuities and other numerical problems in the geometry.
`BOOL SuppressVoids`	If True, any internal void regions in solids being prepared are suppressed - regardless of size.
`BOOL DetectClosePoints`	If True, extra processing is done to detect and improve locations in surfaces which may only be narrow at one point rather than between edges.
`BOOL CutSlivers`	If True, as narrow regions (slivers) are being combined, they may be further cut into multiple pieces to better join with adjacent surfaces.
`BOOL ProcessBlends`	If True, surfaces that are part of blend chains and which are short along the length of the chain are combined with neighboring surfaces in the chain to make better meshing areas.
`BOOL CombineSmallSurfaces`	If True, very small surfaces are combined with neighboring surfaces just like any larger sliver/ narrow surface. If False, very small surfaces are simply suppressed along with all of their curves.
`BOOL RemovePreviousPrep`	If True and the solid has previously been through Geometry Preparation, that previous preparation is removed prior to processing it again.
`BOOL DeletePreviousMesh`	If True, any previous mesh on the part is deleted prior to Geometry Preparation.
`INT4 FixedGroupID`	The ID of a Group that contains Points, Curves and Surfaces to ignore during Geometry Preparation.

Unpublished work. © 2025 Siemens

Property	Description
	This is used to prevent Geometry Preparation from modifying critical areas of the model.
Primary Mesh Sizing Options	
`REAL8 MeshSize`	The size used for Mesh Sizing.
`REAL8 InteriorGrowthFactor`	A multiplier of the Mesh Size used to specify growth of the mesh interior to the surface edges.
`INT4 LengthBasedSizing`	The type of mesh sizing to apply. (0=Parametric, 1=Equal Length, 2=Parametric/Equal Length).
Additional Mesh Sizing Options	
`BOOL MinElemOnEdgeOn`	If True, `MinElemOnEdge` is used.
`BOOL MaxAngleToleranceOn`	If True, `MaxAngleTolerance` is used.
`BOOL MaxElemSmallFeatureOn`	If True, `MaxElemSmallFeature` is used.
`BOOL MaxPctSmallFeatureOn`	If True, `MaxPctSmallFeature` is used.
`BOOL VertexAspectRatioOn`	If True, `VertexAspectRatio` is used.
`BOOL MappedMeshingRefinement`	If True, additional modifications are done to the mesh sizes on a surface that favor mapped meshing. This does not guarantee that mapped meshing sizes will be created. If you must have a mapped mesh, you need to set that approach on the surface before you size.
`BOOL SuppressShortEdgesPctOn`	If True, `SuppressShortEdgesPct` is used.
`BOOL AutoBoundarySmallSurfPctOn`	If True, `AutoBoundarySmallSurfPct` is used.
`INT4 MinElemOnEdge`	The minimum number of elements to mesh along any edge. Only used if `MinElemOnEdgeOn` is True.
`REAL8 MaxAngleTolerance`	The maximum angle tolerance between the geometry tangent vector and the vector connecting consecutive mesh locations along the curves. Additional mesh locations are added along the curves to make sure this criterion is satisfied.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MaxElemSmallFeature`	The maximum number of elements on any feature that is identified as small (depends on `MaxPctSmallFeature`).
`REAL8 MaxPctSmallFeature`	The Percentage of the `MeshSize` that determines whether a feature is small.
`REAL8 VertexAspectRatio`	Used to adjust the mesh sizes at vertices that connect both short and long edges. If used, mesh sizes at vertices are adjusted so that the ratio of the length of the longest element to the shortest element at a vertex is within the specified limit.
`REAL8 SuppressShortEdgesPct`	Edges that are shorter than this percentage of `MeshSize` are suppressed during Mesh Sizing.
`REAL8 AutoBoundarySmallSurfPct`	Surfaces that are smaller than this percentage of `MeshSize` are combined with neighboring surfaces during Mesh Sizing.
`BOOL RefinementRatioOn`	If True, `RefinementRatio` is used.
`REAL8 RefinementRatio`	The curvature-based mesh refinement ratio. Adjusts the mesh sizes along curves in areas of high curvature.
`BOOL RefineSurfaceMesh`	Set to True to also consider curvature requirements when sizing the interior of the mesh. You must specify a nonzero `refineRatio`, or this option will be ignored.
`BOOL AdjacentSurfaceMatching`	Set to True to automatically find adjacent surfaces between multiple solids that you are sizing. The matched surfaces will automatically have a *Linked* approach defined.
`BOOL RemovePreviousSlaving`	Set to True to remove any previous *Linked* approaches on the surfaces of the solids that you are sizing.
`BOOL AdjustColors`	Set to True if you want to change the surface colors to indicate the status and success of the mesh sizing. This is especially important when sizing for hex meshing.
`BOOL SkipSizingOnSlaved`	If True any surfaces with slaved approaches are not re-sized.
Meshing Options	

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MeshType`	Type of mesh to create (0=Tri, 1=Quad, 2=Tet).
`BOOL MidsideNodes`	If True, midside nodes will be added to the mesh.
`BOOL AutoSelectTriMesher`	If True, the type of Tri mesher to use will be automatically selected based on the individual surfaces being meshed. If False, the mesher selected by `TriMesher` is used.
`INT4 TriMesher`	If `AutoSelectTriMesher` is False, then this selects the tri mesher to use (1=Subdivision, 2=Fast Tri, 3=3D Tri).
`BOOL MappedMeshingAllowed`	If True, mapped meshing is allowed.
`INT4 NodeMergeOption`	If tet meshing, 0=No node merging, 1=Merge new nodes, 2=Merge Nodes into Model, 3=Merge All nodes.
`REAL8 TetGrowthRatio`	The tetrahedral growth factor. Numbers larger than 1.0 mean that the elements on the inside of the solid will be larger than those on the outside. It is usually best to use a factor slightly larger than 1.0. Default=1.1.
`REAL8 InitialSizeRatio`	Another growth factor for tet meshing. Not normally used. Default=0.5001.
`BOOL AskAlreadyMeshed`	If True, the user is asked to confirm the action when meshing surfaces that are already meshed.
Surface Meshing Options	
`BOOL PreV10Meshing`	If True, chooses the `PreV10` meshers. Normally False.
`BOOL MidsideMoveToGeometry`	If True, midside nodes are moved toward the geometry (surfaces/curves) being meshed. If False, they are placed at the middle of the element edge.
`BOOL MidsideMaxDistortionAngleOn`	If True, movement of midside nodes toward the geometry is limited by the `MidsideMaxDistortionAngle`. If False, midside nodes are moved to the geometry regardless of distortion to the element edges.
`REAL8 MidsideMaxDistortionAngle`	The maximum angle formed by the edge of the element as the midside nodes are projected onto the geometry.

Unpublished work. © 2025 Siemens

Property	Description
	Only used if `MidsideMoveToGeometry` and `MidsideMaxDistortionAngleOn` are both True.
`BOOL ConnectEdgeNodesOn`	If True, `ConnectEdgeNodes` is used to connect a surface mesh to any existing nodes that have already been created on the surface edges.
`REAL8 ConnectEdgeNodes`	The tolerance used to find and connect to nodes that have already been meshed on surface edges.
`BOOL SmoothingLaplacian`	If True, Laplacian smoothing is used. If False, Centroidal smoothing is used.
`INT4 SmoothingIterations`	The number of smoothing iterations to perform. Set to 0 to skip smoothing.
`REAL8 SmoothingTolerance`	The smoothing tolerance. Smoothing will stop if node movement during a smoothing pass falls below this tolerance.
`INT4 OffsetMeshMode`	Allows creation of offsets on planar elements to move the centerline of the element away from the nodes. The specified location is offset by `OffsetMeshDistance` 0=Centerline, 1=Top Face, 2=Bottom Face.
`REAL8 OffsetMeshDistance`	The distance to offset elements from the nodes. Specify 0.0 to keep the centerline of the element at the nodes.
`REAL8 MapMaxAngleDeviation`	Maximum angle in degrees that the corner of a mapped mesh can deviate and still produce a mapped mesh.
`REAL8 MapMinCornerAngle`	The minimum corner angle above which a surface can be map meshed.
`BOOL MapEqualSidesOnly`	If True, a mapped mesh will be produced only if the opposite sides of the surface have an equal number of elements along their edges. If False, the number of elements along one edge direction must match, but the other can vary somewhat and still produce a mapped mesh.
`BOOL MapSubdivisions`	If True, and if the Subdivision mesher is used, then portions of a surface can be map meshed during the subdivision process.
`BOOL MapSplitQuadMap`	If True and if Tri meshing, then a Quad mesh is produced and split into triangles.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL MapAlternateBias`	If True and if Tri meshing, then the mesher will attempt to alternate the direction of triangles that are side-by-side to eliminate biasing of the mesh.
`BOOL MapRightBias`	When Tri meshing, setting to True or False reverses the diagonals of the first two triangles.
`BOOL PostMeshingCleanup`	If True, additional processing is performed on the completed surface mesh to improve element quality.
`INT4 QuadEdgeLayers`	When Quad meshing, this provides the ability to *pave* a number of element layers around the surface edges. Must be set to 0=Off, 1, 2 or 3.
`REAL8 CutQuadDeviationAngle`	If Quad meshing, quads with corner angles that deviate from 90 deg by more than this amount are split into triangles.
`INT4 MinElemBetweenBoundary`	For the Subdivision mesher, this attempts to ensure a minimum number of elements between opposite surface edges.
`REAL8 MaxElemAspectRatio`	For the Subdivision mesher, controls the interior of the mesh by adding more or less elements to attempt to improve aspect ratio.
`BOOL QuickCutBoundaryOn`	For the Subdivision mesher, setting this to True will reduce the number of possible subdivision combinations that are considered, potentially reducing meshing time.
`INT4 QuickCutMaxNodes`	If `QuickCutBoundaryOn=True` and using the Subdivision mesher, this is the maximum number of nodes on the boundary of a surface subdivision where the quicker subdivision methods will be used.
`BOOL AttributePropOnly`	If True, and the surface has meshing attributes, then only the attribute property will be used and all other options will use their current settings. If False, all options will come from the surface meshing attributes.
Tet Meshing Options	
`BOOL TetStatusMessages`	If True, additional status messages are written to the Message Window during the Tet Meshing process.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL TetQualityToDataTable`	If True, and the Data Table is open, the Tet elements and their element quality are placed in the Data Table during meshing.
`BOOL TetCheckSurfaceElem`	If True, the surface elements are checked to see that all element normals are consistent, there are no free edges and their are no interior walls (non-manifold).
`BOOL TetDeleteSurfaceElem`	If True, the surface mesh is deleted when the Tet mesh is created.
`BOOL TetSmoothSolidElemOn`	If True, the Tet Elements will be smoothed.
`REAL8 TetSmoothSolidElem`	If `TetSmoothSolidElemOn=True`, then this specifies the tolerance at which smoothing will stop.
`BOOL TetMidsideMoveToGeometry`	Same as `MidsideMoveToGeometry`, except for midsides of Solid elements.
`BOOL TetMidsideMaxDistortionAngleOn`	Same as `MidsideMaxDistortionAngleOn`, except for midsides of Solid elements.
`REAL8 TetMidsideMaxDistortionAngle`	Same as `MidsideMaxDistortionAngle`, except for midsides of Solid elements.
`BOOL TetAdjustNodalPrecision`	If True, the nodal precision of the surface mesh nodes is reduced to 6 significant digits when they are supplied to the tet mesher.
`BOOL TetMeshVoids`	If True, internal voids will be meshed along with the solid portions of the model. Voids will only be meshed however if they are not suppressed using SuppressVoids.

Geometry Preparation and Meshing Object Methods

The following methods are available to Prepare, Size and Mesh Geometry.

Note

All methods that are shown in gray are not yet implemented. These methods should not be used. They may have unpredictable results and may be changed prior to their final implementation.

Unpublished work. © 2025 Siemens

AddSolid

AddSolid **(SetID, SetSizes)**	
Description:	
This method selects one or more solids to be processed by the other methods in this object.	
Input:	
`INT4 SetID`	The ID of a set that contains a list of Solid IDs that will be processed. Alternatively, if you specify a negative value, this is simply the ID of the single Solid to process.
`BOOL SetSizes`	If True, the solid(s) are evaluated and default geometry preparation and mesh sizes are determined based on these solids. If False, default sizes are based on the overall model sizes, or any changes you have made to the defaults prior to calling this method.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	The Solid(s) specified do not exist.
Remarks/Usage:	
Typically this is the first method called when using this object.	
Example:	
None	

AddSurface

Unpublished work. © 2025 Siemens

AddSurface	
(SetID, SetSizes)	
Description:	
This method selects one or more surfaces to be processed by the other methods in this object.	
Input:	
`INT4 SetID`	The ID of a set that contains a list of Surface IDs that will be processed. Alternatively, if you specify a negative value, this is simply the ID of the single Surface to process.
`BOOL SetSizes`	If True, the Surface(s) are evaluated and default geometry preparation and mesh sizes are determined based on these Surfaces. If False, default sizes are based on the overall model sizes, or any changes you have made to the defaults prior to calling this method.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	The Surface(s) specified do not exist.
Remarks/Usage:	
In most cases, `AddSolid()` is used instead of `AddSurface()`.	
Example:	
None	

Clear

Clear	
( )	
Description:	

Unpublished work. © 2025 Siemens

This method clears all properties and resets them to default values.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SetSizes

`SetSizes` **`( PrepareSize, MeshSize, InteriorGrowthFactor )`**	
Description:	
This method provides a way to easily set the mesh sizes in a single call.	
Input:	
`REAL8 PrepareSize`	Sets the `PrepareSize` property.
`REAL8 MeshSize`	Sets the `MeshSize` property.
`REAL8 InteriorGrowthFactor`	Sets the `InteriorGrowthFactor`.
Output:	
None	
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
Calling this method is identical to setting the three properties directly.
Example:
None

HasPrevious

`HasPrevious`
 	
`( )`	

Description:	
This method indicates whether the selected solid has previous geometry preparation.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Solid has previous geometry preparation.
`FE_FAIL`	Solid does not have previous geometry preparation.
Remarks/Usage:	
None	
Example:	
None	

RemovePrevious

Unpublished work. © 2025 Siemens

`RemovePrevious`	
`( )`	
Description:	
This method removes previous geometry preparation.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remove previous preparation.
Remarks/Usage:	
None	
Example:	
None	

Prepare

`Prepare`	
`( )`	
Description:	
This method performs Geometry Preparation on the selected geometry using the current options/ properties.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	There are no solids selected.
Remarks/Usage:	
This method will prepare the selected geometry regardless of the setting of the `PrepareGeometry` property.	
Example:	
None	

Size

`Size`	
`( )`	
Description:	
This method performs Mesh Sizing on the selected geometry using the current options/properties.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	There are no surfaces or solids selected.
`FE_FAIL`	Mesh sizing was not successful.
Remarks/Usage:	
This method will apply Mesh Sizes regardless of the setting of the `MeshSizing` property.	
Example:	
None	

Mesh

Unpublished work. © 2025 Siemens

Mesh	
( )	
Description:	
This method Meshes the selected geometry using the current options/properties.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	There are no surfaces or solids selected.
`FE_FAIL`	Meshing was not successful.
Remarks/Usage:	
This method will mesh regardless of the setting of the Meshing property.	
Example:	
None	

Apply

Apply	
( )	
Description:	
This method can remove previous preparation, prepare geometry, apply mesh sizing and mesh selected geometry using the current options/properties.	
Input:	
None	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_NOT_EXIST`	There are no surfaces or solids selected.
`FE_FAIL`	Process was not successful.
Remarks/Usage:	
Four properties control the options that are executed by this method. If `RemovePreviousOnly` is True, then any previous geometry preparation is removed and no other steps are performed. If it is False (the default) then `PrepareGeometry`, `MeshSizing` and `Meshing` control whether each of those steps is performed. **Note** Meshing is currently not implemented in this object.	
Example:	
None	

UI

`UI` **`( MeshingOptions )`**	
Description:	
This method displays the dialog box user interface, just like the **Geometry Preparation** command.	
Input:	
`BOOL MeshingOptions`	If False, the dialog box showing Geometry Preparation and Mesh Sizing options will be used. If True, the dialog box will also contain Meshing Options. **Note** Since meshing is not currently implemented in this object, True should never be used.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_CANCEL`	User cancelled the dialog.
Remarks/Usage:	
This method uses the current object properties to initialize the dialogs and updates the properties to the user's selections.	
Example:	
None	

Geometry Interface Objects

The Geometry Interface object provides programmatic access to the capabilities of the **File** → **Import** → **Geometry** and **File** → **Export** → **Geometry** commands.

Use the `feGeometryInterface` method of the Femap Application object to create GeometryInterface objects.

> **Note**
> This tool provides access to both the most recent interfaces and also to *alternate* older interfaces which have been deprecated. You should only access alternate interfaces if the primary interfaces do not successfully handle your geometry. In future versions of Femap, the alternative interfaces will be removed.

Geometry Interface Object Properties

Property	Description
Primary Import Interfaces	
`REAL8 ImportGeometryScale`	Scale factor to apply to geometry as it is read. Defaults to the geometry scale factor in **File** → **Preferences**.
`STRING ImportTitlePrefix`	A text string that will prefix every solid title. If the body has not title this will be the entire title.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ImportLayer`	ID of the Layer assigned to Geometry.
`INT4 ImportColor`	The color assigned to Geometry if `ImportColorMode` is 2 (single color).
`INT4 ImportColorMode`	The approach used to assign colors (0=From File, 1=Active Colors, 2=Single Color/Use ImportColor property).
`BOOL ImportIncrementColor`	If true, colors are varied for each body in an assembly.
`BOOL ImportIncrementLayer`	If true, layers are varied for each body in an assembly.
`BOOL ImportUpdateExistingGeometry`	If true, an attempt is made to update and replace existing geometry while maintaining associated loads/constraints/etc. rather than creating all new geometry. This option is only used for the Parasolid, Solid Edge, NX and SolidWorks interfaces.
`BOOL ImportBodies`	If true, any solids in the file will be imported.
`BOOL ImportFacetted`	If true, any facetted bodies in the file will be impored.
`BOOL ImportSurfaces`	If true, any surfaces, including those not in a solid, will be imported.
`BOOL ImportCurves`	If true, any curves, including those not in a solid, will be imported.
`BOOL ImportPoints`	If true, any points, including those not in a solid will be imported. (See `ImportPointsCatV5` for a special case).
`BOOL ImportOptimize`	If true, an attempt is made to heal edges, remove redundant topology, and share geometry (See `ImportOptimizeCatV5` and `ImportOptimizeXT` for special cases).
`BOOL ImportSimplify`	If true, attempts to convert B-spline geometry to analytic definitions. (See `ImportSimplifyCatV5` for a special case).
`BOOL ImportShowDialog`	If true, properties from the object are used as the defaults when the `Import( )` method is called, however the options dialogs will still be shown so they can be modified by the user.

Unpublished work. © 2025 Siemens

Property	Description
Special Case Import Properties	
`BOOL ImportPointsCatV5`	Same as `ImportPoints` ... but you must use/update this property if using the Catia V5 interface.
`BOOL ImportOptimizeCatV5`	Same as `ImportOptimize` ... but you must use/update this property if using the Catia V5 interface.
`BOOL ImportSimplifyCatV5`	Same as `ImportSimplify` ... but you must use/update this property if using the Catia V5 interface.
`BOOL ImportOptimizeXT`	Same as `ImportOptimize` ... but you must use/update this property if using the Solid Edge, NX or Parasolid interfaces.
Catia V5 Import Only	
`BOOL CatV5ImportNoShow`	If true, entities on the Catia No Show Layer are imported.
`BOOL CatV5ImportInactiveLayers`	If true, entities on Catia Inactive Layers are imported.
`BOOL CatV5ImportExtTitles`	If true, titles will be appended to include the Part Number (including Embedded Revision), Nomenclature and Description information.
`STRING CatV5ImportAssySearchPath`	A text string that contains one or more paths (separated by semicolons) to search for parts when reading an assembly.
DXF Import Only	
`BOOL DXFImportText`	If true, text or notes in the file will be converted to text in your model.
`BOOL DXFImport3DFace`	If true, 3D Faces will be read and converted to planar elements.
`BOOL DXFImportBlocks`	If true, blocked entities will be read.
`BOOL DXFImportPolyMesh`	If true, polygon/polyface meshes will be read and converted to planar elements.
`BOOL DXFImportTraceBoundary`	If true, creates four lines to represent the rectangular boundary of a trace.
`BOOL DXFImportTraceCenterline`	If true, creates a line to represent the centerline of trances.
`BOOL DXFImportMergePoints`	If true, coincident endpoints of curves are merged.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 DXFImportMergeDistance`	Distance/tolerance used when merging coincident points.
`BOOL DXFImportLayer`	If true, colors and layers are read from the file.
`INT4 DXFImportFirstLayerID`	First Layer number used when importing geometry.
STL Import Only	
`BOOL STLImportSkipShortEdge`	If true, short edges (smaller than `STLImportShortEdgeLength`) are skipped.
`BOOL STLImportMergeNodes`	If true, merges coincident nodes that are created .
`BOOL STLImportCloseGaps`	If true, attempts to close gaps in the mesh
`BOOL STLImportPatranFile`	If true, accepts an STL file in the format created by Patran.
`REAL8 STLImportShortEdgeLength`	The length below which short edges can be skipped.
Alternate Import Interface Controls **Note** **These options and the alternate interfaces should only be used as a last resort if the primary interfaces are unable to properly import your geometry. The alternate interfaces are deprecated and will be removed in a future release.**	
`BOOL ImportUseAlternateInterface`	Set `ImportUseAlternateInterface` = true to use any of the alternate interfaces.
`BOOL ImportUseSimpleAlternateIGESInterface`	In addition to setting `ImportUseAlternateInterface` = true, to use the Old/Simple IGES interface set `ImportUseSimpleAlternateIGESInterface` = true.
Alternate Advanced Options for STEP Import Interface Only	
`BOOL STEPAltAdvImportSolids`	If true, read Class VI solids.
`BOOL STEPAltAdvImportFaceted`	If true, read Class V faceted solids.
`BOOL STEPAltAdvImportSurfaces`	If true, read Class IV Surfaces.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL STEPAltAdvImportWireframeC3`	If true, read Class III wireframe.
`BOOL STEPAltAdvImportWireframeC2`	If true, read Class II wireframe.
`BOOL STEPAltAdvImportCurveBoundSurface`	If true, read Class II Curve Bounded Surfaces.
`BOOL STEPAltAdvImportRectangleSurfaces`	If true, read Class II Rectangle Surfaces.
`BOOL STEPAltAdvImport2DCurves`	If true, read Class II 2D Curves on Surfaces.
`BOOL STEPAltAdvImportSmooth`	If true, attempt to smooth G1 discontinuities.
`BOOL STEPAltAdvImportExplodeAndStitch`	If true, explode and restitch Class VI solids.
`BOOL STEPAltAdvImportSimplify`	If true, convert geometry to analytic formulations.
`BOOL STEPAltAdvImportStitch`	If true, stitch Class II geometry into solids.
`BOOL STEPAltAdvImportFixOrient`	If true, attempt to fix closed surface orientations
`BOOL STEPAltAdvImportProE`	If true, accept files in an old Pro/ENGINEER format.
`BOOL STEPAltAdvImportCheckInput`	If true, check the geometry being read.
`BOOL STEPAltAdvImportCheckOutput`	If true, check the geometry being saved to the model.
Alternate ACIS Import Interface Only	
`REAL8 ACISAltImportScaleFactor`	The scale factor used by the Alternate ACIS interface to scale the model.
`BOOL ACISAltImportClean`	If true, attempt to cleanup sliver faces or other problem geometry.
`BOOL ACISAltImportHeal`	If true, attempt to close gaps and correct geometric inaccuracies.
`BOOL ACISAltImportCheck`	If true, check the converted model to find and report remaining problems.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL ACISAltImportFullCheck`	If true, additional checks are performed for surface discontinuities and other irregularities.
`BOOL ACISAltImportDeleteInvalid`	If true, geometry that does not pass the checking operations is deleted.
`BOOL ACISAltImportKeepFaces`	If true, unstitched surfaces are kept even if the entire solid cannot be converted.
Alternate CATIA Import Interface Only	
`BOOL CATIAAltImportAnalytic`	If true, converts surfaces with spline definitions into an analytic definition.
`BOOL CATIAAltImportTopology`	If true, CATIA topology is read. Typically false, in which case individual surfaces are read and restitched into a solid.
`BOOL CATIAAltImport3DTrimCurves`	If true, import 3D surface trim curves. If false, import 2D trim curves.
`BOOL CATIAAltImportPoints`	If true, standalone points are read.
`BOOL CATIAAltImportNoShow`	If true, No Show CATIA geometry is read.
`BOOL CATIAAltImportNoPick`	If true, No Pick CATIA geometry is read.
`BOOL CATIAAltImportWorkplanes`	If true, CATIA workplanes are read.
`BOOL CATIAAltImportStitch`	If true, an attempt will be made to stitch individual surfaces into a solid, otherwise they will remain unstitched.
`BOOL CATIAAltImportHeal`	If true, an attempt is made to repair tolerance and gap issues.
`BOOL CATIAAltImportCheck`	If true, geometry will be checked for validity as it is imported.
Alternate IGES Import Interface Only	
`BOOL IGESAltImportPoints`	If true, import points that are not part of a solid.
`BOOL IGESAltImportCurves`	If true, import curves that are not part of a solid.
`BOOL IGESAltImportUntrimmedSurfaces`	If true, import untrimmed surfaces that are not part of a solid.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL IGESAltImportTrimmedSurfaces`	If true, import trimmed surfaces that are not part of a solid.
`BOOL IGESAltImportSolids`	If true, import solids.
`INT4 IGESAltImportCustomizationType`	0=None, 1=Special format from older versions of AutoCAD.
`INT4 IGESAltImportTrimCurveOption`	0=Default Trim Curves, 1=2D Trim Curves, 2=3D Trim Curves.
Alternate Old/Simple IGES Import Interface Only	
`BOOL IGESAltOldImportColorLayer`	If true, read colors and layers from the file, otherwise use default colors and layers.
`BOOL IGESAltOldImportGeometryOnly`	If true, removes construction lines and other non-geometry information.
`BOOL IGESAltOldImportSurfaces`	If true, import surfaces.
`INT4 IGESAltOldImportTrimCurveOption`	0=Default Trim Curves, 1=2D Trim Curves, 2=3D Trim Curves.
`BOOL IGESAltOldImportMergePoints`	If true, merge any coincident points that are imported.
`REAL8 IGESAltOldImportMergeDistance`	Distance/Tolerance to be used to merge coincident points.
Alternate Import Interface Controls **Note** **These options should only be used as a last resort if the primary interfaces are unable to properly import your geometry. The alternate interfaces are deprecated and will be removed in a future release.**	
`BOOL ImportUseAlternateInterface`	Set `ImportUseAlternateInterface` = true to use any of the alternate interfaces. In addition, to use the Old/Simple IGES interface set `ImportUseSimpleAlternateIGESInterface` = true.
`BOOL ImportUseSimpleAlternateIGESInterface`	
Primary Export Interface Options	
`REAL8 ExportVersion`	The format version or type of file to export. If set to 0, the default format is used. For Parasolid and ACIS this can be set to a specific version number

Unpublished work. © 2025 Siemens

Property	Description
	of the toolkit. For STEP (0.0=AP203, 1.0=AP203 Edition 2, 2.0=AP214). For VRML (0.0=VRML 2.0/97, 1.0=VRML 1.0).
`BOOL ExportNonSolidPoints`	If true, points that are not part of a solid are exported.
`BOOL ExportNonSolidCurves`	If true, curves that are not part of a solid are exported.
`BOOL ExportForceManifold`	If true, nonmanifold geometry is broken into manifold parts prior to export.
Additional IGES Export Options	
`INT4 ExportIGESCustomization`	0=None, 1=AutoCAD, 2=JAMA-IS, 3=I-DEAS.
`INT4 ExportIGESTrimCurveOption`	0=Both 2D and 3D trim curves, 1=2D only, 2=3D only.
`INT4 ExportIGESGeometryFormat`	0=Solid, 1=Trimmed Surfaces, 2=Wireframe.
`BOOL ExportIGESCurvesAsSplines`	If true, analytic curves are exported as splines.
`BOOL ExportIGESSurfacesAsSplines`	If true, analytic surfaces are exported as splines.
`BOOL ExportIGESBoundedSurfaces`	If true, trimmed surfaces are exported as bounded surfaces.
`BOOL ExportIGESNonSolidPoints`	If true, export points that are not part of a solid.
`BOOL ExportIGESNonSolidCurves`	If true, export curves that are not part of a solid.
Additional VRML Export Options	
`BOOL ExportVRMLSolids`	If true, export solid geometry. If false, export the mesh (planar elements and free faces of solid meshes).
`BOOL ExportVRMLDeformed`	If true, exporting the mesh and showing a deformed view export using the deformed coordinates.
`BOOL ExportVRMLContour`	If true, exporting the mesh and showing a contour view, export the contour colors and a contour legend.

Unpublished work. © 2025 Siemens

Property	Description
	Note Most VRML viewers do not display the contours properly especially in regions of high gradients.
`BOOL ExportVRMLSingleColor`	If true, export using the single color defined by `ExportVMRLColor`. If false, use model colors.
`BOOL ExportVRMLBackground`	If true, export a background.
`INT4 ExportVRMLColor`	The color to be used if `ExportVRMLColor` = true.
`INT4 ExportVRMLBackgroundColor`	The background color if `ExportVRMLBackground` = true.

Geometry Interface Object Methods

The following methods are available for the Geometry Interface Object.

Clear

`Clear` **`( void )`**	
Description:	
Returns all properties in the object to their default values	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Import

Import **(nGeometryFormat, sFilename)**	
Description:	
Imports Geometry from a file.	
Input:	
`INT4 nGeometryFormat`	The format of the geometry in the file being imported. If 0, the format is automatically selected based on the filename extension. Otherwise Parasolid=30, DXF=10, StereoLithography=27, CATIA V4=38, CATIA V5=46, CATIA V6=53, Inventor=51, Rhino=52, Solid Edge=41, NX=43, JT=49, IGES=11, ACIS=29, STEP=34, PROE=39, SOLIDWORKS=47.
`STRING sFilename`	The complete path and filename of the file containing geometry to be imported.
Output:	
None	None
Return Code:	
`FE_INVALID`	An invalid geometry format was specified.
`FE_NOT_AVAILABLE`	The interface selected is not available on your system. Check your installation, some files may be missing.
`FE_NOT_EXIST`	The specified file does not exist.
`FE_FAIL`	Failed to import the specified file.

Unpublished work. © 2025 Siemens

`FE_CANCEL`	Dialog boxes were shown, and the Cancel button was pressed cancelling the import.
Remarks/Usage:	
None	
Example:	
None	

Export

`Export` **`( nGeometryFormat, sFilename, entitySET )`**	
Description:	
Exports selected geometry or a mesh to a file.	
Input:	
`INT4 nGeometryFormat`	The format of the file to be written. Parasolid=0, ACIS=1, StereoLithography=2, VRML=3, STEP=4, IGES=5, JT=6, Criteria=7, Topology Optimization=8
`STRING sFilename`	The complete path and filename of the file containing geometry to be imported.
`INT4 entitySET`	The ID of a Set object containing the entities to be exported. Alternatively, if you specify a negative number it is the ID of a single entity to export. For most formats, these are the IDs of Solids. However, for StereoLithography, Criteria, Topology Optimization and VRML (with ExportVRMLSolids=false) these are the IDs of elements.
Output:	
None	None
Return Code:	
`FE_INVALID`	An invalid geometry format was specified.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	The interface selected is not available on your system. Check your installation, some files may be missing.
`FE_FAIL`	Failed to export to the specified file.
`FE_CANCEL`	Dialog boxes were shown, and the Cancel button was pressed cancelling the export.
Remarks/Usage:	
You can display dialogs so the user can override the default properties, selections, or filename by doing the following: • `nGeometryFormat = 0 (FGXI_NONE)` ... display the dialog to select the format. • `sFilename = ""` ... an empty string...display dialog to select the filename. • `entitySET = 0` ... display dialog to select solids/elements to be exported.	
Example:	
None	

ReadFile Objects

The `ReadFile` object is used to read lines of text from a file and parse their contents. While these functions can be accomplished through other standard Basic or C++ functions, the capabilities of this object are often easier to use. The reading capability in this object also contains its own file buffering, which can be significantly faster than the native read functions.

`ReadFile` can handle different file formats. It reads lines from files using either the UNIX or Windows carriage control conventions. This means that UNIX files that are read on Windows can be transferred either via ASCII or Binary FTP and will work equally well.

Use the `feRead` method of the Femap Application object to create `ReadFile` Objects.

ReadFile Object Properties

Unpublished work. © 2025 Siemens

Property	Description
`STRING line[0..max_length]`	The current line of text that was read from the file. The maximum length is established when you open the file.
`BOOL truncated`	True if any line that has been read to that point was longer than the maximum length that you specified and was truncated.
`BOOL shared`	Set to True prior to calling *Open* if you want the file to be shared for both read and write, otherwise it is only shared for reading.
`BOOL shareddelete`	Set to True prior to calling *Open* if you want the file to be shared for read, write and delete, otherwise it is only shared for reading.
`BOOL Exclusive`	Set to True prior to calling *Open* if you want the file to be opened for exclusive access.

ReadFile Object Methods

Open

`Open` **`( filename, maxLineLength )`**	
Description:	
This function opens a file and prepares for reading.	
Input:	
`STRING filename`	The complete path to the file that you want to read.
`INT4 maxLineLength`	The maximum length of any line of text in this file. Actually, this is simply the maximum length that you are interested in reading. Longer lines may exist in the file, but they will simply be truncated at this length.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Unable to open the file.
`FE_NO_MEMORY`	Unable to allocate memory for file buffering.
Remarks/Usage:	
You must call this function prior to reading from a file. You should always call `Close` when you are finished.	
Example:	
None	

Read

Read
(void)
Description:
This function reads a line from an open file.
Input:
None
Output:
None
Return Code:
`FE_FAIL` — Unable to read from the file.
Remarks/Usage:
This function reads the next line from the open file. The text from that line (without any carriage control) is loaded into the *line* property and can be accessed from there. If you use the various parsing/field methods; you do not need to access the *line* property. You should always call `AtEOF` to check whether you have reached the end of the file.

Unpublished work. © 2025 Siemens

Example:
None

AtEOF

AtEOF **(void)**	
Description:	
This function indicates whether you have reached the end of the file.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	At the end of the file. You should stop reading.
`FE_FAIL`	Not at the end of the file; there is more data to be read.
Remarks/Usage:	
You should always use this check to determine when you reach the end of the file. Depending on how the file was created, `Read` can continue to read and return unwanted characters after reaching the true end of the file. This method will however indicate the end when you reach it.	
Example:	
None	

Close

Unpublished work. © 2025 Siemens

Close	
(void)	
Description:	
This function closes the current file and frees the memory used for file buffering.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
You should always call this method when you are done reading a file. You should not call this method if you have not successfully opened a file.	
Example:	
None	

Position

Position	
(void)	
Description:	
This function returns the current location in the file where you are reading.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`INT4 position`	The current file location.
Remarks/Usage:	
This function can be used with `Jump` to reposition to a specific file location for *rereading*.	
Example:	
None	

Jump

`Jump` **`( jumpto )`**	
Description:	
This function repositions the location in the file where the next read will occur.	
Input:	
`INT4 jumpto`	The position to start reading. This position is the offset from the first byte in the file. Setting `jumpto=0` positions to the beginning of the file.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
You should only specify `jumpto` as a value that you previously retrieved from a call to Position. Computing numbers yourself is prone to error because of handling of carriage control.	
Example:	
None	

JumpToEnd

Unpublished work. © 2025 Siemens

JumpToEnd	
(jumpto)	
Description:	
This function repositions the location in the file where the next read will occur.	
Input:	
`INT4 jumpto`	The position to start reading. This position is the offset from the last byte in the file. Setting `jumpto=0` positions to the end of the file.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
You should only specify `jumpto` as a value that you previously retrieved from a call to Position. Computing numbers yourself is prone to error because of handling of carriage control.	
Example:	
None	

Size

Size	
(nSize)	
Description:	
This function returns the size of the file in bytes.	
Input:	
`INT8 nSize`	The size of the file.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Unable to retrieve file size.
Remarks/Usage:	
Note The file size is an 8-byte integer - this allows support for files over 4 GByte.	
Example:	
None	

TimeCreated

`TimeCreated` **`( nTime )`**	
Description:	
Returns the time that the file was created.	
Input:	
`INT8 nTime`	The time the file was created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to retrieve file information.
Remarks/Usage:	
Note The file size is an 8-byte integer.	
Example:	
None	

Unpublished work. © 2025 Siemens

TimeWritten

TimeWritten

(nTime)

Description:

Returns the time that the file was written.

Input:

`INT8 nTime`	The time the file was created.

Output:

None	

Return Code:

`FE_FAIL`	Unable to retrieve file information.

Remarks/Usage:

Note
The file size is an 8-byte integer.

Example:

None

TimeAccessed

TimeAccessed

(nTime)

Description:

Returns the time that the file was accessed.

Input:

Unpublished work. © 2025 Siemens

`INT8 nTime`	The time the file was created.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to retrieve file information.
Remarks/Usage:	
Note The file size is an 8-byte integer.	
Example:	
None	

Skipped

`Skipped`	
`( void )`	
Description:	
Indicates whether the last read actually skipped over one or more lines.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	One or more lines were skipped.
`FE_FAIL`	No lines were skipped.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method will only indicate lines were skipped if one or more of the methods are being used that causes short, blank or searched lines to be skipped.
Example:
None

ShowStatus

`ShowStatus`	
`( on )`	
Description:	
Turn on or off the status bar in Femap during the read.	
Input:	
`BOOL on`	If True, the Status bar will be displayed showing the reading progress. If False, no Status bar will be displayed.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SetNoBlankLines

Unpublished work. © 2025 Siemens

SetNoBlankLines	
(on)	
Description:	
Indicate whether or not to read blank lines.	
Input:	
`BOOL on`	If True, blank lines will be skipped.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Blank lines are lines containing no text, or lines that because of other processing are reduced to being a blank line.	
Example:	
None	

SetNoShortLines

SetNoShortLines	
(on, minLineLength)	
Description:	
Indicate whether or not to read lines that are shorter than a specified length.	
Input:	
`BOOL on`	If True, short lines will be skipped.
`INT4 minLineLength`	If `BOOL on=True`, lines shorter than this length will be skipped.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SetTrimSpace

`SetTrimSpace` **`( on )`**	
Description:	
Indicate whether or not to trim all white space characters off of the front and back of each line.	
Input:	
`BOOL on`	If True, white space will be trimmed.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Turning this on will delete all leading and trailing spaces. You normally only want to do this if the line is to be parsed using a free format. White space on the *interior* (after the first non-white space character and before the last non-white space character) is not deleted.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetTrimEndSpace

SetTrimEndSpace	
(on)	
Description:	
Indicate whether or not to trim all white space characters off the end of each line.	
Input:	
`BOOL on`	If True, trailing white space will be trimmed.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Turning this on will delete all trailing spaces. This is just like `SetTrimSpace`, but only trims trailing, not leading white space.	
Example:	
None	

SetTrimFront

SetTrimFront	
(on, numChar)	
Description:	
Indicate whether or not to trim a certain number of characters off of the front of each line.	
Input:	
`BOOL on`	If True, leading characters will be trimmed.

Unpublished work. © 2025 Siemens

`INT4 numChar`	The number of characters to trim, regardless of content.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Turning this on trims `numChar` from the front of every line.	
Example:	
None	

SetUppercase

`SetUppercase`	
`( on )`	
Description:	
Returns all text in uppercase.	
Input:	
`BOOL on`	If True, all text will be returned in uppercase.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetSkip

SetSkip	
(on)	
Description:	
Indicates that lines containing the search strings are to be skipped.	
Input:	
`BOOL on`	If True, all lines that contain the search strings are skipped.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This function must be used in combination with the `SetSearchString` method. This cannot be used with the `SetSearch` method.	
Example:	
None	

SetSearch

SetSearch	
(on)	
Description:	
Indicates that lines containing the search strings are to be kept and others are to be skipped.	
Input:	
`BOOL on`	If True, all lines that do not contain the search strings are skipped.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This function must be used in combination with the `SetSearchString` method. This cannot be used with the `SetSkip` method.	
Example:	
None	

SetSearchString

`SetSearchString` **`( mainStr, orStr )`**	
Description:	
Searches for lines containing the specified strings.	
Input:	
`STRING mainStr`	The primary search string.
`STRING orStr`	A second alternate string which could be on the line instead of `mainStr`. This should normally be specified as a blank string to just use the primary search string.
Output:	
None	
Return Code:	
`FE_FAIL`	More than five search string pairs have been specified.
Remarks/Usage:	
This method can be called up to five times before reading to search for multiple strings.	

Unpublished work. © 2025 Siemens

Example:
None

ClearSearch

`ClearSearch`	
`( void )`	
Description:	
Removes any *search string(s)* currently set via `SetSearchString` so they will no longer be considered when reading the file.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	
`FE_FAIL`	
Remarks/Usage:	
After using this method, you can always use `SetSearchString` again to look for the same string(s) or different string(s), then simply use this method again to stop looking for those *search strings*.	
Example:	
None	

SetSkipAfterSearch

Unpublished work. © 2025 Siemens

SetSkipAfterSearch	
(lines)	
Description:	
Specifies the number of lines to skip after finding a line that contains one of the search strings.	
Input:	
`INT4 lines`	The number of lines to skip.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
In some cases you may want to skip certain headers in a file. You can search for a keyword in the header, then use this to skip the remaining header lines. If not specified, no lines are skipped.	
Example:	
None	

SetFormFeed

SetFormFeed	
(on)	
Description:	
Indicates that Form Feed (FF) characters should be treated as end-of-line carriage control.	
Input:	
`BOOL on`	If True, FF characters will also end a line.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
Normally this method can be ignored - most text files do not contain form feed characters. If you are reading the formatted output of some other program, however, this may be important if the program relies on writing a form feed to end certain lines.	
Example:	
None	

SetTabSize

`SetTabSize` **`( tabsize )`**	
Description:	
Specifies the number of spaces to convert tab characters into.	
Input:	
`INT4 tabsize`	The number of spaces to convert into.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The size that you specify will be overridden by any choice that the user has made in the Global property `Pref_ReadTabSize`.	
Example:	
None	

LineInit

Unpublished work. © 2025 Siemens

LineInit	
(maxLineLength)	
Description:	
Used to initialize the necessary properties required for line parsing if you are not reading a file.	
Input:	
`INT4 maxLineLength`	The maximum length of line that you will be parsing.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Normally you will not call this method. If you are reading a file and want to use the parsing/field capabilities, all of that will be handled automatically. If, however, you just want to use the parsing/field capabilities to parse text that you have from some other source, you can call this method to initialize the necessary properties. Then you simply set the *line* property to the text you want to parse and call the various *field* methods.	
Example:	
None	

Reparse

Reparse	
(void)	
Description:	
Used to indicate that you want to reparse the current line.	
Input:	
None	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
None	
Remarks/Usage:	
Use this method if you want to use multiple formats to parse the line. Call the field methods with the first format, change the format, call `Reparse`, and call the field routines again.	
Example:	
None	

FixedFormat

`FixedFormat` **`( numField, width )`**	
Description:	
Used to specify the number of fields and field widths for a fixed format read.	
Input:	
`INT4 numField`	The number of fields to parse from the line. Must be between 1 and 50.
`INT4 width[0..numField-1]`	The widths, in characters, of the fields to parse.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetFixedFormat

SetFixedFormat	
(void)	
Description:	
Indicates that you want to parse a line using fixed field widths.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
You must call `FixedFormat` to specify the field widths.	
Example:	
None	

SetFreeFormat

SetFreeFormat	
(void)	
Description:	
Indicates that you want to parse a line using free format.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
Free format indicates fields with either spaces or commas. Multiple commas create multiple fields. Multiple spaces do not.	
Example:	
None	

SetAutoFormat

`SetAutoFormat`	
`( void )`	
Description:	
Indicates that you want to parse a line using either fixed or free format.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This option checks the line for a comma. If any commas exist, free format is used; otherwise, fixed format is used.	
Example:	
None	

Field

Unpublished work. © 2025 Siemens

Field **(fieldID)**	
Description:	
Parses the current line and returns the indicated field as a text string.	
Input:	
`INT4 fieldID`	The ID of the field on the line (1..49).
Output:	
None	
Return Code:	
`STRING field`	The text in the selected field.
Remarks/Usage:	
There is a limit of 49 fields which can be read by this method.	
Example:	
None	

IntField

IntField **(fieldID, defaultValue)**	
Description:	
Parses the current line and returns the indicated field as an integer.	
Input:	
`INT4 fieldID`	The ID of the field on the line (1..49).
`INT4 defaultValue`	If the selected field is blank, the return value will be this value.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`INT4 field`	An integer representation of the text in the field.
Remarks/Usage:	
If the field is not blank, but does not contain an integer - either a real number or some other text - the return value will be 0, not the default value. There is a limit of 49 fields which can be read by this method.	
Example:	
None	

RealField

`RealField` **`( fieldID, defaultValue )`**	
Description:	
Parses the current line and returns the indicated field as a real number.	
Input:	
`INT4 fieldID`	The ID of the field on the line (1..49).
`REAL8 defaultValue`	If the selected field is blank, the return value will be this value.
Output:	
None	
Return Code:	
`REAL8 field`	An real number representation of the text in the field.
Remarks/Usage:	
If the field is not blank, but does not contain a real number - some other text - the return value will be 0.0, not the default value. There is a limit of 49 fields which can be read by this method.	

Unpublished work. © 2025 Siemens

Example:
None

BoolField

`BoolField` **`( fieldID, defaultValue )`**	
Description:	
Parses the current line and returns the indicated field as a boolean value.	
Input:	
`INT4 fieldID`	The ID of the field on the line (1..49).
`BOOL defaultValue`	If the selected field is blank, the return value will be this value.
Output:	
None	
Return Code:	
`BOOL field`	An Boolean representation of the text in the field.
Remarks/Usage:	
This function will interpret either “1”, “y”, or “yes” as True, and “0”, “n” or “no” as False. There is a limit of 49 fields which can be read by this method.	
Example:	
None	

Selector Objects

The Selector object provides access to the functionality of the selector in your model. It is not a model entity.

Use the `feSelector` method of the Femap Application object to access the Selector object.

Unpublished work. © 2025 Siemens

Selector Object Properties

Property	Description
`BOOL MultipleMode`	If True, the selector will be in **Select Multiple** mode, otherwise it will be in **Select Single Item** mode.
`BOOL AddMode`	If True, the selector will be in **Add to Selection** mode, otherwise it will be in **Remove From Selection** mode.
`INT4 DrillMode`	0=Drill Off (normal selection mode), 1=Drill Query, 2=Drill Front.
`BOOL RelatedMode`	If True, then when an entity is selected, all related entities will also be selected (for example, selecting an element will also select its nodes, properties, materials...)
`INT4 SelectEntity`	The entity type that is being selected.
`BOOL Active`	If True, the selector is active, otherwise it is inactive.
`BOOL ShowInModelInfo`	If True, and `MultipleMode` = False, entity will be highlighted in the Model Info Tree. Only applicable for Solid, CSys, Property, Material, or Layup. Set before picking.

Selector Object Methods

The following methods are available.

ClearAll

`ClearAll`
`( void )`
Description:

Unpublished work. © 2025 Siemens

This method clears all entities of any type from the Selector.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Clear

```
Clear

( entityType )
```

Description:	
This method clears all entities of the specified type from the Selector.	
Input:	
`INT4 entityType`	The entity type to remove from the selector.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Count

Count	
(entityType, nSelected)	
Description:	
This method returns the number of entities of a given type that are currently selected.	
Input:	
`INT4 entityType`	The entity type to query.
Output:	
`INT4 nSelected`	The number of `entityType` entities that are currently selected.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SelectBox

SelectBox
(void)
Description:

Unpublished work. © 2025 Siemens

This method displays the box selection dialog.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method automatically activates **Multiple Mode**.	
Example:	
None	

SelectCircle

```
SelectCircle

( void )
```

Description:	
This method displays the circle selection dialog.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method automatically activates **Multiple Mode**.	
Example:	
None	

Unpublished work. © 2025 Siemens

SelectPolygon

SelectPolygon	
(void)	
Description:	
This method displays the polygon selection dialog.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method automatically activates **Multiple Mode**.	
Example:	
None	

SelectFreehand

SelectFreehand	
(void)	
Description:	
This method displays the freehand selection dialog.	
Input:	
None	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
None	
Remarks/Usage:	
This method automatically activates **Multiple Mode**.	
Example:	
None	

Grow

`Grow`
`( void )`
Description:
This method *grows* the current selection by adding adjacent entities.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
This method automatically activates **Multiple Mode**. It is the same as calling the **Grow** command on the **Selector** toolbar.
Example:
None

Shrink

Unpublished work. © 2025 Siemens

Shrink	
(void)	
Description:	
This method *shrinks* the current selection by removing adjacent entities.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method automatically activates **Multiple Mode**. It is the same as calling the **Shrink** command on the **Selector** toolbar.	
Example:	
None	

Show

Show	
(bShowAll)	
Description:	
This method shows the current selection	
Input:	
`BOOL bShowAll`	If True, all selected entities are shown. If False, only those of the active type (specified by the `SelectEntity` property) are shown.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SaveGroup

SaveGroup **(groupID)**	
Description:	
This method saves the current selection as a group.	
Input:	
`INT4 groupID`	The ID of the group to create.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create group from selection.
Remarks/Usage:	
None	
Example:	
None	

LoadGroup

Unpublished work. © 2025 Siemens

LoadGroup	
(groupID)	
Description:	
This method adds the specified group to the selection.	
Input:	
`INT4 groupID`	The ID of the group to load.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to load the group.
Remarks/Usage:	
This Method automatically activates **Multiple Mode**. It does not clear the current selection before adding the group.	
Example:	
None	

UpdateDataTable

UpdateDataTable	
(void)	
Description:	
This method adds the current selection to the Data Table.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Add

Add **(entityType, ID)**	
Description:	
This method adds an entity to the selector.	
Input:	
`INT4 entityType`	The type of entity to add.
`INT4 ID`	The ID of the entity of the *EntityType* to add.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

AddSet

Unpublished work. © 2025 Siemens

AddSet	
(entityType, nSetID, bUpdateDataTable)	
Description:	
This method adds multiple entities to the selector.	
Input:	
`INT4 entityType`	The type of entity to add.
`INT4 nSetID`	The ID of a Set object that contains the list of IDs of the entities to add.
`BOOL bUpdateDataTable`	If True, the selected entities are automatically added to the Data Table.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetSelected

GetSelected	
(entityType)	
Description:	
This method creates and returns a Set object that contains the IDs of all entities of a specified type that are currently selected.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 entityType`	The type of entity that will be queried.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The Set object is returned in place of the return code.	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim sel As femap.Selector
   Set sel = App.feSelector
   sel.MultipleMode = True
   sel.SelectEntity = FS_ELEM
   sel.RelatedMode = True
   sel.SelectBox()
   Dim e As femap.Set
   Set e = sel.GetSelected( FS_ELEM )
   e.Debug()
   Dim n As femap.Set
   Set n = sel.GetSelected( FS_NODE )
   n.Debug()
   sel.ClearAll()
   sel.SelectEntity = FS_NONE
End Sub
```


Unpublished work. © 2025 Siemens

GetSelectedID

GetSelectedID

(entityType, nNumSelected, nSelectedID)

Description:

This method returns the number of entities of a specified type and fills an array which contains the IDs of all entities of a specified type that are currently selected.

Input:

`INT4 entityTYPE`	The type of entity that will be queried.

Output:

`INT4 nNumSelected`	The number of entries in the `nSelectedID` array.
`INT4 nSelectedID[0..N]`	The IDs of the entities of the specified type currently in the Selector object.

Return Code:

None	

Remarks/Usage:

None

Example:

None

Copy

Copy

(void)

Description:

This method copies all selected entities to the Clipboard.

Unpublished work. © 2025 Siemens

Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The entities are copied in Femap Neutral format and can then be pasted into another model.	
Example:	
None	

Paste

`Paste`
 	
`( void )`

Description:
This method pastes entities from the Clipboard into the current model.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
Data on the clipboard must be in Femap Neutral format, as placed there by the Selector Copy command/ method.
Example:
None

Unpublished work. © 2025 Siemens

Tooltips

Tooltips **(bOn, bIncludeLayersAndGroups)**	
Description:	
This method will turn on or turn off the **Show Tooltips** and **Layers/Groups in Tooltips** options of the **Select** Toolbar.	
Input:	
`BOOL bOn`	If True, the **Show Tooltips** option will be turned on. If False, the **Show Tooltips** options will be turned off.
`BOOL bIncludeLayersAndGroups`	If True, the **Layers/Groups in Tooltips** option will be turned on. If False, the **Layers/Groups in Tooltips** option will be turned off.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The `bOn` option must be set to True for a Tooltip to be shown (i.e., having only `bIncludeLayersAndGroups` set to True will not show a Tooltip containing only the Layer and Group information for an entity).	
Example:	
None	

Set Objects

The Set object is used to contain lists of IDs. Sets, once created, can be passed to many other functions to operate on groups of multiple entities. You can also use Set objects for your own purposes, to make it easier to traverse a list of IDs.

Sets can be persistent objects. They can be saved with a model and can be retrieved at any later time, in a different script or program, or even in a different modeling session.

Unpublished work. © 2025 Siemens

It's recommended that you never hard code set IDs in your program as it may have unintended consequences. When a Set object is created, Femap will auto-assign the next available ID automatically. To reference that set later, simply use the ID property rather than a hard-coded ID.

Use the `feSet` method of the Femap Application object to create Set objects.

In addition to standard Set Objects, developers of applications who are registered with Siemens PLM Software can apply to receive a *Developer ID*, which once entered, allows conversion of any Set Object into a `SavedSet`, then stores the `SavedSet` to the Femap database. A `SavedSet` differs from a normal Set Object by *saving* both the IDs in the Set Object and the Entity Type of the IDs in the Set Object. The `SavedSet` functionality is reserved for developers registered with Siemens PLM Software.

Being a registered developer is not common or in any way required to use the Set Object functionality described in **Set Object Properties** or **Set Object Methods**. If interested in becoming a registered developer, please contact the Siemens Global Technical Access Center (GTAC) for more information.

The methods used to access the `SavedSet` functionality are described in detail in section **Set Object SavedSet Methods**.

Set Object Properties

Property	Description
`INT4 ID`	The internal ID of the set. This ID is needed if you want to reload a set created in a previous session. It is also needed for many of the other methods that operate on sets.
`BOOL Save`	Default=False. Set to True if you want the selection set to persist after the object is destroyed. If you set this option, you can reload the selection set as long as you remember the ID.

`INT4 CurrentID`	The current ID from the Set. This property is updated when you call the Next and Prev methods and provides a simple way to loop through a Set. For example, retrieving a set of nodes...
	
	`While s.Next()`
	`node.Get( s.CurrentID )`
	`...`
	
	`Wend`

Unpublished work. © 2025 Siemens

Set Object Methods

Add

Add	
Description:	
This function adds an ID into a selection set.	
Input:	
`INT4 id`	ID to add to the selection set.
Output:	
None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value.
Remarks/Usage:	
The ID that you are adding to the selection set is just that, an ID. Although it usually represents the ID of some other existing entity in your model, there is no requirement for this.	
Example:	
None	

AddRange

AddRange
`( startID, stopID, increment )`
Description:
This function adds a range of IDs into a selection set.

Unpublished work. © 2025 Siemens

Input:	
`INT4 startID`	The first ID to add to the selection set.
`INT4 stopID`	The last ID to add to the selection set.
`INT4 increment`	The ID increment between `startID` and `stopID`. Must be a positive number.
Output:	
None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value.
Remarks/Usage:	
This is just like the Add Method, but it can be used to add multiple IDs in one call.	
Example:	
None	

AddEntitiesOnLayer

`AddEntitiesOnLayer` **`( laySET, entityTYPE )`**	
Description:	
This function adds all entities of a selected type that are on a set of layers into the set.	
Input:	
`INT4 laySET`	The `setID` that contains the layers to include. Specify a negative number to include entities on a single layer.
`INT4 entityTYPE`	The type of entities being included into the set. Only those entities that reference a layer in `laySET` will be included. For entity information, see **Entity Types**.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	Unable to include entities.
Remarks/Usage:	
None	
Example:	
None	

AddSet

`AddSet` **`( addID )`**	
Description:	
This function adds all of the selected IDs from another set into the current set.	
Input:	
`INT4 addID`	ID of the selection set containing IDs to add. Accessed from the ID property of the set.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add into the specified set.
Remarks/Usage:	
This function simply adds all IDs in the `addID` set into the current set. Duplication does not cause an error; the ID will simply remain selected. The `addID` set remains unchanged during this command - only the current set is updated.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddSetRule

AddSetRule **(addID, ruleID)**	
Description:	
This function adds to a selection set by using one of the standard group selection rules. It uses the IDs from another selection set to create the new one.	
Input:	
`INT4 addID`	The selection set where the IDs will be found. The interpretation of these IDs depends on the type of rule that you select.
`INT4 ruleID`	ID of a selection rule. For more information, see Group Selection Rules.
Output:	
None	
Return Code:	
`FE_FAIL`	Not able to create the requested selection set.
`FE_NOT_EXIST`	Either the `addID` selection set was empty, or there was a problem processing it.
Remarks/Usage:	
This function is just like calling `AddRule( )` once for each of the IDs in the `addID` set. This function adds to the current set; it does not clear previous additions.	
Example:	
None	

AddRule

Unpublished work. © 2025 Siemens

AddRule	
(id, ruleID)	
Description:	
This function adds to a selection set by using one of the standard group selection rules.	
Input:	
`INT4 id`	The ID that you are specifying. The interpretation of this value depends on the type of rule that you select.
`INT4 ruleID`	ID of a selection rule. For more information, see Group Selection Rules.
Output:	
None	
Return Code:	
`FE_FAIL`	Not able to create the requested selection set.
Remarks/Usage:	
This is a very powerful tool to create selections which reference attributes of another entity. There are many group selection rules - review them carefully to choose the appropriate one for your task. This function adds to the set; it does not clear previous additions.	
Example:	
None	

Group Selection Rules

Group Selection Rules		
Rule Type	**Selects**	Which...
`FGD_CSys_byDefCSys`	Coordinate Systems	have selected definition coordinate system
`FGD_CSys_byType`	Coordinate Systems	have the type: 0=Rect, 1=Cyl, 2=Sph
`FGD_Point_ID`	Points	you choose - just matches the ID

Unpublished work. © 2025 Siemens

FGD_Point_byDefCSys	Points	have selected definition coordinate system
FGD_Point_onCurve	Points	are referenced on the curve ID
FGD_Curve_ID	Curves	you choose - just matches the ID
FGD_Curve_byPoint	Curves	are attached to the point ID
FGD_Curve_onSurface	Curves	are referenced on the surface ID
FGD_Surface_ID	Surfaces	you choose - just matches the ID
FGD_Surface_byCurve	Surfaces	are attached to the curve ID
FGD_Surface_onVolume	Surfaces	are referenced on the volume ID
FGD_Volume_ID	Volumes	you choose - just matches the ID
FGD_Volume_bySurface	Volumes	are attached to the surface ID
FGD_Text_ID	Texts	you choose - just matches the ID
FGD_Boundary_ID	Boundaries	you choose - just matches the ID
FGD_Boundary_byCurve	Boundaries	are attached to the surface ID
FGD_Node_ID	Nodes	you choose - just matches the ID
FGD_Node_byDefCSys	Nodes	have selected definition coordinate system
FGD_Node_byOutCSys	Nodes	have selected output coordinate system
FGD_Node_onElem	Nodes	are referenced on the element ID
FGD_Elem_ID	Elements	you choose - just matches the ID
FGD_Elem_byMatl	Elements	use the material ID

Unpublished work. © 2025 Siemens

FGD_Elem_byProp	Elements	use the property ID
FGD_Elem_byType	Elements	have the element type
FGD_Elem_byNode	Elements	reference the node ID
FGD_Matl_ID	Materials	you choose - just matches the ID
FGD_Matl_onProp	Materials	are used by the property ID
FGD_Matl_onElem	Materials	are used by the element ID
FGD_Matl_byType	Materials	have the material type
FGD_Prop_ID	Properties	you choose - just matches the ID
FGD_Prop_onElem	Properties	are used by the elementID
FGD_Prop_byMatl	Properties	reference the material ID
FGD_Prop_byType	Properties	have the property type
FGD_Load_byNode	Loads	are applied on the node ID
FGD_Load_byElem	Loads	are applied on the element ID
FGD_BCo_ID	Constraints	are applied on the node ID
FGD_BEq_byNode	Constraint Equations	reference the node ID
FGD_Node_atPoint	Nodes	are attached to the point ID
FGD_Node_atCurve	Nodes	are attached to the curve ID
FGD_Node_atSurface	Nodes	are attached to the surface ID
FGD_Node_atSolid	Nodes	are attached to the solid/ volume ID
FGD_Elem_atPoint	Elements	are attached to the point ID

Unpublished work. © 2025 Siemens

`FGD_Elem_atCurve`	Elements	are attached to the curve ID
`FGD_Elem_atSurface`	Elements	are attached to the surface ID
`FGD_Elem_atSolid`	Elements	are attached to the solid/ volume ID
`FGD_Load_byPoint`	Loads	are applied at the point ID
`FGD_Load_byCurve`	Loads	are applied to the curve ID
`FGD_Load_bySurface`	Loads	are applied to the surface ID
`FGD_BCo_byPoint`	Constraints	are applied at the point ID
`FGD_BCo_byCurve`	Constraints	are applied to the curve ID
`FGD_BCo_bySurface`	Constraints	are applied to the surface ID
`FGD_Text_byColor`	Text entities	are the same color as the Text ID(s) you select
`FGD_Point_byColor`	Points	are the same color as the Point ID(s) you select
`FGD_Curve_byColor`	Curves	are the same color as the Curve ID(s) you select
`FGD_Surface_byColor`	Surfaces	are the same color as the Surface ID(s) you select
`FGD_Volume_byColor`	Volumes	are the same color as the Volume ID(s) you select
`FGD_Solid_byColor`	Solids	are the same color as the Solid ID(s) you select
`FGD_CSys_byColor`	Coordinate Systems	are the same color as the Coordinate System ID(s) you select
`FGD_Node_byColor`	Nodes	are the same color as the Node ID(s) you select
`FGD_Elem_byColor`	Elements	are the same color as the Element ID(s) you select
`FGD_Prop_byColor`	Properties	are the same color as the Property ID(s) you select

Unpublished work. © 2025 Siemens

`FGD_Matl_byColor`	Materials	are the same color as the Material ID(s) you select
`FGD_Text_byLayer`	Text entities	are on the same layer as the entity you select
`FGD_Point_byLayer`	Points	are on the same layer as the entity you select
`FGD_Curve_byLayer`	Curves	are on the same layer as the entity you select
`FGD_Surface_byLayer`	Surfaces	are on the same layer as the entity you select
`FGD_Volume_byLayer`	Volumes	are on the same layer as the entity you select
`FGD_Solid_byLayer`	Solids	are on the same layer as the entity you select
`FGD_CSys_byLayer`	Coordinate Systems	are on the same layer as the entity you select
`FGD_Node_byLayer`	Nodes	are on the same layer as the entity you select
`FGD_Elem_byLayer`	Elements	are on the same layer as the entity you select
`FGD_Prop_byLayer`	Properties	are on the same layer as the entity you select
`FGD_Matl_byLayer`	Materials	are on the same layer as the entity you select
`FGD_Solid_ID`	Solids	you choose - just matches the ID
`FGD_Solid_byCurve`	Solids	contain the curve ID
`FGD_Solid_bySurface`	Solids	contain the surface ID
`FGD_Curve_onSolid`	Curves	are contained on the solid/ volume ID
`FGD_Surface_onSolid`	Surfaces	are contained on the solid/ volume ID
`FGD_Point_byProp`	Points	have the property ID as a meshing attribute

Unpublished work. © 2025 Siemens

`FGD_Curve_byProp`	Curves	have the property ID as a meshing attribute
`FGD_Surface_byProp`	Surfaces	have the property ID as a meshing attribute
`FGD_Volume_byProp`	Volumes	have the property ID as a meshing attribute
`FGD_Solid_byProp`	Solids	have the property ID as a meshing attribute
`FGD_Contact_ID`	Connection Regions	you choose - just matches the ID
`FGD_Contact_byColor`	Connection Regions	are the same color as the Connection Region ID(s) you select
`FGD_Contact_byLayer`	Connection Regions	are on the same layer as the entity you select
`FGD_CSys_onNode`	Coordinate Systems	are referenced on the nodeID
`FGD_CSys_onPoint`	Coordinate Systems	are referenced on the point ID
`FGD_Elem_byShape`	Elements	have the selected shape
`FGD_NODE_ONELEMASORIENT`	Nodes	are used for element orientation
`FGD_CONNECTIONPROP_ID`	Connection Properties	you choose - just matches the ID
`FGD_CONNECTIONPROP_BYCOLOR`	Connection Properties	are the same color as the Connection Property ID(s) you select
`FGD_CONNECTIONPROP_BYLAYER`	Connection Properties	are on the same layer as the entity you select
`FGD_CONNECTION_ID`	Connectors	you choose - just matches the ID
`FGD_CONNECTION_BYCOLOR`	Connectors	are the same color as the Connector ID(s) you select
`FGD_CONNECTION_BYLAYER`	Connectors	are on the same layer as the entity you select
`FGD_CONNECTION_BYPROP`	Connectors	reference the selected connection property

Unpublished work. © 2025 Siemens

FGD_CONTACT_ONCONNECTION	Connection Regions	are used by the selected connector
FGD_CONNECTION_BYCONTACT	Connectors	reference the selected connection region
FGD_LAYUP_ID	Layups	you choose - just matches the ID
FGD_LAYUP_BYMATL	Layups	reference the selected material
FGD_ELEM_BYLAYUP	Elements	reference a property which references the selected layup
FGD_PROP_BYLAYUP	Properties	reference the selected layup
FGD_LAYUP_ONPROP	Layups	are referenced by the selected property
FGD_NODE_BYSUPERELEMENT	Nodes	have the same Superelement ID
FGD_LOAD_BYREGION	Loads	are applied to selected Bolt Region
FGD_CONTACT_BYELEM	Connection Regions	contain the selected element
FGD_CONTACT_BYNODE	Connection Regions	contain the selected node
FGD_CONTACT_BYCURVE	Connection Regions	contain the selected curve
FGD_CONTACT_BYSURFACE	Connection Regions	contain the selected surface
FGD_CONTACT_BYPROP	Connection Regions	contain the selected property
FGD_ELEM_BYALLNODES	Elements	have all of their nodes also be selected nodes
FGD_ELEM_BYGLOBALPLY	Elements	reference a layup which has the selected global ply
FGD_LAYUP_BYGLOBALPLY	Layups	reference the selected global ply
FGD_AEROPROP_ID	Aero Properties	you choose - just matches the ID
FGD_AEROPROP_BYCOLOR	Aero Properties	are the same color as the Aero Property ID(s) you select

Unpublished work. © 2025 Siemens

FGD_AEROPROP_BYLAYER	Aero Properties	are on the same layer as the entity you select
FGD_AEROPANEL_ID	Aero Panel/Bodies	you choose - just matches the ID
FGD_AEROPANEL_BYCOLOR	Aero Panel/Bodies	are the same color as the Aero Panel ID(s) you select
FGD_AEROPANEL_BYLAYER	Aero Panel/Bodies	are on the same layer as the entity you select
FGD_AEROPANEL_BYPROP	Aero Panel/Bodies	reference the selected connection property
FGD_AEROSPLINE_ID	Aero Splines	you choose - just matches the ID
FGD_AEROSPLINE_BYCOLOR	Aero Splines	are the same color as the Aero Spline ID(s) you select
FGD_AEROSPLINE_BYLAYER	Aero Splines	are on the same layer as the entity you select
FGD_AEROSPLINE_BYPANEL	Aero Splines	reference the selected aero panel/body
FGD_AEROCTRLSURF_ID	Aero Control Surfaces	you choose - just matches the ID
FGD_AEROCTRLSURF_BYCOLOR	Aero Control Surfaces	are the same color as the Aero Control Surface ID(s) you select
FGD_AEROCTRLSURF_BYLAYER	Aero Control Surfaces	are on the same layer as the entity you select
FGD_CSYS_ONPROP	Coordinate Systems	are referenced by the selected property
FGD_CSYS_ONCSYS	Coordinate Systems	use the selected coordinate system as the definition coordinate system
FGD_SURFACE_ONREGION	Surfaces	are referenced by the selected region
FGD_CURVE_ONREGION	Curves	are referenced by the selected region
FGD_ELEM_ONREGION	Elements	are referenced by the selected region
FGD_NODE_ONREGION	Nodes	are referenced by the selected region

Unpublished work. © 2025 Siemens

FGD_PROP_ONREGION	Properties	are referenced by the selected region
FGD_PROP_ONPOINT	Properties	is used as the property meshing attribute of the selected point
FGD_PROP_ONCURVE	Properties	is used as the property meshing attribute of the selected curve
FGD_PROP_ONSURFACE	Properties	is used as the property meshing attribute of the selected surface
FGD_PROP_ONSOLID	Properties	is used as the property meshing attribute of the selected solid
FGD_SOLID_BYTYPE	Solids	are of the same type as the selected type of solid
FGD_NODE_BYTYPE	Nodes	are of the same type as the selected type of node
FGD_POINT_ONSURFACE	Points	are used by the curves of the selected surface
FGD_POINT_ONSOLID	Points	are used by the curves of the selected solid
FGD_CSYS_ONELEM	Coordinate Systems	are referenced by the select element
FGD_CONTACT_ONSOLID	Regions	use a surface on, a curve on, or the mesh associated the selected solid

AddAll

AddAll

(entityTYPE)

Description:

This function adds the IDs of all existing entities of a selected type into the current selection set.

Unpublished work. © 2025 Siemens

Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add to current set.
Remarks/Usage:	
All entity IDs are added to the current set. The set is not emptied before this operation.	
Example:	
None	

AddAllExcept

`AddAllExcept`	
`( entityTYPE, exceptSET )`	
Description:	
This function adds the IDs of all existing entities of a selected type into the current selection set, except those already selected in `exceptSET`.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 exceptSET`	The ID of a set that contains the IDs of entities to not add into this set. Specify a negative number to skip adding a single entity.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add to current set.

Unpublished work. © 2025 Siemens

Remarks/Usage:
The set is not emptied before this operation.
Example:
None

AddAllInRange

AddAllInRange **(entityTYPE, startID, stopID)**	
Description:	
This function adds all IDs in a given range specified by a start ID (first ID in range) and stop ID (last ID in range).	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 startID`	The first ID in the range of IDs to add.
`INT4 stopID`	The last ID in the range of IDs to add.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add to current set.
Remarks/Usage:	
Cannot be properly used with set-based Entity Types (i.e., output vectors, individual loads, individual boundary conditions, etc), as it will always return entities in the given range from Set ID 1.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddAllInRangeInSetV2

AddAllInRangeInSetV2 **(entTYPE, nSetID, startID, stopID)**	
Description:	
This method replaces `AddAllInRangeInSet`. For more information see, **Working with Output in Femap v2020.1 and above**. This method adds a range of entities by ID from an already populated entity set.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 nSetID`	The ID of the set object to add from.
`INT4 startID`	The ID of the entity to start from.
`INT4 stopID`	The ID of the entity to stop at.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

AddCommon

Unpublished work. © 2025 Siemens

AddCommon	
(set1, set2)	
Description:	
This function adds all IDs that are in both `set1` and `set2` to the current selection set.	
Input:	
`INT4 set1`	ID of the first selection set to check.
`INT4 set2`	ID of the second selection set to check.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add IDs to the current set.
Remarks/Usage:	
IDs must exist in both `set1` and `set2` to be added. Only the current set is updated by this command - the other sets remain unchanged.	
Example:	
None	

AddNewRemoveCommonSet

AddNewRemoveCommonSet
(addID)
Description:
This function adds all IDs in `addID` that are not in the current set, and removes any IDs that are common to both sets
Input:

Unpublished work. © 2025 Siemens

`INT4 addID`	ID of the set to combine with the current set.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to add IDs to the current set.
Remarks/Usage:	
This method implements a boolean XOR of the `addID` set and the current set.	
Example:	
None	

AddGroup

`AddGroup`
`( entityTYPE, groupID )`
Description:
This function adds the all of the IDs of a selected type from a group into the current selection set.
Input:

`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 groupID`	ID of the group to copy IDs from.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	Either the group did not exist, or no entities of the selected type were in the group.
`FE_FAIL`	Unable to add to current set.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

All entity IDs from the group are added to the current set. The set is not emptied before this operation.
Example:
None

AddArray

AddArray **(numID, arrayID)**	
Description:	
This function adds an array of IDs into a selection set.	
Input:	
`INT4 numID`	The number of entries in the `arrayID` array.
`INT4 arrayID[0..numID-1]`	An array of IDs to be added to the selection set.
Output:	
None	
Return Code:	
`FE_FAIL`	arrayID not properly defined.
`FE_NO_MEMORY`	unable to allocate working memory to add contents of arrayID to set.
Remarks/Usage:	
This is just like the Add method, but it can be used to add multiple IDs in one call.	
Example:	
None	

AddCoordinate

Unpublished work. © 2025 Siemens

AddCoordinate

(entityTYPE, nCSysID, vbXYZ, vnMode, vdMin, vdMax)

Description:

This function adds entity IDs of entities which are currently *visible* into a set as selected by their coordinate locations

Input:

`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Boundary, Solid, Volume, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, Connection and Connection _Prop entity types are valid.
`INT4 nCSysID`	The ID of the coordinate system to select in. For Rectangular systems, coordinate directions are XYZ. For Cylindrical systems, the coordinate directions are R,Theta,Z, and for Spherical, R,Theta,Phi.
`BOOL vbXYZ[3]`	Flags to consider selecting based on the associated coordinate direction. If True, then the other arguments are used to define the selection. If False, no selections will be done based on that coordinate.
`INT4 vnMode[3]`	0=Select Above Maximum, 1=Select Below Minimum, 2=Select Outside (Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance.
`REAL8 vdMin[3]`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 vdMax[3]`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.

Output:

None	

Return Code:

`FE_FAIL`	arrays not properly specified.

Unpublished work. © 2025 Siemens

`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the coordinate selection available from the entity selection dialog.	
Example:	
None	

AddCoordinate2

`AddCoordinate2` **`( entityTYPE, nCSysID, vbXYZ, vnMode, vdMin, vdMax )`**	
Description:	
This function adds entity IDs into a set as selected by their coordinate locations.	
Input:	
`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Boundary, Solid, Volume, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, Connection and Connection _Prop entity types are valid.
`INT4 nCSysID`	The ID of the coordinate system to select in. For Rectangular systems, coordinate directions are XYZ. For Cylindrical systems, the coordinate directions are R,Theta,Z, and for Spherical, R,Theta,Phi.
`BOOL vbXYZ[3]`	Flags to consider selecting based on the associated coordinate direction. If True, then the other arguments are used to define the selection. If False, no selections will be done based on that coordinate.
`INT4 vnMode[3]`	0=Select Above Maximum, 1=Select Below Minimum, 2=Select Outside (Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance.

Unpublished work. © 2025 Siemens

`REAL8 vdMin[3]`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 vdMax[3]`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.
Output:	
None	
Return Code:	
`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the coordinate selection available from the entity selection dialog.	
Example:	
None	

AddAroundPoint

`AddAroundPoint`	
`( entityTYPE, vBase, nMode, dMin, dMax )`	
Description:	
This function adds entity IDs of entities which are currently *visible* into a set as selected by their proximity to a specified location.	
Input:	
`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Boundary, Solid, Volume, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, Connection and Connection _Prop entity types are valid.

Unpublished work. © 2025 Siemens

`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the point to search around for selections.
`INT4 nMode`	0=Select Above Maximum (Farther Than), 1=Select Below Minimum (Closer Than), 2=Select Outside (Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance.
`REAL8 dMin`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 dMax`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.
Output:	
None	
Return Code:	
`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the **Around Point** coordinate selection available from the entity selection dialog.	
Example:	
None	

AddAroundPoint2

`AddAroundPoint2`
`( entityTYPE, vBase, nMode, dMin, dMax )`
Description:
This function adds entity IDs into a set as selected by their proximity to a specified location.
Input:

Unpublished work. © 2025 Siemens

`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Boundary, Solid, Volume, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, Connection and Connection _Prop entity types are valid.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the point to search around for selections.
`INT4 nMode`	0=Select Above Maximum (Farther Than), 1=Select Below Minimum (Closer Than), 2=Select Outside (Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance.
`REAL8 dMin`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 dMax`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.
Output:	
None	
Return Code:	
`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the **Around Point** coordinate selection available from the entity selection dialog except it selects both visible and non-visible entities.	
Example:	
None	

AddAroundVector

Unpublished work. © 2025 Siemens

AddAroundVector

(entityTYPE, vBase, vVec, nMode, dMin, dMax)

Description:

This function adds entity IDs of entities which are currently *visible* into a set as selected by their proximity to a specified vector.

Input:

`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Boundary, Solid, Volume, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, Connection and Connection _Prop entity types are valid.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the base of the vector around which to search for selections.
`REAL8 vVec[3]`	The components, in Global Rectangular coordinates, of the vector around which to search for selections.
`INT4 nMode`	0=Select Above Maximum (Farther Than), 1=Select Below Minimum (Closer Than), 2=Select Outside (Below Min, Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance
`REAL8 dMin`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 dMax`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.

Output:

None	

Return Code:

`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid **entityTYPE** for this method.

Remarks/Usage:

Unpublished work. © 2025 Siemens

This works just like the **Around Vector** coordinate selection available from the entity selection dialog.
Example:
None

AddAroundVector2

AddAroundVector2 **(entityTYPE, vBase, vVec, nMode, dMin, dMax)**	
Description:	
This function adds entity IDs into a set as selected by their proximity to a specified vector.	
Input:	
`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Solid, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, and Connection types are valid.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the base of the vector around which to search for selections.
`REAL8 vVec[3]`	The components, in Global Rectangular coordinates, of the vector around which to search for selections.
`INT4 nMode`	0=Select Above Maximum (Farther Than), 1=Select Below Minimum (Closer Than), 2=Select Outside (Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance
`REAL8 dMin`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 dMax`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the **Around Vector** coordinate selection available from the entity selection dialog except it selects both visible and non-visible entities.	
Example:	
None	

AddAroundPlane

`AddAroundPlane`	
`( entityTYPE, vBase, vVec, nMode, dMin, dMax )`	
Description:	
This function adds entity IDs of entities which are currently *visible* into a set as selected by their proximity to a specified plane.	
Input:	
`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Boundary, Solid, Volume, Node, Element, Property, Material, Coordinate System, Layup, Text, Connection_Region, Connection and Connection _Prop entity types are valid.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of a point on the plane around which to search for selections.
`REAL8 vVec[3]`	The components, in Global Rectangular coordinates, of the normal to the plane around which to search for selections.
`INT4 nMode`	0=Select Above Maximum (Positive Side), 1=Select Below Minimum (Negative Side), 2=Select Outside

Unpublished work. © 2025 Siemens

	(Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance.
`REAL8 dMin`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 dMax`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.
Output:	
None	
Return Code:	
`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the **Around Plane** coordinate selection available from the entity selection dialog.	
Example:	
None	

AddAroundPlane2

`AddAroundPlane2`
`( entityTYPE, vBase, vVec, nMode, dMin, dMax )`
Description:
This function adds entity IDs into a set as selected by their proximity to a specified plane.
Input:

`INT4 entityTYPE`	The type of entities being included into the set. For entity information, see **Entity Types**. Only Point, Curve, Surface, Solid, Node, Element, Property, Material, Coordinate System, Layup, Text,

Unpublished work. © 2025 Siemens

	Connection_Region, and Connection types are valid.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of a point on the plane around which to search for selections.
`REAL8 vVec[3]`	The components, in Global Rectangular coordinates, of the normal to the plane around which to search for selections.
`INT4 nMode`	0=Select Above Maximum (Positive Side), 1=Select Below Minimum (Negative Side), 2=Select Outside (Below Min,Above Max), 3=Select Between (Above Min and Below Max), 4=Select At within Tolerance.
`REAL8 dMin`	The minimum coordinate for most modes. Not used for **Above Maximum**. For **Select At** the coordinate location to select around.
`REAL8 dMax`	The maximum coordinate for most modes. Not used for **Below Minimum**. For **Select At** the tolerance around the specified location that the coordinate must fall within to be selected.
Output:	
None	
Return Code:	
`FE_FAIL`	arrays not properly specified.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
This works just like the **Around Plane** coordinate selection available from the entity selection dialog except it selects both visible and non-visible entities.	
Example:	
None	

AddNodesOnGeometry

Unpublished work. © 2025 Siemens

AddNodesOnGeometry	
(entityTYPE, entityID, bClear, bFullTree, bBoundaryOnly)	
Description:	
This function adds nodes to the set which are attached to the specified geometry.	
Input:	
`INT4 entityTYPE`	The type of entity to find attached nodes.
`INT4 entityID`	ID of entity to find attached nodes.
`BOOL bClear`	Flag to clear the set before adding nodes.
`BOOL bFullTree`	If True, the nodes attached to the sub entities are also included (Solid = Surf,Curve,Point: Surf=Curve,Point: Curve=Point).
`BOOL bBoundaryOnly`	Flag to only include nodes at the boundary of the entity.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the entity in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.
Remarks/Usage:	
If `bClear` is false this method can be called multiple times to build up an accumulated list.	
Example:	
None	

AddAllTitleV2

Unpublished work. © 2025 Siemens

AddAllTitleV2	
(entityTYPE, nSetID, sFind)	
Description:	
This method replaces `AddAllTitleV2`. For more information see, **Working with Output in Femap v2020.1 and above**. This function adds the selected entities which have matching titles.	
Input:	
`INT4 entityTYPE`	The type of entity to add. The entity type must have a title to be valid for this method.
`INT4 nSetID`	Normally this should be 1. For specific set-based types such as Load and Constraint Definitions, it must be the ID of the set to search for matching titles.
`STRING sFind`	A text string that will be compared to the entity title. If this string is found anywhere in the title, that entity ID will be added to the set. Comparisons are not case sensitive.
Output:	
None	
Return Code:	
`FE_INVALID`	You specified a blank string to match.
`FE_NOT_EXIST`	No entities were found that matched the string you supplied.
Remarks/Usage:	
None	
Example:	
None	

AddAllColor

Unpublished work. © 2025 Siemens

AddAllColor **(entityTYPE, nSetID, nColor, nColorMatch)**	
Description:	
This function adds the selected entities which match a specified color.	
Input:	
`INT4 entityTYPE`	The type of entity to add. The entity type must have a color to be valid for this method.
`INT4 nSetID`	Normally this should be 1. For specific set-based types such as the various load and constraint types, it must be the ID of the set to search for matching colors.
`INT4 nColor`	The color to search for. This number is a combination of the color, linestyle and pattern/ transparency.
`INT4 nColorMatch`	Any combination of 1=Match Color only, 2=Match Pattern only, 4=Match Linestyle only. So 3=Match Color & Pattern, 6=Pattern & Linestyle, 5=Color & Linestyle, 7=Match All.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	No entities were found that matched the color you supplied.
Remarks/Usage:	
None	
Example:	
None	

AddMidsideNodes

Unpublished work. © 2025 Siemens

AddMidsideNodes	
(setID)	
Description:	
This function adds the midside nodes that are on selected elements.	
Input:	
`INT4 setID`	The ID of a Set object that contains the elements that reference the midside nodes to add. Alternatively, if you specify a negative value, this is simply the ID of the single element.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	No midside nodes were found on the elements that you specified.
Remarks/Usage:	
None	
Example:	
None	

AddVisible

AddVisible	
(nVuID, entityTYPE, bUseViewDrawEntityOption)	
Description:	
This function adds entities of the specified type which are visible in the specified view to the current set.	
Input:	
`INT4 nVuID`	ID of View.

Unpublished work. © 2025 Siemens

`INT4 entityType`	Type of entity to select. For more information, see **Entity Types**.
`BOOL bUseViewDrawEntityOption`	Consider the visibility flags when determining visible entities
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	No active model or view does not exist.
Remarks/Usage:	
With respect to this function, visible entities are any entities currently visible in the view, even if the entities are *off screen* because of the current orientation or you are zoomed in and only looking at a portion of the model.	
Example:	
None	

AddConnectedElements

`AddConnectedElements`
`( void )`
Description:
This function adds elements that are connected to the elements in the set to the current set.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
Connected elements are those elements that share at least one node with the specified elements.

Unpublished work. © 2025 Siemens

Example:
None

AddAllConnectedElements

AddAllConnectedElements (void)	
Description:	
This function adds all elements that are connected to the elements in the set to the current set.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Connected elements are those that are connected to the same body as the specified elements, not just the elements that share nodes with the specified elements.	
Example:	
None	

AddSolidElementsAlongVector

AddSolidElementsAlongVector (v1, v2, elemSET, bRequireIntersect, bInfiniteVector, bDirectionalVector)
Description:

Unpublished work. © 2025 Siemens

This function adds solid elements along the specified vector.	
Input:	
`REAL8 p1[0..2]`	The coordinates, in Global Rectangular coordinates, of the base of the vector (point 1 of a straight line).
`REAL8 p2[0..2]`	The components, in Global Rectangular coordinates, of the tip of the vector (point 2 of a straight line).
`INT4 elemSET`	ID of set of elements which can be added. Alternatively, if you specify a negative value, this is simply the ID of a single element. Enter a value of 0 to allow all solid elements to potentially be added.
`BOOL bRequireIntersect`	When True, vector must hit the element. When False, vector must only hit the element box (faster but returns more elements, including ones that don't really touch the vector).
`BOOL bInfiniteVector`	When False, vector is really the line segment between the two points, intersections beyond the ends are not returned. When True, vector extends to infinity.
`BOOL bDirectionalVector`	Only used if bInfiniteVector is True. If True, Vector only extends beyond point 2, not behind point 1. If False, extends in both directions.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	No elements added to set.
Remarks/Usage:	
Midside nodes of parabolic elements are ignored when using the intersection check, therefore, all element faces of parabolic elements are tested the same as those of linear elements.	
Example:	
None	

AddConnectedFillets

Unpublished work. © 2025 Siemens

AddConnectedFillets	
(void)	
Description:	
This function adds all connected fillets to the ones specified in the set and adds them to the set.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No surfaces exist in the current set.
`FE_NOT_EXIST`	No connected fillets have been found.
Remarks/Usage:	
None	
Example:	
None	

AddTangentSurfaces

AddTangentSurfaces	
(dTol)	
Description:	
This method finds all surfaces tangent to those within the current set, within the specified tolerance, and adds them to the set.	
Input:	
`REAL8 dTol`	Tangency tolerance angle, in degrees.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	No surfaces exist in the current set.
`FE_INVALID`	Tolerance is not between 0.0 and 360.0.
`FE_NOT_EXIST`	No tangent surfaces have been found.
Remarks/Usage:	
None	
Example:	
None	

AddConstrained

`AddConstrained` **`( bInAnySet, nSetID, bExpandBC, enOnType, enMatch, bDOF1, bDOF2, bDOF3, bDOF4, bDOF5, bDOF6 )`**	
Description:	
This function adds constrained nodes, points, curves or surfaces to the current set.	
Input:	
`BOOL bInAnySet`	When True, adds entities constrained in any constraint set. When False, only considers entities constrained in the constraint set specified for `nSetID`.
`INT4 nSetID`	ID of Constraint set. Ignored if `bInAnySet` = TRUE
`BOOL bExpandBC`	Expand geometric constraints when searching for constrained nodes
`INT4 enOnType`	Constrained entity type. For more information, see **Entity Types**. Supported types are Nodes (`FT_NODE`), Points (`FT_POINT`), Curves (`FT_CURVE`), and Surfaces (`FT_SURFACE`).

Unpublished work. © 2025 Siemens

`INT4 enMatch`	Match method for degrees of freedom (See Remarks/Usage section). 0=Match Any (`FMTCH_ANY`) 1=Match All (`FMTCH_ALL`) 2=Match Any True (`FMTCH_ANY_TRUE`) 3=Match All True (`FMTCH_ALL_TRUE`) 4=Match Any Constrained (`FMTCH_ANY_CONSTRAINED`) 5=Match All Constrained (`FMTCH_ALL_CONSTRAINED`)
`BOOL bDOF1`	DOF1 (TX)
`BOOL bDOF2`	DOF2 (TY)
`BOOL bDOF3`	DOF3 (TZ)
`BOOL bDOF4`	DOF4 (RX)
`BOOL bDOF5`	DOF5 (RY)
`BOOL bDOF6`	DOF6 (RZ)
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	Unsupported constrained entity type specified.
Remarks/Usage:	

How the six degree-of-freedom flags are used for matching constraints depends on the specified match type for `enMatch`.

`FMTCH_ALL` - Matches when all 6 input DOF exactly match the DOF on an entity (checks for match of all constrained and unconstrained DOF).
`FMTCH_ALL_TRUE` - Matches when all of the input DOF specified as TRUE are also constrained on an entity (only checks for match of all TRUE DOF).
`FMTCH_ALL_CONSTRAINED` - Matches entities that have all six DOF constrained (`bDOF1-6` are ignored).
`FMTCH_ANY` - Matches when any of the 6 input DOF match any of the DOF on an entity (checks for any match of constrained or unconstrained DOF).
`FMTCH_ANY_TRUE` - Matches when any of the 6 input DOF specified as TRUE are also constrained on an entity (checks for any match of TRUE DOF).

Unpublished work. © 2025 Siemens

FMTCH_ANY_CONSTRAINED - Matches entities that have any DOF constrained (bDOF1-6 are ignored).
Example:
None

AddLoaded

AddLoaded	
(bInAnySet, nSetID, bExpandLoad, enLoadType, enRangeOpt, dMin, dMax)	
Description:	
This function adds loaded entities to the current set. The type of entity added depends on the load type specified and if geometric loads are expanded.	
Input:	
BOOL bInAnySet	When True, adds entities loaded in any load set. When False, only considers entities loaded in the load set specified for nSetID.
INT4 nSetID	ID of load set. Ignored if bInAnySet = TRUE.
BOOL bExpandLoad	Expand geometric loads when searching for loaded nodes or elements, as appropriate.
INT4 enLoadType	The type of load being selected. Refer to the load type table for values.
INT4 enRangeOpt	Search Range: 0=Any Load (FRNG_ANY) - matches any load value. 1=Above (FRNG_ABOVE) - matches any load value above dMin. 2=Below (FRNG_BELOW) - matches any load value below dMax. 3=Between (FRNG_BETWEEN) - matches load values between dMin and dMax. 4=Outside (FRNG_OUTSIDE) - matches load values below dMin and above dMax.

Unpublished work. © 2025 Siemens

`REAL8 dMin`	Minimum Load Value.
`REAL8 dMax`	Maximum Load Value.
Output:	
None	
Return Code:	
`FE_INVALID`	`dMin` > `dMax` for `FRNG_BETWEEN` and `FRNG_OUTSIDE`.
`FE_BAD_DATA`	Invalid option specified for `enRangeOpt`.
`FE_NOT_AVAILABLE`	Invalid load set specified or no load sets exist in the model.
Remarks/Usage:	
For loads that consist of multiple components, such as nodal forces, the search is performed on the resultant value, not each individual component.	
Example:	
None	

AddComponentOutputVectorsV2

`AddComponentOutputVectorsV2`
`( nFromOutputSetID )`
Description:
This method replaces `AddComponentOutputVectors`. For more information see, **Working with Output in Femap v2020.1 and above**. This method adds any Component and/or Corner Output Vectors found in the specified Output Set ID, which are component/corner output vectors for Output Vector IDs currently in the Set Object.
Input:

`INT4 nFromOutputSetID`	Output Set ID.

Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	No Output Vector IDs are in the Set Object or the Output Vectors currently in the Set Object do not have any Component or Corner Output Vectors.
Remarks/Usage:	
This method is similar to using the **Add Component/Corner Results** button in dialog boxes which allow you to select any combination of Output Sets and Output Vectors.	
Example:	
None	

AddSimilarOutputVectorsV2

`AddSimilarOutputVectorsV2` **`( nFromOutputSetID )`**	
Description:	
This method replaces `AddSimilarOutputVectors`. For more information see, **Working with Output in Femap v2020.1 and above**. This method adds any *Similar* Output Vectors found in the specified Output Set ID, which are *similar* output vectors for Output Vector IDs currently in the Set Object. Similar Output Vectors include output vectors at different locations and corners/ends (i.e., Top/Bottom for shell elements at Centroid and Corners, All Four Stress Recovery Points on End A and End B for Bar/Beam elements, All Plies for laminate elements, etc).	
Input:	
`INT4 nFromOutputSetID`	Output Set ID.
Output:	
None	
Return Code:	
`FE_FAIL`	No Output Vector IDs are in the Set Object or the Output Vectors currently in the Set Object do not have any Similar Output Vectors.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method is somewhat similar to using the **Add Similar Layer/Ply Results** button and **Add Component/Corner Results** button in dialog boxes which allow you to select any combination of Output Sets and Output Vectors, but may result in more *similar* output vectors then expected to be selected.
Example:
None

AddComplexOutputVectorsV2

AddComplexOutputVectorsV2 **(nFromOutputSetID)**	
Description:	
This method replaces `AddComplexOutputVectors`. For more information see, **Working with Output in Femap v2020.1 and above**. This method adds complex Output Vectors found in the specified Output Set, which correspond to the Output Vector IDs currently in the Set Object. For instance, if Magnitude Output Vectors are currently in the Set Object, then the corresponding Phase Output Vectors will be added or vice versa. Also, if Real Output Vectors are currently in the Set Object, then the corresponding Imaginary Output Vectors will be added and vice versa.	
Input:	
`INT4 nFromOutputSetID`	Output Set ID.
Output:	
None	
Return Code:	
`FE_FAIL`	No Output Vector IDs are in the Set Object or the Output Vectors currently in the Set Object do not have any corresponding complex Output Vectors.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

AddBySize

AddBySize **(entTYPE, nLimitSetID, nMethod, dMinVal, dMaxVal)**	
Description:	
This method adds entities from an existing set by a method type given a minimum and maximum value.	
Input:	
`INT4 entityTYPE`	Type of entity to select. Can only be set to `3 = FT_CURVE`, `4 = FT_SURFACE`, or `39 = FT_SOLID`.
`INT4 nLimitSetID`	The ID of the set object to add from.
`INT4 nMethod`	Type of method to add by. `0 = FPC_ABOVE_MAX`, `1 = FPC_BELOW_MIN`, `2 = FPC_OUTSIDE`, `3 = FPC_BETWEEN`, `4 = FPC_AT`.
`REAL8 dMinVal`	The minimum value used in the method type for adding.
`REAL8 dMaxVal`	The maximum value used in the method type for adding.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Minimum and Maximum values apply to the corresponding entity type, length for Curve, area for Surface, Volume for Solid.	
Example:	
None	

AddNodesOnFreeEdges

Unpublished work. © 2025 Siemens

AddNodesOnFreeEdges

(elemSetID, bParabolicEdges, bLineElem)

Description:

This method adds the nodes found on the free edges in a set of elements to a set.

Input:

`INT4 elemSetID`	The ID of a Set object that contains the elements that reference the nodes on free edges to add. If 0, all elements in the model are used. Alternatively, if you specify a negative value, this is simply the ID of the single element.
`BOOL bParabolicEdges`	If True, the computation of Free Edges includes checking the midside nodes (if any). If False, only corner nodes are considered.
`BOOL bLineElem`	If True, then any line elements that lie along an edge are considered when deciding if the edge is *free*.

Output:

None	

Return Code:

`FE_FAIL`	Unable to determine free edges.

Remarks/Usage:

None

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim nset As femap.Set
   Set nset = App.feSet
   Dim eset As femap.Set
   Set eset = App.feSet
   eset.AddAll (FT_ELEM)
```


Unpublished work. © 2025 Siemens

```
   rc = nset.AddNodesOnFreeEdges (eset.ID, False, True)
   nset.Show (FT_NODE)

End Sub
```

AddNodesOnFreeFaces

AddNodesOnFreeFaces

(elemSetID, bParabolicEdges, bPlaneElem)

Description:	
This method adds the nodes found on the free faces in a set of elements to a set.	
Input:	
`INT4 elemSetID`	The ID of a Set object that contains the elements that reference the nodes on free faces to add. If 0, all elements in the model are used. Alternatively, if you specify a negative value, this is simply the ID of the single element.
`BOOL bParabolicEdges`	If True, the computation of Free Faces includes checking the midside nodes (if any). If False, only corner nodes are considered.
`BOOL bPlaneElem`	If True, then any selected plane elements are considered when deciding if the face is *free*.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to determine free faces.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim nset As femap.Set
   Set nset = App.feSet
   Dim eset As femap.Set

   Set eset = App.feSet
   eset.AddAll (FT_ELEM)

   rc = nset.AddNodesOnFreeFaces (eset.ID, False, True)
   nset.Show (FT_NODE)
End Sub
```

IsAdded

IsAdded

(id)

Description:

This function checks to see if an ID is currently added into a selection set.

Input:

`INT4 id`	ID to check.

Output:

None	

Return Code:

Unpublished work. © 2025 Siemens

`FE_OK`	ID is added into the selection set.
`FE_FAIL`	ID is not added into the selection set.
Remarks/Usage:	
The selection set need not exist to use this function. However, if it does not, the return code will simply be `FE_FAIL` - the ID is not selected.	
Example:	
None	

IsArrayAdded

`IsArrayAdded` **`( numID, arrayID )`**	
Description:	
This function checks to see if all IDs from an array are currently added into a selection set.	
Input:	
`INT4 numID`	The number of IDs in the array.
`INT4 arrayID[0..numID-1]`	The array containing IDs to check to see if they are in the current Set.
Output:	
None	
Return Code:	
`FE_OK`	All IDs in the array are in the current Set.
`FE_FAIL`	At least one ID from the array is not in the current Set.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

IsSetAdded

IsSetAdded **(nSetID)**	
Description:	
This function checks to see if all IDs from a Set are currently added into a selection set.	
Input:	
`INT4 nSetID`	The ID of the Set to check to see if all IDs are in this Set.
Output:	
None	
Return Code:	
`FE_OK`	All IDs in `nSetID` are in the current Set.
`FE_FAIL`	At least one ID from `nSetID` is not in the current Set.
Remarks/Usage:	
None	
Example:	
None	

IsIdenticalSet

Unpublished work. © 2025 Siemens

IsIdenticalSet	
(set1)	
Description:	
This function checks to see if a set has identical entity IDs as another set.	
Input:	
`INT4 set1`	The ID of the Set to check to see if it contains identical entity IDs.
Output:	
None	
Return Code:	
`FE_OK`	Set1 is identical.
`FE_FAIL`	Set1 is not identical.
Remarks/Usage:	
None	
Example:	
None	

Select

Select	
(entityTYPE, clear, title)	
Description:	
This function displays a standard selection dialog box to allow a user to choose entities of a specific type and create a selection set.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.

Unpublished work. © 2025 Siemens

`BOOL clear`	If True, the set is cleared prior to selection, and only the selected entities will be in the set. If False, previously selected entities will be displayed in the dialog box for editing, or for combining with new selections.
`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No entities of the selected type exist. None were selected and the dialog box was not displayed. If `clear=True`, the selection set will be empty.
Remarks/Usage:	
After the dialog box is closed, the set contains the list of all IDs that were selected. If `clear=False`, then some of the entities could have been selected prior to the dialog being displayed. If the user presses **Cancel** to close the dialog, then the set contains whatever it contained before, unless `clear=True`, in which case it contains nothing.	
Example:	
None	

SelectID

`SelectID`
`( entityTYPE, title, ID )`
Description:
This function displays a selection dialog box to allow a user to choose a single entity of a specific type. The ID of the entity is returned and is stored in the current selection set.
Input:

`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.

Unpublished work. © 2025 Siemens

`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
Output:	
`INT4 ID`	The ID of the entity that was selected.
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No entities of the selected type exist. None were selected and the dialog box was not displayed.
Remarks/Usage:	
This function displays one of two different dialog boxes, depending on the entity type that you choose. For entities that have titles, like properties or materials, the dialog box will contain a list of the available entities along with their titles. For other entities, like nodes or elements, a simple dialog box is displayed so that the ID can be typed or picked.	
Example:	
None	

SelectIDWithNew

`SelectIDWithNew` **`( entityTYPE, title, ID )`**	
Description:	
This function displays a selection dialog box (including the ability for the user to create a new entity from within the dialog, when applicable) to allow a user to choose a single entity of a specific type. The ID of the entity is returned and stored in the current selection set.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.

Unpublished work. © 2025 Siemens

Output:	
`INT4 ID`	The ID of the entity that was selected.
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_BAD_TYPE`	An invalid entity type was specified.

Remarks/Usage:

For supported entities, this method is identical to the `SelectID` method with the exceptions of a new entity icon being displayed in the dialog and the dialog will be displayed even if no entities of the specified type exist the database.

Supported entities include:

- Layer
- Property
- Material
- Coordinate System
- Global Ply
- Load Set
- Constraint Set
- Connection Property
- Connection Region

Example:

None

SelectIDInSetV2

`SelectIDInSetV2`

`( entityTYPE, title, nInSetID, ID )`

Description:

This method replaces `SelectIDInSet`. For more information see, **Working with Output in Femap v2020.1 and above**. This function displays a selection dialog box to allow a user to choose a single entity of a specific type. The ID of the entity is returned and is stored in the current selection set.

Input:

Unpublished work. © 2025 Siemens

`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
`INT4 nInSetID`	Select the entity from the specified `SetID` - for example an Output Vector from a specific Output Set, or a Load or Constraint Definition from a specific Load or Constraint Set.
Output:	
`INT4 ID`	The ID of the entity that was selected.
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No entities of the selected type exist. None were selected and the dialog box was not displayed. If `clear=True`, the selection set will be empty.
Remarks/Usage:	
This function is designed to be used to select a single item (i.e., a Load Definition, Constraint Definition, Output Vector) which exists in a specific *Set* in the database (i.e., Load Set, Constraint Set, Output Set). For entities which are not *Set-based*, `nInSetID` must be set to *1*, therefore, there is no benefit to use the method instead of the `SelectID` method. Depending on the type of entity you choose, one of two different dialog boxes will be displayed. For entities that have titles, like properties or materials, the dialog box will contain a list of the available entities along with their titles. For other entities, like nodes or elements, a simple dialog box is displayed so that the ID can be typed or picked.	
Example:	
None	

SelectMultiIDV2

Unpublished work. © 2025 Siemens

SelectMultiIDV2 **(entityTYPE, entitySET, title)**	
Description:	
This method replaces `SelectMultiID`. For more information see, **Working with Output in Femap v2020.1 and above**. This function displays a multi-selection dialog box to allow a user to choose one or more entities of a specific type. The IDs of the selected entities are stored in the current selection set.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 entitySET`	The `setID` associated with the entities you want to select. For most entities this is 1, however for some like Loads and Constraints, they may be stored in other sets.
`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
Remarks/Usage:	
This function will work for most entity types, however in most cases you should only use it for entities with titles. If you use it for entities like Nodes or Elements, you will just see a potentially large list of numbers which are not very useful for selection.	
Example:	
None	

SelectOutputVectorIDV2

Unpublished work. © 2025 Siemens

SelectOutputVectorIDV2

(title, outsetID, limitOutputType, limitComplex, limitToEntity, allowCorner, selID)

Description:

This method replaces `SelectOutputVectorID`. For more information see, **Working with Output in Femap v2020.1 and above**. This function displays a selection dialog box to allow a user to choose an output vector. The ID of the entity is returned and is stored in the current selection set.

Input:

`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
`INT4 outsetID`	The ID of the Output Set containing the list of possible vectors to select.
`INT4 limitOutputType`	Reduces list of output vectors shown to the specified type (0=All, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp).
`INT4 limitComplex`	Reduces list of output vectors shown based on type of real or complex data they contain (0=Magnitude, 1=Phase, 2=Real, 3=Imaginary, 4=Any).
`INT4 limitToEntity`	Reduces list of output vectors to those containing data on the specified entity type.
`BOOL allowCorner`	If True, output vectors at element corners will also be included.

Output:

`INT4 selID`	The ID of the selected output vector.

Return Code:

`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No output vectors existed in the specified output set.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

SelectList

`SelectList`
 	
`( nNumEntity, entityTYPE )`	

Description:	
This function displays a selection dialog box to allow a user to choose a specified number of entities of a specific type. The IDs of the selected entities are stored in the current selection set.	
Input:	
`INT4 nNumEntity`	Number of entities to be selected.
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
Remarks/Usage:	
The dialog box that is displayed by this method automatically closes when `nNumEntity` entities have been selected.	
Example:	
None	

Clear

Unpublished work. © 2025 Siemens

Clear	
(void)	
Description:	
This function simply removes all selections and exclusions from the current set.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If coding in anything other than Visual Basic or WinWrap (language used in the API Programming dockable pane available within the Femap user interface), this method must use all lowercase letters. For more information, see **Other Considerations**.	
Example:	
None	

Remove

Remove	
(id)	
Description:	
This function removes an ID from a selection set.	
Input:	
`INT4 id`	ID to remove from the selection set.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	ID specifies an invalid value.
Remarks/Usage:	
Any ID can be removed from a selection set. Although it has no effect, it is not an error to remove an ID that was not currently added to a selection set, or even removing from a selection set that does not exist.	
Example:	
None	

RemoveRange

`RemoveRange` **`( startID, stopID, increment )`**	
Description:	
This function removes a range of IDs from a selection set.	
Input:	
`INT4 startID`	The first ID to remove from the selection set.
`INT4 stopID`	The last ID to remove from the selection set.
`INT4 increment`	The ID increment between `startID` and `stopID`. Must be a positive number.
Output:	
None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value.
Remarks/Usage:	
This is just like the `Remove` method, but it can be used to remove multiple IDs in one call.	
Example:	
None	

Unpublished work. © 2025 Siemens

RemoveArray

RemoveArray	
(nNumID, arrayID)	
Description:	
This function removes a list of IDs specified in an array from a selection set.	
Input:	
`INT4 nNumID`	The number of IDs in the array to be removed.
`INT4 arrayID[0..nNumID-1]`	The array of IDs to be removed.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to retrieve specified number of IDs from array.
`FE_NO_MEMORY`	Unable to process the array.
Remarks/Usage:	
This is just like the `Remove` method, but it can be used to remove multiple IDs in one call.	
Example:	
None	

RemoveSet

RemoveSet
(removeID)
Description:

Unpublished work. © 2025 Siemens

This function removes all of the selected IDs in another set from the current set.	
Input:	
`INT4 removeID`	ID of the selection set containing the IDs to remove.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remove IDs from the specified set.
Remarks/Usage:	
This function removes all IDs in the `removeID` set that are currently selected. If `removeID` contains IDs that are not currently selected, they are skipped. This is not an error. The `removeID` set remains unchanged during this command - only the current set is updated.	
Example:	
None	

RemoveRule

`RemoveRule`	
`( id, ruleID )`	
Description:	
This function removes from a selection set by using one of the standard group selection rules.	
Input:	
`INT4 id`	The ID that you are specifying. The interpretation of this value depends on the type of rule that you select.
`INT4 ruleID`	ID of a selection rule. For more information, see **Group Selection Rules**.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Rule does not exist or cannot be evaluated.
Remarks/Usage:	
This function is similar to `AddRule`, however instead of adding IDs to the set, it removes them from the set.	
Example:	
None	

RemoveSetRule

`RemoveSetRule` **`( removeID, ruleID )`**	
Description:	
This function removes from a selection set by using one of the standard group selection rules. It uses the IDs from another selection set to operate on the current one.	
Input:	
`INT4 id`	The selection set where the IDs will be found. The interpretation of these IDs depends on the type of rule that you select.
`INT4 ruleID`	ID of a selection rule. For more information, see **Group Selection Rules**.
Output:	
None	
Return Code:	
`FE_FAIL`	Rule does not exist or cannot be evaluated.
Remarks/Usage:	
This function is similar to `AddSetRule`, however instead of adding IDs to the set, it removes them from the set.	
Example:	
None	

Unpublished work. © 2025 Siemens

RemoveNotCommon

RemoveNotCommon	
(nSetID)	
Description:	
This function removes any IDs that are not in both the current Set and the specified Set.	
Input:	
`INT4 nSetID`	ID of a Set that is checked for IDs that are common to the current Set.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remove IDs from the specified set.
Remarks/Usage:	
This function removes all IDs from the current Set that are not in the selected Set. The result is that the current Set will only contain IDs that are common to both Sets. The Set `nSetID` remains unchanged during this command - only the current set is updated.	
Example:	
None	

RemoveNotCommonToGroup

RemoveNotCommonToGroup
(entityType, groupID)
Description:
This function removes any IDs that are not in both the current Set and the specified Group.
Input:

Unpublished work. © 2025 Siemens

`INT4 entityTYPE`	The type of entities in the Group being tested. For entity information, see **Entity Types**.
`INT4 groupID`	The ID of the group to test
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to remove IDs from the specified set.
`FE_NOT_EXIST`	There are no entities of the selected type in the Group.
Remarks/Usage:	
This function removes all IDs from the current Set that are not in the selected Group of the specified `entityTYPE`. The result is that the current Set will only contain IDs that are common to the Set and the Group. The Group remains unchanged during this command - only the current set is updated.	
Example:	
None	

RemoveGroup

`RemoveGroup` **`( entityType, groupID )`**	
Description:	
This function removes any IDs that are in a Group.	
Input:	
`INT4 entityTYPE`	The type of entities in the Group being tested. For entity information, see **Entity Types**.
`INT4 groupID`	The ID of the group to test.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to remove IDs from the specified set.
`FE_NOT_EXIST`	There are no entities of the selected type in the Group.
Remarks/Usage:	
This function removes all IDs from the current Set that are in the selected Group and `entityType`. The Group is unchanged.	
Example:	
None	

RemoveInternalCurves

`RemoveInternalCurves`	
`(  )`	
Description:	
This function removes the IDs of any curves that are internal to a boundary/compound surface.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

RemoveNotVisible

RemoveNotVisible	
( )	
Description:	
This function removes entities of the specified type which are not visible in the specified view from the current set.	
Input:	
`INT4 nVuID`	ID of View.
`INT4 entityType`	The type of entities in the Group being tested. For entity information, see **Entity Types**.
`BOOL bUseViewDrawEntityOption`	Consider the visibility flags when determining visible entities.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	No active model or view does not exist.
Remarks/Usage:	
With respect to this function, visible entities are any entities currently visible in the view, even if the entities are *off screen* because of the current orientation or you are zoomed in and only looking at a portion of the model.	
Example:	
None	

HasCommon

Unpublished work. © 2025 Siemens

HasCommon	
(nSetID)	
Description:	
This function checks to see if there are any common IDs between two Sets.	
Input:	
`INT4 nSetID`	The ID of the Set to check for common IDs with the current Set.
Output:	
None	
Return Code:	
`FE_OK`	Common IDs exist between the two Sets.
`FE_FAIL`	There are no common IDs in the Sets.
Remarks/Usage:	
Neither Set is changed by this method, it simply looks for the same ID in both Sets.	
Example:	
None	

HasNotCommon

HasNotCommon	
(nSetID)	
Description:	
This function checks to see if there are any IDs in the current Set that are not in the specified Set.	
Input:	
`INT4 nSetID`	The ID of the Set to check with the current Set.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_OK`	At least one ID in the current Set is not in the specified Set.
`FE_FAIL`	All IDs in the current Set are also in the specified Set.
Remarks/Usage:	
Neither Set is changed by this method, it simply looks for common IDs in both Sets.	
Example:	
None	

Reload

`Reload` **`( oldSet )`**	
Description:	
This function reloads a saved selection set. This method is obsolete and should be replaced with `Reload2()`.	
Input:	
`INT4 oldSet`	The ID of the selection set to reload. This ID can be determined by querying the ID property of a set.
Output:	
None	
Return Code:	
`FE_FAIL`	The `oldSet` selection set does not exist or is empty.
Remarks/Usage:	
Selection sets can be made to persist across programs or modeling sessions using the Save property. If a set is *saved*, it can be reloaded with this method. If you reload a Set with this method, you must manually also set `Save` to True if you want the set to persist after the object you reload into is destroyed.	

Unpublished work. © 2025 Siemens

Do not use this method in a loop where the Set object is reused to load multiple saved sets. This process is unsupported and will cause random overwrites and loss of saved set data. If this is required you should use `Reload2()` or code so `Reload()` is only called once per Set object, setting the `Save` property then destroying the object. See Example.

Example:

```
Dim App As femap.model

Sub Main

   Set App = feFemap()


   For  i = 1 To 20

       getSet( i )

   Next i

End Sub


Sub getSet( i )


   Dim fs As femap.Set

Set fs = App.feSet

   Dim s As String


   rc = fs.Reload( i )

   fs.Save = True

   s = "ID = " + Str( fs.ID ) + ", Count = " + Str( fs.Count )

   App.feAppMessage ( FCM_COMMAND,  s )


End Sub
```

Reload2

Unpublished work. © 2025 Siemens

Reload2(oldSet)

Description:

This function reloads a saved selection set. This function replaces `Reload()`.

Input:

`INT4 oldSet`	The ID of the selection set to reload. This ID can be determined by querying the ID property of a set.

Output:

None	

Return Code:

`FE_FAIL`	The `oldSet` selection set does not exist or is empty.

Remarks/Usage:

Selection sets can be made to persist across programs or modeling sessions using the Save property. If a set is *saved*, it can be reloaded with this method.

If you reload a Set with this method, you must manually also set `Save` to True if you want the set to persist after the object you reload into is destroyed.

This Method replaces `Reload` and is safe to use in a loop where a single Set object is used to load multiple saved sets.

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim fs As femap.Set
   Set fs = App.feSet
   Dim s As String
   Dim rc As Long

   For  i = 1 To 10000
      rc = fs.Reload2( i )
```


Unpublished work. © 2025 Siemens

```
      If rc = FE_OK Then
          If fs.Count() > 0 Then
            s = "ID = " + Str( fs.ID ) + ", Count = " + Str( fs.Count )
            App.feAppMessage ( FCM_COMMAND,  s )
         End If
      End If
   Next i
End Sub
```

Reset

Reset	
(void)	
Description:	
This function resets the next pointer so that the next call to next will return the first entry in a selection set.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
To find the first ID in a set, you must use the `Reset` method before calling `Next`. If you have just created a set and have not used `Next`, it is already reset.	
Example:	
None	

Unpublished work. © 2025 Siemens

Next

Next	
(void)	
Description:	
This function returns the next selected ID in a selection set.	
Input:	
None	
Output:	
None	
Return Code:	
`Any Positive Number`	The next selected ID.
`FE_FAIL`	No additional IDs were selected.
Remarks/Usage:	
To find the first ID in a set, you must use the `Reset` method before calling `Next`. If you have just created a set and have not used `Next`, it is already reset. This function updates internal pointers in the Set so that if you call it repeatedly it will walk through all IDs.	
Example:	
None	

NextAfter

NextAfter
(nStartID)
Description:

Unpublished work. © 2025 Siemens

This function returns the next higher ID after the specified starting ID.	
Input:	
`INT4 nStartID`	The starting ID.
Output:	
None	
Return Code:	
Any Positive Number	The next ID higher/larger than the starting ID.
`FE_FAIL`	No additional IDs were selected.
Remarks/Usage:	
Unlike `Next( )`, this method does not rely on the internal pointers, nor `Reset( )` - it simply uses the starting ID that you supply.	
Example:	
None	

NextEmptyAfter

`NextEmptyAfter` **`( nStartID )`**	
Description:	
This function returns the first empty ID after the specified starting ID.	
Input:	
`INT4 nStartID`	The starting ID.
Output:	
None	
Return Code:	
Any Positive Number	The next empty ID higher/larger than the starting ID.

Unpublished work. © 2025 Siemens

`FE_FAIL`	No additional IDs were selected.
Remarks/Usage:	
Unlike `Next( )`, this method does not rely on the internal pointers, nor `Reset( )` - it simply uses the starting ID that you supply.	
Example:	
None	

NextID

`NextID` **`( void )`**	
Description:	
This function returns the next selected ID in a selection set.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The next selected ID.
`FE_FAIL`	No additional IDs were selected.
Remarks/Usage:	
This function is just like `Next()` except that it just returns the next ID. It does not increment the internal pointers, so if you repeatedly call `NextID( )`, you will get the same ID every time.	
Example:	
None	

Prev

Unpublished work. © 2025 Siemens

Prev	
(void)	
Description:	
This function returns the previous selected ID in a selection set.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The previous selected ID.
`FE_FAIL`	No additional IDs were selected.
Remarks/Usage:	
To find the last ID in a set, you must use the `Reset` method before calling `Prev`. If you have just created a set and have not used `Prev`, it is already reset. This function updates internal pointers in the Set so that if you call it repeatedly it will walk through all IDs.	
Example:	
None	

PrevBefore

PrevBefore	
(nStartID)	
Description:	
This function returns the next lower ID after the specified starting ID.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 nStartID`	The starting ID.
Output:	
None	
Return Code:	
Any Positive Number	The next ID lower/smaller than the starting ID.
`FE_FAIL`	No additional IDs were selected.
Remarks/Usage:	
Unlike `Prev( )`, this method does not rely on the internal pointers, nor `Reset( )` - it simply uses the starting ID that you supply.	
Example:	
None	

PrevID

`PrevID`
`( void )`
Description:
This function returns the previous selected ID in a selection set.
Input:
None
Output:
None
Return Code:
Any Positive Number — The previous selected ID.
`FE_FAIL` — No additional IDs were selected.
Remarks/Usage:

Unpublished work. © 2025 Siemens

This function is just like `Prev()` except that it just returns the previous ID. It does not increment the internal pointers, so if you repeatedly call `PrevID( )`, you will get the same ID every time.
Example:
None

First

`First` **`( void )`**	
Description:	
This function returns the first selected ID in a selection set.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The first selected ID.
`FE_FAIL`	No IDs were selected.
Remarks/Usage:	
The first ID is the lowest number, not the first entity added to the set. Calling this method is the same as calling `Reset( )` followed by `Next( )`.	
Example:	
None	

Last

Unpublished work. © 2025 Siemens

Last	
(void)	
Description:	
This function returns the last selected ID in a selection set.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The last selected ID.
`FE_FAIL`	No IDs were selected.
Remarks/Usage:	
The last ID is the highest number, not the last entity added to the set.	
Example:	
None	

Match

Match	
(setID)	
Description:	
This function tests to see if the contents of set `setID` match the current set.	
Input:	
`INT4 setID`	ID of the set to compare to this set.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Sets do not match.
Remarks/Usage:	
To match, the two sets must include all of the same IDs.	
Example:	
None	

Copy

Copy **(origID)**	
Description:	
This function makes a duplicate copy of a selection set.	
Input:	
`INT4 origID`	ID of the selection set to duplicate.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to create the specified set.
Remarks/Usage:	
If the current set already existed and contained selected IDs, those will be deleted prior to copying. The resulting copy will always match the original.	
Example:	
None	

CopyToClipboard

Unpublished work. © 2025 Siemens

CopyToClipboard	
(bAsList)	
Description:	
This function copies the IDs of the selected entities to the Clipboard as text.	
Input:	
`BOOL bAsList`	If True, the IDs are placed on the clipboard in a format that is one ID per line. If False, ranges of consecutive IDs are placed on the clipboard in the format `startID`, `stopID`, increment on each line. `stopID` is zero if a single ID is selected in this format.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Count

Count	
(void)	
Description:	
This function counts the number of IDs that are currently selected in the set.	
Input:	
None	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
count	The number of IDs in the set.
Remarks/Usage:	
This function simply looks at each of the selected IDs and counts the number that it finds.	
Example:	
None	

CountCommon

`CountCommon` **`( setID )`**	
Description:	
This function counts the number of IDs that are in the current set that are also in the specified set.	
Input:	
`INT4 setID`	The ID of another set to compare this set to.
Output:	
None	
Return Code:	
count	The number of common IDs.
Remarks/Usage:	
None	
Example:	
None	

CountNotCommon

Unpublished work. © 2025 Siemens

CountNotCommon	
(setID)	
Description:	
This function counts the number of IDs that are in the current set that are not in the specified set.	
Input:	
`INT4 setID`	The ID of another set to compare this set to.
Output:	
None	
Return Code:	
count	The number of IDs that are not in both sets.
Remarks/Usage:	
None	
Example:	
None	

DeleteAll

DeleteAll	
(void)	
Description:	
This function simply deletes Sets which are saved in the model.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

None	
Remarks/Usage:	
Great caution must be used when using this method. It deletes ALL sets in the model without any further notification. Any set objects in your program file will also be cleared.	
Example:	
None	

GetArray

`GetArray` **`( numID, arrayID )`**	
Description:	
This function populates an array of IDs from a selection set.	
Input:	
None	
Output:	
`INT4 numID`	The number of entries in the set.
`INT4 arrayID[0..numID-1]`	The array of all IDs in the set.
Return Code:	
`FE_NOT_EXIST`	No IDs are in the set.
`FE_NO_MEMORY`	Unable to allocate working memory to retrieve contents of set.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

ConvertToAllCurves

ConvertToAllCurves	
(void)	
Description:	
This function expands the current Set to contain all curves referenced by what is currently in the Set.	
Input:	
None	
Output:	
None	
Return Code:	
FE_OK	Expanded Set.
Remarks/Usage:	
If any Combined Curves are in the Set, then all curves used to define them are also added. If any Curves used to define Combined Curves are in the Set, then those Combined Curves are also added.	
Example:	
None	

ConvertToCombinedCurves

ConvertToCombinedCurves	
(void)	
Description:	
This function replaces curves that are used to define Combined Curves with the Combined Curve.	
Input:	
None	

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
FE_OK	Expanded Set.
Remarks/Usage:	
If any Curves used to define Combined Curves are in the Set, then those curves are removed, and the Combined Curves are added. Other Curves, either Combined Curves themselves, or curves not used to define Combined Curves are not changed in the Set.	
Example:	
None	

ConvertToCombinedCurvesOnly

ConvertToCombinedCurvesOnly **(void)**	
Description:	
This function replaces curves that are used to define Combined Curves with the Combined Curve and removes all others.	
Input:	
None	
Output:	
None	
Return Code:	
FE_OK	Expanded Set.
Remarks/Usage:	
If any Curves used to define Combined Curves are in the Set, then those curves are removed, and the Combined Curves are added. Other Curves, either Combined Curves themselves, or curves not used to define Combined Curves are all removed from the Set.	

Unpublished work. © 2025 Siemens

Example:
None

ConvertToInternalCurves

`ConvertToInternalCurves`	
`( void )`	
Description:	
This function replaces any Combined Curves in the Set with the curves used to define them.	
Input:	
None	
Output:	
None	
Return Code:	
FE_OK	Expanded Set.
Remarks/Usage:	
If any Combined Curves are in the Set, then those curves are removed, and the curves used to define them are added. Other curves are not changed in the Set.	
Example:	
None	

ConvertToAllSurfaces

`ConvertToAllSurfaces`
`( void )`
Description:

Unpublished work. © 2025 Siemens

This function expands the current Set to contain all surfaces (Parasolid or boundary surfaces) referenced by the surfaces currently in the set.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Expanded Set.
Remarks/Usage:	
If any Combined Surfaces are in the Set, then all surfaces used to define them are also added. If any Surfaces used to define Combined Surfaces are in the Set, then those Combined Surfaces are also added.	
Example:	
None	

ConvertToBoundarySurfaces

`ConvertToBoundarySurfaces`	
`( void )`	
Description:	
This function replaces surfaces that are used to define Combined Surfaces with the Combined Surface.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Expanded Set.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

If any Surfaces used to define Combined Surfaces are in the Set, then those surfaces are removed, and the Combined Surfaces are added. Other Surfaces, either Combined Surfaces themselves, or surfaces not used to define Combined Surfaces are not changed in the Set.
Example:
None

ConvertToBoundarySurfacesOnly

`ConvertToBoundarySurfacesOnly`	
`( void )`	
Description:	
This function replaces surfaces that are used to define Combined Surfaces with the Combined Surface and removes all others.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Expanded Set.
Remarks/Usage:	
If any Surfaces used to define Combined Surfaces are in the Set, then those surfaces are removed, and the Combined Surfaces are added. Other Surfaces, either Combined Surfaces themselves, or surfaces not used to define Combined Surfaces are all removed from the Set.	
Example:	
None	

ConvertToInternalSurfaces

Unpublished work. © 2025 Siemens

ConvertToInternalSurfaces	
(void)	
Description:	
This function replaces any Combined Surfaces in the Set with the surfaces used to define them.	
Input:	
None	
Output:	
None	
Return Code:	
FE_OK	Expanded Set.
Remarks/Usage:	
If any Combined Surfaces are in the Set, then those surfaces are removed, and the surfaces used to define them are added. Other surfaces are not changed in the Set.	
Example:	
None	

EmptyBlock

EmptyBlock	
(nStartID, nCount, nFirstEmptyID)	
Description:	
This function returns the first empty ID of an empty block of IDs of a specified size, after the specified starting ID.	
Input:	
INT4 nStartID	ID used to begin searching.
INT4 nCount	Size of empty block needed to be found.

Unpublished work. © 2025 Siemens

Output:	
`INT4 nFirstEmptyID`	First available ID in the set where an empty block of the size specified in `nCount` exists.
Return Code:	
`FE_FAIL`	A block of the given size could not be found in the search range.
Remarks/Usage:	
As an example, a set object contains IDs 1-10 and 12-20. If `nCount` is set to 1, then the `nFirstEmptyID` will be returned as 11, as that is the first ID where an empty block of 1 is available. If, however, `nCount` is set to 2, then the `nFirstEmptyID` will be returned as 21, as there are no empty blocks of 2 IDs available within the IDs already entered into the set. Finally, if `nStartID` is set to 12 and `nCount` is set to 1, then `nFirstEmptyID` will be returned as 21, as the available block at 11 is not included in the search.	
Example:	
None	

Debug

Debug **(void)**	
Description:	
This function simply writes all of the IDs contained in a Set to the Message Window.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	
Remarks/Usage:	
This method is really only intended for Debugging. There is no particular formatting of the list of IDs (which can be quite long), it is simply dumped in a comma-separated fashion to the Message Window.	

Unpublished work. © 2025 Siemens

Example:
None

AddAllSavedSetIDs

AddAllSavedSetIDs	
(void)	
Description:	
This method will add the IDs of the Sets currently saved to this set.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Show

Show
(entityType)
Description:
Uses the current settings of the **Window** → **Show Entities** command to highlight all of the entities of the selected entity type currently in the group, in the graphics window.

Unpublished work. © 2025 Siemens

Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see Entity Types.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

ShowAutoscale

`ShowAutoscale`
`( entityType )`
Description:
Uses the current settings of the **Window** → **Show Entities** command, other than the **Autoscale** option, to highlight all of the entities of the selected entity type currently in the group, in the graphics window. This method is essentially identical to the Show method, with the only different being this method always autoscales to show only the entities in the group.
Input:

`INT4 entityTYPE`	Type of entity to select. For more information, see Entity Types.

Output:
None
Return Code:
None
Remarks/Usage:
None

Unpublished work. © 2025 Siemens

Example:
None

IsEmpty

`IsEmpty`
 	
`( void )`	

Description:	
This method checks if a set has no entities added to it, thus is empty.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Set is empty.
`FE_FAIL`	Set is not empty (i.e., has entities).
Remarks/Usage:	
None	
Example:	
None	

IsNotEmpty

Unpublished work. © 2025 Siemens

IsNotEmpty	
(void)	
Description:	
This method checks if a set has one or more entities added to it, thus is NOT empty.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Set is NOT empty.
`FE_FAIL`	Set is empty (i.e., it does NOT have entities).
Remarks/Usage:	
API developers should employ this method instead of testing whether `Count() > 0`.	
Example:	
None	

HasOne

HasOne	
(void)	
Description:	
This method checks if a set has only one entity added to it.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_OK`	Set has one entity added to it.
`FE_FAIL`	Set does NOT have one entity added to it.
Remarks/Usage:	
API developers should employ this method instead of testing whether `Count() == 1`.	
Example:	
None	

HasMultiple

`HasMultiple` **`( void )`**	
Description:	
This method checks if a set has multiple entities added to it.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Set has multiple entities added to it.
`FE_FAIL`	Set does NOT have multiple entities added to it.
Remarks/Usage:	
API developers should employ this method instead of testing whether `Count() > 1`.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddAllAnalysisCases

AddAllAnalysisCases **(nAnalysisSetID)**	
Description:	
This method adds the IDs of all Analysis Cases in the specified Analysis Set to the Set object.	
Input:	
`INT4 nAnalysisSetID`	ID of Analysis Set containing Analysis Cases to be added.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	No analysis cases currently exist in the Analysis Set.
Remarks/Usage:	
None	
Example:	
None	

AddSetFromModel

AddSetFromModel **(addID, nModelID, bClear)**	
Description:	
This method adds the IDs of an set which exists in another model into a set in the current model.	
Input:	
`INT4 addID`	ID of a Set that exists in the model specified in `nModelID`.

Unpublished work. © 2025 Siemens

`INT8 nModelID`	Model identifier.
`BOOL bClear`	If True, clears the set in the current model before adding IDs from set in the other model.
Output:	
None	
Return Code:	
`FE_OK`	Set specified by `addID` has been added from other model. `FE_OK` also returned if the set is empty or set does not exist.
`FE_FAIL`	Anything other the `nModelID` caused the call to not work properly.
`FE_NOT_AVAILABLE`	`nModelID` is not and open document/model.
Remarks/Usage:	
None	
Example:	
None	

Set Object SavedSet Methods

SetDeveloperID

`SetDeveloperID`
`( DeveloperID )`
Description:
This method is used to specify a `DeveloperID` and unlocks the `SavedSet` functionality.
Input:

`INT4 DeveloperID`	The developer ID assigned by Siemens to the registered developer.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	The specified `DeveloperID` is not valid in this version of Femap.
Remarks/Usage:	
This method only needs to be called once per object, but must be called prior to any of the other `SavedSet` type methods otherwise they will all fail. Using this, and the other `SavedSet` methods does not prevent the use of any other Set object functionality.	
Example:	
None	

GetSavedSet

`GetSavedSet` **`( nSavedID, bClear )`**	
Description:	
This method loads the IDs stored in a Saved Set into the current object.	
Input:	
`INT4 nSavedID`	The ID of the Saved Set.
`BOOL bClear`	If True, any previous contents of the object are removed. If False, the Saved Set is merged with the current contents.
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID`, specified an invalid `DeveloperID`, or the selected Saved Set was stored by a different registered developer.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	The specified Saved Set does not exist.
Remarks/Usage:	
You can only access Saved Sets that were stored with your `DeveloperID` using this method.	
Example:	
None	

GetAllSavedSets

`GetAllSavedSets` **`( void )`**	
Description:	
This method loads the IDs of all `SavedSets` into the current object.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID` or specified an invalid `DeveloperID`.
Remarks/Usage:	
Any previous contents of the Set object are automatically cleared prior to loading the IDs of the Saved Sets.	
Example:	
None	

PutSavedSet

Unpublished work. © 2025 Siemens

PutSavedSet

(nPutID, entityTYPE, nSavedID)

Description:

This method saves the contents of the Set object to a selected SavedSet.

Input:

INT4 nPutID	The ID of the Saved Set to create or overwrite. To create a new Saved Set, specify nPutID=0 (or any negative number). If you are overwriting a previously Saved Set, set nPutID to the value that was returned in nSavedID when you created the SavedSet.
INT4 entityTYPE	Type of entity whose IDs are in the Set. For more information, see Entity Types. Currently, the following types will automatically support renumbering and locking entities from delete: FT_POINT, FT_CURVE, FT_SURFACE, FT_SOLID, FT_NODE, FT_ELEM, FT_MATL, FT_PROP, FT_CSYS and FT_FUNCTION_DIR.

Output:

INT4 nSavedID	The ID of the Saved Set that was created or overwritten. If nPutID was 0 (or negative) this is the ID of the new Saved Set that you must remember to retrieve the set later. If nPutID was a positive number nSaveID will simply equal nPutID.

Return Code:

FE_INVALID_DEVELOPER	You either did not specify a DeveloperID, specified an invalid DeveloperID, or the selected SavedSet was already stored by a different registered developer.
FE_BAD_TYPE	The selected SavedSet already exists and contains a different entityTYPE than what you specified.
FE_FAIL	Storing the SavedSet failed.

Remarks/Usage:

If you save a Set using this method, its current contents are copied to the Saved Set. Future changes to the Set object are not reflected in the Saved Set unless you also call PutSavedSet again after those changes.

Unpublished work. © 2025 Siemens

You can specify `nPutID` as a positive number to create a new Saved Set with a fixed ID but this is not recommended. There is no guarantee that some other developer/application has not already used that ID. If you do use a positive number and the Saved Set matches your Developer ID it will be overwritten, but if the Developer ID does not match the *Put* will simply fail. That is why it is always best to allow Femap to find an available ID for new Saved Sets.
Example:
None

DeleteSavedSet

`DeleteSavedSet` **`( nSavedID )`**	
Description:	
This method deletes a `SavedSet` which was previously saved.	
Input:	
`nSavedID`	The ID of the `SavedSet` to delete. You can only delete `SavedSets` that were stored using your `DeveloperID`.
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID`, specified an invalid `DeveloperID`, or the selected `SavedSet` was already stored by a different registered developer.
`FE_NOT_EXIST`	The selected `SavedSet` does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

DeleteAllSavedSets

DeleteAllSavedSets	
(void)	
Description:	
This method deletes all previously `SavedSets`.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID` or specified an invalid `DeveloperID`.
Remarks/Usage:	
Only Saved Sets stored with your `DeveloperID` are deleted by this method.	
Example:	
None	

DeleteOtherSavedSets

DeleteOtherSavedSets	
(void)	
Description:	
This method deletes any previously `SavedSet` that has an ID that is not specified in the current object.	
Input:	
None	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID` or specified an invalid `DeveloperID`.
`FE_BAD_DATA`	The current object is empty. If you want to delete all `SavedSets` use `DeleteAllSavedSets( )` instead.
Remarks/Usage:	
Only `SavedSets` stored with your `DeveloperID` are deleted by this method. This method is intended to be used to cleanup any *leaked* `SavedSets`. Load this Set object with all the `SavedSet` IDs that you want to keep, then call this method... it will delete all others. You should follow this procedure periodically to ensure the database does not contain unnecessary orphaned data. When a user calls the **File→ Rebuild** command is a good time to do this, although it can be done more often.	
Example:	
None	

AddToSavedSet

`AddToSavedSet`	
`( nPutID, entityTYPE, nEntID )`	
Description:	
This method adds a single entity ID to a `SavedSet`.	
Input:	
`INT4 nPutID`	The ID of the `SavedSet` to create or overwrite. To create a new `SavedSet` you should always use `EmptySavedSet` method to find the ID of an available set.
`INT4 entityTYPE`	Type of entity whose IDs are in the Set. For more information, see **Entity Types**. Currently, the following types will automatically support renumbering and locking entities from delete: `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_CSYS` and `FT_FUNCTION_DIR`.

Unpublished work. © 2025 Siemens

`INT4 nEntID`	The ID of the entity to add to the `SavedSet`.
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID`, specified an invalid `DeveloperID`, or the selected `SavedSet` was already stored by a different developer.
`FE_BAD_TYPE`	The selected `SavedSet` already exists and contains a different entityTYPE than what you specified.
`FE_FAIL`	Storing the Set failed.
`FE_INVALID`	`nPutID` is not a valid ID for a `SavedSet`.
Remarks/Usage:	
This method can be called to add the same entity into a `SavedSet` multiple times however it will only be added once. In addition, a single `RemoveFromSavedSet` will eliminate the entity from the `SavedSet`.	
Example:	
None	

AddSetToSavedSet

`AddSetToSavedSet`
`( nPutID, entityTYPE )`
Description:
This method adds the contents of the current Set into a `SavedSet`.
Input:

`INT4 nPutID`	The ID of the `SavedSet` to create or overwrite. To create a new `SavedSet` you should always use `EmptySavedSet` method to find the ID of an available set.

Unpublished work. © 2025 Siemens

`INT4 entityTYPE`	Type of entity whose IDs are in the Set. For more information, see **Entity Types**. Currently, the following types will automatically support renumbering and locking entities from delete: `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_CSYS` and `FT_FUNCTION_DIR`.
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID`, specified an invalid `DeveloperID`, or the selected `SavedSet` was already stored by a different developer.
`FE_BAD_TYPE`	The selected `SavedSet` already exists and contains a different entityTYPE than what you specified.
`FE_FAIL`	Storing the Set failed.
`FE_INVALID`	`nPutID` is not a valid ID for a `SavedSet`.
Remarks/Usage:	
This method can be called to add the same Set into a `SavedSet` multiple times however the entities in the Set will only be added once. In addition, a single `RemoveSetFromSavedSet` will eliminate the entities in the Set from the `SavedSet`.	
Example:	
None	

RemoveFromSavedSet

`RemoveFromSavedSet`
`( nPutID, entityTYPE, nEntID )`
Description:
This method removes a single entity ID from a `SavedSet`.
Input:

Unpublished work. © 2025 Siemens

`nPutID`	The ID of the SavedSet to be modified.
`entityTYPE`	Type of entity whose IDs are in the Set. For more information, see **Entity Types**. Currently, the following types will automatically support renumbering and locking entities from delete:`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, `FT_ELEM`, `FT_MATL`, `FT_PROP`, `FT_CSYS` and `FT_FUNCTION_DIR`.
`nEntID`	The ID of the entity to remove from the `SavedSet`.
Output:	
None	
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID`, specified an invalid `DeveloperID`, or the selected `SavedSet` was already stored by a different developer.
`FE_BAD_TYPE`	The selected `SavedSet` already exists and contains a different entityTYPE than what you specified.
`FE_FAIL`	Storing the Set failed.
`FE_INVALID`	`nPutID` is not a valid ID for a `SavedSet`.
Remarks/Usage:	
This method can be called to remove an entity that is not actually contained in the selected `SavedSet` without causing any harm.	
Example:	
None	

RemoveSetFromSavedSet

Unpublished work. © 2025 Siemens

RemoveSetFromSavedSet

(nPutID, entityTYPE)

Description:

This method removes the contents of the current Set from a SavedSet.

Input:

nPutID	The ID of the Saved Set to be modified.
entityTYPE	Type of entity whose IDs are in the Set. For more information, see **Entity Types**. Currently, the following types will automatically support renumbering and locking entities from delete:FT_POINT, FT_CURVE, FT_SURFACE, FT_SOLID, FT_NODE, FT_ELEM, FT_MATL, FT_PROP, FT_CSYS and FT_FUNCTION_DIR.

Output:

None	

Return Code:

FE_INVALID_DEVELOPER	You either did not specify a DeveloperID, specified an invalid DeveloperID, or the selected SavedSet was already stored by a different developer.
FE_BAD_TYPE	The selected SavedSet already exists and contains a different entityTYPE than what you specified.
FE_FAIL	Storing the Set failed.
FE_INVALID	nPutID is not a valid ID for a SavedSet.

Remarks/Usage:

This method can be called to remove entities that are not actually contained in the selected SavedSet without causing any harm. Only entities that are actually in the SavedSet will be removed.

Example:

None

Unpublished work. © 2025 Siemens

ResetSavedSet

ResetSavedSet	
(void)	
Description:	
This method resets the `SavedSet` next pointer so that the next call to `NextSavedSet` will return the first available `SavedSet`.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
Only `SavedSets` stored with your `DeveloperID` will be accessible by the `NextSavedSet` method.	
Example:	
None	

NextSavedSet

NextSavedSet	
(void)	
Description:	
This method returns the ID of the next `SavedSet`.	
Input:	
None	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
Any Positive Integer	The Next available Saved Set.
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID` or specified an invalid `DeveloperID`.
`FE_FAIL`	No additional `SavedSets` were found.
Remarks/Usage:	
Only `SavedSets` stored with your `DeveloperID` are accessible by this method. To find the first `SavedSet`, you must first use the `ResetSavedSet` method before calling `NextSavedSet` This function updates internal pointers in the Set so that if you call it repeatedly it will walk through all available `SavedSets`.	
Example:	
None	

EmptySavedSet

`EmptySavedSet`
`( void )`
Description:
This method returns the ID of an available/nonexisting `SavedSet`.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
This method always returns the lowest ID available. You do not need to set a `DeveloperID` to use this method.

Unpublished work. © 2025 Siemens

Example:
None

CountSavedSets

CountSavedSets **(AllSets)**	
Description:	
This method returns the number of `SavedSets` in the Database... both the number matching your `DeveloperID` and the total number.	
Input:	
None	
Output:	
`INT4 AllSets`	The total number of `SavedSets` in the database from all developers.
Return Code:	
Any Positive Integer or Zero	The number of `SavedSets` in the database matching your specified `DeveloperID`.
Remarks/Usage:	
You do not need to set a `DeveloperID` to use this method to retrieve `AllSets`, but if you do not, the return code will always be 0.	
Example:	
None	

GetSavedSetType

Unpublished work. © 2025 Siemens

GetSavedSetType **(SavedSetID, enDataType)**	
Description:	
This method returns the type of entities currently stored in a `SavedSet`.	
Input:	
`INT4 SavedSetID`	The ID of the `SavedSet` to check entity type.
Output:	
`INT4 enDataType`	Type of entities found in the Saved Set. For more information, see **Entity Types**.
Return Code:	
`FE_INVALID_DEVELOPER`	You either did not specify a `DeveloperID`, specified an invalid `DeveloperID`, or the selected `SavedSet` was already stored by a different developer.
`FE_NOT_EXIST`	The specified `SavedSet` does not currently exist.
Remarks/Usage:	
This method does not actually get the Saved Set, only the type of entities that are currently stored in the Saved Set.	
Example:	
None	

Sort Objects

The Sort object is similar to the Set object, in that it is used to contain lists of IDs. This object however holds the IDs in a prescribed order - normally the order they were added to the object. The order however can be changed by sorting based on various criteria - hence the name of the object. This object is not used directly as input to other objects or methods, rather it can be used in your code to traverse entities in a particular sorted order.

Use the `feSort` method of the Femap Application object to create Sort objects.

Unpublished work. © 2025 Siemens

Sort Object Properties

Property	Description
`INT4 ID`	The internal ID of the object.
`REAL8 dReal1`	Real Values associated with an entry in the object.
`REAL8 dReal2`	
`REAL8 dReal3`	
`INT4 nInt1`	Integer Values associated with an entry in the object.
`INT4 nInt2`	
`INT4 nInt3`	
`INT4 eDataType`	Entity Type that is associated with an entry in the object. For possible values, refer to **Entity Types**.

Sort Object Methods

Add

Add
`( id )`
Description:
This function adds an ID into a sort object.
Input:

`INT4 id`	ID to add to the sort object.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value or memory can not be allocated for additional IDs.
`FE_NOT_AVAILABLE`	ID has already been added into object. IDs can only be added once.
Remarks/Usage:	
The ID that you are adding to the sort object is just that, an ID. Although it usually represents the ID of some other existing entity in your model, there is no requirement for this. This method adds the ID and associates the current values of the `dRealX`, `nIntX` and `eDataType` properties with this ID. Therefore, if you want to later sort this object, you can first set the property values, then call this method to add an entry. Alternatively, you can use one of the other Add type functions to load both the ID and properties in one call.	
Example:	
None	

AddReal

`AddReal`	
`( id, dReal1, dReal2, dReal3 )`	
Description:	
This function adds an ID and associated real properties into a sort object.	
Input:	
`INT4 id`	ID to add to the sort object.
`REAL8 dReal1`	Real properties to associate with ID.
`REAL8 dReal2`	
`REAL8 dReal3`	

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value or memory can not be allocated for additional IDs.
`FE_NOT_AVAILABLE`	ID has already been added into object. IDs can only be added once.
Remarks/Usage:	
The ID that you are adding to the sort object is just that, an ID. Although it usually represents the ID of some other existing entity in your model, there is no requirement for this. This method adds the ID and sets the values of the `dRealX` properties. The current values of `nIntX` and `eDataType` properties are also associated with this ID.	
Example:	
None	

AddInt

`AddInt` **`( id, nInt1, nInt2, nInt3 )`**	
Description:	
This function adds an ID and associated integer properties into a sort object.	
Input:	
`INT4 id`	ID to add to the sort object.
`INT4 nInt1`	Integer properties to associate with ID.
`INT4 nInt2`	
`INT4 nInt3`	
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value or memory can not be allocated for additional IDs.
`FE_NOT_AVAILABLE`	ID has already been added into object. IDs can only be added once.
Remarks/Usage:	
The ID that you are adding to the sort object is just that, an ID. Although it usually represents the ID of some other existing entity in your model, there is no requirement for this. This method adds the ID and sets the values of the `nIntX` properties. The current values of `dRealX` and `eDataType` properties are also associated with this ID.	
Example:	
None	

AddAll

`AddAll` **`( id, dReal1, dReal2, dReal3, nInt1, nInt2, nInt3, eDataType )`**	
Description:	
This function adds an ID and associated integer properties into a sort object.	
Input:	
`INT4 id`	ID to add to the sort object.
`REAL8 dReal1`	Real properties to associate with ID.
`REAL8 dReal2`	
`REAL8 dReal3`	
`INT4 nInt1`	Integer properties to associate with ID.

Unpublished work. © 2025 Siemens

`INT4 nInt2`	
`INT4 nInt3`	
`INT4 eDataType`	Entity Type that is associated with an entry in the object. For possible values, refer to **Entity Types**.
Output:	
None	
Return Code:	
`FE_FAIL`	ID specifies an invalid value or memory can not be allocated for additional IDs.
`FE_NOT_AVAILABLE`	ID has already been added into object. IDs can only be added once.
Remarks/Usage:	
The ID that you are adding to the sort object is just that, an ID. Although it usually represents the ID of some other existing entity in your model, there is no requirement for this. This method adds the ID and sets the values of all of the properties.	
Example:	
None	

SetData

`SetData` **`( id, dReal1, dReal2, dReal3, nInt1, nInt2, nInt3, eDataType )`**	
Description:	
This function updates the properties associated with an already added ID.	
Input:	
`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
`REAL8 dReal1`	Real properties to associate with ID.

Unpublished work. © 2025 Siemens

`REAL8 dReal2`	
`REAL8 dReal3`	
`INT4 nInt1`	Integer properties to associate with ID.
`INT4 nInt2`	
`INT4 nInt3`	
`INT4 eDataType`	Entity Type that is associated with an entry in the object. For possible values, refer to **Entity Types**.
Output:	
None	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
As an alternative to setting the properties prior to, or at the same time as you add IDs into the object, you can simply add IDs and update the associated values with this method.	
Example:	
None	

SetReal

`SetReal`
`( id, dReal1, dReal2, dReal3 )`
Description:
This function updates the real properties associated with an already added ID.
Input:

`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.

Unpublished work. © 2025 Siemens

`REAL8 dReal1`	Real properties to associate with ID.
`REAL8 dReal2`	
`REAL8 dReal3`	
Output:	
None	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
As an alternative to setting the properties prior to, or at the same time as you add IDs into the object, you can simply add IDs and update the associated real property values with this method.	
Example:	
None	

SetInt

`SetInt` **`( id, nInt1, nInt2, nInt3 )`**	
Description:	
This function updates the integer properties associated with an already added ID.	
Input:	
`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
`INT4 nInt1`	Integer properties to associate with ID.
`INT4 nInt2`	
`INT4 nInt3`	

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
As an alternative to setting the properties prior to, or at the same time as you add IDs into the object, you can simply add IDs and update the associated integer property values with this method.	
Example:	
None	

SetDataType

`SetDataType` **`( id, eDataType )`**	
Description:	
This function updates the entity type associated with an already added ID.	
Input:	
`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
`INT4 eDataType`	Entity Type that is associated with an entry in the object. For possible values, refer to **Entity Types**.
Output:	
None	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
As an alternative to setting the properties prior to, or at the same time as you add IDs into the object, you can simply add IDs and update the associated entity type with this method.	

Unpublished work. © 2025 Siemens

Example:
None

GetData

`GetData` **`( id, dReal1, dReal2, dReal3, nInt1, nInt2, nInt3, eDataType )`**	
Description:	
This function retrieves the properties associated with an already added ID.	
Input:	
`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
Output:	
`REAL8 dReal1`	Real properties associated with ID.
`REAL8 dReal2`	
`REAL8 dReal3`	
`INT4 nInt1`	Integer properties associated with ID.
`INT4 nInt2`	
`INT4 nInt3`	
`INT4 eDataType`	Entity Type that is associated with an entry in the object. For possible values, refer to **Entity Types**.
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetReal

GetReal (id, dReal1, dReal2, dReal3)	
Description:	
This function retrieves the real properties associated with an already added ID.	
Input:	
`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
Output:	
`REAL8 dReal1`	Real properties associated with ID.
`REAL8 dReal2`	
`REAL8 dReal3`	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
None	
Example:	
None	

GetInt

Unpublished work. © 2025 Siemens

GetInt	
(id, nInt1, nInt2, nInt3)	
Description:	
This function retrieves the integer properties associated with an already added ID.	
Input:	
`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
Output:	
`INT4 nInt1`	Integer properties associated with ID.
`INT4 nInt2`	
`INT4 nInt3`	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
None	
Example:	
None	

GetDataType

GetDataType
(id, eDataType
Description:
This function retrieves the entity type associated with an already added ID.
Input:

Unpublished work. © 2025 Siemens

`INT4 id`	ID to update in the sort object. This ID must have been already added to the object.
Output:	
`INT4 eDataType`	Entity Type that is associated with an entry in the object. For possible values, refer to **Entity Types**.
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
Remarks/Usage:	
None	
Example:	
None	

GetSet

`GetSet` **`( nSetID, bClear )`**
Description:
This function populates a set object with the IDs currently in the sort object, with the option to add the IDs to the IDs already in the set object or clear the set object first, so the set object will only contain the entities from the sort object.
Input:

`INT4 nSetID`	ID of the set object.
`BOOL bClear`	Flag to indicate if the set object should be cleared of all existing IDs before IDs from sort object are sent to set object. True=clear, then add, False=add to existing IDs.
Output:	
None	
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Find

Find **(id)**	
Description:	
This function locates an ID within the object.	
Input:	
`INT4 id`	ID to locate.
Output:	
None	
Return Code:	
`FE_FAIL`	The specified ID could not be found in the object.
`FE_NOT_AVAILABLE`	ID specifies an invalid value.
Remarks/Usage:	
If the ID is found, the `dRealX`, `nIntX` and `eDataType` properties are all updated to match those associated with the ID.	
Example:	
None	

Remove

Unpublished work. © 2025 Siemens

Remove	
(id)	
Description:	
This function removes an ID from a sort object.	
Input:	
`INT4 id`	ID to remove from the sort object.
Output:	
None	
Return Code:	
`FE_NOT_AVAILABLE`	ID specifies an invalid value.
Remarks/Usage:	
None	
Example:	
None	

RemoveSet

RemoveSet	
(id)	
Description:	
This function removes all of the selected IDs in another sort object from the current sort object.	
Input:	
`INT4 id`	ID of the sort set containing the IDs to remove.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_AVAILABLE`	Unable to remove IDs from the specified sort set.
Remarks/Usage:	
None	
Example:	
None	

IsAdded

`IsAdded` **`( id )`**	
Description:	
This function checks to see if an ID is currently added into a sort object.	
Input:	
`INT4 id`	ID to check.
Output:	
None	
Return Code:	
`FE_OK`	ID is added into the sort object.
`FE_FAIL`	ID is not added into the sort object.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Clear

Clear	
(void)	
Description:	
This function simply removes all IDs from the current object.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Reset

Reset	
(void)	
Description:	
This function resets the next pointer so that the next call to next will return the first entry in a sort object.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
To find the first ID in a set, you must use the `Reset` method before calling `Next`. If you have just created a set and have not used `Next`, it is already reset.	
Example:	
None	

Next

`Next` **`( void )`**	
Description:	
This function returns the next selected ID in a sort object.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The next ID.
`FE_FAIL`	No additional IDs were available.
Remarks/Usage:	
To find the first ID in a set, you must use the `Reset` method before calling `Next`. If you have just created a set and have not used `Next`, it is already reset. When you retrieve the ID, the properties `dRealX`, `nIntX` and `eDataType` are all udpated to match the values associated with that ID.	
Example:	
None	

Unpublished work. © 2025 Siemens

Prev

Prev	
(void)	
Description:	
This function returns the previous selected ID in a sort object.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The previous ID.
`FE_FAIL`	No additional IDs were available.
Remarks/Usage:	
To find the last ID in a set, you must use the Reset method before calling `Prev`. If you have just created a set and have not used `Prev` (or `Next`), it is already reset. When you retrieve the ID, the properties `dRealX`, `nIntX` and `eDataType` are all udpated to match the values associated with that ID.	
Example:	
None	

Current

Current
(void)
Description:
This function returns the currently selected ID in a sort object.

Unpublished work. © 2025 Siemens

Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The current ID.
`FE_FAIL`	No ID was available.
Remarks/Usage:	
This method works just like `Next()` and `Prev()`, except that it does not move to a different ID. By calling this method, you retrieve the ID and the properties `dRealX`, `nIntX` and `eDataType` are all udpated to match the values associated with that ID.	
Example:	
None	

First

`First` **`( void )`**
Description:
This function returns the first ID in a sort object.
Input:
None
Output:
None
Return Code:
Any Positive Number — The first ID.
`FE_FAIL` — No IDs were available.
Remarks/Usage:

Unpublished work. © 2025 Siemens

The first ID is the first ID in the object, either the first added, or the first after a sort. Calling this method is the same as calling `Reset( )` followed by `Next( )`. When you retrieve the ID, the properties `dRealX`, `nIntX` and `eDataType` are all udpated to match the values associated with that ID.
Example:
None

Last

`Last` **`( void )`**	
Description:	
This function returns the last selected ID in a sort object.	
Input:	
None	
Output:	
None	
Return Code:	
Any Positive Number	The last ID.
`FE_FAIL`	No IDs were available.
Remarks/Usage:	
The last ID is the last ID in the object, either the first added, or the first after a sort. Calling this method is the same as calling `Reset( )` followed by `Prev( )`. When you retrieve the ID, the properties `dRealX`, `nIntX` and `eDataType` are all udpated to match the values associated with that ID.	
Example:	
None	

Unpublished work. © 2025 Siemens

Count

Count	
(void)	
Description:	
This function counts the number of IDs that are currently selected in the object.	
Input:	
None	
Output:	
None	
Return Code:	
count	The number of IDs in the object.
Remarks/Usage:	
This function simply looks at each of the selected IDs and counts the number that it finds.	
Example:	
None	

Select

Select	
(entityTYPE, reload, title)	
Description:	
This function displays a standard selection dialog box to allow a user to choose entities of a specific type and create a sort object.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.

Unpublished work. © 2025 Siemens

`BOOL reload`	If True, previously selected IDs will be displayed in the dialog box for editing, or for combining with new selections.
`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
Output:	
None	
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No entities of the selected type exist. None were selected and the dialog box was not displayed. If `clear=True`, the sort object will be empty.
Remarks/Usage:	
The final selection list contains all of the IDs that were selected when the user pressed OK. If `clear=True`, these are all entities that were just selected. If `clear=False`, it is a combination of old and new entities.	
Example:	
None	

SelectID

`SelectID`	
`( entityTYPE, title, ID )`	
Description:	
This function displays a selection dialog box to allow a user to choose a single entity of a specific type. The ID of the entity is returned and is stored in the current sort object.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.

Unpublished work. © 2025 Siemens

`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
Output:	
`INT4 ID`	The ID of the entity that was selected.
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No entities of the selected type exist. None were selected and the dialog box was not displayed. If `clear=True`, the sort object will be empty.
Remarks/Usage:	
This function displays one of two different dialog boxes, depending on the entity type that you choose. For entities that have titles, like properties or materials, the dialog box will contain a list of the available entities along with their titles. For other entities, like nodes or elements, a simple dialog box is displayed so that the ID can be typed or picked.	
Example:	
None	

SelectList

`SelectList`
`( nNumEntity, entityTYPE )`
Description:
This function displays a selection dialog box to allow a user to choose a specified number of entities of a specific type. The IDs of the selected entities are stored in the current sort set.
Input:

`INT4 nNumEntity`	Number of entities to be selected.
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
Remarks/Usage:	
The dialog box automatically closes when `nNumEntity` entities have been selected.	
Example:	
None	

Sort

`Sort`
`( bSortReal )`
Description:
This function sorts the IDs in the object based on values associated with each ID.
Input:

`BOOL bSortReal`	If True, the IDs are sorted based on the `dRealX` values, if False, the sort is based on the `nIntX` values.
Output:	
None	
Return Code:	
`FE_FAIL`	No IDs are available for sorting.
Remarks/Usage:	
Sorting is always done in increasing order, and always using all three values. If `bSortReal` is True, then the primary sort value is `dReal1`, the second is `dReal2` and third is `dReal3`. The same approach follows for `nInt1`, `nInt2`, and `nInt3`. Even though sorting is always done in one direction, you can traverse the list in either direction using either `Next( )` or `Prev( )`, so you can still retrieve IDs in decreasing order.	

Unpublished work. © 2025 Siemens

Example:
None

SortRemoveDuplicates

SortRemoveDuplicates **(bSortReal)**	
Description:	
This function is the same as `Sort( )` except it also removes duplicate entries from the sorted object, leaving just one of each unique entry.	
Input:	
`BOOL bSortReal`	If True, the IDs are sorted based on the `dRealX` values, if False, the sort is based on the `nIntX` values.
Output:	
None	
Return Code:	
`FE_FAIL`	No IDs are available for sorting.
Remarks/Usage:	
Sorting is always done in increasing order, and always using all three values. If `bSortReal` is True, then the primary sort value is `dReal1`, the second is `dReal2` and third is `dReal3`. The same approach follows for `nInt1`, `nInt2`, and `nInt3`. Even though sorting is always done in one direction, you can traverse the list in either direction using either `Next( )` or `Prev( )`, so you can still retrieve IDs in decreasing order.	
Example:	
None	

ConvertToBoundarySurfaces

Unpublished work. © 2025 Siemens

ConvertToBoundarySurfaces	
(void)	
Description:	
This function replaces surfaces that are used to define Combined Surfaces with the Combined Surface.	
Input:	
None	
Output:	
None	
Return Code:	
FE_OK	Expanded Set.
Remarks/Usage:	
If any Surfaces used to define Combined Surfaces are in the Sort Set, then those surfaces are removed, and the Combined Surfaces are added. Other Surfaces, either Combined Surfaces themselves, or surfaces not used to define Combined Surfaces are not changed in the Sort Set.	
Example:	
None	

AddAroundPoint

AddAroundPoint	
(entityTYPE, nSetID, vBase)	
Description:	
This function adds selected entities with their sort value equal to the distance from the specified location.	
Input:	
INT4 entityTYPE	Type of entity to select. For more information, see **Entity Types**.
INT4 nSetID	The ID of a Set object that contains the entities to add.

Unpublished work. © 2025 Siemens

`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the point used to compute the sort value.
Output:	
None	
Return Code:	
`FE_FAIL`	specified entities do not exist.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
None	
Example:	
None	

AddAroundVector

`AddAroundVector`
`( entityTYPE, nSetID, vBase, vVec )`
Description:
This function adds selected entities with their sort value equal to the distance from the specified vector.
Input:

`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 nSetID`	The ID of a Set object that contains the entities to add.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the base of the vector used to compute the sort value.
`REAL8 vVec[3]`	The components, in Global Rectangular coordinates, of the vector used to compute the sort value.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	specified entities do not exist.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
None	
Example:	
None	

AddAroundPlane

`AddAroundPlane` **`( entityTYPE, nSetID, vBase, vVec, bAbs )`**	
Description:	
This function adds selected entities with their sort value equal to the distance from the specified plane.	
Input:	
`INT4 entityTYPE`	Type of entity to select. For more information, see **Entity Types**.
`INT4 nSetID`	The ID of a Set object that contains the entities to add.
`REAL8 vBase[3]`	The coordinates, in Global Rectangular coordinates, of the base of the vector used to compute the sort value.
`REAL8 vVec[3]`	The components, in Global Rectangular coordinates, of the vector used to compute the sort value.
`BOOL bAbs`	If False, the sort value is computed as positive if the entity is on the side of the plane pointed to by `vVec` and negative if in the other direction. If True, the sort value is simply the distance to the plane.

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
`FE_FAIL`	specified entities do not exist.
`FE_BAD_TYPE`	not a valid `entityTYPE` for this method.
Remarks/Usage:	
None	
Example:	
None	

UserData Objects

The `UserData` object is used to store your own data in the Femap database. You can think of each `UserData` object as a binary file, where you can read and write any data that you need to keep with a model.

Even though you can always store data in your own external files, `UserData` objects provide some very beneficial features. Your data is stored in the same file as the other model data. This eliminates many file/data management issues, and possibilities for errors. Data stored in the same model is always available to that model, and cannot be mixed with other models. Possibly more importantly, however, data stored in `UserData` objects is supported through the Femap Undo/Redo system. If your data is linked to the current model state, then rolling the model back through undo will also roll back your data. This only applies to model that is in the database, not data that is already loaded into a `UserData` Object. Following an Undo/Redo, you must destroy any objects that you are holding, and reload them from the database.

Use the `feUserData` method of the Femap Application object to create `UserData` Objects.

UserData Object Properties

This is a para in the topic template.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title[0..159]`	The title assigned to this object. You can update this title before storing an object and then later retrieve it by specifying just the title.
`INT4 size`	The current size, in bytes, of the data stored in this object.
`INT4 ID`	The internal ID of the Data object. This ID is assigned when you Put and Get the object.
`INT4 setID`	Another name for ID.

UserData Object Methods

Get

`Get` **`( entID )`**	
Description:	
Loads a data object that was previously stored with this `entID`.	
Input:	
`INT4 entID`	The entity ID of the data object that will be loaded.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	Unable to load the data object. Either it does not exist, or there is no data in it.
`FE_NO_MEMORY`	Unable to allocate memory for data buffering.
Remarks/Usage:	
You must call this function, or `GetTitle( )`, to reload objects that have been stored in the model.	

Unpublished work. © 2025 Siemens

Example:
None

GetTitle

GetTitle **(title)**	
Description:	
Loads a data object that was previously stored with this title.	
Input:	
`STRING title[0..159]`	The title of the object to load.
Output:	
None	
Return Code:	
`FE_NOT_EXIST`	Unable to load the data object. Either it does not exist, or there is no data in it.
`FE_NO_MEMORY`	Unable to allocate memory for data buffering.
Remarks/Usage:	
To use this method to retrieve an object, it must have had its title property defined prior to being `Put` to the database. The title specified here must be an exact match, including case, to the title specified when the object was stored. Using titles to store and retrieve data objects can eliminate conflicts between data stored from various programs.	
Example:	
None	

Put

Unpublished work. © 2025 Siemens

Put	
(entID)	
Description:	
Stores data from the current data object.	
Input:	
`INT4 entID`	The ID of the object to store.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to store the data object.
Remarks/Usage:	
You must set the title property prior to using this method if you want to use `GetTitle` to retrieve the object. You do not have to set the title if you plan on using `Get` to retrieve the object. Using this method will always overwrite any data that was previously stored with the same `entID` - even if it was not from your program - for that reason we recommend using `PutTitle` - with unique titles - whenever possible. Data must be `Put` to the model database before it will be kept with the model, and before it will be supported by Undo/Redo.	
Example:	
None	

PutTitle

PutTitle	
(title)	
Description:	
Stores data from the current data object.	
Input:	

Unpublished work. © 2025 Siemens

`STRING title[0..159]`	The title of the object to store.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to store the data object.
Remarks/Usage:	
There is no need to set the title property prior to using this method - that is done automatically. This method will automatically find and overwrite any previous data with the same title - you should always specify a unique title to avoid this possibility - or use `GetTitle` to test if there is other data using the same title. If no data object exists in the database with a matching title, this method simply finds an unused ID and stores the object. Data must be `Put` to the model database before it will be kept with the model, and before it will be supported by Undo/Redo.	
Example:	
None	

Clear

`Clear` **`( void )`**	
Description:	
This function simply removes all data from the current `UserData` object.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Reset

`Reset` **`( void )`**
Description:
This method resets the `Next` pointer so that the next call to `Next` will return the first (lowest ID) `UserData`.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
After calling `Reset`, `Next` will return the first entity and `Prev` will return the last (highest ID) `UserData`.
Example:
None

Next

Unpublished work. © 2025 Siemens

Next	
(void)	
Description:	
This method retrieves the next available `UserData` with a larger ID.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No additional entities were found.
Remarks/Usage:	
Calling this method simply finds the next available `UserData` with an ID that is greater than the current ID, and calls the `Get` method with that ID.	
Example:	
None	

Prev

Prev	
(void)	
Description:	
This method retrieves the next available `UserData` with a smaller ID.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No additional entities were found.
Remarks/Usage:	
Calling this method simply finds the next available `UserData` with an ID that is smaller than the current ID, and calls the `Get` method with that ID.	
Example:	
None	

First

`First` **`( void )`**	
Description:	
This method loads the first (with the lowest ID) available `UserData`.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No entities exist, or unable to load.
Remarks/Usage:	
Calling this method is like calling `Reset`, followed by `Next`.	
Example:	
None	

Last

Unpublished work. © 2025 Siemens

Last	
(void)	
Description:	
This method loads the last (with the highest ID) available `UserData`.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No entities exist, or unable to load.
Remarks/Usage:	
Calling this method is like calling `Reset`, followed by `Prev`.	
Example:	
None	

NextSet

NextSet	
(void)	
Description:	
This method updates the `setID` to be the next existing `UserData` object.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more sets are found, `setID` remains unchanged. This method finds the next set with a higher ID.	
Example:	
None	

PrevSet

`PrevSet` **`( void )`**	
Description:	
This method updates the `setID` to be the previous existing `UserData` object.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more sets are found, `setID` remains unchanged. This method finds the next `UserData` with a lower ID.	
Example:	
None	

NextEmptySet

Unpublished work. © 2025 Siemens

NextEmptySet	
(void)	
Description:	
This method updates the `setID` to be the next empty `UserData` object.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more empty sets are found, `setID` remains unchanged. This method finds the next `setID` that does not contain any `UserData`.	
Example:	
None	

PrevEmptySet

PrevEmptySet	
(void)	
Description:	
This method updates the `setID` to be the previous empty `UserData` object.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more empty sets are found, `setID` remains unchanged. This method finds the previous `setID` that does not contain any `UserData`.	
Example:	
None	

Exist

`Exist` **`( id )`**	
Description:	
This method checks to see if a specific `UserData` exists.	
Input:	
`INT4 id`	The ID of the `UserData` to check.
Output:	
None	
Return Code:	
`FE_FAIL`	The selected entity does not exist.
Remarks/Usage:	
This method does not load any data. It simply returns `FE_OK` if the `UserData` exists, and `FE_FAIL` if it does not.	
Example:	
None	

Deletable

Unpublished work. © 2025 Siemens

Deletable	
(id)	
Description:	
This method checks to see if a specific `UserData` is deletable.	
Input:	
`INT4 id`	The ID of the `UserData` to check.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
This method is currently not used. It will always return `FE_OK`.	
Example:	
None	

Delete

Delete	
(entID)	
Description:	
Deletes the data object from the database with the specified ID.	
Input:	
`INT4 entID`	The ID of the object to delete.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to delete the data object.
Remarks/Usage:	
Deleting a data object is permanent. The data is removed from the database; however, the data can return if you undo past the point of the deletion.	
Example:	
None	

DeleteTitle

`DeleteTitle` **`( title )`**	
Description:	
Deletes the data object from the database with the specified title.	
Input:	
`STRING title[0..159]`	The title of the object to delete.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to delete the data object.
Remarks/Usage:	
Deleting a data object is permanent. The data is removed from the database; however, the data can return if you undo past the point of the deletion.	
Example:	
None	

Position

Unpublished work. © 2025 Siemens

Position	
(void)	
Description:	
Returns the current location in the data object where you are reading or writing.	
Input:	
None	
Output:	
None	
Return Code:	
`INT4 position`	The current data object location.
Remarks/Usage:	
This function can be used with `Jump` to reposition to a specific object location for *rereading* or *rewriting*.	
Example:	
None	

Jump

Jump	
(jumpto)	
Description:	
Repositions the location in the data object where the next read or write will occur.	
Input:	
`INT4 jumpto`	The position to move to. Specify `jumpto=-1` to move to the end of the object, or `jumpto=0` to move to the beginning of the object.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to move to the specified location. Position must be less than the total size of data in the object.
Remarks/Usage:	
You should only specify `jumpto` as a value that you previously retrieved from a call to `Position`. Computing numbers yourself is prone to error because of extra data need to store arrays and certain data types.	
Example:	
None	

AtEOF

`AtEOF`	
`( void )`	
Description:	
Indicates whether the current position for reading/writing is at the end of the data in the object.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	At the end of the object. If you are reading from the object, you should stop at this point. Writes will append to the object.
`FE_FAIL`	Not at the end of the object. You can continue reading. Writes will overwrite previous data.
Remarks/Usage:	
You can use this check to determine when you reach the end of the data in an object.	
Example:	
None	

Unpublished work. © 2025 Siemens

Truncate

Truncate	
(position)	
Description:	
Deletes all data in an object after the specified position.	
Input:	
`INT4 position`	The position to truncate the data in the object. Essentially the size of the data in the remaining object.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to truncate, `Position` probably not between 0 and the size of data in the object.
Remarks/Usage:	
Truncating data in an object only changes the current object. Changes must be `Put` to the database to be made permanent. The current reading/writing position is moved to the end of the object (position) following this method.	
Example:	
None	

WriteDouble

WriteDouble
(val)
Description:
Writes a single *double* value to the current object.

Unpublished work. © 2025 Siemens

Input:	
`REAL8 val`	The value to add to the current object.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

WriteFloat

`WriteFloat`
`( val )`
Description:
Writes a single *float* value to the current object.
Input:

`REAL4 val`	The value to add to the current object.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

WriteLong

`WriteLong`	
`( val )`	
Description:	
Writes a single *long* value to the current object.	
Input:	
`INT4 val`	The value to add to the current object.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

Unpublished work. © 2025 Siemens

WriteShort

`WriteShort`

`( val )`

Description:

Writes a single *short* value to the current object.

Input:

`INT2 val`	The value to add to the current object.

Output:

None	

Return Code:

`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.

Remarks/Usage:

The specified value will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.

Example:

None

WriteBool

`WriteBool`

`( val )`

Description:

Writes a single *bool* value to the current object.

Unpublished work. © 2025 Siemens

Input:	
`BOOL val`	The value to add to the current object.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

WriteDoubleArray

`WriteDoubleArray`
`( num, val )`
Description:
Writes an array of *double* values to the current object.
Input:

`INT4 num`	The number of entries in the *val* array that you want to write.
`REAL8 val[0..num-1]`	The values to add to the current object.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_TOO_SMALL`	The array that you specified is not large enough for the number of entries that you specified.
`FE_BAD_TYPE`	The type of data in the array that you passed is not of the type required for this method.
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified values will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

WriteLongArray

`WriteLongArray`	
`( num, val )`	
Description:	
Writes an array of *long* values to the current object.	
Input:	
`INT4 num`	The number of entries in the *val* array that you want to write.
`INT4 val[0..num-1]`	The values to add to the current object.
Output:	
None	
Return Code:	
`FE_TOO_SMALL`	The array that you specified is not large enough for the number of entries that you specified.

Unpublished work. © 2025 Siemens

`FE_BAD_TYPE`	The type of data in the array that you passed is not of the type required for this method.
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified values will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

WriteBoolArray

`WriteBoolArray` **`( num, val )`**	
Description:	
Writes an array of *bool* values to the current object.	
Input:	
`INT4 num`	The number of entries in the *val* array that you want to write.
`INT4 val[0..num-1]`	The values to add to the current object.
Output:	
None	
Return Code:	
`FE_TOO_SMALL`	The array that you specified is not large enough for the number of entries that you specified.
`FE_BAD_TYPE`	The type of data in the array that you passed is not of the type required for this method.

Unpublished work. © 2025 Siemens

`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified values will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	
Example:	
None	

WriteString

`WriteString` **`( val )`**	
Description:	
Writes a character string to the current object.	
Input:	
`STRING val[0..length]`	The string to add to the current object.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Unable to allocate enough memory to add the specified value.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified values will be added to the object at the current position. You can change this position with the `Position` method prior to calling this method. Data that is added to the object, with any of the write methods, is not be saved to the model database until you call `Put` or `PutTitle`.	

Unpublished work. © 2025 Siemens

Example:
None

ReadDouble

ReadDouble **(val)**	
Description:	
Reads a single *double* value from the current object.	
Input:	
None	
Output:	
`REAL8 val`	The value to read from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

ReadFloat

Unpublished work. © 2025 Siemens

ReadFloat	
(val)	
Description:	
Reads a single *float* value from the current object.	
Input:	
None	
Output:	
`REAL4 val`	The value to read from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

ReadLong

ReadLong
(val)
Description:
Reads a single *long* value from the current object.
Input:

Unpublished work. © 2025 Siemens

None	
Output:	
`INT4 val`	The value to read from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

ReadShort

`ReadShort`
`( val )`
Description:
Reads a single *short* value from the current object.
Input:

None	
Output:	
`INT2 val`	The value to read from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

ReadBool

`ReadBool`	
`( val )`	
Description:	
Reads a single *bool* value from the current object.	
Input:	
None	
Output:	
`BOOL val`	The value to read from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	

Unpublished work. © 2025 Siemens

Example:
None

ReadDoubleArray

`ReadDoubleArray` **`( num, val )`**	
Description:	
Reads an array of *double* values from the current object.	
Input:	
None	
Output:	
`INT4 num`	The number of items in the array that is read.
`REAL8 val[0..num-1]`	The values from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_NO_MEMORY`	Unable to allocate enough memory to read the specified values.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

Unpublished work. © 2025 Siemens

ReadLongArray

ReadLongArray **(num, val)**	
Description:	
Reads an array of *long* values from the current object.	
Input:	
None	
Output:	
`INT4 num`	The number of items in the array that is read.
`REAL8 val[0..num-1]`	The values from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_NO_MEMORY`	Unable to allocate enough memory to read the specified values.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

ReadBoolArray

Unpublished work. © 2025 Siemens

ReadBoolArray	
(num, val)	
Description:	
Reads an array of *bool* values from the current object.	
Input:	
None	
Output:	
`INT4 num`	The number of items in the array that is read.
`BOOL val[0..num-1]`	The values from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_NO_MEMORY`	Unable to allocate enough memory to read the specified values.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

ReadString

Unpublished work. © 2025 Siemens

ReadString **(val)**	
Description:	
Reads a text string from the current object.	
Input:	
None	
Output:	
`STRING val[0..length]`	The string to read from the current object.
Return Code:	
`FE_NOT_AVAILABLE`	There is not enough data left in the object to read, from the current position.
`FE_NO_MEMORY`	Unable to allocate enough memory to read the specified values.
`FE_INVALID`	Invalid size for the data specified.
Remarks/Usage:	
The specified value will be read from the object at the current position. You can change this position with the `Position` method prior to calling this method. Data can be read from an object after a call to `Get`, or after data has been added to the object with one or more `Write` calls (and the position has been relocated before the data).	
Example:	
None	

Results Browsing Objects

The **Results Browsing** object provides a fast, common interface to review analysis results whether they are stored in the Femap database or in an attached external file. Using this object, results can be reviewed and locally modified in the object. Results can be updated and saved back to the database, but not to attached external results files. Results can also be saved to new “user-defined” vector IDs.

This object provides an alternative to using the `Output` and `OutputSet` objects. The `Output` object provides access to a singe output vector, or a single vector and its components. This object provides two different groups of methods. The first simply allows you to directly find information about the Output Sets, Vectors and individual Entity Results (although this last method will be very slow and should only be

Unpublished work. © 2025 Siemens

used in rare cases). The second group of methods allows much faster access to the actual analysis results. You should think of this capability just like a spreadsheet or table - possibly a very large spreadsheet- that you will fill with analysis results. In the *spreadsheet*, the columns represent the output vectors, with one column per vector. The rows of the *spreadsheet* represent the data for an individual entity - either a Node or an Element. The object can only contain columns of one data type - that is, either Nodal Results or Elemental Results. You can not add columns of Nodal Results (for example, Nodal Translations) and columns of Elemental Results (for example, Plate Von Mises Stress) to the same object at the same time.

Results Browsing Object Properties

Most of the functionality of this object is available through the methods which are described below. This property provides additional functionality to support a number of those methods.

Property	Description
`REAL8 ValueForNonExisting`	This property allows you to control the value returned if you use any of the `Get` functions and no results exist for that Row/ID and Column. By default this is 0.0. You should always check the return code of the `Get` methods, but this property provides an additional way to flag individual *cells* that do not exist - especially for methods that return multiple columns, or entire rows in one call.
`BOOL AutoPopulate`	This property allows you to control the whether or not the table is populated automatically after using calls like GetColumnV2 and SetColumnV2.
`INT4 NextSetStartID`	This property sets the pointer to the current Set ID.
`INT4 NextVectorStartID`	This property sets the pointer to the current Vector ID.
`INT4 NextStudyStartID`	This property sets the pointer to the current analysis Study ID.

Limitations and Usage of Methods

While the commands that allow you to review analysis results can be thought of as a rather large spreadsheet, it is still best to not go too far in loading results. Analysis results for large models, or for large numbers of cases or time steps can generate files that are many gigabytes. Attempting to load all of this data at once is not practical. The amount of memory required would be huge. This object will swap its contents to disk allowing very large amounts of data to be loaded, however this will result in significantly degraded performance. In cases where your programming approach would require huge numbers of columns with huge numbers of rows, it may be more efficient to think about a different, more

Unpublished work. © 2025 Siemens

incremental algorithm. For example, if you have a 1 million element model with 1000 time steps of output where you wanted to find the max/min values just for one output vector across all the time steps, loading all the data at once would require 1000 columns (actually 1001, with the 1 extra being to keep the max or min value), and 1 million rows. In spreadsheet terminology, this would require over 1 billion cells. In this case, it might be more efficient to use two results objects. One object would contain the accumulated max/min data, and the other could simply load one (or a few) time steps at a time, and then add logic to incrementally find the max/min. Again, this is not *required*, but it may be more efficient. There are no explicit limits on the numbers of rows or columns. The practical limits will be determined by the amount of memory that you have available.

When programming this object the methods that review the available Output Sets and Output Vectors can be used just like the methods on normal *Entity-based* objects. For example, you can call `NextSetReset()`, followed by repetitive calls to `NextSet(...)` to step thru all the available sets. `NextVectorReset()` and `NextVector(...)` are similar with Output Vectors. With the *spreadsheet* commands, the required approach is to either start with an empty object, or `Clear()` an already used object, then add the desired columns. Adding columns does not load any entity results into the object, it simply defines the structure of the data to be loaded. At this time you can also define any necessary transformations by setting the output orientations and the desired type of transform. After you have defined ALL of the columns and transforms, a single call to `Populate()` must be made. This creates the rows and fills data into *cells* in each defined column. After populating the table with data, you can use the other methods to retrieve values from the table.

Output Vectors

Output Components

The components are 20 vector IDs of other vectors that are *components* of the current vector. There are several possible ways these components are used:

1. The current vector contains the vector magnitude of 3 other vectors that are in the global X, Y, and Z components of similar data:

   Note

   In this case, Components 0, 1, and 2 are the vector IDs of the X, Y, and Z component vectors, respectively. An example of this type of usage is `Vector 1 (TOTAL TRANSLATION)`, which has `Component 0=2(X TRANSLATION)`, `Component 1=3(Y TRANSLATION)` and `Component 2=4(Z TRANSLATION)`. These components MUST be defined for deformed/animated plotting to work correctly. Components 3 through 19 must be 0 for this case.

2. The current vector contains data which should be plotted in a specific global direction:

Unpublished work. © 2025 Siemens

> **Note**
>
> In this case, if plotting should occur in Global X, then `Component 0` must equal the current Vector ID. If Global Y, then `Component 1` = current Vector ID, and if Global Z then `Component 2` = current Vector ID. All other components must equal 0 (i.e. only 1 nonzero component and it is equal to the current Vector ID).

3. The current vector contains elemental centroidal data for elements which also have related data at the element corners:

> **Note**
>
> In this case, the number of nonzero components depends upon the number of nodes on this element type (for example, output for linear plates has 4 components-4 nodes/corners). The nonzero components must equal the Vector IDs of the same type of data at the corners of the elements. For example, if the current vector is `7033 (TOP CENTROIDAL VON MISES STRESS)` then `Component 0` may be `11033 (CORNER1 TOP VON MISES STRESS)` and `Component 1` may be `15033 (CORNER2 TOP VON MISES STRESS)` etc. If you do not have corner data, all components must be 0. If the components are defined, Femap will use the corner data (not the data in the current vector) when calculating data at nodes (like for contouring). If no components are defined, Femap will assign the centroidal value to each node when doing the calculation.

4. The current vector contains line element data at one end of the element:

> **Note**
>
> This is just like the previous case. The first component is the vector ID of the End A data, the second component is the vector ID of the End B data. This type of output also requires special settings in the `comp_dir` variable. Set `comp_dir` to 2 if you want Femap to use the data as it is specified. Set `comp_dir` to 3 if you want Femap to reverse the sign of all End B data when doing a beam diagram. This adjusts for differences in sign conventions between the various analysis programs.

5. All other cases:

> **Note**
>
> All components should be 0. This will force the data in the current vector to be used for every case. The current setting of the View Option Deformed Style - Deformed Direction option will control the deformed plotting direction.

Output Vector IDs

If you are writing your own Neutral files, you can choose any vector IDs that you want, but the normal conventions used by Femap are as follows:

Standard Output Vectors version 2020.1 and above		
Category	**Vector Range**	**Type**

Unpublished work. © 2025 Siemens

1 to 24,999,999 (magnitude, or non-complex)	1 to 2,999	Nodal output
	3,000 to 5,999	Line element output
	6,000 to 9,999	Plate element output
	10,000 to 25,999	Plate corner output
	26,000 to 59,999	Not currently used
	60,000 to 79,999	Solid element output
	80,000 to 89,999	Output on any element type
	90,000 to 99,999	PATRAN elemental output
	100,000 to 999,999	Not currently used
	1,000,000 to 3,999,999	Plate Laminate Ply output
	4,000,000 to 15,999,999	Plate Laminate Ply corner
	16,000,000 to 17,499,999	Solid Laminate Ply output
	17,500,000 to 23,499,999	Solid Laminate Ply corner
	24,000,000 to 24,999,999	User Defined output
25,000,00 to 49,999,999 (complex, phase)	Same definition as magnitude data, just add 25,000,000 to IDs.	
50,000,000 to 74,999,999 (complex, real)	Same definition as magnitude data, just add 50,000,000 to IDs.	
75,000,000 to 99,999,999 (complex, imaginary)	Same definition as magnitude data, just add 75,000,000 to IDs.	

Standard Output Vectors version 2019.1 and prior		
Category	**Vector Range**	**Type**
1 to 9,999,999 (magnitude, or non-complex)	1 to 2,999	Nodal output
	3,000 to 5,999	Line element output
	6,000 to 59,999	Plate element output
	60,000 to 79,999	Solid element output
	80,000 to 89,999	Output on any element type
	90,000 to 99,999	PATRAN elemental output
	100,000 to 299,999	Plate corner output

Unpublished work. © 2025 Siemens

	1,000,000 to 6,000,000	Laminate Ply output
	9,000,000 to 9,999,999	User Defined output
10,000,001 to 20,000,000 (complex, phase)	Same definition as magnitude data, just add 10,000,000 to IDs.	
20,000,001 to 30,000,000 (complex, real)	Same definition as magnitude data, just add 20,000,000 to IDs.	
30,000,001 to 40,000,000 (complex, imaginary)	Same definition as magnitude data, just add 30,000,000 to IDs.	

Results Browsing General Object Methods

The following methods are for general use with the object.

Clear

`Clear`	
`( )`	
Description:	
This method clears all data from the object and reinitializes it to a default state.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetModel

GetModel **(nModelID)**	
Description:	
This method returns the identifier of the model being used by the object.	
Input:	
None	
Output:	
`INT8 nModelID`	Model identifier.
Return Code:	
None	
Remarks/Usage:	
When the object is created, `nModelID` is set to the current/active model.	
Example:	
None	

SetModel

SetModel **(nModelID)**	
Description:	
This method chooses the model to be used for all further operations.	
Input:	
`INT8 nModelID`	Model identifier.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
None	
Remarks/Usage:	
When the object is created, `nModelID` is set to the current/active model.	
Example:	
None	

SendToDataTable

`SendToDataTable` **`( void )`**	
Description:	
This method sends the contents of a populated Results Browsing Object to the Data Table. The Data Table must visible and be unlocked.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_FAIL`	Results Browsing Object has not been populated or Data Table is not visible.
`FE_SECURITY`	Data Table is visible, but locked.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Results Browsing Study Methods

The following methods allow you to create, edit and review the Output Studies that are available in the model.

StudyCreate

StudyCreate **(nStudyID, sTitle, nAnalysisProgram, nAnalysisType)**	
Description:	
This method creates and Analysis Study.	
Input:	
`INT4 nStudyID`	The ID of the Analysis Study.
`STRING sTitle`	The title of the Analysis Study.
`INT4 nAnalysisProgram`	Analysis program where output came from: 0=Unknown, 4=MSC/NASTRAN, 5=ANSYS, 16=ABAQUS, 28=LS-Dyna, 29=MSC.Marc, 31=NE/Nastran, 34=FEMAP Structural. 36=NX Nastran. Use enum `zAnalysisProgram`.
`INT4 nAnalysisType`	Type of analysis the results came from: 0=None, 1=Static, 2=Normal Modes/Eigenvalue, 3=Transient Dynamic/Time History, 4=Frequency/Harmonic Response, 5=Response Spec-trum, 6=Random Response, 7=Buckling, 8=Design Opt, 9=Explicit Transient Dynamics, 10=Nonlinear Static, 11=Nonlinear Buckling, 12=Nonlinear Transient, 13=Implicit Transient Dynamics, 17=Fatigue, 19=Computational Fluid Dynamics, 20=Steady-State Heat Transfer, 21=Transient Heat Transfer, 22=Advanced Nonlinear Static, 23=Advanced Non-linear Transient, 24=Advanced Nonlinear Explicit, 25=Static Aeroelasticity, 26=Aerodynamic Flutter. Use enum `zAnalysisType`.
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to create Analysis Study
Remarks/Usage:	
None	
Example:	
None	

NumberOfStudies

`NumberOfStudies`	
`( )`	
Description:	
This method returns the total number of Analysis Studies in the model.	
Input:	
None	
Output:	
None	
Return Code:	
`Number of Studies`	The number of Studies.
Remarks/Usage:	
None	
Example:	
None	

NextStudyReset

Unpublished work. © 2025 Siemens

NextStudyReset	
( )	
Description:	
This method resets the next pointer so that the next call to `NextStudy` will return the first available Analysis Study.	
Input:	
None	
Output:	
None	
Return Code:	
`Number of Studies`	The number of Studies
Remarks/Usage:	
None	
Example:	
None	

NextStudy

NextStudy	
(nExistState, nLocateOption)	
Description:	
This method returns the next Analysis Study ID.	
Input:	
`INT4 nExistState`	0=ID of Next Existing Analysis Study, 1=ID of next NonExisting Analysis Study.
`INT4 nLocateOption`	1=After, 2=Before, 3=After or Equal, 4=Before or Equal. To increment from smallest to largest ID use 1. To go from largest to smallest use 2. Options 3

Unpublished work. © 2025 Siemens

	and 4 are normally only used for finding the next NonExisting Analysis Study - care must be taken with these options to avoid infinite loops.
Output:	
None	
Return Code:	
`FE_FAIL`	No more Analysis Study IDs are found given the specified options.
`Any other number`	The ID of the Existing/NonExisting Analysis Study found.
Remarks/Usage:	
None	
Example:	
None	

StudyExist

`StudyExist`
`( nStudyID )`
Description:
This method determines if a selected Analysis Study exists.
Input:

`INT4 nStudyID`	The ID of the Analysis Study to check.
Output:	
None	
Return Code:	
`FE_FAIL`	The Analysis Study does not exist.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

StudyInfo

StudyInfo **(nStudyID, pnAnalysisProgram, pnAnalysisType)**	
Description:	
This method retrieves the information of a selected Analysis Study.	
Input:	
`INT4 nStudyID`	The ID of the Analysis Study to check.
Output:	
`INT4 nAnalysisProgram`	Analysis program where output came from: 0=Unknown, 4=MSC/NASTRAN, 5=ANSYS, 16=ABAQUS, 28=LS-Dyna, 29=MSC.Marc, 31=NE/Nastran, 34=Femap Structural. 36=NX Nastran. Use enum `zAnalysisProgram`.
`INT4 nAnalysisType`	Type of analysis the results came from: 0=None, 1=Static, 2=Normal Modes/Eigenvalue, 3=Transient Dynamic/Time History, 4=Frequency/Harmonic Response, 5=Response Spectrum, 6=Random Response, 7=Buckling, 8=Design Opt, 9=Explicit Transient Dynamics, 10=Nonlinear Static, 11=Nonlinear Buckling, 12=Nonlinear Transient, 13=Implicit Transient Dynamics, 17=Fatigue, 19=Computational Fluid Dynamics, 20=Steady-State Heat Transfer, 21=Transient Heat Transfer, 22=Advanced Nonlinear Static, 23=Advanced Nonlinear Transient, 24=Advanced Nonlinear Explicit, 25=Static Aeroelasticity, 26=Aerodynamic Flutter. Use enum `zAnalysisType`.
Return Code:	
`FE_FAIL`	Unable to retrieve information about selected Analysis Study - most likely the study does not exist.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

StudyTitle

StudyTitle **(nStudyID, sTitle)**	
Description:	
This method retrieves the title of a selected Analysis Study.	
Input:	
`INT4 nStudyID`	The ID of the Analysis Study to check.
Output:	
`STRING sTitle`	The title of the Analysis Study.
Return Code:	
`FE_FAIL`	Unable to retrieve title of selected Analysis Study.
Remarks/Usage:	
None	
Example:	
None	

StudyNotes

Unpublished work. © 2025 Siemens

StudyNotes	
(nStudyID, sNotes)	
Description:	
This method retrieves the notes of a selected Analysis Study.	
Input:	
`INT4 nStudyID`	The ID of the Analysis Study to check.
Output:	
`STRING sNotes`	The notes of the Analysis Study.
Return Code:	
`FE_FAIL`	Unable to retrieve notes of selected Analysis Study.
Remarks/Usage:	
None	
Example:	
None	

Results Browsing Set Methods

The following methods allow you to review the Output Sets that are available in the model.

NumberOfSets

NumberOfSets
( )
Description:
This method returns the number of output sets that are currently available.
Input:

Unpublished work. © 2025 Siemens

None	
Output:	
None	
Return Code:	
`Number of Sets`	The number of available Sets.
Remarks/Usage:	
None	
Example:	
None	

NextSetReset

`NextSetReset`
`( )`
Description:
This method resets the next pointer so that the next call to `NextSet` will return the first available Output Set.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

NextSet

NextSet **(nExistState, nLocateOption)**	
Description:	
This method returns the ID of the next available Output Set.	
Input:	
`INT4 nExistState`	0=ID of Next Existing Output Set, 1=ID of next NonExisting Output Set.
`INT4 nLocateOption`	1=After, 2=Before, 3=After or Equal, 4=Before or Equal. To increment from smallest to largest ID use 1. To go from largest to smallest use 2. Options 3 and 4 are normally only used for finding the next NonExisting Output Set - care must be taken with these options to avoid infinite loops.
Output:	
None	
Return Code:	
`FE_FAIL`	No more Output Set IDs are found given the specified options.
`Any other number`	The ID of the Existing/NonExisting Output Set found.
Remarks/Usage:	
None	
Example:	
None	

SetExists

Unpublished work. © 2025 Siemens

SetExists **(nSetID)**	
Description:	
This method determines if a selected Output Set exists.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
Output:	
None	
Return Code:	
`FE_FAIL`	The Output Set does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetInfo

SetInfo **(nSetID, nAnalysisProgram, nAnalysisType, dSetValue)**	
Description:	
This method returns information about a selected Output Set.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
Output:	
`INT4 nAnalysisProgram`	Refer to the `Program` property of the Output Set object.

Unpublished work. © 2025 Siemens

`INT4 nAnalysisType`	Refer to the `Analysis` property of the Output Set object.
`REAL8 dSetValue`	Refer to the `Value` property of the Output Set object.
Return Code:	
`FE_FAIL`	Unable to retrieve information about selected set - most likely the set does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetTitle

`SetTitle`
`( nSetID, sTItle )`
Description:
This method returns the title of a selected Output Set.
Input:

`INT4 nSetID`	The ID of the Output Set to query.
Output:	
`STRING sTitle`	The title of the selected Output Set.
Return Code:	
`FE_FAIL`	Unable to retrieve information about selected set - most likely the set does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetSubcase

SetSubcase	
(nSetID)	
Description:	
This method returns the Nastran subcase ID (Case ID) of the selected Output Set.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
Output:	
None	
Return Code:	
`Value`	The ID of the Nastran subcase associated with the Output Set. If 0, no subcase is associated with the output set.
Remarks/Usage:	
These are automatically created for Nastran Output Sets created from subcases when the **Use Static Subcase IDs** option is on in the **Results** tab of the **File** → **Preference** command.	
Example:	
None	

SetRevision

SetRevision
(nSetID)
Description:
This method returns the revision number of a selected Output Set.
Input:

Unpublished work. © 2025 Siemens

`INT4 nSetID`	The ID of the Output Set to query.
Output:	
None	
Return Code:	
`Value`	The ID of the revision number associated with the Nastran Output Set. If 0, no revision is associated with the output set.
Remarks/Usage:	
These revision numbers are automatically created for Nastran Output Sets when the **Track Revision** option is on in the **Results** tab of the **File** → **Preference** command.	
Example:	
None	

SetNotes

`SetNotes` **`( nSetID, sNotes )`**
Description:
This method returns the notes of a selected Output Set.
Input:

`INT4 nSetID`	The ID of the Output Set to query.
Output:	
`STRING sNotes`	The notes of the selected Output Set.
Return Code:	
`FE_FAIL`	Unable to retrieve information about selected set - most likely the set does not exist.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetLocation

SetLocation **(nSetID, nSetLocation)**	
Description:	
This method identifies whether data for the Output Set is in the database or in an attached external file.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nSetLocation`	1=In Femap Database (FRL_DB) 2=In Nastran OP2 File (FRL_OP2) 3=In FNO File (FRL_FNO) 4=Not Used 5=In Nastran XDB File (FRL_XDB) 6=In Comma-Separated Value (CSV) File (FRL_CSV) 7=In Virtual Output Set (FRL_VIRTUAL) 8=In ABAQUS ODB File (FRL_ODB)
None	
Return Code:	
`FE_FAIL`	Unable to retrieve information about selected set - most likely the set does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetHasVectorsInDB

Unpublished work. © 2025 Siemens

SetHasVectorsInDB

(nSetID)

Description:

This method indicates whether an Output Set contains Output Vectors that are in the Femap Database.

Input:

`INT4 nSetID`	The ID of the Output Set to query.

Output:

None	

Return Code:

`FE_FAIL`	There are no Output Vectors in the Femap Database for this Set.

Remarks/Usage:

If this method returns `FE_OK`, then it contains Output Vectors located in the Femap Database. Even Output Sets that are located in an attached external file can contain certain Output Vectors in the Femap Database, if one of the Femap commands (like **Model** → **Output** → **Process**) was used to create additional Output Vectors in that Output Set.

Example:

None

Sets

Sets

(nSetSetID, bClear)

Description:

This method fills a Set object with the IDs of all of the Output Sets that are available.

Input:

`INT4 nSetSetID`	The ID of the Set Object to fill with the available Output Set IDs.

Unpublished work. © 2025 Siemens

`BOOL bClear`	If True, the Set object is cleared prior to being filled with the available Output Set IDs.
Output:	
None	
Return Code:	
`FE_FAIL`	No Output Sets were found.
Remarks/Usage:	
None	
Example:	
None	

SetCreate

`SetCreate`
`( nSetID, sTitle, nStudyID, nAnalysisProgram, nAnalysisType, dSetValue )`
Description:
This method creates a new Output Set.
Input:

`INT4 nSetID`	The ID of the Output Set to create.
`STRING sTitle`	The title of the Output Set.
`INT4 nStudyID`	The ID of the Study the Output Set is associated with.
`INT4 nAnalysisProgram`	The analysis program of the Output Set. 1=Femap Generated, 2=PAL, 3=PAL 2, 4=MSC/NASTRAN, 5=ANSYS, 6=STARDYNE, 7=COSMOS, 8=PATRAN, 9=Femap Neutral, 10=ALGOR, 11=SSS/NASTRAN, 12=Comma Separated, 13=UAI/NASTRAN, 14=Cosmic NASTRAN, 15=STAAD, 16=ABAQUS, 17=WECAN, 18=MTAB/SAP, 19=CDA/Sprint, 20=CAEFEM, 21=I-DEAS, 22=ME/NASTRAN,

Unpublished work. © 2025 Siemens

	26=CSA/NASTRAN, 28=CFDesign, 31=LS-DYNA, 32=MARC, 33=SINDA, 38=NX Nastran.
`INT4 nAnalysisType`	The analysis type of the Output Set. 0=Unknown, 1=Static, 2=Normal Modes/ Eigenvalue, 3=Transient Dynamic/Time History, 4=Frequency/ Harmonic Response, 5=Response Spectrum, 6=Random Response, 7=Buckling, 8=Design Opt, 9=Explicit Transient Dynamics, 10=Nonlinear Static, 11=Nonlinear Buckling, 12=Nonlinear Transient, 13=Implicit Transient Dynamics, 17=Fatigue, 19=Computational Fluid Dynamics, 20=Steady-State Heat Transfer, 21=Transient Heat Transfer, 22=Advanced Nonlinear Static, 23=Advanced Nonlinear Transient, 24=Advanced Nonlinear Explicit, 25=Static Aero-elasticity, 26=Aerodynamic Flutter.
`REAL8 dSetValue`	The output set value. For modal analyses, the modal frequency. For any transient or nonlinear analysis, the time value at this step. For Buckling, the eigenvalue. For static analysis = 0.0
Output:	
None	
Return Code:	
`FE_FAIL`	Output Set was not created.
Remarks/Usage:	
None	
Example:	
None	

Results Browsing Vector Methods

The following methods allow you to review the Output Vectors that are available in a selected Output Set.

NextVectorResetV2

Unpublished work. © 2025 Siemens

NextVectorResetV2	
(nSetID)	
Description:	
This method resets the next pointer so that the next call to `NextVector` will return the first available Output Vector in the Output Set. This method replaces `NextVectorReset`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

NextVectorV2

NextVectorV2	
(nSetID, nExistState, nLocateOption)	
Description:	
This method returns the ID of the next available Output Vector in a selected Output Set. This method replaces `NextVector`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query for Output Vectors.
`INT4 nExistState`	0=ID of Next Existing Output Set, 1=ID of next Non-Existing Output Set.

Unpublished work. © 2025 Siemens

`INT4 nLocateOption`	1=After, 2=Before, 3=After or Equal, 4=Before or Equal. To increment from smallest to largest ID use 1. To go from largest to smallest use 2. Options 3 and 4 are normally only used for finding the next Non-Existing Output Set - care must be taken with these options to avoid infinite loops.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

VectorExistsV2

`VectorExistsV2`
`( nSetID, nVectorID )`
Description:
This method indicates whether a selected Output Vector exists in a selected Output Set. This method replaces `VectorExist`. For more information see, **Working with Output in Femap v2020.1 and above**.
Input:

`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	The Output Vector does not exist in the selected Output Set.
Remarks/Usage:	
None	
Example:	
None	

VectorInfoV2

`VectorInfoV2` **`( nSetID, nVectorID, nDataType, nOutputType, bCentroidTotal, pbNonlinear, nCompDir )`**	
Description:	
This method returns selected information about a selected Output Vector. This method replaces `VectorInfo`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
Output:	
`INT4 nDataType`	Refer to the *location* property of the Output object.
`INT4 nOutputType`	Refer to the *category* property of the Output object.
`BOOL bCentroidTotal`	Refer to the *centroidtotal* property of the Output object.
`BOOL pbNonlinear`	Refer to the *nonlinear* property of the Output object.
`INT4 nCompDir`	Refer to the *hascomponent* property of the Output object.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.
Remarks/Usage:	
None	
Example:	
None	

VectorTitleV2

`VectorTitleV2` **`( nSetID, nVectorID, sTitle )`**	
Description:	
This method returns the title of the selected Output Vector. This method replaces `VectorTitle`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
Output:	
`STRING sTitle`	The Output Vector Title.
Return Code:	
`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

VectorTitlesV2

VectorTitlesV2 **(nSetID, bIncludeID, minID, maxID, Count, listID, listTITLE)**	
Description:	
This method retrieves two arrays which contain the IDs and titles of all Output Vectors in the selected ID range.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`BOOL bIncludeID`	If True, the titles are in the form “ID..title” (for example, “1..Total Translation”). If False, only the title, without the ID or separator is returned.
`INT4 minID`	The minimum and maximum Output Vector IDs to retrieve. You can set `minID=maxID=0` to retrieve all available Output Vector IDs.
`INT4 maxID`	
Output:	
`INT4 Ccount`	The number of Output Vectors retrieved. This is the number of entries in `listID` and `listTITLE`.
`INT4 listID[0..Count-1]`	The IDs of the Output Vectors.
`STRING listTITLE[0..Count-1]`	The titles of the available Output Vectors. If the Output Vector does not have a title, the string “Untitled” will be returned.
Return Code:	
`FE_NOT_EXIST`	No vectors exist in the selected range.
`FE_NO_MEMORY`	Cannot allocate enough memory for the Titles.
`FE_FAIL`	Failed to retrieve the titles (should not happen).
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

VectorComponentsV2

`VectorComponentsV2`
 	
`( nSetID, nVectorID, nNumComponents, nComponents )`	

Description:	
This method indicates whether a selected Output Vector has Component Output Vectors and, if so, returns the number of Component Output Vectors and their IDs. This method replaces `VectorComponents`. For more information, see **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
Output:	
`INT4 nNumComponents`	The number of components.
`INT4 nComponents[0..nNumComponents-1]`	An array of component vector IDs.
Return Code:	
`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.
Remarks/Usage:	
Components are the IDs of other Output Vectors that are related to the selected Output Vectors. For things like *Total Translation* (or any other Total Nodal output), there will be 3 components that represent the X,Y,Z component of the translation in Global Rectangular coordinates. For Elemental Output such as Plate or Solid Stresses, the Vector that contains the centroidal value will have components that represent the same value at the associated corners (nodes) of the element. For Beam/Bar results, there will be 2 components which are the vectors that contain the results at the two ends of the element. One of these will be the same ID as the selected vector.	

Unpublished work. © 2025 Siemens

Example:
None

VectorLocationV2

`VectorLocationV2`
 	
`( nSetID, nVectorID, nVectorLocation )`	

Description:	
This method identifies whether data for the Output Vector is in the database or in an attached external file. This method replaces `VectorLocation`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
Output:	
`INT4 nVectorLocation`	0=In Femap Database, 1=In Nastran OP2 File, 2=In FNO File.
Return Code:	
`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.
Remarks/Usage:	
None	
Example:	
None	

VectorEntitiesV2

Unpublished work. © 2025 Siemens

VectorEntitiesV2

(nSetID, nVectorID, nEntitySetID, bClear)

Description:

This method loads the IDs of all Nodes or Elements that contain data in the selected vector into a Set Object. This method replaces `VectorEntities`. For more information see, **Working with Output in Femap v2020.1 and above**.

Input:

`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
`INT4 nEntitySetID`	The ID of the Set Object that will receive the IDs of the Nodes or Elements that have data in this Output Vector.
`BOOL bClear`	If True, the Set Object is cleared prior to loading it with the entity IDs.

Output:

None	

Return Code:

`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.

Remarks/Usage:

You can tell whether the IDs reference Nodes or Elements by using the `VectorInfo` method.

Example:

None

VectorsV2

Unpublished work. © 2025 Siemens

VectorsV2	
(nSetID, nVectorSetID, bClear)	
Description:	
This method loads a Set Object with the IDs of all Output Vectors in a selected Output Set. This method replaces `Vectors`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorSetID`	The ID of the Set Object which will receive the Output Vector IDs.
`BOOL bClear`	If True, the Set Object is cleared prior to loading the IDs.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.
Remarks/Usage:	
None	
Example:	
None	

NonExistingUserVectorV2

NonExistingUserVectorV2
(nSetID)
Description:
This method an output vector ID of the next non-existing vector in the user created output vector block. Output Set.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nSetID`	The ID of the Output Set to query.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to return next non-existing vector ID.
Remarks/Usage:	
None	
Example:	
None	

VectorID

`VectorID`
`( )`
Description:
This method returns a `ResultsIDQuery` Object. It is often used for querying an output vector ID without initially declaring a `ResultsIDQuery` object. For more on the `ResultsIDQuery` object, see Results Vector ID Query Objects.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
None

Unpublished work. © 2025 Siemens

Example:
None

Results Browsing Entity Value Methods

The following method allows you to review the Output for individual Nodes or Elements in a selected Output Vector.

EntityValueV2

Warning

This method must be used with great caution - especially with attached results files. Using it may result in reading large amounts of data for every call, and performance will be especially poor. The only appropriate time to use this is if you must dynamically query the results on small numbers of nodes/elements. In almost any other case, it will be more efficient to code an algorithm using the *spreadsheet* capability of this object.

EntityValueV2 **(nSetID, nVectorID, nEntityID, dVal)**	
Description:	
This method returns the Output Value for a node/element in a selected Output Vector. This method replaces `EntityValue`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The ID of the Output Set to query.
`INT4 nVectorID`	The ID of the Output Vector to query.
`INT4 nEntityID`	The ID of the Node or Element to query. If the vector contains nodal data, this must be the ID of a node. Likewise, if the vector contains elemental data, this must be the ID of an element. Vectors that contain element corner data are still elemental vectors, and must be specified by element ID.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 dVal`	The Output Value for the selected Set, Vector and Node/Element.
Return Code:	
`FE_FAIL`	Unable to query the selected Output Vector - probably because it does not exist.
Remarks/Usage:	
See WARNING Above!	
Example:	
None	

Results Browsing Bulk Data Review Methods

The following method allows you to review the Output for individual Nodes or Elements in a manner that is similar to setting up, loading and querying a spreadsheet. For reviewing large amounts of data, these methods are by far the fastest.

AddColumnV2

`AddColumnV2`
`( nSetID, nVectorID, bAddComponentsCorners, nColumnsAdded, nColumnIndices  )`
Description:
This method adds a column to the object that will be filled by a specified Output Vector. This method replaces `AddColumn`. For more information see, **Working with Output in Femap v2020.1 and above**.
Input:

`INT4 nSetID`	The ID of the Output Set to load.
`INT4 nVectorID`	The ID of the Output Vector to Load.
`BOOL bAddComponentsCorners`	If True, additional columns will be added to accommodate the components of the selected vector, if any.

Output:

Unpublished work. © 2025 Siemens

`INT4 nColumnsAdded`	The total number of columns added.
`INT4 nColumnIndices[0..nColumnsAdded-1]`	The indices of the columns that were added. `nColumnIndices[0]` is always the index of the column for the selected vector. If component/corner vectors were added, their indices will be in `nColumnIndices[1]` and beyond.
Return Code:	
`FE_INVALID`	The object has already been populated with data. You can only add columns prior to populating the object.
`FE_NOT_EXIST`	The selected vector does not exist.
`FE_BAD_TYPE`	The selected vector is not of the same type (Nodal or Elemental) as other vectors in the object.
`FE_FAIL`	Unable to add column.
Remarks/Usage:	
Always remember to remember the `nColumnIndices` values - they will be needed to query the actual results after data is loaded by `Populate`.	
Example:	
None	

AddEmptyColumns

`AddEmptyColumns`	
`( nNumberOfColumns, nColumnIndices )`	
Description:	
This method adds one or more columns that are not associated with any Output Vectors.	
Input:	
`INT4 nNumberOfColumns`	The number of columns to add.
Output:	

Unpublished work. © 2025 Siemens

`INT4 nColumnIndices[0..nColumnsAdded-1]`	The indices of the columns that were added.
Return Code:	
`FE_INVALID`	The object has already been populated with data. You can only add columns prior to populating the object.
`FE_NO_MEMORY`	Unable to allocate memory to return the column indices.
Remarks/Usage:	
Use this method to add columns that you want to use to compute other values. When you populate the object, these columns will remain empty, but they will be available for you to store values computed from querying the other values.	
Example:	
None	

AddConversionColumnV2

`AddConversionColumnV2`
`( nSetID, nVectorID, nConversionApproach, nColumnIndex )`
Description:
This method adds one or more columns that are not associated with any Output Vectors. This method replaces `AddConversionColumn`. For more information see, **Working with Output in Femap v2020.1 and above**.
Input:

`INT4 nSetID`	The ID of the Output Set to load.
`INT4 nVectorID`	The ID of the Output Vector to Load.
`INT4 nConversionApproach`	0=Average, 1=Max Value, 2= Average, Skip Corner Results, 3=Max Value, Skip Corner Results, 4=Min Value, 5=Min Value, Skip Corner Results.
Output:	

Unpublished work. © 2025 Siemens

`INT4 nColumnIndex`	The index of the column that was added.
Return Code:	
`FE_INVALID`	The object has already been populated with data. You can only add columns prior to populating the object.
`FE_NOT_EXIST`	The selected vector does not exist.
`FE_BAD_TYPE`	The selected vector (once converted) is not of the same type (Nodal or Elemental) as other vectors in the object.
`FE_FAIL`	Unable to add column.
Remarks/Usage:	
Use this method to add columns that you want to use to convert existing Nodal results to Elemental, or vice versa. When you populate the object, these columns will be automatically populated with the converted results. If you do not want to convert based on results in the entire model, you must call `DataNeeded()` and define the Element ID set prior to calling this method. You must define the Element ID set, no matter whether you are converting Elemental to Nodal results or Nodal to Elemental.	
Example:	
None	

AddEnvelopeColumn

`AddEnvelopeColumn` **`( nEnvelopeType, nDataColumnIndex, nColumnColumnIndex )`**
Description:
This method adds columns that will be `Populated()` with an envelope of the results from the Output Vectors in each row.
Input:

`INT4 nEnvelopeType`	0=Max Value, 1=Min Value, 2=Max Absolute Value.

Unpublished work. © 2025 Siemens

Output:	
`INT4 nDataColumnIndex`	The index of the column that contains the enveloped results value.
`INT4 nColumnColumnIndex`	The index of the column that contains the index of the column where the enveloped value came from.
Return Code:	
`FE_INVALID`	The object has already been populated with data. You can only add columns prior to populating the object.
`FE_BAD_DATA`	An invalid value has been specified for the envelope type.
`FE_FAIL`	Unable to add column.
Remarks/Usage:	
Use this method if you want to find the maximum or minimum value of other Output Vectors. You can Add any other combination of Output Vectors that you want and they will be *enveloped* during `Populate()`. Empty columns are not considered. The envelope is performed as a final step so that all conversions and transforms have already been applied to the populated data prior to enveloping.	
Example:	

```
Sub Main
  Dim App As femap.model
  Set App = feFemap()

  Dim Res As femap.Results
  Set Res = App.feResults

  Dim ResVal1 As Double

  Dim numcolumns As Long
  Dim colindices As Variant
  Dim dataloc As Long
  Dim envcolumn As Long
```


Unpublished work. © 2025 Siemens

```
  Dim ColColIndex As Long


  Dim eset As femap.Set
  Set eset = App.feSet
  eset.AddAll (FT_ELEM)


  rc = Res.AddColumn (1, 7033, False, numcolumns, colindices)
  rc = Res.AddColumn (2, 7033, False, numcolumns, colindices)
  rc = Res.AddColumn (3, 7033, False, numcolumns, colindices)
  rc = Res.AddEnvelopeColumn (0, envcolumn, ColColIndex)
  rc = Res.DataLocation (dataloc)
  rc = Res.Populate


  While eset.Next
     Res.GetValue (eset.CurrentID, envcolumn, ResVal1)
      App.feAppMessage (FCM_NORMAL, _
        "Value"+Str$(eset.CurrentID)+ " is"+Str$(ResVal1))
  Wend


End Sub
```

```
Sub Main
Dim App As femap.model
Set App = feFemap()

Dim Res As femap.Results
Set Res = App.feResults

Dim rc As femap.zReturnCode
Dim nNumCol As Long
Dim vCol As Variant
Dim nEnvCol(1) As Long
Dim i As Long
Dim nID As Long
Dim dVals As Variant
Dim nSet As Long
```


Unpublished work. © 2025 Siemens

```
Dim nVec As Long


rc = Res.AddColumn( 1, 7020, False, nNumCol, vCol )
rc = Res.AddColumn( 1, 7021, False, nNumCol, vCol )
rc = Res.AddColumn( 1, 7023, False, nNumCol, vCol )
rc = Res.AddEnvelopeColumn( FOPA_ALL, nEnvCol(0), nEnvCol(1) )
Res.Populate()

For i=0 To Res.NumberOfRows()
 Res.GetMultipleInRow( i, 2, nEnvCol, nID, dVals )
 Res.GetColumnVector(dVals(1), nSet, nVec )
 App.feAppMessage( FCM_NORMAL, "ID="+Str$(nID)+"    Max="+ _
                Format$(dVals(0)," 0.000E+00;-0.000E+00")+ _
          "    From Set="+Str$(nSet)+" Vector="+Str$(nVec))
Next
End Sub
```

DataNeeded

DataNeeded	
(nDataType, nEntitySetID)	
Description:	
This method defines a set of entity IDs that will be used to `Populate` the object.	
Input:	
`INT4 nDataType`	The only available data types are 7=Node, 8=Element.
`INT4 nEntitySetID`	The ID of a Set object that contains the desired entity IDs.
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	The specified data type is not valid.
Remarks/Usage:	
This method must be called prior to calling `Populate()` and must specify the same data type as the Output Vectors being loaded if it is to be effective. You can also use this method prior to	

Unpublished work. © 2025 Siemens

calling `AddConversionColumn()` to specify Element IDs to be used in converting results during `Populate()`.
Example:
None

SetTri3Orientation

`SetTri3Orientation` **`( nStress, nStrain, nForce )`**	
Description:	
This method sets the default orientations for Elemental Results on 3-noded triangles.	
Input:	
`INT4 nStress`	0=First Edge, 1=Midside Locations, 3=Material Direction
`INT4 nStrain`	0=First Edge, 1=Midside Locations, 3=Material Direction
`INT4 nForce`	0=First Edge, 1=Midside Locations, 3=Material Direction
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	One or more of the arguments has an invalid value.
Remarks/Usage:	
This method is not normally required, as defaults are set as preferences, but if it is used, it must be called prior to calling `Populate()`.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetTri6Orientation

SetTri6Orientation **(nStress, nStrain, nForce)**	
Description:	
This method sets the default orientations for Elemental Results on 6-noded triangles.	
Input:	
`INT4 nStress`	0=First Edge, 1=Midside Locations, 3=Material Direction
`INT4 nStrain`	0=First Edge, 1=Midside Locations, 3=Material Direction
`INT4 nForce`	0=First Edge, 1=Midside Locations, 3=Material Direction
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	One or more of the arguments has an invalid value.
Remarks/Usage:	
This method is not normally required, as defaults are set as preferences, but if it is used, it must be called prior to calling `Populate()`.	
Example:	
None	

SetQuad4Orientation

Unpublished work. © 2025 Siemens

SetQuad4Orientation	
(nStress, nStrain, nForce)	
Description:	
This method sets the default orientations for Elemental Results on 4-noded quadrilateral.	
Input:	
`INT4 nStress`	0=First Edge, 1=Midside Locations, 2=Diagonal Bisector, 3=Material Direction
`INT4 nStrain`	0=First Edge, 1=Midside Locations, 2=Diagonal Bisector, 3=Material Direction
`INT4 nForce`	0=First Edge, 1=Midside Locations, 2=Diagonal Bisector, 3=Material Direction
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	One or more of the arguments has an invalid value.
Remarks/Usage:	
This method is not normally required, as defaults are set as preferences, but if it is used, it must be called prior to calling `Populate()`.	
Example:	
None	

SetQuad8Orientation

SetQuad8Orientation
(nStress, nStrain, nForce)
Description:

Unpublished work. © 2025 Siemens

This method sets the default orientations for Elemental Results on 8-noded quadrilateral.	
Input:	
`INT4 nStress`	0=First Edge, 1=Midside Locations, 2=Diagonal Bisector, 3=Material Direction
`INT4 nStrain`	0=First Edge, 1=Midside Locations, 2=Diagonal Bisector, 3=Material Direction
`INT4 nForce`	0=First Edge, 1=Midside Locations, 2=Diagonal Bisector, 3=Material Direction
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	One or more of the arguments has an invalid value.
Remarks/Usage:	
This method is not normally required, as defaults are set as preferences, but if it is used, it must be called prior to calling `Populate()`.	
Example:	
None	

SetSolidOrientation

`SetSolidOrientation` **`( nIsotropic, nAnisotropic, nHyperelastic )`**	
Description:	
This method sets the default orientations for Elemental Results on Solid elements.	
Input:	
`INT4 nIsotropic`	0=Material Direction, 1=Global Rectangular, 3=Element

Unpublished work. © 2025 Siemens

`INT4 nAnisotropic`	0=Material Direction, 1=Global Rectangular, 3=Element
`INT4 nHyperelastic`	0=Material Direction, 1=Global Rectangular, 3=Element
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	One or more of the arguments has an invalid value.
Remarks/Usage:	
This method is not normally required, as defaults are set as preferences, but if it is used, it must be called prior to calling `Populate()`.	
Example:	
None	

SetEngrShearStrain

`SetEngrShearStrain` **`( bEngrShearStrain )`**	
Description:	
This method sets the assumption for shear strain.	
Input:	
`BOOL bEngrShearStrain`	True=Engineering Shear Strain, False = Actual Shear Strain
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method is not normally required, as defaults are set as preferences, but if it is used, it must be called prior to calling `Populate()`.
Example:
None

SetNodalTransform

`SetNodalTransform` **`( nTransformTo, nCSysID )`**	
Description:	
This method defines parameters to transform nodal results when they are populated.	
Input:	
`INT4 nTransformTo`	0=No Transform, results in Global Rectangular, 1=Results in nCSysID, 2=Results in Nodal Output Coordinate Systems
`INT4 nCSysID`	ID of Coordinate System to transform into when `nTransformTo=1`.
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	An invalid value has been specified for `nTransformTo`.
`FE_NOT_EXIST`	The coordinate system specified by `nCSysID` does not exist.
Remarks/Usage:	
This method must be called prior to calling `Populate()`, but is not required if you do not want to transform results.	

Unpublished work. © 2025 Siemens

Example:
None

SetLineTransform

`SetLineTransform`	
`( nTransformTo, nCSysID )`	
Description:	
This method defines parameters to transform nodal results when they are populated.	
Input:	
`INT4 nTransformTo`	0=No Transform, results in Global Rectangular, 8=Trasform to line CSys.
`INT4 nCSysID`	ID of Coordinate System to transform into when `nTransformTo=1`.
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	An invalid value has been specified for `nTransformTo`.
`FE_NOT_EXIST`	The coordinate system specified by `nCSysID` does not exist.
Remarks/Usage:	
This method must be called prior to calling `Populate()`, but is not required if you do not want to transform results.	
Example:	
None	

SetPlateTransform

Unpublished work. © 2025 Siemens

SetPlateTransform

(nTransformTo, nCSysID, nCSysAxis, dVecX, dVecY, dVecZ, dToleranceAngle)

Description:

This method defines parameters to transform Elemental Results on planar/plate elements when they are populated.

Input:

`INT4 nTransformTo`	0=No Transform, 3=Transform to Material Coordinate System, 4=Transform to CSys `nCSysID`, 5=Transform to specified vector.
`INT4 nCSysID`	ID of Coordinate System to transform into when `nTransformTo=4`.
`INT4 nCSysAxis`	The Axis of `nCSysID` Coordinate System to use as the transform direction (1=X Axis, 2=Y Axis, 3=Z Axis).
`REAL8 dVecX`	The components of the vector to transform into when `nTransformTo=5`.
`REAL8 dVecY`	
`REAL8 dVecZ`	
`REAL8 dToleranceAngle`	This angle corresponds to the **Minimum Angle From Normal** option in the Transform Output Data dialog box. It is used to determine if the vector representing an element's normal direction is too closely aligned with the vector specified for transformation. If the angle between the two vectors is smaller than the specified tolerance then the transformation vector cannot be accurately projected onto the plane of the planar element, thus no transformation will occur. Only used when nTransformTo is set to 4 or 5.

Output:

None	

Return Code:

`FE_INVALID`	The object has already been populated with data.

Unpublished work. © 2025 Siemens

`FE_BAD_DATA`	An invalid value has been specified for `nTransformTo`.
`FE_NOT_EXIST`	The coordinate system specified by `nCSysID` does not exist.
Remarks/Usage:	
This method must be called prior to calling `Populate()`, but is not required if you do not want to transform results. When you transform plate data, you are really specifying a direction that the X component of the stress/strain/... is transformed into.	
Example:	
None	

SetSolidTransform

`SetSolidTransform`	
`( nTransformTo, nCSysID )`	
Description:	
This method defines parameters to transform Solid element results when they are populated.	
Input:	
`INT4 nTransformTo`	0=No Transform, results in Global Rectangular, 6=Results in `nCSysID`, 7=Results in Material Coordinate Systems.
`INT4 nCSysID`	ID of Coordinate System to transform into when `nTransformTo=6`.
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	An invalid value has been specified for `nTransformTo`.
`FE_NOT_EXIST`	The coordinate system specified by `nCSysID` does not exist.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method must be called prior to calling `Populate()`, but is not required if you do not want to transform results.
Example:
None

SetColumnGlobalPly

`SetColumnGlobalPly` **`( nColumnIndex, nGlobalPlyID )`**	
Description:	
This method specifies that a column be populated with data from a Global Ply rather than simply data from a single Output Vector.	
Input:	
`INT4 nColumnIndex`	The index of a column that was created by `AddColumn( )`.
`INT4 nGlobalPlyID`	The ID of a Global Ply or 0=Bottom Ply, or -2=Top Ply
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data.
`FE_BAD_DATA`	An invalid value has been specified for `nColumnIndex`.
Remarks/Usage:	
This method is used for laminate ply results and must be called prior to calling `Populate()`. If you simply want results from a specific ply on all selected elements, you do not need to use this method. However, if you want the column to be filled with data from a Global Ply that is represented by various ply numbers then use this method to choose that Global Ply.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetComplexOptions

SetComplexOptions	
(bExpandComplex , dComplexPhase)	
Description:	
This method defines parameters to expand complex results when they are populated.	
Input:	
`BOOL bExpandComplex`	If True, complex results will be expanded during `Populate` using the value specified for `dComplexPhase`.
`REAL8 dComplexPhase`	Phase value to use during complex expansion.
Output:	
None	
Return Code:	
`FE_FAIL`	No Output Sets were found.
Remarks/Usage:	
None	
Example:	
None	

Populate

Populate
( )
Description:
This method loads analysis results into the columns that you have defined.

Unpublished work. © 2025 Siemens

Input:	
None	
Output:	
None	
Return Code:	
`FE_INVALID`	The object has already been populated with data. You can only populate an object one time. If you want to reuse the same object, you must first call `Clear()`.
`FE_NOT_AVAILABLE`	You have not defined any columns to populate.
`FE_BAD_DATA`	There is a problem with the data you specified when calling `DataNeeded()`.
Remarks/Usage:	
This method must be called once to load results into the object columns. Normally, this method will create and load rows for all available Nodes/Elements. If you only want results for certain entities, call `DataNeeded()` prior to calling this method. If results are attached and there are Femap calculated vectors added into the Results Browsing Object, calling `Populate()` may add addition columns (see help on the `IsAPIColumn` method).	
Example:	
None	

IsPopulated

`IsPopulated`
`( )`
Description:
This method indicates whether `Populate( )` has already been called.
Input:
None
Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_FAIL`	The object has not been populated.
Remarks/Usage:	
None	
Example:	
None	

DataLocation

`DataLocation`
`( )`
Description:
This method indicates whether Nodal or Elemental data is or will be in the object.
Input:
None
Output:
`INT4 nDataType` — 7=Node, 8=Element
Return Code:
`FE_FAIL` — No columns have yet been defined that identify whether nodal or elemental data will be loaded into this object.
Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

NumberOfRows

NumberOfRows	
( )	
Description:	
This method indicates the number of rows that have been defined.	
Input:	
None	
Output:	
None	
Return Code:	
Number	The number of rows of data that have been loaded. Prior to calling `Populate()` this will be 0.
Remarks/Usage:	
None	
Example:	
None	

NumberOfColumns

NumberOfColumns	
( )	
Description:	
This method indicates the number of columns that have been defined.	
Input:	
None	

Unpublished work. © 2025 Siemens

Output:	
None	
Return Code:	
Number	The number of columns of data that have been defined.
Remarks/Usage:	
None	
Example:	
None	

FindColumnV2

`FindColumnV2` **`( nSetID, nVectorID, nColumnIndex )`**	
Description:	
This method locates a column based on the Output Set and Output Vector IDs. This method replaces `FindColumn`. For more information see, **Working with Output in Femap v2020.1 and above**.	
Input:	
`INT4 nSetID`	The desired Output Set ID.
`INT4 nVectorID`	The desired Output Vector ID.
Output:	
`INT4 nColumnIndex`	The zero-based index of the location of the desired column.
Return Code:	
`FE_NOT_AVAILABLE`	No columns have been defined.
`FE_FAIL`	Unable to located the desired column.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetColumnTitle

`GetColumnTitle` **`( nColumnIndex, sTitle )`**	
Description:	
This method returns the title of the Output Vector loaded in the specified column.	
Input:	
`INT4 nColumnIndex`	Zero-based index of the desired column.
Output:	
`STRING sTitle`	Title of the Output Vector in specified column.
Return Code:	
FE_BAD_DATA	Column index is outside of the range of available columns
FE_FAIL	Unable to retrieve title.
Remarks/Usage:	
None	
Example:	
None	

IsApiColumn

Unpublished work. © 2025 Siemens

IsApiColumn **(nColumnIndex)**	
Description:	
This method identifies whether a column was added by an API method or one that was added for internal computations.	
Input:	
`INT4 nColumnIndex`	Zero-based index of the desired column.
Output:	
None	
Return Code:	
`FE_OK`	Column is used for internal calculation.
`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_FAIL`	Not an API Column.
Remarks/Usage:	
In some cases, especially with attached result files, Femap must create additional columns to compute the results that were requested. These columns will return `FE_FAIL` from this method.	
Example:	
None	

GetColumnVectorV2

GetColumnVectorV2 **(nColumnIndex, nSetID, nVectorID)**
Description:
This method returns the Output Set ID and Output Vector ID of a selected column. This method replaces `GetColumnVector`. For more information see, **Working with Output in Femap v2020.1 and above**.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nColumnIndex`	Zero-based index of the desired column.
Output:	
`INT4 nSetID`	ID of the Output Set in the selected column.
`INT4 nVectorID`	ID of the Output Vector in the selected column.
Return Code:	
`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_FAIL`	Unable to retrieve vector info.
Remarks/Usage:	
None	
Example:	
None	

GetColumn

`GetColumn` **`( nColIndex, nIDs, dVals )`**
Description:
This method returns the entity IDs and values from all rows in the selected column.
Input:

`INT4 nColIndex`	Zero-based index of the desired column.
Output:	
`INT4 nIDs [0..nNumberOfRows -1]`	The IDs of the Node/Element.
`REAL8 dVals[0..nNumberOfRows-1]`	The result values.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/columns.
Remarks/Usage:	
This method will only work once the object has been populated. It simply returns values for all rows that have been defined.	
Example:	
None	

GetColumns

`GetColumns` **`( nNumberOfColumns, nColIndices, nIDs, dVals )`**	
Description:	
This method returns the entity IDs and multiple values from the selected columns and all rows.	
Input:	
`INT4 nNumberOfColumns`	The number of column indices in `nColIndices`.
`INT4 nColIndices[0..nNumberOfColumns]`	Zero-based indices of the desired columns.
Output:	
`INT4 nIDs[0..nNumberOfRows-1]`	The Node/Element ID from each row.
`REAL8 dVals[0..nNumberOfColumns*nNumberOfRows-1]`	The result values.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified columns/rows.

Remarks/Usage:

This method will only work once the object has been populated. The entries in `nIDs[ ]` are the Node/Element IDs that directly correspond to the rows of the columns selected in `nColIndices`. The entries in `dVals` are the values from all the selected columns and all rows. Data is stored by rows so that all selected columns for the first row come before the second row, and so on. For example, `dVals[0]` is from the first row of the first selected column, `dVals[1]` is from the first row of the second selected column, then `dVals[nNumberOfColumns]` is from the second row of the first selected column, and so on.

Example:

None

GetColumnMinMax

`GetColumnMinMax`

`( nColumnIndex, nLimitToSet, nMinID, nMaxID, dMinVal, dMaxVal )`

Description:

This method returns the minimum and maximum values in a column.

Input:

`INT4 nColumnIndex`	Zero-based index of the desired column.
`INT4 nLimitToSet`	ID of Set object that contains the entities to consider when searching for min/max. Set to 0 if you want to consider all results that have been loaded in the column.

Output:

`INT4 nMinID`	ID of the Node/Element where the minimum value occurs.

Unpublished work. © 2025 Siemens

`INT4 nMaxID`	ID of the Node/Element where the maximum value occurs.
`REAL8 dMinVal`	The minimum value.
`REAL8 dMaxVal`	The maximum value.
Return Code:	
`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to compute min/max.
Remarks/Usage:	
None	
Example:	
None	

GetColumnAbsMax

`GetColumnAbsMax` **`( nColumnIndex, nLimitToSet, pnAbsMaxID, pdAbsMaxVal )`**	
Description:	
This method returns the maximum absolute value in a column.	
Input:	
`INT4 nColumnIndex`	Zero-based index of the desired column.
`INT4 nLimitToSet`	ID of Set object that contains the entities to consider when searching for min/max. Set to 0 if you want to consider all results that have been loaded in the column.
Output:	

Unpublished work. © 2025 Siemens

`INT4 pnAbsMaxID`	ID of the Node/Element where the maximum absolute value occurs.
`REAL8 pdAbsMaxVal`	The maximum absolute value.
Return Code:	
`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to compute max.
Remarks/Usage:	
None	
Example:	
None	

GetColumnSum

`GetColumnSum`	
`( nColumnIndex, nLimitToSet, nNumVal, dSum, dSumOfSquares )`	
Description:	
This method returns values for the sum and the sum of squares in a column.	
Input:	
`INT4 nColumnIndex`	Zero-based index of the desired column.
`INT4 nLimitToSet`	ID of Set object that contains the entities to consider when calculating sum and sum of squares. Set to 0 if you want to consider all results that have been loaded in the column.
Output:	
`INT4 nNumVal`	Number of values used to compute sum and sum of squares.

Unpublished work. © 2025 Siemens

`REAL8 dSum`	The sum of the column.
`REAL8 dSumOfSquares`	The sum of squares of the column.
Return Code:	
`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to compute sum and/or sum of squares.
Remarks/Usage:	
None	
Example:	
None	

Reset

`Reset`
`( )`
Description:
This method resets internal row pointers so that the next call to `Next()` will return the first row.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
None

Unpublished work. © 2025 Siemens

Example:
None

Next

Next (nColumnIndex, nID, dVal)	
Description:	
This method returns the `entityID` and value from the next row in a selected column.	
Input:	
`INT4 nColumnIndex`	Zero-based index of the desired column.
Output:	
`INT4 nID`	The ID of the Node/Element from the next row.
`REAL8 dVal`	The result value from the selected column in the next row.
Return Code:	
`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	You have reached the end of available rows.
Remarks/Usage:	
This method will only work once the object has been populated. You must call `Reset()` prior to calling this function so that repeated calls will start at the first row. Changing `nColumnIndex` will not automatically reset back to the first row, it will continue from the location of the previous call to `Next()`.	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
   Dim Res As femap.Results
   Set Res = App.feResults
   Dim nNumCol As Long
   Dim vCol As Variant
   Dim nID As Long
   Dim dVal As Double
   Dim rc As femap.zReturnCode

   rc = Res.AddColumn( 1, 7020, False, nNumCol, vCol )
   rc = Res.Populate()
 ' This will retrieve all values from the added column
   Res.Reset()
   While Res.Next( vCol(0), nID, dVal) = FE_OK
     App.feAppMessage( FCM_NORMAL, _
                "ID="+Str$(nID)+"   Value="+Str$(dVal))
   Wend
End Sub
```

GetValue

GetValue

(nID, nColumnIndex, dVal)

Description:

This method returns the value from the selected column for the specified Node/Element.

Input:

`INT4 nID`	The ID of the Node/Element desired.
`INT4 nColumnIndex`	Zero-based index of the desired column.

Output:

`REAL8 dVal`	The result value.

Return Code:

`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to find data for the specified ID.
Remarks/Usage:	
This method will only work once the object has been populated.	
Example:	
None	

GetInRow

`GetInRow` **`( nRowIndex, nColumnIndex, nID, dVal )`**	
Description:	
This method returns the entity ID and value from the selected row and column.	
Input:	
`INT4 nRowIndex`	Zero-based index of the desired row.
`INT4 nColumnIndex`	Zero-based index of the desired column.
Output:	
`INT4 nID`	The ID of the Node/Element.
`REAL8 dVal`	The result value.
Return Code:	
`FE_BAD_DATA`	Either Row or Column index is outside of the range of available rows/columns.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/column.
Remarks/Usage:	
This method will only work once the object has been populated.	

Unpublished work. © 2025 Siemens

Example:
None

GetMultipleInRow

GetMultipleInRow **(nRowIndex, nNumberOfColumns, nColumnIndices, nID, dVals)**	
Description:	
This method returns the entity ID and multiple values from the selected row and columns.	
Input:	
`INT4 nRowIndex`	Zero-based index of the desired row.
`INT4 nNumberOfColumns`	The number of entries in `nColumnIndices`.
`INT4 nColumnIndices [0..nNumberOfColumns-1]`	Zero-based indices of the desired columns.
Output:	
`INT4 nID`	The ID of the Node/Element.
`REAL8 dVals[0..nNumberOfColumns-1]`	The result values, in the same order as `nColumnIndices`.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/columns.
Remarks/Usage:	
This method will only work once the object has been populated.	

Unpublished work. © 2025 Siemens

Example:
None

GetRow

`GetRow` **`( nRowIndex, nID, dVals )`**	
Description:	
This method returns the entity ID and multiple values from the selected row and all columns.	
Input:	
`INT4 nRowIndex`	Zero-based index of the desired row.
Output:	
`INT4 nID`	The ID of the Node/Element.
`REAL8 dVals[0..nNumberOfColumns-1]`	The result values.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/columns.
Remarks/Usage:	
This method will only work once the object has been populated. It is similar to `GetMultipleInRow( )`, except that it simply returns values for all columns that have been defined. You can use the column index values that were returned when columns were added as an index into the returned value array.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetRowByID

GetRowByID **(nID, dVals)**	
Description:	
This method returns multiple values from the selected Node/Element and all columns.	
Input:	
`INT4 nIcD`	The ID of the Node/Element.
Output:	
`REAL8 dVals[0..nNumberOfColumns-1]`	The result values.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified Node/Element.
Remarks/Usage:	
This method will only work once the object has been populated. Rather than using the `rowIndex`, this method directly looks for a specific Node/Element ID. It is similar to `GetMultipleInRow( )`, except that it simply returns values for all columns that have been defined. You can use the column index values that were returned when columns were added as an index into the returned value array.	
Example:	
None	

GetRows

Unpublished work. © 2025 Siemens

GetRows

(nNumberOfRows, nRowIndices, nIDs, dVals)

Description:

This method returns the entity IDs and multiple values from the selected rows and all columns.

Input:

`INT4 nNumberOfRows`	The number of row indices in `nRowIndices`.
`INT4 nRowIndices[0..nNumberOfRows]`	Zero-based indices of the desired rows.

Output:

`INT4 nIDs[0..nNumberOfRows]`	The IDs of the Node/Element from each selected row.
`REAL8 dVals[0..nNumberOfRows*nNumberOfColumns-1]`	The result values.

Return Code:

`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/columns.

Remarks/Usage:

This method will only work once the object has been populated. It is just like `GetRow( )`, except that it will return data from multiple selected rows. The entries in `nIDs[ ]` are the Node/Element IDs that directly correspond to the rows selected in `nRowIndices`. The entries in `dVals` are the values from all the selected rows and all columns. Data is stored by row so that all columns for the first selected row come before the second row, and so on. For example, `dVals[0]` is from the first column of the first selected row, `dVals[1]` is from the second column of the first row ...then `dVals[nNumberOfColumns]` is from the first column of the second row, and so on.

Example:

None

Unpublished work. © 2025 Siemens

GetRowsByID

GetRowsByID **(nEntitySetID, dVals)**	
Description:	
This method returns the values from all columns for the selected entities.	
Input:	
`INT4 nEntitySetID`	The ID of a Set object that contains the Node/Element IDs that are desired.
Output:	
`REAL8 dVals[0..nNumberOfRows*nNumberOfColumns-1]`	The result values.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/columns.
Remarks/Usage:	
This method will only work once the object has been populated. It is just like `GetRowByID( )`, except that it will return data from multiple selected rows. The entries in `dVals` are the values from all the selected rows and all columns. `nNumberOfRows` is the number of IDs in `nEntitySetID`, and IDs/rows are always in increasing order from smallest to largest, no matter how the Set Object was created. Data is stored by row so that all columns for the first selected row come before the second row, and so on. For example, `dVals[0]` is from the first column of the first selected row, `dVals[1]` is from the second column of the first row ...then `dVals[nNumberOfColumns]` is from the first column of the second row, and so on.	
Example:	
None	

GetRowsAndColumns

Unpublished work. © 2025 Siemens

GetRowsAndColumns

(nNumberOfRows, nRowIndices, nNumberOfColumns, nColIndices, nIDs, dVals, bAllExist)

Description:

This method returns the entity IDs and multiple values from selected rows and columns.

Input:

`INT4 nNumberOfRows`	The number of row indices in `nRowIndices`.
`INT4 nRowIndices[0..nNumberOfRows]`	Zero-based indices of the desired rows.
`INT4 nNumberOfColumns`	The number of column indices in `nColIndices`.
`INT4 nColIndices[0..nNumberOfColumns]`	Zero-based indices of the desired columns.

Output:

`INT4 nIDs[0..nNumberOfRows]`	The IDs of the Node/Element from each selected row.
`REAL8 dVals[0..nNumberOfRows*nNumberOfColumns-1]`	The result values.
`BOOL AllExist`	When True, all rows and columns have values. When False, at least one value in the selected columns and rows is missing.

Return Code:

`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to find data for the specified row/columns.

Remarks/Usage:

This method will only work once the object has been populated. It is just like `GetRow( )`, except that it will return data from multiple selected rows and columns. The entries in `nIDs[ ]` are the Node/Element IDs that directly correspond to the rows selected in `nRowIndices`. The entries in `dVals` are the values from all the selected rows and selected columns. Data is stored by row so that all selected columns

Unpublished work. © 2025 Siemens

for the first selected row come before the second row, and so on. For example, `dVals[0]` is from the first column of the first selected row, `dVals[1]` is from the second column of the first row...then `dVals[nNumberOfColumns]` is from the first column of the second row, and so on.
Example:
None

GetRowsAndColumnsByID

`GetRowsAndColumnsByID`	
`( nEntitySetID, nNumberOfColumns, nColIndices, dVals, bAllExist )`	
Description:	
This method returns the values from selected columns for the selected entities.	
Input:	
`INT4 nEntitySetID`	The ID of a Set object that contains the Node/Element IDs that are desired.
`INT4 nNumberOfColumns`	The number of column indices in `nColIndices`.
`INT4 nColIndices[0..nNumberOfColumns]`	Zero-based indices of the desired columns.
Output:	
`REAL8 dVals[0..nNumberOfRows*nNumberOfColumns-1]`	The result values.
`BOOL AllExist`	When True, all rows and columns have values. When False, at least one value in the selected columns and rows is missing.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to find data for the specified row/columns.

Remarks/Usage:

This method will only work once the object has been populated. It is just like `GetRowByID( )`, except that it will return data from multiple selected rows using selected columns. The entries in `dVals` are the values from all the selected rows and selected columns. `nNumberOfRows` is the number of IDs in `nEntitySetID`, and IDs/rows are always in increasing order from smallest to largest, no matter how the Set Object was created. Data is stored by row so that selected columns for the first selected row come before the second row, and so on. For example, `dVals[0]` is from the first column of the first selected row, `dVals[1]` is from the second column of the first row...then `dVals[nNumberOfColumns]` is from the first column of the second row, and so on.

Example:

None

SetValue

`SetValue`

`( nID, nColumnIndex, dVal )`

Description:

This method sets the value in the selected column for the specified Node/Element.

Input:

`INT4 nID`	The ID of the Node/Element desired.
`INT4 nColumnIndex`	Zero-based index of the desired column.
`REAL8 dVal`	The value to set in the row and column.

Output:

None	

Return Code:

`FE_BAD_DATA`	Column index is outside of the range of available columns.
`FE_FAIL`	Unable to find data for the specified ID.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method allows you to update the value in a row/column. It is intended to be used so that you can use the data in other columns, combined with data and logic in your program to fill values in empty columns.
Example:
None

SetColumn

SetColumn **(nColIndex, nNumberOfRows, nIDs, dVals)**	
Description:	
This method fills a column with data.	
Input:	
`INT4 nColIndex`	The column index to fill.
`INT4 nNumberOfRows`	The number of rows that will be filled.
`INT4 nIDs[0..nNumberOfRows-1]`	The IDs of the nodes/elements associated with the filled data.
`REAL8 dVals[0..nNumberOfRows-1]`	The values associated with each of the node/element IDs accordingly.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to fill column with data.
Remarks/Usage:	
The `Save` method must be used after `SetColumn` to put results.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetColumns

SetColumns

(nNumberOfColumns, nColIndices, nNumberOfRows, nIDs, dVals)

Description:

This method fills a column with data.

Input:

`INT4 nNumberOfColumns`	The number of columns to be filled.
`INT4 nColIndices`	An index of the columns that will be filled.
`INT4 nNumberOfRows`	The number of rows that will be filled.
`INT4 nIDs[0..(nNumberOfRows*nNumberOfColumns)-1]`	The IDs of the nodes/elements associated with the filled data.
`REAL8 dVals[0..nNumberOfRows*nNumberOfColumns)-1]`	The values associated with each of the node/element IDs accordingly.

Output:

None	

Return Code:

`FE_FAIL`	Unable to fill columns with data.

Remarks/Usage:

The data for `nIDs` is stored in 1-dimensional array, while the data for dVals is stored in a separate 1-dimensional array using the following format:

```
(Column0,Row0), (Column1,Row0)…(ColumnN,Row0); (Column0,Row1),
(Column1,Row1)…
(ColumnN,Row1);...; (Column0,RowM), (Column1,RowM)…(ColumnN,RowM)
```

In addition, the `Save` method must be used after `SetColumns` to put results.

Unpublished work. © 2025 Siemens

Example:
None

SetInRow

SetInRow **(nRowIndex, nColumnIndex, dVal)**	
Description:	
This method sets the value in the selected column for the specified row.	
Input:	
`INT4 nRowIndex`	Zero-based index of the desired row.
`INT4 nColumnIndex`	Zero-based index of the desired column.
`REAL8 dVal`	The value to set in the row and column.
Output:	
None	
Return Code:	
`FE_BAD_DATA`	Either Row or Column index is outside of the range of available rows/columns.
`FE_FAIL`	Unable to find data for the specified ID.
Remarks/Usage:	
This method allows you to update the value in a row/column. It is intended to be used so that you can use the data in other columns, combined with data and logic in your program to fill values in empty columns.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetMultipleInRow

SetMultipleInRow **(nRowIndex, nNumberOfColumns, nColumnIndices, dVals)**	
Description:	
This method sets the values in multiple columns of a selected row.	
Input:	
`INT4 nRowIndex`	Zero-based index of the desired row.
`INT4 nNumberOfColumns`	The number of entries in `nColumnIndices`.
`INT4 nColumnIndices [0..nNumberOfColumns-1]`	Zero-based indices of the desired columns.
`REAL8 dVals[0..nNumberOfColumns-1]`	The values to define, in the same order as `nColumnIndices`.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to find data for the specified row/columns.
Remarks/Usage:	
This method allows you to update the value in a row/column. It is intended to be used so that you can use the data in other columns, combined with data and logic in your program to fill values in empty columns.	
Example:	
None	

SetRow

Unpublished work. © 2025 Siemens

SetRow	
(nID, dVals)	
Description:	
This method sets the values in multiple columns for a selected node or element ID.	
Input:	
`INT4 nID`	The ID of the node/element.
`REAL8 dVals[0..NumberOfColumns]`	The values to be put in the columns of the specified node/element.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetRows

SetRows
(nNumberOfRows, nIDs, dVals)
Description:
This method sets the values in multiple columns of a selected node or element IDs.
Input:

Unpublished work. © 2025 Siemens

`INT4 nNumberOfRows`	The number or rows or nodes/elements.
`INT4 nIDs[0..(nNumberOfRows*NumberOfColumns)-1]`	The IDs of the nodes/elements.
`REAL8 dVals[0..(nNumberOfRows*NumberOfColumns)-1]`	The values to be put in the columns for the specified node/elements.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	

The data for `nIDs` is stored in 1-dimensional array, while the data for `dVals` is stored in a separate 1-dimensional array using the following format:

```
(Column0,Row0), (Column1,Row0)…(ColumnN,Row0); (Column0,Row1),
(Column1,Row1)…
(ColumnN,Row1);...; (Column0,RowM), (Column1,RowM)…(ColumnN,RowM)
```

This method allows you to update the values of columns for multiple node or element IDs.

Example:

None

SetRowsByID

`SetRowsByID`

`( nEntitySetID, dVals )`

Description:

This method sets the values in multiple columns of a given Entity Set.

Input:

Unpublished work. © 2025 Siemens

`INT4 nEntitySetID`	The ID of the Set of nodes/elements.
`REAL8 dVals[0..(EntitySet.count * NumberOfColumns)]`	The values to be put in the columns for the specified Set.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
This method allows you to update the values of columns for given entities in a Set.	
Example:	
None	

SetRowsAndColumnByID

`SetRowsAndColumnByID` **`( nEntitySetID, nNumberOfColumns, nColIndices, dVals )`**	
Description:	
This method sets the values in multiple columns of a given Entity Set.	
Input:	
`INT4 nEntitySetID`	The ID of the Set of nodes/elements.
`INT4 nNumberOfColumns`	The number of entries in `nColumnIndices`.
`INT4 nColIndices[0..nNumberOfColumns - 1]`	An array of column numbers to be filled.

Unpublished work. © 2025 Siemens

`REAL8 dVals[0..(EntitySet.Count * nNumberOfColumns)]`	The values to be put in the columns for the specified Set.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
This method allows you to update the values of specific columns for given entities in a Set.	
Example:	
None	

Results Browsing Update and Save Methods

The following methods allow you to update specific entries in a manner that is similar to setting up, loading and querying a spreadsheet. These methods will also allow you to initialize and save user created data as output.

AddScalarAtNodeColumnV2

`AddScalarAtNodeColumnV2`	
`( nSetID, nVectorID, title, category, nonlinear, nColumnIndex )`	
Description:	
This method initializes a column to prepare it for creating an output vector containing scalar values at nodes.	
Input:	
`INT4 nSetID`	The ID of the Output Set.
`INT4 nVectorID`	The ID of the Vector to initialize.

Unpublished work. © 2025 Siemens

`STRING title`	The title of columns being added.
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
Output:	
`INT4 nColumnIndex`	The index of the column that was initialized.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

AddVectorAtNodeColumnsV2

`AddVectorAtNodeColumnsV2`
`( nSetID, nVectorID, VecX, VecY, VecZ, title, category, nonlinear, pnColumnIndices )`
Description:
This method initializes columns to prepare it for creating output vectors containing vector values at nodes.
Input:

`INT4 nSetID`	The ID of the Output Set.
`INT4 nVectorID`	The ID of the output vector which will be created to contain the magnitude of the output vectors.

Unpublished work. © 2025 Siemens

`INT4 VecX`	The ID of the output vector which will be created to contain the X components of output.
`INT4 VecY`	The ID of the output vector which will be created to contain the Y components of output.
`INT4 VecZ`	The ID of the output vector which will be created to contain the Z components of output.
`STRING title`	The title of columns being added.
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
Output:	
`INT4 pnColumnIndices`	The indices of the columns that were initialized.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

AddScalarAtElemColumnV2

`AddScalarAtElemColumnV2`
`( nSetID, nVectorID, title, category, nonlinear, nColumnIndex )`
Description:
This method initializes a column to prepare it for creating an output vector containing scalar values at elements.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nSetID`	The ID of the Output Set.
`INT4 nVectorID`	The ID of the Vector to initialize.
`STRING title`	The title of columns being added.
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
Output:	
`INT4 nColumnIndex`	The index of the columns that was initialized.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

AddElemWithCornerColumnsV2

`AddElemWithCornerColumnsV2`
`( nSetID, nVectorID, Vec1, Vec2, Vec3, Vec4, Vec5, Vec6, Vec7, Vec8, title, category, nonlinear, pnColumnIndice )`
Description:
This method initializes a column to prepare it for creating an output vector containing scalar values at elements.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nSetID`	The ID of the Output Set.
`INT4 nVectorID`	The ID of the output vector which will be created containing the results at the corresponding element corners.
`INT4 Vec1`	
`INT4 Vec2`	
`INT4 Vec3`	
`INT4 Vec4`	
`INT4 Vec5`	
`INT4 Vec6`	
`INT4 Vec7`	
`INT4 Vec8`	
`STRING title`	The title of columns being added.
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
Output:	
`INT4 pnColumnIndices`	The indice of the columns that were initialized.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

AddScalarAtBeamColumnsV2

AddScalarAtBeamColumnsV2 **(nSetID, VecA, VecB, title, vecstyle, category, nonlinear, add_endAB_titles, pnColumnIndices)**	
Description:	
This method initialized the column to prepare it for creating Beam output vectors for EndA and EndB.	
Input:	
`INT4 nSetID`	The ID of the Output Set.
`INT4 VecA`	The ID of the EndA output vector to be created.
`INT4 VecB`	The ID of the EndB output vector to be created.
`STRING title`	The title of columns being added.
`INT4 vecstyle`	If 1, component[0..2] are the X,Y,Z component values. If 2, data at end of beams. If 3, reverse data at second end of beam.
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
`BOOL add_endAB_titles`	Flag to prepend End A, End B to the title.
Output:	
`INT4 pnColumnIndices[0..NumberOfColumns - 1]`	The indices of the columns.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
This method allows you to update the values of specific columns for given entities in a Set.	
Example:	
None	

SetColumnInfo

`SetColumnInfo` **`( nColumnIndex, nDataType, nOutputType, bCentroidTotal, bNonlinear )`**	
Description:	
This method set information of output vector stored in a column.	
Input:	
`INT4 nColumnIndex`	The index of column to set output vector info.
`INT4 nDataType`	The data type of the entity. `FT_NODE` for nodal data or `FT_ELEM` for elemental data.
`INT4 nOutputType`	The type of output stored in the column. Displacement, Force, Stress, etc.
`BOOL bCentroidTotal`	If True, this vector has centroidal or nodal output. If False, contains elemental corner output.
`BOOL bNonlinear`	If True, this result cannot be linearly combined explicitly, as it is calculated from underlying vector or tensor components (for example, Total Translation or von Mises Stress).
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_BAD_DATA`	Column Index is invalid. Must be greater than or equal to 0 and less than the number of columns in the object.
`FE_BAD_TYPE`	Data Type must match the type of any existing data in the object.
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

GetColumnInfo

`GetColumnInfo` **`( nColumnIndex, nDataType, nOutputType, bCentroidTotal, bCalcWarn )`**	
Description:	
This method set information of output vector stored in a column.	
Input:	
`INT4 nColumnIndex`	The index of to set.
Output:	
`INT4 nDataType`	The data type of the entity. `FT_NODE` for nodal data or `FT_ELEM` for elemental data.
`INT4 nOutputType`	The type of output stored in the column. Displacement, Force, Stress, etc.
`BOOL bCentroidTotal`	If True, this vector has centroidal or nodal output. If False, contains elemental corner output.
`BOOL bCalcWarn`	If True, this result cannot be linearly combined explicitly, as it is calculated from underlying

Unpublished work. © 2025 Siemens

	vector or tensor components (for example, Total Translation or von Mises Stress).
Return Code:	
FE_BAD_DATA	Column Index is invalid. Must be greater than or equal to 0 and less than the number of columns in the object.
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetColumnComponents

`SetColumnComponents` **`( nColumnIndex, pnNumComponents, pnComponentColumn )`**	
Description:	
This method sets the number of components of an Output Vector and the column index in which the component results are stored.	
Input:	
`INT4 nColumnIndex`	The index of the column.
`INT4 pnNumComponents`	The number of components.
`INT4 pnComponentColumns[0..pnNumComponents - 1]`	The indices of the component columns.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
`SetColumnsComponent` is an identical method to `SetColumnComponents`, thus functions in the same manner, but if possible, it is recommended to use `SetColumnComponents`.	
Example:	
None	

SetColumnComponents2

`SetColumnComponents2` **`( nColumnIndex, nComponentType, pnNumComponents, pnComponentColumn )`**	
Description:	
This method sets the number of components of an Output Vector and the column index in which the component results are stored.	
Input:	
`INT4 nColumnIndex`	The index of the column.
`INT4 nComponentType`	The component type of the column.
`INT4 pnNumComponents`	The number of components.
`INT4 pnComponentColumns[0..pnNumComponents - 1]`	The indices of the component columns.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

`SetColumnsComponent` is an identical method to `SetColumnComponents`, thus functions in the same manner, but if possible, it is recommended to use `SetColumnComponents`.
Example:
None

GetColumnComponents

`GetColumnComponents` **`( nColumnIndex, pnNumComponents, pnComponentColumn )`**	
Description:	
This method returns the number of components of an Output Vector and the column index in which the component results are stored.	
Input:	
`INT4 nColumnIndex`	The index of the column.
Output:	
`INT4 pnNumComponents`	The number of components.
`INT4 pnComponentColumns[0..pnNumComponents - 1]`	The indices of the component columns.
Return Code:	
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetColumnTitle

Unpublished work. © 2025 Siemens

SetColumnTitle	
(nColumnIndex, title)	
Description:	
This method sets the title of a column for naming the Output Vector.	
Input:	
`INT4 nColumnIndex`	The index of the column.
`STRING title`	The title to name the Output Vector.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns,
Remarks/Usage:	
None	
Example:	
None	

SetColumnVectorV2

SetColumnVectorV2	
(nColumnIndex, nSetID, nVectorID)	
Description:	
This method sets the Output Vector ID for a specific column of in an Output Set.	
Input:	
`INT4 nColumnIndex`	The index of the column.

Unpublished work. © 2025 Siemens

`INT4 nSetID`	The ID of the Output Set.
`INT4 nVectorID`	The ID of the Output Vector to set.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetColumnInteger

`SetColumnInteger` **`( nColumnIndex, bIsInteger )`**	
Description:	
This method finds if an Output Vector in a specified column is integer values.	
Input:	
`INT4 nColumnIndex`	The index of the column.
`BOOL bIsInteger`	If True, this vector will be initialized to store integer values.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetColumnInteger

`GetColumnInteger` **`( nColumnIndex, bIsInteger )`**	
Description:	
This method finds if an Output Vector in a specified column is integer values.	
Input:	
`INT4 nColumnIndex`	The index of the column.
Output:	
`BOOL bIsInteger`	If True, this vector was initialized to store integer values.
Return Code:	
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SaveColumn

Unpublished work. © 2025 Siemens

SaveColumn	
(nColumnIndex)	
Description:	
This method saves the data that is filled in a specific columns.	
Input:	
`INT4 nColumnIndex`	The index of the column to save.
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

Save

Save
( )
Description:
This method saves the data that is filled in the columns and rows. This method should be used after modifying or creating any data that is added to the table.
Input:
None
Output:

Unpublished work. © 2025 Siemens

None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

Save2

Save2 **(bAddToExistingData, bDataAddedInRandomIDOrder)**	
Description:	
This method saves the data that is filled in the columns and rows. This method should be used after modifying or creating any data that is added to the table.	
Input:	
`BOOL bAddToExistingData`	
`BOOL bDataAddedInRandomIDOrder`	
Output:	
None	
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetScalarAtNodeColumnV2

`GetScalarAtNodeColumnV2` **`( nColumnIndex, listcount, nID, value )`**	
Description:	
This method returns the entity IDs and values from all rows in the selected column.	
Input:	
`INT4 nColumnIndex`	The index of the column.
Output:	
`INT4 listcount`	The count of nodes in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the Node/Element.
`REAL8 value[0..listcount - 1]`	The results values.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetScalarAtNodeColumnV2

SetScalarAtNodeColumnV2 **(nColumnIndex, listcount, nID, value)**	
Description:	
This method stores output data in a column.	
Input:	
`INT4 nColumnIndex`	The index of the column.
`INT4 listcount`	The count of nodes in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the Node/Element.
`REAL8 value[0..listcount - 1]`	The results values.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

GetScalarAtElemColumnV2

Unpublished work. © 2025 Siemens

GetScalarAtElemColumnV2 **(nColumnIndex, listcount, nID, value)**	
Description:	
This method returns the entity IDs and values from all rows in the selected column.	
Input:	
`INT4 nColumnIndex`	The index of the column.
Output:	
`INT4 listcount`	The count of nodes in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the Node/Element.
`REAL8 value[0..listcount - 1]`	The results values.
Return Code:	
`FE_NO_MEMORY`	Insufficient memory to allocate space for values.
`FE_INVALID`	Object has not yet been populated. This method is only available after results have been loaded.
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetScalarAtElemColumnV2

Unpublished work. © 2025 Siemens

SetScalarAtElemColumnV2	
(nColumnIndex, listcount, nID, value)	
Description:	
This method stores output data in a column.	
Input:	
`INT4 nColumnIndex`	The index of the column.
`INT4 listcount`	The count of nodes in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the Node/Element.
`REAL8 value[0..listcount - 1]`	The results values.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

GetVectorAtNodeColumnsV2

GetVectorAtNodeColumnsV2
(nColumnIndices, listcount, nID, total, x, y, z)
Description:
This method returns the entity IDs and the components of the results stored from all rows in the selected column.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nColumnIndices[0..numColumns]`	The indices of the columns.
Output:	
`INT4 listcount`	The count of nodes in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the Node/Element.
`REAL8 total[0..listcount - 1]`	The total results values.
`REAL8 x[0..listcount - 1]`	The X component results values.
`REAL8 y[0..listcount - 1]`	The Y component results values.
`REAL8 z[0..listcount - 1]`	The Z component results values.
Return Code:	
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetVectorAtColumnsV2

`SetVectorAtColumnsV2`
`( nColumnIndices, listcount, nID, x, y, z )`
Description:
This method sets nodal vector data in a column.
Input:

`INT4 nColumnIndices[0..numColumns]`	The indices of the columns to set.

Unpublished work. © 2025 Siemens

`INT4 listcount`	The count of nodes in the columns
`INT4 nID[0..listcount -1]`	The IDs of the Nodes.
`REAL8 x[0..listcount - 1]`	The X component results values.
`REAL8 y[0..listcount - 1]`	The Y component results values.
`REAL8 z[0..listcount - 1]`	The Z component results values.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

GetElemWithCornerColumnsV2

`GetElemWithCornerColumnsV2`	
`( nColumnIndices, listcount, maxcorner, nID, centroid, c1, c2, c3, c4, c4, c5, c6, c7, c8 )`	
Description:	
This method gets centroid and corner results of elements from given columns.	
Input:	
`INT4 nColumnIndices[0..numColumns]`	The indices of the columns to set.
Output:	
`INT4 listcount`	The count of elements in the column.

Unpublished work. © 2025 Siemens

`INT4 maxcorner`	The ID of the maximum corner available for this object. For example, if this object contains results on quadrilateral plates, `maxcorner` will typically equal 4, since results are only available for corners 1,2,3, and 4.
`INT4 nID[0..listcount - 1]`	The IDs of the elements.
`REAL8 centroid`	The results at the element centroid associated with the corresponding node/element in the ID array.
`REAL8 c1[0..listcount - 1]`	The results at the element corners associated with the corresponding node/element in the ID array. Some elements have missing corners. For example, Tetrahedral results are stored on corners 1,2,3 and 5. Wedge results are stored on 1,2,3 and 5,6,7.
`REAL8 c2[0..listcount - 1]`	
`REAL8 c3[0..listcount - 1]`	
`REAL8 c4[0..listcount - 1]`	
`REAL8 c5[0..listcount - 1]`	
`REAL8 c6[0..listcount - 1]`	
`REAL8 c7[0..listcount - 1]`	
`REAL8 c8[0..listcount - 1]`	
Return Code:	
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

SetElemWithCornerColumnsV2

Unpublished work. © 2025 Siemens

SetElemWithCornerColumnsV2

(nColumnIndices, listcount, maxcorner, nID, centroid, c1, c2, c3, c4, c4, c5, c6, c7, c8)

Description:

This method sets centroid and corner results of elements of given columns.

Input:

`INT4 nColumnIndices[0..numColumns]`	The indices of the columns to set.
`INT4 listcount`	The count of elements in the column.
`INT4 maxcorner`	The ID of the maximum corner available for this object. For example, if this object contains results on quadrilateral plates, `maxcorner` will typically equal 4, since results are only available for corners 1,2,3, and 4.
`INT4 nID[0..listcount - 1]`	The IDs of the elements.
`REAL8 centroid`	The results at the element centroid associated with the corresponding node/element in the ID array.
`REAL8 c1[0..listcount - 1]`	The results at the element corners associated with the corresponding node/element in the ID array. Some elements have missing corners. For example, Tetrahedral results are stored on corners 1,2,3 and 5. Wedge results are stored on 1,2,3 and 5,6,7.
`REAL8 c2[0..listcount - 1]`	
`REAL8 c3[0..listcount - 1]`	
`REAL8 c4[0..listcount - 1]`	
`REAL8 c5[0..listcount - 1]`	
`REAL8 c6[0..listcount - 1]`	
`REAL8 c7[0..listcount - 1]`	
`REAL8 c8[0..listcount - 1]`	

Output:

None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

GetScalarAtBeamColumnsV2

`GetScalarAtBeamColumnsV2` **`( nColumnIndices, listcount, nID, ValA, ValB )`**	
Description:	
This method gets Beam output vectors for EndA and EndB containing a single scalar value.	
Input:	
`INT4 nColumnIndices[0..numColumns]`	The indices of the columns to get.
Output:	
`INT4 listcount`	The count of elements in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the elements.
`REAL8 ValA`	The EndA output values associated with the corresponding element IDs array.
`REAL8 ValB`	The EndB output values associated with the corresponding element IDs array.
Return Code:	
`FE_FAIL`	Unable to get data for the specified row/columns.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetScalarAtBeamColumnsV2

SetScalarAtBeamColumnsV2 **(nColumnIndices, listcount, nID, ValA, ValB)**	
Description:	
This method sets Beam output vectors for EndA and EndB containing a single scalar value.	
Input:	
`INT4 nColumnIndices[0..numColumns]`	The indices of the columns to get.
`INT4 listcount`	The count of elements in the column.
`INT4 nID[0..listcount - 1]`	The IDs of the elements.
`REAL8 ValA`	The EndA output values associated with the corresponding element IDs array.
`REAL8 ValB`	The EndB output values associated with the corresponding element IDs array.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to set data for the specified row/columns.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Sort

Sort **(nColumnIndex, bAscending, bAbsoluteValue)**	
Description:	
This method sorts the data of a specified column.	
Input:	
`INT4 nColumnIndex`	The index of the column to sort.
`BOOL bAscending`	If True the values will be listed in ascending order.
`BOOL bAbsoluteValue`	If True the values will be sorted by the absolute value.
Output:	
None	
Return Code:	
`FE_FAIL`	Unable to sort data.
Remarks/Usage:	
This method should be used before calling `Populate`.	
Example:	
None	

GetScalarAtNodeSetV2

GetScalarAtNodeSetV2 **(nColumnIndex, nNodeSetID, value)**
Description:
This method gets scalar data of a given set of nodes.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nColumnIndex`	The index of the column to get.
`INT4 nNodeSetID`	The ID of the set of nodes.
Output:	
`REAL8 value[0..N]`	The nodal results value of the nodes in the set.
Return Code:	
`FE_FAIL`	Unable to get data.
Remarks/Usage:	
None	
Example:	
None	

GetScalarAtElemSetV2

`GetScalarAtElemSetV2`
`( nColumnIndex, nElemSetID, value )`
Description:
This method gets scalar data of a given set of elements.
Input:

`INT4 nColumnIndex`	The index of the column to get.
`INT4 nElemSetID`	The ID of the set of elements.
Output:	
`REAL8 value[0..N]`	The elemental results value of the nodes in the set.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to get data.
Remarks/Usage:	
None	
Example:	
None	

GetVectorAtNodeSetV2

`GetVectorAtNodeSetV2` **`( nColumnIndices, nNodeSetID, total, x, y, z )`**	
Description:	
This method gets vector data of a given set of nodes.	
Input:	
`INT4 nColumnIndices[0..N]`	The indices of the columns to get.
`INT4 nNodeSetID`	The ID of the set of nodes.
Output:	
`REAL8 total[0..N]`	The total magnitude values associated with the corresponding element in the Set.
`REAL8 x[0..N]`	The X component values associated with the corresponding element in the Set.
`REAL8 y[0..N]`	The Y component values associated with the corresponding element in the Set.
`REAL8 z[0..N]`	The Z component values associated with the corresponding element in the Set.
Return Code:	
`FE_FAIL`	Unable to get data.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetElemWithCornerSetV2

GetElemWithCornerSetV2 **(nColumnIndices, nElemSetID, maxcorner, centroid, c1, c2, c3, c4, c5, c5, c6, c7, c8)**	
Description:	
This method retrieves the centroid and corner results stored in an object that contains elemental data with corners. Results are retrieved for all elements specified in the Set.	
Input:	
`INT4 nColumnIndices[0..N]`	The indices of the columns to get.
`INT4 nElemSetID`	The ID of the set of elements.
Output:	
`INT4 maxcorner`	The ID of the maximum corner available for this object. For example, if this object contains results on quadrilateral plates, maxcorner will typically equal 4, since results are only available for corners 1,2,3, and 4.
`REAL8 c1[0..N]`	The results at the element corners associated with the corresponding node/element in the Set. Some elements have missing corners. For example, Tetrahedral results are stored on corners 1,2,3 and 5. Wedge results are stored on 1,2,3 and 5,6,7.
`REAL8 c2[0..N]`	
`REAL8 c3[0..N]`	
`REAL8 c4[0..N]`	
`REAL8 c5[0..N]`	
`REAL8 c6[0..N]`	

Unpublished work. © 2025 Siemens

`REAL8 c7[0..N]`	
`REAL8 c8[0..N]`	
Return Code:	
`FE_FAIL`	Unable to get data.
Remarks/Usage:	
None	
Example:	
None	

GetScalarAtBeamSetV2

`GetScalarAtBeamSetV2` **`( nColumnIndices, nElemSetID, ValA, ValB )`**	
Description:	
This method gets scalar data of a given set of beam elements.	
Input:	
`INT4 nColumnIndices[0..N]`	The indices of the columns to get.
`INT4 nElemSetID`	The ID of the set of elements.
Output:	
`REAL8 ValA[0..N]`	The EndA values associated with the corresponding element in the Set.
`REAL8 ValB[0..N]`	The EndB values associated with the corresponding element in the Set.
Return Code:	
`FE_FAIL`	Unable to get data.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

MapData Objects

The `MapData` object is essentially a random access array that allows you to store items based on a ID/value pair and quickly retrieve them later. Rather than having fixed values of a certain type, the Femap `MapData` object can hold one or more of several different data types, including Booleans, Strings, Integers, Reals and variants. Each piece of data stored in a value is referred to as a field. Conceptually, data within the `MapData` object can be visualized as a table where the ID is the first column and each field is column 2-n (all columns represent the corresponding value in an ID/value pair, regardless of how many fields are associated with a single ID). IDs do not need to be sequential, nor do they have to be in order (random is okay), however they must be unique for each row.

To use the `MapData` object, first `Initialize()` the object with an approximate guess at how many items (ID/value pairs) will be stored in the object. Next, use the `Define*()` methods to add fields to be associated with each ID added to the `MapData` object. Each call to a `Define*()` method, if successful, will return a *column index* for that field of data. Populate the `MapData` object using the various `Set*()` methods. Finally, retrieve values from the `MapData` object using the `Get*()` methods. Each `Set*()` and `Get*()` method requires an ID and an index where the ID is the unique identifier for each ID/value pair, and the index is the order of the field of interest, obtained from the `Define*()` methods. If you call `Set*()` method with an ID that hasn't been specified yet, it simply adds it to the `MapData` object.

Note that when using the variant data type to hold arrays of data, the `MapData` object requires an array of determinate length. As such, a variant array must be properly dimensioned prior to passing it to the `DefineVariant()` method.

Use the `feMapData` method of the Femap application object to create `MapData` objects.

MapData Object Methods

Clear

Unpublished work. © 2025 Siemens

Clear	
( )	
Description:	
Clears everything in the `MapData` object, including all data definitions, which then must be set again.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
To clear all of the data, but retain the data definitions, use the `RemoveAll` method instead.	
Example:	
None	

Initialize

Initialize	
(nCount)	
Description:	
Sets approximate number of IDs to be added to `MapData` object	
Input:	
`INT4 nCount`	Approximate number of IDs to be added to `MapData` object.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

None	
Remarks/Usage:	
The count supplied is only an estimate to the number of entries the `MapData` object will hold. If more or fewer entries planned to be stored, there is no need to call this method again as the `MapData` will automatically resize itself as appropriate, however a more accurate count will result in more efficient memory utilization and better performance.	
Example:	
None	

Count

Count	
( )	
Description:	
Number of IDs in the `MapData` object.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetIDs

Unpublished work. © 2025 Siemens

GetIDs	
(nIDSet)	
Description:	
Fills Set Object with IDs from `MapData` object.	
Input:	
None	
Output:	
`nIDSet`	ID of Set object to fill with IDs from `MapData` object.
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Remove

Remove	
(nID)	
Description:	
Deletes a selected ID and associated data from the `MapData` object.	
Input:	
`INT4 nID`	ID to delete from `MapData` object.
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

None	
Remarks/Usage:	
None	
Example:	
None	

RemoveAll

`RemoveAll`
`()`
Description:
Deletes a all data associated with all IDs from the `MapData` object, but the field definitions remain.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
None
Example:
None

DefineReal

Unpublished work. © 2025 Siemens

DefineReal	
()	
Description:	
Defines a double field in `MapData` object and returns a number from 0..n that represents the index ID of that piece of data.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
If the data cannot be successfully added to the `MapData` object, -1 is returned.	
Example:	
None	

DefineInt

DefineInt	
()	
Description:	
Defines an integer field in `MapData` object and returns a number from 0..n that represents the index ID of that piece of data.	
Input:	
None	
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
None	
Remarks/Usage:	
If the data cannot be successfully added to the `MapData` object, -1 is returned.	
Example:	
None	

DefineBool

`DefineBool`
`()`
Description:
Defines an Boolean field in `MapData` object and returns a number from 0..n that represents the index ID of that piece of data.
Input:
None
Output:
None
Return Code:
None
Remarks/Usage:
If the data cannot be successfully added to the `MapData` object, -1 is returned.
Example:
None

DefineString

Unpublished work. © 2025 Siemens

DefineString	
(nLength)	
Description:	
Defines an String field of a specified maximum length in `MapData` object and returns a number from 0..n that represents the index ID of that piece of data.	
Input:	
`INT4 nLength`	Maximum length of String.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

DefineVariant

DefineVariant	
(vVar)	
Description:	
Defines an String field of a specified length in `MapData` object and returns a number from 0..n that represents the index ID of that piece of data.	
Input:	
`VARIANT vVar`	Name of the variant.
Output:	

Unpublished work. © 2025 Siemens

None	
Return Code:	
None	
Remarks/Usage:	
The size and data type of the Variant my be defined using the Array call in the API Programming Window.	
Example:	

The following shows how an array can be created to hold Real values, in this case XYZ coordinates, then how those values are set using `SetVariant` in the `MapData` object:

```
Dim fmap As femap.MapData
Set fmap = App.feMapData

Dim fsn As femap.Set
Set fsn = App.feSet

Dim nField1 As Long

Dim nNumNode As Long
Dim vNdID As Variant
Dim vNdXYZ As Variant

Dim vXyz As Variant

Dim i As Long

fsn.AddAll( FT_NODE )

fmap.Initialize( fsn.Count )
vXyz = Array( 0#, 0#, 0# )
```


Unpublished work. © 2025 Siemens

```
nField1 = fmap.DefineVariant( vXyz )


fn.GetCoordArray( fsn.ID, nNumNode, vNdID, vNdXYZ )


For i = 0 To nNumNode - 1 Step 1
vXyz = Array(vNdXYZ(i*3 + 0), vNdXYZ(i*3 + 1), vNdXYZ(i*3 + 2))
fmap.SetVariant( nField1, vNdID( i ), vXyz )
Next
```

SetReal

SetReal **(nIndex, nID, dVal)**	
Description:	
Sets Real value in field defined using `DefineReal` method for `DataMap` object for given index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for real field.
`INT4 nID`	ID value for real field.
`REAL8 dVal`	Real value for real field.
Output:	
None	
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to set Field 4, but only Fields 1-3 are defined).

Unpublished work. © 2025 Siemens

`FE_BAD_TYPE`	Field defined as one data type, but trying to set with a different data type (i.e., tried to set a real value in a field defined for an integer).
Remarks/Usage:	
None	
Example:	
None	

SetInt

`SetInt` **`( nIndex, nID, nVal )`**	
Description:	
Sets Integer value for a field defined using DefineInt method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for integer field.
`INT4 nID`	ID value for integer field.
`INT4 nVal`	Integer value for integer field.
Output:	
None	
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to set Field 4, but only Fields 1-3 are defined).
`FE_BAD_TYPE`	Field defined as one data type, but trying to set with a different data type (i.e., tried to set a real value in a field defined for an integer).
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetBool

SetBool **(nIndex, nID, bVal)**	
Description:	
Sets Boolean value for a field defined using `DefineBool` method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for Boolean field.
`INT4 nID`	ID value for Boolean field.
`BOOL bVal`	Boolean value for Boolean field.
Output:	
None	
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to set Field 4, but only Fields 1-3 are defined).
`FE_BAD_TYPE`	Field defined as one data type, but trying to set with a different data type (i.e., tried to set a real value in a field defined for an integer).
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetString

SetString **(nIndex, nID, sVal)**	
Description:	
Sets String value for a field defined using `DefineString` method for `DataMap` object for given Index-ID data pair	
Input:	
`INT4 nIndex`	Index value for string field.
`INT4 nID`	ID value for string field.
`STRING sVal`	String value for string field.
Output:	
None	
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to set Field 4, but only Fields 1-3 are defined).
`FE_BAD_TYPE`	Field defined as one data type, but trying to set with a different data type (i.e., tried to set a real value in a field defined for an integer).
Remarks/Usage:	
None	
Example:	
None	

SetVariant

Unpublished work. © 2025 Siemens

SetVariant **(nIndex, nID, vVal)**	
Description:	
Sets Variant value for a field defined using `DefineVariant` method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for variant field.
`INT4 nID`	ID value for variant field.
`VARIANT vVal`	Name of variant for variant field.
Output:	
None	
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to set Field 4, but only Fields 1-3 are defined).
`FE_BAD_TYPE`	Field defined as one data type, but trying to set with a different data type (i.e., tried to set a real value in a field defined for an integer).
Remarks/Usage:	
The data type used here needs to be the exact same data type used with the `DefineVariant()` method.	
Example:	
See DefineVariant method for example of using the `SetVariant` method.	

GetReal

Unpublished work. © 2025 Siemens

GetReal **(nIndex, nID, dVal)**	
Description:	
Gets Real value from field filled by SetReal method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for real field.
`INT4 nID`	ID value for real field.
Output:	
`REAL8 dVal`	Real value from real field.
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to get Field 4, but only Fields 1-3 are defined and set).
`FE_BAD_TYPE`	Field defined as one data type, but trying to get a different data type (i.e., tried to get a real value from a field defined and set for an integer).
`FE_NOT_EXIST`	Specified ID does currently does not exist in `DataMap` object.
`FE_TOO_SMALL`	Field exists for some IDs, but not for specified ID.
Remarks/Usage:	
None	
Example:	
None	

GetInt

Unpublished work. © 2025 Siemens

GetInt **(nIndex, nID, nVal)**	
Description:	
Gets Integer value from field filled by `SetInt` method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for integer field.
`INT4 nID`	ID value for integer field.
Output:	
`INT4 nVal`	Integer value from integer field.
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to get Field 4, but only Fields 1-3 are defined and set).
`FE_BAD_TYPE`	Field defined as one data type, but trying to get a different data type (i.e., tried to get a real value from a field defined and set for an integer).
`FE_NOT_EXIST`	Specified ID does currently does not exist in `DataMap` object.
`FE_TOO_SMALL`	Field exists for some IDs, but not for specified ID.
Remarks/Usage:	
None	
Example:	
None	

GetBool

Unpublished work. © 2025 Siemens

GetBool **(nIndex, nID, bVal)**	
Description:	
Gets Boolean value from field filled by `SetBool` method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for Boolean field.
`INT4 nID`	ID value for Boolean field.
Output:	
`BOOL bVal`	Boolean value from Boolean field.
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to get Field 4, but only Fields 1-3 are defined and set).
`FE_BAD_TYPE`	Field defined as one data type, but trying to get a different data type (i.e., tried to get a real value from a field defined and set for an integer).
`FE_NOT_EXIST`	Specified ID does currently does not exist in `DataMap` object.
`FE_TOO_SMALL`	Field exists for some IDs, but not for specified ID.
Remarks/Usage:	
None	
Example:	
None	

GetString

Unpublished work. © 2025 Siemens

GetString **(nIndex, nID, sVal)**	
Description:	
Gets String value from field filled by `SetString` method for `DataMap` object for given Index-ID data pair.	
Input:	
`INT4 nIndex`	Index value for string field.
`INT4 nID`	ID value for string field.
Output:	
`STRING sVal`	String value from string field.
Return Code:	
`FE_INVALID`	Field does not currently exist (i.e., tried to get Field 4, but only Fields 1-3 are defined and set).
`FE_BAD_TYPE`	Field defined as one data type, but trying to get a different data type (i.e., tried to get a real value from a field defined and set for an integer).
`FE_NOT_EXIST`	Specified ID does currently does not exist in `DataMap` object.
`FE_TOO_SMALL`	Field exists for some IDs, but not for specified ID.
Remarks/Usage:	
None	
Example:	
None	

GetVariant

Unpublished work. © 2025 Siemens

GetVariant

(nIndex, nID, vVal)

Description:

Gets Variant from field filled by `SetVarient` method for `DataMap` object for given Index-ID data pair.

Input:

`INT4 nIndex`	Index value for variant field.
`INT4 nID`	ID value for variant field.

Output:

`VARIANT vVal`	Name of variant for variant field.

Return Code:

`FE_INVALID`	Field does not currently exist (i.e., tried to get Field 4, but only Fields 1-3 are defined and set).
`FE_BAD_TYPE`	Field defined as one data type, but trying to get a different data type (i.e., tried to get a real value from a field defined and set as an integer).
`FE_NOT_EXIST`	Specified ID does currently does not exist in `DataMap` object.
`FE_TOO_SMALL`	Field exists for some IDs, but not for specified ID.

Remarks/Usage:

The data type used here needs to be the exact same data type used with the `DefineVariant()` method.

Example:

See `DefineVariant` method for examples of using the `DefineVariant` and `SetVariant` method. This code would be used along with that code from that example to retrieve the information from the Variant DataMap field to list the XYZ coordinates for all nodes in the model.

```
While fsn.Next > 0

   nID = fsn.CurrentID

     App.feAppMessage( FCM_COMMAND, "Node " + Str$( nID ))
```


Unpublished work. © 2025 Siemens

```
   fmap.GetVariant( nField1, nID, vXyz )


   App.feAppMessage(FCM_NORMAL, "X: "+Format$(vXyz(0),"0.000")+ _
   ", Y: "+Format$(vXyz(1),"0.000")+", Z:"+Format$(vXyz(2),"0.000"))


Wend
```

Entity Tracking Objects

The `Entity Tracking` object is used to determine which entities have been created and/or deleted during a particular portion of an API program. Use the `feTrackData` method of the Femap application object to create `TrackData` objects.

To use the `TrackData` object, simply use the `Start` method anywhere in the API program to begin tracking a particular entity type, then use the `Stop` method to stop tracking a particular entity type later in the API program. Entity types which can be tracked individually are points, curves, surfaces, volumes, solids, coordinate systems, nodes, elements, materials, properties, layups, global plies, load sets, constraint sets, functions, data surfaces, analysis sets, output sets, regions, connection properties, connectors, aero panel/bodies, aero properties, aero splines, aero control surfaces, text entities, views, groups, layers, and design optimization entities.

In addition to the `Start` method, the `StartGeometry` method can be used to begin entity tracking of geometric entities (points, curves, surfaces, and solids) and the `StartMesh` method can be used to begin entity tracking of a subset of finite element entities (nodes, elements, materials, and properties). When using one of these methods, or tracking multiple entity types, it may be beneficial to use the `StopAll` method, which will stop entity tracking for all entity types.

At any time during the tracking period or after entity tracking has been completed with the `Stop` or `StopAll` method, use the `Created` method to fill a `Set` object with the entities of a specific type created during the tracking period and/or use the `Deleted` method to fill a `Set` object with the entities of a specific type deleted during the tracking period. Using the `Created` or `Deleted` method will also stop entity tracking of the specified entity type, so the `Start` method must be called again in order to begin entity tracking of the entity type again.

Both the `Created` and `Deleted` methods have an option to *clear* the `Set` object before filling it with the entities of the specified type that have been created or deleted. If the `Set` object is not cleared with these methods, then entity tracking can effectively be done using several different tracking periods. If the `Set` object is cleared, then entity tracking of the specified entity type will only be tracked from the previous call of one of the `Start` methods. Because of this, it is recommended to use unique `Set` objects for tracking purposes (i.e., one `Set` object to track created entities of a specific entity type and a different `Set` object to track deleted entities).

Unpublished work. © 2025 Siemens

The same tracking object can be used multiple times during the execution of an API program. To remove everything from a `TrackData` object and start over at any time, simply use the `Clear` method.

TrackData Object Methods

Clear

`Clear`	
`( )`	
Description:	
Clears everything in the `TrackData` object.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Start

Unpublished work. © 2025 Siemens

Start **(entityType)**	
Description:	
Begins entity tracking for the specified entity type.	
Input:	
`INT4 entityType`	Type of entity to begin tracking. 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 9=Coordinate System, 10=Material, 11=Property, 12=Load Set, 17=Constraint Set, 21=Text, 22=View, 24=Group, 28=Output Set, 32=Boundary Surface, 33=Layer, 35=Function, 39=Solid, 45=Design Optimization, 58=Region, 60=Analysis Set, 69=Data Surface, 71=Connector, 72=Connection Property, 81=Layup, 83=Global Ply, 95=Aero Panel/Body, 96=Aero Property, 98=Aero Spline, 99=Aero Control Surface. For more information, see **Entity Types**.
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	Entity type is not supported by entity tracking.
Remarks/Usage:	
None	
Example:	
None	

StartGeometry

Unpublished work. © 2025 Siemens

StartGeometry	
( )	
Description:	
Begins entity tracking of Point, Curve, Surface, Boundary Surface, and Solid entity types.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

StartMesh

StartMesh	
( )	
Description:	
Begins entity tracking for Node, Element, Material, and Property entity types.	
Input:	
None	
Output:	
None	
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Stop

Stop **(entityType)**	
Description:	
Ends entity tracking for the specified entity type.	
Input:	
`INT4 entityType`	Type of entity to stop tracking. 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 9=Coordinate System, 10=Material, 11=Property, 12=Load Set, 17=Constraint Set, 21=Text, 22=View, 24=Group, 28=Output Set, 32=Boundary Surface, 33=Layer, 35=Function, 39=Solid, 45=Design Optimization, 58=Region, 60=Analysis Set, 69=Data Surface, 71=Connector, 72=Connection Property, 81=Layup, 83=Global Ply, 95=Aero Panel/Body, 96=Aero Property, 98=Aero Spline, 99=Aero Control Surface. For more information, see **Entity Types**.
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	Entity type is not supported by entity tracking.
`FE_NOT_AVAILABLE`	Entity type is not a type currently being tracked, thus tracking cannot be stopped.
`FE_INVALID`	Entity tracking of specified entity type has already been stopped, thus cannot be stopped again.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

StopAll

`StopAll`	
`()`	
Description:	
Ends entity tracking for all entity types.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

Created

`Created`
`( entityType, SetID, bClear )`
Description:

Unpublished work. © 2025 Siemens

Fills the specified Set object with the entities of the specified type which were created during the entity tracking period.	
Input:	
`INT4 entityType`	Type of entity created during tracking period. 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 9=Coordinate System, 10=Material, 11=Property, 12=Load Set, 17=Constraint Set, 21=Text, 22=View, 24=Group, 28=Output Set, 32=Boundary Surface, 33=Layer, 35=Function, 39=Solid, 45=Design Optimization, 58=Region, 60=Analysis Set, 69=Data Surface, 71=Connector, 72=Connection Property, 81=Layup, 83=Global Ply, 95=Aero Panel/Body, 96=Aero Property, 98=Aero Spline, 99=Aero Control Surface. For more information, see **Entity Types**.
`INT4 SetID`	ID of the Set object to fill with entities of the specified entity type created during the entity tracking period. It is recommended to use a unique Set object for tracking created entities of each entity type.
`BOOL bClear`	If True, removes all entities of the specified entity type created in the current tracking period from the Set object specified with `SetID`. If false, entities created during the previous tracking period(s) will remain in the Set object until cleared.
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	Entity type is not supported by entity tracking.
`FE_NOT_AVAILABLE`	Entity type is not a type currently being tracked, thus no information exists about entities of this type created during tracking period.
Remarks/Usage:	
Calling the `Created` method stops tracking of the specified entity type. To begin tracking this entity type again, the `Start`, `StartGeometry`, or `StartMesh` method must be called again.	
Example:	
None	

Unpublished work. © 2025 Siemens

Deleted

Deleted	
(entityType, SetID, bClear)	
Description:	
Fills the specified Set object with the entities of the specified type which were deleted during the entity tracking period.	
Input:	
`INT4 entityType`	Type of entity deleted during tracking period. 3=Point, 4=Curve, 5=Surface, 6=Volume, 7=Node, 8=Element, 9=Coordinate System, 10=Material, 11=Property, 12=Load Set, 17=Constraint Set, 21=Text, 22=View, 24=Group, 28=Output Set, 32=Boundary Surface, 33=Layer, 35=Function, 39=Solid, 45=Design Optimization, 58=Region, 60=Analysis Set, 69=Data Surface, 71=Connector, 72=Connection Property, 81=Layup, 83=Global Ply, 95=Aero Panel/Body, 96=Aero Property, 98=Aero Spline, 99=Aero Control Surface. For more information, see **Entity Types**.
`INT4 SetID`	ID of the Set object to fill with entities of the specified entity type deleted during the entity tracking period. It is recommended to use a unique Set object for tracking deleted entities of each entity type.
`BOOL bClear`	If True, removes all entities of the specified entity type deleted in the current tracking period from the Set object specified with `SetID`. If false, entities deleted during the previous tracking period(s) will remain in the Set object until cleared.
None	
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	Entity type is not supported by entity tracking.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	Entity type is not a type currently being tracked, thus no information exists about entities of this type created during tracking period.
Remarks/Usage:	
Calling the `Deleted` method stops tracking of the specified entity type. To begin tracking this entity type again, the `Start`, `StartGeometry`, or `StartMesh` method must be called again.	
Example:	
None	

Beam Calculator Objects

The `Beam Calculator` object provides access to the Femap Beam Calculator. Results provided via this tool are the same as those found in **View → Advanced Post → Beam Cross Section** command.

Prior to recovering any data for a beam, at the minimum, the Element property must be set. The Element specified will be the element used for all subsequent calls to Beam Calculator Methods (such as `CalculateStress` or `GetElementProperties`) until the Element property is updated and the methods called again. This is generally true for all properties; however, it's not required that their values be set prior to calling any of the Beam Calculator methods. Consult the description for each property to see how changing it may affect the stress or beam property calculations.

Use the `feBeamCalculator` method of the Femap Application object to access Beam Calculator Objects.

Beam Calculator Object Properties

Property	Description
`INT4 Element`	Element ID to use with the beam calculator. Must be a valid beam element with a recoverable cross-section. Used by all Beam Calculator methods.
`REAL8 Position`	The position along the beam, between 0.0 and 1.0 to calculate stresses. Default = 0.0. If not set, `CalculateStress` and `CalculateStressFromForces` will use the default value.

Unpublished work. © 2025 Siemens

Property	Description
	Used by all Beam Calculator methods except `FindMaxMinStress`.
`INT4 MeshFactor`	Setting determines the mesh density used in calculating stresses. Value from 1 (default, coarse) to 5 (fine). Used by all Beam Calculator Methods.
`BOOL IncludeAxialForce`	Consider axial forces when calculating internal stresses. Default True. Used by `CalculateStress` and `FindMaxMinStress` methods.
`BOOL IncludeShearForceY`	Consider shear forces in the element y-direction. Default True. Used by `CalculateStress` and `FindMaxMinStress` methods.
`BOOL IncludeShearForceZ`	Consider shear forces in the element z-direction. Default True. Used by `CalculateStress` and `FindMaxMinStress` methods.
`BOOL IncludeMomentY`	Consider moment about the element y-axis. Default True. Used by `CalculateStress` and `FindMaxMinStress` methods.
`BOOL IncludeMomentZ`	Consider moment about the element z-axis. Default True. Used by `CalculateStress` and `FindMaxMinStress` methods.
`BOOL IncludeTorque`	Consider torque. Default True. Used by `CalculateStress` and `FindMaxMinStress` methods.

Beam Calculator Object Methods

Unpublished work. © 2025 Siemens

GetElementProperties

GetElementProperties **(pdArea, pdIzz, pdIyy, pdIyz, pdJ, pdShearAreaY, pdShearAreaZ, pdWarp, pdNeutralAxisOffsetY, pdNeutralAxisOffsetZ**	
Description:	
Returns beam properties calculated from mesh at the location along the beam specified for the element specified.	
Input:	
None	
Output:	
`REAL8 pdArea`	Area
`REAL8 pdIzz`	Moment of Inertia, Izz
`REAL8 pdIyy`	Moment of Inertia, Iyy
`REAL8 pdIyz`	Moment of Inertia, Iyz
`REAL8 pdJ`	Torsional constant
`REAL8 pdShearAreaY`	Y Shear Area
`REAL8 pdShearAreaZ`	Z Shear Area
`REAL8 pdWarp`	Warping Constant
`REAL8 pdNeutralAxisOffsetY`	Y neutral axis offset
`REAL8 pdNeutralAxisOffsetZ`	Z neutral axis offset
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not retrieve element properties. Either the element was not specified or an invalid element was specified.
Remarks/Usage:	
None	
Example:	
None	

GetMeshInfo

`GetMeshInfo`	
`(pnNumNode, pvCoord, pnNumElem, pvNodes)`	
Description:	
Returns node location element connectivity information about the associated mesh.	
Input:	
None	
Output:	
`INT4 pnNumNode`	Number of nodes.
`REAL8 pvCoord[0..((pnNumNode)*2 - 1)]`	Y and Z coordinates of the each node. First value is the Y coordinate, second value is the Z coordinate. Repeats for each node.
`INT4 pnNumElem`	Number of elements.
`INT4 pvNodes[0..((pnNumElem)*9 - 1)]`	Node indices for 9-noded quad (node 9 is located at the element centroid).
Return Code:	
`FE_FAIL`	Unable to retrieve mesh information. Invalid element likely specified.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

CalculateStress

`CalculateStress`

`(nOutputSetID, enComponent, pnNumNodes, pvCoordinates, pvStresses, pvMaxStress, pvMaxStressInfo, pvMinStress, pvMinStressInfo)`

Description:

Calculates stresses based on forces in one or more specified output sets. Use `Include*` properties to specify which force components are considered in the calculations.

> **Note**
>
> Unless the `Position` property is set, the default position along the beam (End A) is used for all calculations. To recover stresses at different positions along the beam, the `Position` property must be set for each position of interest prior to each corresponding `CalculateStress`.
>
> The `FindMaxMinStress` method automatically will perform calculations for stresses at End A, the beam midpoint, and End B, and return the maximum and minimum stresses.

Input:

`INT4 nOutputSetID`	Output set(s) to use when calculating internal stresses. Use a negative number to specify a single output set ID, otherwise a positive number specifies the Femap Set object ID containing multiple Output Set IDs.
`enum enComponent`	Stress component to return. Use `zBeamCalculatorStressComponent`.

Output:

`INT4 pnNumNodes`	Number of Nodes
`REAL8 pvCoordinates[(0..pnNumNodes)*2 - 1]`	Y and Z coordinate for each node.

Unpublished work. © 2025 Siemens

`REAL8 pvStresses[nOS*nComp*(pnNumNodes) - 1]`	Stresses at each node. Array is organized by: Output sets Stress components Stress `nOS` = number of output sets requested. `nComp` = number of stress components requested. Normally 1, unless `FBMC_SC_ALL` is specified, in which case 8.
`REAL8 pvMaxStress[0..(0 or 7)]`	Maximum stress. If only a single stress component is specified, an array of length 1 is returned containing the maximum stress for the requested stress component. If all stress components are specified, the array is 8 items long and each index corresponds to the values from the `zBeamCalculatorStressComponent` enumeration.
`INT4 pvMaxStressInfo[0..(0 or 7)]`	The values in this array vary based upon the requested data. If a single stress component is specified, the array will be of length 1, otherwise the array will be of length 8. If a single output set is specified the value(s) in the array will be the node index where the maximum stress was found. If multiple output sets are specified the value(s) in the array will be the output set corresponding to the max stress.
`REAL8 pvMinStress[0..(0 or 7)]`	Minimum stress. If only a single stress component is specified, an array of length 1 is returned containing the minimum stress for the requested stress component. If all stress components are specified, the array is 8 items long and each index corresponds to the values from the `zBeamCalculatorStressComponent` enumeration.
`INT4 pvMinStressInfo[0..(0 or 7)]`	The values in this array vary based upon the requested data. If a single stress component is specified, the array will be of length 1, otherwise the array will be of length 8.

Unpublished work. © 2025 Siemens

	If a single output set is specified the value(s) in the array will be the node index where the minimum stress was found. If multiple output sets are specified the value(s) in the array will be the output set corresponding to the minimum stress.
Return Code:	
`FE_FAIL`	Unable to calculate beam stresses.
`FE_INVALID`	Current element is invalid - either it is not a beam element, the beam orientation could not be determined, or the element does not exist.
`FE_NOT_EXIST`	One or more of the specified Output Set IDs does not exist.
`FE_BAD_TYPE`	The specified stress component is invalid. Use `zBeamCalculatorStressComponent` enumeration.
`FE_BAD_DATA`	Invalid output set specified.
Remarks/Usage:	
None	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim fe As femap.Elem
   Dim fbc As femap.BeamCalculator
   Dim fsO As femap.Set
   Dim fsE As femap.Set
   Dim nNumNode As Long
   Dim ddCoord As Variant, ddStress As Variant
   Dim ddMax As Variant, ddMin As Variant
   Dim nnMaxI As Variant, nnMinI As Variant
   Set fe = App.feElem
```


Unpublished work. © 2025 Siemens

```
   Set fbc = App.feBeamCalculator

   Set fsO = App.feSet

   Set fsE = App.feSet

   fsO.AddAll( FT_OUT_CASE )

   fsE.AddRule( FET_L_BEAM, FGD_ELEM_BYTYPE )

   fbc.Position = 0.

   fbc.MeshFactor = 1

    ' Loop through all beam elements, calculate stresses at End A

   fsE.Reset()

   While fsE.Next() <> FE_FAIL

       fbc.Element = fsE.CurrentID

        rc = fbc.CalculateStress( fsO.ID, FBMC_SC_ALL, nNumNode, ddCoord,
ddStress, ddMax, nnMaxI, ddMin, nnMinI )

   Wend

   fbc.Position = 1.

    ' Loop through all beam elements, calculate stresses at End B

   fsE.Reset()

   While fsE.Next() <> FE_FAIL

       fbc.Element = fsE.CurrentID

        rc = fbc.CalculateStress( fsO.ID, FBMC_SC_ALL, nNumNode, ddCoord,
ddStress, ddMax, nnMaxI, ddMin, nnMinI )

   Wend

End Sub
```

CalculateStressFromForces

Unpublished work. © 2025 Siemens

CalculateStressFromForces

(dForcesA, dForcesB, enComponent, pnNumNodes, pvCoordinates, pvStresses, pvMaxStress, pvMaxStressInfo, pvMinStress, pvMinStressInfo)

Description:

Calculates beam stresses using user-defined forces, rather than forces from an output set. `Include*` Beam Calculator properties are ignored when using this method.

Note

Unless the `Position` property is set, the default position along the beam (End A) is used for all calculations. To recover stresses at different positions along the beam, the `Position` property must be set for each position of interest prior to each corresponding `CalculateStress`.

The `FindMaxMinStress` method automatically will perform calculations for stresses at End A, the beam midpoint, and End B, and return the maximum and minimum stresses.

Input:

`REAL8 dForcesA[0..5]`	Forces at end A: 0 - Axial Force 1 - Shear Force Y 2 - Shear Force Z 3 - Moment Y 4 - Moment Z 5 - Torque
`REAL8 dForcesB[0..5]`	Forces at end B: 0 - Axial Force 1 - Shear Force Y 2 - Shear Force Z 3 - Moment Y 4 - Moment Z 5 - Torque
`enum enComponent`	Stress component to return. Use `zBeamCalculatorStressComponent`.

Output:

`INT4 pnNumNodes`	Number of Nodes.

Unpublished work. © 2025 Siemens

`REAL8 pvCoordinates[(0..pnNumNodes)*2 - 1]`	Y and Z coordinate for each node.
`REAL8 pvStresses[nOS*nComp*(pnNumNodes) - 1]`	Stresses at each node. Array is organized by: Output sets Stress components Stress `nOS` = number of output sets requested. `nComp` = number of stress components requested. Normally 1, unless `FBMC_SC_ALL` is specified, in which case 8.
`REAL8 pvMaxStress[0..(0 or 7)]`	Maximum stress. If only a single stress component is specified, an array of length 1 is returned containing the maximum stress for the requested stress component. If all stress components are specified, the array is 8 items long and each index corresponds to the values from the `zBeamCalculatorStressComponent` enumeration.
`INT4 pvMaxStressInfo[0..(0 or 7)]`	The values in this array vary based upon the requested data. If a single stress component is specified, the array will be of length 1, otherwise the array will be of length 8. If a single output set is specified the value(s) in the array will be the node index where the maximum stress was found. If multiple output sets are specified the value(s) in the array will be the output set corresponding to the max stress.
`REAL8 pvMinStress[0..(0 or 7)]`	Minimum stress. If only a single stress component is specified, an array of length 1 is returned containing the minimum stress for the requested stress component. If all stress components are specified, the array is 8 items long and each index corresponds to the values from the `zBeamCalculatorStressComponent` enumeration.
`INT4 pvMinStressInfo[0..(0 or 7)]`	The values in this array vary based upon the requested data

Unpublished work. © 2025 Siemens

	If a single stress component is specified, the array will be of length 1, otherwise the array will be of length 8. If a single output set is specified the value(s) in the array will be the node index where the minimum stress was found. If multiple output sets are specified the value(s) in the array will be the output set corresponding to the minimum stress.
Return Code:	
`FE_FAIL`	Unable to calculate beam stresses.
`FE_INVALID`	Current element is invalid - either it is not a beam element, the beam orientation could not be determined, or the element does not exist.
`FE_NOT_EXIST`	One or more of the specified Output Set IDs does not exist.
`FE_BAD_TYPE`	The specified stress component is invalid. Use `zBeamCalculatorStressComponent` enumeration.
`FE_BAD_TYPE`	Invalid data passed for End A or End B force arrays.
Remarks/Usage:	
None	
Example:	
None	

FindMaxMinStress

```
FindMaxMinStress

(nOutputSetID, enStressComponent, nMaxSetID, enMaxComponent,
dMaxLocation,
dMaxStress, nMinSetID, enMinComponent, dMinLocation, dMinStress)
```

Description:

Unpublished work. © 2025 Siemens

Returns maximum and minimum stress along with the associated stress component for an element across one or more output sets. Max/min stresses are taken from calculations performed at End A, End B and 50% along the length of the element.	
Input:	
`INT4 nOutputSetID`	Output Set ID. Positive value is the ID of a Set object containing one or more output sets. A negative value is the ID of a single Output Set.
`enum enComponent`	Stress component(s) to calculate. Use `zBeamCalculatorStressComponent`.
Output:	
`INT4 nMaxSetID`	Output set ID associated with maximum stress.
`enum enMaxComponent`	Stress component associated with maximum stress. Unless `FBMC_SC_ALL` is specified, this will be the same as the component specified with `enComponent`.
`REAL8 dMaxLocation`	Location along the beam where maximum stress is found. Currently, will return: 0.0 - End A 0.5 - Midpoint 1.0 - End B
`REAL8 dMaxStress`	Maximum stress.
`INT4 nMinSetID`	Output set ID associated with minimum stress.
`enum enMinComponent`	Stress component associated with minimum stress. Unless `FBMC_SC_ALL` is specified, this will be the same as the component specified with `enComponent`.
`REAL8 dMinLocation`	Location along the beam where minimum stress is found. Currently, will return: 0.0 - End A 0.5 - Midpoint 1.0 - End B
`REAL8 dMinStress`	Minimum stress.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to calculate beam stresses.
`FE_INVALID`	Current element is invalid - either it is not a beam element, the beam orientation could not be determined, or the element does not exist.
`FE_NOT_EXIST`	One or more of the specified Output Set IDs does not exist.
`FE_BAD_TYPE`	The specified stress component is invalid. Use `zBeamCalculatorStressComponent` enumeration.
`FE_BAD_DATA`	Invalid output set specified.
Remarks/Usage:	
None	
Example:	
None	

Draw/Erase Objects

The `Draw/Erase` object provides access to the functionality of the **Draw/Erase** Toolbar in your model. It is not a model entity.

Use the `feDrawErase` method of the Femap Application object to access Draw/Erase Objects.

Draw/Erase Object Properties

Property	Description
`BOOL Enable`	When True, Draw/Erase functionality is activated. When False, Draw/Erase functionality is deactivated.
`BOOL EraseMode`	When True, set to *Erase* mode. When False, set to *Draw* mode.
`BOOL AutoSelectMesh`	When True, Auto Select Mesh is activated. When False, option is deactivated.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL EraseGeometry`	When True and `EraseMesh` is set to False, selects Erase Geometry Only. If True and `EraseMesh` is set to True, selects Erase All Selected.
`BOOL EraseMesh`	When True and `EraseGeometry` is set to False, selects Erase Mesh Only. If True and `EraseGeometry` is set to True, selects Erase All Selected.
`BOOL ExclusiveDrawMode`	When True, Exclusive Draw Mode is activated. When False, option is deactivated.
`BOOL DrawModeSelectFromAll`	When True, Draw Mode Select From All is activated. When False, option is deactivated.

Draw/Erase Object Methods

ClearAll

`ClearAll`	
`( void )`	
Description:	
Removes all entities of all types from the Draw/Erase lists.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Clear

Clear (entityType)	
Description:	
Removes all entities of the selected type from the Draw/Erase lists.	
Input:	
`INT4 entityType`	The type of entities to be removed from the Draw/Erase lists. For entity information, see **Entity Types**.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

SaveGroup

SaveGroup (groupID, title)
Description:
Creates a new group of the visible entities and related entities.

Unpublished work. © 2025 Siemens

Input:	
`INT4 groupID`	ID of the group to create. Specifying a value of -1 will use the next available Group ID or a value of 0 to display the dialog box of the **Create Group** command on the **Draw/Erase** Toolbar.
`STRING title`	Title of the Group.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

LoadGroup

`LoadGroup` **`( groupID )`**	
Description:	
Loads a entities found in a specified group into the current *mode* of Draw/Erase Toolbar. When set to *Draw* mode, entities in the group will be drawn. When set to *Erase* mode entities in the group will be erased.	
Input:	
`INT4 groupID`	ID of the group to load into the `Draw/Erase` Toolbar.
Output:	
None	
Return Code:	
None	

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Grow

Grow **(void)**	
Description:	
Expands selection to include the entities connected to the currently selected entities.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The function does the same thing as using the **Grow** command on the **Draw/Erase** Toolbar.	
Example:	
None	

Shrink

Unpublished work. © 2025 Siemens

Shrink	
(void)	
Description:	
Reduces selection to no longer include the entities which are not fully surrounded by other entities of the same type.	
Input:	
None	
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The function does the same thing as using the **Shrink** command on the **Draw/Erase** Toolbar.	
Example:	
None	

HasErased

HasErased	
(void)	
Description:	
Allows you to determine if entities of any type are currently selected by the **Draw/Erase** Toolbar.	
Input:	
None	
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_OK`	One or more entities are selected by the **Draw/Erase** Toolbar.
`FE_FAIL`	No entities are currently selected by the **Draw/Erase** Toolbar.
Remarks/Usage:	
None	
Example:	
None	

HasErasedType

`HasErasedType`	
`( entityType )`	
Description:	
Allows you to determine if entities of the specified type are currently selected by the **Draw/Erase** Toolbar.	
Input:	
`INT4 entityType`	The type of entities to be removed from the Draw/Erase lists. For entity information, see **Entity Types**.
Output:	
None	
Return Code:	
`FE_OK`	One or more entities of the specified type are selected by the **Draw/Erase** Toolbar.
`FE_FAIL`	No entities of the specified type are currently selected by the **Draw/Erase** Toolbar.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

EraseSet

EraseSet **(entityType, nSetID, bAddToSelection, bClear, bRedraw)**	
Description:	
Loads a set object with entity IDs of a particular entity type into the **Draw/Erase** Toolbar. Has options to add the IDs to the *Erase* list or the *Draw* list and also allows you to clear the list before loading the IDs from the set object.	
Input:	
`INT4 entityType`	The type of entities to be removed from the Draw/Erase lists. For entity information, see **Entity Types**.
`INT4 nSetID`	ID of the set filled with entity IDs to load into the **Draw/Erase** Toolbar.
`BOOL bAddToSelection`	When True, adds entity IDs from set to the *Erase* list. When False, adds entity IDs from the set to the *Draw* list.
`BOOL bClear`	When True, clears the *Erase* list or *Draw* list before adding the entity IDs from the set. When False, adds the entity IDs to the *Erase* list or *Draw* list.
`BOOL bRedraw`	When True, a **Window** → **Redraw** command will be done. When False, it will not be done.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
None	
Example:	
None	

GetErased

Unpublished work. © 2025 Siemens

GetErased	
(entityType)	
Description:	
This method creates and returns a Set object that contains the IDs of all entities of a specified type that are currently selected by the **Draw/Erase** Toolbar.	
Input:	
`INT4 entityType`	The type of entities to be removed from the Draw/Erase lists. For entity information, see **Entity Types**.
Output:	
None	
Return Code:	
None	
Remarks/Usage:	
The Set object is returned in place of the return code.	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim de As femap.DrawErase
   Set de = App.feDrawErase
   Dim eset As femap.Set
   Set eset = de.GetErased(FT_ELEM)
End Sub
```

User Defined Graphics Objects

The User Defined Graphics object is used to create graphics data in the Femap graphics windows. You can create any number of User Defined Graphics objects and each can contain multiple collectors of graphics entities such as: points, lines, triangles, text, bitmaps and symbols. All graphics entities can be defined in terms of points in global space. A subset (points, bitmaps, text and symbols) can be defined in terms of

Unpublished work. © 2025 Siemens

locations on Femap entities (`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE` and `FT_ELEMENT`). Trying to add the wrong type of graphical entity to a collector will be ignored.

Graphics entities in a User Defined Graphics object will not be visible until the `Publish` method is called. If any graphical entities are added after calling the `Publish` method, all entities in the User Defined Graphics object will not be visible until the `Publish` method is called again. When you have finished with a collector of graphics data, a bitmap or a symbol, you do not delete them but mark them for deletion. This is to prevent the User Defined Graphics object changing/deleting graphics data while a draw is in progress.

Use the `feUserDefinedGraphics` method of the Femap Application object to create User Defined Graphics objects. When the User Defined Graphics object is destroyed or goes out of scope, the graphics associated will be deleted. No graphics are retained in the database using this object. Graphics data can survive the destruction of the User Defined Graphics object by using the Detach/Attach capability. When graphics data are detached from a User Defined Graphics object, using the `Detach` function, they will survive and be drawn as is until the model is closed and then they will be destroyed. Detached graphics can be reattached using the `Attach` function.

User Defined Graphics Object Methods

SetView

`SetView`	
`( iViewID )`	
Description:	
This function controls which windows the graphics in the User Defined Graphics object are drawn.	
Input:	
`INT4 iViewID`	Specifies the ID of the view(s). 0 = all views -1 = determines active view and sets that view n = nth view
Output:	
None	
Return Code:	

Unpublished work. © 2025 Siemens

`FE_OK`	Function worked.
`FE_FAIL`	Function failed.
Remarks/Usage:	
If `iViewID` is set to 0, then the graphics are drawn in all windows of the active Model. If `iViewID` is set to -1, the active view ID is used. If `iViewID` is set to “n”, then the graphics are drawn in the window that is using view “n”. If an invalid `iViewID` is specified, the graphics will not be drawn in any window. If you change the active view, the graphics will NOT move to the new active view.	
Example:	
None	

GetView

`GetView` **`( iViewID )`**	
Description:	
This function is used to query the view ID of the User Defined Graphics object.	
Input:	
None	
Output:	
`INT4 iViewID`	ID of view.
Return Code:	
`FE_OK`	Function worked.
`FE_FAIL`	Function failed.
Remarks/Usage:	
If you set the active view ID by calling `SetView(-1)`, this function will not provide -1 but will provide the ID of the active view ID when the `SetView(-1)` was called.	

Unpublished work. © 2025 Siemens

Example:
None

SetName

SetName **(pName)**	
Description:	
This function provides the User Defined Graphics object with a name. This is not currently implemented.	
Input:	
`STRING pName`	Name of User Defined Graphics Object.
Output:	
None	
Return Code:	
`FE_OK`	Function worked.
`FE_FAIL`	Function failed.
Remarks/Usage:	
None	
Example:	
None	

Publish

Unpublished work. © 2025 Siemens

Publish	
(void)	
Description:	
This function publishes the User Defined Graphics object graphics data and makes it available to the graphics system.	
Input:	
None	
Output:	
None	
Return Code:	
`FE_OK`	Function worked.
`FE_FAIL`	Function failed.
Remarks/Usage:	
Graphics entities in a User Defined Graphics object will not be visible until the `Publish` method is called. If any graphical entities are added after calling the `Publish` method, all entities in the User Defined Graphics object will not be visible until the `Publish` method is called again.	
Example:	
None	

Blank

Blank	
(qBlanked)	
Description:	
This function blanks the User Defined Graphics object so it will not be drawn until it is un-blanked.	
Input:	

Unpublished work. © 2025 Siemens

`BOOL qBlanked`	If True, the User Defined Graphics object is blanked. If False, the User Defined Graphics object is un-blanked.
Output:	
None	
Return Code:	
`FE_OK`	Function worked.
`FE_FAIL`	Function failed.
Remarks/Usage:	
There is also a `CollectorBlank` method for individual collectors of graphics entities inside the User Defined Graphics object.	
Example:	
None	

SetOpenGLTransformationSpace

`SetOpenGLTransformationSpace`
`( iTransSpace )`
Description:
This function controls which transformation space the graphics in the `UserDefinedGraphics` object are drawn.
Input:

`INT4 iTransSpace`	Specifies the ID of the transformation space. 0=unset 1=model space 2=screen space
Output:	
None	

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
This function should be called immediately after the `UserDefinedGraphics` object is created. If this function is not called, adding the first collector will force it to be model space unless the collector is a `CollectorContourLegendText` which will force it to be screen space. Calling this function a second time will return `FE_FAIL`.	
Example:	
None	

GetOpenGLTransformationSpace

`GetOpenGLTransformationSpace` **`( iTransSpace )`**	
Description:	
This function is used to query the transformation space.	
Input:	
`INT4 iTransSpace`	Specifies the ID of the transformation space. 0=unset 1=model space 2=screen space
Output:	
None	
Return Code:	
`FE_FAIL`	ID of the transformation space could not be retrieved.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

CollectorPoints

CollectorPoints **(pName, nAppearances, fDepthOffset, iCollectorID)**	
Description:	
This function creates a Point collector. Points can be added based on location in global space or relative to Femap entities such as FT_POINT, FT_CURVE, FT_SURFACE, FT_NODE and FT_ELEM.	
Input:	
STRING pName	Name of collector - currently not used.
INT4 nAppearances	Number of appearances that will be added to this collector (see CollectorAppearance).
REAL8 fDepthOffset	Depth offset - currently not used (set to 0.0).
Output:	
INT4 iCollectorID	The ID of the collector created.
Return Code:	
FE_OK	Function worked.
FE_FAIL	Function failed.
Remarks/Usage:	
Point entities added using the functions: CollectorAddPointLocations CollectorAddPointEntityLocations CollectorAddPointEntityFaceLocations	

Unpublished work. © 2025 Siemens

Example:
None

CollectorBitmap

CollectorBitmap **(pName, nAppearances, iBitmapID, fXoffset, fYOffset, fDepthOffset, iCollectorID)**	
Description:	
This function creates a Bitmap collector. Bitmap locations (points) can be added based on location in global space or relative to Femap entities such as FT_POINT, FT_CURVE, FT_SURFACE, FT_NODE and FT_ELEM.	
Input:	
STRING pName	Name of collector - currently not used.
INT4 nAppearances	Number of appearances that will be added to this collector (see CollectorAppearance).
INT4 iBimtapID	ID of bitmap to use (see BitmapCreate).
REAL8 fXOffset	X offset of location compared to bottom left of bitmap.
REAL8 fYOffset	Y offset of location compared to bottom left of bitmap.
REAL8 fDepthOffset	Depth offset - currently not used (set to 0.0).
Output:	
INT4 iCollectorID	The ID of the collector created.
Return Code:	
FE_OK	Function worked.
FE_FAIL	Function failed.

Unpublished work. © 2025 Siemens

Remarks/Usage:

A bitmap must be created using the following method:
 	
`BitmapCreate`
 	
Bitmap location entities added using the following methods:
 	
`CollectorAddPointLocations`
 	
`CollectorAddPointEntityLocations`
 	
`CollectorAddPointEntityFaceLocations`

Example:
None

CollectorSymbol

`CollectorSymbol`

`( pName, nAppearances, iSymbolID, qScreenScale, fScale, fDepthOffset, iCollectorID )`

Description:	
This function creates a Symbol collector. Symbol locations (points) can be added based on location in global space or relative to Femap entities such as `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE` and `FT_ELEM`.	
Input:	
`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see **CollectorAppearance**).
`INT4 iSymbolID`	ID of symbol to use (see **SymbolCreate**).
`BOOL qScreenScale`	If True, `fScale` is a percentage of screen size, If False, `fScale` is a multiplying factor.

Unpublished work. © 2025 Siemens

`REAL8 fScale`	Scale factor for symbols.
`REAL8 fDepthOffset`	Depth offset - currently not used (set to 0.0).
Output:	
`INT4 iCollectorID`	The ID of the collector created.
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	

A symbol must be created using the following method:

```
SymbolCreate
```

Symbol entities added using the following methods:

```
CollectorAddSymbolLocations
CollectorAddSymbolEntityLocations
CollectorAddSymbolEntityFaceLocations
CollectorAddSymbolEntityFaceNormalLocations
```

All symbols in the collector will be scaled by the same amount.

Example:

None

CollectorSymbolREAL8

Unpublished work. © 2025 Siemens

CollectorSymbolREAL8

**(pName, nAppearances, iSymbolID, qScreenScale, fMaxScale, fMinScale, fXOffset,
fYOffset, fZOffset, fDepthOffset, pFormat, iCollectorID)**

Description:

This function creates a Symbol collector that has a REAL8 value associated with each symbol entity. Symbol locations (points) can be added based on location in global space or relative to Femap entities such as FT_POINT, FT_CURVE, FT_SURFACE, FT_NODE and FT_ELEM. The associated REAL8 value can be displayed on each instance of the symbol and can also be used to scale the size of the symbol entities.

Input:

`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see **CollectorAppearance**).
`INT4 iSymbolID`	ID of symbol to use (see **SymbolCreate**).
`BOOL qScreenScale`	If True, `fMaxScale` and `fMinScale` are percentages of screen size. If False, `MaxScale` and `fMinScale` are multiplying factors.
`REAL8 fMaxScale`	Scale factor of largest associated REAL8 value.
`REAL8 fMinScale`	Scale factor of smallest associated REAL8 value.
`REAL8 fXOffset`	X offset of REAL8 value.
`REAL8 fYOffset`	Y offset of REAL8 value.
`REAL8 fZOffset`	Z offset of REAL8 value (set to 0.0).
`REAL8 fDepthOffset`	Depth offset - currently not used (set to 0.0).
`STRING pFormat`	C/C++ format string (should contain a %g or equivalent).

Output:

`INT4 iCollectorID`	The ID of the collector created.

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

A symbol must be created using the following method:

```
SymbolCreate
```

Symbol with REAL8 text entities added using the following methods:

```
CollectorAddSymbolREAL8Locations
CollectorAddSymbolREAL8EntityLocations
CollectorAddSymbolREAL8EntityFaceLocations
CollectorAddSymbolREAL8EntityFaceNormalLocations
```

Individual symbols scaled by associated REAL8 value. If you want all symbols the same size, set `fMinScale` equal to `fMaxScale`.

The `fXOffset` and `fYOffset` are percentages of width and height of the final text string. Values of 0.0 and 0.0 will place the text's center at the specified location. Values of -50.0 and -50.0 will place the bottom left corner of the text at the specified location.

Example:

None

CollectorText

```
CollectorText
```

```
( pName, nAppearances, fXOffset, fYOffset, fZOffset, iCollectorID )
```

Description:

This function create a Text collector. Text locations (points) can be added based on location in global space or relative to Femap entities such as `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE` and `FT_ELEM`.

Input:

Unpublished work. © 2025 Siemens

`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see **CollectorAppearance**)
`REAL8 fXOffset`	X offset of Text location.
`REAL8 fYOffset`	Y offset of Text location.
`REAL8 fZOffset`	Z offset of Text location (set to 0.0).
Output:	
`INT4 iCollectorID`	The ID of the collector created.
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	

Bitmap location entities added using the following methods:

```
CollectorAddTextLocations

CollectorAddTextEntityLocations

CollectorAddTextEntityFaceLocations
```

The `fXOffset` and `fYOffset` are percentages of width and height of the final text string. Values of 0.0 and 0.0 will place the text's center at the specified location. Values of -50.0 and -50.0 will place the bottom left corner of the text at the specified location.

Example:

None

CollectorTextINT4

Unpublished work. © 2025 Siemens

CollectorTextINT4

(pName, nAppearances, fXOffset, fYOffset, fZOffset, pFormat, iCollectorID)

Description:

This function create a TextINT4 collector. Text locations (points) can be added based on location in global space or relative to Femap entities such as FT_POINT, FT_CURVE, FT_SURFACE, FT_NODE and FT_ELEM. This collector is more efficient memory wise that a Text Collector if all you want to write is an INT4 value or an INT4 value with some text that is the same for all INT4s.

Input:

STRING pName	Name of collector - currently not used.
INT4 nAppearances	Number of appearances that will be added to this collector (see **CollectorAppearance**).
REAL8 fXOffset	X offset of Text location.
REAL8 fYOffset	Y offset of Text location.
REAL8 fZOffset	Z offset of Text location (set to 0.0).
STRING pFormat	C/C++ format string (should contain a %d or equivalent)

Output:

INT4 iCollectorID	The ID of the collector created.

Return Code:

FE_FAIL	Call was unsuccessful.

Remarks/Usage:

INT4 text entities added using the following methods:

```
CollectorAddTextINT4Locations

CollectorAddTextINT4EntityLocations

CollectorAddTextINT4EntityFaceLocations
```


Unpublished work. © 2025 Siemens

The `fXOffset` and `fYOffset` are percentages of width and height of the final text string. Values of 0.0 and 0.0 will place the text's center at the specified location. Values of -50.0 and -50.0 will place the bottom left corner of the text at the specified location.
Example:
None

CollectorTextREAL8

`CollectorTextREAL8`	
`( pName, nAppearances, fXOffset, fYOffset, fZOffset, pFormat, iCollectorID )`	
Description:	
This function create a `TextREAL8` collector. Text locations (points) can be added based on location in global space or relative to Femap entities such as `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE` and `FT_ELEM`. This collector is more efficient memory wise that a Text Collector if all you want to write is a REAL8 value or a REAL8 value with some text that is the same for all REAL8s.	
Input:	
`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see **CollectorAppearance**).
`REAL8 fXOffset`	X offset of Text location.
`REAL8 fYOffset`	Y offset of Text location.
`REAL8 fZOffset`	Z offset of Text location (set to 0.0).
`STRING pFormat`	C/C++ format string (should contain a %g or equivalent).
Output:	
`INT4 iCollectorID`	The ID of the collector created.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

REAL8 text entities added using the following methods:

```
CollectorAddTextREAL8Locations

CollectorAddTextREAL8EntityLocations

CollectorAddTextREAL8EntityFaceLocations
```

The `fXOffset` and `fYOffset` are percentages of width and height of the final text string. Values of 0.0 and 0.0 will place the text's center at the specified location. Values of -50.0 and -50.0 will place the bottom left corner of the text at the specified location.

Example:

None

CollectorLines

`CollectorLines`

`( pName, nAppearances, fDepthOffset, iCollectorID )`

Description:

This function create a Line collector.

Input:

`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see CollectorAppearance).
`REAL8 fDepthOffset`	Depth offset - currently not used (set to 0.0).

Output:

`INT4 iCollectorID`	The ID of the collector created.

Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

Line entities added using the following methods:

```
CollectorAddLineLocations
CollectorAddPolyLineLocations
CollectorAddMeshLineLocations
```

Example:

None

CollectorTriangles

`CollectorTriangles`

`( pName, nAppearances, fDepthOffset, iCollectorID )`

Description:

This function create a Triangle collector.

Input:

`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see **CollectorAppearance**).
`REAL8 fDepthOffset`	Depth offset - currently not used (set to 0.0).

Output:

`INT4 iCollectorID`	The ID of the collector created.

Return Code:

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

Unpublished work. © 2025 Siemens

Triangle entities added using the following methods:
 	
`CollectorAddTriangleLocations`
 	
`CollectorAddMeshTriangleLocations`
 	
`CollectorAddMeshTriangleNormalLocations`

Example:
None

CollectorBlank

`CollectorBlank` **`( iCollectorID, qBlanked )`**	
Description:	
This function blanks the specified collector so it will not be drawn until it is un-blanked.	
Input:	
`INT4 iCollectorID`	ID of collector to blank/unblank.
`BOOL qBlanked`	If True, the specified collector is blanked. If False, the specified collector is un-blanked.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
There is also a `Blank` method for the complete User Defined Graphics object.	
Example:	
None	

Unpublished work. © 2025 Siemens

CollectorSingleSymbol

`CollectorSingleSymbol`

`( iCollectorID, qSingle )`

Description:

This function controls whether a single symbol or multiple symbols are drawn on a curve or surface entity

Input:

`INT4 iCollectorID`	ID of collector to control.
`BOOL qSingle`	If True, draw a single symbol. If False, draw multiple symbols.

Output:

None	None

Return Code:

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

By default, entities of type FT_CURVE and FT_SURFACE will draw multiple symbols on the curve or surface. This function enables a single symbol to be drawn at the centroid of the entity.

Example:

None

CollectorLabels

`CollectorLabels`

`( iCollectorID, qLabels )`

Description:

Unpublished work. © 2025 Siemens

This function controls whether labels are drawn for a collector. Currently this only applies to `SymbolREAL8` collectors.	
Input:	
`INT4 iCollectorID`	ID of collector to label or no label.
`BOOL qLabels`	If True, labels are drawn. If False, labels are not drawn.
Output:	
None	None
Return Code:	
`FE_FAIL`	The selected cell does not have a value.
Remarks/Usage:	
None	
Example:	
None	

CollectorMarkForDeletion

`CollectorMarkForDeletion` **`( iCollectorID )`**	
Description:	
This function marks the specified collector for deletion. It will be no longer accessible from the API and will be deleted during the next draw.	
Input:	
`INT4 iCollectorID`	ID of collector to mark for deletion.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAppearance

`CollectorAppearance` **`( iCollectorID, iAppearancID, fColor1, fColor2, fLineWidth, iLineStyle, iFillPattern, qLighting, iCullMode, iFont, fPointSize )`**	
Description:	
This function adds an appearance to a collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add appearance.
`INT4 iAppearanceID`	ID of appearance (Appearance ID's are numbered 0..n-1).
`REAL8 fcColor1[0..3]`	Red, Green, Blue and Alpha values of Color 1. Alpha sets the level of transparency, where 0.0 is not visible and 1.0 is fully visible. In Femap versions prior to v2306, the alpha variable was ignored.
`REAL8 fColor2[0..3]`	Red, Green, Blue and Alpha values of Color 2. Alpha sets the level of transparency, where 0.0 is not visible and 1.0 is fully visible. In Femap versions prior to v2306, the alpha variable was ignored.
`REAL8 fLineWidth`	Width of lines to be drawn in pixels.
`INT4 iLineStyle`	Style of Line to be drawn. 0 = Solid Line; 1 = Long Dash; 2 = Short dash; 3 = Medium dash, Short dash; 4 = Medium dash, Short dash, Short dash.
`INT4 iFillPattern`	Style of Fill Pattern. 0 = Solid, 1 = Horizontal Line, 2 = Vertical Line, 3 = Diagonal Lines Top Left to

Unpublished work. © 2025 Siemens

	Bottom Right, 4 = Diagonal Lines Bottom Left to Top Right, 5 = Check, 6= Diagonal Check.
`BOOL qLighting`	If True, filled faces can be processed with lighting/shading.
`INT4 iCullMode`	Cull Mode. 0 = No Face Culling, 1 = Cull Back Face, 2 = Cull Front Face.
`INT4 iFont`	Text font (see **Label Font** in View Options dialog). Greater than 1000, will use the font specified in the View Options dialog Label Parameters. Greater than or equal to 0 and less than or equal to 76, will use `iFont` (see the View Options dialog Label Parameters option for list). For version Femap 2019.1 and above, it will likely be better to use `CollectorAppearanceSetFontData`, otherwise, you are limited to the fonts available in earlier versions of Femap.
`REAL8 fPointSize`	Size that points are drawn in pixels
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The second color is used when the line style or fill pattern are not 0 and `fColor2` is different from `fColor1`. The second color is also used as the background color for text if that option is selected in View Options and `fColor2` is different from `fColor1`.	
Example:	
None	

CollectorAppearanceSetFontData

Unpublished work. © 2025 Siemens

CollectorAppearanceSetFontData	
(iCollectorID, iAppearanceID, iPointHeight, bBold, bItalic, pcName)	
Description:	
This function defines the font for an existing `CollectorAppearance`.	
Input:	
`INT4 iCollectorID`	ID of collector to add appearance.
`INT4 iAppearanceID`	ID of existing `CollectorAppearance` (Appearance ID's are numbered 0..n-1).
`INT4 iPointHeight`	Size in *point height* (for example, “8 pt”, “9 pt”, “10 pt”, etc) for font requested from Windows via `pcName`.
`BOOL bBold`	If True, attempts to use **Bold** version of font requested from Windows via `pcName`.
`BOOL bItalic`	If True, attempts to use Italic version of font requested from Windows via `pcName`.
`STRING pcName`	Name of font to request from Windows
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If requested font with desired options is not available from Windows on a certain operating system or machine, will use a font as close as possible which is available from Windows on the operating system/ machine.	
Example:	
None	

CollectorContourLegendText

Unpublished work. © 2025 Siemens

CollectorContourLegendText

(pName, nAppearances, piCollectorID)

Description:

This function creates a Text collector used to create custom texts for contour legend and defined level contour legend labels. Text locations can be added relative to the contour legend. The Open GL transformation space must not be set to 1, model space, otherwise this call will not be successful.

Input:

`STRING pName`	Name of collector - currently not used.
`INT4 nAppearances`	Number of appearances that will be added to this collector (see **CollectorAppearance**).

Output:

`INT4 piCollectorID`	The ID of the collector created.

Return Code:

`FE_FAIL`	Open GL transformation space is set to 1, model space.

Remarks/Usage:

Bitmap locations entities added using the following methods:

CollectorAddTextContourLegendLocations

Creating this collector will automatically set the transformation space to 2 i.e. screen space. There is no need to call `SetOpenGLTransformationSpace` to set the transformation space to screen space. However, if this function is called after creating `CollectorContourLegendText` object, it returns `FE_FAIL`.

Example:

None

CollectorAddPointLocations

Unpublished work. © 2025 Siemens

CollectorAddPointLocations	
(iCollectorID, iAppearanceID, nPoints, pfPoints)	
Description:	
This functions adds point locations in global space to a Point collector or to a Bitmap collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..(3*npoints)-1]`	Coordinates of points.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
A point (or bitmap) will be drawn at each location specified using the specified appearance. If the appearance ID is too large, it is wrapped: if there are 5 appearances and the `iAppearanceID` is set to 8 then appearance 3 will be used.	
Example:	
None	

CollectorAddPointEntityLocations

CollectorAddPointEntityLocations
(iCollectorID, iAppearanceID, eType, nIDs, piIDs)
Description:

Unpublished work. © 2025 Siemens

This function adds points specified by existing Femap entities to a Point collector or Bitmap collector	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE`, or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..(nIDs-1)]`	List of entity IDs.
Output:	
None	None
Return Code:	
FE_FAIL	Call was unsuccessful.
Remarks/Usage:	
If `FT_CURVE` or `FT_SURFACE` is used, multiple points are created at the locations used for geometry based loading. If `FT_ELEM` is used, a point is created at the centroid of the element (currently only shells and solids).	
Example:	
None	

CollectorAddPointEntityFaceLocations

```
CollectorAddPointEntityFaceLocations

( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs )
```

Description:
This function adds points specified by a particular face of existing Femap entities to a Point collector or Bitmap collector.
Input:

Unpublished work. © 2025 Siemens

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..(nIDs-1)]`	List of entity IDs.
`INT4 piFaceIDs[0..(nIDs-1)]`	Face ID of entity.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
A point is created at the centroid of the specified face of the specified element (currently only shells and solids).	
Example:	
None	

CollectorAddSymbolLocations

`CollectorAddSymbolLocations`
`( iCollectorID, iAppearanceID, nSymbols, pfPoints, pfDirections, pfOrthogonals )`
Description:
This functions adds symbol locations and orientations in global space to a Symbol collector.
Input:

`INT4 iCollectorID`	ID of collector to add data.

Unpublished work. © 2025 Siemens

`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nSymbols`	Number of symbols to draw.
`REAL8 pfPoints[0..(3*nsymbols)-1]`	Locations to draw symbols.
`REAL8 pfDirections [0..(3*nsymbols)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nsymbols)-1]`	Orthogonal direction of symbol at each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolEntityLocations

`CollectorAddSymbolEntityLocations`
`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, pfDirections, pfOrthogonals )`
Description:
This functions adds symbol locations specified by existing Femap entities to a Symbol collector. Direction and orthogonal direction are provided for each location.
Input:

Unpublished work. © 2025 Siemens

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE`, or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`REAL8 pfDirections [0..(3*nIDs)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nIDs)-1]`	Orthogonal direction of symbol at each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolEntityFaceLocations

`CollectorAddSymbolEntityFaceLocations`

`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, pfDirections, pfOrthogonals )`

Description:

This functions adds symbol locations specified by particular faces of existing Femap entities to a Symbol collector. Direction and orthogonal direction are provided for each location.

Unpublished work. © 2025 Siemens

Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_SURFACE` or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piFaceIDs[0..nIDs-1]`	Face ID of entity.
`REAL8 pfDirections [0..(3*nIDs)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nIDs)-1]`	Orthogonal direction of symbol at each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolEntityFaceNormalLocations

Unpublished work. © 2025 Siemens

CollectorAddSymbolEntityFaceNormalLocations

(iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, pfOrthogonals)

Description:

This functions adds symbol locations specified by particular faces of existing Femap entities to a Symbol collector. The direction is the normal to the particular face of the existing Femap entities. The orthogonal direction is provided for each location.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_CURVE`, `FT_SURFACE`, or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piFaceIDs[0..nIDs-1]`	Face ID of entity.
`REAL8 pfOrthogonals[0..(3*nIDs)-1]`	Orthogonal direction of symbol at each location.

Output:

None	None

Return Code:

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

The face ID is ignored by the `FT_CURVE` entities but the tangential direction on the curve is used to align the symbols.

If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.

Example:

None

Unpublished work. © 2025 Siemens

CollectorAddSymbolEntityFaceNormalAutoLocations

CollectorAddSymbolEntityFaceNormalAutoLocations

(iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, piOrientOption, pdOrientAngle)

Description:

This functions adds symbol locations specified by particular faces of existing Femap entities to a Symbol collector. The direction is the normal to the particular face of the existing Femap entities. The orthogonal direction is controlled by an orientation option, `piOrientOption` (and orientation angle for FT_SURFACE).

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	FT_CURVE, FT_SURFACE, or FT_ELEM.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piFaceIDs[0..nIDs-1]`	Face ID of entity.
`INT4 piOrientOption[0..nIDs-1]`	Orientation option to determine orthogonal direction of symbol at each location. See the Remarks section to see the various options available for each entity type.
`REAL8 pdOrientAngle [0..nIDs-1]`	Rotate orthogonal direction of symbols about normal (FT_SURFACE).

Output:

None	None

Return Code:

`FE_FAIL`	Call was unsuccessful.

Unpublished work. © 2025 Siemens

Remarks/Usage:

The face ID is ignored by the FT_CURVE entities but the tangential direction on the curve is used to align the symbols.

If qScreenOrient = True in the previous call to SymbolCreate, the values of piOrientOption are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.

For eType = FT_CURVE – Symbol direction is local tangent to the curve.

iOrientOption = 0, orthogonal direction as close to global X axis as possible.
iOrientOption = 1, orthogonal direction as close to global Y axis as possible.
iOrientOption = 2, orthogonal direction as close to global Z axis as possible.

For eType = FT_SURFACE – Symbol direction is normal to the surface for Face ID=1, negative normal for Face ID=2.

iOrientOption = 0, orthogonal direction as close to global X axis as possible.
iOrientOption = 1, orthogonal direction as close to global Y axis as possible.
iOrientOption = 2, orthogonal direction as close to global Z axis as possible.
iOrientOption = 3, orthogonal direction as close to local surface s direction.
iOrientOption = 4, orthogonal direction as close to local surface t direction.

For eType = FT_ELEM (Point1)

iOrientOption = MN - Symbol direction in global direction M (0=X, 1=Y, 2=Z), Orthogonal in global direction N.

For eType = FT_ELEM (Line2, Line3) – Symbol direction is along the line element from Node 1 to Node 2.

iOrientOption = 0, orthogonal direction as close to global X axis as possible.
iOrientOption = 1, orthogonal direction as close to global Y axis as possible.
iOrientOption = 2, orthogonal direction as close to global Z axis as possible.

For eType = FT_ELEM (Tria3/6, Quad4/8) – Symbol Direction is normal to Face ID.

For Face ID=1 or 2 (Face ID=0 is same as Face ID=1).

iOrientOption = 1, orthogonal direction is edge 1.
iOrientOption = 2, orthogonal direction is edge 2.
iOrientOption = 3, orthogonal direction is edge 3.
iOrientOption = 4, orthogonal direction is edge 4 (Quad4, Quad8).
iOrientOption = 11, orthogonal direction is vertex 1 to edge 2 (Tria3, Tria6).
iOrientOption = 12, orthogonal direction is vertex 2 to edge 3 (Tria3, Tria6).
iOrientOption = 13, orthogonal direction is vertex 3 to edge 1 (Tria3, Tria6).

Unpublished work. © 2025 Siemens

`iOrientOption` = 11, orthogonal direction is vertex 1 to vertex 3 (`Quad4`, `Quad8`).
`iOrientOption` = 12, orthogonal direction is vertex 2 to vertex 4 (`Quad4`, `Quad8`).
`iOrientOption` = 13, orthogonal direction is edge 1 to edge 3 (`Quad4`, `Quad8`).
`iOrientOption` = 14, orthogonal direction is edge 2 to edge 4 (`Quad4`, `Quad8`).
`iOrientOption` = 15, orthogonal direction is diagonal angle bisector (`Quad4`, `Quad8`).

For Face ID=3, 4, 5 or 6, orthogonal direction is along the corresponding edge.

For `eType = FT_ELEM (Tetra4/10, Wedge6/15, Pyramid5/13, Brick8/20)` – Symbol Direction is normal to Face ID and for each face of element, `iOrientOption` is as for `Tria3/6` and `Quad4/8`.

Example:

None

CollectorAddSymbolEntityEdgeLocations

`CollectorAddSymbolEntityEdgeLocations`

`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piEdgeIDs, pfDirections, pfOrthogonals )`

Description:

This functions adds symbol locations specified by particular edges of existing Femap entities to a Symbol collector. The direction and the orthogonal direction is provided for each location. This currently only applies to `FT_ELEM`.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piEdgeIDs[0..nIDs-1]`	Edge ID of entity.

Unpublished work. © 2025 Siemens

`REAL8 pfDirections [0..(3*nIDs)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nIDs)-1]`	Orthogonal direction of symbol at each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
This function is only valid for `FT_ELEM`. The Edge ID is based on the numbering of the midside nodes of the parabolic versions of the elements. If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolEntityEdgeAutoLocations

`CollectorAddSymbolEntityEdgeAutoLocations`
`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piEdgeIDs, piOrientOption )`
Description:
This functions adds symbol locations specified by particular edges of existing Femap entities to a Symbol collector. The direction and the orthogonal direction is provided for each location. This currently only applies to `FT_ELEM`. The direction and the orthogonal directions are specified using an orientation option, `piOrientOption`.
Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.

Unpublished work. © 2025 Siemens

`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piEdgeIDs[0..nIDs-1]`	Edge ID of entity.
`INT4 piOrientOption[0..nIDs-1]`	Orientation option to determine orthogonal direction of symbol at each location. See Remarks section to see the various options available for each entity type.
Output:	
None	None
Return Code:	
FE_FAIL	Call was unsuccessful.
Remarks/Usage:	

This function is only valid for `FT_ELEM`. The Edge ID is based on the numbering of the midside nodes of the parabolic versions of the elements.

If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `piOrientOption` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.

For `eType = FT_ELEM (Point1)`.

`iOrientOption` = MN, Symbol direction in global direction M (0=X, 1=Y, 2=Z), Orthogonal in global direction N.

For `eType = FT_ELEM` (Line2, Line3) – Symbol direction is along the line element from Node 1 to Node 2

`iOrientOption` = 0, orthogonal direction as close to global X axis as possible.
`iOrientOption` = 1, orthogonal direction as close to global Y axis as possible.
`iOrientOption` = 2, orthogonal direction as close to global Z axis as possible.

For `eType = FT_ELEM (Tria3/6, Quad4/8)`

`iOrientOption` = MN, Symbol direction is controlled by M and orthogonal direction is controlled by N

M = 1, symbol direction is face normal
M = 2, symbol direction is reverse face normal

Unpublished work. © 2025 Siemens

M = 3, symbol direction is edge direction
M = 4, symbol direction is reverse edge direction
M = 5, symbol direction is edge normal
M = 6, symbol direction is reverse edge normal
N = 1, orthogonal direction is face normal
N = 2, orthogonal direction is reverse face normal
N = 3, orthogonal direction is edge direction
N = 4, orthogonal direction is reverse edge direction
N = 5, orthogonal direction is edge normal
N = 6, orthogonal direction is reverse edge normal

For `eType = FT_ELEM (Tetra4/10, Wedge6/15, Pyramid5/13, Brick8/20)`

`iOrientOption` = LMN, Symbol direction is controlled by M and orthogonal direction is controlled by N

L = 0, face normal is average of two faces adjacent to specified edge
L = 1, face normal is normal of face to the left of specified edge
L = 2, face normal is normal of face to the right of specified edge
M and N are the same as for `Tria3/6`, `Quad4/8`

Example:

None

CollectorAddSymbolREAL8Locations

`CollectorAddSymbolREAL8Locations`

`( iCollectorID, iSymbolAppearanceID, iLabelAppearanceID, nSymbols, pfPoints, pfDirections, pfOrthogonals, pfValues )`

Description:

This functions adds symbol locations and orientations in global space to a Symbol collector.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iSymbolAppearanceID`	Appearance ID for symbol.
`INT4 iLabelAppearanceID`	Appearance ID for REAL8 value text.

Unpublished work. © 2025 Siemens

`INT4 nSymbols`	Number of symbols to draw.
`REAL8 pfPoints[0..(3*nsymbols)-1]`	Locations to draw symbols.
`REAL8 pfDirections [0..(3*nsymbols)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nsymbols)-1]`	Orthogonal direction of symbol at each location.
`REAL8 pfValues[0..nsymbols-1]`	Value for each symbol location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolREAL8EntityLocations

`CollectorAddSymbolREAL8EntityLocations`

```
( iCollectorID, iSymbolAppearanceID, iLabelAppearanceID, eType, nIDs,
piIDs,
pfDirections, pfOrthogonals, pfValues )
```

Description:

This functions adds symbol locations specified by existing Femap entities to a Symbol collector. Direction and orthogonal direction are provided for each location. A REAL8 value is provided for each symbol location.

Unpublished work. © 2025 Siemens

Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iSymbolAppearanceID`	Appearance ID for symbol.
`INT4 iLabelAppearanceID`	Appearance ID for REAL8 value text.
`enum zDataType eType`	`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE`, or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`REAL8 pfDirections [0..(3*nsymbols)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nsymbols)-1]`	Orthogonal direction of symbol at each location.
`REAL8 pfValues[0..nsymbols-1]`	Value for each symbol location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolREAL8EntityFaceLocations

Unpublished work. © 2025 Siemens

CollectorAddSymbolREAL8EntityFaceLocations

```
( iCollectorID, iSymbolAppearanceID, iLabelAppearanceID, eType, nIDs,
piIDs,
 piFaceIDs, pfDirections, pfOrthogonals, pfValues )
```

Description:

This functions adds symbol locations specified by particular faces of existing Femap entities to a Symbol collector. Direction and orthogonal direction are provided for each location. A REAL8 value is provided for each symbol location.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iSymbolAppearanceID`	Appearance ID for symbol.
`INT4 iLabelAppearanceID`	Appearance ID for REAL8 value text.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piFaceIDs[0..nIDs-1]`	Face ID of entity.
`REAL8 pfDirections [0..(3*nsymbols)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nsymbols)-1]`	Orthogonal direction of symbol at each location.
`REAL8 pfValues[0..nsymbols-1]`	Value for each symbol location.

Output:

None	None

Return Code:

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

Unpublished work. © 2025 Siemens

If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.
Example:
None

CollectorAddSymbolREAL8EntityFaceNormalLocations

`CollectorAddSymbolREAL8EntityFaceNormalLocations`

`( iCollectorID, iSymbolAppearanceID, iLabelAppearanceID, eType, nIDs, piIDs, piFaceIDs, pfOrthogonals, pfValues )`

Description:

This functions adds symbol locations specified by particular faces of existing Femap entities to a Symbol collector. The direction is the normal to the particular face of the existing Femap entities. The orthogonal direction is provided for each location. A REAL8 value is provided for each symbol location.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iSymbolAppearanceID`	Appearance ID for symbol.
`INT4 iLabelAppearanceID`	Appearance ID for REAL8 value text.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piFaceIDs[0..nIDs-1]`	Face ID of entity.
`REAL8 pfOrthogonals[0..(3*nsymbols)-1]`	Orthogonal direction of symbol at each location.
`REAL8 pfValues[0..nsymbols-1]`	Value for each symbol location.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

CollectorAddSymbolREAL8EntityFaceNormalAutoLocations

`CollectorAddSymbolREAL8EntityFaceNormalAutoLocations` **`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, piOrientOption, pdOrientAngle, pfValues )`**	
Description:	
This functions adds symbol locations specified by particular faces of existing Femap entities to a Symbol collector. The direction is the normal to the particular face of the existing Femap entities. The orthogonal direction is controlled by an orientation option, `piOrientOption` (and orientation angle for FT_SURFACE). A REAL8 value is provided for each symbol location.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	FT_CURVE, FT_SURFACE, or FT_ELEM.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.

Unpublished work. © 2025 Siemens

`INT4 piFaceIDs[0..nIDs-1]`	Face ID of entity.
`INT4 piOrientOption[0..nIDs-1]`	Orientation option to determine orthogonal direction of symbol at each location. See Remarks section to see the various options available for each entity type.
`REAL8 pdOrientAngle [0..nIDs-1]`	Rotate orthogonal direction of symbols about normal (FT_SURFACE).
`REAL8 pfValues[0..nIDs-1]`	Value for each symbol location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	

The face ID is ignored by the FT_CURVE entities but the tangential direction on the curve is used to align the symbols.

If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `piOrientOption` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.

For `eType = FT_CURVE` – Symbol direction is local tangent to the curve.

`iOrientOption` = 0, orthogonal direction as close to global X axis as possible.
`iOrientOption` = 1, orthogonal direction as close to global Y axis as possible.
`iOrientOption` = 2, orthogonal direction as close to global Z axis as possible.

For `eType = FT_SURFACE` – Symbol direction is normal to the surface for Face ID=1, negative normal for Face ID=2.

`iOrientOption` = 0, orthogonal direction as close to global X axis as possible.
`iOrientOption` = 1, orthogonal direction as close to global Y axis as possible.
`iOrientOption` = 2, orthogonal direction as close to global Z axis as possible.
`iOrientOption` = 3, orthogonal direction as close to local surface s direction.
`iOrientOption` = 4, orthogonal direction as close to local surface t direction.

For `eType = FT_ELEM (Point1)`.

`iOrientOption` = MN, Symbol direction in global direction M (0=X, 1=Y, 2=Z), Orthogonal in global direction N.

Unpublished work. © 2025 Siemens

For `eType = FT_ELEM (Line2, Line3)` – Symbol direction is along the line element from Node 1 to Node 2.

`iOrientOption` = 0, orthogonal direction as close to global X axis as possible.
`iOrientOption` = 1, orthogonal direction as close to global Y axis as possible.
`iOrientOption` = 2, orthogonal direction as close to global Z axis as possible.

For `eType = FT_ELEM (Tria3/6, Quad4/8)` – Symbol Direction is normal to Face ID.

For Face ID=1 or 2 (Face ID=0 is same as Face ID=1)

`iOrientOption` = 1, orthogonal direction is edge 1.
`iOrientOption` = 2, orthogonal direction is edge 2.
`iOrientOption` = 3, orthogonal direction is edge 3.
`iOrientOption` = 4, orthogonal direction is edge 4 (`Quad4`, `Quad8`).
`iOrientOption` = 11, orthogonal direction is vertex 1 to edge 2 (`Tria3`, `Tria6`).
`iOrientOption` = 12, orthogonal direction is vertex 2 to edge 3 (`Tria3`, `Tria6`).
`iOrientOption` = 13, orthogonal direction is vertex 3 to edge 1 (`Tria3`, `Tria6`).
`iOrientOption` = 11, orthogonal direction is vertex 1 to vertex 3 (`Quad4`, `Quad8`).
`iOrientOption` = 12, orthogonal direction is vertex 2 to vertex 4 (`Quad4`, `Quad8`).
`iOrientOption` = 13, orthogonal direction is edge 1 to edge 3 (`Quad4`, `Quad8`).
`iOrientOption` = 14, orthogonal direction is edge 2 to edge 4 (`Quad4`, `Quad8`).
`iOrientOption` = 15, orthogonal direction is diagonal angle bisector (`Quad4`, `Quad8`).

For Face ID=3, 4, 5 or 6, orthogonal direction is along the corresponding edge.

For `eType = FT_ELEM (Tetra4/10, Wedge6/15, Pyramid5/13, Brick8/20)` – Symbol Direction is normal to Face ID and for each face of element, `iOrientOption` is as for `Tria3/6` and `Quad4/8`.

Example:

None

CollectorAddSymbolREAL8EntityEdgeLocations

`CollectorAddSymbolREAL8EntityEdgeLocations`

`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piEdgeIDs, pfDirections, pfOrthogonals, pfValues )`

Description:

Unpublished work. © 2025 Siemens

This functions adds symbol locations specified by particular edges of existing Femap entities to a Symbol collector. The direction and the orthogonal direction is provided for each location. This currently only applies to FT_ELEM. A REAL8 value is provided for each symbol location.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piEdgeIDs[0..nIDs-1]`	Edge ID of entity.
`REAL8 pfDirections[0..(3*nIDs)-1]`	Direction of symbol at each location.
`REAL8 pfOrthogonals[0..(3*nIDs)-1]`	Orthogonal direction of symbol at each location.
`REAL8 pfValues[0..nIDs-1]`	Value for each symbol location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
This function is only valid for FT_ELEM. The Edge ID is based on the numbering of the midside nodes of the parabolic versions of the elements. If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `pfOrthogonals` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.	
Example:	
None	

Unpublished work. © 2025 Siemens

CollectorAddSymbolREAL8EntityEdgeAutoLocations

`CollectorAddSymbolREAL8EntityEdgeAutoLocations`

`( iCollectorID, iAppearanceID, eType, nIDs, piIDs, piEdgeIDs, piOrientOption, pfValues )`

Description:

This functions adds symbol locations specified by particular edges of existing Femap entities to a Symbol collector. The direction and the orthogonal direction is provided for each location. This currently only applies to `FT_ELEM`. The direction and the orthogonal directions are specified using an orientation option, `piOrientOption`. A REAL8 value is provided for each symbol location.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`INT4 piEdgeIDs[0..nIDs-1]`	Edge ID of entity.
`INT4 piOrientOption[0..nIDs-1]`	Orientation option to determine orthogonal direction of symbol at each location. See Remarks section to see the various options available for each entity type.
`REAL8 pfValues[0..nIDs-1]`	Value for each symbol location.

Output:

None	None

Return Code:

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

Unpublished work. © 2025 Siemens

This function is only valid for `FT_ELEM`. The Edge ID is based on the numbering of the midside nodes of the parabolic versions of the elements.

If `qScreenOrient` = True in the previous call to `SymbolCreate`, the values of `piOrientOption` are ignored and the symbol XZ plane is rotated in global space to be as close as possible to the plane of the screen.

For `eType = FT_ELEM (Point1)`.

`iOrientOption` = MN, Symbol direction in global direction M (0=X, 1=Y, 2=Z), Orthogonal in global direction N.

For `eType = FT_ELEM (Line2, Line3)` – Symbol direction is along the line element from Node 1 to Node 2

`iOrientOption` = 0, orthogonal direction as close to global X axis as possible.
`iOrientOption` = 1, orthogonal direction as close to global Y axis as possible.
`iOrientOption` = 2, orthogonal direction as close to global Z axis as possible.

For `eType = FT_ELEM (Tria3/6, Quad4/8)`.

`iOrientOption` = MN, Symbol direction is controlled by M and orthogonal direction is controlled by N.

M = 1, symbol direction is face normal.
M = 2, symbol direction is reverse face normal.
M = 3, symbol direction is edge direction.
M = 4, symbol direction is reverse edge direction.
M = 5, symbol direction is edge normal.
M = 6, symbol direction is reverse edge normal.
N = 1, orthogonal direction is face normal.
N = 2, orthogonal direction is reverse face normal.
N = 3, orthogonal direction is edge direction.
N = 4, orthogonal direction is reverse edge direction.
N = 5, orthogonal direction is edge normal.
N = 6, orthogonal direction is reverse edge normal.

For `eType = FT_ELEM (Tetra4/10, Wedge6/15, Pyramid5/13, Brick8/20)`.

`iOrientOption` = LMN, Symbol direction is controlled by M and orthogonal direction is controlled by N.

L = 0, face normal is average of two faces adjacent to specified edge.
L = 1, face normal is normal of face to the left of specified edge.
L = 2, face normal is normal of face to the right of specified edge.

M and N are the same as for `Tria3/6`, `Quad4/8`.

Unpublished work. © 2025 Siemens

Example:
None

CollectorAddTextLocations

`CollectorAddTextLocations` **`( iCollectorID, iAppearanceID, nPoints, pfPoints, ppText )`**	
Description:	
This functions adds text at locations in global space to a Text collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..(3*nPoints)-1]`	Coordinates of points to place text.
`STRING ppText[0..nPoints-1]`	STRING for each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

CollectorAddTextEntityLocations

CollectorAddTextEntityLocations **(iCollectorID, iAppearanceID, eType, nIDs, piIDs, ppText)**	
Description:	
This function adds text at locations specified by existing Femap entities to a Text collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE`, or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`STRING ppText[0..nIDs-1]`	STRING for each location.
Output:	
None	None
Return Code:	
FE_FAIL	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextEntityFaceLocations

Unpublished work. © 2025 Siemens

CollectorAddTextEntityFaceLocations **(iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, ppText)**	
Description:	
This function adds text at locations specified by a particular face of existing Femap entities to a Text collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_ELEM`
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`STRING ppText[0..nIDs-1]`	STRING for each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextINT4Locations

Unpublished work. © 2025 Siemens

CollectorAddTextINT4Locations **(iCollectorID, iAppearanceID, nPoints, pfPoints, piValues)**	
Description:	
This functions adds text based on an `INT4` value at locations in global space to a Text collector. An `INT4` value is supplied for each location.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..(3*nPoints)-1]`	Coordinates of points to place text.
`INT4 piValues[0..nPoints-1]`	INT4 value for each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextINT4EntityLocations

Unpublished work. © 2025 Siemens

CollectorAddTextINT4EntityLocations **(iCollectorID, iAppearanceID, eType, nIDs, piIDs, piValues)**	
Description:	
This function adds text based on an INT4 value at a location specified by existing Femap entities to a TextINT4 collector. An INT4 value is supplied for each location.	
Input:	
INT4 iCollectorID	ID of collector to add data.
INT4 iAppearanceID	ID of appearance to use for these entities.
enum zDataType eType	FT_POINT, FT_CURVE, FT_SURFACE, FT_NODE, or FT_ELEM.
INT4 nIDs	Number of entity IDs.
INT4 piIDs[0..nIDs-1]	List of entity IDs.
STRING piValues[0..nIDs-1]	INT4 value for each location.
Output:	
None	None
Return Code:	
FE_FAIL	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextINT4EntityFaceLocations

Unpublished work. © 2025 Siemens

CollectorAddTextINT4EntityFaceLocations **(iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, piValues)**	
Description:	
This function adds text based on an INT4 value at a location specified by a particular face of existing Femap entities to a TextINT4 collector. An INT4 value is supplied for each location.	
Input:	
INT4 iCollectorID	ID of collector to add data.
INT4 iAppearanceID	ID of appearance to use for these entities.
enum zDataType eType	FT_ELEM
INT4 nIDs	Number of entity IDs.
INT4 piIDs[0..nIDs-1]	List of entity IDs.
INT4 piFaceIDs[0..nIDs-1]	Face ID of entity.
INT4 piValues[0..nIDs-1]	INT4 value for each location.
Output:	
None	None
Return Code:	
FE_FAIL	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextREAL8Locations

Unpublished work. © 2025 Siemens

CollectorAddTextREAL8Locations **(iCollectorID, iAppearanceID, nPoints, pfPoints, pfValues)**	
Description:	
This functions adds text based on a REAL8 value at locations in global space to a `TextREAL8` collector. A REAL8 value is supplied for each location.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..(3*nPoints)-1]`	Coordinates of points to place text.
`REAL8 pfValues[0..nPoints-1]`	REAL8 value for each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextREAL8EntityLocations

Unpublished work. © 2025 Siemens

CollectorAddTextREAL8EntityLocations **(iCollectorID, iAppearanceID, eType, nIDs, piIDs, pfValues)**	
Description:	
This function adds text based on a REAL8 value at a location specified by existing Femap entities to a TextREAL8 collector. A REAL8 value is supplied for each location.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`enum zDataType eType`	`FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_NODE`, or `FT_ELEM`.
`INT4 nIDs`	Number of entity IDs.
`INT4 piIDs[0..nIDs-1]`	List of entity IDs.
`REAL8 pfValues[0..nIDs-1]`	REAL8 value for each location.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTextREAL8EntityFaceLocations

Unpublished work. © 2025 Siemens

CollectorAddTextREAL8EntityFaceLocations

(iCollectorID, iAppearanceID, eType, nIDs, piIDs, piFaceIDs, pfValues)

Description:

This function adds text based on a REAL8 value at a location specified by a particular face of existing Femap entities to a TextREAL8 collector. An REAL8 value is supplied for each location.

Input:

INT4 iCollectorID	ID of collector to add data.
INT4 iAppearanceID	ID of appearance to use for these entities.
enum zDataType eType	FT_ELEM
INT4 nIDs	Number of entity IDs.
INT4 piIDs[0..nIDs-1]	List of entity IDs.
INT4 piFaceIDs[0..nIDs-1]	Face ID of entity.
REAL8 pfValues[0..nIDs-1]	REAL8 value for each location.

Output:

None	None

Return Code:

FE_FAIL	Call was unsuccessful.

Remarks/Usage:

None

Example:

None

CollectorAddTextContourLegendLocations

Unpublished work. © 2025 Siemens

CollectorAddTextContourLegendLocations

(iCollectorID, iAppearanceID, nIDs, piIDs, ppText)

Description:

This function adds text at locations besides contour legend and defined level contour legends.

Input:

`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nIDs`	Number of location IDs.
`INT4 piIDs[0..nIDs-1]`	List of ID locations (See remarks to use `piIDs` to locate texts).
`STRING ppText[0..nIDs-1]`	Text to display for each location.

Output:

None	None

Return Code:

`FE_FAIL`	Call was unsuccessful.

Remarks/Usage:

`CollectorAddTextContourLegendLocations` can only be used with `CollectorContourLegendText` collectors (see **CollectorContourLegendText**).

`nIDs` is the number of `piIDs` and `ppText`. `piIDs` will be the numerical locations beside the contour legend. Positive IDs correspond to locations starting from the bottom and negative numbers correspond to locations starting from the top. Odd numbered IDs correspond to level dividers and even numbered IDs correspond to the mid locations between the level dividers.

Example:

```
Sub Main

Dim App As femap.model

Dim feUDG As femap.UserDefinedGraphics

Dim V As femap.View
```


Unpublished work. © 2025 Siemens

```
Set App = feFemap()
Set V = App.feView
Set feUDG = App.feUserDefinedGraphics
feUDG.SetName("Testing User Defined Graphics")

Dim iTextCollector1 As Long
Dim iAppearanceID As Long
Dim sText(4) As String
Dim pIDs(4) As Long
Dim fCyan(3) As Double
Dim iActiveView As Long

fCyan(0) = 0.0
fCyan(1) = 1.0
fCyan(2) = 1.0
fCyan(3) = 1.0

 'Switch off standard contour legend labels
 'without changing the min max colors
App.feAppGetActiveView(iActiveView)
V.Get(iActiveView)
If (V.ColorMode(FVI_CONTOUR_LEGEND) = 0) Then
   V.ColorMode(FVI_CONTOUR_LEGEND) = 2
ElseIf (V.ColorMode(FVI_CONTOUR_LEGEND) = 1) Then
   V.ColorMode(FVI_CONTOUR_LEGEND) = 3
End If
```


Unpublished work. © 2025 Siemens

```
V.Put(iActiveView)
```

```
'pIDs list
pIDs(0)=1
pIDs(1)=2
pIDs(2)=-1
pIDs(3)=-5
pIDs(4)=-6


 'Test Text
If (feUDG.CollectorContourLegendText
("Text",1,iTextCollector1) = FE_OK)
Then
   feUDG.CollectorAppearance
(iTextCollector1, 0,fCyan,fCyan,1.0,0,0,True,0,5,4.0)
   feUDG.CollectorAppearanceSetFontData
(iTextCollector1, 0,14,True,True,"Arial")
   iAppearanceID = 0
   sText(0) = "Text 1"
   sText(1) = "Text 2"
   sText(2) = "Text -1"
   sText(3) = "Text -5"
   sText(4) = "Text -6"
   feUDG.CollectorAddTextContourLegendLocations
(iTextCollector1,iAppearanceID,5,pIDs,sText)
   iAppearanceID = 0
End If
```


Unpublished work. © 2025 Siemens

```
feUDG.Publish()
App.feViewRegenerate(0)


App.feAppMessageBox (0, "Finished")


feUDG.Detach("MyGraphics")
End Sub
```

CollectorAddLineLocations

CollectorAddLineLocations	
(iCollectorID, iAppearanceID, nLines, pfPoints)	
Description:	
This functions adds lines at locations in global space to a Line collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nLines`	Number of lines to add.
`REAL8 pfPoints[0..nLines*2*3)-1]`	Coordinates of points at each end of each line.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

CollectorAddPolyLineLocations

`CollectorAddPolyLineLocations` **`( iCollectorID, iAppearanceID, nPoints, pfPoints )`**	
Description:	
This functions adds a polyline at locations in global space to a Line collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points in the polyline.
`REAL8 pfPoints[0..(3*nPoints)+2]`	Coordinates of points on the polyline.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddMeshLineLocations

Unpublished work. © 2025 Siemens

CollectorAddMeshLineLocations **(iCollectorID, iAppearanceID, nPoints, pfPoints, nLines, piLines)**	
Description:	
This functions adds a polyline at locations in global space to a Line collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..(3*nPoints)-1]`	Coordinates of points.
`INT4 nLines`	Number of lines.
`INT4 piLines[0..(2*nLines)-1]`	Two indices into `pfPoints` for each line.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CollectorAddTriangleLocations

Unpublished work. © 2025 Siemens

CollectorAddTriangleLocations	
(iCollectorID, iAppearanceID, nTriangles, pfPoints)	
Description:	
This functions adds triangles at locations in global space to a Triangle collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nTriangles`	Number of triangles.
`REAL8 pfPoints[0..nTriangles*3*3)-1]`	Coordinates of points at each corner of each triangle.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If the selected appearance has lighting/shading switched on, the normal used is calculated as the normal of each flat triangle.	
Example:	
None	

CollectorAddMeshTriangleLocations

CollectorAddMeshTriangleLocations
(iCollectorID, iAppearanceID, nPoints, pfPoints, nTriangles, piTriangles)
Description:

Unpublished work. © 2025 Siemens

This functions adds a mesh of triangles at locations in global space to a Triangle collector.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..nPoints*3)-1]`	Coordinates of points.
`INT4 nTriangles`	Number of triangles.
`REAL8 piTriangles[0..nTriangles*3)-1]`	Three indices into `pfPoints` for each triangle.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
If the selected appearance has lighting/shading switched on, the normal used is calculated as the normal of each flat triangle.	
Example:	
None	

CollectorAddMeshTriangleNormalLocations

`CollectorAddMeshTriangleNormalLocations`
`( iCollectorID, iAppearanceID, nPoints, pfPoints, pfNormals, nTriangles, piTriangles )`
Description:

Unpublished work. © 2025 Siemens

This functions adds a mesh of triangles at locations in global space to a Triangle collector. The normal enable the lighting/shading to be smooth over the mesh of triangles.	
Input:	
`INT4 iCollectorID`	ID of collector to add data.
`INT4 iAppearanceID`	ID of appearance to use for these entities.
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..(3*nPoints)-1]`	Coordinates of points.
`REAL8 pfNormals[0..(3*nPoints)-1]`	Normal vector at each point.
`INT4 nTriangles`	Number of triangles.
`INT4 piTriangles[0..(3*nTriangles)-1]`	Three indices into `pfPoints` and `pfNormals` for each triangle.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

BitmapCreate

`BitmapCreate`
`( iWidth, iHeight, pqBits, iBitmapID )`
Description:

Unpublished work. © 2025 Siemens

This function creates a Bitmap that can be used by a Bitmap collector.	
Input:	
`INT4 iWidth`	Width of Bitmap.
`INT4 iHeight`	Height of Bitmap.
`BOOL pqBits [0..(iWidth*iHeight)-1]`	Bitmap data in rows starting bottom left corner.
Output:	
`INT4 iBitmapID`	ID of Bitmap created.
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

BitmapMarkForDeletion

`BitmapMarkForDeletion` **`( iBitmapID )`**	
Description:	
This function marks a Bitmap for deletion. After it has been marked for deletion, it cannot be used by a Bitmap collector.	
Input:	
`INT4 iBitmapID`	ID of Bitmap to mark for deletion
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

SymbolCreate

`SymbolCreate` **`( nAppearances, fLabelLocation, qScreenOrient, iSymbolID )`**	
Description:	
This function creates a Symbol that can be used by a Symbol or `SymbolREAL8` collector.	
Input:	
`INT4 nAppearances`	Number of appearances used by the Symbol.
`REAL8 fLabelLocation[0..2]`	Location in Symbol space that a label can be located.
`BOOL qSreenOrient`	If True, Symbol will be oriented based on the screen normal. If False, Symbol will be oriented in global space.
Output:	
`INT4 iBitmapID`	ID of Symbol created.
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
`fLabelLocation` currently only used by `SymbolREAL8` collector.	

Unpublished work. © 2025 Siemens

A Symbol can have components with different appearances (colors etc.). When Symbol locations are added to a Symbol (or `SymbolREAL8`) collector, an appearance ID is provided and this is used as the base appearance ID. Each component added to a symbol, has an appearance offset that will be added to the base appearance ID to determine the actual appearance used by those locations of the Symbol. `qScreenOrient` is used for *2D Symbols*. Most symbols will be defined using 3D components in Symbol XYZ space. However, you can get screen aligned 2D symbols by defining the components in the XZ plane of symbol space. If `qScreenOrient` = True, then the XZ plane is rotated in global space to align as closely as possible to the direction out of the screen. Components can also be added to be used instead of the usual components if the symbol points straight out of or into the screen (see function `SetAddInScreen`) and these components will usually be in the XY plane of Symbol space. Screen oriented Symbols will be drawn initially oriented based on the screen and when a graphics regeneration (ctrl-g) is processed. Dynamic rotation will not usually keep them oriented based on screen unless the **File** → **Preferences** command is executed, and the **Graphics** tab **Memory Optimization** option is chosen.
Example:
None

SymbolMarkForDeletion

`SymbolMarkForDeletion` **`( iSymbolID )`**	
Description:	
This function marks a Symbol for deletion. After it has been marked for deletion, it cannot be used by a Symbol or `SymbolREAL8` collector.	
Input:	
`INT4 iSymbolID`	ID of Symbol to mark for deletion.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SymbolSetAddInScreen

SymbolSetAddInScreen **(iSymbolID, iInScreenMode)**	
Description:	
This function controls whether Symbol components for screen oriented Symbols are placed.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iInScreenMode`	In Screen mode. 0 = Load Symbol Component to draw when not aligned into or out of screen; > 0 = Load Symbol component to draw when aligned out of screen; < 0 = Load Symbol component to draw when aligned into screen.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

SymbolAddPoints

Unpublished work. © 2025 Siemens

SymbolAddPoints	
(iSymbolID, iAppearanceOffset, nPoints, pfPoints)	
Description:	
This function adds points to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`INT4 nPoints`	Number of points.
`REAL8 pfPoints[0..nPoints*3)-1]`	Coordinates of points in symbol space.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLineStraight

SymbolAddLineStraight
(iSymbolID, iAppearanceOffset, fPoints)
Description:

Unpublished work. © 2025 Siemens

This function adds individual lines to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fPoints[0..nPoints*2*3)-1]`	Coordinates of ends of Line in Symbol Space.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLinePoly

`SymbolAddLinePoly`	
`( iSymbolID, iAppearanceOffset, nPoints, pfPoints )`	
Description:	
This function adds a polyline to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`INT4 nPoints`	Number of points.

Unpublished work. © 2025 Siemens

`REAL8 pfPoints[0..nPoints*3)-1]`	Coordinates of points in symbol space.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLineCircle

`SymbolAddLineCircle`
`( iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fNormal, nDividers )`
Description:
This function adds a circle to a Symbol.
Input:

`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.
`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fNormal[0..2]`	Vector normal to the circle.
`INT4 nDividers`	Number of divisions in a quarter of a circle.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The normal does not have to exactly represent the normal. The normal is used to generate an orthogonal direction that will be in the plane of the circle. The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLineBrick

`SymbolAddLineBrick` **`( iSymbolID, iAppearanceOffset, fPoints )`**	
Description:	
This function adds a wireframe brick to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fPoints[0..23]`	Coordinates of 8 corners of a brick (i.e., 0=X coordinate of corner 1, 1=Ycoordinate of corner 1, 2=Z coordinate of corner 1, 3=X coordinate of corner 2, 4=Ycoordinate of corner 2, 5=Z coordinate of corner 2...21=X coordinate of corner 8, 22=Y coordinate of corner 8, 23=Z coordinate of corner 8).
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLineCylinder

`SymbolAddLineCylinder`
`( iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fDirection, fHeight, nDividers, nSubDividers )`
Description:
This function adds a wireframe cylinder to a Symbol.
Input:

`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.
`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fDirection[0..2]`	Vector along cylinder.
`REAL8 fHeight`	Height of cylinder.
`INT4 nDividers`	Number of lines drawn along cylindrical surface.

Unpublished work. © 2025 Siemens

`INT4 nSubDividers`	Number of divisions along circle between `nDividers`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The angle between the direction of the cylinder and the vector from the center to the point on circle does not have to be 90 degrees (it must not be 0 degrees). If the angle is not 90, the cylinder will be skewed. Internally, `nSubDividers` is increased until `nDividers` multiplied by `nSubDividers` is greater than or equal to 16. The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLineCone

`SymbolAddLineCone`
`( iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fDirection, fHeight, nDividers, nSubDividers )`
Description:
This function adds a wireframe cone to a Symbol.
Input:

`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.

Unpublished work. © 2025 Siemens

`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fDirection[0..2]`	Vector normal to the circle.
`REAL8 fHeight`	Height of cone.
`INT4 nDividers`	Number of lines drawn along conical surface.
`INT4 nSubDividers`	Number of divisions along circle between `nDividers`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The angle between the direction of the cone and the vector from the center to the point on circle does not have to be 90 degrees (it must not be 0 degrees). If the angle is not 90, the cone will be skewed. Internally, `nSubDividers` is increased until `nDividers` multiplied by `nSubDividers` is greater than or equal to 16. The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddLineSphere

`SymbolAddLineSphere`
`( iSymbolID, iAppearanceOffset, fCenter, fPointOnEquator, fDirection, fAngle, nLongDividers, nLongSubDividers, nLatDividers, nLatSubDividers )`
Description:

Unpublished work. © 2025 Siemens

This function adds a wireframe sphere to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of sphere in Symbol space.
`REAL8 fPointOnEquator[0..2]`	Coordinate of point on equator in Symbol space.
`REAL8 fDirection[0..2]`	Vector normal to the equator of the sphere.
`REAL8 fAngle`	Angle of revolution (360.0 is a sphere).
`INT4 nLongDividers`	Number of longitudinal lines drawn on sphere.
`INT4 nLongSubDividers`	Number of divisions between longitudinal lines.
`INT4 nLatDividers`	Number of latitudinal lines drawn on sphere.
`INT4 nLatSubDividers`	Number of divisions between latitudinal lines.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
Internally, `nLongSubDividers` and `nLatSubDividers` is increased until the angle for each subdivision is less than 22.5 degrees. The values of `nLongDividers` and `nLatDividers` control how many lines are drawn and `nLongSubDividers` and `nLatSubDividers` control how smooth they are. The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

Unpublished work. © 2025 Siemens

SymbolAddFillTriangle

SymbolAddFillTriangle **(iSymbolID, iAppearanceOffset, fPoints, fNormals)**	
Description:	
This function adds a filled triangle to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fPoints[0..8]`	Coordinates of triangle corners in Symbol space (i.e., 0=X coordinate of corner 1, 1=Ycoordinate of corner 1, 2=Z coordinate of corner 1, 3=X coordinate of corner 2, 4=Ycoordinate of corner 2, 5=Z coordinate of corner 2, 6=X coordinate of corner 3, 7=Y coordinate of corner 3, 8=Z coordinate of corner 3).
`REAL8 fNormals[0..8]`	Normal vectors at triangle corners in Symbol space (i.e., 0=X coordinate of vector at corner 1, 1=Ycoordinate of vector at corner 1, 2=Z coordinate of vector at corner 1, 3=X coordinate of vector at corner 2, 4=Ycoordinate of vector at corner 2, 5=Z coordinate of vector at corner 2, 6=X coordinate of vector at corner 3, 7=Y coordinate of vector at corner 3, 8=Z coordinate of vector at corner 3).
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The normal do not have to be normal to the triangle. This enables smoother lit/shaded surfaces to be displayed.	

Unpublished work. © 2025 Siemens

The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.
Example:
None

SymbolAddFillCircle

`SymbolAddFillCircle` **`( iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fNormal, nDividers )`**	
Description:	
This function adds a filled circle to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.
`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fNormal[0..2]`	Vector normal to the circle.
`INT4 nDividers`	Number of divisions in a quarter of a circle.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The normal does not have to exactly represent the normal. The normal is used to generate an orthogonal direction that will be in the plane of the circle.	

Unpublished work. © 2025 Siemens

Example:
None

SymbolAddFillBrick

SymbolAddFillBrick **(iSymbolID, iAppearanceOffset, fPoints, qEndCap)**	
Description:	
This function adds a filled brick to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fPoints[0..23]`	Coordinates of 8 corners of a brick (i.e., 0=X coordinate of corner 1, 1=Ycoordinate of corner 1, 2=Z coordinate of corner 1, 3=X coordinate of corner 2, 4=Ycoordinate of corner 2, 5=Z coordinate of corner 2...21=X coordinate of corner 8, 22=Y coordinate of corner 8, 23=Z coordinate of corner 8).
`BOOL qEndCap`	Draw the two end caps. Without end caps, it is a square tube.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	

Unpublished work. © 2025 Siemens

Example:
None

SymbolAddFillCylinder

SymbolAddFillCylinder **(iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fDirection, fHeight, nDividers, nSubDividers, qEndCap)**	
Description:	
This function adds a filled cylinder to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.
`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fDirection[0..2]`	Vector along cylinder.
`REAL8 fHeight`	Height of cylinder.
`INT4 nDividers`	Number of lines drawn along cylindrical surface.
`INT4 nSubDividers`	Number of sub-divisions along circle between `nDividers`.
`BOOL qEndCap`	Draw the two end caps. Without end caps, it is a cylindrical tube.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddFillCone

`SymbolAddFillCone` **`( iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fDirection, fHeight, nDividers, nSubDividers, qEndCap )`**	
Description:	
This function adds a filled cone to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.
`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fDirection[0..2]`	Vector normal to the circle.
`REAL8 fHeight`	Height of cone.
`INT4 nDividers`	Number of lines drawn along conical surface.
`INT4 nSubDividers`	Number of sub-divisions along circle between `nDividers`.
`BOOL qEndCap`	Draw the end cap.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The angle between the direction of the cone and the vector from the center to the point on circle does not have to be 90 degrees (it must not be 0 degrees). If the angle is not 90, the cone will be skewed. Internally, `nSubDividers` is increased until `nDividers` multiplied by `nSubDividers` is greater than or equal to 16. The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddFillTubeBrick

`SymbolAddFillTubeBrick` **`( iSymbolID, iAppearanceOffset, fOuterPoints, fInnerPoints )`**	
Description:	
This function adds a filled quad tube with thickness to a Symbol.	
Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fOuterPoints[0..23]`	Coordinates of outer 8 corners of a brick (i.e., 0=X coordinate of corner 1, 1=Ycoordinate of corner 1, 2=Z coordinate of corner 1, 3=X coordinate of corner 2, 4=Ycoordinate of corner 2, 5=Z coordinate of corner 2...21=X coordinate of corner

Unpublished work. © 2025 Siemens

	8, 22=Y coordinate of corner 8, 23=Z coordinate of corner 8).
`REAL8 fInnerPoints[0..23]`	Coordinates of inner 8 corners of a brick (i.e., 0=X coordinate of corner 1, 1=Ycoordinate of corner 1, 2=Z coordinate of corner 1, 3=X coordinate of corner 2, 4=Ycoordinate of corner 2, 5=Z coordinate of corner 2...21=X coordinate of corner 8, 22=Y coordinate of corner 8, 23=Z coordinate of corner 8).
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddFillTubeCylinder

`SymbolAddFillTubeCylinder`
`( iSymbolID, iAppearanceOffset, fCenter, fPointOnCircle, fDirection, fHeight, fInnerProportion, nDividers, nSubDividers )`
Description:
This function adds a filled cylindrical tube with thickness to a Symbol.
Input:

`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).

Unpublished work. © 2025 Siemens

`REAL8 fCenter[0..2]`	Coordinate of center of circle in Symbol space.
`REAL8 fPointOnCircle[0..2]`	Coordinate of point on circle in Symbol space.
`REAL8 fDirection[0..2]`	Vector along cylinder.
`REAL8 fHeight`	Height of cylinder.
`REAL8 fInnerProportion`	Proportion of inner to outer radius (should be less than 1.0).
`INT4 nDividers`	Number of lines drawn along cylindrical surface.
`INT4 nSubDividers`	Number of sub-divisions along circle between `nDividers`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

SymbolAddFillSphere

`SymbolAddFillSphere`

```
( iSymbolID, iAppearanceOffset, fCenter, fPointOnEquator, fDirection,
fAngle,
nLongDividers, nLongSubDividers, nLatDividers, nLatSubDividers )
```

Description:

This function adds a filled sphere to a Symbol.

Unpublished work. © 2025 Siemens

Input:	
`INT4 iSymbolID`	ID of Symbol to add.
`INT4 iAppearanceOffset`	Appearance offset (see remarks for function `SymbolCreate`).
`REAL8 fCenter[0..2]`	Coordinate of center of sphere in Symbol space.
`REAL8 fPointOnEquator[0..2]`	Coordinate of point on equator in Symbol space.
`REAL8 fDirection[0..2]`	Vector normal to the equator of the sphere.
`REAL8 fAngle`	Angle of revolution (360.0 is a sphere).
`INT4 nLongDividers`	Number of longitudinal lines drawn on sphere.
`INT4 nLongSubDividers`	Number of divisions between longitudinal lines.
`INT4 nLatDividers`	Number of latitudinal lines drawn on sphere.
`INT4 nLatSubDividers`	Number of divisions between latitudinal lines.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
Internally, `nLongSubDividers` and `nLatSubDividers` are increased until the angle for each subdivision is less than 22.5 degrees. The values of `nLongDividers`, `nLatDividers`, `nLongSubDividers` and `nLatSubDividers` control how smooth the surface is. The same variables are used as for `SymbolAddLineSphere` so the faceted lines and faceted filled surface line up. The z axis of the symbol is used as the direction of the symbol, so an arrow symbol would be drawn along the positive z direction.	
Example:	
None	

Unpublished work. © 2025 Siemens

Detach

Detach **(pDetachedName)**	
Description:	
This function detaches the graphics in a User Defined Graphics object. When the graphics are detached, they cannot be controlled and will persist as is, until the model is closed. The graphics data will no longer exist in the User Defined Graphics object. The User Defined Graphics object can continue to be used by adding new graphics data. The `pDetachedName` is copied to the detached graphics data to enable the graphics data to be reattached to a User Defined Graphics object using the Attach function. The main use of the `Detach`/`Attach` capability is to be able to have graphics survive after an API has finished running and to then access them again in another API. It is advised to not use the same `pDetachedName` for more than one `Detach` at a time as a call to `Attach` will not know which order to access them in.	
Input:	
`STRING pDetachedName`	See description.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

Attach

Unpublished work. © 2025 Siemens

Attach **(pDetachedName)**	
Description:	
This function attaches previously detached graphics to a User Defined Graphics object. When the graphics are attached, they will replace all graphics in the User Defined Graphics object at the point `Attach` is called. While graphics are attached, they will be lost when the User Defined Graphics object is destroyed unless `Detach` is called again. The graphics data to attach is specified by `pDetachedName`: if a previously detached graphics data has a matching `pDetachedName`, it will be attached to the User Defined Graphics object. The main use of the `Detach`/`Attach` capability is to be able to have graphics survive after an API has finished running and to then access them again in another API. It is advised to not use the same `pDetachedName` for more than one `Detach` at a time as a call to `Attach` will not know which order to access them in.	
Input:	
`STRING pDetachedName`	See description.
Output:	
None	None
Return Code:	
`FE_FAIL`	Call was unsuccessful.
Remarks/Usage:	
None	
Example:	
None	

CopyTool Objects

The `CopyTool` object offers programmatic access to the capabilities of the commands on the following menus: **Geometry** → **Copy...**; **Geometry** → **Rotate...**; **Geometry** → **Reflect...**; **Mesh** → **Copy...**(**Node** and **Element** commands only); **Mesh** → **Rotate...**; and **Mesh** → **Reflect...**

All of these commands have the same options in the **Numbering** and **Approach** sections of the dialog box. All of the mentioned *Geometry* commands offer the same options in the **Other Entities to Include** section, while all of the mentioned *Mesh* commands offer the same options in the **Other Entities to Include** section. In addition, all of the mentioned *Copy* commands share the same **Orient/Transform** section, while

Unpublished work. © 2025 Siemens

all of the mentioned *Rotate* commands share the same **Orient/Transform** options. Finally, all of the *Reflect* commands have the same option, **Trap Width**, in the **Reflection Options** section.

The `CopyTool` Object Properties are used to set options in the **Approach** section for all commands, along with the **Trap Width** option for the *Reflect* commands. The `CopyTool` Object Options Methods are used to specify options in the **Numbering and Other Entities to Include** sections, while the `CopyTool` Object Operation Methods are used to select the one of the available **Orient/Transform** options for *Copy* or *Rotate* commands, or the *Reflect* operation.

The appropriate process is to have the `CopyTool` Properties (see **CopyTool Object Properties**) and `CopyTool` Option Methods (See **CopyTool Object Options Methods**) specified as desired before calling one of the `CopyTool` Object Operation methods (See **CopyTool Object Operation Methods**).

Use the `feCopyTool` method of the Femap Application object to create `CopyTool` objects.

CopyTool Object Properties

Property	Description
`INT4 Repetitions`	Specifies number of Repetitions of the selected entities for the selection operation to create and specifies **Repetitions** value in the **Approach** section of Copy Options dialog box.
`BOOL CopyInSameLocation`	Flag controls **Copy in Same Location** option in the **Approach** section of Copy Options dialog boxes. When activated, a copy of each entity will be created in each entity's original location (i.e., creates coincident entities).
`BOOL AlwaysCreateParentCSys`	Flag controls **Always Create Parent CSys** option in the **Approach** section of Copy Options dialog boxes. When activated, a *parent* coordinate system is created and referenced by other coordinate systems created by an operation.
`BOOL CopyMeshSize`	Flag controls if Mesh Sizes on curves are copied along with geometry. By default, this option is set to True and Curve Mesh sizes are copied along with Geometry. If set to False, then mesh sizes will not be copied.
`BOOL UsePattern`	Flag controls option selected in the *Approach* section of the Copy Options dialog box. When True, *Pattern* is selected. When False, *Single/Multiple Copies* is selected.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 ReflectionTrapWidth`	Only used when using the `Reflect CopyTool` Operation method. Specifies the **Trap Width** in the **Reflection Options** section of the Copy Options dialog box. See **Reflect**.
`BOOL CopyToActiveLayer`	Flag controls **Assign to Active Layer** option in the **Parameters** section of the Copy Options dialog box. When True, newly created entities will be placed on the active layer. When False, they are placed on the layer of the original entity.
`BOOL CopyActiveColor`	Flag controls **Assign Active Color** option in the **Parameters** section of the Copy Options dialog box. When True, the option is turned on. When False, option is turned off.
`BOOL CopyToActiveCSys`	Flag controls **Assign to Active CSys** option in the **Parameters** section of the Copy Options dialog box. When True, the option is turned on. When False, option is turned off.
`BOOL AddToCopiedEntityGroups`	Flag controls **Add to Copied Entity Groups** option in the **Parameters** section of the Copy Options dialog box. When True, the option is turned on. When False, option is turned off.

CopyTool Object Options Methods

Clear

`Clear`	
`( Void )`	
Description:	
Clears everything in the `CopyTool` object.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Use to clear all values in the `CopyTool` object specified by the `CopyTool` Object Properties and `CopyTool` Object Options Methods. It may be a good idea to use this method to purge the object so no unwanted options are used, especially if performing a different operation (i.e. a *Copy* command after using *Rotate* command with a number of Repetitions.	
Example:	
None	

SetNumbering

`SetNumbering` **`( zCopyRenumMethod, nToBy )`**	
Description:	
Specifies option to use in the Numbering section, along with *To* or *By* value.	
Input:	
`enum zCopyReumMethod`	Numbering method to use for duplicated entities. In the **Numbering** section of Copy Options dialog box: Default = `FCPRNM_DEFAULT (0)` Block = `FCPRNM_BLOCK (1)` Offset = `FCPRNM_OFFSET (2)`
`INT4 nToBy`	When `zCopyReumMethod` is set to `FCPRNM_BLOCK`, used to specify To value. Sets By value when set to `FCPRNM_OFFSET`.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

SetVectorPattern

SetVectorPattern **(nPatternDirection, bOn, nNumberInPattern, dSpacing, vDirection)**	
Description:	
Specifies if *pattern direction* 1, 2, and/or 3 should be used when creating a pattern, as well as how many repetitions, spacing between duplicates, and direction vector for each pattern direction.	
Input:	
`INT4 nPatternDirection`	Pattern Direction. 0 = Direction 1, 1 = Direction 2, 2 = Direction 3.
`BOOL bOn`	When True, the pattern direction specified with `nPatternDirection` will be used when creating a pattern. When False, the pattern direction specified will not be considered.
`INT4 nNumberInPattern`	Number of repetitions to make in the direction specified in `nPatternDirection`.
`REAL8 dSpacing`	The distance between each duplicate created in the direction specified by `nPatternDirection`. The total distance between each instance in the pattern is the product of `dSpacing` and the magnitude of `vDirection`. `dSpacing` is only the actual distance if `vDirection` is specified as a unit vector.
`REAL8 vDirection[0..2]`	The XYZ components of the vector to create copies along the direction specified in `nPatternDirection`. Must be specified in Global Rectangular coordinates.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_INVALID`	Vector specified for `vDirection` is not valid or value < 1 has been defined for `nNumberInPattern`.
Remarks/Usage:	
Once all the desired pattern directions and options have been specified, use `CreatePattern` to create duplicates in the specified pattern.	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim copy As femap.CopyTool
   Set copy = App.feCopyTool

   Dim xyz1(2) As Double
   Dim xyz2(2) As Double
   Dim xyz3(2) As Double

   xyz1(0) = 1.0
   xyz1(1) = 0.0
   xyz1(2) = 0.0

   xyz2(0) = 0.0
   xyz2(1) = 1.0
   xyz2(2) = 0.0

   xyz3(0) = 0.0
   xyz3(1) = 0.0
```


Unpublished work. © 2025 Siemens

```
   xyz3(2) = 1.0


   rc = copy.SetVectorPattern (0, True, 2, 1.5, xyz1)
   rc = copy.SetVectorPattern (1, True, 2, 1.5, xyz2)
   rc = copy.SetVectorPattern (2, True, 2, 1.5, xyz3)


   copy.CreatePattern (FT_SOLID, -1)


End Sub
```

SetCSysPattern

SetCSysPattern

(nPatternDirection, bOn, nNumberInPattern, dSpacing, nCSysID)

Description:

Specifies if *pattern direction* 1, 2, and/or 3, should be used when creating a pattern, as well as how many repetitions, spacing between duplicates, and coordinate system used to define pattern directions.

Input:

`INT4 nPatternDirection`	Pattern Direction: 0 = Direction 1 (X Axis of coordinate system specified in `nCSysID`) 1 = Direction 2 (Y Axis of coordinate system specified in `nCSysID`). 2 = Direction 3 (Z Axis of coordinate system specified in `nCSysID`).
`BOOL bOn`	When True, the pattern direction specified with `nPatternDirection` will be used when creating a pattern. When False, the pattern direction specified will not be considered.
`INT4 nNumberInPattern`	Number of repetitions to make in the direction specified in `nPatternDirection`.

Unpublished work. © 2025 Siemens

`REAL8 dSpacing`	The distance between each duplicate created in the direction specified by `nPatternDirection`. The coordinate system axes are always considered as unit vectors so unlike the `SetVectorPattern` method, `dSpacing` is always the total distance between instances.
`INT4 nCSysID`	Coordinate System used to define the 3 pattern directions.
Output:	
None	None
Return Code:	
`FE_INVALID`	Value < 1 has been defined for `nNumberInPattern`.
`FE_NOT_EXIST`	Coordinate System specified does not exist.
Remarks/Usage:	
Once all the desired pattern directions and options have been specified, use `CreatePattern` to create duplicates in the specified pattern.	
Example:	

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim copy As femap.CopyTool
   Set copy = App.feCopyTool

   rc = copy.SetCSysPattern (0, True, 3, 1.5, 3)
   rc = copy.SetCSysPattern (1, True, 3, 1.5, 3)
   rc = copy.SetCSysPattern (2, True, 2, 2.5, 3)

   copy.CreatePattern(FT_SOLID, -1)
```


Unpublished work. © 2025 Siemens

```
End Sub
```

SetPropertyOption

SetPropertyOption **(zCopyRenumMethod, nOverrideID)**	
Description:	
Specifies which option is used to assign properties to duplicate elements (specified in **Properties/ Material Options** section) or assign properties to duplicate geometric entities when the original geometric entity already have mesh attributes assigned (specified in **Other Entities to Include** section).	
Input:	
`enum zCopyPropApproach`	Approach to use when assigning properties to duplicate elements or as mesh attributes on duplicate geometric entities: Same Properties = `FCPPR_KEEP (0)` Duplicate Properties = `FCPPR_DUP_PROP (1)` Duplicate Materials = `FCPPR_DUP_MATL (2)` Use Property = `FCPPR_OVERRIDE (3)`
`INT4 nOverrideID`	Property to use for duplicates when `zCopyPropApproach` is set to `FCPPR_OVERRIDE`. If type of specified property is not compatible with a duplicate, the duplicate maintains the original property.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Property specified for `nOverrideID` does not exist
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

IncludeOtherEntities

`IncludeOtherEntities` **`( bGeometry, bMesh, bLoads, bConstraints, bConnections )`**	
Description:	
This method has been superseded by `IncludeOtherEntities2`, which has an individual option specifically for coordinate systems. Specifies which other types of entities should be duplicated when finite element entities or geometric entities are duplicated (options are found in the **Other Entities to Include** section of **Copy Options**).	
Input:	
`BOOL bGeometry`	Flag to include geometric entities associated to finite element entities being duplicated (True) or not (False).
`BOOL bMesh`	Flag to include finite element entities associated to geometric entities being duplicated (True) or not (False).
`BOOL bLoads`	Flag to include loads applied to entities being duplicated (True) or not (False).
`BOOL bConstraints`	Flag to include constraints applied to entities being duplicated (True) or not (False).
`BOOL bConnections`	Flag to include regions which reference entities being duplicated (True) or not (False).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

IncludeOtherEntities2

IncludeOtherEntities2 **(bCoordSys, bGeometry, bMesh, bLoads, bConstraints, bConnections)**	
Description:	
Specifies which other types of entities should be duplicated when finite element entities or geometric entities are duplicated (options are found in the **Other Entities to Include** section of **Copy Options**). This method supersedes `IncludeOtherEntities`.	
Input:	
`BOOL bCoordSys`	Flag to include coordinate systems associated to finite element entities being duplicated (True) or not (False).
`BOOL bGeometry`	Flag to include geometric entities associated to finite element entities being duplicated (True) or not (False).
`BOOL bMesh`	Flag to include finite element entities associated to geometric entities being duplicated (True) or not (False).
`BOOL bLoads`	Flag to include loads applied to entities being duplicated (True) or not (False).
`BOOL bConstraints`	Flag to include constraints applied to entities being duplicated (True) or not (False).
`BOOL bConnections`	Flag to include regions which reference entities being duplicated (True) or not (False).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

AllOtherEntities

AllOtherEntities	
(bOn)	
Description:	
Specifies that all entity types which are associated with, referenced by, and applied to the entities being duplicated will also be duplicated.	
Input:	
`BOOL bOn`	Flag to include all entity types which are associated with, referenced by, and applied to the entities being duplicated (True) or not (False).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

CopyTool Object Operation Methods

AlongVector

AlongVector
(nDataType, nEntitySetID, vDirection)
Description:

Unpublished work. © 2025 Siemens

Copies entities of the specified type along a user-defined vector, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy.
`REAL8 vDirection[0..2]`	The XYZ components of the vector to create copies along. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

PointToPoint

`PointToPoint`
`( nDataType, nEntitySetID, vFrom, vTo )`
Description:
Copies entities of the specified type along a vector defined by from one user-defined location to another user-defined location, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods.
Input:

Unpublished work. © 2025 Siemens

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy.
`REAL8 vFrom[0..2]`	The XYZ coordinates of the location at the base of a vector to move along. Must be specified in Global Rectangular coordinates.
`REAL8 vTo[0..2]`	The XYZ coordinates of the location at the tip of a vector to move along. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

BetweenCSys

`BetweenCSys`
`( nDataType, nEntitySetID, nFromCSys, nToCSys )`
Description:
Copies entities of the specified type then orients the duplicate entities by aligning the XYZ axes of the *From* coordinate system to XYZ axes of the *To* coordinate system, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods.
Input:

Unpublished work. © 2025 Siemens

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied and aligned. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy and align.
`INT4 nFromCSys`	The ID of the Coordinate System to use as the *From* Coordinate System
`INT4 nToCSys`	The ID of the Coordinate System to use as the *To* Coordinate System.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

BetweenVectors

`BetweenVectors`
`( nDataType, nEntitySetID, vFromBase, vFromDir, vToBase, vToDir )`
Description:
Copies entities of the specified type then orients the duplicate entities by moving the *From Base* location to the *To Base* location, then aligning the vector specified by the *From* coordinates to the vector specified by *To* coordinates, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods.
Input:

Unpublished work. © 2025 Siemens

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied and aligned. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy and align.
`REAL8 vFromBase[0..2]`	The XYZ coordinates of the location at the base of a vector to align from. Must be specified in Global Rectangular coordinates.
`REAL8 vFromDir[0..2]`	The components of the vector to align from during copy. Must be specified in Global Rectangular coordinates.
`REAL8 vToBase[0..2]`	The XYZ coordinates of the location at the base of a vector to align to. Must be specified in Global Rectangular coordinates.
`REAL8 vToDir[0..2]`	The components of the vector to align to during copy. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

BetweenPlanes

Unpublished work. © 2025 Siemens

BetweenPlanes

(nDataType, nEntitySetID, vFromBase, vFromNormal, vFromX, vToBase, vToNormal, vToX)

Description:

Copies entities of the specified type then orients the duplicate entities by aligning the *From* plane to the *To* plane, including the normal direction of the planes, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods.

Input:

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied and aligned. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy and align.
`REAL8 vFromBase[0..2]`	The XYZ coordinates of location in the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vFromNormal[0..2]`	The components of a vector normal to the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vFromX[0..2]`	The components of a vector that specifies the "X"/ control direction within the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToBase[0..2]`	The XYZ coordinates of location in the plane to align to during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToNormal[0..2]`	The components of a vector normal to the plane to align to during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToX[0..2]`	The components of a vector that specifies the "X"/ control direction within the plane to align to during alignment. Must be specified in Global Rectangular coordinates.

Output:

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

RotateAroundVector

`RotateAroundVector`
`( nDataType, nEntitySetID, vAroundBase, vAroundDir, dAngle, dDistanceAlong )`
Description:
Copies entities of the specified type, then rotates them around a user-defined vector by a specified amount, in degrees, and optionally along the user-defined vector a specified distance, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods.
Input:

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied and rotated/moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy and rotate/move.
`REAL8 vAroundBase[0..2]`	The XYZ coordinates of the location at the base of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vAroundDir[0..2]`	The components of a vector to rotate around. Must be specified in Global Rectangular coordinates.

Unpublished work. © 2025 Siemens

`REAL8 dAngle`	The angle of rotation, in degrees.
`REAL8 dDistanceAlong`	The total distance along the axis of rotation, typically zero.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

RotatePointToPoint

`RotatePointToPoint`
`( nDataType, nEntitySetID, vAroundBase, vAroundDir, vFrom, vTo )`
Description:
Copies entities of the specified type, then rotates them around a user-defined from one location to another location (locations define both angle of rotation and distance along the user-defined vector), using the options specified on the `CopyTool` object by the `CopyTool` Object Properties and `CopyTool` Object Options Methods.
Input:

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be copied and rotated/moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy and rotate/move.

Unpublished work. © 2025 Siemens

`REAL8 vAroundBase[0..2]`	The XYZ coordinates of the location at the base of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vAroundDir[0..2]`	The components of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vFrom[0..2]`	The XYZ coordinates of the location to rotate/move from. Must be specified in Global Rectangular coordinates.
`REAL8 vTo[0..2]`	The XYZ coordinates of the location to rotate/ move to. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

Reflect

`Reflect`	
`( nDataType, nEntitySetID, vAcrossBase, vAcrossDir )`	
Description:	
Copies entities of the specified type by reflecting them across the specified plane, using the options specified on the `CopyTool` object by the `CopyTool` Object Properties, including `ReflectionTrapWidth`, and `CopyTool` Object Options Methods.	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`,

Unpublished work. © 2025 Siemens

	FT_CURVE, FT_SURFACE, FT_SOLID, FT_NODE, or FT_ELEM.
INT4 nEntitySetID	The ID of a set that contains a list of entity IDs that will be copied and reflected. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy and reflect.
REAL8 vAcrossBase[0..2]	The XYZ coordinates of the location at the base of the reflection plane. Must be specified in Global Rectangular coordinates.
REAL8 vAcrossDir[0..2]	The components of a vector normal to the reflection plane. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
FE_BAD_TYPE	Invalid entity type. Must specify one of the types listed in nDataType.
Remarks/Usage:	
None	
Example:	
None	

InPlace

InPlace
(nDataType, nEntitySetID)
Description:
Copies entities of the specified type *in place*, using the options specified on the CopyTool object by the CopyTool Object Properties and CopyTool Object Options Methods.
Input:

INT4 nDataType	The type of entities to be copied. For entity information, see Entity Types. Can be FT_POINT,

Unpublished work. © 2025 Siemens

	FT_CURVE, FT_SURFACE, FT_SOLID, FT_NODE, or FT_ELEM.
INT4 nEntitySetID	The ID of a set that contains a list of entity IDs that will be copied in place. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy in place.
Output:	
None	None
Return Code:	
FE_BAD_TYPE	Invalid entity type. Must specify one of the types listed in nDataType.
Remarks/Usage:	
None	
Example:	
None	

CreatePattern

CreatePattern
(nDataType, nEntitySetID)
Description:
Creates copies using the pattern specified by the SetVectorPattern and/or SetCSysPattern CopyTool Object Options methods and other options specified by various CopyTool Object Options Methods.
Input:

INT4 nDataType	The type of entities to be copied. For entity information, see **Entity Types**. Can be FT_POINT, FT_CURVE, FT_SURFACE, FT_SOLID, FT_NODE, or FT_ELEM.
INT4 nEntitySetID	The ID of a set that contains a list of entity IDs that will be copied to create a pattern. Alternatively, if you specify a negative value, this is simply the ID of the single entity to copy to create a pattern.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
FE_BAD_TYPE	Invalid entity type. Must specify one of the types listed in nDataType.
Remarks/Usage:	
Either SetVectorPattern or SetCSysPattern must be set in at least one direction, otherwise coincident entities will be created.	
Example:	
None	

MoveTool Objects

The MoveTool object offers programmatic access to the capabilities of the **Modify → Move By → Point**; **Modify → Move By → Curve**; **Modify → Move By → Surface**; **Modify → Move By → Solid**; **Modify → Move By → CSys**; **Modify → Move By → Node**; and **Modify → Move By → Element** commands, along with all the commands on the **Modify → Rotate By...**; **Modify → Reflect...**; and **Modify → Align...** menus.

All of the mentioned commands regarding *Geometry* offer the same option in the **Other Entities to Include** section (**Mesh**), while all of the mentioned commands regarding **Mesh** offer the same option in the **Other Entities to Include** section (**Geometry**). In addition, all of the mentioned commands regarding *Coordinate Systems* offer the same option in the **Other Entities to Include** section (**Geometry** and **Mesh**).

In addition, all of the mentioned *Move By* commands share the same **Orient/Transform** section, while all of the mentioned *Rotate By* commands share the same **Orient/Transform** options, as do all of the mentioned *Align* commands. Finally, all of the *Reflect* commands have the same option, **Trap Width**, in the **Reflection Options** section.

The appropriate process is to have the MoveTool Properties (see **MoveTool Object Properties**) and MoveTool Option Methods (See **MoveTool Object Options Methods**) specified as desired before calling one on the MoveTool Object Operation methods (See **MoveTool Object Operation Methods**).

Use the feMoveTool method of the Femap Application object to create MoveTool objects.

MoveTool Object Properties

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AlwaysCreateParentCSys`	Flag controls **Always Create Parent CSys** option in the **Approach** section of Move Options dialog boxes. When activated, a *parent* coordinate system is created and referenced by other coordinate systems created by an operation.
`REAL8 ReflectionTrapWidth`	Only used when using the `Reflect MoveTool` Operation method. Specifies the **Trap Width in the Reflection Options** section of the Move Options dialog box. See **Reflect**.

MoveTool Object Options Methods

Clear

`Clear` **`( Void )`**	
Description:	
Clears everything in the `MoveTool` object.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Use to clear all values in the `MoveTool` object specified by the `MoveTool` Object Properties and `MoveTool` Object Options Methods. It may be a good idea to use this method to purge the object so no unwanted options are used, especially if performing a different operation (i.e. a *Move By* command after using *Rotate By* command).	

Unpublished work. © 2025 Siemens

Example:
None

IncludeOtherEntities

IncludeOtherEntities **(bGeometryOrMesh)**	
Description:	
This method has been superseded by `IncludeOtherEntities2`, which has an individual option for coordinate systems. Specifies which other types of entities should be moved/rotated/reflected/aligned when finite element entities, geometric entities, or coordinate systems are moved/rotated/reflected/aligned (option is found in the **Other Entities to Include** section of Move Options).	
Input:	
`BOOL bGeometryOrMesh`	Flag to include geometric entities associated to finite element entities being moved/rotated/reflected/aligned or to include finite element entities associated to geometric entities being moved/rotated/reflected/aligned (True) or not (False). For coordinate systems, both geometry and mesh will be being moved/rotated/reflected/aligned when this argument is set to True.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

IncludeOtherEntities2

Unpublished work. © 2025 Siemens

IncludeOtherEntities2 **(bCoordSys, bGeometryOrMesh)**	
Description:	
Specifies which other types of entities should be moved/rotated/reflected/aligned when finite element entities, geometric entities, and/or coordinate systems are moved/rotated/reflected/aligned (option is found in the **Other Entities to Include** section of Move Options). This method supersedes `IncludeOtherEntities`.	
Input:	
`BOOL bCoordSys`	Flag to include moving coordinate systems associated with entities when moving geometry, mesh, or coordinate systems.
`BOOL bGeometryOrMesh`	Flag to include geometric entities associated to finite element entities being moved/rotated/reflected/aligned or to include finite element entities associated to geometric entities being moved/rotated/reflected/aligned (True) or not (False). For coordinate systems, both geometry and mesh will be being moved/rotated/reflected/aligned when this argument is set to True.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

MoveTool Object Operation Methods

AlongVector

Unpublished work. © 2025 Siemens

AlongVector **(nDataType, nEntitySetID, vDirection)**	
Description:	
Moves entities of the specified type along a user-defined vector, using the options specified on the MoveTool object by the `MoveTool` Object Properties and `MoveTool` Object Options Methods.	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`REAL8 vDirection[0..2]`	The XYZ components of the vector to move along. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

PointToPoint

Unpublished work. © 2025 Siemens

PointToPoint	
(nDataType, nEntitySetID, vFrom, vTo)	
Description:	
Moves entities of the specified type along a vector defined by from one user-defined location to another user-defined location, using the options specified on the MoveTool object by the `MoveTool` Object Properties and `MoveTool` Object Options Methods.	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`REAL8 vFrom[0..2]`	The XYZ coordinates of the location at the base of a vector to move along. Must be specified in Global Rectangular coordinates.
`REAL8 vTo[0..2]`	The XYZ coordinates of the location at the tip of a vector to move along. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

BetweenCSys

Unpublished work. © 2025 Siemens

BetweenCSys **(nDataType, nEntitySetID, nFromCSys, nToCSys)**	
Description:	
Reorients entities of the specified type by aligning the XYZ axes of the *From* coordinate system to XYZ axes of the *To* coordinate system, using the options specified on the MoveTool object by the MoveTool Object Properties and MoveTool Object Options Methods.	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`INT4 nFromCSys`	The ID of the Coordinate System to use as the *From* Coordinate System.
`INT4 nToCSys`	The ID of the Coordinate System to use as the *To* Coordinate System.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

BetweenVectors

Unpublished work. © 2025 Siemens

BetweenVectors

(nDataType, nEntitySetID, vFromBase, vFromDir, vToBase, vToDir)

Description:

Reorients entities of the specified type by moving the *From Base* location to the *To Base* location, then aligning the vector specified by the *From* coordinates to the vector specified by *To* coordinates, using the options specified on the MoveTool object by the MoveTool Object Properties and MoveTool Object Options Methods.

Input:

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`REAL8 vFromBase[0..2]`	The XYZ coordinates of the location at the base of a vector to align from. Must be specified in Global Rectangular coordinates.
`REAL8 vFromDir[0..2]`	The components of the vector to align from during move. Must be specified in Global Rectangular coordinates.
`REAL8 vToBase[0..2]`	The XYZ coordinates of the location at the base of a vector to align to. Must be specified in Global Rectangular coordinates.
`REAL8 vToDir[0..2]`	The components of the vector to align from during move. Must be specified in Global Rectangular coordinates.

Output:

None	None

Return Code:

`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

BetweenPlanes

BetweenPlanes	
(nDataType, nEntitySetID, vFromBase, vFromNormal, vFromX, vToBase, vToNormal, vToX)	
Description:	
Reorients entities of the specified type by aligning the *From* plane to the *To* plane, including the normal direction of the planes, using the options specified on the MoveTool object by the MoveTool Object Properties and MoveTool Object Options Methods.	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`REAL8 vFromBase[0..2]`	The XYZ coordinates of location in the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vFromNormal[0..2]`	The components of a vector normal to the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vFromX[0..2]`	The components of a vector that specifies the "X"/ control direction within the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToBase[0..2]`	The XYZ coordinates of location in the plane to align to during alignment. Must be specified in Global Rectangular coordinates.

Unpublished work. © 2025 Siemens

`REAL8 vToNormal[0..2]`	The components of a vector normal to the plane to align to during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToX[0..2]`	The components of a vector that specifies the "X"/ control direction within the plane to align to during alignment. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

RotateAroundVector

`RotateAroundVector`
`( nDataType, nEntitySetID, vAroundBase, vAroundDir, dAngle, dDistanceAlong )`
Description:
Rotates entities of the specified type around a user-defined vector by a specified amount, in degrees, and optionally along the user-defined vector a specified distance, using the options specified on the MoveTool object by the MoveTool Object Properties and MoveTool Object Options Methods.
Input:

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a

Unpublished work. © 2025 Siemens

	negative value, this is simply the ID of the single entity to move.
`REAL8 vAroundBase[0..2]`	The XYZ coordinates of the location at the base of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vAroundDir[0..2]`	The components of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 dAngle`	The angle of rotation, in degrees.
`REAL8 dDistanceAlong`	The total distance along the axis of rotation, typically zero.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

RotatePointToPoint

`RotatePointToPoint`
`( nDataType, nEntitySetID, vAroundBase, vAroundDir, vFrom, vTo )`
Description:
Rotates entities of the specified type around a user-defined from one location to another location (locations define both angle of rotation and distance along the user-defined vector), using the options specified on the MoveTool object by the MoveTool Object Properties and MoveTool Object Options Methods.
Input:

`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`,

Unpublished work. © 2025 Siemens

	FT_CURVE, FT_SURFACE, FT_SOLID, FT_NODE, or FT_ELEM.
INT4 nEntitySetID	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
REAL8 vAroundBase[0..2]	The XYZ coordinates of the location at the base of a vector to rotate around. Must be specified in Global Rectangular coordinates.
REAL8 vAroundDir[0..2]	The components of a vector to rotate around. Must be specified in Global Rectangular coordinates.
REAL8 vFrom[0..2]	The XYZ coordinates of the location to rotate/move from. Must be specified in Global Rectangular coordinates.
REAL8 vTo[0..2]	The XYZ coordinates of the location to rotate/move to. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
FE_BAD_TYPE	Invalid entity type. Must specify one of the types listed in nDataType.
Remarks/Usage:	
None	
Example:	
None	

Reflect

Reflect
(nDataType, nEntitySetID, vAcrossBase, vAcrossDir)
Description:

Unpublished work. © 2025 Siemens

Reflects entities of the specified type by reflecting them across the specified plane, using the options specified on the MoveTool object by the MoveTool Object Properties, including `ReflectionTrapWidth`, and MoveTool Object Options Methods.	
Input:	
`INT4 nDataType`	The type of entities to be copied. For entity information, see **Entity Types**. Can be `FT_POINT`, `FT_CURVE`, `FT_SURFACE`, `FT_SOLID`, `FT_NODE`, or `FT_ELEM`.
`INT4 nEntitySetID`	The ID of a set that contains a list of entity IDs that will be moved. Alternatively, if you specify a negative value, this is simply the ID of the single entity to move.
`REAL8 vAcrossBase[0..2]`	The XYZ coordinates of the location at the base of the reflection plane. Must be specified in Global Rectangular coordinates.
`REAL8 vAcrossDir[0..2]`	The components of a vector normal to the reflection plane. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Invalid entity type. Must specify one of the types listed in `nDataType`.
Remarks/Usage:	
None	
Example:	
None	

MergeTool Objects

The `MergeTool` object offers programmatic access to the capabilities of the **File** → **Merge** command.

The `MergeTool` Object Properties are used to set options in the Options section of the Model Merge Manager dialog box and some additional settings when creating a Pattern. The `MergeTool` Object Options Methods are used to specify option in other sections of the Model Merge Manager, while the `MergeTool` Object Operation Methods are used to select the one of option available in the **Orientation/ Transform** section of the Model Merge Manager dialog box.

Unpublished work. © 2025 Siemens

The appropriate process is to have the `MergeTool` Properties (see **MergeTool Object Properties**), `MergeTool` Model Selection Methods (See **MergeTool Object Model Selection Methods**), and `MergeTool` Entity Selection and Status Methods (**MergeTool Object Entity Selection and Status Methods**) specified as desired before calling one of the `MergeTool` Object Operation methods (See **MergeTool Object Operation Methods**).

Use the `feMergeTool` method of the Femap Application object to create `MergeTool` objects.

MergeTool Object Properties

Property	Description
`BOOL CreateGroupForMerged`	Flag controls **Create Group for Merged Model** option in the **Options** section of the Model Merge Manager dialog box. When activated, creates a group containing all of the entities merged from the **Entities to Merge** list into the active or current model.
`BOOL AlwaysCreateParentCSys`	Flag controls **Create Parent CSys for Merged Model** option in the **Options** section of Model Merge Manager dialog box. When activated, creates a *parent* Coordinate Systems in the active or current model for the entities in the **Entities to Merge** list.
`BOOL CondenseMergedGroups`	Flag controls **Condense Transferred Groups** option in the **Options** section of Model Merge Manager dialog box. When activated, any groups brought into the active or current model from the **Entities to Merge** list will be *condensed*.
`BOOL LimitToMergedEntities`	Flag controls **Limit Loads, Constraints and Contact to Merged Entities** option in the **Options** section of Model Merge Manager dialog box. When activated, will only transfer loads, boundary conditions, and connection entities from the **From Model** or current model when the entities referenced by these entity types are also being merged into the active or current model.
`BOOL KeepInOriginalSets`	Flag controls **Keep Loads and Constraints in Original Sets** option in the **Options** section of Model Merge Manager dialog box. When activated, which is the default when using the **Copy in Current Model** option, load sets and constraint sets will not be renumbered.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL UsePattern`	Flag controls if a *pattern* of merge entities is created. When True, a *pattern* of merged entities will be created based on the settings for `SetVectorPattern`, `SetCSysPattern`, and `CreatePattern`.
`BOOL DuplicateFirstInPattern`	Flag controls if the original entities being duplicated in a *pattern* will also be duplicated.

MergeTool Object Model Selection Methods

Clear

`Clear` **`( Void )`**	
Description:	
Clears everything in the MergeTool object.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Use to clear all values in the MergeTool object specified by the MergeTool Object Properties and MergeTool Object Options Methods. It may be a good idea to use this method to purge the object so no unwanted options are used, especially if performing a different Orientation/Transform operation (i.e. a *Copy* after using *Rotate* or setting a different *pattern*). Calling this method also clears the selected model to be merged. You will have to reselect a model prior to calling Merge again.	
Example:	
None	

Unpublished work. © 2025 Siemens

SelectModel

SelectModel **(nModelID)**	
Description:	
Specifies the **From Model in Merge/Extract** section of the Model Merge Manager dialog box using the ID of the model.	
Input:	
`INT8 nModelID`	The ID of the Model to be selected.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The model specified is not open in the current session.
`FE_FAIL`	Unable to merge entities from the specified model.
Remarks/Usage:	
Calling this method not only selects the model to be merged, it also scans the model and automatically sets up merging and renumbering information for the entities in the model. In many cases, if you wanted to merge this entire model, you will not need to call any further status or selection methods prior to merging.	
Example:	
None	

SelectModelByName

Unpublished work. © 2025 Siemens

SelectModelByName

(fName)

Description:

Specifies the **From Model in Merge/Extract** section of the Model Merge Manager dialog box using the full name or partial name of the model.

Input:

`STRING fName`	The name of the Model to be selected. To work properly under all conditions, this string should contain the full pathname to the model file. For example, *C:\MyModels\MyModel.modfem*. If you specify a string that is less than the full path name the model will still be selected as long as only one open model matches the string that you specify. Unsaved models can be selected by specifying an empty string, but again, only if there is only one model that is unsaved.

Output:

None	None

Return Code:

`FE_NOT_AVAILABLE`	The model specified is not open in the current session.
`FE_FAIL`	Unable to merge entities from the specified model.

Remarks/Usage:

Calling this method not only selects the model to be merged, it also scans the model and automatically sets up merging and renumbering information for the entities in the model. In many cases, if you wanted to merge this entire model, you will not need to call any further status or selection methods prior to merging.

While it is possible to use only a portion of the model name to specify the From Model, enough of the title must be used to identify one and only model, otherwise the call will return `FE_FAIL`. If the model has not yet been saved, it will not have a name, entering “Untitled” can potentially work, but is not recommended. You can use `feAppGetAllModels` to retrieve the names of all open models.

Example:

None

Unpublished work. © 2025 Siemens

SelectCurrentModel

`SelectCurrentModel`
 	
`( Void )`	

Description:	
Sets the **Copy in Current** option in the **Merge/Extract** section of the Model Merge Manager dialog box.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The model specified is not open in the current session.
`FE_FAIL`	Unable to merge entities from the specified model.
Remarks/Usage:	
Calling this method not only selects the model to be merged, it also scans the model and automatically sets up merging and renumbering information for the entities in the model. In many cases, if you wanted to merge this entire model, you will not need to call any further status or selection methods prior to merging.	
Example:	
None	

MergeTool Object Entity Selection and Status Methods

GetEntityTypes

Unpublished work. © 2025 Siemens

GetEntityTypes

(nNumTypes, nAvailableTypes)

Description:

Returns available entity types found in the **From Model** or the current model if the **Copy in Current Model** option is specified via `SelectCurrentModel`.

Input:

`INT4 nNumTypes`	Number of available entity types in the **From Model**.
`INT4 nAvailableTypes[0.. nNumTypes -1]`	Available entity types in **From Model**. For entity information, see **Entity Types**.

Output:

None	None

Return Code:

`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_FAIL`	Unable to retrieve the array of available entity types.

Remarks/Usage:

If an entity type is not returned, then none of those entities exist in the **From Model**.

Example:

```
Sub Main

   Dim App as femap.model

   Set App = feFemap()

   Dim mt As femap.MergeTool

   Set mt = App.feMergeTool

   Dim numAvail As Long

   Dim EntTypes As Variant

    'Must specify name of model which is already open in FEMAP

   mt.SelectModelByName ("Test")
```


Unpublished work. © 2025 Siemens

```
   mt.GetEntityTypes (numAvail, EntTypes)

End Sub
```

GetEntityStatus

GetEntityStatus **(entityType, bStatus)**	
Description:	
Retrieves the *status* of a entity type. The status specifies if the entity type will be merged or not into the **To Model**.	
Input:	
`INT4 entityType`	Entity type to retrieve status. Use enumerator `zDataType` to select from available entity types. For entity information, see **Entity Types**.
Output:	
`BOOL bStatus`	If True, entity type will be merged into the **To Model**. If False, entity type will not be merged into the **To Model**.
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_NOT_AVAILABLE`	The entity type specified does not exist in the model to be merged.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetEntityOptions

GetEntityOptions

(entityType, nFromSetID, nToSetID, nRenumStrategy, RenumStartAt, bRenumberDuplicates)

Description:

Retrieves the *status* of the renumbering and duplicate options for a particular entity type.

Input:

`INT4 entityType`	The type of entity to be merged. Use enumerator `zDataType` to select from available entity types. For entity information, see **Entity Types**.
`INT4 nFromSetID`	The ID of a Set object that will be filled with the IDs of the entities of the selected type that will be merged. **Note** This Set object must have been created when the current model was active even though it will contain IDs from the model being merged.
`INT4 nToSetID`	The ID of a Set object that will be filled with the IDs that merged entities will be renumbered to when merged into the current model. **Note** This Set object must have been created when the current model was active.

Output:

`INT4 nRenumStrategy`	Current renumbering option specified for a particular entity type. Use enumerator `zMergeRenumMethod`. None = 0 (`FMRRNM_NONE`) Minimal Renumbering = 1 (`FMRRNM_MINIMAL`) Block Renumbering = 2 (`FMRRNM_BLOCK`) Offset Renumbering = 3 (`FMRRNM_OFFSET`)

Unpublished work. © 2025 Siemens

	Compress = 4 (FMRRNM_COMPRESS)
`INT4 RenumStartAt`	Current ID specified for Renumber To for Minimal Renumbering, Block Renumbering, and Compress or Offset By for Offset Renumbering.
`BOOL bRenumberDuplicates`	Flag indicating if Renumber Duplicates (True) or Overwrite Duplicates (False) is currently specified.
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_NOT_AVAILABLE`	The entity type specified does not exist in the model to be merged.
Remarks/Usage:	
None	
Example:	
None	

SetAllStatus

`SetAllStatus` **`( bStatus )`**	
Description:	
For the all entity types sets the option to merge or not merge the type.	
Input:	
`BOOL bStatus`	If True, all available entity types will be merged. If False, no types will be merged.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.

Remarks/Usage:

Normally this method is used to turn off merging of all entity types prior to selecting a few individual entity types to be merged. Since the status of most entity types is automatically set to True when a model is selected, it is often not necessary to call this method.

Example:

None

SetStatus

`SetStatus`

`( bGeometry, bModel, bConnection, bAeroElasticity, bAnalysisAndSimulation, bResults, bOthers )`

Description:

For categories of entity types sets the option to merge or ignore the type

Input:

`BOOL bGeometry`	If True, geometry types will be merged. This includes: Points, Curves, Surfaces, Solids and Mesh Points.
`BOOL bModel`	If True, model types will be merged. This includes: Coordinate Systems, Nodes, Elements, Materials, Properties, Layups, Global Plies, Load Sets, Constraint Sets, Functions, Tables and Data Surfaces.
`BOOL bConnection`	If True, connection types will be merged. This includes: Connection Properties, Regions and Connections.
`BOOL bAeroElasticity`	If True, aeroelasticity types will be merged. This includes: Aero Panels, Properties, Splines and Surfaces.
`BOOL bAnalysisAndSimulation`	If True, analysis and simulation types will be merged. This includes: Analysis Manager Sets,

Unpublished work. © 2025 Siemens

	Design Optimization Data, Monitor Points and Direct Matrix Input Data.
`BOOL bResults`	If True, analysis result types will be merged. This includes: Analysis Studies and Result Sets.
`BOOL bOthers`	If True, all other types will be merged. This includes: Text, Groups, Layers, Views and more.
Output:	
None	None
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
None	None
Remarks/Usage:	
None	
Example:	
None	

SetEntityStatus

`SetEntityStatus`	
`( entityType, bStatus )`	
Description:	
For the specified entity type, sets the option to merge or ignore the type	
Input:	
`INT4 entityType`	Entity type to set status. Use enum zDataType to select from available entity types. For entity information, see **Entity Types**.
`BOOL bStatus`	If True, the selected entity type will be merged. If False, that type will not be merged.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
FE_NO_FILENAME	No model has been selected to merge. You must always select the model to merge first before other methods.
FE_NOT_AVAILABLE	The entity type specified does not exist in the model to be merged.

Remarks/Usage:

None

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim mt As femap.MergeTool
   Set mt = App.feMergeTool
   ‘Select the From Model by name "wingpost"
   mt.SelectModelByName("wingpost")
   mt.SetStatus(False, False, False, False, False, False, False)

   mt.SetEntityStatus(FT_NODE, True)

   mt.Merge( )

End Sub
```

SelectEntity

Unpublished work. © 2025 Siemens

SelectEntity

**(bThisEntityOnly, entityType, nFromSetID, bSetInCurrentModel, nRenumStrategy,
nRenumSrartAt, bRenumberDuplicates)**

Description:

Selects a Set of entities of a specified type to be merged. Also specifies the renumbering and overwriting options for that entity type.

Input:

`BOOL bThisEntityOnly`	If True, automatically deselects all other entity types, which is the same as calling:
	
	`SetAllStatus(False)`
	
	`SelectEntity(False, …)`

`INT4 entityType`	The type of entities to be merged. Use enum `zDataType` to select from available entity types. For entity information, see **Entity Types**.
`INT4 nFromSetID`	The ID of a Set object that contains the IDs of the entities to be merged. This Set object can be created in either model (see `bSetInCurrentModel` argument), but must contain the IDs of entities in the model being merged.
`BOOL bSetInCurrentModel`	If True, the `nFromSetID` Set object must have been created when the current model was active. If False, it must have been created when the model being merged was active. Set objects are linked to the model that is active when they are created, not when IDs are added, so it is important to be careful which model is active when creating objects.
`INT4 nRenumStrategy`	Current renumbering option specified for a particular entity type. Use enum `zMergeRenumMethod`. None = 0 (`FMRRNM_NONE`) Minimal Renumbering = 1 (`FMRRNM_MINIMAL`) Block Renumbering = 2 (`FMRRNM_BLOCK`) Offset Renumbering = 3 (`FMRRNM_OFFSET`) Compress = 4 (`FMRRNM_COMPRESS`)

Unpublished work. © 2025 Siemens

`INT4 RenumStartAt`	Current ID specified for Renumber To for Minimal Renumbering, Block Renumbering, and Compress or Offset By for Offset Renumbering.
`BOOL bRenumberDuplicates`	Flag indicating if Renumber Duplicates (True) or Overwrite Duplicates (False) is currently specified.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_NOT_AVAILABLE`	The entity type specified has not been selected to be merged.
`FE_NOT_EXIST`	No entities of the specified type exist in the model to be merged.
Remarks/Usage:	
Calling this method will automatically turn on the status for the specified entity type. There is no need for an additional call to `SetEntityStatus`. Selecting single entity types via this method does not ensure that you will be able to merge a valid model or portion of a model. Unless you really just want what you are selecting, you will probably need to call `AddRelated` prior to merging.	
Example:	

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim mt As femap.MergeTool
   Set mt = App.feMergeTool
   Dim ndSet As femap.Set
   Set ndSet = App.feSet
   ‘Set a model by name "wingpost"
   App.feAppSetModelByName("wingpost")
   ndSet.Select(FT_NODE, True, "Select Nodes to Merge")
   ‘Set a different model by name "MergeModel"
```


Unpublished work. © 2025 Siemens

```
   App.feAppSetModelByName("MergeModel")

   ‘Select the From Model by name "wingpost"

   mt.SelectModelByName("wingpost")

    mt.SelectEntity(True,FT_NODE,ndSet.ID,False,FMRRNM_OFFSET, 100000,
True)


   mt.Merge( )


End Sub
```

SelectFromGroup

SelectFromGroup

**(bThisGroupOnly, bAllEentityTypes, entityType, nFromGroupID, nRenumStrategy,
 nRenumSrartAt, bRenumberDuplicates)**

Description:

Selects entities of one or more types that are included in a Group in the model to be merged. Also specifies the renumbering and overwriting options for those entity types.

Input:

`BOOL bThisGroupOnly`	If True, automatically deselects all entity types other than the ones selected from the group, which is the same as calling:
	`SetAllStatus(False)`
	`SelectFromGroup(False, …)`

`BOOL bAllEntityTypes`	If True, all entity types that are referenced in the Group are selected. If False, only the type specified by entityType is selected.

Unpublished work. © 2025 Siemens

`INT4 entityType`	Only used if `bAllEntityTypes=False`. Use enum `zDataType` to select from available entity types. For entity information, see **Entity Types**.
`INT4 nFromGroupID`	The ID of a Group that is defined in the model being merged.
`INT4 nRenumStrategy`	Current renumbering option specified for a particular entity type. Use enum `zMergeRenumMethod`. None = 0 (`FMRRNM_NONE`) Minimal Renumbering = 1 (`FMRRNM_MINIMAL`) Block Renumbering = 2 (`FMRRNM_BLOCK`) Offset Renumbering = 3 (`FMRRNM_OFFSET`) Compress = 4 (`FMRRNM_COMPRESS`)
`INT4 RenumStartAt`	Current ID specified for Renumber To for Minimal Renumbering, Block Renumbering, and Compress or Offset By for Offset Renumbering.
`BOOL bRenumberDuplicates`	Flag indicating if Renumber Duplicates (True) or Overwrite Duplicates (False) is currently specified.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_NOT_AVAILABLE`	The entity type specified does not exist in the model to be merged.
`FE_NOT_EXIST`	No entities of the specified type exist in the model to be merged.
Remarks/Usage:	
Calling this method will automatically turn on the status for the specified entity types. There is no need for additional calls to `SetEntityStatus`. Selecting single entity types or even entire groups via this method does not ensure that you will be able to merge a valid model or portion of a model. Unless you really just want what you are selecting, you will probably need to call `AddRelated` prior to merging.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddRelated

AddRelated

(bAddAssociated)

Description:

Selects all entities that are related (used by) the entities that are already selected to be merged.

Input:

`BOOL bAddAssociated`	If True, additional associated entities will be added to the selection. For example, if you have selected surfaces, then if this is True any nodes and elements meshed on those surfaces are considered to be *associated* and will be included.

Output:

None	None

Return Code:

`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.

Remarks/Usage:

Use of this method is usually required if you are adjusting the entity selection to ensure that a complete/valid portion of your model will be merged. Otherwise, you can end up merging entities that still reference other entities that do not exist in the current model.

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim mt As femap.MergeTool
   Set mt = App.feMergeTool
   'Select From Model by name "wingpost"
   mt.SelectModelByName("wingpost")
   mt.SetStatus(False, False, False, False, False, False, False)
```


Unpublished work. © 2025 Siemens

```
   mt.SetEntityStatus(FT_ELEM, True)

   mt.AddRelated(True)

   mt.Merge( )

End Sub
```

ShowDialog

ShowDialog	
(Void)	
Description:	
Displays the same dialog used by the **File** → **Merge** command	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_CANCEL`	The dialog box was canceled.
Remarks/Usage:	
You can use this method to display the dialog at any time. Any selections or changes to options that you do prior to calling this method will be shown in the dialog when it is displayed, and changes in the dialog will reflect in the object if you press OK. No merging is done when the dialog is closed. You must still manually call one of the merging methods.	

Unpublished work. © 2025 Siemens

Example:
None

SetVectorPattern

SetVectorPattern **(nPatternDirection, bOn, nNumberInPattern, dSpacing, vDirection)**	
Description:	
Specifies if *pattern direction* 1, 2, and/or 3 should be used when creating a pattern, as well as how many repetitions, spacing between duplicates, and direction vector for each pattern direction.	
Input:	
`INT4 nPatternDirection`	Pattern Direction. 0 = Direction 1, 1 = Direction 2, 2 = Direction 3
`BOOL bOn`	When True, the pattern direction specified with `nPatternDirection` will be used when creating a pattern. When False, the pattern direction specified will not be considered.
`INT4 nNumberInPattern`	Number of repetitions to make in the direction specified in `nPatternDirection`.
`REAL8 dSpacing`	The distance between each duplicate created in the direction specified by `nPatternDirection`. The total distance between each instance in the pattern is the product of `dSpacing` and the magnitude of `vDirection`. `dSpacing` is only the actual distance if `vDirection` is specified as a unit vector.
`REAL8 vDirection[0..2]`	The XYZ components of the vector to create copies along the direction specified in `nPatternDirection`. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Vector specified for `vDirection` is not valid or value < 1 has been defined for `nNumberInPattern`.
`FE_TOO_SMALL,FE_BAD_TYPE`	`vDirection` is not properly specified as a vector.
`FE_BAD_DATA`	`nNumberInPattern` must be greater than 0.

Remarks/Usage:

Once all the desired pattern directions and options have been specified, use `CreatePattern` to create duplicates in the specified pattern.

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim mt As femap.MergeTool
   Set mt = App.feMergeTool

   Dim xyz1(2) As Double
   Dim xyz2(2) As Double
   Dim xyz3(2) As Double

   xyz1(0) = 1.0
   xyz1(1) = 0.0
   xyz1(2) = 0.0

   xyz2(0) = 0.0
   xyz2(1) = 1.0
   xyz2(2) = 0.0
```


Unpublished work. © 2025 Siemens

```
   xyz3(0) = 0.0
   xyz3(1) = 0.0
   xyz3(2) = 1.0

   rc = mt.SetVectorPattern (0, True, 2, 1.5, xyz1)
   rc = mt.SetVectorPattern (1, True, 2, 1.5, xyz2)
   rc = mt.SetVectorPattern (2, True, 2, 1.5, xyz3)

   mt.CreatePattern (FT_SOLID, -1)

End Sub
```

SetCSysPattern

SetCSysPattern

(nPatternDirection, bOn, nNumberInPattern, dSpacing, nCSysID)

Description:

Specifies if *pattern direction* 1, 2, and/or 3, should be used when creating a pattern, as well as how many repetitions, spacing between duplicates, and coordinate system used to define pattern directions.

Input:

`INT4 nPatternDirection`	Pattern Direction: 0 = Direction 1 (X Axis of coordinate system specified in `nCSysID`) 1 = Direction 2 (Y Axis of coordinate system specified in `nCSysID`) 2 = Direction 3 (Z Axis of coordinate system specified in `nCSysID`)
`BOOL bOn`	When True, the pattern direction specified with `nPatternDirection` will be used when creating

Unpublished work. © 2025 Siemens

	a pattern. When False, the pattern direction specified will not be considered.
`INT4 nNumberInPattern`	Number of repetitions to make in the direction specified in `nPatternDirection`.
`REAL8 dSpacing`	The distance between each duplicate created in the direction specified by `nPatternDirection`. The coordinate system axes are always considered as unit vectors so unlike the `SetVectorPattern` method, `dSpacing` is always the total distance between instances.
`INT4 nCSysID`	Coordinate System used to define the 3 pattern directions.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly dSpacing is too small or negative, or a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_NOT_EXIST`	The specified coordinate system does not exist.
`FE_BAD_DATA`	`nNumberInPattern` must be greater than 0.
Remarks/Usage:	
Once all the desired pattern directions and options have been specified, use `CreatePattern` to create duplicates in the specified pattern.	
Example:	

```
Sub Main

   Dim App As femap.model

   Set App = feFemap()


   Dim mt As femap.MergeTool
```


Unpublished work. © 2025 Siemens

```
    Set mt = App.feMergeTool

    rc = mt.SetCSysPattern (0, True, 3, 1.5, 3)
    rc = mt.SetCSysPattern (1, True, 3, 1.5, 3)
    rc = mt.SetCSysPattern (2, True, 2, 2.5, 3)

    mt.CreatePattern(FT_SOLID, -1)

End Sub
```

MergeTool Object Operation Methods

Merge

Merge	
(Void)	
Description:	
Merges all of the selected entities into the current model.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with

Unpublished work. © 2025 Siemens

	the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.
Remarks/Usage:	
This method is used to merge entities into the current model at the same location where they exist in the selected model. Other methods allow you to specify transforms to apply during the merge.	
Example:	
None	

AlongVector

`AlongVector` **`( vDirection )`**	
Description:	
Merges all of the selected entities into the current model and transforms them along a vector.	
Input:	
`REAL8 vDirection[0..2]`	The components of a vector specifying the direction and distance to offset the merged entities. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.

Unpublished work. © 2025 Siemens

`FE_TOO_SMALL,FE_BAD_TYPE`	`vDirection` is not properly specified as a vector.
Remarks/Usage:	
None	
Example:	
None	

PointToPoint

`PointToPoint` **`( vFrom, vTo )`**	
Description:	
Merges all of the selected entities into the current model and transforms them from one specified location to another.	
Input:	
`REAL8 vFrom[0..2]`	The XYZ coordinates of the location at the base of a vector to move along. Must be specified in Global Rectangular coordinates.
`REAL8 vTo[0..2]`	The XYZ coordinates of the location at the tip of a vector to move along. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.

Unpublished work. © 2025 Siemens

`FE_TOO_SMALL,FE_BAD_TYPE`	`vDirection` is not properly specified as a vector.
Remarks/Usage:	
None	
Example:	
None	

BetweenCSys

`BetweenCSys` **`( nFromCSys, nToCSys )`**	
Description:	
Merges all of the selected entities into the current model and transforms them to align from one location and orientation to another.	
Input:	
`INT4 nFromCSys`	The ID of the Coordinate System to use as the *From* Coordinate System.
`INT4 nToCSys`	The ID of the Coordinate System to use as the *To* Coordinate System.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_NOT_EXIST`	One or both of the coordinate systems do not exist.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

BetweenVectors

BetweenVectors **(vFromBase, vFromDir, vToBase, vToDir)**	
Description:	
Merges all of the selected entities into the current model and transforms them by moving the `FromBase` location to the `ToBase` location and then aligning the `FromDir` vector to the `ToDir` vector.	
Input:	
`REAL8 vFromBase[0..2]`	The XYZ coordinates of the location at the base of a vector to align from. Must be specified in Global Rectangular coordinates.
`REAL8 vFromDir[0..2]`	The components of the vector to align from during merge. Must be specified in Global Rectangular coordinates.
`REAL8 vToBase[0..2]`	The XYZ coordinates of the location at the base of a vector to align to. Must be specified in Global Rectangular coordinates.
`REAL8 vToDir[0..2]`	The components of the vector to align to during merge. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with

Unpublished work. © 2025 Siemens

	the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.
`FE_TOO_SMALL,FE_BAD_TYPE`	One of the arguments is not properly specified as a location or vector.
Remarks/Usage:	
None	
Example:	
None	

BetweenPlanes

`BetweenPlanes`	
`( vFromBase, vFromNormal, vFromX, vToBase, vToNormal, vToX )`	
Description:	
Merges all of the selected entities into the current model and transforms them by aligning the *From* plane to the *To* plane, including the normal direction of the planes.	
Input:	
`REAL8 vFromBase[0..2]`	The XYZ coordinates of location in the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vFromNormal[0..2]`	The components of a vector normal to the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vFromX[0..2]`	The components of a vector that specifies the "X"/ control direction within the plane to align from during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToBase[0..2]`	The XYZ coordinates of location in the plane to align to during alignment. Must be specified in Global Rectangular coordinates.

Unpublished work. © 2025 Siemens

`REAL8 vToNormal[0..2]`	The components of a vector normal to the plane to align to during alignment. Must be specified in Global Rectangular coordinates.
`REAL8 vToX[0..2]`	The components of a vector that specifies the "X"/ control direction within the plane to align to during alignment. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.
`FE_TOO_SMALL,FE_BAD_TYPE`	One of the arguments is not properly specified as a location or vector.
Remarks/Usage:	
None	
Example:	
None	

RotateAroundVector

`RotateAroundVector`
`( vAroundBase, vAroundDir, dAngle, dDistanceAlong )`
Description:
Merges all of the selected entities into the current model and transforms them by rotating around and optionally moving along the specified vector.

Unpublished work. © 2025 Siemens

Input:	
`REAL8 vAroundBase[0..2]`	The XYZ coordinates of the location at the base of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vAroundDir[0..2]`	The components of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 dAngle`	The angle of rotation, in degrees.
`REAL8 dDistanceAlong`	The total distance along the axis of rotation, typically zero.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.
`FE_TOO_SMALL,FE_BAD_TYPE`	One of the arguments is not properly specified as a location or vector.
Remarks/Usage:	
None	
Example:	
None	

RotatePointToPoint

Unpublished work. © 2025 Siemens

RotatePointToPoint

(vAroundBase, vAroundDir, vFrom, vTo)

Description:

Merges all of the selected entities into the current model and transforms them by rotating around a specified vector from one location to another.

Input:

`REAL8 vAroundBase[0..2]`	The XYZ coordinates of the location at the base of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vAroundDir[0..2]`	The components of a vector to rotate around. Must be specified in Global Rectangular coordinates.
`REAL8 vFrom[0..2]`	The XYZ coordinates of the location to rotate/move from. Must be specified in Global Rectangular coordinates.
`REAL8 vTo[0..2]`	The XYZ coordinates of the location to rotate/move to. Must be specified in Global Rectangular coordinates.

Output:

None	None

Return Code:

`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_FAIL`	Merging of selected entities failed.
`FE_TOO_SMALL,FE_BAD_TYPE`	One of the arguments is not properly specified as a location or vector.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

Reflect

Reflect (vAcrossBase, vAcrossDir)	
Description:	
Merges all of the selected entities into the current model and transforms them by reflecting across a specified plane.	
Input:	
REAL8 vAcrossBase[0..2]	The XYZ coordinates of the location at the base of the reflection plane. Must be specified in Global Rectangular coordinates.
REAL8 vAcrossDir[0..2]	The components of a vector normal to the reflection plane. Must be specified in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
FE_NO_FILENAME	No model has been selected to merge. You must always select the model to merge first before other methods.
FE_INVALID	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
FE_FAIL	Merging of selected entities failed.
FE_TOO_SMALL,FE_BAD_TYPE	One of the arguments is not properly specified as a location or vector.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

CreatePattern

CreatePattern **(vInitialOffset)**	
Description:	
Merges multiple copies of the selected entities using the Pattern options specified by the `SetVectorPattern` or `SetCSysPattern` methods.	
Input:	
`REAL8 vInitialOffset[0..2]`	The components of a vector used to offset the entire pattern from the merged entities location in the model being merged.
Output:	
None	None
Return Code:	
`FE_NO_FILENAME`	No model has been selected to merge. You must always select the model to merge first before other methods.
`FE_NOT_AVAILABLE`	No pattern info has been specified.
`FE_INVALID`	Invalid merging options have been specified. Possibly a renumbering problem, a problem with the transformation or you are attempting to merge incomplete geometric information.
`FE_TOO_SMALL,FE_BAD_TYPE`	The specified offset or one of the pattern vectors not properly specified or is too small.
`FE_BAD_DATA`	`nNumberInPattern` must be greater than 0.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Results Vector ID Query Objects

The Results Vector ID Query Object is used to determine the ID of an output vector for a nodal output vector, an elemental output vector (based on element topology and/or type), a complex output vector, or user output vector. There are also methods available which return a range of related Output Vector IDs for the specified output quantity. The purpose of this object is to allow the user to find specific output vector IDs without having to open the Femap user interface and view the IDs in an existing put set.

In addition, there is a method which *finds* an output vector ID based on the output vector title.

To make these methods easier to use, constants (enumerations) have been created for the various arguments, which are described in **Results Vector ID Query Constants (Enumerations)**.

There are no properties on Results Vector ID Query Object, but the available methods can be found in **Results Vector ID Query Object Methods**.

Use the `feResultsIDQuery` method of the Femap Application object to create Results Vector ID Query objects.

Results Vector ID Query Constants (Enumerations)

This section contains constants (enumerations) for the arguments of all methods of the Results Vector ID Query Object, other than the Find method. For more information about the **Find** method.

For more information on each method's enumerations:

`Nodal` and `NodalWithComponents` - See **Constants for Nodal and NodalWithComponents Methods**.

`NodalOther` - See **Constants for NodalOther Method**.

`Line` and `LineAllLocations` - See **Constants for Line and LineAllLocations Methods**.

`LineOther` - See **Constants for LineOther Method**.

`Plate`, `PlateWithCorners`, `Laminate`, `LaminateWithCorners`, `LaminateGlobalPly`, and `LaminateGolbolPlyWithCorners` - See **Constants for Planar Element Methods**.

`PlateOther` - See **Constants for PlateOther Method**.

`Solid` and `SolidWithCorners` - See **Constants for SolidOther Method**.

`SolidOther` - See **Constants for SolidOther Method**.

Unpublished work. © 2025 Siemens

`LaminateSolid`, `LaminateSolidWithCorners`, `LaminateSolidGlobalPly` and `LaminateSolidGlobalPlyWithCorners` - See **Constants for Sold Laminate Element Methods**.

`Elemental` - See **Constants for Elemental Method**.

`Complex` - Only has 4 constants, which are documented in the method. See **Complex**.

`User` - Has no constants. **User**.

Constants for Nodal and NodalWithComponents Methods

The following table provides the list of available constants for vector-based nodal output quantities (enum `zVecNodalResult`).

Constant Name		Constant Name	
VNV_TRANSLATION	0	VNV_GP_MPC_FORCE	18
VNV_ROTATION	1	VNV_GP_MPC_MOMENT	19
VNV_VELOCITY	2	VNV_GP_DRILLING_FORCE	20
VNV_ANG_VELOCITY	3	VNV_GP_DRILLING_MOMENT	21
VNV_ACCELERATION	4	VNV_GP_CONTACT_FORCE	22
VNV_ANG_ACCELERATION	5	VNV_GP_CONTACT_MOMENT	23
VNV_APPLIED_FORCE	6	VNV_CONTACT_TRACTION	24
VNV_APPLIED_MOMENT	7	VNV_CONTACT_FORCE	25
VNV_CONSTRAINT_FORCE	8	VNV_GLUE_TRACTION	26
VNV_CONSTRAINT_MOMENT	9	VNV_GLUE_FORCE	27
VNV_MPC_FORCE	10	VNV_GLUE_MOMENT	28
VNV_MPC_MOMENT	11	VNV_CONTACT_INCSLIDEDISTANCE	29
VNV_GP_TOTAL_FORCE	12	VNV_CONTACT_SLIDEDISTANCE	30

Unpublished work. © 2025 Siemens

VNV_GP_TOTAL_MOMENT	13	VNV_GLUE_INCSLIDEDISTANCE	31
VNV_GP_APPLIED_FORCE	14	VNV_GLUE_SLIDEDISTANCE	32
VNV_GP_APPLIED_MOMENT	15	VNV_AERO_TRANSLATION	33
VNV_GP_CONSTRAINT_FORCE	16	VNV_AERO_ROTATION	34
VNV_GP_CONSTRAINT_MOMENT	17		

The following table provides the list of available constants for vector-based nodal output types, component or resultant (enum `zVecNodalType`).

Constant Name		Constant Name	
VNT_TOTAL_NONE	0	VNT_Y	2
VNT_X	1	VNT_Z	3

Constants for NodalOther Method

The following table provides the list of available constants for scalar nodal output quantities (enum `zVecNodalScalar`):

Constant Name		Constant Name	
VNS_TEMPERATURE	0	VNS_CONTACT_NORMALFORCE	12
VNS_THERMAL_CONSTRAINTFORCE	1	VNS_CONTACT_FRICTIONFORCE	13
VNS_THERMAL_APPLIEDLOAD	2	VNS_CONTACT_NORMALSTRESS	14
VNS_PRESSURE	3	VNS_CONTACT_FRICTIONSTRESS1	15
VNS_ENTHALPY	4	VNS_CONTACT_FRICTIONSTRESS2	16

Unpublished work. © 2025 Siemens

VNS_ENTHALPY_CHANGE	5	VNS_CONTACT_X_FORCE	17
VNS_THERMAL_MULTIPOINTFORCE	6	VNS_CONTACT_Y_FORCE	18
VNS_CONTACT_PRESSURE	7	VNS_CONTACT_Z_FORCE	19
VNS_CONTACT_INISEPDIST	8	VNS_CONTACT_X_FRICTIONFORCE	20
VNS_CONTACT_DEFSEPDIST	9	VNS_CONTACT_Y_FRICTIONFORCE	21
VNS_GLUE_PRESSURE	10	VNS_CONTACT_Z_FRICTIONFORCE	22
VNS_CONTACT_STATUS	11	VNS_COMPLEX_MODE_SHAPE	23

Constants for Line and LineAllLocations Methods

The following table provides the list of available constants for output quantities which may be available for Bar, Beam, or Weld elements (enum `zVecLineResult`):

Constant Name		Constant Name	
VLV_AXIAL_FORCE	0	VLV_STRESS_TENSION_MARGIN	11
VLV_TORQUE	1	VLV_STRESS_COMPRESSION_MARGIN	12
VLV_TORQUE_WARPING	2	VLV_STRAIN_AXIAL	13
VLV_MOMENT_PLANE1	3	VLV_STRAIN_BENDING	14
VLV_MOMENT_PLANE2	4	VLV_STRAIN_COMBINED	15
VLV_SHEAR_PLANE1	5	VLV_STRAIN_PLASTIC	16
VLV_SHEAR_PLANE2	6	VLV_STRAIN_CREEP	17
VLV_STRESS_AXIAL	7	VLV_STRAIN_TOTAL	18

Unpublished work. © 2025 Siemens

VLV_STRESS_BENDING	8	VLV_STRAIN_TENSION_MARGIN	19
VLV_STRESS_COMBINED	9	VLV_STRAIN_COMPRESSION_MARGIN	20
VLV_STRESS_EQUIVALENT	10		

The following table provides the list of available constants for element type for output quantities on line elements (enum `zVecLineElemType`):

Constant Name		Constant Name	
VLT_BAR	0	VLT_WELD	2
VLT_BEAM	1		

The following table provides the list of available constants to specify which end of the line element for output quantities on line elements (enum `zVecLineEnd`):

Constant Name		Constant Name	
VLE_OVERALL	0	VLE_B	2
VLE_A	1		

The following table provides the list of available constants for stress recovery location for output quantities on line elements (enum `zVecLineLoc`):

Constant Name		Constant Name	
VLL_OVERALL	0	VLL_2	4
VLL_MAX	1	VLL_3	5
VLL_MIN	2	VLL_4	6
VLL_1	3		

Unpublished work. © 2025 Siemens

Constants for LineOther Method

The following table provides the list of available constants to select a scalar output quantity on line elements (enum `zVecLineOther`).

Spring Element (non-CBUSH) Output:

Constant Name		Constant Name	
VLO_SPRING_FORCE	0	VLO_SPRING_STRAIN	2
VLO_SPRING_STRESS	1		

Rod Element Output:

Constant Name		Constant Name	
VLO_ROD_AXIAL_FORCE	3	VLO_ROD_STRAIN_AXIAL	10
VLO_ROD_TORQUE	4	VLO_ROD_STRAIN_TORSIONAL	11
VLO_ROD_STRESS_AXIAL	5	VLO_ROD_STRAIN_PLASTIC	12
VLO_ROD_STRESS_TORSIONAL	6	VLO_ROD_STRAIN_CREEP	13
VLO_ROD_STRESS_EQUIVALENT	7	VLO_ROD_STRAIN_TOTAL	14
VLO_ROD_STRESS_AXIAL_MARGIN	8	VLO_ROD_STRAIN_AXIAL_MARGIN	15
VLO_ROD_STRESS_TORSIONAL_MARGIN	9	VLO_ROD_STRAIN_TORSIONAL_MARGIN	16

Gap Element Output:

Constant Name		Constant Name	
VLO_GAP_DISPLACEMENT_AXIAL	17	VLO_GAP_SLIP_W	21

Unpublished work. © 2025 Siemens

VLO_GAP_DISPLACEMENT_V	18	VLO_GAP_FORCE_X	22
VLO_GAP_DISPLACEMENT_W	19	VLO_GAP_FORCE_Y	23
VLO_GAP_SLIP_V	20	VLO_GAP_FORCE_Z	24

Bush Element Output:

Constant Name		Constant Name	
VLO_BUSH_AXIAL_FORCE	25	VLO_BUSH_AXIAL_STRESS	28
VLO_BUSH_AXIAL_DISPLACEMENT	26	VLO_BUSH_AXIAL_STRAIN	29
VLO_BUSH_AXIAL_VELOCITY	27	VLO_BUSH_AXIAL_PLASTIC_STRAIN	30

Bush/Fastener Element Output:

Constant Name		Constant Name	
VLO_BUSHFAST_FORCE_X	31	VLO_BUSHFAST_STRESS_RX	40
VLO_BUSHFAST_FORCE_Y	32	VLO_BUSHFAST_STRESS_RY	41
VLO_BUSHFAST_FORCE_Z	33	VLO_BUSHFAST_STRESS_RZ	42
VLO_BUSHFAST_MOMENT_X	34	VLO_BUSHFAST_STRAIN_TX	43
VLO_BUSHFAST_MOMENT_Y	35	VLO_BUSHFAST_STRAIN_TY	44
VLO_BUSHFAST_MOMENT_Z	36	VLO_BUSHFAST_STRAIN_TZ	45
VLO_BUSHFAST_STRESS_TX	37	VLO_BUSHFAST_STRAIN_RX	46

Unpublished work. © 2025 Siemens

VLO_BUSHFAST_STRESS_TY	38	VLO_BUSHFAST_STRAIN_RY	47
VLO_BUSHFAST_STRESS_TZ	39	VLO_BUSHFAST_STRAIN_RZ	48

Other Weld Element Output:

Constant Name		Constant Name	
VLO_WELD_AXIAL_STRESS	49	VLO_WELD_BEARING_STRESS	51
VLO_WELD_MAX_SHEAR_STRESS	50		

Constants for Planar Element Methods

The following table provides the list of available constants for vector-based and tensor output quantities on planar elements and laminates, with or without global piles (enum `zVecPlateResult`).

Constant Name		Constant Name	
VPV_FORCE	0	VPV_NONLINEAR_STRESS	6
VPV_MOMENT	1	VPV_NONLINEAR_STRAIN	7
VPV_STRESS	2	VPV_CURVATURE	8
VPV_STRAIN	3	VPV_ELASTIC_CURVATURE	9
VPV_ELASTIC_STRAIN	4	VPV_THERMAL_CURVATURE	10
VPV_THERMAL_STRAIN	5		

The following table provides the list of available constants for the various types of output quantities on planar elements and laminates, with or without global piles (enum `zVecPlateType`).

Unpublished work. © 2025 Siemens

Constant Name		Constant Name	
VPT_VON_MISES	0	VPT_X	7
VPT_MAX_PRIN	1	VPT_Y	8
VPT_MIN_PRIN	2	VPT_Z	9
VPT_MID_PRIN	3	VPT_XY	10
VPT_MAX_SHEAR	4	VPT_YZ	11
VPT_ANG_PRIN	5	VPT_XZ	12
VPT_MEAN	6		

The following table provides the list of available constants to specify if the output is at the top, middle, or bottom of a plate element (enum `zVecPlatePly`).

Constant Name		Constant Name	
VPP_TOP	0	VPP_BOT	3
VPP_MID	1		

The following table provides the list of available constants to specify if the output is at the centroid or an individual corner of a planar or laminate element (enum `zVecPlateLoc`).

Constant Name		Constant Name	
VPL_CENTROID	0	VPL_3	4
VPL_1	1	VPL_4	5
VPL_2	3		

Constants for PlateOther Method

The following table provides the list of available constants to select a scalar output quantity on planar elements (enum `zVecPlateScalar`).

Unpublished work. © 2025 Siemens

Shear Element Output:

Constant Name		Constant Name	
VPS_SHEAR_1FROM4	0	VPS_SHEAR_KICK3	12
VPS_SHEAR_1FROM2	1	VPS_SHEAR_KICK4	13
VPS_SHEAR_2FROM1	2	VPS_SHEAR_FLOW12	14
VPS_SHEAR_2FROM3	3	VPS_SHEAR_FLOW23	15
VPS_SHEAR_3FROM2	4	VPS_SHEAR_FLOW34	16
VPS_SHEAR_3FROM4	5	VPS_SHEAR_FLOW41	17
VPS_SHEAR_4FROM3	6	VPS_SHEAR_MAXSHEAR	18
VPS_SHEAR_4FROM1	7	VPS_SHEAR_AVGSHEAR	19
VPS_SHEAR_13DIAG	8	VPS_SHEAR_MAXSHEARSTRAIN	20
VPS_SHEAR_24DIAG	9	VPS_SHEAR_AVGSHEARSTRAIN	21
VPS_SHEAR_KICK1	10	VPS_SHEAR_MARGINOFSAFETY	22
VPS_SHEAR_KICK2	11	VPS_SHEAR_STRAINMARGINOFSAFETY	23

Axisymmetric Element Linear Stress Output:

Constant Name		Constant Name	
VPS_AXISYM_RADIALSTRESS	24	VPS_AXISYM_MINPRINSTRESS	29
VPS_AXISYM_AZIMUTHSTRESS	25	VPS_AXISYM_MAXSHEARSTRESS	30
VPS_AXISYM_AXIALSTRESS	26	VPS_AXISYM_OCTAHEDRALSTRESS	31

Unpublished work. © 2025 Siemens

VPS_AXISYM_SHEARSTRESS	27	VPS_AXISYM_VONMISESSTRESS	32
VPS_AXISYM_MAXPRINSTRESS	28		

Axisymmetric Element Linear Strain Output:

Constant Name		Constant Name	
VPS_AXISYM_RADIALSTRAIN	33	VPS_AXISYM_MINPRINSTRAIN	38
VPS_AXISYM_AZIMUTHSTRAIN	34	VPS_AXISYM_MAXSHEARSTRAIN	39
VPS_AXISYM_AXIALSTRAIN	35	VPS_AXISYM_OCTAHEDRALSTRAIN	40
VPS_AXISYM_SHEARSTRAIN	36	VPS_AXISYM_VONMISESSTRAIN	41
VPS_AXISYM_MAXPRINSTRAIN	37		

Axisymmetric Element Nonlinear Stress Output:

Constant Name		Constant Name	
VPS_AXISYM_RADIALSTRESSNL	42	VPS_AXISYM_SHEARSTRESSNL	45
VPS_AXISYM_AZIMUTHSTRESSNL	43	VPS_AXISYM_VONMISESSTRESSNL	46
VPS_AXISYM_AXIALSTRESSNL	44		

Axisymmetric Element Nonlinear Strain Output:

Unpublished work. © 2025 Siemens

Constant Name		Constant Name	
VPS_AXISYM_RADIALSTRAINNL	47	VPS_AXISYM_AXIALSTRAINNL	49
VPS_AXISYM_AZIMUTHSTRAINNL	48	VPS_AXISYM_SHEARSTRAINNL	50

Axisymmetric Element Linear and Nonlinear Effective Plastic Strain Output:

Constant Name		Constant Name	
VPS_AXISYM_EFFPLASTICSTRAIN	51	VPS_AXISYM_EFFPLASTICSTRAINNL	53

Axisymmetric Element Linear and Nonlinear Effective Creep Strain Output:

Constant Name		Constant Name	
VPS_AXISYM_EFFCREEPSTRAIN	52	VPS_AXISYM_EFFCREEPSTRAINNL	54

Axisymmetric Element Radial Force Output:

Constant Name		Constant Name	
VPS_AXISYM_RADIALFORCE1	55	VPS_AXISYM_RADIALFORCE3	57
VPS_AXISYM_RADIALFORCE2	56	VPS_AXISYM_RADIALFORCE4	58

Axisymmetric Element Azimuth Force Output:

Constant Name		Constant Name	
VPS_AXISYM_AZIMUTHFORCE1	59	VPS_AXISYM_AZIMUTHFORCE3	61
VPS_AXISYM_AZIMUTHFORCE2	60	VPS_AXISYM_AZIMUTHFORCE4	62

Unpublished work. © 2025 Siemens

Axisymmetric Element Axial Force Output:

Constant Name		Constant Name	
VPS_AXISYM_AXIALFORCE1	63	VPS_AXISYM_AXIALFORCE3	65
VPS_AXISYM_AXIALFORCE2	64	VPS_AXISYM_AXIALFORCE4	66

Axisymmetric Element Linear and Nonlinear Mean Stress and Volumetric Strain Output:

Constant Name		Constant Name	
VPS_AXISYM_MEANPRESSURE	67	VPS_AXISYM_VOLUMETRICSTRAIN	68
VPS_AXISYM_MEANPRESSURENL	69	VPS_AXISYM_VOLUMETRICSTRAINNL	70

Plate Fiber Output:

Constant Name		Constant Name	
VPS_FIBER_TOP	70	VPS_FIBER_TOPTHERMAL	74
VPS_FIBER_BOTTOM	71	VPS_FIBER_BOTTOMTHERMAL	75
VPS_FIBER_TOPNL	72	VPS_FIBER_TOPELASTIC	76
VPS_FIBER_BOTTOMNL	73	VPS_FIBER_BOTTOMELASTIC	77

Laminate Max Failure Indices and Min Strength Ratio Output:

Constant Name		Constant Name	
VPS_LAMINATE_MAXFAILUREINDEX	78	VPS_LAMINATE_MINSTRENGTHRATIO	79

Unpublished work. © 2025 Siemens

Constants for Solid Element Methods

The following table provides the list of available constants for stress, nonlinear stress, strain, and nonlinear strain output quantities on solid elements and solid laminates, with or without global piles (enum `zVecSolidResult`).

Constant Name		Constant Name	
`VSV_STRESS`	0	`VSV_NONLINEAR_STRESS`	2
`VSV_STRAIN`	1	`VSV_NONLINEAR_STRAIN`	3

The following table provides the list of available constants for the various types of output quantities on solid elements and solid laminates, with or without global piles (enum `zVecSolidType`).

Constant Name		Constant Name	
`VST_X`	0	`VST_INT_PRIN`	8
`VST_Y`	1	`VST_MAX_SHEAR`	9
`VST_Z`	2	`VST_MEAN`	10
`VST_XY`	3	`VST_VON_MISES`	11
`VST_YZ`	4	`VST_OCT_SHEAR`	12
`VST_ZX`	5	`VST_PLASTIC`	13
`VST_MAX_PRIN`	6	`VST_CREEP`	14
`VST_MIN_PRIN`	7	`VST_VOLUMETRIC`	15

The following table provides the list of available constants to specify if the output is at the centroid or an individual corner of a solid or solid laminate element (enum `zVecSolidLoc`).

Constant Name		Constant Name	
`VPL_CENTROID`	0	`VPL_5`	5
`VPL_1`	1	`VPL_6`	6

Unpublished work. © 2025 Siemens

VPL_2	2	VPL_6	7
VPL_3	3	VPL_8	8
VPL_4	4		

Constants for SolidOther Method

The following table provides the list of available constants to select a scalar output quantity on solid elements (enum `zVecSolidOther`).

Solid Bolt Output:

Constant Name		Constant Name	
VSO_BOLT_AXIALFORCE	0	VSO_BOLT_MOMENT1	3
VSO_BOLT_SHEARFORCE1	1	VSO_BOLT_MOMENT2	4
VSO_BOLT_SHEARFORCE2	2	VSO_BOLT_INcITAXIALSTRAIN	5

Cohesive Element Output:

Constant Name		Constant Name	
VSO_COHESIVE_NORMRELSEP	6	VSO_COHESIVE_YSHEARSTRESS	16
VSO_COHESIVE_XRELDISP	7	VSO_COHESIVE_ZSHEARSTRESS	17
VSO_COHESIVE_YRELDISP	8	VSO_COHESIVE_MATCSID	18
VSO_COHESIVE_ZRELDISP	9	VSO_COHESIVE_XSTRESS	19
VSO_COHESIVE_RELMATCS	10	VSO_COHESIVE_YSTRESS	20

Unpublished work. © 2025 Siemens

VSO_COHESIVE_XRELDISPMATCS	11	VSO_COHESIVE_ZSTRESS	21
VSO_COHESIVE_YRELDISPMATCS	12	VSO_COHESIVE_DAMAGE1	22
VSO_COHESIVE_ZRELDISPMATCS	13	VSO_COHESIVE_DAMAGE2	23
VSO_COHESIVE_NORMALSTRESS	14	VSO_COHESIVE_DAMAGE3	24
VSO_COHESIVE_XSHEARSTRESS	15		

Constants for Sold Laminate Element Methods

The following table provides the list of available constants for stress, strain, failure, and strength ratio output quantities on solid laminate elements, with or without global piles (enum `zVecSolidLamResult`).

Constant Name		Constant Name	
VSLV_STRESS	0	VSLV_FAILURE	2
VSLV_STRAIN	1	VSLV_STRENGTHRATIO	3

The following table provides the list of available constants for the various types of output quantities on solid elements and solid laminates, with or without global piles (enum `zVecSolidLamType`).

Constant Name		Constant Name	
VSLT_X	0	VSLT_MEAN	10
VSLT_Y	1	VSLT_VON_MISES	11
VSLT_Z	2	VSLT_PLY_11	12
VSLT_XY	3	VSLT_PLY_22	13

Unpublished work. © 2025 Siemens

VSLT_YZ	4	VSLT_PLY_33	14
VSLT_ZX	5	VSLT_PLY_12	15
VSLT_MAX_PRIN	6	VSLT_PLY_23	16
VSLT_MIN_PRIN	7	VSLT_PLY_13	17
VSLT_MID_PRIN	8	VSLT_BOND	18
VSLT_MAX_SHEAR	9		

The following table provides the list of available constants to specify if the output is at the top, middle, or bottom of a ply in a solid laminate element (enum `zVecSolidLamLoc`).

Constant Name		Constant Name	
VSLL_TOP	0	VSLL_BOT	3
VSLL_MID	1		

Constants for Elemental Method

The following table provides the list of available constants to select output quantities for elements which are not available using the other methods for elemental output, by shape (enum `zVecElemOther`).

Strain Energy Output:

Constant Name		Constant Name	
VEO_STRAIN_ENERGY	0	VEO_STRAIN_ENERGY_DENSITY	2
VEO_STRAIN_ENERGY_PERCENT	1		

Heat Transfer Output:

Unpublished work. © 2025 Siemens

Constant Name		Constant Name	
VEO_TEMP_GRADIENT_X	3	VEO_HEAT_FLUX_TOTAL	10
VEO_TEMP_GRADIENT_Y	4	VEO_APPLIED_LOAD	11
VEO_TEMP_GRADIENT_Z	5	VEO_CONVECTION_FREE	12
VEO_TEMP_GRADIENT_TOTAL	6	VEO_CONVECTION_FORCED	13
VEO_HEAT_FLUX_X	7	VEO_RADIATION	14
VEO_HEAT_FLUX_Y	8	VEO_HEAT_FLOW_TOTAL	15
VEO_HEAT_FLUX_Z	9		

Kinetic Energy and Energy Loss Output:

Constant Name		Constant Name	
VEO_KINETIC_ENERGY	16	VEO_ENERGY_LOSS_PERCENT	28
VEO_ENERGY_LOSS	26	VEO_KINETIC_ENERGY_DENSITY	29
VEO_ENERGY_LOSS_DENSITY	27	VEO_KINETIC_ENERGY_PERCENT	30

Crack Stress Output:

Constant Name		Constant Name	
VEO_CRACK_STRESS_1	17	VEO_CRACK_STRESS_6	22
VEO_CRACK_STRESS_2	18	VEO_CRACK_STRESS_7	23
VEO_CRACK_STRESS_3	19	VEO_CRACK_STRESS_8	24

Unpublished work. © 2025 Siemens

VEO_CRACK_STRESS_4	20	VEO_CRACK_STRESS_9	25
VEO_CRACK_STRESS_5	21		

Gasket Element Output:

Constant Name		Constant Name	
VEO_GASKET_PRESSURE	31	VEO_GASKET_YIELD_STRESS	34
VEO_GASKET_CLOSURE	32	VEO_GASKET_STATUS	35
VEO_GASKET_PLASTIC_CLOSURE	33		

Fluid Pressure and Normalized Mass Density (Topology Optimization) Output:

Constant Name		Constant Name	
VEO_FLUID_PRESSURE	36	VEO_NORMALIZED_MASS_DENSITY	37

Results Vector ID Query Object Methods

Find

Find
(nRVOutSetID, sRVTitle)
Description:
Determines the ID of an output vector which matches the specified title that exists in the specified output set. Attempting to *Exact Matches* is the best approach, but this method can sometimes use a *Partial Match* to determine the output vector ID, as long as there is only a single *Partial Match*. See Remarks/Usage section.
Input:

Unpublished work. © 2025 Siemens

`INT4 nRVOutSetID`	ID of Output Set to find output vector ID.
`STRING sRVTitle`	Title of Output Vector to find in specified output set.
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	ID of Femap Output Vector.
`FE_FAIL`	ID could not be determined using entered text. See Remarks/Usage Section.
Remarks/Usage:	
Using more characters for `sRVTitle` increases the chances of finding the correct Output Vector ID. Some Examples: *Translation* = `FE_FAIL`, Total Translation = 1 *Beam EndA Moment* = `FE_FAIL`, Beam EndA Plane1 Moment = 3014 *Membrane Force* = `FE_FAIL`, X Membrane Force = 8006 *Plate VonMises Stress* = `FE_FAIL`, Plate Top VonMisesStress = 7033 *Solid Von Mises* = `FE_FAIL` (when Strains are also in Output Set) or 60031 (no Strains in output set), therefore it is always better to use Solid Von Mises Stress, as it will always return 60031. Also, this method does not consider any corner output vectors when attempting to determine an output vector ID from a *Partial Match*, therefore an *Exact Match* will be needed for corner output vectors.	
Example:	
None	

Nodal

`Nodal`

`( zVecNodalResult, zVecNodalType )`

Description:
Determines the ID of the output vector which contains vector-based output data on nodes by entering a type of vector output (Translations, Rotations, Applied Forces, Grid Point Forces, Contact Forces, etc) and

Unpublished work. © 2025 Siemens

if a specific component direction (X,Y, or Z) or the resultant (Total) is needed. See **Constants for Nodal and NodalWithComponents Methods**	
Input:	
`enum zVecNodalResult`	The vector-based nodal output quantity.
`enum zVecNodalType`	Specific component direction or resultant: Total/None = VNT_TOTAL_NONE T1/R1/X = VNT_X T2/R2/Y = VNT_Y T3/R3/Z = VNT_Z
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

NodalWithComponents

`NodalWithComponents` **`( zVecNodalResult, VectorIDs )`**
Description:
Determines the IDs of the output vectors which contain vector-based output data on nodes by entering a type of vector output (Translations, Rotations, Applied Forces, Grid Point Forces, Contact Forces, etc). See **Constants for Nodal and NodalWithComponents Methods**.
Input:

Unpublished work. © 2025 Siemens

`enum zVecNodalResult`	The vector-based nodal output quantity.
Output:	
`VARIANT VectorIDs[0..3]`	Array of four output vector IDs: 0 = Total/None 1 = T1/R1/X 2 = T2/R2/Y 3 = T3/R3/Z
Return Code:	
`FE_FAIL`	Output Vector IDs could not be determined.
Remarks/Usage:	
None	
Example:	
None	

NodalOther

`NodalOther` **`( zVecNodalScalar )`**	
Description:	
Determines the ID of the output vector which contains scalar output data on nodes by entering a type of vector output (Temperature, Enthalpy, Contact Pressure, Contact Normal Force, etc). See **Constants for NodalOther Method**	
Input:	
`enum zVecNodalScalar`	The scalar nodal output quantity.
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

Line

`Line` **`( zVecLineElemType, zVecLineResult, zVecLineEnd, zVecLineLoc )`**	
Description:	
Determines the ID of the output vector which contains output data for Bar, Beam, or Weld elements by entering the type of element, the type of results (Axial Force, Torque, Moment, Shear, Stresses, and Strains), which end of the line element, and stress recovery locations. See **Constants for Line and LineAllLocations Methods**.	
Input:	
`enum zVecLineElemType`	Type of line element: Bar Element = `VLT_BAR` Beam Element = `VLT_BEAM` Weld Element = `VLT_WELD`
`enum zVecLineResult`	Type of results on line element.
`enum zVecLineEnd`	Specifies end of the line element for the results: No End = `VLE_OVERALL` End A= `VLE_A` End B= `VLE_B`
`enum zVecLineLoc`	Specifies stress recovery location of the line element for results: Overall = `VLL_OVERALL` Max = `VLL_MAX` Min = `VLL_MIN`

Unpublished work. © 2025 Siemens

	Pt1 = VLL_1 Pt2 = VLL_2 Pt3 = VLL_3 Pt4 = VLL_4
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LineAllLocations

`LineAllLocations` **`( zVecLineElemType, zVecLineResult, VectorIDs )`**
Description:
Determines the IDs of the output vectors which contain output data for Bar, Beam, or Weld elements, at all locations, by entering the type of element and the type of results (Axial Force, Torque, Moment, Shear, Stresses, and Strains). See **Constants for Line and LineAllLocations Methods**.
Input:

`enum zVecLineElemType`	Type of line element: Bar Element = VLT_BAR Beam Element = VLT_BEAM Weld Element = VLT_WELD
`enum zVecLineResult`	Type of results on line element.
Output:	

Unpublished work. © 2025 Siemens

`VARIANT VectorIDs[0..7]`	Array of eight output vector IDs: 0 = End A: Stress Recovery Point 1 1 = End A: Stress Recovery Point 2 2 = End A: Stress Recovery Point 3 3 = End A: Stress Recovery Point 4 4 = End B: Stress Recovery Point 1 5 = End B: Stress Recovery Point 2 6 = End B: Stress Recovery Point 3 7 = End B: Stress Recovery Point 4
Return Code:	
`FE_FAIL`	Output Vector IDs could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LineOther

`LineOther` **`( zVecLineOther )`**
Description:
Determines the ID of the output vector which contains output data for line elements other than Bars or Beams, (i.e., Springs, Rods, Gaps, CBUSH, Bush/Fastener), and some for Welds. See **Constants for LineOther Method**.
Input:

`enum zVecLineOther`	The line element output quantity.
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

Plate

`Plate` **`( zVecPlateResult, zVecPlateType, zVecPlatePly, zVecPlateLoc )`**	
Description:	
Determines the ID of the output vector which contains output data for non-laminate planar elements (Force, Moment, Stress, Strain, etc) at a specified position (Top/Middle/Bottom) and location (Centroid or Corner). See **Constants for Planar Element Methods**.	
Input:	
`enum zVecPlateResult`	Type of results on non-laminate planar element.
`enum zVecPlateType`	Sub-type of non-laminate planar results (Tensor, Invariant, or Other).
`enum zVecPlatePly`	Position on element for the results: Top = VPP_TOP Middle = VPP_MID Bottom = VPP_BOT
`enum zVecPlateLoc`	Location on element for the results: Centroid = VPL_CENTROID Corner 1 = VPL_1 Corner 2 = VPL_2 Corner 3 = VPL_3 Corner 4 = VPL_4
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

PlateWithCorners

`PlateWithCorners`	
`( zVecPlateResult, zVecPlateType, zVecPlatePly, VectorIDs )`	
Description:	
Determines the IDs of the output vectors which contain output data for non-laminate planar elements (Force, Moment, Stress, Strain, etc) for a specified position (Top/Middle/Bottom). See **Constants for Planar Element Methods**.	
Input:	
`enum zVecPlateResult`	Type of results on non-laminate planar element.
`enum zVecPlateType`	Sub-type of non-laminate planar results (Tensor, Invariant, or Other).
`enum zVecPlatePly`	Position on element for the results: Top = VPP_TOP Middle = VPP_MID Bottom = VPP_BOT
Output:	
`VARIANT VectorIDs[0..4]`	Array of five output vector IDs: 0 = Centroid 1 = Corner 1 2 = Corner 2

Unpublished work. © 2025 Siemens

	3 = Corner 3 4 = Corner 4
Return Code:	
`FE_FAIL`	Output Vector IDs could not be determined.
Remarks/Usage:	
None	
Example:	
None	

Laminate

`Laminate` **`( zVecPlateResult, zVecPlateType, nRVPly, zVecPlateLoc )`**	
Description:	
Determines the ID of the output vector which contains output data for laminate planar elements (Force, Moment, Stress, Strain, etc) at a specified location (Centroid or individual Corner), for a specified laminate ply ID. See **Constants for Planar Element Methods**.	
Input:	
`enum zVecPlateResult`	Type of results on laminate planar element.
`enum zVecPlateType`	Sub-type of laminate planar results (Tensor, Invariant, or Other).
INT4 nRVPly	ID of laminate ply
`enum zVecPlateLoc`	Location on element for the results: Centroid = `VPL_CENTROID` Corner 1 = `VPL_1` Corner 2 = `VPL_2` Corner 3 = `VPL_3` Corner 4 = `VPL_4`
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LaminateWithCorners

`LaminateWithCorners` **`( zVecPlateResult, zVecPlateType, nRVPly, VectorIDs )`**	
Description:	
Determines the IDs of the output vectors which contain output data for laminate planar elements (Force, Moment, Stress, Strain, etc) for a specified laminate ply ID. See **Constants for Planar Element Methods**.	
Input:	
`enum zVecPlateResult`	Type of results on laminate planar element.
`enum zVecPlateType`	Sub-type of laminate planar results (Tensor, Invariant, or Other).
`INT4 nRVPly`	ID of laminate ply.
Output:	
`VARIANT VectorIDs[0..4]`	Array of five output vector IDs: 0 = Centroid 1 = Corner 1 2 = Corner 2 3 = Corner 3 4 = Corner 4
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Output Vector IDs could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LaminateGlobalPly

`LaminateGlobalPly` **`( zVecPlateResult, zVecPlateType, nRVGlobalPly, zVecPlateLoc )`**	
Description:	
Determines the ID of the output vector which contains output data for laminate planar elements (Force, Moment, Stress, Strain, etc) at a specified location (Centroid or individual Corner), for a specified laminate global ply ID. See **Constants for Planar Element Methods**.	
Input:	
`enum zVecPlateResult`	Type of results on laminate planar element.
`enum zVecPlateType`	Sub-type of laminate planar results (Tensor, Invariant, or Other).
`INT4 nRVGlobalPly`	ID of laminate global ply.
`enum zVecPlateLoc`	Location on element for the results: Centroid = `VPL_CENTROID` Corner 1 = `VPL_1` Corner 2 = `VPL_2` Corner 3 = `VPL_3` Corner 4 = `VPL_4`
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LaminateGlobalPlyWithCorners

`LaminateGlobalPlyWithCorners` **`( zVecPlateResult, zVecPlateType, nRVGlobalPly, VectorIDs )`**	
Description:	
Determines the IDs of the output vectors which contain output data for laminate planar elements (Force, Moment, Stress, Strain, etc) for a specified laminate global ply ID. See **Constants for Planar Element Methods**.	
Input:	
`enum zVecPlateResult`	Type of results on laminate planar element.
`enum zVecPlateType`	Sub-type of laminate planar results (Tensor, Invariant, or Other).
`INT4 nRVGlobalPly`	ID of laminate global ply.
Output:	
`VARIANT VectorIDs[0..4]`	Array of five output vector IDs: 0 = Centroid 1 = Corner 1 2 = Corner 2 3 = Corner 3 4 = Corner 4
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Output Vector IDs could not be determined.
Remarks/Usage:	
None	
Example:	
None	

PlateOther

`PlateOther` **`( zVecPlateScalar )`**	
Description:	
Determines the ID of the output vector which contains output data for planar elements other than plates and laminates, with or without global plies including Shear and Axisymmetric elements. Also, this method can be used to determine the output vector ID for scalar output on plate or laminate elements, with or without global plies (i.e., only one output value exists for the element). See **Constants for PlateOther Method**.	
Input:	
`enum zVecPlateScalar`	The planer element output quantity (typically a scalar).
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Solid

Solid **(zVecSolidResult, zVecSolidType, zVecSolidLoc)**	
Description:	
Determines the ID of the output vector which contains output data for non-laminate solid elements (Stress, Nonlinear Stress, Strain, and Nonlinear Strain) at specified location (Centroid or Corner). See **Constants for Solid Element Methods**.	
Input:	
`enum zVecSolidResult`	Type of results on non-laminate solid element.
`enum zVecSolidType`	Sub-type of non-laminate solid results (Tensor, Invariant, or Other).
`enum zVecSolidLoc`	Location on element for the results: Centroid = VSL_CENTROID Corner 1 = VSL_1 Corner 2 = VSL_2 Corner 3 = VSL_3 Corner 4 = VSL_4 Corner 5 = VSL_5 Corner 6 = VSL_6 Corner 7 = VSL_7 Corner 8 = VSL_8
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SolidWithCorners

SolidWithCorners **(zVecSolidResult, zVecSolidType, VectorIDs)**
Description:
Determines the IDs of the output vectors which contain output data for non-laminate solid elements (Stress, Nonlinear Stress, Strain, and Nonlinear Strain) at all locations. See **Constants for Solid Element Methods**.
Input:

`enum zVecSolidResult`	Type of results on non-laminate solid element.
`enum zVecSolidType`	Sub-type of non-laminate solid results (Tensor, Invariant, or Other).

Output:

`VARIANT VectorIDs[0..4]`	Array of nine output vector IDs: 0 = Centroid 1 = Corner 1 2 = Corner 2 3 = Corner 3 4 = Corner 4 5 = Corner 5 6 = Corner 6 7 = Corner 7 8 = Corner 8

Return Code:

`FE_FAIL`	Output Vector IDs could not be determined.

Remarks/Usage:
None
Example:
None

Unpublished work. © 2025 Siemens

SolidOther

SolidOther

(zVecSolidOther)

Description:

Determines the ID of the output vector which contains specialized output data for solid elements, such as solid elements used as Bolts, as well as Solid Cohesive elements. See **Constants for SolidOther Method**.

Input:

`enum zVecSolidOther`	The specialized solid element output quantity.

Output:

None	None

Return Code:

`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.

Remarks/Usage:

None

Example:

None

LaminateSolid

LaminateSolid

(zVecSolidLamResult, zVecSolidLamType, nRVPly, zVecSolidLamLoc, zVecSolidLoc)

Description:

Determines the ID of the output vector which contains output data for solid laminate planar elements (Stress, Strain, Failure, or Strength Ratio) at a specified location (Centroid or individual Corner), for the

Unpublished work. © 2025 Siemens

top, middle, or bottom of the specified solid laminate ply ID. See **Constants for Sold Laminate Element Methods**.	
Input:	
`enum zVecSolidLamResult`	Type of results on solid laminate element.
`enum zVecSolidLamType`	Sub-type of solid laminate results (Tensor, Invariant, or Other).
`INT4 nRVPly`	ID of solid laminate ply.
`enum zVecSolidLamLoc`	Results at Top, Middle, or Bottom of specified solid laminate ply.
`enum zVecSolidLoc`	Location on element for the results: Centroid = `VSL_CENTROID` Corner 1 = `VSL_1` Corner 2 = `VSL_2` Corner 3 = `VSL_3` Corner 4 = `VSL_4` Corner 5 = `VSL_5` Corner 6 = `VSL_6` Corner 7 = `VSL_7` Corner 8 = `VSL_8` See **Constants for Solid Element Methods**.
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LaminateSolidWithCorners

Unpublished work. © 2025 Siemens

LaminateSolidWithCorners

(zVecSolidLamResult, zVecSolidLamType, nRVPly, zVecSolidLamLoc, VectorIDs)

Description:

Determines the IDs of the output vectors which contain output data for solid laminate elements (Stress, Strain, Failure, or Strength Ratio) for the top, middle, or bottom of the specified solid laminate ply ID. See Constants for Sold Laminate Element Methods.

Input:

`enum zVecSolidLamResult`	Type of results on solid laminate element.
`enum zVecSolidLamType`	Sub-type of solid laminate results (Tensor, Invariant, or Other).
`INT4 nRVPly`	ID of solid laminate ply.
`enum zVecSolidLamLoc`	Results at Top, Middle, or Bottom of specified solid laminate ply.

Output:

`VARIANT VectorIDs[0..4]`	Array of nine output vector IDs: 0 = Centroid 1 = Corner 1 2 = Corner 2 3 = Corner 3 4 = Corner 4 5 = Corner 5 6 = Corner 6 7 = Corner 7 8 = Corner 8

Return Code:

`FE_FAIL`	Output Vector IDs could not be determined.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

LaminateSolidGlobalPly

LaminateSolidGlobalPly

(zVecSolidLamResult, zVecSolidLamType, nRVGlobalPly, zVecSolidLamLoc, zVecSolidLoc)

Description:

Determines the ID of the output vector which contains output data for solid laminate planar elements (Stress, Strain, Failure, or Strength Ratio) at a specified location (Centroid or individual Corner), for the top, middle, or bottom of the specified solid laminate global ply ID. See **Constants for Sold Laminate Element Methods**.

Input:

`enum zVecSolidLamResult`	Type of results on solid laminate element.
`enum zVecSolidLamType`	Sub-type of solid laminate results (Tensor, Invariant, or Other).
`INT4 nRVGlobalPly`	ID of solid laminate global ply.
`enum zVecSolidLamLoc`	Results at Top, Middle, or Bottom of specified solid laminate global ply.
`enum zVecSolidLoc`	Location on element for the results: Centroid = `VSL_CENTROID` Corner 1 = `VSL_1` Corner 2 = `VSL_2` Corner 3 = `VSL_3` Corner 4 = `VSL_4` Corner 5 = `VSL_5` Corner 6 = `VSL_6` Corner 7 = `VSL_7` Corner 8 = `VSL_8` See **Constants for Solid Element Methods**.

Output:

None	None

Return Code:

`INT4 1 to Maximum ID`	Output Vector ID.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	
None	
Example:	
None	

LaminateSolidGlobalPlyWithCorners

`LaminateSolidGlobalPlyWithCorners` **`( zVecSolidLamResult, zVecSolidLamType, nRVPly, zVecSolidLamLoc, VectorIDs )`**	
Description:	
Determines the IDs of the output vectors which contain output data for solid laminate elements (Stress, Strain, Failure, or Strength Ratio) for the top, middle, or bottom of the specified solid laminate global ply ID. See **Constants for Sold Laminate Element Methods**.	
Input:	
`enum zVecSolidLamResult`	Type of results on solid laminate element.
`enum zVecSolidLamType`	Sub-type of solid laminate results (Tensor, Invariant, or Other).
`INT4 nRVGlobalPly`	ID of solid laminate global ply.
`enum zVecSolidLamLoc`	Results at Top, Middle, or Bottom of specified solid laminate global ply.
Output:	
`VARIANT VectorIDs[0..4]`	Array of nine output vector IDs: 0 = Centroid 1 = Corner 1 2 = Corner 2 3 = Corner 3 4 = Corner 4 5 = Corner 5 6 = Corner 6

Unpublished work. © 2025 Siemens

	7 = Corner 7 8 = Corner 8
Return Code:	
`FE_FAIL`	Output Vector IDs could not be determined.
Remarks/Usage:	
None	
Example:	
None	

Elemental

`Elemental` **`( zVecElemOther )`**
Description:
Determines the ID of the output vector which contains specialized output data for elements which is not able to be determined using the `Line`, `LineOther`, `Plate`, `Laminate`, `LaminateGlobalPly`, `PlateOther`, `Solid`, `SolidOther`, `LaminateSolid`, or `LaminateSolidGlobalPly` methods. See **Constants for Elemental Method**. Also, see Remarks/Usage section.
Input:

`enum zVecElemOther`	The element output quantity.
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.

Remarks/Usage:
This method is used to determine the output vector ID for almost any Strain Energy, Heat Transfer, Kinetic Energy, Energy Loss, Crack Stress, or Fluid Pressure output quantity. In addition, it can be used to determine the Output Vector ID for any Gasket element output quantity. Finally, it can also determine the

Unpublished work. © 2025 Siemens

output vector ID for Normalized Mass Density, which is useful when reviewing results recovered from a Topology Optimization run performed with SOL 200 of Simcenter Nastran.
Example:
None

Complex

Complex	
(zVecComplex, nRVOriginalVector)	
Description:	
Determines the ID of the output vector which is the corresponding complex output vector to the output vector ID specified as the *Original Output Vector* (i.e., the ID of the phase output vector when a magnitude output vector has been specified or vice versa). See Remarks/Usage section.	
Input:	
`enum zVecComplex`	Used to determine which type of complex output vector ID should be found to correspond with the *Original Vector ID*. To determine corresponding output vector, use: `VCX_MAGNITUDE` for Magnitude `VCX_PHASE` for Phase `VCX_REAL` for Real `VCX_IMAGINARY` for Imaginary
`INT4 nRVOriginalVector`	Original Vector ID
Output:	
None	None
Return Code:	
`INT4 1 to Maximum ID`	Output Vector ID.
`FE_FAIL`	Output Vector ID could not be determined.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method can be used as a secondary step to determine the corresponding complex output vector after another method on the Results ID Query object has been used to find the ID of the *Original Vector*. Typically, if a magnitude vector ID has been found first, set `zVecComplex` to `VCX_PHASE` to find the corresponding phase vector ID. Another use might be to find the ID of the real output vector if the ID of the imaginary vector is already known.

See example. This would return a value of 74000001, the first value in the User Output range for Real.

Example:

This method and be used in conjunction with `User` method to do this:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim r As femap.ResultsIDQuery
   Set r = App.feResultsIDQuery

   Dim outvec As Long
   outvec = r.Complex(VCX_REAL, r.User(1))
End Sub
```

User

User

(nRVUserVector)

Description:

Determines the ID an output vector in the *User Output* range for the specified output vector ID.

Input:

`INT4 nRVUserVector`	ID of output vector to determine output vector in *User Output* range. The only values which can be used are:

Unpublished work. © 2025 Siemens

	Magnitude = 1 to 999,999 Phase = 25,000,001 to 25,999,999 Real = 50,000,0001 to 50,999,999 Imaginary = 75,000,001 to 75,999,999
Output:	
None	None
Return Code:	
`INT4` 24000001 to 24999999 49000001 to 49999999 74000001 to 74999999 99000001 to 99999999	Output Vector ID in *User Output* range. There are 999,999 available vectors in each *User Output* range: Magnitude = 24,000,001 to 24,999,999 Phase = 49,000,001 to 49,999,999 Real = 74,000,0001 to 74,999,999 Imaginary = 99,000,001 to 99,999,999
`FE_FAIL`	Output Vector ID could not be determined
Remarks/Usage:	
See example. This would return a value of 49000001, the first value in the User Output range for Phase.	
Example:	

This method and be used in conjunction with `Complex` method to do this:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim r As femap.ResultsIDQuery
   Set r = App.feResultsIDQuery

   Dim outvec As Long
   outvec = r.User(r.Complex(VCX_PHASE,1))
End Sub
```


Unpublished work. © 2025 Siemens

Stress Linearization (StressLinear) Objects

The Stress Linearization (`StressLinear`) object offers programmatic access to the **View → Advanced Post → Stress Linearization** command.

The Femap Stress Linearization Tool is based on the ASME Boiler and Pressure Vessel Code, 2007 SECTION VIII, DIVISION 2, ANNEX 5.1 LINEARIZATION OF STRESS RESULTS FOR STRESS CLASSIFICATION.

The ASME Code, and this tool, are based on the user choosing an appropriate Stress Classification Line (SCL) by choosing two nodes.

Stress Linearization relies on transforming FEA tensor stresses to be aligned with the SCL. For this calculation to be correct, the reference coordinate system for the stress data must be known. The base reference system can be specified in the Femap user interface by either using **File → Preferences**, choosing the **Geometry/Model** tab, then clicking the **Output Orientation** button or using the **Transform** button found in various dialog boxes, then clicking the **Output Orientation** button. In all cases, the Current Output Orientation dialog will be displayed to set the base orientation.

The base orientation can also be set/updated via the API through Global Integer Properties such as `Info_OrientSolidIsoOutput`, `Info_OrientQuad4StressOutput`, etc as detailed in this manual.

Note

You should be familiar with the requirements and recommendations of the ASME Code, the proper choice of the SCL is key to applying this standard.

Note

For more information about how the **Stress Linearization** command is calculated, see "The Algorythm" section of the View → Advanced Post → Stress Linearization... topic of the Femap Commands Guide.

The appropriate process is to have the `StressLinear` Properties (see **StressLinear Object Properties**) specified as desired before calling a `StressLinear` Method (see **StressLinear Object Methods**). Currently, there is only one `StressLinear` method (`CalcStressLinearization`).

Use the `feStressLinear` method of the Femap Application object to create Stress Linearization objects.

StressLinear Object Properties

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ndID1`	Node ID at the beginning of the SCL.
`INT4 ndID2`	Node ID at the end of the SCL.
`INT4 ouSetID`	ID of the Output Set with stresses to be linearized.
`INT4 nElemType`	Type of Elements to consider for calculations. 0=Solid, 1= Plates, 2=Axisymmetric.
`INT4 nStressMode`	Type of stress used for calculations. 0=von Mises, 1=Tresca.
`BOOL bNonlinerMode`	Controls whether standard linear stress data or nonlinear stress data will be used from the output set specified for `ouSetID`.
`BOOL bFullStressTensor`	Default/Recommended value is FALSE to follow the ASME code. Setting to TRUE will use all stress tensor values when linearizing bending stress.
Calculated Stress Output (Read-Only)	
`REAL8 dMembraneStress`	Calculated linearized Membrane Stress per the ASME Code (will be 0.0 until `CalcStressLinearization` is used).
`REAL8 dBendingStress`	Calculated linearized Bending Stress per the ASME Code (will be 0.0 until `CalcStressLinearization` is used).
`REAL8 dMaxMemStress`	Calculated linearized Membrane and Bending Stress per the ASME Code (will be 0.0 until `CalcStressLinearization` is used).
`REAL8 dMaxStress`	Actual maximum stress along the SCL from the FEA results (will be 0.0 until `CalcStressLinearization` is used).

StressLinear Object Methods

CalcStressLinearization

Unpublished work. © 2025 Siemens

CalcStressLinearization

(Void)

Description:

Calculates the ASME Stress Linearization, if successful, with a return code of FE_OK, the Calculated Stress Output values on the StressLinear Object will be updated.

Input:

None	None

Output:

None	None

Return Code:

FE_OK	Stress Linearization successfully calculated
FE_FAIL	Error in calculations
FE_BAD_DATA	One of the following has occurred: Node IDs not unique, or at least one node ID does not exist. Output Set does not exist. Nodes not connected by mesh. Axisymmetric elements not on proper global planes.
FE_BAD_TYPE	Wrong element type specified.

Remarks/Usage:

None

Example:

Sample: In this sample, two SCL's are defined, and two stress linearizations are calculated.

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()
   Dim fSL As femap.StressLinear
```


Unpublished work. © 2025 Siemens

```
Set fSL = App.feStressLinear


fSL.ouSetID = 1
fSL.nElemType = 0
fSL.ndID1 = 2789
fSL.ndID2 = 2899
fSL.bNonlinearMode = False
If fSL.CalcStressLinearization() = FE_OK Then
    Msg = "Maxium Stress = " +Str$(fSL.dMaxStress)
    App.feAppMessage( FCM_NORMAL, Msg )
    Msg = "Membrane Stress = " + Str$(fSL.dMembraneStress)
    App.feAppMessage( FCM_NORMAL, Msg )
    Msg = "Bending Stress = " + Str$(fSL.dBendingStress )
    App.feAppMessage( FCM_NORMAL, Msg )
End If

fSL.ouSetID = 1
fSL.nElemType = 0
fSL.ndID1 = 2791
fSL.ndID2 = 2896
If fSL.CalcStressLinearization() = FE_OK Then
    Msg = "Maxium Stress = " +Str$(fSL.dMaxStress)
    App.feAppMessage( FCM_NORMAL, Msg )
    Msg = "Membrane Stress = " + Str$(fSL.dMembraneStress)
    App.feAppMessage( FCM_NORMAL, Msg )
    Msg = "Bending Stress = " + Str$(fSL.dBendingStress )
    App.feAppMessage( FCM_NORMAL, Msg )
End If
```


Unpublished work. © 2025 Siemens

```
End Sub
```

PublishTool Objects

The `PublishTool` object provides capabilities to publish customized HTML and RTF reports to either the clipboard or to a file.

Use the `fePublishTool` method of the Femap application object to create `PublishTool` objects.

PublishTool Object Methods

Clear

```
Clear

( )
```

Description:	
Clears everything in the `PublishTool` object.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetFormat

SetFormat **(nFormat)**	
Description:	
Specifies the format of data to be published to the Clipboard or to a File.	
Input:	
`INT4 nFormat`	The format to publish (0=RTF, 1=HTML).
Output:	
None	None
Return Code:	
`FE_INVALID`	The specified format does not correspond to either RTF or HTML.
Remarks/Usage:	
None	
Example:	
See `AddParagraph()` method.	

SetParagraphAlignment

SetParagraphAlignment **(nAlign)**	
Description:	
Specifies how paragraphs and other objects will be aligned as they are added to the Tool.	
Input:	
`INT4 nAlign`	0=Left, 1=Right, 2=Center

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_INVALID`	The alignment method is not valid.
Remarks/Usage:	
This method can be called as many times as necessary, but must be called prior to adding the data that is supposed to be aligned. Default alignment is Left.	
Example:	
None	

SetParagraphIndent

`SetParagraphIndent` **`( nLeft, nRight, nFirstLine )`**	
Description:	
Specifies how paragraphs and other objects will be indented as they are added to the Tool.	
Input:	
`INT4 nLeftIndent`	The space between the margin and left side of a paragraph, image or table. This is specified in points (1/72 inch).
`INT4 nRightIndent`	The space between the margin and right side of a paragraph. This is specified in points (1/72 inch).
`INT4 nFirstLineIndent`	Additional space between the indented left side of the paragraph and the beginning of the first line. That is, the first line is indented by `nLeftIndent+nFirstLineIndent`. This is specified in points (1/72 inch).
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method can be called as many times as necessary, but must be called prior to adding the data that is supposed to be indented. `SetParagraphIndent(0,0,0)` eliminates all indents.
Example:
None

SetParagraphSpacing

`SetParagraphSpacing` **`( nSpaceAbove, nSpaceBelow )`**	
Description:	
Specifies how space will be added above and below paragraphs and other objects as they are added to the Tool.	
Input:	
`INT4 nSpaceAbove`	The space above the top of a paragraph, image or table. This is specified in points (1/72 inch).
`INT4 nSpaceBelow`	The space below the bottom of a paragraph, image or table. This is specified in points (1/72 inch).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method can be called as many times as necessary, but must be called prior to adding the data that will have spacing above or below. `SetParagraphSpacing(0,0)` eliminates spacing.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddStyle

AddStyle **(sFontName, nFontSize, nColor, nHighlightColor, bBold, bItalic, bUnderline bPreserveFormatting)**	
Description:	
Defines a style that can then be applied to text that is added to the Tool.	
Input:	
`STRING sFontName`	The name of a font to be used for text. This must match the name of an existing Font. Alternatively, this can be a blank string if you want to continue to use the current font.
`INT4 nFontSize`	The size of the font to be used in points (1/72 inch). Specify as 0 to use the size of the current font.
`INT4 nColor`	The color of text.
`INT4 nHighlightColor`	The background color of text.
`BOOL bBold`	Text will be bold if True.
`BOOL bItalic`	Text will be italic if True.
`BOOL bUnderline`	Text will be underlined if True.
`BOOL bPreserveFormatting`	If True, line breaks that are included in a paragraph that is added will be maintained so the text appears *as specified*. If False, line breaks are ignored and the text will be continuous, only breaking at margins.
Output:	
None	None
Return Code:	
`INT4 nStyleID`	The ID of the specified style which can later be used to apply this style.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method does not activate a style, it merely defines it. The *Style ID* that is returned is then used as text is defined to apply the style to the text.
Example:
See `AddParagraph()` method.

AddParagraph

`AddParagraph` **`( sText, nStyle )`**	
Description:	
Adds a paragraph of text to the Publish Tool.	
Input:	
`STRING sText`	The text of the paragraph.
`INT4 nStyle`	The style to be applied to the paragraph. This is the value returned by the `AddStyle( )` method. Specify as 0 to use the default style.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The active alignments, indents and spacing are applied to the paragraph as it is added.	
Example:	

```
Sub Main

   Dim App as femap.model

   Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
   Dim pTool As femap.PublishTool

   Set pTool = App.fePublishTool()


   rc = pTool.SetFormat( PUB_RTF )


    'Create a style for the Heading

   bBold = True : bUnderline = True : bItalic = True : bPres = False


    aStyle = pTool.AddStyle( "Arial", 20, FCL_BLACK, FCL_WHITE, bBold,
bItalic, bUnderline, bPres )


   pTool.AddParagraph( "Heading", aStyle )

   pTool.AddParagraph( " ", 0)


    'Create other styles for the Text

    bStyle = pTool.AddStyle( "Arial", 15, FCL_RED, FCL_YELLOW, bBold,
bItalic, bUnderline, bPres )

    cStyle = pTool.AddStyle( "Arial", 15, FCL_FORESTGREEN, FCL_YELLOW,
bBold, bItalic, bUnderline, bPres )


    ' Add paragraphs of text with different style applied

   For i = 0 To 5


       If i Mod 2 = 0 Then

           pTool.AddParagraph("Test"+Str( i ), bStyle)

       Else

           pTool.AddParagraph("Test"+Str( i ), cStyle)

       End If
```


Unpublished work. © 2025 Siemens

```
        pTool.AddParagraph( " ", 0)


   Next


    'Send the text & other contents to the clipboard
   rc = pTool.PublishToClipboard()


End Sub
```

AddPicture

AddPicture **(sFileName, nFormat, nSizePct)**	
Description:	
Adds an image from a file.	
Input:	
`STRING sFileName`	The full pathname to the image file to be included.
`INT4 nFormat`	0=Bitmap, 1=JPEG, 2=PNG, 3=GIF, 4=TIFF
`INT4 nSizePct`	A scale factor that can be applied to the natural size of the image. If specified as 0 (or 100) the image will not be scaled.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	The specified image file does not exist.
`FE_FAIL`	The image can not be added.

Unpublished work. © 2025 Siemens

Remarks/Usage:
The active alignments, left indent and spacing are applied to the image as it is added.
Example:
None

AddView

AddView **(nVuID, nSizePct)**	
Description:	
Adds an image created from a View stored in the current model.	
Input:	
`INT4 nVuID`	The ID of the View to be used to create the image.
`INT4 nSizePct`	A scale factor that can be applied to the natural size of the image. If specified as 0 (or 100) the image will not be scaled.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	The specified view does not exist.
`FE_NOT_AVAILABLE`	There is no active View in the current model.
`FE_FAIL`	The image can not be added.
Remarks/Usage:	
The active alignments, left indent and spacing are applied to the image as it is added.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddDataTable

AddDataTable **(bCopyAllRows, bCopyHeader, bCopyFooter, nNumColumns, sColumnTitles)**	
Description:	
Adds a copy of the current Data Table.	
Input:	
`BOOL bCopyAllRows`	If True, all rows in the Data Table will be copied. If False, only selected rows will be copied.
`BOOL bCopyHeader`	If True, any Data Table Column or Table header rows will be included.
`BOOL bCopyFooter`	If True, any Data Table Footer rows will be included.
`INT32 nNumColumns`	If 0, all Columns will be included. If >0, this is the number of titles included in `sColumnTItles`.
`STRING sColumnTitles[0..nNumColumns-1]`	If `nNumColumns` > 0 this is a list of the column titles that will be copied. The titles must be specified in full, exactly matching what is in the Data Table, however it is case insensitive.
Output:	
None	None
Return Code:	
`FE_BAD_DATA`	Invalid column title specified.
`FE_NO_MEMORY`	Insufficient memory to retrieve Column titles.
`FE_FAIL`	The Data Table is not available or can not be added.
Remarks/Usage:	
The active alignments, left indent and spacing are applied to the data table as it is added.	
Example:	
None	

Unpublished work. © 2025 Siemens

AddFunctionTableEditor

AddFunctionTableEditor **(bCopyHeader)**	
Description:	
Adds a copy of the current Function/Table Editor.	
Input:	
`BOOL bCopyHeader`	If True, any Column or Table header rows will be included.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Function/Table Editor is not available or can not be added.
Remarks/Usage:	
The active alignments, left indent and spacing are applied to the function or table as it is added.	
Example:	
None	

AddEntityEditor

AddEntityEditor **(void)**	
Description:	
Adds a copy of the current Entity Editor.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	The Entity Editor is not available or can not be added.
Remarks/Usage:	
The active alignments, left indent and spacing are applied to the image as it is added.	
Example:	
None	

AddMessages

`AddMessages`
`( void )`
Description:
Adds selected text from the Message Window.
Input:

`INT4 nFirstLine`	The first line of text to select from the Message WIndow. Specify as <= 0 to select the first available line in the Window.
`INT4 nLastLine`	The last line of text to select from the Message WIndow. Specify as <= 0 to select the last available line in the Window.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Message Window is not available or can not be added.
Remarks/Usage:	
The active alignments, left indent and spacing are applied to the image as it is added.	

Unpublished work. © 2025 Siemens

You can use the method `feFileMessageLineNumber( )` to retrieve the current line number in the Message Window prior to and after an operation to use as the first and last lines for this method.
Example:
None

AddChart

`AddChart`	
`( nSizePct )`	
Description:	
Adds an image of the current Charting Pane.	
Input:	
`INT4 nSizePct`	A scale factor that can be applied to the natural size of the image. If specified as 0 (or 100) the image will not be scaled.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Charting Pane is not available or can not be added.
Remarks/Usage:	
The active alignments, left indent and spacing are applied to the image as it is added.	
Example:	
None	

UserTable

Unpublished work. © 2025 Siemens

UserTable	
(void)	
Description:	
Creates and returns a `PublishTable` object.	
Input:	
None	None
Output:	
None	None
Return Code:	
`PublishTable` object	None
Remarks/Usage:	
Refer to the documentation of the `PublishTable` object to configure and publish the table.	
Example:	
None	

PublishToClipboard

PublishToClipboard	
(void)	
Description:	
Publishes the contents of the current object to the Clipboard for pasting to other applications.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to publish current object.
Remarks/Usage:	
None	
Example:	
None	

PublishToFile

`PublishToFile` **`( sFileName )`**	
Description:	
Publishes the contents of the current object to a specified file.	
Input:	
`STRING sFileName`	The name of the file to contain the published data.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to publish current object.
Remarks/Usage:	
For easiest access later, the file extension of the specified file should match the normal usage for the publishing format. For example, .html for HTML format or .rtf for RTF format.	
Example:	
None	

PublishTable Objects

One or more `PublishTable` object is created by using the `UserTable` method of the `PublishTool` Object. These objects are then associated with that specific `PublishTool` object.

Unpublished work. © 2025 Siemens

PublishTable Object Methods

AddColumn

AddColumn

(sColumnTitle, nAlign, dWidth)

Description:

Adds an empty column to the table.

Input:

`STRING sColumnTitle`	A title to add to the column header.
`INT4 nAlign`	Alignment of text in the column 0=Left, 1=Right, 2=Center.
`REAL8 dWidth`	Width of the column in inches. Specify as 0.0 for default width..

Output:

None	None

Return Code:

`INT4 nColumnID`	The column ID of the added column. -1 if adding the column failed.

Remarks/Usage:

The column ID returned by this method must be used when adding text/values to the column.

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim pTool As femap.PublishTool
```


Unpublished work. © 2025 Siemens

```
   Dim pTable As femap.PublishTable


   Set pTool = App.fePublishTool()

   Set pTable = pTool.UserTable


    'Create a style for the Heading & Body of pTable

   bBold = True : bUnderline = False : bItalic = True :
bPreserveFormatting = False


    headerStyle = pTool.AddStyle( "Arial", 15, FCL_BLACK,
FCL_PALESKYBLUE, bBold, bItalic, bUnderline, bPreserveFormatting )

    bodyStyle = pTool.AddStyle( "Arial", 20, FCL_BLACK, FCL_WHITE, bBold,
bItalic, bUnderline, bPreserveFormatting )


    'Set the style of the table

   rc = pTable.SetTableStyle( headerStyle, bodyStyle )


    'AddColumn() method returns the zero-based index

   col0 = pTable.AddColumn("Column 1", PUBALN_CENTER, 2.0)

   col1 = pTable.AddColumn("Column 2", PUBALN_CENTER, 2.0 )

   rc = pTable.AddColumnGroupTitle(col0, col1, "Column Group")


    'Add row titles

   rc = pTable.SetRowTitle(0, "Row 1")

   rc = pTable.SetRowTitle(1, "Row 2")


    'Set some mock data

   rc = pTable.SetRealFormat( 10, 3 )

   rc = pTable.SetCellText(0, col0, "Data")
```


Unpublished work. © 2025 Siemens

```
    rc = pTable.SetCellReal(0, col1, 5.45677777)


    rc = pTable.Publish()
    rc = pTool.PublishToClipboard()


     'Now it's on the Clipboard. Paste anywhere to view.


End Sub
```

AddColumnGroupTitle

AddColumnGroupTitle **(nStartColumn, nEndColumn, sColumnGroupTitle)**	
Description:	
Adds a second title above a contiguous range of columns.	
Input:	
`INT4 nStartColumn`	The ID of the column where the Group Title starts.
`INT4 nEndColumn`	The ID of the column where the Group Title ends.
`STRING sColumnGroupTitle`	The title to add above the headers of the selected columns.
Output:	
None	None
Return Code:	
`FE_FAIL`	Adding the column header failed.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method adds a joined group title above all of the selected columns. This creates a second row of headers above the column titles. It is not possible to create additional rows of headers nor to add multiple group titles to the same column. Columns must be added prior to calling this method. You can specify `nStartColumn=nEndColumn` if you just want a second header above a single column. Any column that does not have a Group Title will have its header text centered vertically between the two header rows.
Example:
See `AddColumn()` method.

AddRowNumbering

`AddRowNumbering` **`( nFirstRowNumber )`**	
Description:	
Adds row numbers as a left side header in the table.	
Input:	
`INT4 nFirstRowNumber`	The number used for the first row of the table. Typically, this would be either 0 or 1, however you may specify any positive value. Successive rows increment this number by 1. Specifying a negative value turns off row numbering.
Output:	
None	None
Return Code:	
`FE_FAIL`	Adding the column header failed.
Remarks/Usage:	
Defining Row Numbering using this method does not change the way rows must be referenced by the other methods of this object. Actual row numbers always start at 0 and increase by 1. This method simply defines the values that are shown in the left side header. `SetRowTitle( )` is an alternative to this method that lets you specify arbitrary left side header information.	

Unpublished work. © 2025 Siemens

Example:
None

SetRowTitle

SetRowTitle (nRow, sRowTitle)	
Description:	
Adds a string as a left side header for a particular row in the table.	
Input:	
INT4 nRow	The row of the table where the left side header is being defined. The first row in the table is always row 0.
STRING sRowTitle	Title of the row header.
Output:	
None	None
Return Code:	
FE_NOT_AVAILABLE	Invalid row number. Row numbers must be >= 0.
Remarks/Usage:	
AddRowNumbering( ) is an alternative to this method that can be used if you simply want row numbers in the left side header.	
Example:	
See AddColumn() method.	

SetRealFormat

Unpublished work. © 2025 Siemens

SetRealFormat	
(nWidth, nPrecision)	
Description:	
Specifies how real numbers are formatted when added to the cells of the table.	
Input:	
`INT4 nWidth`	The number of characters to use in writing the value.
`INT4 nPrecision`	The number of digits beyond the decimal point when writing the value.
Output:	
None	None
Return Code:	
`FE_INVALID`	An invalid format has been specified. Both nWidth and `nPrecision` must be positive, `nWidth` > `nPrecision` and `nWidth` <= 20.
Remarks/Usage:	
In the user interface, values shown in dialog boxes typically use a format similar to `SetRealFormat( 10, 7 )`, while values in the listing commands often use `SetRealFormat( 12, 7 )`. When real numbers are formatted, values may switch between a standard floating point and exponential representations based on the specified format sizes and the numeric values.	
Example:	
See `AddColumn()` method.	

SetTableStyle

SetTableStyle	
(nHeaderStyle, nBodyStyle)	
Description:	

Unpublished work. © 2025 Siemens

Specifies the styles used by the header and body sections of the table.	
Input:	
`INT4 nHeaderStyle`	The ID of a style that is defined in the `PublishTool` object that is used for both the top and left side header of the table.
`INT4 nBodyStyle`	The ID of a style that is defined in the `PublishTool` object that is used for the body portion of the table.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The styles specified must be defined using the `AddStyle( )` method of the associated `PublishTool` object. These styles are generally applied, but can be overridden for specific cells using the `SetCellStyle( )` method.	
Example:	
See `AddColumn()` method.	

SetCellStyle

`SetCellStyle` **`( nRow, nColumn, nStyle )`**	
Description:	
Selects a style to apply to a table cell.	
Input:	
`INT4 nRow`	The ID of the row to be updated. The first body row of the table is `nRow=0`.
`INT4 nColumn`	The ID of the column to be updated. This value is the return code from the `AddColumn()` method. However, just like rows, the first body column of the table is `nColumn=0`.

Unpublished work. © 2025 Siemens

`INT4 nStyle`	The ID of a style that is defined in the `PublishTool` object that is used for the selected cell.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method merely defines potential styles to cells, the style is not applied until the table is published. Styles selected by this method override the header and body styles selected by `SetTableStyle()`. You can override the header styles by specifying the following... `nRow = -1` The column title row. `nRow = -2` The column group title row. `nColumn = -1` The left side header column.	
Example:	
None	

SetCellReal

`SetCellReal` **`( nRow, nColumn, dValue )`**	
Description:	
Adds a real number value to a cell.	
Input:	
`INT4 nRow`	The ID of the row to be updated. The first body row of the table is `nRow=0`.
`INT4 nColumn`	The ID of the column to be updated. This value is the return code from the `AddColumn()` method. However, just like rows, the first body column of the table is `nColumn=0`.

Unpublished work. © 2025 Siemens

`REAL8 dValue`	A real number value to assign to the specified cell.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The specified row is not defined or could not be added.
`FE_FAIL`	Unable to assign the specified value to the cell.
Remarks/Usage:	
The format of the values shown in cells defined by this method is controlled by the `SetRealFormat( )` method. Only a single real, integer or text value can be assigned to a table cell.	
Example:	
See `AddColumn( )` method.	

SetCellInteger

`SetCellInteger`
`( nRow, nColumn, nValue )`
Description:
Adds an integer value to a cell.
Input:

`INT4 nRow`	The ID of the row to be updated. The first body row of the table is `nRow=0`.
`INT4 nColumn`	The ID of the column to be updated. This value is the return code from the `AddColumn( )` method. However, just like rows, the first body column of the table is `nColumn=0`.
`INT4 nValue`	An integer value to assign to the specified cell.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_AVAILABLE`	The specified row is not defined or could not be added.
`FE_FAIL`	Unable to assign the specified value to the cell.
Remarks/Usage:	
Only a single real, integer or text value can be assigned to a table cell.	
Example:	
None	

SetCellText

`SetCellText` **`( nRow, nColumn, sText )`**	
Description:	
Adds a text string to a cell.	
Input:	
`INT4 nRow`	The ID of the row to be updated. The first body row of the table is `nRow=0`.
`INT4 nColumn`	The ID of the column to be updated. This value is the return code from the `AddColumn()` method. However, just like rows, the first body column of the table is `nColumn=0`.
`STRING sText`	A text string to assign to the specified cell.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The specified row is not defined or could not be added.
`FE_FAIL`	Unable to assign the specified value to the cell.

Unpublished work. © 2025 Siemens

Remarks/Usage:
Only a single real, integer or text value can be assigned to a table cell. If more are required, you must format them as a string and use this method to apply them to the cell.
Example:
See `AddColumn()` method.

SetMultiCellStyle

`SetMultiCellStyle` **`( nCount, vRow, vColumn, vStyle )`**	
Description:	
Selects a style to apply to multiple table cells.	
Input:	
`INT4 nCount`	The number of entries in the `vRow`, `vColumn` and `vStyle` arrays.
`INT4 vRow[0..nCount-1]`	The IDs of the rows to be updated. The first body row of the table is `nRow=0`.
`INT4 vColumn[0..nCount-1]`	The IDs of the columns to be updated. This value is the return code from the `AddColumn()` method. However, just like rows, the first body column of the table is `nColumn=0`.
`INT4 vStyle[0..nCount-1]`	The IDs of styles that are defined in the `PublishTool` object that is used for the selected cell.
Output:	
None	None
Return Code:	
`FE_INVALID`	`nCount` must be greater than zero.
`FE_NO_MEMORY`	Insufficient memory to retrieve the multi cell information.

Unpublished work. © 2025 Siemens

`FE_TOO_SMALL`	One or more of the arrays does not contain enough values.
`FE_NOT_AVAILABLE`	Unable to access one or more of the arrays.
`FE_BAD_TYPE`	One or more of the arrays contains values of an incorrect type.
Remarks/Usage:	
This method operates just like calling `SetCellStyle( vRow(i), vColumn(i), vStyle(i) )` multiple times.	
Example:	
None	

SetMultiCellReal

`SetMultiCellReal` **`( nCount, vRow, vColumn, vValue )`**	
Description:	
Adds real number values to multiple table cells.	
Input:	
`INT4 nCount`	The number of entries in the `vRow`, `vColumn` and `vStyle` arrays.
`INT4 vRow[0..nCount-1]`	The IDs of the rows to be updated. The first body row of the table is `nRow=0`.
`INT4 vColumn[0..nCount-1]`	The IDs of the columns to be updated. This value is the return code from the `AddColumn()` method. However, just like rows, the first body column of the table is `nColumn=0`.
`INT4 vValue[0..nCount-1]`	The values to add to the specified cells.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

FE_INVALID	nCount must be greater than zero.
FE_NO_MEMORY	Insufficient memory to retrieve the multi cell information.
FE_TOO_SMALL	One or more of the arrays does not contain enough values.
FE_NOT_AVAILABLE	Unable to access one or more of the arrays.
FE_BAD_TYPE	One or more of the arrays contains values of an incorrect type.
Remarks/Usage:	
This method operates just like calling SetCelReal(vRow(i), vColumn(i), vValue(i)) multiple times.	
Example:	
None	

SetMultiCellInteger

SetMultiCellInteger	
(nCount, vRow, vColumn, vValue)	
Description:	
Adds integer values to multiple table cells.	
Input:	
INT4 nCount	The number of entries in the vRow, vColumn and vStyle arrays.
INT4 vRow[0..nCount-1]	The IDs of the rows to be updated. The first body row of the table is nRow=0.
INT4 vColumn[0..nCount-1]	The IDs of the columns to be updated. This value is the return code from the AddColumn() method. However, just like rows, the first body column of the table is nColumn=0.
INT4 vValue[0..nCount-1]	The values to add to the specified cells.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_INVALID`	`nCount` must be greater than zero.
`FE_NO_MEMORY`	Insufficient memory to retrieve the multi cell information.
`FE_TOO_SMALL`	One or more of the arrays does not contain enough values.
`FE_NOT_AVAILABLE`	Unable to access one or more of the arrays.
`FE_BAD_TYPE`	One or more of the arrays contains values of an incorrect type.
Remarks/Usage:	
This method operates just like calling `SetCellInteger( vRow(i), vColumn(i), vValue(i) )` multiple times.	
Example:	
None	

SetMultiCellText

`SetMultiCellText`
`( nCount, vRow, vColumn, vText )`
Description:
Adds text values to multiple table cells.
Input:

`INT4 nCount`	The number of entries in the `vRow`, `vColumn` and `vText arrays`.
`INT4 vRow[0..nCount-1]`	The IDs of the rows to be updated. The first body row of the table is `nRow=0`.
`INT4 vColumn[0..nCount-1]`	The IDs of the columns to be updated. This value is the return code from the `AddColumn()` method.

Unpublished work. © 2025 Siemens

	However, just like rows, the first body column of the table is `nColumn=0`.
`INT4 vText[0..nCount-1]`	The text to add to the specified cells.
Output:	
None	None
Return Code:	
`FE_INVALID`	`nCount` must be greater than zero.
`FE_NO_MEMORY`	Insufficient memory to retrieve the multi cell information.
`FE_TOO_SMALL`	One or more of the arrays does not contain enough values.
`FE_NOT_AVAILABLE`	Unable to access one or more of the arrays.
`FE_BAD_TYPE`	One or more of the arrays contains values of an incorrect type.
Remarks/Usage:	
This method operates just like calling `SetCellText( vRow(i), vColumn(i), vText(i) )` multiple times.	
Example:	
None	

Publish

`Publish`
`( void )`
Description:
Adds the contents of the current table into the associated `PublishTool` object.
Input:
None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	Current object is not properly associated with a `PublishTool` object.
`FE_FAIL`	Unable to add table into the `PublishTool` object.
Remarks/Usage:	
This method must be called after the entire contents of the table have been defined. The contents of the table are inserted into the object as it is called... just like the various *Add* methods in the `PublishTool` object.	
Example:	
See `AddColumn()` method.	

BodyMesher Objects

The `BodyMesher` object offers programmatic access to the capabilities of the **Mesh → Bodies** and **Mesh → Mesh on Mesh** commands.

The `BodyMesher` Object Properties are used to set options in the Automesh Bodies dialog box used by the **Mesh → Bodies** and **Mesh → Mesh on Mesh** commands. The `BodyMesher` Object Option Methods are used to set multiple properties on the `BodyMesher` object using a single method, calculate a default mesh size, set sizing on additional curves and/or surfaces, while the `BodyMesher` Object Operation Methods are used to mesh selected bodies or remesh the selected elements.

The appropriate process is to have the `BodyMesher` Properties (see **BodyMesher Object Properties**) specified or use the `BodyMesher` Option Methods (see **BodyMesher Object Option Methods**) to specify multiple parameters before calling one of the one of the `BodyMesher` Object Operation Methods (see **BodyMesher Object Operation Methods**).

Use the `feBodyMesher` method of the Femap Application object to create `BodyMesher` objects.

BodyMesher Object Properties

Unpublished work. © 2025 Siemens

Property	Description
Mesh Type	
`enum MeshType`	Specifies Type of Elements to create: Tri = `FTO_TRIA3 (2)` or `FTO_TRIA6 (3)` Quad = `FTO_QUAD4 (4)` or `FTO_QUAD8 (5)` Tet (**Mesh → Bodies** command only) = `FTO_TETRA4 (6)` or `FTO_TETRA10 (10)`
`BOOL MidsideNodes`	When True, activates the **Midside Nodes** option.
Mesh Sizing	
`REAL8 TargetSize`	Specifies value for **Target Element Size**.
`BOOL PointsAroundCircleOn`	When True, activates **Points Around Circle** option.
`INT4 PointsAroundCircle`	Specifies value for **Points Around Circle**.
`BOOL MinSizeFactorOn`	When True, activates **Min Element Size Factor** option.
`REAL8 MinSizeFactor`	Specifies value (percentage) for **Min Element Size Factor**.
`BOOL GrowthFactorOn`	When True, activates **Growth Factor** option.
`REAL8 GrowthFactor`	Specifies value (percentage) for **Growth Factor**.
Geometry tab - Mesh Associativity (Mesh → Bodies command only).	
`enum AssociateTo`	Specifies option for Mesh Associativity: All Geometry = `MPAS_ALL (0)` Required Geometry Only = `MPAS_REQD (1)` Additional Selected Geometry = `MPAS_SELECT (2)`
`BOOL AssociateAngleOn`	When True, activates **Additional Geometry by Angle** option.
`REAL8 AssociateAngle`	Specifies value for **Additional Geometry by Angle**.
Geometry tab - On Associated Geometry (Mesh → Bodies command only).	
`BOOL UseCurveSizing`	When True, activates **Use Curve Sizing** option.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL UseSurfaceSizing`	When True, activates **Use Surface Sizing** option.
Curvature/Proximity tab - Surface Curvature.	
`BOOL CurvatureAlignment`	When True, activates **Curvature Alignment** option.
`BOOL CurvatureRefinement`	When True, activates **Curvature Refinement** option.
`INT4 CurvatureRefinementMaxPoints`	Specifies value for **Max Points Around Circle**.
`REAL8 CurvatureRefinementLimit`	Specifies value (percentage) for **Refinement Deviation Limit**.
Curvature/Proximity tab - Surface Proximity.	
`BOOL ProximityRefinement`	When True, activates **Proximity Refinement** option.
`BOOL ProximityElemAcrossOn`	When True, activates **Elements Across** option.
`INT4 ProximityElemAcross`	Specifies value for **Elements Across**.
`BOOL ProximityDistanceOn`	When True, activates **Max Search Distance** option.
`REAL8 ProximityDistance`	Specifies value for **Max Search Distance**.
`INT4 ProximityDirection`	Specifies option for **Search Direction**: Inside = 0, Outside = 1, Both = 2
Mesh Quality tab - Mesh Quality .	
`BOOL MinQualityOn`	When True, activates **Min Tri/Quad Quality** option.
`REAL8 MinQuality`	Specifies value for **Min Tri/Quad Quality**.
`BOOL EnhancedQualityMesh`	When True, activates **Enhanced Quality Mesh** option.
`BOOL CompatibilityRefinement`	When True, activates **Compatibility Refinement** option.
`BOOL AllowMeshIntersections`	When True, activates **Allow Mesh Intersections** option.
Mesh Quality tab - Faceting Tolerances.	

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 FacetMaxSize`	Specifies value (percentage) for **Max Facet Size**.
`REAL8 FacetDistanceError`	Specifies value (percentage) for **Distance Error**.
`REAL8 FacetAngleError`	Specifies value (percentage) for **Angle Error**.
`REAL8 FacetChordError`	Specifies value (percentage) for **Chord Error**.
`REAL8 FacetCurveFactor`	Specifies value for **Curve Factor**.
`BOOL UseFaceter`	When True, activates **Use Faceter** option.
Midside Nodes tab (Mesh → Bodies command only).	
`Enum MidsideOntoGeometry`	`MBMM_NONE (0)` ... midside nodes along the element edges, this is the default. `MBMM_ASSOCIATED (1)` ... project midside nodes toward associated geometry.
`BOOL MidsideAngleLimitOn`	If True, allows for use of `MidsideAngleLimit`.
`REAL8 MidsideAngleLimit`	The maximum edge distortion angle when projecting midside nodes toward associated geometry.
Tet Options tab (Mesh → Bodies command only).	
`BOOL TetSliverRemoval`	When True, activates **Tet Sliver Removal** option.
`BOOL TetAllowVoids`	When True, activates **Allow Void Regions** option.
`BOOL TetMultipleThruThicknessOn`	When True, activates **Multiple Tet thru Thickness** option.
`INT4 TetMultipleThruThickness`	Specifies value for **Multiple Tet thru Thickness** (value should be specified with a value from 2 to 10).
`INT4 TetOptimizationLevel`	Specifies option for **Tet Optimization**: 0 = None, 1 = Minimal, 2 = between Minimal and Default, 3 = Default, 4 = between Default and Maximum, 5 = Maximum.
`REAL8 TetGrowthRatio`	Specifies value for **Tet Growth Ratio**.

Unpublished work. © 2025 Siemens

Property	Description
Mesh Features tab - Element Attributes to Retain (Mesh → Mesh on Mesh command only).	
`BOOL RetainProperty`	When True, enables **Property** option.
`BOOL RetainElementType`	When True, activates **Element Type** option, but is only used when `RetainProperty` is also set to True.
`BOOL RetainCornerThickness`	When True, activates **Corner Thickness** option.
`BOOL RetainOffset`	When True, activates **Offset** option.
`BOOL RetainNormalDirection`	When True, activates **Normal Direction** option.
`BOOL RetainColor`	When True, activates **Color** option.
`BOOL RetainLayer`	When True, activates **Layer** option.
`BOOL RetainAssociativity`	When True, activates **Associativity** option.
Mesh Features tab - Feature Control (Mesh → Mesh on Mesh command only).	
`REAL8 FeatureEdgeBreakAngle`	Specifies value (degrees) for **Edge Break**.
`BOOL KeepMeshOnFreeEdges`	When True, activates **Keep Mesh on Free Edges** option.
Mesh Features tab - Connectivity Mesh → Mesh on Mesh command only).	
`BOOL MergeNodes`	When True, activates **Merge Nodes** option.

BodyMesher Object Option Methods

Clear

Unpublished work. © 2025 Siemens

Clear **(bClearAll)**	
Description:	
Reset the contents of the `BodyMesher` object.	
Input:	
`BOOL bClearAll`	If True, resets all parameters and mesh contents in the `BodyMesher` object back to their default/ empty state. If False, only the mesh contents are cleared.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Setting `Clear` (FALSE) allows the object to be reused for multiple meshes without resetting all of the parameters.	
Example:	
None	

Dialog

Dialog **(bMeshingSolids, nSolidOrElemSet)**
Description:
Displays the dialog box shown in the **Mesh** → **Bodies** and **Mesh** → **Mesh on Mesh** commands to collect parameter settings.
Input:

Unpublished work. © 2025 Siemens

`BOOL bMeshingSolids`	If True displays the dialog box used by the **Mesh → Bodies** command. If False, displays the dialog box used by the **Mesh → Mesh on Mesh** command.
`INT4 nSolidOrElemSet`	The ID of a Set object containing the IDs of Solids (can include Sheet Solids and General Bodies as well) or Elements to be meshed. Alternatively, if you specify a negative value, it is simply the ID of the single entity to mesh.
Output:	
None	None
Return Code:	
`FE_INVALID`	`nSolidOrElemSet` must be nonzero.
`FE_CANCEL`	User canceled the dialog box.
Remarks/Usage:	
The entities specified in `nSolidOrElemSet` are not remeshed or added to the object in this method, they are simply used to properly initialize the dialog box.	
Example:	
None	

Size

`Size` **`( dTargetSize, dMinSizeFactor, dGrowthFactor, nPointsAroundCircle )`**
Description:
Single method to set multiple properties on the object to set mesh sizing parameters.
Input:

`REAL8 dTargetSize`	Sets value for `TargetSize` property on the object.
`REAL8 dMinSizeFactor`	Sets the `MinSizeFactorOn` and `MinSizeFactor` properties on the object. If

Unpublished work. © 2025 Siemens

	`dMinSizeFactor=0.0`, then `MinSizeFactorOn` = False.
`REAL8 dGrowthFactor`	Sets the `GrowthFactorOn` and `GrowthFactor` properties on the object. If `dGrowthFactor = 0.0`, `GrowthFactorOn` = False.
`INT4 nPointsAroundCircle`	Sets the `PointsAroundCircleOn` and `PointsAroundCircle` properties on the object. If `nPointsAroundCircle` < 3, `PointsAroundCircleOn` = False.
Output:	
None	None
Return Code:	
`FE_INVALID`	One or more parameters is specified with an invalid or unreasonable value.
Remarks/Usage:	
None	
Example:	
None	

UseSizing

`UseSizing`
`( bUseCurveSizing, bUseSurfaceSizing )`
Description:
Single method to set multiple properties on the object in the **On Associated Geometry** parameters.
Input:

`BOOL bUseCurveSizing`	Sets the `UseCurveSizing` property on the object.
`BOOL bUseSurfaceSizing`	Sets the `UseSurfaceSizing` property on the object.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Associate

Associate **(nAssociateTo, nExtraCurveSetID, nExtraSurfaceSetID, bAssociateAngleOn, dAssociateAngle)**	
Description:	
Single method to set properties on the object to set **Mesh Associativity** parameters and add extra geometry to be retained.	
Input:	
`INT4 nAssociateTo`	Sets the level of automatic mesh associativity to be retained. 0=All Geometry, 1=Required Geometry Only, 2=Retain additional geometry selected by `nExtraCurveSetID` and/or `nExtraSurfaceSetID`.
`INT4 nExtraCurveSetID`	The ID of a Set object containing the IDs of additional Curves to be retained in the final mesh when `nAssoicateTo` is set to a value of 2. Use 0 for None.
`INT4 nExtraSurfaceSetID`	The ID of a Set object containing the IDs of additional Surfaces to be retained in the final mesh when `nAssoicateTo` is set to a value of 2. Use 0 for None.
`BOOL bAssociateAngleOn`	If True, select extra geometry to retain by the angle between surfaces adjacent to curves.
`REAL8 dAssociateAngle`	The angle to be used to select extra geometry to be retained. Ignored if `bAssociateAngleOn` = False.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_INVALID`	Either invalid value for `nAssociateTo` or `dAssociateAngle`.
Remarks/Usage:	
None	
Example:	
None	

CalculateDefaultMeshSize

`CalculateDefaultMeshSize` **`( enDataType, nSolidOrElemSet, dSize )`**	
Description:	
Find the default mesh size that is used to initialize the target mesh size.	
Input:	
`enum enDataType`	Either `FT_ELEM` or `FT_SOLID`.
`INT4 nSolidOrElemSet`	The ID of a Set object containing the IDs of Solids (can include Sheet Solids and/or General Bodies) or Elements to be used to compute the target size.
Output:	
`REAL8 dSize`	The computed target size for meshing.
Return Code:	
`FE_INVALID`	Invalid entity type selected for `enDataType`.
Remarks/Usage:	
Calling this method will return the default target size and will also update the target size in the object.	

Unpublished work. © 2025 Siemens

Example:
None

SetSizeOnCurve

SetSizeOnCurve **(nCurveSet, dSize)**	
Description:	
Sets mesh sizing on selected curves.	
Input:	
`INT4 nCurveSet`	The ID of a Set object containing the IDs of Curves to set sizing. Alternatively, if you specify a negative value, it is simply the ID of the single curve to be set.
`REAL8 dSize`	The nominal element size along the curves.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set sizes on curves.
Remarks/Usage:	
This method is equivalent to calling the `feMeshSizeCurve` method with no biasing or other options. Mesh size along curves are only used if `UseCurveSizing` is set to True and the curves are retained in the final mesh.	
Example:	
None	

SetSizeOnSurfaces

Unpublished work. © 2025 Siemens

SetSizeOnSurfaces **(nSurfaceSet, dSize)**	
Description:	
Sets mesh sizing on selected surfaces.	
Input:	
`INT4 nSurfaceSet`	The ID of a Set object containing the IDs of Surfaces to set sizing. Alternatively, if you specify a negative value, it is simply the ID of the single surface to be set.
`REAL8 dSize`	The nominal element size along the curves of the surfaces.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set sizes on surfaces.
Remarks/Usage:	
This method is equivalent to calling the `feMeshSizeSurface` method with no options other than sizing. Mesh sizes on surfaces are only used if `UseSurfaceSizing` is True and the surfaces are retained in the final mesh.	
Example:	
None	

BodyMesher Object Operation Methods

MeshBodies

Unpublished work. © 2025 Siemens

MeshBodies

(nSolidSet, enElemShape, nPropID, nZeroAttrPropID)

Description:

Mesh Solids, Sheet Solids, and/or General Bodies with the selected parameters.

Input:

`INT4 nSolidSet`	The ID of a Set object containing the IDs of Solids (can include Sheet Solids and General Bodies as well) to be meshed. Alternatively, if you specify a negative value, it is simply the ID of the single entity to mesh.
`enum enElemShape`	The type of mesh to create. This can be `FTO_TRIA3 (2)`, `FTO_TRIA6 (3)`, `FTO_QUAD4 (4)`, `FTO_QUAD8 (5)`, `FTO_TETRA4 (6)`, or `FTO_TETRA10 (10)`.
`INT4 nPropID`	The ID of an existing Property to apply to the mesh elements. Specify `nPropID=0` to use the Meshing Attributes that have been previously applied to the geometry.
`INT4 nZeroAttachPropID`	If `nPropID=0`, and Meshing Attributes are being used, this property will be used for any geometry where Meshing Attributes are not specified.

Output:

None	None

Return Code:

`FE_INVALID`	Invalid element shape or no geometry specified to be meshed.
`FE_BAD_TYPE`	The specified property does not correspond to the element shape that was selected.
`FE_NOT_EXIST`	The specified property does not exist.
`FE_FAIL`	Meshing failed.

Remarks/Usage:

Calls the **Mesh** → **Bodies** command programmatically using the Object Properties currently specified on the `BodyMesher` object.

Unpublished work. © 2025 Siemens

Example:
None

MeshOnMesh

`MeshOnMesh` **`( nElemSet, enElemShape )`**	
Description:	
Remesh Elements with the selected parameters.	
Input:	
`INT4 nElemSet`	The ID of a Set object containing the IDs of Elements to be remeshed. Alternatively, if you specify a negative value, it is simply the ID of the single entity to remesh.
`enum enElemShape`	The type of mesh to create. This can be `FTO_TRIA3 (2)`, `FTO_TRIA6 (3)`, `FTO_QUAD4 (4)`, or `FTO_QUAD8 (5)`.
Output:	
None	None
Return Code:	
`FE_INVALID`	Invalid element shape or no geometry specified to be meshed.
`FE_FAIL`	Meshing failed.
Remarks/Usage:	
Calls the **Mesh → Mesh on Mesh** command programmatically using the Object Properties currently specified on the `BodyMesher` object.	
Example:	
None	

Unpublished work. © 2025 Siemens

SolidCleanupTool Objects

The SolidCleanupTool object offers programmatic access to the capabilities of the **Geometry → Solid → Cleanup** command.

All defaults in the object are set the same way that they are presented in the command's dialogs. The Object Properties can be used to change those options just as the check boxes and edit controls change them in the user interface. The object `Cleanup` method is used to process the selected solids similar to what happens when pressing OK in the user interface dialog box.

Use the `feSolidCleanupTool` method of the Femap Application object to create SolidCleanupTool objects.

> **Note**
> NonManifold and Curve-Only solids are not supported by this tool and if selected will be skipped.

Solid Cleanup Tool Object Properties

Property	Description
Basic Dialog	
`BOOL CheckGeometry`	True or False for checking the validity of a solid that you are creating.
`BOOL MatchModelScaleFactor`	True or False to adjust the internal scale factor of each piece of selected geometry to match the Solid Geometry Scale Factor that is currently set.
`BOOL RemoveRedundantGeometry`	True or False for removing geometry that is not required to define the volume of the solid.
`BOOL ShowProgressMessages`	True or False to show progress of the selected geometry validation and/or geometry cleanup operations.
`BOOL AdvCleanInvalidGeometry`	True or False to activate the Geometry Cleaning options in Advanced Cleanup Options.
`BOOL AdvRemoveSmallFeatures`	True or False to activate the Small Feature Removal options in Advanced Cleanup Options.
`BOOL AdvOptimize`	True or False to activate the Geometry Optimization options in Advanced Cleanup Options.
`BOOL AdvStitch`	True or False to activate the Stitch Sheets into Solid options in Advanced Cleanup Options.

Unpublished work. © 2025 Siemens

Property	Description
	Note For option to become available, at least one Sheet Solid must be selected.
Advanced Options Dialog	
`BOOL AdvCleanValidGeometry`	True or False to use cleaning operations or clean if errors are found.
`BOOL AdvRepairEdges`	True or False to repair edges with a defined tolerance.
`BOOL AdvSmoothDiscontinuity`	True or False to remove surface or curve G1 discontinuities based on a defined tolerance.
`BOOL AdvRemoveSelfIntersection`	True or False to remove self-intersections, which lie outside face boundaries. The portion of the surface will be removed by splitting the surface.
`BOOL AdvAllowSurfaceMods`	True or False to allow geometry to be modified or be completely left alone during cleaning.
`BOOL AdvRemoveSpikes`	True or False to attempt to heal surface trimming curves that have “spikes”.
`REAL8 AdvRemoveSpikesTolerance`	Tolerance for healing surface trimming curves that have “spikes”.
`BOOL AdvRemoveSmallEdges`	True or False to remove very short edges below the length specified.
`REAL8 AdvRemoveSmallEdgesTolerance`	Tolerance (max edge length) for removal of very short edges.
`BOOL AdvRemoveSmallFaces`	True or False to remove very small faces below the tolerance specified.
`REAL8 AdvRemoveSmallFacesTolerance`	Tolerance (radius of encompassing sphere) for removal of very small faces.
`BOOL AdvRemoveSlivers`	True or False to remove insignificant faces with high aspect ratios, and a small area.
`REAL8 AdvRemoveSliversTolerance`	Tolerance (area) for removal of very small faces with high aspect ratios, and a small area.
`BOOL AdvRemoveGashes`	True or False to remove insignificant gashes (openings) with high aspect ratios, and a small area.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 AdvRemoveGashesTolerance`	Tolerance (area) for removal of very small openings with high aspect ratios, and a small area.
`BOOL AdvSimplifyGeometry`	True or False to convert B-Spline curves and surfaces to simplified analytic geometry whenever possible.
`REAL8 AdvSimplifyGeometryTolerance`	Tolerance for conversion of B-Spline curves and surfaces to simplified analytic geometry.
`BOOL AdvImproveEdges`	True or False to heal inaccuracies in the edges of a solid or surface. Repairs edge and vertex geometry by recalculating geometry that does not meet precisely.
`REAL8 AdvImproveEdgesTolerance`	Tolerance to which the edges are recomputed to meet the other constraints imposed by the model.
`BOOL AdvMergeSharedGeometry`	True or False to remove redundant edges in the model after healing inaccurate edges/surfaces.
`BOOL AdvReapplyBlends`	True or False to identify blend faces and attempt to simplify, then reapply blended surfaces.
`REAL8 AdvStitchTolerance`	Tolerance for stitching sheet solids.
`BOOL AdvStitchSmoothDiscontinuity`	True or False for removing G1-discontinuities in curves and surfaces, and to make closed geometry periodic.
`REAL8 AdvStitchSmoothDiscontinuityTolerance`	Tolerance in degrees for removing G1-discontinuities in curves and surfaces.
`BOOL AdvStitchReplaceMissingGeometry`	True or False for fitting surfaces in any remaining holes in the model to close it into a solid.

Solid Cleanup Tool Object Methods

SetTolerances

Unpublished work. © 2025 Siemens

SetTolerances	
(dTolerance)	
Description:	
Updates all tolerance properties in the object to be a specified value.	
Input:	
`REAL8 dTolerance`	Tolerance to apply throughout the object.
Output:	
None	None
Return Code:	
`FE_FAIL`	FE_FAIL Tolerance is outside of acceptable range.
Remarks/Usage:	
Use this method to quickly update all of the tolerances then, if required, update individual property tolerances to other values. Use of this method is not necessary if the default tolerances are acceptable.	
Example:	
None	

Dialog

Dialog	
(void)	
Description:	
Displays the same dialogs as the user interface to allow user specification of options.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	User pressed Cancel to close the dialog.
Remarks/Usage:	
None	
Example:	
None	

Cleanup

`Cleanup` **`( nSolidSet, bEnableAdvancedCleanup )`**	
Description:	
This method enables the Advanced Cleanup Options for the Solid Cleanup command.	
Input:	
`INT4 nSolidSet`	The ID of a Set object containing the IDs of Solids to clean. Alternatively, if you specify a negative value, it is the ID of the single solid to clean.
`BOOL bEnableAdvancedCleanup`	Set to True to process whatever Advanced Cleanup options chosen by the SolidCleanupTool object properties. If False, no advanced processing is performed regardless of the property settings.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	No acceptable solids selected.
`FE_FAIL`	Cleanup failed.
Remarks/Usage:	
Only call this method if you want user specification/verification of the tool properties.	

Unpublished work. © 2025 Siemens

Example:
None

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

5. Femap Entity Objects

In addition to the various methods and properties that are available directly from the main Femap application object, and the tool objects described earlier, Femap also provides objects that allow you to query and manipulate the data in your model. You will find that these objects give you the detailed control that is often necessary to implement your own functionality.

Using the Documentation for Entity Objects

All of the entity objects share common functionality to retrieve, store and search your model data. Rather than describe all of this functionality for each object, these common functions are described in **Common Entity Properties and Methods**. Each of the individual entity objects then describes only the additional functionality provided by that object.

Creating an Object

To link your application to Femap, you had to declare an object and then connect it to Femap. In Basic, this is done via the `GetObject` or `CreateObject` functions. Creating these additional Femap objects is similar. First you must declare an object. Then, however, instead of using the standard system level functions, you use one of the following Femap functions to actually create the object. Once the object has been created, all of the properties and methods for that type of object are available for use.

All of the following methods are available from the Femap application object, and return the object that they create.

Entity Objects

Method	Description	Other Methods
`feAeroPanel` (**Aero Panel/Body Objects**)	Creates an Aero Panel/ Body Object	
`feAeroProp` (**Aero Property Objects**)	Creates an Aero Property Object	
`feAeroSpline` (**Aero Spline Objects**)	Creates an Aero Spline Object	
`feAeroSurf` (**Aero Surface Objects**)	Creates an Aero Control Surface Object	
`feAnalysisCase` (**AnalysisCase Objects**)	Creates an AnalysisCase Object	`AnalysisMgr.Case`
`feAnalysisMgr` (**AnalysisMgr Objects**)	Creates an AnalysisMgr Object	

Unpublished work. © 2025 Siemens

Method	Description	Other Methods
`feAnalysisStep` (AnalysisStep Objects)	Creates an AnalysisStep Object	`AnalysisCase.PutStep`
`feAnalysisStudy` (AnalysisStudy Objects)	Creates an AnalysisStudy Object	
`feBCDefinition` (BCDefinition Objects)	Creates a BCDefinition Object	`BCSet.NextBCDef`
`feBCEqn` (BCEqn Objects)	Creates a BCEqn Object	`BCSet.NextBC`, `BCDefinition.NextBC`
`feBCGeom` (BCGeom Objects)	Creates a BCGeom Object	`BCSet.NextBC`, `BCDefinition.NextBC`
`feBCNode` (BCNode Objects)	Creates a BCNode Object	`BCSet.NextBC`, `BCDefinition.NextBC`
`feBCSet` (BCSet Objects)	Creates a BCSet Object	
`feChart` (Chart Objects	Creates a Chart Object	
`feChartSeries` (Chart Data Series Objects)	Creates a Chart Data Series Object	
`feComputedResultsVectors` (Computed Vector Objects	Creates a Computed Vector Object	
`feConnectionProp` (Connection Property Objects)	Creates a Connection Property	`Connection.Prop`
`feConnectionRegion` (Connection Region Objects)	Creates a Connection Region / Contact Object	`feContact`
`feConnection` (Connection Objects)	Creates a Connection	
`feCSys` (CSys Objects)	Creates a CSys Object	
`feCurve` (Curve Objects)	Creates a Curve Object	
`feDataSurface` (Data Surface Objects)	Creates a Data Surface Object	
`feFlexibleSlider` (Design Equation Objects)	Creates a Flexible Slider Object.	
`feMatrixInput` (Direct Matrix Input Objects)	Creates a Direct Matrix Input Object	
`feDiscreteValueSet` (Discrete Value Set Object)	Creates a Discrete Value Set Object	

Unpublished work. © 2025 Siemens

Method	Description	Other Methods
`feElem` (Element Objects)	Creates an Element Object	
`feElemAddRemove` (Element Add | Remove Objects)	Creates an Element Add | Remove Set	
`feFlexibleSlider` (Flexible Slider Objects)	Creates an FlexibleSlider Object	
`feFreebody` (Freebody Objects)	Creates an Freebody Object	
`feFrequency` (Frequency Objects)	Creates an Frequency Object	
`feFunction` (Function Objects)	Creates a Function Object	
`feGFXArrow` (GFXArrow Objects)	Creates a GFXArrow Object	
`feGFXLine` (GFXLine Objects)	Creates a GFXLine Object	
`feGFXPoint` (GFXPoint Objects)	Creates a GFXPoint Object	
`feGFXQuad4` (GFXQuad4 Objects)	Creates a GFXQuad4 Object	
`feGFXTria3` (GFXTria3 Objects)	Creates a GFXTria3 Object	
`feGlobalPly` (GlobalPly Objects)	Creates a GlobalPly Object	
`feGlobalStep` (GlobalStep Objects)	Creates an GlobalStep Object	
`feGroup` (Group Objects)	Creates a Group Object	
`feJoint` (Kinematic Joint and Joint Connection Objects)	Creates a Joint Object	
`feLayer` (Layer Objects)	Creates a Layer Object	
`feLayup` (Layup Objects)	Creates a Layup Object	`Prop.Layup`
`feLoadBolt` (LoadBolt Object)	Creates a LoadBolt Object	

Unpublished work. © 2025 Siemens

Method	Description	Other Methods
`feLoadDefinition` (LoadDefinition Objects)	Creates a LoadDefinition Object	`LoadSet.NextLoadDef`
`feLoadETemp` (LoadETemp Objects)	Creates a LoadETemp Object	`LoadSet.NextLoad`, `LoadDefinition.NextLoad`
`feLoadGeom` (LoadGeom Objects)	Creates a LoadGeom Object	`LoadSet.NextLoad`, `LoadDefinition.NextLoad`
`feLoadMesh` (LoadMesh Objects)	Creates a LoadMesh Object	`LoadSet.NextLoad`, `LoadDefinition.NextLoad`
`feLoadNTemp` (LoadNTemp Objects)	Creates a LoadNTemp Object	`LoadSet.NextLoad`, `LoadDefinition.NextLoad`
`feLoadSet` (LoadSet Objects)	Creates a LoadSet Object	
`feMapOutput` (MapOutput Objects)	Creates a MapOutput Object	
`feMatl` (Material Objects)	Creates a Material Object	`Element.Matl`, `Prop.Matl`
`feMeshHardPoint` (Mesh Point Objects)	Creates a Mesh Point Object	
`feMeshHardPointDefinition` (Mesh Point Definition Objects)	Creates a Mesh Point Definition Object	
`feMonitorPoint` (Monitor Point Objects)	Creates a Monitor Point Object	
`feNode` (Node Objects)	Creates a Node Object	
`feOptim` (Optim Objects)	Creates an Optimization Object	
`feOptMC` (Optimization Manufacturing Constraints Objects)	Creates an Optimization Manufacturing Constraint Object	
`feOptRel` (Optimization Relationship (Variable) Objects)	Creates an Optimization Relationship (Variable) Object	

Unpublished work. © 2025 Siemens

Method	Description	Other Methods
`feOptResp` (Optimization Response (Limits) Objects)	Creates an Optimization Response (Limits) Object	
`feOutput` (Output Objects (OBSOLETE))	Creates an Output Object	`Output.Corner`, `Output.X`, `Output.Y`, `Output.Z`
`feOutputSet` (OutputSet Objects)	Creates an OutputSet Object	`Output.Vector`
`fePlane` (Plane Objects)	Creates a Plane Object	
`fePlyMaterial` (Ply Material Objects)	Creates a Ply Material Object	
`fePoint` (Point Objects)	Creates a Point Object	
`feProp` (Property Objects)	Creates a Property Object	`Element.Prop`
`feReference` (Reference Objects)	Creates a Reference Object	
`feReport` (Report Objects)	Creates a Report Object	
`feSEReference` (Superelement Reference Objects)	Creates an Superelement Object	
`feSolid` (Solid Objects)	Creates a Solid/Volume Object	
`feSurface` (Surface Objects)	Creates a Surface Object	
`feTableData` (Table Data Objects)	Creates a Table Data Object	
`feText` (Text Objects)	Creates a Text Object	
`feTMGBC` (TMGBC Objects)	Creates a TMGBc Object	
`feTMGCtrl` (TMGCtrl Objects)	Creates a TMGCtrl Object	
`feTMGInt` (TMGInt Objects)	Creates a TMGInt Object	
`feTMGOpt` (TMGOpt Objects)	Creates a TMGOpt Object	

Unpublished work. © 2025 Siemens

Method	Description	Other Methods
`feTMGReal` (TMGReal Objects)	Creates a TMGReal Object	
`feVar` (Variable Objects)	Creates a Var Object	
`feVector` (Vector Objects)	Creates a Vector Object	
`feView` (View Objects)	Creates a View Object	
`feViewOrient` (ViewOrient Objects)	Creates a View Orient Object	

Other Objects

Method	Description	Other Methods
`feUserData` (UserData Objects)	Creates a UserData Object.	

Example of Creating an Object

The following shows a sample (in Basic) of how to create a Node object, and use it to retrieve Node 100, update its X coordinate, and save the updated location in your model. This example assumes that Femap is running and has a model that contains Node 100.

```
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim nd As femap.Node
Set nd = App.feNode

rc     = nd.Get(100)
nd.x   = 50.5
rc     = nd.Put(100)

End Sub
```

Destroying an Object

Objects are destroyed automatically when your application ends. However, you can, and should, destroy them yourself as soon as you are done using them. By destroying objects sooner, you will minimize the amount of memory that is required at any one time by your application and by Femap.

Unpublished work. © 2025 Siemens

From Basic, you can destroy an object simply by changing its value. For example, if you had:

```
Dim femap As Object
Set femap = GetObject(, "femap.model")
Dim nd As Object
Set nd = femap.feNode
...
Set nd = femap.feNode
...
```

The first `nd` object was destroyed as soon as you made the second `Set` statement. This, however, creates a second node object that will be destroyed when your application terminates. Similarly, you can do the following:

```
Dim femap As Object
Set femap = GetObject(, "femap.model")
Dim nd As Object
Set nd = femap.feNode
...
Set nd = Nothing
...
```

This destroys the `nd` object without creating another.

Common Entity Properties and Methods

Entity objects are ones that contain the data that is part of your Femap model. This includes nodes, elements, materials, properties, points, and curves. All of the properties and methods described here can be accessed from any entity object, no matter what its type. As you will see, most of these methods deal with traversing the entities in the model, or retrieving or storing entity data. You will find properties and methods that are specific to an entity type documented with that object.

Common Entity Properties

Property	Description
`INT4 ID`	The ID of the entity that has been loaded. This property can be retrieved any time. It is automatically updated by any method that loads or saves entity data. It is not recommended to change this property directly. You can do it, but simply

Unpublished work. © 2025 Siemens

Property	Description
	doing that does not load the data for the entity with the new ID.
`INT4 SetID`	The `SetID` of the entity that has been loaded. For many entity types, this will always be 1. All entities that are not stored in sets are stored with a `SetID` = 1. Loads, constraints, and other set-based data are stored with `SetID` equal to the ID of the appropriate set. Unlike the ID property, you can and should update this property to be the set that you want to use to retrieve and store entities. You can do this either directly or indirectly with the `NextSet`, `PrevSet`, `NextEmptySet`, or `PrevEmpty` Set methods.
`INT4 Active`	The ID of the active entity. This is normally used for set-based entities like load sets, output sets, etc. to choose the active set. It is also used for properties and materials to choose the active entity.

Common Entity Methods

Get

`Get` **`( id )`**	
Description:	
This method retrieves an entity from your model database with the specified ID.	
Input:	
`INT4 id`	ID of the entity to retrieve. If you specify `id=0`, then the current value of the ID property will be used instead.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
This method retrieves the data for the selected entity and updates the ID property.	
Example:	
None	

GetLibrary

`GetLibrary` **`( libINDEX, libFile )`**	
Description:	
This method retrieves an entity from the associated library.	
Input:	
`INT4 libINDEX`	The index of the entity stored in the library. The first entity stored is `libINDEX` = 0, the second has `libINDEX` = 1, and so on. If you specify `libINDEX` = -1, a dialog box will be displayed showing all entities in the library, and will allow the user to select the entity to load.
`INT4 or enum libFILE`	Use `zLibraryFile` enumeration. Alternatively, enter value for Type of library: 0=Open Dialog Box, 1=Personal, 2=Shared, 3=Femap Standard Libraries.
Output:	
None	None
Return Code:	
`FE_FAIL`	No Material was selected. Either the library is empty or does not contain any Materials of the matching type (e.g. 3D Orthotropic).
`FE_CANCEL`	User canceled the dialog box to select a Material from the library.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Library file was opened but isn't a valid library of the correct type.
`FE_NOT_EXIST`	The selected index in the library was not found.
`FE_NOT_AVAILABLE`	The library file could not be accessed. Either the filename is specified incorrectly in Femap, or there is some other permission or corruption problem with the file.
`FE_BAD_TYPE`	Tried to call `GetLibrary` from an entity/object type that does not support libraries

Remarks/Usage:

When using this method with a Material object, you must set the *Active Material Type* using `App.Info_ActiveType (FT_MATL)` = *Type of Material to Get from Library* before calling the `GetLibrary` method. To avoid setting the *Active Material Type*, use the `GetLibraryOfType` method on the Material Object.

When using this method with a Property object, you must set the *Active Element Type* using `App.Info_ActiveType (FT_ELEM)` = *Type of Element/Property to Get from Library* before calling the `GetLibrary` method. Setting the *Active Property Type* will NOT work as expected. To avoid setting the *Active Element Type*, use the `GetLibraryOfType` method on the Property Object.

Example:

None

Put

`Put`

`( id )`

Description:

This method stores an entity in the model database with the specified ID.

Input:

`INT4 id`	The ID of the entity to store. If you specify id=0, then the current value of the ID property will be used instead.

Output:

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
All of the data that has been currently loaded/updated is stored as the entity with the specified ID. The ID where you store the data does not have to be the same as the ID that you used in a previous Get. For example, calling `Get(3)`, then `Put(6)`, creates a new entity with an ID of 6 that is a copy of entity 3.	
Example:	
None	

PutLibrary

`PutLibrary` **`( libINDEX, libFILE )`**	
Description:	
This method stores an entity in the selected index of the specified library file.	
Input:	
`INT4 libINDEX`	If `libINDEX` = -1 (actually just < 0), the entity will be added to the end of the library. If a library does not exist, a new library will be created. If `libINDEX` = 0, the entity will be inserted as the first entry in the library. Also, if no library currently exists or the library is empty creates a new library. If `libINDEX` > 0, the entity is inserted before the `libINDEX'th` entry in the existing library.
`INT4 or enum libFILE`	Use `zLibraryFile` enumeration. Alternatively, enter value for Type of library: 0=Open Dialog Box, 1=Personal, 2=Shared, 3=Femap Standard Libraries.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to complete the insertion of the entity into the library.
`FE_NOT_EXIST`	The library does not exist. Can only happen when `libINDEX` > 0.
`FE_NOT_AVAILABLE`	Less than `libINDEX` entries were found in the library so the entity cannot be inserted at that location.

Remarks/Usage:

For most entity types, the current data that you have loaded into the object can be stored directly to the library. There is no need to necessarily store it in the model database. For several entity types, such as functions and analysis sets, the object must first be stored in the database. For these entities, it is actually the data from the database that is written to the library, not the data in the current object. The ID of the current object must identify the database entity that you want written in the library.

Example:

None

GetTitleList

`GetTitleList`

`( minID, maxID, count, listID, listTITLE )`

Description:

This method retrieves two arrays which contain the IDs and titles of all entities (in the selected ID range) with the same type as the current object.

Input:

`INT4 minID`	The minimum and maximum entity IDs to retrieve. You can set `minID=maxID=0` to retrieve all available entities.
`INT4 maxID`	

Output:

`INT4 count`	The number of entities retrieved. This is the number of entries in `listID` and `listTITLE`.
`INT4 listID[0..count-1]`	The IDs of the available entities.

Unpublished work. © 2025 Siemens

`STRING listTITLE[0..count-1]`	The titles of the available entities. If the entity does not have a title, the string “Untitled” will be returned.
Return Code:	
`FE_FAIL`	This function is only available for entity types that have titles (**Material** → **Property** → **View**, etc.). If you call this method from another entity type, it will fail.
Remarks/Usage:	
This method is identical to calling `GetTitleIDList( False, ....)`. See that method for more information.	
Example:	
None	

GetTitleIDList

`GetTitleIDList` **`( bIncludeID, minID, maxID, count, listID, listTITLE )`**	
Description:	
This method retrieves two arrays which contain the IDs and titles of all entities (in the selected ID range) with the same type as the current object.	
Input:	
`BOOL bIncludeID`	If True, the titles are in the form “ID..title” (for example, “1..Steel”). If False, only the title, without the ID or separator is returned.
`INT4 minID`	The minimum and maximum entity IDs to retrieve. You can set `minID=maxID=0` to retrieve all available entities.
`INT4 maxID`	
Output:	
`INT4 count`	The number of entities retrieved. This is the number of entries in `listID` and `listTITLE`.

Unpublished work. © 2025 Siemens

`INT4 listID[0..count-1]`	The IDs of the available entities.
`STRING listTITLE[0..count-1]`	The titles of the available entities. If the entity does not have a title, the string “Untitled” will be returned.
Return Code:	
`FE_FAIL`	This function is only available for entity types that have titles (**Material** → **Property** → **View**, etc.). If you call this method from another entity type, it will fail.
Remarks/Usage:	
This method is somewhat different than the other entity methods. It does not really operate on, or tell about the current entity. It simply uses it to select the type of entity that you are interested in, and to choose the `setID` of interest for certain entity types - like output objects. Calling this method with `bIncludedID=False`, is the same as calling `GetTitleList( )`.	
Example:	
None	

ParseTitleID

`ParseTitleID` **`( sTitleID )`**	
Description:	
Returns the ID from a string that was formatted by `GetTitleIDList` in the form “ID..title”	
Input:	
`STRING sTitleID`	This must be a string in the format “ID..title”
Output:	
None	None
Return Code:	
`ID`	This is the ID contained in the string. 0 if the string has the wrong format.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:

```
Sub ListOutputVectors()

'Attach to FEMAP.....................
Dim femap As Object
Set App = feFemap()

'Look at Output Set 3.................
Dim oc As Object

Set oc = femap.feOutputSet()
oc.Get (1)
Dim od As Object

Set od = oc.Vector(1)

Dim rc As Integer
Dim Count As Long
Dim ID As Variant
Dim Title As Variant
Dim Row As Integer

'Retrieve ID and Title of all available Output Vectors...
rc = od.GetTitleIDList(True, 0, 0, Count, ID, Title)
' Display list in cells of a spreadsheet.................
For Row = 1 To Count
  Worksheets(1).Cells(Row, 1).Value = _
     od.ParseTitleID(Title(Row - 1))
  Worksheets(1).Cells(Row, 2).Value = Title(Row - 1)
Next Row

End Sub
```

Exist

Exist

(id)

Description:
This method checks to see if a specified entity currently exists.

Unpublished work. © 2025 Siemens

Input:	
`INT4 id`	The ID of the entity to check. If you specify id=0, then the current value of the ID property will be used instead.
Output:	
None	None
Return Code:	
`FE_FAIL`	The selected entity does not exist.
Remarks/Usage:	
This method does not load any data. It simply returns `FE_OK` if the entity exists, and `FE_FAIL` if it does not.	
Example:	
None	

Deletable

`Deletable`
`( id )`
Description:
This method checks to see if an entity can be deleted.
Input:

`INT4 id`	The ID of the entity to check. If you specify `id=0`, then the current value of the ID property will be used instead.
Output:	
None	None
Return Code:	
`FE_FAIL`	The entity is not deletable.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Entities are deletable only if there are no other entities in the model that require them to exist. For example, nodes cannot be deleted until after all elements that reference them have been deleted.
Example:
None

Delete

`Delete` **`( id )`**	
Description:	
This method deletes an entity from the model database.	
Input:	
`INT4 id`	The ID of the entity to delete. If you specify `id=0`, then the current value of the ID property will be used instead.
Output:	
None	None
Return Code:	
`FE_OK`	The entity was deleted.
`FE_FAIL`	The entity could not be deleted.
Remarks/Usage:	
Entities are not deletable if there are other entities in the Femap model that require them to exist. Certain objects, like `LoadSet` and `BCSet` objects, will also automatically delete all of the other entities that they contain if you delete them. For example, if you delete a `LoadSet`, all of the individual loads on nodes, elements or geometry in that set will also be deleted.	
Example:	
None	

DeleteLibrary

Unpublished work. © 2025 Siemens

DeleteLibrary	
(libINDEX, libFILE)	
Description:	
This method deletes an entity from the selected index of the specified library file.	
Input:	
`INT4 libINDEX`	The index of the entity stored in the library. The first entity stored is `libINDEX` = 0, the second has `libINDEX` = 1, and so on. If you specify `libINDEX` = -1, a dialog box will be displayed showing all entities in the library, and will allow the user to select the entity to delete.
`INT4 or enum libFILE`	Use `zLibraryFile` enumeration. Alternatively, enter value for Type of library: 0=Open Dialog Box, 1=Personal, 2=Shared, 3=Femap Standard Libraries.
Output:	
None	None
Return Code:	
`FE_FAIL`	This function is only available for entity types that have libraries (Material, Property, View, etc.). If you call this method from another entity type, it will fail.
Remarks/Usage:	
You must use this method with caution. There is no option to allow the user to confirm the deletion and no option to undo the operation. Once entities are deleted from a library, they cannot be recovered.	
Example:	
None	

Reset

Unpublished work. © 2025 Siemens

Reset	
(void)	
Description:	
This method resets the `Next` pointer so that the next call to `Next` will return the first (lowest ID) entity.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
After calling `Reset`, `Next` will return the first entity and `Prev` will return the last (highest ID) entity.	
Example:	
None	

Next

Next	
(void)	
Description:	
This method retrieves the next available entity with a larger ID.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No additional entities were found.
Remarks/Usage:	
Calling this method simply finds the next available entity with an ID that is greater than the current ID, and calls the Get method with that ID.	
Example:	
None	

NextInSet

`NextInSet` **`( setID )`**	
Description:	
This method retrieves the next available entity with a larger ID that is referenced in the Set.	
Input:	
`INT4 setID`	The ID of a Set object that contains the IDs of the entities that you want to iterate through.
Output:	
None	None
Return Code:	
`FE_FAIL`	No additional entities were found.
Remarks/Usage:	
Calling this method simply finds the next available entity that is in the specified Set object that has an ID that is greater than the current ID, and calls the Get method with that ID. So, to start at the beginning of the Set, first call the `Reset()` method on the current entity (not on the Set), or call `FirstInSet()`.	
Example:	
None	

Prev

Unpublished work. © 2025 Siemens

Prev	
(void)	
Description:	
This method retrieves the next available entity with a smaller ID.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	No additional entities were found.
Remarks/Usage:	
Calling this method simply finds the next available entity with an ID that is smaller than the current ID, and calls the `Get` method with that ID.	
Example:	
None	

First

First	
(void)	
Description:	
This method loads the first (with the lowest ID) available entity.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No entities exist, or unable to load.
Remarks/Usage:	
Calling this method is like calling `Reset`, followed by `Next`.	
Example:	
None	

FirstInSet

`FirstInSet` **`( setID )`**	
Description:	
This method loads the first (with the lowest ID) available entity that is referenced in the Set.	
Input:	
`INT4 setID`	The ID of the set that contains the entities that you want to iterate through.
Output:	
None	None
Return Code:	
`FE_FAIL`	No entities exist, or unable to load.
Remarks/Usage:	
Calling this method is like calling `Reset`, followed by `NextInSet()`.	
Example:	
None	

Last

Unpublished work. © 2025 Siemens

Last	
(void)	
Description:	
This method loads the last (with the highest ID) available entity.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	No entities exist, or unable to load.
Remarks/Usage:	
Calling this method is like calling `Reset`, followed by `Prev`.	
Example:	
None	

NextSet

NextSet	
(void)	
Description:	
This method updates the `setID` to be the next set that contains entities of this type.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more sets are found, `setID` remains unchanged. This method finds the next set with a higher ID.	
Example:	
None	

PrevSet

`PrevSet` **`( void )`**	
Description:	
This method updates the `setID` to be the previous set that contains entities of this type.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more sets are found, `setID` remains unchanged. This method finds the next set with a lower ID.	
Example:	
None	

NextEmptySet

Unpublished work. © 2025 Siemens

NextEmptySet	
(void)	
Description:	
This method updates the `setID` to be the next set of this entity type that is empty.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more empty sets are found, `setID` remains unchanged. This method finds the next set with a higher ID.	
Example:	
None	

PrevEmptySet

PrevEmptySet	
(void)	
Description:	
This method updates the `setID` to be the previous set of this entity type that is empty.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No more sets available.
Remarks/Usage:	
If no more empty sets are found, `setID` remains unchanged. This method finds the next set with a lower ID.	
Example:	
None	

NextID

`NextID` **`( void )`**	
Description:	
This method finds the ID of the next available entity with a higher ID.	
Input:	
None	None
Output:	
None	None
Return Code:	
`ID`	If return value is less than or equal to `MAX_LABEL`, this is the entity that was found. Otherwise, there are no more available entities.
Remarks/Usage:	
This method is the same as `Next`, except that it does not update the current ID, nor does it load the data for the entity.	
Example:	
None	

PrevID

Unpublished work. © 2025 Siemens

PrevID	
(void)	
Description:	
This method finds the ID of the next available entity with a lower ID.	
Input:	
None	None
Output:	
None	None
Return Code:	
`ID`	If return value is greater than or equal to 1, this is the entity that was found. Otherwise, there are no more available entities.
Remarks/Usage:	
This method is the same as `Prev`, except that it does not update the current ID, nor does it load the data for the entity.	
Example:	
None	

NextEmptyID

NextEmptyID	
(void)	
Description:	
This method finds the ID of the next nonexisting entity with a higher ID.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`ID`	If return value is less than or equal to `MAX_LABEL`, this is the entity ID that was empty. Otherwise, there are no more available empty entities.
Remarks/Usage:	
None	
Example:	
None	

PrevEmptyID

`PrevEmptyID`	
`( void )`	
Description:	
This method finds the ID of the next nonexisting entity with a lower ID.	
Input:	
None	None
Output:	
None	None
Return Code:	
`ID`	If return value is greater than or equal to 1, this is the entity that was empty. Otherwise, there are no more available empty entities.
Remarks/Usage:	
None	
Example:	
None	

SelectID

Unpublished work. © 2025 Siemens

SelectID **(title)**	
Description:	
This function displays a selection dialog box to allow a user to choose a single entity of a specific type. The selected entity is automatically loaded into the calling entity.	
Input:	
`STRING title`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.
Output:	
None	None
Return Code:	
`FE_CANCEL`	The user cancelled the selection.
`FE_NOT_EXIST`	No entities of the selected type exist. None were selected and the dialog box was not displayed.
Remarks/Usage:	
This function displays one of two different dialog boxes, depending on the entity type that you choose. For entities that have titles, like properties or materials, the dialog box will contain a list of the available entities along with their titles. For other entities, like nodes or elements, a simple dialog box is displayed so that the ID can be typed or picked. This method is the same as the `SelectID` method in the Set Object, however instead of simply adding the entity ID into the Set, it retrieves the selected entity.	
Example:	
None	

SelectIDWithNew

Unpublished work. © 2025 Siemens

SelectIDWithNew

(sTitle)

Description:

This function displays a selection dialog box (including the ability for the user to create a new entity from within the dialog, when applicable) to allow a user to choose a single entity of a specific type. The selected entity is automatically loaded into the calling entity.

Input:

`STRING sTitle`	A text string that will be added to the title bar of the dialog box to give the user more information about what is being selected.

Output:

None	None

Return Code:

`FE_CANCEL`	The user cancelled the selection.
`FE_BAD_TYPE`	An invalid entity type was specified.

Remarks/Usage:

For supported entities, this method is identical to the `SelectID` method with the exceptions of a new entity icon being displayed in the dialog and the dialog will be displayed even if no entities of the specified type exist the database.

Supported entities include:

Layer
Property
Material
Coordinate System
Global Ply
Load Set
Constraint Set
Connection Property
Connection Region

Example:

None

Unpublished work. © 2025 Siemens

CountSet

`CountSet`
 	
`( void )`	

Description:	
Returns the number of entities in the database of the current type.	
Input:	
None	None
Output:	
None	None
Return Code:	
`Count`	The number of entities of the current type.
Remarks/Usage:	
This function only counts entities that are in the set specified by the `SetID` entity property. For most entities (Curves, Surfaces, Nodes, Elements, ...) this counts all entities in the model. For other *set-based* entities - like Loads and Constraints - it only counts the entities in the one set, so you must specify which set you want prior to calling this function.	
Example:	
None	

Aero Panel/Body Objects

Aero Panel/Body objects correspond to the Aero Panels and Bodies in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Aero Panel/Body objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAeroPanel` method of the Femap Application object to create Aero Panel/Body objects.

Aero Panel/Body Object Properties

Aero Panel/Bodies are always stored with a `SetID=1`, and the ID equal to the Aero Panel/Body ID.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 color`	The panel/body color.
`INT4 layer`	The layer associated with the panel/body.
`STRING title`	Aero Panel/Body title.
`INT4 propID`	ID of Aero Property referenced by the panel/body.
`INT4 icp`	The ID of the coordinate system in which points 1,4 are defined with `Pt1`, `vPt1`, `Pt4`, and/or `vPt4`.
`INT4 nSpan`	Number of spanwise boxes.
`INT4 nChord`	Number of chordwise boxes.
`INT4 nIgid`	Interference group identification

`REAL8 Pt1[0..2]`
or
`VARIANT vPt1`
	Location of point 1 in coordinate system, icp.

`REAL8 Pt4[0..2]`
or
`VARIANT vPt4`
	Location of point 4 in coordinate system, icp.

`REAL8 dChord12`	Edge chord length 1-2.
`REAL8 dChord34`	Edge chord length 3-4.
`INT4 nLspanID`	ID of Aero Division List.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 nLchordID`	ID of Aero Division List.
`INT4 type`	Type of panel: Panel=0, Body=1

Aero Panel/Body Object Methods

GetDivisionList

`GetDivisionList` **`( nChord, cval, nSpan, sval )`**	
Description:	
This method retrieves Chord /Span division arrays for the Aero Panel.	
Input:	
None	None
Output:	
`INT4 nChord`	The number of chord divisions for which data is being returned.
`REAL8 cval[0..(nChord-1]`	The division location along the chord as a percentage along the edge - i.e. 0.8 mean a division 80% of the way between points.
`INT4 nSpan`	The number of span divisions for which data is being returned.
`REAL8 sval[0..(nSpan-1]`	The division location along the span as a percentage along the edge - i.e. 0.8 mean a division 80% of the way between points.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the panel.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	No divisions have been defined.
`FE_OK`	
Remarks/Usage:	
None	
Example:	
None	

PutDivisionList

`PutDivisionList` **`( nChord, cval, nSpan, sval )`**	
Description:	
This method retrieves Chord/Span division arrays for the Aero Panel.	
Input:	
`INT4 nChord`	The number of chord divisions for which data is being stored.
`REAL8 cval[0..(nChord-1]`	The division location along the chord as a percentage along the edge - i.e. 0.8 mean a division 80% of the way between points.
`INT4 nSpan`	The number of span divisions for which data is being stored.
`REAL8 sval[0..(nSpan-1]`	The division location along the span as a percentage along the edge - i.e. 0.8 mean a division 80% of the way between points.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to save data to the panel.

Unpublished work. © 2025 Siemens

`FE_OK`	
Remarks/Usage:	
None	
Example:	
None	

SlenderBodyCount

`SlenderBodyCount` **`( void )`**	
Description:	
This method retrieves the number of Slender Body elements defined.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	Number of Slender Body elements.
Remarks/Usage:	
None	
Example:	
None	

InterferenceBodyCount

Unpublished work. © 2025 Siemens

`InterferenceBodyCount`	
`( void )`	
Description:	
This method retrieves the number of Interference Body elements defined.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	Number of Interference Body elements.
Remarks/Usage:	
None	
Example:	
None	

PanelSpanCount

`PanelSpanCount`	
`( void )`	
Description:	
This method retrieves the number of Span elements defined.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`count`	Number of Span elements.
Remarks/Usage:	
None	
Example:	
None	

PanelChordCount

`PanelChordCount` **`( void )`**	
Description:	
This method retrieves the number of Chord elements defined.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	Number of Chord elements.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetBoxSet

GetBoxSet	
(iBox1, iBox2)	
Description:	
This method creates and returns a `Set` object that contains the IDs of the aero elements included within the box set bounded by BOX1 - BOX2.	
Input:	
`INT4 iBox1`	ID of Box 1.
`INT4 iBox2`	ID of Box 2.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method is used to determine the panel elements chosen based on a Aero Spline BOX1, BOX2 definition.	
Example:	
None	

AeroChordXYZ

AeroChordXYZ
(nNumChord , dChrodLoc)
Description:
This methods retrieve the locations of the Chord edge locations in the global coordinate system.
Input:

Unpublished work. © 2025 Siemens

None	None
Output:	
`INT4 nNumChord`	Number of Chord edge locations.
`REAL8 dChordLoc[0.. nNumChord -1]`	The global coordinates of the Chord point location.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AeroSpanXYZ

`AeroSpanXYZ`
`( nNumSpan , dSpanLoc )`
Description:
This methods retrieve the locations of the Span edge locations in the global coordinate system.
Input:
None / None
Output:
`INT4 nNumSpan` — Number of Span edge locations.
`REAL8 dSpanLoc[0.. nNumChord -1]` — The global coordinates of the Span point location.
Return Code:
None / None
Remarks/Usage:
None

Unpublished work. © 2025 Siemens

Example:
None

Aero Property Objects

Aero Panel/Body objects correspond to the Aero Properties in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Aero Property objects. For more information, see **Common Entity Properties and Methods**.

Use the `Prop` method of the Femap Application object to create Aero Property objects.

Aero Property Object Properties

Aero Properties are always stored with a `SetID=1`, and the ID equal to the Aero Property ID.

Property	Description
`INT4 color`	The property color.
`INT4 layer`	The layer associated with the property.
`STRING title`	Aero Property title.

`REAL8 pdval[0..9]`
or
`VARIANT vpdval`
	values associated with property.

`INT4 pnval[0..14]`
or
`VARIANT vpnval`
	values associated with property.

`REAL8 ap_d_width`	Reference half-width.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 ap_d_ar`	Aspect ratio of the interference tube.
`INT4 ap_i_orient`	Orientation flag Z = 0, Y = 1, ZY = 2.
`INT4 ap_i_lrsb`	Slender Body Radius: ID of Aero Division List.
`INT4 ap_i_lrib`	Interference Body Radius: ID of Aero Division List.
`INT4 ap_i_lth1`	Theta Array 1: ID of Aero Division List.
`INT4 ap_i_lth2`	Theta Array 2: ID of Aero Division List.
`INT4 ap_i_thi1`	First Interference element, Pair 1.
`INT4 ap_i_thi2`	First Interference element, Pair 2.
`INT4 ap_i_thi3`	First Interference element, Pair 3.
`INT4 cap_i_thn1`	Last Interference element, Pair 1.
`INT4 ap_i_thn2`	Last Interference element, Pair 2.
`INT4 ap_i_thn3`	Last Interference element, Pair 3.
`INT4 type`	Type of property: Panel=0, Body=1.

Aero Property Object Methods

GetThetaList

Unpublished work. © 2025 Siemens

GetThetaList	
(nListID, ncount, nval)	
Description:	
This method retrieves theta arrays for the Aero Property	
Input:	
`INT4 nListID`	0 = `Theta1`, 1 = `Theta2`
Output:	
`INT4 ncount`	The number of theta values for which data is being returned.
`REAL8 nval[0..(ncount-1]`	Theta values .
Return Code:	
`FE_FAIL`	Unable to retrieve data from the property.
`FE_NOT_EXIST`	No theta values have been defined.
`FE_OK`	
Remarks/Usage:	
None	
Example:	
None	

PutThetaList

PutThetaList
(nListID, ncount, nval)
Description:
This method saves theta arrays to the Aero Property.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nListID`	0 = Theta1, 1 = Theta2
`INT4 ncount`	The number of theta values being defined.
REAL8 nval[0..(ncount-1]	Theta values.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to save data to the property.
`FE_OK`	
Remarks/Usage:	
None	
Example:	
None	

GetRadiList

`GetRadiList`
`( nListID, ncount, nval )`
Description:
This method retrieves Radi arrays for the Aero Property.
Input:

`INT4 nListID`	0 = Radi at Slender Body, 1 = Radi at Interference Body
Output:	
`INT4 ncount`	The number of values for which data is being returned.

Unpublished work. © 2025 Siemens

`REAL8 nval[0..(ncount-1]`	Radi values.
Return Code:	
None	None
Remarks/Usage:	
`FE_FAIL`	Unable to retrieve data from the property.
`FE_NOT_EXIST`	No values have been defined.
`FE_OK`	
Example:	
None	

PutRadiList

`PutRadiList` **`( nListID, ncount, nval )`**	
Description:	
This method saves Radi arrays to the Aero Property.	
Input:	
`INT4 nListID`	0 = Theta1, 1 = Theta2
`INT4 ncount`	The number of theta values being defined.
`REAL8 nval[0..(ncount-1]`	Theta values.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to save data to the property.

Unpublished work. © 2025 Siemens

FE_OK	
Remarks/Usage:	
None	
Example:	
None	

ClearSbList

ClearSbList	
(void)	
Description:	
This method clears the Slender Body Radi arrays in the Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
FE_OK	
Example:	
None	

ClearIbList

Unpublished work. © 2025 Siemens

`ClearIbList`	
`( void )`	
Description:	
This method clears the Interference Body Radi arrays in the Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
None	
Example:	
None	

ClearTheta1List

`ClearTheta1List`	
`( void )`	
Description:	
This method clears the Theta 1 arrays in the Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

FE_OK	None
Remarks/Usage:	
None	
Example:	
None	

ClearTheta2List

ClearTheta2List	
(void)	
Description:	
This method clears the Theta 2 arrays in the Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
None	
Example:	
None	

SlenderBodyDivisionsByProp

Unpublished work. © 2025 Siemens

SlenderBodyDivisionsByProp	
(void)	
Description:	
This method returns the number of Slender Body Divisions on an Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	Number of Slender Body Divisions.
Remarks/Usage:	
None	
Example:	
None	

InterferenceBodyDivisionsByProp

InterferenceBodyDivisionsByProp	
(void)	
Description:	
This method returns the number of Interference Body Divisions on an Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`count`	Number of Interference Body Divisions.
Remarks/Usage:	
None	
Example:	
None	

GetSBRadiCount

`GetSBRadiCount` **`( void )`**	
Description:	
This method returns the number of Slender Body Radii specified on an Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	Number of Slender Body Radii specified in Create Custom Cross Section dialog box in the GUI.
`FE_OK`	Number of Slender Body Radii defined by number of Interference Elements.
Remarks/Usage:	
None	
Example:	
None	

GetIBRadiCount

Unpublished work. © 2025 Siemens

`GetIBRadiCount`	
`( void )`	
Description:	
This method returns the number of Interference Body Radii specified on an Aero Property.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	Number of Interference Body Radii specified in Create Custom Cross Section dialog box in the GUI.
`FE_OK`	Number of Interference Body Radii defined by number of Interference Elements.
Remarks/Usage:	
None	
Example:	
None	

Aero Spline Objects

Aero Spline objects correspond to the Aero Splines in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Aero Spline objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAeroSpline` method of the Femap Application object to create Aero Spline objects.

Aero Spline Object Properties

Aero Splines are always stored with a `SetID=1`, and the ID equal to the Aero Spline ID.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 color`	The spline color.
`INT4 layer`	The layer associated with the spline.
`STRING title`	Spline title.
`INT4 type`	Type of spline: Surface=0, Beam=1
`INT4 icaero`	Aero element ID.
`INT4 ibox1`	First box in interpolation.
`INT4 ibox2`	Last box in interpolation.
`INT4 isetg`	Group ID.
`REAL8 dz`	Linear attachment flexibility.
`INT4 meth`	IPS = 0,TPS = 1, FPS = 2
`INT4 nelem`	The number of elements along the local spline x-axis.
`INT4 melem`	The number of elements along the local spline y-axis.
`INT4 usage`	FORCE =0, DISP=1, BOTH=2
`REAL8 dtor`	Torsional flexibility ratio.
`INT4 cid`	`CAERO2 CID`
`REAL8 dthx`	RX Rotational attachment flexibility.
`REAL8 dthy`	RY Rotational attachment flexibility.

Aero Spline Object Methods

Unpublished work. © 2025 Siemens

GetNodeSet

GetNodeSet	
(void)	
Description:	
This method creates and returns a Set object that contains the IDs of the Nodes in the Structural Grid Group.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetBoxSet

GetBoxSet	
(void)	
Description:	
This method creates and returns a Set object that contains the IDs of the aero elements included within the box set bounded by Box1 - Box2 belonging to the Aero Panel defined by `icaero` in the Spline properties.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method is used to determine the panel elements chosen based on a Aero Spline BOX1, BOX2 definition.	
Example:	

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Dim aSpline As femap.AeroSpline
    Set aSpline = App.feAeroSpline

    Dim abSet As femap.Set

    aSpline.SelectID("Select Aero Spline")

    Set abSet = aSpline.GetBoxSet()

    App.feAppMessage(FCM_NORMAL, "Box Count = " + Str$(abSet.Count()))

    While abSet.Next
        App.feAppMessage(FCM_NORMAL, Str$(abSet.CurrentID()))
    Wend

End Sub
```

Aero Surface Objects

Aero Surface objects correspond to the Aero Control Surfaces in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Aero Surface objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAeroSurf` method of the Femap Application object to create Aero Surface objects.

Aero Surface Object Properties

Aero Surfaces are always stored with a `SetID=1`, and the ID equal to the Aero Surface ID.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 color`	The control surface color.
`INT4 layer`	The layer associated with the control surface.
`STRING title`	Aero Control Surface title.
`INT4 csys`	The ID of the coordinate system which defines y axis of control surface 1.
`INT4 csys1`	The ID of the coordinate system which defines y axis of control surface 2.
`INT4 aeid`	The ID of the division list that defines control surface 1.
`INT4 aeid1`	The ID of the division list that defines control surface 2.
`REAL8 eff`	Control surface effectiveness.
`BOOL ldw`	Linear downwash flag, `NOLDW = FALSE`, `LDW = TRUE`
`REAL8 crefc`	Reference chord length for the control surface.
`REAL8 crefs`	Reference surface area for the control surface.
`REAL8 pllim`	Lower deflection limits for the control surface.
`REAL8 pulim`	Upper deflection limits for the control surface.
`REAL8 hmllim`	Lower hinge moment limits.
`REAL8 hmulim`	Upper hinge moment limits.
`INT4 tqllim`	Femap function defining deflection limits vs dynamic pressure.
`INT4 tqulim`	Femap function defining deflection limits vs dynamic pressure.
`STRING label`	Controller name, 8 characters max.

Unpublished work. © 2025 Siemens

Aero Surface Object Methods

PutSurfaceSet1

PutSurfaceSet1 **(nSetID)**	
Description:	
This method loads Aero Panel elements into Control Surface 1 arrays.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the elements.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
None	
Example:	
None	

PutSurfaceSet2

PutSurfaceSet2 **(nSetID)**
Description:
This method loads Aero Panel elements into Control Surface 2 arrays.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the elements.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
None	
Example:	
None	

GetSurfaceSet1

`GetSurfaceSet1`
`( void )`
Description:
This method creates and returns a Set object that contains the IDs of the Aero Panel elements in Control Surface 1.
Input:
None | None
Output:
None | None
Return Code:
`FE_OK` | None
Remarks/Usage:
None

Unpublished work. © 2025 Siemens

Example:
None

GetSurfaceSet2

GetSurfaceSet2	
(void)	
Description:	
This method creates and returns a Set object that contains the IDs of the Aero Panel elements in Control Surface 2.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
None	
Example:	
None	

ClearSurfaceSet1

ClearSurfaceSet1
(nSetID)
Description:

Unpublished work. © 2025 Siemens

This method clears the array that contains the IDs of the Aero Panel elements in Control Surface 1.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
None	
Example:	
None	

ClearSurfaceSet2

ClearSurfaceSet2 **(nSetID)**	
Description:	
This method clears the array that contains the IDs of the Aero Panel elements in Control Surface 2.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

AnalysisCase Objects

AnalysisCase objects correspond to the analysis cases in the analysis sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for AnalysisCase objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAnalysisCase` method of the Femap Application object to create AnalysisCase objects. Alternatively, you can create an AnalysisCase object by calling the Case method of the AnalysisMgr object. This approach has the added advantage of ensuring that the case is from the correct analysis set, and preloading the case data.

AnalysisCase Object Properties

AnalysisCases are stored with their SetID equal to the ID of the analysis set, and the ID equal to the analysis case that you are trying to access. Note that the master case is not accessed from an AnalysisCase object. Those properties are directly available from the AnalysisMgr object.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties**.

Property	Description
Analysis Case Options - General	
`STRING CaseTitle[80]`	Title of analysis case (maximum 80 characters).
`STRING CaseLabel[80]`	Label of analysis case (maximum 80 characters).
`INT4 caseID`	ID of analysis case (Not Available for master, caseID for others).
`enum SubcaseAnalysisType`	Analysis Type for Subcase (only used by certain solvers for certain analysis types). See **Analysis Types** in **Femap Constants**.
`INT4 CaseType`	Type of analysis case (0=Normal, 1=Subcom).

Unpublished work. © 2025 Siemens

Property	Description

`INT4 BCSet[0..20]`
 	
or
 	
`VARIANT vBCSet`
	SetIDs of load and constraint sets for this case: [0]=Constraints, [1]=Constraint Equations, [2]=Loads, [3]=Initial Conditions, [4]=Master DOF, [5]=Base/Support DOF, [6]=Omit DOF, [7]=QSET DOF, [8]=CSET DOF, [9]=BSET DOF, [10]=Bolt Preloads, [11]=SUPORT1 DOF, [12]=Temperatures, [13]=Connection Group (MSC Only), [14]=Loads (Time), [15]=Temperatures (Init), [16]=Glue Set, [17]=Contact Set, others not currently used.

`INT4 Output[0..30]`
 	
or
 	
`VARIANT vOutput`
	Output requests for this case: 0=off,-1=full model, >0=groupID, [0]=Nodal Applied Load, [1]=Nodal Reaction Force, [2]=Nodal Displacement, [3]=Nodal Velocity, [4]=Nodal Acceleration, [5]=Nodal Temperature, [6]=Nodal Heat Flux, [7]=Nodal Constraint Equation Forces, [8]=Nodal Force Balance, [9]=Nodal Kinetic Energy, [15]=Element Force, [16]=Element Stress, [17]=Element Strain, [18]=Element Plastic Strain, [19]=Element Strain Energy, [20]=Element Heat Flux, [21]=Element Enthalpy, [22]=Element Enthalpy Rate, [23]=Element Contact, [24]=Element Temperature, [25]=Kinetic Energy, [26]=Energy Loss, [27]=Fluid Pressure, others not currently used.

`STRING Echo[80]`	String for ECHO option (maximum 80 characters).
`BOOL SkipStandard`	Flag to skip standard output for this case.
`BOOL NasCaseAlternateTextLocation`	Flag to use Start Text Inside Case option.
`INT4 StartText`	ID of starting text.
`INT4 EndText`	ID of ending text.
`INT4 Destination`	Output destination: 0=Default, 1=Print, 2=Post, 3=Print+Post, 4=Punch, 5=Punch+Post.
`INT4 Imaginary`	Format for complex output: 0=Magnitude/Phase, 1=Real/Imaginary.
`INT4 CornerOutput`	Flag to request output at element corners.
`INT4 IntegrationOutput`	Flag to request output at element integration points.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasRelEMotion`	Flag to request Relative Enforced Motion Results.
`BOOL NasCurvatureStrain`	Flag to request Curvature Strain Results. True = Curvature, False = Fiber.
ABAQUS Case Options	
ABAQUS Step Options	
`BOOL AbaHistOn`	Flag indicating ABAQUS History Options have been defined.
`REAL8 AbaHistTimePeriod`	Time period.
`REAL8 AbaHistInitTimeInc`	Initial time increment.
`REAL8 AbaHistMinTimeInc`	Minimum time increment.
`REAL8 AbaHistMaxTimeInc`	Maximum time increment.
`INT4 AbaHistEigenvectors`	Number of eigenvectors.
`REAL8 AbaHistMinFreq`	Dynamic analysis minimum frequency.
`REAL8 AbaHistMaxFreq`	Dynamic analysis maximum frequency.
`REAL8 AbaHistShiftFreq`	Dynamic analysis shift frequency.
`REAL8 AbaHistScaleFactor`	Time step scale factor.
`INT4 AbaHistOutInterval`	Time step number of output intervals.
`BOOL AbaHistNonlinGeom`	Flag to specify nonlinear geometry.
`BOOL AbaHistLinearPert`	Flag to specify linear perturbation.
`BOOL AbaHistAutoContact`	Flag to specify auto contact.
`BOOL AbaHistStabilize`	Flag to stabilize contact.
`BOOL AbaHistOutZeroInc`	Flag to write results for zero time increment.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AbaHistStepAmp`	Step amplitude: 0=Default, 1=Step, 2=Ramp.
`INT4 AbaHistStepLoad`	Step loads: 0=New, 1=Modify.
`INT4 AbaHistStepConstr`	Step constraints: 0=New, 1=Modify.
`INT4 AbaHistAnalysisType`	Step based analysis type (see *Analysis Type* property above): 0=Default=Use *Analysis Type*.
`BOOL AbaUsePreviousOutput`	Flag to propagate the results from the previous step.
ANSYS Case Options	
ANSYS Load Step - Case Options	

`INT4 BCSet[0..20]`
 	
or
 	
`VARIANT vBCSet`
	SetIDs of load and constraint sets for this case: [0]=Constraints, [1]=Constraint Equations, [2]=Loads, [3]=Initial Conditions, [10]=Bolt Preloads, [12]=Temperatures, [16]=Glue Set, [17]=Contact Set

`BOOL AnsLdstepOn`	Flag indicating ANSYS Load Step options have been defined.
`BOOL AnsKbc`	Flag to use Amplitude (KBC) option.
`BOOL AnsPstres`	Flag to use Include Prestress Effect (PSTRES) option.
`INT4 AnsEqslv`	Specifies Equation Solver (EQSLV): 0=Sparse Direct, 1=Jacobi Conjugate Gradient iterative, 2=Incomplete Cholesky Conjugate Gradient iterative, 3=Quasi-Minimal Residual iterative, 4=Preconditioned Conjugate Gradient iterative.
`BOOL AnsNewConstraint`	Flag to specify New (True) or Modify (False) for Constraints.
`BOOL AnsNewLoad`	Flag to specify New (True) or Modify (False) for Loads.
`BOOL AnsNewContacct`	Flag to specify New (True) or Modify (False) for Contacts.

Unpublished work. © 2025 Siemens

Property	Description
ANSYS Time Step Options - Case Options	
`BOOL AnsTimestepOn`	Flag indicating ANSYS Time Step options have been defined.
`REAL8 AnsTime`	Time Duration.
`BOOL AnsAutots`	Flag to use Automatic Time Stepping (AUTOTS) option.
`BOOL AnsNsubs`	Flag to set Number of Steps (True) or Size of Steps (False).
`INT4 AnsNsbstep`	Used when `AnsNsubs` is True, sets Number of Steps.
`INT4 AnsNsbmx`	Used when `AnsNsubs` is True, sets Max Number of Steps.
`INT4 AnsNsbmn`	Used when `AnsNsubs` is True, sets Min Number of Steps.
`REAL8 AnsDtime`	Used when `AnsNsubs` is False, sets Time Step.
`REAL8 AnsDtmin`	Used when `AnsNsubs` is False, sets Min Time Step.
`REAL8 AnsDtmax`	Used when `AnsNsubs` is False, sets Max Time Step.
`BOOL AnsPlslimit`	Flag to use Max Plastic Strain option with specified value.
`REAL8 AnsPlslimitvalue`	Max Plastic Strain value.
`BOOL AnsCrplimit`	Flag to use Max Creep Strain option with specified value.
`REAL8 AnsCrplimitvalue`	Max Creep Strain value
`BOOL AnsDpplimit`	Flag to use Max Pressure Change option with specified value.
`REAL8 AnsDpplimitvalue`	Max Pressure Change value.
`BOOL AnsDsplimit`	Flag to use Max Disp Change option with specified value.
`REAL8 AnsDsplimitvalue`	Max Disp Change value.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsBisect`	Flag to use Bisection Factor option with specified value.
`REAL8 AnsBisectfactor`	Bisection Factor value.
`BOOL AnsPredictor`	Flag to use Predict to Bisect option.
`BOOL AnsMidtol`	Flag to use Midstep Tolerance option with specified value.
`REAL8 AnsTolerb`	Midstep Tolerance value.
`BOOL AnsResfq`	Flag to use Response Frequency option.
ANSYS Modal Analysis Options - Case Options	
`BOOL AnsModalOptOn`	Flag indicating ANSYS Modal Analysis options have been defined.
`INT4 AnsModalMethod`	Select Method for model extraction: 0=Block Lanczos, 1=PCG Lanczos, 2=Supernode, 3=Subspace, 4=Unsymmetrical, 5=Damped, 6=Damped w/ QR.
`INT4 AnsModalNmode`	Number Frequency Modes.
`REAL8 AnsModalFreqb`	Frequency value for From.
`REAL8 AnsModalFreqe`	Frequency value for To.
`BOOL AnsModalNrmkey`	Flag to use Mass (True) or Max=Unity (False) for Mode Normalization.
`BOOL AnsModalStrmck`	Flag to enable Strum Sequence Check.
`REAL8 AnsModalRangefact`	`RangeFact` value (only used when `AnsModalMethod=2`).
`INT4 AnsModalBlocksize`	`Blocksize` value (only used when `AnsModalMethod=2`).
`INT4 AnsModalRobustlev`	`RobustLev` value (only used when `AnsModalMethod=2`).
`INT4 AnsModalCompute`	Specifies if Eigenvalues and/or Eigenvectors options should on/off. 0=Both Off, 1=Eigenvalues Only On,

Unpublished work. © 2025 Siemens

Property	Description
	2=Eignevectors Only On, 3=Both On (only used when AnsModalMethod=2).
BOOL AnsModalReusekey	Flag to enable Eigen Reuse (only used when AnsModalMethod=6).
INT4 AnsModalSymmeth	Select Mode Extract symmetrical method for Damped w/QR: 0=Block Lanczos, 1=Subspace, 2=Supernode (only used when AnsModalMethod=6).
ANSYS Buckling Analysis Options - Case Options	
BOOL AnsBuckleOptOn	Flag indicating ANSYS Buckling Analysis options have been defined.
INT4 AnsBuckleMethod	Select Method for model extraction: 0=Block Lanczos, 1=Subspace.
INT4 AnsBuckleNmode	Number of Buckle Modes.
REAL8 AnsBuckleShift	Specifies Shift value when AnsBuckleRangekey is True (Center option) or Lower Bound when AnsBuckleRangekey is False (Range option).
REAL8 AnsBuckleLdmulte	Specifies Bound value when AnsBuckleRangekey is True (Center option) or Upper Bound when AnsBuckleRangekey is False (Range option).
BOOL AnsBuckleRangekey	Flag to use Center (True) or Range (False) for *Range Type*.
BOOL AnsBuckleStrmck	Flag to enable Strum Sequence Check.
ANSYS Harmonic Analysis Options - Case Options	
BOOL AnsHarmonicOptOn	Flag indicating ANSYS Harmonic Analysis options have been defined.
BOOL AnsHarmonicMethod	Flag for Modal (True) or Physical (False) option for Frequency Response analysis.
INT4 AnsHarmonicMinmode	Min Mode Value (only used AnsHarmonicMethod = True).
INT4 AnsHarmonicMaxmode	Max Mode Value (only used AnsHarmonicMethod = True).
REAL8 AnsHarmoniccFreqb	Frequency value for Hz From.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 AnsHarmonicFreqe`	Frequency value for Hz To.
`INT4 AnsHarmonicLogopt`	Specifies Log Frequency Span: 0=Logarithm, 1=Octave band, 2=1/2 Octave band, 3=1/3 Octave band, 4=1/6 Octave band, 5=1/12 Octave band, 6=1/24 Octave band.
`BOOL AnsHarmonicReimky`	Flag for Real/Imag (True) or Ampl/Phase (False) option for Complex Key.
`BOOL AnsHarmonicClust`	Flag for Cluster (True) or Uniform (False) option for Cluster.
ANSYS Transient Dynamics Options - Case Options	
`BOOL AnsTransOptOn`	Flag indicating ANSYS Transient Dynamics options have been defined.
`BOOL AnsTransMethod`	Flag for Modal (True) or Physical (False) option for Transient analysis.
`INT4 AnsTransMinmode`	Min Mode Value (only used `AnsTransMethod` = True).
`INT4 AnsTransMaxmode`	Max Mode Value (only used `AnsTransMethod` = True).
`BOOL AnsTransTintopt`	Flag for HHT (True) or Newmark (False) option for Time Integration.
`REAL8 AnsTransDmpsfreq`	Excitation Frequency value.
`BOOL AnsTransLumpm`	Flag to enable Lumped option for Mass Matrix.
`REAL8 AnsTransAlphad`	Mass value (Damping).
`REAL8 AnsTransBetad`	Stiffness value (Damping).
`REAL8 AnsTransDmpstr`	Struct value (Damping).
`REAL8 AnsTransDmprat`	Ratio value (Damping).
`BOOL AnsTransGamma`	Flag to use gamma option with specified value (when enabled, alpha, delta, alphaf, and alpham are ignored).
`REAL8 AnsTransGammavalue`	Gamma value.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 AnsTransAlpha`	Alpha value (only used when `AnsTransGamma` is False).
`REAL8 AnsTransDelta`	Delta value (only used when `AnsTransGamma` is False).
`REAL8 AnsTransAlphaf`	Alphaf value (only used when `AnsTransGamma` is False).
`REAL8 AnsTransAlpham`	Alpham value (only used when `AnsTransGamma` is False).
ANSYS Nonlinear Options - Case Options	
`BOOL AnsNlOptOn`	Flag indicating ANSYS Nonlinear options have been defined.
`INT4 AnsNropt`	Specifies Method for Stiffness Updates: 0=ANSYS choose, 1=Full Newton Raphson, 2=Modified Newton Raphson, 3=Initial Stiffness, 4=Full Newton Raphson Unsymmetrical.
`INT4 AnsNeqit`	Max Iterations value.
`BOOL AnsCnvtolU`	Flag to use Displacement option (Convergence Tolerances) with specified value.
`REAL8 AnsUtoler`	Displacement value.
`BOOL AnsCnvtolRot`	Flag to use Rotation option (Convergence Tolerances) with specified value.
`REAL8 AnsRottoler`	Rotation value.
`BOOL AnsCnvtolF`	Flag to use Force option (Convergence Tolerances) with specified value.
`REAL8 AnsFtoler`	Force value.
`BOOL AnsCnvtolM`	Flag to use Moment option (Convergence Tolerances) with specified value.
`REAL8 AnsMtoler`	Moment value.
`BOOL AnsCnvtolDvol`	Flag to use Volume Change option (Convergence Tolerances) with specified value.
`REAL8 AnsDvoltoler`	Volume Change value.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsCnvtolHdsp`	Flag to use Hydro Pressure option (Convergence Tolerances) with specified value.
`REAL8 AnsHdsptoler`	Hydro Pressure value.
`BOOL AnsNlgeom`	Flag to use Nonlinear Geometry option.
`BOOL AnsArclen`	Flag to use Arc-Length option.
`BOOL AnsLnsrch`	Flag to use Line Search option.
`BOOL AnsPred`	Flag to use Predictor option.
ANSYS Output Requests - Case Options	
`BOOL AnsOutputOn`	Flag indicating ANSYS Output Requests have been defined.
ANSYS Output Requests - Binary Result File (OUTRES) Section	
`BOOL AnsOutresAll`	Flag to select All Results option.
`INT4 AnsOutresAllFreq`	Specify FREQ value for All Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresAllNth`	Specify integer value for OR STEPS for All Results, only used when `AnsOutresAllFreq`=3.
`BOOL AnsOutresBasic`	Flag to select Basic Results option.
`INT4 AnsOutresBasicFreq`	Specify FREQ value for Basic Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresBasicNth`	Specify integer value for OR STEPS for Basic Results, only used when `AnsOutresBasicFreq`=3.
`BOOL AnsOutresRsol`	Flag to select Nodal Reaction option.
`INT4 AnsOutresRsolFreq`	Specify FREQ value for Nodal Reaction: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresRsolNth`	Specify integer value for OR STEPS for Nodal Reaction, only used when `AnsOutresRsolFreq`=3.
`BOOL AnsOutresNsol`	Flag to select Nodal DOF option.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AnsOutresNsolFreq`	Specify FREQ value for Nodal DOF: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresNsolNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutresNsolFreq`=3.
`BOOL AnsOutresVel`	Flag to select Velocity option.
`INT4 AnsOutresVelFreq`	Specify FREQ value for Velocity: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresVelNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutresVelFreq`=3.
`BOOL AnsOutresAcc`	Flag to select Acceleration option.
`INT4 AnsOutresAccFreq`	Specify FREQ value for Acceleration: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresAccNth`	Specify integer value for OR STEPS for Acceleration, only used when `AnsOutresAccFreq`=3.
`BOOL AnsOutresEsol`	Flag to select Element Results option.
`INT4 AnsOutresEsolFreq`	Specify FREQ value for Element Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresEsolNth`	Specify integer value for OR STEPS for Element Results, only used when `AnsOutresEsolFreq`=3.
ANSYS Output Requests - Print Result File (OUTPR) Section	
`BOOL AnsOutprAll`	Flag to select All Results option.
`INT4 AnsOutprAllFreq`	Specify FREQ value for All Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprAllNth`	Specify integer value for OR STEPS for All Results, only used when `AnsOutprAllFreq`=3.
`BOOL AnsOutprBasic`	Flag to select Basic Results option.
`INT4 AnsOutprBasicFreq`	Specify FREQ value for Basic Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth
`INT4 AnsOutprBasicNth`	Specify integer value for OR STEPS for Basic Results, only used when `AnsOutprBasicFreq`=3.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsOutprRsol`	Flag to select Nodal Reaction option.
`INT4 AnsOutprRsolFreq`	Specify FREQ value for Nodal Reaction: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprRsolNth`	Specify integer value for OR STEPS for Nodal Reaction, only used when `AnsOutprRsolFreq`=3.
`BOOL AnsOutprNsol`	Flag to select Nodal DOF option.
`INT4 AnsOutprNsolFreq`	Specify FREQ value for Nodal DOF: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprNsolNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutprNsolFreq`=3.
`BOOL AnsOutprVel`	Flag to select Velocity option.
`INT4 AnsOutprVelFreq`	Specify FREQ value for Velocity: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprVelNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutprVelFreq`=3.
`BOOL AnsOutprAcc`	Flag to select Acceleration option.
`INT4 AnsOutprAccFreq`	Specify FREQ value for Acceleration: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprAccNth`	Specify integer value for OR STEPS for Acceleration, only used when `AnsOutprAccFreq`=3.
`BOOL AnsOutprEsol`	Flag to select Element Results option.
`INT4 AnsOutprEsolFreq`	Specify FREQ value for Element Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprEsolNth`	Specify integer value for OR STEPS for Element Results, only used when `AnsOutprEsolFreq`=3.
NASTRAN Case Options	
Analysis Case Options - NASTRAN Specific	
`BOOL NasOmodesOn`	Flag indicating an OMODES request has been defined.
`STRING NasOmodes[160]`	Expression used to select OMODES output.

Unpublished work. © 2025 Siemens

Property	Description
Analysis Case Options - SUBCOM	
`INT4 NasSubSeqCase[200]`	IDs of Subcases to be combined in this SUBCOM.
`REAL8 NasSubSeqCase[200]`	Scale factors applied to Subcases combined in SUBCOM.
Analysis Case Options - Direct Matrix Input (DMIG)	
`BOOL NasEnableDMIG`	Flag indicating Enable Matrix Input.
`INT4 NasDMIGK2GG`	ID of Direct Matrix Input entity referenced by Stiffness drop-down in G-Type Matrices. Writes K2GG Case Control entry.
`INT4 NasDMIGM2GG`	ID of Direct Matrix Input entity referenced by Mass drop-down in G-Type Matrices. Writes M2GG Case Control entry.
`INT4 NasDMIGB2GG`	ID of Direct Matrix Input entity referenced by Damping drop-down in G-Type Matrices. Writes B2GG Case Control entry.
`INT4 NasDMIGK42GG`	ID of Direct Matrix Input entity referenced by Structural Damping drop-down in G-Type Matrices. Writes K42GG Case Control entry.
`INT4 NasDMIGP2G`	ID of Direct Matrix Input entity referenced by Loading drop-down in G-Type Matrices. Writes P2G Case Control entry.
`INT4 NasDMIGK2PP`	ID of Direct Matrix Input entity referenced by Stiffness drop-down in P-Type Matrices. Writes K2PP Case Control entry.
`INT4 NasDMIGM2PP`	ID of Direct Matrix Input entity referenced by Mass drop-down in P-Type Matrices. Writes M2PP Case Control entry.
`INT4 NasDMIGB2PP`	ID of Direct Matrix Input entity referenced by Damping drop-down in P-Type Matrices. Writes B2PP Case Control entry.
Analysis Case Options - Monitor Points	
`BOOL NasCaseMonitorEnabled`	Flag indicating Monitor Point Output Requests will be written to the analysis case, when applicable.
`BOOL NasCaseMonitorLoadEnabled`	Flag to set Print Load Monitor Results option.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasCaseMonitorAeroEnabled`	Flag to set Print Aero Monitor Results option.
`BOOL NasCaseMonitorElemEnabled`	Flag to set Print Element Monitor Results option.
NASATRAN Nonlinear Analysis - Case Options	
`BOOL NasCnlOn`	Flag indicating the Nonlinear Case Options has been defined.
`BOOL NasCnlUseLoadSet`	Flag to use the Nonlinear options from the Load Set.
`INT4 NasCnlIncrements`	Number of Time Increments.
`REAL8 NasCnlTime_Increment`	Value for Time Step Size.
`INT4 NasCnlMaxIter`	Nonlinear - Max Iterations per Step.

`BOOL NasCnlConvergenceFlags[0..2]`
or
`VARIANT vNasCnlConvergenceFlags`
	Nonlinear - Convergence Flags for 0=Displacement, 1=Load and 2=Work.

`REAL8 NasCnlConvergenceValue[0..2]`
or
`VARIANT vNasCnlConvergenceValue`
	Nonlinear - Convergence Tolerances for Displacement, Load and Work.

`BOOL MSCNasCnlConvergenceF►lags[0..1]`
or
`VARIANT vMSCNasCnlConvergenceFlags`
	Nonlinear - Convergence Flags for 0=Vector and 1=Length.
	
	MSC Nastran Only and there are no tolerance values.

`INT4 NasCnlStiffnessMethod`	Nonlinear - Stiffness Update Method.
`INT4 NasCnlKstep`	Nonlinear - Iterations before Stiffness Update.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasCnlIntermediateOutput`	Nonlinear - Intermediate Output Type.
`INT4 NasCnlOutputInterval`	Nonlinear - Output Interval.
`INT4 NasCnlSolutionStrategy`	Nonlinear - Nonlinear Arc-Length Solution Strategy (Arc-Length if nonzero).
`INT4 NasCnlSolutionOverrides`	Nonlinear - Solution Strategy Overrides: 0=none/ advanced, 1=Full Newton Raphson, 2=Modified Newton Raphson.
`BOOL NasCnlModnewtonLineSearch`	Nonlinear - Modified Newton Line Search Override (1=Skip).
`BOOL NasCnlModnewtonQuasiNewton`	Nonlinear - Modified Newton Quasi-Newton Override (1=Skip).
`BOOL NasCnlModnewtonBisection`	Nonlinear - Modified Newton Bisection Override (1=Skip).
`INT4 NasCnlArcConstraintType`	Nonlinear - Arc Length Constraint Type.
`REAL8 NasCnlArcMinAdjust`	Nonlinear - Min Arc Length Adjust.
`REAL8 NasCnlArcMaxAdjust`	Nonlinear - Max Arc Length Adjust.
`REAL8 NasCnlArcLoadScale`	Nonlinear - Arc Length Method Constraint Load Scale .
`INT4 NasCnlArcDesiredIter`	Nonlinear - Arc Length Desired Iterations.
`INT4 NasCnlArcMaxStep`	Nonlinear - Arc Length Max Steps.
`INT4 NasCnlTimeSkipAdjust`	Nonlinear - Transient Time Step Skip Factor.
`INT4 NasCnlDominantPeriodSteps`	Nonlinear - Transient Steps for Dominant Period.
`REAL8 NasCnlBoundsRb`	Nonlinear - Transient Bounds to Maintain Step.
`REAL8 NasCnlStabilityTolerance`	Nonlinear - Transient Minimum Stability Tolerance.
`INT4 NasCnlDivergenceLimit`	Nonlinear - Max Diverging Conditions.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasCnlQuasiNewtonVectors`	Nonlinear - Number of Quasi-Newton Vectors.
`INT4 NasCnlMaxLineSearch`	Nonlinear - Max Line Search per Iteration.
`REAL8 NasCnlLineSearchTolerance`	Nonlinear - Line Search Tolerance.
`INT4 NasCnlMaxBisections`	Nonlinear - Max Bisections per Increment.
`REAL8 NasCnlMaxRotation`	Nonlinear - Max Rotation per Bisection.
`REAL8 NasCnlFstress`	Nonlinear - Stress Fraction Limit.
`REAL8 NasCnlMaxAdjust`	Nonlinear - Max Adjusted vs. Initial Increment.
`BOOL NasCnlkDAMP`	Nonlinear - Flag to Include Differential Stiffness in Damping.
`BOOL NasCnlCreep`	Flag Indicating this a Creep Analysis.
`BOOL NasCnlSkipNLPARM`	Flag to disable writing of the `NLPARM` for this case.
NASTRAN Modal Analysis - Case options	
`BOOL NasModeOn`	Flag indicating NASTRAN Modal Options have been defined for subcase.
`INT4 NasModeEigrSet`	SetID to be used for EIGR.
`INT4 NasModeMethod`	Modal Solution method: 0=Givens,1=Modified Givens, 2=Inverse Power, 3=Inverse Power/ Strum, 4=Householder, 5=Modified Householder, 6=Lanczos, 7=Hessenberg, 8=Complex Inverse Power, 9=Complex Lanczos.
`BOOL NasModeSolutionType`	Solution type: False=Direct, True=Modal.

`REAL8 NasModeFreqRange[2]`
or
`VARIANT vNasModeFreqRange`
	Frequency range of interest, [0]=Min, [1]=Max.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasModeEstRoots`	Estimated number of eigenvalues.
`INT4 NasModeDesiredRoots`	Number of desired eigenvalues/modes.
`INT4 NasModeNormOpt`	Normalization method: 0=Mass,1=Max, 2=Point.
`INT4 NasModeNormNode`	Node ID for use with point normalization method.
`INT4 NasModeNormDOF`	Nodal DOF for use with point normalization method.
`INT4 NasModeMassForm`	Mass matrix formulation: 0=Default, 1=Lumped, 2=Coupled.
`BOOL NasModesSkipEigr`	If True, skips writing the EIGR card for specifying modal analysis options.

`REAL8 NasModeImagFreqRange[2]`
or
`VARIANT vNasModeImagFreqRange`
	For Complex Modes analysis, specifies the imaginary part of the Frequency Range of interest.

`REAL8 NasModesComplexConverge`	Specifies Complex Modes convergence value.
`REAL8 NasModesComplexRegionWidth`	Specifies Region Width for Complex Modes analysis.
`REAL8 NasModeDampOverall`	Specifies PARAM,G
Nastran STATSUB Options	
`BOOL NasStatsubOn`	Flag indicating STATSUB Options have been defined for subcase.
`INT4 NasStatsubBuckle`	ID of Static Subcase in Buckling Subcase list.
`INT4 NasStatsubPreload`	ID of Static Subcase in Preload Subcase list.
NASTRAN Multi-Step Control Options (SOL 401) - Case	
`BOOL NasMsnlCntOn`	Flag indicating Multi-Step Control Options for Multi-Step Structural have been defined for subcase.

Unpublished work. © 2025 Siemens

Property	Description
NASTRAN Multi-Step Control Options (SOL 401) - Case - General options	
`BOOL NasMsnlCntSkipNLCNTL`	Flag to disable writing of the `NLCNTL` for this case.
NASTRAN Multi-Step Control Options (SOL 401) - Case - Time Step Control	
`BOOL NasMsnlCntAutotim`	Flag to use Auto Time Step (`AUTOTIM`) option.
`REAL8 NasMsnlCntDtinit`	Initial Time Step (`DMINIT`) value.
`REAL8 NasMsnlCntDtmin`	Min Time Step (`DMIN`) value.
`REAL8 NasMsnlCntDtmax`	Max Time Step (`DMAX`) value.
`BOOL NasMsnlCntDtsbcdt`	Flag to specify New (False) or Modify (True) for Subcase Time Step (`DTSBCDT`).
`REAL8 NasMsnlCntEqmfmin`	Equilibrium Min Factor (`EQMFMIN`) value.
`REAL8 NasMsnlCntEqmfmx`	Equilibrium Max Factor (`EQMFMAX`) value.
`REAL8 NasMsnlCntRTOLB`	Sets value for **Incremental Rotation Limit in Iteration (RTOLB)** Time Control strategy parameter (real numbers > 0.0, default 20)
`INT4 NasMsnlCntTsccr`	Specifies Time Step Method option: 0=Constant, 1=Adaptive, Truncation Error, 2=Adaptive, Maximum Creep Increment to Elastic Strain Ratio, 3=Adaptive, Maximum Creep Increment, 12=Combination of 1 and 2, 13=Combination of 1 and 3, 23=Combination of 2 and 3, 123=Combination of 1, 2 and 3.
`BOOL NasMsnlCntTsceq`	Flag to use Equilibrium Iteration Stepping (`TSCEQ`).
`BOOL NasMsnlCntLvar`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Loads (`LVAR`).
`BOOL NasMsnlCntTvar`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Temp Loads (`TVAR`).
`BOOL NasMsnlCntDtsbcdt`	Flag to activate stepping based on UMAT supplied time increments (`TSCUMAT`).
`REAL8 NasMsnlCntEqmfmin`	Minimum time step factor for UMAT stepping (`UMFMIN`) value.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMsnlCntEqmfmx`	Minimum time step factor for UMAT stepping (UMFMX) value.
NASTRAN Multi-Step Control Options (SOL 401) - Case - Other Options	
`BOOL NasMsnlCntPlastic`	Flag to activate **Include Plasticity** option.
`BOOL NasMsnlCntCreep`	Flag to activate **Include Creep** option.
NASTRAN Multi-Step Control Options (SOL 401) - Case - Solution and Convergence	
Solution and Convergence - Solution Parameters	
`INT4 NasMsnlCntLOADOFF`	Disable Mechanical Loads (LOADOFF) option: 0=No Loads Turned Off 1=Body Loads Turned Off 2=Boundary Loads Turned Off 3=Both Boundary and Body Loads Turned Off
`INT4 NasMsnlCntSolver`	Specifies Solver (SOLVER) option: 0=SPARSE, 1=PARDISO, 2=MUMPS, 3=ELEMITER.
`INT4 NasMsnlCntRFVAR`	Specifies **Rot. Load Inertia Scaling** (RFVAR) option: 0=Scale Angular Velocity and Acceleration 1=Scale Force
`BOOL NasMsnlCntINACCN`	Flag to use Compute Initial Acceleration in Dynamics (INACCN).
`BOOL NasMsnlCntINERTIA`	Flag to use Enable Inertia in Dynamics (INERTIA).
`INT32 NasMsnlCntMsglvl`	Flag to use Thermal Strain Loading (TSCEQ).
`BOOL NasMsnlCntThrmst`	Flag to use Diagnostic Level (MSGLVL).
`BOOL NasMsnlCntLSTCONV`	When on, Femap always outputs results at the converged last step, even when you did not request output for that step. When off, Femap only outputs results at the converged last step when you requested output for that step on the TSTEP1 bulk entry.
Solution and Convergence - Stiffness Parameters	

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasMsnlCntKupdate`	0=Auto Stiffness Update, 1=Full Newton-Raphson, 2=Quasi Newton-Raphson.
`INT4 NasMsnlCntQuasiNewtonIter`	Quasi Newton-Rhapson Iterations. Must be a value greater than or equal to 2. Only used when `NasMsnlCntKupdate` is set to 2=Quasi Newton-Raphson.
`INT4 NasMsnlCntStfoptn`	1=Elastic Stiffness Matrix, 2=Tangent Stiffness Matrix, 3=Elastic/Tangent Stiffness Matrix.
`BOOL NasMsnlCntFollowk`	Flag to use Follow Stiffness (`FOLLOWK`).
`BOOL NasMsnlCntSpink`	Flag to use Spin Softening (`SPINK`).
`BOOL NasMsnlCntStressk`	Flag to use Stress Stiffening (`STRESSK`).
`BOOL NasMsnlCntTstepk`	Flag to use Follow Stiffness (`TSTEPK`).
`BOOL NasMsnlCntMSTAB`	Flag to use Enable Stiffness Matrix Stabilization (`MSTAB`).
`REAL8 NasMsnlCntMSFAC`	Value for Matrix Stabilization Factor (`MSFAC`) parameter.
`BOOL NasMsnlCntSTKFRST`	Flag to include Stress Stiffening at the first time step. If "Yes", the **Include Stress Stiffeneing at First Iteration (STKFRST)** Stiffness Parameter is activated.
Solution and Convergence - Convergence Parameters	
`VARIENT vNasMsnlCntConv_flags`	Flags to use convergence option and specified value: [0]=Displacement (`EPSU`) [1]=Load (`EPSP`) [2]=Work (`EPSW`)
`VARIENT vNasMsnlCntConv_value`	Values for use corresponding convergence option [0]=Displacement (`EPSU`) [1]=Load (`EPSP`) [2]=Work (`EPSW`)
`INT4 NasMsnlCntMaxbis`	Max Bisection (`MAXBIS`) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasMsnlCntMaxdiv`	Max Divergences (MAXDIV) value.
`INT4 NasMsnlCntMaxiter`	Max Iterations/Step (MAXITER) values.
`INT4 NasMsnlCntMaxqn`	Max Quasi Newton Vectors (MAXQN).
`INT4 NasMsnlCntNORMP`	Specifies Norm Criteria for Force (NORMP) option: 0=Norm 1 Criteria, 1.=Norm 2 Criteria, 2.=Norm Infinity Criteria.
`INT4 NasMsnlCntNORMU`	Specifies Norm Criteria for Displacement (NORMU) option: 0=Norm 1 Criteria, 1.=Norm 2 Criteria, 2.=Norm Infinity Criteria
`REAL8 NasMsnlCntREFP`	Value for Force Error Function Denominator (REFP).
`REAL8 NasMsnlCntREFU`	Value for Force Error Function Denominator (REFU).
Solution and Convergence - Time Integration	
`INT4 NasMsnlCntTINTMTH`	Specifies Integration Scheme. 1=Newmark, 2=Hilber-Hughes-Taylor, 3=Generalized alpha method, 4=Modified generalized alpha method.
`REAL8 NasMsnlCntBETA`	Value for BETA time integration parameter.
`REAL8 NasMsnlCntGAMA`	Value for GAMA time integration parameter.
`REAL8 NasMsnlCntALFA`	Value for ALFA time integration parameter.
`REAL NasMsnlCntTETA`	Value for TETA time integration parameter.
`REAL NasMsnlCntRHOINF`	Value for RHOINF time integration parameter.
`REAL8 NasMsnlkCnt2RHOINF`	Value for RHOINF time integration parameter.
Solution and Convergence - Result Storage Control	
`INT4 NasMsnlkCnt2IA16`	Storage cycle for grid point results.
`INT4 NasMsnlkCnt2IA19`	Storage cycle for XY plotting results requested by OUTMGT bulk entries.

Unpublished work. © 2025 Siemens

Property	Description
NASTRAN Multi-Step Control Options (SOL 401) - Case - Contact/Bolt Preload	
Contact/Bolt Preload - Contact	
`INT4 NasMsnlCntCntmdiv`	Num Divergence for Bisection (CNTMDIV) value.
`REAL8 NasMsnlCntFsymtol`	Frict Threshold (FSYMTOL) value.
`INT4 NasMsnlCntKsym`	Unsymmetric Matrix option (KSYM) option:0=AUTO, 1=SYM, 2=UNSYM.
`REAL8 NasMsnlCntKsymtol`	Symmetrizing Tol (KSYMTOL) value.
`INT4 NasMsnlCntMsglvlc`	Contact Diagnostics (MSGLVLC) option: 0=None, 1=Convergence, 2=Convergence and Contact Pair.
`INT4 NasMsnlCntUsolver`	Unsymmetric Solver (USOLVER) option: 0=HYBRID, 1=SPARSE, 2=PARDISO, 3=MUMPS.
`INT4 NasMsnlCntKMODTN`	Tangential contact stiffness (KMODTN) option: 1=With Friction - applies a uniform tangential contact stiffness of (0.1 * normal stiffness) in a modal subcase, 2=With Friction - applies the actual final tangential contact stiffness from the end of a nonlinear subcase in a modal subcase, 3=With Friction - applies the actual final tangential contact stiffness from the end of a nonlinear subcase in a modal subcase, 4=Both Cases - applies a uniform tangential contact stiffness of (0.1 * normal stiffness) in a modal subcase, 5=Both Cases - replaces the final stiffness of any active contact element at the end of the nonlinear subcase with a no-separation, tied-contact stiffness in both the normal and tangential directions, regardless of the friction value.
`REAL8 NasMsnlCntKMODSCL`	Scale factor for the computed normal contact stiffness for the modal subcase (Real > 0.0; Default = 1.0).
Contact/Bolt Preload - Bolt/Preload	
`REAL8 NasMsnlCntEpsbolt`	Nonzero Preload Convergence Tol (EPSBOLT) value.
`REAL8 NasMsnlCntZerbolt`	Zero Preload Tol (ZERBOLT) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasMsnlCntItrbolt`	Max Iterations for Convergence (`ITRBOLT`) value.
`REAL8 NasMsnlCntMisfblt`	Bolt Strain Limit (`MISFBLT`) value.
`INT4 NasMsnlCntMsglvlb`	Diagnostics (`MSGLVLB`) option: 0=Summary at Convergence, 1=Summary Each Iteration, 2=Summary and Load Tol Each Iteration.
NASTRAN Multi-Step Control Options (SOL 401) - Case - Creep Options	
`REAL8 NasMsnlCntCrcerat`	Max Creep Ratio (`CRCERAT`) value.
`REAL8 NasMsnlCntCrcinc`	Max Creep Inc (`CRCINC`) value.
`REAL8 NasMsnlCntCricoff`	Creep Strain Inc (`CRICOFF`) value.
`REAL8 NasMsnlCntCrinfac`	Integration Factor (`CRINFAC`) value.
`REAL8 NasMsnlCntCrmfmn`	Min Step Factor (`CRMFMN`) value.
`REAL8 NasMsnlCntCrmfmx`	Max Step Factor (`CRMFMX`) value.
`REAL8 NasMsnlCntCrteabs`	Max Truncation Error (`CRTEABS`) value.
`REAL8 NasMsnlCntdCrteco`	Time Step for Next Truncation Step (`CRTECO`) value.
`REAL8 NasMsnlCntCrterel`	Max Relative Truncation Error (`CRTEREL`) value.
`REAL8 NasMsnlCntCRLIMR`	Integration Error (`CRLIMR`) used for adjusting the creep strain increment (Real ≥ 0.0; Default = 0.0).
`REAL8 NasMsnlCntPLLIM`	Maximum equivalent plastic strain increment allowed within each time step (Real ≥ 0.0; Default=0.0, which turns off the capability).
`REAL8 NasMsnlCntPLLIMF`	Multiplying factor (`PLLIMF`) of maximum equivalent plastic strain (`PLLIM`) when adjusting the time step increment. (Real > 0.0; Default=1.0)
`REAL8 NasMsnlkCnt2PLLIM`	Maximum equivalent plastic strain increment allowed within each time step (`PLLIM`). Real ≥ 0.0; Default=0.0, which turns off the capability.
Arc-Length Control Options (SOL 401)	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasMsnlArcLenOn`	Flag indicating Arc-Length Control Options have been defined for subcase (i.e., sets **Enable NLARCL** option)
`INT4 NasMsnlArcLenConstr`	Constraint Type: 0=Default, 1=Riks, 2=Modified Riks, 3=Crisfield.
`REAL8 NasMsnlArcLenMinAlr`	Min ArcLen Adjust Ration (`MINALR`) value.
`REAL8 NasMsnlArcLenMaxAlr`	Max ArcLen Adjust Ration (`MAXALR`) value.
`REAL8 NasMsnlArcLenMaxR`	Load/ArcLen Bound (`MAXR`) value.
`REAL8 NasMsnlArcLenScale`	Scale for Constraint Load (`SCALE`) value.
`INT4 NasMsnlArcLenDIter`	Desired Iterations (`DESITER`) value.
`INT4 NasMsnlArcLenMxInc`	Max Load Increments (`MXINC`) value.
`REAL8 NasMsnlArcLenInitLdfac`	Initial Load Factor (`LDFACIN`) value.
`REAL8 NasMsnlArcLenMxLdfac`	Max Load Factor (`MXLDFAC`) value.
`INT4 NasMsnlArcLenSkipFac`	Output Skip Factor (`NOUTAL`) value.
NASTRAN Multi-Step Control Options (SOL 402) - Case	
`BOOL NasMsnlCnt2On`	Flag indicating Multi-Step Control Options for Multi-Step Nonlinear Kinematic have been defined for subcase
NASTRAN Multi-Step Control Options (SOL 402) - Case - General options	
`BOOL NasMsnlCnt2SkipNLCNTL2`	Flag to disable writing of the `NLCNTL2` for this case.
`BOOL NasMsnlCnt2LSTCONV`	TBD
NASTRAN Multi-Step Control Options (SOL 402) - Case - Time Step	
`BOOL NasMsnlkCnt2CIBL`	Flag to use Auto Time Stepping (`CIBL`) option (version 12), Auto Time Stepping (`AUTOTIM`) option (2019.1 and above).

Unpublished work. © 2025 Siemens

Property	Description
or `BOOL NasMsnlkCnt2AUTOTIM`	

`REAL8 NasMsnlkCnt2DTI0`
 	
or
 	
`REAL8 NasMsnlkCnt2DTINIT`
	First Time Step (DTI0) value (version 12), First Time Step (`DTINIT`) value (2019.1 and above).

`REAL8 NasMsnlkCnt2HMIN`
 	
or
 	
`REAL8 NasMsnlkCnt2DTMIN`
	Minimum Time Step (`HMIN`) value (version 12), Minimum Time Step (`DTMIN`) value (2019.1 and above).

`REAL8 NasMsnlkCnt2HMAX`
 	
or
 	
`REAL8 NasMsnlkCnt2DTMAX`
	Maximum Time Step (`HMAX`) value (version 12), Maximum Time Step (`DTMAX`) value (2019.1 and above).

`REAL8 NasMsnlkCnt2RUP`
 	
or
 	
`REAL8 NasMsnlkCnt2EQMFMX`
	Maximum Increase Ratio (`RUP`) value (version 12), Maximum Increase Ratio (`EQMFMX`) value (2019.1 and above).

`REAL8 NasMsnlkCnt2RDOW`
 	
or
 	
`REAL8 NasMsnlkCnt2EQMFMIN`
	Maximum Decrease Ratio (`RDOW`) value (version 12), Maximum Decrease Ratio (`EQMFMIN`) value (2019.1 and above).

`INT4 NasMsnlkCnt2ITEREF`	Number of Iterations (`ITEREF`) value.
`INT4 NasMsnlkCnt2RJPN`	Specifies Negative Pivot Time Step Rejection (`RJPN`) option:

Unpublished work. © 2025 Siemens

Property	Description
	0=Never 1=Last Iteration Only 2=Print State At Last Iteration 3=Always
`BOOL NasMsnlkCnt2RJPZ`	Zero Pivot Time Step Rejection (RJPZ) value.
`BOOL NasMsnlkCnt2TSDYN`	Integration Error Control (TSDYN) value.
`BOOL NasMsnlkCnt2ERCD`	Flag to use Activate Time Stepping Based on Disp/Temp Error.
`REAL8 NasMsnlkCnt2PRED`	Value for Allowable Change/Step (PRED).
NASTRAN Multi-Step Control Options (SOL 402) - Case - Viscous Material Options	
`BOOL NasMsnlkCnt2TSVSC`	Flag to use Integration Error Control (TSVSC) option.
`REAL8 NasMsnlkCnt2VSCOTE`	Error Threshold (VSCOTE).
`INT4 NasMsnlkCnt2VSCOSN`	Min Stress Factor (VSCOSN).
NASTRAN Multi-Step Control Options (SOL 402) - Case - Analysis Control	
`REAL8 NasMsnlkCnt2DISLIM`	Maximum Displacement (DISLIM) value.
`REAL8 NasMsnlkCnt2ROTLIM`	Maximum Rotation (ROTLIM) value.
`REAL8 NasMsnlkCnt2DEFLIM`	Maximum Deformation (DEFLIM) value.
`BOOL NasMsnlkCnt2LVAR`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Loads (LVAR).
`BOOL NasMsnlkCnt2TVAR`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Temp Loads (TVAR).
`BOOL NasMsnlkCnt2DIPR`	Flag to use Predict Displacement (DIPR) option.
`BOOL NasMsnlkCnt2NORM`	Flag to use Force Normalization (NORM) option.
`BOOL NasMsnlkCnt2MATSYM`	Flag to use Unsymmetrical Matrices for Complex Modes (MATSYM).

Unpublished work. © 2025 Siemens

Property	Description
`INT32 NasMsnlkCnt2INERTIA`	Flag to use Enable Inertia in Dynamics (`INERTIA`).
`BOOL NasMsnlkCnt2IAR0`	Flag to use Save Data at Beginning of Computation (`IAR0`).
`INT4 NasMsnlkCnt2ITHE`	Manage thermal loads to improve convergence (-1 = Free thermal expansion, 0 = Constrained thermal expansion). Default = 0.
`INT4 NasMsnlkCnt2LL2`	Sets the space required to store the laws of excitation. If the value is too small, a warning is issued, and the run is interrupted. Allowable values: Integers greater than 0. Default = 20000.
`INT4 NasMsnlkCnt2RFVAR`	Time variation (RFVAR) when interpolating rotational inertia loads: 0=Scale angular velocity and acceleration, 1=Scale force.
NASTRAN Multi-Step Control Options (SOL 402) - Case - Plasticity and Creep Control	
`BOOL NasMsnlkCnt2PLASTIC`	Flag to use Enable Plasticity option.
`BOOL NasMsnlkCnt2CREEP`	Flag to use Enable Creep option.
`REAL8 NasMsnlkCnt2CRINFAC`	Creep Integration Factor (`CRINFAC`) value.
`REAL8 NasMsnlCnt2PLLIM`	Maximum equivalent plastic strain increment allowed within each time step. When greater than 0.0, the solution can adjust the time step increment. Allowable values: Real numbers greater than or equal to 0.0. Default=0.0, which deactivates the capability.
`REAL8 NasMsnlCnt2PLLIMF`	Multiplying factor of `PLLIM` when adjusting the time step increment.
`REAL8 NasMsnlkCnt2CRICOFF`	Creep strain increment below which the next time step is the product of the current time step and the maximum time step multiplying factor. Valid for creep analysis only. (0.0 < Real < 1.0; Default = 1.0E-6).
`REAL8 NasMsnlkCnt2CRLIMR`	Time stepping method for creep analysis. Adjusts the integration error used for adjusting the creep strain increment. (Real ≥ 0.0; Default = 0.0)
NASTRAN Multi-Step Control Options (SOL 402) - Case - Internal Restart	

Unpublished work. © 2025 Siemens

Property	Description
INT4 NasMsnlkCnt2RSUB	Specifies if the Restart (RSUB) should be activated and the value to use for Restart Subcase. If value is set to 0, the Restart (RSUB) option will be deactivated. If value is set to a value above 0, the Restart (RSUB) option will be enabled and the value specified will be used as the value for Restart Subcase.
NASTRAN Multi-Step Control Options (SOL 402) - Case - Solution and Convergence	
Solution and Convergence - Analysis Options	
REAL8 NasMsnlkCnt2STAB	Stabilization Factor (STAB) value.
REAL8 NasMsnlkCnt2MADI	Minimum Disp Variation (MADI) value.
Solution and Convergence - Equilibrium and Convergence	
INT4 NasMsnlkCnt2ILNS	Line Search (ILNS) value.
REAL8 NasMsnlkCnt2PRLN	Line Search Convergence Tol (PRLN) value.
REAL8 NasMsnlkCnt2AMIN	Min Line Search Factor (AMIN) value.
REAL8 NasMsnlkCnt2AMAX	Max Line Search Convergence Force (AMAX) value.
INT4 NasMsnlkCnt2ITMA	Max Iter (ITMA) value.
REAL8 NasMsnlkCnt2PRCR	Relative Force Tol (PRCR) value.
REAL8 NasMsnlkCnt2REFP	Relative Force (REFP) value.
REAL8 NasMsnlkCnt2PRCQ	Relative Disp Force (PRCQ) value.
REAL8 NasMsnlkCnt2REFU	Reference Disp (REFU) value.
REAL8 NasMsnlkCnt2PRCE	Rel Energy Tol (PRCE) value.
REAL8 NasMsnlkCnt2REFE	Reference Energy (REFE) value.
INT4 NasMsnlkCnt2IT1K	Stiffness Update Param 1 (IT1K) value.

Unpublished work. © 2025 Siemens

Property	Description
INT4 NasMsnlkCnt2IT2K	Stiffness Update Param 2 (IT2K) value.
INT4 NasMsnlkCnt2IT3K	Stiffness Update Param 3 (IT3K) value.
INT4 NasMsnlkCnt2PLAS	Specifies First Iteration Strategy (PLAS) option: -1=Elastic tangent matrix, plastic stress variation -2=Elastic tangent matrix, elastic stress variation -3=Plastic tangent matrix, elastic stress variation 1=Plastic tangent matrix, plastic stress variation
INT4 NasMsnlkCnt2OTRE	Specifies Rigid Body Motion (OTRE) option: 0=Disable Convergence 1=Enable based on PRCE, PRCQ, and PRCR criteria. 2=Enable based on PRCE, PRCQ, PRCR, and REFP.
INT4 NasMsnlkCnt2MAXBIS	Maximum number of times Femap can reduce a time step consecutively. By default, equal to 0, no limit.
INT4 NasMsnlkCnt2KUPDATE	Stiffness update strategy. (Integer; Default = 0).
Solution and Convergence - Time Integration	
INT4 NasMsnlkCnt2IMPL	Specifies Integration Scheme (IMPL) option (version 12): 1=NewMark implicit predictor-corrector. 2=Newmark implicit predictor-corrector with automatic time stepping. 21=Hilber-Hughes-Taylor (HHT) implicit predictor-corrector scheme. 22=Hilber-Hughes-Taylor (HHT) implicit predictor-corrector with automatic time stepping. 31=Generalized midpoint. 32=Generalized midpoint with automatic time stepping. 61=Generalized Alpha. 62=Generalized Alpha with automatic time stepping.
INT4 NasMsnlkCnt2TINTMTH	Specifies Integration Scheme (c) option (2019.1 and above):

Unpublished work. © 2025 Siemens

Property	Description
	1=Newmark 2=Hilber-Hughes-Taylor 3=Generalized Midpoint 4=Generalized Alpha
`REAL8 NasMsnlkCnt2BETA`	Param1 (BETA) value.
`REAL8 NasMsnlkCnt2GAMA`	Param2 (GAMA) value.
`REAL8 NasMsnlkCnt2ALFA`	HHT Scheme Param (ALFA) value.
`REAL8 NasMsnlkCnt2TETA`	Generalized Scheme Param (TETA) value.
`REAL8 NasMsnlkCnt2PRCO`	Intg Control Param 1 (PRCO) value.
`REAL8 NasMsnlkCnt2HPRCO`	Intg Control Param 2 (HPRCO) value.
Solution and Convergence - Contact	
`REAL8 NasMsnlkCnt2RELC`	Value for Relaxation Force (RELC).
`REAL8 NasMsnlkCnt2DCON`	Value for Characteristic Length (DCON).
`REAL8 NasMsnlkCnt2PRCS`	Value for Ing Regularization Factor (PRCS).
`REAL8 NasMsnlkCnt2PRCF`	Value for Threshold for Contact Force Variation (PRCF).
`INT4 NasMsnlCnt2KMODTN`	tangential contact stiffness for the modal subcase. Useful, for example, when the nonlinear subcase requires frictionless contact.
`REAL8 NasMsnlCnt2KMODSCL`	Scales normal contact stiffness for the modal subcase.
`INT4 NasMsnlkCnt2KMODTN`	Independently control the tangential contact stiffness of the contact pairs with friction, and without friction (Integer; Default = 1).
`INT4 NasMsnlkCnt2MSGLVLC`	Diagnostic output for contact. (Integer = 0, 1 or 2; Default = 1) 0: No additional diagnostic output. 1: Convergence information is listed for each iteration. Changes in contact status are also

Unpublished work. © 2025 Siemens

Property	Description
	listed. Before the first iteration, a summary of the number of sliding/sticking/active/inactive elements is listed. The number of augmentations and number of pair updates is also listed. 2: Everything described in option 1 and after each iteration, detailed information for every contact pair is listed.
Solution and Convergence - Diagnostic	
`INT4 NasMsnlkCnt2IMPG`	Print Frequency (IMPG) value.
`INT4 NasMsnlkCnt2IMPR`	Max Residual Printout (IMPR) value.
`INT4 NasMsnlkCnt2IMPV`	Max Pivots (IMPV) value.
NASTRAN Multi-Step Time Steps (SOL 401 and SOL 402) - Case	
`BOOL NasMsnlTSOn`	Flag indicating Multi-Step Time Steps for Multi-Step Structural or Multi-Step Nonlinear Kinematic have been defined for subcase.
`BOOL NasMsnlTSSkipTSTEP`	Flag to disable writing of the TSTEP for this case.
`REAL8 NasMsnlTSEndTime[0..9]`	Value in End Time Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc.
`INT4 NasMsnlTSNumIncrements[0..9]`	Value in Number of Increments Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc.
`INT4 NasMsnlTSOutputFreq[0..9]`	Value in Output Frequency Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc: 0=Requested Increments, 1=Solution Step End Time, 2=All Times, 3=Coupling Times, 4=Skip Factor .
`INT4 NasMsnlTSSkipFactor[0..9]`	Value in Number of Increments Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc. Only used when `NasMsnlTSOutputFreq`=4.
NASTRAN Aeroelastic Trim Parameters - Case Options	
`BOOL NasCaeOn`	Flag indicating NASTRAN Trim Options have been defined for subcase.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasCaeMachNumber`	Mach number.
`REAL8 NasCaeDynPressure`	Dynamic pressure.
`BOOL NasCaeRigidTrim`	Flag to request a rigid trim analysis.
`BOOL NasCaeWrtieTrim`	Flag to enable writing of the TRIM for this case.
NASTRAN Flutter Parameters - Case Options	
`BOOL NasCflOn`	Flag indicating NASTRAN Flutter Options have been defined for subcase.
`INT4 NasCflMethod`	Flutter analysis method.
`INT4 NasCflDenID`	Femap function defining density ratios.
`INT4 NasCflMachFactID`	Femap function defining density ratios.
`INT4 NasCflRfreqFactID`	Femap function defining density ratios.
`INT4 NasCflFliMethod`	Interpolation method.
`INT4 NasCflEig`	Number Eigenvalues.
`REAL8 NasCflEps`	Convergence.
`BOOL NasCflWriteFlutter`	Flag to enable writing of the FLUTTER for this case. .
`INT4 NasSELoad`	Superelement load set ID.
`INT4 NasCflSdampID`	ID of Femap function defining the Modal Damping Table.
MSC NASTRAN Implicit Nonlinear (SOL 400) - Control Parameters	
`BOOL NasInlStepOptOn`	Flag indicating Control Parameters for Implicit Nonlinear Analysis have been defined for the subcase.
`BOOL NasInlStepOptSkipNLSTEP`	Flag to disable writing of NLSTEP for this subcase

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptID`	ID of NLSTEP for this subcase.
Control Parameters - Step Parameters	
`REAL8 NasInlStepOptTOTTIM`	Total Time (TOTTIM) value.
`INT4 NasInlStepOptCTRLDEF`	Specifies **Nonlinearity** (CTRLDEF) option: 0=Default 1=Linear 2=Mildly Nonlinear 3=Severely Nonlinear
Control Parameters - General Parameters	
`INT4 NasInlStepOptMAXITER`	Max Iterations/Increment (MAXITER) value.
`INT4 NasInlStepOptMINITER`	Min Iterations/Increment (MINITER) value.
`INT4 NasInlStepOptMAXBIS`	Max Bisections for Step (MAXBIS) value.
`BOOL NasInlStepOptCREEP`	Flag to enable Include Creep (CREEP) option.
Control Parameters - Analysis Parameter	
`INT4 NasInlStepOptLoadStepping`	Specifies **Increment Type** option: 0=Fixed 1=Adaptive 2=Arc-Length
Control Parameters - Analysis Parameters - Fixed Time Stepping Parameters	
`INT4 NasInlStepOptNINC`	Number of Increments (NINC) value.
`INT4 NasInlStepOptNO`	Output Interval (NO) value.
Control Parameters - Analysis Parameters - Adaptive Load Stepping Parameters	
`REAL8 NasInlStepOptDTINITF`	Initial Fraction (DTINITF) value
`REAL8 NasInlStepOptDTMINF`	Min Fraction (DTMINF) value
`REAL8 NasInlStepOptDTMAXF`	Max Fraction (DTMAXF) value

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasInlStepOptSFACT`	Increase Factor (SFACT) value
`INT4 NasInlStepOptADJUST`	Auto Skip Factor (ADJUST) value
`INT4 NasInlStepOptMSTEP`	Dominant Period Resp. Steps (MSTEP) value
`REAL8 NasInlStepOptRB`	Adaptive Time Step Bound (RB) value
`INT4 NasInlStepOptNDESIR`	Desired Iterations/Increments (NDESIR) value
`INT4 NasInlStepOptNSMAX`	Max Increments (NSMAX) value
`REAL8 NasInlStepOptUTOL`	Displacement Tolerance (UTOL) value
`INT4 NasInlStepOptIDAMP`	Specifies Static Damping (IDMAP) option: 0=No Added Damping 4=Time Step Control Using Damping 5=Damping-based Time Step Control with No Damping 6=Add Damping when Minimum Time Step is Reached
`REAL8 NasInlStepOptDAMP`	Damping Ratio (DAMP) value.
`INT4 NasInlStepOptINTOUT`	Specifies Output Flag (INTOUT) option: -1=Last Increment 0=Every Load Increment >0=Equally Spaced Intervals (value is used to specify value for Output Every Nth Step)
Adaptive Load Stepping Parameters - Criteria	
`BOOL NasInlStepOptEnableCriteria`	Flag to enable the Enable User Criteria option
`INT4 NasInlStepOptIPHYS`	Specifies Physical Criteria (IPHYS) option: -2=No Auto Physical Criteria; Continue When User Criteria Exceeded -1=Add Auto Physical Criteria; Continue When User Criteria Exceeded 1=1..No Auto Physical Criteria; Stop When User Criteria Exceeded

Unpublished work. © 2025 Siemens

Property	Description
	2=2..Add Auto Physical Criteria; Stop When User Criteria Exceeded
`INT4 NasInlStepOptLIMTAR`	Specifies Definition (`LIMTAR`) option: 0=Criteria as Limits 1=Criteria as Targets
`REAL8 NasInlStepOptRSMALL`	Min. Ratio (`RSMALL`) value
`REAL8 NasInlStepOptRBIG`	Max Ration (`RBIG`) value
`BOOL NasInlStepOptIncrements[0..9]`	Flag in Status column of row specified by zero-based index value in list of Criteria. When True, Criterion is enabled. 0=Elem. Strain 1=Elem. Plastic Strain 2=Elem. Creep Strain 3=Elem. Normalized Creep Strain 4=Elem. Strain Energy 5=Elem. Stress 6=Elem. Normalized Stress 7=Nodal Temperature 8=Nodal Displacement 9=Nodal Rotation
`INT4 NasInlStepOptGroupIDs[0..9]`	ID of existing Group in Group column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT1_i[0..9]`	Value in Y1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT1_i[0..9]`	Value in M1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT2_i[0..9]`	Value in Y2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT2_i[0..9]`	Value in M2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasInlStepOptYT3_i[0..9]`	Value in Y3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT3_i[0..9]`	Value in M3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT4_i[0..9]`	Value in Y4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT4_i[0..9]`	Value in M4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
Control Parameters - Analysis Parameters - Arc-Length Load Stepping Parameters	
`INT4 NasInlStepOptTYPE`	Specifies Constraint Type option: 0=Crisfield 1=Riks 2=Modifed Riks
`REAL8 NasInlStepOptDTINITFA`	Initial Time Step Fraction (`DTINITFA`) value.
`REAL8 NasInlStepOptMINALR`	Min ArcLenAdjust Ratio (`MINALR`) value.
`REAL8 NasInlStepOptMAXALR`	Max ArcLenAdjust Ratio (`MAXALR`) value.
`REAL8 NasInlStepOptSCALEA`	Scale Factor (`SCALEA`) value.
`INT4 NasInlStepOptNDESIRA`	Desired Iterations (`NDESIRA`) value.
`INT4 NasInlStepOptNSMAXA`	Max Increments (`NSMAXA`) value.
Control Parameters - Analysis Parameters - Mechanical Analysis Parameters	
`BOOL NasInlStepOptMECH`	Flag to activate the Enable MECH option.
`INT4 NasInlStepOptKMETHOD`	Specifies Method (`KMETHOD`) option: 0=Full Newton-Raphson 1=Quasi Newton-Raphson

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptKSTEP`	Iteration Interval (KSTEP) value.
`INT4 NasInlStepOptMAXQN`	QN Vectors (MAXQN) value.
`INT4 NasInlStepOptMAXLS`	Max Line Searches (MAXLS) value.
`REAL8 NasInlStepOptLSTOL`	Line Search Tol (LSTOL) value.
`REAL8 NasInlStepOptFSTRESS`	Fraction Limit (FTRESS) value.
`BOOL NasInlStepOptConvU`	Flag to activate Displacement option.
`REAL8 NasInlStepOptEPSU`	Displacement value.
`BOOL NasInlStepOptConvP`	Flag to enable Load option.
`REAL8 NasInlStepOptEPSP`	Load value.
`BOOL NasInlStepOptConvW`	Flag to enable Work option.
`REAL8 NasInlStepOptEPSW`	Work value.
`BOOL NasInlStepOptConvV`	Flag to enable Vector option.
`BOOL NasInlStepOptConvN`	Flag to enable Length option.
`BOOL NasInlStepOptConvA`	Flag to enable Automatic Switching option.
`INT4 NasInlStepOptMRCONV`	Specifies Checking (MRCONV) option: 0=Forces, Moments, Displacements, and Rotations 1=Forces, Moments, and Displacements 2=Forces, Displacements, and Rotations 3=Forces and Displacements
MSC Nastran Boundary Conditions	
`INT4 ContactOption`	Boundary Conditions dialog box for MSC Nastran Only. Sets BCONTACT Option (0=Default, 1=Inital Contact, 2=All Body, 3=Auto).

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ContactSetType`	Boundary Conditions dialog box for MSC Nastran Only. Sets Contact/Glue Sets option (0=All Connectors, 1=Connection Group, 2=All Body). ID of group for Connection Group set using `BCSet[13]`.
ANSYS Case Options (Legacy)	
Analysis Case Options - ANSYS Specific (Legacy)	
`BOOL AnsDOFOn`	Flag indicating Dynamic DOF Options have been defined.
`BOOL AnsDOFDoSet`	Flag indicating a master DOF set is specified and in use.
`BOOL AnsDOFAutomatic`	Indicates automatic dynamic DOF selection.
`BOOL AnsDOFNoRotation`	Excludes rotational DOF from automatic DOF selection.
`INT4 AnsDOFSetID`	Constraint set ID to be used as a master DOF set.
`INT4 AnsDOFNumber`	Number of degrees of freedom to automatically select.
Modal Options - ANSYS (Legacy)	
`BOOL AnsModesOn`	Flag indicating modal expansion options have been defined.
`INT4 AnsModesNumModes`	Number of modes/frequencies to expand.
`REAL8 AnsModesMinFreq`	Minimum frequency of interest.
`REAL8 AnsModesMaxFreq`	Maximum frequency of interest.
`REAL8 AnsModesModalMethod`	Modal expansion method.
`BOOL AnsModesElemResults`	Flag indicating whether elemental results (stresses, strains...) will be calculated.
Heat Transfer Options ANSYS (Legacy)	
`BOOL AnsHeatOn`	Flag indicating Heat Transfer Options have been defined.
`REAL8 AnsHeatEndTime`	Time at end of this load step.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 AnsHeatTimeStep`	Initial time increment.
`INT4 AnsHeatOutputFreq`	Output step interval.
Analysis Case Options - MARC Specific	
`BOOL MarHistOn`	Flag indicating MARC History options have been defined.
`INT4 MarHistContactProp`	ID of Contact Property written to the CONTACT TABLE.
`INT4 MarHistCtrlMethod`	ID of control type written to the CONTROL option. 0=None, 1=Full Newton-Raphson, 2=Modified Newton-Raphson, 3=Strain Correction Newton-Raphson, 8=Secant Method,
`INT4 MarHistSolverMeth`	ID of solver type written to the SOLVER option. 0=None, 1=Profile Direct, 2=Sparse Iterative, 3=Sparse Direct, 4=Hardware Provided Sparse Direct, 5=MultiFrontal Direct Sparse.
`INT4 MarHistCtrlMaxStep`	Number of Max Steps written to CONTROL option.
`INT4 MarHistCtrlMaxRecycle`	Number of Max Recycles written to CONTROL option.
`INT4 MarHistCtrlMinRecycle`	Number of Min Recycles written to CONTROL option.
`REAL8 MarHistMaxResidual`	Number of Max Conv. Residual written to CONTROL option.
`INT4 MarHistConvergeMeth`	ID of method written to CONTROL option. 0=Rel Force Residuals, 1=Rel Disp Residuals, 2=Rel Strain Ener Res, 3=Abs Force Residuals, 4=Abs Disp Residuals, 5=Abs Strain Ener Res.
`BOOL MarHistNonPosDefinite`	Flag indicating Nonpositive Def will be used on SOLVER option.
`BOOL MarHistNonSymmetric`	Flag indicating NonSymmetric will be used on SOLVER option.
`REAL8 MarHistModalMinFreq`	Number indicating the Min Freq for a modal analysis.
`REAL8 MarHistModalMaxFreq`	Number indicating the Max Freq for a modal analysis.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MarHistModalEigen`	Number indicating the Number of eigenvectors to be found in modal analysis.
`INT4 MarHistChecking`	ID indicating checking will be used. 0=None, 1=Strum Sequence.
`BOOL MarHistAutoSwitch`	Flag indicating Auto Switch will be used on CONTROL option.
`INT4 MarHistAnalCaseSol`	ID indicating the Analysis type for this Subcase 1=Static, 2=Normal Modes, 3=Buckling.
`INT4 MarHistBuckleNumModes`	Number of Buckling modes.
`INT4 MarHistBucklePosEigen`	Number of Modes With positive eigenvalues.
`BOOL MarIncOn`	Flag indicating MARC Increment options have been defined
`INT4 MarIncArcLenMeth`	ID indicating the Arclength Method. 0=None(AUTO STEP), 1=Crisfield, 2=Riks, 3=Modified Riks, 4=Crisfield/Modified Riks.
`REAL8 MarIncTimePeriod`	Number indicating AUTO STEP time period.
`REAL8 MarIncInitTimeInc`	Number indicating AUTO STEP initial time increment.
`REAL8 MarIncMinTimeInc`	Number indicating AUTO STEP min time increment.
`REAL8 MarIncMaxTimeInc`	Number indicating AUTO STEP max time increment.
`REAL8 MarIncScaleFact`	Number indicating AUTO STEP scale factor.
`INT4 MarIncMaxStep`	Number indicating AUTO STEP max number of steps.
`INT4 MarIncNumOutInc`	Number indicating number of output intervals.
`BOOL MarIncQuasiStatDamp`	Flag to include Damping scheme for statics.
`REAL8 MarIncFracFirst`	Number indicating AUTO INCREMENT fraction of total load applied in first step.
`INT4 MarIncMaxNumInc`	Number indicating AUTO INCREMENT max num increments.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MarIncCtrlMaxRecycle`	Number indicating AUTO INCREMENT max recycles.
`REAL8 MarIncMaxFracForAll`	Number indicating AUTO INCREMENT max fraction of load for any step.
`REAL8 MarIncTotalTime`	Number indicating AUTO INCREMENT total time.
`BOOL MarCtblOn`	Flag indicating the MARC Contact Table has been defined.
`BOOL MarCtblWrite`	Flag indicating the MARC Contact Table is to be written.

AnalysisCase Object Methods

In addition to the standard methods, the following methods are also available.

InitAnalysisCase

`InitAnalysisCase` **`( nSetID, nCaseID )`**	
Description:	
This method initializes the default values for an analysis case.	
Input:	
`INT4 nSetID`	The Analysis SetID containing the case to be initialized.
`INT4 nCaseID`	The Case ID to be initialized.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to initialize the case. Probably the Analysis Set you specified does not exist.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

DeleteAnalysisCase

`DeleteAnalysisCase` **`( nSetID, nCaseID )`**	
Description:	
Use this method to delete an analysis case.	
Input:	
`INT4 nSetID`	The Analysis SetID containing the case to be deleted.
`INT4 nCaseID`	The Case ID to be deleted.
Output:	
None	None
Return Code:	
`FE_FAIL`	None
Remarks/Usage:	
This method takes care of deleting any start/end text or Marc contact table records that referenced the case.	
Example:	
None	

ActiveContactEntities

Unpublished work. © 2025 Siemens

ActiveContactEntities **(bContact, nConnectorSetID, nRegionSetID, nConPropSetID)**	
Description:	
This method is used to populate Set Objects with the Connectors, Connection Regions, and Connection Properties that will be exported for Contact or Glue Contact by an analysis case. This method takes into consideration the settings in the Contact Sets or Glue Sets section of the Boundary Conditions dialog box for Simcenter SOL 401/402 or setting for Contact/Glue Sets of the Boundary Conditions dialog box for MSC Nastran.	
Input:	
`BOOL bContact`	If True, populates sets with connection entities for Contact. If False, populates sets with connection entities for Glued Contact.
`INT4 nConnectorSetID`	Set ID of Set Object to populate with Connectors which will be exported by this analysis case.
`INT4 nRegionSetID`	Set ID of Set Object to populate with Connection Regions which will be exported by this analysis case.
`INT4 nConPropSetID`	Set ID of Set Object to populate with Connection Properties which will be exported by this analysis case.
Output:	
None	None
Return Code:	
`FE_FAIL`	No Connection Regions were detected.
Remarks/Usage:	
It is important to use three uniques Set Objects when using this method, otherwise the IDs of different entity types may contaminate a Set. Also, it is recommend to use *empty sets* to ensure that only the Connection entities from a single analysis case populate the three sets.	
Example:	
None	

GetContactTable

Unpublished work. © 2025 Siemens

GetContactTable	
(count, rowSeg, colSeg, tieType, distTol, sepForce, friction, interClose)	
Description:	
This method retrieves data from the Contact Table.	
Input:	
`INT4 count`	The number of entries in each of the other vectors.
`INT4 rowSeg[0..count-1]`	The IDs of the contact segments referenced in rows of the table.
`INT4 colSeg[0..count-1]`	The IDs of the contact segments referenced in columns of the table.
`INT4 tieType[0..count-1]`	The tying type (0,1=None, 2=Touch, 3=Glue).
`REAL8 distTol[0..count-1]`	The distance tolerance.
`REAL8 sepForce[0..count-1]`	The separation force.
`REAL8 friction[0..count-1]`	The friction values.
`REAL8 interClose[0..count-1]`	The interference closure.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	The contact table does not exist.
`FE_FAIL`	Unable to allocate memory to return the contact table data.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutContactTable

PutContactTable **(count, rowSeg, colSeg, tieType, distTol, sepForce, friction, interClose)**	
Description:	
This method replaces data in the Contact Table.	
Input:	
`INT4 count`	The number of entries in each of the other vectors.
`INT4 rowSeg[0..count-1]`	The IDs of the contact segments referenced in rows of the table.
`INT4 colSeg[0..count-1]`	The IDs of the contact segments referenced in columns of the table.
`INT4 tieType[0..count-1]`	The tying type (0,1=None, 2=Touch, 3=Glue).
`REAL8 distTol[0..count-1]`	The distance tolerance.
`REAL8 sepForce[0..count-1]`	The separation force.
`REAL8 friction[0..count-1]`	The friction values.
`REAL8 interClose[0..count-1]`	The interference closure.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to update the contact table.
Remarks/Usage:	
This function completely replaces the contact table - it does not add to data that is already there.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetStandardTrim

GetStandardTrim **(useTrim, TrimID, TrimVal)**	
Description:	
This method retrieves data from the standard trim definition.	
Input:	
`INT4 useTrim[0..11]`	0=Off,1=Use, 2= Fixed. See Remarks for array index value.
`INT4 TrimID[0..11]`	Not currently used.
`REAL8 TrimVal[0..11]`	Magnitude value when `UseTrim[i]` = Fixed. See Remarks for array index value.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Standard Trim values have not been defined.
`FE_FAIL`	Unable to allocate memory to return the trim data.
Remarks/Usage:	
Valid TrimID values: 0 = NAS_AERO_TRIM_ANGLEA 1 = NAS_AERO_TRIM_SIDES 2 = NAS_AERO_TRIM_ROLL 3 = NAS_AERO_TRIM_PITCH 4 = NAS_AERO_TRIM_YAW 5 = NAS_AERO_TRIM_URDD1 6 = NAS_AERO_TRIM_URDD2 7 = NAS_AERO_TRIM_URDD3 8 = NAS_AERO_TRIM_URDD4 9 = NAS_AERO_TRIM_URDD5 10 - NAS_AERO_TRIM_URDD6	

Unpublished work. © 2025 Siemens

Example:
None

PutStandardTrim

`PutStandardTrim` **`( useTrim, TrimID, TrimVal )`**	
Description:	
This method loads standard trim data into the trim definition.	
Input:	
`INT4 useTrim[0..11]`	0=Off, 1=Use, 2= Fixed. See Remarks for array index value
`INT4 TrimID[0..11]`	Not currently used
`REAL8 TrimVal[0..11]`	Magnitude value when `UseTrim[i]` = Fixed. See Remarks for array index value
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Standard Trim values have not been defined.
`FE_FAIL`	Unable to allocate memory to return the trim data.
Remarks/Usage:	
Valid index values: 0 = NAS_AERO_TRIM_ANGLEA 1 = NAS_AERO_TRIM_SIDES 2 = NAS_AERO_TRIM_ROLL 3 = NAS_AERO_TRIM_PITCH 4 = NAS_AERO_TRIM_YAW 5 = NAS_AERO_TRIM_URDD1 6 = NAS_AERO_TRIM_URDD2 7 = NAS_AERO_TRIM_URDD3	

Unpublished work. © 2025 Siemens

8 = NAS_AERO_TRIM_URDD4 9 = NAS_AERO_TRIM_URDD5 10 - NAS_AERO_TRIM_URDD6
Example:
None

GetTrimControlSurf

GetTrimControlSurf	
(useTrim, TrimID, TrimVal)	
Description:	
This method retrieves data from the trim definition for user defined Aero Control Surfaces.	
Input:	
None	None
Output:	
`INT4 useTrim[0..29]`	0=Off,1=Use, 2= Fixed.
`INT4 TrimID[0..29]`	ID of the Femap Aero Control Surface Entity.
`REAL8 TrimVal[0..29]`	Magnitude.
Return Code:	
`FE_NOT_EXIST`	Trim values have not been defined.
`FE_FAIL`	Unable to allocate memory to return the trim data.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutTrimControlSurf

PutTrimControlSurf **(useTrim, TrimID, TrimVal)**	
Description:	
This method loads user defined Aero Control Surface data into the Trim definition.	
Input:	
`INT4 useTrim[0..29]`	0=Off,1=Use, 2= Fixed.
`INT4 TrimID[0..29]`	ID of the Femap Aero Control Surface Entity.
`REAL8 TrimVal[0..29]`	Magnitude.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the trim data.
Remarks/Usage:	
None	
Example:	
None	

GetOptLimits

GetOptLimits **(nSetID, bClear)**
Description:
Gets Optimization Limits (Optimization Response objects) enabled for an analysis case.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nSetID`	ID of Set Object to be populated.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	
`FE_INVALID`	Analysis type is not design optimization.
`FE_NOT_EXIST`	No Optimization Limits (Responses) have been activated.
Remarks/Usage:	
None	
Example:	
None	

PutOptLimits

`PutOptLimits` **`( nSetID, bClear )`**	
Description:	
Sets Optimization Limits (Optimization Response objects) enabled for an analysis case.	
Input:	
`INT4 nSetID`	ID of Set Object containing Optimization Limits (Response) IDs. Alternatively, specify a negative number to put a single optimization variable.
`BOOL bClear`	Clear existing entities.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_INVALID`	Analysis type is not design optimization.
`FE_NOT_EXIST`	No Optimization Limits (Responses) have been activated.
Remarks/Usage:	
None	
Example:	
None	

GetStep

`GetStep` **`( StepID )`**	
Description:	
This method creates and returns an `AnalysisStep` object that is loaded with the options referenced by an analysis step.	
Input:	
`INT4 entID`	The ID of Analysis Step in the current Case to load.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current `AnalysisCase` object must be loaded with the data from the case that contains the step that you want to retrieve. The `AnalysisCase.ID` property is used to define the case that you want to retrieve.	
Example:	
`Sub Main`	

Unpublished work. © 2025 Siemens

```
    Dim App as femap.model
    Set App = feFemap()

    Dim rc As Integer

    Dim aCase As femap.AnalysisCase
    Set aCase = App.feAnalysisCase

    Dim aStep As femap.AnalysisStep

    aCase.SetID = 1
    rc = aCase.Get( 1 )
    If (rc = FE_OK) Then
        Set aStep = aCase.GetStep( 1 )
        App.feAppMessage(FCM_NORMAL, aStep.StepTitle )
    End If
End Sub
```

PutStep

PutStep **(pDISPATCH)**	
Description:	
Saves a fully populated `AnalysisStep` Object to the database.	
Input:	
`OBJECT pDISPATCH`	`AnalysisStep` Object to save to the database.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This can be used in conjunction with `GetStep` to create a fully populated `AnalysisStep` object, make changes to the object, then save the changes to the database.	
Example:	
None	

DeleteStep

`DeleteStep`
`( entID )`
Description:
Use this method to delete an analysis step.
Input:

`INT4 entID`	The Analysis Step to be deleted.
Output:	
None	None
Return Code:	
`FE_FAIL`	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

AnalysisCase Object - ANSYS Specific Methods

In addition to the standard methods, the following methods are also available specifcially for use with ANSYS translator.

KBC

KBC	
(key)	
Description:	
This method sets the **Amplitude (KBC)** option on ANSYS Load Step Options dialog box.	
Input:	
BOOL key	When True, activates the **Amplitude (KBC)** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PSTRES

PSTRES
(key)
Description:

Unpublished work. © 2025 Siemens

This method sets the **Include Prestress Effects (PSTRES)** option on ANSYS Load Step Options dialog box.	
Input:	
`BOOL key`	When True, activates **Include Prestress Effects (PSTRES)** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

TIME

`TIME` **`( time )`**	
Description:	
This method sets the **Time Duration** value on ANSYS Time Step Options dialog box.	
Input:	
`REAL8 time`	**Time Duration** value.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

AUTOTS

AUTOTS	
(key)	
Description:	
This method sets the **Automatic Time Stepping (AUTOTS)** option on ANSYS Load Step Options dialog box	
Input:	
`BOOL key`	When True, activates **Automatic Time Stepping (AUTOTS)** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

LANBOPTION

LANBOPTION
(strmck)
Description:

Unpublished work. © 2025 Siemens

This method sets the **Strum Sequence Check** option on ANSYS Modal Analysis dialog box for use with the **Block Lanczos** mode extraction method.	
Input:	
`BOOL key`	When True, activates the **Strum Sequence Check** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PCGOPT

PCGOPT **(strmck)**	
Description:	
This method sets the **Strum Sequence Check** option on ANSYS Modal Analysis dialog box for the **PCG Lanczos** mode extraction method.	
Input:	
`BOOL key`	When True, activates **Strum Sequence Check** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

SUBOPT

SUBOPT	
(strmck)	
Description:	
This method sets the **Strum Sequence Check** option on ANSYS Modal Analysis dialog box for use the **Subspace** mode extraction method.	
Input:	
`BOOL key`	When True, activates **Strum Sequence Check** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

LUMPM

Unpublished work. © 2025 Siemens

LUMPM	
(key)	
Description:	
This method sets the **Lumper** option on ANSYS Transient Dynamics dialog box.	
Input:	
`BOOL key`	When True, Activates **Lumped** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ALPHAD

ALPHAD	
(alpha)	
Description:	
This method sets the **Mass value (Damping)** on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 alpha`	**Mass value (Damping)**
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

BETAD

BETAD **(beta)**	
Description:	
This method sets the **Stiffness value (Damping)** on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 beta`	**Stiffness value (Damping)**
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DMPSTR

Unpublished work. © 2025 Siemens

DMPSTR	
(dmp)	
Description:	
This method sets the **Struct value (Damping)** on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 dmp`	**Struct value (Damping)**.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DMPRAT

DMPRAT	
(rat)	
Description:	
This method sets the **Ratio value (Damping)** on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 rat`	**Ratio value (Damping)**.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

NEQIT

NEQIT **(neqit)**	
Description:	
This method sets the **Max Iterations** value on ANSYS Nonlinear dialog box.	
Input:	
`INT4 neqit`	**Max Iterations** value.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

NLGEOM

Unpublished work. © 2025 Siemens

NLGEOM	
(key)	
Description:	
This method sets the **Nonlinear Geometry** option on ANSYS Nonlinear dialog box.	
Input:	
BOOL key	When True, activates **Nonlinear Geometry** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ARCLEN

ARCLEN	
(key)	
Description:	
This method sets the **Arc-Length** option on ANSYS Nonlinear dialog box.	
Input:	
BOOL key	When True, activates **Arc-Length** option.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

LNSRCH

LNSRCH	
(key)	
Description:	
This method sets the **Line Search** option on ANSYS Nonlinear dialog box.	
Input:	
`BOOL key`	When True, activates **Line Search** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PRED

Unpublished work. © 2025 Siemens

PRED	
(sskey)	
Description:	
This method sets the **Predictor** option on ANSYS Nonlinear dialog box.	
Input:	
`BOOL sskey`	When True, activates **Predictor** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

SPOPT

SPOPT	
(sptype, nmode, elcalc)	
Description:	
This method sets the **Spectrum** options on ANSYS Random Vibration Analysis Options dialog box.	
Input:	
`INT4 sptype`	ID of the spectrum type option.
`INT4 nmode`	Specifies the number of modes.
`BOOL elcalc`	When True, Element Results is on.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PSDUNIT

PSDUNIT **(tblno, type)**	
Description:	
This method sets the PSD table on the ANSYS Random Vibration Analysis Options dialog box	
Input:	
`INT4 tblno`	ID of the function, 0=DISP, 1=VEL, 2=ACEL (Acc2/Hz), 3=ACCG (g2/Hz), 4=FORC, 5=PRES.
`INT4 type`	Specifies the PSD type.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PFACT

PFACT

(tblno, excit)

Description:

This method sets the PSD table on the ANSYS Random Vibration Analysis Options dialog box.

Input:

`INT4 tblno`	ID of the function, 0=DISP, 1=VEL, 2=ACEL (Acc2/Hz), 3=ACCG (g2/Hz), 4=FORC, 5=PRES.
`BOOL excit`	Sets the Mode Factors, True = Node, False = Base.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

None

Example:

None

PSDCOM

PSDCOM

(signif, type)

Description:

This method sets the **Mode Combinations** options on the ANSYS Random Vibration Analysis Options dialog box.

Unpublished work. © 2025 Siemens

Input:	
`REAL8 signif`	Sets the Significance value.
`INT4 comode`	Sets the Mode Number value.
`BOOLforcetype`	Sets the Combine Force, True = Static, False = Total.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnalysisMgr Objects

While maybe not entirely obvious, AnalysisMgr objects correspond to the analysis sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for AnalysisMgr objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAnalysisMgr` method of the Femap Application object to create AnalysisMgr objects.

Note

While a method called `feAnalysisSet` also exists, it is only used to create AnalysisMgr objects, thus it is redundant and should NOT be used. It has not been removed for the sole purpose of allowing any programs which have it specified to continue to run properly.

AnalysisMgr Object Properties

`AnalysisMgr` objects are always stored with their `SetID` = 1, and the ID equal to the analysis set that you are trying to access.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title`	Analysis set title (maximum 79 characters).
`INT4 Solver`	Analysis program to be used for analysis: 0=Unknown, 4=MSC/NASTRAN, 5=ANSYS, 16=ABAQUS, 28=LS-Dyna, 29=MSC.Marc, 31=NE/Nastran, 34=FEMAP Structural. 36=NX Nastran.
`INT4 AnalysisType`	Type of analysis: 0=Unknown, 1=Static, 2=Normal Modes/Eigenvalue, 3=Transient Dynamic/Time History, 4=Frequency/Harmonic Response, 5=Response Spectrum, 6=Random Response, 7=Buckling, 8=Design Opt, 9=Explicit Transient Dynamics, 10=Nonlinear Static, 11=Nonlinear Buckling, 12=Nonlinear Transient, 13=Implicit Transient Dynamics, 17=Fatigue, 19=Computational Fluid Dynamics, 20=Steady-State Heat Transfer, 21=Transient Heat Transfer, 22=Advanced Nonlinear Static, 23=Advanced Nonlinear Transient, 24=Advanced Nonlinear Explicit, 25=Static Aeroelasticity, 26=Aerodynamic Flutter, 27=Multi-Step Structural, 28=Multi-Step Nonlinear Kinematic, 30=Implicit Nonlinear, 31=Aeroelastic Frequency Response, 32=Aeroelastic Transient Response, 33=Aeroelastic Random Response
`INT4 LinkedSolverOption`	Sets Solve Using option for analysis set: 0=Integrated Solver (`FS_INTEGRATED` enum), 1=Linked Solver (`FS_LINKED` enum), 2=VisQ (`FS_VISQ` enum)
(Obsolete) `BOOL LaunchWithVisQ`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use `LinkedSolverOption` instead to launch the analysis using VisQ.
Master Case Options - Master Case and Boundary Conditions	
`STRING CaseTitle[80]`	Title of analysis set Master (maximum 80 characters).
`STRING CaseLabel[80]`	Label of analysis set Master (maximum 80 characters).
`enum SubcaseAnalysisType`	Analysis Type for Subcase (only used by certain solvers for certain analysis types). See **Analysis Types** in **Femap Constants** for available enumerations.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 CaseType`	0=Normal, 1=Subcom (not used)
`BOOL SkipStandard`	Flag to skip standard output for this case.
`INT4 StartText`	ID of starting text.
`INT4 EndText`	ID of ending text.

`INT4 BCSet[20]`
or
`VARIANT vBCSet`
	SetIDs of load and constraint sets for this case: [0]=Constraints, [1]=Constraint Equations, [2]=Loads, [3]=Initial Conditions, [4]=Master DOF, [5]=Base/Support DOF, [6]=Omit DOF, [7]=QSET DOF, [8]=CSET DOF, [9]=BSET DOF, [10]=Bolt Preloads, [11]=SUPORT1 DOF, [12]=Temperatures, [13]=Connection Group (MSC Only), [14]=Loads (Time), [15]=Temperatures (Init), [16]=Glue Set, [17]=Contact Set, others not currently used.

Master Case Options - Output Requests (Multiple Solvers other than ANSYS)	

`INT4 Output[35]`
or
`VARIANT vOutput`
	Output requests for this case: 0=off,-1=full model, >0=groupID, [0]=Nodal Applied Load, [1]=Nodal Reaction Force, [2]=Nodal Displacement, [3]=Nodal Velocity, [4]=Nodal Acceleration, [5]=Nodal Temperature, [6]=Nodal Heat Flux, [7]=Nodal Constraint Equation Forces, [8]=Nodal Force Balance, [9]=Nodal Kinetic Energy, [15]=Element Force, [16]=Element Stress, [17]=Element Strain, [18]=Element Plastic Strain, [19]=Element Strain Energy, [20]=Element Heat Flux, [21]=Element Enthalpy, [22]=Element Enthalpy Rate, [23]=Element Contact, [24]=Element Temperature, [25]=Element Kinetic Energy, [26]=Element Energy Loss, [27]=Glue, others not currently used.

`STRING Echo[80]`	String for ECHO option.
`INT4 Destination`	Output destination: 0=Default, 1=Print Only, 2=PostProcess Only, 3=Print and PostProcess, 4=Punch Only, 5=Punch and PostProcess, 6=XDB, 7=Output Database (ABAQUS ODB File), 8=Print and Output Database (ABAQUS ODB File)
`INT4 Imaginary`	Format for complex output: 0=Magnitude/Phase, 1=Real/Imaginary.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 CornerOutput`	Flag to request output at element corners.
`INT4 IntegrationOutput`	Flag to request output at element integration points.
`BOOL NasRelEMotion`	Flag to request Relative Enforced Motion Results.
`BOOL NasCurvatureStrain`	Flag to request Curvature Strain Results. True = Curvature, False = Fiber.
ABAQUS Options	
Analysis Set Options - ABAQUS Specific	
`BOOL AbaModOn`	Flag indicating ABAQUS Model options have been specified.
`BOOL AbaModGroupsAsSets`	Flag to write all groups as sets.
`BOOL AbaModTitleSetNames`	Flag to write entity titles as set names.
`BOOL AbaModCbushAsMatrix`	Flag to write all CBUSH elements as MATRIX (True), otherwise, writes connections entries.
`BOOL AbaModSkipStandard`	Flag to skip output of standard ABAQUS model.
`INT4 AbaModStartText`	ID of text written at beginning of ABAQUS model
`INT4 AbaModEndText`	ID of text written at end of ABAQUS model.
`INT4 AbaModVersion`	Version number of ABAQUS to translate for (61=Version 6.1).
ABAQUS Step Options - Master Case Options	
`BOOL AbaHistOn`	Flag indicating ABAQUS History Options have been defined.
`REAL8 AbaHistTimePeriod`	Time period.
`REAL8 AbaHistInitTimeInc`	Initial time increment.
`REAL8 AbaHistMinTimeInc`	Minimum time increment.
`REAL8 AbaHistMaxTimeInc`	Maximum time increment.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AbaHistEigenvectors`	Number of eigenvectors.
`REAL8 AbaHistMinFreq`	Dynamic analysis minimum frequency.
`REAL8 AbaHistMaxFreq`	Dynamic analysis maximum frequency.
`REAL8 AbaHistShiftFreq`	Dynamic analysis shift frequency.
`REAL8 AbaHistScaleFactor`	Time step scale factor.
`INT4 AbaHistOutInterval`	Time step number of output intervals.
`BOOL AbaHistNonlinGeom`	Flag to specify nonlinear geometry
`BOOL AbaHistLinearPert`	Flag to specify linear perturbation.
`BOOL AbaHistAutoContact`	Flag to specify auto contact.
`BOOL AbaHistStabilize`	Flag to stabilize contact.
`BOOL AbaHistOutZeroInc`	Flag to write results for zero time increment.
`INT4 AbaHistStepAmp`	Step amplitude: 0=Default, 1=Step, 2=Ramp.
`INT4 AbaHistStepLoad`	Step loads: 0=New, 1=Modify.
`INT4 AbaHistStepConstr`	Step constraints: 0=New, 1=Modify.
`INT4 AbaHistAnalysisType`	Step-based analysis type (see *Analysis Type* property above): 0=Default="Use Analysis Type"
ANSYS Options	
ANSYS Command and Model Control and ANSYS Command Control (Legacy)	
`BOOL AnsCmlOn`	Flag indicates ANSYS Command and Model Control or ANSYS Command Control options (Legacy) are specified.
`INT4 AnsCmlVersion`	Version of ANSYS.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AnsCmlPid`	Value of item in ANSYS Product drop-down.
`STRING AnsCmlProd`	Title of item in ANSYS Product drop-down.
`BOOL AnsCmlList`	Flag to List Input File in Output Text File.
`STRING AnsCmlJobname`	Jobname (maximum 259 characters).
`STRING AnsCmlDir`	Output Directory (maximum 259 characters).
`INT4 AnsCmlDBmem`	Database Memory (in Megabytes).
`INT4 AnsCmlWSmem`	Workspace Memory (in Megabytes.)
`STRING AnsCmlUser`	Other Command Line Inputs (maximum 259 characters).
ANSYS Command and Model Control - Model Control, Result Control, Manual Control	
`INT4 AnsModelGroupID`	Specifies group ID for Portion of Model to Write option. 0=Entire Model, above 0=ID of group to export.
`BOOL AnsModelSkipCsys`	Flag to Skip Coordinate Systems during export.
`BOOL AnsModelSkipNode`	Flag to Skip Nodes during export.
`BOOL AnsModelSkipElem`	Flag to Skip Elements during export.
`BOOL AnsModelSkipMatl`	Flag to Skip Materials during export.
`BOOL AnsModelSkipProp`	Flag to Skip Properties during export.
`BOOL AnsModelSkipGroup`	Flag to Skip Groups during export.
`BOOL AnsModelSkipMisc`	Flag to Skip Miscellaneous text during export.
`INT4 AnsModelRSTcomp`	Specifies File Compression Level option. 0=Sparse, 1=No Compression.
`BOOL AnsModSkipStandard`	Flag to skip output of standard ANSYS model.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AnsModStartText`	ID of text written at beginning of ANSYS model.
`INT4 AnsModEndText`	ID of text written at end of ANSYS model.
ANSYS Load Step - Master Case Options	

`INT4 BCSet[0..20]`
or
`VARIANT vBCSet`
	SetIDs of load and constraint sets for this case: [0]=Constraints, [1]=Constraint Equations, [2]=Loads, [3]=Initial Conditions, [10]=Bolt Preloads, [12]=Temperatures, [16]=Glue Set, [17]=Contact Set

`BOOL AnsLdstepOn`	Flag indicating ANSYS Load Step options have been defined.
`BOOL AnsKbc`	Flag to use Amplitude (KBC) option.
`BOOL AnsPstres`	Flag to use Include Prestress Effect (PSTRES) option.
`INT4 AnsEqslv`	Specifies Equation Solver (EQSLV): 0=Sparse Direct, 1=Jacobi Conjugate Gradient iterative, 2=Incomplete Cholesky Conjugate Gradient iterative, 3=Quasi-Minimal Residual iterative, 4=Preconditioned Conjugate Gradient iterative.
`BOOL AnsNewConstraint`	Flag to specify New (True) or Modify (False) for Constraints.
`BOOL AnsNewLoad`	Flag to specify New (True) or Modify (False) for Loads.
`BOOL AnsNewContact`	Flag to specify New (True) or Modify (False) for Contacts.
ANSYS Time Step Options - Master Case Options	
`BOOL AnsTimestepOn`	Flag indicating ANSYS Time Step options have been defined.
`REAL8 AnsTime`	Time Duration.
`BOOL AnsAutots`	Flag to use Automatic Time Stepping (AUTOTS) option.
`BOOL AnsNsubs`	Flag to set Number of Steps (True) or Size of Steps (False).

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AnsNsbstep`	Used when AnsNsubs is True, sets Number of Steps.
`INT4 AnsNsbmx`	Used when AnsNsubs is True, sets Max Number of Steps.
`INT4 AnsNsbmn`	Used when AnsNsubs is True, sets Min Number of Steps.
`REAL8 AnsDtime`	Used when AnsNsubs is False, sets Time Step.
`REAL8 AnsDtmin`	Used when AnsNsubs is False, sets Min Time Step.
`REAL8 AnsDtmax`	Used when AnsNsubs is False, sets Max Time Step.
`BOOL AnsPlslimit`	Flag to use Max Plastic Strain option with specified value.
`REAL8 AnsPlslimitvalue`	Max Plastic Strain value.
`BOOL AnsCrplimit`	Flag to use Max Creep Strain option with specified value.
`REAL8 AnsCrplimitvalue`	Max Creep Strain value.
`BOOL AnsDpplimit`	Flag to use Max Pressure Change option with specified value.
`REAL8 AnsDpplimitvalue`	Max Pressure Change value.
`BOOL AnsDsplimit`	Flag to use Max Disp Change option with specified value.
`REAL8 AnsDsplimitvalue`	Max Disp Change value.
`BOOL AnsBisect`	Flag to use Bisection Factor option with specified value.
`REAL8 AnsBisectfactor`	Bisection Factor value.
`BOOL AnsPredictor`	Flag to use Predict to Bisect option.
`BOOL AnsMidtol`	Flag to use Midstep Tolerance option with specified value.
`REAL8 AnsTolerb`	Midstep Tolerance value.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsResfq`	Flag to use Response Frequency option.
ANSYS Modal Analysis Options - Master Case Options	
`BOOL AnsModalOptOn`	Flag indicating ANSYS Modal Analysis options have been defined.
`INT4 AnsModalMethod`	Select Method for model extraction: 0=Block Lanczos, 1=PCG Lanczos, 2=Supernode, 3=Subspace, 4=Unsymmetrical, 5=Damped, 6=Damped w/ QR.
`INT4 AnsModalNmode`	Number Frequency Modes.
`REAL8 AnsModalFreqb`	Frequency value for From.
`REAL8 AnsModalFreqe`	Frequency value for To.
`BOOL AnsModalNrmkey`	Flag to use Mass (True) or Max=Unity (False) for Mode Normalization.
`BOOL AnsModalStrmck`	Flag to enable Strum Sequence Check.
`REAL8 AnsModalRangefact`	RangeFact value (only used when `AnsModalMethod`=2).
`INT4 AnsModalBlocksize`	Blocksize value (only used when `AnsModalMethod`=2).
`INT4 AnsModalRobustlev`	RobustLev value (only used when `AnsModalMethod`=2).
`INT4 AnsModalCompute`	Specifies if Eigenvalues and/or Eigenvectors options should on/off. 0=Both Off, 1=Eigenvalues Only On, 2=Eignevectors Only On, 3=Both On (only used when `AnsModalMethod`=2).
`BOOL AnsModalReusekey`	Flag to enable Eigen Reuse (only used when `AnsModalMethod`=6).
`INT4 AnsModalSymmeth`	Select Mode Extract symmetrical method for Damped w/QR: 0=Block Lanczos, 1=Subspace, 2=Supernode (only used when `AnsModalMethod`=6).
ANSYS Buckling Analysis Options - Master Case Options	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsBuckleOptOn`	Flag indicating ANSYS Buckling Analysis options have been defined.
`INT4 AnsBuckleMethod`	Select Method for model extraction: 0=Block Lanczos, 1=Subspace.
`INT4 AnsBuckleNmode`	Number of Buckle Modes.
`REAL8 AnsBuckleShift`	Specifies Shift value when `AnsBuckleRangekey` is True (Center option) or Lower Bound when `AnsBuckleRangekey` is False (Range option).
`REAL8 AnsBuckleLdmulte`	Specifies Bound value when `AnsBuckleRangekey` is True (Center option) or Upper Bound when `AnsBuckleRangekey` is False (Range option).
`BOOL AnsBuckleRangekey`	Flag to use Center (True) or Range (False) for *Range Type*.
`BOOL AnsBuckleStrmck`	Flag to enable Strum Sequence Check.
ANSYS Harmonic Analysis Options - Master Case Options	
`BOOL AnsHarmonicOptOn`	Flag indicating ANSYS Harmonic Analysis options have been defined.
`BOOL AnsHarmonicMethod`	Flag for Modal (True) or Physical (False) option for Frequency Response analysis.
`INT4 AnsHarmonicMinmode`	Min Mode Value (only used `AnsHarmonicMethod` = True).
`INT4 AnsHarmonicMaxmode`	Max Mode Value (only used `AnsHarmonicMethod` = True).
`REAL8 AnsHarmonicFreqb`	Frequency value for Hz From.
`REAL8 AnsHarmonicFreqe`	Frequency value for Hz To.
`INT4 AnsHarmonicLogopt`	Specifies Log Frequency Span: 0=Logarithm, 1=Octave band, 2=1/2 Octave band, 3=1/3 Octave band, 4=1/6 Octave band, 5=1/12 Octave band, 6=1/24 Octave band.
`BOOL AnsHarmonicReimky`	Flag for Real/Imag (True) or Ampl/Phase (False) option for Complex Key.
`BOOL AnsHarmonicClust`	Flag for Cluster (True) or Uniform (False) option for Cluster.

Unpublished work. © 2025 Siemens

Property	Description
ANSYS Transient Dynamics Options - Master Case Options	
`BOOL AnsTransOptOn`	Flag indicating ANSYS Transient Dynamics options have been defined.
`BOOL AnsTransMethod`	Flag for Modal (True) or Physical (False) option for Transient analysis.
`INT4 AnsTransMinmode`	Min Mode Value (only used `AnsTransMethod` = True).
`INT4 AnsTransMaxmode`	Max Mode Value (only used `AnsTransMethod` = True).
`BOOL AnsTransTintopt`	Flag for HHT (True) or Newmark (False) option for Time Integration.
`REAL8 AnsTransDmpsfreq`	Excitation Frequency value.
`BOOL AnsTransLumpm`	Flag to enable Lumped option for Mass Matrix.
`REAL8 AnsTransAlphad`	Mass value (Damping).
`REAL8 AnsTransBetad`	Stiffness value (Damping).
`REAL8 AnsTransDmpstr`	Struct value (Damping).
`REAL8 AnsTransDmprat`	Ratio value (Damping).
`BOOL AnsTransGamma`	Flag to use gamma option with specified value (when enabled, alpha, delta, alphaf, and alpham are ignored).
`REAL8 AnsTransGammavalue`	gamma value.
`REAL8 AnsTransAlpha`	alpha value (only used when `AnsTransGamma` is False).
`REAL8 AnsTransDelta`	delta value (only used when `AnsTransGamma` is False).
`REAL8 AnsTransAlphaf`	alphaf value (only used when `AnsTransGamma` is False).
`REAL8 AnsTransAlpham`	alpham value (only used when `AnsTransGamma` is False).
ANSYS Nonlinear Options - Master Case Options	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsNlOptOn`	Flag indicating ANSYS Nonlinear options have been defined.
`INT4 AnsNropt`	Specifies Method for Stiffness Updates: 0=ANSYS choose, 1=Full Newton Raphson, 2=Modified Newton Raphson, 3=Initial Stiffness, 4=Full Newton Raphson Unsymmetrical.
`INT4 AnsNeqit`	Max Iterations value.
`BOOL AnsCnvtolU`	Flag to use Displacement option (Convergence Tolerances) with specified value.
`REAL8 AnsUtoler`	Displacement value.
`BOOL AnsCnvtolRot`	Flag to use Rotation option (Convergence Tolerances) with specified value.
`REAL8 AnsRottoler`	Rotation value.
`BOOL AnsCnvtolF`	Flag to use Force option (Convergence Tolerances) with specified value.
`REAL8 AnsFtoler`	Force value.
`BOOL AnsCnvtolM`	Flag to use Moment option (Convergence Tolerances) with specified value.
`REAL8 AnsMtoler`	Moment value.
`BOOL AnsCnvtolDvol`	Flag to use Volume Change option (Convergence Tolerances) with specified value.
`REAL8 AnsDvoltoler`	Volume Change value.
`BOOL AnsCnvtolHdsp`	Flag to use Hydro Pressure option (Convergence Tolerances) with specified value.
`REAL8 AnsHdsptoler`	Hydro Pressure value.
`BOOL AnsNlgeom`	Flag to use Nonlinear Geometry option.
`BOOL AnsArclen`	Flag to use Arc-Length option.
`BOOL AnsLnsrch`	Flag to use Line Search option.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsPred`	Flag to use Predictor option.
ANSYS Output Requests - Master Case Options	
`BOOL AnsOutputOn`	Flag indicating ANSYS Output Requests have been defined.
ANSYS Output Requests - Binary Result File (OUTRES) Section	
`BOOL AnsOutresAll`	Flag to select All Results option.
`INT4 AnsOutresAllFreq`	Specify FREQ value for All Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresAllNth`	Specify integer value for OR STEPS for All Results, only used when `AnsOutresAlclFreq`=3.
`BOOL AnsOutresBasic`	Flag to select Basic Results option.
`INT4 AnsOutresBasicFreq`	Specify FREQ value for Basic Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresBasicNth`	Specify integer value for OR STEPS for Basic Results, only used when `AnsOutresBasicFreq`=3.
`BOOL AnsOutresRsol`	Flag to select Nodal Reaction option.
`INT4 AnsOutresRsolFreq`	Specify FREQ value for Nodal Reaction: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresRsolNth`	Specify integer value for OR STEPS for Nodal Reaction, only used when `AnsOutresRsolFreq`=3.
`BOOL AnsOutresNsol`	Flag to select Nodal DOF option.
`INT4 AnsOutresNsolFreq`	Specify FREQ value for Nodal DOF: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutresNsolNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutresNsolFreq`=3
`BOOL AnsOutresVel`	Flag to select Velocity option
`INT4 AnsOutresVelFreq`	Specify FREQ value for Velocity: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.

Unpublished work. © 2025 Siemens

Property	Description
INT4 AnsOutresVelNth	Specify integer value for OR STEPS for Velocity, only used when AnsOutresVelFreq=3.
BOOL AnsOutresAcc	Flag to select Acceleration option.
INT4 AnsOutresAccFreq	Specify FREQ value for Acceleration: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
INT4 AnsOutresAccNth	Specify integer value for OR STEPS for Acceleration, only used when AnsOutresAccFreq=3.
BOOL AnsOutresEsol	Flag to select Element Results option.
INT4 AnsOutresEsolFreq	Specify FREQ value for Element Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
INT4 AnsOutresEsolNth	Specify integer value for OR STEPS for Element Results, only used when AnsOutresEsolFreq=3.
ANSYS Output Requests - Print Result File (OUTPR) Section	
BOOL AnsOutprAll	Flag to select All Results option.
INT4 AnsOutprAllFreq	Specify FREQ value for All Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
INT4 AnsOutprAllNth	Specify integer value for OR STEPS for All Results, only used when AnsOutprAllFreq=3.
BOOL AnsOutprBasic	Flag to select Basic Results option.
INT4 AnsOutprBasicFreq	Specify FREQ value for Basic Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
INT4 AnsOutprBasicNth	Specify integer value for OR STEPS for Basic Results, only used when AnsOutprBasicFreq=3.
BOOL AnsOutprRsol	Flag to select Nodal Reaction option.
INT4 AnsOutprRsolFreq	Specify FREQ value for Nodal Reaction: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
INT4 AnsOutprRsolNth	Specify integer value for OR STEPS for Nodal Reaction, only used when AnsOutprRsolFreq=3.
BOOL AnsOutprNsol	Flag to select Nodal DOF option.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 AnsOutprNsolFreq`	Specify FREQ value for Nodal DOF: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprNsolNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutprNsolFreq`=3.
`BOOL AnsOutprVel`	Flag to select Velocity option
`INT4 AnsOutprVelFreq`	Specify FREQ value for Velocity: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprVelNth`	Specify integer value for OR STEPS for Velocity, only used when `AnsOutprVelFreq`=3.
`BOOL AnsOutprAcc`	Flag to select Acceleration option.
`INT4 AnsOutprAccFreq`	Specify FREQ value for Acceleration: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprAccNth`	Specify integer value for OR STEPS for Acceleration, only used when `AnsOutprAccFreq`=3.
`BOOL AnsOutprEsol`	Flag to select Element Results option.
`INT4 AnsOutprEsolFreq`	Specify FREQ value for Element Results: 0=None, 1=Write All, 2=Write Last, 3=Write Every Nth.
`INT4 AnsOutprEsolNth`	Specify integer value for OR STEPS for Element Results, only used when `AnsOutprEsolFreq`=3.
LS-DYNA Options	
LS-DYNA Model Options	
`BOOL DynModOn`	Flag Indicating LS-DYNA parameters have been specified.
`BOOL DynModExportRigidNodes`	Flag to export Rigid Reference Nodes.
`BOOL DynModMatlBasedParts`	Flag to write Material Based Plate Parts.
`INT4 DynModelGroupID`	Specifies group ID for Portion of Model to Write option. 0=Entire Model, above 0=ID of group to export.
`BOOL DynModelSkipCsys`	Flag to Skip Coordinate Systems during export.
`BOOL DynModelSkipNode`	Flag to Skip Nodes during export.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL DynModelSkipElem`	Flag to Skip Elements during export.
`BOOL DynModelSkipMatl`	Flag to Skip Materials during export.
`BOOL DynModelSkipProp`	Flag to Skip Properties during export.
`BOOL DynModelSkipGroup`	Flag to Skip Groups during export.
`BOOL DynModelSkipMisc`	Flag to Skip Miscellaneous text during export.
`BOOL DynModSkipStandard`	Flag to skip the standard LS-DYNA model section.
`INT4 DynModStartText`	ID of text written at the beginning of the model section.
`INT4 DynModEndText`	ID of text written at the end of the model section.
`BOOL DynModGroupsAsSets`	(Obsolete) Flag to write Femap groups as sets.
Master Case Options - LS-DYNA Analysis Control	
`BOOL DynHistOn`	Flag indicating an Dyna History section has been defined.
`INT4 DynHistBodyLoadGrp`	Initial Body Load Group.
`BOOL DynHistMassDampOn`	Flag indicating Mass Damping value has been defined.
`REAL8 DynHistMsssDamp`	Mass Damping Value.
`BOOL DynHistImplicit`	Flag indicating use of Implicit Solver.
`BOOL DynHistSpringback`	Flag indicating Springback option has been defined.
`BOOL DynHistStrinOutput`	Flag indicating Strain will be output.
`REAL8 DynHistTotalTime`	Value defining total solution time.
`REAL8 DynHistTimeHistOuput`	Number indicating output time interval.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 DynHistPrintHistOutput`	Number indicating print output time interval.
`INT4 DynHistPlateIntgPts`	Number indicating number of plate integration points.
`INT4 DynHistGroupHistId`	ID of group to limit output request.
NASTRAN Options	
Master Case Options - NASTRAN Specific	
`BOOL NasOmodesOn`	Flag indicating an OMODES request has been defined.
`STRING NasOmodes[160]`	Expression used to select OMODES output.
Analysis Set Options - NASTRAN Specific	
NASTRAN Executive Control Options	
`BOOL NasExecOn`	Flag indicating NASTRAN Executive Options have been defined.
`INT4 NasExecVersion`	Version of NASTRAN to support - Possible values 2004, 2001 and 70 (for all previous versions).
`STRING NasExecOutDir[260]`	Path to directory for output.
`STRING NasExecAnalyzeFilename[80]`	Base File Name for input file (maximum 80 characters).
`STRING NasExecUserCmdLine[260]`	Additional Command Line Arguments.
`STRING NasExecProblem[160]`	Problem ID.
`STRING NasExecSolName[80]`	Name of solution - specified as text, but could be a number.
`REAL8 NasExecMaxTime`	Maximum execution time.
`STRING NasExecDiag[160]`	String specifying diagnostics to include in the solution.
`BOOL NasExecIterSolve`	Flag to select iterative solver.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasExecIterSolve2`	Similar to `NasExecIterSolve`, but allows you to set 0=Off, 1=On, 2=NX Nastran Elemental Iterative Solver.
`INT4 NasExecNumCPU`	Number of SMP Cores (Processors) to use during SMP solution: 0=Default=1 Processor. **Note** SMP is the default setup when using Simcenter Nastran installed with Femap.
`INT4 NasExecNumDmpCPU`	Number of DMP CPUs to use during DMP solution: 0=Default=1 CPU. **Note** DMP requires additional setup.
`BOOL NasExecSaveDB`	Flag to save solution databases for later restart.
`BOOL NasExecRestart`	Flag indicating you want to restart a previous analysis.
`STRING NasExecRestartDB[260]`	Path to solution database that you want to restart from.
`BOOL NasExecRestartReadOnly`	Flag indicating you want to perform a read only restart.
`INT4 NasExecRestartVersion`	ID of restart version in master file.
`INT4 NasExecRestartSubcase`	ID of subcase to start from when restarting a nonlinear analysis.
`BOOL NasExecSkipStandard`	Flag to skip writing standard Executive Control.
`INT4 NasExecStartText`	ID of text to include at beginning of Executive Control.
`INT4 NasExecEndText`	ID of text to include at end of Executive Control.
`INT4 NasExecMemory`	Value of solution memory in Mb.
`BOOL NasExecGPU`	Flag to enable GPU Computing.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasExecGPUOpt`	Sets the GPU Option:0=DCMP, FRRD1;1=DCMP; 2=FRRD1.
`BOOL NasExecExtendedMessages`	Flag to enable Extended Error Messages.
`BOOL NasExecSolutionMonitor`	(OBSOLETE) Use `NasSolutionMonitorOpt` instead to select option for Solution Monitor.
`INT4 NasExecSolutionMonitorOpt`	Sets the Solution Monitor option: 0=No Extended Solution Status Monitoring. 1=Generate Monitor files for Femap Solution Monitor. 2=Launch Simcenter Nastran's Solution Monitor.
`STRING NasExecSystemCell[160]`	Specifies System Cell options for Nastran.
`STRING NasExecScratchDir1[260]`	Specifies primary scratch directory.
`STRING NasExecScratchDir2[260]`	Specifies secondary scratch directory.
`INT4 NasExecScratchSize1`	Size of primary scratch directory (in MBytes).
`INT4 NasExecScratchSize2`	Size of secondary scratch directory (in MBytes).
`INT4 NasExtSEOutAssignForm`	Sets the Form option in the Create Assign Statement dialog box accessed by clicking **Start Text**, then clicking **Create ASSIGN**. 0=None, 1=FORMATTED, 2=UNFORMATTED, 3=LITTLEENDIAN, 4=BIGENDIAN.
NASTRAN Bulk Data Options	
`BOOL NasBulkOn`	Flag indicating NASTRAN Bulk Data Options have been defined.
`INT4 NasBulkGroupID`	ID of the group of entities to be written to the NASTRAN file. 0 = entire model.
`BOOL NasBulkAutospc`	Flag to write PARAM,AUTOSPC.
`INT4 NasBulkAutospcType`	ID of Autospc method 0=Eigenvalue, 1=1..Singular Value Decomposition.
`BOOL NasBulkGrdpnt`	Flag to write PARAM,GRDPNT.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasBulkGrdpntVal`	Value to include on PARAM,GRDPNT.
`BOOL NasBulkWtmass`	Flag to write PARAM,WTMASS.
`REAL8 NasBulkWtmassVal`	Value to include on PARAM,WTMASS.
`BOOL NasBulkK6rot`	Flag to write PARAM,K6ROT.
`REAL8 NasBulkK6rotVal`	Value to include on PARAM,K6ROT.
`BOOL NasBulkMaxratio`	Flag to write PARAM,MAXRATIO.
`REAL8 NasBulkMaxratioVal`	Value to include on PARAM,MAXRATIO.
`BOOL NasBulkNdamp`	Flag to write PARAM,NDAMP.
`REAL8 NasBulkNdampVal`	Value to include on PARAM,NDAMP.
`BOOL NasBulkInrel`	Flag to write PARAM,INREL.
`INT4 NasBulkInrelVal`	ID of Inertia Relief option: -1=On, -2=Automatic.
`BOOL NasBulkBoltFact`	Flag to write PARAM, BOLTFACT.
`REAL8 NasBulkBoltFactVal`	Value to be written with PARAM, BOLTFACT option.
`BOOL NasBulkEnfmotn`	Flag to write NASTRAN command option for NX Nastran or PARAM,ENFMOTN for MSC Nastran.
`INT4 NasBulkEnfmotnOpt`	ID of ENFMOTN option: 0,1, 2.
`BOOL BulkSwpangle`	Flag to write PARAM, SWPANGLE.
`REAL8 NasBulkSwpangleVal`	Value to be written with PARAM, SWPANGLE option.
`BOOL NasBulkMgrid`	Flag to write PARAM, MGRID.
`INT4 NasBulkMgridID`	ID of Node to write with PARAM, MGRID.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasBulkMdofVal`	DOF written with PARAM, MGRID, must be an integer 1-6.
`INT4 NasBulkMatnl`	When turned on and set to 1, writes PARAM,MATNL, 1. When activated and set to -1, writes PARAM,MATNL,-1. When deactivated, writes nothing and default is used.
`BOOL NasBulkTENSOQD`	If True, the TENSOQD option is turned on in the dialog box.
`INT4 NasBulkTENSOQDval`	When option is turned on and set to 1, writes PARAM,TENSOQD, 1, which turns off the tension-only behavior of shell elements in SOL 401, even if appropriate values have been specified on Plate Property. When option is turned on and set to -1, writes PARAM,TENSOQD,-1., which allows the tension-only behavior to be invoked in SOL 401 as long a appropriate values have been specified on Plate Property.
`BOOL NasBulkLangle`	Flag to write PARAM,LANGLE.
`BOOL NasBulkLgdisp`	Flag to write PARAM,LGDISP.
`BOOL NasBulkLgstrn`	If True, writes PARAM,LGSTRN,1 to include effects of large strain.
`BOOL NasBulkPrgpst`	Flag to write PARAM,PRGPST.
`BOOL NasBulkOgeom`	Flag to write PARAM,OGEOM.
`BOOL NasBulkSrcomps`	If True, writes PARAM,SRCOMPS,YES to output Laminate Strength Ratios.
`BOOL NasBulkNofisr`	If True, writes PARAM,NOFISR,1 to skip failure indices to the F06 file.
`BOOL NasBulkPARAMBailout`	Flag to write PARAM,BAILOUT,-1.
`BOOL NasBulkAutompc`	Flag to write PARAM,AUTOMPC.
`BOOL NasBulkDdrmmOff`	Flag to control PARAM,DDRMM.
`BOOL NasBulkModacc`	Flag to write PARAM,MODACC.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasBulkResVec`	Flag to write PARAM,RESVEC.
`BOOL NasBulkResVecOn`	Value to write with PARAM,RESVEC.
`BOOL NasBulkResViner`	Flag to write PARAM,RESVINER.
`BOOL NasBulkCntAset`	Flag to write PARAM,CNTASET.
`BOOL NasBulkSecomb`	Flag to write PARAM,SECOMB.
`BOOL NasBulkAlpha1`	Flag to write PARAM,ALPHA1.

`REAL8 NasBulkAlpha1Val[0..1]`
or
`VARIANT vNasBulkAlpha1Val`
	Value to be written with PARAM, ALPHA1 option, 0=Real, 1=Complex.

`BOOL NasBulkAlpha2`	Flag to write PARAM,ALPHA2.

`REAL8 NasBulkAlpha2Val[0..1]`
or
`VARIANT vNasBulkAlpha2Val`
	Value to be written with PARAM, ALPHA2 option, 0=Real, 1=Complex.

`INT4 NasBulkLargeField`	Selection of field format: 0=Small,1=Large Csys,Matl,Prop, 2=Large All But Elem, 3=Large, 4 = Large Csys, Node.
`BOOL NasBulkQuadr`	Flag selecting CQUADR/CTRIAR as the preferred plate/shell elements.
`BOOL NasBulkSkipBeamShape`	Flag to skip writing Beam Shapes as PBEAML/PBARL.
`BOOL NasBulkRigidThermalExp`	If True includes effects of thermal expansion for rigid elements.
`BOOL NasBulkGapsAsContact`	Flag to write Gap Elements as contact.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasBulkDynLdAsLOADSET`	Flag to write Dynamic Loads using LOADSET/LSEQ method.
`BOOL NasBulkWriteAllLoadBCSets`	Flag to write all Static Loads and BC Sets.
`BOOL NasBulkSkipStandard`	Flag to skip standard bulk data.
`BOOL NasBulkEndTextOutsideBulk`	Flag to write End Text at End of File Outside Bulk.
`INT4 NasBulkStartText`	ID of text written at the beginning of the Bulk Data.
`INT4 NasBulkEndText`	ID of text written at the end of the Bulk Data.
`BOOL NasBulkPARAMBuckle`	Flag to write PARAM, BUCKLE.
Monitor Points	
`BOOL NasMonitorEnabled`	Flag indicating Monitor Point Output Requests will be written to the analysis case, when applicable.
`INT4 NasMptSetID`	ID of the Set Object containing the Monitor Points selected for Export, which can be used in conjunction with the Methods of the Analysis Set Object to Get, Put, and Clear the list.
`BOOL NasMonitorLoadEnabled`	Flag to set Print Load Monitor Results option.
`BOOL NasMonitorAeroEnabled`	Flag to set Print Aero Monitor Results option.
`BOOL NasMonitorElemEnabled`	Flag to set Print Element Monitor Results option.
Incompressible Fluid Options	
`BOOL NasBulkSLOSHCMS`	Flag to write PARAM, SLOSHCMS.
`REAL8 NasBulkSLOSHCMSVal`	Value to be written for SLOSHCMS incompressible fluid option.
`BOOL NasACMODLExport`	Flag to export Fluid-Structure Interface Modeling Parameters.
`REAL8 NasACMODLNORMAL`	Value to be written for the Outward Normal Search Distance option of the FSI Modeling Parameters dialog box (Real > 0.0).

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasACMODLOVLPANG`	Value to be written for the Angular Tolerance option of the FSI Modeling Parameters dialog box.
`INT4 NasACMODLSRCHUNIT`	Sets the Search Units option of the FSI Modeling Parameters dialog box. The choices are 0..Relative and 1..Absolute.
`REAL8 NasACMODLINTOL`	Value to be written for the Inward Normal Search Distance option of the FSI Modeling Parameters dialog box (Real > 0.0).
`INT4 NasACMODLAREAOP`	Sets the Coupling Method option of the FSI Modeling Parameters dialog box. The choices are 0..Recommended Method and 1..RBE3 Method.
`INT4 NasACMODLCTYPE`	Sets the Type of Coupling option of the FSI Modeling Parameters dialog box. The choices are 0..Two-way Coupling (STRONG) and 1..Effect of Structure on Fluid Only (WEAK).
`NasACMODLFSETEntityOpt`	
`BOOL NasACMODLFSETOpt`	When True, activates FLSTCNT options in the FSI Control Parameters dialog box.
`INT4 NasACMODLFSET`	SID of a SET1 or SET3 bulk entry that defines the fluid elements to include in the coupling evaluation. A negative sign in front of the SID instructs the software to include all fluid elements minus those associated with the SET1 or SET3 bulk entry in the coupling evaluation. (Integer ≠ 0; Default = Include all fluid elements in the coupling evaluation).
`NasACMODLSSETEntityOpt`	
`BOOL NasACMODLSSETOpt`	When True, activates ACMODL options in the FSI Modeling Parameters dialog box.
`INT4 NasACMODLSSET`	SID of a SET1 or SET3 bulk entry that defines the structural elements for the coupling evaluation. A negative sign in front of the SID instructs the software to include all structural elements minus those associated with the SET1 or SET3 bulk entry in the coupling evaluation. (Integer ≠ 0; Default = Include all structural elements in the coupling evaluation).

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasFLSTCNTExport`	Flag to export Fluid-Structure Interface Control Parameters.
`INT4 NasFLSTCNTACSYM`	Sets the Symmetric Request option of the FSI Control Parameters dialog box. The choices are 0..NO and 1..YES (default = YES).
`INT4 NasFLSTCNTACOUT`	Sets the Particle Vel Output Type option of the FSI Control Parameters dialog box. The choices are 0..Peak value output and 1..Root-mean-square output.
`INT4 NasFLSTCNTASCOUP`	Sets the Coupled Solution option of the FSI Control Parameters dialog box. The choices are 0..NO and 1..YES (default = YES).
`REAL8 NasFLSTCNTREFDB`	Value to be written for the Reference Pressure option of the FSI Control Parameters dialog box.
`INT4 NasFLSTCNTSKINOUT`	Sets the Coupling Information option of the FSI Control Parameters dialog box. The choices are 0..NONE, 1..PUNCH, 2..FREEFACE, and 3..STOP.
`INT4 NasFLSTCNTAGGPCH`	Sets the Coupling Matrix to PCH option of the FSI Control Parameters dialog box. The choices are 0..NO and 1..YES (default = NO).
`INT4 NasFLSTCNTFEF70`	Sets the Matrix from SFE AKUSMOD option of the FSI Control Parameters dialog box. The choices are 0..NO and 1..YES (default = NO).
`BOOL NasINCMOPTOn`	Flag to write PARAM, INCMOPT
`BOOL NasINCMOPTExport`	Flag to export Incompressible Fluid Options.
`INT4 NasINCMOPTMETHOD`	Sets the Modal Calc Method option of the Incompressible Fluid Options dialog box. The choices are 0..Subspace Iteration (SUBS), 1..Dry Modes plus Residual Vectors (DRYMODES), and 2..ARPACK.
`REAL8 NasINCMOPTSF`	Value to be written for the Scale Factor option of the Incompressible Fluid Options dialog box. (Real > 1.0)
`REAL8 NasINCMOPTSFTOL`	Value to be written for the Surface Flatness Tolerance option of the Incompressible Fluid Options dialog box. (Real > 0.0)

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasINCMOPTBSIZE`	Value to be written for the Block Size option of the Incompressible Fluid Options dialog box. (Integer > 0)
`INT4 NasINCMOPTNITER`	Value to be written for the Max Iterations for SUBS option of the Incompressible Fluid Options dialog box. (Integer > 0)
`INT4 NasINCMOPTRBFLMASS`	Sets the Compute G-set INCMPFL Mass option of the Incompressible Fluid Options dialog box. The choices are 0..NO and 1..YES (default = NO).
`BOOL NasINCMOPTVMESH`	If True, the output visualization mesh for sloshing surfaces will be requested.
`INT4 NasINCMOPTGOFFSET`	The ID offset for nodes in the output visualization mesh.
`INT4 NasINCMOPTEOFFSET`	The ID offset for elements in the output visualization mesh.
Direct Matrix Input (DMIG)	
`BOOL NasEnableDMIG`	Flag indicating Enable Matrix Input.
`INT4 NasDMIGK2GG`	ID of Direct Matrix Input entity referenced by Stiffness drop-down in G-Type Matrices. Writes K2GG Case Control entry.
`INT4 NasDMIGM2GG`	ID of Direct Matrix Input entity referenced by Mass drop-down in G-Type Matrices. Writes M2GG Case Control entry.
`INT4 NasDMIGB2GG`	ID of Direct Matrix Input entity referenced by Damping drop-down in G-Type Matrices. Writes B2GG Case Control entry.
`INT4 NasDMIGK42GG`	ID of Direct Matrix Input entity referenced by Structural Damping drop-down in G-Type Matrices. Writes K42GG Case Control entry.
`INT4 NasDMIGP2G`	ID of Direct Matrix Input entity referenced by Loading drop-down in G-Type Matrices. Writes P2G Case Control entry.
`INT4 NasDMIGK2PP`	ID of Direct Matrix Input entity referenced by Stiffness drop-down in P-Type Matrices. Writes K2PP Case Control entry.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasDMIGM2PP`	ID of Direct Matrix Input entity referenced by Mass drop-down in P-Type Matrices. Writes M2PP Case Control entry.
`INT4 NasDMIGB2PP`	ID of Direct Matrix Input entity referenced by Damping drop-down in P-Type Matrices. Writes B2PP Case Control entry.
NASTRAN Nonlinear Analysis Options - Master Case Options	
`BOOL NasCnlOn`	Flag indicating the Nonlinear Case Options has been defined.
`BOOL NasCnlUseLoadSet`	Flag to use the Nonlinear options from the Load Set.
`INT4 NasCnlIncrements`	Number of Time Increments.
`REAL8 NasCnlTime_Increment`	Value for Time Step Size.
`INT4 NasCnlMaxIter`	Nonlinear - Max Iterations per Step.

`BOOL NasCnlConvergenceFlags[0..2]`
or
`VARIANT vNasCnlConvergenceFlags`
	Nonlinear - Convergence Flags for 0=Displacement, 1=Load and 2=Work.

`REAL8 NasCnlConvergenceValue[0..2]`
or
`VARIANT vNasCnlConvergenceValue`
	Nonlinear - Convergence Tolerances for Displacement, Load and Work.

`BOOL MSCNasCnlConvergenceFlags[0..1]`
or
`VARIANT vMSCNasCnlConvergenceFlags`
	Nonlinear - Convergence Flags for 0=Vector and 1=Length.
	
	MSC Nastran Only and there are no tolerance values.

`INT4 NasCnlStiffnessMethod`	Nonlinear - Stiffness Update Method. 0=Default, 1=AUTO, 2=Full.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasCnlKstep`	Nonlinear - Iterations before Stiffness Update.
`INT4 NasCnlIntermediateOutput`	Nonlinear - Intermediate Output Type.
`INT4 NasCnlOutputInterval`	Nonlinear - Output Interval.
`INT4 NasCnlSolutionStrategy`	Nonlinear - Nonlinear Arc-Length Solution Strategy (Arc- Length if nonzero).
`INT4 NasCnlSolutionOverrides`	Nonlinear - Solution Strategy Overrides: 0=none/ advanced, 1=Full Newton Raphson, 2=Modified Newton Raphson.
`BOOL NasCnlModnewtonLineSearch`	Nonlinear - Modified Newton Line Search Override (1=Skip).
`BOOL NasCnlModnewtonQuasiNewton`	Nonlinear - Modified Newton Quasi-Newton Override (1=Skip).
`BOOL NasCnlModnewtonBisection`	Nonlinear - Modified Newton Bisection Override (1=Skip).
`INT4 NasCnlArcConstraintType`	Nonlinear - Arc Length Constraint Type.
`REAL8 NasCnlArcMinAdjust`	Nonlinear - Min Arc Length Adjust.
`REAL8 NasCnlArcMaxAdjust`	Nonlinear - Max Arc Length Adjust.
`REAL8 NasCnlArcLoadScale`	Nonlinear - Arc Length Method Constraint Load Scale.
`INT4 NasCnlArcDesiredIter`	Nonlinear - Arc Length Desired Iterations.
`INT4 NasCnlArcMaxSteps`	Nonlinear - Arc Length Max Steps.
`INT4 NasCnlTimeSkipAdjust`	Nonlinear - Transient Time Step Skip Factor.
`INT4 NasCnlDominantPeriodSteps`	Nonlinear - Transient Steps for Dominant Period.
`REAL8 NasCnlBoundsRb`	Nonlinear - Transient Bounds to Maintain Step.
`REAL8 NasCnlStabilityTolerance`	Nonlinear - Transient Minimum Stability Tolerance.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasCnlDivergenceLimit`	Nonlinear - Max Diverging Conditions.
`INT4 NasCnlQuasiNewtonVectors`	Nonlinear - Number of Quasi-Newton Vectors.
`INT4 NasCnlMaxLineSearch`	Nonlinear - Max Line Search per Iteration.
`REAL8 NasCnlLineSearchTolerance`	Nonlinear - Line Search Tolerance.
`INT4 NasCnlMaxBisections`	Nonlinear - Max Bisections per Increment.
`REAL8 NasCnlMaxRotation`	Nonlinear - Max Rotation per Bisection.
`REAL8 NasCnlFstress`	Nonlinear - Stress Fraction Limit.
`BOOL NasCnlkDAMP`	Nonlinear - Flag to Include Differential Stiffness in Damping.
`REAL8 NasCnlMaxAdjust`	Nonlinear - Max Adjusted vs. Initial Increment.
`BOOL NasCnlCreep`	Flag Indicating this a Creep Analysis.
`BOOL NasCnlSkipNLPARM`	Flag to deactivate writing of the NLPARM for this case.
NASTRAN Nonlinear Options	
`BOOL NasNonlinOn`	Flag indicating NASTRAN Nonlinear Options have been defined.
`INT4 NasNonlinNDT`	Number of time steps.
`REAL8 NasNonlinDT`	Initial time increment. In Femap version 11.2.2 and below, Positive value = Adaptive option selected in Nonlinear Control Options dialog box, Negative value = Constant Interval option selected.
`BOOL NasNonlinAdapt`	Flag to turn on **Adaptive in Step Control** section of Nonlinear Control Options dialog box.
`INT4 NasNonlinNO`	Output step interval.
`INT4 NasNonlinMaxIter`	Max iterations per step.

Unpublished work. © 2025 Siemens

Property	Description

`BOOL NasNonlinEpsFlag[3]`
 	
or
 	
`VARIANT vNasNonlinEpsFlag`
	Flags indicating data to use for checking convergence: 0=Temperature, 1=Load, 2=Work.

`REAL8 NasNonlinEps[3]`
 	
or
 	
`VARIANT vNasNonlinEps`
	Convergence values to check with the convergence flags.

`INT4 NasNonlinKUPDATE`	Stiffness Update Method. 0=Default, 1=AUTO, 2=Full.
`BOOL NasNonlinKDAMP`	Flag to Include Differential Stiffness in Damping.
NASTRAN Modal Options	
`BOOL NasModeOn`	Flag indicating NASTRAN Modal Options have been defined.
`BOOL NasModesSkipEigr`	If True, skips writing the EIGR card for specifying modal analysis options.
`INT4 NasModeEigrSet`	SetID to be used for EIGR (Method ID field).
`INT4 NasModeMethod`	Modal Solution method: 0=Givens,1=Modified Givens, 2=Inverse Power, 3=Inverse Power/ Strum, 4=Householder, 5=Modified Householder, 6=Lanczos, 7=Hessenberg, 8=Complex Inverse Power, 9=Complex Lanczos, 10=Auto(HOU/MHOU)/ Auto (NEi), 11=Subspace (NEi Nastran only).
`BOOL NasModeSolutionType`	Solution type: False=Direct, True=Modal.

`REAL8 NasModeFreqRange[2]`
 	
or
 	
`VARIANT NasModeFreqRange`
	Frequency range of interest, [0]=Min, [1]=Max.

Unpublished work. © 2025 Siemens

Property	Description

`REAL8 NasModeImagFreqRange[2]`
 	
or
 	
`VARIANT vNasModeImagFreqRange`
	For Complex Modes analysis, specifies the imaginary part of the Frequency Range of interest.

`INT4 NasModeEstRoots`	Estimated number of eigenvalues.
`INT4 NasModeDesiredRoots`	Number of desired eigenvalues/modes.
`INT4 NasModeNormOpt`	Normalization method: 0=Mass,1=Max, 2=Point.
`INT4 NasModeNormNode`	Node ID for use with point normalization method.
`INT4 NasModeNormDOF`	Nodal DOF for use with point normalization method.
`INT4 NasModeMassForm`	Mass matrix formulation: 0=Default, 1=Lumped, 2=Coupled.
`BOOL NasModeSelOn`	If True, option in **Response Mode Selection** section is activated.
`BOOL NasModeSelInc`	If True, selects Include. If False, selects Exclude.
`STRING NasModeSel`	String specifying IDs of modes for dynamic response.
`REAL8 NasModesComplexConverge`	Specifies Complex Modes convergence value.
`REAL8 NasModesComplexRegionWidth`	Specifies Region Width for Complex Modes analysis.
`REAL8 NasModeDampOverall`	Specifies PARAM,G.
NASTRAN Dynamic Options	
`BOOL NasDynOn`	Flag indicating NASTRAN Dynamic Options have been defined.
`BOOL NasDynUseLoadSet`	Flag to use dynamic options from the load set.
`REAL8 NasDynDampOverall`	Value for Overall Structural Damping.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasDynDampW3`	Value for System Damping.
`REAL8 NasDynDampW4`	Value for Element Damping.

`REAL8 NasDynKeepFreq[2]`
or
`VARIANT vNasDynKeepFreq`
	Max and Min frequencies used to limit response.

`REAL8 NasDynTranDT`	Value used for Transient Time Increment.
`INT4 NasDynFreqTbl`	ID of Function defining the Solution frequencies.
`INT4 NasDynDampModalTbl`	ID of Function defining the Modal Damping Table.
`INT4 NasDynKeepModes`	Number of Modes used to limit response.
`INT4 NasDynTranTimeSteps`	Number of Transient Time Steps.
`INT4 NasDynTransOutInt`	Number for Transient Output Interval.
`BOOL NasDynDampModalMethod`	TBD
`INT4 NasDynRespSpect`	ID of Function defining the Modal Frequency Correlation for Response Spectrum.
`BOOL NasDynFzero`	Flag to write PARAM,FZERO.
`REAL8 NasDynFzeroVal`	Value of PARAM, FZERO.
`BOOL NasDynWmodal`	Flag to write PARAM,WMODAL.
`INT4 NasDynFreqID`	Value for Frequency ID.
(Obsolete) `INT4 NasDynNoFreq[2]`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use Solution Frequency definition instead. Number of Frequency Intervals.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vNasDynNoFreq`	
(Obsolete) `BOOL NasDynLogInterp[2]` or `VARIANT vNasDynLogInterp`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use Solution Frequency definition instead. Flag to use Logarithmic Interpolation.
(Obsolete) `INT4 NasDynFreqType[2]` or `VARIANT vNasDynFreqType`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use Solution Frequency definition instead. Input type for definition of Solution Frequencies.
(Obsolete) `REAL8 NasDynMinFreq[2]` or `VARIANT vNasDynFreqType`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use Solution Frequency definition instead. Min of frequency range for solution frequencies.
(Obsolete) `REAL8 NasDynMaxFreq[2]` or `VARIANT vNasDynMaxFreq`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use Solution Frequency definition instead. Max of frequency range for solution frequencies.
(Obsolete) `REAL8 NasDynSpreadCluster[2]` or `VARIANT vNasDynSpreadCluster`	*This property is obsolete and has been removed. It will not work in Femap version 11.3 and above. Use Solution Frequency definition instead. Value defining frequency spread around modes.
NASTRAN Modal XY Plot Options	
`BOOL NasModeXYOn`	Flag indicating NASTRAN Modal XY Plot Options have been defined.
`BOOL NasModeXYRequest[0..4]`	Flags selecting types of output for XY plotting. Options: [0]=Modal Participation Factors, [1]=Modal Effective Mass, [2]=Modal Effective

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vNasModeXYRequest`	Weight, [3]=Modal Effective Mass Fraction, [4]=Summary.
`INT4 NasModeXYRefNode`	ID of Reference Node.
NASTRAN Random OUTPUT2 Options	
`BOOL NasRandomOn`	Flag indicating NASTRAN Random Output2 options have been defined.

`INT4 NasRandomRequest[0..1]`
 	
or
 	
`VARIANT vNasRandomRequest`
	Output requests for Output2: [0]=Nodal Output, [1]=Elemental Output. 0=None, 1=Power Spectral Density Functions, 2=Autocorrelation Functions, 3=All, 4=Root Mean Square,

`BOOL NasModeXYRefNode`	Flag indicating output of summary data only.
NASTRAN Random XY Plot Options	
`BOOL NasRandXYOn`	Flag indicating NASTRAN Random XY Plot Options have been defined.

`BOOL NasRandXYRequest[48]`
 	
or
 	
`VARIANT vNasRandXYRequest`
	Flags selecting nodal and elemental output for XY plotting.

`BOOL NasRandXYSummary`	Flag indicating output of summary data only.
`BOOL NasRandXYPsdf`	Flag requesting PSDF XY output.
`BOOL NasRandXYAuto`	Flag requesting Autocorrelation XY output.
`INT4 NasRandXYNodeGroup`	ID of group containing node IDs for XY plot output.
`INT4 NasRandXYElemGroup`	ID of group containing element IDs for XY plot output.
NASTRAN Power Spectral Density Factors	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasRandCorrOn`	Flag indicating Autocorrelation Function Time Lag options have been defined. `NasRandXYOn` or `NasRandomOn` Flag must also be set to True for this Flag to have any effect.
`INT4 NasRandCorrLagInterval`	Number of Lag Intervals.
`REAL8 NasRandCorrLagStart`	Value for Starting Lag.
`REAL8 NasRandCorrLagMax`	Value for Max Lag.
NASTRAN Response Spectrum Generation	
`BOOL NasGenSpecOn`	Flag indicating NASTRAN Response Spectrum Generation Options have been specified.

`BOOL NasGenSpecRequest[18]`
or
`VARIANT vNasGenSpecRequest`
	Flags selecting spectrum generation for displacement, velocity and acceleration DOF.

`BOOL NasGenSpecAbsolute`	Flag indicating absolute or relative.
`INT4 NasGenSpecNodeGroup`	ID of group containing node IDs for spectrum generation.
NASTRAN Response Spectrum Application	
`BOOL NasAppSpecOn`	Flag indicating NASTRAN Response Spectrum Application Options have been specified.
`INT4 NasAppSpecType`	Spectrum type.
`INT4 NasAppSpecFunction`	Spectrum function ID.
`REAL8 NasAppSpecScale`	Scale factor to apply to spectrum.
`INT4 NasAppSpecMethod`	Modal combination method: 0=ABS, 1=SRSS, 2=NRL, 3=NRLO.
`REAL8 NasAppSpecClose`	Modal combination closeness.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasAppSpecSuport`	ID of constraint set to be used as a Base/SUPORT.
`INT4 NasAppSpecDamp`	Function ID to be used for modal damping.
NASTRAN Optimization Options	
`BOOL NasOptimOn`	Flag indicating NASTRAN Optimization Options have been specified.
`BOOL NasOptimIsTopology`	Flag indicating Topology optimization (True) or Design optimization (False).
`INT4 NasOptimAnalysisType`	Sets analysis type for design optimization. 1 = Linear Statics, 2 = Normal Modes/Eigenvalue.
`BOOL NasOptimTrackModes`	Enables mode shape tracking for normal modes design optimization.
`INT4 NasOptimGoal`	Design objective. Use enum zOptGoal: User-defined goal = `FOG_DRESP(0)` Weight = `FOG_WEIGHT(1)` Volume = `FOG_VOLUME(2)` Compliance = `FOG_COMPLIANCE(3)`
`INT4 NasOptimCycles`	Cycles value (Max number of design cycles).
`BOOL NasOptimMinDesobj`	Flag to use Minimize Design Objective (True) or Maximum (False).
`INT4 NasOptimDresp`	Design objective response ID if `NasOptimGoal = 0 (FOG_DRESP)`.
`BOOL NasOptimInterval`	Flag to use **1..Interval** option (True) or not (False).
`INT4 NasOptimIntervalVal`	Value for optimization output interval
`BOOL NasOptimEcho`	Flag to Echo a punch file of the optimized mode (True) or not Echo (False). This cannot be accessed with the GUI.
`BOOL NasOptimMCDelay`	Flag to use Delay until cycle and specified value for manufacturing constraints.
`INT4 NasOptimMCDelayVal`	First design cycle to apply manufacturing constraints.
NASTRAN GEOMCHECK Options	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasGCheckOn`	Flag indicating Geomtery checking is specified.
`BOOL NasGCheckNone`	Flag to write GEOMCHECK, NONE.

`BOOL NasGCheckTest[21]`
 	
or
 	
`VARIANT vNasGCheckTest`
	Flag indicating type of Geomtery checking specified. `[0]=Q4_SKEW`, `[1]=Q4_TAPER`, `[2]=Q4_WARP`, `[3]=Q4_IAMIN`, `[4]=Q4_IAMAX`, `[5]=T3_SKEW`, `[6]=T3_IAMAX`, `[7]=TET_AR`, `[8]=TET_EPLR`, `[9]=TET_DETJ`, `[10]=TET_DETG`, `[11]=HEX_AR`, `[12]=HEX_EPLR`, `[13]=HEX_DETJ`, `[14]=HEX_WARP`, `[15]=PEN_AR`, `[16]=PEN_EPLR`, `[17]=PEN_DETJ`, `[18]=PEN_WARP`, `[19]=BEAM_OFF`, `[20]=BAR_OFF`.
	
	Used in conjunction with `NasGCheckTol` and `NasGCheckMsg`.

`BOOL NasGCheckTest2[3]`
 	
or
 	
`VARIANT vNasGCheckTest2`
	Flag indicating type of Geomtery checking specified. `[0]=PYR_AR`, `[1]=PYR_EPLR`, `[2]=PYR_WARP`, `[3]=PYR_DETJ`.
	
	Used in conjunction with `NasGCheckTol2` and `NasGCheckMsg2`.

`REAL8 NasGCheckTol[21]`
 	
or
 	
`VARIANT vNasGCheckTol`
	Number indicating tolerance value for each `GEOMCHEC`. Used in conjunction with `NasGCheckTest` and `NasGCheckMsg`.

`REAL8 NasGCheckTol2[3]`
 	
or
 	
`VARIANT vNasGCheckTol2`
	Number indicating tolerance value for each `GEOMCHECK`. Used in conjunction with `NasGCheckTest2` and `NasGCheckMsg2`.

`INT4 NasGCheckMsg[21]`	0=Fatal, 1=Inform, 2=Warn. Used in conjunction with `NasGCheckTest` and `NasGCheckTol`.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vNasGCheckMsg`	

`INT4 NasGCheckMsg2[3]`
 	
or
 	
`VARIANT vNasGCheckMsg2`
	0=Fatal, 1=Inform, 2=Warn. Used in conjunction with `NasGCheckTest2` and `NasGCheckTol2`.

`BOOL NasGCheckSummary`	Flag to request Geomtery Check summary.
`INT4 NasGCheckMsgLimit`	Number indicating max number of messages.
NASTRAN MODELCHECK Options	
`BOOL NasMCheckOn`	Flag indicating NASTRAN Model Check options have been specified.

`BOOL NasMCheckWtDOFSet[6]`
 	
or
 	
`VARIANT vNasMCheckWtDOFSet`
	Flag indicating selected Model Checks. `[0]=G`, `[1]=N`, `[2]=N+AUTOSPC`, `[3]=F`, `[4]=A`, `[5]=V`.

`BOOL NasMCheckWtCG`	Flag to select CGI option.
`INT4 NasMCheckWtRefNode`	ID of Model Check Ref Node.
`INT4 NasMCheckWtUnits`	Number indicating Units to be used. 0=MASS, 1=WEIGHT.
`INT4 NasMCheckGrndRefNode`	ID of Ground Check Ref Node.

`BOOL NasMCheckGrndDOFSet[5]`
 	
or
 	
`VARIANT vNasMCheckGrndDOFSet`
	Flag indicating selected Ground Checks. `[0]=G`, `[1]=N`, `[2]=N+AUTOSPC`, `[3]=F`, `[4]=A`.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMCheckMaxStrnEn`	Number indicating Max Strain Energy.
`BOOL NasMCheckDataRec`	Flag to use DATAREC.
`REAL8 NasMCheckDataTol`	Tolerance Number to print forces above.
Mass and Damping Options (SOL 401)	
`BOOL NasMsnlModalMassForm`	Mass matrix formulation: 0=Default, 1=Lumped, 2=Coupled.
NASTRAN Multi-Step Control Options (SOL 401)	
`BOOL NasMsnlCntOn`	Flag indicating Multi-Step Control Options for Multi-Step Structural have been defined for master case.
NASTRAN Multi-Step Control Options (SOL 401) - General options	
`BOOL NasMsnlCntSkipNLCNTL`	Flag to deactivate writing of the `NLCNTL` for master case.
`BOOL NasMsnlCnt2LSTCONV`	TBD
NASTRAN Multi-Step Control Options (SOL 401) - Time Step Control	
`BOOL NasMsnlCntAutotim`	Flag to use **Auto Time Step (AUTOTIM)** option.
`REAL8 NasMsnlCntDtinit`	Initial Time Step (DMINIT) value.
`REAL8 NasMsnlCntDtmin`	Min Time Step (DMIN) value.
`REAL8 NasMsnlCntDtmax`	Max Time Step (DMAX) value.
`BOOL NasMsnlCntDtsbcdt`	Flag to specify New (False) or Modify (True) for Subcase Time Step (DTSBCDT).
`REAL8 NasMsnlCntEqmfmin`	Equilibrium Min Factor (EQMFMIN) value.
`REAL8 NasMsnlCntEqmfmx`	Equilibrium Max Factor (EQMFMAX) value.
`INT4 NasMsnlCntTsccr`	Specifies Time Step Method option: 0=Constant, 1=Adaptive, Truncation Error, 2=Adaptive, Maximum Creep Increment to Elastic Strain Ratio, 3=Adaptive, Maximum Creep Increment, 12=Combination of 1 and 2, 13=Combination

Unpublished work. © 2025 Siemens

Property	Description
	of 1 and 3, 23=Combination of 2 and 3, 123=Combination of 1, 2 and 3.
`BOOL NasMsnlCntTsceq`	Flag to use Equilibrium Iteration Stepping (TSCEQ).
`BOOL NasMsnlCntLvar`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Loads (LVAR).
`BOOL NasMsnlCntTvar`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Temp Loads (TVAR).
`BOOL NasMsnlCntDtsbcdt`	Flag to activate stepping based on UMAT supplied time increments (TSCUMAT).
`REAL8 NasMsnlCntEqmfmin`	Minimum time step factor for UMAT stepping (UMFMIN) value.
`REAL8 NasMsnlCntEqmfmx`	Minimum time step factor for UMAT stepping (UMFMX) value.
`REAL8 NasMsnlCntRTOLB`	Sets value for **Incremental Rotation Limit in Iteration (RTOLB)** Time Control strategy parameter (real numbers > 0.0, default 20)
NASTRAN Multi-Step Control Options (SOL 401) - Other Options	
`BOOL NasMsnlCntPlastic`	Flag to activate **Include Plasticity** option.
`BOOL NasMsnlCntPLSHUT`	Flag to activate **Hydrostatic Pressure Plasticity** option.
`BOOL NasMsnlCntCreep`	Flag to activate **Include Creep** option.
NASTRAN Multi-Step Control Options (SOL 401) - Case - Solution and Convergence	
Solution and Convergence - Solution Parameters	
`INT4 NasMsnlCntLOADOFF`	Deactivate Mechanical Loads (LOADOFF) option: 0=No Loads Turned Off 1=Body Loads Turned Off 2=Boundary Loads Turned Off 3=Both Boundary and Body Loads Turned Off
`INT4 NasMsnlCntSolver`	Specifies Solver (SOLVER) option: 0=SPARSE, 1=PARDISO, 2=MUMPS, 3=ELEMITER
`INT4 NasMsnlCntRFVAR`	Specifies Rot. Load Inertia Scaling (RFVAR) option: 0=Scale Angular Velocity and Acceleration

Unpublished work. © 2025 Siemens

Property	Description
	1=Scale Force
`BOOL NasMsnlCntINACCN`	Flag to use Compute Initial Acceleration in Dynamics (INACCN).
`BOOL NasMsnlCntINERTIA`	Flag to use Enable Inertia in Dynamics (INERTIA).
`INT4 NasMsnlCntMsglvl`	Diagnostic level (Integer = 0, 1, or 2; Default = 1). 0: No additional diagnostic output, 1: Convergence information is output for each iteration, 2: Convergence information is output for each iteration plus the location and value of the maximum translational displacement. In addition, the solution accuracy indicators table is output.
`BOOL NasMsnlCntThrmst`	Flag to use Diagnostic Level (MSGLVL).
`BOOL NasMsnlCntLSTCONV`	When on, Femap always outputs results at the converged last step, even when you did not request output for that step. When off, Femap only outputs results at the converged last step when you requested output for that step on the `TSTEP1` bulk entry.
Solution and Convergence - Stiffness Parameters	
`INT4 NasMsnlCntKupdate`	0=Auto Stiffness Update, 1=Full Newton-Raphson, 2=Quasi Newton-Raphson.
`INT4 NasMsnlCntQuasiNewtonIter`	Quasi Newton-Rhapson Iterations. Must be a value greater than or equal to 2. Only used when NasMsnlCntKupdate is set to 2=Quasi Newton-Raphson.
`INT4 NasMsnlCntStfoptn`	1=Elastic Stiffness Matrix, 2=Tangent Stiffness Matrix, 3=Elastic/Tangent Stiffness Matrix.
`BOOL NasMsnlCntFollowk`	Flag to use Follow Stiffness (FOLLOWK).
`BOOL NasMsnlCntSpink`	Flag to use Spin Softening (SPINK).
`BOOL NasMsnlCntStressk`	Flag to use Stress Stiffening (STRESSK).
`BOOL NasMsnlCntTstepk`	Flag to use Follow Stiffness (TSTEPK).
`BOOL NasMsnlCntMSTAB`	Flag to use Enable Stiffness Matrix Stabilization (MSTAB).

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMsnlCntMSFAC`	Value for Matrix Stabilization Factor (MSFAC) parameter.
`BOOL NasMsnlCntSTKFRST`	Flag to include Stress Stiffening at the first time step. If "Yes", the **Include Stress Stiffeneing at First Iteration (STKFRST)** Stiffness Parameter is activated.
Solution and Convergence - Convergence Parameters	
`VARIENT vNasMsnlCntConv_flags`	Flags to use convergence option and specified value: [0]=Displacement (EPSU) [1]=Load (EPSP) [2]=Work (EPSW)
`VARIENT vNasMsnlCntConv_value`	Values for use corresponding convergence option: [0]=Displacement (EPSU) [1]=Load (EPSP) [2]=Work (EPSW)
`INT4 NasMsnlCntMaxbis`	Max Bisection (MAXBIS) value.
`INT4 NasMsnlCntMaxdiv`	Max Divergences (MAXDIV) value.
`INT4 NasMsnlCntMaxiter`	Max Iterations/Step (MAXITER) values.
`INT4 NasMsnlCntMaxqn`	Max Quasi Newton Vectors (MAXQN).
`INT4 NasMsnlCntNORMP`	Specifies Norm Criteria for Force (NORMP) option: 0=Norm 1 Criteria, 1.=Norm 2 Criteria, 2.=Norm Infinity Criteria.
`INT4 NasMsnlCntNORMU`	Specifies Norm Criteria for Displacement (NORMU) option: 0=Norm 1 Criteria, 1.=Norm 2 Criteria, 2.=Norm Infinity Criteria.
`REAL8 NasMsnlCntREFP`	Value for Force Error Function Denominator (REFP).
`REAL8 NasMsnlCntREFU`	Value for Force Error Function Denominator (REFU).
Solution and Convergence - Time Integration	

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasMsnlCntTINTMTH`	Specifies Integration Scheme. 1=Newmark, 2=Hilber-Hughes-Taylor, 3=Generalized alpha method, 4=Modified generalized alpha method.
`REAL8 NasMsnlCntBETA`	Value for BETA time integration parameter.
`REAL8 NasMsnlCntGAMA`	Value for GAMA time integration parameter.
`REAL8 NasMsnlCntALFA`	Value for ALFA time integration parameter.
`REAL NasMsnlCntTETA`	Value for TETA time integration parameter.
`REAL NasMsnlCntRHOINF`	Value for RHOINF time integration parameter.
Solution and Convergence - Result Storage Control	
`INT4 NasMsnlkCnt2IA16`	Storage cycle for grid point results.
`INT4 NasMsnlkCnt2IA19`	Storage cycle for XY plotting results requested by `OUTMGT` bulk entries.
NASTRAN Multi-Step Control Options (SOL 401) - Contact/Bolt Preload	
Contact/Bolt Preload - Contact	
`INT4 NasMsnlCntCntmdiv`	Num Divergence for Bisection (CNTMDIV) value.
`REAL8 NasMsnlCntFsymtol`	Frict Threshold (FSYMTOL) value.
`INT4 NasMsnlCntKsym`	Unsymmetric Matrix option (KSYM) option:0=AUTO, 1=SYM, 2=UNSYM.
`REAL8 NasMsnlCntKsymtol`	Symmetrizing Tol (KSYMTOL) value.
`INT4 NasMsnlCntMsglvlc`	Contact Diagnostics (MSGLVLC) option: 0=None, 1=Convergence, 2=Convergence and Contact Pair.
`INT4 NasMsnlCntUsolver`	Unsymmetric Solver (USOLVER) option: 0=HYBRID, 1=SPARSE, 2=PARDISO, 3=MUMPS.
`INT4 NasMsnlCntKMODTN`	Tangential contact stiffness (`KMODTN`) option: 1=With Friction - applies a uniform tangential contact stiffness of (0.1 * normal stiffness) in a modal subcase, 2=With Friction - applies the actual final tangential contact stiffness from the end of a nonlinear subcase in a modal subcase,

Unpublished work. © 2025 Siemens

Property	Description
	3=With Friction - applies the actual final tangential contact stiffness from the end of a nonlinear subcase in a modal subcase, 4=Both Cases - applies a uniform tangential contact stiffness of (0.1 * normal stiffness) in a modal subcase, 5=Both Cases - replaces the final stiffness of any active contact element at the end of the nonlinear subcase with a no-separation, tied-contact stiffness in both the normal and tangential directions, regardless of the friction value.
`REAL8 NasMsnlCntKMODSCL`	Scale factor for the computed normal contact stiffness for the modal subcase (Real > 0.0; Default = 1.0).
Contact/Bolt Preload - Bolt/Preload	
`REAL8 NasMsnlCntEpsbolt`	Nonzero Preload Convergence Tol (EPSBOLT) value.
`REAL8 NasMsnlCntZerbolt`	Zero Preload Tol (ZERBOLT) value.
`INT4 NasMsnlCntItrbolt`	Max Iterations for Convergence (ITRBOLT) value.
`REAL8 NasMsnlCntMisfblt`	Bolt Strain Limit (MISFBLT) valu.e
`INT4 NasMsnlCntMsglvlb`	Diagnostics (MSGLVLB) option: 0=Summary at Convergence, 1=Summary Each Iteration, 2=Summary and Load Tol Each Iteration.
NASTRAN Multi-Step Control Options (SOL 401) - Creep Options	
`REAL8 NasMsnlCntCrcerat`	Max Creep Ratio (CRCERAT) value.
`REAL8 NasMsnlCntCrcinc`	Max Creep Inc (CRCINC) value.
`REAL8 NasMsnlCntCricoff`	Creep Strain Inc (CRICOFF) value.
`REAL8 NasMsnlCntCrinfac`	Integration Factor (CRINFAC) value.
`REAL8 NasMsnlCntCrmfmn`	Min Step Factor (CRMFMN) value.
`REAL8 NasMsnlCntCrmfmx`	Max Step Factor (CRMFMX) value.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMsnlCntCrteabs`	Max Truncation Error (CRTEABS) value.
`REAL8 NasMsnlCntdCrteco`	Time Step for Next Truncation Step (CRTECO) value.
`REAL8 NasMsnlCntCrterel`	Max Relative Truncation Error (CRTEREL) value.
`REAL8 NasMsnlCntCRLIMR`	Integration Error (`CRLIMR`) used for adjusting the creep strain increment (Real ≥ 0.0; Default = 0.0).
`REAL8 NasMsnlCntPLLIM`	Maximum equivalent plastic strain increment allowed within each time step (Real ≥ 0.0; Default=0.0, which turns off the capability).
`REAL8 NasMsnlCntPLLIMF`	Multiplying factor (`PLLIMF`) of maximum equivalent plastic strain (`PLLIM`) when adjusting the time step increment. (Real > 0.0; Default=1.0)
NASTRAN Multi-Step Strategy Parameters - Global Parameters	
`BOOL NasMsNLKGlobalOn`	Flag indicating Multi-Step Global Control Options for Multi-Step Nonlinear Kinematic have been defined.
`INT4 NasMsNLKGlobalRESO`	Specifies Solver (RESO) option (SOL 402 Only): 0=Auto 1=Skyline 2=Sparse 3=Parallel (SOL 402 Only)
`INT4 NasMsNLKGlobalSTRMEAS`	Specifies Stress-Strain Measure for Plastic Matl. (STRMEAS) option (SOL 402 Only): 0=Green Strain, PK2 Stress 1=Log Strain, Cauchy Stress 2=Biot Strain, Biot Stress 3=Log Strain, Kirchhoff Stress
`INT4 NasMsNLKGlobalSTROUT`	Specifies Stress-Strain Measure for Output (STROUT) option: 0=Green Strain, PK2 Stress 1=Log Strain, Cauchy Stress 2=Biot Strain, Biot Stress 3=Log Strain, Kirchhoff Stress
`INT4 NasMsNLKGlobalSTRCONV`	Specifies Stress-Strain Conversion Method (STRCONV) option:

Unpublished work. © 2025 Siemens

Property	Description
	0=Exact Method 1=Standard Method
`INT4 NasMsNLKGlobalIREF`	Specifies Computation (IREF) option (SOL 402 Only): 0=No initial static or steady-state computation 1=Initial static computation only 2=Steady-state computation only 3=Initial Static and steady-state computation
`INT4 NasMsNLKGlobalRESXTRP`	Specifies Solid Elem. Output Strategy (RESXTRP) option: 0=Default(SOL 401 Only): 1=Deactivate when hydrostatic pressure changes sign. 2=Retain when hydrostatic pressure changes sign.
`INT4 NasMsNLKGlobalBOLTMETH`	Specifies Bolt Preload Method (BOLTMETH) option (SOL 402 Only): 0=Default 1=RBE3 Elements and Sensor along the Cut 2=Contact Elements along the Cut
`INT4 NasMsNLKGlobalPLSHSOL`	Specifies Plasticity Computation (PLSHSOL) option: 0=Default 1=Deactivate when hydrostatic pressure changes sign. 2=Retain when hydrostatic pressure changes sign.
`BOOL NasMsNLKGlobalINLY`	Flag to use Unsymmetric Solver (INLY) option (SOL 402 Only).
`BOOL NasMsNLKGlobalKINI`	Specifies the **Check Initial Stiffness** option. TRUE = The checks are performed. FALSE = The checks are not performed.
`INT4 NasMsNLKGlobalFHMETH`	Specifies the Multiharmonic Options (computation options). 1 = The zero, positive, and negative harmonics are computed and are fully coupled.

Unpublished work. © 2025 Siemens

Property	Description
	2 = The zero, positive, and negative harmonics are computed and are uncoupled. 3 = The zero, positive, and negative harmonics are computed, and positive and negative pairs are coupled. 11 = The zero and positive harmonics are computed and are coupled. 12 = The zero and positive harmonics are computed and are uncoupled. 13 = Same as FHMETH=12.
NASTRAN Multi-Step Strategy Parameters - Advanced Parameters (SOL 402 Only)	
`REAL8 NasMsNLKGlobalPRECPIVO`	Threshold for Zero Pivots (PRECPIVO) value.
`INT4 NasMsNLKGlobalNKINE`	Modes to Save as Output (NKINE).
`INT4 NasMsNLKGlobalMODEVERS`	Specifies Version for Param. Defaults (MODEVERS) option: 0=Default, 1=2019.1, 2=2019.2
NASTRAN Multi-Step Strategy Parameters - Restart Parameters (Only Available when `NasExecRestart` is turned on)	
`BOOL NasMsNLKGlobalRSTGEN`	Flag to use Save Restart Data (RSTGEN) option.
`INT4 NasMsNLKGlobalRSTUNIT`	Restart File Unit Number (RSTUNIT).
`INT4 NasMsNLKGlobalRSTFROM`	Subcase to Restart From (RSTFROM).
`INT4 NasMsNLKGlobalEXEFROM`	Subcase to Execute From (EXEFROM).
`BOOL NasMsNLKGlobalMDLVAL`	Flag to use Disable Model Validation for Restart Run (MDLVAL) option.
NASTRAN Multi-Step Control Options (SOL 402)	
`BOOL NasMsnlCnt2On`	Flag indicating Multi-Step Control Options for Multi-Step Nonlinear Kinematic have been defined.
`INT4 NasMsnlkCnt2ISTO`	Specifies the **Stop Run Ooption (ISTO)**. 2 = Stop the run after estimating how much space to reserve in the central memory and on the disk. (2 is the default when a METHOD case control command is not present.)

Unpublished work. © 2025 Siemens

Property	Description
	3 = Stop the run after a complete or partial calculation of the dynamic response. (3 is the default when a METHOD case control command is present.)"
`INT4 NasMsnlkCnt2BALF`	Specifies **Solution Frequency Option (BALF)**, 0 = Calculates the harmonic response at the solution frequencies. 1 = Calculate2 the harmonic response at the solution frequencies and modal frequencies that lie within the range of solution frequencies."
`INT4 NasMsnlkCnt2IA4`	Specifies the **Storage Cycle for Element Stress (IA4)** option.
`INT4 NasMsnlkCnt2IA12`	Specifies the **Complex Eigenvalue Frequency Step** calculation controls.
NASTRAN Multi-Step Control Options (SOL 402) - General options	
`BOOL NasMsnlCnt2SkipNLCNTL2`	Flag to deactivate writing of the `NLCNTL2` for this case.
NASTRAN Multi-Step Control Options (SOL 402) - Time Step	

`BOOL NasMsnlkCnt2CIBL`
 	
or
 	
`BOOL NasMsnlkCnt2AUTOTIM`
	Flag to use Auto Time Stepping (CIBL) option (version 12), Auto Time Stepping (AUTOTIM) option (Femap 2019.1 and above).

`REAL8 NasMsnlkCnt2DTI0`
 	
or
 	
`REAL8 NasMsnlkCnt2DTINIT`
	First Time Step (DTI0) value (version 12), First Time Step (DTINIT) value (Femap 2019.1 and above).

`REAL8 NasMsnlkCnt2HMIN`
 	
or
 	
`REAL8 NasMsnlkCnt2DTMIN`
	Minimum Time Step (HMIN) value (version 12), Minimum Time Step (DTMIN) value (Femap 2019.1 and above).

Unpublished work. © 2025 Siemens

Property	Description

`REAL8 NasMsnlkCnt2HMAX`
 	
or
 	
`REAL8 NasMsnlkCnt2DTMAX`
	Maximum Time Step (HMAX) value (version 12), Maximum Time Step (DTMAX) value (Femap 2019.1 and above).

`REAL8 NasMsnlkCnt2RUP`
 	
or
 	
`REAL8 NasMsnlkCnt2EQMFMX`
	Maximum Increase Ratio (RUP) value (version 12), Maximum Increase Ratio (EQMFMX) value (Femap 2019.1 and above).

`REAL8 NasMsnlkCnt2RDOW`
 	
or
 	
`REAL8 NasMsnlkCnt2EQMFMIN`
	Maximum Decrease Ratio (RDOW) value (version 12), Maximum Decrease Ratio (EQMFMIN) value (Femap 2019.1 and above).

`INT4 NasMsnlkCnt2ITEREF`	Number of Iterations (ITEREF) value.
`INT4 NasMsnlkCnt2RJPN`	Specifies Negative Pivot Time Step Rejection (RJPN) option: 0=Never 1=Last Iteration Only 2=Print State At Last Iteration 3=Always
`BOOL NasMsnlkCnt2RJPZ`	Zero Pivot Time Step Rejection (RJPZ) value.
`BOOL NasMsnlkCnt2TSDYN`	Integration Error Control (TSDYN) value.
`BOOL NasMsnlkCnt2ERCD`	Flag to use Activate Time Stepping Based on Disp/Temp Error.
`REAL8 NasMsnlkCnt2PRED`	Value for Allowable Change/Step (PRED).
NASTRAN Strategy Parameters (SOL 402) - Viscous Material Options	
`BOOL NasMsnlkCnt2TSVSC`	Flag to use Integration Error Control (TSVSC) option.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMsnlkCnt2VSCOTE`	Error Threshold (VSCOTE).
`INT4 NasMsnlkCnt2VSCOSN`	Min Stress Factor (VSCOSN).
NASTRAN Strategy Parameters (SOL 402) - Analysis Control	
`REAL8 NasMsnlkCnt2DISLIM`	Maximum Displacement (DISLIM) value.
`REAL8 NasMsnlkCnt2ROTLIM`	Maximum Rotation (ROTLIM) value.
`REAL8 NasMsnlkCnt2DEFLIM`	Maximum Deformation (DEFLIM) value.
`BOOL NasMsnlkCnt2LVAR`	Flag to specify “0..Ramp” (False) or “1..Step” (True) for Time Unassigned Loads (LVAR).
`BOOL NasMsnlkCnt2TVAR`	Flag to specify **0..Ramp** (False) or **1..Step** (True) for Time Unassigned Temp Loads (TVAR).
`BOOL NasMsnlkCnt2DIPR`	Flag to use Predict Displacement (DIPR) option.
`BOOL NasMsnlkCnt2NORM`	Flag to use Force Normalization (NORM) option.
`BOOL NasMsnlkCnt2MATSYM`	Flag to use Unsymmetrical Matrices for Complex Modes (MATSYM).
`INT4 NasMsnlkCnt2INERTIA`	Flag to use Enable Inertia in Dynamics (INERTIA).
`BOOL NasMsnlkCnt2IAR0`	Flag to use Save Data at Beginning of Computation (IAR0).
`INT4 NasMsnlkCnt2ITHE`	Manage thermal loads to improve convergence (-1 = Free thermal expansion, 0 = Constrained thermal expansion). Default = 0.
`INT4 NasMsnlkCnt2LL2`	Sets the space required to store the laws of excitation. If the value is too small, a warning is issued, and the run is interrupted. Allowable values: Integers greater than 0. Default = 20000.
`INT4 NasMsnlkCnt2RFVAR`	Changes how Femap interpolates time variation when interpolating rotational inertia loads (see RFVAR).
NASTRAN Strategy Parameters (SOL 402) - Plasticity and Creep Control	
`BOOL NasMsnlkCnt2PLASTIC`	Flag to use Enable Plasticity option.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasMsnlkCnt2PLSHUT`	Flag to use Hydrostatic Pressure Plasticity option.
`BOOL NasMsnlkCnt2CREEP`	Flag to use Enable Creep option.
`REAL8 NasMsnlkCnt2CRINFAC`	Creep Integration Factor (CRINFAC) value.
`REAL8 NasMsnlkCnt2PLLIM`	Maximum equivalent plastic strain increment allowed within each time step (`PLLIM`). Real ≥ 0.0; Default=0.0, which turns off the capability.
NASTRAN Strategy Parameters (SOL 402) - Internal Restart	
`INT4 NasMsnlkCnt2RSUB`	Specifies if the Restart (RSUB) should be activated and the value to use for Restart Subcase. If value is set to 0, the Restart (RSUB) option will be deactivated. If value is set to a value above 0, the Restart (RSUB) option will be activated and the value specified will be used as the value for Restart Subcase.
`REAL8 NasMsnlCnt2CRLIMR`	
`REAL8 NasMsnlCnt2PLLIM`	Maximum equivalent plastic strain increment allowed within each time step. When greater than 0.0, the solution can adjust the time step increment. Allowable values: Real numbers greater than or equal to 0.0. Default=0.0, which deactivates the capability.
`REAL8 NasMsnlCnt2PLLIMF`	Multiplying factor of `PLLIM` when adjusting the time step increment.
`REAL8 NasMsnlCnt2CRICOFF`	Maximum equivalent plastic strain increment allowed within each time step. When greater than 0.0, the solution can adjust the time step increment. Allowable values: Real numbers greater than or equal to 0.0. Default=0.0, which deactivates the capability.
`REAL8 NasMsnlkCnt2CRLIMR`	Time stepping method for creep analysis. Adjusts the integration error used for adjusting the creep strain increment. (Real ≥ 0.0; Default = 0.0)
NASTRAN Strategy Parameters (SOL 402) - Solution and Convergence	
Solution and Convergence - Analysis Options	

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMsnlkCnt2STAB`	Stabilization Factor (STAB) value.
`REAL8 NasMsnlkCnt2MADI`	Minimum Disp Variation (MADI) value.
Solution and Convergence - Equilibrium and Convergence	
`INT4 NasMsnlkCnt2ILNS`	Line Search (ILNS) value.
`REAL8 NasMsnlkCnt2PRLN`	Line Search Convergence Tol (PRLN) value.
`REAL8 NasMsnlkCnt2AMIN`	Min Line Search Factor (AMIN) value.
`REAL8 NasMsnlkCnt2AMAX`	Max Line Search Convergence Force (AMAX) value.
`INT4 NasMsnlkCnt2ITMA`	Max Iter (ITMA) value.
`REAL8 NasMsnlkCnt2PRCR`	Relative Force Tol (PRCR) value.
`REAL8 NasMsnlkCnt2REFP`	Relative Force (REFP) value.
`REAL8 NasMsnlkCnt2PRCQ`	Relative Disp Force (PRCQ) value.
`REAL8 NasMsnlkCnt2REFU`	Reference Disp (REFU) value.
`REAL8 NasMsnlkCnt2PRCE`	Rel Energy Tol (PRCE) value.
`REAL8 NasMsnlkCnt2REFE`	Reference Energy (REFE) value.
`INT4 NasMsnlkCnt2IT1K`	Stiffness Update Param 1 (IT1K) value.
`INT4 NasMsnlkCnt2IT2K`	Stiffness Update Param 2 (IT2K) value.
`INT4 NasMsnlkCnt2IT3K`	Stiffness Update Param 3 (IT3K) value.
`INT4 NasMsnlkCnt2PLAS`	Specifies First Iteration Strategy (PLAS) option: -1=Elastic tangent matrix, plastic stress variation -2=Elastic tangent matrix, elastic stress variation -3=Plastic tangent matrix, elastic stress variation 1=Plastic tangent matrix, plastic stress variation

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasMsnlkCnt2OTRE`	Specifies Rigid Body Motion (OTRE) option: 0=Disable Convergence 1=Enable based on PRCE, PRCQ, and PRCR criteria 2=Enable based on PRCE, PRCQ, PRCR, and REFP
`INT4 NasMsnlkCnt2MAXBIS`	Maximum number of times Femap can reduce a time step consecutively. By default, equal to 0, no limit.
`INT4 NasMsnlkCnt2KUPDATE`	Stiffness update strategy. (Integer; Default = 0).
`REAL8 NasMsnlkCnt2PRDR`	Specifies relaxed force tolerance (PRDR).
`REAL8 NasMsnlkCnt2PRDE`	Specifies relaxed energy tolerance (PRDE).
Solution and Convergence - Time Integration	

`INT4 NasMsnlkCnt2IMPL`
 	
or
 	
`INT4 NasMsnlkCnt2TINTMTH`
	Specifies Integration Scheme (TINTMTH) option (Femap 2019.1 and above):
	
	1=Newmark
	2=Hilber-Hughes-Taylor
	3=Generalized Midpoint
	4=Generalized Alpha
	
	Specifies Integration Scheme (IMPL) option:
	
	1=NewMark implicit predictor-corrector, 2=Newmark implicit predictor-corrector with automatic time stepping, 21=Hilber-Hughes-Taylor (HHT) implicit predictor-corrector scheme. 22=Hilber-Hughes-Taylor (HHT) implicit predictor-corrector with automatic time stepping, 31=Generalized midpoint, 32=Generalized midpoint with automatic time stepping, 61=Generalized Alpha, 62=Generalized Alpha with automatic time stepping.

`REAL8 NasMsnlkCnt2BETA`	Param1 (BETA) value.
`REAL8 NasMsnlkCnt2GAMA`	Param2 (GAMA) value.
`REAL8 NasMsnlkCnt2ALFA`	HHT Scheme Param (ALFA) value.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMsnlkCnt2TETA`	Generalized Scheme Param (TETA) value.
`REAL8 NasMsnlkCnt2PRCO`	Intg Control Param 1 (PRCO) value.
`REAL8 NasMsnlkCnt2HPRCO`	Intg Control Param 2 (HPRCO) value.
`REAL8 NasMsnlkCnt2RHOINF`	Value for RHOINF time integration parameter.
Solution and Convergence - Contact	
`REAL8 NasMsnlkCnt2RELC`	Value for Relaxation Force (RELC).
`REAL8 NasMsnlkCnt2DCON`	Value for Characteristic Length (DCON).
`REAL8 NasMsnlkCnt2PRCS`	Value for Ing Regularization Factor (PRCS).
`REAL8 NasMsnlkCnt2PRCF`	Value for Threshold for Contact Force Variation (PRCF).
`INT4 NasMsnlCnt2KMODTN`	tangential contact stiffness for the modal subcase. Useful, for example, when the nonlinear subcase requires frictionless contact.
`REAL8 NasMsnlCnt2KMODSCL`	Scales normal contact stiffness for the modal subcase.
`INT4 NasMsnlkCnt2KMODTN`	Independently control the tangential contact stiffness of the contact pairs with friction, and without friction (Integer; Default = 1).
`INT4 NasMsnlkCnt2MSGLVLC`	Diagnostic output for contact. (Integer = 0, 1 or 2; Default = 1) 0: No additional diagnostic output. 1: Convergence information is listed for each iteration. Changes in contact status are also listed. Before the first iteration, a summary of the number of sliding/sticking/active/inactive elements is listed. The number of augmentations and number of pair updates is also listed. 2: Everything described in option 1 and after each iteration, detailed information for every contact pair is listed.
Solution and Convergence - Diagnostic	

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasMsnlkCnt2IMPG`	Print Frequency (IMPG) value.
`INT4 NasMsnlkCnt2IMPR`	Max Residual Printout (IMPR) value.
`INT4 NasMsnlkCnt2IMPV`	Max Pivots (IMPV) value.
Joint Time Constraint (SOL402)	
`BOOL NasMsnlkJCON`	If True, the Activate JCON option is turned on, which causes JCONSET* case control and JCON bulk data entries to be written.
NASTRAN Multi-Step Time Steps (SOL 401 and SOL 402)	
`BOOL NasMsnlTSOn`	Flag indicating Multi-Step Time Steps for Multi-Step Structural or Multi-Step Nonlinear Kinematic have been defined for subcase.
`BOOL NasMsnlTSSkipTSTEP`	Flag to disable writing of the TSTEP for this case.
`REAL8 NasMsnlTSEndTime[0..9]`	Value in End Time Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc.
`INT4 NasMsnlTSNumIncrements[0..9]`	Value in Number of Increments Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc.
`INT4 NasMsnlTSOutputFreq[0..9]`	Value in Output Frequency Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc: 0=Requested Increments, 1=Solution Step End Time, 2=All Times, 3=Coupling Times, 4=Skip Factor.
`INT4 NasMsnlTSSkipFactor[0..9]`	Value in Number of Increments Column of row specified by zero-based index value in Time Steps dialog box. 0=Row 1, 1=Row 2, etc. Only used when `NasMsnlTSOutputFreq`=4.
NASTRAN NXSTRAT Options	
`BOOL NasNXStratOn`	Flag indicating NASTRAN Advanced Nonlinear options have been defined.
`INT4 NasNXStratSolver`	Solver type 0=Direct Sparse 1=Multigrid.
`INT4 NasNXStratAuto`	Auto Incrementation.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasNXStratNposit`	Continue when not positive definite.
`BOOL NasNXStratMasstyp`	Mass Matrix Type.
`INT4 NasNXStratTint`	Integration order thru Shell thick.
`BOOL NasNXStratMstab`	Matrix Stabilization.
`REAL8 NasNXStratMsfac`	Matrix Stabilization Factor.
`REAL8 NasNXStratDtdelay`	Element Death Time delay.
`REAL8 NasNXStratSdofang`	Shell DOF Angle.
`BOOL NasNXStratUpform`	u/p formulation.
`BOOL NasNXStratDispopt`	Prescribed displacement option.
`BOOL NasNXStratLoadopt`	Applied Load option.
`REAL8 NasNXStratAlpha`	Newmark Time Integ alpha coefficient.
`REAL8 NasNXStratDelta`	Newmark Time Integ delta coefficient.
`INT4 NasNXStratItemax`	Max iter for multigrid solver.
`REAL8 NasNXStratEpsia`	Convergence Tolerance epsia.
`REAL8 NasNXStratEpsib`	Convergence Tolerance epsib.
`REAL8 NasNXStratEpsii`	Convergence Tolerance epsii.
`INT4 NasNXStratLsearch`	Line Search Flag.
`INT4 NasNXStratMaxite`	Max Iter per time step.
`INT4 NasNXStratConvcri`	Convergence Criteria.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasNXStratPlasAlg`	Plasticity Algorithm.
`REAL8 NasNXStratEtol`	Relative Energy Tolerance.
`REAL8 NasNXStratRtol`	Relative Force Tolerance.
`REAL8 NasNXStratRnorm`	Reference Force.
`REAL8 NasNXStratRmnorm`	Reference Moment.
`REAL8 NasNXStratRctol`	Relative Contact Force Tolerance.
`REAL8 NasNXStratDtol`	Relative Displacement Tolerance.
`REAL8 NasNXStratDnorm`	Reference Translation.
`REAL8 NasNXStratDmnorm`	Reference Rotation.
`REAL8 NasNXStratStol`	Line Search Convergence Tolerance.
`REAL8 NasNXStratRconsm`	Reference Contact Force.
`REAL8 NasNXStratEnlsth`	Line Search Energy Threshold.
`INT4 NasNXStratAtssubd`	Auto Time Step Small Step Size.
`REAL8 NasNXStratAtsmxdt`	Auto Time Step Largest Step Size.
`INT4 NasNXStratAtsnext`	Auto Time Step Size Flag.
`REAL8 NasNXStratAtsdfac`	Division Factor for Sub Increment.
`BOOL NasNXStratAtslows`	Low Speed Dynamics Flag.
`REAL8 NasNXStratAtsdamp`	Low Speed Dynamics Damping Factor.
`INT4 NasNXStratLdcgrid`	Node ID for Prescribed Displacement.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasNXStratLdcdof`	DOF for Prescribed Displacement.
`REAL8 NasNXStratLdcdisp`	Magnitude of Prescribed Displacement.
`REAL8 NasNXStratLdcimax`	Displacement Convergence Factor.
`REAL8 NasNXStratLdcdmax`	Max Displacement.
`BOOL NasNXStratLdccont`	Solution Termination after Critical.
`INT4 NasNXStratLdcsubd`	Max Arc Length Subdivisions.
`INT4 NasNXStratImpact`	Impact Control Scheme.
`INT4 NasNXStratNsupp`	Num Iterations for Pairing.
`BOOL NasNXStratModex`	Restart Flag - Obsolete, use `NasNXStratModexNew`.
`INT4 NasNXStratModexNew`	Restart Flag/Option - 0=Normal, 1=Restart, 2=Recover Results.
`REAL8 NasNXStratAtsmass`	Value for Auto Time Stepping Inertia Factor.
`REAL8 NasNXStratTstart`	Restart Solution Time.
`INT4 NasNXStratIrint`	Frequency of Results in File.
`INT4 NasNXStratNsubgrp`	Number of Subgroups.
`BOOL NasNXStratElcv`	Conversion Flag.
`INT4 NasNXStratEqrbar`	RBAR element option.
`INT4 NasNXStratEqrbe2`	RBE2 element option.
`REAL8 NasNXStratSpringk`	Stiffness of RBAR/RBE2 springs.
`REAL8 NasNXStratBeame`	Young’s modulus for RBAR/RBE2 elements.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasNXStratBeama`	Effective cross section for RBAR/RBE2 elements.
`REAL8 NasNXStratRblcrit`	Critical Length for RBAR/RBE2 elements.
`INT4 NasNXStratXstep`	Time Step Method.
`INT4 NasNXStratXdtcal`	Critical time step recalculation every XDTCAL steps.
`REAL8 NasNXStratXdtfac`	Critical time step factor.
`REAL8 NasNXStratXmscale`	Mass scale factor.
`REAL8 NasNXStratXdtmin1`	Min steps size to use mass scaling.
`REAL8 NasNXStratXdtmin2`	Max steps size to use mass scaling.
`INT4 NasNXStratIcmode`	Incompatible mode flag 0=off 1=on.
`BOOL NasNXStratBeamalg`	When True, Use NXN v8.5 Elastic Beam Formulation option is activated.
`INT4 NasNXStratUlform`	Large strain formulation used.
`INT4 NasNXStratTimeSteps[0..9]`	Number of Steps.
`REAL8 NasNXStratTimeIncrement[0..9]`	Time increment.
`INT4 NasNXStratSkipFactor[0..9]`	Output results every Nth step.
`INT4 NasNXStratCstype`	Selects the type of contact segment to use.
`BOOL NasNXStratElrescs`	For NX Nastran Advanced Nonlinear solver, if True, results are in elemental coordinate system, if False, in the Material coordinate system.
`REAL8 NasNXStratMaxDisp`	Limit for max incremental displacement/ equilibrium iteration.
`INT4 NasNXStratBoltstp`	Number of steps for applying bolt pre-load force.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasNXStratCvssval`	Flag to convert TABLES1 from engineering stress strain to true stress strain.
`BOOL NasNXStratRtalg`	Flag selecting the rigid target method: 0=Current, 1=Old(v4).
`BOOL NasNXStratTnslcf`	Flag allowing tensile consistent contact forces on quadratic 3D contact segments.
`REAL8 NasNXStratDrillkf`	Maximum Rotational Stiffness Factor used when drilling stiffness is zero.
`REAL8 NasNXStratLslower`	Value for Lower bound of line search.
`REAL8 NasNXStratLsupper`	Value for Upper bound of line search.
`BOOL NasNXStratXtcurve`	Flag to extend TABLES1 by linear extrapolation.
NASTRAN DDAM Options	
`BOOL NasDdamOn`	Flag indicating Nastran DDAM Options have been specified.
`BOOL NasDdamEnabled`	If True, DDAM analysis is activated.
`BOOL NasDdamIncludePath`	If True, full paths are specified in all file names.
`BOOL NasDdamIsDDAM`	If True, this is a DDAM analysis, otherwise it is a non-DDAM spectrum analysis.
`INT4 NasDdamUseBuiltInCoeff`	0=Specify Coefficients, 1=External File, 2=Built-In.
`STRING NasDdamSpecCoeffFile[260]`	Filename (and path) of file containing spectrum coefficients (only used if `NasDdamUseBuiltInCoeff`=1).
`INT4 NasDdamFileUnit`	Specify value for Spectrum File Unit in Spectrum/ Coefficient Options section. Only used when DDAMCTR/SPDIR is selected and Specify Coefficients or Coefficients from External File is selected.
`INT4 NasDdamEquationType`	0=DDS-072, 1=NRL 1396 (Only used for MSC Nastran).
`INT4 NasDdamSurfaceShip`	0=Surface, 1=Submerged.
`INT4 NasDdamEquipmentLocation`	0=Deck, 1=Hull, 2=Shell.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasDdamElasticPlastic`	0=Elastic, 1=Elastic-Plastic.
`INT4 NasDdamForeAftDirection`	Fore/Aft Axis (0=X, 1=Y, 2=Z).
`INT4 NasDdamVerticalDirection`	Vertical Axis (0=X, 1=Y, 2=Z).
`INT4 NasDdamDirectionSequence`	Number specifying sequence of axes to apply (X=1,Y=2,Z=3), so 132 means X then Z, then Y.
`REAL8 NasDdamMassToWeight`	Mass to Weight conversion factor.
`REAL8 NasDdamWeightCutoffPct`	Modal Mass Cutoff Percentage.
`REAL8 NasDdamGCutoff`	Minimum Acceleration in Gs.

`BOOL NasDdamCoeffOn[14]`
or
`VARIANT vNasDdamCoeffOn`
	Flags to turn on/off specified spectrum coefficients.

`REAL8 NasDdamCoeffVal[14]`
or
`VARIANT vNasDdamCoeffVal`
	Values of specified spectrum coefficients.

`REAL8 NasDdamForceConversion`	Value for Force Unit Conversion.
`REAL8 NasDdamAccelConversion`	Value for Acceleration Unit Conversion.
`INT4 NasDdamSpdirSCSID`	Specify value for Output CSys in Excitation section.
`INT4 NasDdamSpdirCOMP`	Specify value for Components in Excitation section. Currently, only valid value is 123.
`INT4 NasDdamSelModeMethod`	Specifies Selection Method option: 0=None 1=CUTOFF 2=PERCENT

Unpublished work. © 2025 Siemens

Property	Description
	3=TOP
`REAL8 NasDdamSelModeVal`	Specify value for Value in Mode Selection section.
NASTRAN Rotor Dynamics Options	
`BOOL NasRotorOn`	Flag indicating Nastran Rotor Dynamics Options have been specified.
`BOOL NasRotorEnabled`	If True, Rotor Dynamics analysis is enabled.
`BOOL NasRotorIncludePath`	If True, full paths are specified in all filenames.
`BOOL NasRotorSingleRotor`	If True, this is a single rotor analysis of the full model, if False, multiple rotors must be specified with rotor regions.
`BOOL NasRotorFixedRefSys`	True = Fixed Reference CSys, False = Rotating.
`BOOL NasRotorAsyncResponse`	True = Asynchronous Response Calculations.
`BOOL NasRotorSteinerInertia`	True = Include Steiner Inertia Terms.
`BOOL NasRotorForceExcitation`	True = Force Excitation, False = Mass Unbalance.
`BOOL NasRotorDoCSV`	True = Create comma separated output file.
`REAL8 NasRotorWhirlThreshold`	Whirl Threshold.
`REAL8 NasRotorExcitationOrder`	Excitation Order (Default = 1.0, 0.0=Forward Whirl, 2.0=Backward Whirl).
`REAL8 NasRotorSpeedStart`	Rotor Speed Start Value.
`REAL8 NasRotorSpeedStep`	Rotor Speed Step Size.
`STRING NasRotorModeSelection[60]`	String specifying IDs of modes for dynamic response.
NASTRAN SOL 414 Rotor Dynamics	
`BOOL fhpost`	Flag to write PARAM,FHPOST.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasBulkFHPOSTVal`	Value to include on PARAM,FHPOST.
`BOOL NasBulkFHPNST`	Flag to write PARAM,FHPNST.
`INT4 NasBulkFHPNSTVal`	Value to include on PARAM,FHPNST
`BOOL NasBulkDFREQ`	Flag to write PARAM,DFREQ
`REAL8 NasBulkDFREQVal`	Value to include on PARAM,DFREQ
`BOOL NasBulkRDMTRAK`	Flag to write PARAM,RDMTRAK
`REAL8 NasBulkRDMTRAKVal`	Value to include on PARAM,RDMTRAK

`VARIANT vNasRotorRegionSelection`
or
`INT4 NasRotorRegionSelection`
	ID of rotor region to include on ROTORD

`VARIANT vNasRotorSpeedMultiplier`
or
`REAL8 NasRotorSpeedMultiplier`
	Value to include as rotor speed multiplier on ROTORD

`VARIANT vNasRotorSpeedMultiplierFn`
or
`INT4 NasRotorSpeedMultiplierFn`
	ID to include as rotor speed multipliers by Table ID on ROTORD

NASTRAN Aerodynamic Data Options - General (SOL 144, SOL 145, and SOL 146)	
`BOOL NasRotorDoGPF`	True = Create GPF output file.
`INT4 NasRotorInputUnits`	Speed Input Units 0=Rev/Min, 1=Cycle/Sec, 2=Hz, 3=Radians/Sec.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasRotorOutputUnits`	Output Units 0=Rev/Min, 1=Cycle/Sec, 2=Hz, 3=Radians/Sec.
`INT4 NasRotorPrintedOutput`	Printed Output 0=None, 1=Generalized Matrices, 2=Eigenvalue Summary/Eigenvectors, 3=Both.
`INT4 NasRotorSpeedNumSteps`	Number of Steps in Rotor Speed.
`REAL8 NasRotorComplexModeOutput`	Mode/Whirl Output (0.0=None, <0=All RPM, >0=Specified Speed).
`BOOL NasAeroOn`	Flag indicating NASTRAN Aerodynamic Data Options have been defined.
`INT4 NasAeroCsID`	Aerodynamic coordinate system.
`REAL8 NasAeroRefLength`	For Static Aeroelasticity (SOL 144), specifies Chord Length. For Flutter (SOL 145) and Aeroelastic Response (SOL 146), specifies Ref Length.
`INT4 NasAeroSymXY`	Symmetry flag for the aero coordinate x-y plane. -1 = Symmetry, 0 = No Symmetry, 1 = Anti Symmetry (also used in Flutter).
`INT4 NasAeroSymxz`	Symmetry flag for the aero coordinate z-x plane 1 = Symmetry, 0 = No Symmetry, -1 = Anti Symmetry (also used in Flutter).
NASTRAN Aerodynamic Data Options - Static Aeroelasticity (SOL 144 Only)	
`INT4 NasAeroRefCsID`	Reference coordinate system.
`REAL8 NasAeroRefSpan`	Reference span.
`REAL8 NasAeroRefArea`	Reference wing area.
`BOOL NasAeroAeunit`	PARAM, AUNITS flag.
`REAL8 NasAeroAeunitVal`	PARAM, AUNITS val.
NASTRAN Aerodynamic Data Options - Flutter (SOL 145) and Aeroelastic Response (SOL 146)	
`REAL8 NasAeroVelo`	Velocity for aerodynamic force.
`REAL8 NasAeroRefDens`	Reference density.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasAeroMkFuID`	Mach number/Reduced Frequency function.
`INT4 NasAeroModesKeep`	Dynamic Options: Number of Modes. When non-zero, writes PARAM,LMODES with specified value.
`REAL8 vNasAeroFreqKeep[0..1]`	Dynamic Options: Lowest Freq (Hz) and Highest Freq (Hz). [0] sets value to write with PARAM,LFREQ. [1] sets value to write with PARAM,HFREQ.
`BOOL NasAerobPARAMfzero`	Dynamic Options: Zero Modes Val (FZERO) flag. When True writes PARAM,FZERO with value specified for `NasAerodPARAMfzero`.
`REAL8 NasAerodPARAMfzero`	Dynamic Options: Zero Modes Val (FZERO) value.
`BOOL NasAeroDampMethod`	Dynamic Options: As Structural (KDMAP). When True, writes PARAM,KDAMP,-1.
`BOOL NasAerobOPPHIPA`	Dynamic Options: PARAM OPPHIPA. When True, writes PARAM,OPPHIPA,1.
NASTRAN Aerodynamic Data Options - Aeroelastic Response (SOL 146 Only)	
`REAL8 NasAerodPARAMmach`	Dynamic Options: Mach Number (MACH). When non-zero, writes PARAM,MACH with specified value.
`REAL8 NasAerodPARAMq`	Dynamic Options: Dynamic Pressure (Q). When non-zero, writes PARAM,Q with specified value.
`INT4 NasAeronPARAMgustaero`	Dynamic Options: Gust Load (GUSTAERO). When set to *-1..Compute*, writes PARAM,GUSTAERO,-1, while *1..Do Not Compute* writes PARAM,GUSTAERO,1.
Aeroelastic Trim Case Options - NASTRAN Specific (SOL 144 Only)	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasCaeOn`	Flag indicating NASTRAN Trim Options have been defined.
`REAL8 NasCaeMachNumber`	Mach number.
`REAL8 NasCaeDynPressure`	Dynamic pressure.
`BOOL NasCaeRigidTrim`	Flag to request a rigid trim analysis.
`BOOL NasCaeWrtieTrim`	Flag to enable writing of the TRIM for this case.
Aerodynamic Flutter Case Options - NASTRAN Specific (SOL 145 Only)	
`BOOL NasCflOn`	Flag indicating NASTRAN Flutter Options have been defined.
`INT4 NasCflMethod`	Flutter analysis method.
`INT4 NasCflDenID`	Femap function defining density ratios.
`INT4 NasCflMachFactID`	Femap function defining density ratios.
`INT4 NasCflRfreqFactID`	Femap function defining density ratios.
`INT4 NasCflFliMethod`	Interpolation method.
`INT4 NasCflEig`	Number Eignevalues.
`REAL8 NasCflEps`	Convergence.
`BOOL NasCflWriteFlutter`	Flag to enable writing of the FLUTTER for this case.
`INT4 NasCflSdampID`	ID of Femap function defining the Modal Damping Table.
NASTRAN External Superelement Creation Options (Nastran SOL 101 and 103 Only)	
`BOOL NasExtSEOutOn`	Enable creating of external superelement.
`INT4 NasExtSEOutExtID`	External super element ID.
`BOOL NasExtSEOutMatStiffness`	Store boundary stiffness matrix.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasExtSEOutMatMass`	Store boundary mass matrix.
`BOOL NasExtSEOutMatK4damp`	Store boundary structural damping matrix.
`BOOL NasExtSEOutMatLoads`	Store loads matrix.
`INT4 NasExtSEOutTo`	External superelement output location. 0=DMIGPCH, 1=DMIGOP2, 2=MATOP4.
`INT4 NasExtSEOutUnitID`	Unit ID.
`STRING NasExtSEOutAssignTitle`	Title for filename when writing to DMIGOP2 or MATOP4.
`BOOL NasExtSEOutOptGeom`	Store geometry.
`BOOL NasExtSEOutOptAsmbulk`	Generate bulk entries related to superelement assembly process.
`BOOL NasExtSEOutOptExtbulk`	Generate bulk entries for external superelement.
NASTRAN External Superelement Reference Options	
`BOOL NasBulkSecomb`	Write PARAM,SECOMB when writing super element references.
`BOOL NasExecAllowDuplicateNodes`	Allow duplicate node IDs across super elements.
`REAL8 NasExecDuplicateTolerance`	Specify duplicate node tolerance.
`BOOL NasExecWriteSERefPath`	Write full path for external super element references.
MSC Nastran Contact Solver Parameters Options	
`BOOL NasMscCtOn`	Flag indicating MSC Nastran Contact Solver Parameters options have been defined.
`BOOL NasMscCtEnable`	If True, selected options in MSC Nastran Contact Solver Parameters will be used.
`INT4 NasMscCtMETHOD`	Specifies Contact Method (METHOD) option, 0=`NODESURF`, 1=`SEGSMALL`, 2=`SEGLARGE`.
`REAL8 NasMscCtERROR`	Specify value for Touch Dist (ERROR).

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasMscCtBIAS`	Specify value for Cont Tolerance (BIAS), use 0.9 for default.
`REAL8 NasMscCtSLDLMT`	Specify value for Max Slide Dist (SLDLMT).
`REAL8 NasMscCtSTKSLP`	Specify value for Max Slip Dist (STKSLP).
`BOOL NasMscCtNLGLUE`	If True, All Glue (NLGLUE) is activated.
`BOOL NasMscCtERRBAS`	If True, Error by Pair (ERRBAS) is activated.
`BOOL NasMscCtBEAMB`	If True, Include 3D Beam (BEAMB) is activated.
`BOOL NasMscCtTHKOFF`	If True, Ignore Tolerance Thickness (THKOFF) is activated.
`INT4 NasMscCtAUGMENT`	Specifies the Augment Method (AUGMENT), 0=None, 1=Linear-Constant, Quadratic-Bilinear, 2=Constant, 3=Bilinear.
`REAL8 NasMscCtAUGDIST`	Specify value for Aug Pen Dist (AUGDIST).
`REAL8 NasMscCtPENALT`	Specify value for Augment PenFactor (PENALT).
`REAL8 NasMscCtTPENALT`	Specify value for Stick Pen Fact (TPENALT).
`REAL8 NasMscCtSFNPNLT`	Specify value for Normal Scale Factor (SFNPNLT).
`REAL8 NasMscCtSFTPNLT`	Specify value for Tangent Scale Factor (SFTPNLT).
`REAL8 NasMscCtDDULMT`	Specify value for Max DDU (DDULMT).
`BOOL NasMscCtTAUGMNT`	If True, Sticking Augment (TAUGMNT) is activated.
`BOOL NasMscCtLINCNT`	If True, Infinitesimal Sliding (LINCNT) is activated.
`INT4 NasMscCtFTYPE`	Specifies Friction (FTYPE) type, 0=None, 6=Bilinear Coulomb, 7=Bilinear Shear.
`REAL8 NasMscCtRVCNST`	Specify value for Bilin (RVCNST).

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasMscCtSEGSYM`	If True, Non Symmetric Matrix (SEGSYM) is activated.
`INT4 NasMscCtICSEP`	Specifies Sep Control (ICSEP) option, 0=Nodal Force > Separation Force, 1=Prevent Separation Until Next Increment, 2=Prevent New Node Separation, 3=Both 1 and 2.
`INT4 NasMscCtIBSEP`	Specifies Separation (IBSEP) option, 0=Force Based, 1=Absolute Stress Based, 2=Absolute Intg Pt Stress, 3=Relative Nodal Stress, 4=Relative Intg Pt Stress.
`INT4 NasMscCtMAXSEP`	Integer value for Max Sep (MAXSEP), use 9999 for default.
`REAL8 NasMscCtFNTOL`	Specify value for Sep Force/Stress (FNTOL).
`INT4 NasMscCtNODSEP`	Integer value for Skip Sep Check (NODSEP), use 2 for default.
MSC Nastran Contact Solver Parameters Options - Cohesive Contact Options	
`BOOL NasMscCtFGCFLG`	If True, Enable Cohesive Contact (FGCFLG) is enabled.
`REAL8 NasMscCtFGCNST`	Specify value for Normal Stiffness (FGCNST).
`REAL8 NasMscCtFGCTST`	Specify value for Tangent Stiffness (FGCTST).
`INT4 NasMscCtFGCNSTR`	ID of function for Normal Stress Function (FGCNSTR).
`INT4 NasMscCtFGCTSTR`	ID of function for Tangent Stress Function (FGCTSTR).
`INT4 NasMscCtFGCNSTI`	ID of function for Normal Stiffness Function (FGCNSTI).
`INT4 NasMscCtFGCTSTI`	ID of function for Tangent Stiffness Function (FGCTSTI).
`INT4 NasMscCtFGCRCEN`	ID of Reference Node.
MSC NASTRAN Implicit Nonlinear (SOL 400) - Nonlinear Behavior Options	
`BOOL NasNonlinBehaviorOn`	Flag indicating Nonlinear Behavior Options for Implicit Nonlinear Analysis have been defined.
`BOOL NasNonlinBehaviorEnable`	Flag to activate the Enable NLMOPTS option.

Unpublished work. © 2025 Siemens

Property	Description
Nonlinear Behavior Options - Nonlinear Properties	
`INT4 NasNonlinBehaviorPROPMAP`	Specifies Mapping (PROPMAP) option: -1=No mapping considered 0=Full integration elements when necessary 1=Reduced integration elements when necessary 2=Full integration elements when possible 3=Reduced integration elements when possible
`INT4 NasNonlinBehaviorPROPBEH`	Specifies Behavior (PROPBEH) option: 0=1D 1=3D 2=PLST 3=PSTR 4=AXI
`INT4 NasNonlinBehaviorIPRINT`	Specifies Print Flag (IPRINT) option: 0=Default 1=Print 2=Punch 3=Print and Punch
`INT4 NasNonlinBehaviorDIRECT`	Specify value for Layer Direction (DIRECT).
`REAL8 NasNonlinBehaviorTHICKOP`	Specify value for Thickness (THICKOP).
Nonlinear Behavior Options - Composite Options	
`INT4 NasNonlinBehaviorVCOORD`	Specifies Output (VCOORD) option: 0=Use Original CSys 1=Use Ply CSys of bottom layer 2=Use Ply CSys of above layer
`BOOL NasNonlinBehaviorCPROJ`	If True, X-axis Projection (CPROJ) is activated.
`BOOL NasNonlinBehaviorTSHEAR`	If True, Parabolic Shear Distribution (TSHEAR) is activated.
Nonlinear Behavior Options - Strain Formulations	
`INT4 NasNonlinBehaviorLRGSTRN`	Specifies Large Strain (LRGSTRN) option: -1=No Large Strain Formulation

Unpublished work. © 2025 Siemens

Property	Description
	0=Mean Normal Return 1=Hypoelasticity and additive plasticity with mean normal return 2=Hyperelasticity and multiplicative plasticity with radial return
`INT4 NasNonlinBehaviorASSM`	Specifies Assumed Strain (ASSM) option: 0=Default 1=Use assumed strain formulation 2=Do not use assumed strain formulation
Nonlinear Behavior Options - Element Deactivation	
`INT4 NasNonlinBehaviorVALD1`	Specifies Rules (VALD1) option: -1=Default deactivation rules are always ignored 0=Default deactivation rules are always applied 1=Default deactivation rules are applied in a mechanical pass only 2=Default deactivation rules are applied in a thermal pass only
`INT4 NasNonlinBehaviorVALD2`	Specifies Diag. Output (VALD2) option: 0=Summary of deactivated elements 1=Detailed list of deactivated elements 2=Summary of deactivated and remaining elements 3=Detailed list of deactivated and remaining elements
Nonlinear Behavior Options - Creep	
`BOOL NasNonlinBehaviorCreepOn`	If True, Enable Creep Formulations (CREEP) is enabled
`BOOL NasNonlinBehaviorCreepType`	Specifies Creep Type: True = Explicit Creep, False = Implicit Creep
`INT4 NasNonlinBehaviorCreepExp`	Specifies Explicit Creep option: 0=Maxwell Model Creep 1=Explicit Viscoplastic Creep
`INT4 NasNonlinBehaviorCreepImp`	Specifies Implicit Creep option: 0=Elastic Tangent

Unpublished work. © 2025 Siemens

Property	Description
	1=Secant Tangent 2=Radial Return
MSC NASTRAN Implicit Nonlinear (SOL 400) - Automatic Contact Parameters	
`BOOL NasAutoCtOn`	Flag indicating Automatic Contact Parameters for Implicit Nonlinear Analysis have been defined.
`BOOL NasAutoCtEnabled`	Flag to activate the Enable BCAUTOP option.
Automatic Contact Parameters - Primary Parameters	
`INT4 NasAutoCtBodyVal`	Specifies Automatic Contact Body Type option: 0=Elements (ESET) 1=Elements as Seeds (SeedESet) 2=Nodes as Seeds (SeedGSet) 3=Properties (PropSet)
`INT4 NasAutoCtESET`	ID of Group for Elements (`ESET`).
`INT4 NasAutoCtSeedESet`	ID of Group for Elements as Seeds (`SeedESet`).
`INT4 NasAutoCtSeedGSet`	ID of Group for Nodes as Seeds (`SeedGSet`).
`INT4 NasAutoCtPropSet`	ID of Group for Properties (`PropSet`).
`INT4 NasAutoCtRigidSet`	ID of Group for Rigid Regions (`RigidSet`).
`INT4 NasAutoCtCTYPE`	Specifies Type (CTYPE) option: 0=Touching 1=Permanent Glue 2=Step Glue 3=General Glue
`REAL8 NasAutoCtDISTANCE`	Specify value for Distance Tolerance.
Automatic Contact Parameters - Auxiliary Parameters	
`INT4 NasAutoCtConProp`	ID of Connection Property for Connection Property.
`INT4 NasAutoCtRegion`	ID of Connection Region for Region Property.
Automatic Contact Parameters - Secondary Parameters	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasAutoCt2D`	Specifies options for Dimension (`DIMENS`): True=2D, False=3D
`BOOL NasAutoCtBEAMCNT`	If True, Consider Beam Contact (`BEAMCNT`) is activated.
`BOOL NasAutoCtBEAMCAP`	If True, Detection at Beam Free End (`BEAMCAP`) is activated.
`BOOL NasAutoCtEDGECNT`	If True, Consider Edge Contact (`EDGECNT`) is activated.
`BOOL NasAutoCtIGNTHK`	If True, Ignore Shell Thickness (`IGNTHK`) is activated.
`BOOL NasAutoCtINISTF`	If True, Initial Stress Free Contact (`INISTF`) is activated.
`BOOL NasAutoCtSelfCont`	If True, Self Contact (`SelfCont`) is activated.
Automatic Contact Parameters - Friction	
`INT4 NasAutoCtFTYPE`	Specifies Type (FTYPE) option: 0=No Friction 6=Bilinear Coulomb Friction 7=Bilinear Shear Friction
`BOOL NasAutoCtFRIC`	Friction Option: True = Value Coefficient, False = Table Coefficient.
`INT4 NasAutoCtFRICTABLE`	ID of Function for Table Coefficient.
`REAL8 NasAutoCtFRICVal`	Specify value for Value Coefficient.
MSC NASTRAN Implicit Nonlinear (SOL 400) - Control Parameters	
`BOOL NasInlStepOptOn`	Flag indicating Control Parameters for Implicit Nonlinear Analysis have been defined for the subcase.
`BOOL NasInlStepOptSkipNLSTEP`	Flag to disable writing of NLSTEP for this subcase.
`INT4 NasInlStepOptID`	ID of NLSTEP for this subcase.
Control Parameters - Step Parameters	
`REAL8 NasInlStepOptTOTTIM`	Total Time (TOTTIM) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptCTRLDEF`	Specifies Nonlinearity (CTRLDEF) option: 0=Default 1=Linear 2=Mildly Nonlinear 3=Severely Nonlinear
Control Parameters - General Parameters	
`INT4 NasInlStepOptMAXITER`	Max Iterations/Increment (MAXITER) value.
`INT4 NasInlStepOptMINITER`	Min Iterations/Increment (MINITER) value.
`INT4 NasInlStepOptMAXBIS`	Max Bisections for Step (MAXBIS) value.
`BOOL NasInlStepOptCREEP`	Flag to enable Include Creep (CREEP) option.
Control Parameters - Analysis Parameter	
`INT4 NasInlStepOptLoadStepping`	Specifies Increment Type option: 0=Fixed 1=Adaptive 2=Arc-Length
Control Parameters - Analysis Parameters - Fixed Time Stepping Parameters	
`INT4 NasInlStepOptNINC`	Number of Increments (NINC) value.
`INT4 NasInlStepOptNO`	Output Interval (NO) value.
Control Parameters - Analysis Parameters - Adaptive Load Stepping Parameters	
`REAL8 NasInlStepOptDTINITF`	Initial Fraction (DTINITF) value.
`REAL8 NasInlStepOptDTMINF`	Min Fraction (DTMINF) value.
`REAL8 NasInlStepOptDTMAXF`	Max Fraction (DTMAXF) value.
`REAL8 NasInlStepOptSFACT`	Increase Factor (SFACT) value.
`INT4 NasInlStepOptADJUST`	Auto Skip Factor (ADJUST) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptMSTEP`	Dominant Period Resp. Steps (MSTEP) value.
`REAL8 NasInlStepOptRB`	Adaptive Time Step Bound (RB) value.
`INT4 NasInlStepOptNDESIR`	Desired Iterations/Increments (NDESIR) value.
`INT4 NasInlStepOptNSMAX`	Max Increments (NSMAX) value.
`REAL8 NasInlStepOptUTOL`	Displacement Tolerance (UTOL) value.
`INT4 NasInlStepOptIDAMP`	Specifies Static Damping (IDMAP) option: 0=No Added Damping 4=Time Step Control Using Damping 5=Damping-based Time Step Control with No Damping 6=Add Damping when Minimum Time Step is Reached
`REAL8 NasInlStepOptDAMP`	Damping Ratio (DAMP) value.
`INT4 NasInlStepOptINTOUT`	Specifies Output Flag (INTOUT) option: -1=Last Increment 0=Every Load Increment >0=Equally Spaced Intervals (value is used to specify value for Output Every Nth Step)
Adaptive Load Stepping Parameters - Criteria	
`BOOL NasInlStepOptEnableCriteria`	Flag to enable the Enable User Criteria option.
`INT4 NasInlStepOptIPHYS`	Specifies Physical Criteria (IPHYS) option: -2=No Auto Physical Criteria; Continue When User Criteria Exceeded -1=Add Auto Physical Criteria; Continue When User Criteria Exceeded 1=1..No Auto Physical Criteria; Stop When User Criteria Exceeded 2=2..Add Auto Physical Criteria; Stop When User Criteria Exceeded
`INT4 NasInlStepOptLIMTAR`	Specifies Definition (LIMTAR) option:

Unpublished work. © 2025 Siemens

Property	Description
	0=Criteria as Limits 1=Criteria as Targets
`REAL8 NasInlStepOptRSMALL`	Min. Ratio (RSMALL) value.
`REAL8 NasInlStepOptRBIG`	Max Ration (RBIG) value.
`BOOL NasInlStepOptIncrements[0..9]`	Flag in Status column of row specified by zero-based index value in list of Criteria. When True, Criterion is enabled. 0=Elem. Strain 1=Elem. Plastic Strain 2=Elem. Creep Strain 3=Elem. Normalized Creep Strain 4=Elem. Strain Energy 5=Elem. Stress 6=Elem. Normalized Stress 7=Nodal Temperature 8=Nodal Displacement 9=Nodal Rotation
`INT4 NasInlStepOptGroupIDs[0..9]`	ID of existing Group in Group column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT1_i[0..9]`	Value in Y1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT1_i[0..9]`	Value in M1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT2_i[0..9]`	Value in Y2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT2_i[0..9]`	Value in M2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT3_i[0..9]`	Value in Y3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasInlStepOptMT3_i[0..9]`	Value in M3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT4_i[0..9]`	Value in Y4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT4_i[0..9]`	Value in M4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
Control Parameters - Analysis Parameters - Arc-Length Load Stepping Parameters	
`INT4 NasInlStepOptTYPE`	Specifies Constraint Type option: 0=Crisfield 1=Riks 2=Modifed Riks
`REAL8 NasInlStepOptDTINITFA`	Initial Time Step Fraction (DTINITFA) value.
`REAL8 NasInlStepOptMINALR`	Min ArcLenAdjust Ratio (MINALR) value.
`REAL8 NasInlStepOptMAXALR`	Max ArcLenAdjust Ratio (MAXALR) value.
`REAL8 NasInlStepOptSCALEA`	Scale Factor (SCALEA) value.
`INT4 NasInlStepOptNDESIRA`	Desired Iterations (NDESIRA) value.
`INT4 NasInlStepOptNSMAXA`	Max Increments (NSMAXA) value.
Control Parameters - Analysis Parameters - Mechanical Analysis Parameters	
`BOOL NasInlStepOptMECH`	Flag to enable the Enable MECH option.
`INT4 NasInlStepOptKMETHOD`	Specifies Method (KMETHOD) option: 0=Full Newton-Raphson 1=Quasi Newton-Raphson
`INT4 NasInlStepOptKSTEP`	Iteration Interval (KSTEP) value.
`INT4 NasInlStepOptMAXQN`	QN Vectors (MAXQN) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptMAXLS`	Max Line Searches (MAXLS) value.
`REAL8 NasInlStepOptLSTOL`	Line Search Tol (LSTOL) value.
`REAL8 NasInlStepOptFSTRESS`	Fraction Limit (FTRESS) value.
`BOOL NasInlStepOptConvU`	Flag to activate Displacement option.
`REAL8 NasInlStepOptEPSU`	Displacement value.
`BOOL NasInlStepOptConvP`	Flag to activate Load option.
`REAL8 NasInlStepOptEPSP`	Load value.
`BOOL NasInlStepOptConvW`	Flag to activate Work option.
`REAL8 NasInlStepOptEPSW`	Work value.
`BOOL NasInlStepOptConvV`	Flag to activate Vector option.
`BOOL NasInlStepOptConvN`	Flag to activate Length option.
`BOOL NasInlStepOptConvA`	Flag to activate Automatic Switching option.
`INT4 NasInlStepOptMRCONV`	Specifies Checking (MRCONV) option: 0=Forces, Moments, Displacements, and Rotations 1=Forces, Moments, and Displacements 2=Forces, Displacements, and Rotations 3=Forces and Displacements
MSC Nastran Boundary Conditions	
`INT4 ContactOption`	Boundary Conditions dialog box for MSC Nastran Only. Sets BCONTACT Option (0=Default, 1=Inital Contact, 2=All Body, 3=Auto).
`INT4 ContactSetType`	Boundary Conditions dialog box for MSC Nastran Only. Sets Contact/Glue Sets option (0=All Connectors, 1=Connection Group, 2=All Body). ID of group for Connection Group set using `BCSet[13]`.

Unpublished work. © 2025 Siemens

Property	Description
ANSYS Analysis Set Options (Legacy)	
Master Case Options - ANSYS Specific (Legacy)	
`BOOL AnsDOFOn`	Flag indicating dynamic DOF options have been defined.
`BOOL AnsDOFDoSet`	Flag indicating a master DOF set is specified and in use.
`BOOL AnsDOFAutomatic`	Indicates automatic dynamic DOF selection.
`BOOL AnsDOFNoRotation`	Excludes rotational DOF from automatic DOF selection.
`INT4 AnsDOFSetID`	Constraint set ID to be used as a master DOF set.
`INT4 AnsDOFNumber`	Number of degrees of freedom to automatically select.
ANSYS Model Write (Legacy)	
`BOOL AnsModOn`	Flag indicating ANSYS Model options have been specified.
`BOOL AnsModTitleSetNames`	Flag to Write Femap titles as set names.
`BOOL AnsModSkipPreprocessCMD`	Flag to skip writing `/PREP7` command.
`BOOL AnsModGroupsAsSets`	Flag to write Groups As Sets.
`BOOL AnsModSkipStandard`	Flag to skip output of standard ANSYS model.
`INT4 AnsModStartText`	ID of text written at beginning of ANSYS model.
`INT4 AnsModEndText`	ID of text written at end of ANSYS model.
`STRING AnsModTitle`	Title for analysis (maximum 79 characters).
`BOOL AnsModWriteSolCmd`	Flag to write commands for automatic solve of model.
`BOOL AnsModWriteIter`	Flag to choose the iterative solver in ANSYS 5.3 and above.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsModWriteLargeDef`	Flag to include large deformation effects in nonlinear.
`INT4 AnsModMemory`	Analysis memory: 0=Default.
`INT4 AnsModVersion`	ANSYS version (4=ANSYS 4.4, 5=ANSYS 5.x).
ANSYS Modal Options (Legacy)	
`BOOL AnsModesOn`	Flag indicating modal expansion options have been defined.
`INT4 AnsModesNumModes`	Number of modes/frequencies to expand.
`REAL8 AnsModesMinFreq`	Minimum frequency of interest.
`REAL8 AnsModesMaxFreq`	Maximum frequency of interest.
`REAL8 AnsModesModalMethod`	Modal expansion method.
`BOOL AnsModesElemResults`	Flag indicating whether elemental results (stresses, strains...) will be calculated.
ANSYS Heat Transfer Options (Legacy)	
`BOOL AnsHeatOn`	Flag indicating heat transfer options have been defined.
`REAL8 AnsHeatEndTime`	Time at end of this load step.
`REAL8 AnsHeatTimeStep`	Initial time increment.
`INT4 AnsHeatOutputFreq`	Output step interval.
ANSYS Random Analysis Options (Legacy)	
`BOOL AnsRandOn`	Flag indicating ANSYS Random Analysis Options have been specified.
`BOOL AnsRandDisp`	Flag requesting displacement output.
`BOOL AnsRandVel`	Flag requesting velocity output.
`BOOL AnsRandAccel`	Flag requesting acceleration output.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL AnsRandDispRelative`	Flag indicating displacement output should be relative or absolute.
`BOOL AnsRandVelRelative`	Flag indicating velocity output should be relative or absolute.
`BOOL AnsRandAccelRelative`	Flag indicating acceleration output should be relative or absolute.
Analysis Set Options - Femap Structural Specific	
Femap Structural File Operations	
`BOOL FStFilOn`	Flag indicating FEMAP Structural File Operation Options have been specified.
`STRING FStFilLogFile[260]`	Full path to the log file (blank=default).
`BOOL FStFilKeepLogFile`	Flag indicating that the log file should be kept.
`BOOL FStFilEchoLogFile`	
`STRING FStFilHyper1[260]`	Filename for first Hypermatrix file (blank=default).
`STRING FStFilHyper2[260]`	Filename for second Hypermatrix file (blank=default).
`BOOL FStFilKeepHyper`	Flag indicating that Hypermatrix files should be kept.
`STRING FStFilScratch1[260]`	Path to sparse solver scratch directory.
`STRING FStFilScratch2[260]`	Path to additional sparse solver scratch directory.
Femap Structural Heat Transfer Options	
`BOOL FStHeatOn`	Flag indicating Femap Structural Heat Transfer Options have been specified.
`REAL8 FStHeatTempTol`	Temperature convergence tolerance.
`INT4 FStHeatConvergeType`	Convergence norm: 0=Absolute, 1=Euclidean, 2=Maximum.
`INT4 FStHeatMaxIter`	Maximum iterations.
FEMAP Structural Dynamic Solution Options	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL FStDynOn`	Flag indicating FEMAP Structural Dynamic Solution options have been specified.
`INT4 FStDynSolMethod`	Solution method: 0=Guyan, 1=SVI, 2=Lanczos.
`BOOL FStDynMassForm`	Mass formulation: 0=Consistent, 1=Lumped.
`BOOL FStDynModalDamp`	Flag indicating that modal damping should be included.
`BOOL FStDynModalNorm`	Flag indicating mode normalization method.
`INT4 FStDynNumModes`	Number of flexible modes.
`INT4 FStDynRigidModes`	Number of rigid body modes.
`INT4 FStDynFreqMethod`	Frequency range method: 0=Above, 1=Below, 2=Around, 3=Between.

`REAL8 FStDynFreqRange[2]`
or
`VARIANT vFStDynFreqRange`
	Frequency range: [0]=Single or Min Frequency, [1]=Max Frequency.

`INT4 FStDynMassDir`	Effective mass direction: 0=None, 1=X, 2=Y, 3=Z, 4=X+Y, 5=X+Z, 6=Y+Z, 7=X+Y+Z.
`REAL8 FStDynMassTol`	Effective mass lower limit.
`INT4 FStDynIterVec`	Number of iteration vectors.
`INT4 FStDynMaxIter`	Max iterations.
`INT4 FStDynSigDigits`	Significant digits.
Femap Structural Solver Options	
`BOOL FStSolOn`	Flag indicating FEMAP Structural Solver Options have been specified.
`BOOL FStSolIter`	Flag choosing sparse or iterative solver.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 FStSolIterConvTol`	Iterative solver convergence tolerance.
`INT4 FStSolIterMaxIter`	Iterative solver max iterations.
`INT4 FStSolMemIterMax`	Max memory for iterative solver in MBytes.
`INT4 FStSolMemVirtMax`	Total virtual memory limit in MBytes.
`INT4 FStSolMemAppPct`	Application dynamic memory in percent.
`INT4 FStSolMemFortranPct`	Fortran workspace in percent.
`BOOL FStSolStressStiff`	Flag to turn on stress stiffening.
`BOOL FStSolSpinSoft`	Flag to turn on spin softening.
`INT4 FStSolStiffSoftLoad`	Load set ID for use with stress stiffening or spin softening.
`BOOL FStSolShellAuto`	Shell autorestraint option.
`BOOL FStSolSingularity`	Flag to select singularity removal.
`BOOL FStSolVerification`	Flag indicating that a verification-only analysis.
`REAL8 FStSolShellAutoAng`	Shell autorestraint angle.
`REAL8 FStSolSingCriteria`	Singularity removal criteria.
`INT4 FStSolNodeGroup`	ID of group containing node IDs for nodal output recovery (0=All Nodes).
`INT4 FStSolElemGroup`	ID of group containing element IDs for elemental output recovery (0=All Elements).
Master Case Options - MARC Specific	
`BOOL MarHistOn`	Flag indicating MARC History options have been defined.
`INT4 MarHistContactProp`	ID of Contact Property written to the CONTACT TABLE.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MarHistCtrlMethod`	ID of control type written to the CONTROL option. 0=None, 1=Full Newton-Raphson, 2=Modified Newton-Raphson, 3=Strain Correction Newton-Raphson, 8=Secant Method.
`INT4 MarHistSolverMeth`	ID of solver type written to the SOLVER option. 0=None, 1=Profile Direct, 2=Sparse Iterative, 3=Sparse Direct, 4=Hardware Provided Sparse Direct, 5=MultiFrontal Direct Sparse.
`INT4 MarHistCtrlMaxStep`	Number of Max Steps written to CONTROL option.
`INT4 MarHistCtrlMaxRecycle`	Number of Max Recycles written to CONTROL option.
`INT4 MarHistCtrlMinRecycle`	Number of Min Recycles written to CONTROL option.
`REAL8 MarHistMaxResidual`	Number of Max Conv. Residual written to CONTROL option.
`INT4 MarHistConvergeMeth`	ID of method written to CONTROL option. 0=Rel Force Residuals, 1=Rel Disp Residuals, 2=Rel Strain Ener Res, 3=Abs Force Residuals, 4=Abs Disp Residuals, 5=Abs Strain Ener Res.
`BOOL MarHistNonPosDefinite`	Flag indicating Nonpositive Def will be used on SOLVER option.
`BOOL MarHistNonSymmetric`	Flag indicating NonSymmetric will be used on SOLVER option.
`REAL8 MarHistModalMinFreq`	Number indicating the Min Freq for a modal analysis.
`REAL8 MarHistModalMaxFreq`	Number indicating the Max Freq for a modal analysis.
`INT4 MarHistModalEigen`	Number indicating the Number of eigenvectors to be found in modal analysis.
`INT4 MarHistChecking`	ID indicating checking will be used. 0=None, 1=Strum Sequence.
`BOOL MarHistAutoSwitch`	Flag indicating Auto Switch will be used on CONTROL option.
`INT4 MarHistAnalCaseSol`	ID indicating the Analysis type for this Subcase 1=Static, 2=Normal Modes, 3=Buckling.
`INT4 MarHistBuckleNumModes`	Number of Buckling modes.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MarHistBucklePosEigen`	Number of Modes With positive eigenvalues.
`BOOL MarIncOn`	Flag indicating MARC Increment options have been defined.
`INT4 MarIncArcLenMeth`	ID indicating the Arclength Method. 0=None(AUTO STEP), 1=Crisfield, 2=Riks, 3=Modified Riks, 4=Crisfield/Modified Riks.
`REAL8 MarIncTimePeriod`	Number indicating AUTO STEP time period.
`REAL8 MarIncInitTimeInc`	Number indicating AUTO STEP initial time increment.
`REAL8 MarIncMinTimeInc`	Number indicating AUTO STEP min time increment.
`REAL8 MarIncMaxTimeInc`	Number indicating AUTO STEP max time increment.
`INT4 MarIncScaleFact`	Number indicating AUTO STEP scale factor.
`INT4 MarIncMaxStep`	Number indicating AUTO STEP max number of steps.
`INT4 MarIncNumOutInc`	Number indicating number of output intervals.
`BOOL MarIncQuasiStatDamp`	Flag to include Damping scheme for statics.
`REAL8 MarIncFracFirst`	Number indicating AUTO INCREMENT fraction of total load applied in first step.
`INT4 MarIncMaxNumInc`	Number indicating AUTO INCREMENT max num increments.
`INT4 MarIncCtrlMaxRecycle`	Number indicating AUTO INCREMENT max recycles.
`REAL8 MarIncMaxFracForAll`	Number indicating AUTO INCREMENT max fraction of load for any step.
`REAL8 MarIncTotalTime`	Number indicating AUTO INCREMENT total time.
`BOOL MarCtblOn`	Flag indicating the MARC Contact Table has been defined.
`BOOL MarCtblWrite`	Flag indicating the MARC Contact Table is to be written.

Unpublished work. © 2025 Siemens

Property	Description
Analysis Set Options - MARC Specific	
`BOOL MarModOn`	Flag Indicating MARC parameters have been specified.
`BOOL MarModSkipStandard`	Flag to skip the standard MARC model section.
`INT4 MarModStartText`	ID of text written at the begining of the model section.
`INT4 MarModEndText`	ID of text written at the end of the model section.
`char MarModTitle[160]`	Title of model specified as text.
`INT4 MarModWorkSpVec`	Size of Work Space Vector on the SIZING command.
`INT4 MarModMaxConstrain`	Max Number of constraints written to the SIZING command.
`BOOL MarModFolOn`	Flag indicating Follower Force has been specified.
`INT4 MarModFolOpt`	ID of Follower Option 1=Off Incremental, 2=Last Iter Inc, 3=Beg Disp Incr, 4=Off Total, 5=Last Iter Total, 6=Beg Disp To
`BOOL MarModPlasOn`	Flag indicating Plasticity has been specified.
`INT4 MarModPlasOpt`	ID of Plasticity Option 1=Add Normal Small Strain 2=Add Radial Small Strain, 3=Add Normal Large Strain, 4=Add Radial Large Strain, 5=Multiplicative.
`BOOL MarModElasOn`	Flag indicating Elasticity Force has been specified.
`BOOL MarModElasTotal`	Total Lagrange=TRUE, Updated Lagrange=FALSE
`BOOL MarModElasIs`	Flag indicating ELASTIC parameter will be used.
`BOOL MarModConstDil`	Flag indicating CONSTANT parameter will be used.
`BOOL MarModLgDisp`	Flag indicating LARGE DISP parameter will be used.
`BOOL MarModUpdate`	Flag indicating UPDATE parameter will be used.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL MarModFinite`	Flag indicating FINITE parameter will be used.
`BOOL MarModEchoInput`	Flag to Echo the Input.
`BOOL MarModWriteGroupSet`	Flag to write groups as sets.
`BOOL MarModExtInput`	Flag indicating the EXTENDED parameter will be used.
`BOOL MarModShlTrShear`	Flag indicating the TSHEAR parameter will be used.
`INT4 MarModPltIntegPt`	Number of SHELL SECT integration points.
`BOOL MarModDistLoadOn`	Flag indicating Load List information has been specified.
`INT4 MarModDistLoadList`	Number indicating the max number of Load List in the model.
`INT4 MarModDistLoadElem`	Number indicating the max number of Elements in any list.
`INT4 MarModDistLoadNode`	Number indicating the max number of Nodes in any list.
`BOOL MarModTyingOn`	Flag indicating TIE parameter will be used.
`INT4 MarModTyingCe`	Number of tying type constraint equations.
`INT4 MarModTyingType`	Number of different types of tying constraints.
`INT4 MarModTyingServoNd`	Max Number of retained nodes for the tying type.
`INT4 MarModTyingServoLk`	Max Number of Servo Links.
`BOOL MarModOutOfCore`	Flag indicating the ELSTO parameter will be used.
`INT4 MarModCoreVal`	Number indicating the Out Of Core buffer size.
`BOOL MarModProcessors`	Flag indicating the PROCESSOR parameter will be used.
`INT4 MarModNoCpu`	Number of CPU's to be used.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 MarModVecLength`	Number indicating Vector Length for the PROCESSOR param.
`BOOL MarModParallel`	Flag if matrices are to be formed in parallel.
`BOOL MarModBWOpt`	Flag indicating the OPTIMIZE parameter will be used.

AnalysisMgr Object Methods

InitAnalysisMgr

`InitAnalysisMgr` **`( analysis_program, analysis_type, InitMasterCase )`**	
Description:	
This method initializes default options in the AnalysisMgr object.	
Input:	
`INT4 analysis_program`	Refer to the Solver property.
`INT4 analysis_type`	Refer to the AnalysisType property.
`BOOL InitMasterCase`	If true, also initializes the Master Case in in the object.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to initialize the current object.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetText

GetText **(textID, text)**	
Description:	
This method returns a string containing the text associated with a specified `textID`.	
Input:	
`INT4 textID`	A textID returned from one of the text properties.
Output:	
`STRING text`	A string containing the text. This string will contain carriage control characters if there are multiple lines in the text.
Return Code:	
`FE_FAIL`	Unable to get the text. Check to see this is a valid ID.
Remarks/Usage:	
Text IDs are returned from various `AnalysisMgr` properties. These properties have names ending with `StartText` or `EndText`.	
Example:	
None	

PutText

Unpublished work. © 2025 Siemens

PutText	
(textID, text)	
Description:	
This method updates or creates text to associate with a text property.	
Input:	
`INT4 textID`	A textID used to store the text string. Specify the same value as the text property returned if you simply want to update the text. Specify 0 if you want to create a new text entity. In this case, the return value will be the textID that was created. You should set the text property to this value.
`STRING text`	A string containing the text. May contain carriage control to signify multiple lines.
Output:	
None	None
Return Code:	
`textID`	The ID.
Remarks/Usage:	
Text IDs are returned from various `AnalysisMgr` properties. These properties have names ending with `StartText` or `EndText`.	
Example:	
None	

AllFreqOn

AllFreqOn
(void)
Description:

Unpublished work. © 2025 Siemens

This method turns on all available *lists of solution frequencies* (FREQ and FREQi entries) in the **Solution Frequencies** tab of the NASTRAN Dynamic Analysis dialog box.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	All available lists of solution frequencies have been turned on.
Remarks/Usage:	
This method applies only to the analysis set currently loaded into a AnalysisMgr object.	
Example:	
None	

AllFreqOff

AllFreqOff **(void)**	
Description:	
This method turns off all available lists of solution frequencies (FREQ and FREQi entries) in the **Solution Frequencies** tab of the NASTRAN Dynamic Analysis dialog box.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	All available lists of solution frequencies have been turned off.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method applies only to the analysis set currently loaded into a AnalysisMgr object.
Example:
None

TurnOnFreq

TurnOnFreq **(freqID)**	
Description:	
This method turns on any number of *lists of solution frequencies* (FREQ and FREQi entries) in the **Solution Frequencies** tab of the NASTRAN Dynamic Analysis dialog box.	
Input:	
freqID	The ID of a set that contains a list of solution frequency lists to activate. Alternatively, if you specify a negative value, this is simply the ID of the single list of solution frequencies.
Output:	
None	None
Return Code:	
FE_OK	Specified lists of solution frequencies have been turned on.
Remarks/Usage:	
This method applies only to the analysis set currently loaded into a AnalysisMgr object.	
Example:	
None	

TurnOffFreq

Unpublished work. © 2025 Siemens

TurnOffFreq	
(freqID)	
Description:	
This method turns off any number of *lists of solution frequencies* (FREQ and FREQi entries) in the **Solution Frequencies** tab of the NASTRAN Dynamic Analysis dialog box.	
Input:	
`freqID`	The ID of a set that contains a list of solution frequency lists to deactivate. Alternatively, if you specify a negative value, this is simply the ID of the single list of solution frequencies.
Output:	
None	None
Return Code:	
`FE_OK`	Specified lists of solution frequencies have been turned off.
Remarks/Usage:	
This method applies only to the analysis set currently loaded into a AnalysisMgr object.	
Example:	
None	

GetUsedFREQS

GetUsedFREQS	
(nSetID)	
Description:	
This method fills a set object with the FREQS that are turned on in the specified analysis set.	
Input:	
`INT4 nSetID`	The ID of the set to fill the FREQ IDs with.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_OK`	Specified lists of solution frequencies have been turned on.
Remarks/Usage:	
This method applies only to the analysis set currently loaded into a AnalysisMgr object.	
Example:	
None	

PutUsedFREQS

`PutUsedFREQS`	
`( nSetID )`	
Description:	
This method puts a set object filled with FREQ IDs to be turned on in the specified analysis set.	
Input:	
`INT4 nSetID`	The set ID filled with the FREQ IDs to turn on.
Output:	
None	None
Return Code:	
`FE_OK`	Specified lists of solution frequencies have been turned on.
Remarks/Usage:	
This method applies only to the analysis set currently loaded into a AnalysisMgr object.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetCorrelate

GetCorrelate **(count, excitedCase, appliedCase, realScale, imagScale, realFunc, imagFunc)**	
Description:	
This method retrieves data from the Random Analysis Correlation Table.	
Input:	
None	None
Output:	
`INT4 count`	The number of entries in each of the other vectors.
`INT4 excitedCase[0..count-1]`	The IDs of the analysis case being excited.
`INT4 appliedCase[0..count-1]`	The IDs of the analysis case where the PSD is applied.
`REAL8 realScale[0..count-1]`	The scale factor for the real portion of the PSD.
`REAL8 imagScale[0..count-1]`	The scale factor for the imaginary portion of the PSD.
`INT4 realFunc[0..count-1]`	The IDs of functions defining the real PSDs.
`INT4 imagFunc[0..count-1]`	The IDs of functions defining the imaginary PSDs.
Return Code:	
`FE_NOT_EXIST`	The correlation table does not exist.
`FE_FAIL`	Unable to allocate memory to return the correlation data.
Remarks/Usage:	
This function has been replace by `GetCorrelate2` in Femap 10.1. The AnalysisMgr object must be loaded with data from the associated set before this function can be used.	

Unpublished work. © 2025 Siemens

Example:
None

GetCorrelate2

GetCorrelate2 **(count, excitedCase, appliedCase, realScale, imagScale, realFunc, imagFunc, realInterp, imagInterp)**	
Description:	
This method retrieves data from the Random Analysis Correlation Table.	
Input:	
None	None
Output:	
`INT4 count`	The number of entries in each of the other vectors.
`INT4 excitedCase[0..count-1]`	The IDs of the analysis case being excited.
`INT4 appliedCase[0..count-1]`	The IDs of the analysis case where the PSD is applied.
`REAL8 realScale[0..count-1]`	The scale factor for the real portion of the PSD.
`REAL8 imagScale[0..count-1]`	The scale factor for the imaginary portion of the PSD.
`INT4 realFunc[0..count-1]`	The IDs of functions defining the real PSDs.
`INT4 imagFunc[0..count-1]`	The IDs of functions defining the imaginary PSDs.
`INT4 realInterp[0..count-1]`	The IDs of Interpolation method defining the real PSD. 0=Log Log, 1=Linear, 2=XLog YLin, 3=XLin YLog.
`INT4 imagInterp[0..count-1]`	The IDs of Interpolation method defining the imaginary PSD. 0=Log Log, 1=Linear, 2=XLog YLin, 3=XLin YLog.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_EXIST`	The correlation table does not exist.
`FE_FAIL`	Unable to allocate memory to return the correlation data.
Remarks/Usage:	
The AnalysisMgr object must be loaded with data from the associated set before this function can be used.	
Example:	
None	

PutCorrelate

`PutCorrelate` **`( count, excitedCase, appliedCase, realScale, imagScale, realFunc, imagFunc )`**	
Description:	
This method replaces data in the Random Analysis Correlation Table.	
Input:	
`INT4 count`	The number of entries in each of the other vectors.
`INT4 excitedCase[0..count-1]`	The IDs of the analysis case being excited.
`INT4 appliedCase[0..count-1]`	The IDs of the analysis case where the PSD is applied.
`REAL8 realScale[0..count-1]`	The scale factor for the real portion of the PSD.
`REAL8 imagScale[0..count-1]`	The scale factor for the imaginary portion of the PSD.
`INT4 realFunc[0..count-1]`	The IDs of functions defining the real PSDs.
`INT4 imagFunc[0..count-1]`	The IDs of functions defining the imaginary PSDs.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to update the correlation table.
Remarks/Usage:	
This function has been replace by `PutCorrelate2` in Femap 10.1. The AnalysisMgr object must be loaded with data from the associated set before this function can be used. This function completely replaces the correlation table - it does not add to data that is already there.	
Example:	
None	

PutCorrelate2

`PutCorrelate2` **`( count, excitedCase, appliedCase, realScale, imagScale, realFunc, imagFunc, realInterp, imagInterp )`**	
Description:	
This method replaces data in the Random Analysis Correlation Table.	
Input:	
`INT4 count`	The number of entries in each of the other vectors.
`INT4 excitedCase[0..count-1]`	The IDs of the analysis case being excited.
`INT4 appliedCase[0..count-1]`	The IDs of the analysis case where the PSD is applied.
`REAL8 realScale[0..count-1]`	The scale factor for the real portion of the PSD.
`REAL8 imagScale[0..count-1]`	The scale factor for the imaginary portion of the PSD.
`INT4 realFunc[0..count-1]`	The IDs of functions defining the real PSDs.

Unpublished work. © 2025 Siemens

`INT4 imagFunc[0..count-1]`	The IDs of functions defining the imaginary PSDs.
`INT4 realInterp[0..count-1]`	The IDs of Interpolation method defining the real PSD. 0=Log Log, 1=Linear, 2=XLog YLin, 3=XLin YLog.
`INT4 imagInterp[0..count-1]`	The IDs of Interpolation method defining the imaginary PSD. 0=Log Log, 1=Linear, 2=XLog YLin, 3=XLin YLog.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to update the correlation table.
Remarks/Usage:	
The AnalysisMgr object must be loaded with data from the associated set before this function can be used. This function completely replaces the correlation table - it does not add to data that is already there.	
Example:	
None	

ActiveContactEntities

`ActiveContactEntities`
`( bContact, nConnectorSetID, nRegionSetID, nConPropSetID )`
Description:
This method is used to populate Set Objects with the Connectors, Connection Regions, and Connection Properties that will be exported for Contact or Glue Contact by an analysis set. This method takes into consideration the settings in the Contact Sets or Glue Sets section of the Boundary Conditions dialog box for Simcenter SOL 401/402 or setting for Contact/Glue Sets of the Boundary Conditions dialog box for MSC Nastran.
Input:

`BOOL bContact`	If True, populates sets with connection entities for Contact. If False, populates sets with connection entities for Glued Contact.

Unpublished work. © 2025 Siemens

`INT4 nConnectorSetID`	Set ID of Set Object to populate with Connectors which will be exported by this analysis set.
`INT4 nRegionSetID`	Set ID of Set Object to populate with Connection Regions which will be exported by this analysis set.
`INT4 nConPropSetID`	Set ID of Set Object to populate with Connection Properties which will be exported by this analysis set.
Output:	
None	None
Return Code:	
`FE_FAIL`	No Connection Regions were detected.
Remarks/Usage:	
It is important to use three uniques Set Objects when using this method, otherwise the IDs of different entity types may contaminate a Set. Also, it is recommend to use *empty sets* to ensure that only the Connection entities from a single Analysis Set populate the three sets.	
Example:	
None	

GetContactTable

`GetContactTable` **`( count, rowSeg, colSeg, tieType, distTol, sepForce, friction, interClose )`**	
Description:	
This method retrieves data from the Contact Table.	
Input:	
None	None
Output:	
`INT4 count`	The number of entries in each of the other vectors.

Unpublished work. © 2025 Siemens

`INT4 rowSeg[0..count-1]`	The IDs of the contact segments referenced in rows of the table.
`INT4 colSeg[0..count-1]`	The IDs of the contact segments referenced in columns of the table.
`INT4 tieType[0..count-1]`	The tying type (0,1=None, 2=Touch, 3=Glue).
`REAL8 distTol[0..count-1]`	The distance tolerance.
`REAL8 sepForce[0..count-1]`	The separation force.
`REAL8 friction[0..count-1]`	The friction values.
`REAL8 interClose[0..count-1]`	The interference closure.
Return Code:	
`FE_NOT_EXIST`	The contact table does not exist.
`FE_FAIL`	Unable to allocate memory to return the contact table data.
Remarks/Usage:	
None	
Example:	
None	

PutContactTable

`PutContactTable`
`( count, rowSeg, colSeg, tieType, distTol, sepForce, friction, interClose )`
Description:
This method replaces data in the Contact Table.
Input:

Unpublished work. © 2025 Siemens

`INT4 count`	The number of entries in each of the other vectors.
`INT4 rowSeg[0..count-1]`	The IDs of the contact segments referenced in rows of the table.
`INT4 colSeg[0..count-1]`	The IDs of the contact segments referenced in columns of the table.
`INT4 tieType[0..count-1]`	The tying type (0,1=None, 2=Touch, 3=Glue).
`REAL8 distTol[0..count-1]`	The distance tolerance.
`REAL8 sepForce[0..count-1]`	The separation force.
`REAL8 friction[0..count-1]`	The friction values.
`REAL8 interClose[0..count-1]`	The interference closure.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to update the contact table.
Remarks/Usage:	
This function completely replaces the contact table - it does not add to data that is already there.	
Example:	
None	

Case

`Case`
`( caseID )`
Description:
This method creates and returns an AnalysisCase object that is loaded with the options referenced by a case.

Unpublished work. © 2025 Siemens

Input:	
`INT4 caseID`	The ID of the Analysis Case to load.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current AnalysisMgr object must be loaded with the data from the set that contains the case that you want to retrieve. The `AnalysisMgr.ID` property is used to define the set that you want to retrieve.	
Example:	

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim rc As Integer

   Dim aMgr As femap.AnalysisMgr
   Set aMgr = App.feAnalysisMgr

   Dim aCase As femap.AnalysisCase

   rc = aMgr.Get( 1 )
   If (rc = FE_OK) Then
       Set aCase = aMgr.Case( 1 )
       App.feAppMessage(FCM_NORMAL, aCase.CaseTitle )
   End If
End Sub
```


Unpublished work. © 2025 Siemens

Analyze

Analyze **(setID)**	
Description:	
Selects and analyzes an Analysis Set.	
Input:	
`INT4 setID`	The ID of the Analysis Set to be analyzed.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method can be used to select an analysis set and run the associated analysis program.	
Example:	
None	

AnalyzeMultiple

AnalyzeMultiple **(setID)**	
Description:	
Selects and sequentially analyzes one or more Analysis Sets.	
Input:	
`INT4 setID`	The ID of a Set that contains the IDs of the Analysis Sets to be analyzed.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method can be used to select multiple analysis sets and run the associated analysis program.	
Example:	

Script to create five Nastran Static analysis type analysis sets and run them:

```
Sub Main
       Dim App As femap.model
       Set App = feFemap()

       Dim am As femap.AnalysisManager
       Set am = App.feAnalysisMgr

       Dim aMgrSet As femap.Set
       Set aMgrSet = App.feSet

       am. InitAnalysisMgr(FAP_NX_NASTRAN, FAT_STATIC, True)

       Dim aID As Long
       Dim I1 As Integer

       For I1 = 0 to 5
           aID = App.Info_NextID(FT_AMGR_DIR)
           am. Put(aID)
           aMgrSet. Add(aID)

          Dim ac As femap.AnalysisCase
          Set ac = App. feAnalysisCase

          Dim aCase As Long

           aID = App. Info_ActiveID(FT_AMGR_DIR)

           ac. InitAnalysisCase(aID,1)
           ac. Put(1)

           ac. InitAnalysisCase(aID,2)
           ac. Put(2)
         Next

     am. AnalyzeMultiple(aMgrSet.ID)

End Sub
```


Unpublished work. © 2025 Siemens

ClearAnalysisQueue

ClearAnalysisQueue **(void)**	
Description:	
Clears the Analysis Job Queue of jobs.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
None	
Example:	
None	

ClearJobMonitorData

ClearJobMonitorData **(void)**	
Description:	
Delete all job monitor related data of analyzed analysis sets. Calling this method clears the job monitor data from memory and also resets the tree icons of the analysis sets to *Not Analyzed*.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
None	
Example:	

```
Sub Main
       Dim App As femap.model
       Set App = feFemap()

       Dim am As femap.AnalysisManager
       Set am = App.feAnalysisMgr

       am. ClearJobMonitorData( )
```

Preview

Preview	
(SetID)	
Description:	
Calls Preview Input in the Analysis Set Manager.	
Input:	
`INT4 setID`	The ID of a Analysis Set to be previewed in the Preview Analysis Input File window. Enter -1 for the Active Analysis Set.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetStandardTrim

`GetStandardTrim` **`( useTrim, TrimID, TrimVal )`**	
Description:	
This method retrieves data from the standard trim definition.	
Input:	
None	None
Output:	
`INT4 useTrim[0..11]`	0=Off,1=Use, 2= Fixed. See Remarks for array index value.
`INT4 TrimID[0..11]`	Not currently used.
`REAL8 TrimVal[0..11]`	Magnitude value when `UseTrim[i]` = Fixed. See Remarks for array index value.
Return Code:	
`FE_NOT_EXIST`	Standard Trim values have not been defined.
`FE_FAIL`	Unable to allocate memory to return the trim data.
Remarks/Usage:	
Valid `TrimID` values: 0 = NAS_AERO_TRIM_ANGLEA 1 = NAS_AERO_TRIM_SIDES 2 = NAS_AERO_TRIM_ROLL 3 = NAS_AERO_TRIM_PITCH 4 = NAS_AERO_TRIM_YAW 5 = NAS_AERO_TRIM_URDD1	

Unpublished work. © 2025 Siemens

6 = NAS_AERO_TRIM_URDD2
7 = NAS_AERO_TRIM_URDD3
8 = NAS_AERO_TRIM_URDD4
9 = NAS_AERO_TRIM_URDD5
10 - NAS_AERO_TRIM_URDD6

Example:

None

PutStandardTrim

PutStandardTrim

(useTrim, TrimID, TrimVal)

Description:

This method loads standard trim data into the trim definition.

Input:

`INT4 useTrim[0..11]`	0=Off, 1=Use, 2= Fixed. See Remarks for array index value.
`INT4 TrimID[0..11]`	Not currently used.
`REAL8 TrimVal[0..11]`	Magnitude value when `UseTrim[i]` = Fixed. See Remarks for array index value.

Output:

None	None

Return Code:

`FE_NOT_EXIST`	Standard Trim values have not been defined.
`FE_FAIL`	Unable to allocate memory to return the trim data.

Remarks/Usage:

Valid index values:

0 = NAS_AERO_TRIM_ANGLEA
1 = NAS_AERO_TRIM_SIDES

Unpublished work. © 2025 Siemens

2 = NAS_AERO_TRIM_ROLL 3 = NAS_AERO_TRIM_PITCH 4 = NAS_AERO_TRIM_YAW 5 = NAS_AERO_TRIM_URDD1 6 = NAS_AERO_TRIM_URDD2 7 = NAS_AERO_TRIM_URDD3 8 = NAS_AERO_TRIM_URDD4 9 = NAS_AERO_TRIM_URDD5 10 - NAS_AERO_TRIM_URDD6
Example:
None

GetTrimControlSurf

GetTrimControlSurf **(useTrim, TrimID, TrimVal)**	
Description:	
This method retrieves data from the trim definition for user defined Aero Control Surfaces.	
Input:	
None	None
Output:	
INT4 useTrim[0..29]	0=Off,1=Use, 2= Fixed.
INT4 TrimID[0..29]	ID of the Femap Aero Control Surface Entity.
REAL8 TrimVal[0..29]	Magnitude.
Return Code:	
FE_NOT_EXIST	Trim values have not been defined.
FE_FAIL	Unable to allocate memory to return the trim data.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

PutTrimControlSurf

`PutTrimControlSurf` **`( useTrim, TrimID, TrimVal )`**	
Description:	
This method loads user defined Aero Control Surface data into the Trim definition.	
Input:	
`INT4 useTrim[0..29]`	0=Off,1=Use, 2= Fixed.
`INT4 TrimID[0..29]`	ID of the Femap Aero Control Surface Entity.
`REAL8 TrimVal[0..29]`	Magnitude.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the trim data.
Remarks/Usage:	
None	
Example:	
None	

GetOptVars

Unpublished work. © 2025 Siemens

GetOptVars	
(nSetID, bClear)	
Description:	
Gets Optimization Variables (Optimization Relationship objects) enabled for an analysis.	
Input:	
`INT4 nSetID`	ID of Set to be populated.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	
`FE_INVALID`	Analysis type is not design optimization.
`FE_NOT_EXIST`	No Optimization Variables (Relationship) have been enabled.
Remarks/Usage:	
None	
Example:	
None	

PutOptVars

PutOptVars	
(nSetID, bClear)	
Description:	
Sets Optimization Variables (Optimization Relationship objects) enabled for an analysis.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 nSetID`	ID of Set Object containing Optimization Variable (Relationship) IDs. Alternatively, specify a negative number to put a single optimization variable.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	
`FE_INVALID`	Analysis type is not design optimization.
`FE_NOT_EXIST`	No Optimization Variables (Relationship) have been enabled.
Remarks/Usage:	
None	
Example:	
None	

GetOptLimits

`GetOptLimits`
`( nSetID, bClear )`
Description:
Gets Optimization Limits (Optimization Response objects) enabled for an analysis.
Input:

`INT4 nSetID`	ID of Set Object to be populated.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_INVALID`	Analysis type is not design optimization.
`FE_NOT_EXIST`	No Optimization Limits (Responses) have been enabled.
Remarks/Usage:	
None	
Example:	
None	

PutOptLimits

`PutOptLimits` **`( nSetID, bClear )`**	
Description:	
Sets Optimization Limits (Optimization Response objects) enabled for an analysis.	
Input:	
`INT4 nSetID`	ID of Set Object containing Optimization Limits (Response) IDs. Alternatively, specify a negative number to put a single optimization variable.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	
`FE_INVALID`	Analysis type is not design optimization.
`FE_NOT_EXIST`	No Optimization Limits (Responses) have been enabled.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetOptMCs

GetOptMCs **(nSetID, bClear)**	
Description:	
Gets Topology Optimization Manufacturing Constraints enabled for an analysis.	
Input:	
`INT4 nSetID`	ID of Set Object to be populated.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	
`FE_INVALID`	Analysis type is not design/topology optimization.
`FE_NOT_EXIST`	No Manufacturing Constraints have been enabled.
Remarks/Usage:	
None	
Example:	
None	

PutOptMCs

Unpublished work. © 2025 Siemens

PutOptMCs	
(nSetID, bClear)	
Description:	
Sets Topology Optimization Manufacturing Constraints enabled for an analysis.	
Input:	
`INT4 nSetID`	ID of Set Object containing Manufacturing Constraint IDs. Alternatively, specify a negative number to put a single manufacturing constraint.
`BOOL bClear`	Clear existing entities.
Output:	
None	None
Return Code:	
`FE_INVALID`	Analysis type is not design/topology optimization.
`FE_NOT_EXIST`	No Manufacturing Constraints have been enabled.
Remarks/Usage:	
None	
Example:	
None	

GetMptSet

GetMptSet	
(pSelectedMptSet)	
Description:	
Gets Monitor Points enabled for an analysis.	
Input:	

Unpublished work. © 2025 Siemens

None	None
Output:	
`OBJECT pSelectedMptSet`	Set object that contains the IDs of the Monitor Points enabled for analysis.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutMptSet

`PutMptSet` **`( pSelectedMptSet )`**
Description:
Sets Monitor Points enabled for an analysis.
Input:

`OBJECT pSelectedMptSet`	Set Object containing the IDs of the monitor points to enable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutMptSetID

PutMptSetID **(nSetID)**	
Description:	
Alternative method for specifying Monitor Points enabled for an analysis.	
Input:	
`INT4 nSetID`	ID of Set Object containing Monitor Point IDs. Alternatively, specify a negative number to enable a single monitor point.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ClearMptSet

ClearMptSet **(void)**	
Description:	
This method clears the array that contains the IDs of the Monitor Points currently enabled for analysis.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetFLXSLISet

GetFLXSLISet **(pSelectedFLXSLISet)**	
Description:	
Gets Flexible Sliders enabled for an analysis.	
Input:	
None	None
Output:	
`OBJECT pSelectedFLXSLISet`	Set object that contains the IDs of the Flexible Sliders enabled for analysis.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutFLXSLISet

Unpublished work. © 2025 Siemens

PutFLXSLISet **(pSelectedFLXSLISet)**	
Description:	
Sets Flexible Sliders enabled for an analysis.	
Input:	
`OBJECT pSelectedFLXSLISet`	Set Object containing the IDs of the flexible sliders to enable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutFLXSLISetID

PutFLXSLISetID **(nSetID)**	
Description:	
Alternative method for specifying Flexible Sliders enabled for an analysis.	
Input:	
`INT4 nSetID`	ID of Set Object containing Flexible Slider IDs. Alternatively, specify a negative number to enable a single flexible slider.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ClearFLXSLISet

`ClearFLXSLISet`	
`( void )`	
Description:	
This method clears the array that contains the IDs of the Flexible Sliders currently enabled for analysis.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DeleteAnalysisCase

Femap API Guide
Unpublished work. © 2025 Siemens

DeleteAnalysisCase	
(nSetID, nCaseID)	
Description:	
Use this method to delete an analysis case.	
Input:	
`INT4 nSetID`	The Analysis SetID containing the case to be deleted.
`INT4 nCaseID`	The Case ID to be deleted.
Output:	
None	None
Return Code:	
`FE_FAIL`	None
Remarks/Usage:	
This method takes care of deleting any start/end text or Marc contact table records that referenced the case.	
Example:	
None	

IsGust

IsGust	
(void)	
Description:	
This method indicates if the Correlation Table in the NASTRAN Power Spectral Density Factors (SOL 146) dialog box is set to Gust PSD Input or Tabular PSD Input.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_OK`	Correlation table option is set to Gust PSD Input.
`FE_FAIL`	Correlation table option is set to Tabular PSD Input.
`FE_NOT_EXIST`	The correlation table does not exist.
Remarks/Usage:	
Only used when Analysis Type for the Analysis Set is set to **33..Aeroelastic Random Response**. When `FE_FAIL` is returned, use `GetCorrelate2` and `PutCorrelate2` methods to retrieve or set PDS information.	
Example:	
None	

GetCorrelateGust

`GetCorrelateGust` **`( gustTableID, realScale, gustPSDType, turbFactor, rmsGustVel )`**	
Description:	
This method retrieves data from the Correlation Table in the NASTRAN Power Spectral Density Factors (SOL 146) dialog box when set to Gust PSD Input.	
Input:	
None	None
Output:	
`INT4 gustTableID`	Retrieves ID of the `TABRNDG` entry exported to the Nastran input file.
`REAL8 realScale`	Retrieves value for Factor.
`INT4 gustPSDType`	Retrieves option for Gust PSD Model, 0=von Karman, 1=Dryden.

Unpublished work. © 2025 Siemens

`REAL8 turbFactor`	Retrieves value for Turbulence Factor.
`REAL8 rmsGustVel`	Retrieves value for Gust RMS Velocity.
Return Code:	
`FE_NOT_EXIST`	The correlation table does not exist.
Remarks/Usage:	
Only used when Analysis Type for the Analysis Set is set to **33..Aeroelastic Random Response**.	
Example:	
None	

PutCorrelateGust

`PutCorrelateGust` **`( gustTableID, realScale, gustPSDType, turbFactor, rmsGustVel )`**	
Description:	
This method sets option in the Correlation Table in the NASTRAN Power Spectral Density Factors (SOL 146) dialog box when set to Gust PSD Input.	
Input:	
`INT4 gustTableID`	Sets ID of the TABRNDG entry exported to the Nastran input file.
`REAL8 realScale`	Sets value for Factor.
`INT4 gustPSDType`	Sets option for Gust PSD Model, 0=von Karman, 1=Dryden.
`REAL8 turbFactor`	Sets value for Turbulence Factor.
`REAL8 rmsGustVel`	Sets value for Gust RMS Velocity.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_EXIST`	The correlation table does not exist.
Remarks/Usage:	
Only used when Analysis Type for the Analysis Set is set to **33..Aeroelastic Random Response**.	
Example:	
None	

Note

In addition to the standard methods, the following topics describe methods which are also available specifically for use with ANSYS translator.

PutOUTMGTSetID

`PutOUTMGTSetID` **`( pSelectedOUTMGTSet )`**	
Description:	
Alternative method for specifying XY Plot Definitions enabled for an analysis.	
Input:	
`OBJECT pSelectedOUTMGTSet`	Set object containing the IDs of the XY Plot Definitions to enable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetOUTMGTSet

GetOUTMGTSet **(pSelectedOUTMGTSet)**	
Description:	
Gets XY Plot Definitions enabled for an analysis.	
Input:	
`OBJECT pSelectedOUTMGTSet`	Set object that contains the IDs of the XY Plot Definitions enabled for analysis.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ClearOUTMGTSet

ClearOUTMGTSet **(Void)**	
Description:	
This method clears the array that contains the IDs of the XY Plot Definitions currently enabled for analysis.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutOUTMGTSet

`PutOUTMGTSet` **`( pSelectedOUTMGTSet )`**	
Description:	
Sets XY Plot Definitions enabled for an analysis.	
Input:	
`OBJECT pSelectedOUTMGTSet`	Set Object containing the IDs of the XY Plot Definitions to enable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutOMEGASetID

Unpublished work. © 2025 Siemens

PutOMEGASetID	
(pSelectedOMEGASet)	
Description:	
Sets Rotational Speed Definitions enabled for an analysis.	
Input:	
`OBJECT pSelectedOMEGASet`	Set object containing the IDs of the Rotational Speed Definitions to enable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetOMEGASet

GetOMEGASet	
(pSelectedOMEGASet)	
Description:	
Gets Rotational Speed Definitions enabled for an analysis.	
Input:	
`OBJECT pSelectedOMEGASet`	Set object that contains the IDs of the Rotational Speed Definitions enabled for analysis.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ClearOMEGASet

```
ClearOMEGASet

( Void )
```

Description:	
This method clears the array that contains the IDs of the Rotational Speed Definitions currently enabled for analysis.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutOMEGASet

Unpublished work. © 2025 Siemens

PutOMEGASet	
(pSelectedOMEGASet)	
Description:	
Sets Rotational Speed Definitions enabled for an analysis.	
Input:	
`OBJECT pSelectedOMEGASet`	Set Object containing the IDs of the Rotational Speed Definitions to enable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnalysisMgr Objects - ANSYS Specific Methods

KBC

KBC	
(key)	
Description:	
This method sets the **Amplitude (KBC)** option on ANSYS Load Step Options dialog box.	
Input:	
`BOOL key`	When True, activates **Amplitude (KBC)** option.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PSTRES

PSTRES	
(key)	
Description:	
This method sets the **Include Prestress Effects (PSTRES)** option on ANSYS Load Step Options dialog box.	
Input:	
`BOOL key`	When True, activates **Include Prestress Effects (PSTRES)** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

TIME

Unpublished work. © 2025 Siemens

TIME	
(time)	
Description:	
This method sets the Time Duration value on ANSYS Time Step Options dialog box.	
Input:	
`REAL8 time`	Time Duration value.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AUTOTS

AUTOTS	
(key)	
Description:	
This method sets the **Automatic Time Stepping (AUTOTS)** option on ANSYS Load Step Options dialog box.	
Input:	
`BOOL key`	When True, activates **Automatic Time Stepping (AUTOTS)** option.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

LANBOPTION

LANBOPTION **(strmck)**	
Description:	
This method sets the **Strum Sequence Check** option on ANSYS Modal Analysis dialog box for use with the Block Lanczos mode extraction method.	
Input:	
`BOOL key`	When True, activates **Strum Sequence Check** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PCGOPT

Unpublished work. © 2025 Siemens

PCGOPT	
(strmck)	
Description:	
This method sets the **Strum Sequence Check** option on ANSYS Modal Analysis dialog box for the PCG Lanczos mode extraction method.	
Input:	
`BOOL key`	When True, activates **Strum Sequence Check** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

SUBOPT

SUBOPT	
(strmck)	
Description:	
This method sets the **Strum Sequence Check** option on ANSYS Modal Analysis dialog box for use the Subspace mode extraction method.	
Input:	
`BOOL key`	When True, activates **Strum Sequence Check** option.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

LUMPM

LUMPM **(key)**	
Description:	
This method sets the **Lumper** option on ANSYS Transient Dynamics dialog box.	
Input:	
`BOOL key`	When True, activates **Lumped** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ALPHAD

Unpublished work. © 2025 Siemens

ALPHAD	
(alpha)	
Description:	
This method sets the Mass value (Damping) on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 alpha`	Mass value (Damping).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

BETAD

BETAD	
(beta)	
Description:	
This method sets the Stiffness value (Damping) on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 beta`	Stiffness value (Damping).
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

DMPSTR

DMPSTR **(dmp)**	
Description:	
This method sets the Struct value (Damping) on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 dmp`	Struct value (Damping).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DMPRAT

Unpublished work. © 2025 Siemens

DMPRAT	
(rat)	
Description:	
This method sets the Ratio value (Damping) on ANSYS Transient Analysis dialog box.	
Input:	
`REAL8 rat`	Ratio value (Damping).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

NEQIT

NEQIT	
(neqit)	
Description:	
This method sets the Max Iterations value on ANSYS Nonlinear dialog box.	
Input:	
`INT4 neqit`	Max Iterations value.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

NLGEOM

NLGEOM **(key)**	
Description:	
This method sets the **Nonlinear Geometry** option on ANSYS Nonlinear dialog box.	
Input:	
`BOOL key`	When True, activates **Nonlinear Geometry** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ARCLEN

Unpublished work. © 2025 Siemens

ARCLEN	
(key)	
Description:	
This method sets the **Arc-Length** option on ANSYS Nonlinear dialog box.	
Input:	
`BOOL key`	When True, activates **Arc-Length** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

LNSRCH

LNSRCH	
(key)	
Description:	
This method sets the **Line Search** option on ANSYS Nonlinear dialog box.	
Input:	
`BOOL key`	When True, activates **Line Search** option.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

PRED

PRED **(sskey)**	
Description:	
This method sets the **Predictor** option on ANSYS Nonlinear dialog box.	
Input:	
`BOOL sskey`	When True, activates **Predictor** option.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

SPOPT

Unpublished work. © 2025 Siemens

SPOPT

(sptype, nmode, elcalc)

Description:	
This method sets the **Spectrum** options on ANSYS Random Vibration Analysis Options dialog box.	
Input:	
`INT4 sptype`	ID of the spectrum type option.
`INT4 nmode`	Specifies the number of modes.
`BOOL elcalc`	When True, Element Results is on.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PSDUNIT

PSDUNIT

(tblno, type)

Description:	
This method sets the PSD table on the ANSYS Random Vibration Analysis Options dialog box.	
Input:	
`INT4 tblno`	ID of the function, 0=DISP, 1=VEL, 2=ACEL (Acc2/Hz), 3=ACCG (g2/Hz), 4=FORC, 5=PRES.

Unpublished work. © 2025 Siemens

`INT4 type`	Specifies the PSD type.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PFACT

PFACT **(tblno, excit)**	
Description:	
This method sets the PSD table on the ANSYS Random Vibration Analysis Options dialog box.	
Input:	
`INT4 tblno`	ID of the function, 0=DISP, 1=VEL, 2=ACEL (Acc2/Hz), 3=ACCG (g2/Hz), 4=FORC, 5=PRES.
`BOOL excit`	Sets the Mode Factors, True = Node, False = Base.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

PSDCOM

PSDCOM **(signif, comode, forcetype)**	
Description:	
This method sets the **Mode Combinations** options on the ANSYS Random Vibration Analysis Options dialog box.	
Input:	
`REAL8 signif`	Sets the Significance value.
`INT4 comode`	Sets the Mode Number value.
`BOOL forcetype`	Sets the Combine Force, True = Static, False = Total.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsWriteGroup

Unpublished work. © 2025 Siemens

AnsWriteGroup	
(group)	
Description:	
This method specifies a Group ID for the **Portion of Model to Write** option in the ANSYS Command and Model Options dialog box.	
Input:	
`INT4 group`	ID of Group to write to the input file. To write out the entire model, set this value to 0.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipCSys

AnsSkipCSys	
(key)	
Description:	
This method specifies setting for **Skip CSys** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipNode

AnsSkipNode **(key)**	
Description:	
This method specifies setting for **Skip Node** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipElem

Unpublished work. © 2025 Siemens

AnsSkipElem	
(key)	
Description:	
This method specifies setting for **Skip Elem** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipMatl

AnsSkipMatl	
(key)	
Description:	
This method specifies setting for **Skip Matl** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipProp

AnsSkipProp **(key)**	
Description:	
This method specifies setting for **Skip Prop** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipGroup

Unpublished work. © 2025 Siemens

AnsSkipGroup	
(key)	
Description:	
This method specifies setting for **Skip Group** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsSkipMisc

AnsSkipMisc	
(key)	
Description:	
This method specifies setting for **Skip Misc** option in the ANSYS Command and Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnsRSTComp

AnsRSTComp **(level)**	
Description:	
This method specifies the **File Compression Level** option in the ANSYS Command and Model Options dialog box.	
Input:	
`INT4 level`	Specifies the value for **File Compression Level**: 0=Sparse, 1=No Compression
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnalysisMgr Objects - LS-Dyna Specific Methods

Unpublished work. © 2025 Siemens

DynWriteGroup

DynWriteGroup	
(group)	
Description:	
This method specifies a Group ID for the **Portion of Model to Write** option in the LS-DYNA Model Options dialog box.	
Input:	
`INT4 group`	ID of Group to write to the input file. To write out the entire model, set this value to 0.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DynSkipCSys

DynSkipCSys
(key)
Description:
This method specifies setting for **Skip CSys** option in the LS-DYNA Model Options dialog box.
Input:

Unpublished work. © 2025 Siemens

`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DynSkipNode

`DynSkipNode`	
`( key )`	
Description:	
This method specifies setting for **Skip Node** option in the LS-DYNA Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

DynSkipElem

DynSkipElem	
(key)	
Description:	
This method specifies setting for **Skip Elem** option in the LS-DYNA Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DynSkipMatl

DynSkipMatl	
(key)	
Description:	
This method specifies setting for **Skip Matl** option in the LS-DYNA Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DynSkipProp

DynSkipProp **(key)**	
Description:	
This method specifies setting for **Skip Prop** option in the LS-DYNA Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DynSkipGroup

Unpublished work. © 2025 Siemens

DynSkipGroup	
(key)	
Description:	
This method specifies setting for **Skip Group** option in the LS-DYNA Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DynSkipMisc

DynSkipMisc	
(key)	
Description:	
This method specifies setting for **Skip Misc** option in the LS-DYNA Model Options dialog box.	
Input:	
`BOOL key`	When True, option is activated. When False, option is deactivated.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AnalysisStep Objects

AnalysisStep objects correspond to the analysis steps within subcases which are within analysis sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for AnalysisStep objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAnalysisStep` method of the Femap Application object to create AnalysisStep objects. Alternatively, you can create an AnalysisStep object by calling the `GetStep` method of the AnalysisCase object. This approach has the added advantage of ensuring that the step is from the correct analysis case, and preloading the step data.

Note

Currently, AnalysisStep Objects are only used for MSC Nastran Implicit Nonlinear Analysis (SOL 400).

AnalysisStep Object Properties

AnalysisStep Objects are stored with their SetID equal to the ID of the analysis set, CaseID equal to the analysis case, and StepID equal to the analysis step that you are trying to access. Note that the neither the analysis set, nor subcase is not accessed from an AnalysisStep object. Those properties are directly available from the AnalysisMgr object or AnalysisCase object, respectively.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
Analysis Step Options - General	
`STRING StepTitle[80]`	Title of analysis step (maximum 80 characters).

Unpublished work. © 2025 Siemens

Property	Description
`STRING StepLabel[80]`	Label of analysis step (maximum 80 characters).
`INT4 CaseID`	ID of analysis case (subcase).
`INT4 InternalStepID`	ID of analysis step stored within Femap. It is highly recommended to simply use `NextExmptyID` when creating a new step to avoid accidentally overwriting an existing step.
`INT4 StepID`	ID of analysis step as seen in the user interface (i.e., ID of Step when written to the MSC Nastran SOL 400 input file).
`BOOL UseGlobalStep`	Flag to use Global Step. True = Use Global Step.
`INT4 GlobalStepID`	ID of Global Step to use when `UseGlobalStep` is True.
`enum StepAnalysisType`	Analysis Type for Step (only used by certain solvers for certain analysis types). See **Analysis Types** in **Femap Constants** for available enumerations.

`INT4 BCSet[0..20]`
or
`VARIANT vBCSet`
	SetIDs of load and constraint sets for this step: [0]=Constraints, [1]=Constraint Equations, [2]=Loads, [3]=Initial Conditions, [4]=Master DOF, [5]=Base/Support DOF, [6]=Omit DOF, [7]=QSET DOF, [8]=CSET DOF, [9]=BSET DOF, [10]=Bolt Preloads, [11]=SUPORT1 DOF, [12]=Temperatures, [13]=Connection Group (MSC Only), [14]=Loads (Time), [15]=Temperatures (Init), [16]=Glue Set, [17]=Contact Set, others not currently used.

`INT4 Output[0..30]`
or
`VARIANT vOutput`
	Output requests for this step: 0=off,-1=full model, >0=groupID, [0]=Nodal Applied Load, [1]=Nodal Reaction Force, [2]=Nodal Displacement, [3]=Nodal Velocity, [4]=Nodal Acceleration, [5]=Nodal Temperature, [6]=Nodal Heat Flux, [7]=Nodal Constraint Equation Forces, [8]=Nodal Force Balance, [9]=Nodal Kinetic Energy, [15]=Element Force, [16]=Element Stress, [17]=Element Strain, [18]=Element Plastic Strain, [19]=Element Strain Energy, [20]=Element Heat Flux, [21]=Element Enthalpy, [22]=Element Enthalpy Rate, [23]=Element Contact, [24]=Element Temperature, [25]=Kinetic Energy, [26]=Energy Loss, [27]=Fluid Pressure, others not currently used.

Unpublished work. © 2025 Siemens

Property	Description
`STRING Echo[80]`	String for ECHO option (maximum 80 characters).
`BOOL SkipStandard`	Flag to skip standard output for this step.
`BOOL NasStepAlternateTextLocation`	Flag to use **Start Text Inside Step** option.
`INT4 StartText`	ID of starting text.
`INT4 EndText`	ID of ending text.
`INT4 Destination`	Output destination: 0=Default, 1=Print, 2=Post, 3=Print+Post, 4=Punch, 5=Punch+Post.
`INT4 Imaginary`	Format for complex output: 0=Magnitude/Phase, 1=Real/Imaginary.
`INT4 CornerOutput`	Flag to request output at element corners.
`INT4 IntegrationOutput`	Flag to request output at element integration points.
`BOOL NasCurvatureStrain`	Flag to request Curvature Strain Results. True = Curvature, False = Fiber.
NASTRAN Step Options	
Analysis Step Options - NASTRAN Specific	
`BOOL NasOmodesOn`	Flag indicating an OMODES request has been defined.
`STRING NasOmodes[160]`	Expression used to select OMODES output.
Analysis Case Options - Monitor Points	
`BOOL NasCaseMonitorEnabled`	Flag indicating Monitor Point Output Requests will be written to the analysis case, when applicable.
`BOOL NasCaseMonitorAeroEnabled`	Flag to set **Print Aero Monitor Results** option.
`BOOL NasCaseMonitorLoadEnabled`	Flag to set **Print Load Monitor Results** option.
NASTRAN Modal Analysis - Step options	
`BOOL NasModeOn`	Flag indicating NASTRAN Modal Options have been defined for subcase.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasModeEigrSet`	SetID to be used for EIGR.
`INT4 NasModeMethod`	Modal Solution method: 0=Givens,1=Modified Givens, 2=Inverse Power, 3=Inverse Power/ Strum, 4=Householder, 5=Modified Householder, 6=Lanczos, 7=Hessenberg, 8=Complex Inverse Power, 9=Complex Lanczos.
`BOOL NasModeSolutionType`	Solution type: False=Direct, True=Modal.

`REAL8 NasModeFreqRange[2]`
or
`VARIANT vNasModeFreqRange`
	Frequency range of interest, [0]=Min, [1]=Max.

`INT4 NasModeEstRoots`	Estimated number of eigenvalues.
`INT4 NasModeDesiredRoots`	Number of desired eigenvalues/modes.
`INT4 NasModeNormOpt`	Normalization method: 0=Mass,1=Max, 2=Point.
`INT4 NasModeNormNode`	Node ID for use with point normalization method.
`INT4 NasModeNormDOF`	Nodal DOF for use with point normalization method.
`INT4 NasModeMassForm`	Mass matrix formulation: 0=Default, 1=Lumped, 2=Coupled.
`BOOL NasModesSkipEigr`	If True, skips writing the EIGR card for specifying modal analysis options.

`REAL8 NasModeImagFreqRange[2]`
or
`VARIANT vNasModeImagFreqRange`
	For Complex Modes analysis, specifies the imaginary part of the Frequency Range of interest.

`REAL8 NasModesComplexConverge`	Specifies Complex Modes convergence value.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasModesComplexRegionWidth`	Specifies Region Width for Complex Modes analysis.
`REAL8 NasModeDampOverall`	Specifies PARAM,G.
MSC NASTRAN Implicit Nonlinear (SOL 400) - Control Parameters	
`BOOL NasInlStepOptOn`	Flag indicating Control Parameters for Implicit Nonlinear Analysis have been defined for the subcase.
`BOOL NasInlStepOptSkipNLSTEP`	Flag to disable writing of NLSTEP for this subcase.
`INT4 NasInlStepOptID`	ID of NLSTEP for this subcase.
Control Parameters - Step Parameters	
`REAL8 NasInlStepOptTOTTIM`	Total Time (TOTTIM) value.
`INT4 NasInlStepOptCTRLDEF`	Specifies Nonlinearity (CTRLDEF) option: 0=Default 1=Linear 2=Mildly Nonlinear 3=Severely Nonlinear
Control Parameters - General Parameters	
`INT4 NasInlStepOptMAXITER`	Max Iterations/Increment (MAXITER) value.
`INT4 NasInlStepOptMINITER`	Min Iterations/Increment (MINITER) value.
`INT4 NasInlStepOptMAXBIS`	Max Bisections for Step (MAXBIS) value.
`BOOL NasInlStepOptCREEP`	Flag to activate **Include Creep (CREEP)** option.
Control Parameters - Analysis Parameter	
`INT4 NasInlStepOptLoadStepping`	Specifies Increment Type option: 0=Fixed 1=Adaptive 2=Arc-Length
Control Parameters - Analysis Parameters - Fixed Time Stepping Parameters	

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptNINC`	Number of Increments (NINC) value.
`INT4 NasInlStepOptNO`	Output Interval (NO) value.
Control Parameters - Analysis Parameters - Adaptive Load Stepping Parameters	
`REAL8 NasInlStepOptDTINITF`	Initial Fraction (DTINITF) value.
`REAL8 NasInlStepOptDTMINF`	Min Fraction (DTMINF) value.
`REAL8 NasInlStepOptDTMAXF`	Max Fraction (DTMAXF) value.
`REAL8 NasInlStepOptSFACT`	Increase Factor (SFACT) value.
`INT4 NasInlStepOptADJUST`	Auto Skip Factor (ADJUST) value.
`INT4 NasInlStepOptMSTEP`	Dominant Period Resp. Steps (MSTEP) value.
`REAL8 NasInlStepOptRB`	Adaptive Time Step Bound (RB) value.
`INT4 NasInlStepOptNDESIR`	Desired Iterations/Increments (NDESIR) value.
`INT4 NasInlStepOptNSMAX`	Max Increments (NSMAX) value.
`REAL8 NasInlStepOptUTOL`	Displacement Tolerance (UTOL) value.
`INT4 NasInlStepOptIDAMP`	Specifies **Static Damping (IDMAP)** option: 0=No Added Damping 4=Time Step Control Using Damping 5=Damping-based Time Step Control with No Damping 6=Add Damping when Minimum Time Step is Reached
`REAL8 NasInlStepOptDAMP`	Damping Ratio (DAMP) value.
`INT4 NasInlStepOptINTOUT`	Specifies **Output Flag (INTOUT)** option: -1=Last Increment 0=Every Load Increment

Unpublished work. © 2025 Siemens

Property	Description
	>0=Equally Spaced Intervals (value is used to specify value for Output Every Nth Step)
Adaptive Load Stepping Parameters - Criteria	
`BOOL NasInlStepOptEnableCriteria`	Flag to activate the **Enable User Criteria** option.
`INT4 NasInlStepOptIPHYS`	Specifies **Physical Criteria (IPHYS)** option: -2=No Auto Physical Criteria; Continue When User Criteria Exceeded -1=Add Auto Physical Criteria; Continue When User Criteria Exceeded 1=1..No Auto Physical Criteria; Stop When User Criteria Exceeded 2=2..Add Auto Physical Criteria; Stop When User Criteria Exceeded
`INT4 NasInlStepOptLIMTAR`	Specifies **Definition (LIMTAR)** option: 0=Criteria as Limits 1=Criteria as Targets
`REAL8 NasInlStepOptRSMALL`	Min. Ratio (RSMALL) value.
`REAL8 NasInlStepOptRBIG`	Max Ration (RBIG) value.
`BOOL NasInlStepOptIncrements[0..9]`	Flag in Status column of row specified by zero-based index value in list of Criteria. When True, Criterion is activate. 0=Elem. Strain 1=Elem. Plastic Strain 2=Elem. Creep Strain 3=Elem. Normalized Creep Strain 4=Elem. Strain Energy 5=Elem. Stress 6=Elem. Normalized Stress 7=Nodal Temperature 8=Nodal Displacement 9=Nodal Rotation
`INT4 NasInlStepOptGroupIDs[0..9]`	ID of existing Group in Group column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasInlStepOptYT1_i[0..9]`	Value in Y1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT1_i[0..9]`	Value in M1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT2_i[0..9]`	Value in Y2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT2_i[0..9]`	Value in M2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT3_i[0..9]`	Value in Y3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT3_i[0..9]`	Value in M3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT4_i[0..9]`	Value in Y4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT4_i[0..9]`	Value in M4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
Control Parameters - Analysis Parameters - Arc-Length Load Stepping Parameters	
`INT4 NasInlStepOptTYPE`	Specifies **Constraint Type** option: 0=Crisfield 1=Riks 2=Modifed Riks
`REAL8 NasInlStepOptDTINITFA`	Initial Time Step Fraction (DTINITFA) value.
`REAL8 NasInlStepOptMINALR`	Min ArcLenAdjust Ratio (MINALR) value.
`REAL8 NasInlStepOptMAXALR`	Max ArcLenAdjust Ratio (MAXALR) value.
`REAL8 NasInlStepOptSCALEA`	Scale Factor (SCALEA) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptNDESIRA`	Desired Iterations (NDESIRA) value.
`INT4 NasInlStepOptNSMAXA`	Max Increments (NSMAXA) value.
Control Parameters - Analysis Parameters - Mechanical Analysis Parameters	
`BOOL NasInlStepOptMECH`	Flag to activater the **Enable MECH** option.
`INT4 NasInlStepOptKMETHOD`	Specifies **Method (KMETHOD)** option: 0=Full Newton-Raphson 1=Quasi Newton-Raphson
`INT4 NasInlStepOptKSTEP`	Iteration Interval (KSTEP) value.
`INT4 NasInlStepOptMAXQN`	QN Vectors (MAXQN) value.
`INT4 NasInlStepOptMAXLS`	Max Line Searches (MAXLS) value.
`REAL8 NasInlStepOptLSTOL`	Line Search Tol (LSTOL) value.
`REAL8 NasInlStepOptFSTRESS`	Fraction Limit (FTRESS) value.
`BOOL NasInlStepOptConvU`	Flag to activate **Displacement** option.
`REAL8 NasInlStepOptEPSU`	Displacement value.
`BOOL NasInlStepOptConvP`	Flag to activate **Load** option.
`REAL8 NasInlStepOptEPSP`	Load value.
`BOOL NasInlStepOptConvW`	Flag to activate **Work** option.
`REAL8 NasInlStepOptEPSW`	Work value.
`BOOL NasInlStepOptConvV`	Flag to activate **Vector** option.
`BOOL NasInlStepOptConvN`	Flag to activate **Length** option.
`BOOL NasInlStepOptConvA`	Flag to activate **Automatic Switching** option.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptMRCONV`	Specifies **Checking (MRCONV)** option: 0=Forces, Moments, Displacements, and Rotations 1=Forces, Moments, and Displacements 2=Forces, Displacements, and Rotations 3=Forces and Displacements
MSC Nastran Boundary Conditions	
`INT4 ContactOption`	Boundary Conditions dialog box for MSC Nastran Only. Sets **BCONTACT** Option (0=Default, 1=Inital Contact, 2=All Body, 3=Auto).
`INT4 ContactSetType`	Boundary Conditions dialog box for MSC Nastran Only. Sets **Contact/Glue Sets** option (0=All Connectors, 1=Connection Group, 2=All Body). ID of group for Connection Group set using `BCSet[13]`.

AnalysisStep Object Methods

In addition to the standard methods, the following methods are also available.

InitAnalysisStep

`InitAnalysisStep` **`( nSetID, nCaseID, nStepID )`**	
Description:	
This method initializes the default values for an analysis step.	
Input:	
`INT4 nSetID`	The Analysis Set ID (SetID) containing the case to be initialized.
`INT4 nCaseID`	The Analysis Case ID (CaseID) containing the step to be initialized.

Unpublished work. © 2025 Siemens

`INT4 nStepID`	The Analysis Step ID (StepID) to be initialized.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to initialize the case. Probably the Analysis Set or Analysis Case you specified does not exist.
Remarks/Usage:	
None	
Example:	
None	

DeleteAnalysisStep

`DeleteAnalysisStep` **`( nSetID, nCaseID, nStepID )`**	
Description:	
Use this method to delete an analysis step.	
Input:	
`INT4 nSetID`	The Analysis Set ID (SetID) containing the case to be deleted.
`INT4 nCaseID`	The Analysis Case ID (CaseID) containing the step to be initialized.
`INT4 nStepID`	The Analysis Step ID (StepID) to be deleted.
Output:	
None	None
Return Code:	
`FE_FAIL`	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method takes care of deleting any start/end text or Marc contact table records that referenced the case.
Example:
None

ActiveContactEntities

`ActiveContactEntities` **`( bContact, nConnectorSetID, nRegionSetID, nConPropSetID )`**	
Description:	
This method is used to populate Set Objects with the Connectors, Connection Regions, and Connection Properties that will be exported for Contact or Glue Contact by an analysis step. This method takes into consideration the settings in the **Contact Sets or Glue Sets** section of the Boundary Conditions dialog box for Simcenter SOL 401/402 or setting for **Contact/Glue Sets** of the Boundary Conditions dialog box for MSC Nastran.	
Input:	
`BOOL bContact`	If True, populates sets with connection entities for Contact. If False, populates sets with connection entities for Glued Contact.
`INT4 nConnectorSetID`	Set ID of Set Object to populate with Connectors which will be exported by this analysis step.
`INT4 nRegionSetID`	Set ID of Set Object to populate with Connection Regions which will be exported by this analysis step.
`INT4 nConPropSetID`	Set ID of Set Object to populate with Connection Properties which will be exported by this analysis step.
Output:	
None	None
Return Code:	
`FE_FAIL`	No Connection Regions were detected.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

It is important to use three uniques Set Objects when using this method, otherwise the IDs of different entity types may contaminate a Set. Also, it is recommend to use *empty sets* to ensure that only the Connection entities from a single analysis step populate the three sets.
Example:
None

AnalysisStudy Objects

AnalysisStudy objects correspond to the analysis studies in your model. They are derived from Entity objects, so in addition to the properties and methods described here, essentially all of the properties and methods defined for Entity objects are also available for AnalysisStudy objects. For more information, see **Common Entity Properties and Methods**.

Use the `feAnalysisStudy` method of the Femap Application object to create AnalysisStudy objects.

AnalysisStudy Object Properties

AnalysisStudy objects are always stored with their SetID = 1, and the ID equal to the analysis study that you are trying to access. AnalysisStudy objects do not contain any of the actual results data or output sets; they simply contain the header information for the analysis study.

Output Sets are *added* to an analysis study by specifying the Analysis Study ID on the output set (*study* property on the OutputSet object).

Property	Description
`STRING Title`	Analysis Study title (maximum 79 characters).
`INT4 AnalysisProg`	Analysis program where output came from: 0=Unknown, 4=MSC/NASTRAN, 5=ANSYS, 16=ABAQUS, 28=LS-Dyna, 29=MSC.Marc, 31=NE/Nastran, 34=FEMAP Structural. 36=NX Nastran. Use enum `zAnalysisProgram`.
`INT4 AnalysisType`	Type of analysis the results came from: 0=None, 1=Static, 2=Normal Modes/Eigenvalue, 3=Transient Dynamic/Time History, 4=Frequency/Harmonic Response, 5=Response Spectrum, 6=Random Response, 7=Buckling, 8=Design Opt, 9=Explicit Transient Dynamics, 10=Nonlinear Static, 11=Nonlinear Buckling, 12=Nonlinear Transient, 13=Implicit Transient Dynamics, 17=Fatigue,

Unpublished work. © 2025 Siemens

Property	Description
	19=Computational Fluid Dynamics, 20=Steady-State Heat Transfer, 21=Transient Heat Transfer, 22=Advanced Nonlinear Static, 23=Advanced Nonlinear Transient, 24=Advanced Nonlinear Explicit, 25=Static Aeroelasticity, 26=Aerodynamic Flutter. Use enum `zAnalysisType`.
`INT4 FileTime`	Returns time as an integer which can be mapped to *Windows Time* (time_t data type) using certain programming languages.
`STRING Notes`	One or more lines of text. Maximum 256 characters. Each line is separated by a CR-LF (0x0D, 0x0A) character pair. The end of the string is terminated by a NULL character.
`INT4 AnalysisSet`	ID of the Femap Analysis Set, if the input file created using the Analysis Set Manager and automatic creation of studies is enabled. If 0, the input file was likely created in a different manner or the analysis study was created manually.

AnalysisStudy Object Methods

CountOutputSets

`CountOutputSets` **`( void )`**	
Description:	
This method returns the number of output sets which reside in an analysis study.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`INT4 count`	The number of output sets in the analysis study currently in the AnalysisStudy object. If 0, no output sets are in the analysis study.
Remarks/Usage:	
None	
Example:	
None	

OutputSets

`OutputSets` **`( nOuptutSetSetID )`**	
Description:	
This method loads an existing Set object with the output set IDs for the analysis study currently loaded in the AnalysisStudy object.	
Input:	
`INT4 nOutputSetSetID`	The ID of a Set object to be filled with output set IDs currently residing an analysis study.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

HasOutputSet

Unpublished work. © 2025 Siemens

HasOutputSet	
(nSetID)	
Description:	
This method returns a Boolean indicating if the specified output set reside in the analysis study currently loaded in the AnalysisStudy object.	
Input:	
`INT4 nSetID`	The ID of an output set.
Output:	
None	None
Return Code:	
`BOOL`	True = output set resides in analysis study, False = output set does not reside in analysis study.
Remarks/Usage:	
None	
Example:	
None	

AddOutputSets

AddOutputSets	
(nSetID)	
Description:	
The method adds all output sets in a set object to an analysis study.	
Input:	
`INT4 nSetID`	The ID of a Set that contains output set IDs to add to analysis study. Alternatively, if you specify a negative value, this is simply the ID of a single output set to add to the analysis study.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

RemoveOutputSets

`RemoveOutputSets` **`( nSetID )`**	
Description:	
The method removes all output sets in a set object from an analysis study.	
Input:	
`INT4 nSetID`	The ID of a Set that contains output set IDs to remove from analysis study. Alternatively, if you specify a negative value, this is simply the ID of a single output set to remove from the analysis study.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

RemoveAllOutputSets

RemoveAllOutputSets	
(void)	
Description:	
The method removes all output sets from an analysis study.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Delete

Delete	
(nSetID)	
Description:	
The method deletes all analysis studies in a set object from the model and optionally delete the output sets in those studies from the model at the same time.	
Input:	
`INT4 nSetID`	The ID of a Set that contains analysis study IDs to delete from the model. Alternatively, if you specify

Unpublished work. © 2025 Siemens

	a negative value, this is simply the ID of a single analysis set to delete from the analysis study.
`BOOL bDeleteOutput`	When True, will also delete any output sets currently residing in an analysis study which is deleted. When False, output sets will remain in the model.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

BCDefinition Objects

BCDefinition objects correspond to the constraint definitions in your model. They group individual constraints into related sets and provide a title for the set. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for BCDefinition objects. For more information, see **Common Entity Properties and Methods**.

Use the `feBCDefinition` method of the Femap Application object to create BCDefinition objects.

BCDefinition Object Properties

BCDefinition objects are always stored with their SetID equal to the setID of the constraint set; however, their IDs simply go from 1 to N. To find a particular object, you must retrieve each object in the set and check the constraints it contains.

Property	Description
`STRING title`	Constraint Definition title.
`INT4 OnType`	The type of entity that the constraint is applied to in this constraint definition. For more information

Unpublished work. © 2025 Siemens

Property	Description
	on Entity Types refer to the table in **Entity Types**. This must be `FT_POINT`, `FT_CURVE`, `FT_SURFACE` or `FT_NODE`.
`INT4 DataType`	The type of entities that are referenced by this constraint definition. For more information on Entity Types refer to the table in **Entity Types**. This must be `FT_BCO`, `FT_BCO_GEOM` or `FT_BEQ`.

BCDefinition Object Methods

PutAll

`PutAll` **`( entID, enDataType, enOnType, sTitle  )`**	
Description:	
This method stores a Constraint Definition with the specified ID along with all the available options.	
Input:	
`INT4 entID`	The ID of the Constraint Definition to store.
`INT4 DataType`	Refer to the property descriptions for each of these items.
`INT4 OnType`	
`STRING Title`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.
Example:
None

ResetNextBC

`ResetNextBC`	
`( void )`	
Description:	
This method initializes all options prior to calling the `NextBC` method below.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
This method is used prior to calling `NextBC` to specify which constraints will be retrieved. It simply resets this object so that the next call to `NextBC` will return the first constraint available in the object.	
Example:	
None	

NextBC

Unpublished work. © 2025 Siemens

NextBC	
(pDISPATCH)	
Description:	
This method returns the next available constraint that is contained in the Constraint Definition.	
Input:	
None	None
Output:	
`OBJECT pDISPATCH`	The next constraint object. Depending on the type of the constraint definition this could be a BCNode, BCGeom or BEqn object.
Return Code:	
`FE_FAIL`	There are no more constraints of the specified type to retrieve. Call `ResetNextBC` to go back to the first constraint, if you want to retrieve the constraints again.
Remarks/Usage:	
This method, coupled with `ResetNextBC` provides an easy way to retrieve all constraints from a Constraint Definition. For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.	
Example:	
None	

CountBCs

CountBCs	
(void)	
Description:	

Unpublished work. © 2025 Siemens

This method returns the number of constraints referenced by a constraint definition, as a return code.	
Input:	
None	None
Output:	
None	None
Return Code:	
`INT4 numBCs`	The number of constraints referenced by this constraint definition.
Remarks/Usage:	
This method relies on the current Constraint Definition ID to count the referenced constraints.	
Example:	
None	

BCEqn Objects

BCEqn objects correspond to the constraint equations in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for BCEqn. For more information, see **Common Entity Properties and Methods**.

Use the `feBCEqn` method of the Femap Application object to create BCEqn objects.

BCEqn Object Properties

BCEqns are always stored with their SetID equal to the setID of the constraint set; however, their IDs simply go from 1 to N. To find a particular object, you must retrieve each object in the set and check its nodes.

When you create a new BCEqn, it will always reference the active constraint set. You can change the set that it references simply by changing the setID parameter.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description
Description:	

`INT4 dof[0..5999]`
 	
or
 	
`VARIANT vdof`
	An integer from 1 to 6 indicating the DOF associated with the corresponding term in the equation.

`INT4 node[0..5999]`
 	
or
 	
`VARIANT vnode`
	The node ID referenced by each term in the equation.

`REAL8 coefficient[0..5999]`
 	
or
 	
`VARIANT vcoefficient`
	The coefficient associated with each term in the equation.

`INT4 color`	The constraint color.
`INT4 layer`	The ID of the layer associated with the constraint.
`INT4 BCDefinitionID`	The ID of the constraint definition that contains this constraint (0=None).
Additional Properties	
`INT4 count`	The number of nonzero terms in the equation. This property is read-only and cannot be changed.

BCEqn Object Methods

GetAll

Unpublished work. © 2025 Siemens

GetAll

(nID, nSetID, pnDefID, pnCount, pvNode, pvDof, pvCoeff, peColor, pnLayer)

Description:

Retrieves all constraint equation properties in a single call.

Input:

`INT4 nID`	ID of Constraint equation.
`INT4 nSetID`	ID of Constraint set.

Output:

`INT4 pnDefID`	ID of Constraint definition.
`INT4 pnCount`	Number of terms in constraint equation.
`INT4 pvNode [0..nCount-1]`	Array containing node IDs.
`INT4 pvDof [0..nCount-1]`	Array containing DOFs.
`REAL8 pvCoeff [0..nCount-1]`	Array containing Coefficients.
`INT4 peColor`	Color of constraint equation.
`INT4 pnLayer`	Layer of constraint equation.

Return Code:

`FE_NOT_EXIST`	Constraint equation does not exist in the specified constraint set.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

PutAll

PutAll **(nID, nSetID, nDefID, nCount, vNode, vDof, vCoeff, eColor, nLayer)**	
Description:	
Sets all constraint equation properties in a single call.	
Input:	
`INT4 nID`	ID of Constraint equation.
`INT4 nSetID`	ID of Constraint set. If non-existing, one is created.
`INT4 nDefID`	ID of Constraint definition. If non-existing, one is created. If existing, must be a nodal constraint equation definition.
`INT4 nCount`	Total number of terms in the constraint equation.
`INT4 vNode [0..nCount-1]`	Array containing node IDs.
`INT4 vDof [0..nCount-1]`	Array containing DOFs, corresponding to each node.
`REAL8 vCoeff [0..nCount-1]`	Array containing Coefficients, corresponding to each node.
`INT4 eColor`	Color of constraint equation.
`INT4 nLayer`	Layer of constraint equation.
Output:	
None	None
Return Code:	
`FE_FAIL`	Specified layer doesn't exist.
`FE_INVALID`	Set or Definition ID is invalid.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	One or more nodes in the node array does not exist.
`FE_TOO_SMALL`	Insufficient terms (< 2) have been supplied for the constraint equation.
`FE_BAD_DATA`	Array for nodes, DOFs, or coefficients is invalid.
`FE_BAD_TYPE`	Definition ID exists in the current constraint set, however is invalid for constraint equations.
Remarks/Usage:	
None	
Example:	
None	

NextOnNode

`NextOnNode` **`( nodeSET )`**	
Description:	
Gets the next constraint equation that is defined on a selected set of nodes.	
Input:	
`INT4 nodeSET`	The ID of a Set object that contains the IDs of the entities to be searched. Alternatively, if you specify a negative value, this is simply the ID of the single entity to search.
Output:	
None	None
Return Code:	
`FE_FAIL`	No additional constraint equations can be found in this Constraint Set.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Before calling this method, the setID property of the object must be defined to search the desired Constraint Set. Also, you should call `Reset` before the first call to this method so it will retrieve the first available load.

Example:

Retrieve all of the constraint equations in Constraint Set 7, defined on Surfaces 1 and 2:

```
Dim b As femap.BCEqn
   Set b = App.feBCEqn
   Dim s As femap.Set
   Set s = App.feSet

   s.AddRange(  1, 2, 1 )

   b.SetID = 1
   b.Reset()
   While b.NextOnNode( s.ID ) = FE_OK
       . . .
   Wend
```

BCGeom Objects

BCGeom objects correspond to the geometry-based constraints in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for BCGeom. For more information, see **Common Entity Properties and Methods**.

Use the `feBCGeom` method of the Femap Application object to create BCGeom objects.

BCGeom Object Properties

BCGeoms are always stored with their SetID equal to the setID of the constraint set; however, their IDs simply go from 1 to N. To find a particular object, you must retrieve each object in the set and check its `geomID` and `geomType`.

Unpublished work. © 2025 Siemens

When you create a new BCGeom, it will always reference the active constraint set. You can change the set that it references simply by changing the setID parameter.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`BOOL dof[0..5]`
 	
or
 	
`VARIANT vdof`
	An array of flags telling whether or not the corresponding degrees of freedom are constrained (True) or free (False), in this set.

`INT4 color`	The constraint color.
`INT4 layer`	The ID of the layer associated with the constraint.
`BOOL expanded`	True if this geometry-based constraint is currently expanded.
`INT4 geomType`	The type of geometry that is constrained by this object: 3=Point, 4=Curve, 5=Surface.
`INT4 geomID`	The ID of the geometry that is constrained by this object.
`INT4 type`	The type of constraint being applied (0=DOF Pinned or Fixed, 1=General 6 DOF Control in output_sys, 2=Constrained normal to surface, 3=Constrained in all *in-surface* directions, 4=Allow sliding in a direction, 5=Cylinder, dof control directions).

`REAL8 direction[0..2]`
 	
or
 	
`VARIANT vdirection`
	The direction of sliding for type=4 constraints.

`INT4 outCSys`	ID of constraint coordinate system for type=1 constraints.
`INT4 BCDefinitionID`	The ID of the constraint definition that contains this constraint (0=None).

Unpublished work. © 2025 Siemens

Property	Description
Additional Properties	
`BOOL dofvec[0..5]`	Accesses all six components of the dof vector and returns them as a variant. Same as using dof independently.

BCGeom Object Methods

GetSearch

`GetSearch`	
`( geomType, geomID )`	
Description:	
This method searches for and retrieves an entity that is constraining a specified geometric entity.	
Input:	
`INT4 geomType`	Type of geometry being constrained: 3=Point, 4=Curve, 5=Surface.
`INT4 geomID`	ID of geometry being constrained.
Output:	
None	None
Return Code:	
`FE_FAIL`	The specified entity does not exist or is not constrained.
Remarks/Usage:	
Since BCGeom objects are not stored by a specified ID, but rather in sequential order, the normal Get method may not be useful if you are trying to find the constraints on a specific geometric entity. The only way to do that would be to `Get` every constraint and check for the geometry that you want. This method does that for you. You specify the geometry you want to search for, and it returns the constraints on that entity. Be aware that this method does still search the entire set. If you are going to retrieve constraints	

Unpublished work. © 2025 Siemens

on a large number of geometry entities, it could be faster to retrieve them all using `Get`, and discard the ones you do not want.
Example:
None

SetSurfSliding

`SetSurfSliding` **`( INT4 surfID )`**	
Description:	
This method initializes the current object to allow sliding along a surface.	
Input:	
`INT4 surfID`	The ID of the surface to slide along.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method does not store the constraint. You must still call `Put` to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

SetSurfNormal

Unpublished work. © 2025 Siemens

SetSurfNormal	
(INT4 surfID)	
Description:	
This method initializes the current object to allow movement normal to a surface.	
Input:	
`INT4 surfID`	The ID of the surface to move normal to.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method does not store the constraint. You must still call `Put` to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

SetSurfDirection

SetSurfDirection	
(INT4 surfID, REAL8 x, REAL8 y, REAL8 z)	
Description:	
This method initializes the current object to allow sliding of a surface in a specified direction.	
Input:	
`INT4 surfID`	The ID of the surface to slide.
`REAL8 x`	The Global X component of the permitted sliding direction.

Unpublished work. © 2025 Siemens

`REAL8 y`	The Global Y component of the permitted sliding direction.
`REAL8 z`	The Global Z component of the permitted sliding direction.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method does not store the constraint. You must still call `Put` to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

SetCylinder

`SetCylinder` **`( INT4 surfID, BOOL radius, BOOL rotation, BOOL sliding )`**	
Description:	
This method initializes the current object to allow sliding of a surface in cylindrical directions.	
Input:	
`INT4 surfID`	The ID of the surface to slide.
`BOOL radius`	If True, allow growth of the radius of the cylinder.
`BOOL rotation`	If True, allow rotation around the axis of the cylinder.
`BOOL sliding`	If True, allow sliding along the axis of the cylinder.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	The specified surface is not cylindrical.
Remarks/Usage:	
This method does not store the constraint. You must still call `Put` to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

SetFixed

`SetFixed` **`( geomType , geomID )`**	
Description:	
This method initializes the current object to apply *Fixed* boundary conditions (All 6 DOFs constrained) on a geometric entity of the specified type.	
Input:	
`INT4 geomType`	The type of entity to constrain. For entity information, see **Entity Types**. Can be 3=Point (`FT_POINT`), 4=Curve (`FT_CURVE`), or 5=Surface (`FT_SURFACE`).
`INT4 geomID`	The ID of the point, curve, or surface to constrain
Output:	
None	None
Return Code:	
`FE_INVALID`	The specified geomType is not a point, curve, or surface.
`FE_NOT_EXIST`	The specified point, curve, or surface ID does not exist.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method does not store the constraint. You must still call Put to do that. It simply initializes the properties necessary to define this type of constraint.
Example:
None

SetPinned

`SetPinned` **`( geomType , geomID )`**	
Description:	
This method initializes the current object to apply *Pinned* boundary conditions (constrains the 3 translational DOFs) on a geometric entity of the specified type.	
Input:	
`INT4 geomType`	The type of entity to constrain. For entity information, see **Entity Types**. Can be 3=Point (`FT_POINT`), 4=Curve (`FT_CURVE`), or 5=Surface (`FT_SURFACE`).
`INT4 geomID`	The ID of the point, curve, or surface to constrain
Output:	
None	None
Return Code:	
`FE_INVALID`	The specified geomType is not a point, curve, or surface.
`FE_NOT_EXIST`	The specified point, curve, or surface ID does not exist.
Remarks/Usage:	
This method does not store the constraint. You must still call Put to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetNoRotation

SetNoRotation **(geomType , geomID)**	
Description:	
This method initializes the current object to apply *No Rotation* boundary conditions (constrains the 3 rotational DOFs) on a geometric entity of the specified type.	
Input:	
`INT4 geomType`	The type of entity to constrain. For entity information, see **Entity Types**. Can be 3=Point (`FT_POINT`), 4=Curve (`FT_CURVE`), or 5=Surface (`FT_SURFACE`).
`INT4 geomID`	The ID of the point, curve, or surface to constrain
Output:	
None	None
Return Code:	
`FE_INVALID`	The specified geomType is not a point, curve, or surface.
`FE_NOT_EXIST`	The specified point, curve, or surface ID does not exist.
Remarks/Usage:	
This method does not store the constraint. You must still call Put to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

SetArbitrary

Unpublished work. © 2025 Siemens

SetArbitrary

(geomType , geomID, csysID, DOF1, DOF2, DOF3, DOF4, DOF5, DOF6)

Description:

This method initializes the current object to apply boundary conditions in a specified coordinate system, using the selected DOF, on a geometric entity of the specified type.

Input:

`INT4 geomType`	The type of entity to constrain. For entity information, see **Entity Types**. Can be 3=Point (`FT_POINT`), 4=Curve (`FT_CURVE`), or 5=Surface (`FT_SURFACE`).
`INT4 geomID`	The ID of the point, curve, or surface to constrain.
`INT4 csysID`	The ID of a coordinate system. Specify -1 to use the output coordinate of each node associated to the specified geometric entity.
`BOOL DOF1`	True = TX constrained in coordinate system specified in csysID.
`BOOL DOF2`	True = TY constrained in coordinate system specified in csysID.
`BOOL DOF3`	True = TZ constrained in coordinate system specified in csysID.
`BOOL DOF4`	True = RX constrained in coordinate system specified in csysID.
`BOOL DOF5`	True = RY constrained in coordinate system specified in csysID.
`BOOL DOF6`	True = RZ constrained in coordinate system specified in csysID.

Output:

None	None

Return Code:

`FE_INVALID`	The specified geomType is not a point, curve, or surface.
`FE_NOT_EXIST`	The specified point, curve, or surface ID does not exist.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	The specified coordinate system does not exist or cannot be used.
Remarks/Usage:	
This method does not store the constraint. You must still call `Put` to do that. It simply initializes the properties necessary to define this type of constraint.	
Example:	
None	

NextOnEntity

`NextOnEntity` **`( geomTYPE, geomSET )`**	
Description:	
Gets the next constraint that is defined on a selected set of geometric entities.	
Input:	
`INT4 geomTYPE`	The type of geometry where the constraints are defined. Must be Point, Curve, or Surface. For entity information, see **Entity Types**.
`INT4 geomID`	The ID of a Set object that contains the IDs of the entities to be searched. Alternatively, if you specify a negative value, this is simply the ID of the single entity to search.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	`geomTYPE` is not one of the available entity types.
`FE_FAIL`	No additional loads can be found in this Load Set.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Before calling this method, the setID property of the object must be defined to search the desired Constraint Set. Also, you should call `Reset` before the first call to this method so it will retrieve the first available load.

Example:

Retrieve all of the constraints in Constraint Set 7, defined on Surfaces 1 and 2:

```
Dim b As femap.BCGeom
   Set b = App.feBCGeom
   Dim s As femap.Set
   Set s = App.feSet

   s.AddRange(  1, 2, 1 )

   b.SetID = 1
   b.Reset()
   While b.NextOnEntity( FT_SURFACE, s.ID ) = FE_OK
      . . .
   Wend
```

BCNode Objects

BCNode objects correspond to the nodal constraints in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for BCNodes. For more information, see **Common Entity Properties and Methods**.

Use the `feBCNode` method of the Femap Application object to create BCNode objects.

BCNode Object Properties

BCNodes are always stored with their SetID equal to the setID of the constraint set, and the ID equal to the node ID.

Unpublished work. © 2025 Siemens

When you create a new BCNode, it will always reference the active constraint set. You can change the set that it references simply by changing the setID parameter.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`BOOL dof[0..5]`
 	
or
 	
`VARIANT vdof`
	An array of flags telling whether or not the corresponding degrees of freedom are constrained (True) or free (False) in this set.

`INT4 color`	The constraint color.
`INT4 layer`	The ID of the layer associated with the constraint.
`BOOL expanded`	True if this is a constraint from an expanded geometric constraint.
`INT4 BCDefinitionID`	The ID of the constraint definition that contains this constraint (0=None).
`BOOL NonZeroConstraint`	True if constraint is a `NonZeroConstraint`.

`REAL8 value[0..5]`
 	
or
 	
`VARIANT vvalue`
	An array of values containing the non-zero values at each corresponding degrees of freedom. `NonZeroConstraint` must be True and each DOF must also be constrained using dof or vdof when creating a new `NonZeroConstraint` when only using BCNode object properties.

Additional Properties	
`BOOL dofvec[0..5]`	Accesses all six components of the dof vector and returns them as a Variant. Same as using dof independently.

BCNode Object Methods

Unpublished work. © 2025 Siemens

Add

Add	
(setID, bDOF1, bDOF2, bDOF3, bDOF4, bDOF5, bDOF6)	
Description:	
This method creates nodal constraints on all selected nodes.	
Input:	
`INT4 setID`	The ID of a Set object that contains the nodes to constrain. Alternatively, if you specify a negative value, this is simply the ID of the single node to constrain.
`BOOL bDOF1`	The constraints to apply to the nodes. If True, the corresponding degree of freedom is constrained.
`BOOL bDOF2`	
`BOOL bDOF3`	
`BOOL bDOF4`	
`BOOL bDOF5`	
`BOOL bDOF6`	
Output:	
None	None
Return Code:	
`FE_FAIL`	The Constraints could not be created.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color, layer and setID, and any other types required for your constraints. This method simply uses the data you specify here to overwrite specific values in the BCNode object, and create a new constraint for each entry in the set of nodes.	

Unpublished work. © 2025 Siemens

Example:
None

AddArray

AddArray **(count, doDOF, NodeID, DOF)**	
Description:	
This method creates multiple nodal constraints. This method is obsolete and should be replaced by `PutArray`.	
Input:	
`INT4 count`	The number of constraints to create, and the number of IDs in NodeID.
`BOOL doDOF`	If True, then the DOF array must contain degree-of-freedom flags to be used. If False, then whatever is specified in the current object is used and DOF is ignored.
`INT4 NodeID[0..count-1]`	The Node IDs where constraints will be created. One constraint is created on each ID.
`INT4 DOF[0..(6*count)-1]`	Flags specifying constraints on each specified node. 6 flags per node. Refer to DOF property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Constraints could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color, layer and setID, and any other types required for your constraints. This method simply uses the data you specify here to overwrite specific values in the BCNode object, and create a new constraint for each entry in the arrays.	

Unpublished work. © 2025 Siemens

Example:
None

AddNonZero

AddNonZero **(setID, bDOF1, dVal1, bDOF2, dVal2, bDOF3, dVal3, bDOF4, dVal4, bDOF5, dVal5, bDOF6, dVal6)**	
Description:	
This method creates non-zero nodal constraints on all selected nodes.	
Input:	
`INT4 setID`	The ID of a Set object that contains the nodes to constrain. Alternatively, if you specify a negative value, this is simply the ID of the single node to constrain.
`BOOL bDOF1`	If True, TX degree of freedom is constrained and corresponding non-zero value will be applied.
`REAL8 dVal1`	Non-zero value for TX degree of freedom.
`BOOL bDOF2`	If True, TY degree of freedom is constrained and corresponding non-zero value will be applied.
`REAL8 dVal2`	Non-zero value for TY degree of freedom.
`BOOL bDOF3`	If True, TZ degree of freedom is constrained and corresponding non-zero value will be applied.
`REAL8 dVal3`	Non-zero value for TZ degree of freedom.
`BOOL bDcOF4`	If True, RX degree of freedom is constrained and corresponding non-zero value will be applied.
`REAL8 dVal4`	Non-zero value for RX degree of freedom.
`BOOL bDOF5`	If True, RY degree of freedom is constrained and corresponding non-zero value will be applied.

Unpublished work. © 2025 Siemens

`REAL8 dVal5`	Non-zero value for RY degree of freedom.
`BOOL bDOF6`	If True, RZ degree of freedom is constrained and corresponding non-zero value will be applied.
`REAL8 dVal6`	Non-zero value for RZ degree of freedom.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Constraints could not be created.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color, layer, and setID, and any other types required for your constraints. This method simply uses the data you specify here to overwrite specific values in the BCNode object, and create a new constraint for each entry in the set of nodes. Also, if a value is specified for a particular DOF which is not currently set to True, but then is set to True later, the value will then also be applied.	
Example:	
None	

PutArray

`PutArray`	
`( count, doDOF, NodeID, DOF )`	
Description:	
This method creates multiple nodal constraints.	
Input:	
`INT4 count`	The number of constraints to create, and the number of IDs in NodeID.
`BOOL doDOF`	If True, then the DOF array must contain degree-of-freedom flags to be used. If False, then whatever is specified in the current object is used and DOF is ignored.

Unpublished work. © 2025 Siemens

`INT4 NodeID[0..count-1]`	The Node IDs where constraints will be created. One constraint is created on each ID.
`INT4 DOF[0..(6*count)-1]`	Flags specifying constraints on each specified node. 6 flags per node. Refer to *dof* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Constraints could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object.	
Example:	
None	

GetAllArray

`GetAllArray`
`( count, NodeID, Layer, Color, BCDefinitionID, bNonZero, DOF, NonZeroValues )`
Description:
This method finds information on nodal constraints.
Input:
None — None
Output:
`INT4 count` — The number of constraints to create, and the number of IDs in NodeID.
`INT4 NodeID[0..count-1]` — The Node IDs where constraints will be created. One constraint is created on each ID.
`INT4 Layer[0..count-1]` — The IDs of the layers the loads are on.

Unpublished work. © 2025 Siemens

`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 BCDefinitionID[0..count-1]`	The ID of the constraint definition that contains this constraint (0=None).
`BOOL bNonZero[0..(6*count)-1]`	True if constraint is a `NonZeroConstraint`.
`INT4 DOF[0..(6*count)-1]`	Flags specifying constraints on each specified node. 6 flags per node. Refer to *dof* property.
`REAL8 NonZeroValues[0..(6*count)-1]`	An array of values containing the non-zero values at each corresponding degrees of freedom. `NonZeroConstraint` must be True and each DOF must also be constrained using dof or vdof when creating a new `NonZeroConstraint` when only using BCNode object properties.
Return Code:	
`FE_NOT_AVAILABLE`	No contraints exist in the Constriant Set.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object.	
Example:	
None	

PutAllArray

`PutAllArray`
`( count, NodeID, Layer, Color, BCDefinitionID, bNonZero, DOF, NonZeroValues )`
Description:
This method creates multiple nodal constraints.
Input:

`INT4 count`	The number of constraints to create, and the number of IDs in NodeID.

Unpublished work. © 2025 Siemens

`INT4 NodeID[0..count-1]`	The Node IDs where constraints will be created. One constraint is created on each ID.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 BCDefinitionID[0..count-1]`	The ID of the constraint definition that contains this constraint (0=None).
`BOOL bNonZero[0..(6*count)-1]`	True if constraint is a `NonZeroConstraint`.
`INT4 DOF[0..(6*count)-1]`	Flags specifying constraints on each specified node. 6 flags per node. Refer to *dof* property.
`INT4 NonZeroValues[0..(6*count)-1]`	An array of values containing the non-zero values at each corresponding degrees of freedom. `NonZeroConstraint` must be True and each DOF must also be constrained using dof or vdof when creating a new `NonZeroConstraint` when only using BCNode object properties.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Constraints could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object.	
Example:	
None	

BCSet Objects

BCSet objects correspond to the constraint sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for constraint sets. For more information, see **Common Entity Properties and Methods**.

Use the `feBCSet` method of the Femap Application object to create BCSet objects.

Unpublished work. © 2025 Siemens

BCSet Object Properties

BCSets are always stored with a SetID=1, and the ID equal to the constraint set ID.

BCSet objects are really simply placeholders for global data for the constraint set. The actual constraints are defined in other objects.

Property	Description
`STRING title`	Constraint set title (maximum 79 characters).
`BOOL IsCombination`	This is a Nastran SPCADD/MPCADD combination set if True.

BCSet Object Methods

Expand

Expand	
`( void )`	
Description:	
This method expands all of the geometric constraints in a selected constraint set.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	The specified expansion did not work.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method takes the current constraint set object (uses the ID), and expands all of the geometric constraints in that set, so that they may be retrieved as normal mesh-based constraints. If you are using advanced geometric constraints (not just fixed and/or pinned), the resulting nodal output coordinate systems, and expanded constraints may depend on the sets you choose to expand. Expanding one set at a time may not yield the result that you expect.

Note

This method only works with BCSets after they have been stored in the model database. If you are creating a new BCSet, only call this method after you have called `Put()`.

Example:

None

Compress

`Compress`

`( void )`

Description:

This method compresses all of the geometric constraints in a selected constraint set.

Input:

None	None

Output:

None	None

Return Code:

`FE_FAIL`	The specified compression did not work.

Remarks/Usage:

This method takes the current constraint set object (uses the ID), and compresses (deletes) any expanded constraints. You are left with the unexpanded geometric constraints.

Note

This method only works with BCSets after they have been stored in the model database. If you are creating a new BCSet, only call this method after you have called `Put()`.

Unpublished work. © 2025 Siemens

Example:
None

ResetNextBC

`ResetNextBC` **`( enDataType, bOtherBCOnly, bIncludeExpanded )`**	
Description:	
This method initializes all options prior to calling the `NextBC` method below.	
Input:	
`INT4 enDataType`	The type of constraint entities to be retrieved. For entity information, see **Entity Types**. Can be `FT_BCO`, `FT_BCO_GEOM`, or `FT_BEQ`.
`BOOL bOtherBCOnly`	If true, then only constraints that are not part of a constraint definition will be retrieved. Otherwise all are available.
`BOOL bIncludeExpanded`	If True, expanded geometric constraints (if any) will be returned along with normal constraints. Ignored if `enDataType = FT_BCO_GEOM`.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
This method is used prior to calling `NextBC` to specify which constraints will be retrieved. It also resets this object so that the next call to `NextBC` will return the first constraint available in the object.	
Example:	
None	

Unpublished work. © 2025 Siemens

NextBC

NextBC

(pDISPATCH)

Description:

This method returns the next available constraint that matches the criteria established by `ResetNextBC`.

Input:

None	None

Output:

`OBJECT pDISPATCH`	The next constraint object. Depending on the options specified, this could be a BCNode, BCGeom or BCEqn object.

Return Code:

`FE_BAD_DATA`	The data type that you are trying to retrieve is invalid.
`FE_FAIL`	There are no more constraints of the specified type to retrieve. Call `ResetNextBC` to go back to the first constraint, if you want to retrieve the constraints again.

Remarks/Usage:

This method, coupled with `ResetNextBC` provides an easy way to retrieve all constraints from a Constraint Set.

For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.

Example:

```
‘ This example loops through all constraint sets retrieving all

‘ BCNode objects. For each it displays a message box.

Sub Main
Dim App As femap.model
Set App = feFemap()
```


Unpublished work. © 2025 Siemens

```
Dim bs As femap.BCSet
Set bs = App.feBCSet


 Dim b As femap.BCNode


While bs.Next()
 bs.ResetNextBC( FT_BCO, False, False )
  While bs.NextBC( b )
     MsgBox(Str$(b.setID)+""+Str$(b.ID) )

   Wend
 Wend
End Sub
```

ResetNextBCDef

ResetNextBCDef	
(bAllDataTypes, enDataType)	
Description:	
This method initializes all options prior to calling the `NextBCDef` method below.	
Input:	
`BOOL bAllDataTypes`	Used to limit the type of constraint definitions to be retrieved. If True, all selected constraint definitions will be returned. If False, only constraint definitions that contain constraints of the type specified in `enDataType` will be returned.
`INT4 enDataType`	The type of constraint entities to be retrieved. For entity information, see **Entity Types**. Can be `FT_BCO`, `FT_BCO_GEOM`, or `FT_BEQ`.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method is used prior to calling `NextBCDef` to specify which constraint definitions will be retrieved. It also resets this object so that the next call to `NextBCDef` will return the first constraint definition available in the object.
Example:
None

NextBCDef

`NextBCDef` **`( pDISPATCH )`**	
Description:	
This method returns the next available constraint definition that matches the criteria established by `ResetNextBCDef`.	
Input:	
None	None
Output:	
`OBJECT pDISPATCH`	The next constraint definition object. This is always a BCDefinition object.
Return Code:	
`FE_FAIL`	There are no more constraint definitions of the specified type to retrieve. Call `ResetNextBCDef` to go back to the first constraint definition, if you want to retrieve the constraints again.
Remarks/Usage:	
This method, coupled with `ResetNextBCDef` provides an easy way to retrieve all constraint definitions from a Constraint Set. For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.	

Unpublished work. © 2025 Siemens

Example:

```
‘ This example loops through all constraint sets retrieving all
‘ nodal BCDefinition objects, then loops through each
‘ BCDefinition to retrieve the individual constraints. For each ‘ it
displays a message box.
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim bs As femap.BCSet
Set bs = App.feBCSet


Dim bd As femap.BCDefinition
Dim b As femap.BCNode


While bs.Next()
bs.ResetNextBCDef( False, FT_BCO )
  While bs.NextBCDef( bd )
     MsgBox( "BC Set"+Str$(bd.setID)+" BC Def"+Str$(bd.ID))
     While bd.NextBC( b )
        MsgBox(  "BC On Node "+Str$(b.ID) )
      Wend
  Wend
Wend
End Sub
```

GetCombination

GetCombination

(nCombo, nSetID)

Description:	
This method returns the data for a Nastran SPCADD/MPCADD Combination Set.	
Input:	
None	None
Output:	
`INT4 nCombo`	Number of entries in the combination.

Unpublished work. © 2025 Siemens

`INT4 nSetID [0..count-1]`	The IDs of the constraint sets in the combination.
Return Code:	
`FE_BAD_TYPE`	This is not a Combination Constraint Set.
`FE_NOT_AVAILABLE`	No constraint sets are in the Combination.
Remarks/Usage:	
None	
Example:	
None	

PutCombination

`PutCombination` **`( nCombo, nSetID )`**	
Description:	
This method sets the data in a Nastran SPCADD/MPCADD Combination Set.	
Input:	
`INT4 nCombo`	Number of entries in the combination.
`INT4 nSetID[0..count-1]`	The IDs of the constraint sets in the combination.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	One or more Combined Constraint Sets specified do not exist.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

NodesAsSet

NodesAsSet **(bExpandedGeomBC, bNodalBC, bEquationBC, nNodeSetID, bClear)**	
Description:	
This method adds the IDs of nodes constrained in the Constraint Set currently loaded in the BCSet Object to a specified Set. There are options to include nodes which are constrained after geometric constraints are expanded, individual nodes which are constrained, and/or nodes used in constraint equations.	
Input:	
`BOOL bExpandedGeomBC`	When True, the IDs of any nodes which are constrained after geometric constraints are expanded will be added to the Set.
`BOOL bNodalBC`	When True, the IDs of any individual nodes which are constrained will be added to the Set.
`BOOL bEquationBC`	When True, the IDs of any nodes which are used in a constraint equation will be added to the Set.
`INT4 nNodeSetID`	ID of Set object to add node IDs.
`BOOL bClear`	When True, any IDs which are in the set are removed before adding the node IDs to the Set.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Chart Objects

Chart objects correspond to the charts in **Charting** pane. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for chart objects. For more information, see **Common Entity Properties and Methods**.

Use the `feChart` method of the Femap Application object to create Chart objects.

Chart Object Properties

Charts are always stored with a SetID=1, and the ID equal to the Chart ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 Style`	Chart Style: 0=Point, 1=Line, 2=Line-Fast, 3=Line-Step, 4=Area, 5=Area-Step
`STRING Title`	The Chart Title.
`BOOL TitleVisible`	Controls Chart Title visibility.
`BOOL ScaleInherit`	Use scale factors from individual data series when true. When false, MasterScale value is used for all displayed data series, regardless of individual data series scale factors.
`REAL8 MasterScale`	Scale factor for all data series displayed on the chart. Only used when ScaleInherit is set to False.
`INT4 TitleLocation`	Chart Title Location: 0=Top, 1=Left, 2=Right, 3=Bottom.
`INT4 TitleJustification`	Chart Title Justification: 0=Left, 1=Center, 2=Right.
`REAL8 TitleFontSize`	Title Font Size.
`BOOL TitleFontBold`	Controls if Font is Bold in Chart Title (TRUE=Bold).

Unpublished work. © 2025 Siemens

Property	Description
`BOOL TitleFontItalic`	Controls if Font is Italic in Chart Title (TRUE=Italic).
`BOOL TitleFontUnderline`	Controls if Font is Underlined in Chart Title (TRUE=underline)
`INT4 TitleFontColor`	Controls Chart Title Font Color
`STRING Font`	Controls overall Font for the Chart. Controls Title, Axis Titles, and Axis Labels.
`INT4 Palette`	Chart Palette: 0=Femap, 1=Gray, 2=Office, 3=Vibrant, 4=Blue, 5=Brown, 6=Green, 7=Red.
`BOOL PaletteFromChart`	When TRUE, all Data Series use Chart Palette Colors. When FALSE, Data Series colors controlled by individual Data Series.
`BOOL DarkBackground`	When TRUE, Chart switches to have a Dark Background.
`BOOL LegendVisible`	Controls Chart Legend visibility.

`INT4 LegendLocation [0..1]`
or
`VARIANT vLegendLocation`
	Chart Legend Location:
	[0] = Vertical Location - 0= Top Outside, 1=Top, 2=Center, 3=Bottom, 4=Bottom Outside
	[1] = Horizontal Location - 0= Left Outside, 1=Left, 2=Center, 3=Right, 4=Right Outside

`INT4 LegendDirection`	Chart Legend Direction: 0=Top to Bottom, 1=Left to Right.
`REAL8 LegendFontSize`	Font size for legend labels. Must be greater than 0.0.

`BOOL AxisAutoscale[0..1]`
or
`VARIANT vAxisAutoscale`
	Controls if Autoscale is on for an Axis (TRUE=On) :
	[0]=X Axis
	[1]=YAxis

`INT4 AxisStyle[0..1]`	Controls Axis Style. 0=Linear, 1=Logarithmic: [0]=X Axis [1]=YAxis

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vAxisStyle`	

`REAL8 AxisRange[0..1][0..1]`
 	
or
 	
`VARIANT vAxisRange`
	Controls minimum and maximum values for each axis:
	
	[0][0]=X Axis, minimum [0][1] =X Axis, maximum
	[1][0]=Y Axis, minimum [1][1] =Y Axis, maximum

`BOOL AxisRangePad[0..1]`
 	
or
 	
`VARIANT vAxisRangePad`
	Controls autoscale padding for axis:
	
	[0]=X Axis
	[1]=YAxis

`BOOL AxisVisible[0..1]`
 	
or
 	
`VARIANT vAxisVisible`
	Controls if an Axis is visible (TRUE=On):
	
	[0]=X Axis
	[1]=YAxis

`BOOL AxisMajorGridlineVisible[0..1]`
 	
or
 	
`VARIANT vAxisMajorGridlineVisible`
	Controls if major grid lines on an Axis are visible (TRUE=On):
	
	[0]=X Axis
	[1]=YAxis

`REAL8 AxisMajorInterval[0..1]`
 	
or
 	
`VARIANT vAxisMajorIntervcl`
	Sets the interval value for major grid lines on an axis:
	
	[0]=X Axis
	[1]=YAxis

`BOOL AxisMajorTickmarkVisible[0..1]`	Controls if major grid line tick marks are visible on an axis (TRUE=On): [0]=X Axis

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vAxisMajorTickmarkVisible`	[1]=YAxis
`INT4 AxisMajorColor`	Controls Major Grid Line Color.
`BOOL AxisMajorColorAutoDim`	Controls Dim for Dark Backgrounds option (TRUE=On).

`BOOL AxisMinorGridlineVisible[0..1]`
 	
or
 	
`VARIANT vAxisMinorGridlineVisible`
	Controls if minor grid lines on an Axis are visible (TRUE=On) The major grid lines must be on for the minor grid lines to be visible.
	
	[0]=X Axis
	[1]=YAxis

`INT4 AxisMinorCount[0..1]`
 	
or
 	
`VARIANT vAxisMinorCount`
	Sets the value for number of minor grid lines between each major grid line on an axis for linear scale.
	
	[0]=X Axis
	[1]=YAxis

`INT4 AxisMinorCountLog[0..1]`
 	
or
 	
`VARIANT vAxisMinorCountLog`
	Sets the value for number of minor grid lines between each major grid line on an axis for logarithmic scale.
	
	[0]=X Axis
	[1]=YAxis

`BOOL AxisMinorTickmarkVisible[0..1]`
 	
or
 	
`VARIANT vAxisMinorTickmarkVisible`
	Controls if minor grid line tick marks are visible on an axis (TRUE=On).
	
	[0]=X Axis
	[1]=YAxis

`BOOL AxisZoom[0..1]`
 	
or
 	
`VARIANT vAxisZoom`
	Controls Allow Zooming option for an axis. TRUE = Allow Zooming on, FALSE = Allow Zooming Off.
	
	[0]=X Axis
	[1]=YAxis

Unpublished work. © 2025 Siemens

Property	Description
`STRING AxisTitleX`	Title of X Axis.
`STRING AxisTitleY`	Title of Y Axis.

`BOOL AxisTitleVisible[0..1]`
or
`VARIANT vAxisTitleVisible`
	Controls visibility of Axis Title:
	[0]=X Axis
	[1]=YAxis

`INT4 AxisTitleJustification[0..1]`
or
`VARIANT vAxisTitleJustification`
	Axis Title Justification: 0=Left, 1=Center, 2=Right:
	[0]=X Axis
	[1]=YAxis

`INT4 AxisTitleFontSize[0..1]`
or
`VARIANT vAxisTitleFontSize`
	Axis Title Font Size.
	[0]=X Axis
	[1]=YAxis

`BOOL AxisTitleFontBold`
or
`VARIANT vAxisTitleFontBold`
	Controls if Font is Bold in Axis Title (TRUE=Bold).
	[0]=X Axis
	[1]=YAxis

`BOOL AxisTitleFontItalic`
or
`VARIANT vAxisTitleFontItalic`
	Controls if Font is Italic in Axis Title (TRUE=Italic).
	[0]=X Axis
	[1]=YAxis

`BOOL AxisTitleFontUnderline`	Controls if Font is Underlined in Axis Title (TRUE=underline). [0]=X Axis

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vAxisTitleFontUnderline`	[1]=YAxis

`INT4 AxisTitleFontColor`
 	
or
 	
`VARIANT vAxisTitleFontColor`
	Controls Axis Title Font Color.
	
	[0]=X Axis
	[1]=YAxis

`INT4 AxisLabelDecimal[0..1]`
 	
or
 	
`VARIANT vAxisLabelDecimal`
	Controls number of digits after the decimal point. Set to -1 for automatic mode.
	
	[0]=X Axis
	[1]=YAxis

`INT4 AxisLabelFormat[0..1]`
 	
or
 	
`VARIANT vAxisLabelFormat`
	Controls Label Format on an axis. 0=Standard, 1=Scientific, 2=Percentage.
	
	[0]=X Axis
	[1]=YAxis

`REAL8 AxisValueFontSize`	Font size for axis value labels. Must be greater than 0.0.
`BOOL MarkerFromChart`	When TRUE, overrides all data series marker settings and uses only chart marker settings.
`BOOL MarkerVisible`	Controls visibility of Data Series Markers being controlled by the Chart (`MarkerFromChart` = TRUE). When chart style is set to `FCH_STYLE_LINEFAST`, markers are unavailable and this setting is ignored.
`INT4 MarkerSize`	Data Series Marker size, 1 to 10
`INT4 MarkerStyle`	Data Series Marker style. 0=Circle, 1=Square, 2=Diamond, 3=Triangle, 4=Pentagon, 5=Hexagon, 6=Star.
`BOOL LabelFromChart`	When TRUE, overrides all data series label settings and uses only chart marker settings.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL LabelVisible`	Controls visibility of Data Series Labels being controlled by the Chart (`LabelFromChart` = TRUE). When chart style is set to `FCH_STYLE_LINEFAST`, labels are unavailable and this setting is ignored.
`REAL8 LabelFontSize`	Font size for point labels. Must be greater than 0.0.
`BOOL ShowLabelX`	Show X value on label (TRUE=ON). Also used to control value shown in Tooltip labels.
`BOOL ShowLabelY`	Show Y value on label (TRUE=ON). Also used to control value shown in Tooltip labels.
`BOOL ShowLabelMaxMin`	Show data series max/min values only. ShowLabelX = TRUE, ShowLabelY = TRUE - XY data shown at data series max/min Y values ShowLabelX = TRUE, ShowLabelY = FALSE - X data shown at data series max/min X values ShowLabelX = FALSE, ShowLabelY = TRUE - Y data shown at data series max/min Y values
`BOOL OutputSetInherit`	Use output set / vector data specified on the Chart Data Series (TRUE = Inherit from Data Series). Only for XY vs ID, XY vs Set, XY vs Set Value, and XY vs Position).
`BOOL OutputSet`	Output Set ID. Only used when `OutputSetInherit` = FALSE. When using Type = XY vs ID or XY vs Position, sets output set. When using Type = XY vs. Set or XY vs. Set Value, sets the Start Output Set.
`BOOL OutputSetEnd`	Ending Output Set ID. Only used when `OutputSetInherit` = FALSE. When using Type = XY vs. Set or XY vs. Set Value, sets the End Output Set.
`BOOL OutputSetAll`	For XY vs Set and XY vs Set Value only. When TRUE, all output sets in the model will be used when plotting. Only used when `OutputSetInherit` = FALSE.
Secondary Plot	
`enum zChartComplexLocation ComplexPlotLocation`	Specifies location of Complex Plot (Secondary Plot). 0=None (`FCCL_OFF`), 1=Top (`FCCL_TOP`), and 2=Bottom (`FCCL_BOTTOM`).

Unpublished work. © 2025 Siemens

Property	Description

`BOOL AxisAutoscale2[0..1]`
 	
or
 	
`VARIANT vAxisAutoscale2`
	Controls if Autoscale is on for an Axis (TRUE=On) in secondary chart.
	
	[0]=X Axis (Currently not used)
	[1]=YAxis (Y Complex)

`INT4 AxisStyle2[0..1]`
 	
or
 	
`VARIANT vAxisStyle2`
	Controls Axis Style in secondary chart. 0=Linear, 1=Logarithmic.
	
	[0]=X Axis (Currently not used)
	[1]=YAxis (Y Complex)

`REAL8 AxisRange2[0..1][0..1]`
 	
or
 	
`VARIANT vAxisRange2`
	Controls minimum and maximum values for each axis in secondary chart.
	
	[0][0]=X Axis, minimum [0][1] =X Axis, maximum (Currently not used)
	[1][0]=Y Axis, minimum [1][1] =Y Axis, maximum (Y Complex)

`BOOL AxisRangePad2[0..1]`
 	
or
 	
`VARIANT vAxisRangePad2`
	Controls autoscale padding for axis in secondary chart.
	
	[0]=X Axis (Currently not used)
	[1]=YAxis (Y Complex)

`INT4 AxisLabelDecimal2[0..1]`
 	
or
 	
`VARIANT vAxisLabelDecimal2`
	Controls number of digits after the decimal point in secondary chart. Set to -1 for automatic mode.
	
	[0]=X Axis (Currently not used)
	[1]=YAxis (Y Complex)

`INT4 AxisLabelFormat2[0..1]`
 	
or
 	
`VARIANT vAxisLabelFormat2`
	Controls Label Format on an axis in secondary chart. 0=Standard, 1=Scientific, 2=Percentage.
	
	[0]=X Axis (Currently not used)
	[1]=YAxis (Y Complex)

Unpublished work. © 2025 Siemens

Chart Object Methods

AddDataSeries

AddDataSeries **(SetID)**	
Description:	
Adds one or more Data Series to the Chart.	
Input:	
`INT4 SetID`	ID of set containing Data Series IDs to add. When value is negative, ID of Data Series to add.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	When value of SetID is negative, `FE_NOT_EXIST` is returned if the Data Series does not exist.
Remarks/Usage:	
None	
Example:	
None	

RemoveDataSeries

RemoveDataSeries **(SetID)**
Description:

Unpublished work. © 2025 Siemens

Removes one or more Data Series from the Chart.	
Input:	
`INT4 SetID`	ID of set containing data series IDs to Remove. When value is negative, ID of data series to remove.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

RemoveAllDataSeries

`RemoveAllDataSeries`	
`( void )`	
Description:	
Removes all data series from the chart.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetDataSeries

`GetDataSeries` **`( numDS, dsID )`**	
Description:	
Returns an array of Data Series displayed on the Chart.	
Input:	
None	None
Output:	
`INT4 numDS`	Number of Data Series on Chart.
`INT4 dsID[0..numDS-1]`	Data Series IDs.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ShowPane

Unpublished work. © 2025 Siemens

ShowPane **(void)**	
Description:	
Displays the Chart in the Charting pane.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Charting pane is not displayed.
Remarks/Usage:	
The charting pane will display the chart as it exists in memory at the time this method is called. Any subsequent changes to the chart object will not be displayed.	
Example:	
None	

Chart Data Series Objects

Chart Data Series objects correspond to Data Series objects which are displayed in the **Charting** pane. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for chart objects. For more information, see **Common Entity Properties and Methods**.

Use the `feChartSeries` method of the Femap Application object to create Chart objects.

Chart Data Series Object Properties

Chart Data Series are always stored with a SetID=1, and the ID equal to the Data Series ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description
`STRING Title`	Data Series title
`INT4 SeriesType`	Data Series type: 0 = `FCHD_ID` (Vector vs. Entity - Entity ID) 1 = `FCHD_SET` (Vector vs. Set - Output Set ID) 2 = `FCHD_SETVAL` (Vector vs. Set - Output Set Value) 3 = `FCHD_POSITION` (Vector vs. Entity - Position) 4 = `FCHD_FUNCTION` (Function) 7 =`FCHD_VECVEC` (Vector vs. Vector) 10 = `FCHD_CPLXEXP` (Expand Complex)
`INT4 StudyID`	Study ID for `FCHD_SET`, `FCHD_SETVAL`, and `FCHD_VECVEC`. Only used when `OutputSetAll` is set to False. When StudyID is non-zero, `OutputSet` and `OutputSetEnd` are ignored.
`INT4 OutputSet`	Output Set ID for `FCHD_ID`, `FCHD_POSITION`, and `FCHD_CPLXEXP`. Starting Set ID for `FCHD_SET`, `FCHD_SETVAL`, and `FCHD_VECVEC` (only used when `OuptutSetAll` = False).
`INT4 OutputSetEnd`	Ending Set ID for `FCHD_SET`, `FCHD_SETVAL`, and `FCHD_VECVEC` (only used when `OuptutSetAll` = False).
`BOOL OutputSetAll`	When TRUE, all Use All Output Sets is activated and all output sets in the model are used for `FCHD_SET`, `FCHD_SETVAL`, and `FCHD_VECVEC`.
`INT4 OutputVectorV2`	Dependent Vector ID for `FCHD_VECVEC`. Vector ID for all others, except `FCHD_FUNCTION`. `OutputVector` has been deprecated. For more information, see **Working with Output in Femap v2020.1 and above**.
`INT4 OutputVector2V2`	Independent Vector ID for `FCHD_VECVEC`. `OutputVector2` has been deprecated. For more information, see **Working with Output in Femap v2020.1 and above**.
`INT4 Interval`	Interval for limiting retrieval of data when using `FCHD_SET`, `FCHD_SETVAL`, and `FCHD_VECVEC`.

Unpublished work. © 2025 Siemens

Property	Description
	Regardless of the interval set, the Starting Set and Ending Set are always included.
`INT4 FunctionID`	ID of Function for `FCHD_FUNCTION`.
`INT4 Group`	ID of Group (optional) for `FCHD_ID` and `FCHD_POSITION`.
`INT4 Position`	Component of Position for `FCHD_POSITION`. 0=X, 1=Y, 2=Z.
`INT4 PositionIsRelative`	When TRUE, enables the x-axis points to be Relative to Node specified in the Location property.
`INT4 PositionCoordSys`	Coordinate System ID for `FCHD_POSITION`.
`INT4 Location`	For `FCHD_SET`, `FCHD_SETVAL`, and `FCHD_ECPLXEXP`: Location to obtain data. Value must be a valid Node or Element ID and correspond with selected OutputVector. For `FCHD_VECVEC`: Independent Entity location. For `FCHD_POSITION`: Node/Element ID of Relative to Node/Element. Value must be a valid Node or Element ID.
`INT4 Location2`	For `FCHD_VECVEC`: Dependent entity location.
`INT4 TransformNodalMode`	Use enum `zOutputDestination`: For `FCHD_ID`, `FCHD_POSITION`, and `FCHD_ECPLXEXP`: Nodal vector transform mode for Vector For `FCHD_SET` and `FCHD_SETVAL`: Nodal vector transform mode for Vector across all selected output sets. For `FCHD_VECVEC`: Nodal vector transform mode for Independent Vector across all selected output sets.
`INT4 TransformNodalCSys`	Coordinate system for nodal transformations when `TransformNodalMode` is `FOD_VECTOR_TO_CSYS`.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 TransformNodalMode2`	Use enum zOutputDestination. For FCHD_VECVEC: Nodal vector transform mode for Dependent Vector across selected output sets.
`INT4 TransformNodalCSys2`	Coordinate system for nodal transformations when `TransformNodalMode2` is FOD_VECTOR_TO_CSYS.
`INT4 TransformPlateMode`	Use enum `zOutputDestination`: For FCHD_ID, FCHD_POSITION, and FCHD_ECPLXEXP: Plate vector transform mode for Vector. For FCHD_SET and FCHD_SETVAL: Plate vector transform mode for Vector across all selected output sets. For FCHD_VECVEC: Plate vector transform mode for Independent Vector across all selected output sets.
`INT4 TransformPlateCSys`	Plate vector transformation coordinate system when `TransformPlateMode = FOD_PLATE_TO_CSYS`.
`INT4 TransformPlateDOF`	Coordinate system axis for plate transformations when `TransformPlateMode = FOD_PLATE_TO_CSYS`.

`REAL8 TransformPlateVector[0..2]`
 	
or
 	
`VARIANT vTransformPlateVector`
	Components of vector for plate transformations when `TransformPlateMode = FOD_PLATE_TO_VECTOR`.

`REAL8 TransformPlateTolerance`	Tolerance Angle for plate transformations.
`INT4 TransformPlateMode2`	Use enum `zOutputDestination`: For FCHD_VECVEC: Plate vector transform mode for Dependent Vector across all selected output sets.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 TransformPlateCSys2`	Plate vector transformation coordinate system when `TransformPlateMode2 = FOD_PLATE_TO_CSYS`.
`INT4 TransformPlateDOF2`	Coordinate system axis for plate transformations when `TransformPlateMode2 = FOD_PLATE_TO_CSYS`.

`REAL8 TransformPlateVector2[0..2]`
 	
or
 	
`VARIANT vTransformPlateVector2`
	Components of vector for plate transformations when `TransformPlateMode2 = FOD_PLATE_TO_VECTOR`.

`REAL8 TransformPlateTolerance2`	Tolerance Angle for plate transformations of Dependent Vector.
`INT4 TransformSolidMode`	Use enum zOutputDestination: For `FCHD_ID`, `FCHD_POSITION`, and `FCHD_ECPLXEXP`: Solid vector transform mode for Vector. For `FCHD_SET` and `FCHD_SETVAL`: Solid vector transform mode for Vector across all selected output sets. For `FCHD_VECVEC`: Solid vector transform mode for Independent Vector across all selected output sets.
`INT4 TransformSolidCSys`	Solid vector transformation coordinate system when `TransformSolidMode = FOD_SOLID_TO_CSYS`.
`INT4 TransformSolidMode2`	Use enum `zOutputDestination`: For `FCHD_VECVEC`: Solid vector transform mode for Independent Vector across all selected output sets.
`INT4 TransformSolidCSys2`	Solid vector transformation coordinate system when `TransformSolidMode2 = FOD_SOLID_TO_CSYS`.
`INT4 ConvertMethod`	Use enum `zResultsConvert` (`FRC_NONE`, `FRC_AVG`, and `FRC_MAX` only):

Unpublished work. © 2025 Siemens

Property	Description
	For FCHD_ID, FCHD_POSITION, and FCHD_ECPLXEXP: Nodal / elemental conversion method for Vector. For FCHD_SET and FCHD_SETVAL: Nodal / elemental conversion method for Vector across all selected output sets. For FCHD_VECVEC: Nodal / elemental conversion method for Independent Vector across all selected output sets
INT4 ConvertMethod2	Use enum zResultsConvert (FRC_NONE, FRC_AVG, and FRC_MAX only). For FCHD_VECVEC: Nodal / elemental conversion method for dependent vector across all selected output sets.
INT4 ComplexMethod	Use enum zChartSeriesComplexMethod: For FCHD_ID and FCHD_POSITION: Method used for Complex Expansion of Vector. For FCHD_SET, FCHD_SETVAL, and FCHD_VECVEC: Method used for Complex Expansion of Vector across all selected output sets.
REAL8 ComplexPhase	For FCHD_ID, FCHD_POSITION, FCHD_SET, FCHD_SETVAL, and FCHD_VECVEC: Phase Angle value used when ComplexMethod = FCSCM_SYNC. For FCHD_CPLXEXP: always value for Increment.
REAL8 ComplexStart	For FCHD_CPLXEXP: value for Start.
REAL8 ComplexEnd	For FCHD_CPLXEXP: value for Stop.
REAL8 Scale	Scale factor applied to all values in an individual Data Series.
BOOL SortData	When TRUE, use Sort Data option for FCHD_VECVEC.
BOOL ColorInherit	When TRUE, use Palette colors from the Chart.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 Color`	When `ColorInherit` = FALSE, sets color of individual Data Series.
`BOOL MarkerInherit`	When TRUE, use Marker settings from the Chart.
`BOOL MarkerVisible`	When MarkerInherit = FALSE, controls visibility of markers for individual Data Series.
`INT4 MarkerStyle`	When `MarkerInherit` = FALSE, controls style of markers for individual Data Series.
`INT4 MarkerSize`	When `MarkerInherit` = FALSE, controls size of markers for individual Data Series.
`BOOL LabelInherit`	When TRUE, use Label settings from the Chart.
`BOOL LabelVisible`	When `LabelInherit` = FALSE, setting this to TRUE will display labels for individual Data Series.
`BOOL ShowLabelX`	When `LabelInherit` = FALSE and `LabelVisible` = TRUE, setting this to TRUE will display the X value in labels for individual Data Series.
`BOOL ShowLabelY`	When `LabelInherit` = FALSE and `LabelVisible` = TRUE, setting this to TRUE will display the Y value in labels for individual Data Series.
`BOOL ShowLabelMaxMin`	When `LabelInherit` = FALSE and `LabelVisible` = TRUE, setting this to TRUE will display only the Max/Min labels based on selected X and Y components.
`INT4 CombinationType`	The operation to perform on two chart series: 0=Add, 1=Subtract, 2=Multiply, 3=Divide.
`BOOL AbsoluteCombination`	Flag to convert combination result to absolute value upon completion.

Chart Data Series Object Methods

GetXY

Unpublished work. © 2025 Siemens

GetXY	
(numPoints, xyValues, xRange, yRange)	
Description:	
Retrieves all XY values from Data Series	
Input:	
None	None
Output:	
`INT4 numPoints`	Number of points returned.
`VARIANT xyValues[0..2*numpoints-1]`	`VARIANT xRange[0..1].`
`VARIANT xRange[0..1]`	Min/Max X Values.
`VARIANT xRange[0..1]`	Min/Max Y Values.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

CopyToClipboard

CopyToClipboard	
(void)	
Description:	
Copies all of the XY values to the clipboard.	
Input:	

Unpublished work. © 2025 Siemens

None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to copy to clipboard/no values to copy.
Remarks/Usage:	
None	
Example:	
None	

GetEntityType

`GetEntityType` **`( chartID )`**	
Description:	
Returns either Node or Element for the specified Chart.	
Input:	
`chartID`	ID of Chart.
Output:	
None	None
Return Code:	
Integer	7 = Node, 8 = Element.
`FE_FAIL`	Chart has no data or the type of data cannot be determined.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Computed Vector Objects

Computed Vector objects correspond to defined vectors which can be stored in the database to calculate additional output vectors when an analysis is done. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Vector objects. For more information, see **Common Entity Properties and Methods**.

Use the **feComputedResultsVectors** method of the Femap Entity object to create Computed Vector objects.

Computed Vector Object Properties

Computed Vectors are always stored with a SetID=1, and the ID equal to the reference ID.

Property	Description
Elemental Results (Automatic)	
`BOOL PlateSolidPrincipals`	If True, Plate/Solid Principals and Invariants will be turned on.
`BOOL PlateAveragedMid`	If True, Plate Avarged Mid Stress/Strain will be turned on.
`BOOL BeamVonMises`	If True, Beam/Bar Von Mises Stress will be turned on.
BOOL Plate/Solid Failure Theory (Automatic)	
`BOOL PlateSolidMaxVonMises`	If True, Max Von Mises failure theory will be turned on.
`BOOL PlateSolidTresca`	If True, Tresca failure theory will be turned on.
`BOOL PlateSolidMohrCoulomb`	If True, Mohr Coulomb failure theory will be turned on.
`BOOL PlateSolidMaxNormal`	If True, Max Normal failure theory will be turned on.
Plate/Solid Failure Theory Options (Automatic)	

Unpublished work. © 2025 Siemens

Property	Description
`BOOL PlateSolidUseCorners`	If True, plate and solid and solid corner results will be considered for overall failure theory computed vectors.
`INT4 DesignFactorOfSafety`	Set the value for the design factor of safety value used in the margin of safety calculations.
BOOL Laminate Failure Theory (Automatic)	
`BOOL LaminateHill`	If True, Hill failure theory will be turned on.
`BOOL LaminateHoffmann`	If True, Hoffman failure theory will be turned on.
`BOOL LaminateTsaiWu`	If True, Tsai-Wu failure theory will be turned on.
`BOOL LaminateMaxStrain`	If True, Max Strain failure theory will be turned on.
Enveloped Results (Automatic)	
`BOOL EnvelopeBeamAll`	If True, Beam All Locations will be turned on.
`BOOL EnvelopePlateLamPly`	If True, Plate Top.Bot, Laminate All Ply will be turned on.
`BOOL EnvelopePlateLamAll`	If True, Plate/Laminate All Location will be turned on.
`BOOL EnvelopeSolidAll`	If True, Solid All Location will be turned on.
Elemental Results (Now)	
`BOOL NowPlateSolidPrincipals`	If True, Plate/Solid Principals and Invariants will be turned on.
`BOOL NowPlateAveragedMid`	If True, Plate Avarged Mid Stress/Strain will be turned on.
`BOOL NowBeamVonMises`	If True, Beam/Bar Von Mises Stress will be turned on.
BOOL Plate/Solid Failure Theory (Now)	
`BOOL NowPlateSolidMaxVonMises`	If True, Max Von Mises failure theory will be turned on.
`BOOL NowPlateSolidTresca`	If True, Tresca failure theory will be turned on.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NowPlateSolidMohrCoulomb`	If True, Mohr Coulomb failure theory will be turned on.
`BOOL NowPlateSolidMaxNormal`	If True, Max Normal failure theory will be turned on.
Plate/Solid Failure Theory Options (Now)	
`BOOL NowPlateSolidUseCorners`	If True, plate and solid and solid corner results will be considered for overall failure theory computed vectors.
`INT4 NowDesignFactorOfSafety`	Set the value for the design factor of safety value used in the margin of safety calculations.
BOOL Laminate Failure Theory (Now)	
`BOOL NowLaminateHill`	If True, Hill failure theory will be turned on.
`BOOL NowLaminateHoffmann`	If True, Hoffman failure theory will be turned on.
`BOOL NowLaminateTsaiWu`	If True, Tsai-Wu failure theory will be turned on.
`BOOL NowLaminateMaxStrain`	If True, Max Strain failure theory will be turned on.
Enveloped Results (Now)	
`BOOL NowEnvelopeBeamAll`	If True, Beam All Locations will be turned on.
`BOOL NowEnvelopePlateLamPly`	If True, Plate Top.Bot, Laminate All Ply will be turned on.
`BOOL NowEnvelopePlateLamAll`	If True, Plate/Laminate All Location will be turned on.
`BOOL NowEnvelopeSolidAll`	If True, Solid All Location will be turned on.

Computed Vector Object Methods

There following methods are available in addition to the standard entity methods.

Clear

Unpublished work. © 2025 Siemens

Clear	
(bClearAll)	
Description:	
Removes all user defined computed vectors from the list.	
Input:	
`BOOL bClearAll`	When True, all computed vectors are removed.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Dialog

Dialog	
(void)	
Description:	
Opens the dialog.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

GetCountUserDefined

`GetCountUserDefined` **`( nCount )`**	
Description:	
Gets the count of how many user defined vectors are created.	
Input:	
None	None
Output:	
`INT4 nCount`	Count of user defined computed vectors.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetUserDefinedInfo

Unpublished work. © 2025 Siemens

GetUserDefinedInfo

**(nID, pbEnabled, pbRunNowEnabled, pnType, pnRangeType, pnRangeStart, pnRangeEnd, sTitle, pbSetInfo,
 pnCount, pdScale, pnVectorID)**

Description:

Gets information about user defined vectors.

Input:

INT4 nID	The ID of the User Define Vector, also the position..

Output:

BOOL pbEnabled	If True, user defined vector is enabled for Auto Run.
BOOL pbRunNowEnabled	If True, user define vector is enable for Run Now.
INT4 pnType	The type of user defined vector. 0= Linear Combination, 1 = RSS Combination, 2 = Max Envelope, 3 = Min Envelope, 4 = Absolute Max Envelope, 5 = API.
INT4 pnRangeType	Type of range for vector destination. 0 = Auto, 1 = Single, 2 = Range.
INT4 pnRangeStart	If pnRangeType = 2, the vector start ID to use.
INT4 pnRangeEnd	If pnRangeType = 2, the vector end ID to use.
STRING sTitle	The title of the user defined vector.
BOOL pbSetInfo	If pnType is an Envelope type, when True, the Store Location Vector option is turned on.
INT4 pnCount	If pnType is an Envelope or Combination type, the number of vectors Enveloped or Combined.
INT8 (pnCount – 1) pdScale	If pnType = 0 Linear Combination, the scale factors for the combination.
INT4 (pnCount – 1) pnVectorID	If pnType = 0 Linear Combination, the vector IDs to combine.

Return Code:

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	`nID` User Defined does not exist.
Remarks/Usage:	
None	
Example:	
None	

EnableUserDefined

`( nID, bSetAutoRun, bEnabled )`	
Description:	
Enables the User Defined computed vector creation options.	
Input:	
`INT4 nID`	The ID of the User Define Vector, also the position.
`BOOL bSetAutoRun`	If True, will be set for Auto Run, if False determines to Run Now.
`BOOL bEnabled`	If True, user define vector will be turned on.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	`nID` User Defined does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

EnableNowAll

EnableNowAll **(bEnableStd, bEnableUser)**	
Description:	
Enables all the standard or all the user defined Computed Vectors.	
Input:	
`BOOL bEnableStd`	If True, enables all Standard Computed Vectors.
`BOOL bEnableUser`	If True, enables all User Defined Computed Vectors.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

FindUserDefined

FindUserDefined **(sTitle, pnID)**
Description:
Method to return the ID of a computed vector by the title.
Input:

Unpublished work. © 2025 Siemens

`STRING sTitle`	Title of the vector.
Output:	
`INT4 pnID`	The ID of the User Define Vector, also the position.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

NewUserDefinedAPI

`NewUserDefinedAPI` **`( bEnabled, sTitle, sFilename, pnNewID )`**	
Description:	
This method creates a new API Computed Vector.	
Input:	
`BOOL bEnabled`	If True, user defined vector will be enabled.
`STRING sTitle`	Title of the user define vector.
`STRING sFilename`	Full path of the API file.
Output:	
`INT4 pnNewID`	Gives the ID of the newly created user define vector. Will be next available ID.
Return Code:	
`FE_NOT_EXIST`	API file name does not exist.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

NewUserDefinedCombine

NewUserDefinedCombine **(bEnabled, sTitle, bLinearCombine, nCount, dScale, nVectorID, pnNewID)**	
Description:	
Creates an Combination type Computed Vector (linear or RSS).	
Input:	
`BOOL bEnabled`	If True, user defined vector will be enabled.
`STRING sTitle`	Title of the user define vector.
`BOOL bLinearCombine`	If True, a linear combination is done, if False, an RSS combination is done.
`INT4 nCount`	The number of vectors to combine.
`INT8 (nCount – 1) dScale`	The scale factors to use for combination.
`INT4 (nCount – 1) nVectorID`	The vectors IDs to combine
Output:	
`INT4 pnNewID`	Gives the ID of the newly created user define vector. Will be next available ID.
Return Code:	
`FE_INVALID`	Not a valid nCount number.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

NewUserDefinedEnvelope

`NewUserDefinedEnvelope` **`( bEnabled, sTitle, nEnvType, bSaveSetInfod, nCount, nVectorID, pnNewID )`**	
Description:	
Creates an Envelope type Computed Vector.	
Input:	
`BOOL bEnabled`	If True, user defined vector will be enabled.
`STRING sTitle`	Title of the user define vector.
`INT4 nEnvType`	0 = Max Envelope, 1 = Min Envelope, 2 = Absolute Max Envelope.
`BOOL bSaveSetInfo`	When True, the Store Location Vector option is turned on.
`INT4 nCount`	Number of vectors to envelope.
`INT4 (nCount – 1) nVectorID`	The vectors IDs to envelope.
Output:	
`INT4 pnNewID`	Gives the ID of the newly created user define vector. Will be next available ID.
Return Code:	
`FE_INVALID`	Not a valid `nCount` number or `nEnvType` is invalid.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetUserDefinedVectorIDRange

SetUserDefinedVectorIDRange **(nID, nRangeType, nStartID, nEndID)**	
Description:	
Sets the vector ID range for API created user defined Computed Vectors.	
Input:	
`INT4 nID`	Vector ID for Single vector range type.
`INT4 nRangeType`	Vector range type; Auto, Single, Multiple
`INT4 nStartID`	Start ID for multiple vectors.
`INT4 nEndID`	End ID for multiple vectors.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

DeleteUserDefined

Unpublished work. © 2025 Siemens

DeleteUserDefined	
(nID)	
Description:	
Initializes the delete user defined icon tool in the User Defined tab of the Computed Results Vectors dialog.	
Input:	
`INT4 nID`	The ID of the user define computed vector definition to be deleted.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

RunNow

RunNow	
(nResultSetSet)	
Description:	
Computes vectors in the output set.	
Input:	
`INT4 nResultSetSet`	ID of the set object containing the IDs of output sets. Alternatively, a negative value will specify a single output set to calculate for.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Connection Property Objects

Connection Property objects correspond to the connection properties in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Connection Property objects. For more information, see **Common Entity Properties and Methods**.

Use the `feConnectionProp` method of the Femap Application object to create Connection Property objects.

Connection Property Object Properties

Connection Properties are always stored with a `SetID=1`, and the ID equal to the connection property ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The connection property color.
`INT4 layer`	The layer associated with the connection property.
`INT4 type`	Type of Connection, 0=Contact, 1=Glued.
`STRING title`	Connection property title (maximum 79 characters).
`BOOL flag[0..3]`	Array of connection property flags.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vflag`	

`INT4 flagI[0..3]`
or
`VARIANT vflagI`
	Array of connection property flags accessed as an INT4. Most flags are simply true/false and can be accessed with `flag( )`.

`REAL8 pval[0..399]`
or
`VARIANT vpval`
	Array of connection property values.

`INT4 plyval[0..1]`
or
`VARIANT vplyval`
	Array of material IDs.

Additional Properties	
`INT4 plymat[0..1]`	Same as `plyval`, but returns/sets entire array as a VARIANT.
`INT4 pmat[0..234]`	Same as `pval`, but returns/sets entire array as a VARIANT.

Accessing Connection Property Properties

You can access the individual properties for the Property object by referencing the properties described above, and the data contained in the table below. For example, to retrieve the coefficient of static friction in direction 1, use:

```
mu = pr.pval(0)
```


Unpublished work. © 2025 Siemens

This approach provides great flexibility, but can be slow if you are going to work with a large number of properties. In this case it is best to access the data using *Additional Properties*. These all set and return entire arrays of values in one reference. For example, you could write:

```
Dim pr As femap.ConnectionProp
Set pr = App.feConnectionProp
Dim v As Variant
pr.Get(1)
v = pr.pmat
v(0)=0.3
v(1)=0.35
pr.pmat = v
pr.Put(1)
```

This code loads Connection Property 1, retrieves the entire array of real values into **"v"**, updates the two coefficients of friction, puts the entire array back, and saves the updated Connection Property definition. While the code is slightly longer than simply accessing the individual properties, performance will be better using this approach.

Connection Property Array Index Values

In general, most of the property data is simply stored in the various array properties. The location of the specific property data is defined in the following table.

Property Values (continued)	CONTACT
Flags	
0	ABAQUS GAP CONDUCTANCE Active
1	ABAQUS GAP CONDUCTANCE is Pressure based
2	ABAQUS GAP RADIATION Active
Values	
0	Static Friction 1
1	Static Friction 2
2	Dynamic Friction
3	Decay
4	Viscous
5	Damping
6	

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
7	
8	
9	
10	
11	
12	
13	
14	
15	Abaqus Small Slide
16	Abaqus Hcrit
17	Abaqus Ext Zone
18	Abaqus Smoothing
19	Abaqus Tied
20	Abaqus Weight
21	Abaqus Thickness
22	Abaqus Slide Dist
23	Abaqus Friction Type
24	Abaqus Friction Value
25	Abaqus Max Shear
26	Abaqus Adjust
27	Abaqus Approach
28	
29	
30	Dyna Type (0=Default/General, 1=Automatic, 2=Old (Non-Automatic, 3=Constraint, 4=Eroding, 5=Force Transducer, 6=Forming, 7=Rigid, 8=Single Edge, 9=Sliding Only, 10=Tiebreak, 11=Tied, 12=Tied Shell Edge, 13=Tied Shell Edge to Surface)
31	Dyna One Way Contact
32	Dyna Offset

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
33	Dyna Penetration (0=Option Off, 1=Option On, 2=Both Penetration and Shortest Diagonal Options On)
34	Dyna Birth (Value of 0.0=Option Off, Value greater than 0.0=Option On, Value=Time)
35	Dyna Death (Value of 0.0=Option Off, Value greater than 0.0=Option On, Value=Time)
36	Dyna Scale Factors Slave Penalty Stiffness
37	Dyna Scale Factors Master Penalty Stiffness
38	Dyna Thickness Overrides Slave (Value of 0.0=Option Off, Value greater than 0.0=Option On, Value=Override Value)
39	Dyna Thickness Overrides Master (Value of 0.0=Option Off, Value greater than 0.0=Option On, Value=Override Value)
40	Dyna Scale Factors Slave Thickness
41	Dyna Scale Factors Master Thickness
42	Dyna Scale Factors Coulomb Friction
43	Dyna Scale Factors Viscous Friction
44	Dyna Output Slave Forces
45	Dyna Output Master Forces
46	Dyna Rigid Function Force vs. Penetration (ID of Femap Function)
47	Dyna Rigid Force Method (1=Total Force/Penetration, 2=Nodal Force/Penetration, 3=Pressure/Penetration, 4=Total Force/Penetration)
48	Dyna Rigid Unloading Stiffness
49	Dyna Tiebreak Normal Failure
50	Dyna Tiebreak Shear Failure
51	Dyna Tiebreak Normal Exponent
52	Dyna Tiebreak Shear Exponent
53	Dyna Tiebreak Function - Resisting Stress/Opening (ID of Femap Function)

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
54	Dyna Eroding Symmetry On
55	Dyna Eroding Interior Node Erosion
56	Dyna Eroding Interior Solid Erosion
57	Dyna Constraint (0=Symmetric Constraint, 1=Slave Nodes Constrained, 2=Master Nodes Constrained)
58	Dyna Friction Static
59	Dyna Friction Dynamic
60	Dyna Friction Exp. Decay
61	Dyna Friction Viscous
62	Dyna Friction Viscous Damping
...	
65	Marc Tolerance
66	Marc Sep Force
67	Marc Inter
68	Marc Disp
69	Marc Friction
70	Marc Stick Vel
71	Marc Fr Stress
72	Marc Con Type
73	Marc Con Shell
74	Marc Tol Bias
75	Marc Beam
76	Marc Sep
77	Marc Sep Max
78	Marc Sep Stress
79	Marc Slip Coeff
80	Marc Slip F
81	Marc Slip Trans
82	Marc Rig Avg

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
83	Marc Rig Cutoff
...	
100	Sinda Type
101	Sinda Coeff1
102	Sinda View
103	Sinda Coeff2
104	Sinda Cfunc1
105	Sinda Vfunc
106	Sinda Cfunc2
107	Sinda Cfunc3
...	
115	Ansys Closure
116	Ansys Min Pen
117	Ansys Max Pen
118	Ansys Close Stiff
119	Ansys Open Stiff
120	Ansys Norm Tol
121	Ansys Pinball
122	Ansys Surf Off
123	Ansys Max Fric
124	Ansys Penalty Type
125	Ansys Node Cont
126	Ansys Unsym
127	Ansys No Spurious
128	Ansys Shell Thick
129	Ansys Cont Surf
130	Ansys Init Pen
131	Ansys Time Inc
...	

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
135	NE/Nastran Scale Fact
136	NE/Nastran Fric Stiff
137	NE/Nastran Static Friction
138	NE/Nastran Pen Type
139	(9.2+) NE/Nastran Max Penetration
140	(9.2+) NE/Nastran Max Adjust Ratio
141	(9.2+) NE/Nastran Max Penetration Fraction
142	(9.2+) NE/Nastran Penetration Surface Offset
143	(9.2+) NE/Nastran Max Activation Distance
144	(9.2+) NE/Nastran Max Activation Distance (Automatic)
...	
145	NX/Nas601 Type
146	NX/Nas601 NSide
147	NX/Nas601 Birth Time
148	NX/Nas601 Death Time
149	NX/Nas601 Initial Penetration
150	NX/Nas601 Penetration Depth
151	NX/Nas601 Segment Normal
152	NX/Nas601 Offset Type
153	NX/Nas601 Offset Distance
154	NX/Nas601 Initial Penetration Duration
155	NX/Nas601 Consistant Stiffness
156	NX/Nas601 Tied
157	NX/Nas601 Tied Tolerance
158	NX/Nas601 Surface Extention Factor
159	NX/Nas601 Friction Model
160	NX/Nas601 Friction Parameter 1
161	NX/Nas601 Friction Parameter 2

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
162	NX/Nas601 Friction Parameter 3
163	NX/Nas601 Friction Parameter 4
164	NX/Nas601 Friction Parameter 5
165	NX/Nas601 Eps N
166	NX/Nas601 Eps T
167	NX/Nas601 CFactor 1
168	NX/Nas601 Penetration Tolerance
169	NX/Nas601 Normal Modulus
170	NX/Nas601 Tangential Modulus
171	NX/Nas601 Tensile Freeing Force
172	NX/Nas601 Total Freeing Force
173	(9.3+)NX/Nas601 DISP
174	(9.3+)NX/Nas601 GAPVAL
...	
175	(9.1+) NX/Nas101 Max Force Iterations
176	(9.1+) NX/Nas101 Max Status Iterations
177	(9.1+) NX/Nas101 Normal Penalty Factor
178	(9.1+) NX/Nas101 Tangential Penalty Factor
179	(9.1+) NX/Nas101 Force Convergence Tolerance
180	(9.1+) NX/Nas101 Number of Allow Contact Changes
181	(9.1+) NX/Nas101 Contact Precentage
182	(9.1+) NX/Nas101 Shell Offset
183	(9.1+) NX/Nas101 Contact Status
184	(9.1+) NX/Nas101 Friction
185	(9.1+) NX/Nas101 Min Contact Search Dist
186	(9.1+) NX/Nas101 Max Contact Search Dist
187	(9.1+) NX/Nas101 Averaging Method
188	(9.1+) NX/Nas101 Initial Penetration
189	(9.2+) NX/Nas101 Gluing Search Distance

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
190	(9.2+) NX/Nas101 Gluing Penalty Factor
191	(9.3+) NX/Nas101 Gluing Zoffset
192	(9.3+) NX/Nas101 INTORD
193	(9.3+) NX/Nas101 REFINE
194	(10.0+) NX/Nas101 NCHG TYPE
195	(10.0+) NX/Nas101 PENTYPE
196	(10.0+) NX/Nas101 CSTRAT
197	(10.1+) NX/Nas101 GLUETYPE
198	(10.1+) NX/Nas101 PENGLUE
199	(10.1+) NX/Nas101 PENAUTO
...	
200	(9.2+) NX/Nas701 Friction
201	(9.2+) Marc Friction
202	(9.2+) AnsyNX/Nas601s Friction
203	(10.0+)NX/Nas601BOND_EXT
204	(10.0+)NX/Nas601SLIDVEL
205	(10.0+)NX/Nas601OCHECK
206	(10.0+)NX/Nas601GAPBIAS
207	(10.0+)NX/Nas601OFFDET
208	(10.0+)NX/Nas601RTPCHECK
209	(10.0+)NX/Nas601RTPMAX
210	(10.0+)NX/Nas601FRICDLY
211	(10.0+)NX/Nas601TFORCE
212	(10.0+)NX/Nas701 SLIDVEL
213	(10.0+)NX/Nas701 OFFDET
214	(10.0+)NX/Nas701 GAPBIAS
215	(10.1+) NX/Nas101 PENADAPT
216	(10.1+)NX/Nas101 PENETFAC
217	(10.3+)Dyna A Soft Constraint

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
218	(10.3+)Dyna A Soft Constraint Scale Factor
219	(10.3+)Dyna A Airbag Load Curve ID
220	(10.3+)Dyna A Maximum Parametric Segment Search
221	(10.3+)Dyna A Segment Based Contact Options
222	(10.3+)Dyna A Search Depth
223	(10.3+)Dyna A Number of Cycles Between Sorts (or Frequency Curve IDx (-1))
224	(10.3+)Dyna A Cycles Between Contact Force Updates
225	(10.3+)Dyna B Maximum Penetration Distance
226	(10.3+)Dyna B Thickness Option
227	(10.3+)Dyna B Shell Thickness Considerations
228	(10.3+)Dyna B Shooting Node Logic (0=Enabled)
229	(10.3+)Dyna B Symmetry Plane (0=Off)
230	(10.3+)Dyna B Primary Segment Searching Option (0=2D; 1=3D)
231	(10.3+)Dyna B Solid Element Thickness
232	(10.3+)Dyna B Solid Element Stiffness
233	(10.3+)Dyna C Improve Implicit Convergence (1=On; 2=Off)
234	(10.3+)Dyna C Ignore Initial Penetration Options
235	(10.3+)Dyna C Penetration Depth Reduction Factor (or Time Curve ID X(-1))
236	(10.3+)Dyna C Stiffness Time Step (or Time Curve IDx (-1))
237	(10.3+)Dyna C Angle Tolerance for Feature Line (radians)
238	(10.3+)Dyna D Split Quadrilateral Contact Segments into 2 triangles
239	(10.3+)Dyna D Time Interval between Shell Penetration Reports
240	(10.3+)Dyna D Scale Factor for Neighbor Segment Contact

Unpublished work. © 2025 Siemens

Property Values (continued)	CONTACT
241	(10.3+)Dyna D Incremental Displacement Update for Tied Contacts (1=On)
242	(10.3+)Dyna ABCD Level (0=Off, 1=A, 2=AB, 3=ABC, 4=ABCD)
243	(10.3+)NE/Nastran Maximum Normal Activation Distance
244	(10.3+)NE/Nastran Maximum Radial Activation Distance
245	(10.3+)NE/Nastran Maximum Allowable Slip
246	(10.3+)NE/Nastran MAXAD OR MAXNAD/MAXRAD toggle
262	(2019.1+)ABAQUS GAP CONDUCTANCE: Function defining Conductance, Gap Clearance, Temperature relationship
263	(2019.1+)ABAQUS GAP RADIATION: Emissivity A
264	(2019.1+)ABAQUS GAP RADIATION: Emissivity B
265	(2019.1+)ABAQUS GAP RADIATION: Function defining Effective View Factor, Gap Clearance relationship
...	
400	(2019.1+)Adaptively Modify Penalty Factor (1=On)
401	(2019.1+)Adjustment Tolerance
402	(2019.1+)Const Offset Distance
403	(2019.1+)End Time of Initial Penetration
404	(2019.1+)Sliding Contact Formulation 0=Automatically Update Contact Pairing 1=Update Contact Pairing Based on Initial Geometry 2=Update Contact Pairing with Sliding Displacements

Connection Property Object Methods

The following methods are available in addition to the standard entity methods.

Unpublished work. © 2025 Siemens

SetDefaults

`SetDefaults`
 	
`( ContactTolerance )`	

Description:	
This method initializes default values for the connection property.	
Input:	
`REAL8 ContactTolerance`	Specifies the **Max Contact Search Distance** for NX Nastran Linear Contact, the **Search Distance** for NX Nastran Glued Contact, and the **Tied Tolerance** for NX Nastran Advnaced Nonlinear Contact. If you specify 0, Femap will automatically calculate the value (10 x model merge tolerance).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This is a simple method that allows default values to be placed into the object.	
Example:	
None	

Connection Region Objects

Connection Region objects correspond to the connection regions in your model (in earlier Femap versions, they were called Contact Objects and Contact Segments). They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Connection Region objects. For more information, see **Common Entity Properties and Methods**.

Use the `feConnectionRegion` method of the Femap Application object to create Connection Region objects. For compatibility with earlier releases, you can also use `feContact` to create this object, however this method is obsolete.

Unpublished work. © 2025 Siemens

Connection Region Object Properties

Connection Regions are always stored with a `SetID=1`, and the ID equal to the region ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
Connection Region Options	
`INT4 color`	The connection region color.
`INT4 layer`	The layer associated with the connection region.
`STRING title`	The connection region title (maximum 79 characters).
`INT4 refNode`	Reference node ID.
`INT4 output`	Output on nodes(7) or elements(8).
`BOOL rigid`	Flag to indicate rigid segment.

`REAL8 box[0..1][0..2]`
or
`VARIANT vbox`
	Locations at corners of contact box: `box[0][xyz]` is the minimum dimension, `box[1][xyz]` is the maximum.

`INT4 count`	The number of items in the connection region contact list.
`INT4 type`	Type of Region (0=Normal/Contact, 1=Fluid, 2=Bolt, 3=Rotor).
`INT4 refCSys`	Reference Coordinate System.
`BOOL Disabled`	Flag to indicate a region is disabled.
Region Options button - Simcenter Nastran tab	

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 offset`	Region Offset Distance.
Region Options button - MSC Nastran tab	
`INT4 SurfaceContactFlag`	Flag to indicate how body surfaces may contact. Writes COPTB and corresponding value to BCBDPRP entry.
`INT4 FrictionType`	0=Friction Value (FRIC), 1=Friction Table (TABLE)
`REAL8 FrictionValue`	Specifies a friction coefficient and writes FRIC and a real number to BCBDPRP entry.
`INT4 FrictionTable`	Specify the ID of an existing function. Writes FRIC and the integer ID value to BCBDPRP.
`INT4 SmoothingControl`	Writes IDSPL and corresponding value to BCBDPRP entry. Default=0.
`INT4 MidNodeProjection`	Writes MIDNOD and corresponding value to BCBDPRP entry. Only used when Smoothing Control IDSPL does not equal 0.
`REAL8 DiscontinuityValue`	Writes SANGLE and corresponding value to BCBDPRP entry. Default=60.0.
`INT4 ContactCheck`	Writes ISTYP to BCBDPRP entry: 0=Check Each Body, 1=Optimized Contact. Default=0.
Region Options button - MSC Nastran tab - Rigid Options button	
`INT4 ControlNode`	ID of the Control Node (CGID) for Rigid Motion Control.
`INT4 MotControlType`	Control Type (CONTROL) for Rigid Motion Control. -1=Position Control, 0=Velocity Control, 2=Load Control
`INT4 RotControlNode`	ID of the Rotation Control Node (NLOAD) for Rigid Motion Control.
`BOOL AngVelFlag`	When True, sets Angular Velocity Value (ANGVEL) option.
`INT4 AngVelTable`	Writes ID of existing function to (ANGVEL), VX (VELRB1), VY (VELRB2), and/or VZ (VELRB3) fields as an integer.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 AngVelValue`	Set or get the Angular Velocity Value (ANGVEL) option.
`BOOL RotAxisFlag[0..2]`	Set or get, Rotation Axis options, whether values or time dependent. For example, to set on X (DCOS1), use `RotAxisFlag[0]=1`.
`INT4 RotAxisTable[0..2]`	Set or get the ID(s) of existing function(s) as integers. Specify a function ID at index [0] for DCOS1, a function ID at index [1] for DCOS2, a function ID at index [2] for DCOS3.
`REAL8 RotAxisValue[0..2]`	Set or get the real values for X,Y, Z Rotation Axis options. Specify a real value at index [0] for X (DCOS1), a real value at index [1] for Y (DCOS2), a real value at index [2] for Z (DCOS3).
`BOOL MotionFlag[0..2]`	Set or get, Angular Velocity (VELRBX) options. For example, to set on VX (VELRB1), use `MotionFlag[0]=1`.
`INT4 MotionTable[0..2]`	Set or get the ID(s) of existing function(s) as integers. Specify a function ID at index [0] for Time Dependent VX, a function ID at index [1] for Time Dependent VY, a function ID at index [2] for Time Dependent VZ.
`REAL8 MotionValue[0..2]`	Set or get the real values for VX,VY, VZ Motion Value Angular Velocity options. Specify a real value at index [0] for VX (VELRB1), a real value at index [1] for VY (VELRB2), a real value at index [2] for VZ (VELRB3).
`REAL8 AppAngVelVal`	Set or get the value of Approach Angular Velocity.
`REAL8 AppRotAxisVal[0..2]`	Set or get the real values for X (N1),Y (N2), Z (N3) of the Approach Velocity Rotation Axes.
`REAL8 AppVelVal[0..2]`	Set or get the real values for X (V1),Y (V2), Z (V3) of the Approach Velocity.
`REAL8 GrowthComp[0..2]`	Set or get the real values for Growth Factor options. Specify a real value at index [0] for Component 1 (GF1), a real value at index [1] for Component 2 (GF2), a real value at index [2] for Component 3 (GF3).
`INT4 GrowthTable[0..2]`	Set or get the ID(s) of existing function(s) as integers. Specify a function ID at index [0] for Table 1 (TAB-GF1), a function ID at index [1] for Table

Unpublished work. © 2025 Siemens

Property	Description
	2 (TAB-GF2), a function ID at index [2] for Table 3 (TAB-GF3).
Region Options button - ABAQUS tab	
This is done via `SetAnalyticSurfaceType` and `GetAnalyticSurfaceType`.	
Fluid Region Options	
`BOOL FluidZFreeOn`	True = Fluid Z Free Surface is On.
`REAL8 FluidZFree`	Z-Depth of Fluid Free Surface.
`REAL8 FluidDensity`	Density of Fluid.
`INT4 FluidXZSymmetry`	Symmetry Condition for XZ Plane (0=None, 1=Symmetric, 2=Antisymmetric).
`INT4 FluidYZSymmetry`	Symmetry Condition for YZ Plane (0=None, 1=Symmetric, 2=Antisymmetric).
`REAL8 FluidCharLength`	Fluid Characteristic Length.
`REAL8 FluidExactIntegFact`	Exact Integration factor for fluid.
`BOOL IsINCMPFL`	When True, activates the Incompressible Fluid Surface options in the Fluid Region dialog box.
`INT4 INCMPFLSurfType`	Sets the Free Surface option of the Fluid Region dialog box. The choices are 0..Allow sloshing, 1..Disallow sloshing, and 3..Treat the fluid as completely enclosed by the structure.
`REAL8 INCMPFLGrav`	Value to be written for the Gravity (acceleration due to gravity) option of the Fluid Region dialog box (Real > 0.0).
`INT4 INCMPFLCmsModes`	Value to be written for the Comp. Modes (number of modes associated with the sloshing surface after component mode synthesis CMS reduction) option of the Fluid Region dialog box (Integer > 0).
Rotor Region Options	
`INT4 RotorLoadSet`	ID of Load Set containing Rotational body loads to be applied.
`REAL8 RotorOverallDamp`	Frequency for Overall Damping (W3).

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 RotorMatlDamp`	Frequency for Material Damping (W4).
`REAL8 RotorSpeed`	Rotor speed (not currently used).
NonStructural Mass Region Options	
`INT4 MassType`	The type of Nonstructural Mass Region 0=Mass per length, 1=mass per area, 2=total mass on length, 3=total mass on area.
`REAL8 MassNSM`	Nonstructural mass value.
Bolt Region Options	
`INT4 BoltType`	Type of bolt 1=Beam 2=Solid.
`INT4 BoltDir`	Bolt Direction 1=X, 2=Y, 3=Z.
`BOOL UseSolverAxes`	If True, allows Nastran to automatically determine bolt axis/direction (CSID, IDIR fields) for BOLT entries. Valid for Bolt Type: Solid.

Connection Region Object Methods

The following methods are available in addition to the standard entity methods.

GetContactList

`GetContactList`	
`(  listcount, listTYPE, entityID, faceID, expanded )`	
Description:	
This method retrieves information from the contact list in a connection region.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`INT4 listcount`	The number of entries in the other arrays.
`INT4 listTYPE[0..N]`	The type of entity in each list entry. Refer to **Entity Types** for a list of types.
`INT4 entityID[0..N]`	The ID of the entity in each list entry.
`INT4 faceID[0..N]`	The ID of the face referenced by each entity (if applicable). For Elements, this typically should specify the element face that is in the region (1 through 6). However, in cases like Bolt and NonStructural Mass Regions, along with Connection Regions where Output is set to *Element – No Faces*, faceID should be set to 0 to indicate the entire element is in the Region. For Curves and Surfaces, faceID 1 = Positive Side, faceID 2 = Negative Side.
`INT4 expanded[0..N]`	Flag indicating that this is an expanded node/element entry from a geometry contact.
Return Code:	
`FE_NOT_EXIST`	There are no entries in the list.
Remarks/Usage:	
This method simply retrieves the information about the contact lists from the current object. You must first load that data - either using the `Get( )` method, or by loading it yourself with `PutContactList( )`.	
Example:	
None	

PutContactList

`PutContactList`
`( listcount, listTYPE, entityID, faceID, expanded )`
Description:

Unpublished work. © 2025 Siemens

This method loads information into the contact list for a connection region.	
Input:	
`INT4 listcount`	The number of entries in the other arrays.
`INT4 listTYPE[0..N]`	The type of entity specified in each entry. Refer to **Entity Types** for a list of types.
`INT4 entityID[0..N]`	The ID of the entity specified in each entry.
`INT4 faceID[0..N]`	The ID of the face referenced by each entity (if applicable). For Elements, this typically should specify the element face that is in the region (1 through 6). However, in cases like Bolt and NonStructural Mass Regions, along with Connection Regions where Output is set to *Element – No Faces*, faceID should be set to 0 to indicate the entire element is in the Region. For Curves and Surfaces, faceID 1 = Positive Side, faceID 2 = Negative Side.
`INT4 expanded[0..N]`	Flag indicating whether each entry is an expanded node/element entry from a geometric contact.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method stores information into the list of the current object. Any previous list information in that object is lost. This method does not store that information to the model database. After using this function, you must call `Put( )` if you want to store the updated list in the database.	
Example:	
None	

GetEntities

Unpublished work. © 2025 Siemens

GetEntities

(entityTYPE, bGetExpanded)

Description:

This retrieves a Sort Object (SortSet) that contains all of the entities of a given type referenced by the region.

Input:

`INT4 entityTYPE`	The type of entity that you want to select. Refer to **Entity Types** for a list of types.
`BOOL bGetExpanded`	If True, the region is first expanded - any selected geometric entities will also select associated nodes or elements.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

Once a Sort Object has been populated with entities, the entity ID can be accessed via the `nInt1` property of the Sort Object, the FaceID via the `nInt2` property, and the Expansion Status via the `nInt3` property.

Example:

This method must be used with a Set statement to define the Sort object (`SortSet`).

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim regn As femap.ConnectionRegion
   Set regn = App.feConnectionRegion
   regn.Get(1)
   Dim sorts As femap.SortSet
   Set sorts = regn.GetEntities(FT_ELEM, True)
   Dim s As femap.Set
```


Unpublished work. © 2025 Siemens

```
   Set s = App.feSet
   While sorts.Next() <> FE_FAIL
       s.Add(sorts.nInt1)
   Wend
End Sub
```

This sequence creates the **"s"** Set object with the list of elements that are referenced by Region 1.

GetEntitiySet

GetEntitiySet

(entityTYPE, bGetExpanded)

Description:	
This retrieves a set that contains all of the entity IDs of a given type referenced by the region.	
Input:	
`INT4 entityTYPE`	The type of entity that you want to select. Refer to **Entity Types** for a list of types.
`BOOL bGetExpanded`	If True, the region is first expanded - any selected geometric entities will also select associated nodes or elements.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

This method must be used with a Set statement to define the Set object.

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim regn As femap.ConnectionRegion
   Set regn = App.feConnectionRegion

   regn.Get(1)

   Dim s As femap.Set
   Set s = regn.GetEntitySet(FT_ELEM, True)

End Sub
```

This sequence creates the **"s"** Set object with the list of elements that are referenced by Region 1.

AddContact

`AddContact`

`( listTYPE, entityID, faceID )`

Description:

This method loads information into the contact list for a connection region. Unlike `PutContactList`, this method loads one entry at a time.

Input:

`INT4 listTYPE`	The type of entity to be added. Refer to **Entity Types** for a list of types.

Unpublished work. © 2025 Siemens

`INT4 entityID`	The ID of the entity specified.
`INT4 faceID`	The ID of the face referenced by each entity (if applicable). For Elements, this typically should specify the element face that is in the region (1 through 6). However, in cases like Bolt and NonStructural Mass Regions, along with Connection Regions where Output is set to *Element – No Faces*, faceID should be set to 0 to indicate the entire element is in the Region. For Curves and Surfaces, faceID 1 = Positive Side, faceID 2 = Negative Side.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method stores information into the list of the current object. After using this function, you must call `Put( )` if you want to store the updated list in the database.	
Example:	
None	

SetOutputType

`SetOutputType` **`( entityTYPE )`**	
Description:	
This method sets the output type for the connection region.	
Input:	
`INT4 entityTYPE`	The type of entity for output. Refer to **Entity Types** for a list of types.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
For both Elements and Elements - No Faces, `entityTYPE` should be set to `FT_ELEM`. When Face ID = 0 for the element IDs, that indicates the region has output set to Elements - No Faces.	
Example:	
None	

SetOffset

`SetOffset` **`( offset )`**	
Description:	
This method sets the offset for the connection region.	
Input:	
`REAL8 offset`	The value of the region offset.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetOffset

GetOffset	
(offset)	
Description:	
This method returns the offset for the connection region.	
Input:	
None	None
Output:	
`REAL8 offset`	The value of the region offset.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

SetFriction

SetFriction	
(body_friction)	
Description:	
This method sets the friction for the connection region.	
Input:	
`REAL8 body_friction`	The value of the region friction.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetFriction

`GetFriction` **`(  body_friction )`**	
Description:	
This method returns the friction for the connection region.	
Input:	
None	None
Output:	
`REAL8 offset`	The value of the region friction.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

SetAnalyticSurfaceType

Unpublished work. © 2025 Siemens

SetAnalyticSurfaceType **(body_friction)**	
Description:	
This method sets the Analytic Surface Type for the connection region.	
Input:	
`INT4 aba_rigSurfType`	The type of Analytic Rigid Surface. Refer to **Femap Constants**: `0=FARS_CYLINDER`, `1=FARS_REVOLUTION`, `2=FARS_SEGMENTS`.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetAnalyticSurfaceType

GetAnalyticSurfaceType **(void)**	
Description:	
This method returns the Analytic Surface Type for the connection region.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`INT4`	0=CYLINDER, 1=SEGMENTS, 2=REVOLUTION
Remarks/Usage:	
None	
Example:	
None	

SetRigidType

`SetRigidType` **`( offset )`**	
Description:	
This method defines whether a connection region is rigid.	
Input:	
`BOOLEAN isRigid`	Set to True for a rigid region, False otherwise.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

IsRigidType

Unpublished work. © 2025 Siemens

IsRigidType	
(void)	
Description:	
This method indicates whether a connection region is rigid.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_FAIL	Region is not rigid (FE_OK = Rigid).
Remarks/Usage:	
None	
Example:	
None	

Expand

Expand	
(void)	
Description:	
This method expands all of the geometric connection regions in the model.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	The specified expansion did not work.
Remarks/Usage:	
This method expands all connection regions, not just the one in the current object.	
Example:	
None	

Compress

`Compress` **`( void )`**	
Description:	
This method compresses all of the geometric connection regions in the model.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	The specified compression did not work.
Remarks/Usage:	
This method compresses all connection regions, not just the one loaded into the current object.	
Example:	
None	

Clear

Unpublished work. © 2025 Siemens

`Clear`	
`( void )`	
Description:	
This method clears and resets the contents of the connection region.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Reverse

`Reverse`	
`( void )`	
Description:	
This method reverses the direction of the connection region.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Enable

`Enable` **`( bEnable )`**	
Description:	
This method is used to activate or deactivate a Connection, Fluid, Bolt, NonStructrual Mass, or Rotor region.	
Input:	
`BOOL bEnable`	Flag to set the state of the connection region.
Output:	
None	None
Return Code:	
`FE_FAIL`	Region is not proper type.
Remarks/Usage:	
None	
Example:	
None	

IsEnabled

Unpublished work. © 2025 Siemens

IsEnabled	
(void)	
Description:	
This method checks the state of the connection region.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Region is disabled.
`FE_OK`	Region is enabled.
Remarks/Usage:	
None	
Example:	
None	

BoundingBox

BoundingBox	
(Box)	
Description:	
This method returns the coordinates of a rectangular box that encloses a connection region.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`REAL8 Box[0..5]`	The dimensions of the bounding box, in Global Rectangular coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
`FE_INVALID`	Connection Region contains invalid entity types.
Remarks/Usage:	
This method always retrieves a box that is aligned with Global Rectangular coordinates, not an aligned box of minimum volume.	
Example:	
None	

BoundingBoxInCSys

`BoundingBoxInCSys` **`( nCSysID, Box )`**	
Description:	
This method returns the coordinates of a rectangular box that encloses a connection region, in coordinates of the selected coordinate system.	
Input:	
`INT4 nCSysID`	ID of coordinate system to return minimum and maximum
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in “coordinate system specified with `nCSysID`” coordinates.

Unpublished work. © 2025 Siemens

	Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
`FE_INVALID`	Connection Region contains invalid entity types.
Remarks/Usage:	
This method always retrieves a box that is aligned with the selected coordinate system, not an aligned box of minimum volume.	
Example:	
None	

IsEmpty

`IsEmpty` **`( void )`**	
Description:	
This method checks if a connection region has no entities selected, thus is empty.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Region is not empty (i.e., has entities).
`FE_OK`	Region is empty.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

RemoveDuplicates

`RemoveDuplicates`	
`( void )`	
Description:	
This method removes any duplicate entities which exist in a region.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Could not create the reduced list.
`FE_NOT_AVAILABLE`	There are no selected surfaces/elements/etc in the region.
FE_OK	Region contains no duplicates.
Remarks/Usage:	
None	
Example:	

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim c As ConnectionRegion
```


Unpublished work. © 2025 Siemens

```
    Set c = App.feConnectionRegion


    While c.Next()
        c.RemoveDuplicates()
        c.Put(c.ID)
    Wend
End Sub
```

NodesAsSet

NodesAsSet

(bExpandGeom, nNodeSetID, bClear)

Description:	
This method adds the IDs of nodes referenced by the region (Connection, Bolt, or Rotor only) currently loaded in the ConnectionRegion Object to a specified Set. There is an option to include the IDs of nodes which are referenced by a region after geometric portions of the region are expanded.	
Input:	
`BOOL bExpandGeom`	When True, the IDs of any nodes which are referenced by a region after geometric portions of the region are expanded will be added to the Set.
`INT4 nNodeSetID`	ID of Set object to add node IDs.
`BOOL bClear`	When True, any IDs which are in the set are removed before adding the node IDs to the Set.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

ElementsAsSet

`ElementsAsSet` **`( bExpandGeom, nElemSetID, bClear )`**	
Description:	
This method adds the IDs of elements referenced by the region (any Type other than Rotor) currently loaded in the ConnectionRegion Object to a specified Set. There is an option to include the IDs of elements which are referenced by a region after geometric portions of the region are expanded.	
Input:	
`BOOL bExpandedGeomLoads`	When True, the IDs of any elements which are referenced by a region after geometric portions of the region are expanded will be added to the Set.
`INT4 nElemSetID`	ID of Set object to add element IDs.
`BOOL bClear`	When True, any IDs which are in the set are removed before adding the element IDs to the Set.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Connection Objects

Connection objects correspond to the connections in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Connection objects. For more information, see **Common Entity Properties and Methods**.

Use the `feConnection` method of the Femap Application object to create Connection objects.

Connection Object Properties

Connections are always stored with a `SetID=1`, and the ID equal to the connection ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The connection color.
`INT4 layer`	The layer associated with the connection.
`INT4 propID`	ID of the connection property referenced by the connection.
`STRING title`	The title of the connection (max 79 char).

`INT4 contact[0..1]`
or
`VARIANT vcontact`
	The IDs of the Primary and Secondary connection regions.

`BOOL IsCombination`	Flag to indicate whether a connection is made of referenced connectors.

Connection Object Methods

The following methods are available in addition to the standard entity methods.

Unpublished work. © 2025 Siemens

Prop

Prop	
(void)	
Description:	
This method creates and returns a Connection Property object that is loaded with the property data referenced by a connection.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Enable

Enable	
(bEnable)	
Description:	
This method activates and deactivates connections.	
Input:	
`BOOL bEnable`	Activate connection if True, deactivate it if False.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Activated connections are translated for analysis, deactivated ones are not.	
Example:	
None	

IsEnabled

`IsEnabled`	
`( void )`	
Description:	
This method indicates whether a connection is activated.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Indicates connection is deactivated (`FE_OK` if activated).
Remarks/Usage:	
None	
Example:	
None	

BoundingBox

Unpublished work. © 2025 Siemens

BoundingBox	
(Box)	
Description:	
This method returns the coordinates of a rectangular box that encloses a connector.	
Input:	
None	None
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in Global Rectangular coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
Remarks/Usage:	
This method always retrieves a box that is aligned with Global Rectangular coordinates, not an aligned box of minimum volume.	
Example:	
None	

BoundingBoxInCSys

BoundingBoxInCSys
(nCSysID, Box)
Description:
This method returns the coordinates of a rectangular box that encloses a connector, in coordinates of the selected coordinate system.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nCSysID`	ID of coordinate system to return minimum and maximum.
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in Global Rectangular coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
Remarks/Usage:	
This method always retrieves a box that is aligned with the selected coordinate system, not an aligned box of minimum volume.	
Example:	
None	

ConnectionsByType

`ConnectionsByType` **`( bGlue, bAll, nConnectorSetID, bClear )`**
Description:
Add the IDs of connector(s) to a user-created set object based on contact or glued types.
Input:

`BOOL bGlue`	If set to False, only connector(s) with a contact type property will be added to the `nConnectorSet`. If set to True, only connector(s) with a glued type property will be added to the set specified by `nConnectorSetID`.

Unpublished work. © 2025 Siemens

`BOOL bAll`	If set to True, `bGlue` is ignored and all connector(s) of either type are added to the set specified by `nConnectorSetID`.
`LONG nConnectorSetID`	The set ID of a Femap Set Object to be populated.
`BOOL bClear`	If set to True, the set specified by `nConnectorSetID` will be cleared before new connector ID(s) are added.
Output:	
None	None
Return Code:	
`INT4 rc`	The number of IDs added to the set specified by `nConnectorSetID`.
Remarks/Usage:	
See `CombinationsByType` for similar example.	
Example:	
None	

CombinationsByType

`CombinationsByType` **`( bGlue, bAll, nConnectorSetID, bClear )`**	
Description:	
Add the IDs of connector combination(s) (referenced) to a user-created set object based on contact or glued types.	
Input:	
`BOOL bGlue`	If set to False, only connector combination(s) that have referenced connectors with a contact type property will be added to the `nConnectorSet`. If set to True, only connector combination(s) that have referenced connectors with a glued type property will be added to the set specified by `nConnectorSetID`.

Unpublished work. © 2025 Siemens

`BOOL bAll`	If set to True, `bGlue` is ignored and all connector combinations of either type are added to the set specified by `nConnectorSetID`.
`LONG nConnectorSetID`	The set ID of a Femap Set Object to be populated.
`BOOL bClear`	If set to True, the set specified by `nConnectorSetID` will be cleared before new connector combination ID(s) are added.
Output:	
None	None
Return Code:	
`INT4 rc`	The number of IDs added to the set specified by `nConnectorSetID`.
Remarks/Usage:	
In the example below, `rc` would equal `cSet.Count()` because `cSet` starts empty.	
Example:	

```
' ================================================
' ** Need existing Connectors and Connector Sets before using
' ================================================
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim connectionObj As femap.Connection
   Dim cSet As femap.Set
   Dim rc As femap.zReturnCode
   Set connectionObj = App.feConnection
   Set cSet = App.feSet
' ================================================
' cSet is an empty FEMAP Set Object that will be populated with
' the ID(s) of glued connection combinations
' ================================================
```


Unpublished work. © 2025 Siemens

```
   rc = connectionObj.CombinationsByType(True,False,cSet.ID,True)
End Sub
```

IsGlueConntor

IsGlueConntor **(nConnectorID)**	
Description:	
Determines if connector has a glued property type or not.	
Input:	
`INT4 nConnectorID`	The ID of the connector to check.
Output:	
None	None
Return Code:	
`BOOL`	True if connector has a glue property type.
Remarks/Usage:	
None	
Example:	
None	

GetCombinations

GetCombinations **(nCombinationsSetID)**
Description:

Unpublished work. © 2025 Siemens

Populates a set object with the IDs of all connector combinations (referenced set) that use one particular connector.	
Input:	
`INT4 nCombinationsSetID`	The ID of a set to populate with the ID(s) of all connector combinations (connector referenced sets) the connector exists in.
Output:	
None	None
Return Code:	
`INT4 rc`	The number of IDs added to the set specified by `nCombinationsSetID`.
Remarks/Usage:	
This call must be proceeded by a `.Get` on any valid connector object. The set specified by `nCombinationSetID` is always cleared before populating.	
Example:	
None	

CSys Objects

CSys objects correspond to the coordinate systems in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for CSys objects. For more information, see **Common Entity Properties and Methods**.

Use the `feCSys` method of the Femap Application object to create CSys objects.

CSys Object Properties

CSys are always stored with a `SetID=1`, and the ID equal to the coordinate system ID.

The predefined global coordinate systems cannot be accessed or modified using this object. Therefore you must always specify an ID > 2.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 x`	The coordinates of the origin of the coordinate system. They are always specified in global rectangular coordinates.
`REAL8 y`	
`REAL8 z`	
`REAL8 rotx`	The rotations, in degrees, of the coordinate system axes, from the global coordinate axes. These are ordered rotations about X, then the rotated Y axis, then the doubly-rotated Z. No matter what definition coordinate system is used, these are always relative to global coordinates.
`REAL8 roty`	
`REAL8 rotz`	
`INT4 layer`	The ID of the layer associated with the coordinate system.
`INT4 color`	The coordinate system color.
`INT4 type`	The coordinate system type: 0=Rectangular, 1=Cylindrical, 2=Spherical.
`INT4 dcefCSys`	The ID of the definition coordinate system.
`STRING title`	Coordinate system title (maximum 79 characters).
Additional Properties	
`REAL8 origin[0..2]`	Accesses all three coordinates of the origin as a Variant. Same as using x, y, and z independently.
`REAL8 rotation[0..2]`	Accesses all three components of the rotations as a Variant. Same as using rotx, roty, and rotz independently.
`REAL8 matrix[0..8]`	The 3 x 3 direction cosine matrix that corresponds to the orientation of the coordinate system in global rectangular. The rows of the array are stored sequentially.

CSys Object Methods

Unpublished work. © 2025 Siemens

XYPoints

XYPoints **(origin, xaxis, xyplane)**	
Description:	
This method automatically updates the origin and rotations of a coordinate system based on three locations that you specify.	
Input:	
`REAL8 origin[0..2]`	The origin of the coordinate system. This, and all other locations, must be specified in global rectangular coordinates.
`REAL8 xaxis[0..2]`	The coordinates of a point along the X Axis of the coordinate system.
`REAL8 xyplane[0..2]`	The coordinates of a point in the XY Plane of the coordinate system. This point does not need to lie along any of the coordinate axes.
Output:	
None	None
Return Code:	
`FE_FAIL`	The operation failed. You probably did not correctly specify one of the coordinate locations.
Remarks/Usage:	
None	
Example:	
None	

Axis

Unpublished work. © 2025 Siemens

Axis

(nCSysType, nAxisDir, origin, Axis)

Description:

This method creates or updates a coordinate system by specifying a location on the X, Y, or Z Axis.

Input:

`INT4 nCSysType`	Coordinate system type. 0=Rectangular, 1=Cylindrical, 2=Spherical
`INT4 nAxisDir`	Specifies which axis of the coordinate system will be aligned with the location specified in Axis: 0 - X Axis 1 - Y Axis 2 - Z Axis
`REAL8 origin[0..2]`	The origin of the coordinate system. Must be specified in Global Rectangular coordinates. Must be specified in Global Rectangular coordinates.
`REAL8 Axis[0..2]`	The XYZ coordinates of a point along the Axis of the coordinate system specified by `nAxisDir`.

Output:

None	None

Return Code:

`FE_BAD_TYPE`	Invalid value for coordinate system type specified.
`FE_BAD_DATA`	Invalid value for axis direction (`nAxisDir`) specified.

Remarks/Usage:

When specifying a point on the X, Y or Z axes, a corresponding point on the XY, YZ, or ZX plane, respectively, is automatically determined by finding the closest orthogonal global axis. This method is best used when only the direction of the specified axis is of consequence. For additional control over the direction of the normal axis, use either or XYPoints (**XYPoints** or TwoAxes (**TwoAxes**).

Example:

None

Unpublished work. © 2025 Siemens

TwoAxes

TwoAxes

(nCSysType, nNormalAxisDir, origin, axis1, axis2)

Description:

This method creates or updates a coordinate system by specifying two vectors which are used in a defined manner based on an option specified by the user.

Input:

`INT4 nCSysType`	Coordinate system type. 0=Rectangular, 1=Cylindrical, 2=Spherical
`INT4 nNormalAxisDir`	Specifies the normal axis and how axis1 and axis2 will be used. 0 - X = normal, axis1 = Y, axis2 = YZ plane 1 - Y = normal, axis1 = X, axis2 = XZ plane 2 - Z = normal, axis1 = X, axis2 = XY plane
`REAL8 origin[0..2]`	The origin of the coordinate system. Must be specified in Global Rectangular coordinates.
`REAL8 axis1[0..2]`	The XYZ components of the vector of Axis 1. Must be specified in Global Rectangular coordinates.
`REAL8 axis2[0..2]`	The XYZ components of the vector of Axis 2. Must be specified in Global Rectangular coordinates.

Output:

None	None

Return Code:

`FE_BAD_TYPE`	Invalid value for coordinate system type specified.
`FE_BAD_DATA`	Invalid value for normal axis direction specified.

Remarks/Usage:

Unpublished work. © 2025 Siemens

None
Example:
None

Curve Objects

Curve objects correspond to the curves in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for curve objects. For more information, see **Common Entity Properties and Methods**.

Use the `feCurve` method of the Femap Application object to create Curve objects.

Curve Object Properties

Curves are always stored with a `SetID=1`, and the ID equal to the Curve ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 layer`	The ID of the layer associated with the curve.
`INT4 color`	The curve color.
`INT4 type`	The curve type: 0=Line, 1=Arc, 2=Circle, 3=Spline, 4=BSpline, 5=Solid, 6=Combined.

`INT4 StdPoint[0..4]`
or
`VARIANT vStdPoint`
	The IDs of the points referenced by lines, arcs, circles, splines and solid lines.

`INT4 attrPID`	The Property ID of the meshing attribute.
`REAL8 attrOrient[0..2]`	The orientation vector meshing attribute. Either a location or vector (based on `attrOffsetType`), specified in global rectangular coordinates.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vattrOrient`	

`REAL8 attrOffset[0..1][0..2]`
 	
or
 	
`VARIANT vattrOffset`
	The meshing attribute defining the offset to be applied to elements. `attrOffset[0][0..2]` is at the beginning of the curve, `attrOffset[1][0..2]` is at the end. Specified in global rectangular coordinates. If doing radial offsets, `attrOffset[0][0..2]` contains the radial origin, `attrOffset[1][0]` contains the radial offset distance.

`INT4 attrOrientType`	Type of orientation for property attribute: 0=Orient By Vector, 1=Orient By Location, 2=Orient By Vector, Reversed Element Direction, 3=Orient By Location, Reversed Element Direction.
`INT4 attrOffsetType`	Type of offset for property attribute: 0=Offset by Vector, 1=Offset Radial, 2=Offset by Location.
`INT4 attrTopology`	Topology parameter.

`INT4 attrRelease[0..1][0..5]`
 	
or
 	
`VARIANT vattrRelease`
	The release flags for the start and end of the curve. Nonzero if the associated degree of freedom is released.

`INT4 Engine`	The ID of the solid modeling engine used with this curve: 0=None, 1=Parasolid, 2=ACIS.
`INT4 Address`	The solid modeling engine geometry address.
`BOOL Suppressed`	True if this curve is suppressed (will not be meshed).
`INT4 InternalToBoundary`	If this curve is internal to a multi-surface boundary, this property contains the ID of the boundary, otherwise 0.
`INT4 InCombinedCurve`	If this curve is used to define a combined curve, this property contains the ID of the combined curve, otherwise 0

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 ScaleFactor`	The Solid Geometry Scale Factor.
Mesh Sizing	

`INT4 MeshElem[0..159]`
or
`VARIANT vMeshElem`
	The number of elements associated with the curve or this segment of a custom mesh size.

`REAL8 MeshBias[0..159]`
or
`VARIANT vMeshBias`
	The bias factor for these elements.

`REAL8 MeshMaxParam[0..159]`
or
`VARIANT vMeshMaxParam`
	The s parametric value along the curve at the end of this segment. If you do not use a custom mesh size, this will always have `MeshMaxParam[0]` = 1.0.

`BOOL LengthSpacing[0..159]`
or
`VARIANT vLengthSpacing`
	True if elements are sized with equal length spacing. False if parametric spacing is used.

`BOOL CenterBias[0..159]`
or
`VARIANT vCenterBias`
	True if center biasing is used. False if end biasing is used.

`BOOL GeometricBias[0..159]`	True if geometric biasing is used. False if linear biasing is used.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vGeometricBias`	
`BOOL HasCustomMesh`	True if this is a custom mesh size, False otherwise.

`INT4 MeshPointID[0..1224]`
 	
or
 	
`VARIANT vMeshPointID`
	There is a maximum 1225 hard points on curves.

For BSplines only	
`INT4 BSplineOrder`	The order of the BSpline.
`INT4 BSplineNumPoints`	The number of keypoints used to define the BSpline.

`INT4 BSplinePoint[0..109]`
 	
or
 	
`VARIANT vBSplinePoint`
	The IDs of the points used to define the Bspline.

`REAL8 BSplineKnot[0..120]`
 	
or
 	
`VARIANT vBSplineKnot`
	The knot values associated with the BSpline.

`REAL8 BSplineWeight[0..120]`
 	
or
 	
`VARIANT vBSplineWeight`
	The weight values associated with the control points of the BSpline.

`REAL8 BSplineT0`	

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 BSplineT1`	
`BOOL BSplinePlanar`	True if the BSpline is planar.
`BOOL BSplineClosed`	True if the BSpline is closed.
`BOOL BSplineRational`	True if the BSpline is rational.
`BOOL BSplinePeriodic`	True if the BSpline is periodic.

The Curve Types

If you look at the curve Type property, you will see a list of seven available types. In fact, these seven types really comprise three categories: Standard Curves, BSplines and Solid Curves. Each category stores its point definitions differently. The following table shows the point definition for each curve type.

Curve Type	Point Location	Description
Line	StdPoint	[0]=start, [1]=end
Arc	StdPoint	[0]=center, [1]=start, [2]=along in direction from start to end, [3]=end
Circle	StdPoint	[0]=center, [1]=start/end, [2]=90 deg., [3]=180 deg., [4]=270 deg.
Spline	StdPoint	[0]=start, [1]=first keypoint, [2]=second keypoint, [3]=end
BSpline	BSplinePoint	[0]=start, ... [`BSplineNumPoints`-1]=end
Solid		Points are not defined, unless this is a solid curve that represents a line, in which case [0]=start,[1]=end.
Combined	StdPoint	[0]=start, [1]=end

Curve Object Methods

Unpublished work. © 2025 Siemens

Reverse

Reverse	
(void)	
Description:	
This method reverses the direction of a curve.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to reverse the curve, probably a solid curve.
Remarks/Usage:	
This method will not work for solid curves.	
Example:	
None	

EndPoints

EndPoints	
(startPt, endPt)	
Description:	
This method retrieves the IDs of the endpoints of the current curve.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`INT4 startPt`	The IDs of the starting and ending points on the curve.
`INT4 endPt`	
Return Code:	
`FE_FAIL`	Unable to get the endpoints. Possibly curve has not been loaded, or not stored.
Remarks/Usage:	
This method works for any stored curve. You must first call `Get` to load the curve from the database. It will not work with data you have loaded into a curve object but not yet stored.	
Example:	
None	

PointsAsSet

`PointsAsSet` **`( SetID, bClear)`**	
Description:	
This method adds the end points of a curve into the specified Set object with the option to clear the set before adding the end points.	
Input:	
`INT4 SetID`	ID of Set object to add end points.
`BOOL bClear`	If True, the Set object will be cleared before the end points are added. If False, the end points are simply added to the specified Set object.
Output:	
None	None
Return Code:	
`FE_FAIL`	End Points could not be determined.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

ArcCircleInfo

`ArcCircleInfo` **`( center, normal, startPt, endPt, angle, radius )`**	
Description:	
This method retrieves information about arcs and circles.	
Input:	
None	None
Output:	
`REAL8 center[0..2]`	The coordinates of the center of the circle.
`REAL8 nocrmal[0..2]`	The vector components of the arc/circle normal direction.
`REAL8 startPt[0..2]`	The coordinates of the starting point of the curve.
`REAL8 endPt[0..2]`	The coordinates of the ending point of the curve.
`REAL8 angle`	The subtended angle of the curve.
`REAL8 radius`	The radius of the curve.
Return Code:	
`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded, or is not the correct type.
Remarks/Usage:	
This method will only work for arc and circle curve types - not for solids that represent those types.	

Unpublished work. © 2025 Siemens

Example:
None

ParamToXYZ

ParamToXYZ **(s, xyz)**	
Description:	
This method retrieves the coordinates along the curve of a specified parametric location.	
Input:	
`REAL8 s`	The parametric value (0<=s<=1) along the curve. 0=Start, 1=End.
Output:	
`REAL8 xyz[0..2]`	The coordinates of the parametric value along the curve.
Return Code:	
`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

XYZToParam

Unpublished work. © 2025 Siemens

XYZToParam	
(xyz, s)	
Description:	
This method retrieves the parametric location along a curve closest to the specified coordinates.	
Input:	
`REAL8 xyz[0..2]`	The coordinates of the location of interest.
Output:	
`REAL8 s`	The parametric value (0<=s<=1) along the curve. 0=Start, 1=End.
Return Code:	
`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

Project

Project	
(fromXYZ, toXYZ)	
Description:	
This method projects a location onto a curve (This method is OBSOLETE, use `ProjectToClosest` method for access to all functionality of the **Modify** → **Project...** commands).	
Input:	
`REAL8 fromXYZ[0..2]`	The coordinates of the initial location.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 toXYZ[0..2]`	The coordinates of the projected location.
Return Code:	
`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

ProjectToClosest

`ProjectToClosest` **`( bExtendCurve, vOrig, vProj )`**	
Description:	
This method projects a location to the closest location that lies on a curve (i.e., Femap determines vector used for projection).	
Input:	
`BOOL bExtendCurve`	If True, a straight curve will be extended infinitely in both directions and a circular curve will be extended a full 360 degrees. The closest location will then be determined using the *extended curve* (i.e., the projected location could lie outside the end points of the curve). If False, the location will be forced onto the curve itself and in some cases this will result in the location being coincident with one of the curve's end points.
`REAL8 vOrig[0..2]`	The coordinates of the initial location.
Output:	
`REAL8vProj[0..2]`	The coordinates of the projected location.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

ProjectAlongVector

`ProjectAlongVector` **`( vAlong, vOrig, vProj )`**	
Description:	
This method projects a location along a specified vector to a location that lies on a curve (i.e., user supplies vector used for projection).	
Input:	
`REAL8 vAlong [0..2]`	The vector supplied by the user to project along.
`REAL8 vOrig [0..2]`	The coordinates of the initial location.
Output:	
`REAL8 vProj [0..2]`	The coordinates of the projected location, on the curve.
Return Code:	
`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Tangent

Tangent	
(outward, fromXYZ, tangent)	
Description:	
This method finds the tangent to a curve at a specified location.	
Input:	
`BOOL outward`	If True, the tangent along the first *half* of the curve (s<0.5) is computed in the opposite direction from the curve. If False, all tangents are computed in the curve direction from start to end.
`REAL8 fromXYZ[0..2]`	The coordinates of the initial location.
Output:	
`REAL8 tangent[0..2]`	The vector components of the tangent.
Return Code:	
`FE_FAIL`	Unable to get the information. Possibly the curve has not been loaded.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

Length

Length
(void)
Description:

Unpublished work. © 2025 Siemens

This method returns the length of a curve.	
Input:	
None	None
Output:	
None	None
Return Code:	
`Length`	The length of the curve. 0.0 for any invalid curve.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

PartialLength

`PartialLength`	
`( s0, s1 )`	
Description:	
This method returns the length of a curve between two parametric locations.	
Input:	
`REAL8 s0`	The starting parametric location.
`REAL8 s1`	The ending parametric location.
Output:	
None	None
Return Code:	
`Length`	The length of the curve. 0.0 for any invalid curve.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method will only work for any curve type. The values for `s0` and `s1` must be between 0.0 and 1.0. `s1` must be greater than `s0`.
Example:
None

ParamAtLength

`ParamAtLength`	
`( fromstart, length )`	
Description:	
This method returns the parameter value at a specified distance from the end of a curve.	
Input:	
`BOOL fromstart`	If True, length is measured from the start of the curve. If False, it is measured from the end.
`REAL8 length`	The distance along the curve where the parameter value is determined.
Output:	
None	None
Return Code:	
`S`	The parametric value corresponding to the location along the curve.
Remarks/Usage:	
This method will only work for any curve type. The length must be less than the total length of the curve.	
Example:	
None	

MeshParam

Unpublished work. © 2025 Siemens

MeshParam

(numParam, meshParam)

Description:

This method returns an array of parametric locations along the curve that are currently established as mesh locations, based on the mesh size specifications that are defined.

Input:

`INT4 numParam`	The number of parametric locations in `meshParam`.
`REAL8 meshParam[0..numParam-1]`	The parametric locations along the curve where nodes will be meshed.

Output:

None	None

Return Code:

`FE_FAIL`	Unable to determine the mesh locations. Possibly the curve has not been loaded or saved.

Remarks/Usage:

This method will only work for any curve type. The curve must however be Put to the database after any changes have been made to mesh sizing or this command may not return the proper locations.

Example:

```
Sub Main()
' This function creates nodes at mesh locations
' on all curves in the model


Dim App As femap.model
Set App = feFemap()


Dim cu As femap.Curve
Set cu = App.feCurve


Dim nd As femap.Node
Set nd = App.feNode


Dim rc As Long
Dim n As Long
Dim meshloc As Variant
Dim ndloc As Variant
```


Unpublished work. © 2025 Siemens

```
' Loop over all curves
While cu.Next
    ' Get the mesh locations on the curves
   rc = cu.MeshParam(n, meshloc)
   If rc Then
       For i = 0 To n - 1
            ' Find the nodal location and create a node
           rc = cu.ParamToXYZ(meshloc(i), ndloc)
           nd.xyz = ndloc
           nd.Put (nd.NextEmptyID)
       Next i
   End If
Wend

App.feViewRegenerate(0)

End Sub
```

AddMeshLoc

AddMeshLoc

(s, numElem, spacing, biasMethod, bias, biasLoc)

Description:

This method adds a custom mesh size location at the specified parametric location.

Input:

`REAL8 s`	The parametric location along the curve - must be between 0.0 and 1.0 - where the mesh location will be added.
`INT4 numElem`	The number of elements between this location and the preceding one (at the next lower s value).
`INT4 spacing`	If 0, then parametric spacing is used; otherwise, constant length spacing is used.
`INT4 biasMethod`	0=Equal length spacing, 1=Linear Bias, 2=Geometric Bias.
`REAL8 bias`	The bias value - greater than or equal to 1.0. A value of 1.0 is not biased. A value of 2.0 means that the largest element is approximately 2.0 times the length of the smallest.

Unpublished work. © 2025 Siemens

`INT4 biasLoc`	0=Small elements at start of curve, 1=Small elements at end of curve, 2=Small elements at center of curve, 3=Small elements at both ends of curve.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to add the mesh location. Possibly the curve has not been loaded or saved. Possibly there are already too many locations defined.
Remarks/Usage:	
The curve should be loaded prior to using this method. This will always result in a custom mesh size being defined for the curve. You must always Put the curve to the database after finishing for the mesh locations to be saved. To redefine a mesh location, simply call this method again with the same value for s.	
Example:	
None	

AddMeshLocXYZ

`AddMeshLocXYZ`	
`( xyz, numElem, spacing, biasMethod, bias, biasLoc )`	
Description:	
This method adds a custom mesh size location at the specified coordinate location.	
Input:	
`REAL8 xyz[0..2]`	The coordinates of the location for the mesh location. Must be in global rectangular. These coordinates are projected onto the curve to determine the corresponding parametric location.
`INT4 numElem`	The number of elements between this location and the preceding one (at the next lower s value).
`INT4 spacing`	If 0, then parametric spacing is used. Otherwise, constant length spacing is used.

Unpublished work. © 2025 Siemens

`INT4 biasMethod`	0=Equal length spacing, 1=Linear Bias, 2=Geometric Bias.
`REAL8 bias`	The bias value - greater than or equal to 1.0. A value of 1.0 is not biased. A value of 2.0 means that the largest element is approximately 2.0 times the length of the smallest.
`INT4 biasLoc`	0=Small elements at start of curve, 1=Small elements at end of curve, 2=Small elements at center of curve, 3=Small elements at both ends of curve.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to add the mesh location. Possibly the curve has not been loaded or saved. Possibly there are already too many locations defined.
Remarks/Usage:	
This method is essentially the same as `AddMeshLoc`, except that it uses a coordinate location instead of a parametric location.	
Example:	
None	

Nodes

`Nodes` **`( getBoundary, getInterior, numNodes, nodeID )`**	
Description:	
This method returns the nodes that are attached to this curve.	
Input:	
`BOOL getBoundary`	If True, the nodes attached to the curve endpoints are selected.
`BOOL getInterior`	If True, the nodes attached to the interior of the curve (not the endpoints) are selected.

Unpublished work. © 2025 Siemens

Output:	
`INT4 numNodes`	The number of nodes returned.
`INT4 nodeID[0..numNodes-1]`	The IDs of the selected nodes which are attached to the curve.
Return Code:	
`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the curve in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.
Remarks/Usage:	
None	
Example:	

The following codes modifies the color of all nodes on Curve 4.

```
Sub Main


Dim App As femap.model
Set App = feFemap()

Dim cu As femap.Curve
Set cu = App.feCurve


Dim nd As femap.Node
Set nd = App.feNode


Dim rc As Long
Dim Count As Long
Dim Nodes As Variant
Dim i As Long

rc = cu.Get (4)
rc = cu.Nodes(True, True, Count, Nodes)

For i = 0 To Count - 1
  nd.Get (Nodes(i))
 nd.color = 4
 nd.Put (0)
Next i

App.feViewRegenerate(0)
```


Unpublished work. © 2025 Siemens

```
End Sub
```

NodesAsSet

NodesAsSet

(getBoundary, getInterior, nSetID, bClear)

Description:	
This method returns the nodes that are attached to this curve in a Set Object.	
Input:	
`BOOL getBoundary`	If True, the nodes attached to the curve endpoints are selected.
`BOOL getInterior`	If True, the nodes attached to the interior of the curve (not the endpoints) are selected.
`INT4 nSetID`	The ID of a Set Object to receive the Node IDs.
`BOOL bClear`	If True, the Set is cleared prior to loading the Node IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return nodes, possibly none exist.
Remarks/Usage:	
Same as Nodes, except that the IDs are returned in a Set Object rather than an array.	
Example:	
None	

Elements

Unpublished work. © 2025 Siemens

Elements	
(numElems, elemID)	
Description:	
This method returns the elements that are attached to this curve.	
Input:	
None	None
Output:	
`INT4 numElems`	The number of elements returned.
`INT4 elemID[0..numElems-1]`	The IDs of the selected elements which are attached to the curve.
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the curve in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.
Remarks/Usage:	
Only elements that have all of their nodes on the curve will be returned by this method. Elements that simply touch the curve at one edge or one node will not be returned.	
Example:	
None	

ElementsAsSet

ElementsAsSet
(nSetID, bClear)
Description:

Unpublished work. © 2025 Siemens

This method returns the elements that are attached to this curve in a Set Object.	
Input:	
`INT4 nSetID`	The ID of the Set Object that receives the Element IDs.
`BOOL bClear`	If True, the Set is cleared before it is loaded with Element IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
Remarks/Usage:	
Only elements that have all of their nodes on the curve will be returned by this method. Elements that simply touch the curve at one edge or one node will not be returned. Same as Elements method except that IDs are returned in a Set rather than an array.	
Example:	
None	

ElementsWithFaces

`ElementsWithFaces` **`( numElems, elemID )`**	
Description:	
This method returns the elements that have faces which are attached to this curve.	
Input:	
None	None
Output:	
`INT4 numElems`	The number of elements returned.
`INT4 elemID[0..numElems-1]`	The IDs of the selected elements which have faces attached to the curve.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the curve in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.
Remarks/Usage:	
This will return elements that have any face on the curve - including planar elements that have one edge along the curve. Planar elements that simply touch the curve at one corner/node will not be returned.	
Example:	
None	

BoundingBox

`BoundingBox` **`( Box )`**	
Description:	
This method returns the coordinates of a rectangular box the encloses this geometry.	
Input:	
None	None
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in Global Rectangular coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Object does not contain valid geometry. Curve type is not a Solid or Composite curve. Other types are not supported
Remarks/Usage:	
This method always retrieves a box that is aligned with Global Rectangular coordinates, not an aligned box of minimum volume.	
Example:	
None	

BoundingBoxInCSys

`BoundingBoxInCSys` **`( nCSysID, Box )`**	
Description:	
This method returns the coordinates of a rectangular box the encloses this geometry, in coordinates of the selected coordinate system.	
Input:	
`INT4 nCSysID`	ID of coordinate system to return minimum and maximum
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in “coordinate system specified with `nCSysID`" coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Object does not contain valid geometry. Curve type is not a Solid or Composite curve. Other types are not supported.
Remarks/Usage:	
This method always retrieves a box that is aligned with the selected coordinate system, not an aligned box of minimum volume.	
Example:	
None	

GetMeshLoc

`GetMeshLoc` **`( numLoc, meshParam )`**	
Description:	
Retrieves the parametric values along the curve of the mesh locations.	
Input:	
None	None
Output:	
`INT4 numLoc`	The number of mesh locations on the curve.
`REAL8 meshParam[0..numLoc-1]`	The parametric locations.
Return Code:	
`FE_FAIL`	Unable to find mesh locations.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetMeshLocXYZ

GetMeshLocXYZ	
(numLoc, meshXYZ)	
Description:	
Retrieves the global coordinates of the mesh locations along the curve.	
Input:	
None	None
Output:	
`INT4 numLoc`	The number of mesh locations on the curve.
`REAL8 mceshXYZ[0..numLoc-1]`	The global coordinates of the mesh locations.
Return Code:	
`FE_FAIL`	Unable to find mesh locations.
Remarks/Usage:	
Similar to `GetMeshLoc`, but this method returns coordinate rather than parametric values.	
Example:	
None	

IsSmoothEdge

IsSmoothEdge
(dAngle, nConvexityType)
Description:
Determines whether the adjacent surfaces of a solid meet smoothly at this curve.
Input:

Unpublished work. © 2025 Siemens

`REAL8 dAngle`	The maximum angle that will be considered as smooth.
Output:	
`INT4 nConvexityType`	0=Smooth and Flat, 1=Smooth and Convex, 2=Smooth and Concave, 3=Smooth with Inflection, 4=Smooth and Variable Convex/Concave.
Return Code:	
`FE_FAIL`	Unable to test smoothness and convexity.
`FE_INVALID`	Not smooth.
`FE_NOT_AVAILABLE`	Curve is not a curve on a solid.
Remarks/Usage:	
None	
Example:	
None	

Surfaces

`Surfaces` **`( nCombinedMode, numSurfaces, surfaceID )`**	
Description:	
Returns an array of surfaces that reference this curve	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are adjacent to the curve, 1=Any surfaces that are adjacent that are in a Combined Surface are replaced by the ID of the Combined Surface, 2=List contains both the underlying and combined surfaces.
Output:	
`INT4 numSurfaces`	The number of surfaces in the returned array.

Unpublished work. © 2025 Siemens

`INT4 surfaceID[0..numSurfaces-1]`	The IDs of the surfaces.
Return Code:	
`FE_FAIL`	Unable to get surfaces.
Remarks/Usage:	
This function only works for Solid or Composite Curves that are defined on Solids. Same as `SurfacesAsSet` except IDs are returned in an Array rather than in a Set.	
Example:	
None	

SurfacesAsSet

`SurfacesAsSet` **`( nCombinedMode, nSetID, bClear )`**	
Description:	
Returns surfaces that reference this curve in a Set Object.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are adjacent to the curve, 1=Any surfaces that are adjacent that are in a Combined Surface are replaced by the ID of the Combined Surface, 2=List contains both the underlying and combined surfaces.
`INT4 nSetID`	ID of the Set Object to receive the Surface IDs.
`BOOL bClear`	If True, the Set is cleared before loading the surfaces.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get surfaces.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This function only works for Solid or Composite Curves that are defined on Solids. Same as `Surfaces`, except that IDs are returned in the Set rather than in an array.
Example:
None

Normal

`Normal` **`( xyz, vNormal )`**	
Description:	
Returns the average of the surface normals at a specified location along the curve.	
Input:	
`REAL8 xyz[0..2]`	The Global Rectangular coordinates of a location along the curve where you want to find the normal.
Output:	
`REAL8 vNormal[0..2]`	The average surface normal at that location.
Return Code:	
`FE_FAIL`	Unable to get surfaces.
Remarks/Usage:	
This function finds all of the surfaces adjoining the curve and computes a normal to each surface at the specified location. The returned normal is simply the average of those surface normals.	
Example:	
None	

IsCombinedCurve

Unpublished work. © 2025 Siemens

IsCombinedCurve	
(void)	
Description:	
Indicates whether or not the curve is a combined curve.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Curve is a combined curve.
`FE_FAIL`	Curve is not a combined curve.
Remarks/Usage:	
You can also use the *type* property of the curve to determine whether a curve is a combined curve.	
Example:	
None	

GetCombinedCurves

GetCombinedCurves	
(nNumCurves, curveID)	
Description:	
Returns the IDs of the curves used to define a combined curve.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`INT4 nNumCurves`	The number of curves returned in curveID.
`INT4 curveID[0..nNumCurves]`	The IDs of the curves used to define a combined curve. The curves in this array are listed in the order they are used along the combined curve - starting at the beginning of the curve. If the direction of the returned curve is opposite that of the composite curve, then the ID in the array will be negative (i.e. if you see an entry in curveID like -7, that means that curve 7 is used in its reversed direction).
Return Code:	
`FE_FAIL`	Not a combined curve.
`FE_INVALID`	Invalid number of curves defining the composite curve.
Remarks/Usage:	
None	
Example:	
None	

CombinedCurvesAsSet

`CombinedCurvesAsSet`	
`( nSetID, bClear )`	
Description:	
Returns the IDs of the curves used to define a combined curve	
Input:	
`INT4 nSetID`	The ID of a Set that will receive the curve IDs.
`BOOL bClear`	If True, the Set is cleared prior to loading the curve IDs.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	Not a combined curve.
`FE_INVALID`	Invalid number of curves defining the composite curve.
Remarks/Usage:	
This method is similar to `GetCombinedCurves`, however there is no indication of forward/reverse direction when using this method. All IDs are positive and stored in the Set Object.	
Example:	
None	

CombineCurves

`CombineCurves` **`( nNumCurves, curveID )`**	
Description:	
Creates a combined curve from an array of underlying curves.	
Input:	
`INT4 nNumCurves`	The number of curves supplied in `curveID`.
`INT4 curveID[0..nNumCurves]`	The IDs of the curves used to define a combined curve. The curves in this array must be listed in the order they will be used along the combined curve - starting at the beginning of the curve. If you need to reverse a curve, then you must supply its ID as a negative number (i.e. -ID) - then the usage of the curve in the combined curve will be in the opposite direction from the original curve.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not create combined curve.
`FE_INVALID`	Too many curves specified.
`FE_NOT_EXIST`	A supplied curve does not exist.
Remarks/Usage:	
The curves supplied to this method must be properly ordered, connected and able to create a combined curve.	
Example:	
None	

Facets

`Facets`	
`( numPoints, xyzPoint )`	
Description:	
Returns the global rectangular coordinates of the locations along the curve that are used to draw the curve.	
Input:	
None	None
Output:	
`INT4 numPoints`	The number of points returned in `xyzPoint`.
`REAL8 xyzPoint[0..(3*numPoints)-1]`	The locations along the curve, stored [x0,y0,z0,x1,y1,z1,...].
Return Code:	
`FE_FAIL`	Could not retrieve facets.
Remarks/Usage:	
The curve is drawn by simply connecting these locations sequentially.	

Unpublished work. © 2025 Siemens

Example:
None

PartialLengthXYZ

PartialLengthXYZ	
(xyz0, xyz1)	
Description:	
This method returns the length of a curve between two XYZ locations.	
Input:	
`REAL8 xyz0[0..2]`	The coordinates of the location of interest.
`REAL8 xyz1[0..2]`	The coordinates of the location of interest.
Output:	
None	None
Return Code:	
`Length`	The length of the curve. 0.0 for any invalid curve.
Remarks/Usage:	
This method will only work for any curve type.	
Example:	
None	

PartialLengthNode

Unpublished work. © 2025 Siemens

PartialLengthNode	
(nID1, nID2)	
Description:	
This method returns the length of a curve between the Node locations that are closest to the curve.	
Input:	
`INT4 nID1`	The ID of the first Node.
`INT4 nID2`	The ID of the second Node.
Output:	
None	None
Return Code:	
`Length`	The length of the curve. 0.0 for any invalid curve.
Remarks/Usage:	
This method will work for any curve type.	
Example:	
None	

IsStraight

IsStraight	
( )	
Description:	
Determines whether the curve is a straight line.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_OK`	Curve is a straight line.
`FE_FAIL`	Not a straight line.
Remarks/Usage:	
None	
Example:	
None	

IsArc

`IsArc` **`( )`**	
Description:	
Determines whether the curve is an arc.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Curve is an arc.
`FE_FAIL`	Not an arc.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

IsNonManifold

IsNonManifold	
( )	
Description:	
Determines whether the curve is nonmanifold.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	Manifold curve.
FE_FAIL	Curve is nonmanifold.
FE_BAD_TYPE	Curve is not a supported type for this method. This method only works with Solid and Composite curves.
Remarks/Usage:	
Nonmanifold curves are ones that bound more than 2 surfaces.	
Example:	
None	

IsGeneral

IsGeneral
( )
Description:

Unpublished work. © 2025 Siemens

Determines whether the curve is nonmanifold (on a general body) by querying Parasolid.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Manifold curve.
`FE_FAIL`	Curve is nonmanifold.
`FE_BAD_TYPE`	Curve is not a supported type for this method. This method only works with Solid and Composite curves.
Remarks/Usage:	
Nonmanifold curves are ones that bound more than 2 surfaces. This command provides an alternate method to the IsNonManifold method.	
Example:	
None	

IsSmooth

`IsSmooth` **`( dAngle )`**	
Description:	
Determines whether the surfaces bounded by this curve are tangent within a specified angle along the curve.	
Input:	
`REAL8 dAngle`	The maximum angle below which the curve is considered to be a smooth connection between the adjoining surfaces.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_OK`	Curve is smooth.
`FE_FAIL`	Not a smooth curve.
`FE_BAD_TYPE`	This method only works with Solid and Composite curves.
Remarks/Usage:	
None	
Example:	
None	

IsMergeable

`IsMergeable`
`( void )`
Description:
Determines whether a curve is mergeable, thus can be removed by geometry simplification and cleanup commands.
Input:

None	None
Output:	
None	None
Return Code:	
`FE_OK`	Curve is mergeable.
`FE_FAIL`	Not is not mergeable.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

IsFreeEdge

`IsFreeEdge` **`( void )`**	
Description:	
Determines whether a curve is a free edge.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Curve is a Free Edge.
`FE_FAIL`	Curve is not a Free Edge.
`FE_BAD_TYPE`	Curve type must be 5=Solid or 6=Combined.
Remarks/Usage:	
None	
Example:	
None	

IsOnSolidRegion

Unpublished work. © 2025 Siemens

IsOnSolidRegion	
(void)	
Description:	
Determines whether a curve is on a solid region.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	Curve is on a Solid Region.
FE_FAIL	Curve is not on a Solid Region.
FE_BAD_TYPE	Curve type must be 5=Solid or 6=Combined.
Remarks/Usage:	
None	
Example:	
None	

TangentAtEnds

TangentAtEnds
(outward, pvEnd1Tan, pvEnd2Tan)
Description:
Determines whether the surfaces bounded by this curve are tangent within a specified angle along the curve.
Input:

Unpublished work. © 2025 Siemens

`BOOL outward`	If true, the outward pointing tangents are computed at each end. If False, tangents along the curve in the parametric direction of the curve are computed.
Output:	
`REAL8 pvEnd1Tan[0..2]`	The tangent at the start of the curve.
`REAL8 pvEnd2Tan[0..2]`	The tangent at the end of the curve.
Return Code:	
`FE_BAD_TYPE`	Curve type is not supported for this method.
`FE_FAIL`	Unable to compute tangents.
Remarks/Usage:	
This method is only supported for Solid and Composite Curves.	
Example:	
None	

CurvatureAcrossEdge

`CurvatureAcrossEdge` **`( pdMinVal, pdMaxVal, pdAngleBetweenNormals )`**	
Description:	
Finds the minimum and maximum curvature along the curve and the maximum angle between normals of the adjacent surfaces.	
Input:	
None	None
Output:	
`REAL8 pdMinVal`	Minimum radius of curvature.
`REAL8 pdMaxVal`	Maximum radius of curvature.

Unpublished work. © 2025 Siemens

`REAL8 pdAngleBetweenNormals`	Maximum angle between the adjacent surface normals.
Return Code:	
`FE_BAD_TYPE`	Curve type is not supported for this method.
`FE_FAIL`	Unable to compute values.
Remarks/Usage:	
This method is only supported for Solid and Composite Curves.	
Example:	
None	

ClosestPointToCurve

`ClosestPointToCurve` **`( nCurveID, pdX, pdY, pdZ, pdDistance, pdAngleOfTangents )`**	
Description:	
Finds the closest point on the curve to a selected curve.	
Input:	
`INT4 nCurveID`	The curve to test for closest approach.
Output:	
`REAL8 pdX`	The coordinates of the closest location on the curve to the selected curve.
`REAL8 pdY`	
`REAL8 pdZ`	
`REAL8 pdDistance`	The distance from the point to the selected curve.
`REAL8 pdAngleOfTangents`	The Angle between the Curve Tangent and the curve tangent of the selected curve at the location of closest approach.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_BAD_TYPE`	Curve type is not supported for this method.
`FE_FAIL`	Unable to compute values.
`FE_INVALID`	Unable to find closest approach.
`FE_NOT_AVAILABLE`	Unable to locate selected curve.
Remarks/Usage:	
This method is only supported for Solid and Composite Curves.	
Example:	
None	

ClosestPointToSurface

`ClosestPointToSurface` **`( nSurfaceID, pdX, pdY, pdZ, pdDistance, pdAngleOfTangentToSurfaceNormal )`**	
Description:	
Finds the closest point on the curve to a selected surface.	
Input:	
`INT4 nSurfaceID`	The surface to test for closest approach.
Output:	
`REAL8 pdX`	The coordinates of the closest location on the curve to the surface.
`REAL8 pdY`	
`REAL8 pdZ`	
`REAL8 pdDistance`	The distance from the point to the surface.

Unpublished work. © 2025 Siemens

`REAL8 pdAngleOfTangentToSurfaceNormal`	The Angle between the Curve Tangent and the surface normal at the location of closest approach.
Return Code:	
`FE_BAD_TYPE`	Curve type is not supported for this method.
`FE_FAIL`	Unable to compute values.
`FE_INVALID`	Unable to find closest approach.
`FE_NOT_AVAILABLE`	Unable to locate selected surface.
Remarks/Usage:	
This method is only supported for Solid and Composite Curves.	
Example:	
None	

MinRadius

`MinRadius`	
`( pdMinRadius)`	
Description:	
Finds the minimum radius of a Solid curve or a Composite curve.	
Input:	
None	None
Output:	
`REAL8 pdMinRadius`	Minimum radius of curve.
Return Code:	
`FE_BAD_TYPE`	Curve type is not supported for this method.
`FE_FAIL`	Unable to compute values.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method is only supported for Solid and Composite Curves.
Example:
None

ParamOnCombinedCurve

ParamOnCombinedCurve **(s, pnCurveID)**	
Description:	
This method returns the ID of the underlying curve of a *composite* curve at a specified parametric location.	
Input:	
`REAL8 s`	Parametric location along *composite* curve from 0.0 to 1.0.
Output:	
`INT4 pnCurveID`	Curve ID of underlying curve at specified parametric location.
Return Code:	
`FE_BAD_TYPE`	Curve type is not supported for this method.
`FE_FAIL`	Unable to determine underlying curve ID.
Remarks/Usage:	
This method is only supported for Composite Curves.	
Example:	
None	

IsSuppressed

Unpublished work. © 2025 Siemens

IsSuppressed	
(void)	
Description:	
Indicates whether or not the curve is suppressed.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	Curve is suppressed.
FE_FAIL	Curve is not suppressed.
Remarks/Usage:	
None	
Example:	
None	

ClearMeshLoc

ClearMeshLoc	
(void)	
Description:	
Clears all previous mesh sizing on curve.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_OK`	Mesh sizing has been removed from curve.
Remarks/Usage:	
The curve should be loaded prior to using this method. You must always Put the curve to the database after finishing for removal of assigned mesh sizing. Sets number of elements to 0.	
Example:	
None	

HasMeshSize

`HasMeshSize`	
`( void )`	
Description:	
Indicates if a curve has assigned mesh sizing.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Curve has assigned mesh sizing.
`FE_FAIL`	Curve does not have assigned mesh sizing.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetSimpleMeshLoc

SetSimpleMeshLoc

(nNumElem, spacing, biasMethod, bias, biasLoc)

Description:

This method sets mesh sizing on a curve using only **Number of Elements** and **Node Spacing** options. Similar to using **Mesh** → **Mesh Control** → **Size Along Curve** command.

Input:

`INT4 nNumElem`	The number of elements on the curve.
`INT4 spacing`	If 0, then parametric spacing is used; otherwise, constant length spacing is used.
`INT4 biasMethod`	0=Equal length spacing, 1=Linear Bias, 2=Geometric Bias.
`REAL8 bias`	The bias value - greater than or equal to 1.0. A value of 1.0 is not biased. A value of 2.0 means that the largest element is approximately 2.0 times the length of the smallest.
`INT4 biasLoc`	0=Small elements at start of curve, 1=Small elements at end of curve, 2=Small elements at center of curve, 3=Small elements at both ends of curve.

Output:

None	None

Return Code:

`FE_FAIL`	Unable to set mesh location. Possibly the curve has not been loaded or saved.

Remarks/Usage:

The curve should be loaded prior to using this method. You must always `Put` the curve to the database after finishing for the mesh locations to be saved.

Example:

None

Unpublished work. © 2025 Siemens

MeshCount

MeshCount	
(NumElem, CustomSize)	
Description:	
This method returns the number of elements specified via mesh sizing on a curve and if custom mesh sizing has been assigned.	
Input:	
None	None
Output:	
`INT4 NumElem`	Number of elements assigned via mesh sizing on curve.
`BOOL CustomSize`	If True, the curve has custom mesh sizing assigned. If False, standard mesh sizing has been assigned.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

CommonPoint

CommonPoint
(OtherCurveID, CommonPoint, MultipleCommon)
Description:
The method returns the ID of a common point found between the curve currently loaded in the object and another specified curve. In addition, it can also be used to determine if multiple points are common between the curves.

Unpublished work. © 2025 Siemens

Input:	
`INT4 OtherCurveID`	ID of curved used to determine if there are any common points.
Output:	
`INT4 CommonPoint`	ID of point common between curve currently loaded in object and `OtherCurveID`.
`BOOL MultipleCommon`	If True, more than one point is common between the curve currently loaded in the object and `OtherCurveID`. If False, then only ID of `CommonPoint` is common between the curves.
Return Code:	
`FE_FAIL`	No points are common between the curves.
Remarks/Usage:	
None	
Example:	
None	

Solid

`Solid`
`( nSolidID )`
Description:
This method returns the ID of the Solid that contains the current Curve.
Input:

None	None
Output:	
`INT4 nSolidID`	The ID of the Solid that references the current Curve.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to get the solid.
Remarks/Usage:	
This method will only work with a Curve and Combined Curve that is defined on a Solid.	
Example:	
None	

Midpoint

`Midpoint` **`( Mipoint )`**	
Description:	
This method returns the coordinates of the midpoint of a curve or combined curve.	
Input:	
None	None
Output:	
`REAL8 midpoint[0..2]`	The coordinates of the midpoint of the current curve or combined curve. Always returned in the global rectangular coordinate system.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Center

Unpublished work. © 2025 Siemens

Center **(center)**	
Description:	
This method returns the coordinates of the center point of a circular arc (curve or combined curve).	
Input:	
None	None
Output:	
`REAL8 center[0..2]`	The coordinates of the center point of the current curve or combined curve. Always returned in the global rectangular coordinate system.
Return Code:	
`FE_FAIL`	Curve or combined curve is not a circular arc or does not exist.
Remarks/Usage:	
None	
Example:	
None	

GetPrecision

GetPrecision **(void)**	
Description:	
This method returns the current Parasolid precision of a curve.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	Current Parasolid precision of curve.
Remarks/Usage:	
None	
Example:	
None	

ResetPrecision

`ResetPrecision`	
`( void )`	
Description:	
This method resets the Parasolid precision of a curve to a value equal to 5.0E-9*current Solid Geometry Scale Factor.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
If Solid Geometry Scale Factor is equal to 39.37 (Inches), then this value will be 1.9685E-7. If Solid Geometry Scale Factor is equal to 1.0 (Meters), then this value will be 5.0E-9. If Solid Geometry Scale Factor is equal to 1000.0 (Millimeters), then this value will be 5.0E-6.	
Example:	
None	

Unpublished work. © 2025 Siemens

InitMeshAttributes

InitMeshAttributes	
(nPropID)	
Description:	
Initialize Mesh Attributes on the curve.	
Input:	
`INT4 nPropID`	Property ID used for mesh attributes on curve.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method sets the default `attrTopology = FTO_LINE2`.	
Example:	
None	

ManifoldType

ManifoldType
(pnType)
Description:
This method is used to determine the type of surface connectivity for a curve. Types are: Free - *Edge Curve* not shared with any other surface. Manifold - *Edge Curve* shared by two different surfaces. NonManifold - *Edge Curve* is shared by more than two surfaces OR *Edge Curve* of one surface is shared with *Interior Curve* of another surface.

Unpublished work. © 2025 Siemens

Wirebody - Indicates a Parasolid wirebody (very uncommon, as they can only be imported, not created within Femap). Unknown - Curve connectivity cannot be determined, such as a curve not attached to a surface and is also not a Parasolid wirebody.	
Input:	
None	None
Output:	
`INT4 pnType`	Type of Curve Connectivity. Use enum `zCurveManifoldType`: Unknown = `FCMT_UNKNOWN (-1)` Wirebody = `FCMT_WIREEDGE (0)` Free = `FCMT_FREEEDGE (1)` Manifold = `FCMT_MANIFOLD (2)` NonManifold = `FCMT_NONMANIFOLD (3)`
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

IsFacet

`IsFacet` **`( void )`**
Description:
Indicates whether or not the curve is a curve on a faceted surface.
Input:
None | None
Output:
None | None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_OK`	Curve is a curve on a faceted surface.
`FE_FAIL`	Curve is not a curve on a faceted surface.
`FE_INVALID`	The current object does not contain a valid Parasolid curve.
Remarks/Usage:	
None	
Example:	
None	

GetAllArray

`GetAllArray`	
`( setID, numCurve, entID, exist, curvetype, layer, color, donotmesh, nInCompositeCurve, nInternalToBoundary, points )`	
Description:	
This method retrieves arrays of values for a set of curves, including if a curve exists in the database.	
Input:	
`INT4 setID`	ID of the Set object that contains the curve IDs to retrieve. If 0, retrieve all curves in the model.
Output:	
`INT4 numCurve`	The number of curves for which data is being returned.
`INT4 entID[0..numCurve-1]`	The IDs of the curves being retrieved.
`BOOL exist[0..numCurve-1]`	True=Curve exists, False=Curve does not exist.
`INT4 curvetype[0..numCurve-1]`	Type of Curve (0=Line, 1=Arc, 2=Circle, 3=Spline, 4=BSpline, 5=Solid, 6=Combined).
`INT4 layer[0..numCurve-1]`	The ID of the layer for the curve.

Unpublished work. © 2025 Siemens

`INT4 color[0..numCurve-1]`	The color of the curve.
`BOOL donotmesh[0..numCurve-1]`	True=curve is suppressed for meshing, False=not suppressed.
`INT4 nInCompositeCurve[0..numCurve-1]`	If nonzero, the ID of a Composite Curve that references this curve.
`INT4 nInternalToBoundary[0..numCurve-1]`	If nonzero, the ID of a Composite Surface where this curve is used but not on an outer boundary of the surface.
`INT4 points[0..(5*numCurve)-1]`	The IDs of points used to define the curve. See Remarks/Usage.
Return Code:	
`FE_FAIL`	Unable to retrieve curve info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve curve info.
Remarks/Usage:	
For Solid curves other than *lines*, including BSplines which are *straight*, these points are not used or specified, as Solid curves cannot be edited by changing points, therefore all values for *points* are 0.	
Example:	
None	

PutAllArray

`PutAllArray`
`( numCurve, entID, bUpdateLayer, layer, bUpdateColor, color, bUpdateDoNotMesh, donotmesh )`
Description:
This method updates properties of multiple selected curves.
Input:

`INT4 numCurve`	The number of curves for which data is being updated.

Unpublished work. © 2025 Siemens

`INT4 entID[0..numCurve-1]`	The IDs of the curves to update.
`BOOL bUpdateLayer[0..numCurve-1]`	If True, update Layer ID.
`INT4 layer[0..numCurve-1]`	The ID of the layer for the curve. Only assigned if `bUpdateLayer`=True.
`BOOL bUpdateColor[0..numCurve-1]`	If True, update Color
`INT4 color[0..numCurve-1]`	The color of the curve. Only assigned if `bUpdateColor`=True.
`BOOL bUpdateDoNotMesh[0..numCurve-1]`	If True, update donotmesh property.
`BOOL donotmesh[0..numCurve-1]`	True=curve is suppressed for meshing, False=not suppressed. Only assigned if `bUpdateDoNotMesh`=True.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to update one or more curve with the specified IDs.
`FE_NO_MEMORY`	Unable to allocate memory to update curves.
Remarks/Usage:	
None	
Example:	
None	

GetAllAttrArray

Unpublished work. © 2025 Siemens

GetAllAttrArray

(setID, numCurve, entID, exist, pid, orient_type, orient, offset_type, offset, release)

Description:

This method retrieves arrays of meshing attributes assigned to a set of curves.

Input:

INT4 setID	ID of the Set object that contains the curve IDs to retrieve. If 0, retrieve all curves in the model.

Output:

`INT4 numCurve`	The number of curves for which data is being returned.
`INT4 entID[0..numCurve-1]`	The IDs of the curves being retrieved.
`BOOL exist[0..numCurve-1]`	True=Curve exists, False=Curve does not exist.
`INT4 pid[0..numCurve-1]`	The ID of a Property assigned as a meshing attribute.
`INT4 orient_type[0..numCurve-1]`	The orientation type. 0=Orient By Vector, 1=Orient By Location, 2=Orient By Vector, Reversed Element Direction, 3=Orient By Location, Reversed Element Direction.
`REAL8 orient[0..(3*numCurve)-1]`	The orientation vector meshing attribute. Either a location or vector (based on orient_type), specified in global rectangular coordinates.
`INT4 offset_type[0..numCurve-1]`	Type of offset for property attribute: 0=Offset by Vector, 1=Offset Radial, 2=Offset by Location.
`REAL8 offset [0..(6*numCurve)-1]`	The meshing attribute defining the offset to be applied to elements. `attrOffset[0][0..2]` is at the beginning of the curve, `attrOffset[1][0..2]` is at the end. Specified in global rectangular coordinates. If doing radial offsets, `attrOffset[0][0..2]` contains the radial origin, `attrOffset[1][0]` contains the radial offset distance.
`BOOL release[0..(12*numCurve)-1]`	The release flags for the start and end of the curve. Nonzero if the associated degree of freedom is released.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to retrieve curve info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve curve info.
Remarks/Usage:	
None	
Example:	
None	

PutAllAttrArray

`PutAllAttrArray`	
`( numCurve, entID, bUpdatePID, pid, bUpdateOrient, orient_type, orient, bUpdateOffset, offset_type, offset, bUpdateRelease, release )`	
Description:	
This method updates meshing attributes of multiple selected curves.	
Input:	
`INT4 numCurve`	The number of curves for which data to be updated.
`INT4 entID[0..numCurve-1]`	The IDs of the curves to update.
`BOOL bUpdatePID[0..numCurve-1]`	If True, update the property meshing attribute.
`INT4 pid[0..numCurve-1]`	The ID of a Property assigned as a meshing attribute.
`BOOL bUpdateOrient[0..numCurve-1]`	If True, update the orientation meshing attributes.
`INT4 orient_type[0..numCurve-1]`	The orientation type. 0=Orient By Vector, 1=Orient By Location, 2=Orient By Vector, Reversed Element Direction, 3=Orient By Location, Reversed Element Direction. Only assigned if `bUpdateOrient`=True.

Unpublished work. © 2025 Siemens

`REAL8 orient[0..(3*numCurve)-1]`	The orientation vector meshing attribute. Either a location or vector (based on orient_type), specified in global rectangular coordinates. Only assigned if `bUpdateOrient`=True.
`BOOL bUpdateOffset[0..numCurve-1]`	If True, update the offset meshing attributes.
`INT4 offset_type[0..numCurve-1]`	Type of offset for property attribute: 0=Offset by Vector, 1=Offset Radial, 2=Offset by Location. Only assigned if `bUpdateOffset`=True.
`REAL8 offset [0..(6*numCurve)-1]`	The meshing attribute defining the offset to be applied to elements. `attrOffset[0][0..2]` is at the beginning of the curve, `attrOffset[1][0..2]` is at the end. Specified in global rectangular coordinates. If doing radial offsets, `attrOffset[0][0..2]` contains the radial origin, `attrOffset[1][0]` contains the radial offset distance. Only assigned if `bUpdateOffset`=True.
`BOOL bUpdateRelease[0..numCurve-1]`	If True, update the release meshing attribute.
`BOOL release[0..(12*numCurve)-1]`	The release flags for the start and end of the curve. Nonzero if the associated degree of freedom is released. Only assigned if `bUpdateRelease`=True.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store one or more curve with the specified IDs.
`FE_NO_MEMORY`	Unable to allocate memory to update curves.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Data Surface Objects

Data Surface objects correspond to Femap Data Surfaces which are used to create spatial and entity based variations.

They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Data Surface objects. For more information, see **Common Entity Properties and Methods**.

Use the `feDataSurf` method of the Femap Application object to create Data Surface objects.

Data Surface Object Properties

Data Surfaces are always stored with a `SetID=1`, and the ID equal to the surface ID.

Property	Description
`STRING title`	Data Surface title (maximum 79 characters).
`INT4 csys`	Coordinate System in which the Data Surface should be evaluated in.
`BOOL vector`	Flag indicating that this data surface contains vector data.
`INT4 AlongCoordProjectID`	(Along Coordinates) Surface ID in which to project Along Curve Data surface drive spline.
`INT4 ParamCurveSurfID`	(Between, Tabular) Curve ID or Surface ID that defines the parametric space.
`BOOL ExtendEdgeValues`	(Between) When True, enables Extend Edge Values Outside Data Surface.
`REAL8 InterpPercent`	(Arbituary 3D) Percent of variation locations to include in interpolation.
`INT4 InterpMin`	(Arbitrary 3D) Minimum number of variation locations to include in interpolation.
`INT4 TabularUdefOption`	(Tabular XYZ) 0 = Interpolate from closest, 1 = Constant value.
`REAL8 TabularUdefVal`	(Tabular XYZ)Value for `TabularUdefOpt`.
`INT4 OutMapOption`	(Output Map) Method used to determine values for target locations that fall outside of source mesh:

Unpublished work. © 2025 Siemens

Property	Description
	0=FMO_ZERO; 1=FMO_VALUE; 2=FMO_EXTEND; 3=FMO_INTERP; 4=FMO_NONE.
`REAL8 DefOutMapValue1`	(Output Map)'X' value used if target mapping falls outside of source mesh and `MapOption` is set to 1 (FMO_VALUE)
`REAL8 DefOutMapValue2`	(Output Map)'Y' value used if target mapping falls outside of source mesh and `MapOption` is set to 1 (FMO_VALUE).
`REAL8 DefOutMapValue3`	(Output Map)'Z' value used if target mapping falls outside of source mesh and `MapOption` is set to 1 (FMO_VALUE).
`REAL8 MapOffsetTolerance`	Specifies the Mapping Tolerance value.
`enum DataConversion`	Data Conversion method for output: 0=FMDC_VU (From View) 1=FMDC_AVG (Average) 2=FMDC_MAX (Max Absolute Value) 3=FMDC_MIN (Min Absolute Value) 4=FMDC_AVG_SKIP_CORNER (Average, Centroid Only) 5=FMDC_MAX_SKIP_CORNER (Max Absolute Value, Centroid Only) 6=FMDC_MIN_SKIP_CORNER (Min Absolute Value, Centroid Only) 7=FMDC_MAX_NOABS (Max Value) 8=FMDC_MIN_NOABS (Min Value) 9=FMDC_MAX_NOABS_SKIP_CORNER (Max Value, Centroid Only) 10=FMDC_MIN_NOABS_SKIP_CORNER (Min Value, Centroid Only)

Data Surface Methods

The following methods are available in addition to the standard entity methods.

GetDataSurf

Unpublished work. © 2025 Siemens

GetDataSurf	
(EntID)	
Description:	
This method retrieves the Data Surface ID.	
Input:	
`INT4 EntID`	ID of the Data Surface to retrieve.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the data surface.
Remarks/Usage:	
None	
Example:	
None	

PutDataSurf

PutDataSurf	
(EntID)	
Description:	
This method stores the Data Surface in the model database with the specified ID.	
Input:	
`INT4 EntID`	ID of the Data Surface to store.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to store the data surface.
Remarks/Usage:	
None	
Example:	
None	

VarAlongCoord

`VarAlongCoord` **`(numEntries, xyz, val_xyz)`**	
Description:	
This method creates a Along Coordinates Data Surface.	
Input:	
`INT4 numEntries`	Number of variation locations used to create Data Surface.

`REAL8 xyz[0..(3*numEntries)-1]`
 	
or
 	
`VARIANT vxyz`
	`xyz` is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*numEntries )-1]`
 	
or
 	
`VARIANT vval_xyz`
	`val_xyz` is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `AlongCoordProjectID`	
Example:	
None	

VarBetween2Pts

`VarBetween2Pts` **`(xyz, val_xyz)`**	
Description:	
This method creates a Between Coordinates Data Surface with the 2 Point Linear Variation Type.	
Input:	

`REAL8 xyz[0..(3*2)-1]`
 	
or
 	
`VARIANT vxyz`
	xyz is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*2)-1]`
 	
or
 	
`VARIANT vval_xyz`
	`val_xyz` is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

FE_FAIL	Unable to create the Data Surface.

Remarks/Usage:

The following Data Surface Object properties that are valid:

1) csys

Example:

```
Sub Main
Dim App As femap.model
Set App = feFemap

Dim ds As femap.DataSurf
Set ds = App.feDataSurf

Dim xyz(6) As Double
Dim xyzVal(6) As Double

For j = 0 To 2-1
   xyz(j*3+0) =  j*2
   xyz(j*3+1) =  0
   xyz(j*3+2) =  0

   xyzVal(j*3+0) =  j*1
   xyzVal(j*3+1) =  j*2
   xyzVal(j*3+2) =  0
Next j

ds.vector=True

rc = ds.VarBetween2Pts( xyz, xyzVal )
rc =ds.PutDataSurf( App.Info_NextID(FT_DATA_SURF) )

End Sub
```

VarBetween4Pts

VarBetween4Pts

(xyz, val_xyz)

Description:

This method creates a Between Coordinates Data Surface with the 4 Point Bilinear Variation Type.

Unpublished work. © 2025 Siemens

Input:	

`REAL8 xyz[0..(3*4)-1]`
 	
or
 	
`VARIANT vxyz`
	xyz is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*4)-1]`
 	
or
 	
`VARIANT vval_xyz`
	val_xyz is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
Data Surface Object properties that are valid: 1) csys	
Example:	
None	

VarBetween8Pts

VarBetween8Pts **(xyz, val_xyz**
Description:
This method creates a Between Coordinates Data Surface with the 8 Point Trilinear Variation Type.
Input:

Unpublished work. © 2025 Siemens

`REAL8 xyz[0..(3*8)-1]`
 	
or
 	
`VARIANT vxyz`
	`xyz` is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*8)-1]`
 	
or
 	
`VARIANT vval_xyz`
	`val_xyz` is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
Data Surface Object properties that are valid: 1) `csys`	
Example:	
None	

VarParametric2Pt

`VarParametric2Pt` **`(xyz, val_xyz)`**
Description:
This method creates a Between Coordinates Data Surface with the 2 Point Parametric Variation Type.
Input:

Unpublished work. © 2025 Siemens

`REAL8 xyz[0..(3*2)-1]`
 	
or
 	
`VARIANT vxyz`
	`xyz` is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*2)-1]`
 	
or
 	
`VARIANT vval_xyz`
	`val_xyz` is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `csys` 2) `ParamCurveSurfID`	
Example:	
None	

VarParametric4Pt

`VarParametric4Pt` **`(xyz, val_xyz)`**
Description:
This method creates a Between Coordinates Data Surface with the 4 Point Parametric Variation Type.
Input:

Unpublished work. © 2025 Siemens

`REAL8 xyz[0..(3*4)-1]`
 	
or
 	
`VARIANT vxyz`
	`xyz` is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*4)-1]`
 	
or
 	
`VARIANT vval_xyz`
	`val_xyz` is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `csys` 2) `ParamCurveSurfID`	
Example:	
None	

VarOutputMapV2

`VarOutputMapV2` **`(nModelID, groupID, outSetID, outVecID)`**
Description:
This method creates a Output Map Data Surface. This method replaces (insert method name without V2 here). For more information see, **Working with Output in Femap v2020.1 and above**.
Input:

Unpublished work. © 2025 Siemens

`INT8 nModelID`	ID of the source model containing output to be mapped.
`INT4 groupID`	ID of group of entities in source model from which to map output.
`INT4 outSetID`	ID of output set containing data from which to map output.
`INT4 outVecID`	ID of output vector containing data from which to map output.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `csys` 2) `OutMapOption` 3) `DefOutMapValue1` 4) `DefOutMapValue2` 5) `DefOutMapValue3`	
Example:	
None	

VarMeshSurface

`VarMeshSurface`
`(setID, eDataType, sX, sY, sZ)`
Description:
This method creates a Output Map Data Surface.
Input:

`INT4 setID`	ID of the femap set containing the Data Surface entities.

Unpublished work. © 2025 Siemens

`INT4 eDataType`	Type of entity in Data Surface. For more information, see **Entity Types**. Only Node and Element types are valid.
`STRING sX`	String containing value or expression for x cell in Data Surface.
`STRING sY`	String containing value or expression for y cell in Data Surface.
`STRING sZ`	String containing value or expression for z cell in Data Surface.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `csys`	
Example:	
None	

VarSpatialUVtable

`VarSpatialUVtable` **`(uDivisions, vDivisions)`**	
Description:	
This method creates a Tabular Parametric uv Data Surface.	
Input:	
`INT4 uDivisions`	Number of divisions in the u direction.
`INT4 vDivisions`	Number of divisions in the v direction.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `TabularUdefOption` 2) `TabularUdefVal`	
Example:	
None	

VarSpatialXYZTable

`VarSpatialXYZTable` **`(xDivisions, yDivisions, zDivisions)`**	
Description:	
This method creates a Output Map Data Surface.	
Input:	
`INT4 xDivisions`	Number of divisions in the 1st dimension.
`INT4 yDivisions`	Number of divisions in the 2st dimension.
`INT4 zDivisions`	Number of divisions in the 3st dimension.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

The following Data Surface Object properties that are valid: 1) `csys` 2) `TabularUdefOption` 3) `TabularUdefVal`
Example:
None

VarArbitrary3DCoord

`VarArbitrary3DCoord` **`(numEntries, xyz, val_xyz)`**	
Description:	
This method creates a Arbitrary 3D Data Surface.	
Input:	
`INT4 numEntries`	Number of variation locations in xyz, val_xyz arrays.

`REAL8 xyz[0..(3*numEntries)-1]`
 	
or
 	
`VARIANT vxyz`
	xyz is an array of the variation coordinates. In the array, there must be 3 entries for each location, followed by the 3 for the next location.

`REAL8 val_xyz[0..(3*numEntries)-1]`
 	
or
 	
`VARIANT vval_xyz`
	val_xyz is an array of the variation values. In the array, there must be 3 entries for the variation location, followed by the 3 variation values for the next location. If Data Surface is a Scalar then just define the y,z values as zero.

Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `csys` 2) `InterpPercent` 3) `InterpMin`	
Example:	
None	

VarEquation

`VarEquation` **`(equation)`**	
Description:	
This method creates a Arbitrary 3D Data Surface.	
Input:	
`STRING`	String containing equation.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the Data Surface.
Remarks/Usage:	
The following Data Surface Object properties that are valid: 1) `csys`	
Example:	
None	

Unpublished work. © 2025 Siemens

GetCellValue

GetCellValue **(nPage, nRow, nCol, nComponent, dValue)**	
Description:	
This method retrieves the value of a cell.	
Input:	
`INT4 nPage`	Page containing cell to retrieve value from.
`INT4 nRow`	Row containing cell to retrieve value from.
`INT4 nCol`	Column containing cell to retrieve value from.
`INT4 nComponent`	Component of the value vector in which to retrieve value from. Component: Scalar = 0, Vector = 0..3
Output:	
`REAL8 dValue`	Value of the cell.
Return Code:	
`FE_NOT_AVAILABLE`	Cell is Blank.
Remarks/Usage:	
None	
Example:	
None	

SetCellValue

Unpublished work. © 2025 Siemens

SetCellValue	
(nPage, nRow, nCol, nComponent, rVal)	
Description:	
This method sets the value of a cell.	
Input:	
`INT4 nPage`	Page containing cell to set value.
`INT4 nRow`	Row containing cell to set value.
`INT4 nCol`	Column containing cell to set value.
`INT4 nComponent`	Component of the value vector in which to set value for. Components: Scalar = 0, Vector = 0..3
`REAL8 rVal`	Value to set.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	Cell does not exist.
Remarks/Usage:	
None	
Example:	
None	

SetCellEquation

SetCellEquation
(nPage, nRow, nCol, nComponent, equation)
Description:

Unpublished work. © 2025 Siemens

This method sets the equation of a cell.	
Input:	
`INT4 nPage`	Page containing cell to set.
`INT4 nRow`	Row containing cell to set.
`INT4 nCol`	Column containing cell to set.
`INT4 nComponent`	Component of the value vector in which to set. Components: Scalar = 0, Vector = 0..3
`STRING equation`	Equation to set the cell to.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	Cell does not exist.
Remarks/Usage:	
None	
Example:	
None	

GetColumnTitle

`GetColumnTitle` **`(nPage, nCol, title)`**	
Description:	
This method retrieves the title of a column.	
Input:	
`INT4 nPage`	Page containing column to retrieve title from.

Unpublished work. © 2025 Siemens

`INT4 nCol`	Column retrieve title from.
Output:	
`STRING Title`	Title of column.
Return Code:	
`FE_FAIL`	Unable to get title.
Remarks/Usage:	
None	
Example:	
None	

GetColumnValue

`GetColumnValue` **`(nPage, nCol, rVal)`**	
Description:	
This method retrieves the value of a column.	
Input:	
`INT4 nPage`	Page containing column to retrieve value from.
`INT4 nCol`	Column retrieve value from.
Output:	
`REAL8 rVal`	Value of column.
Return Code:	
`FE_NOT_AVAILABLE`	Column Value is blank.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetRowTitle

GetRowTitle **(nPage, nRow, sTitle)**	
Description:	
This method retrieves the title of a row.	
Input:	
`INT4 nPage`	Page containing row to retrieve title from.
`INT4 nRow`	Row to retrieve title from.
Output:	
`STRING sTitle`	Title of row.
Return Code:	
`FE_FAIL`	Unable to get title.
Remarks/Usage:	
None	
Example:	
None	

SetRowTitle

Unpublished work. © 2025 Siemens

SetRowTitle	
(nPage, nRow, sTitle)	
Description:	
This method sets the title of a row.	
Input:	
`INT4 nPage`	Page containing row to set title for.
`INT4 nRow`	Row to set title for.
`STRING sTitle`	Title of row.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get title.
Remarks/Usage:	
None	
Example:	
None	

GetRowValue

GetRowValue	
(nPage, nRow, rVal)	
Description:	
This method retrieves the title of a row.	
Input:	

Unpublished work. © 2025 Siemens

`INT4 nPage`	Page containing row to retrieve value from.
`INT4 nRow`	Row to retrieve value from.
Output:	
`REAL8 rVal`	Value of row.
Return Code:	
`FE_NOT_AVAILABLE`	Row is blank.
Remarks/Usage:	
None	
Example:	
None	

SetRowValue

`SetRowValue`	
`(nPage, nRow, sTitle)`	
Description:	
This method sets the value of a row.	
Input:	
`INT4 nPage`	Page containing row to set value for.
`INT4 nRow`	Row to set value for.
`REAL8 rVal`	Value of row.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to set value.
Remarks/Usage:	
None	
Example:	
None	

GetPageTitle

`GetPageTitle` **`(nPage, sTitle)`**	
Description:	
This method retrieves the title of a page.	
Input:	
`INT4 nPage`	Page to retrieve title from.
Output:	
`STRING sTitle`	Title of page.
Return Code:	
`FE_FAIL`	Unable to get title.
Remarks/Usage:	
None	
Example:	
None	

SetPageTitle

Unpublished work. © 2025 Siemens

SetPageTitle	
(nPage, sTitle)	
Description:	
This method sets the title of a page.	
Input:	
`INT4 nPage`	Page to set title.
`STRING sTitle`	Title of page.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set page title.
Remarks/Usage:	
None	
Example:	
None	

GetPageValue

GetPageValue	
(nPage, rVal)	
Description:	
This method retrieves the value of a page.	
Input:	
`INT4 nPage`	Page to retrieve value from.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 rVal`	Value of page.
Return Code:	
`FE_NOT_AVAILABLE`	Unable to get value.
Remarks/Usage:	
None	
Example:	
None	

SetPageValue

`SetPageValue`	
`(nPage, rval)`	
Description:	
This method sets the value of a page.	
Input:	
`INT4 nPage`	Page to set value.
`REAL8 rVal`	Value of page.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set page value.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetColumTitle

SetColumTitle **(nPage, nCol, sTitle)**	
Description:	
This method set the title of a column.	
Input:	
`INT4 nPage`	Page that contains the column to set.
`INT4 nCol`	Column to set.
`STRING sTitle`	Title to set.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set title.
Remarks/Usage:	
None	
Example:	
None	

SetColumValue

SetColumValue **(nPage, nCol, rVal)**
Description:
This method sets the value of a column.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nPage`	Page that contains the column to set.
`INT4 nCol`	Column to set.
`REAL8`	Value to set.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set value.
Remarks/Usage:	
None	
Example:	
None	

SetCellBlank

`SetCellBlank` **`(nPage, nRow, nCol, nComponent)`**	
Description:	
This method set a cell to blank.	
Input:	
`INT4 nPage`	Page that contains the cell to set.
`INT4 nRow`	Row containing cell to set.
`INT4 nCol`	Column containing cell to set.
`INT4 nComponent`	Component of value vector to set.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set cell to blank.
Remarks/Usage:	
None	
Example:	
None	

GetDataSurfInfo

`GetDataSurfInfo`	
`(nPages, nComponent, sTitle)`	
Description:	
This method retrieves Data Surface information.	
Input:	
None	None
Output:	
`INT4 nPages`	Number of Pages in Data Surface.
`INT4 nComponent`	Number of components for Data Surface.
`STRING sTitle`	Title of Data Surface.
Return Code:	
`FE_FAIL`	Unable to get Data Surface info.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetPageInfo

`GetPageInfo`
 	
`(nPage, nRows, nCols)`	

Description:	
This method retrieves page information.	
Input:	
`INT4 nPage`	Page to retrieve info for.
Output:	
`INT4 nRows`	Number of Row in page.
`INT4 nCols`	Number of Columns in page.
Return Code:	
`FE_FAIL`	Unable to get page info.
Remarks/Usage:	
None	
Example:	
None	

Resize

Unpublished work. © 2025 Siemens

Resize	
(nPage, nRow, nCol, bVector)	
Description:	
This method resizes the Data Surface grid.	
Input:	
`INT4 nPage`	Number of Pages.
`INT4 nRow`	Number of Rows.
`INT4 nCol`	Number of Columns.
`BOOL bVector`	Flag for vector Data Surface.
Output:	
None	None
Return Code:	
`FE_TOO_SMALL`	Grid size is invalid.
Remarks/Usage:	
None	
Example:	
None	

Evaluate

Evaluate	
(geomID, eDataType, MapSetID, isElemental)	
Description:	
This method Evaluates the Data Surface prior to calling `GetData()`.	

Unpublished work. © 2025 Siemens

Input:	
`INT4 geomID`	Geometry supplying parametric space. Between 2 Point Parametric Data Surface: ID of Curve Between 4 Point Parametric Data Surface: ID of Surface
`INT4 DataType`	Type of entity Data Surface needs to be evaluated for. Between 2 Point Parametric Data Surface Type = Curve Between 4 Point Parametric Data Surface Type = Surface Mesh Data Surface Type = Node or Element For more information, see **Entity Types**. Only Node and Element types are valid.
`INT4 MapSetID`	(Output Map Data Surface) ID of the Femap Set containing entities to map the Data Surface to.
`BOOL isElemental`	(Output Map Data Surface) Flag for elemental output.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to evaluate Data Surface.
Remarks/Usage:	
None	
Example:	
None	

GetData

Unpublished work. © 2025 Siemens

GetData

(geomID, feaID, Coord, answer, valXYZ)

Description:

This method gets the computed Data Surface value at the independent variable.

Input:

`INT4 geomID`	Between 2,4 Point Parametric Data Surface: ID of curve/surface supplying parametric space.
`INT4 feaID`	Mesh, Output Map Data Surface: ID of mesh entity. Equation Data Surface: ID of Mesh entity for id variable.

`IREAL8 Coord[0..3]`
or
`VARIANT Coord`
	Along Coordinates, Between Coordinates, Tabular, Arbitrary 3D, Equation Data Surfaces: Location where the Data Surface will be evaluated at.

Output:

`REAL8 answer`	Computed value for Scalar Data Surface. Magnitude for Vector Data Surface.

`REAL8 valXYZ[0..3]`
or
`VARIANT valXYZ`
	Computed vector components for Vector Data Surfaces.

Return Code:

None	None

Remarks/Usage:

Before calling `GetData` the Data Surface must first have been evaluated with `Evaluate()`.

Example:

```
Sub Main
Dim App As femap.model
Set App = feFemap

Dim ds As femap.DataSurf
```


Unpublished work. © 2025 Siemens

```
Set ds = App.feDataSurf
Dim Coord(3) As Double
Dim ans As Double
Dim entSet As femap.Set
Set entSet = App.feSet
```

```
Dim nd As femap.Node

Set nd = App.feNode

Dim RBO As femap.Results

Set RBO = App.feResults

Dim LCol As Long

Dim setID As Long

Dim outset As femap.OutputSet

Set outset = App.feOutputSet

Dim ndIDV As Variant

Dim thicknessV As Variant

Dim ndLIST(100000) As Long

Dim thick(100000) As Double

Dim k As Long

Dim i As Long


' Create Output Set

setID = outset.NextEmptyID()

outset.title = "Data Surface"

outset.value = 0

outset.analysis = 0

outset.Put (setID)


rc = RBO.AddScalarAtNodeColumnV2( setID2, 2147000000, "Interpolated
Data", FOT_TEMP, False, LCol )

k = i
```


Unpublished work. © 2025 Siemens

```
' Get Nodes
entSet.Select( FT_NODE, True, "Select Nodes to Get Data Surface Data" )

' Get Data Surface and evaluate
rc =ds.GetDataSurf(1)
rc = ds.Evaluate( 0, 0, 0, False )

i = 0
```

```
feaID = entSet.First()
    While feaID > 0
       nd.Get(feaID)
       Coord(0) = nd.x
       Coord(1) = nd.y
       Coord(2) = nd.z

        ' Get Data Surface value at nodal location
       rc = ds.GetData( 0, feaID, Coord, ans, vXYZVal )

       ndLIST(i) = feaID
       thick(i) = ans

       i = i +1
       feaID = entSet.Next
   Wend

k = entSet.count
ndIDV = ndLIST
```


Unpublished work. © 2025 Siemens

```
thicknessV = thick


rc = RBO.SetScalarAtNodeColumnV2( LCol, i, ndIDV, thicknessV)
rc = RBO.Save()


End Sub
```

GetDataArray

GetDataArray	
(nCount, vGeomID, vfeaID, vCoord, sVal, cgXYZ)	
Description:	
This method is similar to `GetData()`, but allows for retrieval of data at multiple geometry/fea entities/ coordinates.	
Input:	
`INT4 nCount`	Number of entities being supplied in `vGeomID`, vfeaID or 1/3 the number in `vCoord`. Note that the size of arrays has to be consistent with this value.
`INT4 vGeomID [0..nCount-1]`	Between 2-Point or 4-Point Parametric Data Surface: ID of curve/surface(s) supplying parametric space.
`INT4 vfeaID [0..nCount-1]`	Mesh or Output Map Data Surface: ID of mesh entity or entities. Equation Data Surface: ID of Mesh entity for id variable.
`REAL8 vCoord[0..3*nCount-1]`	Along Coordinates, Between Coordinates, Tabular, Arbitrary 3D, or Equation Data Surfaces: Location where the Data Surface will be evaluated at in global rectangular coordinates.
Output:	
`VARIANT sVal`	Computed values for Scalar Data Surface. Magnitudes for Vector Data Surface.

Unpublished work. © 2025 Siemens

`VARIANT cgXYZ`	Computed vector components for Vector Data Surfaces.
Return Code:	
None	None
Remarks/Usage:	
Before calling `GetDataArray()` the Data Surface must first have been evaluated with `Evaluate()`.	
Example:	
None	

GetDataSurfType

`GetDataSurfType`	
`(void)`	
Description:	
This method returns the Data Surface Type.	
Input:	
None	None
Output:	
None	None
Return Code:	
`type`	The ID of the Data Surface Type. 0=Tabular Data Surface 1=Equation Data Surface 2=Nodes Data Surface 3=Elements Data Surface 4=Parametric 2 Point Data Surface 5=Parametric 4 Point Data Surface 6=Along Coordinates Data Surface 7=4 Point Bilinear Data Surface 8=8 Point Trilinear Data Surface 9=2 Point Linear Data Surface 10=Arbitrary 3-D Location Data Surface

Unpublished work. © 2025 Siemens

	11=Output Map Data Surface
Remarks/Usage:	
None	
Example:	
None	

Design Equation Objects

FlexibleSlider objects correspond to the flexible slider simulation objects in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Joint objects. For more information, see **Common Entity Properties and Methods**.

Use the `feFlexibleSlider` method of the Femap Application object to create FlexibleSlider objects.

FlexibleSlider Object Properties

Flexible Sliders are always stored with a `SetID=1`, and the ID equal to the Flexible Slider ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The element color.

`INT4 formulation[0..2]`
 	
or
 	
`VARIANT vformulation`
	Additional flags defining the element type.

Unpublished work. © 2025 Siemens

Accessing Element Nodes

For most elements, the node property will give you access to the IDs of the nodes that define the element. Before you can use this data, however, you must know where to find the nodes in the node array. The following table defines the layout for the various element types:

Topology	Array Entries (optional/midside nodes in parentheses)
Point	0
Line	0,1
Tri3	0,1,2
Tri6	0,1,2, (4,5,6)
Quad4	0,1,2,3
Quad8	0,1,2,3,(4,5,6,7)
Tetra4	0,1,2, 4
Pyramid5	0,1,2,3, 4
Wedge6	0,1,2, 4,5,6
Brick8	0,1,2,3,4,5,6,7
Tetra10	0,1,2, 4 (8,9,10, 12,13,14)
Pyramid13	0,1,2,3, 4 (8,9,10,11, 12,13,14,15)
Wedge15	0,1,2, 4,5,6 (8,9,10, 12,13,14, 16,17,18)
Brick20	0,1,2,3,4,5,6,7, (8,9,10,11,12,13,14,15,16,17,18,19)
RigidList	0=Independent, dependent nodes use element lists, not array entries.
MultiList	Uses element lists, not array entries.
Contact	References contact segments, not nodes.
Weld	Weld Axis = 0,1 SegA= 4,5,6,(7,8,9,10,11) SegB=12,13,14,(15,16,17,18,19).

Unpublished work. © 2025 Siemens

Direct Matrix Input Objects

Direct Matrix Input objects correspond to Direct Matrix Input Simulation Entities in the model, often known as DMIG in Nastran, which are found on the **Model → Simulation Entities** menu. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Direct Matrix Input objects. For more information, see Common Entity Properties and Methods.

Use the `feMatrixInput` method of the Femap Application object to create Direct Matrix Input objects.

Direct Matrix Input Object Properties

Direct Matrix Input objects are always stored with a `SetID=1`, and the ID equal to the Direct Matrix Input ID.

Property	Description
`STRING title`	Direct Matrix Input Reference title (maximum 56 characters).
`INT4 DataSource`	Data Source. Use enum zMatSrcForm: Matrix using Internalized Data = `MAT_INT (0)` Matrix referencing External File = `MAT_EXT (1)` Matrix Combination = `MAT_COMB (2)`
`INT4 FileInputForm`	Only used when Source Data is set to External File. Use enum `zMatFileForm`: 0..NASTRAN Punch (*.*pch*) = `MAT_PCH (0)` 1..NASTRAN OP2 (*.*op2*) = `MAT_OP2 (1)`
`STRING matrixDir`	Path to external Source Data file (.*op2* or .*pch* file)
`INT4 UnitID`	FORTRAN Unit ID, which is only used when Source Data is from an external .op2 file
`INT4 AssignForm`	Only used when Source Data is from an external .*op2* file. Form used for ASSIGN statements. Use enum `zAnalysisAssignForm`. 0..None = `AAF_NONE` 1..FORMATTED = `AAF_FORMATTED`

Unpublished work. © 2025 Siemens

Property	Description
	2..UNFORMATTED = AAF_UNFORMATED 3..LITTLEENDIAM = AAF_LITTLEENDIAN 4..BIGENDIAN = AAF_BIGENDIAN
`STRING Name`	Direct Matrix Input Reference Name (maximum 8 characters).
`INT4 InputForm`	Matrix Form. Use enum `zMatInputForm`: Square = `MAT_SQUARE (0)` Symmetric = `MAT_SYMMETRIC (1)` Rectangular Type 2 = `MAT_RECT2 (2)` Rectangular Type 9 = `MAT_RECT9 (3)`
`INT4 InputPrecision`	Input Format/Precision. Use enum `zMatIOType`: Machine = `MAT_MACHPRES (0)` Real Single = `MAT_REAL_SINGLE (1)` Real Double = `MAT_REAL_DOUBLE (2)` Complex Single = `MAT_CPLX_SINGLE (3)` Complex Double = `MAT_CPLX_DOUBLE (4)`
`INT4 OutputPrecision`	Output Format/Precision. Use enum `zMatIOType`: Machine = `MAT_MACHPRES (0)` Real Single = `MAT_REAL_SINGLE (1)` Real Double = `MAT_REAL_DOUBLE (2)` Complex Single = `MAT_CPLX_SINGLE (3)` Complex Double = `MAT_CPLX_DOUBLE (4)`
`INT4 ComplexForm`	Complex Format. Use `zMatCplxForm`: Real/Imaginary = `MAT_REAL_IMAG (0)` Amplitude/Phase = `MAT_AMP_PHASE (1)`
`INT4 colCount`	Number of Columns.
`REAL8 ScaleFactor`	Scale Factor in Selection Data.

Direct Matrix Input Object Methods

There following methods are available in addition to the standard entity methods.

Unpublished work. © 2025 Siemens

PutMatrixEntry

PutMatrixEntry

**(nSetID, NodeColumnIndex, NodeColumnComponent, NodeRowIndex, NodeRowComponent,
RealMatrixValue, ImagMatrixValue)**

Description:

This method is used to store a matrix entry in an internalized Matrix Input entity.

Input:

`INT4 nSetIDn`	The ID of the Matrix Input entity to store the entry.
`INT4 NodeColumnIndex`	The ID for the node that represents the column index for the entry.
`INT4 NodeColumnComponent`	The component number for the node that represents the column index.
`INT4 NodeRowIndex`	The ID for the node that represents the row index.
`INT4 NodeRowComponent`	The component number for the node that represents the row index.
`REAL8 RealMatrixValue`	The real (or amplitude) part of a matrix element.
`REAL8 ImagMatrixValue`	The imaginary (or phase) part of a matrix element.

Output:

None	None

Return Code:

`FE_NOT_EXIST`	The Matrix Input entity does not exist, or the node that represents the row or column index does not exist.
`FE_BAD_DATA`	The Matrix Input entity's data source is not internal.

Remarks/Usage:

None

Unpublished work. © 2025 Siemens

Example:
None

GetMatrixEntry

GetMatrixEntry

```
( nSetID, NodeColumnIndex, NodeColumnComponent, NodeRowIndex,
NodeRowComponent,
 RealMatrixValue, ImagMatrixValue )
```

Description:

This method is used to retrieve the real and imaginary matrix entry values from an internalized Matrix Input entity, given its row and column index.

Input:

`INT4 nSetIDn`	The ID of the Matrix Input entity to store the entry.
`INT4 NodecColumnIndex`	The ID for the node that represents the column index for the entry.
`INT4 NodeColumnComponent`	The component number for the node that represents the column index.
`INT4 NodeRowIndex`	The ID for the node that represents the row index.
`INT4 NodeRowComponent`	The component number for the node that represents the row index.

Output:

`REAL8 RealMatrixValue`	The real (or amplitude) part of a matrix element.
`REAL8 ImagMatrixValue`	The imaginary (or phase) part of a matrix element.

Return Code:

`FE_NOT_EXIST`	The Matrix Input entity does not exist, the node that represents the row or column index does not exist, or a matrix entry for that set of components does not exist.

Unpublished work. © 2025 Siemens

`FE_BAD_DATA`	The Matrix Input entity's data source is not internal.
Remarks/Usage:	
None	
Example:	
None	

PutMatrixList

`PutMatrixList` **`( listcount, NodeColumnIndex, NodeColumnComponent, NodeRowIndex, NodeRowComponent, RealMatrixValue, ImagMatrixValue )`**	
Description:	
This method stores information to the matrix entries in a Matrix Input.	
Input:	
`INT4 listcount`	The number of entries in the other arrays.
`INT4 NodeColumnIndex[0..listcount -1]`	The ID for the node that represents the column index for the entry.
`INT4 NodeColumnComponent[0..listcount -1]`	The component number for the node that represents the column index.
`INT4 NodeRowIndex[0..listcount-1]`	The ID for the node that represents the row index.
`INT4 NodeRowComponent[0..listcount-1]`	The component number for the node that represents the row index.
`INT4 RealMatrixValue[0..listcount-1]`	The real (or amplitude) part of a matrix element.
`INT4 ImagMatrixValue[0..listcount-1]`	The imaginary (or phase) part of a matrix element.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	No matrix entries exist in the Matrix Input entity.
Remarks/Usage:	
This method stores information into the list of the current object. Any previous list information in that object is lost. This method does not store that information to the model database. After using this function, you must call `Put` if you want to store the updated list in the database.	
Example:	
None	

GetMatrixList

`GetMatrixList` **`( listcount, NodeColumnIndex, NodeColumnComponent, NodeRowIndex, NodeRowComponent,`** **` RealMatrixValue, ImagMatrixValue )`**	
Description:	
This method retrieves information from the matrix entries in a Matrix Input.	
Input:	
None	None
Output:	
`INT4 listcount`	The number of entries in the other arrays.
`INT4 NodeColumnIndex[0..listcount -1]`	The ID for the node that represents the column index for the entry.
`INT4 NodeColumnComponent[0..listcount -1]`	The component number for the node that represents the column index.
`INT4 NodeRowIndex[0..listcount-1]`	The ID for the node that represents the row index.

Unpublished work. © 2025 Siemens

`INT4 NodeRowComponent[0..listcount-1]`	The component number for the node that represents the row index.
`INT4 RealMatrixValue[0..listcount-1]`	The real (or amplitude) part of a matrix element.
`INT4 ImagMatrixValue[0..listcount-1]`	The imaginary (or phase) part of a matrix element.
Return Code:	
`FE_NOT_EXIST`	No matrix entries exist in the Matrix Input entity.
Remarks/Usage:	
This method simply retrieves the information about the matrix entries from the current object. You must first load that data, either by using the `Get` method or by loading it yourself with `PutMatrixList`.	
Example:	
None	

PutCombination

`PutCombination`	
`( nSetID, dScale, MatrixInputID, InputName )`	
Description:	
This method stores matrix combinations in a combined Matrix Input entity.	
Input:	
`INT4 nSetIDn`	The ID of the combined Matrix Input entity where to store the combination.
`REAL8 dScale`	The scale factor of the combination.
`INT4 MatrixInputID`	The ID of the referenced Matrix Input (if it exists).
`STRING InputName`	The name of the matrix referenced by the combination (if specified as direct name input).
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_NOT_EXIST`	The specified Matrix Input entity did not exist.
`FE_BAD_DATA`	If a Matrix Input entity was not specified, the length of the direct name input was zero.
Remarks/Usage:	
This method stores matrix combinations to a combined Matrix Input. The IDs of the combination must first be loaded into a Set Object using `GetCombinationSet`.	
Example:	
None	

GetCombination

`GetCombination` **`( nSetID, entID, dScale, MatrixInputID, InputName )`**	
Description:	
This method retrieves information from the matrix combinations in a combined Matrix Input entity.	
Input:	
`INT4 nSetIDn`	The ID of the combined Matrix Input entity from where to retrieve the combination.
`INT4 entID`	The ID of the combination (i.e., place in the list of Referenced Inputs).
Output:	
`REAL8 dScale`	The scale factor of the combination.
`INT4 MatrixInputID`	The ID of the referenced Matrix Input (if it exists).
`STRING InputName`	The name of the matrix referenced by the combination (if specified as direct name input).
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	The specified combination did not exist in the specified matrix input or specified matrix input is not a combined Matrix Input entity.
Remarks/Usage:	
This method retrieves a single line from the Referenced Inputs section from a combined Matrix Input. The IDs of the combination can first be loaded into a Set Object using `GetCombinationSet`.	
Example:	
None	

PutCombinations

`PutCombinations`	
`( listcount, MatrixEntityIDs, DirectInputs, ScaleFactors )`	
Description:	
This method stores matrix combinations in a combined Matrix Input entity.	
Input:	
`INT4 listcount`	The number of matrix input entities in the combination.
`INT4 MatrixEntityIDs[0..list► count-1]`	The IDs of the matrix input entities in the combination.
`STRING DirectInputs[0..list► count-1]`	The titles of the Direct Name Input.
`REAL8 ScaleFactor[0..listcount-1]`	The scale factors of the combinations.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The Matrix Input is not a combined matrix input.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetCombinations

GetCombinations **(MatrixComboEntityID, listcount, MatrixEntityIDs, DirectInputs, ScaleFactors)**	
Description:	
This method retrieves information from the matrix combinations in a combined Matrix Input entity.	
Input:	
`INT4 MatrixComboEntityID`	The ID of the combined Matrix Input entity where to store the combination.
Output:	
`INT4 listcount`	The number of matrix input entities in the combination.
`INT4 MatrixEntityIDs[0..list▶ count-1]`	The IDs of the matrix input entities in the combination.
`STRING DirectInputs[0..list▶ count-1]`	The titles of the Direct Name Input.
`REAL8 ScaleFactor[0..listcount-1]`	The scale factors of the combinations.
Return Code:	
`FE_NOT_EXIST`	The combined Matrix entity does not contain matrix input entities.
`FE_BAD_TYPE`	If a Matrix Combo Entity ID is not a combined matrix.
`FE_FAIL`	If a Matrix Combo Entity ID does not exist.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetCombinationSet

`GetCombinationSet` **`( pCombinationSet )`**	
Description:	
This method loads a Set Object with the IDs of matrix input entries in a combined Matrix Input entity currently loaded in the Matrix Input Object.	
Input:	
None	None
Output:	
`OBJECT pCombinationSet`	The Set Object that will receive the IDs of the matrix combinations.
Return Code:	
`FE_FAIL`	The set could not be loaded.
Remarks/Usage:	
None	
Example:	
None	

Discrete Value Set Object

The Discrete Value Set object is used to create a set of *discrete values* which can then be plotted using the **View → Advanced Post → Contour Model Data** command.

API access for plotting discrete values is in two locations. This section details the creation and editing of a set of discrete values. The controls used to turn on/off a discrete value plot, choose the type of

Unpublished work. © 2025 Siemens

plot to display and other options, as well as which discrete value set to use, are properties on the View Object. See the "Model Data Contour and Discrete Value controls" portion of **View Object Properties**, **SetModelDataContourType**, and **Model Contour Data Types** for information about selecting a model data item to use when plotting a set of discrete values.

Use the `feDiscreteValueSet` method of the Femap Application object to create `DiscreteValueSet` objects.

Discrete Value Set Object Properties

Discrete Value Sets are always stored with a `SetID=1`, and the ID equal to the discrete value set ID.

Property	Description
`BOOL IntegerType`	Flag to indicate if discrete value set contains Integer values (True) or Floating Point (i.e., Real Number) values (False).
`BOOL RangeType`	Flag to indicate if discrete value set is made up of Ranges of values (True) or Individual values (False).

Discrete Value Set Object Methods

This is a para in the topic template.

GetNumberLevels

`GetNumberLevels` **`( pnLevels )`**	
Description:	
Returns the number of levels, ranges of values or individual values, in the discrete value set (i.e., rows shown in the Discrete Value Set dialog box).	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`INT4 pnLevels`	Number of levels
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetLevelInteger

`GetLevelInteger`	
`( iLevelID, iValue1, iValue2, iColor )`	
Description:	
Returns the integer value(s) and color ID for a given level in the discrete value set.	
Input:	
`INT4 iLevelID`	Level ID.
Output:	
`INT4 iValue1`	For a discrete value set with individual values, returns the integer value for the specified level. For a discrete value set with ranges of values, returns the integer value which is the lower bound of the range for the specified level.
`INT4 iValue2`	Only used when a discrete value set has ranges of values. Returns the integer value which is the upper bound of the range for the specified level.
`INT4 iColor`	ID of the Color currently assigned to the specified level.
Return Code:	
`FE_FAIL`	Level does not contain Integer Values.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

SetLevelInteger

`SetLevelInteger` **`( iLevelID, iValue1, iValue2, iColor )`**	
Description:	
Sets integer value(s) and color ID for the specified level in the discrete value set.	
Input:	
`INT4 iLevelID`	Level ID.
`INT4 iValue1`	For a discrete value set with individual values, sets the integer value for the specified level. For a discrete value set with ranges of values, sets integer value which is the lower bound of the range for the specified level.
`INT4 iValue2`	Only used when a discrete value set has ranges of values. Sets the integer value which is the upper bound of the range for the specified level.
`INT4 iColor`	ID of the Color to assign the specified level.
Output:	
None	None
Return Code:	
`FE_FAIL`	Level does not contain Integer Values.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

AddLevelInteger

`AddLevelInteger`
 	
`( iValue1, iValue2, iColor )`	

Description:	
Adds a new level to the bottom of the discrete value set containing the specified integer value(s) and assigns the specified color to the new level.	
Input:	
`INT4 iValue1`	For a discrete value set with individual values, sets the integer value for the new level. For a discrete value set with ranges of values, sets integer value which is the lower bound of the range for the new level.
`INT4 iValue2`	Only used when a discrete value set has ranges of values. Sets the integer value which is the upper bound of the range for the new level.
`INT4 iColor`	ID of the Color to assign the new level.
Output:	
None	None
Return Code:	
`FE_FAIL`	Level does not contain Integer Values.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetLevelFloat

GetLevelFloat **(iLevelID, pdValue1, pdValue2, iColor)**	
Description:	
Returns the floating point (real number) value(s) and color ID for a given level in the discrete value set	
Input:	
`INT4 iLevelID`	Level ID.
Output:	
`REAL8 iValue1`	For a discrete value set with individual values, returns the floating point value for the specified level. For a discrete value set with ranges of values, returns the floating point value which is the lower bound of the range for the specified level.
`REAL8 iValue2`	Only used when a discrete value set has ranges of values. Returns the floating point value which is the upper bound of the range for the specified level.
`INT4 iColor`	ID of the Color currently assigned to the specified level.
Return Code:	
`FE_FAIL`	Level does not contain Floating Point Values.
Remarks/Usage:	
None	
Example:	
None	

SetLevelFloat

Unpublished work. © 2025 Siemens

SetLevelFloat	
(iLevelID, pdValue1, pdValue2, iColor)	
Description:	
Sets floating point (real number) value(s) and color ID for the specified level in the discrete value set.	
Input:	
`INT4 iLevelID`	Level ID.
`REAL8 iValue1`	For a discrete value set with individual values, sets the floating point value for the specified level. For a discrete value set with ranges of values, sets floating point value which is the lower bound of the range for the specified level.
`REAL8 iValue2`	Only used when a discrete value set has ranges of values. Sets the floating point value which is the upper bound of the range for the specified level.
`INT4 iColor`	ID of the Color to assign the specified level.
Output:	
None	None
Return Code:	
`FE_FAIL`	Level does not contain Floating Point Values.
Remarks/Usage:	
None	
Example:	
None	

AddLevelFloat

Unpublished work. © 2025 Siemens

AddLevelFloat **(pdValue1, pdValue2, iColor)**	
Description:	
Adds a new level to the bottom of the discrete value set containing the specified floating point (real number) value(s) and assigns the specified color to the new level.	
Input:	
`REAL8 iValue1`	For a discrete value set with individual values, sets the floating point value for the new level. For a discrete value set with ranges of values, sets floating point value which is the lower bound of the range for the new level.
`REAL8 iValue2`	Only used when a discrete value set has ranges of values. Sets the floating point value which is the upper bound of the range for the new level.
`INT4 iColor`	ID of the Color to assign the new level.
Output:	
None	None
Return Code:	
`FE_FAIL`	Level does not contain Floating Point Values.
Remarks/Usage:	
None	
Example:	
None	

Sort

Unpublished work. © 2025 Siemens

Sort	
(void)	
Description:	
Sorts the entries in the discrete value set into ascending order based on the first and second values.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_FAIL	None
Remarks/Usage:	
A Sort is automatically called in the Put function for Discrete Value Set Objects.	
Example:	
None	

GetTitle

GetTitle	
(sTitle)	
Description:	
Returns the title of the discrete value set.	
Input:	
None	None
Output:	
STRING sTitle	Title of discrete value set.
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

SetTitle

SetTitle **(sTitle)**	
Description:	
Sets the title of the discrete value set.	
Input:	
`STRING sTitle`	Title of discrete value set.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

CreateRangeInteger

Unpublished work. © 2025 Siemens

CreateRangeInteger	
(nRanges, iMinimum, iMaximum)	
Description:	
Automatically create a sequential range of integer values in a discrete value set based on the provided number of ranges and Maximum/Minimum values.	
Input:	
`INT4 nRanges`	Number of ranges to create.
`INT4 iMinimum`	Minimum integer value.
`INT4 iMaximum`	Maximum integer value.
Output:	
None	None
Return Code:	
`FE_FAIL`	The maximum value is less than the minimum value or the value specified for `nRanges` is < 1.
Remarks/Usage:	
Colors are automatically assigned to the levels	
Example:	
None	

CreateRangeIFloat

CreateRangeIFloat
(nRanges, dMinimum, dMaximum)
Description:
Automatically create a sequential range of floating point (real number) values in a discrete value set based on the provided number of ranges and Maximum/Minimum values.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nRanges`	Number of ranges to create.
`REAL8 dMinimum`	Minimum floating point (real number) value.
`REAL8 dMaximum`	Maximum floating point (real number) value.
Output:	
None	None
Return Code:	
`FE_FAIL`	The maximum value is less than the minimum value or the value specified for `nRanges` is < 1.
Remarks/Usage:	
Colors are automatically assigned to the levels.	
Example:	
None	

AutoCreateLevels

`AutoCreateLevels` **`( iVuID, iGroupID, sTitle, zMDCType, nLevels, iRangeOption, iLowestValue, dLowestValue, zElementType, zMaterialType, iLocation )`**
Description:
Automatically creates a discrete value set using the specified model data item, which can be created using data from the whole model or a group.
Input:

`INT4 iVuID`	ID of View is required to use.
`INT4 iGroupID`	ID of Group to use to limit model data values and should correspond to the value specified for `nModelDataContourGroup` on the View Object.

Unpublished work. © 2025 Siemens

`STRING sTitle`	Title of discrete value set. If not specified, a title will automatically be created based on the model data item and other options specified.
`enum zMDCType`	Used to specify which model data item to display as a contour, criteria, beam diagram. See **Model Contour Data Types** for more information about selecting the appropriate model data item. If using any option other than `FVMDC_MATL_TABLE` or `FVMDC_PROP_TABLE`, the `zElementType`, `zMaterialType`, and `iLocation` arguments in this method are ignored.
`INT4 nLevels`	Specifies the number of levels to create in the Discrete Value Set.
`INT4 iRangeOption`	Specifies the option to use for Range Type. 0=Individual Value, 1=Range by equal value, 2=Range by equal number, 3=Range by largest gap.
`INT4 iLowestValue`	Lowest value to use when, such as property ID or material ID
`REAL8 dLowestValue`	Lowest value to use when, such as plate thickness.
`enum zElementType`	Only used when `zMDCType` is set to `FVMDC_PROP_TABLE`. Indicates the element/ property type to use for the index value specified in `iLocation` (See **Element Types** in **Femap Constants**.
`enum zMaterialType`	Only used when `zMDCType` is set to `FVMDC_MATL_TABLE`. Indicates the material type to use for the index value specified in `iLocation` (See **"Material Types"** in **Femap Constants**.
`iLocation`	Only used when `zMDCType` is set to `FVMDC_MATL_TABLE` or `FVMDC_PROP_TABLE`. ID of the index in the Material Array Index (See **Material Array Index Values**) or Property Array Index (See **Property Array Index Values**) to use to create the plot.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Colors are automatically assigned to the levels.

Example:

```
Sub Main

  Dim App As femap.model

  Set App = feFemap()


  Dim dvs As femap.DiscreteValueSet

  Set dvs = App.feDiscreteValueSet


‘Creates a new Discrete Value Set with 17 individual

‘values For Property ID in the next empty ID

 rc = dvs.AutoCreateLevels (0, 0, "", FVMDC_PROP_ID, 17, 0, 0,_ 0.0, 0,
0, 0)

   dvs.Put (dvs.NextEmptyID)


'Creates a new Discrete Value Set with 10 ranges of

'Plate Thickness in the next empty ID

rc = dvs.AutoCreateLevels (0, 0, "", FVMDC_PROP_TABLE, 10, 1,0,_ 0.0,
FET_L_PLATE, 0, 0)


dvs.Put (dvs.NextEmptyID)


End Sub
```

Element Add | Remove Objects

Element Add | Remove objects correspond to SOL 401’s element addition and removal capabilities. Both Element Add | Remove Entities *and* Sets referencing multiple Entities can be created. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Element objects. For more information, see **Common Entity Properties and Methods**.

Unpublished work. © 2025 Siemens

Use the `feElemAddRemove` method of the Femap Application object to create Element Add | Remove objects.

Element Add | Remove Properties

Element Add | Remove Properties are always stored with a `SetID=1`, and the ID equal to the Element Add | Remove property ID.

In this table, you may see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`STRING title`	Element Add | Remove title.
`INT4 AltType`	Element Add | Remove type enumerated value (0 = ELAR, 1 = ELAR2, 2 = ELARADD).
`INT4 NasTYPE`	Adition/Removal criteria type (0 = Time, 1 = Inelastic Strain, 2 = Mechanical Strain).
`INT4 NasGRPID`	ID of the Femap Group which selects the elements for adding or removing.
`INT4 NasHasAC`	Specification of Addition Time flag.
`INT4 NasAC`	Addition Time.
`INT4 NasADOpt`	Addition Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`INT NasAD`	Addition Delta Time.
`INT4 NasHasRC`	Specification of Removal Time (or Strain) flag.
`INT4 NasRC`	Removal Time (or Strain).
`INT4 NasRDOpt`	Removal Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`INT4 NasRD`	Removal Delta Time.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasSTNFREE`	Addition with or without Strain flag (TRUE = Strain free addition, FALSE = Addition with strain).
`BOOL HasElemList`	When TRUE, the Set Object containing element IDs referenced by `ElemSetID` will be used for ELAR Selection. When FALSE, the Group specified by `NasGRPID` will be used.
`INT4 ElemSetID`	ID of Set Object containing Element IDs to be used by `HasElemList` is set to TRUE.

Element Add | Remove Object Methods

The following methods are available in addition to the standard entity methods.

GetELAR2Entry

`GetELAR2Entry`	
`( nElementID, TYPE, HasAC, AC, ADOpt, AD, HasRC, RC, RDOpt, RD, IsSTNFREE )`	
Description:	
This method is used to retrieve the option data for an ELAR2 list entry, given its element ID.	
Input:	
`INT4 nElementID`	The ID of element selected by this ELAR2 entry.
Output:	
`INT4 TYPE`	Adition/Removal criteria type (0 = Time, 1 = Inelastic Strain, 2 = Mechanical Strain).
`INT HasAC`	Specification of Addition Time flag.
`REAL8 AC`	Addition Time.
`INT4 ADOpt`	Addition Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).

Unpublished work. © 2025 Siemens

`REAL8 AD`	Addition Delta Time
`INT HasRC`	Specification of Removal Time (or Strain) flag.
`REAL8 RC`	Removal Time (or Strain).
`INT4 RDOpt`	Removal Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`REAL8 RD`	Removal Delta Time.
`INT IsSTNFREE`	Addition with or without Strain flag (TRUE = Strain free addition, FALSE = Addition with strain).
Return Code:	
`FE_BAD_TYPE`	The ELAR entity's type is not ELAR2
`FE_INVALID`	The ELAR2 entity did not have any list entries
`FE_NOT_EXIST`	The specified element was not selected by this ELAR2
Remarks/Usage:	
None	
Example:	
None	

PutELAR2Entry

`PutELAR2Entry`
`( nElementID, TYPE, HasAC, AC, ADOpt, AD, HasRC, RC, RDOpt, RD, IsSTNFREE )`
Description:
This method stores information to the ELAR2 list entries in an ELAR2.
Input:

Unpublished work. © 2025 Siemens

`INT4 nElementID`	The ID of element selected by this ELAR2 entry
`INT4 TYPE`	Adition/Removal criteria type (0 = Time, 1 = Inelastic Strain, 2 = Mechanical Strain).
`INT HasAC`	Specification of Addition Time flag.
`REAL8 AC`	Addition Time.
`INT4 ADOpt`	Addition Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`REAL8 AD`	Addition Delta Time
`INT HasRC`	Specification of Removal Time (or Strain) flag.
`REAL8 RC`	Removal Time (or Strain).
`INT4 RDOpt`	Removal Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`REAL8 RD`	Removal Delta Time.
`INT IsSTNFREE`	Addition with or without Strain flag (TRUE = Strain free addition, FALSE = Addition with strain).
Output:	
None	
Return Code:	
`FE_BAD_TYPE`	The ELAR entity's type is not ELARADD
`FE_FAIL`	Failed to add the specified combination to the ELARADD's combination entries
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutCombination

PutCombination **(nELARCombinationID)**	
Description:	
This method stores ELARADD combinations in a combined ELARADD entity.	
Input:	
`INT4 nELARCombinationID`	The ID of ELAR selected by this ELARADD entry
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The ELAR entity's type is not ELARADD
`FE_FAIL`	Failed to add the specified combination to the ELARADD's combination entries
Remarks/Usage:	
None	
Example:	
None	

GetELAR2List

GetELAR2List **(listcount, nElementID, TYPE, HasAC, AC, ADOpt, AD, HasRC, RC, RDOpt, RD, IsSTNFREE)**
Description:
This method is used to retrieve the option data for all ELAR2 list entries.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
`INT4 listcount`	The number of entries in the other arrays.
`INT4[0-listcount-1] nElementID`	The ID of element selected by this ELAR2 entry
`INT4[0-listcount-1] TYPE`	Adition/Removal criteria type (0 = Time, 1 = Inelastic Strain, 2 = Mechanical Strain).
`INT[0-listcount-1] HasAC`	Specification of Addition Time flag.
`REAL8[0-listcount-1] AC`	Addition Time.
`INT4[0-listcount-1] ADOpt`	Addition Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`REAL8[0-listcount-1] AD`	Addition Delta Time
`INT[0-listcount-1] HasRC`	Specification of Removal Time (or Strain) flag.
`REAL8[0-listcount-1] RC`	Removal Time (or Strain).
`INT4[0-listcount-1] RDOpt`	Removal Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`REAL8[0-listcount-1] RD`	Removal Delta Time.
`INT[0-listcount-1] IsSTNFREE`	Addition with or without Strain flag (TRUE = Strain free addition, FALSE = Addition with strain).
Return Code:	
`FE_BAD_TYPE`	The ELAR entity's type is not ELAR2
`FE_NOT_EXIST`	The ELAR2 entity did not have any list entries
`FE_FAIL`	Failed to add the specified option data to the ELAR2's list entries
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

PutELAR2List

PutELAR2List	
(listcount, nElementID, TYPE, HasAC, AC, ADOpt, AD, HasRC, RC, RDOpt, RD, IsSTNFREE)	
Description:	
This method is used to define the ELAR2 list entries and their option data for an ELAR2 entity.	
Input:	
`INT4 listcount`	The number of entries in the other arrays.
`INT4[0-listcount-1] nElementID`	The ID of element selected by this ELAR2 entry
`INT4[0-listcount-1] TYPE`	Adition/Removal criteria type (0 = Time, 1 = Inelastic Strain, 2 = Mechanical Strain).
`INT[0-listcount-1] HasAC`	Specification of Addition Time flag.
`REAL8[0-listcount-1] AC`	Addition Time.
`INT4[0-listcount-1] ADOpt`	Addition Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).
`REAL8[0-listcount-1] AD`	Addition Delta Time
`INT[0-listcount-1] HasRC`	Specification of Removal Time (or Strain) flag.
`REAL8[0-listcount-1] RC`	Removal Time (or Strain).
`INT4[0-listcount-1] RDOpt`	Removal Delta Time option (-1 = Ramp over remaining subcase time, 0 = No Delta Time, 1 = Specify Delta Time).

Unpublished work. © 2025 Siemens

`REAL8[0-listcount-1] RD`	Removal Delta Time.
`INT[0-listcount-1] IsSTNFREE`	Addition with or without Strain flag (TRUE = Strain free addition, FALSE = Addition with strain).
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The ELAR entity's type is not ELAR2
`FE_FAIL`	Failed to add the specified option data to the ELAR2's list entries
Remarks/Usage:	
None	
Example:	
None	

GetCombinationSet

`GetCombinationSet` **`( pCombinationSet )`**	
Description:	
This method is used to retrieve the ELARADD combinations as a Set in a combined ELARADD entity	
Input:	
None	None
Output:	
`OBJECT pCombinationSet`	The Set Object that will receive the IDs of the ELARADD combinations
Return Code:	
`FE_FAIL`	The set could not be loaded.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

GetCombinations

GetCombinations **(listcount, ELAREntityIDs)**	
Description:	
This method is used to retrieve the ELARADD combination data for all ELARADD list entries.	
Input:	
None	None
Output:	
`INT4 listcount`	The number of entries in the other arrays.
`INT4 ELAREntityIDs`	The ID of ELAR selected by this ELARADD entry
Return Code:	
`FE_NOT_EXIST`	The ELARADD entity did not have any combination entries
`FE_FAIL`	Failed to add the specified option data to the ELARADD's combination entries
Remarks/Usage:	
None	
Example:	
None	

PutCombinations

Unpublished work. © 2025 Siemens

PutCombinations	
(listcount, ELARADDEntityIDs)	
Description:	
This method stores ELARADD combinations in a combined ELARADD entity.	
Input:	
`listcount`	The number of entries in the other arrays.
`ELARADDEntityIDs[0-listcount-1]`	The ID of ELAR selected by this ELARADD entry
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The ELAR entity's type is not ELARADD
`FE_FAIL`	Failed to add the specified combination to the ELARADD's combination entries
Remarks/Usage:	
None	
Example:	
None	

GetComponentSet

GetComponentSet	
(pSelectedComponentSet)	
Description:	
This method loads a Set object that contains the IDs of the Elements List used for ELAR selection.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
`OBJECT pSelectedComponentSet`	Set Object containing the IDs of the entities.
Return Code:	
`FE_FAIL`	Failed to generate a set with the selected Element IDs.
Remarks/Usage:	
None	
Example:	
None	

ClearComponentSet

`ClearComponentSet`	
`( void )`	
Description:	
This method clears the array that contains the IDs of the Elements in the Element List used for ELAR selection.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Tried to clear components of an Element Add | Remove that did not have a selection list.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutComponentSet

PutComponentSet **(pSelectedComponentSet)**	
Description:	
This method loads Element IDs into the Element List for ELAR selection.	
Input:	
`OBJECT pSelectedComponentSet`	Set Object containing the IDs of the entities.
Output:	
None	None
Return Code:	
`FE_FAIL`	Failed to load Element IDs into the Element List.
Remarks/Usage:	
None	
Example:	
None	

Element Objects

Element objects correspond to the elements in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Element objects. For more information, see **Common Entity Properties and Methods**.

Use the `feElem` method of the Femap Application object to create Element objects.

Element Object Properties

Elements are always stored with a `SetID=1`, and the ID equal to the element ID.

Unpublished work. © 2025 Siemens

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The element color.
`INT4 layer`	The layer associated with the element.
`INT4 type`	Type of element: Rod=1, Bar=2, Tube=3, Link=4, Beam=(Lin=5 Para=37), Spring/Damper=6, DOF Spring=7, Curved Beam=8, Gap=9, Plot Only=10, Shear Panel (Lin=11 Para=12), Membrane (Lin=13 Para=14), Bending Only (Lin=15 Para=16), Plate (Lin=17 Para=18), Plane Strain (Lin=19 Para=20), Laminate (Lin=21 Para=22), Axisymmetric (Lin=23 Para=24), Solid (Lin=25 Para=26), Mass=27, Mass Matrix=28, Rigid=29, General Matrix=30, Curved Tube=31, Plot Only Plate=32, Slide-Line=33, Contact=34, Axisymmetric Shell (Lin=35 Para=36), Weld/Fastener=38, Solid Laminate (Lin=39, Para=40), Spring/Damper to Ground=41, DOF Spring to Ground=42, Cohesive (Lin=43, Para=44), Bearing = 46, Gear = 47, Fourier = 48
`INT4 propID`	ID of property referenced by the element. This is not required for certain property types, like plot-only and rigid elements. These elements have no properties or materials.
`INT4 topology`	The shape of the element: 0=Line2, 1=Line3, 2=Tri3, 3=Tri6, 4=Quad4, 5=Quad8, 6=Tetra4, 7=Wedge6, 8=Brick8, 9=Point, 10=Tetra10, 11=Wedge15, 12=Brick20, 13=Rigid, 14=Pyramid5, 15=MultiList, 16=Contact, 17=Weld, 18=Rigid (RBE1 elements only), 19=Pyramid13.
`INT4 orientID`	The orientation node for elements like bars and beams that require an orientation. This can be 0, in which case the orient property specifies an orientation vector.
`BOOL MaterialAngleFlag`	If True, the `MaterialAngle` property has been set using an Angle. This property has been superseded by the `MaterialOrientType` property.
`INT4 MaterialOrientType`	The material orientation for the element. 0=None, 1=Oriented using an Angle (`MaterialAngle`

Unpublished work. © 2025 Siemens

Property	Description
	property), 2=Oriented using a Coordinate System (`MaterialCSys` property)
`INT4 MaterialCSys`	The ID of the coordinate system used for material orientation if the `MaterialOrientType`=2.
`INT4 geomID`	The ID of the geometry associated to this element. This is a `curveID` for line elements, a surface ID for planar elements, and a solid/volume ID for solid elements.

`INT4 formulation[0..2]`
or
`VARIANT vformulation`
	Additional flags defining the element type.

`INT4 node[0..19]`
or
`VARIANT vnode`
	Nodes referenced by element. This array does not contain the nodes referenced by rigid or slide line elements, which store their nodes in a separate node list. Likewise, contact elements are handled separately.

`REAL8 orient[0..2]`
or
`VARIANT vorient`
	Element orientation vector for bar/beam. `[0]` contains material orientation angle for planar elements. In global rectangular coordinates.

`REAL8 offset[0..1][0..2]`
or
`VARIANT voffset`
	Offset vectors at ends of bar/beam elements. In global rectangular coordinates. `offset[0][...]` is at the first end of the element, `offset[1][...]` is at the second end.

`BOOL release[0..1][0..5]`
or
`VARIANT vrelease`
	Release flags at ends of bar/beam elements. `release[0][...]` is at the first end of the element, `release[1][...]` is at the second end. The 6 second indices correspond to the degrees of freedom. This is also used for the DOFs of the independent node of Rigid Element `using [0][...]`.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL RigidRodDependentDofLoc`	If True, the Rigid Rod element's dependent DOF is on End B of the element. If False, it is on End A of the element
`INT4 RigidRodDependentDof`	The component number of the one dependent translation DOF. Can either be 1, 2, or 3
`BOOL[0..3][0..5] vRigidBarDOFs`	Flags for the independent DOFs and dependent DOFs of End A and End B for the Rigid Bar element. Row 0 = Independent End A, Row 1 = Independent End B, Row 2 = Dependent End A, Row 3 = Dependent End B
Additional Properties	
`INT4 nodes[0..19]`	Same as the node property but accesses the entire array as a Variant.
`BOOL RigidInterpolate`	Flag specifying whether an element with type=Rigid is really an interpolation element.
`REAL8 RigidThermalExpansion`	Coefficient of thermal expansion for rigid elements.
`REAL8 RigidInterpolationFactor`	Scale factor for Rigid Interpolation elements.
`REAL8 PlateOffset`	Specifies the offset value for plate elements.
`REAL8 MaterialAngle`	Specifies the material orientation angle for planar elements if `MaterialAngleFlag`=True.

`INT4 connectTYPE[0..1]`
 	
or
 	
`VARIANT vconnectTYPE`
	`connectTYPE[0]` specifies type of weld element.
	
	`connectTYPE[1]` Weld Location by: Projection=0 Axis=1.
	
	`connectTYPE[0]` must be set to 100 for Gear element.

`INT4 connectSEG[0..1]`
 	
or
 	
`VARIANT vconnectSEG`
	Node, Element or Property ID for weld patch A and B, respectively, for individual of weld types.

`INT4 warping[0..1]`	ID of warping node on beam element: warping(0)=End A

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vwarping`	warping(1)=End B
`BOOL SpringNoOrient`	Spring/Damper element has no orientation. Node, vector and CID are ignored during translation.
`BOOL SpringUseCID`	Use SpringCID for orientation.
`INT4 SpringCID`	Coordinate system for spring/damper orientation.
`BOOL SpringUseLocation`	Use SpringLocation for location.
`REAL8 SpringLocation`	Spring/damper location, between 0.0 and 1.0.
`BOOL SpringUsePropCID`	Use spring/damper coordinate system from property.
`BOOL SpringUsePropLocation`	Use spring/damper location from property.
`INT4 vkeyopt[0..11]`	Ansys Element keyopts.

Accessing Element Nodes

For most elements, the node property will give you access to the IDs of the nodes that define the element. Before you can use this data, however, you must know where to find the nodes in the node array. The following table defines the layout for the various element types:

Topology	Array Entries (optional/midside nodes in parentheses)
Point	0
Line	0,1
Tri3	0,1,2
Tri6	0,1,2, (4,5,6)
Quad4	0,1,2,3
Quad8	0,1,2,3,(4,5,6,7)

Unpublished work. © 2025 Siemens

Topology	Array Entries (optional/midside nodes in parentheses)
Tetra4	0,1,2, 4
Pyramid5	0,1,2,3, 4
Wedge6	0,1,2, 4,5,6
Brick8	0,1,2,3,4,5,6,7
Tetra10	0,1,2, 4 (8,9,10, 12,13,14)
Pyramid13	0,1,2,3, 4 (8,9,10,11, 12,13,14,15)
Wedge15	0,1,2, 4,5,6 (8,9,10, 12,13,14, 16,17,18)
Brick20	0,1,2,3,4,5,6,7, (8,9,10,11,12,13,14,15,16,17,18,19)
RigidList	0=Independent, dependent nodes use element lists, not array entries
MultiList	Uses element lists, not array entries
Contact	References contact segments, not nodes
Weld	Weld Axis = 0,1 SegA= 4,5,6,(7,8,9,10,11) SegB=12,13,14,(15,16,17,18,19)

Element Object Methods

The following methods are available in addition to the standard entity methods.

GetAllArray

GetAllArray

```
( setID, numElem, entID, propID, elemTYPE, topology, layer, color,
formulation, orient, offset,
release, orientSET, orientID, Nodes, connectTYPE, connectSEG )
```

Description:

This method retrieves arrays of values for a set of elements.

Input:

Unpublished work. © 2025 Siemens

`INT4 setID`	ID of the set that contains the element IDs to retrieve. If 0, retrieve all elements in the model.
Output:	
`INT4 numElem`	The number of elements for which data is being returned.
`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element.
`INT4 propID[0..numElem-1]`	The property ID of the element.
`INT4 elemTYPE[0..numElem-1]`	The element type.
`INT4 topology[0..numElem-1]`	The element topology/shape.
`INT4 layer[0..numElem-1]`	The ID of the layer for the element.
`INT4 color[0..numElem-1]`	The color of the element.
`INT4 formulation[0..(2*numElem)-1]`	The element formulation. There are 2 values per element.
`REAL8 orient[0..(3*numElem)-1]`	The element orientation vector (3 values per line element) or Material Angle (1 value per element, stored in the first of the three slots per element).
`REAL8 offset[0..(6*numElem)-1]`	The element offset vectors, 6 values per element. The first 3 at end 1, the second 3 at end 2.
`INT4 release[0..(12*numElem)-1]`	Flags indicating end releases. 12 values per element. The first 6 correspond to the elemental degrees of freedom at the first end of the element. The final 6 to the second end. For spring/damper elements: release(0)=True - Use `SpringLocation` for location. release(1)=True - Use spring/damper location from property. release(2)=True - Use `SpringCID` for orientation. release(3)=True - Spring/Damper element has no orientation. Node, vector and CID are ignored during translation.

Unpublished work. © 2025 Siemens

	Use `AddArray3` to return values for spring locations (`springLocation`) and material coordinate systems (`materialcsys`).
`INT4 orientSET[0..numElem-1]`	Flag indicating whether material orientation is on for this element.
`INT4 orientID[0..numElem-1]`	The ID of the third/orientation node for the element.
`INT4 Nodes[0..(20*numElem)-1]`	The node IDs used to define the element. Stored 20 values per element. Unused nodes are 0. Element types that reference lists of nodes (like rigid, slide lines or contact), will not have the nodes from their node lists in this array - you must use other methods to retrieve those nodes.
`INT4 connectTYPE[0..(2*numElem)-1]`	There are 2 values per element. `connectTYPE[0]` specifies type of weld element(0 thru 6). `connectTYPE[1]` Weld Location by: Projection=0 Axis=1.
`INT4 connectSEG[0..(2*numElem)-1]`	There are 2 values per element. Node, Element or Property ID for weld patch A and B respectively for individual of weld types.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected elements.
Remarks/Usage:	
This method is the fastest way to retrieve information about a large number of elements. If a retrieved element ID does not exist, then all of the values for that element ID will be 0.	
Example:	

```
Sub Main
Dim App As femap.model
Set App = feFemap()

Const MAX_NODES_PER_ELEM = 20

Dim El As femap.Elem
Set El = App.feElem


Dim elSet As femap.Set
Set elSet = App.feSet
```


Unpublished work. © 2025 Siemens

```
Dim numElem As Long
Dim entID As Variant
Dim propID As Variant
Dim elemType As Variant
Dim topology As Variant
Dim Lyr As Variant
Dim color As Variant
Dim formulation As Variant
Dim orient As Variant
Dim offset As Variant
Dim release As Variant
Dim orientSET As Variant
Dim orientID As Variant
Dim nodes As Variant
Dim connectTYPE As Variant
Dim connectSEG As Variant


Dim Msg As String
```

```
Dim findElemType As Integer
findElemType = 17 'Linear Plate

rc = elSet.AddRule(findElemType, FGD_ELEM_BYTYPE)
rc = El.GetAllArray(elSet.ID, numElem, entID, propID, _
elemType, topology, Lyr, color, formulation,orient,offset, _
release, orientSET, orientID, nodes, connectTYPE, connectSEG )

App.feAppMessage(FCL_NORMAL, _
" Number of elements: " + Str$(numElem))
For i = 0 To numElem-1
  ' Load ID, property, type, and nodes into String for display
 Msg = Str$(entID(i)) + ", "+Str$(propID(i)) + ", "+ _
        Str$(elemType(i))


  For u = 0 To MAX_NODES_PER_ELEM-1
   If nodes(i*MAX_NODES_PER_ELEM + u) <> 0 Then
     Msg = Msg + ", "+ Str$(nodes(i*MAX_NODES_PER_ELEM + u))
   End If
 Next u
 App.feAppMessage(FCM_NORMAL, Msg)
Next i



End Sub
```

GetAllArray2

Unpublished work. © 2025 Siemens

GetAllArray2

**(setID, numElem, entID, exist, propID, elemTYPE, topology, layer, color, formulation,
orient, offset, release, orientSET, orientID, Nodes, connectTYPE, connectSEG)**

Description:

This method retrieves arrays of values for a set of elements, including if an element exists in database.

Input:

`INT4 setID`	ID of the set that contains the element IDs to retrieve. If 0, retrieve all elements in the model.

Output:

`INT4 numElem`	The number of elements for which data is being returned.
`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element.
`BOOL exist[0..numElem-1]`	Element existence, True=Element exists, False=Element does not exist.
`INT4 propID[0..numElem-1]`	The property ID of the element.
`INT4 elemTYPE[0..numElem-1]`	The element type.
`INT4 topology[0..numElem-1]`	The element topology/shape.
`INT4 layer[0..numElem-1]`	The ID of the layer for the element.
`INT4 color[0..numElem-1]`	The color of the element.
`INT4 formulation[0..(2*numElem)-1]`	The element formulation. There are 2 values per element.
`REAL8 orient[0..(3*numElem)-1]`	The element orientation vector (3 values per line element) or Material Angle (1 value per element, stored in the first of the three slots per element).
`REAL8 offset[0..(6*numElem)-1]`	The element offset vectors, 6 values per element. The first 3 at end 1, the second 3 at end 2.
`INT4 release[0..(12*numElem)-1]`	Flags indicating end releases. 12 values per element. The first 6 correspond to the elemental

Unpublished work. © 2025 Siemens

	degrees of freedom at the first end of the element. The final 6 to the second end. For spring/damper elements: release(0)=True - Use `SpringLocation` for location. release(1)=True - Use spring/damper location from property. release(2)=True - Use `SpringCID` for orientation. release(3)=True - Spring/Damper element has no orientation. Node, vector and CID are ignored during translation. Use `AddArray3` to return values for spring locations (`springLocation`) and material coordinate systems (`materialcsys`).
`INT4 orientSET[0..numElem-1]`	Flag indicating whether material orientation is on for this element.
`INT4 orientID[0..numElem-1]`	The ID of the third/orientation node for the element.
`INT4 Nodes[0..(20*numElem)-1]`	The node IDs used to define the element. Stored 20 values per element. Unused nodes are 0. Element types that reference lists of nodes (like rigid, slide lines or contact), will not have the nodes from their node lists in this array - you must use other methods to retrieve those nodes.
`INT4 connectTYPE[0..(2*numElem)-1]`	There are 2 values per element. `connectTYPE[0]` specifies type of weld element (0 thru 6). `connectTYPE[1]` Weld Location by: Projection=0 Axis=1.
`INT4 connectSEG[0..(2*numElem)-1]`	There are 2 values per element. Node, Element or Property ID for weld patch A and B respectively for individual of weld types.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected elements.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method is the fastest way to retrieve information about a large number of elements. If a retrieved element ID does not exist, then all of the values for that element ID will be 0. If it is known that all of the elements in the element set exist, use `GetAllArray`.
Example:
None

GetAllArray3

`GetAllArray3` **`( setID, numElem, entID, exist, propID, elemTYPE, topology, layer, color, formulation, orient, offset, release, orientSET, orientID, Nodes, connectTYPE, connectSEG, materialOrientType, materialCSys, warpingID, SpringLocation )`**	
Description:	
This method retrieves arrays of values for a set of elements, including if an element exists in database.	
Input:	
`INT4 setID`	ID of the set that contains the element IDs to retrieve. If 0, retrieve all elements in the model.
Output:	
`INT4 numElem`	The number of elements for which data is being returned.
`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element.
`BOOL exist[0..numElem-1]`	Element existence, True=Element exists, False=Element does not exist.
`INT4 propID[0..numElem-1]`	The property ID of the element.
`INT4 elemTYPE[0..numElem-1]`	The element type.
`INT4 topology[0..numElem-1]`	The element topology/shape.
`INT4 layer[0..numElem-1]`	The ID of the layer for the element.

Unpublished work. © 2025 Siemens

`INT4 color[0..numElem-1]`	The color of the element.
`INT4 formulation[0..(2*numElem)-1]`	The element formulation. There are 2 values per element.
`REAL8 orient[0..(3*numElem)-1]`	The element orientation vector (3 values per line element) or Material Angle (1 value per element, stored in the first of the three slots per element).
`REAL8 offset[0..(6*numElem)-1]`	The element offset vectors, 6 values per element. The first 3 at end 1, the second 3 at end 2.
`INT4 release[0..(12*numElem)-1]`	Flags indicating end releases. 12 values per element. The first 6 correspond to the elemental degrees of freedom at the first end of the element. The final 6 to the second end. For spring/damper elements: release(0)=True - Use SpringLocation for location. release(1)=True - Use spring/damper location from property. release(2)=True - Use `SpringCID` for orientation. release(3)=True - Spring/Damper element has no orientation. Node, vector and CID are ignored during translation. Use `SpringLocation` array for spring locations and `materialCsys` array for spring/damper coordinate system IDs.
`INT4 orientSET[0..numElem-1]`	Flag indicating whether material orientation is on for this element.
`INT4 orientID[0..numElem-1]`	The ID of the third/orientation node for the element.
`INT4 Nodes[0..(20*numElem)-1]`	The node IDs used to define the element. Stored 20 values per element. Unused nodes are 0. Element types that reference lists of nodes (like rigid, slide lines or contact), will not have the nodes from their node lists in this array - you must use other methods to retrieve those nodes.
`INT4 connectTYPE[0..(2*numElem)-1]`	There are 2 values per element.

Unpublished work. © 2025 Siemens

	`connectTYPE[0]` specifies type of weld element(0 thru 6). `connectTYPE[1]` Weld Location by: Projection=0 Axis=1.
`INT4 connectSEG[0..(2*numElem)-1]`	There are 2 values per element. Node, Element or Property ID for weld patch A and B respectively for individual of weld types.
`INT4 materialOrientType [0..numElem-1]`	The material orientation for the element. 0=None, 1=Oriented using an Angle (`MaterialAngle` property), 2=Oriented using a Coordinate System (`MaterialCSys` property).
`INT4 materialCSys [0..numElem-1]`	The ID of the coordinate system used for material orientation if the `MaterialOrientType`=2.
`INT4 warpingID [0..numElem-1]`	The ID of the node associated to beam elements for warping.
`REAL8 SpringLocation[0..numElem-1]`	Spring/damper location, between 0.0 and 1.0.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected elements.
Remarks/Usage:	
This method is the fastest way to retrieve information about a large number of elements. If a retrieved element ID does not exist, then all of the values for that element ID will be 0. If it is known that all of the elements in the element set exist, use `GetAllArray`.	
Example:	
None	

PutAllArray

`PutAllArray`
`( numElem, entID, propID, elemTYPE, topology, layer, color, formulation, orient, offset, release, orientSET, orientID, Nodes, connectTYPE, connectSEG )`
Description:

Unpublished work. © 2025 Siemens

This method stores/creates multiple elements based on the values passed in various arrays.	
Input:	
`INT4 numElem`	The number of elements to create - the number of entries in the arrays.
`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element to be created.
`INT4 propID[0..numElem-1]`	The property ID of the element.
`INT4 elemTYPE[0..numElem-1]`	The element type.
`INT4 topology[0..numElem-1]`	The element topology/shape.
`INT4 layer[0..numElem-1]`	The ID of the layer for the element.
`INT4 color[0..numElem-1]`	The color of the element.
`INT4 formulation[0..(2*numElem)-1]`	The element formulation. There are 2 values per element.
`REAL8 orient[0..(3*numElem)-1]`	The element orientation vector, 3 values per element.
`REAL8 offset[0..(6*numElem)-1]`	The element offset vectors, 6 values per element. The first 3 at end 1, the second 3 at end 2.
`INT4 release[0..(12*numElem)-1]`	Flags indicating end releases. 12 values per element. The first 6 correspond to the elemental degrees of freedom at the first end of the element. The final 6 to the second end.
`INT4 orientSET[0..numElem-1]`	Flag indicating whether material orientation is on for this element.
`INT4 orientID[0..numElem-1]`	The ID of the third/orientation node for the element.
`INT4 Nodes[0..(20*numElem)-1]`	The node IDs used to define the element. Stored 20 values per element. Unused nodes are 0. Element types that reference lists of nodes (like rigid, slide lines or contact), can not be created using this method - you must use `Put` to create those element types.
`INT4 connectTYPE[0..(2*numElem)-1]`	There are 2 values per element.

Unpublished work. © 2025 Siemens

	`connectTYPE[0]` specifies type of weld element(0 thru 6). `connectTYPE[1]` Weld Location by: Projection=0 Axis=1.
`INT4 connectSEG[0..(2*numElem)-1]`	There are 2 values per element. Node, Element or Property ID for weld patch A and B respectively for individual of weld types.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to create the specified elements.
Remarks/Usage:	
This method is the fastest way to create a large number of elements.	
Example:	
None	

GetGeomPropArray

`GetGeomPropArray`
`(setID, numElem, entID, propID, matlID, elemTYPE, topology, vCG, length, area, volume)`
Description:
This method returns various geometric properties for a selected set of elements.
Input:

`INT4 setID`	ID of the set that contains the element IDs to retrieve. If 0, retrieve all elements in the model.
Output:	
`INT4 numElem`	The number of elements for which data is being returned.
`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element.

Unpublished work. © 2025 Siemens

`INT4 propID[0..numElem-1]`	The property ID of the element.
`INT4 matlID[0..numElem-1]`	The material ID of the element.
`INT4 elemTYPE[0..numElem-1]`	The element type.
`INT4 topology[0..numElem-1]`	The element topology/shape.
`REAL8 vCG[0..(3*numElem)-1]`	The coordinates of the centroid of the element. Always reported in Global Rectangular coordinates.
`REAL8 length[0..numElem-1]`	The length of each element. Only nonzero for line elements.
`REAL8 area[0..numElem-1]`	The area of each element.
`REAL8 volume[0..numElem-1]`	The volume of each element.
Return Code:	
`FE_NOT_EXIST`	No Geometric properties found for selected elements.
`FE_NO_MEMORY`	Unable to allocate sufficient memory to return properties.
`FE_INVALID`	No Elements selected.
Remarks/Usage:	
This method is the fastest way to retrieve information about a large number of elements. If a retrieved element ID does not exist, then all of the values for that element ID will be 0.	
Example:	
None	

GetNodeList

`GetNodeList`
`(  listINDEX, nodecount, node, face, weight, dof )`
Description:

Unpublished work. © 2025 Siemens

This method retrieves information from the node lists on rigid and slide line elements.	
Input:	
`INT4 listINDEX`	The ID of the list to retrieve. For rigid elements, only list 0 is available. For slide lines, lists 0 and 1 are available.
Output:	
`INT4 nodecount`	The number of entries in the other arrays (except dof, which has 6*nodecount entries).
`INT4 node[0..(N-1)]`	The IDs of the nodes in the list.
`INT4 face[0..(N-1)]`	The face numbers of any elements (not used).
`REAL8 weight[0..(N-1)]`	The weighting factors associated with the nodes for interpolation elements.
`INT4 dof[0..(6N - 1)]`	The degree of freedom flags associated with the nodes. This array has six sequential entries for each node. For example, node[0] has entries dof[0]..dof[5] as its degree of freedom flags. node[1] has dof[6]..dof[11], and so on.
Return Code:	
`FE_NOT_EXIST`	There are no entries in the list.
`FE_NOT_AVAILABLE`	You have requested an invalid list, check `listINDEX`.
Remarks/Usage:	
This method simply retrieves the information about the node lists from the current object. You must first load that data - either using the `Get( )` method, or by loading it yourself with `PutNodeList( )`.	
Example:	
None	

PutNodeList

Unpublished work. © 2025 Siemens

PutNodeList

(listINDEX, nodecount, node, face, weight, dof)

Description:

This method loads information into the node lists on rigid and slide line elements.

Input:

`INT4 listINDEX`	The ID of the list to retrieve. For rigid elements, only list 0 is available. For slide lines, lists 0 and 1 are available.
`INT4 nodecount`	The number of entries in the other arrays (except dof, which has 6*nodecount entries).
`INT4 node[0..(N-1)]`	The IDs of the nodes in the list.
`INT4 face[0..(N-1)]`	The face numbers of any elements (not used - must be NULL).
`REAL8 weight[0..(N-1)]`	The weighting factors associated with the nodes for interpolation elements.
`INT4 dof[0..(6N - 1)]`	The degree of freedom flags associated with the nodes. This array has six sequential entries for each node. For example, node[0] has entries dof[0]..dof[5] as its degree of freedom flags. node[1] has dof[6]..dof[11], and so on.

Output:

None	None

Return Code:

`FE_NOT_AVAILABLE`	You have requested an invalid list, check `listINDEX`.

Remarks/Usage:

This method stores information into the lists of the current object. Any previous list information in that object is lost. This method does not store that information to the model database. After using this function, you must call `Put( )` if you want to store the updated lists in the database.

Example:

None

Unpublished work. © 2025 Siemens

ClearNodeList

`ClearNodeList`

`( listINDEX )`

Description:

This method clears node lists on rigid and slide line elements.

Input:

`INT4 listINDEX`	The ID of the list to retrieve. For rigid elements, only list 0 is available. For slide lines, lists 0 and 1 are available. You may specify -1 to clear all lists in one call.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

If there are node lists in the object, either because you added them with `PutNodeList()`, or because you called `Get()` on an element that had lists, this method will simply erase the selected list. You can use this if you want to clear the lists because you want to reuse the object for a different element type.

Example:

None

GetSpringOrient

`GetSpringOrient`

`( enOrient, nOrient, dVec1, dVec2, dVec3 )`

Description:

Returns the orientation for the spring/damper element.

Input:

Unpublished work. © 2025 Siemens

None	None
Output:	
`INT4 enOrient`	Spring/damper orientation. 0=None (FESO_NONE), 1=Node (FESO_NODE), 2=Vector (FESO_VECTOR), 3=Coordinate System on Element (FESO_ELCID), 4=Coordinate System on Property (FESO_PRCID).
`INT4 nOrient`	Orientation entity for FESO_NODE (node ID), FESO_ELCID (CSys ID), FESO_PRCID (CSys ID from property)
`REAL8 dVec1`	Vector X orientation in global coordinate system (FESO_VECTOR).
`REAL8 dVec2`	Vector Y orientation in global coordinate system (FESO_VECTOR).
`REAL8 dVec3`	Vector Z orientation in global coordinate system (FESO_VECTOR).
Return Code:	
`FE_FAIL`	Element is not a spring/damper.
`FE_INVALID`	Property does not have coordinate system flag activated.
Remarks/Usage:	
None	
Example:	
None	

SetSpringOrient

`SetSpringOrient`
`( enOrient, nOrient, dVec1, dVec2, dVec3 )`
Description:
Set orientation for spring/damper element.
Input:

Unpublished work. © 2025 Siemens

`INT4 enOrient`	Spring/damper orientation. 0=None (`FESO_NONE`), 1=Node (`FESO_NODE`), 2=Vector (`FESO_VECTOR`), 3=Coordinate System on Element (`FESO_ELCID`), 4=Coordinate System on Property (`FESO_PRCID`).
`INT4 nOrient`	Orientation entity for FESO_NODE (node ID), FESO_ELCID (CSys ID), FESO_PRCID (CSys ID from property)
`REAL8 dVec1`	Vector X orientation in global coordinate system (`FESO_VECTOR`).
`REAL8 dVec2`	Vector Y orientation in global coordinate system (`FESO_VECTOR`).
`REAL8 dVec3`	Vector Z orientation in global coordinate system (`FESO_VECTOR`).
Output:	
None	None
Return Code:	
`FE_FAIL`	Element is not a spring/damper or invalid orientation method specified.
`FE_INVALID`	`FESO_PRCID` specified and property doesn't have CSys activated.
`FE_NOT_EXIST`	Node doesn't exist (`FESO_NODE`) or CSys doesn't exist (`FESO_ELCID`),
Remarks/Usage:	
Use of this method is recommended over setting individual spring orientation properties on the element object as it ensures all the correct flags will be set for the selected orientation	
Example:	
None	

GetSpringOffset

`GetSpringOffset`
`(  enOffset, dVec1, dVec2, dVec3 )`
Description:

Unpublished work. © 2025 Siemens

Returns the offset for spring/damper element.	
Input:	
None	None
Output:	
`INT4 enOffset`	Spring/damper orientation. 0=Default (FESL_DEFAULT), 1=Location (FESL_ELLOC), 2=From Property (FESL_PRLOC), 3=Vector (FESL_OFFSET).
`REAL8 dVec1`	Offset vector X component in global rectangular (FESL_OFFSET). Location for FESL_ELLOC or FESL_PRLOC.
`REAL8 dVec2`	Offset vector Y component in global rectangular (FESL_OFFSET).
`REAL8 dVec3`	Offset vector Z component in global rectangular (FESL_OFFSET).
Return Code:	
`FE_FAIL`	Element is not a spring/damper.
`FE_INVALID`	Property does not have coordinate system flag activated.
Remarks/Usage:	
None	
Example:	
None	

SetSpringOffset

`SetSpringOffset`
`( enOffset, dVec1, dVec2, dVec3 )`
Description:
Set offset for spring/damper element.

Unpublished work. © 2025 Siemens

Input:	
`INT4 enOffset`	Spring/damper orientation. 0=Default (FESL_DEFAULT), 1=Location (FESL_ELLOC), 2=From Property (FESL_PRLOC), 3=Vector (FESL_OFFSET).
`REAL8 dVec1`	Offset vector X component in global rectangular (FESL_OFFSET). Location for FESL_ELLOC or FESL_PRLOC.
`REAL8 dVec2`	Offset vector Y component in global rectangular (FESL_OFFSET).
`REAL8 dVec3`	Offset vector Z component in global rectangular (FESL_OFFSET).
Output:	
None	None
Return Code:	
`FE_FAIL`	Element is not a spring/damper or invalid offset method specified.
`FE_INVALID`	Property does not have location enabled when FESL_PRLOC is specified.
Remarks/Usage:	
Use of this method is recommended over setting individual spring/damper offset properties on the element object as it ensures all the correct flags will be set for the selected offset.	
Example:	
None	

Prop

`Prop`
`( void )`
Description:
This method creates and returns a Property object that is loaded with the property data referenced by an element.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	

This method must be used with a Set statement to define the Property object.

```
Sub Main
Dim App As femap.model
Set App = feFemap()

Dim pr As femap.Prop
Dim el As femap.Elem


Set el = App.feElem

el.Get(83)
Set pr = el.Prop

App.feAppMessage(FCM_NORMAL, "Property: " + Str$(pr.ID)+ _
    " "+pr.title)


End Sub
```

This sequence creates the `pr` Property object with the property data that is referenced by element 83.

Matl

`Matl`

`( void )`

Description:

Unpublished work. © 2025 Siemens

This method creates and returns a Material object that is loaded with the material data referenced by an element.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	

This method must be used with a Set statement to define the Material object.

```
Sub Main()
Dim App As femap.model
  Set App = feFemap()

Dim el As femap.Elem
Dim mt As femap.Matl
Set el = App.feElem

el.Get(83)
Set mt = el.Matl

App.feAppMessage(FCM_NORMAL, "Material: " + Str$(mt.ID)+ _
       " "+mt.title)


End Sub
```

This sequence creates the `mt` Material object with the property data that is referenced by element 83.

GetCentroid

Unpublished work. © 2025 Siemens

GetCentroid	
(cgXYZ)	
Description:	
This method returns the centroid of the current element.	
Input:	
None	None
Output:	
`REAL8 cgXYZ[0..2]`	The coordinates of the centroid of the element. Always reported in Global Rectangular coordinates.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetCentroidArray

GetCentroidArray	
(setID, numElem, entID, vCG)	
Description:	
This method retrieves arrays of centroid values for a set of elements.	
Input:	
`INT4 setID`	ID of the set that contains the element IDs to retrieve. If 0, retrieve all elements in the model.
Output:	
`INT4 numElem`	The number of elements for which data is being returned.

Unpublished work. © 2025 Siemens

`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element.
`REAL8 vCG[0..(3*numElem)-1]`	The coordinates of the centroid of the element. Always reported in Global Rectangular coordinates.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected elements.
Remarks/Usage:	
This method is the fastest way to retrieve centroid information about a large number of elements. If a retrieved element ID does not exist, then all of the values for that element ID will be 0.	
Example:	
None	

GetFaceCentroid

`GetFaceCentroid`	
`( nFace, cgXYZ )`	
Description:	
This method returns the centroid of a selected element face.	
Input:	
`INT4 nFace`	The identifier of the face that you want to select.
Output:	
`REAL8 cgXYZ[0..2]`	The coordinates of the centroid of the element face. Always reported in Global Rectangular coordinates.
Return Code:	
None	None
Remarks/Usage:	
The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid.	

Unpublished work. © 2025 Siemens

Example:
None

GetFaceNormal

GetFaceNormal **(nFace, normalXYZ)**	
Description:	
This method returns the normal vector of a selected element face.	
Input:	
`INT4 nFace`	The identifier of the face that you want to select.
Output:	
`REAL8 normalXYZ[0..2]`	The vector components of the normal of the element face. Always reported in Global Rectangular coordinates. These vector components are always normalized.
Return Code:	
None	None
Remarks/Usage:	
The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid. For nonplanar faces, an approximate normal is computed as an average normal of the normals at the face corners.	
Example:	
None	

GetFaceArea

Unpublished work. © 2025 Siemens

GetFaceArea	
(nFace, dArea)	
Description:	
This method returns the area of a selected element face.	
Input:	
`INT4 nFace`	The identifier of the face that you want to select.
Output:	
`REAL8 dArea`	The area of the element face.
Return Code:	
None	None
Remarks/Usage:	
The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid. If the element has thickness, its average thickness is used along with the edge length to compute edge area. For axisymmetric elements, a thickness of 1.0 is used.	
Example:	
None	

GetSingleFaceInfoArray

GetSingleFaceInfoArray	
(setID, nFace, numElem, entID, cgXYZ, normalXYZ, dArea)	
Description:	
This method returns arrays of centroid, normal, and area values of an identical element face for a set of elements.	
Input:	
`INT4 setID`	ID of the set that contains the element IDs to retrieve. If 0, retrieve all elements in the model.

Unpublished work. © 2025 Siemens

`INT4 nFace`	The identifier of the face that you want to select. Information on this face is returned for all elements.
Output:	
`INT4 numElem`	The number of element faces for which data is being returned.
`INT4 entID[0..numElem-1]`	Each entry is the ID of the associated element.
`REAL8 cgXYZ[0..(3*numElem)-1]`	The coordinates of the centroid of the element faces, 3 values per element. Always reported in Global Rectangular coordinates.
`REAL8 normalXYZ[0..(3*numElem)-1]`	The vector components of the normal of the element faces, 3 values per element. Always reported in Global Rectangular coordinates. These vector components are always normalized.
`REAL8 dArea[0..numElem-1]`	The areas of the element faces.
Return Code:	
`FE_OK`	Success.
`FE_BAD_DATA`	Unable to retrieve face information. Ensure nFace is a valid face ID.
`FE_INVALID`	Element set was empty or there are no elements in the model.
Remarks/Usage:	
The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid. For nonplanar faces, an approximate normal is computed as an average normal of the normals at the face corners.	
Example:	

```
Sub Main

   Dim App as femap.model

   Set App = feFemap()


   Dim e As femap.Elem
```


Unpublished work. © 2025 Siemens

```
   Set e = App.feElem


   Dim eSet As femap.Set
   Set eSet = App.feSet


   rc = eSet.Select( FT_ELEM, True, "Select Element(s)" )
   If rc <> FE_OK Then End


   Dim nFace, numElem As Long
   Dim vID, vCGXYZ, vNormalXYZ, vArea As Variant


   nFace = 1


   rc = e.GetSingleFaceInfoArray(eSet.ID, nFace, numElem, vID, vCGXYZ,
vNormalXYZ, vArea )
   If rc <> FE_OK Then End


   msg = "ID, Centroid X,Y,Z"
   App.feAppMessage(FCM_WARNING, msg )


   For i = 0 To numElem-1


      eID = vID( i )


      x = vCGXYZ(3*i )
      y = vCGXYZ(3*i +1)
      z = vCGXYZ(3*i +2)
```


Unpublished work. © 2025 Siemens

```
msg = Format(eID)+",  "+vbTab+Format(x)+",  "+vbTab+Format(y)+",
"+vbTab+Format(z)

      App.feAppMessage(FCM_NORMAL, msg )


   Next


End Sub
```

GetMultiFaceInfoArray

GetMultiFaceInfoArray

(nNumFaces, nElemID, nFaceIDs, faceOK, cgXYZ, normalXYZ,dArea)

Description:

This method returns arrays of centroid, normal, and area values on different faces specified in an array for a set of elements.

Input:

`INT4 nNumFaces`	The number of face entries in the `nFaceIDs` array.
`INT4 nElemIDs`	Each entry is the ID of the associated element.
`INT4 nFaceIDs`	Each entry is the ID of the face to query.

Output:

`BOOL faceOK[0..nNumFaces-1]`	Returns an array of True or False values if valid element ID and face ID are provided as input.
`REAL8 cgXYZ[0..(3*numElem)-1]`	The coordinates of the centroid of the element faces, 3 values per element. Always reported in Global Rectangular coordinates.
`REAL8 normalXYZ[0..(3*numElem)-1]`	The vector components of the normal of the element faces, 3 values per element. Always reported in Global Rectangular coordinates. These vector components are always normalized.

Unpublished work. © 2025 Siemens

`REAL8 dArea[0..numElem-1]`	The areas of the element faces.
Return Code:	
`FE_OK`	Success.
`FE_BAD_DATA`	Unable to retrieve face information. Ensure valid element IDs and valid Face IDs are populated in the input arrays.
`FE_INVALID`	nNumFaces must be greater than zero.
Remarks/Usage:	
The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid. For nonplanar faces, an approximate normal is computed as an average normal of the normals at the face corners.	
Example:	

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim e As femap.Elem
   Set e = App.feElem

   Dim nNumFaces As Long
   Dim vID, vFaceID As Variant
   Dim vFaceOk, vCGXYZ, vNormalxyz, vArea As Variant

   rc = App.feGetElementFaces("Select Element Face(s)", 0, 0, 0, _
                              nNumFaces&, vID, vFaceID )

   rc = e.GetMultiFaceInfoArray(  nNumFaces, vID, vFaceID, _
```


Unpublished work. © 2025 Siemens

```
                                vFaceOk, vCGXYZ,  vNormalxyz, vArea )

   msg = "[ID], [Face], ( Centroid X,Y,Z )..."
   rc = App.feAppMessage(FCM_WARNING, msg )

   For i = 0 To nNumFaces-1

      eID = vID( i )
      fID = vFaceID( i )

      x = vCGXYZ(3*i )
      y = vCGXYZ(3*i +1)
      z = vCGXYZ(3*i +2)

      msg = "["+Format(eID)+"],"+" ["+Format(fID)+"], "
      msg += "("+Format(x)+",   "+Format(y)+",   "+Format(z)+")"
      rc = App.feAppMessage(FCM_NORMAL, msg )

   Next

End Sub
```

GetFaceNodes

Unpublished work. © 2025 Siemens

GetFaceNodes	
(nFace, nodes)	
Description:	
This method returns the nodes which define the selected element face.	
Input:	
`INT4 nFace`	The identifier of the face that you want to select.
Output:	
`INT4 nodes[0..7]`	The nodes used to define the selected face. Corner nodes always come first, followed by midside nodes. Most faces use less than 8 nodes, and the final node IDs will all be 0.
Return Code:	
None	None
Remarks/Usage:	
This method only desinged for Planar (Tria3, Tria6, Quad4 and Quad8) and Solid (Tetra4, Tetra10, Pyramid5, Pyramid13, Wedge6, Wedge15, Brick8 and Brick20) shaped elements. The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid. Node order for Face 1 on planar elements follows the Right Hand Rule, while node order for Face 2 is opposite. For solid elements, node order always follows the Right hand Rule around an inward facing normal.	
Example:	
None	

GetFaceNodes2

Unpublished work. © 2025 Siemens

GetFaceNodes2

(bCornerPressureOrder, nFace, nodes)

Description:

Similar to `GetFaceNodes`, this method returns the nodes which define the selected element face, which can optionally be returned in the order used to apply pressure loads on element corners.

Input:

`BOOL bCornerPressureOrder`	If False, nodes are returned in the same order as `GetFaceNodes`, and is described in the Remarks/Usage section. If True, Planar element node order follows the right-hand rule around the element normal for both Face 1 and Face 2. For Solid Elements, node order always follows the right-hand rule around an outward facing normal.
`INT4 nFace`	The identifier of the face that you want to select.

Output:

`INT4 nodes[0..7]`	The nodes used to define the selected face. Corner nodes always come first, followed by midside nodes. Most faces use less than 8 nodes, and the final node IDs will all be 0.

Return Code:

None	None

Remarks/Usage:

This method is only designed for Planar (Tria3, Tria6, Quad4 and Quad8) and Solid (Tetra4, Tetra10, Pyramid5, Pyramid13, Wedge6, Wedge15, Brick8 and Brick20) shaped elements.

The face identifiers are the same as the face numbers used when applying elemental loads. These are shown in the Element Reference in the Femap User Guide. Only planar or solid elements can be used, but the edges of planar elements are valid.

Node order for Face 1 on planar elements follows the Right Hand Rule, while node order for Face 2 is opposite. For solid elements, node order always follows the Right hand Rule around an inward facing normal.

Example:

None

Unpublished work. © 2025 Siemens

GetEdgeNodes

GetEdgeNodes **(nEdge, n1, n2, nmid)**	
Description:	
This method returns the nodes on the selected edge of the element.	
Input:	
`INT4 nEdge`	The identifier of the edge that you want to select.
Output:	
`INT4 n1`	The first node used to define the edge.
`INT4 n2`	The last node used to define the edge.
`INT4 nmid`	The midside node on the edge (0 if edge has no midside node).
Return Code:	
None	None
Remarks/Usage:	
This method only works for Line (Line2 and Line3), Planar (Tria3, Tria6, Quad4 and Quad8) and Solid (Tetra4, Tetra10, Pyramid5, Pyramid13, Wedge6, Wedge15, Brick8 and Brick20) shaped elements. For Line elements, only edge 1 is available. For Planar elements, edge 1 connects the first and second nodes on the element, and proceed in order around the element connectivity. For Solid elements, the first edges are around the bottom face, followed by the edges connecting the bottom and top faces, and finally the top face edges.	
Example:	
None	

IsParabolic

Unpublished work. © 2025 Siemens

IsParabolic	
(void)	
Description:	
This method indicates if the loaded element is parabolic or not.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Element is Parabolic.
`FE_FAIL`	Element is not Parabolic.
Remarks/Usage:	
None	
Example:	
None	

Thickness

Thickness	
(dAvg, dAtC1, dAtC2, dAtC3, dAtC4)	
Description:	
This method returns the thickness of the element.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`REAL8 dAvg`	The average thickness of the element.
`REAL8 dAtC1`	The element thickness at corner 1.
`REAL8 dAtC2`	The element thickness at corner 2.
`REAL8 dAtC3`	The element thickness at corner 3.
`REAL8 dAtC4`	The element thickness at corner 4.
Return Code:	
`FE_OK`	
`FE_BAD_TYPE`	Element type does not have thickness.
`FE_NOT_AVAILABLE`	Element does not reference a property.
Remarks/Usage:	
None	
Example:	
None	

Length

`Length` **`( dLength )`**	
Description:	
This method returns the length of a Line Element.	
Input:	
None	None
Output:	
`REAL8 dLength`	The element Length.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_BAD_TYPE`	Element is not correct type.
Remarks/Usage:	
None	
Example:	
None	

Area

`Area` **`( dAreaA, dAreaB )`**	
Description:	
This method returns the area at EndA and EndB of Line Element.	
Input:	
None	None
Output:	
`REAL8 dAreaA`	The element area at EndA.
`REAL8 dAreaB`	The element area at EndB.
Return Code:	
`FE_OK`	
`FE_BAD_TYPE`	Element is not correct type.
`FE_NOT_AVAILABLE`	Element does not reference a property.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Inertia

Inertia **(dI1A, dI2A, dI12A, dJA, dI1B, dI2B, dI12B, dJB)**	
Description:	
This method returns line element properties at EndA and EndB.	
Input:	
None	None
Output:	
`REAL8 dI1A`	Area Moment of Inertia for plane 1 EndA.
`REAL8 dI2A`	Area Moment of Inertia for plane 2 EndA.
`REAL8 dI12A`	Cross product of inertia at EndA.
`REAL8 dIJA`	Torsional constant EndA.
`REAL8 dI1B`	Area Moment of Inertia for plane 1 EndB.
`REAL8 dI2B`	Area Moment of Inertia for plane 2 EndB.
`REAL8 dI12B`	Cross product of inertia at EndB.
`REAL8 dIJB`	Torsional constant EndB.
Return Code:	
`FE_OK`	
`FE_BAD_TYPE`	Element is not a line type.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	Element does not reference a property.
Remarks/Usage:	
None	
Example:	
None	

GetMinMaxEdgeLength

`GetMinMaxEdgeLength` **`( dMinLen, dMaxLen )`**	
Description:	
Finds the minimum and maximum length of edges in the element.	
Input:	
None	None
Output:	
`REAL8 dMinLen`	The minimum edge length.
`REAL8 dMaxLen`	The maximum edge length.
Return Code:	
`FE_OK`	
`FE_INVALID`	Element is not a supported type - only line, planar and solid elements are supported.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

ElementHasThickness

`ElementHasThickness`

`( void )`

Description:

This method indicates if the loaded element has corner thickness values defined on the element.

Input:

None	None

Output:

None	None

Return Code:

`FE_OK`	Element has corner thickness values defined on the element.
`FE_FAIL`	Element does not have corner thickness values defined on the element.

Remarks/Usage:

It may be beneficial to use this call before using the Thickness method to determine if the corner thickness values being returned are coming from the element or the property referenced by the element.

Example:

None

SetPlateThickness

`SetPlateThickness`

`( dAtC1, dAtC2, dAtC3, dAtC4 )`

Description:

Sets the element corner thickness on Plate elements.

Input:

Unpublished work. © 2025 Siemens

`REAL8 dAtC1`	The element thickness at corner 1.
`REAL8 dAtC2`	The element thickness at corner 2.
`REAL8 dAtC3`	The element thickness at corner 3.
`REAL8 dAtC4`	The element thickness at corner 4.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Element is not a supported type - only plate and plane strain elements are supported.
Remarks/Usage:	
None	
Example:	
None	

GetAnsysFormulationArray

`GetAnsysFormulationArray`
`( setID, numElem, entID, exist, formulation )`
Description:
Gets the ANSYS formulations of a given set of elements.
Input:

`INT4 setID`	The ID of the set object containing the elements.
Output:	
`INT4 numElem`	The number of elements returned.
`INT4 entID[0..numElem-1]`	An array of the IDs of the elements returned.

Unpublished work. © 2025 Siemens

`INT4 exist[0..numElem-1]`	An array of the elements that indicated if the formulation exists.
`INT4 formulation[0..numElem-1]`	An array of the element formulation values.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutAnsysFormulationArray

`PutAnsysFormulationArray` **`( numElem, entID, exist, formulation )`**	
Description:	
Sets the ANSYS formulations for a given array of elements.	
Input:	
`INT4 numElem`	The number of elements to put.
`INT4 entID[0..numElem-1]`	An array with the element IDs to put.
`INT4 formulation[0..numElem-1]`	An array with the ANSYS formulations to put.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetAnsysKeyoptArray

GetAnsysKeyoptArray **(numElem, entID, exist, keyopt)**	
Description:	
Gets the ANSYS keyopts for a given set of elements.	
Input:	
`INT4 setID`	The ID of the set object containing the elements.
Output:	
`INT4 numElem`	The number of elements returned.
`INT4 entID[0..numElem-1]`	An array of the IDs of the elements returned.
`INT4 exist[0..numElem-1]`	An array of the elements that indicated if the formulation exists.
`INT4 keyopt[0..12*numElem-1]`	An array of the element formulation values.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutAnsysKeyoptArray

Unpublished work. © 2025 Siemens

PutAnsysKeyoptArray	
(numElem, entID, keyopt)	
Description:	
Sets the ANSYS keyopts for a given array of elements.	
Input:	
`INT4 numElem`	The number of elements to put.
`INT4 entID[0..numElem-1]`	An array with the element IDs to put.
`INT4 keyopt[0..12*numElem-1]`	An array with the ANSYS keyopts to put.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

HasProp

HasProp	
( )	
Description:	
Returns if the element has an associated property.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_OK`	Element has a property.
`FE_FAIL`	Element does not have a property.
Remarks/Usage:	
None	
Example:	
None	

HasMatl

`HasMatl`	
`( )`	
Description:	
Return if the element has an associated material.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Element has a material.
`FE_FAIL`	Element does not have a material.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetMatlOrientVec

GetMatlOrientVec **(vMatOrientVec)**	
Description:	
This method returns the vector that is the material orientation direction.	
Input:	
None	None
Output:	
`REAL8 vMatOrientVec[0..2]`	The vector components of the material orientation vector. For laminate elements, this is the 0 degree reference direction.
Return Code:	
`FE_OK`	The element has a material orientation.
`FE_FAIL`	The element does not have a material orientation.
Remarks/Usage:	
None	
Example:	
None	

ElementalCSys

Unpublished work. © 2025 Siemens

ElementalCSys

(bCreateCSys, nCSysType, bUsePlateMatlDirection, nCSysID, vOrigin, vRotation)

Description:

This method creates a coordinate system along the element axis or material direction. This method can also return element orientation values.

Input:

`BOOL bCreateCSys`	If True, a Coordinate System will be created and its ID will be returned in `nCSysID`. If False, no Coordinate System will be created and only element orientation values will be returned.
`INT4 nCSysType`	Type of Coordinate System: Rectangular=0, Cylindrical=1, Spherical=2.
`BOOL bUsePlateMatlDirection`	If False, the Coordinate System represents the elemental axes. If True, the Coordinate System Z axis is aligned with the elemental axes, but the Coordinate System X axis is in the material orientation direction. This option is only used for Planar elements and is ignored for Solid elements.

Output:

`INT4 nCSysID`	If `bCreateCSys`=True, this is the ID of the Coordinate System that was created.
`REAL8 vOrigin[0..2]`	The coordinates in Global Rectangular of the origin of the Coordinate System.
`REAL8 vRotation[0..2]`	The triple rotation angles used to define the orientation of the Coordinate System.

Return Code:

`FE_OK`	Coordinate System was created and/or values were returned.
`FE_FAIL`	Can not determine the elemental Coordinate System.
`FE_NOT_EXIST`	Can not retrieve the nodes that define the element.
`FE_BAD_TYPE`	Element Type is not supported.

Remarks/Usage:

Unpublished work. © 2025 Siemens

None

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim e As femap.Elem
   Set e = App.feElem

   Dim rc As femap.zReturnCode
   Dim bCreateCSys, bUsePlateMatDir As Boolean
    Dim nCSysID As Long 'returned
    Dim vOrigin, vRotation As Variant 'returned

   bCreateCSys = True
   bUsePlateMatDir = True

   If e.SelectID("") <> FE_OK Then End

   rc = e.ElementalCSys( bCreateCSys, FCS_RECTANGULAR,  bUsePlateMatDir, _
                              nCSysID, vOrigin, vRotation )

End Sub
```

Flexible Slider Objects

FlexibleSlider objects correspond to the flexible slider simulation objects in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and

Unpublished work. © 2025 Siemens

methods defined for Entity objects are also available for FlexibleSlider objects. For more information, see **Common Entity Properties and Methods**.

Use the `feFlexibleSlider` method of the Femap Application object to create FlexibleSlider objects.

Flexible Slider Object Properties

Flexible Sliders are always stored with a `SetID=1`, and the ID equal to the Flexible Slider ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Also, all properties which have a "`Nas`" prefix correspond to the name of a field on the `FLXSLI` entry for Simcenter Nastran.

Property	Description
`INT4 color`	The flexible slider color (currently unused).
`INT4 layer`	The layer associated with the flexible slider (currently unused).
`STRING title`	Flexible Slider title (maximum 79 characters).
`INT4 NasTYPE`	The type of flexible slider: `FLXSLI_TYPE_FLEX` = Spherical (0) `FLXSLI_TYPE_CYLF` = Cylindrical (1) `FLXSLI_TYPE_PRIF` = Prismatic (2) `FLXSLI_TYPE_TWIF` = Universal (3) `FLXSLI_TYPE_UNKNOWN` = Unknown (-1)
`INT4 NasNGRPID`	ID of group containing nodes to use for Displacement Entities.
`INT4 NasBGRPID`	ID of group containing elements to use for Slider Trajectory.
`INT4 NasSGID`	ID of node to use as the Sensor Node for Displacement Entities.
`BOOL NasPROJ`	When True, **Projection Alignment** option is turned on.
`INT4 NasNLIM`	Value for Beam Limit in Slider Trajectory.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasDTYPE`	In Driver Load section, Driver Type: `FLXSLI_DRIVER_FORCE` = Force (0) `FLXSLI_DRIVER_DISP` = Displacement (1) `FLXSLI_DRIVER_SENSOR` = Sensor (2) `FLXSLI_DRIVER_UNKNOWN` = None (-1)
`INT4 NasDGID`	ID of node to use as Driver Node for Driver Load.
`REAL8 NasDVAL`	Value of Load for Driver Load.
`INT4 NasDTID`	ID of Femap function to use for Table for Driver Load.
`INT4 NasFOP`	In Friction section, Option: `FLXSLI_FRICTION_NONE` = No Friction (0) `FLXSLI_FRICTION_INF` = Infinite Friction (1) `FLXSLI_FRICTION_VELO` = Velocity (2) `FLXSLI_FRICTION_DISP` = Displacement (3) `FLXSLI_FRICTION_UNKNOWN` = Unknown (-1)
`REAL8 NasCF`	Value of Coefficient for Friction. Only used when `FOP` is set to `FLXSLI_FRICTION_VELO` or `FLXSLI_FRICTION_DISP`.
`REAL8 NasTOL`	Value of Reg. Velocity for Friction. Only used when `FOP` is set to `FLXSLI_FRICTION_VELO`.
`REAL8 NasKCF`	Value of Reg. Stiffness for Friction. Only used when `FOP` is set to `FLXSLI_FRICTION_DISP`.
`BOOL HasNodeList`	When True, the Set Object containing node IDs referenced by `NodeSetID` will be used for Displacement Entities. When False, the group specified by `NodeGroup` will be used.
`BOOL HasElemList`	When True, the Set Object containing element IDs referenced by `ElemSetID` will be used for Slider Trajectory. When False, the group specified by `ElemGroup` will be used.
`INT4 NodeSetID`	ID of Set Object containing Node IDs to be used by `HasNodeSet` is set to True.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ElemSetID`	ID of Set Object containing Element IDs to be used by `HasElemSet` is set to True.
`BOOL NasHasNLIM`	When True, the value for NLIM will be taken into account, otherwise value for NLIM will be ignored.

Flexible Slider Object Methods

The following methods are available in addition to the standard entity methods.

PutComponentSet

`PutComponentSet` **`( entityTYPE, pSelectedComponentSet )`**	
Description:	
This method loads Node IDs into the Node List for Displacement Entities or Element IDs into the Element List for Slider Trajectory.	
Input:	
`INT4 entityType`	The entity type: 7=Node (`FT_NODE`), 8=Element (`FT_ELEMENT`).
`OBJECT pSelectedComponentSet`	Set Object containing the IDs of the entities.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The data type for the component did not match the type of Flexible Sliders.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetComponentSet

GetComponentSet

(entityTYPE, pSelectedComponentSet)

Description:

This method loads a Set object that contains the IDs of the Nodes in the Node List used for Displacement Entities or IDs of the Elements in the Element List used for Slider Trajectory.

Input:

`INT4 entityType`	The entity type: 7=Node (`FT_NODE`), 8=Element (`FT_ELEMENT`).

Output:

`OBJECT pSelectedComponentSet`	Set Object containing the IDs of the entities.

Return Code:

`FE_BAD_TYPE`	The data type for the component did not match the type of Flexible Sliders.

Remarks/Usage:

None

Example:

None

ClearComponentSet

ClearComponentSet

(entityTYPE)

Description:

This method clears the array that contains the array that contains the IDs of the Nodes in the Node List used for Displacement Entities or IDs of the Elements in the Element List used for Slider Trajectory.

Unpublished work. © 2025 Siemens

Input:	
`INT4 entityType`	The entity type: 7=Node (`FT_NODE`), 8=Element (`FT_ELEMENT`).
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The data type for the component did not match the type of Flexible Sliders.
Remarks/Usage:	
None	
Example:	
None	

Freebody Objects

Freebody Objects refer to Freebody Entities in the model, which are used to create freebody diagrams in the model for post-processing. For more information, see **Common Entity Properties and Methods**.

Use the `feFreebody` method of the Femap Application object to create Freebody objects.

Freebody Object Properties

Freebody entities are always stored with a `SetID=1`, and the ID equal to the Freebody ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`STRING title`	Freebody title (maximum 79 characters).
`INT4 CSys`	ID of coordinate system used by the Freebody.
`BOOL UseNodalOutputCS`	Use nodal output coordinate system for freebody nodal vectors. Freebody coordinate system is used when FALSE.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 Layer`	ID of the layer containing the Freebody.
`INT4 DisplayMode`	The Display Mode of the Freebody (0=Freebody, 1=Interface Load, 2=Section Cut). Use enum `zFbdDisplayMode`.
`INT4 Group`	Sets the Entity Selection Mode of the Freebody (-1=Group Select: use Active Group, 0=Entity Select, 1+ = Group Select: use value to specify Group ID).
`INT4 SectionMode`	Section Cut Entity Selection Mode. 0=plane/normal, 1=plane/vector, 2=vector, 3=curve. Use enum `zFbdSectionMode`.

`REAL8 SectionPlaneBase[0..2]`
or
`VARIANT vSectionPlaneBase`
	Coordinates, in Global Rectangular Coordinate System, of section cut plane base. Only used when section cut mode (SectionMode) is 0 (plane/normal) or 1 (plane/vector).

`REAL8 SectionPlaneNormal[0..2]`
or
`VARIANT vSectionPlaneNormal`
	Unit vector for section cut plane normal. Only used when section cut mode (SectionMode) is 0 (plane/normal) or 1 (plane/vector).

`REAL8 SectionVectorBase[0..2]`
or
`VARIANT vSectionVectorBase`
	Coordinates, in Global Rectangular Coordinate System, of section cut vector base. Only used when section cut mode (SectionMode) is 1 (plane/vector) or 2 (vector).

`REAL8 SectionVectorTip[0..2]`
or
`VARIANT vSectionVectorTip`
	Unit vector for section cut vector normal. Only used when section cut mode (SectionMode) is 1 (plane/vector) or 2 (vector).

`INT4 SectionCurve`	ID of curve, only used when section cut mode (SectionMode) is 3 (curve).

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 SectionTolerance`	Entity search tolerance for section cut plane. Value > 0.0.
`BOOL SectionReverse`	Reverse the normal direction of the section cut plane when searching for entities. Does not modify the normal values in the object, only the search direction used internally to determine freebody entities.
`REAL8 SectionRadius`	Search radius for freebody section cut entities. Entities must be less than this distance from the cutting plane base to be included in the freebody (centroidal location is used for element location). Value > 0.0.
`BOOL SectionLimitToGroup`	Limit entities for freebody section cut to those contained within a specified group. Both nodes and elements must exist in the specified group to be considered.
`INT4 SectionGroup`	ID of group to limit freebody section cut entities.
`BOOL SectionIncludeClippedElem`	Include elements clipped by cutting plane in as freebody elements.
`BOOL SectionRotateSums`	Rotate nodal sums and total summation to tangent vector at section cut plane/path intersection.
`INT4 SectionSumLocation`	Location of total summation vector when in section cut mode. 0=plane/path intersection, 1=centroid, 2=static location (defined x, y, z). Use enum zFbdSectionSumLoc
`REAL8 SectionLocation`	Location, along section cut path, of cutting plane
`INT4 NodeMarkerColor`	Sets the color of the Node Marker(s) for the Freebody. Only used when the Color Mode for the Node Makers is set to **Freebody Entity Color**.

`INT4 TotalVectorMode[0..1]`
 	
or
 	
`VARIANT vTotalVectorMode`
	Array of Total Summation Vector display mode.
	
	`TotalVectorMode[0]` = Force Vector Display mode.
	`TotalVectorMode[1]` = Moment Vector Display mode.
	
	Set value to 0 = Off, 1 = Display Components, 2 = Display Resultants.

Unpublished work. © 2025 Siemens

Property	Description

`BOOL ShowTotalVec[0..5]`
 	
or
 	
`VARIANT vShowTotalVec`
	Array of on/off flags for Displayed Forces and Displayed Moments for Total Summation Vector.
	
	`ShowTotalVec[0] = FX (FBC_FX)`
	`ShowTotalVec[1] = FY (FBC_FY)`
	`ShowTotalVec[2] = FZ (FBC_FZ)`
	`ShowTotalVec[3] = MX (FBC_MX)`
	`ShowTotalVec[4] = MY (FBC_MY)`
	`ShowTotalVec[5] = MZ (FBC_MZ)`

`BOOL SumComponents[0..5]`
 	
or
 	
`VARIANT vSumComponents`
	Array of on/off flags for Summed Components for Total Summation Vector:
	
	`SumComponents[0] = Forces X (FBC_FX)`
	`SumComponents[1] = Forces Y (FBC_FY)`
	`SumComponents[2] = Forces Z (FBC_FZ)`
	`SumComponents[3] = Moments X (FBC_MX)`
	`SumComponents[4] = Moments Y (FBC_MY)`
	`SumComponents[5] = Moments Z (FBC_MZ)`

`INT4 TotalVecColor[0..1]`
 	
or
 	
`VARIANT vTotalVecColor`
	Array of Freebody Entity Colors for Total Summation Vector.
	
	`TotalVecColor[0]` = Total Force Vector Freebody Color.
	`TotalVecColor[1]` = Total Moment Vector Freebody Color.
	
	Only used when the Color Mode for the Total Force Vector and/or Total Moment Vector is set to **Freebody Entity Color**.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 x`	X coordinate of freebody total summation vector in coordinate system 0. Only used when Display Mode = Interface Load.
`REAL8 y`	Y coordinate of freebody total summation vector in coordinate system 0. Only used when Display Mode = Interface Load.
`REAL8 z`	Z coordinate of freebody total summation vector in coordinate system 0. Only used when Display Mode = Interface Load.

`INT4 NodalVectorMode[0..1]`
 	
or
 	
`VARIANT vNodalVectorMode`
	Array of Nodal Vector(s) display mode.
	
	`NodalVectorMode[0]` = Force Vector Display mode.
	`NodalVectorMode[1]` = Moment Vector Display mode.
	
	Set value to 0 = Off, 1 = Display Components, 2 = Display Resultants.

`BOOL ShowNodalVec[0..5]`
 	
or
 	
`VARIANT vShowNodalVec`
	Array of on/off flags for Displayed Forces and Displayed Moments for Nodal Vector(s):
	
	`ShowNodalVec[0] = FX (FBC_FX)`
	`ShowNodalVec[1] = FY (FBC_FZ)`
	`ShowNodalVec[2] = FZ (FBC_FZ)`
	`ShowNodalVec[3] = MX (FBC_MX)`
	`ShowNodalVec[4] = MY (FBC_MY)`
	`ShowNodalVec[5] = MZ (FBC_MZ)`

`INT4 NodalVecColor[0..1]`
 	
or
 	
`VARIANT vNodalVecColor`
	Array of Freebody Entity Colors for Total Summation Vector.
	
	`NodalVecColor[0]` = Nodal Force Vector(s) Freebody Color.
	`NodalVecColor[1]` = Nodal Moment Vector(s) Freebody Color.

Unpublished work. © 2025 Siemens

Property	Description
	Only used when the Color Mode for the Nodal Force Vector(s) and/or Nodal Moment Vector(s) is set to **Freebody Entity Color**.

`BOOL SumContributions[0..7]`
 	
or
 	
`VARIANT vSumContributions`
	Array of on/off flags for Freebody Contributions From:
	
	`SumContributions[0] = Applied (FBL_APPLIED)`
	
	`SumContributions[1] = Reaction (FBL_SPC)`
	
	`SumContributions[2] = MultiPoint Reaction (FBL_MPC)`
	
	`SumContributions[3] = Freebody Elements (FBL_EL_FB)`
	
	`SumContributions[4] = Peripheral Elements (FBL_EL_PER)`
	
	`SumContributions[5] = Nodal Summation (FBL_TOTAL)`
	
	`SumContributions[6] = Contact (FBL_CONTACT)`
	
	`SumContributions[7] = Glue (FBL_GLUE)`

`BOOL AlternateApplied`	Allow use of alternate vector for load contribution component for Applied.
`BOOL AlternateSPC`	Allow use of alternate vector for load contribution component for Reaction.
`BOOL AlternateMPC`	Allow use of alternate vector for load contribution component for MultiPoint Reaction.
`BOOL ReverseTotalValues`	Reverse values for freebody nodal summation contribution.

Freebody Object Methods

The following methods are available in addition to the standard entity methods.

Unpublished work. © 2025 Siemens

GetElements

GetElements **(numElem, elemID)**	
Description:	
This method retrieves the number of elements and an array of element IDs selected by the Freebody.	
Input:	
None	None
Output:	
`INT4 numElem`	The number of elements for which IDs are being returned.
`INT4 elemID[0..numElem-1]`	The IDs of the elements.
Return Code:	
`FE_FAIL`	Unable to retrieve elements from freebody.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	
Example:	
None	

SetElements

SetElements **(bClear, setID)**
Description:

Unpublished work. © 2025 Siemens

Sets the elements stored in the freebody object.	
Input:	
`BOOL bClear`	Clear any existing elements from the freebody.
`INT4 setID`	Set ID containing elements to add to the freebody.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store elements in Freebody entity.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	
Example:	
None	

ClearElements

`ClearElements` **`( void )`**	
Description:	
Resets elements stored in the freebody object.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	No elements in Freebody entity.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	
Example:	
None	

GetNodes

`GetNodes`	
`( numNode, NodeID )`	
Description:	
This method retrieves the number of nodes and an array of node IDs selected by the Freebody.	
Input:	
None	None
Output:	
`INT4 numElem`	The number of nodes for which IDs are being returned.
`INT4 elemID[0..numElem-1]`	The IDs of the nodes.
Return Code:	
`FE_FAIL`	Unable to retrieve nodes from freebody.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	

Unpublished work. © 2025 Siemens

Example:
None

SetNodes

`SetNodes` **`( bClear, setID )`**	
Description:	
Sets the elements stored in the freebody object.	
Input:	
`BOOL bClear`	Clear any existing nodes from the freebody.
`INT4 setID`	Set ID containing nodes to add to the freebody.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store nodes in Freebody entity.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	
Example:	
None	

ClearNodes

Unpublished work. © 2025 Siemens

ClearNodes	
(void)	
Description:	
Resets nodes stored in the freebody object.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	No nodes stored in the freebody object.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	
Example:	
None	

CalculateNodalCenter

CalculateNodalCenter	
(bUpdate, xyz)	
Description:	
Calculates the center of the nodes stored in the freebody object and optionally updates the summation location.	
Input:	
`BOOL bUpdate`	Update the location of the summation vector.

Unpublished work. © 2025 Siemens

Output:	
`VARIANT xyz`	Center of nodes in freebody in global rectangular coordinate system.
Return Code:	
`FE_FAIL`	No nodes stored in freebody.
`FE_INVALID`	Freebody is not set to entity select mode (group is not 0).
`FE_NOT_AVAILABLE`	Freebody contains nodes that are not available in the database.
Remarks/Usage:	
The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.	
Example:	
None	

CalculateSummation

`CalculateSummation`
`( outsetID, calculations)`
Description:
This method is obsolete. For new applications, you should use `CalculateSummation2` Calculates the total summation vector based on the settings in the freebody for the given output set.
Input:
`INT4 outSetID` — Output set ID for calculation.
Output:
`VARIANT calculations` — Calculated summed forces and moments.
Return Code:
`FE_FAIL` — Freebody summation has been calculated with potential errors (ie missing contributions of load,

Unpublished work. © 2025 Siemens

	missing elements, etc.) The values need to be verified.
`FE_NOT_EXIST`	Specified output set does not exist.
`FE_INVALID`	Freebody is not set to interface mode.

Remarks/Usage:

The method only works when the freebody is set to entity select mode (group is set to 0), otherwise `FE_INVALID` is returned.

Example:

None

CalculateSummation2

`CalculateSummation2`

`( nSetID, bUseFBCSys, nNumSets, OutSetIDs, calculations)`

Description:

Calculates the total summation vector based on the settings in the freebody for one or more output sets in either the freebody coordinate system, or the global coordinate system.

Input:

`INT4 nSetID`	The ID of a Set object containing a list of output sets. Alternatively, specify a negative number to specify the ID of a single output set.
`BOOL bUseFBCSys`	Use the freebody coordinate system to return the calculations. If FALSE, calculations are returned in the global coordinate system.

Output:

`INT4 nNumSets`	Number of output sets returned.
`VARIANT outsetIDs[0..nNumSets-1]`	IDs of output sets returned.
`VARIANT calculations[0..6*(nNumSets)-1]`	Calculated total summation vectors Fx, Fy, Fz, Mx, My, Mz, for each output set returned in either the global or freebody coordinate system.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Freebody summation has been calculated with potential errors (ie missing contributions of load, missing elements, etc.). The values may need to be verified.
`FE_NOT_EXIST`	One or more of the specified output sets do not exist.
`FE_INVALID`	Freebody is not set to interface mode.
Remarks/Usage:	
This method only works when the freebody is set to interface load mode, otherwise `FE_INVALID` is returned.	
Example:	
None	

GetNodalSums

`GetNodalSums`	
`( nSetID, nNumNodes, nNodeIDs, dVals )`	
Description:	
This method Returns values for summations calculated at nodes for all freebody nodes for one or more output sets.	
Input:	
`INT4 nSetID`	The ID of a Set object containing a list of output sets. Alternatively, specify a negative number to specify the ID of a single output set.
Output:	
`INT4 nNumNodes`	Number of nodes in freebody.
`INT4 nNodeIDs[0..nNumNodes-1]`	The IDs of the nodes.
`INT4 dVals[(0..(6*numElem)-1) *Number of Output Sets]`	Each node returns six values (Fx, Fy, Fz, Mx, My, Mz), so the data in the array is stored as 6 *

Unpublished work. © 2025 Siemens

	`nNumNode`. This is repeated for each output set requested.
Return Code:	
`FE_NOT_EXIST`	One or more of the specified output sets does not exist in the database.
`FE_FAIL`	The current freebody contains no nodes.
Remarks/Usage:	
This method returns the values that Femap calculates as a sum at a node, dependent on the selected components and contributions. It does not return the solver-calculated nodal imbalance at a node.	
Example:	
None	

GetSumAtNode

`GetSumAtNode` **`( nNodeID, nSetID, dVals  )`**	
Description:	
Returns the calculated summation at a single node for one or more output sets.	
Input:	
`INT4 nNodeID`	Node ID
`INT4 nSetID`	The ID of a Set object containing a list of output sets. Alternatively, specify a negative number to specify the ID of a single output set.
Output:	
`INT4 dVals[0..6*Number of Output Sets-1]`	Six values (Fx, Fy, Fz, Mx, My, Mz) for each output set.
Return Code:	
`FE_NOT_EXIST`	Specified node does not exist in the freebody.

Unpublished work. © 2025 Siemens

`FE_FAIL`	One or more of the specified output sets does not exist in the database.
Remarks/Usage:	
This method returns the values that Femap calculates as a sum at a node, dependent on the selected components and contributions. It does not return the solver-calculated nodal imbalance at a node.	
Example:	
None	

GetSectionCutSums

`GetSectionCutSums` **`( nOutSetSetID, nCuts, pnNumCuts, pCutLocs, pSumLocs, pSums )`**	
Description:	
Calculates the total summation for a freebody in section cut mode across one or more output sets.	
Input:	
`INT4 nOutSetSetID`	The ID of a Set that contains output set IDs to use for calculating total summation. Alternatively, if you specify a negative value, this is simply the ID of a single output set.
`INT4 nCuts`	Positive value: number of equally spaced cuts to make. Negative value: ID of Along Coordinates Data Surface to use for locating section cuts. When section cut (SectionMode) is 0 (plane/normal), 1 (plane/vector) or 2 (vector), specify XYZ values in the coordinate system specified for the total summation vector. When section cut is (SectionMode) is 3 (curve), specify a value between 0 and 1 representing the distance along the curve for the X value in the Data Surface.
Output:	
`INT4 pnNumCuts`	Number of section cuts returned.

Unpublished work. © 2025 Siemens

`REAL8 pCutLocs[0..3*Number of Output Sets-1]`	Section cut locations in coordinate system specified for total summation. Three values (X, Y, Z) for each section cut.
`REAL8 pSumLocs[0..3*Number of Output Sets-1]`	Total summation location in coordinate system specified for total summation. Three values (X,Y, Z) for each section cut.
`REAL8 pSums[0..6*Number of Output Sets-1]`	Total summation for each section cut. Six values (Fx, Fy, Fz, Mx, My, Mz) for each section cut.
Return Code:	
`FE_FAIL`	Unable to calculate section cut summations.
`FE_INVALID`	Freebody is not in section cut mode.
`FE_BAD_DATA`	Data surface is specified and is not along coordinates, or data surface contains invalid data.
Remarks/Usage:	
Only valid for use with freebodies in section cut mode.	
Example:	
None	

SetLocationFromCoord

`SetLocationFromCoord`	
`( x, y, z, pLoc )`	
Description:	
Set the section cut location by specifying x, y, z coordinates.	
Input:	
`REAL8 x`	Coordinates in global rectangular coordinate system.
`REAL8 y`	

Unpublished work. © 2025 Siemens

`REAL8 z`	
Output:	
`REAL8 pLoc`	Calculated location.
Return Code:	
`FE_FAIL`	Freebody is not in section cut mode.
`FE_INVALID`	Freebody section cut is not defined by plane/normal, plane/vector or vector.
Remarks/Usage:	
Only valid for use with freebody entities in section cut mode with section cut (SectionMode) defined by plane/normal (0), plane/vector (1), or vector (2).	
Example:	
None	

Show

Show **`( void )`**	
Description:	
Highlights both the nodes and elements of the freebody entity currently loaded in the Freebody Object.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to show entities.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

To ensure a particular *Show Mode* and options are used for highlighting, set any combination of the available `Info_ViewShow...` options before calling this method.
Example:
None

Frequency Objects

Frequency objects correspond to lists of solution frequencies that are used in the frequency response analysis of NX NASTRAN. The lists are input as FREQ, FREQ1, FREQ2, FREQ3, FREQ4 and FREQ5. They Frequency objects are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Frequency objects, see Common Entity Properties and Methods. These lists of solution frequencies can also be used by the **Model → Output → Forced Response** command.

Use the `feFreq` method of the Femap Application object to create Frequency objects.

Frequency Object Properties

Frequency input is always stored with a `SetID=1`, and an ID automatically incremented from 1.

Property	Description
`INT4 type`	This must be FREQ, FREQ1, FREQ2, FREQ3, FREQ4 or FREQ5.
`REAL8 F1`	F1 in FREQ1-5 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ2-5 of NX NASTRAN.
`REAL8 DF`	DF in FREQ1 of NX NASTRAN.
`REAL8 FSPD`	FSPD in FREQ4 of NX NASTRAN.
`REAL8 CLUSTER`	CLUSTER in FREQ3.
`INT4 NF`	NDF in FREQ1, NF in FREQ2, NEF in FREQ3 or NFM in FREQ4.
`BOOL LOG`	LOG=TRUE sets TYPE=LOG in FREQ3, otherwise TYPE=LINEAR.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NDF`	NDF in FREQ1 or Number of Increments.
`INT4 NEF`	NEF in FREQ3 or Number of Frequencies.
`INT4 NFM`	NFM in FREQ4 or Number of Frequencies.

Frequency Object Methods

The following methods are available in addition to the standard entity methods.

AddFreq

AddFreq **(NF, Fi, asetID, on)**	
Description:	
This method adds a FREQ list of solution frequencies, with NF frequency values stored in Fi. If on=TRUE, it means the FREQ list of solution frequencies is used in the analysis set specified by `asetID`.	
Input:	
`INT4 NF`	The number of frequency values in Fi.
`REAL8 Fi[0..NF-1]`	Frequency values (F1,F2,...) of FREQ.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ is used in the analysis set specified by `asetID`.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to add FREQ Frequency object.
Remarks/Usage:	
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0.	
Example:	
None	

AddFreqByFuncID

`AddFreqByFuncID` **`( funcID, asetID, on )`**	
Description:	
This method adds a FREQ list of solution frequencies, with the frequency values defined by a function specified by funcID. If on=TRUE, it means the FREQ input is used in the analysis set specified by `asetID`.	
Input:	
`INT4 funcID`	The ID of the function.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	
`FE_OK`	The current FREQ Frequency object is updated successfully.
`FE_FAIL`	Unable to add FREQ Frequency object.

Unpublished work. © 2025 Siemens

Remarks/Usage:
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0. Refer to **Function Objects**. In this usage, x values of the function is not relevant.
Example:
None

UpdFreq

UpdFreq **(NF, Fi, asetID, on)**	
Description:	
This method updates the current list of solution frequencies to FREQ type, with NF frequency values stored in Fi. If on=TRUE, it means the FREQ input is used in the analysis set specified by asetID.	
Input:	
`INT4 NF`	The number of frequency values in Fi.
`REAL8 Fi[0..NF-1]`	Frequency values (F1,F2,...) of FREQ.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to update FREQ Frequency object.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

The current list of solution frequencies is selected by the `Get` method of the Entity object. It needs to call the Put method of the Entity object to update.
Example:
None

UpdFreqByFuncID

`UpdFreqByFuncID` **`( funcID, asetID, on )`**	
Description:	
This method updates the current list of solution frequencies to a FREQ list of solution frequencies, with the frequency values defined by a function specified by funcID. If on=TRUE, it means the FREQ input is used in the analysis set specified by `asetID`.	
Input:	
`INT4 NF`	The ID of the function.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	
`FE_OK`	The current FREQ Frequency object is updated successfully.
`FE_FAIL`	Unable to add FREQ Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the `Get()` method of the Entity object. It needs to call the `Put()` method of the Entity object to update. Refer to **Function Objects**.	

Unpublished work. © 2025 Siemens

In this usage, x values of the function is not relevant.
Example:
None

AddFreq1

AddFreq1 **(F1, DF, NDF, asetID, on)**	
Description:	
This method adds a FREQ1 list of solution frequencies. If on=TRUE, it means the FREQ1 input is used in the analysis set specified by asetID.	
Input:	
REAL8 F1	F1 in FREQ1 of NX NASTRAN.
REAL8 DF	DF in FREQ1 of NX NASTRAN.
INT4 NDF	NDF in FREQ1 of NX NASTRAN.
INT4 asetID	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
BOOL on	If on=TRUE, it sets the flag that the input FREQ1 is used in the analysis set specified by asetID.
Output:	
None	None
Return Code:	
FE_FAIL	Unable to add FREQ1 Frequency object.
Remarks/Usage:	
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0.	

Unpublished work. © 2025 Siemens

Example:
None

UpdFreq1

UpdFreq1 **(F1, DF, NDF, asetID, on)**	
Description:	
This method updates the current list of solution frequencies to a FREQ1 list of solution frequencies. If on=TRUE, it means the FREQ1 input is used in the analysis set specified by asetID.	
Input:	
REAL8 F1	F1 in FREQ1 of NX NASTRAN.
REAL8 DF	DF in FREQ1 of NX NASTRAN.
INT4 NDF	NDF in FREQ1 of NX NASTRAN.
INT4 asetID	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
BOOL on	If on=TRUE, it sets the flag that the input FREQ1 is used in the analysis set specified by asetID.
Output:	
None	None
Return Code:	
FE_FAIL	Unable to add FREQ1 Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the Get method of the Entity object. It needs to call the Put method of the Entity object to update.	

Unpublished work. © 2025 Siemens

Example:
None

AddFreq2

`AddFreq2`
 	
`( F1, F2, NF, asetID, on )`	

Description:	
This method adds a FREQ2 list of solution frequencies. If on=TRUE, it means the FREQ2 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ2 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ2 of NX NASTRAN.
`INT4 NF`	NF in FREQ2 of NX NASTRAN.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ2 is used in the analysis set specified by asetID.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to add FREQ2 Frequency object.
Remarks/Usage:	
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0.	

Unpublished work. © 2025 Siemens

Example:
None

UpdFreq2

UpdFreq2 **(F1, F2, NF, asetID, on)**	
Description:	
This method updates the current list of solution frequencies to a FREQ2 list of solution frequencies. If on=TRUE, it means the FREQ2 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ2 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ2 of NX NASTRAN.
`INT4 NF`	NF in FREQ2 of NX NASTRAN.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ2 is used in the analysis set specified by asetID.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to add FREQ2 Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the `Get` method of the Entity object. It needs to call the `Put` method of the Entity object to update.	

Unpublished work. © 2025 Siemens

Example:
None

AddFreq3

AddFreq3 **(F1, F2, NEF, LOG, CLUSTER, asetID, on)**	
Description:	
This method adds a FREQ3 list of solution frequencies. If on=TRUE, it means the FREQ3 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ3 of NX NASTRAN
`REAL8 F2`	F2 in FREQ3 of NX NASTRAN
`INT4 NEF`	NEF in FREQ3 of NX NASTRAN
`BOOL LOG`	If LOG=TRUE, it sets TYPE=LOG in FREQ3. Otherwise it sets TYPE=LINEAR.
`REAL8 CLUSTER`	CLUSTER in FREQ3 of NX NASTRAN
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ3 is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to add FREQ3 Frequency object.

Unpublished work. © 2025 Siemens

Remarks/Usage:
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0.
Example:
None

UpdFreq3

UpdFreq3 **(F1, F2, NEF, LOG, CLUSTER, asetID, on)**	
Description:	
This method updates the current list of solution frequencies to a FREQ3 list of solution frequencies. If on=TRUE, it means the FREQ3 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ3 of NX NASTRAN
`REAL8 F2`	F2 in FREQ3 of NX NASTRAN
`INT4 NEF`	NEF in FREQ3 of NX NASTRAN
`BOOL LOG`	If LOG=TRUE, it sets TYPE=LOG in FREQ3. Otherwise it sets TYPE=LINEAR.
`REAL8 CLUSTER`	CLUSTER in FREQ3 of NX NASTRAN
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ3 is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to add FREQ3 Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the `Get` method of the Entity object. It needs to call the `Put` method of the Entity object to update.	
Example:	
None	

AddFreq4

`AddFreq4` **`( F1, F2, NFM, FPSD, asetID, on )`**	
Description:	
This method adds a FREQ4 list of solution frequencies. If on=TRUE, it means the FREQ4 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ4 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ4 of NX NASTRAN.
`INT4 NFM`	NFM in FREQ4 of NX NASTRAN.
`REAL8 FPSD`	FPSD in FREQ4 of NX NASTRAN.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ4 is used in the analysis set specified by asetID.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
FE_FAIL	Unable to add FREQ4 Frequency object.
Remarks/Usage:	
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0.	
Example:	
None	

UpdFreq4

UpdFreq4 **(F1, F2, NFM, FPSD, asetID, on)**	
Description:	
This method updates the current list of solution frequencies to a FREQ4 list of solution frequencies. If on=TRUE, it means the FREQ4 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ4 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ4 of NX NASTRAN.
`INT4 NFM`	NFM in FREQ4 of NX NASTRAN.
`REAL8 FPSD`	FPSD in FREQ4 of NX NASTRAN.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ4 is used in the analysis set specified by asetID.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
FE_FAIL	Unable to add FREQ4 Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the `Get` method of the Entity object. It needs to call the `Put` method of the Entity object to update.	
Example:	
None	

AddFreq5

AddFreq5 **(F1, F2, NFR, FRi, asetID, on)**	
Description:	
This method adds a FREQ5 list of solution frequencies, with NFR frequency values stored in `FRi`. If on=TRUE, it means the FREQ5 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ5 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ5 of NX NASTRAN.
`INT4 NFR`	The number of frequency values in `FRi`.
`REAL8 FRi[0..NFR-1]`	Frequency values (F1,F2,...) of FREQ5 in the range of F1 and F2.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ5 is used in the analysis set specified by `asetID`.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
FE_FAIL	Unable to add FREQ5 Frequency object.
Remarks/Usage:	
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0.	
Example:	
None	

AddFreq5ByFuncID

AddFreq5ByFuncID **(F1, F2, funcID, asetID, on)**	
Description:	
This method adds a FREQ5 list of solution frequencies, with the frequency values defined by a function specified by `funcID`. If on=TRUE, it means the FREQ5 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ5 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ5 of NX NASTRAN.
`INT4 funcID`	The ID of the function.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`BOOL on`	If on=TRUE, it sets the flag that the input FREQ5 is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_OK`	The current FREQ5 Frequency object is updated successfully.
`FE_FAIL`	Unable to add FREQ5 Frequency object.
Remarks/Usage:	
The ID of the newly added list of solution frequencies is the unused maximum ID counting from 0. Refer to **Function Objects**. In this usage, x values of the function is not relevant.	
Example:	
None	

UpdFreq5

`UpdFreq5` **`( F1, F2, NFR, FRi, asetID, on )`**	
Description:	
This method updates the current list of solution frequencies to FREQ5 type, with NFR frequency values stored in `FRi`. If on=TRUE, it means the FREQ5 input is used in the analysis set specified by `asetID`.	
Input:	
`REAL8 F1`	F1 in FREQ5 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ5 of NX NASTRAN.
`INT4 NFR`	The number of frequency values in `FRi`.
`REAL8 FRi[0..NFR-1]`	Frequency values (F1,F2,...) of FREQ5 in the range of F1 and F2.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.

Unpublished work. © 2025 Siemens

`BOOL on`	If on=TRUE, it sets the flag that the input FREQ5 is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to add FREQ5 Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the `Get` method of the Entity object. It needs to call the `Put` method of the Entity object to update.	
Example:	
None	

UpdFreq5ByFuncID

`UpdFreq5ByFuncID`
`( F1, F2, funcID, asetID, on )`
Description:
This method updates the current list of solution frequencies to a FREQ5 list of solution frequencies, with the frequency values defined by a function specified by `funcID`. If on=TRUE, it means the FREQ5 input is used in the analysis set specified by `asetID`.
Input:

`REAL8 F1`	F1 in FREQ5 of NX NASTRAN.
`REAL8 F2`	F2 in FREQ5 of NX NASTRAN.
`INT4 funcID`	The ID of the function.
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.

Unpublished work. © 2025 Siemens

`BOOL on`	If on=TRUE, it sets the flag that the input FREQ5 is used in the analysis set specified by `asetID`.
Output:	
None	None
Return Code:	
`FE_OK`	The current FREQ5 Frequency object is updated successfully.
`FE_FAIL`	Unable to add FREQ5 Frequency object.
Remarks/Usage:	
The current list of solution frequencies is selected by the `Get` method of the Entity object. It needs to call the `Put` method of the Entity object to update. Refer to **Function Objects**. In this usage, x values of the function is not relevant.	
Example:	
None	

AllOn

`AllOn` **`( asetID )`**
Description:
This method sets the flag to True for all solution frequencies list(s) defined in the model, in the analysis set(s) specified by `asetID`.
Input:

`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to set flags.
Remarks/Usage:	
Opposite operation to `AllOff`.	
Example:	
None	

AllOff

`AllOff` **`( asetID )`**	
Description:	
This method sets the flag to False for all solution frequencies list(s) defined in the model, in the analysis set(s) specified by `asetID`.	
Input:	
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set flags.
Remarks/Usage:	
Opposite operation to `AllOn`.	
Example:	
None	

Unpublished work. © 2025 Siemens

TurnOn

TurnOn	
(asetID, freqID)	
Description:	
This method sets the flag to True for the solution frequencies list(s) specified by `freqID`, in the analysis set(s) specified by `asetID`.	
Input:	
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`INT4 freqID`	The ID of a set that contains IDs of lists of solution frequencies. Alternatively, if you specify a negative value, this is simply the ID of the single list of solution frequencies, while a value of 0 selects all available lists of solution frequencies. Refer to **Set Objects**.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set flags.
Remarks/Usage:	
Opposite operation to `TurnOff`.	
Example:	
None	

TurnOff

Unpublished work. © 2025 Siemens

TurnOff **(asetID, freqID)**	
Description:	
This method sets the flag to False for the solution frequencies list(s) specified by `freqID`, in the analysis set(s) specified by `asetID`.	
Input:	
`INT4 asetID`	The ID of a set that contains analysis set IDs. Alternatively, if you specify a negative value, this is simply the ID of the single analysis set, while a value of 0 indicates all analysis sets. Refer to **Set Objects**.
`INT4 freqID`	The ID of a set that contains IDs of lists of solution frequencies. Alternatively, if you specify a negative value, this is simply the ID of the single list of solution frequencies, while a value of 0 selects all available lists of solution frequencies. Refer to **Set Objects**.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set flags.
Remarks/Usage:	
Opposite operation to `TurnOn`.	
Example:	
None	

GetFREQType

Unpublished work. © 2025 Siemens

GetFREQType **(FREQ_Type)**	
Description:	
This method returns the FREQ type.	
Input:	
None	None
Output:	
`INT4 FREQ_Type`	The Freq type - FREQ, FREQ1, FREQ2, FREQ3, FREQ4 or FREQ5.
Return Code:	
`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

GetFREQData

GetFREQData **(NF, Fi, funcID)**	
Description:	
This method returns the FREQ data.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`NF`	The number of frequency values in `Fi`.
`Fi`	Frequency values (F1,F2,...) of FREQ.
`funcID`	The ID of the function.
Return Code:	
`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

GetFREQ1Data

`GetFREQ1Data`	
`( F1, DF, NDF)`	
Description:	
This method returns the FREQ1 data.	
Input:	
None	None
Output:	
`REAL8 F1`	Value of First Frequency (`F1`).
`INT4 DF`	Value of Frequency Increment (`DF`).
`INT4 NDF`	Value of Number of Increments (`NDF`).
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

GetFREQ2Data

`GetFREQ2Data` **`( F1, F2, NF)`**	
Description:	
This method returns the FREQ2 data.	
Input:	
None	None
Output:	
`REAL8 F1`	Value of First Frequency (`F1`).
`REAL8 F2`	Value of Last Frequency (`F2`).
`INT4 NF`	Value for the Number of Log Interval (`NF`).
Return Code:	
`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetFREQ3Data

GetFREQ3Data **(F1, F2, NEF, LOG, CLUSTER)**	
Description:	
This method returns the FREQ3 data.	
Input:	
None	None
Output:	
`REAL8 F1`	Value of First Frequency (F1).
`REAL8 F2`	Value of Last Frequency (F2).
`INT4 NEF`	Value of Number of Frequencies (NEF).
`BOOL LOG`	If True, Logarithmic option is checked.
`REAL8 CLUSTER`	Value of Cluster.
Return Code:	
`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

GetFREQ4Data

Unpublished work. © 2025 Siemens

GetFREQ4Data	
(F1, F2, NFM, FPSD)	
Description:	
This method returns the FREQ4 data.	
Input:	
None	None
Output:	
`REAL8 F1`	Value of First Frequency (`F1`).
`REAL8 F2`	Value of Last Frequency (`F2`).
`INT4 NFM`	Value of Number of Frequencies (`NFM`).
`REAL8 FPSD`	Value of Spread (`FSPD, +/- %`).
Return Code:	
`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

GetFREQ5Data

GetFREQ5Data
(F1, F2, NFR, FRi, funcID)
Description:
This method returns the FREQ5 data.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
`REAL8 F1`	Value of First Frequency (`F1`).
`REAL8 F2`	Value of Last Frequency (`F2`).
`INT4 NFR`	Value of Number of Frequencies (`NFR`).
`REAL8 FRi[0..NFR-1]`	Frequency values (`F1`,`F2`,...) of `FREQ5` in the range of `F1` and `F2`.
`INT4 funcID`	ID of Femap function specified in **Function ID** drop-down.
Return Code:	
`FE_FAIL`	Unable to get FREQ type.
Remarks/Usage:	
This method simply retrieves the information about the Freq from the current object. You must first load that data by using the `Get( )` method.	
Example:	
None	

SetFREQData

`SetFREQData`
`( NF, Fi, funcID)`
Description:
This method sets the FREQ data.
Input:

`NF`	The number of frequency values in `Fi`.

Unpublished work. © 2025 Siemens

`Fi`	Frequency values (F1,F2,...) of FREQ.
`funcID`	The ID of the function.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set FREQ.
Remarks/Usage:	
This method does not store that information to the model database. After using this function, you must call `Put()` if you want to store the lists in the database.	
Example:	
None	

SetFREQ1Data

`SetFREQ1Data`
`( F1, DF, NDF)`
Description:
This method sets the FREQ1 data.
Input:

`REAL8 F1`	Value of First Frequency (F1).
`INT4 DF`	Value of Frequency Increment (DF).
`INT4 NDF`	Value of Number of Increments (NDF).
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to set FREQ 1.
Remarks/Usage:	
This method does not store that information to the model database. After using this function, you must call `Put()` if you want to store the lists in the database.	
Example:	
None	

SetFREQ2Data

`SetFREQ2Data` **`( F1, F2, NF)`**	
Description:	
This method sets the FREQ2 data.	
Input:	
`REAL8 F1`	Value of First Frequency (F1).
`REAL8 F2`	Value of Last Frequency (F2).
`INT4 NF`	Value for the Number of Log Interval (NF).
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set FREQ2.
Remarks/Usage:	
This method does not store that information to the model database. After using this function, you must call `Put()` if you want to store the lists in the database.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetFREQ3Data

SetFREQ3Data **(F1, F2, NEF, LOG, CLUSTER)**	
Description:	
This method sets the FREQ3 data.	
Input:	
`REAL8 F1`	Value of First Frequency (F1).
`REAL8 F2`	Value of Last Frequency (F2).
`INT4 NEF`	Value of Number of Frequencies (NEF).
`BOOL LOG`	If True, checks Logarithmic option.
`REAL8 CLUSTER`	Value of Cluster.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set FREQ3.
Remarks/Usage:	
This method does not store that information to the model database. After using this function, you must call `Put()` if you want to store the lists in the database.	
Example:	
None	

SetFREQ4Data

Unpublished work. © 2025 Siemens

SetFREQ4Data	
(F1, F2, NFM, FPSD)	
Description:	
This method sets the FREQ4 data.	
Input:	
`REAL8 F1`	Value of First Frequency (F1).
`REAL8 F2`	Value of Last Frequency (F2).
`INT4 NFM`	Value of Number of Frequencies (NFM).
`REAL8 FPSD`	Value of Spread (FSPD, +/- %).
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set FREQ4.
Remarks/Usage:	
This method does not store that information to the model database. After using this function, you must call `Put()` if you want to store the lists in the database.	
Example:	
None	

SetFREQ5Data

SetFREQ5Data	
(F1, F2, NFR, FRi, funcID)	
Description:	
This method sets the FREQ5 data.	

Unpublished work. © 2025 Siemens

Input:	
`REAL8 F1`	Value of First Frequency (F1).
`REAL8 F2`	Value of Last Frequency (F2).
`INT4 NFR`	Value of Number of Frequencies (NFR).
`REAL8 FRi[0..NFR-1]`	Frequency values (F1,F2,...) of FREQ5 in the range of F1 and F2.
`INT4 funcID`	ID of Femap function specified in **Function ID** drop-down
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to set FREQ5.
Remarks/Usage:	
This method does not store that information to the model database. After using this function, you must call `Put()` if you want to store the lists in the database.	
Example:	
None	

Function Objects

Function objects correspond to the functions in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Function objects. For more information, see **Common Entity Properties and Methods**.

Use the `feFunction` method of the Femap Application object to create Function objects.

Function Object Properties

Functions are always stored with a `SetID=1`, and the ID equal to the function ID.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title`	The function title (maximum 79 characters).
`INT4 type`	Type of function: 0=Dimensionless, 1=vs. Time, 2=vs. Temp, 3=vs. Freq, 4=vs. Stress, 5=Func vs. Temp, 6=Struct Damp vs. Freq, 7=Crit Damp vs. Freq, 8=Q Damp vs. Freq, 9=vs. Strain Rate, 10=Func vs. Strain Rate, 11=vs. Curve Length, 12=vs. Curve Param, 13=Stress vs. Strain, 14=Stress vs. Plastic Strain, 15=Function vs Value, 16=Function vs. Critical Damping, 17=vs. Angle of Incidence, 18=vs. Direction of Incidence, 19=vs. Temp (TABLEM1 Linear,Linear), 20=vs. Temp (TABLEM1 Log,Linear), 21=vs. Temp (TABLEM1 Linear,Log), 22=vs. Temp (TABLEM1 Log,Log), 23=vs. Mode, 24=Displacement vs. Freq, 25=Rotation vs. Frequency, 26=Velocity vs. Freq, 27=Angular Vel vs. Freq, 28=Acceleration vs. Freq, 29=Angular Acc vs. Freq, 30=Force vs. Freq, 31=Moment vs. Freq, 32=Stress vs. Freq, 33=Strain vs.Freq, 34=Mach Number vs. Freq, 35=vs. Aerodynamic Factor, 36=Acceleration vs. Location

Function Object Methods

The following methods are available in addition to the standard entity methods.

GetFunctionList

`GetFunctionList`	
`( listcount, x, y )`	
Description:	
This method retrieves function pair information.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
`INT4 listcount`	The number of entries in the other arrays.
`REAL8 x[0..N]`	The x and y values for the function..
`REAL8 y[0..N]`	
Return Code:	
`FE_NOT_EXIST`	There are no entries in the list.
Remarks/Usage:	
This method simply retrieves the information about the function pairs from the current object. You must first load that data - either using the `Get( )` method, or by loading it yourself with `PutFunctionList( )`. Data retrieved from the database using `Get`, will always be in order of ascending X values.	
Example:	
None	

PutFunctionList

`PutFunctionList` **`( listcount, x, y )`**	
Description:	
This method saves function pair information.	
Input:	
`INT4 listcount`	The number of entries in the other arrays.
`REAL8 x[0..N]`	The x and y values for the function.
`REAL8 y[0..N]`	
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
This method stores information into the list of the current object. Any previous list information in that object is lost. This method does not store that information to the model database. After using this function, you must call `Put( )` if you want to store the updated list in the database. Data to be stored in the database will be sorted into order of increasing x values. It is not required, however, `Put` performance will be improved if you provide the x,y pairs in sorted order.	
Example:	
None	

GetValue

`GetValue` **`( nMethod, x, y )`**	
Description:	
This method evaluates the function to find a y value given an x value.	
Input:	
`INT4 nMethod`	The method to be used to find the y value. 0=Before (return the y value of the function at the x value before the specified “x”). 1=After (return the y value of the function at the xvalue after the specified “x”). 2=Closest (return the y value of the function at the x value closest to the specified “x”). 3=Interpolate, Linear-Linear (return the y value at the specified "x" location by linearly interpolating the function x values and the function y values). 4=Interpolate, Log-Log (return the y value at the specified "x" location by linearly interpolating the log of the function x values and the log of the function y values). 5=Interpolate, Linear-Log (return the y value at the specified "x" location by linearly interpolating

Unpublished work. © 2025 Siemens

	the function x values and the log of the function y values). 6=Interpolate, Log-Linear (return the y value at the specified "x" location by linearly interpolating the log of the function x values and the function y values).
`REAL8 x`	The location where the function is evaluated.
Output:	
`REAL8 y`	The function value at the specified x location.
Return Code:	
`FE_NOT_AVAILABLE`	If you use `nMethod`=0 then x must be above the lowest x value in the function, or for `nMethod`=1 x must be below the highest x value, or else the function can not be evaluated with this method.
FE_FAIL	Evaluation failed. Probably because insufficient (or no) points have been defined in the function.
Remarks/Usage:	
The Closest method returns one of the endpoints of the function if you specify an x value that is outside the defined range of the function. The Interpolate method uses the first/last two points in the function and linearly extrapolates to find values beyond the defined range of the function. If you attempt to interpolate on a function with only one defined value, you will simply get that value for every location.	
Example:	
None	

GFXArrow Objects

GFXArrow objects correspond to user-creatable graphics arrows in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for nodes. For more information, see **Common Entity Properties and Methods**.

Simply creating user-graphics objects will not make them visible in your model. You must also use the methods described in **User Graphics Methods**, to select the data/set to display and the views where you want to display your graphics.

Use the `feGFXArrow` method of the Femap Application object to create GFXArrow objects.

Unpublished work. © 2025 Siemens

GFXArrow Object Properties

User graphics data can be stored in any Set, but only one set can be selected for display at a time.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`REAL8 x`	These are the coordinates of the end of an arrow. They are always specified in global rectangular coordinates.
`REAL8 y`	
`REAL8 z`	
`REAL8 dx`	These are the direction components of the arrow. They are always specified in global rectangular coordinates.
`REAL8 dy`	
`REAL8 dz`	
`REAL8 length`	The length of the arrow.
`REAL8 lengthmode`	0=Absolute (length is a fixed size in model units), 1=Scaled (length is scaled to view vector size).
`INT4 layer`	The ID of the layer associated with the arrow.
`INT4 color`	The arrow color.
`INT4 style`	The arrow/arrowhead style (0=Line, 1=Solid, 2=Double-headed line, 3=Double-headed Solid, 4=Reversed Double-headed line, 5=Reversed Double-headed Solid).

`REAL8 xyz[0..3]`
or
`VARIANT vxyz`
	Accesses the coordinates of the end of an arrow as an array or Variant.

Unpublished work. © 2025 Siemens

Property	Description

`REAL8 dxyz[0..3]`
 	
or
 	
`VARIANT vdxyz`
	Accesses the direction components of the end of an arrow as an array or Variant.

`REAL8 location`	Controls location of arrowhead. 0.0=base of arrow at "end of arrow" location, 1.0=tip of arrow at *end of arrow*. Must be set and `Put` after using `PutAll`, `PutAllArray`, or `PutCoordArray`.

GFXArrow Object Methods

PutAll

`PutAll` **`( entID, x, y, z, dx, dy, dz, length, lengthmode, layer, color, style )`**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`REAL8 x`	Refer to the property descriptions for each of these items.
`REAL8 y`	
`REAL8 z`	
`REAL8 dx`	
`REAL8 dy`	

Unpublished work. © 2025 Siemens

`REAL8 dz`	
`REAL8 length`	
`INT4 lengthmode`	
`INT4 layer`	
`INT4 color`	
`INT4 style`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually. If attempting to use the *location* property of the GFXArrow object in conjunction with `PutAll`, it must be done afterwards by setting the *location* value, then using `Put` again.	
Example:	
None	

PutAllArray

`PutAllArray`
`( numAr, entID, xyz, dxyz, length, lengthmode, layer, color, style )`
Description:
This method stores multiple GFXArrow entities with the specified data.

Unpublished work. © 2025 Siemens

Input:	
`INT4 numAr`	The number of GFXArrows to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numAr-1]`	The IDs of the GFXArrows to store.
`REAL8 xyz[0..(3*numar)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 3 entries for a entity, followed by the 3 for the next entity, ...The dxyz array is stored similarly. The other arrays have just one entry per arrow. All must be in global rectangular coordinates.
`REAL8 dxyz[0..(3*numar)-1]`	
`REAL8 length[0..numAr-1]`	
`INT4 lengthmode[0..numAr-1]`	
`INT4 layer[0..numAr-1]`	
`INT4 color[0..numAr-1]`	
`INT4 style[0..numAr-1]`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This is the fastest `Put` method if you have a large number of items to create, but still need full control over all options. All data can be accumulated in arrays and then transferred in one call. If attempting to use the *location* property of the GFXArrow object in conjunction with `PutAllArray`, it must be done afterwards by using `Get` for each individual arrow, setting the *location* value, then using `Put` again for each arrow.	
Example:	
None	

PutCoordArray

Unpublished work. © 2025 Siemens

PutCoordArray

(numAr, xyz, dxyz, length, lengthmode, layer, color, style)

Description:

This is a simpler version of `PutAllArray` that allows multiple entities to be stored.

Input:

`INT4 numAr`	The number of GFXArrows to create (i.e. the number of entries in the arrays).
`REAL8 xyz[0..(3*numar)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 3 entries for a entity, followed by the 3 for the next entity, ...The dxyz array is stored similarly. The other arrays have just one entry per arrow. All must be in global rectangular coordinates.
`REAL8 dxyz[0..(3*numar)-1]`	
`REAL8 length[0..numAr-1]`	
`INT4 lengthmode`	
`INT4 layer`	
`INT4 color`	
`INT4 style`	

Output:

None	None

Return Code:

`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.

Remarks/Usage:

This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates are input as an array.

If attempting to use the *location* property of the GFXArrow object in conjunction with `PutCoordArray`, it must be done afterwards by using `Get` for each individual arrow, setting the *location* value, then using `Put` again for each arrow.

Unpublished work. © 2025 Siemens

Example:
None

DeleteAll

`DeleteAll`
 	
`( bAllSets, nSetID )`	

Description:	
Deletes all GFXArrows in one or more sets.	
Input:	
`BOOL bAllSets`	If True, all entities, in all sets are deleted. If False, then all entities in Set `nSetID` are deleted.
`INT4 nSetID`	The setID to delete if `bAllSets`=False.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to delete entities.
Remarks/Usage:	
None	
Example:	
None	

GFXLine Objects

GFXLine objects correspond to user-creatable graphics lines in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for nodes. For more information, see **Common Entity Properties and Methods**.

Unpublished work. © 2025 Siemens

Simply creating user-graphics objects will not make them visible in your model. You must also use the methods described in **User Graphics Methods**, to select the data/set to display and the views where you want to display your graphics.

Use the `feGFXLine` method of the Femap Application object to create GFXLine objects.

GFXLine Object Properties

User graphics data can be stored in any Set, but only one set can be selected for display at any time.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`REAL8 x1`	These are the coordinates of the first end of a line. They are always specified in global rectangular coordinates.
`REAL8 y1`	
`REAL8 z1`	
`REAL8 x2`	These are the coordinates of the second end of a line. They are always specified in global rectangular coordinates.
`REAL8 y2`	
`REAL8 z2`	
`INT4 layer`	The ID of the layer associated with the line.
`INT4 color`	The line color.

`REAL8 xyz[0..5]`
or
`VARIANT vxyz`
	Accesses the coordinates of a line as an array or Variant. End 1 is store first, followed by end 2.

Unpublished work. © 2025 Siemens

GFXLine Object Methods

PutAll

PutAll **(entID, x1, y1, z1, x2, y2, z2, layer, color)**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`REAL8 x1`	Refer to the property descriptions for each of these items.
`REAL8 y1`	
`REAL8 z1`	
`REAL8 x2`	
`REAL8 y2`	
`REAL8 z2`	
`INT4 layer`	
`INT4 color`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.
Example:
None

PutAllArray

`PutAllArray`	
`( numLn, entID, xyz, layer, color )`	
Description:	
This method stores multiple GFXLine entities with the specified data.	
Input:	
`INT4 numLn`	The number of GFXLines to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numLn-1]`	The IDs of the GFXLines to store.
`REAL8 xyz[0..(2*3*numLn)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 6 entries for a entity (3 for the first end then 3 for the second end), followed by the 6 for the next entity, ... All must be in global rectangular coordinates.
`INT4 layer[0..numLn-1]`	
`INT4 color[0..numLn-1]`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This is the fastest `Put` method if you have a large number of items to create, but still need full control over all options. All data can be accumulated in arrays and then transferred in one call.	

Unpublished work. © 2025 Siemens

Example:
None

PutCoordArray

PutCoordArray **(numLn, xyz, layer, color)**	
Description:	
This is a simpler version of `PutAllArray` that allows multiple entities to be stored.	
Input:	
`INT4 numLn`	The number of GFXLines to create (i.e. the number of entries in the arrays).
`REAL8 xyz[0..(2*3*numLn)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 6 entries for a entity (3 for the first end then 3 for the second end), followed by the 6 for the next entity, ... All must be in global rectangular coordinates.
`INT4 layer`	
`INT4 color`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates are input as an array.	
Example:	
None	

Unpublished work. © 2025 Siemens

DeleteAll

`DeleteAll`
 	
`( bAllSets, nSetID )`	

Description:	
Deletes all GFXLines in one or more sets.	
Input:	
`BOOL bAllSets`	If True, all entities, in all sets are deleted. If False, then all entities in Set `nSetID` are deleted.
`INT4 nSetID`	The setID to delete if `bAllSets`=False.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to delete entities.
Remarks/Usage:	
None	
Example:	
None	

GFXPoint Objects

GFXPoint objects correspond to user-creatable graphics points in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for nodes. For more information, see **Common Entity Properties and Methods**.

Simply creating user-graphics objects will not make them visible in your model. You must also use the methods described in **User Graphics Methods**, to select the data/set to display and the views where you want to display your graphics.

Use the `feGFXPoint` method of the Femap Application object to create GFXPoint objects.

Unpublished work. © 2025 Siemens

GFXPoint Object Properties

User graphics data can be stored in any Set, but only one set can be selected for display at any time.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`REAL8 x`	These are the coordinates of the point. They are always specified in global rectangular coordinates.
`REAL8 y`	
`REAL8 z`	
`INT4 layer`	The ID of the layer associated with the point.
`INT4 color`	The point color.
`INT4 symbol`	The symbol to be used to display the point. (0=Point, 1=Square, 2=Filled Square, 3=Diamond, 4=X, 5=Triangle, 6=Small Triangle, 7=Cross, 8=Circle, 9=Filled Circle, 10=Blank).

`REAL8 xyz[0..3]`
or
`VARIANT vxyz`
	Accesses all three coordinates of a point as an array or Variant. Same as using x, y, and z independently.

GFXPoint Object Methods

PutAll

Unpublished work. © 2025 Siemens

PutAll **(entID, x, y, z, layer, color, symbol)**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`REAL8 x`	Refer to the property descriptions for each of these items.
`REAL8 y`	
`REAL8 z`	
`INT4 layer`	
`INT4 color`	
`INT4 symbol`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.	
Example:	
None	

PutAllArray

Unpublished work. © 2025 Siemens

PutAllArray

(numPoint, entID, xyz, layer, color, symbol)

Description:

This method stores multiple GFXPoint entities with the specified data.

Input:

`INT4 numPoint`	The number of GFXPoints to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numPoint-1]`	The IDs of the GFXPoints to store.
`REAL8 xyz[0..(3*numPoint)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 3 entries for a entity, followed by the 3 for the next entity, ... All must be in global rectangular coordinates.
`INT4 layer[0..numPoint-1]`	
`INT4 color[0..numPoint-1]`	
`INT4 symbol[0..numPoint-1]`	

Output:

None	None

Return Code:

`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.

Remarks/Usage:

This is the fastest `Put` method if you have a large number of items to create, but still need full control over all options. All data can be accumulated in arrays and then transferred in one call.

Example:

None

PutCoordArray

Unpublished work. © 2025 Siemens

PutCoordArray	
(numPt, xyz, layer, color, symbol)	
Description:	
This is a simpler version of `PutAllArray` that allows multiple entities to be stored.	
Input:	
`INT4 numPoint`	The number of GFXPoints to create (i.e. the number of entries in the arrays).
`REAL8 xyz[0..(3*numPoint)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 3 entries for a entity, followed by the 3 for the next entity. All must be in global rectangular coordinates.
`INT4 layer`	
`INT4 color`	
`INT4 symbol`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates are input as an array.	
Example:	
None	

DeleteAll

Unpublished work. © 2025 Siemens

DeleteAll **(bAllSets, nSetID)**	
Description:	
Deletes all GFXPoints in one or more sets.	
Input:	
`BOOL bAllSets`	If True, all entities, in all sets are deleted. If False, then all entities in Set `nSetID` are deleted.
`INT4 nSetID`	The setID to delete if `bAllSets`=False.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to delete entities.
Remarks/Usage:	
None	
Example:	
None	

GFXQuad4 Objects

GFXQuad4 objects correspond to user-creatable graphics quads in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for nodes. For more information, see **Common Entity Properties and Methods**.

Simply creating user-graphics objects will not make them visible in your model. You must also use the methods described in **User Graphics Methods**, to select the data/set to display and the views where you want to display your graphics.

Use the `feGFXQuad4` method of the Femap Application object to create GFXQuad4 objects.

Unpublished work. © 2025 Siemens

GFXQuad4 Object Properties

User graphics data can be stored in any Set, but only one set can be selected for display at any time.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`REAL8 xyz[0..11]`
 	
or
 	
`VARIANT vxyz`
	The coordinates of the corners of a quad. Values are stored in the order (x1,y1,z1,x2,y2,z2,...). All values are specified in global rectangular coordinates.

`REAL8 normal[0..11]`
 	
or
 	
`VARIANT vnormal`
	The components of the normal vector to the quad at each corner. Stored in thes same order as the coordinates. These values are used for smooth shading and are only used if `autonormal`=False. All values are specified in global rectangular coordinates.

`INT4 layer`	The ID of the layer associated with the quad.
`INT4 fillcolor`	The color used to fill the interior of the quad.
`INT4 edgecolor`	The color used for the edge lines of the quad.
`INT4 edgeflags`	Any combination of the following (0=No Edges, 0x0F=All Edges Shown, 0x01=Edge 1 On, 0x02=Edge 2 On, 0x04=Edge 3 On, 0x08=Edge 4 On - for quads). These are used to control which edges of the quad are drawn. By turning off edges, multiple quads (and quads) can be combined to appear as one larger shape.
`BOOL autonormal`	If True, a single normal is computed for the quad and used for shading. If False, you must supply normals in the *normal* property for shading.

GFXQuad4 Object Methods

Unpublished work. © 2025 Siemens

PutAll

PutAll

(entID, xyz, edgeflags, normal, autonormal, layer, fillcolor, edgecolor)

Description:

This method stores an entity with the specified ID.

Input:

INT4 entID	The ID of the entity to store.
REAL8 xyz[0..11]	Refer to the property descriptions for each of these items.
INT4 edgeflags	
REAL8 normal[0..11]	
BOOL autonormal	
INT4 layer	
INT4 fillcolor	
INT4 edgecolor	

Output:

None	None

Return Code:

FE_FAIL	Unable to store the entity with the specified ID.

Remarks/Usage:

Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.

Unpublished work. © 2025 Siemens

Example:
None

PutAllArray

PutAllArray **(numQuad, entID, xyz, edgeflags, normal, autonormal, layer, fillcolor, edgecolor)**	
Description:	
This method stores multiple GFXQuad4 entities with the specified data.	
Input:	
`INT4 numQuad`	The number of GFXQuad4s to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numQuad-1]`	The IDs of the GFXQuad4s to store.
`REAL8 xyz[0..(4*3*numQuad)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 12 entries for a entity (3 for the first corner, 3 for the 2nd, ...), followed by the 12 for the next entity. The normal array is stored similarly, but contains the components of the normal vectors. All other arrays contain one entry per quad. All must be in global rectangular coordinates.
`INT4 edgeflags[0..numQuad-1]`	
`REAL8 normal [0..(4*3*numQuad)-1]`	
`BOOL autonormal[0..numQuad-1]`	
`INT4 layer[0..numQuad-1]`	
`INT4 fillcolor[0..numQuad-1]`	
`INT4 edgecolor[0..numQuad-1]`	
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.

Remarks/Usage:

This is the fastest `Put` method if you have a large number of items to create, but still need full control over all options. All data can be accumulated in arrays and then transferred in one call.

Example:

None

PutCoordNormalArray

`PutCoordNormalArray`

`( numQuad, xyz, edgeflags, normal, autonormal, layer, fillcolor, edgecolor )`

Description:

This is a simpler version of `PutAllArray` that allows multiple entities to be stored, and still allows you to specify the quad normal vectors.

Input:

`INT4 numQuad`	The number of GFXQuad4s to create (i.e. the number of entries in the arrays).
`REAL8 xyz[0..(4*3*numQuad)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 12 entries for a entity (3 for the first corner, 3 for the 2nd, ...), followed by the 12 for the next entity, ... The normal array is stored similarly, but contains the components of the normal vectors. All other arrays contain one entry per quad. All must be in global rectangular coordinates.
`INT4 edgeflags[0..numQuad-1]`	
`REAL8 normal [0..(4*3*numQuad)-1]`	
`BOOL autonormal`	
`INT4 layer`	
`INT4 fillcolor`	
`INT4 edgecolor`	

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates, edgeflags and normals are input as arrays.	
Example:	
None	

PutCoordArray

`PutCoordArray` **`( numQuad, xyz, layer, fillcolor, edgecolor )`**	
Description:	
This is a simpler version of PutAllArray that allows multiple entities to be stored.	
Input:	
`INT4 numQuad`	The number of GFXQuad4s to create (i.e. the number of entries in the arrays).
`REAL8 xyz[0..(4*3*numQuad)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 12 entries for a entity (3 for the first corner, 3 for the 2nd, ...), followed by the 12 for the next entity, ... All must be in global rectangular coordinates.
`INT4 layer`	
`INT4 fillcolor`	
`INT4 edgecolor`	
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates are input as an array.	
Example:	
None	

DeleteAll

`DeleteAll` **`( bAllSets, nSetID )`**	
Description:	
Deletes all GFXQuad4s in one or more sets.	
Input:	
`BOOL bAllSets`	If True, all entities, in all sets are deleted. If False, then all entities in Set `nSetID` are deleted.
`INT4 nSetID`	The setID to delete if `bAllSets`=False.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to delete entities.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GFXTria3 Objects

GFXTria3 objects correspond to user-creatable graphics triangles in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for nodes. For more information, see **Common Entity Properties and Methods**.

Simply creating user-graphics objects will not make them visible in your model. You must also use the methods described in **User Graphics Methods**, to select the data/set to display and the views where you want to display your graphics.

Use the `feGFXTria3` method of the Femap Application object to create GFXTria3 objects.

GFXTria3 Object Properties

User graphics data can be stored in any Set, but only one set can be selected for display at any time.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`REAL8 xyz[0..8]`
 	
or
 	
`VARIANT vxyz`
	The coordinates of the corners of a triangle. Values are stored in the order (x1,y1,z1,x2,y2,z2,...). All values are specified in global rectangular coordinates.

`REAL8 normal[0..8]`
 	
or
 	
`VARIANT vnormal`
	The components of the normal vector to the triangle at each corner. Stored in thes same order as the coordinates. These values are used for smooth shading and are only used if `autonormal`=False. All values are specified in global rectangular coordinates.

`INT4 layer`	The ID of the layer associated with the triangle.
`INT4 fillcolor`	The color used to fill the interior of the triangle.
`INT4 edgecolor`	The color used for the edge lines of the triangle.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 edgeflags`	Any combination of the following (0=No Edges, 0x0F=All Edges Shown, 0x01=Edge 1 On, 0x02=Edge 2 On, 0x04=Edge 3 On, 0x08=Edge 4 On - for quads). These are used to control which edges of the triangle are drawn. By turning off edges, multiple triangles (and quads) can be combined to appear as one larger shape.
`BOOL autonormal`	If True, a single normal is computed for the triangle and used for shading. If False, you must supply normals in the *normal* property for shading.

GFXTria3 Object Methods

PutAll

`PutAll`	
`( entID, xyz, edgeflags, normal, autonormal, layer, fillcolor, edgecolor )`	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`REAL8 xyz[0..8]`	Refer to the property descriptions for each of these items.
`INT4 edgeflags`	
`REAL8 normal[0..8]`	
`BOOL autonormal`	
`INT4 layer`	

Unpublished work. © 2025 Siemens

`INT4 fillcolor`	
`INT4 edgecolor`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.	
Example:	
None	

PutAllArray

`PutAllArray` **`( numTria, entID, xyz, edgeflags, normal, autonormal, layer, fillcolor, edgecolor )`**	
Description:	
This method stores multiple GFXTria3 entities with the specified data.	
Input:	
`INT4 numTria`	The number of GFXTria3s to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numTria-1]`	The IDs of the GFXTria3s to store.
`REAL8 xyz[0..(3*3*numTria)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 9 entries for a entity (3 for the first corner, 3 for the 2nd, ...), followed by the 9 for the next entity, ... The normal array is stored similarly, but contains the components of
`INT4 edgeflags[0..numTria-1]`	

Unpublished work. © 2025 Siemens

`REAL8 normal [0..(3*3*numTria)-1]`	the normal vectors. All other arrays contain one entry per triangle. All must be in global rectangular coordinates.
`BOOL autonormal[0..numTria-1]`	
`INT4 layer[0..numTria-1]`	
`INT4 fillcolor[0..numTria-1]`	
`INT4 edgecolor[0..numTria-1]`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This is the fastest `Put` method if you have a large number of items to create, but still need full control over all options. All data can be accumulated in arrays and then transferred in one call.	
Example:	
None	

PutCoordNormalArray

`PutCoordNormalArray`
`( numTria, xyz, edgeflags, normal, autonormal, layer, fillcolor, edgecolor )`
Description:
This is a simpler version of `PutAllArray` that allows multiple entities to be stored, and still allows you to specify the triangle normal vectors.
Input:

`INT4 numTria`	The number of GFXTria3s to create (i.e. the number of entries in the arrays).

Unpublished work. © 2025 Siemens

`REAL8 xyz[0..(3*3*numTria)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 9 entries for a entity (3 for the first corner, 3 for the 2nd, ...), followed by the 9 for the next entity, ... The normal array is stored similarly, but contains the components of the normal vectors. All other arrays contain one entry per triangle. All must be in global rectangular coordinates.
`INT4 edgeflags[0..numTria-1]`	
`REAL8 normal [0..(3*3*numTria)-1]`	
`BOOL autonormal`	
`INT4 layer`	
`INT4 fillcolor`	
`INT4 edgecolor`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates, edgeflags and normals are input as arrays.	
Example:	
None	

PutCoordArray

`PutCoordArray`
`( numTria, xyz, layer, fillcolor, edgecolor )`
Description:
This is a simpler version of `PutAllArray` that allows multiple entities to be stored.

Unpublished work. © 2025 Siemens

Input:	
`INT4 numTria`	The number of GFXTria3s to create (i.e. the number of entries in the arrays).
`REAL8 xyz[0..(3*3*numTria)-1]`	Refer to the property descriptions for each of these items. xyz is an array of the entity coordinates. In the array, there must be 9 entries for a entity (3 for the first corner, 3 for the 2nd, ...), followed by the 9 for the next entity, ... All must be in global rectangular coordinates.
`INT4 layer`	
`INT4 fillcolor`	
`INT4 edgecolor`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID. Probably unable to allocate memory for all of the arrays.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. Entities are simply stored in the next available ID, and the same layer, color... are used for every entity - only the coordinates are input as an array.	
Example:	
None	

DeleteAll

`DeleteAll`
`( bAllSets, nSetID )`
Description:
Deletes all GFXTria3s in one or more sets.
Input:

Unpublished work. © 2025 Siemens

`BOOL bAllSets`	If True, all entities, in all sets are deleted. If False, then all entities in Set `nSetID` are deleted.
`INT4 nSetID`	The setID to delete if `bAllSets`=False.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to delete entities.
Remarks/Usage:	
None	
Example:	
None	

GlobalPly Objects

Global Ply objects correspond to the global plys in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for GlobalPly objects. For more information, see **Common Entity Properties and Methods**.

Use the `feGlobalPly` method of the Femap Application object to create GlobalPly objects.

GlobalPly Object Properties

GlobalPlys are always stored with a `SetID=1`, and the ID equal to the global ply ID.

Property	Description
`STRING title`	Global Ply title.
`REAL8 thickness`	The global ply thickness.
`INT4 matlID`	The ID of the material referenced by the global ply.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 failuretheory`	(OBSOLETE) Failure Theory for global ply. 0=From Property, 1=Hill, 2=Hoffman, 3=Tsai-Wu, 4=Max Strain.
`BOOL validate`	When True, flag to validate global ply will be turned on.

GlobalPly Object Methods

There are no GlobalPly object-specific methods. Only the standard entity methods are available.

GlobalStep Objects

GlobalStep objects correspond to the global step entities which can be referenced by analysis steps within subcases which are within analysis sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for GlobalStep objects. For more information, see **Common Entity Properties and Methods**.

Use the `feGlobalStep` method of the Femap Application object to create GlobalStep objects.

Note

Currently, GlobalStep objects are only used for MSC Nastran Implicit Nonlinear Analysis (SOL 400), which is the only analysis type that used AnalysisStep Objects.

GlobalStep Object Properties

GlobalStep Objects are stored with their `globalstepID` equal to the analysis step that you are trying to access.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
Global Step Options - General	
`STRING GlobalStepTitle[80]`	Title of analysis Global step (maximum 80 characters).

Unpublished work. © 2025 Siemens

Property	Description
`STRING GlobalStepLabel[80]`	Label of analysis step (maximum 80 characters).
`INT4 globalstepID`	ID of Global Step.
`enum GlobalStepAnalysisType`	Analysis Type for Global Step (only used by certain solvers for certain analysis types). See **Analysis Types** in **Femap Constants** for available enumerations.

`INT4 BCSet[0..20]`
or
`VARIANT vBCSet`
	SetIDs of load and constraint sets for this step: [0]=Constraints, [1]=Constraint Equations, [2]=Loads, [3]=Initial Conditions, [4]=Master DOF, [5]=Base/Support DOF, [6]=Omit DOF, [7]=QSET DOF, [8]=CSET DOF, [9]=BSET DOF, [10]=Bolt Preloads, [11]=SUPORT1 DOF, [12]=Temperatures, [13]=Connection Group (MSC Only), [14]=Loads (Time), [15]=Temperatures (Init), [16]=Glue Set, [17]=Contact Set, others not currently used.

`INT4 Output[0..30]`
or
`VARIANT vOutput`
	Output requests for this step: 0=off,-1=full model, >0=groupID, [0]=Nodal Applied Load, [1]=Nodal Reaction Force, [2]=Nodal Displacement, [3]=Nodal Velocity, [4]=Nodal Acceleration, [5]=Nodal Temperature, [6]=Nodal Heat Flux, [7]=Nodal Constraint Equation Forces, [8]=Nodal Force Balance, [9]=Nodal Kinetic Energy, [15]=Element Force, [16]=Element Stress, [17]=Element Strain, [18]=Element Plastic Strain, [19]=Element Strain Energy, [20]=Element Heat Flux, [21]=Element Enthalpy, [22]=Element Enthalpy Rate, [23]=Element Contact, [24]=Element Temperature, [25]=Kinetic Energy, [26]=Energy Loss, [27]=Fluid Pressure, others not currently used.

`STRING Echo[80]`	String for ECHO option (maximum 80 characters).
`BOOL SkipStandard`	Flag to skip standard output for this step.
`BOOL NasGlobalAlternateTextLocation`	Flag to use Start Text Inside the Global Step option.
`INT4 StartText`	ID of starting text.
`INT4 EndText`	ID of ending text.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 Destination`	Output destination: 0=Default, 1=Print, 2=Post, 3=Print+Post, 4=Punch, 5=Punch+Post.
`INT4 Imaginary`	Format for complex output: 0=Magnitude/Phase, 1=Real/Imaginary.
`INT4 CornerOutput`	Flag to request output at element corners.
`INT4 IntegrationOutput`	Flag to request output at element integration points.
`BOOL NasCurvatureStrain`	Flag to request Curvature Strain Results. True = Curvature, False = Fiber.
NASTRAN Global Step Options	
Analysis Step Options - NASTRAN Specific	
`BOOL NasOmodesOn`	Flag indicating an OMODES request has been defined.
`STRING NasOmodes[160]`	Expression used to select OMODES output.
NASTRAN Modal Analysis - Global Step options	
`BOOL NasModeOn`	Flag indicating NASTRAN Modal Options have been defined for subcase.
`INT4 NasModeEigrSet`	SetID to be used for EIGR.
`INT4 NasModeMethod`	Modal Solution method: 0=Givens,1=Modified Givens, 2=Inverse Power, 3=Inverse Power/ Strum, 4=Householder, 5=Modified Householder, 6=Lanczos, 7=Hessenberg, 8=Complex Inverse Power, 9=Complex Lanczos.
`BOOL NasModeSolutionType`	Solution type: False=Direct, True=Modal.

`REAL8 NasModeFreqRange[2]`
or
`VARIANT vNasModeFreqRange`
	Frequency range of interest, [0]=Min, [1]=Max.

`INT4 NasModeEstRoots`	Estimated number of eigenvalues.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasModeDesiredRoots`	Number of desired eigenvalues/modes.
`INT4 NasModeNormOpt`	Normalization method: 0=Mass,1=Max, 2=Point.
`INT4 NasModeNormNode`	Node ID for use with point normalization method.
`INT4 NasModeNormDOF`	Nodal DOF for use with point normalization method.
`INT4 NasModeMassForm`	Mass matrix formulation: 0=Default, 1=Lumped, 2=Coupled.
`BOOL NasModesSkipEigr`	If True, skips writing the EIGR card for specifying modal analysis options.

`REAL8 NasModeImagFreqRange[2]`
 	
or
 	
`VARIANT vNasModeImagFreqRange`
	For Complex Modes analysis, specifies the imaginary part of the Frequency Range of interest.

`REAL8 NasModesComplexConverge`	Specifies Complex Modes convergence value.
`REAL8 NasModesComplexRegionWidth`	Specifies Region Width for Complex Modes analysis.
`REAL8 NasModeDampOverall`	Specifies PARAM,G.
MSC NASTRAN Implicit Nonlinear (SOL 400) - Control Parameters	
`BOOL NasInlStepOptOn`	Flag indicating Control Parameters for Implicit Nonlinear Analysis have been defined for the subcase.
`BOOL NasInlStepOptSkipNLSTEP`	Flag to disable writing of NLSTEP for this subcase.
`INT4 NasInlStepOptID`	ID of NLSTEP for this subcase.
Control Parameters - Step Parameters	
`REAL8 NasInlStepOptTOTTIM`	Total Time (TOTTIM) value
`INT4 NasInlStepOptCTRLDEF`	Specifies Nonlinearity (CTRLDEF) option: 0=Default

Unpublished work. © 2025 Siemens

Property	Description
	1=Linear 2=Mildly Nonlinear 3=Severely Nonlinear
Control Parameters - General Parameters	
`INT4 NasInlStepOptMAXITER`	Max Iterations/Increment (MAXITER) value.
`INT4 NasInlStepOptMINITER`	Min Iterations/Increment (MINITER) value.
`INT4 NasInlStepOptMAXBIS`	Max Bisections for Step (MAXBIS) value.
`BOOL NasInlStepOptCREEP`	Flag to activate Include Creep (CREEP) option.
Control Parameters - Analysis Parameter	
`INT4 NasInlStepOptLoadStepping`	Specifies Increment Type option: 0=Fixed 1=Adaptive 2=Arc-Length
Control Parameters - Analysis Parameters - Fixed Time Stepping Parameters	
`INT4 NasInlStepOptNINC`	Number of Increments (NINC) value.
`INT4 NasInlStepOptNO`	Output Interval (NO) value.
Control Parameters - Analysis Parameters - Adaptive Load Stepping Parameters	
`REAL8 NasInlStepOptDTINITF`	Initial Fraction (DTINITF) value.
`REAL8 NasInlStepOptDTMINF`	Min Fraction (DTMINF) value.
`REAL8 NasInlStepOptDTMAXF`	Max Fraction (DTMAXF) value.
`REAL8 NasInlStepOptSFACT`	Increase Factor (SFACT) value.
`INT4 NasInlStepOptADJUST`	Auto Skip Factor (ADJUST) value.
`INT4 NasInlStepOptMSTEP`	Dominant Period Resp. Steps (MSTEP) value.
`REAL8 NasInlStepOptRB`	Adaptive Time Step Bound (RB) value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NasInlStepOptNDESIR`	Desired Iterations/Increments (NDESIR) value.
`INT4 NasInlStepOptNSMAX`	Max Increments (NSMAX) value.
`REAL8 NasInlStepOptUTOL`	Displacement Tolerance (UTOL) value.
`INT4 NasInlStepOptIDAMP`	Specifies Static Damping (IDMAP) option: 0=No Added Damping 4=Time Step Control Using Damping 5=Damping-based Time Step Control with No Damping 6=Add Damping when Minimum Time Step is Reached
`REAL8 NasInlStepOptDAMP`	Damping Ratio (DAMP) value.
`INT4 NasInlStepOptINTOUT`	Specifies Output Flag (INTOUT) option: -1=Last Increment 0=Every Load Increment >0=Equally Spaced Intervals (value is used to specify value for Output Every Nth Step)
Adaptive Load Stepping Parameters - Criteria	
`BOOL NasInlStepOptEnableCriteria`	Flag to turn on the Enable User Criteria option.
`INT4 NasInlStepOptIPHYS`	Specifies Physical Criteria (IPHYS) option: -2=No Auto Physical Criteria; Continue When User Criteria Exceeded. -1=Add Auto Physical Criteria; Continue When User Criteria Exceeded. 1=1..No Auto Physical Criteria; Stop When User Criteria Exceeded. 2=2..Add Auto Physical Criteria; Stop When User Criteria Exceeded.
`INT4 NasInlStepOptLIMTAR`	Specifies Definition (LIMTAR) option: 0=Criteria as Limits 1=Criteria as Targets
`REAL8 NasInlStepOptRSMALL`	Min. Ratio (RSMALL) value

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasInlStepOptRBIG`	Max Ration (RBIG) value
`BOOL NasInlStepOptIncrements[0..9]`	Flag in Status column of row specified by zero-based index value in list of Criteria. When True, Criterion is turned on. 0=Elem. Strain 1=Elem. Plastic Strain 2=Elem. Creep Strain 3=Elem. Normalized Creep Strain 4=Elem. Strain Energy 5=Elem. Stress 6=Elem. Normalized Stress 7=Nodal Temperature 8=Nodal Displacement 9=Nodal Rotation
`INT4 NasInlStepOptGroupIDs[0..9]`	ID of existing Group in Group column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT1_i[0..9]`	Value in Y1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT1_i[0..9]`	Value in M1 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT2_i[0..9]`	Value in Y2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT2_i[0..9]`	Value in M2 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT3_i[0..9]`	Value in Y3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptMT3_i[0..9]`	Value in M3 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
`REAL8 NasInlStepOptYT4_i[0..9]`	Value in Y4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 NasInlStepOptMT4_i[0..9]`	Value in M4 column of row specified by zero-based index value in list of Criteria. For description of each row, See `NasInlStepOptIncrements`.
Control Parameters - Analysis Parameters - Arc-Length Load Stepping Parameters	
`INT4 NasInlStepOptTYPE`	Specifies Constraint Type option: 0=Crisfield 1=Riks 2=Modifed Riks
`REAL8 NasInlStepOptDTINITFA`	Initial Time Step Fraction (DTINITFA) value.
`REAL8 NasInlStepOptMINALR`	Min ArcLenAdjust Ratio (MINALR) value.
`REAL8 NasInlStepOptMAXALR`	Max ArcLenAdjust Ratio (MAXALR) value.
`REAL8 NasInlStepOptSCALEA`	Scale Factor (SCALEA) value.
`INT4 NasInlStepOptNDESIRA`	Desired Iterations (NDESIRA) value.
`INT4 NasInlStepOptNSMAXA`	Max Increments (NSMAXA) value.
Control Parameters - Analysis Parameters - Mechanical Analysis Parameters	
`BOOL NasInlStepOptMECH`	Flag to enable the Enable MECH option.
`INT4 NasInlStepOptKMETHOD`	Specifies Method (KMETHOD) option: 0=Full Newton-Raphson 1=Quasi Newton-Raphson
`INT4 NasInlStepOptKSTEP`	Iteration Interval (KSTEP) value.
`INT4 NasInlStepOptMAXQN`	QN Vectors (MAXQN) value.
`INT4 NasInlStepOptMAXLS`	Max Line Searches (MAXLS) value.
`REAL8 NasInlStepOptLSTOL`	Line Search Tol (LSTOL) value.
`REAL8 NasInlStepOptFSTRESS`	Fraction Limit (FTRESS) value.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL NasInlStepOptConvU`	Flag to turn on Displacement option.
`REAL8 NasInlStepOptEPSU`	Displacement value.
`BOOL NasInlStepOptConvP`	Flag to turn on Load option.
`REAL8 NasInlStepOptEPSP`	Load value.
`BOOL NasInlStepOptConvW`	Flag to turn on Work option.
`REAL8 NasInlStepOptEPSW`	Work value.
`BOOL NasInlStepOptConvV`	Flag to turn on Vector option.
`BOOL NasInlStepOptConvN`	Flag to turn on Length option.
`BOOL NasInlStepOptConvA`	Flag to turn on Automatic Switching option.
`INT4 NasInlStepOptMRCONV`	Specifies Checking (MRCONV) option: 0=Forces, Moments, Displacements, and Rotations 1=Forces, Moments, and Displacements 2=Forces, Displacements, and Rotations 3=Forces and Displacements
MSC Nastran Boundary Conditions	
`INT4 ContactOption`	Boundary Conditions dialog box for MSC Nastran Only. Sets BCONTACT Option (0=Default, 1=Inital Contact, 2=All Body, 3=Auto) .
`INT4 ContactSetType`	Boundary Conditions dialog box for MSC Nastran Only. Sets Contact/Glue Sets option (0=All Connectors, 1=Connection Group, 2=All Body). ID of group for Connection Group set using `BCSet[13]`.

GlobalStep Object Methods

In addition to the standard methods, the following methods are also available.

Unpublished work. © 2025 Siemens

InitGlobalStep

InitGlobalStep **(nCaseID)**	
Description:	
This method initializes the default values for a global step.	
Input:	
`INT4 nCaseID`	The Global Step to be initialized.
Output:	
None	None
Return Code:	
`FE_FAIL`	None
Remarks/Usage:	
None	
Example:	
None	

DeleteGlobalStep

DeleteGlobalStep **(nCaseID)**	
Description:	
Use this method to delete a global step.	
Input:	
`INT4 nCaseID`	The Global Step to be deleted.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	None
Remarks/Usage:	
None	
Example:	
None	

ActiveContactEntities

`ActiveContactEntities` **`( bContact, nConnectorSetID, nRegionSetID, nConPropSetID )`**	
Description:	
This method is used to populate Set Objects with the Connectors, Connection Regions, and Connection Properties that will be exported for Contact or Glue Contact by an analysis step which references this global step. This method takes into consideration the settings in the Contact Sets or Glue Sets section of the Boundary Conditions dialog box for Simcenter SOL 401/402 or setting for Contact/Glue Sets of the Boundary Conditions dialog box for MSC Nastran.	
Input:	
`BOOL bContact`	If True, populates sets with connection entities for Contact. If False, populates sets with connection entities for Glued Contact.
`INT4 nConnectorSetID`	Set ID of Set Object to populate with Connectors which will be exported by this analysis step which references this global step.
`INT4 nRegionSetID`	Set ID of Set Object to populate with Connection Regions which will be exported by this analysis step which references this global step.
`INT4 nConPropSetID`	Set ID of Set Object to populate with Connection Properties which will be exported by this analysis step which references this global step.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	No Connection Regions were detected.
Remarks/Usage:	
It is important to use three uniques Set Objects when using this method, otherwise the IDs of different entity types may contaminate a Set. Also, it is recommend to use *empty sets* to ensure that only the Connection entities from a single analysis step which references this global step populate the three sets.	
Example:	
None	

Group Objects

Group objects correspond to the groups in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Group objects. For more information, see **Common Entity Properties and Methods**.

Use the `feGroup` method of the Femap Application object to create Group objects.

Group Object Properties

Groups are always stored with a `SetID=1`, and the ID equal to the group ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 LayerMode`	The mode to use layers to limit group contents: 0=All Layers, 1=Equal or Above Max, 2=Equal or Below Min, 3=Between, 4=Outside, 5=Single Layer.
`INT4 LayerMin`	The minimum layer ID used to limit layer selection in certain `LayerModes`.
`INT4 LayerMax`	The maximum layer ID used to limit layer selection in certain `LayerModes`.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 CoordClipOn`	Flag used to indicate that coordinate clipping is being used.
`INT4 CoordClipMode`	The mode used for coordinate clipping: 0=Greater, 1=Less, 2=Between, 3=Outside.
`REAL8 CoordClipMin`	The minimum coordinate location, used to limit coordinate clipping in certain `CoordClipModes`.
`REAL8 CoordClipMax`	The maximum coordinate location, used to limit coordinate clipping in certain `CoordClipModes`.
`INT4 CoordClipCSys`	The coordinate system in which coordinate clipping is being performed. The `CoordClipMin` and `CoordClipMax` values refer to coordinates in this coordinate system.
`INT4 CoordClipDir`	The coordinate axis along which coordinate clipping occurs: 0=X (or R), 1=Y (or theta), 2=Z (or phi).
`INT4 PlaneClipMode`	The mode used for plane clipping: 0=Off, 1=Screen, 2=Plane, 3=Volume.
`BOOL PlaneClipInside`	Flag indicating whether to retain the inside or outside of the clipping region.

`INT4 PlaneClipOn[0..5]`
or
`VARIANT vPlaneClipOn`
	Flag used to indicate whether each clipping plane is being used.

`BOOL PlaneClipNeg[0..5]`
or
`VARIANT vPlaneClipNeg`
	Flags indicating whether to use the negative sides of the planes for clipping.

`REAL8 PlaneClipBase[0..5][0..2]`
or
`VARIANT vPlaneClipBase`
	The coordinate locations defining a point on each of the clipping planes. The first index indicates the clipping plane. The second index chooses the x, y, or z coordinates. In global rectangular.

Unpublished work. © 2025 Siemens

Property	Description

`REAL8 PlaneClipDir[0..5][0..2]`
 	
or
 	
`VARIANT vPlaneClipDir`
	The components of vectors normal to each of the clipping planes. The first index indicates the clipping plane. The second index chooses the x,y or z components. In global rectangular.

`STRING title`	The group title (maximum 79 characters).
`BOOL Renumber`	Flag indicating whether this group should be renumbered when other entities are renumbered.
`BOOL NeedEval`	Flag indicating that the current group needs to be evaluated before use. This is automatically set to True every time you `Put` the group to the database. Evaluation computes the list of objects selected into a group by rules, clipping, or any other selection approach.
`BOOL EvalAlways`	If you set this to True, then when you `Put` a group, it will be marked so that it will be automatically evaluated every time it is used. With this set to False, the default, the group will be evaluated once, and then never again until it is modified.
`BOOL Condense`	Set this to True if you want the group to be condensed when you `Put` it back into the database. Condensing means converting all possible selection methods into equivalent, by ID rules. To see the effect of this operation, you must `Put` the group.
`BOOL ReduceToExisting`	Set this to True if you want the group to be reduced to existing when you `Put` it back into the database. Reduced to existing means converting all possible selection methods update each rule type to contain its existing entities. To see the effect of this operation, you must `Put` the group.

Group Range Types

All of the Group methods that access ranges require you to specify which range you want to access. The following table provides the `rangeType` values that you should use.

Unpublished work. © 2025 Siemens

Group Range Types					
0	CSys_ID	26	Matl_ID	52	Text_byColor
1	CSys_byDefCSys	27	Matl_onProp	53	Point_byColor
2	CSys_byType	28	Matl_onElem	54	Curve_byColor
3	Point_ID	29	Matl_byType	55	Surface_byColor
4	Point_byDefCSys	30	Prop_ID	56	Volume_byColor
5	Point_onCurve	31	Prop_onElem	57	Solid_byColor
6	Curve_ID	32	Prop_byMatl	58	CSys_byColor
7	Curve_byPoint	33	Prop_byType	59	Node_byColor
8	Curve_onSurface	34	Load_byNode	60	Elem_byColor
9	Surface_ID	35	Load_byElem	61	Prop_byColor
10	Surface_byCurve	36	BCo_ID	62	Matl_byColor
11	Surface_onVolume	37	BEq_byNode	63	Text_byLayer
12	Volume_ID	38	Node_atPoint	64	Point_byLayer
13	Volume_bySurface	39	Node_atCurve	65	Curve_byLayer

Unpublished work. © 2025 Siemens

Group Range Types					
14	Text_ID	40	Node_atSurface	66	Surface_byLayer
15	Not Used	41	Node_atSolid	67	Volume_byLayer
16	Not Used	42	Elem_atPoint	68	Solid_byLayer
17	Node_ID	43	Elem_atCurve	69	CSys_byLayer
18	Node_byDefCSys	44	Elem_atSurface	70	Node_byLayer
19	Node_byOutCSys	45	Elem_atSolid	71	Elem_byLayer
20	Node_onElem	46	Load_byPoint	72	Prop_byLayer
21	Elem_ID	47	Load_byCurve	73	Matl_byLayer
22	Elem_byMatl	48	Load_bySurface	74	Solid_ID
23	Elem_byProp	49	BCo_byPoint	75	Solid_byCurve
24	Elem_byType	50	BCo_byCurve	76	Solid_bySurface
25	Elem_byNode	51	BCo_bySurface	77	Curve_onSolid
78	Surface_onSolid	102	Layup_byMatl	126	AEROSPLINE_BYPANEL
79	Point_byProp	103	Layup_byProp	127	AEROCTRLSURF_ID

Unpublished work. © 2025 Siemens

Group Range Types					
80	`Curve_byProp`	104	`Elem_byLayup`	128	`AEROCTRLSURF_BYCOLOR`
81	`Surface_byProp`	105	`Prop_byLayup`	129	`AEROCTRLSURF_BYLAYER`
82	`Volume_byProp`	106	`Node_bySuperelement`	130	`CSYS_ONPROP`
83	`Solid_byProp`	107	`Load_byRegion`	131	`CSYS_ONCSYS`
84	`Contact_ID`	108	`Contact_byElem`	132	`SURFACE_ONREGION`
85	`Contact_byColor`	109	`Contact_byNode`	133	`CURVE_ONREGION`
86	`Contact_byLayer`	110	`Contact_byCurve`	134	`ELEM_ONREGION`
87	`CSys_onNode`	111	`Contact_bySurface`	135	`NODE_ONREGION`
88	`CSys_onPoint`	112	`Contact_byProp`	136	`PROP_ONREGION`
89	`Elem_byShape`	113	`Elem_byAllNodes`	137	`PROP_ONPOINT`
90	`Node_onElemAsOrient`	114	`Elem_byGlobalPly`	138	`PROP_ONCURVE`
91	`ConnectionProp_ID`	115	`Layup_byGlobalPly`	139	`PROP_ONSURFACE`
92	`ConnectionProp_byColor`	116	`AEROPROP_ID`	140	`PROP_ONSOLID`

Unpublished work. © 2025 Siemens

Group Range Types					
93	ConnectionProp_byLayer	117	AEROPROP_BYCOLOR	141	SOLID_BYTYPE
94	Connector_ID	118	AEROPROP_BYLAYER	142	NODE_BYTYPE
95	Connector_byColor	119	AEROPANEL_ID	143	POINT_ONSURFACE
96	Connector_byLayer	120	AEROPANEL_BYCOLOR	144	POINT_ONSOLID
97	Connector_byProp	121	AEROPANEL_BYLAYER	145	CSYS_ONELEM
98	Contact_onConnector	122	AEROPANEL_BYPROP	146	CONTACT_ONSOLID
99	ConnectionProp_onConnector	123	AEROSPLINE_ID	147	MESHPOINT_ID
100	Connector_byContact	124	AEROSPLINE_BYCOLOR	148	MESHPOINT_BYCOLOR
101	Layup_ID	125	AEROSPLINE_BYLAYER	149	MESHPOINT_BYLAYER
150	MESHPOINT_BYPOINT	159	FGD_ELEM_ATPOINTSOFSOLID	168	FGD_CURVE_FROMELEM
151	_MESHPOINTBYCURVE	160	FGD_ELEM_ATCURVESOFSOLID	169	FGD_SURFAC_FROMNODE
152	MESHPOINT_BYSURFACE	161	FGD_ELEM_ATSURFACESOFSOLID	170	FGD_SURFACE_FROMELEM

Unpublished work. © 2025 Siemens

Group Range Types					
153	`_MESHPOINTBYSOLID`	162	`FGD_JOINT_ID`	171	`FGD_SOLID_FROMNODE`
154	`FGD_ELEM_BYORIENTNODE`	163	`FGD_JOINT_BYCOLOR`	172	`FGD_SOLID_FROMELEM`
155	`FGD_ELEM_ALLNODESONSOLID`	164	`FGD_JOINT_BYLAYER`	173	`FGD_ELEM_BYPLYMATERIAL`
156	`FGD_NODE_ATPOINTSOFSOLID`	165	`FGD_POINT_FROMNODE`	174	`FGD_LAYUP_BYPLYMATERIAL`
157	`FGD_NODE_ATCURVESOFSOLID`	166	`FGD_POINT_FROMELEM`		
158	`FGD_NODE_ATSURFACESOFSOLID`	167	`FGD_CURVE_FROMNODE`		

Group List Types

The Group `List` method returns a set of selected entities. You must, however, specify which list you want to access. The following table provides the `listType` values that you should use.

Group List Types					
0	CSys	10	Property	20	Surface Const.
1	Point	11	Nodal Load	21	Solids
2	Curve	12	Elem Load	22	Connection Regions
3	Surface	13	Constraint	23	Connectors
4	Volume	14	Constraint Equations	24	Connection Properties
5	Text	15	Point Loads	25	Layups

Unpublished work. © 2025 Siemens

Group List Types					
6	Unused	16	Curve Loads	26	Region Loads (Bolt)
7	Node	17	Surface Loads	27	Mesh Point
8	Elem	18	Point Constraints	28	Kinematic Joint
9	Material	19	Curve Constraints		

Group Object Methods

The following methods are available in addition to the standard entity methods.

List

`List`
 	
`( listType )`	

Description:	
This method returns a Set object that contains all of the entities currently in the selected group list.	
Input:	
`INT4 listType`	The type of group list that you want to retrieve. Refer to **Group List Types** for allowable list types.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

This method must be used with a Set statement to define the Set object.

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Const  DELIM = " "
    Dim gr As group
    Dim grID As Long
    grID = 1
    Dim s As femap.Set
    Set gr = App.feGroup

    gr.Get(grID)
    Set s = gr.List(FGR_NODE)

    If s Is Nothing  Then
        App.feAppMessageBox(0, "Group has no nodes")
        End
    End If

    'Print out IDs in set
    Dim msg As String
    Dim ndID As Long

    msg = "Nodes in group " +  Str$(gr.ID)
    App.feAppMessage(FCM_NORMAL, msg)
    msg = ""

    'Get first in set
    ndID = s.First
    msg = Str$(ndID) + DELIM

    ' prints out 10 ids per line
    For i =0 To s.Count-1
         If ( (i+1) Mod 10 = 0)  Or i=s.Count-1 Then
               App.feAppMessage(FCM_NORMAL, msg)

            'get next in set
            ndID = s.Next
            msg = ""
            msg = msg +  Str$(ndID)+ DELIM
         Else
        'get next in set
          ndID = s.Next
          msg = msg +  Str$(ndID)+ DELIM
         End If
    Next i
End Sub
```


Unpublished work. © 2025 Siemens

This sequence creates the “s” Set object with the list of nodes (`listType`=7) that are selected in Group 14.

GetList

`GetList`	
`( listType, setID )`	
Description:	
This method is similar to the `List` method, but instead of returning a new Set object, it simply adds the contents of the list into a Set object that already exists.	
Input:	
`INT4 listType`	The type of group list that you want to retrieve. Refer to **Group List Types** for allowable list types.
`INT4 setID`	The ID of the Set object to fill.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

HasList

Unpublished work. © 2025 Siemens

HasList	
(listType)	
Description:	
This method is used to determine if a list is present in the current Group.	
Input:	
`INT4 listType`	The type of group list that you want to retrieve. Refer to **Group List Types** for allowable list types.
Output:	
None	None
Return Code:	
`FE_FAIL`	The specified list does not exist in the current Group.
`FE_OK`	The specified list does exist in the current Group.
Remarks/Usage:	
None	
Example:	
None	

CountList

CountList	
(listType)	
Description:	
This method returns the number of entities selected in the specified Group list.	
Input:	
`INT4 listType`	The type of group list that you want to retrieve. Refer to **Group List Types** for allowable list types.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`count`	The number of entities selected in the Group List (0 if the list does not exist)
Remarks/Usage:	
None	
Example:	
None	

Range

Range **(rangeType)**	
Description:	
This method returns a Set object that contains all of the entities currently selected by the specified `rangeType`.	
Input:	
`INT4 rangeType`	The type of group range list that you want to retrieve. Refer to **Group List Types** for allowable `rangeTypes`.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:

This method must be used with a Set statement to define the Set object.

```
Sub Main
Const  DELIM = " "
Dim App As femap.model
Set App = feFemap()


Dim gr As Group
Dim grID As Long

grID = 14
Dim s As Set
Set gr = App.feGroup
gr.Get(grID)
Set s = gr.Range(FGD_ELEM_BYMATL)

'Print out IDs in set
Dim msg As String
Dim ndID As Long
msg = "Materials in group " +  Str$(gr.ID)
App.feAppMessage(FCM_NORMAL, msg)
msg = ""

'Get first in set
ndID = s.First
msg = Str$(ndID) + DELIM

' print out 10 ids per line
For i =0 To s.count-1
 If ( (i+1) Mod 10 = 0) Or i=s.count-1 Then
   App.feAppMessage(FCM_NORMAL, msg)

    ndID = s.Next  'get next in set
   msg = ""
   msg = msg +  Str$(ndID)+ DELIM

 Else
    'get next in set
   ndID = s.Next
   msg = msg +  Str$(ndID)+ DELIM
 End If
Next i

End Sub
```

This sequence creates the “s” Set object with the list of materials used to select elements (`rangeType`=22) that are selected in Group 14.

Unpublished work. © 2025 Siemens

RangeReset

RangeReset	
(void)	
Description:	
This method resets the next pointer so that the next call to `RangeNext` will return the first (lowest ID) range.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method does not change the next pointer for the Group object, just for the ranges within the group. The next method will still return the next group.	
Example:	
None	

RangeGetAll

RangeGetAll	
(rangeType, rangeCount, startID, stopID, increment, include)	
Description:	
This method returns all of the ranges of a specific type in the form of arrays.	
Input:	
`INT4 rangeType`	This is the type of range that you are retrieving.

Unpublished work. © 2025 Siemens

Output:	
`INT4 rangeCount`	The number of entries in each of the other arrays. That is, the number of ranges that are returned.
`INT4 startID[0..N]`	The starting ID in the range.
`INT4 stopID[0..N]`	The ending ID in the range. This will be 0 for single entry ranges.
`INT4 increment[0..N]`	The increment to count by between startID and stopID.
`INT4 include[0..N]`	The include flag 0=Remove, 1=Add, Exclude=-1.
Return Code:	
`FE_NOT_AVAILABLE`	The rangeType that you specified is invalid.
`FE_FAIL`	Unable to retrieve the data for ranges.
Remarks/Usage:	
Use the `Reset` method to go to the beginning of the lists of ranges, then keep calling `RangeNext` to retrieve entries from a list of ranges one-by-one. Alternatively, you can call `RangeGetAll` to get all of the entries in one call.	
Example:	
None	

RangeNext

`RangeNext`
`( rangeType,  startID, stopID, increment, include )`
Description:
This method returns the values from the next range of a specific type.
Input:

`INT4 rangeType`	This is the type of range that you are retrieving.

Unpublished work. © 2025 Siemens

Output:	
`INT4 startID`	The starting ID in the range.
`INT4 stopID`	The ending ID in the range. This will be 0 for single entry ranges.
`INT4 increment`	The increment to count by between startID and stopID.
`INT4 include`	The include flag: 0=Remove, 1=Add, Exclude=-1.
Return Code:	
`FE_NOT_AVAILABLE`	The rangeType that you specified is invalid.
`FE_FAIL`	Unable to retrieve the data for ranges, or you are at the end of the list.
Remarks/Usage:	
Use the `Reset` method to go to the beginning of the lists of ranges, then keep calling `RangeNext` to retrieve entries from a list of ranges one-by-one. Alternatively, you can call `RangeGetAll` to get all of the entries in one call.	
Example:	
None	

RangeAdd

RangeAdd	
`( rangeType,  startID, stopID, increment, include )`	
Description:	
This method adds a range selection to the end of a selected list.	
Input:	
`INT4 rangeType`	This is the type of range that you are creating.
`INT4 startID`	The starting ID in the range.

Unpublished work. © 2025 Siemens

`INT4 stopID`	The ending ID in the range. This should be 0 for single entry ranges.
`INT4 increment`	The increment to count by between startID and stopID, usually 1.
`INT4 include`	The include flag :0=Remove, 1=Add, Exclude=-1.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The rangeType that you specified is invalid.
`FE_FAIL`	Unable to save the data for range.
Remarks/Usage:	
If the entities that you want to add to the group are in a set, you can use the `SetAdd` method.	
Example:	
None	

RangeDelete

`RangeDelete`
`( rangeType )`
Description:
This method deletes a single range from the selected list.
Input:

`INT4 rangeType`	This is the type of range that you are deleting.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	The `rangeType` that you specified is invalid.
Remarks/Usage:	
This function deletes the range at the current location. To use this function, you must use the `RangeNext` function to retrieve ranges until you retrieve the one you want to delete, then call `RangeDelete` and that range will be removed.	
Example:	
None	

RangeDeleteAll

`RangeDeleteAll`	
`( rangeType )`	
Description:	
This method deletes all ranges from the selected list.	
Input:	
`INT4 rangeType`	This is the type of range that you are deleting. Specify -1 to delete all ranges of all types.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The `rangeType` that you specified is not valid.
Remarks/Usage:	
Deleting all ranges of all types (`rangeType=-1`) is equivalent to the **Group → Operations → Reset Rules** command.	
Example:	
None	

SetAdd

Unpublished work. © 2025 Siemens

SetAdd **(entityTYPE, SetID)**	
Description:	
This method adds all entities in a set to a group.	
Input:	
`INT4 entityTYPE`	Type of entity in the selected set. For more information, see **Entity Types**.
`INT4 SetID`	The ID of the set containing the entities to add to the group.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `entityTYPE` to add to a group.
Remarks/Usage:	
This method creates one or more *entity ID* range type rules to select the entities into the group.	
Example:	
None	

SetAdd2

SetAdd2 **(entityTYPE, SetID)**	
Description:	
This method adds all entities in a set to a group, based on the specified Group Data Type.	
Input:	
`enum entityTYPE(zGroupDataType)`	Type of entity in the selected set specified by `zGroupDataType`. For more information, see **Group Data Types**.

Unpublished work. © 2025 Siemens

`INT4 SetID`	The ID of the set containing the entities to add to the group.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `GroupDataType` to add to a group.
Remarks/Usage:	
This method creates one or more *entity ID* range type rules to select the entities into the group.	
Example:	
None	

SetAddOpt

`SetAddOpt`	
`( entityTYPE, SetID, addOpt )`	
Description:	
This method adds (or removes) all entities in a set to a group.	
Input:	
`INT4 entityTYPE`	Type of entity in the selected set. For more information, see **Entity Types**.
`INT4 SetID`	The ID of the set containing the entities to add to, remove from, or exclude from the group.
`INT4 addOpt`	0=Remove, 1=Add, -1= Exclude.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `entityTYPE` to add to, remove from, or exclude from a group.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method works just like the RangeAdd method except that all entities in the set are either added or removed.
Example:
None

SetAddOpt2

SetAddOpt2 **(entityTYPE, SetID, addOpt)**	
Description:	
This method adds (or removes) all entities in a set to a group, based on the specified Group Data Type.	
Input:	
`enum entityTYPE(zGroupDataType)`	Type of entity in the selected set specified by `zGroupDataType`. For more information, see Group Data Types.
`INT4 SetID`	The ID of the set containing the entities to add to, remove from, or exclude from the group.
`INT4 addOpt`	0=Remove, 1=Add, -1= Exclude.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `entityTYPE` to add to, remove from, or exclude from a group.
Remarks/Usage:	
This method works just like the RangeAdd method except that all entities in the set are either added or removed.	
Example:	
None	

Unpublished work. © 2025 Siemens

Add

Add **(entityTYPE, nEntityID)**	
Description:	
This method adds a single entity to a group.	
Input:	
`INT4 entityTYPE`	Type of entity in the selected set. For more information, see **Entity Types**.
`INT4 nEntityID`	The ID of the entity to add to the group.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `entityTYPE` to add to a group.
Remarks/Usage:	
None	
Example:	
None	

Add2

Add2 **(entityTYPE, nEntityID)**
Description:
This method adds a single entity of a specific Group Data Type to a group.
Input:

Unpublished work. © 2025 Siemens

`enum entityTYPE(zGroupDataType)`	Type of entity in the selected set specified by zGroupDataType. For more information, see **Group Data Types** in **Femap Constants**.
`INT4 nEntityID`	The ID of the entity to add to the group.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `GroupDataType` to add to a group.
Remarks/Usage:	
None	
Example:	
None	

AddOpt

`AddOpt` **`( entityTYPE, nEntityID, addOpt )`**	
Description:	
This method adds (or removes) an entity to a group using a user-defined option.	
Input:	
`INT4 entityTYPE`	Type of entity in the selected set. For more information, see **Entity Types**.
`INT4 nEntityID`	The ID of the entity to add to, remove from, or exclude from the group.
`INT4 addOpt`	0=Remove, 1=Add, -1= Exclude.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Invalid `entityTYPE` to add to, remove from, or exclude from a group.
Remarks/Usage:	
This method works just like the `Add` method except that you can choose to Add, Remove, or Exclude instead of only being able to Add.	
Example:	
None	

AddOpt2

`AddOpt2`	
`( entityTYPE, nEntityID, addOpt )`	
Description:	
This method adds (or removes) all entities in a set to a group, based on the specified Group Data Type.	
Input:	
`enum entityTYPE(zGroupDataType)`	Type of entity in the selected set specified by `zGroupDataType`. For more information, see Group Data Types.
`INT4 nEntityID`	The ID of the entity to add to, remove from, or exclude from the group.
`INT4 addOpt`	0=Remove, 1=Add, -1= Exclude.
Output:	
None	None
Return Code:	
`FE_FAIL`	Invalid `entityTYPE` to add to, remove from, or exclude from a group.
Remarks/Usage:	
This method works just like the `Add` method except that you can choose to Add, Remove, or Exclude instead of only being able to Add.	

Unpublished work. © 2025 Siemens

Example:
None

AddRelated

AddRelated	
(void)	
Description:	
This method adds entities related to the entities already in the group currently in the object.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_FAIL	None
Remarks/Usage:	
Performs the same function as the **Group → Operations → Add Related Entities** command.	
Example:	
None	

SelectModel

SelectModel
(void)
Description:
This method creates rules that select all possible entities in a model.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method is like the **Group** → **Operations** → **Select Model** command. The group must be `Put` to the database before these selections will take effect.	
Example:	
None	

SelectMesh

`SelectMesh`	
`( elemSET )`	
Description:	
This method creates a group that contains all of the nodes, properties, materials, coordinate systems, loads, and constraints that are referenced by the selected elements.	
Input:	
`INT4 elemSET`	The set of elements to use to compute the desired subset of the model.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This is like the **Group** → **Operations** → **Select Mesh** command. The group must be `Put` to the database before these selections will take effect.	

Unpublished work. © 2025 Siemens

Example:
None

SelectContact

SelectContact **(contactSET)**	
Description:	
This method creates a group that contains all of the entities referenced by a selected set of contact segments/surfaces.	
Input:	
`INT4 contactSET`	The set of contact segments/surfaces used to generate the group.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This is like the **Group → Operations → Contact Segment/Surface** command. The group must be `Put` to the database before these selections will take effect.	
Example:	
None	

SelectAllOnLayer

Unpublished work. © 2025 Siemens

SelectAllOnLayer	
(startLayer, endLayer)	
Description:	
This method selects all of the entities in the current model that are on layers `startLayer` thru `endLayer`.	
Input:	
`INT4 startLayer`	The ID of the lowest layer to select.
`INT4 endLayer`	The ID of the highest layer to select (If 0, then just entities on `startLayer` are selected).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This is like the **Group** → **Operations** → **Generate Entities On Layer** command, except the entities are selected into the current Group object.	
Example:	
None	

ReferencedGroups

ReferencedGroups	
(void)	
Description:	
This method creates and returns a Set object that contains the IDs of all sub-groups that are referenced by this group.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The Set object is returned in place of the return code.	
Example:	

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim gr As femap.group
   Set gr = App.feGroup

   gr.Get (14)
   Set gset = gr.ReferencedGroups
   gset.Debug
End Sub
```

SetReferencedGroups

SetReferencedGroups

(setID, clear)

Description:
This method is used to specify the IDs of all sub-groups that are referenced by this group.
Input:

Unpublished work. © 2025 Siemens

`INT4 setID`	ID of Set Object containing the IDs of the sub-groups to be referenced by a group. Alternatively, entering a negative value will only consider a single sub-group ID.
`BOOL clear`	When True, all sub-group IDs which were referenced by the group will be removed before adding the IDs specified in Set Object specified for setID. When False, the IDs specified in the Set Object specified for setID will simply be added.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Can be used in conjunction with `ReferencedGroups` to essentially create a copy of a group which has referenced sub-groups.	
Example:	
None	

Clear

`Clear`	
`( void )`	
Description:	
This function simply removes all rules and entities from the Group object.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Kinematic Joint and Joint Connection Objects

Joint objects correspond to the kinematic joints or joint connections in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Joint objects. For more information, see **Common Entity Properties and Methods**.

Use the `feJoint` method of the Femap Application object to create Joint objects.

Kinematic Joint and Joint Connection Object Properties

Kinematic Joints and Joint Connections are always stored with a `SetID=1`, and the ID equal to the Kinematic Joint or Joint Connection ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The joint color.
`INT4 layer`	The joint layer.
`STRING title`	Title of joint

`INT4 type`	Kinematic Joint or Joint Connection:
	`KINEMATIC_UNKNOWN (0)`
	`KINEMATIC_JOINT (1)`
	`KINEMATIC_CONNECTION (2)`

`INT4 JointBehavior`	Joint Type:

Unpublished work. © 2025 Siemens

Property	Description
	JOINT_UNKNOWN (0=Unkown) JOINT_REVOLUTE (1=Revolute) JOINT_INLINE (2=Inline) JOINT_SLIDER (3=Slider) JOINT_SPHERE (4=Spherical) JOINT_CYLDR (5=Cylindrical) JOINT_SLIUNV (6=Slider-Universal) JOINT_FIXED (7=Fixed) JOINT_SCREW (8=Screw) JOINT_UNIVSEL (9=Universal) JOINT_CONVEL (10=Constant Velocity) JOINT_GENERAL (11=General)
BOOL HasCID1	If True, Joint CSys at Node 1 (Define Kinematic Joint dialog box) or CSys in Orientation 1 section (Define Joint Connection dialog box) can be specified.
INT4 CID1	ID of Joint CSys at Node 1 (Define Kinematic Joint dialog box) or ID of CSys in Orientation 1 section (Define Joint Connection dialog box).
BOOL HasCID2	If True, Joint CSys at Node 2 (Define Kinematic Joint dialog box) or CSys in Orientation 2 section (Define Joint Connection dialog box) can be specified. Only used when JointBehavior is set to JOINT_UNIVSEL (9=Universal) or JOINT_CONVEL (10=Constant Velocity).
INT4 CID2	ID of Joint CSys at Node 2 (Define Kinematic Joint dialog box) or ID of CSys in Orientation 2 section (Define Joint Connection dialog box). Only used when JointBehavior is set to JOINT_UNIVSEL (9=Universal) or JOINT_CONVEL (10=Constant Velocity).
Kinematic Joint	
INT4 G1	Nodes - First Field ID.
INT4 G2	Nodes - Second Field ID.
INT4 PID	Joint Property ID.
Joint Connection	
BOOL HasTarget1List	If True, entities have been specified for Reference Attachment.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 Target1Type`	Type of Entity or To Ground option for Reference Attachment. `JOINT_ATTACH_UNKNOWN` (0=Unknown) `JOINT_ATTACH_POINTS` (1=Points) `JOINT_ATTACH_SURFACES` (2=Surfaces) `JOINT_ATTACH_CURVES` (3=Curves) `JOINT_ATTACH_NODES` (4=Nodes) `JOINT_ATTACH_ELEMENTS` (5=Elements) `JOINT_ATTACH_GROUND` (6=To Ground)
`INT4 Target1SetID`	ID of Set containing entities used for Reference Attachment.
`BOOL HasTarget2List`	If True, entities have been specified for Motion Attachment.
`INT4 Target2Type`	Type of Entity or To Ground option for Motion Attachment. `JOINT_ATTACH_UNKNOWN` (0=Unknown) `JOINT_ATTACH_POINTS` (1=Points) `JOINT_ATTACH_SURFACES` (2=Surfaces) `JOINT_ATTACH_CURVES` (3=Curves) `JOINT_ATTACH_NODES` (4=Nodes) `JOINT_ATTACH_ELEMENTS` (5=Elements)
`INT4 Target2SetID`	ID of Set containing entities used for Motion Attachment.

`REAL8 JointLoc1[0..2]`
 	
or
 	
`VARIANT vJointLoc1`
	Array of values to specify X, Y, and Z Locations of Point 1 or the coordinates of Point 1 as a Variant. They are always specified in global rectangular coordinates.

`REAL8 JointLoc2[0..2]`
 	
or
 	
`VARIANT vJointLoc2`
	Array of values to specify X, Y, and Z Locations of Point 2 or the coordinates of Point 2 as a Variant. They are always specified in global rectangular coordinates.

`REAL8 JointLoc1_x`	These are the coordinates of Point 1. They are always specified in global rectangular coordinates.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 JointLoc1_y`	
`REAL8 JointLoc1_z`	
`REAL8 JointLoc2_x`	These are the coordinates of Point 2. They are always specified in global rectangular coordinates.
`REAL8 JointLoc2_y`	
`REAL8 JointLoc2_z`	
Joint Properties - General tab	
`INT4 CurveExpansionOpt`	From Curve option in Connection Expansion section. `EXPAND_BOUNDRY` (0=Nodes on boundary only `EXPAND_INTERIOR` (1=Nodes in interior only) `EXPAND_BOTH` (2=Nodes on boundary and interior)
`INT4 SurfaceExpansionOpt`	From Surface option in Connection Expansion section. `EXPAND_BOUNDRY` (0=Nodes on boundary only `EXPAND_INTERIOR` (1=Nodes in interior only) `EXPAND_BOTH` (2=Nodes on boundary and interior)

`INT4 NasExapnsionOpt`	Connection (Nastran) option in Solver Expansion section.
	
	`0=CJOINT + RBE2`
	`1=CJOINT + RBE3`

`INT4 ElemExpansionOpt`	NOT CURRENTLY USED.
Joint Properties - Simcenter Nastran tab	
`BOOL HasG3`	Control Node (G3) ID.
`INT4 G3`	Control Node (G3) ID.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 KT`	Value for Translational (KT).
`REAL8 KR`	Value for Rotational (KR).
`REAL8 CF`	Value for Coefficient (CF).
`REAL8 FR0`	Value for Tightening Force (FR0).
`REAL8 LR`	Value for Length Radius (LR).
`REAL8 LX`	Value for Length X (LX).
`REAL8 LY`	Value for Length X (LY).
`REAL8 LZ`	Value for Length X (LZ).
`BOOL RegularizationOpt`	If True, regularization is based on velocity. If False, regularization based on stiffness.
`REAL8 KCF`	Value for Reg. Stiffness (KCF).
`REAL8 TOL`	Value for Reg. Velocity (TOL).
`REAL8 LIBL`	Value for Liberation Length (LIBL).
`REAL8 PITCH`	Value for Pitch (PITCH).
`BOOL OPT`	If True, sets 0..Boolean Identification for DOF Option (OPT). If False, sets 1..Lagrangian Multipliers.
`BOOL HasSPR1TRA`	NOT CURRENTLY USED. When True, items in the Translational Spring Properties section can be specified.
`REAL8 K1T`	Value Translational (K1T).
`INT4 K1TTID`	ID of function to use for Force vs. Disp (K1TTID).
`INT4 SK1TTID`	ID of function to use for Scale Factor (SK1TTID).

Unpublished work. © 2025 Siemens

Property	Description
`BOOL HasSPR1ROT`	NOT CURRENTLY USED. When True, items in the Rotational Spring Properties section can be specified.
`REAL8 K1R`	Value of Rotational (K1R).
`INT4 K1RTID`	ID of function to use for Torque vs. Disp (K1RTID).
`INT4 SK1RTID`	ID of function to use for Scale Factor (SK1RTID).
`BOOL HasDAM1TRA`	NOT CURRENTLY USED. When True, items in the Translational Damping Properties section can be specified.
`REAL8 B1T`	Value of Translational (B1T).
`INT4 B1TTID`	ID of function to use for Force vs. Vel (B1TTID).
`INT4 SB1TTID`	ID of function to use for Scale Factor (SB1TTID).
`BOOL HasDAM1ROT`	NOT CURRENTLY USED. When True, items in the Rotational Damping Properties section can be specified.
`REAL8 B1R`	Value of Rotational (B1R).
`INT4 B1RTID`	ID of function to use for Torque vs. Vel (B1RTID).
`INT4 SB1RTID`	ID of function to use for Scale Factor (SB1RTID).
Joint Properties - ANSYS	

`BOOL JointProp[0..37]`
 	
or
 	
`VARIANT vJointProp`
	Array of values used to specify values on the **ANSYS** tab of the Joint Properties dialog box.
	
	[0] = Not currently used
	
	Reference Lengths/Angles (SECDATA) Section:
	
	[1..3] = Reference Lengths (L, L2, and L3)
	[4..6] = Angles (Phi1, Phi2, and Phi3)
	
	Misc Data (SECJ, TB) Section:

Unpublished work. © 2025 Siemens

Property	Description
	[7] = Screw Pitch, [8] = Outer Radius, [9] = Inner Radius [10] = Effective Length, [11] = Stiffness, [12] = Damping, [13] = Friction mu Locks (SECLOCK) section: [14..15] UX Minimum and Maximum values [16..17] UY Minimum and Maximum values [18..19] UZ Minimum and Maximum values [20..21] RX Minimum and Maximum values [22..23] RY Minimum and Maximum values [24..25] RZ Minimum and Maximum values Stops (SECSTOP) section: [26..27] UX Minimum and Maximum values [28..29] UY Minimum and Maximum values [30..31] UZ Minimum and Maximum values [32..33] RX Minimum and Maximum values [34..35] RY Minimum and Maximum values [36..37] RZ Minimum and Maximum values

`BOOL flag[0..18]`
 	
or
 	
`VARIANT vflag`
	Array of Flags to turn on certain check boxes in the Define Kinematic Joint and Define Joint Connection dialog boxes, as well as various tabs of the Joint Properties dialog box.
	
	Define Kinematic Joint and Define Joint Connection dialog boxes:
	
	[1..6] = TX, TY, TZ, RX, RY, and RZ Relative DOF. Only available when **Joint Type** is set to **11..General**.
	
	Joint Properties dialog box (**General** Tab):
	
	[0] = Connection ANSYS option
	
	Joint Properties dialog box (**ANSYS** Tab):
	
	[7..12] = UX, UY, UZ, RX, RY, and RZ in Locks (SECLOCK) section
	[13..18] = UX, UY, UZ, RX, RY, and RZ in Stops (SECSTOP) section

Unpublished work. © 2025 Siemens

Joint Object Methods

The following methods are available in addition to the standard entity methods.

PutReferenceSet

PutReferenceSet **(pReferenceSet)**	
Description:	
This method loads Entity IDs into the Entity List for Reference Attachment for Joint Connections.	
Input:	
`OBJECT pReferenceSet`	Set Object containing the IDs of the entities.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetReferenceSet

GetReferenceSet **(pReferenceSet)**
Description:

Unpublished work. © 2025 Siemens

This method loads a Set object that contains the IDs of the Entities in the Entity List used for Reference Attachment for Joint Connections.	
Input:	
None	None
Output:	
`OBJECT pReferenceSet`	Set Object containing the IDs of the entities.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ClearReferenceSet

`ClearReferenceSet`
 	
`( void )`	

Description:	
This method clears the array that contains the IDs of the Entities used by Reference Attachment for Joint Connections.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

PutMotionSet

PutMotionSet **(pMotionSet)**	
Description:	
This method loads Entity IDs into the Entity List for Motion Attachment for Joint Connections.	
Input:	
`OBJECT pReferenceSet`	Set Object containing the IDs of the entities.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetMotionSet

GetMotionSet **(pMotionSet)**
Description:

Unpublished work. © 2025 Siemens

This method loads a Set object that contains the IDs of the Entities in the Entity List used for Motion Attachment for Joint Connections.	
Input:	
None	None
Output:	
`OBJECT pReferenceSet`	Set Object containing the IDs of the entities.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ClearMotionSet

```
ClearMotionSet

( void )
```

Description:	
This method clears the array that contains the IDs of the Entities used by Motion Attachment.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Layer Objects

Layer objects correspond to the layer definitions in your model. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Layer objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLayer` method of the Femap Application object to create Layer Objects.

Layer Object Properties

Layers are always stored with their SetID equal to the setID of the load set, and the ID equal to the Layer ID.

Property	Description
`INT4 color`	The layer color.
`STRING title`	The layer title (maximum 79 characters).

Layer Object Methods

The following methods are available in addition to the standard entity methods.

InitAllLayerInfo

`InitAllLayerInfo` **`( bContents, bGroupReference, bViewReference )`**
Description:

Unpublished work. © 2025 Siemens

This method scans the model and builds the data structure for the Contents, Group References, and/or View References for a layer, then loads this data into the Layer Object. This method is the same as using a combination of the `InitLayerContentsInfo`, `InitLayerGroupReferenceInfo`, and/or `InitLayerViewReferenceInfo` methods with some specific values specified.	
Input:	
`BOOL bContents`	When True, builds the data structure for contents (entities) of the layer.
`BOOL bGroupReference`	When True, builds the data structure for group references of the layer.
`BOOL bViewReference`	When True, builds the data structure for view references of the layer.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

This method or a combination of the `InitLayerContentsInfo`, `InitLayerGroupReferences`, and/or `InitLayerViewReferenceInfo` methods set to the following values:
 	
`InitLayerContentsInfo(0, 0, NULL, 0)`
 	
`InitLayerGroupReferenceInfo (0, 0)`
 	
`InitLayerViewReferenceInfo (0, 0)`
 	
...and this method or one of the individual methods MUST be called in order for the other methods of the Layer Object to function properly, otherwise they will return `FE_INVALID`. For more information of the individual methods, see **InitLayerContentsInfo**, **InitLayerGroupReferenceInfo**, and/or **InitLayerViewReferenceInfo**.	

Example:	
None	

InitLayerContentsInfo

Unpublished work. © 2025 Siemens

InitLayerContentsInfo

(nLayerSetID, nNumEntityTypes, EntityTypes, nSetSetID)

Description:

This method scans the model and builds the data structure for Contents (entities) for a layer, then loads this data into the Layer Object.

Input:

`INT4 nLayerSetID`	ID of a Set Object containing IDs of layers. Entering a value of 0 will consider all layers. Alternatively, entering a negative value will only consider a single layer.
`INT4 nNumEntityTypes`	Number of entity types in EntityTypes. Entering a value of 0 will consider all entity types.
`VARIANT EntityTypes`	Array of entity type IDs. If a value of 0 has been entered for `nNumEntityTypes`, this argument can be set to NULL.
`INT4 nSetSetID`	ID of a Set Object containing IDs of Load/Constraint Sets. Entering a value of 0 will consider all Load/Constraint Sets. Alternatively, entering a negative value will only consider a single Load/Constraint Set.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

This method, when used with these values:

```
InitLayerContentsInfo(0, 0, NULL, 0)
```

...is equivalent to using `InitAllLayerInfo` with True for the `bContents` argument and either of these methods MUST be called in order for the other methods of the Layer Object to function properly, otherwise they will return `FE_INVALID`. For more information about the combined method for initializing and populating layer information, see **InitLayerContentsInfo**.

Example:

None

Unpublished work. © 2025 Siemens

InitLayerGroupReferenceInfo

`InitLayerGroupReferenceInfo`

`( nLayerSetID, nGroupSetID )`

Description:	
This method scans the model and builds the data structure for groups that reference the layers, then loads this data into the Layer Object.	
Input:	
`INT4 nLayerSetID`	ID of a Set Object containing IDs of layers. Entering a value of 0 will consider all layers. Alternatively, entering a negative value will only consider a single layer.
`INT4 nGroupSetID`	ID of a Set Object containing IDs of groups. Entering a value of 0 will consider all groups. Alternatively, entering a negative value will only consider a single group.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method, when used with these values: `InitLayerGroupReferenceInfo(0, 0)` ...is equivalent to using `InitAllLayerInfo` with True for the `bGroupReference` argument and either of these methods MUST be called in order for the other methods of the Layer Object to function properly, otherwise they will return `FE_INVALID`. For more information about the combined method for initializing and populating layer information, see **InitLayerContentsInfo**.	
Example:	
None	

Unpublished work. © 2025 Siemens

InitLayerViewReferenceInfo

`InitLayerViewReferenceInfo`

`( nLayerSetID, nViewSetID )`

Description:

This method scans the model and builds the data structure for layers displayed on views, then loads this data into the Layer Object.

Input:

`INT4 nLayerSetID`	ID of a Set Object containing IDs of layers. Entering a value of 0 will consider all layers. Alternatively, entering a negative value will only consider a single layer.
`INT4 nViewSetID`	ID of a Set Object containing IDs of views. Entering a value of 0 will consider all views. Alternatively, entering a negative value will only consider a single view.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

This method, when used with these values:

```
InitLayerViewReferenceInfo(0, 0)
```

...is equivalent to using `InitAllLayerInfo` with True for the `bViewReference` argument and either of these methods MUST be called in order for the other methods of the Layer Object to function properly, otherwise they will return `FE_INVALID`. For more information about the combined method for initializing and populating layer information, see **InitLayerContentsInfo**.

Example:

None

Unpublished work. © 2025 Siemens

ClearLayerInfo

ClearLayerInfo	
(void)	
Description:	
This method clears the layer information from a Layer Object.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method only needs to be used to clear information from the Layer Object which was placed there by the `InitAllLayerInfo` method (InitLayerContentsInfo) or any combination of the individual `InitLayerContentsInfo` (InitLayerContentsInfo), `InitLayerGroupReferences` (InitLayerGroupReferenceInfo), and/or `InitLayerViewReferenceInfo` (InitLayerViewReferenceInfo) methods.	
Example:	
None	

ResetNextLayerInfo

ResetNextLayerInfo
(void)
Description:
This method resets the data structure with the information and contents of a layer.
Input:

Unpublished work. © 2025 Siemens

None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

NextLayerInfo

`NextLayerInfo` **`( nDesiredLayerID, nEntitySetID, nDataType, nInSetID, nOnType )`**	
Description:	
This method fills a Set Object with the next type of entity in the data structure of the desired layer.	
Input:	
`INT4 nDesiredLayerID`	The ID of the layer to retrieve information on.
`INT4 nEntitySetID`	The ID of the Set Object to fill.
Output:	
`INT4 nDataType`	The enum of the data type the Set Object was filled with. For all data types, see **Entity Types**.
`INT4 nInSetID`	The ID of the load or constraint set associated to `nDataType`. All data types other than loads or constraints `nInSetID` = 1.
`INT4 nOnType`	The enum of the data type the entities in `nEntitySet` are associated to. For example, `nInSetID` would be the ID of the load/constraint set, and `nOnType` would be the data type the load or constraint is on, `FT_NODE`, `FT_ELEM`,

Unpublished work. © 2025 Siemens

	FT_CURVE, etc. All data types other than loads or constraints nOnType = nType.
Return Code:	
FE_INVALID	Information about the *contents* layers (entities), layers referenced by groups, and/or views where the layer is visible has not been properly added to the Layer Object. See Remarks/Usage section.
Remarks/Usage:	
This method requires information to be loaded into the Layer Object by the InitAllLayerInfo method (InitLayerContentsInfo) or any combination of the individual InitLayerContentsInfo (InitLayerContentsInfo), InitLayerGroupReferences (InitLayerGroupReferenceInfo), and/or InitLayerViewReferenceInfo (InitLayerViewReferenceInfo) methods If this is not done, prior to using this method, a return code of FE_INVALID will be returned.	
Example:	
None	

FindLayerInfo

FindLayerInfo	
(nLayerID, nDataType, nInSetID, nOnType, nEntitySetID, bClearEntitySet)	
Description:	
This method fills a Set Object with the entities in a load or constraint on a specify layer.	
Input:	
INT4 nLayerID	The ID of the layer to retrieve information on.
INT4 nDataType	The enum of the data type the Set Object was filled with. 13 = FT_SURF_LOAD, 14 = FT_GEOM_LOAD, 15 = FT_NTHERM_LOAD, 16 = FT_ETHERM_LOAD, 18 = FT_BCO, 19 = FT_BC_GEOM, 20 = FT_BEQ.
INT4 nInSetID	The ID of the Set Object to fill.
INT4 nOnType	The enum of the data type the entities in nEntitySet are associated to. For example, nInSetID would be the ID of the load/constraint set, and nOnType would be the data type the

Unpublished work. © 2025 Siemens

	load or constraint is on, FT_NODE, FT_ELEM, FT_CURVE, etc.
INT4 nEntitySetID	The ID of the Set Object to fill.
BOOL bClearEntitySet	If True, nEntitySet will be cleared of all entities beforehand.
Output:	
None	None
Return Code:	
FE_INVALID	Information about the *contents* layers (entities), layers referenced by groups, and/or views where the layer is visible has not been properly added to the Layer Object. See Remarks/Usage section.
Remarks/Usage:	
This method requires information to be loaded into the Layer Object by the InitAllLayerInfo method (InitLayerContentsInfo) or any combination of the individual InitLayerContentsInfo (InitLayerContentsInfo), InitLayerGroupReferences (InitLayerGroupReferenceInfo), and/or InitLayerViewReferenceInfo (InitLayerViewReferenceInfo) methods If this is not done, prior to using this method, a return code of FE_INVALID will be returned.	
Example:	
None	

FindAllReferencedLayers

FindAlIReferencedLayers
(nLayerSetID)
Description:
This method fills the Set Object of the specified ID with all the layers which reference any information currently loaded in the Layer Object.
Input:

INT4 nLayerSetID	ID of an existing set object to fill with the IDs of all the layers which currently have information loaded into the Layer Object. See Remarks/Usage section.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_INVALID`	Information about the *contents* layers (entities), layers referenced by groups, and/or views where the layer is visible has not been properly added to the Layer Object. See Remarks/Usage section.
Remarks/Usage:	
This method requires information to be loaded into the Layer Object by the `InitAllLayerInfo` method (**InitLayerContentsInfo**) or any combination of the individual `InitLayerContentsInfo` (**InitLayerContentsInfo**), `InitLayerGroupReferences` (**InitLayerGroupReferenceInfo**), and/or `InitLayerViewReferenceInfo` (**InitLayerViewReferenceInfo**) methods If this is not done, prior to using this method, a return code of `FE_INVALID` will be returned.	
Example:	
None	

FindEmptyLayers

`FindEmptyLayers` **`( nLayerSetID )`**	
Description:	
This method fills the Set Object of the specified ID with all the layers in the model which are currently empty, based on the information currently loaded in the Layer Object.	
Input:	
`INT4 nLayerSetID`	ID of an existing set object to fill with the IDs of the layers which are currently empty based on the information currently loaded into in the Layer Object by the "`Init`" methods. See Remarks/Usage section.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_INVALID`	Information about the *contents* layers (entities), layers referenced by groups, and/or views where the layer is visible has not been properly added to the Layer Object. See Remarks/Usage section.

Remarks/Usage:

This method requires information to be loaded into the Layer Object by the `InitAllLayerInfo` method (**InitLayerContentsInfo**) or any combination of the individual `InitLayerContentsInfo` (**InitLayerContentsInfo**), `InitLayerGroupReferences` (**InitLayerGroupReferenceInfo**), and/or `InitLayerViewReferenceInfo` (**InitLayerViewReferenceInfo**) methods If this is not done, prior to using this method, a return code of `FE_INVALID` will be returned.

Example:

None

FindMissingLayers

`FindMissingLayers`

`( nLayerSetID )`

Description:

This method fills the Set Object of the specified ID with all the layers in the model which have entities assigned to them, but do not currently exist in the model.

Input:

`INT4 nLayerSetID`	ID of an existing set object to fill with the IDs of the layers which have entities assigned to them, based on the information currently loaded into in the Layer Object by the "`Init`" methods, but do not currently exist in the model. See Remarks/Usage section.

Output:

None	None

Return Code:

`FE_INVALID`	Information about the *contents* layers (entities), layers referenced by groups, and/or views where the layer is visible has not been properly added to the Layer Object. See Remarks/Usage section.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method requires information to be loaded into the Layer Object by the `InitAllLayerInfo` method (**InitLayerContentsInfo**) or any combination of the individual `InitLayerContentsInfo` (**InitLayerContentsInfo**), `InitLayerGroupReferences` (**InitLayerGroupReferenceInfo**), and/or `InitLayerViewReferenceInfo` (**InitLayerViewReferenceInfo**) methods If this is not done, prior to using this method, a return code of `FE_INVALID` will be returned.
Example:
None

Layup Objects

Layup objects correspond to the layups in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Layup objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLayup` method of the Femap Application object, or the Layup method of the Property object to create Layup objects.

Layup Object Properties

Layups are always stored with a `SetID=1`, and the ID equal to the layup ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

The ply information is stored as an array - with globalply, matlID, thickness, angle, failure theory, and on/off status for each ply.

Property	Description
`STRING title`	Layup title.
`INT4 NumberOfPlys`	The number of plys in the layup definition.
`REAL8 LayupThickness`	The overall layup thickness (sum of all the ply thicknesses). Read-only.
`BOOL IncludeOffPlys`	When False, which is the default behavior, only plies set to *on* will be loaded into the Layup

Unpublished work. © 2025 Siemens

Property	Description
	Object when using `Get`. When True, plies which are currently set to *off* will be also be loaded into the Layup Object, thus this must be specified before the Layup Object is loaded with plies.

`INT4 globalply[0..NumberOfPlys-1]`
or
`VARIANT vglobalply`
	Array of global ply IDs referenced by each ply. The VARIANT form can only be used with layups of 500 plies or less. To set ply data using the VARIANT form, you must first set `NumberOfPlys` to the number of plies you want to load from the VARIANT.

`INT4 matlID[0..NumberOfPlys-1]`
or
`VARIANT vmatlID`
	Array of material IDs referenced by each ply. The VARIANT form can only be used with layups of 500 plies or less. To set ply data using the VARIANT form, you must first set `NumberOfPlys` to the number of plies you want to load from the VARIANT.

`REAL8 thickness[0..NumberOfPlys-1]`
or
`VARIANT vthickness`
	Array of thicknesses referenced by each ply. The VARIANT form can only be used with layups of 500 plies or less. To set ply data using the VARIANT form, you must first set `NumberOfPlys` to the number of plies you want to load from the VARIANT.

`REAL8 angle[0..NumberOfPlys-1]`
or
`VARIANT vangle`
	Array of orientation angles referenced by each ply. The VARIANT form can only be used with layups of 500 plies or less. To set ply data using the VARIANT form, you must first set `NumberOfPlys` to the number of plies you want to load from the VARIANT.

`REAL8 failuretheory[0..NumberOfPlys-1]`
or
`VARIANT vfailuretheory`
	Array of failure theory methods referenced by each ply. The VARIANT form can only be used with layups of 500 plies or less. To set ply data using the VARIANT form, you must first set `NumberOfPlys` to the number of plies you want to load from the VARIANT.

`BOOL OnOff[0..NumberOfPlys-1]`	Array of booleans to specify if ply is on or off for each ply. The VARIANT form can only be used with layups of 500 plies or less. To set ply data using the VARIANT form, you must first set `NumberOfPlys`

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vOnOff`	to the number of plies you want to load from the VARIANT.

`REAL8 AMatrix[0..2,0..2]`
or
`VARIANT vAMatrix`
	Computed Value - [A] Matrix

`REAL8 BMatrix[0..2,0..2]`
or
`VARIANT vBMatrix`
	Computed Value - [B] Matrix

`REAL8 DMatrix[0..2,0..2]`
or
`VARIANT vDMatrix`
	Computed Value - [D] Matrix

`REAL8 AInvMatrix[0..2,0..2]`
or
`VARIANT vAInvMatrix`
	Computed Value - [A]-1 Matrix

`REAL8 BInvMatrix[0..2,0..2]`
or
`VARIANT vBInvMatrix`
	Computed Value - [B]-1 Matrix

`REAL8 DInvMatrix[0..2,0..2]`	Computed Value - [D]-1 Matrix

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vDInvMatrix`	

`REAL8 InPlaneProp[0..7]`
 	
or
 	
`VARIANT vInPlaneProp`
	In-Plane Properties (Ex, Ey, Gxy, NUxy, NUyx, Alphax, Alphay, Alphaxy). The method `.Compute2` should be called beforehand to return real values.

`REAL8 InPlanePropExcMemBend[0..4]`
 	
or
 	
`VARIANT vInPlanePropExcMemBend`
	For informational purposes only. Excluded In-Plane Properties (Ex, Ey, Gxy, NUxy, NUyx, Alphax, Alphay, Alphaxy). The method `.Compute2` should be called beforehand to return real values.

`REAL8 BendingProp[0..7]`
 	
or
 	
`VARIANT vBendingProp`
	Bending Properties (Exb, Eyb, Gxyb, NUxyb, NUyxb, Alphaxb, Alphayb, Alphaxyb).

`REAL8 BendingPropExcMemBend[0..7]`
 	
or
 	
`VARIANT vBendingPropExcMemBend`
	For informational purposes only. Excluded Bending Properties (Exb, Eyb, Gxyb, NUxyb, NUyxb, Alphaxb, Alphayb, Alphaxyb).

Layup Object Methods

The following methods are available in addition to the standard entity methods.

AddPly

Unpublished work. © 2025 Siemens

AddPly	
(nMatlID, dThickness, dAngle, nGlobalPly)	
Description:	
This method adds a ply to the layup. If you need to specify a different Failure Theory for each ply, see, AddPly2.	
Input:	
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This function simply adds a ply to the top of the layup. Same as calling `InsertPly` with `nPlyID = NumberOfPlys+1`.	
Example:	
None	

AddPly2

AddPly2	
(nMatlID, dThickness, dAngle, nGlobalPly, nFailureTheory)	
Description:	

Unpublished work. © 2025 Siemens

This method adds a ply to the layup.	
Input:	
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
`INT4 nFailureTheory`	The ID of a Failure Theory referenced by this ply, if any.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This function simply adds a ply to the top of the layup. Same as calling `InsertPly2` with `nPlyID = NumberOfPlys+1`.	
Example:	
None	

AddPlyMaterialPly

`AddPlyMaterialPly`
`( nPlyMaterialID, dAngle, nGlobalPly )`
Description:
This method adds a ply material ply to the layup.
Input:

`INT4 nPlyMaterialID`	The ID of a ply material.

Unpublished work. © 2025 Siemens

`REAL8 dAngle`	The orientation angle of the ply material ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply material ply, if any.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Ply Material ID doesn't exist.
`FE_INVALID`	Global Ply ID does not exist.
Remarks/Usage:	
This function simply adds a ply material ply to the top of the layup.	
Example:	
None	

AddLayupPly

`AddLayupPly`
`( nLayupID, nSym, dAngle )`
Description:
This method adds a layup ply to the layup with an option to make the layup be symmetric, anti, or antisymmetric.
Input:

`INT4 nLayupID`	The ID of a layup
`INT4 nSym`	Symmetry option for layup ply: 0=No Symmetry Option (`FPLS_ASIS`). 1=Symmetric (`FPLS_SYMMETRIC`). 2=Anti (`FPLS_ANTI`). 3=Anti-Symmetric (`FPLS_ANTISYMMETRIC`).

Unpublished work. © 2025 Siemens

`REAL8 dAngle`	The orientation angle of the layup ply.
Output:	
None	None
Return Code:	
`FE_INVALID`	Value specified for `nSym` is not 0, 1, 2, or 3.
`FE_NOT_EXIST`	Layup does not exist.
Remarks/Usage:	
This function simply adds a layup ply to the top of the layup.	
Example:	
None	

AddMatchedPlys

`AddMatchedPlys`
`( nNumPly, nPlyID, nSym )`
Description:
This method adds any number of matched plies to the layup with an option to make the matched plies be symmetric, anti, or antisymmetric.
Input:

`INT4 nNumPly`	The number of matched plies being added.
`INT4 nPlyID[0..nNumPly-1]`	The IDs of original plies being used to create matched plies.
`INT4 nSym`	Symmetry option for matched plies: 0=No Symmetry Option (`FPLS_ASIS`). 1=Symmetric (`FPLS_SYMMETRIC`). 2=Anti (`FPLS_ANTI`). 3=Anti-Symmetric (`FPLS_ANTISYMMETRIC`).
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_INVALID`	One or more selected plies is a *Child Ply* or a *Matched Ply*, which cannot be added as a *Matched Ply*.
`FE_NOT_EXIST`	One of more ply IDs is set to 0 or a negative value.
Remarks/Usage:	
This function simply adds the matched plies to the top of the layup.	
Example:	
None	

InsertPly

`InsertPly` **`( nPlyID, nMatlID, dThickness, dAngle, nGlobalPly )`**
Description:
This method inserts a ply into the layup. If you need to specify a different Failure Theory for each ply, see, InsertPly2.
Input:

`INT4 nPlyID`	The index of the ply to insert. Ply 1 is at the bottom. The IDs of all plys at and above this number are increased by one.
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

InsertPly2

`InsertPly2` **`( nPlyID, nMatlID, dThickness, dAngle, nGlobalPly, nFailureTheory )`**	
Description:	
This method inserts a ply into the layup.	
Input:	
`INT4 nPlyID`	The index of the ply to insert. Ply 1 is at the bottom. The IDs of all plys at and above this number are increased by one.
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
`INT4 nFailureTheory`	The ID of a Failure Theory referenced by this ply, if any.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

SetPly

SetPly **(nPlyID, nMatlID, dThickness, dAngle, nGlobalPly)**	
Description:	
This method replaces the data for a ply in the layup. If you need to specify a different Failure Theory for each ply, see **SetPly2**.	
Input:	
`INT4 nPlyID`	The index of the ply to replace/set. Ply 1 is at the bottom.
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	`nPlyID` does not specify a ply that currently exists in the layup. Use `AddPly2` or `InsertPly2` to create a new ply.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetPly2

SetPly2 **(nPlyID, nMatlID, dThickness, dAngle, nGlobalPly, nFailureTheory)**	
Description:	
This method replaces the data for a ply in the layup.	
Input:	
`INT4 nPlyID`	The index of the ply to replace/set. Ply 1 is at the bottom.
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
`INT4 nFailureTheory`	The ID of a Failure Theory referenced by this ply, if any.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	`nPlyID` does not specify a ply that currently exists in the layup. Use `AddPly2` or `InsertPly2` to create a new ply.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetAllPly

SetAllPly

(nNumPly, nMatlID, dThickness, dAngle, nGlobalPly)

Description:

This method replaces the data for all plys in the layup. If you need to specify different Failure Theories for each ply, see SetAllPly2.

Input:

`INT4 nNumPly`	The number of plys being specified.
`INT4 nMatlID[0..nNumPly-1]`	The IDs of materials referenced by the plys.
`REAL8 dThickness[0..nNumPly-1]`	The thicknesses of the plys.
`REAL8 dAngle[0..nNumPly-1]`	The orientation angles of the plys.
`INT4 nGlobalPly[0..nNumPly-1]`	The IDs of Global Plys referenced by the plys, if any.

Output:

None	None

Return Code:

`FE_FAIL`	Unable to set the ply data.

Remarks/Usage:

Unlike , this method simply clears all previous ply definitions and replaces the entire layup with the definition supplied in the arrays. The first entry in the arrays becomes the bottom of the layup.

Example:

None

SetAllPly2

Unpublished work. © 2025 Siemens

SetAllPly2

(nNumPly, nMatlID, dThickness, dAngle, nGlobalPly, nFailureTheory)

Description:

This method replaces the data for all plys in the layup.

Input:

`INT4 nNumPly`	The number of plys being specified.
`INT4 nMatlID[0..nNumPly-1]`	The IDs of materials referenced by the plys.
`REAL8 dThickness[0..nNumPly-1]`	The thicknesses of the plys.
`REAL8 dAngle[0..nNumPly-1]`	The orientation angles of the plys.
`INT4 nGlobalPly[0..nNumPly-1]`	The IDs of Global Plys referenced by the plys, if any.
`INT4 nFailureTheory[0..nNumPly-1]`	The IDs of Failure Theories referenced by the plys, if any.

Output:

None	None

Return Code:

`FE_FAIL`	Unable to set the ply data.

Remarks/Usage:

Unlike `SetPly2`, this method simply clears all previous ply definitions and replaces the entire layup with the definition supplied in the arrays. The first entry in the arrays becomes the bottom of the layup.

Example:

None

GetPly

Unpublished work. © 2025 Siemens

GetPly **(nPlyID, nMatlID, dThickness, dAngle, nGlobalPly)**	
Description:	
Returns all of the data for a selected ply. If you need to retrieve a different Failure Theory for each ply, see, GetPly2.	
Input:	
`INT4 nPlyID`	The index of the ply to retrieve. Ply 1 is at the bottom.
Output:	
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
Return Code:	
`FE_NOT_AVAILABLE`	The ply ID that you specified for `nPlyID` does not exist. It must be between 1 and the number of plys in the layup.
Remarks/Usage:	
None	
Example:	
None	

GetPly2

Unpublished work. © 2025 Siemens

GetPly2 **(nPlyID, nMatlID, dThickness, dAngle, nGlobalPly, nFailureTheory)**	
Description:	
Returns all of the data for a selected ply.	
Input:	
`INT4 nPlyID`	The index of the ply to retrieve. Ply 1 is at the bottom.
Output:	
`INT4 nMatlID`	The ID of a material referenced by the ply.
`REAL8 dThickness`	The thickness of the ply.
`REAL8 dAngle`	The orientation angle of the ply.
`INT4 nGlobalPly`	The ID of a Global Ply referenced by this ply, if any.
`INT4 nFailureTheory`	The ID of a Failure Theory referenced by this ply, if any.
Return Code:	
`FE_NOT_AVAILABLE`	The ply ID that you specified for `nPlyID` does not exist. It must be between 1 and the number of plys in the layup.
Remarks/Usage:	
None	
Example:	
None	

GetPlyInfo

Unpublished work. © 2025 Siemens

GetPlyInfo

(nPlyID, bGetParentPlyInfo, nPlyType, nRefID)

Description:

Returns ply type and reference ID, if applicable, for either the specified ply itself or the *Parent* of a specified *Child Ply*.

Input:

`INT4 nPlyID`	The ID of the ply to retrieve. Ply 1 is at the bottom.
`BOOL bGetParentPlyInfo`	When True, returns the info of the *Parent* (ply or layup) if `nPlyID` is a *Child Ply*. If `nPlyID` has no *Parent*, `FE_NOT_AVAILABLE` will be returned. When False, returns the info of the specified ply, regardless of the ply type.

Output:

`INT4 or enum nPlyType`	The type of ply. 0=Ply (`FPLT_PLY`) 1=Ply Material (`FPLT_PLYMATERIAL`) 2=Layup (`FPLT_LAYUP`) 3=Layup-Symmetric (`FPLT_LAYUPSYMMETRIC`) 4=Layup-Anti (`FPLT_LAYUPANTI`) 5=Layup-AntiSymmetric (`FPLT_LAYUPANTISYMMETRIC`) 6=Matched Ply (`FPLT_MATCH`) 7=Matched Ply-Symmetric (`FPLT_MATCHSYMMETRIC`) 8=Matched Ply-Anti (`FPLT_MATCHANTI`) 9=Matched Ply-AntiSymmetric (`FPLT_MATCHANTISYMMETRIC`) 10=Child Ply (`FPLT_CHILD`)
`INT4 nRefID`	The ID of the referenced Ply Material, Layup, or Matched Ply.

Return Code:

`FE_NOT_EXIST`	The ply ID that you specified for `nPlyID` does not exist. It must be between 1 and the number of plys in the layup.
`FE_NOT_AVAILABLE`	Issued if `bGetParentPlyInfo` is set to True and ply has no *Parent*.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

GetAllPlyDefinition

GetAllPlyDefinition **(nNumPly, nPlyID, bOnOff, nPlyType, nRefID, nGlobalPly, nMatlID, dThickness, dAngle, nFailureTheory)**	
Description:	
Returns all of the data for all plys.	
Input:	
None	None
Output:	
`INT4 nNumPly`	The number of plys, and the number of entries in each of the arrays.
`INT4 nPlyID`	The ID of each ply.
`BOOL bOnOff`	Flag to indicate if a ply is *on* (True) or `off` (False) in the layup.
`INT4 nPlyType`	The type of ply. 0=Ply (`FPLT_PLY`) 1=Ply Material (`FPLT_PLYMATERIAL`) 2=Layup (`FPLT_LAYUP`) 3=Layup-Symmetric (`FPLT_LAYUPSYMMETRIC`) 4=Layup-Anti (`FPLT_LAYUPANTI`) 5=Layup-AntiSymmetric (`FPLT_LAYUPANTISYMMETRIC`) 6=Matched Ply (`FPLT_MATCH`) 7=Matched Ply-Symmetric (`FPLT_MATCHSYMMETRIC`) 8=Matched Ply-Anti (`FPLT_MATCHANTI`)

Unpublished work. © 2025 Siemens

	9=Matched Ply-AntiSymmetric (FPLT_MATCHANTISYMMETRIC) 10=Child Ply (FPLT_CHILD)
INT4 nRefID	The ID of the referenced Ply Material, Layup, or Matched Ply.
INT4 nGlobalPly[0..nNumPly-1]	The IDs of Global Plys referenced by each ply, if any.
INT4 nMatlID[0..nNumPly-1]	The IDs of materials referenced by each ply.
REAL8 dThickness[0..nNumPly-1]	The thicknesses of each ply.
REAL8 dAngle[0..nNumPly-1]	The orientation angles of each ply.
INT4 nFailureTheory[0..nNumPly-1]	The IDs of Failure Theory referenced by each ply, if any.
Return Code:	
FE_FAIL	Unable to retrieve ply data.
Remarks/Usage:	
Plies are stored in arrays from Top to Bottom, so index 0 is for Ply n, where n is the maximum ply ID, index 1 is for ply n-1, etc.	
Example:	
None	

GetAllPly

GetAllPly **(nNumPly, nMatlID, dThickness, dAngle, nGlobalPly)**	
Description:	
OBSOLETE Use GetAllPlyDefinitions instead (see GetAllPlyDefinition). Returns all of the data for all plys. If you need to review the Failure Theory for each ply (see GetAllPly2).	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
`INT4 nNumPly`	The number of plys, and the number of entries in each of the arrays.
`INT4 nMatlID[0..nNumPly-1]`	The IDs of materials referenced by each ply.
`REAL8 dThickness[0..nNumPly-1]`	The thicknesses of each ply.
`REAL8 dAngle[0..nNumPly-1]`	The orientation angles of each ply.
`INT4 nGlobalPly[0..nNumPly-1]`	The IDs of Global Plys referenced by each ply, if any.
Return Code:	
`FE_FAIL`	Unable to retrieve ply data.
Remarks/Usage:	
Plies are stored in arrays from Bottom to Top, so index 0 is for Ply 1, index 1 is for ply 2, etc	
Example:	
None	

GetAllPly2

`GetAllPly2` **`( nNumPly, nMatlID, dThickness, dAngle, nGlobalPly, nFailureTheory )`**	
Description:	
OBSOLETE Use `GetAllPlyDefinitions` instead (see **GetAllPlyDefinition**). Returns all of the data for all plys.	
Input:	
None	None
Output:	
`INT4 nNumPly`	The number of plys, and the number of entries in each of the arrays.

Unpublished work. © 2025 Siemens

`INT4 nMatlID[0..nNumPly-1]`	The IDs of materials referenced by each ply.
`REAL8 dThickness[0..nNumPly-1]`	The thicknesses of each ply.
`REAL8 dAngle[0..nNumPly-1]`	The orientation angles of each ply.
`INT4 nGlobalPly[0..nNumPly-1]`	The IDs of Global Plys referenced by each ply, if any.
`INT4 nFailureTheory[0..nNumPly-1]`	The IDs of Failure Theory referenced by each ply, if any.
Return Code:	
`FE_FAIL`	Unable to retrieve ply data.
Remarks/Usage:	
Plies are stored in arrays from Bottom to Top, so index 0 is for Ply 1, index 1 is for ply 2, etc.	
Example:	
None	

DeletePly

`DeletePly` **`( nPlyID )`**	
Description:	
This method removes a ply from the layup.	
Input:	
`INT4 nPlyID`	The index of the ply to delete. Ply 1 is at the bottom. The IDs of all plys at and above this number are decreased by one.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	The ply ID that you specified did not exist. It must be between 1 and the number of plys in the layup.
Remarks/Usage:	
None	
Example:	
None	

MaterialsUsed

`MaterialsUsed` **`( nMatlSetID, bUseOnPlysOnly)`**	
Description:	
This method retrieves material IDs used by a layup. Also, there is an option to only retrieve material IDs from plies which are *on* in the layup or from all plies, regardless of status.	
Input:	
`INT4 nMatlSetID`	ID of Set Object to store the IDs of all materials used in a layup.
`BOOL bUseOnPlysOnly`	Flag to indicate if only the material IDs which are used by plies which are *on* should be added to the Set Object. When True, only materials of *on* plies are added to the Set. When False, material IDs from all plies in the layup will be added to the Set.
Output:	
None	None
Return Code:	
`FE_FAIL`	Layup Object has no data.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GlobalPlysUsed

GlobalPlysUsed **(nGPSetID, bUseOnPlysOnly)**	
Description:	
This method retrieves global ply IDs used by a layup. Also, there is an option to only retrieve global ply IDs from plies which are *on* in the layup or from all plies, regardless of status.	
Input:	
`INT4 nGPSetID`	ID of Set Object to store the IDs of global plies used in a layup.
`BOOL bUseOnPlysOnly`	Flag to indicate if only the global ply IDs which are used by plies which are *on* should be added to the Set Object. When True, only global plies of *on* plies are added to the Set. When False, global ply IDs from all plies in the layup will be added to the Set.
Output:	
None	None
Return Code:	
`FE_FAIL`	Layup Object has no data.
Remarks/Usage:	
None	
Example:	
None	

PlyMaterialsUsed

Unpublished work. © 2025 Siemens

PlyMaterialsUsed	
(nPMSetID, bUseOnPlysOnly)	
Description:	
This method retrieves ply material IDs used by a layup. Also, there is an option to only retrieve ply material IDs from plies which are *on* in the layup or from all plies, regardless of status.	
Input:	
`INT4 nPMSetID`	ID of Set Object to store the IDs of ply materials used in a layup.
`BOOL bUseOnPlysOnly`	Flag to indicate if only the ply material IDs which are used by plies which are *on* should be added to the Set Object. When True, only ply material IDs of *on* plies are added to the Set. When False, ply material IDs from all plies in the layup will be added to the Set.
Output:	
None	None
Return Code:	
`FE_FAIL`	Layup Object has no data.
Remarks/Usage:	
None	
Example:	
None	

HasGlobalPly

HasGlobalPly	
(nGlobalPlyID, nPlyNum)	
Description:	
This method indicates whether or not a selected global ply is used in a layup.	

Unpublished work. © 2025 Siemens

Input:	
`INT4 nGlobalPlyID`	The ID of the Global Ply that you want to search for.
`BOOL bIndex`	If True, `nPlyNum` will be the value of a zero-based index in which the actual ply IDs are stored (i.e., ply 1 will return a value of 0, ply 2 will return a value of 1, etc). If False, `nPlyNum` is the actual ply ID.
Output:	
`INT4 nPlyNum`	Depending on the value set for `bIndex`, is either the index (0 is the bottom ply) or ID (1 is the bottom ply) of the ply that references `nGlobalPlyID` - if any.
Return Code:	
`FE_NOT_AVAILABLE`	The layup has no plys to search.
`FE_FAIL`	The selected global ply was not used.
Remarks/Usage:	
This method was changed for Femap version 2019.1, as previous versions did not include the second argument, `bIndex`. This was changed because the call actually returned a zero-based index value for `nPlyNum`, not the actual ply number. The `bIndex` argument now allows the user to choose to return the zero-based index or the actual ply number.	
Example:	
None	

HasMaterial

`HasMaterial`
`( nMatlID )`
Description:
This method indicates whether or not a selected material is used in a layup.
Input:

Unpublished work. © 2025 Siemens

`INT4 nMatlID`	The ID of the Material that you want to search for.
Output:	
None	None
Return Code:	
`FE_NOT_AVAILABLE`	The layup has no plys to search.
`FE_FAIL`	The selected material was not used.
Remarks/Usage:	
None	
Example:	
None	

HasFullGlobalPly

`HasFullGlobalPly`	
`( )`	
Description:	
This method indicates whether or not every ply in a layup references a global ply.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Every ply references a global ply.
`FE_FAIL`	At least one ply does not reference a global ply.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Clear

Clear (void)	
Description:	
This method removes all plys from the layup.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Compute

Compute (dynamic)
Description:
This method computes overall layup properties.

Unpublished work. © 2025 Siemens

Input:	
`BOOL dynamic`	If True, the results will be immediately sent to the **Entity Info** window (if it is open). If False, the results will be sent to the **Message** window.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Compute2

`Compute2` **`( nOption )`**
Description:
This method computes overall layup properties.
Input:

`INT4 nOption`	0=No Echo, just compute. 1=Compute and Echo to Entity Info. 2=Compute and Echo to **Message** Window.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

LoadBolt Object

LoadBolt objects correspond to bolt preloads in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for LoadBolt objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadBolt` method of the Femap Application object to create LoadBolt objects.

LoadBolt Object Properties

LoadBolts are always stored with their SetID equal to the setID of the load set, and the ID equal to the bolt region ID where they are applied.

When you create a new LoadBolt, it will always reference the active load set. You can change the set that it references simply by changing the setID parameter.

Property	Description
`INT4 color`	The load color.
`INT4 layer`	The ID of the layer associated with the load.
`REAL8 preload`	The elemental temperature.
`INT4 LoadDefinitionID`	The ID of the load definition that contains this load (0=None).
`INT4 PreloadType`	The Type of Bolt Preload. 0=Force, 1=Strain, 2=Displacement.

LoadBolt Object Methods

Unpublished work. © 2025 Siemens

AddArray

AddArray **(count, RegionID, Values)**	
Description:	
This method creates multiple Bolt Preloads. This method is obsolete and should be replaced by `PutArray`.	
Input:	
`INT4 count`	The number of bolt preloads to create, and the number of IDs in `RegionID`.
`INT4 RegionID[0..count-1]`	The Bolt Region IDs where preloads will be created. One preload is created on each ID.
`REAL8 Values[0..count-1]`	Bolt Preload Values. Refer to *preload* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color and setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the LoadBolt object, and create a new preload for each entry in the arrays.	
Example:	
None	

PutArray

Unpublished work. © 2025 Siemens

PutArray **(count, RegionID, Values)**	
Description:	
This method creates multiple Bolt Preloads.	
Input:	
`INT4 count`	The number of bolt preloads to create, and the number of IDs in `RegionID`.
`INT4 RegionID[0..count-1]`	The Bolt Region IDs where preloads will be created. One preload is created on each ID.
`REAL8 Values[0..count-1]`	Bolt Preload Values. Refer to *preload* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color and setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the LoadBolt object, and create a new preload for each entry in the arrays. For more information on how to create loads, see **Working with Loads and Constraints**.	
Example:	
None	

GetAllArray

Unpublished work. © 2025 Siemens

GetAllArray

(count, RegionID, Layer, Color, LoadDefinitionID, BoltType, BoltPreload, BoltLength)

Description:

This method returns information on multiple Bolt Preloads.

Input:

None	None

Output:

`INT4 count`	The number of bolt preloads to create, and the number of IDs in `RegionID`.
`INT4 RegionID[0..count-1]`	The Bolt Region IDs where preloads will be created. One preload is created on each ID.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.

`INT4 BoltType[0..count-1]`	The type of bolt.
	`[0] = Load_bp`
	`[1] = Strain_bp`
	`[2] = Disp_bp.`

`REAL8 BoltPreload[0..count-1]`	The Preload Values.
`REAL8 BoltLength[0..count-1]`	The Length Values.

Return Code:

`FE_NOT_AVAILABLE`	No bolt preloads on the mesh exist in the Load Set.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.

Remarks/Usage:

Unpublished work. © 2025 Siemens

Prior to using this method, you must properly specify the setID parameter in the object.
Example:
None

PutAllArray

PutAllArray **(count, RegionID, Layer, Color, LoadDefinitionID, BoltType, BoltPreload, BoltLength)**	
Description:	
This method creates multiple Bolt Preloads.	
Input:	
`INT4 count`	The number of bolt preloads to create, and the number of IDs in `RegionID`.
`INT4 RegionID[0..count-1]`	The Bolt Region IDs where preloads will be created. One preload is created on each ID.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.

`INT4 BoltType[0..count-1]`	The type of bolt.
	
	`[0] = Load_bp`
	`[1] = Strain_bp`
	`[2] = Disp_bp.`

`REAL8 BoltPreload[0..count-1]`	The Preload Values.
`REAL8 BoltLength[0..count-1]`	The Length Values.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	The bolt preloads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object. For more information on how to create loads, see **Working with Loads and Constraints**.	
Example:	
None	

LoadDefinition Objects

LoadDefinition objects correspond to the load definitions in your model. They group individual loads into related sets and provide a title for the set. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Load Definition objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadDefinition` method of the Femap Application object to create LoadDefinition objects.

LoadDefinition Object Properties

LoadDefinition objects are always stored with their SetID equal to the setID of the load set; however, their IDs simply go from 1 to N. To find a particular object, you must retrieve each object in the set and check the loads it contains.

Property	Description
`STRING title`	Load Definition title.
`INT4 LoadType`	The type of load that is referenced by this load definition. For more information on Load Types refer to the table in **Femap Constants**.
`INT4 DataType`	The type of entities that are referenced by this load definition. For more information on Entity Types refer to the table in **Entity**

Unpublished work. © 2025 Siemens

Property	Description
	Types. Can be FT_SURF_LOAD, FT_GEOM_LOAD, FT_NTHERM_LOAD, or FT_ETHERM_LOAD.

LoadDefinition Object Methods

PutAll

PutAll **(entID, enDataType, enLoadType, sTitle)**	
Description:	
This method stores a Load Definition with the specified ID along with all the available options.	
Input:	
INT4 entID	The ID of the Load Definition to store.
INT4 DataType	Refer to the property descriptions for each of these items.
INT4 LoadType	
STRING Title	
Output:	
None	None
Return Code:	
FE_FAIL	Unable to store the entity with the specified ID.
Remarks/Usage:	
Unlike the standard Put method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.	

Unpublished work. © 2025 Siemens

Example:
None

ResetNextLoad

ResetNextLoad	
(void)	
Description:	
This method initializes all options prior to calling the `NextLoad` method.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	None
Remarks/Usage:	
This method is used prior to calling `NextLoad` to specify which loads will be retrieved. It simply resets this object so that the next call to `NextLoad` will return the first load available in the object.	
Example:	
None	

NextLoad

NextLoad
(pDISPATCH)
Description:

Unpublished work. © 2025 Siemens

This method returns the next available load that is contained in the Load Definition.	
Input:	
None	None
Output:	
`OBJECT pDISPATCH`	The next load object. Depending on the type of the load definition this could be a `LoadGeom`, `LoadMesh`, `LoadNTemp` or `LoadGeom` object.
Return Code:	
`FE_FAIL`	There are no more loads of the specified type to retrieve. Call `ResetNextLoad` to go back to the first load, if you want to retrieve the loads again.
Remarks/Usage:	
None	
Example:	
This method, coupled with `ResetNextLoad` provides an easy way to retrieve all loads from a Load Definition. For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.	

CountLoads

`CountLoads` **`( void )`**	
Description:	
This method returns the number of loads referenced by a load definition, as a return code.	
Input:	
None	None
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`INT4 numLoads`	The number of loads referenced by this load definition.
Remarks/Usage:	
This method relies on the current Load Definition ID to count the referenced loads.	
Example:	
None	

IsTotalLoad

`IsTotalLoad`	
`( )`	
Description:	
This method indicates whether the current load definition represents a total load applied to geometry.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Load Definition represents a total load.
`FE_FAIL`	Not a total load.
`FE_INVALID`	Load is not a applied to geometry.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetTotalLoad

SetTotalLoad **(bTotalLoad)**	
Description:	
This method sets whether the current load definition is a total load applied on the geometry.	
Input:	
`BOOL bTotalLoad`	If TRUE, load is set to total load.
Output:	
None	None
Return Code:	
`FE_OK`	Load Definition is set to a total load.
`FE_INVALID`	Load is not a applied to geometry.
Remarks/Usage:	
None	
Example:	
None	

LoadETemp Objects

LoadETemp objects correspond to the elemental temperatures in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for LoadETemps objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadETemp` method of the Femap Application object to create LoadETemp objects.

Unpublished work. © 2025 Siemens

LoadETemp Object Properties

LoadETemps are always stored with their SetID equal to the setID of the load set, and the ID equal to the element ID where they are applied.

When you create a new LoadETemp, it will always reference the active load set. You can change the set that it references simply by changing the setID parameter.

Property	Description
`INT4 color`	The load color.
`INT4 layer`	The ID of the layer associated with the load.
`BOOL expanded`	True if this is a load from an expanded geometric load.
`REAL8 temp`	The elemental temperature.
`REAL8 phase`	The phase value associated with the temperature.
`INT4 function`	The ID of a function that defines time dependency of the temperature.
`INT4 LoadDefinitionID`	The ID of the load definition that contains this load (0=None).
`REAL8 gradient`	The elemental linear thermal gradient.

LoadETemp Object Methods

AddArray

`AddArray`
`( count, doValues, doFunctions, ElemID, Values, FunctionID )`
Description:

Unpublished work. © 2025 Siemens

This method creates multiple elemental temperatures. This method is obsolete and should be replaced by `PutArray`.	
Input:	
`INT4 count`	The number of temperatures to create, and the number of IDs in `ElemID`.
`BOOL doValues`	If True, then the Values array must contain load values to be used. If False, then whatever is specified in the current object is used and Values is ignored.
`BOOL doFunctions`	If True, then the `FunctionID` array must contain Function IDs to be used. If False, then whatever is specified in the current object is used and `FunctionID` is ignored.
`INT4 ElemID[0..count-1]`	The Element IDs where temperatures will be created. One temperature is created on each ID.
`REAL8 Values[0..count-1]`	Temperature Values. Refer to *temp* property.
`INT4 FunctionID[0..count-1]`	IDs of functions associated with each temperature. Specify 0 for constant values. Refer to *function* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color and setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the LoadETemp object, and create a new temperature for each entry in the arrays. This method puts data directly into the database, so there is no need to call `Put` after using this method.	
Example:	
None	

Unpublished work. © 2025 Siemens

PutArray

PutArray **(count, doValues, doFunctions, ElemID, Values, FunctionID)**	
Description:	
This method creates multiple elemental temperatures.	
Input:	
`INT4 count`	The number of temperatures to create, and the number of IDs in `ElemID`.
`BOOL doValues`	If True, then the Values array must contain load values to be used. If False, then whatever is specified in the current object is used and Values is ignored.
`BOOL doFunctions`	If True, then the `FunctionID` array must contain Function IDs to be used. If False, then whatever is specified in the current object is used and `FunctionID` is ignored.
`INT4 ElemID[0..count-1]`	The Element IDs where temperatures will be created. One temperature is created on each ID.
`REAL8 Values[0..count-1]`	Temperature Values. Refer to *temp* property.
`INT4 FunctionID[0..count-1]`	IDs of functions associated with each temperature. Specify 0 for constant values. Refer to *function* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color and setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the LoadETemp object, and create a new temperature for each entry in the	

Unpublished work. © 2025 Siemens

arrays. This method puts data directly into the database, so there is no need to call `Put` after using this method. For more information on how to create loads, see **Working with Loads and Constraints**.
Example:
None

GetAllArray

`GetAllArray` **`( count, doValues, doFunctions, ElemID, Layer, Color, LoadDefinitionID, Temperature, TempGradient, FunctionID )`**	
Description:	
This method creates returns elemental temperatures.	
Input:	
None	None
Output:	
`INT4 count`	The number of temperatures to create, and the number of IDs in `ElemID`.
`BOOL ExpandedGeomLoad[0..count-1]`	If True, the load was expanded from geometry.
`INT4 ElemID[0..count-1]`	The IDs of the nodes found.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.
`REAL8 Temperature[0..count-1]`	The elemental temperature.
`REAL8 TempGradient[0..count-1]`	The elemental linear thermal gradient.
`INT4 FunctionID[0..count-1]`	The ID of a function that defines time dependency of the temperature.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_NOT_AVAILABLE`	No temperature loads on mesh exist in the Load Set.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object.	
Example:	
None	

PutAllArray

`PutAllArray`	
`( count, doValues, doFunctions, ElemID, Layer, Color, LoadDefinitionID, Temperature, TempGradient, FunctionID )`	
Description:	
This method creates multiple elemental temperatures.	
Input:	
INT4 count	The number of temperatures to create, and the number of IDs in `ElemID`.
`BOOL ExpandedGeomLoad[0..count-1]`	If True, the load was expanded from geometry.
`INT4 ElemID[0..count-1]`	The IDs of the nodes found.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.
`REAL8 Temperature[0..count-1]`	The elemental temperature.

Unpublished work. © 2025 Siemens

`REAL8 TempGradient[0..count-1]`	The elemental linear thermal gradient.
`INT4 FunctionID[0..count-1]`	The ID of a function that defines time dependency of the temperature.
Output:	
None	None
Return Code:	
`FE_FAIL`	The temperature loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object. For more information on how to create loads, see **Working with Loads and Constraints**.	
Example:	
None	

LoadGeom Objects

LoadGeom objects correspond to the geometry-based loads in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for LoadGeom objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadGeom` method of the Femap Application object to create LoadGeom objects.

LoadGeom Object Properties

LoadGeom objects are always stored with their SetID equal to the setID of the load set; however, their IDs simply go from 1 to N. To find a particular object you must retrieve each object in the set and check its type and geomID.

When you create a new LoadGeom, it will always reference the active load set. You can change the set that it references simply by changing the setID parameter.

All of the data in a LoadGeom object can be accessed through the first set of properties listed below. The *Additional Properties* simply provide other names, which may be more convenient or descriptive, to access

Unpublished work. © 2025 Siemens

the same information. You will notice that the primary properties for LoadGeom records are the same as those for LoadMesh objects.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 geomID`	ID of the point, curve or surface that is loaded.

`INT4 addlID[0..1]`
or
`VARIANT vaddlID`
	Additional IDs. These are not used for most loads. They contain the additional nodes for nonlinear force loads.

`INT4 csys`	The ID of the coordinate system used to define the load. Regardless of this value, all loads that are vector-based are stored in global rectangular coordinates.

`INT4 function[0..4]`
or
`VARIANT vfunction`
	The IDs of functions associated with the various load values:
	[0]=Load func, Emissivity func
	[1]=Absorbtivity vs. Temp
	[2]=Temp vs Temp
	[3]=View Factor vs Time
	[4]=Phase vs. Freq

`INT4 dof[0..2]`
or
`VARIANT vdof`
	Array containing the face number (0) for face-based loads, or flags indicating which components are enabled for vector loads.

`BOOL flag[0..2]`
or
`VARIANT vflag`
	Flags specifying various load options.

Unpublished work. © 2025 Siemens

Property	Description
`vINT4 type`	The type of load being defined. For values, see **LoadGeom Load Types**.
`INT4 subtype`	An additional flag specifying more information about the type of load. Used for nonlinear transient force loads.
`INT4 geomtype`	The type of geometric entity that this load is applied to. For more information on Entity Types refer to the table in **Entity Types**. Can be `FT_SURFACE`, `FT_CURVE`, or `FT_POINT`. This property is Read-Only.
`INT4 layer`	The layer associated with the load.

`REAL8 load[0..4]`
or
`VARIANT vload`
	The load values. The location of specific values varies for each type of load, but generally follows the order of the values in the load creation dialog box.
	
	For loads with a single value, it is located in load[0]. X,Y,Z components are located in load[0],load[1],load[2].

`REAL8 fluxdir[0..2]`
or
`VARIANT vfluxdir`
	The components of the direction for certain elemental loads. For vector fluxes, the initial direction. This does not contain the direction for nodal vector loads like force or displacement. Those values are in the *load* values.

`INT4 DataSurface[0..2]`
or
`VARIANT vDataSurface`
	The IDs of Data Surfaces referenced by the various *load* values.

`INT4 color`	The load color.
`BOOL expanded`	True if this is an expanded geometric load.
`INT4 LoadDefinitionID`	The ID of the load definition that contains this load (0=None).
Geometry Load Specific Properties	

Unpublished work. © 2025 Siemens

Property	Description
`INT4 dirmode`	The direction setting for the load: 0=None, 1=Vector, 2=AlongCurve, 3= Normal to Plane, 4=Normal to Surface.

`REAL8 dirbase[0..2]`
or
`VARIANT vdirbase`
	The location of the base of the direction vector, if `dirmode`=1 or 3.

`REAL8 direction[0..2]`
or
`VARIANT vdirection`
	The components of the direction vector, if dirmode = 1 or 3.

`INT4 dirID`	The curve or surface ID if dirmode = 2 or 4.
`INT4 variation`	The load variation mode: 0=None/Constant, 1=Equation, 2=Function, 3=Interpolation.
`STRING varname`	The variable name used for load variation.
`STRING vareqn`	The equation defining the load variation if variation=1.
`INT4 varfunction`	The ID of the function used for load variation if variation=2.

`REAL8 varlocate [0..3][0..2]`
or
`VARIANT vvarlocate`
	The coordinates of the locations used for interpolation if variation=3. The x, y and z coordinates of a single point are stored in `varlocate(i,0)`, `varlocate(i,1)` and `varlocate(i,2)` respectively.

`REAL8 varvalue[0..3]`
or
`VARIANT vvarvalue`
	The values corresponding to each of the `varlocate` locations.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL midside`	If True, then loads at midside nodes are adjusted during load expansion to provide uniform loading. If False, midside nodes are treated just like other nodes during expansion.
Additional Properties	
`INT4 FaceNumber`	The element face number for face-based loads, like pressure.
`BOOL XOn`	True if the X component of a nodal load, like force or displacement is active. This refers to the X component in the definition CSys.
`BOOL YOn`	True if the Y component of a nodal load, like force or displacement is active. This refers to the Y component in the definition CSys.
`BOOL ZOn`	True if the Z component of a nodal load, like force or displacement is active. This refers to the Z component in the definition CSys.
`REAL8 X`	The X component of a nodal load, like force or displacement. In global rectangular coordinates.
`REAL8 Y`	The Y component of a nodal load, like force or displacement. In global rectangular coordinates.
`REAL8 Z`	The Z component of a nodal load, like force or displacement. In global rectangular coordinates.
`REAL8 Temp`	The temperature associated with the load.
`REAL8 Phase`	The phase associated with the load.
`INT4 LoadFunction`	The ID of the function associated with the primary load value.
`INT4 TempVsTime`	The ID of the function specifying the time dependence of temperature.
`INT4 PhaseVsFreq`	The ID of the function specifying the frequency dependence of phase.
`REAL8 LoadValue`	The primary load value.
`BOOL CornerPressure`	Flag specifying whether a pressure load has corner values, or a single constant value.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 Pressure`	The pressure value for a pressure load.
`REAL8 HeatGen`	The heat generation value for a heat generation load.
`REAL8 HeatFlux`	The heat flux value for a heat flux load.
`REAL8 Absorptivity`	The absorptivity for a radiation load.
`INT4 AbsorptivityVsTemp`	The ID of a function specifying temperature dependence of absorptivity.
`BOOL VectorFlux`	Flag specifying that an elemental flux load is a vector flux.
`REAL8 FluidValue`	Primary load value for a fluid load.
`BOOL PeriodicMaster`	Flag specifying a periodic condition load as Master.
`BOOL PeriodicDirection`	Flag specifying the periodic condition direction.
`REAL8 ConvectionCoeff`	The convection coefficient for a convection load.
`BOOL ForcedConvection`	Flag specifying that a convection load defines forced convection.
`BOOL FCDisableAdvection`	Flag specifying that a forced convection load is disabling advection.
`BOOL FCDisableConvection`	Flag specifying that a forced convection load is disabling convection.
`REAL8 FlowRate`	The flow rate for forced convection.
`REAL8 FlowDiameter`	The flow diameter for forced convection.
`REAL8 FlowAreaFactor`	The flow area factor for forced convection.
`REAL8 Emissivity`	The emissivity for a radiation load.
`INT4 EmissivityFunc`	The ID of function specifying the variation of emissivity.
`REAL8 ViewFactor`	The radiation view factor.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ViewFactorVsTime`	ID of the function specifying the time dependence of the view factor.
`BOOL EnclosureRadiation`	Flag specifying a radiation load as enclosure radiation.
`BOOL CanShade`	Flag specifying a radiation load as shading other regions.
`BOOL CanBeShaded`	Flag specifying a radiation load as being shaded by other regions.
`INT4 CavityNumber`	The cavity number for enclosure radiation.

`INT4 addl_ptID[0..3]`
or
`VARIANT vaddl_ptID`
	Additional Point IDs. These are not used for most loads. They contain the additional points for follower force and follower moment. By default, there are 2 points, but can be 4 points when the Normal to Plane option is enabled.

LoadGeom Object Methods

GetSearch

`GetSearch` **`( loadType, geomID )`**	
Description:	
This method searches for and retrieves an entity that is loading a specified geometric entity.	
Input:	
`INT4 loadType`	Type of load being applied. For values, see **LoadGeom Load Types**.
`INT4 geomID`	ID of geometry loaded. The `loadType` determines whether this should be a point, curve or surface ID.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	The specified entity does not exist or is not loaded.
Remarks/Usage:	
Since LoadGeom objects are not stored by a specified ID, but rather in sequential order, the normal `Get` method may not be useful if you are trying to find the loads on a specific curve or surface. The only way to do that would be to `Get` every load and check for the references that you want. This method does that for you. You specify the load type and point/curve/surface you want to search for, and it returns the loads on that entity. Be aware that this method does still search the entire set. If you are going to retrieve loads on a large number of geometry entities, it could be faster to retrieve them all using `Get`, and discard the ones you do not want.	
Example:	
None	

NextOnEntity

`NextOnEntity` **`( geomTYPE, geomSET )`**	
Description:	
Gets the next load that is defined on a selected set of geometric entities.	
Input:	
`INT4 geomTYPE`	The type of geometry where the loads are defined. Must be Point, Curve, or Surface. For entity information, see **Entity Types**.
`INT4 geomID`	The ID of a Set object that contains the IDs of the entities to be searched. Alternatively, if you specify a negative value, this is simply the ID of the single entity to search.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

FE_BAD_TYPE	geomTYPE is not one of the available entity types.
FE_FAIL	No additional loads can be found in this Load Set.

Remarks/Usage:

Before calling this method, the setID property of the object must be defined to search the desired Load Set. Also, you should call Reset before the first call to this method so it will retrieve the first available load.

Example:

Retrieve all of the loads in Load Set 7, defined on Surfaces 1 and 2:

```
Dim l As femap.LoadGeom
   Set l = App.feLoadGeom
   Dim s As femap.Set
   Set s = App.feSet

   s.AddRange(  1, 2, 1 )

   l.SetID = 7
   l.Reset()
   While l.NextOnEntity( FT_SURFACE, s.ID ) = FE_OK
       ...
   Wend
```

LoadGeom Load Types

The following table provides the list of available load types on geometry.

Unpublished work. © 2025 Siemens

Geometry Load Types (p = Point ; c = Curve ; s = Surface, n=on Node, e=on Element)					
81	pnForce	130	cnRotVelocity	167	snDisp
82	pnMoment	131	cnAccel	168	snRotDisp
83	pnDisp	132	cnRotAccel	169	snVelocity
84	pnRotDisp	133	cnTemp	170	snRotVelocity
85	pnVelocity	134	cnHeatFlux	171	snAccel
86	pnRotVelocity	135	cnHeatFluxPerLength	172	snRotAccel
87	pnAccel	136	cnHeatFluxAtNode	173	snTemp
88	pnRotAccel	137	cnHeatGen	174	snHeatFlux
89	pnTemp	138	cePressure	175	snHeatFluxPerArea
90	pnHeatFlux	139	ceTemp	176	snHeatFluxAtNode
91	pnHeatGen	140	ceHeatFlux	177	snHeatGen
92	pnPressure	141	ceConvection	178	sePressure
93	pnTotalPressure	142	ceRadiation	179	seTemp
94	pnScalar	143	ceHeatGen	180	seHeatFlux
95	pnSteamQuality	144	cnPressure	181	seConvection

Unpublished work. © 2025 Siemens

Geometry Load Types (p = Point ; c = Curve ; s = Surface, n=on Node, e=on Element)					
96	pnHumidity	145	cnTotalPressure	182	seRadiation
97	pnFluidHeight	146	cnScalar	183	seHeatGen
98	pnUnknownCondition	147	cnSteamQuality	184	snPressure
99	pnSlipCondition	148	cnHumidity	185	snTotalPressure
100	pnFanCurve	149	cnFluidHeight	186	snScalar
101	pnPeriodic	150	cnUnknownCondition	187	snSteamQuality
121	cnForce	151	cnSlipCondition	188	snHumidity
122	cnForcePerLength	152	cnFanCurve	189	snFluidHeight
123	cnForceAtNode	153	cnPeriodic	190	snUnknownCondition
124	cnMoment	161	snForce	191	snSlipCondition
125	cnMomentPerLength	162	snForcePerArea	192	snFanCurve
126	cnMomentAtNode	163	snForceAtNode	193	snPeriodic
127	cnDisp	164	snMoment	194	snTorque

Unpublished work. © 2025 Siemens

Geometry Load Types (p = Point ; c = Curve ; s = Surface, n=on Node, e=on Element)					
128	`cnRotDisp`	165	`snMomentPerArea`	195	`snBearing`
129	`cnVelocity`	166	`snMomentAtNode`		

LoadMesh Objects

LoadMesh objects correspond to the nodal and elemental loads (other than temperatures) in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for LoadMesh objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadMesh` method of the Femap Application object to create LoadMesh objects.

LoadMesh Object Properties

LoadMesh objects are always stored with their SetID equal to the setID of the load set; however, their IDs simply go from 1 to N. To find a particular object, you must retrieve each object in the set and check its type and meshID.

When you create a new LoadMesh, it will always reference the active load set. You can change the set that it references simply by changing the setID parameter.

All of the data in a LoadMesh object can be accessed through the first set of properties listed below. The *Additional Properties* simply provide other names, which may be more convenient or descriptive, to access the same information.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 meshID`	ID of the node or element that is loaded.
`INT4 addlID[0..3]`	Additional IDs. These are not used for most loads. They contain the additional nodes for nonlinear force loads, follower forces, and follower moments.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vaddlID`	
`INT4 csys`	The ID of the coordinate system used to define the load. Regardless of this value, all loads that are vector-based are stored in global rectangular coordinates.

`INT4 function[0..7]`
 	
or
 	
`VARIANT vfunction`
	The IDs of functions associated with the various load values:
	
	[0]=Load func, Emissivity func
	[1]=Absorbtivity vs. Temp
	[2]=Temp vs Temp
	[3]=View Factor vs Time
	[4]=Phase vs. Freq
	[5]=X component time function for Heat Flux.
	[6]=Y component time function for Heat Flux.
	[7]=Z component time function for Heat Flux.

`INT4 dof[0..2]`
 	
or
 	
`VARIANT vdof`
	Array containing the face number (0) for face based loads, or flags indicating which components are enabled for vector loads. Also, `dof[0]` is used to specify the *Distributed Load Direction* for a *Distributed Load* on a line element (type = 41, `eLineLoad`): 1 = Element X, 2 = Element Y, 3 = Element Z, 4 = Global X, 5 = Global Y, 6 = Global Z.

`BOOL flag[0..2]`
 	
or
 	
`VARIANT vflag`
	Flags specifying various load options.

`INT4 type`	The type of load being defined. Refer to the load type table for values.
`INT4 subtype`	An additional flag specifying more information about the type of load. Used for nonlinear transient force loads.
`INT4 layer`	The layer associated with the load.
`REAL8 load[0..]`	The load values. The location of specific values varies for each type of load, but generally follows

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vload`	the order of the values in the load creation dialog box. For loads with a single value, it is located in load[0]. X,Y,Z components are located in load[0],load[1],load[2].

`REAL8 fluxdir[0..2]`
 	
or
 	
`VARIANT vfluxdir`
	The components of the direction for certain elemental loads. For vector fluxes, the initial direction. This does not contain the direction for nodal vector loads like force or displacement. Those values are in the *load* values.

`BOOL VectorPressure`	If True, pressure load has a direction. Use `PressureDirection` or `vPressureDirection` to assign the direction of the pressure load.

`REAL8 PressureDirection[0..2]`
 	
or
 	
`VARIANT vPressureDirection`
	The components of the direction for elemental pressure loads.

`INT4 color`	The load color.
`BOOL expanded`	True if this is an expanded geometric load.
`INT4 LoadDefinitionID`	The ID of the load definition that contains this load (0=None).
Additional Properties	
`INT4 FaceNumber`	The element face number for face-based loads, like pressure.
`BOOL XOn`	True if the X component of a nodal load, like force or displacement, is active. This refers to the X component in the definition CSys.
`BOOL YOn`	True if the Y component of a nodal load, like force or displacement, is active. This refers to the Y component in the definition CSys.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL ZOn`	True if the Z component of a nodal load, like force or displacement, is active. This refers to the Z component in the definition CSys.
`REAL8 X`	The X component of a nodal load, like force or displacement. In global rectangular coordinates.
`REAL8 Y`	The Y component of a nodal load, like force or displacement. In global rectangular coordinates.
`REAL8 Z`	The Z component of a nodal load, like force or displacement. In global rectangular coordinates.
`REAL8 Temp`	The temperature associated with the load.
`REAL8 Phase`	The phase associated with the load.
`INT4 LoadFunction`	The ID of the function associated with the primary load value.
`INT4 TempVsTime`	The ID of the function specifying the time dependence of temperature.
`INT4 PhcaseVsFreq`	The ID of the function specifying the frequency dependence of phase.
`REAL8 LoadValue`	The primary load value.
`BOOL CornerPressure`	Flag specifying whether a pressure load has corner values, or a single constant value.
`REAL8 Pressure`	The pressure value for a pressure load.
`REAL8 HeatGen`	The heat generation value for a heat generation load.
`REAL8 HeatFlux`	The heat flux value for a heat flux load.
`REAL8 Absorptivity`	The absorptivity for a radiation load.
`INT4 AbsorptivityVsTemp`	The ID of a function specifying temperature dependence of absorptivity.
`BOOL VectorFlux`	Flag specifying that an elemental flux load is a vector flux.
`REAL8 FluidValue`	Primary load value for a fluid load.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL PeriodicMaster`	Flag specifying a periodic condition load as Master.
`BOOL PeriodicDirection`	Flag specifying the periodic condition direction.
`REAL8 ConvectionCoeff`	The convection coefficient for a convection load.
`BOOL ForcedConvection`	Flag specifying that a convection load defines forced convection.
`BOOL FCDisableAdvection`	Flag specifying that a forced convection load is disabling advection.
`BOOL FCDisableConvection`	Flag specifying that a forced convection load is disabling convection.
`REAL8 FlowRate`	The flow rate for forced convection.
`REAL8 FlowDiameter`	The flow diameter for forced convection.
`REAL8 FlowAreaFactor`	The flow area factor for forced convection.
`REAL8 Emissivity`	The emissivity for a radiation load.
`INT4 EmissivityFunc`	The ID of function specifying the variation of emissivity.
`REAL8 ViewFactor`	The radiation view factor.
`INT4 ViewFactorVsTime`	ID of the function specifying the time dependence of the view factor.
`BOOL EnclosureRadiation`	Flag specifying a radiation load as enclosure radiation.
`BOOL CanShade`	Flag specifying a radiation load as shading other regions.
`BOOL CanBeShaded`	Flag specifying a radiation load as being shaded by other regions.
`INT4 CavityNumber`	The cavity number for enclosure radiation.

LoadMesh Object Methods

Unpublished work. © 2025 Siemens

GetSearch

GetSearch **(loadType, meshID, faceID)**	
Description:	
This method searches for and retrieves an entity that is loading a specified node or element entity and loads the LoadMesh Object with that entity.	
Input:	
`INT4 loadType`	Type of load being applied. For values, see LoadMesh Load Types.
`INT4 meshID`	ID of node or element being loaded. The `loadType` determines whether this should be a node ID or an element ID.
`INT4 faceID`	For certain face-based elemental loads (pressure, flux, etc.) you must also specify the element face that is being loaded. For all other load types, this option is ignored.
Output:	
None	None
Return Code:	
`FE_FAIL`	The specified entity does not exist or is not loaded.
Remarks/Usage:	
Since LoadMesh objects are not stored by a specified ID, but rather in sequential order, the normal `Get` method may not be useful if you are trying to find the loads on a specific node or element. The only way to do that would be to `Get` every load and check for the references that you want. This method does that for you. You specify the load type and node/element you want to search for, and it returns the loads on that entity. Be aware that this method does still search the entire set. If you are going to retrieve loads on a large number of mesh entities, it could be faster to retrieve them all using `Get`, and discard the ones you do not want.	
Example:	
None	

Unpublished work. © 2025 Siemens

Add

Add

(setID, loadType, nCSys, DOF, Values, FunctionID)

Description:

This method creates multiple nodal or elemental loads

Input:

`INT4 setID`	The ID of a Set object that contains the nodes or elements where loads will be applied. Alternatively, if you specify a negative value, this is simply the ID of the single node or element to load.
`INT4 loadType`	The type of load being defined. Refer to *type* property.
`INT4 nCSys`	The ID of the Coordinate System where load values are being defined.
`INT4 DOF[0..2]`	Degree of Freedom flags for forces (True=On, False=Off). For Pressures and other face-based loads the first DOF for each element is the faceID (1-6) for the load. Refer to *dof* property.
`REAL8 Values[0..4]`	Load Values, 5 per load. Refer to *load* property.
`INT4 FunctionID[0..4]`	IDs of functions associated with each load value. Specify 0 for constant values. 5 per load. Refer to *function* property.

Output:

None	None

Return Code:

`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.

Remarks/Usage:

Prior to using this method, you must properly specify the other parameters in the object - such as, color, setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the loadmesh object, and create a new load for each entry in the set.

Unpublished work. © 2025 Siemens

Example:
None

AddArray

AddArray **(count, doDOF, doValues, doFunctions, NodeElemID, DOF, Values, FunctionID)**	
Description:	
This method creates multiple nodal or elemental loads. This method is obsolete and should be replaced by `PutArray`.	
Input:	
`INT4 count`	The number of loads to create, and the number of IDs in `NodeElemID`.
`BOOL doDOF`	If True, then the DOF array must contain the DOF flags/Face IDs to be used. If False, then whatever is specified in the current object is used, and DOF is ignored.
`BOOL doValues`	If True, then the Values array must contain load values to be used. If False, then whatever is specified in the current object is used and Values is ignored.
`BOOL doFunctions`	If True, then the FunctionID array must contain Function IDs to be used. If False, then whatever is specified in the current object is used and FunctionID is ignored.
`INT4 NodeElemID[0..count-1]`	The Node or Element IDs where loads will be created. One load is created on each ID.
`INT4 DOF[0..(3*count)-1]`	Degree of Freedom flags for forces (True=On, False=Off). For Pressures and other face-based loads the first DOF for each element is the faceID (1-6) for the load. There are three entries per load. Refer to *dof* property.
`REAL8 Values[0..(5*count)-1]`	Load Values, 5 per load. Refer to “load” property.

Unpublished work. © 2025 Siemens

`INT4 FunctionID[0..(5*count)-1]`	IDs of functions associated with each load value. Specify 0 for constant values. 5 per load. Refer to “function” property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, load type, color, setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the loadmesh object, and create a new load for each entry in the arrays.	
Example:	
None	

GetAllArray

`GetAllArray`

```
( count, ExpandedGeomLoad, loadType, loadSubType, NodeElemID, Layer,
Color, LoadDefinitionID,
nCSysID, DOFFace, Values, FunctionID, NodeElemAddID, flag, Direction )
```

Description:	
This method fills information about all of the LoadMesh objects in a Load Set.	
Input:	
None	None
Output:	
`INT4 count`	The number of loads to create, and the number of IDs in `NodeElemID`.
`BOOL ExpandedGeomLoad[0..count-1]`	If True, the load was expanded from geometry.

Unpublished work. © 2025 Siemens

`INT4 loadType[0..count-1]`	The type of load being defined. Refer to the load type table for values. For all load types, see **LoadMesh Load Types**.
`INT4 loadSubType[0..count-1]`	An additional flag specifying more information about the type of load. Used for nonlinear transient force loads.
`INT4 NodeElemID[0..count-1]`	The IDs of the nodes or elements found.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.
`INT4 nCSysID[0..count-1]`	The ID of the coordinate system used to define the load. Regardless of this value, all loads that are vector-based are stored in global rectangular coordinates.
`INT4 DOFFace[0..(3*count)-1]`	Array containing the face number (0) for face based loads, or flags indicating which components are enabled for vector loads. Also, `dof[0]` is used to specify the *Distributed Load Direction* for a *Distributed Load* on a line element (type = 41, `eLineLoad`): 1 = Element X, 2 = Element Y, 3 = Element Z, 4 = Global X, 5 = Global Y, 6 = Global Z.
`REAL8 Values[0..(5*count)-1]`	The load values. The location of specific values varies for each type of load, but generally follows the order of the values in the load creation dialog box. For loads with a single value, it is located in load[0]. X,Y,Z components are located in load[0],load[1],load[2].
`INT4 FunctionID[0..(8*count)-1]`	The IDs of functions associated with the various load values: [0]=Load func, Emissivity func [1]=Absorbtivity vs. Temp [2]=Temp vs Temp [3]=View Factor vs Time [4]=Phase vs. Freq [5]=X component time function for Heat Flux. [6]=Y component time function for Heat Flux.

Unpublished work. © 2025 Siemens

	[7]=Z component time function for Heat Flux.
`INT4 NodeElemAddID[0..count-1]`	Additional IDs. These are not used for most loads. They contain the additional nodes for nonlinear force loads.
`BOOL flag[0..(3*count)-1]`	Flags specifying various load options.
`REAL8 Direction[0..(3*count)-1]`	The components of the direction for certain elemental loads. For vector fluxes, the initial direction. This does not contain the direction for nodal vector loads like force or displacement. Those values are in the *load* values.
Return Code:	
`FE_NOT_AVAILABLE`	No loads on mesh exist in the Load Set.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the setID parameter in the object.	
Example:	
None	

PutArray

`PutArray`
`( count, doDOF doValues, doFunctions, NodeElemID, DOF, Values, FunctionID )`
Description:
This method creates multiple nodal or elemental loads.
Input:

`INT4 count`	The number of loads to create, and the number of IDs in `NodeElemID`.
`BOOL doDOF`	If True, then the DOF array must contain the DOF flags/Face IDs to be used. If False, then whatever is

Unpublished work. © 2025 Siemens

	specified in the current object is used, and DOF is ignored.
`BOOL doValues`	If True, then the Values array must contain load values to be used. If False, then whatever is specified in the current object is used and Values is ignored.
`BOOL doFunctions`	If True, then the FunctionID array must contain Function IDs to be used. If False, then whatever is specified in the current object is used and FunctionID is ignored.
`INT4 NodeElemID[0..count-1]`	The Node or Element IDs where loads will be created. One load is created on each ID.
`INT4 DOF[0..(3*count)-1]`	Degree of Freedom flags for forces (True=On, False=Off). For Pressures and other face-based loads the first DOF for each element is the faceID (1-6) for the load. There are three entries per load. Refer to `dof` property.
`REAL8 Values[0..(5*count)-1]`	Load Values, 5 per load. Refer to `load` property.
`INT4 FunctionID[0..(5*count)-1]`	IDs of functions associated with each load value. Specify 0 for constant values. 5 per load. Refer to *function* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, load type, color, setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the loadmesh object, and create a new load for each entry in the arrays. For more information on how to create loads, see **Working with Loads and Constraints**.	
Example:	
None	

Unpublished work. © 2025 Siemens

PutAllArray

PutAllArray

(bDeleteAllExistingLoads, count, ExpandedGeomLoad, loadType, loadSubType, NodeElemID, Layer, Color, LoadDefinitionID, nCSysID, DOFFace, Values, FunctionID, NodeElemAddID, flag, Direction)

Description:

This method creates multiple nodal or elemental loads.

Input:

`BOOL bDeleteAllExistingLoads`	If True, all loads in the setID will be deleted beforehand.
`INT4 count`	The number of loads to create, and the number of IDs in `NodeElemID`.
`BOOL ExpandedGeomLoad[0..count-1]`	If True, the load was expanded from geometry.
`INT4 loadType[0..count-1]`	The type of load being defined. Refer to the load type table for values. For all load types, see LoadMesh Load Types.
`INT4 loadSubType[0..count-1]`	An additional flag specifying more information about the type of load. Used for nonlinear transient force loads.
`INT4 NodeElemID[0..count-1]`	The IDs of the nodes or elements found.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.
`INT4 nCSysID[0..count-1]`	The ID of the coordinate system used to define the load. Regardless of this value, all loads that are vector-based are stored in global rectangular coordinates.
`INT4 DOFFace[0..(3*count)-1]`	Array containing the face number (0) for face based loads, or flags indicating which components are enabled for vector loads. Also, `dof[0]` is used

Unpublished work. © 2025 Siemens

	to specify the *Distributed Load Direction* for a *Distributed Load* on a line element (type = 41, `eLineLoad`): 1 = Element X, 2 = Element Y, 3 = Element Z, 4 = Global X, 5 = Global Y, 6 = Global Z.
`REAL8 Values[0..(5*count)-1]`	The load values. The location of specific values varies for each type of load, but generally follows the order of the values in the load creation dialog box. For loads with a single value, it is located in load[0]. X,Y,Z components are located in load[0],load[1],load[2].
`INT4 FunctionID[0..(8*count)-1]`	The IDs of functions associated with the various load values: [0]=Load func, Emissivity func [1]=Absorbtivity vs. Temp [2]=Temp vs Temp [3]=View Factor vs Time [4]=Phase vs. Freq [5]=X component time function for Heat Flux. [6]=Y component time function for Heat Flux. [7]=Z component time function for Heat Flux.
`INT4 NodeElemAddID[0..(2*count)-1]`	Additional IDs. These are not used for most loads. They contain the additional nodes for nonlinear force loads.
`BOOL flag[0..(3*count)-1]`	Flags specifying various load options.
`REAL8 Direction[0..(3*count)-1]`	The components of the direction for certain elemental loads. For vector fluxes, the initial direction. This does not contain the direction for nodal vector loads like force or displacement. Those values are in the *load* values.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Prior to using this method, you must properly specify the setID parameter in the object. For more information on how to create loads, see **Working with Loads and Constraints**.
Example:
None

NextOnEntity

`NextOnEntity` **`( entityTYPE, meshSET )`**	
Description:	
Gets the next load that is defined on a selected set of nodes or elements.	
Input:	
`INT4 entityTYPE`	The type of geometry where the loads are defined. Must be Node or Element. For entity information, see **Entity Types**.
`INT4 geomID`	The ID of a Set object that contains the IDs of the entities to be searched. Alternatively, if you specify a negative value, this is simply the ID of the single entity to search.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	`entityTYPE` is not one of the available entity types.
`FE_FAIL`	No additional loads can be found in this Load Set.
Remarks/Usage:	
Before calling this method, the setID property of the object must be defined to search the desired Load Set. Also, you should call Reset before the first call to this method so it will retrieve the first available load.	

Unpublished work. © 2025 Siemens

Example:

Retrieve all of the loads in Load Set 7, defined on Surfaces 1 and 2:

```
Dim l As femap.LoadMesh
   Set l = App.feLoadMesh
   Dim s As femap.Set
   Set s = App.feSet

   s.AddRange(  1, 2, 1 )

   l.SetID = 7
   l.Reset()
   While l.NextOnEntity( FT_ELEM, s.ID ) = FE_OK
       . . .
   Wend
```

LoadMesh Load Types

The following table provides the list of available load types on nodes and elements.

Mesh Load Types **(n = Nodal; e = Elemental)**					
1	`nForce`	11	`nHeatGen`	20	`nSlipCondition`
2	`nMoment`	12	`Transient`	21	`nFanCurve`
3	`nDisplacement`	13	`nPressure`	22	`nPeriodic`

Unpublished work. © 2025 Siemens

Mesh Load Types (n = Nodal; e = Elemental)					
4	`nRotDisplacement`	14	`nTotalPressure`	41	`eLineLoad`
5	`nVelocity`	15	`nScalar`	42	`ePressure`
6	`nRotVelocity`	16	`nSteamQuality`	44	`eHeatFlux`
7	`nAcceleration`	17	`nHumidity`	45	`eConvection`
8	`nRotAcceleration`	18	`nFluidHeight`	46	`eRadiation`
10	`nHeatFlux`	19	`nUnknownCondition`	47	`eHeatGen`

LoadNTemp Objects

LoadNTemp objects correspond to the nodal temperatures in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for LoadNTemps objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadNTemp` method of the Femap Application object to create LoadNTemp objects.

LoadNTemp Object Properties

LoadNTemps are always stored with their SetID equal to the setID of the load set, and the ID equal to the node ID where they are applied.

When you create a new LoadNTemp, it will always reference the active load set. You can change the set that it references simply by changing the *setID* parameter.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 color`	The load color.
`INT4 layer`	The ID of the layer associated with the load.
`BOOL expanded`	True if this is a load from an expanded geometric load.
`REAL8 temp`	The nodal temperature.
`REAL8 phase`	The phase value associated with the temperature.
`INT4 function`	The ID of a function that defines time dependency of the temperature.
`INT4 LoadDefinitionID`	The ID of the load definition that contains this load (0=None).

LoadNTemp Object Methods

AddArray

`AddArray` **`( count, doValues, doFunctions, NodeID, Values, FunctionID )`**	
Description:	
This method creates multiple nodal temperatures. This method is obsolete and should be replaced by `PutArray`.	
Input:	
`INT4 count`	The number of temperatures to create, and the number of IDs in NodeID.
`BOOL doValues`	If True, then the Values array must contain load values to be used. If False, then whatever is specified in the current object is used and Values is ignored.

Unpublished work. © 2025 Siemens

`BOOL doFunctions`	If True, then the FunctionID array must contain Function IDs to be used. If False, then whatever is specified in the current object is used and FunctionID is ignored.
`INT4 NodeID[0..count-1]`	The Node IDs where temperatures will be created. One temperature is created on each ID.
`REAL8 Values[0..count-1]`	Temperature Values. Refer to *temp* property.
`INT4 FunctionID[0..count-1]`	IDs of functions associated with each temperature. Specify 0 for constant values. Refer to *function* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color and setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the LoadNTemp object, and create a new temperature for each entry in the arrays. This method puts data directly into the database, so there is no need to call `Put` after using this method.	
Example:	
None	

PutArray

`PutArray`
`( count, doValues, doFunctions, NodeID, Values, FunctionID )`
Description:
This method creates multiple nodal temperatures.

Unpublished work. © 2025 Siemens

Input:	
INT4 count	The number of temperatures to create, and the number of IDs in NodeID.
`BOOL doValues`	If True, then the Values array must contain load values to be used. If False, then whatever is specified in the current object is used and Values is ignored.
`BOOL doFunctions`	If True, then the FunctionID array must contain Function IDs to be used. If False, then whatever is specified in the current object is used and FunctionID is ignored.
`INT4 NodeID[0..count-1]`	The Node IDs where temperatures will be created. One temperature is created on each ID.
`REAL8 Values[0..count-1]`	Temperature Values. Refer to *temp* property.
`INT4 FunctionID[0..count-1]`	IDs of functions associated with each temperature. Specify 0 for constant values. Refer to *function* property.
Output:	
None	None
Return Code:	
`FE_FAIL`	The Loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	
Prior to using this method, you must properly specify the other parameters in the object - such as, color and setID, and any other types required for your loads. This method simply uses the data you specify here to overwrite specific values in the LoadNTemp object, and create a new temperature for each entry in the arrays. This method puts data directly into the database, so there is no need to call `Put` after using this method. For more information on how to create loads, see **Working with Loads and Constraints**.	
Example:	
None	

GetAllArray

Unpublished work. © 2025 Siemens

GetAllArray

(count, ExpandedGeomLoad, NodeID, Layer, Color, LoadDefinitionID, Temperature, FunctionID)

Description:

This method finds all nodal temperatures.

Input:

None	None

Output:

`INT4 count`	The number of temperatures, and the number of IDs in NodeID.
`BOOL ExpandedGeomLoad[0..count-1]`	If True, the load was expanded from geometry.
`INT4 NodeID[0..count-1]`	The IDs of the nodes found.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The IDs of the Load Definitions.
`REAL8 Temperature[0..count-1]`	The nodal temperature.
`INT4 FunctionID[0..count-1]`	The ID of a function that defines time dependency of the temperature.

Return Code:

`FE_NOT_AVAILABLE`	No temperature loads on mesh exist in the Load Set.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.

Remarks/Usage:

Prior to using this method, you must properly specify the setID parameter in the object.

Example:

None

Unpublished work. © 2025 Siemens

PutAllArray

PutAllArray **(count, ExpandedGeomLoad, NodeID, Layer, Color, LoadDefinitionID, Temperature, FunctionID)**	
Description:	
This method creates multiple nodal temperatures.	
Input:	
`INT4 count`	The number of temperatures to create, and the number of IDs in NodeID.
`BOOL ExpandedGeomLoad[0..count-1]`	If True, the load was expanded from geometry.
`INT4 NodeID[0..count-1]`	The IDs of the nodes found.
`INT4 Layer[0..count-1]`	The IDs of the layers the loads are on.
`INT4 Color[0..count-1]`	The IDs of the load color.
`INT4 LoadDefinitionID[0..count-1]`	The ID of the coordinate system used to define the load. Regardless of this value, all loads that are vector-based are stored in global rectangular coordinates.
`REAL8 Temperature[0..count-1]`	The nodal temperature.
`INT4 FunctionID[0..count-1]`	The ID of a function that defines time dependency of the temperature.
Output:	
None	None
Return Code:	
`FE_FAIL`	The temperature loads could not be created.
`FE_NO_MEMORY`	Unable to allocate memory to store arrays of data.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Prior to using this method, you must properly specify the setID parameter in the object. For more information on how to create loads, see **Working with Loads and Constraints**.
Example:
None

LoadSet Objects

LoadSet objects correspond to the load sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Load Set objects. For more information, see **Common Entity Properties and Methods**.

Use the `feLoadSet` method of the Femap Application object to create LoadSet objects.

LoadSet Object Properties

LoadSets are always stored with a `SetID=1`, and the ID equal to the load set ID.

LoadSet objects are really simply placeholders for global data for the load set. The actual loads are defined in other objects.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`STRING title`	Set title (maximum 79 characters).
`BOOL IsCombination`	If True, this is a Nastran LOAD Combination Load Set.
`BOOL IsSequence`	If True, this is a Nastran DTEMP Sequence Load Set.
`REAL8 CombinedScaleFactor`	Overall scale factor for Nastran LOAD combination.
`INT4 layer`	The layer associated with the set.
`BOOL BodyAccelOn`	Body Load - Accelerations are active if True.

Unpublished work. © 2025 Siemens

Property	Description

`REAL8 BodyAccel[0..5]`
 	
or
 	
`VARIANT vBodyAccel`
	Body Load - Translational and rotational acceleration values.

`INT4 BodyAccelFunction[0..5]`
 	
or
 	
`VARIANT vBodyAccelFunction`
	Body Load - IDs of functions associated with Translational and rotational acceleration values.

`BOOL BodyVaryingAccelOn`	Body Load - Varying Translational Accelerations are active if True.

`REAL8 BodyVaryingAccel[0..2]`
 	
or
 	
`VARIANT vBodyVaryingAccel`
	Body Load - Varying Translational acceleration values.

`REAL8 BodyVaryingAccelAxis`	Body Load - Varying Translational acceleration axis to vary along (0=X, 1=Y, 2=Z).
`INT4 BodyVaryingAccelFunction`	Body Load - IDs of function associated with Varying Translational acceleration values.
`BOOL BodyVelocityOn`	Body Load - Rotational Velocities are active if True.

`REAL8 BodyVelocity[0..2]`
 	
or
 	
`VARIANT vBodyVelocity`
	Body Load - Rotational Velocity.

`INT4 BodyVelocityFunction[0..2]`	Body Load - IDs of functions associated with Rotational Velocity values.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vBodyVelocityFunction`	

`REAL8 BodyOrigin[0..2]`
 	
or
 	
`VARIANT vBodyOrigin`
	Body Load - The center of rotation.

`INT4 BodyLoadCSys`	The ID of the coordinate system used to define the various body loads.
`BOOL BodyDefTempOn`	Body Load - Default Temperature is active if True.
`REAL8 BodyDefTemp`	Body Load - The default temperature.
`BOOL BodyRefTempOn`	Body Load - Reference Temperature is active if True.
`REAL8 BodyRefTemp`	Body Load - The reference temperature.
`BOOL BodyGustOn`	Body Load - Vertical Gust Load is active if True.
`REAL8 BodyGustWG`	Create Gust Load - Specifies value for Scale Factor (Gust Vel/Forward Vel).
`REAL8 BodyGustX0`	Create Gust Load - Specifies value for Streamwise Location.
`INT4 BodyGustVelocityOpt`	Create Gust Load - When set to **0..Match Analysis**, uses Velocity value from NASTRAN Aerodynamic Data dialog box in the Analysis Set Manager. When set to **1..On Load**, uses Vehicle Velocity specified by `BodyGustVelocity`.
`REAL8 BodyGustVelocity`	Create Gust Load - Specifies value for Vehicle Velocity. Only used when `BodyGustVelocityOpt` is set to **1..On Load**.
`INT4 BodyGustAppliedNode`	Create Gust Load - Specifies ID of node to use for Applied Node.
`INT4 BodyGustFunction`	Create Gust Load - Specifies ID of function to use Time/Frequency Dependence.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 StefanBoltzmann`	Heat Transfer - Stefan-Boltzmann Constant.
`INT4 RadiationAmbientElem`	Heat Transfer - ID of element used for ambient temp in enclosure radiation.
`BOOL bRadViewFactorZTOL`	Heat Transfer - Flag to write specified ZTOL value Nastran input file (Not in Femap User Interface).
`REAL8 RadViewFactorZTOL`	Heat Transfer - ZTOL value (Not in Femap User Interface).
`REAL8 TempOffset`	Heat Transfer - Offset of Temperatures from Absolute Zero.
`REAL8 FreeConvExponent`	Heat Transfer - Free Convection Exponent.
`BOOL FreeConvAlternateForm`	Heat Transfer - Flag for Alternate Free Convection Formulation.
`REAL8 FCFluidConductivity`	Heat Transfer - Forced Convection Fluid Conductivity.
`REAL8 FCFluidHeatCapacity`	Heat Transfer - Forced Convection Fluid Heat Capacity.
`REAL8 FCFluidViscosity`	Heat Transfer - Forced Convection Fluid Viscosity.
`REAL8 FCFluidDensity`	Heat Transfer - Forced Convection Fluid Density.
`INT4 FCFluidConductivityFunc`	Heat Transfer - Forced Convection Fluid Conductivity Function.
`INT4 FCFluidHeatCapacityFunc`	Heat Transfer - Forced Convection Fluid Heat Capacity Function.
`INT4 FCFluidViscosityFunc`	Heat Transfer - Forced Convection Fluid Viscosity Function.
`INT4 FCFluidDensityFunc`	Heat Transfer - Forced Convection Fluid Density Function.
`REAL8 FCConstantCoeff`	Heat Transfer - Forced Convection Equation Constant Coefficient.
`REAL8 FCReynoldsExp`	Heat Transfer - Forced Convection Reynolds Exponent.
`REAL8 FCPrandtlIn`	Heat Transfer - Forced Convection Prandtl Exponent into Fluid.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 FCPrandtlOut`	Heat Transfer - Forced Convection Prandtl Exponent out of Fluid.
`BOOL FCAlternateForm`	Heat Transfer - Flag for Alternate Forced Convection Formulation.
`BOOL FCExcludeConvective`	Heat Transfer - Flag to Exclude Forced Convection Convective Energy Flow.
`INT4 NLOn`	Flag for Nonlinear Analysis: 0=Off, 1=Static, 2=Creep, 3=Transient.
`INT4NLIncrements`	Nonlinear - Number of Increments for Nonlinear Analysis.
`REAL8NLTimeIncrement`	Nonlinear - Time Increment.
`INT4 NLMaxIterPerStep`	Nonlinear - Max Iterations per Step.

`BOOL NLConvergenceFlag[0..2]`
or
`VARIANT vNLConvergenceFlag`
	Nonlinear - Convergence Flags for 0=Displacement, 1=Load and 2=Work.

`REAL8 NLConvergenceVal[0..2]`
or
`VARIANT vNLConvergenceVal`
	Nonlinear - Convergence Tolerances for Displacement, Load and Work.

`INT4 NLStiffnessMethod`	Nonlinear - Stiffness Update Method.
`INT4 NLStiffnessIter`	Nonlinear - Iterations before Stiffness Update.
`INT4 NLIntermediateOutput`	Nonlinear - Intermediate Output Type.
`INT4 NLOutputInterval`	Nonlinear - Output Interval.
`INT4 NLArcLengthMethod`	Nonlinear - Nonlinear Arc-Length Solution Strategy (Arc-Length if nonzero).

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NLSolutionOverride`	Nonlinear - Solution Strategy Overrides: 0=none/ advanced, 1=Full Newton Raphson, 2=Modified Newton Raphson.
`BOOL NLNewtRaphLineSearch`	Nonlinear - Modified Newton Line Search Override (1=Skip).
`BOOL NLNewtRaphQuasiNewton`	Nonlinear - Modified Newton Quasi-Newton Override (1=Skip).
`BOOL NLNewtRaphBisection`	Nonlinear - Modified Newton Bisection Override (1=Skip).
`REAL8 NLArcScaleConstLoad`	Nonlinear - Arc Length Method Constraint Load Scale.
`REAL8 NLArcMinAdjust`	Nonlinear - Min Arc Length Adjust.
`REAL8 NLArcMaxAdjust`	Nonlinear - Max Arc Length Adjust.
`INT4 NLArcConstraintType`	Nonlinear - Arc Length Constraint Type.
`INT4 NLArcDesiredIter`	Nonlinear - Arc Length Desired Iterations.
`INT4 NLArcMaxIncrement`	Nonlinear - Arc Length Max Steps.
`INT4 NLMaxDivergeCond`	Nonlinear - Max Diverging Conditions.
`INT4 NLQuasiNewtonVec`	Nonlinear - Number of Quasi-Newton Vectors.
`INT4 NLMaxLineSearch`	Nonlinear - Max Line Search per Iteration.
`REAL8 NLLineSearchTolerance`	Nonlinear - Line Search Tolerance.
`INT4 NLMaxBisection`	Nonlinear - Max Bisections per Increment.
`REAL8 NLMaxRotation`	Nonlinear - Max Rotation per Bisection.
`REAL8 NLStressFraction`	Nonlinear - Stress Fraction Limit.
`REAL8 NLMaxAdjusted`	Nonlinear - Max Adjusted vs. Initial Increment.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 NLTransStepSkipFactor`	Nonlinear - Transient Time Step Skip Factor.
`INT4 NLTransDominantSteps`	Nonlinear - Transient Steps for Dominant Period.
`REAL8 NLTransBoundsMaintain`	Nonlinear - Transient Bounds to Maintain Step.
`REAL8 NLTransStabilityTol`	Nonlinear - Transient Minimum Stability Tolerance.
`REAL8 DYNStructDamp`	Dynamics - Overall Structural Damping Coefficient, G.
`REAL8 DYNSystemFreqDamp`	Dynamics - Frequency for System Damping, W3.
`REAL8 DYNElemFreqDamp`	Dynamics - Frequency for Element Damping, W4.
`INT4 DYNOn`	Dynamics - Flag for Dynamic Analysis: 0=Off, 1=Direct, 2=Modal.
`INT4 DYNType`	Dynamics - Addl Flag for Dynamic Analysis: 0=Off, 1=Transient, 2=Freq.
`INT4 DYNMassFormulation`	Dynamics - Dynamic Mass Formulation: 0=Default, 1=Lumped, 2=Coupled.
`INT4 DYNDataRecovery`	Dynamics - Dynamic Data Recovery: 0=Mode Displacement, 1=Mode Acceleration, 2=Matrix.
`INT4 DYNSolFreqType`	Dynamics - Solution Frequency Input Type.
`INT4 DYNPSDType`	Dynamics - PSD Data Type.
`INT4 DYNPSDInterp`	Dynamics - PSD Interpolation Method.
`REAL8 DYNSolFreqSpread`	Dynamics - Spread for frequency clusters.
`INT4 DYNSolFreqNumFreq`	Dynamics - Solution Frequencies per Mode.
`REAL8 DYNSolFreqMin`	Dynamics - Minimum Frequency to Compute.
`REAL8 DYNSolFreqMax`	Dynamics - Maximum Frequency to Compute.
`BOOL DYNSolFreqLogInterp`	Dynamics - Frequency Logarithmic Interpolation.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DYNTransTimeSteps`	Dynamics - Transient Analysis Number of Steps.
`REAL8 DYNTransTimePerStep`	Dynamics - Transient Analysis Time Step.
`INT4 DYNTransOutputInterval`	Dynamics - Transient Analysis Output Interval.
`INT4 DYNFrequencyTable`	Dynamics - Function ID of the Frequency Table.
`INT4 DYNDampingTable`	Dynamics - Function ID of the Damping Table.
`INT4 DYNRandomPSD`	Dynamics - Function ID of the PSD.
`INT4 DYNResponseModes`	Dynamics - Number of Modes to use in Response Calculations.
`REAL8 DYNResponseMinFreq`	Dynamics - Minimum Response Frequency to Keep for further analysis.
`REAL8 DYNResponseMaxFreq`	Dynamics - Maximum Response Frequency to Keep for further analysis.

LoadSet Object Methods

Expand

Expand	
`( void )`	
Description:	
This method expands all of the geometric loads in a selected load set.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	The specified expansion did not work.
Remarks/Usage:	
This method takes the current load set object (uses the ID), and expands all of the geometric loads in that set, so that they may be retrieved as normal mesh-based loads.	
Example:	
None	

Compress

`Compress` **`( void )`**	
Description:	
This method compresses all of the geometric loads in a selected load set.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	The specified compression did not work.
Remarks/Usage:	
This method takes the current load set object (uses the ID), and compresses (deletes) any expanded loads. You are left with the unexpanded geometric loads.	
Example:	
None	

Unpublished work. © 2025 Siemens

ResetNextLoad

ResetNextLoad **(enDataType, bAllLoadTypes, enLoadType, bOtherLoadsOnly, bIncludeExpanded)**	
Description:	
This method initializes all options prior to calling the `NextLoad` method below.	
Input:	
`INT4 enDataType`	The type of load entities to be retrieved. For entity information, see **Entity Types**. Can be `FT_SURF_LOAD`, `FT_GEOM_LOAD` (default), `FT_NTHERM_LOAD`, or `FT_ETHERM_LOAD`.
`BOOL bAllLoadTypes`	Used to limit the type of loads to be retrieved. If True, all selected loads will be returned. If False, only loads that match the type specified in `enLoadType` will be returned.
`INT4 enLoadType`	The type of loads to return. Only used if `bAllLoadTypes` is False. Refer to the Load Types table in **Femap Constants** for more information.
`BOOL bOtherLoadsOnly`	If true, then only loads that are not part of a load definition will be retrieved. Otherwise all are available.
`BOOL bIncludeExpanded`	If True, expanded geometric loads (if any) will be returned along with normal loads. Ignored if `enDataType=FT_GEOM_LOAD`.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
This method is used prior to calling `NextLoad` to specify which loads will be retrieved. It also resets this object so that the next call to `NextLoad` will return the first load available in the object.	

Unpublished work. © 2025 Siemens

Example:
None

NextLoad

NextLoad (pDISPATCH)	
Description:	
This method returns the next available load that matches the criteria established by `ResetNextLoad`.	
Input:	
None	None
Output:	
`OBJECT pDISPATCH`	The next load object. Depending on the options specified, this could be a LoadGeom, LoadMesh, LoadNTemp or LoadGeom object.
Return Code:	
`FE_BAD_DATA`	The data type that you are trying to retrieve is invalid.
`FE_FAIL`	There are no more loads of the specified type to retrieve. Call `ResetNextLoad` to go back to the first load, if you want to retrieve the loads again.
Remarks/Usage:	
This method, coupled with `ResetNextLoad` provides an easy way to retrieve all loads from a Load Set. For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.	
Example:	

```
‘ This example loops through all load sets retrieving all

‘ LoadMesh objects. For each it displays a message box.

Sub Main
```


Unpublished work. © 2025 Siemens

```
Dim App As femap.model
Set App = feFemap()


Dim ls As femap.LoadSet
Set ls = App.feLoadSet


 Dim l As femap.LoadMesh


While ls.Next()
 ls.ResetNextLoad( FT_SURF_LOAD, True, 0, False, False )
  While ls.NextLoad( l )
     MsgBox(Str$(l.setID)+""+Str$(l.meshID) )

   Wend
 Wend
End Sub
```

ResetNextLoadDef

ResetNextLoadDef

(bAllDataTypes, enDataType, bAllLoadTypes, enLoadType)

Description:

This method initializes all options prior to calling the `NextLoadDef` method below.

Input:

`BOOL bAllDataTypes`	Used to limit the type of load definitions to be retrieved. If True, all selected load definitions will be returned. If False, only load definitions that contain loads of the type specified in `enDataType` will be returned.
`INT4 enDataType`	The type of load entities to be retrieved. For entity information, see **Entity Types**. Can be `FT_SURF_LOAD`, `FT_GEOM_LOAD` (default), `FT_NTHERM_LOAD`, or `FT_ETHERM_LOAD`.
`BOOL bAllLoadTypes`	Used to limit the type of load definitions to be retrieved. If True, all selected load definitions will be returned. If False, only load definitions that

Unpublished work. © 2025 Siemens

	contain loads of the type specified in `enLoadType` will be returned.
`INT4 enLoadType`	The type of loads to return. Only used if `bAllLoadTypes` is False. Refer to the Load Types table in **Femap Constants** for more information.
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
This method is used prior to calling `NextLoadDef` to specify which load definitions will be retrieved. It also resets this object so that the next call to `NextLoadDef` will return the first load definition available in the object.	
Example:	
None	

NextLoadDef

`NextLoadDef`	
`( pDISPATCH )`	
Description:	
This method returns the next available load definition that matches the criteria established by `ResetNextLoadDef`.	
Input:	
None	None
Output:	
`OBJECT pDISPATCH`	The next load definition object. This is always a LoadDefinition object.
Return Code:	
`FE_FAIL`	There are no more load definitions of the specified type to retrieve. Call `ResetNextLoadDef` to go

Unpublished work. © 2025 Siemens

	back to the first load definition, if you want to retrieve the loads again.

Remarks/Usage:

This method, coupled with `ResetNextLoadDef` provides an easy way to retrieve all load definitions from a Load Set.

For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.

Example:

```
‘ This example loops through all load sets retrieving all nodal
‘ force LoadDefinition objects, then loops through each
‘ LoadDefinition to retrieve the individual loads. For each it
‘ displays a message box.
Sub Main
Dim App As femap.model
Set App = feFemap()


Dim ls As femap.LoadSet
Set ls = App.feLoadSet


Dim ld As femap.LoadDefinition
Dim l As femap.LoadMesh


While ls.Next()
ls.ResetNextLoadDef( False, FT_SURF_LOAD, False, FLT_NFORCE )
  While ls.NextLoadDef( ld )
     MsgBox( "Load Set"+Str$(ld.setID)+" Load Def"+Str$(ld.ID))
     While ld.NextLoad( l )
        MsgBox(  "Load On Node "+Str$(l.meshID) )
      Wend
  Wend
Wend
End Sub
```

GetCombination

Unpublished work. © 2025 Siemens

GetCombination	
(dOverallScale, nCombo, dScale, nSetID)	
Description:	
This method returns the data for a Nastran LOAD Combination Set.	
Input:	
None	None
Output:	
`REAL8 dOverallScale`	Overall Scale Factor for Nastran LOAD Combination.
`INT4 nCombo`	Number of entries in the LOAD Combination.
`REAL8 dScale[0..count-1]`	The Scale Factor for the load sets in the Combination.
`INT44 nSetID[0..count-1]`	The IDs of the load sets in the Nastran LOAD Combination.
Return Code:	
`FE_BAD_TYPE`	This is not a LOAD Combination Set.
`FE_NOT_AVAILABLE`	No load sets are in the combination.
Remarks/Usage:	
None	
Example:	
None	

PutCombination

PutCombination	
(dOverallScale, nCombo, dScale, nSetID)	
Description:	

Unpublished work. © 2025 Siemens

This method sets the data in a Load Combination Set.	
Input:	
`REAL8 dOverallScale`	Overall Scale Factor for Nastran LOAD Combination.
`INT4 nCombo`	Number of entries in the LOAD Combination.
`REAL8 dScale[0..count-1]`	The Scale Factor for the load sets in the Combination.
`INT4 nSetID[0..count-1]`	The IDs of the load sets in the Nastran LOAD Combination.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	One or more Combined Load Sets specified do not exist.
Remarks/Usage:	
None	
Example:	
None	

NodesAsSet

`NodesAsSet` **`( bExpandedGeomLoads, bNodeElemLoads, bTemperature, nNodeSetID, bClear )`**	
Description:	
This method adds the IDs of nodes loaded in the Load Set currently loaded in the LoadSet Object to a specified Set. There are options to include nodes which are loaded after geometric loads are expanded, nodes which have non-temperature loads applied, and/or nodes which have temperature loads applied.	
Input:	
`BOOL bExpandedGeomLoads`	When True, the IDs of any nodes which are loaded after geometric constraints are expanded will be added to the Set.

Unpublished work. © 2025 Siemens

`BOOL bNodeElemLoads`	When True, the IDs of any individual nodes which have non-Temperature loads applied will be added to the Set. If `bExpandedGeomLoads` is True, then this also controls if node IDs which are loaded with non-Temperature loads after geometric loads have been expanded will be added to the Set.
`BOOL bTemperatures`	When True, the IDs of any individual nodes which have Temperature loads applied will be added to the Set. If `bExpandedGeomLoads` is True, then this also controls if node IDs which are loaded with Temperature loads after geometric loads have been expanded will be added to the Set.
`INT4 nNodeSetID`	ID of Set object to add node IDs.
`BOOL bClear`	When True, any IDs which are in the set are removed before adding the node IDs to the Set.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
If Temperature loads are applied as geometric loads and `bTemperature` is set to False, then those node IDs will NOT be added to the Set. If non-Temperature loads are applied as geometric loads and `bNodeElemLoads` is set to False, then those node IDs will NOT be added to the Set.	
Example:	
None	

ElementsAsSet

`ElementsAsSet`
`( bExpandedGeomLoads, bNodeElemLoads, bTemperature, nElemSetID, bClear )`
Description:
This method adds the IDs of elements loaded in the Load Set currently loaded in the LoadSet Object to a specified Set. There are options to include elements which are loaded after geometric loads are

Unpublished work. © 2025 Siemens

expanded, elements which have non-temperature loads applied, and/or elements which have temperature loads applied.	
Input:	
`BOOL bExpandedGeomLoads`	When True, the IDs of any elements which are loaded after geometric constraints are expanded will be added to the Set.
`BOOL bNodeElemLoads`	When True, the IDs of any individual elements which have non-Temperature loads applied will be added to the Set. If `bExpandedGeomLoads` is True, then this also controls if element IDs which are loaded with non-Temperature loads after geometric loads have been expanded will be added to the Set.
`BOOL bTemperatures`	When True, the IDs of any individual elements which have Temperature loads applied will be added to the Set. If `bExpandedGeomLoads` is True, then this also controls if element IDs which are loaded with Temperature loads after geometric loads have been expanded will be added to the Set.
`INT4 nElemSetID`	ID of Set object to add element IDs.
`BOOL bClear`	When True, any IDs which are in the set are removed before adding the element IDs to the Set.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
If Temperature loads are applied as geometric loads and `bTemperature` is set to False, then those element IDs will NOT be added to the Set. If non-Temperature loads are applied as geometric loads and `bNodeElemLoads` is set to False, then those element IDs will NOT be added to the Set.	
Example:	
None	

MapOutput Objects

MapOutput objects correspond to the process of mapping output from a different model as loads into your model. They are derived from Entity objects, so in addition to the properties and methods described

Unpublished work. © 2025 Siemens

here, all of the properties and methods defined for Entity objects are also available for solids. For more information, see **Common Entity Properties and Methods**.

Use the `feMapOutput` method of the Femap Application object to create MapOutput objects.

MapOutput Object Properties

MapOutput are always stored with a `SetID=1`, and the ID equal to the entity ID.

Property	Description
`BOOL UseCriteria(12.0+)`	Flag to indicate if a Criteria approach (True) or Contour approach should be used for Data Conversion.

`enum MapOption`	Method used to determine values for target locations that fall outside of source mesh:
	`0=FMO_ZERO`
	`1=FMO_VALUE`
	`2=FMO_EXTEND`
	`3=FMO_INTERP`
	`4=FMO_NONE`
	Only used when `UseCriteria` is set to False.

`REAL8 DefaultMapValue1`	'X' value used if target mapping falls outside of source mesh and `MapOption` is set to 1 (`FMO_VALUE`). Only used when `UseCriteria` is set to False.
`REAL8 DefaultMapValue2`	'Y' value used if target mapping falls outside of source mesh and `MapOption` is set to 1 (`FMO_VALUE`). Only used when `UseCriteria` is set to False
`REAL8 DefaultMapValue3`	'Z' value used if target mapping falls outside of source mesh and `MapOption` is set to 1 (`FMO_VALUE`). Only used when `UseCriteria` is set to False
`enum DataConversion`	Data Conversion method for source model:

Unpublished work. © 2025 Siemens

Property	Description
	0=FMDC_VU (From View) 1=FMDC_AVG (Average) 2=FMDC_MAX (Maximum) 3=FMDC_MIN (Minimum) 4=FMDC_AVG_SKIP_CORNER (Average, Centroid Only) 5=FMDC_MAX_SKIP_CORNER (Max, Centroid Only) 6=FMDC_MIN_SKIP_CORNER (Min, Centroid Only) Only used when `UseCriteria` is set to False.
`INT4 TargetSet`	The ID of the target model's set of nodes or elements.

`enum LoadType`	Target model mapped load type:
	
	`0=FMLT_NTEMPERATURE`
	`1=FMLT_NFORCE`
	`2=FMLT_NDISPLACEMENT`
	`3=FMLT_NROTDISPLACEMENT`
	`4=FMLT_NVELOCITY`
	`5=FMLT_NROTVELOCITY`
	`6=FMLT_NACCELERATION`
	`7=FMLT_NROTACCELERATION`
	`8=FMLT_ETEMPERATURE`
	`9=FMLT_EHEATFLUX`
	`10=FMLT_EHEATGEN`
	`11=FMLT_EPRESSURE`
	`12=FMLT_NPRESSURE`
	
	Only elemental options available when `UseCriteria` set to True.

Unpublished work. © 2025 Siemens

Property	Description
`enum MapToType`	Mapped (target load) entity type applied to: `0=FMP_STANDARD`; `1=FMP_NODE`; `2=FMP_ELEMENT`. Note: Value used for cross-entity type mapping. If load will be applied to the same entity type as the output: e.g. source NODE to target NODE , use `MapToType=FMP_STANDARD`. (for *same type* mapping). However, for source NODE output mapping to target `ELEMENT` load, use `FMP_ELEMENT`.
`REAL8 MapOffsetTolerance(10.3+)`	Maximum distance between Source and Target data locations where locations are considered coincident and direct mapping without interpolation occurs.

MapOutput Object Methods

MapFromModelToSetV2

`MapFromModelToSetV2` **`( SourceModel, Group ID, OutputSetID, OutputVector )`**	
Description:	
This method replaces `MapFromModelToSet`. For more information see, **Working with Output in Femap v2020.1 and above**. This method maps output data from a source model to loads on a target model. It performs the same function as Femap menu command **Model → Load → Map Output From Model...**.	
Input:	
`INT8 SourceModel`	ID of the source model containing output to be mapped.
`INT4 GroupID`	ID of group of entities in source model from which to map output.

Unpublished work. © 2025 Siemens

`INT4 OutputSetID`	ID of output set in source model containing data from which to map output.
`INT4 OutputVector`	ID of output vector in source model containing data from which to map output.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the map data. Possibly data is not stored or source group does not exist in source model or required properties are set incorrectly.
Remarks/Usage:	
This method will work with any output, but will only map to types specified in LoadType. All listed properties must be set on the MapOutput object before calling this method.	
Example:	

```
Sub Main


Dim App As femap.model
Set App = feFemap()
   Dim ret As Integer
   Dim ret As Integer
   Set map = App.feMapOutput
   Dim s As femap.Set
   Set s = App.feSet
```

```
'1 Create target Set
     rc = s.Select(FT_NODE, True, "Nodes" )
     If s.count = 0 Then
           Exit Sub
     End If
     map.TargetSet = s.ID
```


Unpublished work. © 2025 Siemens

```
'2 Set No-map preference
  map.MapOption = femap.FMO_INTERP

'3 Set Default values
  map.DefaultMapValue1 = 0.0
  map.DefaultMapValue2 = 0.0
  map.DefaultMapValue3 = 0.0

'4 Set element/Node conversion If any
     map.MapToType = femap.FMP_STANDARD

'5 Set output load Type
  map.loadTYPE = femap.FMLT_NDISPLACEMENT

'6 Get Model ID
     Dim modelID As Long
     Dim vModelID As Variant
     Dim num As Long
     Dim modelName As String
     modelName  =""
     rc = App.feAppGetAllModels(num, vModelID)
     For e=0 To num
           rc = App.feAppGetModelName(vModelID(e), modelName)
           If modelName <> App.ModelName Then
                 modelID = vModelID(e)
                 Debug.Print modelID;  " - "; modelName
                 Exit For
           End If
```


Unpublished work. © 2025 Siemens

```
     Next
```

```
Dim srcGroup As Long
     Dim srcSet As Long
     Dim srcVector As Long
     srcGroup =1
     srcSet = 1
     srcVector = 1
      ret = map.MapFromModelToSet(modelID,  srcGroup, srcSet, srcVector)
End Sub
```

MapFromModelToSet2V2

MapFromModelToSet2V2

(SourceModel, Group ID, OutputSetID, OutputVector, vID, vEIFace)

Description:

This method replaces `MapFromModelToSet2`. For more information see, **Working with Output in Femap v2020.1 and above**. This method maps output data from a source model to loads on a target model. It performs the same function as Femap menu command **Model → Load → Map Output From Model...**

Similar to `MapFromModelToSet`, but allows specification of Element or Node IDs, along with Element Face IDs, if needed, instead of specifying the `TargetSet` in the MapOutput object.

Input:

`INT8 SourceModel`	ID of the source model containing output to be mapped.
`INT4 GroupID`	ID of group of entities in source model from which to map output.
`INT4 OutputSetID`	ID of output set in source model containing data from which to map output.

Unpublished work. © 2025 Siemens

`INT4 OutputVector`	ID of output vector in source model containing data from which to map output.
`VARIANT vID`	Array of element or node IDs where data is to be mapped.
`VARIANT vElFace`	Array of element faces where data is to be mapped.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the map data. Possibly data is not stored or source group does not exist in source model or required properties are set incorrectly.
Remarks/Usage:	
This method will work with any output, but will only map to types specified in `LoadType`. All listed properties in the MapOutput object, except `TargetSet`, must be set before calling this method.	
Example:	

```
Sub Main
 Dim App As femap.model
 Set App = feFemap()

 Dim rc As Integer
 Dim Velems As Variant
 Dim Vfaces As Variant
 Dim num As Long

 Dim map As femap.MapOutput
 Set map = App.feMapOutput

 Dim targElemSet As femap.Set
 Set targElemSet = App.feSet
```


Unpublished work. © 2025 Siemens

```
  '1 Pick Target Elements and Faces
  App.feGetElementFaces(" Pick Faces", 0, , , num, Velems, Vfaces )

  '2 Set No-map preference
 map.MapOption = femap.FMO_INTERP

  '3 Set Default values
 map.DefaultMapValue1 = 0.0
 map.DefaultMapValue2 = 0.0
 map.DefaultMapValue3 = 0.0

  '4 Set element/Node conversion If any
 map.MapToType = femap.FMP_STANDARD

'5 Set output load Type
 map.loadType = femap.FMLT_EPRESSURE
```

```
  '6 Get Model ID
 Dim modelID As Long
 Dim vModelID As Variant
 Dim numM As Long
 Dim modelName As String
 modelName =""
 rc = App.feAppGetAllModels(numM, vModelID)
 For e=0 To numM
   rc = App.feAppGetModelName(vModelID(e), modelName)
     If modelName <> App.ModelName Then
       modelID = vModelID(e)
       Debug.Print modelID; " - "; modelName
```


Unpublished work. © 2025 Siemens

```
      Exit For
    End If
 Next


  '7 Output data to map from
 Dim srcGroup As Long
 Dim srcSet As Long
 Dim srcVector As Long
  srcGroup = 1
 srcSet = 2
 srcVector = 9000000


 ret = map.MapFromModelToSet2( modelID, srcGroup, srcSet,srcVector,
Velems, Vfaces)


End Sub
```

MapFromModelToLocationV2

MapFromModelToLocationV2

**(SourceModel, Group ID, OutputSetID, OutputVector, XLoc, YLoc, ZLoc, vMapVal,
vUnitX, vUnitY, vUnitZ)**

Description:

This method replaces MapFromModelToLocation. For more information see, **Working with Output in Femap v2020.1 and above**. This method maps output data from a source model to any locations in space.

Input:

Unpublished work. © 2025 Siemens

`INT8 SourceModel`	ID of the source model containing output to be mapped.
`INT4 GroupID`	ID of group of entities in source model from which to map output.
`INT4 OutputSetID`	ID of output set in source model containing data from which to map output.
`INT4 OutputVector`	ID of output vector in source model containing data from which to map output.

`REAL8 XLoc[0...N-1]`
 	
or
 	
`VARIANT vXLoc`
	Array of global rectangular x-coordinates for locations where data is to be mapped.
	
	N = number of locations for which mapped values are returned.

`REAL8 YLoc[0...N-1]`
 	
or
 	
`VARIANT vYLoc`
	Array of global rectangular y-coordinates for locations where data is to be mapped.
	
	N = number of locations for which mapped values are returned.

`REAL8 ZLoc[0...N-1]`
 	
or
 	
`VARIANT vZLoc`
	Array of global rectangular z-coordinates for locations where data is to be mapped.
	
	N = number of locations for which mapped values are returned.

Output:	
`VARIANT vMapVal`	Output value mapped to location. Values ordered 0..N-1 in `VARIANT`.
`VARIANT vUnitX`	Unit x vector (global, rectangular) of corresponding mapped output value.
`VARIANT vUnitY`	Unit y vector (global, rectangular) of corresponding mapped output value.
`VARIANT vUnitZ`	Unit z vector (global, rectangular) of corresponding mapped output value.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to get the map data. Possibly data is not stored or group does not exist in model or required MapObject properties are set incorrectly.
Remarks/Usage:	
This method will work with any output, but will only map to types specified in `LoadType`. All listed properties must be set on the MapOutput object before calling this method.	
Example:	
None	

GetOutputDataSurface

`GetOutputDataSurface` **`( dataSurfID )`**	
Description:	
This method retrieves the Data Surface ID from current model for use in the MapOutput Object.	
Input:	
`INT4 dataSurfaceID`	ID of output data surface.
Output:	
None	None
Return Code:	
`FE_FAIL`	Data Surface does not exist or is not an Output Map Data Surface.
Remarks/Usage:	
Used in conjunction with `MapOutputDataToLocation`.	
Example:	
None	

MapOutputDataToLocation

Unpublished work. © 2025 Siemens

MapOutputDataToLocation

(count, vXYZ, vValXYZ, qValid)

Description:

This method maps values to locations specified in vXYZ using local Data Surface retrieved with `GetOutputDataSurface`.

Input:

`INT4 count`	The number of locations to map in vXYZ.
`REAL8 vXYZ[0..(3*count)-1]`	Array of point coordinates in XYZ order where data is to be mapped.

Output:

`REAL8 vValXYZ[0..(3*count)-1]`	Array of output values mapped in valX, valY, valZ order.
`BOOL qValid[0..(3*count)-1]`	Array of validation indicators. If `qValid` = FALSE (0), value was not mapped.

Return Code:

`FE_OK`	Mapping performed successfully.
`FE_FAIL`	Data Surface invalid or not previously retrieved.
`FE_NO_MEMORY`	Resources are not sufficient to perform mapping operation.

Remarks/Usage:

Used for mapping locations to Data Surface in current model.

Example:

```
Sub Main
     Dim App As femap.model
     Set App = GetObject(,"femap.model")
     Dim ret As Integer
     Dim map As MapOutput
     Set map = App.feMapOutput
```


Unpublished work. © 2025 Siemens

```
      Dim pt As Point
      Set pt = App.fePoint
      Dim s As Set
      Set s = App.feSet
```

```
Const  np As Integer = 8
      Dim locxyz(np*3) As Double
      Dim ptID(np) As Integer
      Dim vLoc As Variant

      For i = 1 To 8
            pt.Get(i)
            ptID(i-1) = pt.ID
            locxyz(3*i-3) = pt.x
            locxyz(3*i-2) = pt.y
            locxyz(3*i-1) = pt.z
      Next

       'test situation where no map exists
      locxyz(23) = 100.0
      locxyz(21) = 2.25

      vLoc = locxyz

      Dim vLoad As Variant
      Dim vIsValid As Variant

      ret = map.GetOutputDataSurface(1)
```


Unpublished work. © 2025 Siemens

```
      'test situation where no map exists
     map.MapOption = FMO_NONE

     'test situation where set value if no map
      'map.MapOption = FMO_VALUE
     map.DefaultMapValue1=999
     map.DefaultMapValue2=-999
     map.DefaultMapValue3 = 1000
```

```
map.MapOption = FMO_EXTEND
    'map.MapOption = FMO_INTERP
   If(ret = FE_OK) Then
   ret = map.MapOutputDataToLocation(np, vLoc, vLoad, vIsValid)
   End If

   Dim msg As String
   Dim msg2 As String

   msg = "Pt   X     Y     Z     vX   vY    vZ    Valid"
   App.feAppMessage(FCL_BLACK, msg)
   For i=1 To np
     msg2 = Str$(vIsValid(i-1))
     msg = Str$(ptID(i-1)) + "   " +Str$(vLoc(3*i-3))+ "   "+ _
     Str$(vLoc(3*i-2))+ "      "+Str$(vLoc(3*i-1))+ "  " + _
     Str$(vLoad(3*i-3)) + "    "+ Str$(vLoad(3*i-2)) + "     "+ _
     Str$(vLoad(3*i-1)) + "  "+ msg2
    App.feAppMessage(FCL_BLACK, msg)
   Next
```


Unpublished work. © 2025 Siemens

```
End Sub
```

Material Objects

Material objects correspond to the materials in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for materials. For more information, see **Common Entity Properties and Methods**.

Use the `feMatl` method of the Femap Application object, or the `Matl` method of the Element object to create Material objects.

Material Object Properties

Materials are always stored with a `SetID=1`, and the ID equal to the material ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The material color.
`INT4 type`	Type of material: 0=Isotropic, 1=2D Orthotropic, 2=3D Orthotropic, 3=2D Anisotropic, 4=3D Anisotropic, 5=Hyperelastic, 6=General, 7=Fluid.
`INT4 subtype`	Subtype for general materials.
`INT4 layer`	The ID of the layer associated with the material.
`STRING title`	Material title (maximum 79 characters).
`BOOL bval[0..9]`	Array of Boolean flags.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vbval`	

`INT4 ival[0..24]`
 	
or
 	
`VARIANT vival`
	Array of integer values.

`REAL8 mval[0..199]`
 	
or
 	
`VARIANT vmval`
	Array of real values.

`INT4 fval[0..49]`
 	
or
 	
`VARIANT vfval`
	Array of function IDs.

`INT4 tval[0..69]`
 	
or
 	
`VARIANT vtval`
	Additional function IDs.

Additional Properties	
`REAL8 mmat[0..199]`	Same as mval, but returns/sets entire array as a VARIANT.
`INT4 imat[0..24]`	Same as ival but returns/sets entire array as a VARIANT.
`INT4 fmat[0..49]`	Same as fval, but returns/sets entire array as a VARIANT.
`INT4 tmat[0..69]`	Same as tval, but returns/sets entire array as a VARIANT.

Unpublished work. © 2025 Siemens

Property	Description
The following properties can be accessed directly from the value arrays, or by name using the following references.	
`REAL8 Ex`	Young's Modulus
`REAL8 Ey`	
`REAL8 Ez`	
`REAL8 Gx`	Shear Modulus
`REAL8 Gy`	
`REAL8 Gz`	
`REAL8 Nuxy`	Poisson's Ratio
`REAL8 Nuyz`	
`REAL8 Nuxz`	
`REAL8 Alphaxx`	Coefficient of Thermal Expansion
`REAL8 Alphayy`	
`REAL8 Alphazz`	
`REAL8 Alphaxy`	
`REAL8 Alphayz`	
`REAL8 Alphazx`	
`REAL8 Kxx`	Thermal Conductivity
`REAL8 Kyy`	
`REAL8 Kzz`	

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 Kxy`	
`REAL8 Kyz`	
`REAL8 Kzx`	
`REAL8 Cp`	Heat Capacity
`REAL8 Density`	Density
`REAL8 Damping`	Damping Coefficient
`REcAL8 RefTemp`	Reference Temperature
`REAL8 TesionLimit1`	Limit Stress/Strain in Tension
`REAL8 TensionLimit2`	
`REAL8 CompressionLimit1`	Limit Stress/Strain in Compression
`REAL8 CompressionLimit2`	
`REAL8 ShearLimit`	Limit Stress/Strain in Shear
`REAL8 TsaiWu`	Tsai-Wu Interaction Factor

Accessing Material Properties

You can access the individual properties for the Material object by referencing the properties described previously, and the data contained in the table below. For example, to retrieve the Young's Modulus for an Isotropic material, use:

```
E = mt.mval(0)
```

or

```
rho = mt.mval(49)
```


Unpublished work. © 2025 Siemens

for density.

This method provides great flexibility, but can be slow if you are going to work with a large number of properties. In this case, it is best to access the data using *Additional Properties*. These all set and return entire arrays of values in one reference. For example, you could write:

```
Dim mt as Object
Set mt = femap.feMatl
Dim v  as Variant
mt.Get(1)
v = mt.mmat
v(0)=30.0E6
v(49)=0.1
mt.mmat = v
mt.Put(1)
```

This code loads Material 1, retrieves the entire array of real values into "v", updates the Young's modulus and density, puts the entire array back, and saves the updated material definition. While the code is slightly longer than simply accessing the individual properties, performance will be better using this approach.

Material Array Index Values

In general, most of the material data is simply stored in the various array properties. The location of the specific property data is defined in the following table.

The IDs of the functions used for function dependence of the standard properties (accessed in Femap by pressing the **Function** button on the Material dialog boxes) can be accessed by looking in the following table under the *Real Values* section. There you should identify the property that you want, and call the tval property with the same index as the mval property that you are looking for.

Property	Description	Locations
Boolean Values		
`HYPER_POLYORD(i)`	Strain Energy Polynomial Order for Hyperelastic.	`ival[0],ival[1]`
`NONLINEAR_TYPE`	Nonlinear Type (0=None/Linear, 1=Nonlinear Elastic, 2=Plastic, 3=Elasto-Plastic).	`ival[2]`
`HARDENING_TYPE`	Hardening Data Type.	`ival[3]`
`YIELD_TYPE`	Yield Criterion (0=von Mises, 1=Tresca, 2=Mohr-Coloumb, 3=Drucker-Prager).	`ival[4]`

Unpublished work. © 2025 Siemens

Property	Description	Locations
CREEP_TYPE	Creep Type (0=None, 1=Empirical, 2=Tabular).	ival[5]
CREEP_EMPIRICAL_FORM(i)	Empirical Creep Law Format Flags.	ival[6..9]
DP_HARDENING_DATA_TYPE	Drucker-Prager Hardening data type.	ival[10]
PLY_FAILURE_MODE	Ply/Bond Failure - Ply Failure Theory (0=None, 1=Hill, 2=Hoffman, 3=Tsai-Wu, 4=Max Strain, 5=Max Stress, 6=Max Transverse Shear).	ival[11]
BOND_FAILURE_MODE	Ply/Bond Failure - Bond Failure Theory (0=None, 1=Transverse Shear, 2=Normal Stress).	ival[12]
Boolean Values		
HAS_STRAIN_LIMITS	Flag if material uses strain limits instead of stress limits.	bval[0]
FLUID IS LIQUID	Flag if a fluid type material is a liquid vs. a gas.	bval[1]
OPT_FRONT_SIDE		bval[2]
OPT_REV_SIDE		bval[3]
DMATRIX_HYP_D1_ZERO	Flag to write D1 in Nastran Hyperelastic material as Zero.	bval[4]
Real Values		
E(i)	Young's modulus.	mval[0..2]
G(i)	Shear modulus.	mval[3..5]
NU(i)	Poisson's ratio.	mval[6..8]
GMATRIX_3D(i)	Upper triangle of 6x6 3D anisotropic elastic matrix.	mval[9..29]
GMATRIX_2D(i)	Upper triangle of 3x3 2D anisotropic elastic matrix.	mval[30..35]

Unpublished work. © 2025 Siemens

Property	Description	Locations
THERMAL_EXPANSION(i)	Thermal expansion coefficients.	mval[36..41]
THERMAL_CONDUCTIVITY(i)	Thermal conductivity coefficients.	mval[42..47]
THERMAL_CAPACITY	Specific Heat.	mval[48]
DENSITY	Material density.	mval[49]
DAMPING	Damping coefficient.	mval[50]
TEMPERATURE	Reference temperature.	mval[51]
TENSION_LIMIT(i)	Stress allowable in tension.	mval[52..53]
COMPRESSION_LIMIT(i)	Stress allowable in compression.	mval[54..55]
SHEAR_LIMIT	Stress allowable in shear.	mval[56]
TSAI_WU_INTERACTION	Tsai_wu interaction factor.	mval[57]
AMATRIX_HYP(i)	Hyperelastic Amatrix values.	mval[58..78]
DMATRIX_HYP(i)	Hyperelastic Dmatrix values.	mval[79..83]
HARDENING_SLOPE	Plasticity Hardening Slope.	mval[84]
YIELD_LIMITS(i)	Plasticity Yield Limits - Initial Yield Stress, Friction Angle, additional yield limits.	mval[85..89]
CREEP_THRESHOLD_STRESS	Creep Threshold Stress.	mval[90]
CREEP_REF_TEMP	Creep Reference Temperature.	mval[91]
CREEP_TEMP_DEP_RATE	Temperature Dependent Creep Rate	mval[92]
CREEP_EMPIRICAL_COEFF(i)	Empirical Creep Law Coefficients.	mval[93..99]

Unpublished work. © 2025 Siemens

Property	Description	Locations
HEAT GENERATION	Heat Generation Value.	mval[100] or tval[57]
REFERENCE ENTHALPY	Enthalpy for Phase Change.	mval[101]
PHASE CHANGE TEMP	Temperature for Phase Change Onset.	mval[102]
PHASE CHANGE TEMP RANGE	Temperature Range for Phase Change.	mval[103]
LATENT HEAT OF FUSION	Latent heat of Fusion.	mval[104]
SPECIFIC HEAT ABOVE PHASE	Specific heat above Phase Change Temp.	mval[105]
OPT EMISSIVITY	Optical - Emissivity.	mval[106]
OPT EMISSIVITY REV	Optical - Emissivity, Reverse Side.	mval[107]
OPT ABSORPTIVITY	Optical - Absorptivity.	mval[108]
OPT ABSORPTIVITY REV	Optical - Absorptivity, Reverse Side.	mval[109]
OPT TRANSMISSIVITY SOLAR	Optical - Solar Transmissivity.	mval[110]
OPT TRANSMISSIVITY IR	Optical - InfraRed Transmissivity.	mval[111]
OPT REFLECTIVITY SOLAR	Optical - Solar Reflectivity.	mval[112]
OPT REFLECTIVITY SOLAR REV	Optical - Solar Reflectivity, Reverse Side	mval[113]
OPT REFLECTIVITY IR	Optical - InfraRed Reflectivity	mval[114]

Unpublished work. © 2025 Siemens

Property	Description	Locations
OPT REFLECTIVITY IR REV	Optical - InfraRed Reflectivity, Reverse.	mval[115]
GAS CONSTANT	Fluid Gas Constant.	mval[116] or tval[58]
DYNAMIC VISCOSITY	Fluid Dynamic Viscosity.	mval[117] or tval[59]
PRANDTL NUMBER	Prandtl Number.	mval[118] or tval[60]
ELECTRICAL_RESISTIVITY	Optical - Electrical Resistivity.	mval[119]
VOL_SPECIFIC_HEAT	Fluid Specific Heat, Cv.	mval[120]
OPT_REFRACTION	Optical - Index of Refraction.	mval[121]
OPT_REFRACTION_REV	Optical - Index of Refraction, Reverse.	mval[122]
BULK_MODULUS	Other Types - Bulk Modulus.	mval[123]
SPEED_OF_SOUND	Other Types - Speed of Sound - C on NX Nastran Fluid Material (MAT10) and on MSC Nastran Fluid Material (MAT10).	mval[124]
PLY_FAILURE_STRESS_XC	Ply/Bond Failure - Compressive Stress Limits Value 1.	mval[125]

Unpublished work. © 2025 Siemens

Property	Description	Locations
PLY_FAILURE_STRESS_YC	Ply/Bond Failure - Compressive Stress Limits Value 2.	mval[126]
PLY_FAILURE_STRESS_ZC	Ply/Bond Failure - Compressive Stress Limits Value 3.	mval[127]
PLY_FAILURE_STRESS_XT	Ply/Bond Failure - Tensile Stress Limits Value 1.	mval[128]
PLY_FAILURE_STRESS_YT	Ply/Bond Failure - Tensile Stress Limits Value 2.	mval[129]
PLY_FAILURE_STRESS_ZT	Ply/Bond Failure - Tensile Stress Limits Value 3.	mval[130]
PLY_FAILURE_STRESS_XY SHR	Ply/Bond Failure - Shear Stress Limits Value 12.	mval[131]
PLY_FAILURE_STRESS_YZ SHR	Ply/Bond Failure - Shear Stress Limits Value 23.	mval[132]
PLY_FAILURE_STRESS_XZ SHR	Ply/Bond Failure - Shear Stress Limits Value 13.	mval[133]
PLY_FAILURE_STRAIN_XC	Ply/Bond Failure - Compressive Strain Limits Value 1.	mval[134]
PLY_FAILURE_STRAIN_YC	Ply/Bond Failure - Compressive Strain Limits Value 2.	mval[135]
PLY_FAILURE_STRAIN_ZC	Ply/Bond Failure - Compressive Strain Limits Value 3.	mval[136]
PLY_FAILURE_STRAIN_XT	Ply/Bond Failure - Tensile Strain Limits Value 1.	mval[137]
PLY_FAILURE_STRAIN_YT	Ply/Bond Failure - Tensile Strain Limits Value 2.	mval[138]
PLY_FAILURE_STRAIN_ZT	Ply/Bond Failure - Tensile Strain Limits Value 3.	mval[139]
PLY_FAILURE_STRAIN_XY SHR	Ply/Bond Failure - Shear Strain Limits Value 12.	mval[140]
PLY_FAILURE_STRAIN_YZ SHR	Ply/Bond Failure - Shear Strain Limits Value 23.	mval[141]

Unpublished work. © 2025 Siemens

Property	Description	Locations
PLY_FAILURE_STRAIN_XZ SHR	Ply/Bond Failure - Shear Strain Limits Value 13.	mval[142]
PLY_FAILURE_TSAI_XY	Ply/Bond Failure - Tsai-Wu Interaction Value 12.	mval[143]
PLY_FAILURE_TSAI_YZ	Ply/Bond Failure - Tsai-Wu Interaction Value 23.	mval[144]
PLY_FAILURE_TSAI_XZ	Ply/Bond Failure - Tsai-Wu Interaction Value 13.	mval[145]
BULK_MODULUS_RATIO	Other Types - Bulk Mod Ratio - GAMMA on NX Nastran Fluid Material (MAT10).	mval[170]
ADMITTANCE_COEFFICIEN T	Other Types - Admittance Coefficient - ALPHA on MSC Nastran Fluid Material (MAT10).	mval[171]
Function Values		
EXPERIMENTAL_FUNCTION S(i)	IDs of Hyperelastic Experimental Data Functions.	fval[0..4]
NONLINEAR_FUNCTION	ID of function for Plasticity.	fval[5]
CREEP_FUNCTIONS(i)	Tabular Model Creep Function IDs.	fval[6..8]
DP_YIELD_FUNCTION	ID of Drucker-Prager Yield Function.	fval[9]
OPT EMISSIVITY FUNCTION	ID of Optical Emissivity Function.	fval[10]
OPT_EMISSIVITY REV FUNCTION	ID of Optical Emissivity Rev Side Function.	fval[11]
OPT ABSORPTIVITY FUNCTION	ID of Optical Absorptivity Function.	fval[12]
OPT ABSORPTIVITY REV FUNC	ID of Optical Absorptivity Rev Side Function.	fval[13]
OPT TRANSMISSIVITY SOLAR FUNC	ID of Solar Transmissivity Function.	fval[14]

Unpublished work. © 2025 Siemens

Property	Description	Locations
OPT TRANSMISSIVITY IR FUNC	ID of InfraRed Transmissivity Function.	fval[15]
OPT REFLECTIVITY SOLAR FUNC	ID of Solar Reflectivity Function.	fval[16]
OPT REFLECTIVITY SOLAR REV FUNC	ID of Solar Reflectivity, Rev Side Function.	fval[17]
OPT REFLECTIVITY IR FUNC	ID of InfraRed Reflectivity Function.	fval[18]
OPT REFLECTIVITY IR REV FUNC	ID of InfraRed Reflectivity Rev Side Function.	fval[19]
ELECTRICAL RESISTIVITY FUNC	ID of Electrical Resistivity Function.	fval[20]
OPT_REFRACTION_FUNC	ID of Index of Refraction Function.	fval[21]
OPT_REFRACTION_REV_FUNC	ID of Index of Refraction Rev Side Function.	fval[22]
BULK_MODULUS_FUNC	ID of Bulk Modulus Function - NX Nastran Fluid Material (MAT10).	fval[39]
DAMPING_COEFFICIENT_FUNC	ID of Damping Coeff Function - NX Nastran Fluid Material (MAT10).	fval[40]
BULK_MODULUS_RATIO_FUNC	ID of Bulk Mod Ration Function - NX Nastran Fluid Material (MAT10).	fval[41]
DENSITY_FUNC	ID of Density Function - NX Nastran Fluid Material (MAT10).	fval[49]

Material Object Methods

AutoComplete

Unpublished work. © 2025 Siemens

AutoComplete	
(void)	
Description:	
This method automatically fills out additional properties in the material object.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Often you want to define a material using a particular type - for example, orthotropic. In some cases however Femap will need to use that material as another type - for example, anisotropic. To do this successfully, all of the properties, not just the ones for the defined type need to be completed. This method will automatically compute, using the standard definitions in Femap, the required properties. Only standard structural properties are supported - `E`, `G`, `NU`, `GMATRIX_2D`, `GMATRIX_3D`, `THERMAL_EXPANSION`, and `THERMAL_CONDUCTIVITY`, and the `TENSION_LIMIT` and `COMPRESSION_LIMIT` are currently supported. You should call this method after completing your material definition, and immediately before calling `Put` to store the material in the database.	
Example:	
None	

AreDuplicate

AreDuplicate	
(nMatl1, nMatl2, ignoretitle)	
Description:	

Unpublished work. © 2025 Siemens

This method determines if two materials have identical values and includes an option to also consider the titles of the two materials.	
Input:	
`INT4 nMatl1`	ID of a material.
`INT4 nMatl2`	ID of a material to compare to the material specified in `nMatl1`.
`BOOL ignoretitle`	Should title be ignored when determining if the 2 materials are duplicates (True=titles ignored, False=titles considered).
Output:	
None	None
Return Code:	
`FALSE / 0`	Materials are not duplicates (i.e., different).
`TRUE / 1`	Materials are duplicates.
Remarks/Usage:	
Returns TRUE or FALSE instead of Femap Return Code.	
Example:	
None	

GetLibraryOfType

`GetLibraryOfType` **`( libINDEX, libTYPE, libFILE )`**	
Description:	
This method retrieves a material of a specified type from the Material library.	
Input:	
`INT4 libINDEX`	The index of the entity stored in the library. The first material of the specified type stored is `libINDEX` = 0, the second has `libINDEX` = 1,

Unpublished work. © 2025 Siemens

	and so on. If you specify `libINDEX` = -1, a dialog box will be displayed showing all materials of the specified type in the library, and will allow the user to select the material to load.
`INT4 or enum libTYPE`	Use `zMaterialType` enumeration. Alternatively, enter value for Type of material: 0=Isotropic, 1=2D Orthotropic, 2=3D Orthotropic, 3=2D Anisotropic, 4=3D Anisotropic, 5=Hyperelastic, 6=General, 7=Fluid.
`INT4 or enum libFILE`	Use `zLibraryFile` enumeration. Alternatively, enter value for Type of library: 0=Open Dialog Box, 1=Personal, 2=Shared, 3=Femap Standard Libraries.
Output:	
None	None
Return Code:	
`FE_FAIL`	No Material was selected. Either the library is empty or does not contain any Materials of the matching type (e.g. 3D Orthotropic).
`FE_CANCEL`	User canceled the dialog box to select a Material from the library.
`FE_INVALID`	Library file was opened but isn't a valid library of the correct type.
`FE_NOT_EXIST`	The selected index in the library was not found.
`FE_NOT_AVAILABLE`	The library file could not be accessed. Either the filename is specified incorrectly in Femap, or there is some other permission or corruption problem with the file.
`FE_BAD_TYPE`	`libTYPE` value does not correspond to an available material type.
Remarks/Usage:	
This method can be used instead of specifying the *Active Material Type* before using the `GetLibrary` method to load a material from the Material library.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetValueArray

GetValueArray

(numValue, nMatlID, nMatlIndex, dValue)

Description:

This method returns an array of material property values using any combination of material IDs and material array index values.

Input:

`INT4 numValue`	The number of entries in `nMatlID` and `nMatlIndex`.
`INT4 nMatlID[0..numValue-1]`	The IDs of the materials.
`INT4 nMatlIndex[0..numValue-1]`	The ID(s) of the material array index value(s).

Output:

`REAL8 dValue[0..numvalue-1]`	The array of all the material property values.

Return Code:

`FE_BAD_DATA`	One or more `nMatlIndex` values are outside the allowable range.

Remarks/Usage:

See **Accessing Material Properties** and **Material Array Index Values** for more information about accessing material property values using the material array index values.

Example:

```
Sub Main

   Dim App As femap.model

   Set App = feFemap()

   Dim mt As femap.Matl

   Set mt = App.feMatl

   Dim mtset As femap.Set

   Set mtset = App.feSet
```


Unpublished work. © 2025 Siemens

```
   Dim Mats As Variant
   Dim MatInd As Variant
   Dim MatVals As Variant
   Dim numMat As Long
   mtset.AddAll (FT_MATL)
   numMat = mtset.Count
   MatCount = numMat*2-1
   ReDim MatInd(MatCount) As Long
   ReDim Mats(MatCount) As Long
   mtset.First
   For i = 0 To numMat-1
MatInd(i*2) = 0
      MatInd(i*2+1) = 6
      Mats(i*2) = mtset.CurrentID
      Mats(i*2+1) = mtset.CurrentID
      mtset.Next
   Next
   rc = mt.GetValueArray (numMat*2, Mats, MatInd, MatVals)
   For i = 0 To numMat-1
       rc = App.feAppMessage(FCM_NORMAL,"Material ID" _       +Str$
(Mats(i*2))+" E is"+ Str$(MatVals(i*2)))
   rc = App.feAppMessage(FCM_NORMAL,"Material ID" _
+Str$(Mats(i*2+1))+" nu is" + Str$(MatVals(i*2+1)))
   Next
End Sub
```

PutValueArray

Unpublished work. © 2025 Siemens

PutValueArray

(numValue, nMatlID, nMatlIndex, dValue)

Description:

This method updates any number of material property values on any number of materials.

Input:

`INT4 numValue`	The number of entries in `nMatlID`, `nMatlIndex`, and `dValue`.
`INT4 nMatlID[0..numValue-1]`	The IDs of the materials.
`INT4 nMatlIndex[0..numValue-1]`	The IDs of the material array index value(s).
`REAL8 dValue[0..numvalue-1]`	The material property values.

Output:

None	None

Return Code:

`FE_NOT_EXIST`	One or more Materials being updated do not exist.
`FE_BAD_DATA`	One or more `nMatlIndex` values are outside the allowable range.

Remarks/Usage:

See **Accessing Material Properties** and **Material Array Index Values** for more information about accessing material property values using the material array index values.

Example:

```
Sub Main

   Dim App As femap.model

   Set App = feFemap()

   Dim mt As femap.Matl

   Set mt = App.feMatl

   Dim mtset As femap.Set

   Set mtset = App.feSet
```


Unpublished work. © 2025 Siemens

```
   Dim Mats As Variant
   Dim MatInd As Variant
   Dim MatVals As Variant
   Dim numMat As Long
   mtset.AddAll (FT_MATL)
   numMat = mtset.Count
   MatCount = numMat*2-1
   ReDim MatInd(MatCount) As Long
   ReDim Mats(MatCount) As Long
   ReDim MatVals(MatCount) As Double
   mtset.First
   For i = 0 To numMat-1
     MatInd(i*2) = 0
     MatInd(i*2+1) = 6
     MatVals(i*2) = 500000
     MatVals(i*2+1) = .33
     Mats(i*2) = mtset.CurrentID
     Mats(i*2+1) = mtset.CurrentID
mtset.Next
   Next
   rc = mt.PutValueArray (numMat*2, Mats, MatInd, MatVals)
End Sub
```

PutValue

Unpublished work. © 2025 Siemens

PutValue	
(nMatlSetID, nMatlIndex, dValue)	
Description:	
This method updates a single material property value on any number of materials.	
Input:	
`INT4 nMatlSetID`	The ID of set of material IDs.
`INT4 nMatlIndex`	The ID of the material array index value.
`REAL8 dValue`	The material property value.
Output:	
None	None
Return Code:	
`FE_BAD_DATA`	One or more `nMatlIndex` values are outside the allowable range.
Remarks/Usage:	
See **Accessing Material Properties** and **Material Array Index Values** for more information about accessing material property values using the material array index values.	
Example:	
None	

CopyArray

CopyArray	
(bUpdateReferences, bCopyToStartID, nCopyToStartID, numID, nOrigID, pnNewID)	
Description:	
This method copies any number of materials and allows you to select a starting ID for the copies or provide all IDs for all copies. It also allows you to choose if the *new material* should replace the *original*	

Unpublished work. © 2025 Siemens

material anywhere the original material was referenced (i.e., property "n" would now reference copied material instead of original material).	
Input:	
`BOOL bUpdateReferences`	Flag to indicate if *copied material* should replace *original material* anywhere the *original material* was referenced. True = original material references are replaced with copied material.
`BOOL bCopyToStartID`	Flag controls IDs of *copied materials*. When True, uses value of `nCopyToStartID` as the first ID of the copies. If a material ID already exists in the range where new materials will be created, that ID is skipped and the material will be placed in the next empty ID. When False, then `pnNewID` becomes an input instead of an output and must be populated with the same number of entries as `nOrigID` and cannot contain any IDs of materials which already exist.
`INT4 nCopyToStartID`	Starting ID for copied materials.
`INT4 numID`	Number of original material IDs in `nOrig` array.
`INT4 nOrigID[numID-1]`	The IDs of the original materials.
Output:	
`INT4 pnNewID[numID-1]`	The IDs of the newly created (copied) materials. If `bCopyToStartID` is False, this is treated as an input instead of an output, which contains the new IDs off all copied materials. It must have the same number of entries as `nOrigID` and cannot contain any IDs of existing materials.
Return Code:	
`FE_NOT_AVAILABLE`	One or more of the materials to be created already exist.
`FE_NOT_EXIST`	One or more of the materials being copied do not exist.
`FE_NO_MEMORY`	Unable to allocate memory to process this operation.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetAllArray

`GetAllArray`
 	
`( setID, numMatl, entID, exist, matltype, matlsubtype, layer, color, title )`	

Description:	
This method retrieves arrays of values for a set of materials, including if a material exists in the database.	
Input:	
`INT4 setID`	ID of the set that contains the material IDs to retrieve. If 0, retrieve all materials in the model.
Output:	
`INT4 numMatl`	The number of materials for which data is being returned..
`INT4 entID[0..numMatl-1]`	The IDs of the materials being retrieved.
`BOOL exist[0..numMatl-1]`	True=Material exists, False=Material does not exist.
`INT4 matltype[0..numMatl-1]`	Type of material: 0=Isotropic, 1=2D Orthotropic, 2=3D Orthotropic, 3=2D Anisotropic, 4=3D Anisotropic, 5=Hyperelastic, 6=General, 7=Fluid.
`INT4 matlsubtype[0..numMatl-1]`	Subtype ... only nonzero for General Materials.
`INT4 layer[0..numMatlt-1]`	The ID of the layer for the material.
`INT4 color[0..numMatl-1]`	The color of the material.
`STRING title[0..numMatl-1]`	The title of the material.
Return Code:	
`FE_FAIL`	Unable to retrieve material info.

Unpublished work. © 2025 Siemens

`FE_NO_MEMORY`	Unable to allocate memory to retrieve material info.
Remarks/Usage:	
None	
Example:	
None	

PutAllArray

`PutAllArray` **`( numMatl, entID, bUpdateType, matltype, matlsubtype, bUpdateLayer, layer,`** **`bUpdateColor, color, bUpdateTitle, title )`**	
Description:	
This method updates properties of multiple selected materials.	
Input:	
INT4 numMatl	The number of materials for which data is being updated.
`INT4 entID[0..numMatl-1]`	The IDs of the materials to update.
`BOOL bUpdateType[0..numMatl-1]`	If True, update `matltype` and `matlsubtype`.
`INT4 matltype[0..numMatl-1]`	Type of material: 0=Isotropic, 1=2D Orthotropic, 2=3D Orthotropic, 3=2D Anisotropic, 4=3D Anisotropic, 5=Hyperelastic, 6=General, 7=Fluid. Only assigned if `bUpdateType` is set to True.
`INT4 matlsubtype[0..numMatl-1]`	Material Subtype. Only assigned if `bUpdateType` is set to True.
`BOOL bUpdatLayer[0..numMatl-1]`	If True, update layer ID
`INT4 layer[0..numMatlt-1]`	The ID of the layer for the material. Only assigned if `bUpdateLayer` is set to True.

Unpublished work. © 2025 Siemens

`BOOL bUpdateColor[0..numMatl-1]`	If True, update color.
`INT4 color[0..numMatl-1]`	The color of the material. Only assigned if `bUpdateColor` is set to True.
`BOOL bUpdateTitle[0..numMatl-1]`	If True, update title.
`STRING title[0..numMatl-1]`	The title of the material. Only assigned if `bUpdateTitle` is set to True.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store one or more material with the specified IDs.
`FE_NO_MEMORY`	Unable to allocate memory to update materials.
Remarks/Usage:	
None	
Example:	
None	

Mesh Point Objects

Mesh Point (`MeshHardPoint`) objects correspond to the mesh points in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for point objects. For more information, see **Common Entity Properties and Methods**.

Use the `feMeshHardPoint` method of the Femap Application object to create Mesh Point objects.

MeshHardPoint Object Properties

Mesh Points are always stored with a `SetID=1`, and the ID equal to the Mesh Point ID.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DefinitionID`	ID of mesh point definition associated with a mesh point. It is not required for a mesh point to reference a mesh point definition.
`INT PointID`	ID of mesh point's underlying geometric point.
`INT4 layer`	The ID of the layer associated with the mesh point.
`INT4 color`	The mesh point color.
`INT4 OnGeometryType`	Type of geometric entity the mesh point is associated. 0=None, 3=Point, 4=Curve, 5=Surface, 39=Solid.
`INT4 OnGeometryID`	ID of the geometric entity (i.e. point ID, curve ID, surface ID, or solid ID).
`BOOL NodeOnGeometry`	Flag telling if the **Node on Geometry** option is activated (True) deactivated (False) for the mesh point.
`REAL8 DistanceToGeometry`	Distance from underlying point to mesh point location on the associated geometry.
`BOOL Locked`	Flag telling if the **Locked** option is activated (True) deactivated (False) for the mesh point.

`REAL8 LocationOnGeometry[0..2]`
or
`VARIANT vLocationOnGeometry`
	These are the coordinates of the mesh point on geometry, not the coordinates of the underlying point. They are always specified in global rectangular coordinates.

`BOOL AutoAssign`	When a Mesh Point is within the tolerance specified in the Geometry Connections section of the Mesh Point Options dialog box, automatically associates that Mesh Point to the closest piece of geometry.

MeshHardPoint Object Methods

Enable

Unpublished work. © 2025 Siemens

Enable **(bEnable)**	
Description:	
This method is used to activate or deactivate a Mesh Point.	
Input:	
`BOOL bEnable`	Flag to set the state of the mesh point.
Output:	
None	None
Return Code:	
`FE_FAIL`	Mesh point is not proper type.
Remarks/Usage:	
None	
Example:	
None	

IsEnabled

IsEnabled **(void)**	
Description:	
This method checks the state of the mesh point.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Mesh point is deactivated.
`FE_OK`	Mesh point is activated.
Remarks/Usage:	
None	
Example:	
None	

OnPoint

`OnPoint` **`( pointID )`**	
Description:	
This method sets the ID of the underlying point of a mesh point.	
Input:	
`INT4 pointID`	ID of geometric point to use for underlying point of mesh point.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Specified underlying geometric point does not exist.
Remarks/Usage:	
None	
Example:	
None	

OnPoint2

Unpublished work. © 2025 Siemens

OnPoint2

(pointID, bAutoAssign)

Description:

This method sets the ID of the underlying point of a mesh point.

Input:

`INT4 pointID`	ID of geometric point to use for underlying point of mesh point.
`BOOL bAutoAssign`	When True, if a Mesh Point is within the tolerance specified in the Geometry Connections section of the Mesh Point Options dialog box, it is automatically associates that Mesh Point to the closest piece of geometry. When False, the auto assign functionality is turned off.

Output:

None	None

Return Code:

`FE_NOT_EXIST`	Specified underlying geometric point does not exist.

Remarks/Usage:

None

Example:

None

AutoDistanceToGeometry

AutoDistanceToGeometry

(void)

Description:

This method automatically calculates a distance, then sets the *Distance to Geometry* value for the mesh point object which is associated with a geometric entity.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Mesh point is not associated with any geometric entity.
`FE_OK`	Calculation of distance and setting of value successful.
Remarks/Usage:	
None	
Example:	
None	

UpdateMeshPoints

`UpdateMeshPoints` **`( bDoAll, nMeshPointSetID, nDuplicatesOnSolidID )`**
Description:
This method automatically determines which geometric entity all or the selected mesh point(s) should be associated.
Input:

`BOOL bDoAll`	Flag to have all mesh points be updated (True) or only the mesh points included in the set object specified for `nMeshPointSetID` (False).
`INT4 nMeshPointSetID`	The ID of a set that contains a list of mesh point IDs that will be automatically associated with geometric entities, if they are not already associated with a geometric entity. Alternatively, if you specify a negative value, this is simply the ID of the single mesh point to associate to a geometric entity.

Unpublished work. © 2025 Siemens

`INT4 nDuplicatesOnSolidID`	ID of solid to automatically attach Mesh Point(s) currently in set object specified for `nMeshPointSetID`. Mesh points in set will automatically attach to points, curves, and/or surfaces of the specified solid, or the solid itself. Setting this to a value of 0 will consider all solids.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

UpdateMeshPointsOnGeometry

`UpdateMeshPointsOnGeometry`
`( nSolidSetID, nSurfaceSetID, nCurveSetID, nPointSetID, nDuplicatesOnSolidID, bUpdateUnassignedPoints, bUpdatePointsOnDeletedGeometry, bResizeCurves, bEnabledPointsOnly )`
Description:
This method updates mesh points associated with specific solids, surfaces, curves, and/or points while considering a number of additional options.
Input:

`INT4 nSolidSetID`	The ID of a set that contains a list of solid IDs to consider when updating mesh points. Alternatively, if you specify a negative value, this is simply the ID of the single solid to consider when updating.
`INT4 nSurfaceSetID`	The ID of a set that contains a list of surface IDs to consider when updating mesh points. Alternatively, if you specify a negative value, this is simply the ID of the single surface to consider when updating.

Unpublished work. © 2025 Siemens

`INT4 nCurveSetID`	The ID of a set that contains a list of curve IDs to consider when updating mesh points. Alternatively, if you specify a negative value, this is simply the ID of the single curve to consider when updating.
`INT4 nPointSetID`	The ID of a set that contains a list of point IDs to consider when updating mesh points. Alternatively, if you specify a negative value, this is simply the ID of the single point to consider when updating.
`INT4 nDuplicatesOnSolidID`	ID of solid to automatically attach Mesh Point(s). Only mesh points currently attached to the points, curves, surfaces and/or solids specified for `nPointSet`, `nCurveSet`, `nSurfaceSet`, and `nSolidSet` will be considered. Setting this to a value of 0 will consider all solids.
`BOOL bUpdateUnassignedPoints`	Flag to have unassigned mesh points be updated (True) or only mesh points which are currently assigned to geometry (False).
`BOOL bUpdatePointsOnDeletedGeometry`	Flag to have update points on geometric entities which have been deleted (True) or not (False).
`BOOL bResizeCurves`	Flag to have update mesh size on curves which now have a mesh point associated (True) or not (False).
`BOOL bEnabledPointOnly`	Flag to only consider mesh points which are currently enabled (True) or all mesh points (False).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Solid

Unpublished work. © 2025 Siemens

Solid	
(pnSolidID)	
Description:	
This method retrieves the ID of the Solid associated with a mesh point.	
Input:	
None	None
Output:	
`INT4 pnSolidID`	ID of Solid associated with mesh point.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

AtCoordinates

AtCoordinates	
(nCount, vXYZ, bAutoAssign)	
Description:	
None	
Input:	
`INT4 nCount`	None
`VARIANT vXYZ`	
`BOOL bAutoAssign`	

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Clear

```
Clear

( void )
```

Description:	
None	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

OnSurface

Unpublished work. © 2025 Siemens

OnSurface **(nSurfaceID, nCount, vUV, bAutoAssign)**	
Description:	
None	
Input:	
`INT4 nSurfaceID`	None
`INT4 nCount`	
`VARIANT vUV`	
`BOOL bAutoAssign`	
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Mesh Point Definition Objects

MeshPointDefinition (`MeshHardPointDefinition`) objects correspond to the mesh point definitions in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for point objects. For more information, see **Common Entity Properties and Methods**.

Use the `feMeshHardPointDefinition` method of the Femap Application object to create Mesh Point Definition objects.

Unpublished work. © 2025 Siemens

MeshHardPointDefinition Object Properties

MeshPointDefinition objects are always stored with their ID equal to the ID of the Mesh Point Definition; however, the IDs of the Mesh Points which reference a Mesh Point Definition can have any ID. To find a particular Mesh Point which references, use the `NextMeshPoint` to get a fully populated MeshPoint object and check what it contains.

Property	Description
`STRING title`	Title of Mesh Point Definition.

MeshHardPointDefinition Object Methods

ResetNextMeshPoint

`ResetNextMeshPoint` **`( void )`**	
Description:	
This method initializes all options prior to calling the `NextMeshPoint` method.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	None
Remarks/Usage:	
This method is used prior to calling `NextMeshPoint` to specify which MeshPoint will be retrieved. It simply resets this object so that the next call to `NextMeshPoint` will return the first MeshPoint available in the object.	

Unpublished work. © 2025 Siemens

Example:
None

NextMeshPoint

`NextMeshPoint` **`( pDISPATCH )`**	
Description:	
This method returns the next available mesh point that is contained in the Mesh Point Definition.	
Input:	
None	None
Output:	
`OBJECT pDISPATCH`	The next mesh point object.
Return Code:	
`OBJECT pDISPATCH`	The next mesh point object.
Remarks/Usage:	
This method, coupled with `ResetNextMeshPoint` provides an easy way to retrieve all mesh points from a Mesh Point Definition. For any method where API objects are returned as parameters, such as this one, it is your responsibility to free that object when you are finished using the object. Some programming languages like Basic will do this automatically, but others like C++, require you to do this manually. The objects are only created or returned when the Return Code is `FE_OK`.	
Example:	
None	

CountMeshPoints

Unpublished work. © 2025 Siemens

`CountMeshPoints`	
`( void )`	
Description:	
This method returns the number of mesh points referenced by a mesh point definition, as a return code.	
Input:	
None	None
Output:	
None	None
Return Code:	
`INT4 numMeshPoints`	The number of mesh points referenced by this mesh point definition.
Remarks/Usage:	
This method relies on the current Mesh Point Definition ID to count the referenced mesh points.	
Example:	
None	

Monitor Point Objects

Monitor Point objects correspond to Monitor Points in the model, which are found on the **Model → Simulation Entities** menu. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Monitor Point objects. For more information, see **Common Entity Properties and Methods**.

Use the `feMonitorPoint` method of the Femap Application object to create Monitor Point objects.

Monitor Point Object Properties

Monitor Points are always stored with a `SetID=1`, and the ID equal to the monitor point ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 color`	The monitor point color.
`INT4 layer`	The layer associated with the monitor point.
`STRING title`	Monitor Point title (maximum 56 characters).
`STRING Name`	Monitor Point Name (maximum 8 characters).
`INT4 type`	Monitor Point type = enum `zMptType`: Aerodynamic Control Monitor Point: Component - 0..Aero Panel = `MPT_1_AEROPANEL (0)` Component - 1..Aero Mesh = `MPT_1_AEROMESH (1)` Component - 2..Nodes = `MPT_1_STRUCT (2)` Element Monitor Point = `MPT_2 (4)` Integrated Load Monitor Point = `MPT_3 (3)`

`INT4 SumContributions[0..3]`
 	
or
 	
`VARIANT vSumContributions`
	Only used when type = `MONPNT3`.
	
	Array of on/off flags for Monitor Point Contributions (enum `zMptContribution`):
	
	`SumComponents[0]` = Reaction (`MPT_SPC`)
	`SumComponents[1]` = MultiPoint Reaction (`MPT_MPC`)
	`SumComponents[2]` = Applied Loads (`MPT_APPLIED`)
	`SumComponents[3]` = DMIG (`MPT_DMIG`)

`INT4 ElemGroup`	Only used when type = `MONPNT3`. ID of Group of Included Elements. 0 = None, therefore look for List of Included Elements (see `HasElemList` and `ElemSetID`).
`BOOL HasElemList`	Only used when type = `MONPNT3`. When True, a List of Included Elements has been defined. See `ElemSetID`. When False, either a ElemGroup has been defined or there are no Included Elements.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ElemSetID`	Only used when type = `MONPNT3`. ID of Set Object containing IDs of Included Elements.
`STRING CompTitle`	Monitor Point List Name (maximum 8 characters).
`INT4 NodeGroup`	Used when type = `MONPNT1_STRUCT` or `MONPNT3`. ID of Group of Included Nodes. 0 = None, therefore look for List of Included Nodes (see `HasNodeList` and `NodeSetID`).
`BOOL HasNodeList`	Used when type = `MONPNT1_STRUCT` or `MONPNT3`. When True, a List of Included Nodes has been defined. See `NodeSetID`. When False, either a NodeGroup has been defined or there are no Included Nodes.
`INT4 NodeSetID`	Used when type = `MONPNT1_STRUCT` or `MONPNT3`. ID of Set Object containing IDs of Included Nodes.
`BOOL HasPanelList`	Only used when type = `MONPNT1_AEROPANEL`. When True, a List of Aero Panels has been defined. See `PanelSetID`. When False, there are no Included Aero Panels.
`INT4 PanelSetID`	Only used when type = `MONPNT1_AEROPANEL`. ID of Set Object containing IDs of Included Aero Panels.
`BOOL HasAeroMeshList`	Only used when type = `MONPNT1_AEROMESH`. When True, a List of Aero Mesh has been defined. See `AeroMeshSetID`. When False, there are no Included Aero Mesh entities.
`INT4 AeroMeshSetID`	Only used when type = `MONPNT1_AEROMESH`. ID of Set Object containing IDs of Included Aero Mesh entities.
`INT4 SumComponents[0..5]`	Array of flags for Summation Axes (enum `zMptComponent`):

Unpublished work. © 2025 Siemens

Property	Description

or
`VARIANT vSumComponents`
	`SumComponents[0] = FX (MPT_FX)`
	`SumComponents[1] = FY (MPT_FY)`
	`SumComponents[2] = FZ (MPT_FZ)`
	`SumComponents[3] = MX (MPT_MX)`
	`SumComponents[4] = MY (MPT_MY)`
	`SumComponents[5] = MZ (MPT_MZ)`

`REAL8 x`	These are the individual coordinates of the Summation Point. They are always specified in global rectangular coordinates.
`REAL8 y`	
`REAL8 z`	

`REAL8 xyz[0..2]`
or
`VARIANT vxyz`
	The coordinates of the Summation Point as an Array/Variant.

`INT4 defCSys`	ID of Coordinate System used for Summation Point.
`INT4 outCSys`	ID of Output Coordinate System for Summation Axes Results (-1 = Use Same CSys as Summation Point).
`INT4 NumberOfContributions`	Number of Contributions in list found in NASTRAN Element Monitor Point dialog box.

Monitor Point Object Methods

There following methods are available in addition to the standard entity methods.

PutComponentSet

Unpublished work. © 2025 Siemens

PutComponentSet	
(entityTYPE, pSelectedComponentSet)	
Description:	
This method loads Node IDs into the Node List (type = MPT_1_STRUCT or MPT_3), Element IDs into the Element List (type = MPT_3), or Aero Panels into the Aero Panel List (type = MPT_1_AEROPANEL) from a Set Object.	
Input:	
INT4 entityTYPE	The entity type: 7=Node (FT_NODE), 8=Element (FT_ELEMENT), 95=Aero Panel/Body (FT_AERO_PANEL).
OBJECT pSelectedComponentSet	Set Object containing the IDs of the entities.
Output:	
None	None
Return Code:	
FE_BAD_TYPE	The data type for the component did not match the type of Monitor Point.
Remarks/Usage:	
Cannot be used when type = MPT_1_AEROMESH. Use PutAeroMeshSet instead. For more information, see GetComponentSet.	
Example:	
None	

GetComponentSet

GetComponentSet	
(entityTYPE, pSelectedComponentSet)	
Description:	

Unpublished work. © 2025 Siemens

This method loads a Set object that contains the IDs of the Nodes in the Node List (type = MPT_1_STRUCT or MPT_3), Elements in the Element List (type = MPT_3), or Aero Panels in the Aero Panel List (type = MPT_1_AEROPANEL).	
Input:	
`INT4 entityTYPE`	The entity type: 7=Node (FT_NODE), 8=Element (FT_ELEMENT), 95=Aero Panel/Body (FT_AERO_PANEL).
Output:	
`OBJECT pSelectedCompenentSet`	Set object that contains the IDs of the Nodes in the Node Lis, Elements in the Element List, or Aero Panels in the Aero Panel List.
Return Code:	
`FE_BAD_TYPE`	The data type for the component did not match the type of Monitor Point.
Remarks/Usage:	
Cannot be used when type = MPT_1_AEROMESH. Use `GetAeroMeshSet` instead. See **GetAeroMeshSet**.	
Example:	
None	

ClearComponentSet

`ClearComponentSet` **`( entityTYPE )`**	
Description:	
This method clears the array that contains the array that contains the IDs of the Nodes in the Node List (type = MPT_1_STRUCT or MPT_3), Elements in the Element List (type = MPT_3), or Aero Panels in the Aero Panel List (type = MPT_1_AEROPANEL).	
Input:	
`INT4 entityTYPE`	The entity type: 7=Node (FT_NODE), 8=Element (FT_ELEMENT), 95=Aero Panel/Body (FT_AERO_PANEL).

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	The data type for the component did not match the type of Monitor Point.
Remarks/Usage:	
Cannot be used when type = `MPT_1_AEROMESH`. Use `ClearAeroMeshSet` instead. See ClearAeroMeshSet.	
Example:	
None	

PutAeroMeshSet

`PutAeroMeshSet` **`( nSetID )`**	
Description:	
This method loads a set of Aero Mesh entity IDs into the Aero Mesh List.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the aero mesh entities.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Only valid when type = `MPT_1_AEROMESH`. For all other types, use `PutComponentSet`. For more information, see PutComponentSet.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetAeroMeshSet

GetAeroMeshSet

(pAeroMeshSet)

Description:

This method Loads a Set object that contains the IDs of the Aero Mesh entities in the Aero Mesh List.

Input:

None	None

Output:

`OBJECT pAeroMeshSet`	Set object that contains the IDs of the Aero Mesh entities in the Aero Mesh list.

Return Code:

`FE_BAD_TYPE`	The data type for the component did not match the type of Monitor Point.

Remarks/Usage:

Only valid when type = `MPT_1_AEROMESH`. For all other types, use `GetComponentSet`.

Example:

None

ClearAeroMeshSet

ClearAeroMeshSet

(void)

Description:

This method clears the array that contains the array that contains the IDs of the Aero Mesh entities in the Aero Mesh List.

Input:

None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_OK`	
Remarks/Usage:	
Only valid when type = `MPT_3`. Cannot be used when type = `MPT_1_AEROMESH` or `MPT_2`. Use `ClearComponentSet` instead.	
Example:	
None	

AddManualContribution

`AddManualContribution`	
`( nElemID, cTABLE, cTYPE, cNDDLitem )`	
Description:	
This method adds a single line to the list in the NASTRAN Element Monitor Point dialog box by specifying a value for Type, Table, Component, and Element ID using the available options described in the Nastran documentation. See Remarks/Usage section for more information.	
Input:	
`INT4 nElemID`	ID of Element to specify or update manual contribution details.
`STRING cTABLE`	Text in *Table* column of the NASTRAN Element Monitor Point dialog box. Text must be one of the available options for the TABLE field for the MONPNT2 entry, which are restricted to FORCE, STRESS, or STRAIN only.
`STRING cTYPE`	Text in *Type* column of the NASTRAN Element Monitor Point dialog box. Text must be one of the available options for the TYPE field for the MONPNT2 entry, which is the element type. See Remarks/Usage section for more information.
`STRING cNDDLitem`	Text in *Component* column of the NASTRAN Element Monitor Point dialog box. Text must be

Unpublished work. © 2025 Siemens

	one of the available options for COMP field for the MONPNT2 entry, which is the NDDL entity. See Remarks/Usage section for more information.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Only valid when type = `MPT_2`. For Simcenter Nastran, see the MONPNT2 entry in the Quick Reference Guide. For MSC Nastran, see the MSC Nastran DMAP Programmer's Guide, specifically the OEF datablock description for FORCE and the OES datablock description for STRESS and STRAIN.	
Example:	
None	

SetManualContributions

`SetManualContributions` **`( listcount, nElemID, cTABLE, cTYPE, cNDDLitem )`**	
Description:	
This method adds any number of lines to the list in the NASTRAN Element Monitor Point dialog box by specifying values in arrays for Type, Table, Component, and Element ID using the available options described in the Nastran documentation. See Remarks/Usage section for more information.	
Input:	
`INT4 listcount`	Number of lines to add to the list in the NASTRAN Element Monitor Point dialog box.
`INT4 nElemID[0..listcount-1]`	Array containing Element IDs to specify manual contribution details. An Element ID can appear in the list more than once.
`STRING cTABLE[0..listcount-1]`	Array containing text in *Table* column of the NASTRAN Element Monitor Point dialog box. Text must be one of the available options for the TABLE

Unpublished work. © 2025 Siemens

	field for the MONPNT2 entry, which are restricted to FORCE, STRESS, or STRAIN only.
`STRING cTYPE[0..listcount-1]`	Array containing text for the *Type* column. Text must be one of the available options for the TYPE field for the MONPNT2 entry, which is the element type. See Remarks/Usage section for more information.
`STRING cNDDLitem[0..listcount-1]`	Array containing text in *Component* column of the NASTRAN Element Monitor Point dialog box. Text must be one of the available options for COMP field for the MONPNT2 entry, which is the NDDL entity. See Remarks/Usage section for more information.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Only valid when type = `MPT_2`. For Simcenter Nastran, see the MONPNT2 entry in the Quick Reference Guide. For MSC Nastran, see the MSC Nastran DMAP Programmer's Guide, specifically the OEF datablock description for FORCE and the OES datablock description for STRESS and STRAIN.	
Example:	
None	

GetElemContributions

`GetElemContributions`

`( nCount, nElemID, cTABLE, cTYPE, cNDDLitem )`

Description:
This method retrieves values for Type, Table, Component, and Element ID from all lines in the list found in the NASTRAN Element Monitor Point dialog box. Retrieved text for Type, Table, and Component matches available options described in the Nastran documentation. See Remarks/Usage section for more information.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nCount`	Number of lines in the list in the NASTRAN Element Monitor Point dialog box.
`INT4 nElemID[0..nCount-1]`	Array containing the Element ID from each line. An Element ID can appear in the list more than once.
`STRING cTABLE[0..nCount-1]`	Array containing text in *Table* column for each line. Text will be one of the available options for the TABLE field for the MONPNT2 entry (FORCE, STRESS, or STRAIN only). See Remarks/Usage section for more information.
`STRING cTYPE[0..nCount-1]`	Array containing text for the *Type* column for each line. Text will be one of the available options for the TYPE field for the MONPNT2 entry, which is the element type. See Remarks/Usage section for more information.
`STRING cNDDLitem[0..nCount-1]`	Array containing text in *Component* column for each line. Text will be one of the available options for COMP field for the MONPNT2 entry, which is the NDDL entity. See Remarks/Usage section for more information.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Only valid when type = `MPT_2`. For Simcenter Nastran, see the MONPNT2 entry in the Quick Reference Guide. For MSC Nastran, see the MSC Nastran DMAP Programmer's Guide, specifically the OEF datablock description for FORCE and the OES datablock description for STRESS and STRAIN.	
Example:	
None	

GetAtIndex

Unpublished work. © 2025 Siemens

GetAtIndex

(nIndex, nElemID, cTABLE, cTYPE, cNDDLitem)

Description:

This method retrieves the value for Type, Table, Component, and Element ID from a single line in the list found in the NASTRAN Element Monitor Point dialog box, based on the index value of the line (0=Top of list, "n" = Bottom of list). Retrieved text for Type, Table, and Component matches available options described in the Nastran documentation. See Remarks/Usage section for more information.

Input:

`INT4 nIndex`	Line in the NASTRAN Element Monitor Point dialog box to retrieve information. 0=Top of list, "n" = Bottom of list.

Output:

`INT4 nElemID`	Element ID for line specified by `nIndex`.
`STRING cTABLE`	Text in the *Table* column for the specified line. Text will be one of the available options for the TABLE field for the MONPNT2 entry (FORCE, STRESS, or STRAIN only). See Remarks/Usage section for more information.
`STRING cTYPE`	Text in the *Type* column for the specified line. Text will be one of the available options for the TYPE field for the MONPNT2 entry, which is the element type. See Remarks/Usage section for more information.
`STRING cNDDLitem`	Text in the *Component* column for the specified line. Text will be one of the available options for COMP field for the MONPNT2 entry, which is the NDDL entity. See Remarks/Usage section for more information.

Return Code:

None	None

Remarks/Usage:

Only valid when type = `MPT_2`.

For Simcenter Nastran, see the MONPNT2 entry in the Quick Reference Guide. For MSC Nastran, see the MSC Nastran DMAP Programmer's Guide, specifically the OEF datablock description for FORCE and the OES datablock description for STRESS and STRAIN.

Unpublished work. © 2025 Siemens

Example:
None

SetAtIndex

`SetAtIndex`
 	
`( nIndex, UpdateElemID, nElemID, UpdateTABLE, cTABLE, UpdateTYPE, cTYPE, UpdateNDDLitem, cNDDLitem )`	

Description:	
This method updates the value for Type, Table, Component, and/or Element ID for a single line in the list found in the NASTRAN Element Monitor Point dialog box, based on the index value of the line (0=Top of list, "n" = Bottom of list). Retrieved text for Type, Table, and Component matches available options described in the Nastran documentation. See Remarks/Usage section for more information.	
Input:	
`INT4 nIndex`	Line in the NASTRAN Element Monitor Point dialog box to update information. 0=Top of list, "n" = Bottom of list.
`BOOL UpdateElemID`	If True, Element ID value is updated. If False, it is not.
`INT4 nElemID`	Element ID to set for line specified by `nIndex`.
`BOOL UpdateTABLE`	If True, "Table" text is updated. If False, it is not.
`STRING cTABLE`	Text to set *Table* column for the specified line. Text must be one of the available options for the TABLE field for the MONPNT2 entry (FORCE, STRESS, or STRAIN only). See Remarks/Usage section for more information.
`BOOL UpdateTYPE`	If True, "Type" text is updated. If False, it is not.
`STRING cTYPE`	Text to set in *Type* column for the specified line. Text must be one of the available options for the TYPE field for the MONPNT2 entry, which is the element type. See Remarks/Usage section for more information.

Unpublished work. © 2025 Siemens

`BOOL UpdateNDDLitem`	If True, *Component* text is updated. If False, it is not.
`STRING cNDDLitem`	Text to set in *Component* column for the specified line. Text must be one of the available options for COMP field for the MONPNT2 entry, which is the NDDL entity. See Remarks/Usage section for more information.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Only valid when type = `MPT_2`. For Simcenter Nastran, see the MONPNT2 entry in the Quick Reference Guide. For MSC Nastran, see the MSC Nastran DMAP Programmer's Guide, specifically the OEF datablock description for FORCE and the OES datablock description for STRESS and STRAIN.	
Example:	
None	

ClearElemContributions

`ClearElemContributions` **`( void )`**	
Description:	
This method clears the list of element contributions.	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_OK`	
Remarks/Usage:	
Only valid when type = `MPT_2`.	
Example:	
None	

Node Objects

Node objects correspond to the nodes in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for nodes. For more information, see **Common Entity Properties and Methods**.

Use the `feNode` method of the Femap Application object to create Node objects.

Node Object Properties

Nodes are always stored with a `SetID=1`, and the ID equal to the node ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`REAL8 x`	These are the coordinates of the node. They are always specified in global rectangular coordinates.
`REAL8 y`	
`REAL8 z`	
`VARIANT xyz`	The coordinates of the node as a Variant.
`INT4 layer`	The ID of the layer associated with the node.
`INT4 color`	The node color.

Unpublished work. © 2025 Siemens

Property	Description

`BOOL permBC[0..5]`
 	
or
 	
`VARIANT vpermBC`
	An array of flags telling whether the corresponding degrees of freedom are constrained (True) or free (False) as permanent constraints.

`INT4 type`	The node type (0=Node, 1=Scalar, 2=Extra).
`INT4 defCSys`	The ID of the definition coordinate system.
`INT4 outCSys`	The ID of the output coordinate system.
`INT4 superelementID`	The ID of the superelement that contains this node (0=None/Residual).
`INT4 NumberOfElements`	Number of elements using the node.
`INT4 NumberOfLoads`	Total number of nodal loads using the node from all load sets.
`INT4 NumberOfConstraints`	Total number of nodal constraints and/or constraint equations using the node from all constraint sets.
`INT4 NumberOfMatrices`	Total number of Direct Matrix Input entities which reference the node.
Additional Properties	
`REAL8 xyz[0..2]`	Accesses all three coordinates of a node as a Variant. Same as using x, y, and z independently.

Node Object Methods

GetAll

Unpublished work. © 2025 Siemens

GetAll

(entID, x, y, z, layer, color, type, defCSys, outCSys, permBC)

Description:

This method retrieves an entity with the specified ID.

Input:

`INT4 entID`	ID of the entity to retrieve.

Output:

`REAL8 x`	Refer to the property descriptions for each of these items. `permBC` is returned in an integer array, even though it contains only True/False values.
`REAL8 y`	
`REAL8 z`	
`INT4 layer`	
`INT4 color`	
`INT4 type`	
`INT4 defCSys`	
`INT4 outCSys`	
`INT4 permBC[0..5]`	

Return Code:

`FE_FAIL`	The specified entity does not exist.

Remarks/Usage:

Unlike the standard `Get` method, this function returns all of the properties as arguments. If you need to get all of the property data anyway, calling this will be faster than multiple calls to retrieve each property individually.

Example:

None

Unpublished work. © 2025 Siemens

GetAllArray

GetAllArray **(setID, numNode, entID, xyz, layer, color, type, defCSys, outCSys, permBC)**	
Description:	
This method retrieves arrays of values for a set of nodes.	
Input:	
`INT4 setID`	ID of the set that contains the node IDs to retrieve. If 0, retrieve all nodes in the model.
Output:	
`INT4 numNode`	The number of nodes for which data is being returned.
`INT4 entID[0..numNode-1]`	Each entry is the ID of the associated node.
`REAL8 xyz[0..(3*numNode)-1]`	The coordinates of the node. There are 3 entries, x, y, z for each node.
`INT4 layer[0..numNode-1]`	The ID of the layer for the node.
`INT4 color[0..numNode-1]`	The color of the node.
`INT4 type[0..numNode-1]`	The type of the node.
`INT4 defCSys[0..numNode-1]`	The definition coordinate system of the node.
`INT4 outCSys[0..numNode-1]`	The output coordinate system of the node.
`INT4 permBC[0..(6*numNode)-1]`	Flags indicating the permanent constraints on the node. 0=off, 1=on. There are 6 entries, corresponding to the degrees of freedom for each node.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected nodes.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method is the fastest way to retrieve information about a large number of nodes. If a retrieved node ID does not exist, then all of the values for that node ID will be 0.
Example:
None

GetAllArray2

GetAllArray2	
(setID, numNode, entID, exist, xyz, layer, color, type, defCSys, outCSys, permBC)	
Description:	
This method retrieves arrays of values for a set of nodes, including if a node exists in the database.	
Input:	
`INT4 setID`	ID of the set that contains the node IDs to retrieve. If 0, retrieve all nodes in the model.
Output:	
`INT4 numNode`	The number of nodes for which data is being returned.
`INT4 entID[0..numNode-1]`	Each entry is the ID of the associated node.
`BOOL exist[0..numNode-1]`	Node existence, True=Node exists, False=Node does not exist.
`REAL8 xyz[0..(3*numNode)-1]`	The coordinates of the node. There are 3 entries, x, y, z for each node.
`INT4 layer[0..numNode-1]`	The ID of the layer for the node.
`INT4 color[0..numNode-1]`	The color of the node.
`INT4 type[0..numNode-1]`	The type of the node.
`INT4 defCSys[0..numNode-1]`	The definition coordinate system of the node.

Unpublished work. © 2025 Siemens

`INT4 outCSys[0..numNode-1]`	The output coordinate system of the node.
`INT4 permBC[0..(6*numNode)-1]`	Flags indicating the permanent constraints on the node. 0=off, 1=on. There are 6 entries, corresponding to the degrees of freedom for each node.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected nodes.
Remarks/Usage:	
This method is the fastest way to retrieve information about a large number of nodes. If a retrieved node ID does not exist, then all of the values for that node ID will be 0. If it is known that all of the nodes in the node set exist, use `GetAllArray`.	
Example:	
None	

GetCoordArray

`GetCoordArray`
`( setID, numNode, entID, xyz )`
Description:
This method retrieves arrays of coordinates for a set of Nodes.
Input:

`INT4 setID`	ID of the set that contains the node IDs to retrieve. If 0, retrieve all nodes in the model.
Output:	
`INT4 numNode`	The number of nodes for which data is being returned.
`INT4 entID[0..numNode-1]`	Each entry is the ID of the associated node.
`REAL8 xyz[0..(3*numNode)-1]`	The coordinates of the node. There are 3 entries, x, y, z for each node.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to retrieve data from the selected nodes.
Remarks/Usage:	
This is a simpler form of `GetAllArray`, if you simply need the nodal coordinates. If a retrieved node ID does not exist, then all of the values for that node ID will be 0.	
Example:	
None	

GetCoordArray2

`GetCoordArray2` **`( setID, numNode, entID, exist, xyz )`**	
Description:	
This method retrieves arrays of coordinates for a set of Nodes and an additional array designating if a node exists in the database.	
Input:	
`INT4 setID`	ID of the set that contains the node IDs to retrieve. If 0, retrieve all nodes in the model.
Output:	
`INT4 numNode`	The number of nodes for which data is being returned.
`INT4 entID[0..numNode-1]`	Each entry is the ID of the associated node.
`BOOL exist[0..numNode-1]`	Node existence, True=Node exists, False=Node does not exist.
`REAL8 xyz[0..(3*numNode)-1]`	The coordinates of the node. There are 3 entries, x, y, z for each node.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected nodes.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This is a simpler form of `GetAllArray2`, if you simply need the nodal coordinates and also want to know if a node exists in the database. If a retrieved node ID does not exist, then all of the values for that node ID will be 0.
Example:
None

PutAll

`PutAll`	
`( entID,  x, y, z, layer, color, type, defCSys, outCSys, permBC )`	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`REAL8 x`	Refer to the property descriptions for each of these items. `permBC` is loaded in an integer array, even though it contains only True/False values.
`REAL8 y`	
`REAL8 z`	
`INT4 layer`	
`INT4 color`	
`INT4 type`	
`INT4 defCSys`	
`INT4 outCSys`	
`INT4 permBC[0..5]`	
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
Unlike the standard `Put` method, this function loads all of the properties as arguments. If you are updating all of the properties before storing an entity, this function will be faster than multiple calls to load each property individually.	
Example:	
None	

PutAllArray

`PutAllArray`	
`( numNode, entID,  xyz, layer, color, type, defCSys, outCSys, permBC )`	
Description:	
This method stores multiple node entities with the specified data.	
Input:	
`INT4 numNode`	The number of nodes to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numNode-1]`	The IDs of the nodes to store.
`REAL8 xyz[0..(3*numNode)-1]`	Refer to the property descriptions for each of these items. `xyz` is an array of the nodal coordinates, there are 3 entries for each node (must be in global rectangular coordinates). `permBC` is loaded in an integer array, even though it contains only True/False values. It contains six values per node.
`INT4 layer[0..numNode-1]`	
`INT4 color[0..numNode-1]`	
`INT4 type[0..numNode-1]`	
`INT4 defCSys[0..numNode-1]`	

Unpublished work. © 2025 Siemens

`INT4 outCSys[0..numNode-1]`	
`INT4 permBC[0..(6*numNode)-1]`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
This is the fastest `Put` method if you have a large number of nodes to create. All data can be accumulated in arrays and then transferred in one call.	
Example:	
None	

PutCoordArray

`PutCoordArray`	
`( numNode, entID,  xyz )`	
Description:	
This is a simpler version of `PutAllArray` that allows multiple node entities to be stored with the specified data.	
Input:	
`INT4 numNode`	The number of nodes to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numNode-1]`	The IDs of the nodes to store.
`REAL8 xyz[0..(3*numNode)-1]`	The coordinates of the nodes to store.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. The same layer, color... are used for every entity - only the coordinates are input as an array.	
Example:	
None	

GetClosest

`GetClosest` **`( xyz )`**	
Description:	
This method finds the Node closest to the desired location and loads it into the current object.	
Input:	
`REAL8 xyz[0..2]`	The coordinates of the location of interest.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Unable to find closest Node.
Remarks/Usage:	
None	
Example:	
None	

IsMidsideNode

Unpublished work. © 2025 Siemens

IsMidsideNode	
(elemSET)	
Description:	
This method determines if the current Node is used as a midside node on any elements.	
Input:	
`INT4 elemSET`	The ID of a Set Object used to limit the elements to consider when determining if the current Node is a midside node. Set this to 0 if you simply want to consider all elements.
Output:	
None	None
Return Code:	
`FE_OK`	Current node is used as a midside node on one or more elements.
`FE_FAIL`	Current node is not used as a midside node.
Remarks/Usage:	
If you are going to call this method repeatedly for multiple nodes performance may not be as good as using the `AddMidsideNodes()` method on the Set Object to find all midside nodes and then testing if this node is in the Set.	
Example:	
None	

GetSuperelementIDArray

GetSuperelementIDArray
(setID, numNode, entID, superelementID)
Description:
This method retrieves arrays of Node IDs and Superelement IDs for a set of Nodes.
Input:

Unpublished work. © 2025 Siemens

`INT4 setID`	ID of the set that contains the node IDs to retrieve. If 0, retrieve all nodes in the model.
Output:	
`INT4 numNode`	The number of nodes for which data is being returned.
`INT4 entID[0..numNode-1]`	Each entry is the ID of the associated node.
`INT4 superelementID[0..numNode-1]`	Each entry is the Superelement ID of the associated node.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected nodes.
Remarks/Usage:	
If a retrieved node ID does not exist, then all of the values for that node ID will be 0.	
Example:	
None	

PutSuperelementIDArray

`PutSuperelementIDArray` **`( numNode, entID, superelementID )`**	
Description:	
This method stores multiple node entities with the specified Superelement ID data.	
Input:	
`INT4 numNode`	The number of nodes to create (i.e. the number of entries in the arrays).
`INT4 entID[0..numNode-1]`	The IDs of the nodes to store.
`INT4 superelementID[0..numNode-1]`	The Superelement ID of the nodes to store.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
This is the fastest `Put` method if you have a large number of nodes to create with a Superelement ID, as all data can be accumulated in arrays and then transferred in one call.	
Example:	
None	

Optim Objects

> **Caution**
>
> THIS OBJECT AND ALL PROPERTIES AND METHODS ARE OBSOLETE AND SHOULD NOT BE USED IN VERSIONS OF Femap AFTER 11.4.2!!!

Please see **Optimization Relationship (Variable) Objects**, **Optimization Response (Limits) Objects**, and **Optimization Manufacturing Constraints Objects** for more information about the appropriate Objects to use to create Optimization entities in Femap. In addition, see **AnalysisMgr Objects** and **AnalysisCase Objects** for more information about setting up an Optimization analysis for Nastran using the **Analysis Set Manager**.

Optim objects correspond to the Design Optimization definitions in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Optim objects. For more information, see **Common Entity Properties and Methods**.

Use the `feOptim` method of the FEMAP Application object to create Optim objects.

Optim Object Properties

Optim are always stored with their `SetID=1`, and the ID varying from 1 to N, depending on the order in which the item was created. Each object contains a specific type of information: goal, vary or limit. You must look at the total collection of Optim objects to assemble the full design optimization definition.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 type`	The Optim object type: 1=Goal, 2=Vary, 3=Limit.
`INT4 goal`	Goal of optimization (only if type=1): 0=None,1=MinWeight
`INT4 vary`	Optimization variable: 0=None,1=RodArea,2=RodTorsion, 3=BarArea, 4=BarI1, 5=BarI2, 6=BarTorsion, 7=PlateThickness
`INT4 limit`	Optimization limit: 0=None, 1=NodXDisp, 2=NodYDisp,3=NodZDisp, 4=NodXRDisp, 5=NodYRDisp,6=NodZRDisp, 7=RodAxialStress,8=RodTorsionStress, 9=RodAxialStrain, 10=RodTorsionStrain, 11=BarAxialStress, 12=BarMaxStress, 13=BarMinStress, 14=BarAxialStrain, 15=BarMaxStrain, 16=BarMinStrain, 17=PltXNormalStress, 18=PltYNormalStress, 19=PltXYShearStress, 20=PltMaxPrinStress, 21=PltMinPrinStress, 22=PltVonMisesStress, 23=PltXNormalStrain, 24=PltYNormalStrain, 25=PltXYShearStrain, 26=PltMaxPrinStrain, 27=PltMinPrinStrain, 28=PltVonMisesStrain
`INT4 cycles`	The number of design cycles.
`INT4 varyType`	The type of data to vary. 11=Property.
`INT4 varyID`	Property ID to vary.
`INT4 varyExtraID`	Not used.
`BOOL varyPct`	If True, max/min specified as %, otherwise as values.
`REAL8 varyMin`	Minimum allowable value for variable.
`REAL8 varyMax`	Maximum allowable value for variable.
`REAL8 varyDelta`	Maximum change in variable before new analysis is required.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 respType`	Response on: Node=7 or Property=11.
`INT4 respID`	ID of node or property for response.
`REAL8 respMin`	Minimum allowable response.
`REAL8 respMax`	Maximum allowable response.

Optim Object Methods

There are no Optim object-specific methods. Only the standard entity methods are available.

Optimization Manufacturing Constraints Objects

Optimization Manufacturing Constraint objects correspond to Optimization Manufacturing Constraints in the model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Optimization Manufacturing Constraint objects. For more information, see **Common Entity Properties and Methods**. While Optimization Manufacturing Constraints can be setup by individually setting the appropriate properties, however using the predefined `Set*`/`Get*` methods is recommended.

Use the `feOptMC` method of the Femap Application object to create Optimization Manufacturing Constraint objects.

Optimization Manufacturing Constraint Object Properties

Optimization Manufacturing Constraints are always stored with a `SetID=1`, and the ID equal to the manufacturing constraint ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title`	Title of Manufacturing Constraint.
`INT4 ManConType`	Manufacturing constraint type. Use enum `zOptMnConType`: Additive Manufacturing = `FOPMC_ADDM (0)` Casting Die Direction = `FOPMC_CAST (1)` Checkerboarding Control = `FOPMC_CHECK (2)` Cyclic Symmetry = `FOPMC_CSYM (3)` Extrusion = `FOPMC_EXTR (4)` Minimum Size = `FOPMC_MINS (5)` Maximum Size = `FOPMC_MAXS (6)` Planar Symmetry = `FOPMC_PSYM (7)`

`REAL8 Vec1[0..2]`
 	
or
 	
`VARIANT Vec1`
	The XYZ components of the various vectors used by some of the manufacturing constraints. In addition, all planes are stored in terms of vectors. Must be specified in Global Rectangular coordinates.
	
	Used when `ManConTyp` = `FOPMC_ADDM`, `FOPMC_CAST`, `FOPMC_CSYM`, `FOPMC_EXTR`, or `FOPMC_PSYM`.

`REAL8 Vec2[0..2]`
 	
or
 	
`VARIANT Vec2`	

`REAL8 Vec3[0..2]`
 	
or
 	
`VARIANT Vec3`	

`REAL8 Vec4[0..2]`
 	
or
 	
`VARIANT Vec4`	

`INT4 nField1`	Integer value 1. When `ManConTyp = FOPMC_CSYM`, used to set Number of Sectors.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 dField1`	Real value 1, which is used when `ManConTyp` = `FOPMC_ADDM`, `FOPMC_MINS`, or `FOPMC_MAXS`. Specify 0.0 to turn off check box for fields where a real values is entered.
`REAL8 dField2`	Real value 2, which can potentially be used when `ManConTyp = FOPMC_ADDM`. Specify 0.0 to turn off check box for fields where a real value are entered.

Optimization Manufacturing Constraints Object Methods

The following methods are available in addition to the standard entity methods.

SetAddm

`SetAddm`	
`( dAngle, dMin, vBase, vDir )`	
Description:	
Sets up an additive manufacturing constraint.	
Input:	
`REAL8 dAngle`	Maximum overhang angle. Specify 0.0 for no constraint.
`REAL8 dMin`	Minimum allowed size. Specify 0.0 for no constraint.
`VARIANT vBase[0..2]`	Coordinates of a point on a plane for the base plate. Must be specified in Global Rectangular coordinates.
`VARIANT vDir[0..2]`	Components of a vector normal to the base plate in the direction of material addition.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

GetAddm

GetAddm **(pdAngle, pdMin, pvBase, pvDir)**	
Description:	
Retrieves an additive manufacturing constraint.	
Input:	
None	None
Output:	
`REAL8 pdAngle`	Maximum overhang angle. Specify 0.0 for no constraint.
`REAL8 pdMin`	Minimum allowed size. Specify 0.0 for no constraint.
`VARIANT pvBase`	Coordinates of a point on a plane for the base plate in Global Rectangular coordinates.
`VARIANT pvDir`	Components of a vector normal to the base plate in the direction of material addition.
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetCast

SetCast **(vCastBase, vCastDir, vMoldDir, vMoldDir2)**	
Description:	
Sets up a casting die direction manufacturing constraint.	
Input:	
`VARIANT vCastBase[0..2]`	Coordinates of a point on the casting plane, in Global Rectangular.
`VARIANT vCastDir[0..2]`	Components of a vector normal to the casting plane.
`VARIANT vMoldDir[0..2]`	Components of a vector that defines the first direction of mold removal.
`VARIANT vMoldDir2[0..2]`	Components of an optional secondary mold removal vector. Specify 0., 0., 0. to turn off.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetCast

Unpublished work. © 2025 Siemens

GetCast **(pvCastBase, pvCastDir, pvMoldDir, pvMoldDir2)**	
Description:	
Retrieves a casting die direction manufacturing constraint.	
Input:	
None	None
Output:	
`VARIANT pvCastBase`	Coordinates of a point on the casting plane, in Global Rectangular.
`VARIANT pvCastDir`	Components of a vector normal to the casting plane.
`VARIANT pvMoldDir`	Components of a vector that defines the first direction of mold removal.
`VARIANT pvMoldDir2`	Components of an optional secondary mold removal vector.
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	
Example:	
None	

SetCheckerboard

SetCheckerboard **(bEnable)**
Description:
Sets up a checkerboarding manufacturing constraint.

Unpublished work. © 2025 Siemens

Input:	
`BOOL bEnable`	Turn on checkerboarding control (True).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
Checkerboarding is on by default and a manufacturing constraint does not need to be created to turn on.	
Example:	
None	

GetCheckerboard

`GetCheckerboard` **`( pbEnable )`**	
Description:	
Retrieves a checkerboarding manufacturing constraint.	
Input:	
None	None
Output:	
`BOOL pbEnable`	Checkerboarding control is enabled (True).
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetCycSym

SetCycSym **(vBase, vRot, vNorm, nSect)**	
Description:	
Sets up a cyclic symmetry manufacturing constraint.	
Input:	
`VARIANT vBase[0..2]`	Coordinate of a point on a cyclic symmetry plane, in global rectangular.
`VARIANT vRot[0..2]`	Components of a vector defining the axis of rotation.
`VARIANT vNorm[0..2]`	Components of vector perpendicular to the axis of rotation.
`INT4 nSect`	Number of sectors.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetCycSym

Unpublished work. © 2025 Siemens

GetCycSym	
(pvBase, pvRot, pvNorm, pnSect)	
Description:	
Retrieves a cyclic symmetry manufacturing constraint.	
Input:	
None	None
Output:	
`VARIANT vBase`	Coordinate of a point on a cyclic symmetry plane, in global rectangular.
`VARIANT vRot`	Components of a vector defining the axis of rotation.
`VARIANT vNorm`	Components of vector perpendicular to the axis of rotation.
`INT4 nSect`	Number of sectors.
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	
Example:	
None	

SetExtrusion

SetExtrusion
(vDir)
Description:
Sets up a extrusion manufacturing constraint.

Unpublished work. © 2025 Siemens

Input:	
`VARIANT vDir`	Components of a vector defining the extrusion direction.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetExtrusion

`GetExtrusion`
`( pvDir )`
Description:
Retrieves a casting die direction constraint.
Input:

None	None
Output:	
`VARIANT vDir`	Components of a vector defining the extrusion direction.
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetMinSize

SetMinSize **(dSize)**	
Description:	
Sets up a minimum size manufacturing constraint.	
Input:	
`REAL8 dSize`	Minimum Size value.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetMinSize

GetMinSize **(pdSize)**
Description:
Retrieves a minimum size manufacturing constraint.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
`REAL8 pdSize`	Minimum Size value.
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	
Example:	
None	

SetMaxSize

`SetMaxSize` **`( dSize )`**	
Description:	
Sets up a maximum size manufacturing constraint.	
Input:	
`REAL8 dSize`	Maximum Size value.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

GetMaxSize

GetMaxSize **(pdSize)**	
Description:	
Retrieves a maximum size manufacturing constraint.	
Input:	
None	None
Output:	
`REAL8 pdSize`	Maximum Size value.
Return Code:	
`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	
Example:	
None	

SetPlSym

SetPlSym **(vBase, vDir)**
Description:

Unpublished work. © 2025 Siemens

Sets up a planar symmetry manufacturing constraint.	
Input:	
`VARIENT vBase[0..2]`	Coordinate of a point on a symmetry plane, in global rectangular.
`VARIENT vDir[0..2]`	Components of a vector normal to the symmetry plane.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetPlSym

`GetPlSym` **`( vBase, vDir )`**	
Description:	
Retrieves a planar symmetry manufacturing constraint.	
Input:	
None	None
Output:	
`VARIENT vBase[0..2]`	Coordinate of a point on a symmetry plane, in global rectangular.
`VARIENT vDir[0..2]`	Components of a vector normal to the symmetry plane.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_INVALID`	Invalid manufacturing constraint type.
Remarks/Usage:	
None	
Example:	
None	

Optimization Relationship (Variable) Objects

Optimization Relationship objects correspond to Optimization Variables in the model, including Material, Property and Element relations and Topology Optimization regions. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Optimization Relationship objects. For more information, see **Common Entity Properties and Methods**.

Use the `feOptRel` method of the Femap Application object to create Optimization Relationship objects.

Optimization Relationship Object Properties

Optimization Relationships (Variables) are always stored with a `SetID=1`, and the ID equal to the optimization relationship ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`STRING title`	Title of Optimization Relationship.
`INT4 RelationshipType`	Optimization relationship type. Use enum `zOptRelType`: Property = `FOPRT_PROPERTY(0)` Material = `FOPRT_MATERIAL(1)` Topology = `FOPRT_TOPOLOGY(2)` Element = `FOPRT_ELEM(3)`
`INT4 RelationshipID`	Material ID or Property ID for Material and Property relationship types.

Unpublished work. © 2025 Siemens

Property	Description
`STRING TypeField`	Optimization relationship type name for Material and Property relationship types.
`STRING NameField`	Optimization relationship name field for Material and Property relationship types. Either field names or numbers may be specified, however numbers must be specified using string values.
`BOOL TopologyActive`	Defines if topology regions are active or frozen. Only used for Topology relationship type.

`REAL8 Bounds[0..1]`
 	
or
 	
`VARIANT vBounds`
	Optimization bounds:
	
	[0]=Lower Bound value
	[1]=Upper Bound value

`INT4 BoundType`	Bound type. Only valid for Material and Property relationship types. Use enum `zOptBoundType`: Values = `FOPBT_VALUES (0)` Offset = `FOPBT_RELATIVE (1)` Percentage = `FOPBT_PCT (2)`

Optimization Relationship Object Methods

The following methods are available in addition to the standard entity methods.

SetupTopology

`SetupTopology` **`( nSetID, bActive)`**
Description:
Sets up the Optimization Relationship as a topology optimization region in a single call.
Input:

Unpublished work. © 2025 Siemens

`INT4 nSetID`	ID of a Set object containing elements for topology optimization.
`BOOL bActive`	Specifies if elements are active (True) or frozen (False).
Output:	
None	None
Return Code:	
`FE_FAIL`	No elements specified.
`FE_INVALID`	Elements have been specified that are not valid for topology optimization.
Remarks/Usage:	
None	
Example:	
None	

GetTopology

`GetTopology`
`( nSetID, bClear, bActive )`
Description:
Sets up the Optimization Relationship as a topology optimization region in a single call.
Input:

`INT4 nSetID`	ID of a Set object to be populated with topology optimization entities.
`BOOL bClear`	Clear any existing IDs from the Set.
Output:	
`BOOL bActive`	Entities are active (True) or frozen (False).
Return Code:	

Unpublished work. © 2025 Siemens

`FE_BAD_TYPE`	Optimization relationship object is not setup as topology optimization region.
Remarks/Usage:	
None	
Example:	
None	

SetupShellT

`SetupShellT` **`( nSetID, dLower, dUpper )`**	
Description:	
Sets up a shell thickness relationship in a single call.	
Input:	
`INT4 nSetID`	ID of Set object containing elements to optimize for thickness.
`REAL8 dLower`	Shell thickness lower bound.
`REAL8 dUpper`	Shell thickness upper bound.
Output:	
None	None
Return Code:	
`FE_FAIL`	No elements specified.
`FE_INVALID`	Invalid bounds specified for thickness.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetShellT

GetShellT **(nSetID, bClear, pdLower, pdUpper)**	
Description:	
Get shell thickness relationship parameters.	
Input:	
`INT4 nSetID`	ID of a Set object to be populated with topology optimization entities.
`BOOL bClear`	Clear any existing IDs from the Set.
Output:	
`REAL8 pdLower`	Shell thickness lower bound.
`REAL8 pdUpper`	Shell thickness upper bound.
Return Code:	
`FE_BAD_TYPE`	Optimization relationship object is not setup as a shell thickness relationship.
Remarks/Usage:	
None	
Example:	
None	

SetupProp

Unpublished work. © 2025 Siemens

SetupProp

(nPropID, sType, sName, eBoundType, dMin, dMax)

Description:

Sets property relationship parameters in a single call.

Input:

`INT4 nPropID`	Property ID.
`STRING sType`	Property type.
`STRING sName`	Property field name.
`INT4 eBoundType`	Bound type. Use enum `zBoundType`: Values = `FOPBT_VALUES (0)` Offset = `FOPBT_RELATIVE (1)` Percentage = `FOPBT_PCT (2)`
`REAL8 dMin`	Property value lower bound.
`REAL8 dMax`	Property value upper bound.

Output:

None	None

Return Code:

`FE_INVALID`	Invalid values for bounds.
`FE_NOT_EXIST`	Specified property does not exist.

Remarks/Usage:

When specifying property types or names that are not natively supported by Femap, these will show up as *Custom* optimization variables in the Femap User Interface.

Field IDs may be specified for the field name parameter, however they must be passed in as strings.

Example:

None

Unpublished work. © 2025 Siemens

GetProp

GetProp **(pnPropID, sType, sName, peBoundType, pdMin, pdMax)**	
Description:	
Get property relationship parameters.	
Input:	
None	None
Output:	
`INT4 pnPropID`	Property ID.
`STRING sType`	Property type.
`STRING sName`	Property field name.
`INT4 peBoundType`	Bound type. Use enum `zBoundType`: Values = `FOPBT_VALUES (0)` Offset = `FOPBT_RELATIVE (1)` Percentage = `FOPBT_PCT (2)`
`REAL8 pdMin`	Property value lower bound.
`REAL8 pdMax`	Property value upper bound.
Return Code:	
`FE_BAD_TYPE`	Optimization relationship object is not setup as a property relationship.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetupMatl

SetupMatl **(nMatlID, sType, sName, eBoundType, dMin, dMax)**	
Description:	
Sets material relationship parameters in a single call.	
Input:	
`INT4 nMatlID`	Material ID.
`STRING sType`	Material type.
`STRING sName`	Material field name.
`INT4 eBoundType`	Bound type. Use enum `zBoundType`: Values = `FOPBT_VALUES (0)` Offset = `FOPBT_RELATIVE (1)` Percentage = `FOPBT_PCT (2)`
`REAL8 dMin`	Material value lower bound.
`REAL8 dMax`	Material value upper bound.
Output:	
None	None
Return Code:	
`FE_INVALID`	Invalid values for bounds.
`FE_NOT_EXIST`	Specified material does not exist.
Remarks/Usage:	
When specifying material types or names that are not natively supported by Femap, these will show up as *Custom* optimization variables in the Femap User Interface. Field IDs may be specified for the field name parameter, however they must be passed in as strings.	

Unpublished work. © 2025 Siemens

Example:
None

GetMatl

GetMatl **(pnMatlID, sType, sName, peBoundType, pdMin, pdMax)**	
Description:	
Get material relationship parameters.	
Input:	
None	None
Output:	
`INT4 pnMatlID`	Material ID.
`STRING sType`	Material type.
`STRING sName`	Material field name.
`INT4 peBoundType`	Bound type. Use enum `zBoundType`: Values = `FOPBT_VALUES (0)` Offset = `FOPBT_RELATIVE (1)` Percentage = `FOPBT_PCT (2)`
`REAL8 pdMin`	Material value lower bound.
`REAL8 pdMax`	Material value upper bound.
Return Code:	
`FE_BAD_TYPE`	Optimization relationship object is not setup as a material relationship.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

CalculateBounds

`CalculateBounds` **`( pdLower, pdUpper, pdInit )`**	
Description:	
Returns the lower and upper bounds, as well as the initial value that will be written to the analysis deck. Note, this does not update any object properties.	
Input:	
None	None
Output:	
`REAL8 pdLower`	Lower bound.
`REAL8 pdUpper`	Upper bound.
`REAL8 pdInit`	Initial value.
Return Code:	
`FE_INVALID`	Invalid optimization relationship type or unable to calculate bounds.
Remarks/Usage:	
None	
Example:	
None	

Optimization Response (Limits) Objects

Optimization Response objects correspond to Optimization Limits in the model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties

Unpublished work. © 2025 Siemens

and methods defined for Entity objects are also available for Optimization Response objects. For more information, see **Common Entity Properties and Methods**. While Optimization Responses can be set up by setting the individual property values, use of the individual setup methods is recommended.

Use the `feOptResp` method of the Femap Application object to create Optimization Response objects.

Optimization Response (Limits) Object Properties

Optimization Responses (limits) are always stored with a SetID=1, and the ID equal to the optimization response ID.

Property	Description
`STRING title`	Title of Optimization Response.
`INT4 ResponseCategory`	Optimization response type. This is different than the response type field. Use enum `zOptRespType`: User-Defined = `FOPRSP_CUSTOM(0)` Nodal Displacement = `FOPRSP_NDISP (1)` SPC Forces = `FOPRSP_SPCFORCE (2)` Element Force, Stress, Strain = `FOPRSP_ELEM (3)` Strain Energy = `FOPRSP_ESE (4)` Frequency = `FOPRSP_FREQ (5)` Eigenvalue = `FOPRSP_EIGEN (6)` Buckling = `FOPRSP_BUCKLE (7)` Compliance = `FOPRSP_COMPL (8)` Volume = `FOPRSP_VOL (9)` Weight = `FOPRSP_WEIGHT (10)`
`STRING rtype`	Response type field.
`STRING ptype`	Response property name or element flag.
`INT32 atta`	Response attribute A.
`BOOL attaBlank`	Flag indicating atta should be written as a blank field.
`STRING attb`	Response attribute B. Data type is defined using the `attbFieldType` property, however must always be specified as a string value.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 attbFieldType`	attb field data type. Use enum `zOptRespAttType`: Blank Field = `FOPATT_BLANK (0)` Text Field = `FOPATT_CHAR (1)` Integer Field = `FOPATT_INT (2)` Real Number Field = `FOPATT_REAL (3)`
`INT4 attiDataType`	Data type for selected entities. Use enum `zDataType`. See **Entity Types**.
`INT4 VectorIDV2`	This property replaces `VectorID`. For more information see, **Working with Output in Femap v2020.1 and above**. Output vector ID for element optimization responses

Optimization Response (Limits) Object Methods

The following methods are available in addition to the standard entity methods.

SetEntities

`SetEntities` **`( nSetID, bClear, enDataType )`**	
Description:	
Sets the entities for optimization responses.	
Input:	
`INT4 nSetID`	ID of Set object containing entities for optimization responses.
`BOOL bClear`	Clear existing entities from Set Object.
`INT4 enDataType`	Data type of entities. Use enDataType. See **Entity Types**.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	No entities specified.
`FE_NOT_EXIST`	One or more of the specified entities does not exist.
Remarks/Usage:	
This method can be used to specify entities for optimization responses, however it is recommended to use the setup methods instead.	
Example:	
None	

5.58.2.2 GetEntities

This is a para in the topic template.

SetupDisplacement

`SetupDisplacement` **`( nSetID, nDof )`**	
Description:	
Sets up a displacement optimization response in a single call.	
Input:	
`INT4 nSetID`	The ID of a set that contains a list of node IDs. Alternatively, if you specify a negative value, this is simply the ID of the single node.
`INT4 nDof`	Displacement degree of freedom: Tx = 1, Ty = 2, Tz = 3, Rx = 4, Ry = 5, Rz = 6
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	No entities specified.
Remarks/Usage:	
Optimization response constraints are specified separately using the `SetConstraints` method.	
Example:	
None	

GetDisplacement

`GetDisplacement` **`( nSetID, bClear, nDof )`**	
Description:	
Retrieves attributes for displacement response in a single call.	
Input:	
`INT4 nSetID`	ID of Set object to be populated with displacement response nodes.
`BOOL bClear`	Clear Set Object of entities prior to populating.
Output:	
`INT4 nDof`	Displacement degree of freedom: Tx = 1, Ty = 2, Tz = 3, Rx = 4, Ry = 5, Rz = 6
Return Code:	
`FE_FAIL`	Invalid response type.
Remarks/Usage:	
Optimization response constraints are retrieved separately using the `GetConstraints` method.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetupSPCForce

SetupSPCForce **(nSetID, bValidate, nDof)**	
Description:	
Sets up an SPC force / moment optimization response in a single call	
Input:	
`INT4 nSetID`	The ID of a set that contains a list of node IDs. Alternatively, if you specify a negative value, this is simply the ID of the single node.
`BOOL bValidate`	Validate node set
`INT4 nDof`	Force/Moment Component. Fx = 1, Fy = 2, Fz = 3, Mx = 4, My = 5, Mz = 6
Output:	
None	None
Return Code:	
`FE_FAIL`	No entities specified
`FE_BAD_DATA`	One or more unconstrained nodes has been specified
Remarks/Usage:	
Optimization response constraints are specified separately using the `SetConstraints` method.	
Example:	
None	

Unpublished work. © 2025 Siemens

Output Objects (OBSOLETE)

Note

Starting with Femap version 2020.1, the entire Output Object has been deprecated and the Results Browsing Object MUST be used in place of the Output Object. The Results Browsing Object provides more capability when reviewing existing results and methods were added for Femap version 2020.1 to provide superior functionality to the Output Object.

Output objects correspond to the Output data in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Output objects. For more information, see **Common Entity Properties and Methods**.

Use the `feOutput` method of the Femap Application object to create Output objects.

Output Object Properties (OBSOLETE)

Output objects are always stored with their SetID equal to the ID of the output set/step that they reference, and the ID equal to the output vector ID of the type of data stored in the vector. Output vector IDs can vary from version to version of Femap. The best way to determine the standard IDs used is to simply read results into Femap and see what vectors are created.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`STRING title`	Output vector title (maximum 79 characters).

`INT4 component[0..19]`
 	
or
 	
`VARIANT vcomponent`
	Component vectors. Either zero, or the IDs of the X,Y,Z components, or the IDs of the corresponding elemental corner output. See below.

`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`INT4 location`	The location where the output is computed and stored. Nodal = 7, Elemental Centroid or Element Corner = 8.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL nonlinear`	If True, this result cannot be linearly combined.
`INT4 hascomponent`	If 1, `component[0..2]` are the X,Y,Z component values for nodal output. If 1, for centroidal plate and centroidal solid element data, this indicates that the component property contains the IDs of Output Vectors that contain the associated element corner data. If 2, `component[0..1]` contain the vector IDs of data at end of beams. If 3, `component[0..1]` contain the vector IDs of data at end of beams, but the data at the second end of beam is reversed in sign.
`BOOL centroidtotal`	If True, this vector has centroidal or nodal output.
`BOOL InitAsInteger`	If True, this vector is initialized to store integer values.
`REAL8 maxval` (Read Only)	The maximum value in the output vector.
`REAL8 minval` (Read Only)	The minimum value in the output vector.
`REAL8 absmaxval` (Read Only)	The maximum absolute value of any value in the output vector.
`INT4 maxvalID` (Read Only)	The ID of the node/element where the maxval occurs.
`INT4 minvalID` (Read Only)	The ID of the node/element where the minval occurs.
`REAL8 Value[1..N]`	The output values. The index must be the ID of the node/element where the output value is stored. The value for any node or element where no output exists will be 0.0. This property can be used to manipulate results one value at a time. In general, however, it will be faster to use one or more of the output methods that manipulate entire output vectors.

Output Components (OBSOLETE)

The components are 20 vector IDs of other vectors that are *components* of the current vector. There are several possible ways these components are used:

1. The current vector contains the vector magnitude of 3 other vectors that are in the global X, Y, and Z components of similar data:

Unpublished work. © 2025 Siemens

In this case, Components 0, 1, and 2 are the vector IDs of the X, Y, and Z component vectors, respectively. An example of this type of usage is Vector 1 (TOTAL TRANSLATION), which has Component 0=2(X TRANSLATION), Component 1=3(Y TRANSLATION) and Component 2=4(Z TRANSLATION). These components MUST be defined for deformed/animated plotting to work correctly. Components 3 through 19 must be 0 for this case.

2. The current vector contains data which should be plotted in a specific global direction:

   In this case, if plotting should occur in Global X, then Component 0 must equal the current Vector ID. If Global Y, then Component 1 = current Vector ID, and if Global Z then Component 2 = current Vector ID. All other components must equal 0 (i.e. only 1 nonzero component and it is equal to the current Vector ID).

3. The current vector contains elemental centroidal data for elements which also have related data at the element corners:

   In this case, the number of nonzero components depends upon the number of nodes on this element type (for example, output for linear plates has 4 components-4 nodes/corners). The nonzero components must equal the Vector IDs of the same type of data at the corners of the elements. For example, if the current vector is 7033 (CENTROIDAL VON MISES STRESS) then component 0 may be 20133 (CORNER1 VON MISES STRESS) and component 1 may be 30133 (CORNER2 VON MISES STRESS) etc. If you do not have corner data, all components must be 0. If the components are defined, Femap will use the corner data (not the data in the current vector) when calculating data at nodes (like for contouring). If no components are defined, Femap will assign the centroidal value to each node when doing the calculation.

4. The current vector contains line element data at one end of the element:

   This is just like the previous case. The first component is the vector ID of the End A data, the second component is the vector ID of the End B data. This type of output also requires special settings in the `comp_dir` variable. Set `comp_dir` to 2 if you want Femap to use the data as it is specified. Set `comp_dir` to 3 if you want Femap to reverse the sign of all End B data when doing a beam diagram. This adjusts for differences in sign conventions between the various analysis programs.

5. All other cases:

   All components should be 0. This will force the data in the current vector to be used for every case. The current setting of the **View Option Deformed Style - Deformed Direction** option will control the deformed plotting direction.

Output Vector IDs (OBSOLETE)

If you are writing your own Neutral files, you can choose any vector IDs that you want, but the normal conventions used by Femap are as follows:

Unpublished work. © 2025 Siemens

Standard Output Vectors		
Category	**Vector Range**	**Type**
1 to 9,999,999 (magnitude, or non-complex)	1 to 2,999	Nodal output
	3,000 to 5,999	Line element output
	6,000 to 59,999	Plate element output
	60,000 to 79,999	Solid element output
	80,000 to 89,999	Output on any element type
	90,000 to 99,999	PATRAN elemental output
	100,000 to 299,999	Plate corner output
	1,000,000 to 6,000,000	Laminate Ply output
	9,000,000 to 9,999,999	User Defined output
10,000,001 to 20,000,000 (complex, phase)	Same definition as magnitude data, just add 10,000,000 to IDs.	
20,000,001 to 30,000,000 (complex, real)	Same definition as magnitude data, just add 20,000,000 to IDs.	
30,000,001 to 40,000,000 (complex, imaginary)	Same definition as magnitude data, just add 30,000,000 to IDs.	

Output Object Methods (OBSOLETE)

The following methods are available in addition to the standard entity methods.

GetFromSet (OBSOLETE)

```
GetFromSet
```

```
( setID, entID )
```

Description:

This method retrieves data from a specified output vector, in a specified set.

Unpublished work. © 2025 Siemens

Input:	
`INT4 setID`	The Output Set containing the output vector to retrieve
`INT4 entID`	The Output Vector to retrieve
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

This method is simply a shortcut to replace the standard Get() method. Assuming “Out” is an Output object, the following two sequences are identical...

```
Out.GetFromSet( s, v )
```

and

```
Out.setID = s
  Out.Get( v )
```

Example:
None

GetOutputList (OBSOLETE)

```
GetOutputList
```

```
(  listcount, ID, value )
```

Description:
This method retrieves an output vector.
Input:

None	None
Output:	

Unpublished work. © 2025 Siemens

`INT4 listcount`	The number of entries in the ID and value arrays.
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is available.
`REAL8 value[0..N]`	The output values associated with the corresponding node/element in the ID array.
Return Code:	
`FE_NOT_EXIST`	There are no entries in the list.
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself.	
Example:	
None	

GetOutputListAtID (OBSOLETE)

`GetOutputListAtID`	
`( ID, defaultVal, value )`	
Description:	
This method retrieves data from an output vector at specified IDs.	
Input:	
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is being requested.
`REAL8 defaultVal[0..N]`	The default values to be returned if no output is found for a specified ID. This can be specified as NULL, in which case all default values will be 0.0.
Output:	
`REAL8 value[0..N]`	The output values associated with the corresponding node/element in the ID array.
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to retrieve IDs from the list or allocate memory for processing.
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself.	
Example:	
None	

GetOutputListAtSet (OBSOLETE)

`GetOutputListAtSet` **`( idSET, defaultVal, value )`**	
Description:	
This method retrieves data from an output vector at IDs selected in a Set.	
Input:	
`INT4 idSET`	The ID of a Set object that contains the IDs of the entities to retrieve.
`REAL8 defaultVal[0..N]`	The default values to be returned if no output is found for a specified ID. This can be specified as NULL, in which case all default values will be 0.0.
Output:	
`REAL8 value[0..N]`	The output values associated with the corresponding node/element in the ID array.
Return Code:	
`FE_FAIL`	Unable to retrieve IDs from the list or allocate memory for processing.
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetModel (OBSOLETE)

GetModel **(nModelID)**	
Description:	
This method returns the identifier of the model being used by the object.	
Input:	
None	None
Output:	
`INT8 nModelID`	Model identifier.
Return Code:	
None	None
Remarks/Usage:	
When the object is created, `nModelID` is set to the current/active model.	
Example:	
None	

SetModel (OBSOLETE)

SetModel **(nModelID)**	
Description:	
This method chooses the model to be used for all further operations.	
Input:	
`INT8 nModelID`	Model identifier.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
When the object is created, `nModelID` is set to the current/active model.	
Example:	
None	

SetComplexOptions (OBSOLETE)

`SetComplexOptions` **`( bExpandComplex, dComplexPhase )`**	
Description:	
This method sets a phase angle to use for any complex output vector (magnitude, phase, real or imaginary) which will cause the output to be returned as the expanded complex magnitude at the specified phase angle.	
Input:	
`BOOL bExpandComplex`	When True, complex expansion will occur.
`REAL8 dComplexPhase`	Value of phase angle to use for complex expansion.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method MUST be called before any `Get` call in order for the expanded complex output to be loaded into the output object.	

Unpublished work. © 2025 Siemens

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim out As femap.Output
   Set out = App.feOutput

   out.setID = 11
   out.SetComplexOptions (True, 15.0)
   out.Get(10000024)

   For i = 0 To 1000
   outval = out.Value(i)
   Next

End Sub
```

Clear (OBSOLETE)

Clear

( )

Description:

This method clears all data from the object and reinitializes it to a default state.

Input:

None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutOutputList (OBSOLETE)

PutOutputList (OBSOLETE) **(listcount, ID, value)**	
Description:	
This method stores data to an output vector.	
Input:	
`INT4 listcount`	The number of entries in both the ID and value arrays.
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is being created.
`REAL8 value[0..N]`	The output values associated with the corresponding node/element in the ID array.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	There are no entries in the list - you specified listcount = 0.
Remarks/Usage:	
This method stores information into the current object. Any previous output information in that object is lost. You cannot use this method to add to an existing output vector. This method does not store that	

Unpublished work. © 2025 Siemens

information to the model database. After using this function, you must call `Put( )` if you want to store the updated lists in the database.
Example:
None

InitScalarAtNode (OBSOLETE)

`InitScalarAtNode` **`( outputSET, vec, title, category, nonlinear )`**	
Description:	
This method initializes the current object to prepare it for creating an output vector containing a single scalar value at each node.	
Input:	
`INT4 outputSET`	The ID of the output set where this vector will be stored.
`INT4 vec`	The ID of the output vector which will be created.
`STRING title`	The Output vector title (maximum 79 characters).
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method simply sets up the various properties in a convenient manner for this style of output. You can override the parameters that are created automatically by directly manipulating the properties after the object has been initialized. Calling this method destroys any output that was previously loaded into the object.	

Unpublished work. © 2025 Siemens

Example:
None

GetScalarAtNode (OBSOLETE)

GetScalarAtNode	
(listcount, ID, value)	
Description:	
This method is identical to `GetOutputList`. It is simply an alias available for use with objects initialized with `InitScalarAtNode`.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetScalarAtNodeSet (OBSOLETE)

GetScalarAtNodeSet (OBSOLETE)
(idSET, value)
Description:

Unpublished work. © 2025 Siemens

This method is identical to `GetOutputListAtSet` - except that the default values are not specified, and are always just 0.0. It is simply an alias available for use with objects initialized with `InitScalarAtNode`.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutScalarAtNode (OBSOLETE)

`PutScalarAtNode` **`( listcount, ID, value )`**	
Description:	
This method is identical to `PutOutputList`. It is simply an alias available for use with objects initialized with `InitScalarAtNode`.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

InitVectorAtNode (OBSOLETE)

InitVectorAtNode **(outputSET, vec, vecX, vecY, vecZ, title, category, nonlinear)**	
Description:	
This method initializes the current object to prepare it for creating output containing vector data at each node. Actually four output values are created for each node, the magnitude, and the X, Y and Z components.	
Input:	
`INT4 outputSET`	The ID of the output set where this vector will be stored.
`INT4 vec`	The ID of the output vector which will be created to contain the magnitude of the output vectors.
`INT4 vecX`	The ID of the output vector which will be created to contain the X components of output.
`INT4 vecY`	The ID of the output vector which will be created to contain the Y components of output.
`INT4 vecZ`	The ID of the output vector which will be created to contain the Z components of output.
`STRING title`	The output vector title (maximum 79 characters).
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, the components of this result cannot be linearly combined. The magnitude can never be linearly combined.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
This method initializes the current object to contain the magnitude results, and creates three additional objects which are the X, Y and Z components of the vector results. All of the *component* properties are properly initialized to indicate that this is nodal vector data.	
Example:	
None	

GetVectorAtSingleNode (OBSOLETE)

`GetVectorAtSingleNode` **`( ID, x, y, z )`**	
Description:	
This method retrieves the components of the results stored in an object that contains nodal vector data.	
Input:	
`INT4 ID`	The ID of the node to retrieve output.
Output:	
`REAL8 x`	The X component of the results.
`REAL8 y`	The Y component of the results.
`REAL8 z`	The Z component of the results.
Return Code:	
None	None
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself. The object must contain nodal vector data for this method to work properly. Only the X,Y,Z components are returned. If you want to retrieve the magnitude data, you can do that by calling `GetScalarAtNode` - even though the object contains vector data.	

Unpublished work. © 2025 Siemens

All results are returned in the coordinate system in which they were stored. Results must be stored in global rectangular coordinates to be useful in other post-processing functions.
Example:
None

GetVectorAtNode (OBSOLETE)

`GetVectorAtNode` **`( listcount, ID, x, y, z )`**	
Description:	
This method retrieves the components of the results stored in an object that contains nodal vector data.	
Input:	
None	None
Output:	
`INT4 listcount`	The number of entries in the ID and x,y,z arrays.
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is available.
`REAL8 x[0..N]`	The X component of the values associated with the corresponding node/element in the ID array.
`REAL8 y[0..N]`	The Y component of the values associated with the corresponding node/element in the ID array.
`REAL8 z[0..N]`	The Z component of the values associated with the corresponding node/element in the ID array.
Return Code:	
None	None
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself. The object must contain nodal vector data for this method to work properly. Only the X,Y,Z components are returned. If you want to retrieve the magnitude data, you can do that by calling `GetScalarAtNode` - even though the object contains vector data.	

Unpublished work. © 2025 Siemens

All results are returned in the coordinate system in which they were stored. Results must be stored in global rectangular coordinates to be useful in other post-processing functions.
Example:
None

GetVectorAtNodeSet (OBSOLETE)

GetVectorAtNodeSet	
(idSET, x, y, z)	
Description:	
This method retrieves the components of the results stored in an object that contains nodal vector data for all IDs stored in the specifed Set.	
Input:	
`INT4 idSET`	The ID of a Set object that contains the IDs of the entities to retrieve.
Output:	
`REAL8 x[0..N]`	The X component of the values associated with the corresponding node/element in the Set.
`REAL8 y[0..N]`	The Y component of the values associated with the corresponding node/element in the Set.
`REAL8 z[0..N]`	The Z component of the values associated with the corresponding node/element in the Set.
Return Code:	
None	None
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself. The object must contain nodal vector data for this method to work properly. Only the X,Y,Z components are returned. If you want to retrieve the magnitude data, you can do that by calling `GetScalarAtNodeSet` - even though the object contains vector data. All results are returned in the coordinate system in which they were stored. Results must be stored in global rectangular coordinates to be useful in other post-processing functions.	

Unpublished work. © 2025 Siemens

Example:
None

PutVectorAtNode (OBSOLETE)

PutVectorAtNode **(listcount, ID, x, y, z)**	
Description:	
This method loads nodal vector data into an object. The object must first be initialized with `InitVectorAtNode`.	
Input:	
`INT4 listcount`	The number of entries in the ID and x,y,z arrays.
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is being created.
`REAL8 x[0..N]`	The X component of the values associated with the corresponding node/element in the ID array.
`REAL8 y[0..N]`	The Y component of the values associated with the corresponding node/element in the ID array.
`REAL8 z[0..N]`	The Z component of the values associated with the corresponding node/element in the ID array.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method stores information into the current object. Any previous output information in that object is lost. You cannot use this method to add to an existing output vector. This method does not store that information to the model database. After using this function, you must call `Put( )` if you want to store the updated lists in the database. You do not have to specify the vector magnitudes with this method - they are computed and created automatically from the components.	

Unpublished work. © 2025 Siemens

All results are stored in the coordinate system in which they were supplied. Results must be stored in global rectangular coordinates to be useful in other post-processing functions.
Example:
None

InitScalarAtElem (OBSOLETE)

`InitScalarAtElem` **`( outputSET, vec, title, category, nonlinear )`**	
Description:	
This method initializes the current object to prepare it for creating an output vector containing a single scalar value at each element.	
Input:	
`INT4 outputSET`	The ID of the output set where this vector will be stored.
`INT4 vec`	The ID of the output vector which will be created.
`STRING title`	The output vector title (maximum 79 characters).
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method simply sets up the various properties in a convenient manner for this style of output. You can override the parameters that are created automatically by directly manipulating the properties after the object has been initialized. Calling this method destroys any output that was previously loaded into the object.	

Unpublished work. © 2025 Siemens

Example:
None

GetScalarAtElem (OBSOLETE)

GetScalarAtElem	
(listcount, ID, value)	
Description:	
This method is identical to `GetOutputList`. It is simply an alias available for use with objects initialized with `InitScalarAtElem`.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetScalarAtElemSet (OBSOLETE)

GetScalarAtElemSet
(idSET, value)
Description:

Unpublished work. © 2025 Siemens

This method is identical to `GetOutputListAtSet` - except that the default values are not specified, and are always just 0.0. It is simply an alias available for use with objects initialized with `InitScalarAtElem`.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

PutScalarAtElem (OBSOLETE)

```
PutScalarAtElem

( listcount, ID, value )
```

Description:	
This method is identical to `PutOutputList`. It is simply an alias available for use with objects initialized with `InitScalarAtElem`.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

InitElemWithCorner (OBSOLETE)

InitElemWithCorner **(outputSET, vec, vec1, vec2, vec3, vec4, vec5, vec6, vec7, vec8, title, category, nonlinear)**	
Description:	
This method initializes the current object to prepare it for creating output containing a scalar value at the element centroid, and a scalar value at each element corner.	
Input:	
`INT4 outputSET`	The ID of the output set where this vector will be stored.
`INT4 vec`	The ID of the output vector which will be created to contain the centroidal results.
`INT4 vec1`	The ID of the output vector which will be created to contain the results at the corresponding element corners.
`INT4 vec2`	
`INT4 vec3`	
`INT4 vec4`	
`INT4 vec5`	
`INT4 vec6`	
`INT4 vec7`	
`INT4 vec8`	
`STRING title`	The output vector title (maximum 79 characters).

Unpublished work. © 2025 Siemens

`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, the components of this result cannot be linearly combined.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method initializes the current object to contain the centroid results, and creates additional objects which are the corner results. All of the *component* properties are properly initialized to indicate that this is elemental data with corner values.	
Example:	
None	

GetElemWithCorner (OBSOLETE)

`GetElemWithCorner`	
`( listcount, maxcorner, ID, centroid, c1, c2, c3, c4, c5, c6, c7, c8 )`	
Description:	
This method retrieves the centroid and corner results stored in an object that contains elemental data with corners.	
Input:	
None	None
Output:	
`INT4 listcount`	The number of entries in the ID and results arrays.
`INT4 maxcorner`	The ID of the maximum corner available for this object. For example, if this object contains results on quadrilateral plates, maxcorner will typically

Unpublished work. © 2025 Siemens

	equal 4, since results are only available for corners 1,2,3, and 4.
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is available.
`REAL8 centroid[0..N]`	The results at the element centroid associated with the corresponding node/element in the ID array.
`REAL8 c1[0..N]`	The results at the element corners associated with the corresponding node/element in the ID array. Some elements have missing corners. For example, Tetrahedral results are stored on corners 1,2,3 and 5. Wedge results are stored on 1,2,3 and 5,6,7.
`REAL8 c2[0..N]`	
`REAL8 c3[0..N]`	
`REAL8 c4[0..N]`	
`REAL8 c5[0..N]`	
`REAL8 c6[0..N]`	
`REAL8 c7[0..N]`	
`REAL8 c8[0..N]`	

Return Code:

None	None

Remarks/Usage:

This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself. The object must contain elemental data with corners for this method to work properly. If you only want to retrieve the centroidal data, you can do that by calling `GetScalarAtElem` - even though the object contains corner data.

For models where the output vectors may only contain partial corner data - for example mixed quad/tri models where there is no corner output for the triangles, or at least one missing corner - this method will return the centroidal value for all missing corner data. This only applies to corners where there are some results (below `maxcorner`). Corners that have no output for all elements are still not returned.

Example:

None

Unpublished work. © 2025 Siemens

GetElemWithCornerSet (OBSOLETE)

GetElemWithCornerSet **(idSET, maxcorner, centroid, c1, c2, c3, c4, c5, c6, c7, c8)**	
Description:	
This method retrieves the centroid and corner results stored in an object that contains elemental data with corners. Results are retrieved for all elements specified in the Set.	
Input:	
`INT4 idSET`	The ID of a Set object that contains the IDs of the entities to retrieve.
Output:	
`INT4 maxcorner`	The ID of the maximum corner available for this object. For example, if this object contains results on quadrilateral plates, maxcorner will typically equal 4, since results are only available for corners 1,2,3, and 4.
`REAL8 centroid[0..N]`	The results at the element centroid associated with the corresponding node/element in the Set.
`REAL8 c1[0..N]`	The results at the element corners associated with the corresponding node/element in the Set. Some elements have missing corners. For example, Tetrahedral results are stored on corners 1,2,3 and 5. Wedge results are stored on 1,2,3 and 5,6,7.
`REAL8 c2[0..N]`	
`REAL8 c3[0..N]`	
`REAL8 c4[0..N]`	
`REAL8 c5[0..N]`	
`REAL8 c6[0..N]`	
`REAL8 c7[0..N]`	
`REAL8 c8[0..N]`	
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
This method simply retrieves the information about the output from the current object. You must first load that data - either using the `Get( )` method, or by creating it yourself. The object must contain elemental data with corners for this method to work properly. If you only want to retrieve the centroidal data, you can do that by calling `GetScalarAtElemSet` - even though the object contains corner data. For models where the output vectors may only contain partial corner data - for example mixed quad/tri models where there is no corner output for the triangles, or at least one missing corner - this method will return the centroidal value for all missing corner data. This only applies to corners where there are some results (below `maxcorner`). Corners that have no output for all elements are still not returned.	
Example:	
None	

PutElemWithCorner (OBSOLETE)

`PutElemWithCorner` **`( listcount, maxcorner, ID, centroid, c1, c2, c3, c4, c5, c6, c7, c8 )`**	
Description:	
This method loads elemental centroidal and corner data into an object. The object must first be initialized with `InitElemWithCorner`.	
Input:	
`INT4 listcount`	The number of entries in the ID and results arrays.
`INT4 maxcorner`	The ID of the maximum corner available for this object. For example, if this object contains results on quadrilateral plates, maxcorner will typically equal 4, since results are only available for corners 1,2,3, and 4. If you are trying to store results with less than 8 corners, you must still specify all 8 corner values, but you can simply provide dummy values for all corners past maxcorner.
`INT4 ID[0..N]`	The IDs of the nodes/elements where output is available.

Unpublished work. © 2025 Siemens

`REAL8 centroid[0..N]`	The results at the element centroid associated with the corresponding node/element in the ID array.
`REAL8 c1[0..N]`	The results at the element corners associated with the corresponding node/element in the ID array. Some elements have missing corners. For example, Tetrahedral results are stored on corners 1,2,3 and 5. Wedge results are stored on 1,2,3 and 5,6,7.
`REAL8 c2[0..N]`	
`REAL8 c3[0..N]`	
`REAL8 c4[0..N]`	
`REAL8 c5[0..N]`	
`REAL8 c6[0..N]`	
`REAL8 c7[0..N]`	
`REAL8 c8[0..N]`	
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method stores information into the current object. Any previous output information in that object is lost. You cannot use this method to add to an existing output vector. This method does not store that information to the model database. After using this function, you must call `Put( )` if you want to store the updated lists in the database.	
Example:	
None	

InitScalarAtBeam (OBSOLETE)

Unpublished work. © 2025 Siemens

InitScalarAtBeam

(outputSET, vec1, vec2, title, vecstyle, category, nonlinear, bAddEndTitles)

Description:

This method initializes the current object to prepare it for creating Beam output vectors for EndA and EndB containing a single scalar value at each element.

Input:

`INT4 outputSET`	The ID of the output set where this vector will be stored.
`INT4 vec1`	The ID of the EndA output vector which will be created.
`INT4 vec2`	The ID of the EndB output vector which will be created.
`STRING title`	The output vector title (maximum 79 characters).
`INT4 vecstyle`	If 1, component[0..2] are the X,Y,Z component values. If 2, data at end of beams. If 3, reverse data at second end of beam
`INT4 category`	General category of output: 0=Any, 1=Disp, 2=Accel, 3=Force, 4=Stress, 5=Strain, 6=Temp, others=User.
`BOOL nonlinear`	If True, this result cannot be linearly combined.
`BOOL bAddEndTitles`	Flag to prepend *End A*, *End B* to the title.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

PutScalarAtBeam (OBSOLETE)

PutScalarAtBeam **(listcount, ID, v1, v2)**	
Description:	
This method stores data to an output vectors for beam elements.	
Input:	
`INT4 listcount`	The number of the elements where output is being created.
`INT4 ID[0..listcount-1]`	The IDs of the elements where output is being created.
`REAL8 v1[0..listcount-1]`	The EndA output values associated with the corresponding element in the ID array.
`REAL8 v2[0..listcount-1]`	The EndB output values associated with the corresponding element in the ID array.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

FindMaxMin (OBSOLETE)

Unpublished work. © 2025 Siemens

FindMaxMin (OBSOLETE)	
(idSET, bUseCornersOrComponents, minID, minValue, maxID, maxValue)	
Description:	
This method searches the data that has been loaded to find minimum and maximum values.	
Input:	
`INT4 idSET`	The ID of a Set object that contains the node or element IDs to search
`BOOL bUseCornersOrComponents`	If True, and the object has corner or component data loaded, then those corners will also be searched to find the max/min values. If False, then only the primary vector will be searched.
Output:	
`INT4 minID`	The ID of node/element where the minimum value occurs.
`REAL8 minValue`	The minimum output value.
`INT4 maxID`	The ID of the node/element where the maximum value occurs.
`REAL8 maxValue`	The maximum output value.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

OutputSet Objects

OutputSet objects correspond to the output/results sets in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for OutputSet objects. For more information, see **Common Entity Properties and Methods**.

Unpublished work. © 2025 Siemens

Use the `feOutputSet` method of the Femap Application object to create OutputSet objects.

OutputSet Object Properties

OutputSets are always stored with their `SetID=1`, and the ID equal to the output set or time step that you are trying to access. OutputSet objects do not contain any of the actual results data; they simply contain the header information for the set.

Property	Description
`STRING title`	Output set title (maximum 79 characters).
`INT4 program`	Analysis program where output came from: 0=Unknown, 1=Femap Generated, 2=PAL, 3=PAL 2, 4=MSC/NASTRAN, 5=ANSYS, 6=STARDYNE, 7=COSMOS, 8=PATRAN, 9=Femap Neutral, 10=ALGOR, 11=SSS/NASTRAN, 12=Comma Separated, 13=UAI/NASTRAN, 14=Cosmic NASTRAN, 15=STAAD, 16=ABAQUS, 17=WECAN, 18=MTAB/SAP, 19=CDA/Sprint, 20=CAEFEM, 21=I-DEAS, 22=ME/NASTRAN, 26=CSA/NASTRAN, 28=CFDesign, 31=LS-DYNA, 32=MARC, 33=SINDA, 38=NX Nastran.
`INT4 analysis`	Type of analysis: 0=Unknown, 1=Static, 2=Normal Modes/Eigenvalue, 3=Transient Dynamic/Time History, 4=Frequency/Harmonic Response, 5=Response Spectrum, 6=Random Response, 7=Buckling, 8=Design Opt, 9=Explicit Transient Dynamics, 10=Nonlinear Static, 11=Nonlinear Buckling, 12=Nonlinear Transient, 13=Implicit Transient Dynamics, 17=Fatigue, 19=Computational Fluid Dynamics, 20=Steady-State Heat Transfer, 21=Transient Heat Transfer, 22=Advanced Nonlinear Static, 23=Advanced Nonlinear Transient, 24=Advanced Nonlinear Explicit, 25=Static Aeroelasticity, 26=Aerodynamic Flutter
`REAL8 value`	The output set value. For modal analyses, the modal frequency. For any transient or nonlinear analysis, the time value at this step. For Buckling, the eigenvalue. For static analysis = 0.0.
`STRING notes`	One or more lines of text. Maximum 256 characters. Each line is separated by a `CR-LF`

Unpublished work. © 2025 Siemens

Property	Description
	`(0x0D, 0x0A)` character pair. The end of the string is terminated by a NULL character.
`INT4 study`	ID of analysis study where the output set currently resides, 0 indicates output set is not in an analysis study.
`INT4 nas_case`	Case ID for Nastran Output Sets
`INT4 nas_revision`	Revision ID for Nastran Output Sets
`INT4 combination_type`	Type of output processing operation used to create the output set: 0=None, 1=Linear Combination, 2=RSS Combination, 3=Max Envelope, 4=Min Envelope, 5=MaxAbs Envelope, 6=SetID Max Envelope, 7=SetID Min Envelope, 8=SetID MaxAbs Envelope

OutputSet Object Methods

VectorV2

`VectorV2` **`( vectorID )`**
Description:
This method replaces `Vector`. For more information see, **Working with Output in Femap v2020.1 and above**. This method creates and returns an Output object that is loaded with the data for a specific output vector.
Input:

`INT4 caseID`	The ID of the output vector to load.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
The current OutputSet must be loaded with the Set data from the set where you want to retrieve the vector prior to calling this method. The OutputSet ID property is used to define the set where the vector is retrieved.	
Example:	
None	

OutputVectorsV2

`OutputVectorsV2` **`( nVectorSetID )`**	
Description:	
This method replaces `OutputVectors`. For more information see, **Working with Output in Femap v2020.1 and above**. This method fills a Set object with the IDs of all Output vectors that are in the currently selected Output Set.	
Input:	
`INT4 nVectorSetID`	The ID of a Set Object that will receive the IDs of the Output vectors.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
You must set the *ID* property of the current object to match the Output Set where you want to retrieve the vectors - this happens automatically if you `Get` the Output Set object.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetComplexOptions

SetComplexOptions	
(bExpandComplex, dComplexPhase)	
Description:	
Used to control the output returned when using the `OutputVectors` method. This method sets a phase angle to use for any complex output (magnitude, phase, real or imaginary) which will cause the output to be returned in `OutputVectors` as the expanded complex magnitude at the specified phase angle.	
Input:	
`BOOL bExpandComplex`	When True, complex expansion will occur.
`REAL8 dComplexPhase`	Value of phase angle to use for complex expansion.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method MUST be called before the `Get` call in order for the expanded complex output to be loaded into the output set object.	
Example:	
None	

SetCombination

SetCombination
(enProcessType, nNumFactors, vnSetID, vdFactorArray)
Description:

Unpublished work. © 2025 Siemens

This method sets the output processing operation, output set IDs, and scale factors to create a new processed output set or for an existing output set currently in the OutputSet object.	
Input:	
`enum zResultsProcessType enProcessType`	Type of output processing operation, 1=Linear Combination, 2=RSS Combination, 3=Max Envelope, 4=Min Envelope, 5=MaxAbs Envelope, 6=SetID Max Envelope, 7=SetID Min Envelope, 8=SetID MaxAbs Envelope.
`INT4 nNumFactors`	Number of output set and scale factor pairs in the combination.
`INT4 vnSetID[0..numFactors-1]`	The IDs of the output sets used in the combination.
`REAL8 xyz[0..numFactors-1]`	The scale factors for each output set used in the combination.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
If using this method to create a new processed output set, call `ExpandCombination` immediately after this method to create a new output set (i.e. a `Put` is not needed). If using this method to modify an existing *Create All Immediately* processed output set, you will need to call `ExpandCombination` to update the values in the output set (i.e., a Put is not needed). If using this method to modify an existing *As Needed/Temporary* processed output set, you will not want to use `ExpandCombination`, but instead will need to use `Put` to update the processed output set. If you call `ExpandCombination`, it will change the processed output set from *As Needed/Temporary* to *Create All Immediately*.	
Example:	
None	

SetStudyCombination

Unpublished work. © 2025 Siemens

SetStudyCombination	
(enProcessType, nStudyID)	
Description:	
This method sets the output processing operation for an analysis study.	
Input:	
`enum zResultsProcessType enProcessType`	Type of output processing operation, 1=Linear Combination, 2=RSS Combination, 3=Max Envelope, 4=Min Envelope, 5=MaxAbs Envelope, 6=SetID Max Envelope, 7=SetID Min Envelope, 8=SetID MaxAbs Envelope.
`INT4 nStudyID`	ID of Analysis Study.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

ExpandCombination

ExpandCombination	
(bSetTitles)	
Description:	
This method can be used in conjunction with `SetCombination` to create a new processed output set in the next available output set ID. Also, it is used update the values in a *Create All Immediately* type of processed output set after using `SetCombination`.	

Unpublished work. © 2025 Siemens

Alternatively, it can be used convert an existing *As Needed/Temporary* type of processed output set to a *Create All Immediately* type of processed output set.	
Input:	
`BOOL bSetTitles`	When True, an descriptive title based on the process type and output sets used in the combination will be applied to the new processed output set. When False, a default title will be created. If modifying an existing output set, this option does nothing, as the title will not be changed.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method deletes all existing output in an existing *Create All Immediately* processed output set, then recreates all of the output. Also, after using this method, the output data for the *expanded* output set will be stored in the database.	
Example:	
None	

Plane Objects

Plane objects correspond to defined planes which can be stored in the database for future use in commands. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Plane objects. For more information, see **Common Entity Properties and Methods**.

Use the `fePlane` method of the Femap Application object to create Plane objects.

Plane Object Properties

Planes are always stored with a `SetID=1`, and the ID equal to the reference ID.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title`	Title of saved plane (Maximum 79 Characters).

`REAL8 base[0..2]`
or
`VARIANT vBase`
	Plane base location in global rectangular coordinates.

`REAL8 norm[0..2]`
or
`VARIANT vNorm`
	Components of the vector normal to the plane.

`REAL8 axis[0..2]`
or
`VARIANT vAxis`
	Components of a vector along the plane X axis.

Plane Object Methods

PutAll

`PutAll` **`( ID, sTitle, vBase, vNorm, vAxis  )`**
Description:
This method saves a plane to the database with all the available options using a single call.
Input:

Unpublished work. © 2025 Siemens

`INT4 ID`	ID of plane.
`STRING title`	Title of plane (Maximum 79 Characters).
`REAL8 vBase[0..2]`	Plane base location in global rectangular coordinates.
`REAL8 vNorm[0..2]`	Components of the vector normal to the plane.
`REAL8 vAxis[0..2]`	Components of a vector along the plane X axis.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to save plane.
Remarks/Usage:	
None	
Example:	
None	

Ply Material Objects

Ply Material objects correspond to the ply materials in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Property objects. For more information, see **Common Entity Properties and Methods**.

Use the `fePlyMaterial` method of the Femap Application object to create Ply Material objects.

Ply Material Object Properties

Ply Materials are always stored with a `SetID=1`, and the ID equal to the ply material ID.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title`	Ply Material title (maximum 79 characters).
`INT4 matlID`	ID of material referenced by Ply Material.
`REAL8 thickness`	Ply Material Thickness value.
`INT4 failuretheory`	Failure theory option: 0=From Property 1=Hill 2=Hoffman 3=Tsai-Wu 4=Max Strain

Point Objects

Point objects correspond to the points in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for point objects. For more information, see **Common Entity Properties and Methods**.

Use the `fePoint` method of the Femap Application object to create Point objects.

Point Object Properties

Points are always stored with a SetID=1, and the ID equal to the Point ID.

Property	Description
`REAL8 x`	These are the coordinates of the point. They are always specified in global rectangular coordinates.
`REAL8 y`	
`REAL8 z`	
`INT4 layer`	The ID of the layer associated with the point.
`INT4 color`	The point color.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 type`	The point type: 0=Default, 1=Solid.
`INT4 defCSys`	The ID of the definition coordinate system.
`INT4 attrPID`	The Property ID of the meshing attribute.
`INT4 engine`	The ID of the solid modeling engine used with this point: 0=None, 1=Parasolid, 2=ACIS.
`INT4 address`	The solid modeling engine geometry address.
`REAL8 meshsize`	MeshSize at the point. 0.0, unless a point-based mesh size has been defined.
`INT4 InCombinedCurve`	If the point is internal to a combined curve, this propety contains the ID of the combined curve, otherwise it is 0.
`REAL8 ScaleFactor`	The Solid Geometry Scale Factor.
Additional Properties	
`REAL8 xyz[0..2]`	Accesses all three coordinates of a point as a Variant. Same as using x, y, and z independently.

Point Object Methods

Nodes

`Nodes`	
`( numNodes, nodeID )`	
Description:	
This method returns the nodes that are attached to this point.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
`INT4 numNodes`	The number of nodes returned.
`INT4 nodeID[0..numNodes-1]`	The IDs of the selected nodes which are attached to the point.
Return Code:	
`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the point in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.
Remarks/Usage:	
None	
Example:	

The following codes modifies the color of all nodes on Point 4.

```
Sub Main
     Dim App As femap.model
     Set App = feFemap()

     Dim pt As femap.Point
     Set pt = App.fePoint

     Dim nd As femap.Node
     Set nd = App.feNode

     Dim rc As Long
     Dim Count As Long
     Dim Nodes As Variant
     Dim i As Long
```


Unpublished work. © 2025 Siemens

```
     rc = pt.Get (4)
     rc = pt.Nodes(Count, Nodes)

     For i = 0 To Count - 1
           nd.Get (Nodes(i))
           nd.color = 4
           nd.Put (0)
      Next i
     App.feViewRegenerate(0)
End Sub
```

NodesAsSet

NodesAsSet

(nSetID, bClear)

Description:	
This method returns the nodes that are attached to this point in a Set Object.	
Input:	
`INT4 nSetID`	The ID of the Set Object which receives the Node IDs.
`BOOL bClear`	If True, the Set is cleared before loading the Node IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return nodes, possibly none exist.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

Curves

Curves **(nCombinedMode, numCurves, curveID)**	
Description:	
This method returns the curves that are attached to this point.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic curves that are adjacent to the point, 1=Any curves that are adjacent that are in a Combined Curve are replaced by the ID of the Combined Curve, 2=List contains both the underlying and combined curves.
Output:	
`INT4 numCurves`	The number of curves in the returned array.
`INT4 curveID[0..numCurves-1]`	The IDs of the curves.
Return Code:	
`FE_FAIL`	Unable to return curves, possibly none exist.
Remarks/Usage:	
Only points that are on Solids/Sheet Solids can be used with this method.	
Example:	
None	

CurvesAsSet

Unpublished work. © 2025 Siemens

CurvesAsSet	
(nCombinedMode, nSetID, bClear)	
Description:	
This method returns the curves that are attached to this point in a Set Object.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic curves that are adjacent to the point, 1=Any curves that are adjacent that are in a Combined Curve are replaced by the ID of the Combined Curve, 2=List contains both the underlying and combined curves.
`INT4 nSetID`	ID of the Set Object to receive the Curve IDs.
`BOOL bClear`	If True, the Set is cleared before loading the curves.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return curves, possibly none exist.
Remarks/Usage:	
Similar to `Curves`, except that the attached curves are returned in a Set Object instead of an array. Only points that are on Solids/Sheet Solids can be used with this method.	
Example:	
None	

SurfacesAsSet

SurfacesAsSet
(nCombinedMode, nSetID, bClear)
Description:

Unpublished work. © 2025 Siemens

This method returns the surfaces that are attached to this point in a Set Object.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are adjacent to the curve, 1=Any surfaces that are adjacent that are in a Combined Surface are replaced by the ID of the Combined Surface, 2=List contains both the underlying and combined surfaces.
`INT4 nSetID`	ID of the Set Object to receive the Surface IDs.
`BOOL bClear`	If True, the Set is cleared before loading the surfaces.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return surfaces, possibly none exist.
Remarks/Usage:	
Only points that are on Solids/Sheet Solids can be used with this method.	
Example:	
None	

Solid

`Solid` **`( nSolidID )`**	
Description:	
This method returns the ID of the Solid that contains the current point.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`INT4 nSolidID`	The ID of the Solid that references the current Point.
Return Code:	
`FE_FAIL`	Unable to get the solid.
Remarks/Usage:	
This method will only work with a point that is defined on a Solid.	
Example:	
None	

GetClosest

`GetClosest`	
`( xyz )`	
Description:	
This method finds the Point closest to the desired location and loads it into the current object.	
Input:	
`REAL8 xyz[0..2]`	The coordinates of the location of interest.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	Unable to find closest Point.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetAllArray

GetAllArray **(SetID, numPoint, entID, Exist, xyz, layer, color, ptType, defCSys)**	
Description:	
This method retrieves arrays of values for a set of points, including if a point exists in the database.	
Input:	
`INT4 SetID`	ID of the set that contains the point IDs to retrieve. If 0, retrieve all points in the model.
Output:	
`INT4 numPoint`	The number of points for which data is being returned.
`INT4 entID[0..numPoint-1]`	Each entry is the ID of the associated point.
`BOOL Exist[0..numPoint-1]`	Point existence, True=Point exists, False=Point does not exist.
`REAL8 xyz[0..(3*numPoint)-1]`	The coordinates of the point. There are 3 entries, x, y, z for each point.
`INT4 layer[0..numPoint-1]`	The ID of the layer for the point.
`INT4 color[0..numPoint-1]`	The color of the point.
`INT4 ptType[0..numPoint-1]`	The type of the point.
`INT4 defCSys[0..numPoint-1]`	The definition coordinate system of the point.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected points.
Remarks/Usage:	
This method is the fastest way to retrieve information about a large number of points. If a retrieved point ID does not exist, then all of the values for that point ID will be 0.	

Unpublished work. © 2025 Siemens

Example:
None

PutAllArray

PutAllArray **(numPoint, entID, xyz, layer, color, ptType, defCSys)**	
Description:	
This method stores multiple point entities with the specified data.	
Input:	
`INT4 numPoint`	The number of points to create.
`INT4 entID[0..numPoint-1]`	Each entry is the ID of the point to create.
`REAL8 xyz[0..(3*numPoint)-1]`	The coordinates of the point. There are 3 entries, x, y, z for each point.
`INT4 layer[0..numPoint-1]`	The ID of the layer for the point.
`INT4 color[0..numPoint-1]`	The color of the point.
`INT4 ptType[0..numPoint-1]`	The type of the point. **Note** The `ptType` argument is required, however its values are not currently used. All points are created as the default type.
`INT4 defCSys[0..numPoint-1]`	The definition coordinate system of the point.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to store one or more points with the specified IDs.
Remarks/Usage:	
This is the fastest *Put* method if you have a large number of points to create. All data can be accumulated in arrays and transferred in one call.	
Example:	
None	

GetCoordArray

`GetCoordArray` **`( SetID, numPoint, entID, Exist, xyz )`**	
Description:	
This method retrieves arrays of coordinates for a set of Points.	
Input:	
`INT4 SetID`	ID of the set that contains the point IDs to retrieve. If 0, retrieve all points in the model.
Output:	
`INT4 numPoint`	The number of points for which data is being returned.
`INT4 entID[0..numPoint-1]`	Each entry is the ID of the associated point.
`BOOL Exist[0..numPoint-1]`	Point existence, True=Point exists, False=Point does not exist.
`REAL8 xyz[0..(3*numPoint)-1]`	The coordinates of the point. There are 3 entries, x, y, z for each point.
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected points.
Remarks/Usage:	
This is a simpler form of `GetAllArray`, if you simply need the point coordinates. If a retrieved point ID does not exist, then all of the values for that point ID will be 0.	

Unpublished work. © 2025 Siemens

Example:
None

PutCoordArray

`PutCoordArray`	
`( numPoint, entID, xyz )`	
Description:	
This is a simpler version of `PutAllArray` that allows multiple point entities to be stored with the specified data.	
Input:	
`INT4 numPoint`	The number of points to create.
`INT4 entID[0..numPoint-1]`	Each entry is the ID of a point.
`REAL8 xyz[0..(3*numPoint)-1]`	The coordinates of the point. There are 3 entries, x, y, z for each point.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to retrieve data from the selected points.
Remarks/Usage:	
This method is even faster than `PutAllArray`, and requires less memory, it does not however give you full control. The same layer, color... are used for every entity - only the IDs and coordinates are input as an array.	
Example:	
None	

Unpublished work. © 2025 Siemens

Property Objects

Property objects correspond to the properties in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Property objects. For more information, see **Common Entity Properties and Methods**.

Use the `fePropmethod` of the Femap Application object, or the Prop method of the Element object to create Property objects.

Property Object Properties

Properties are always stored with a `SetID=1`, and the ID equal to the property ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 color`	The property color.
`INT4 layer`	The layer associated with the property.
`INT4 type`	Type of property: Rod=1, Bar=2, Tube=3, Link=4, Beam=5, Spring/Damper=6, DOFSpring=7, CurvedBeam=8, Gap=9, PlotOnly=10, ShearPanel (Lin=11 Para=12), Membrane (Lin=13 Para=14), BendingOnly (Lin=15 Para=16), Plate (Lin=17 Para=18), PlaneStrain (Lin=19 Para=20), Laminate (Lin=21 Para=22), Axisymmetric (Lin=23 Para=24), Solid (Lin=25 Para=26), Mass=27, MassMatrix=28, Rigid=29, StiffnessMatrix=30, CurvedTube=31, PlotOnlyPlate=32, SlideLine=33, Contact=34, Axisymmetric Shell (Lin=35, Para=36), Para Beam=37, Weld/Fastener=38, Solid Laminate (Lin=39, Para=40), Spring/Damper to Ground=41, DOF Spring to Ground=42, Bearing = 46, Gear = 47, Fourier = 48
`INT4 matlID`	ID of material referenced by property. This is not used for certain property types, like masses

Unpublished work. © 2025 Siemens

Property	Description
	and laminates. These properties do not directly reference material data.
`INT4 layupID`	ID of the layup referenced by property. This is only used for laminates.
`STRING title`	Property title (maximum 79 characters).
`INT4 cbush`	Flag to indicate if Spring/Damper Property has Type set to CBUSH. 1=CBUSH, 0=Other (NASTRAN CROD/CVISC).
`INT4 refCS`	Reference coordinate system. Used to specify the coordinate system used on the Weld/Fastener property in the **Fastener Property Values** section.

`BOOL flag[0..7]`
 	
or
 	
`VARIANT vflag`
	Array of property flags.

`INT4 flagI[0..7]`
 	
or
 	
`VARIANT vflagI`
	Array of property flags accessed as an INT4. Most flags are simply true/false and can be accessed with `flag( )`.
	
	For bars, beams and curved beams, where shapes are available, `flagI(1)` defines the type of shape selected must be accessed as an INT4.
	
	Beam/Bar Sections: 1 = Rectangular Bar, 2 = Rectangular Tube, 3 = Trapezoidal Bar, 4 = Trapezoidal Tube, 5 = Circular Bar, 6 = Circular Tube, 7 = Hex Bar, 8 = Hex Tube, 9 = I, 10 = Channel, 11 = Angle, 12 = T, 13 = Z, 14 = Hat, 15 = General
	
	NASTRAN PBEAML/PBARL sections: 16 = ROD, 17 = TUBE, 18 = L, 19 = I, 20 = CHAN, 21 = T, 22 = BOX, 23 = BAR, 24 = CROSS, 25 = H, 26 = T1, 27 = I1, 28 = CHAN1, 29 = Z, 30 = CHAN2, 31 = T2, 32 = BOX1, 33 = HEXA, 34 = HAT, 35 = HAT1
	
	For bars, beams and curved beams, `flagI(2) (End A) & flagI(3) (End B)` define the Section Property Generation Method.

Unpublished work. © 2025 Siemens

Property	Description
	Beam/Bar Section Property Methods: 1 = Original FEMAP, 2 = Alternate FEMAP, 3 = NASTRAN PBARL/ PBEAML For beams, `flagI(4)` defines the setting for Write Zeros at End B(Off=Blank) option. For laminates... Failure Theory: 0 = None, 1 = Hill, 2 = Hoffman, 3 = Tsai-Wu, 4 = Max Strain NEi Only(10.3+):(5 = Max Stress, 6 = NASA LaRC (LARC02), 7 = Puck PCP (PUCK), 8 = Multicontinuum (MCT)) Options: 0 = As Specified, 1 = Symmetry, 2 = Membrane Only, 3 = Bending Only, 4 = Smear, 5 = Smear-core

`REAL8 pval[0..188]`
 	
or
 	
`VARIANT vpval`
	Array of property values.

`INT4 pival[0..109]`
 	
or
 	
`VARIANT vipval`
	Array of integer property values. Currently used for Gear element.

`INT4 ExtraMatlID[0..9]`
 	
or
 	
`VARIANT vExtraMatlID`
	Array of extra material IDs.
	
	For Plate elements...
	
	`ExtraMatlID[0]` = Bending Material Specification
	
	`ExtraMatlID[1]` = Transverse Shear Material Specification
	
	`ExtraMatlID[2]` = Membrane-Bending Coupling Specification

Unpublished work. © 2025 Siemens

Property	Description
	Link property... `ExtraMatlID[0]` = Conduction Calculated Material ID `ExtraMatlID[1]` = Convection Fluid Material ID `ExtraMatlID[2]` = Advection Fluid Material ID `ExtraMatlID[3]` = Radiation Coating Material ID *Advanced Thermal* Interface Style Only: Mass property... `ExtraMatlID[0]` = Mass Material Array of Property Extension material IDs. For Rod, Bar, Beam, Plane Strain, Axisymmetric, and Solid: `ExtraMatlID[4]` = Overrides - Material For Membrane, Bending Only, Plate, and Laminate: `ExtraMatlID[4]` = Material Overrides - Membrane `ExtraMatlID[5]` = Material Overrides - Bending
Additional Properties	
`INT4 extramat[0..9]`	Same as `plyval`, but returns/sets entire array as a VARIANT.
`INT4 pmat[0..74]`	Same as `pval`, but returns/sets entire array as a VARIANT.
`INT4 numOutline`	This is a read-only property that specifies the number of points available in the current outline. You can multiply this value by 3 to get the number of real numbers that are available in *outline*.
`INT4 outline[0..3N]`	Gets or sets the outline drawn for a general shape. The outline array consists of real numbers. For each point in the outline, there are three consecutive

Unpublished work. © 2025 Siemens

Property	Description
	entries in the array. The first two entries are the x,y coordinates (in the 2D coordinates of the outline) of the location on the outline. The third entry is a flag indicating the type of location being specified. Possible values include: 0=Off, 1=Normal, 2=Start of Loop, 3=End of Loop, 4=Stress Recovery, 5=No Draw (used by Femap to reduce complexity of beam elements displayed with cross-sections in the graphics window, cannot be a Stress Recovery). Points on each loop in the outline must be specified in consecutive order.

Accessing Property Properties

You can access the individual properties for the Property object by referencing the properties described above, and the data contained in the table below. For example, to retrieve the cross-sectional area for a bar property, use:

```
A = pr.pval(0)
```

or...

```
bend = pr.pval(10)
```

...for the bending stiffness parameter of a plate property.

This approach provides great flexibility, but can be slow if you are going to work with a large number of properties. In this case it is best to access the data using *Additional Properties*. These all set and return entire arrays of values in one reference. For example, you could write:

```
Dim pr as Object
Set pr = femap.feProp
Dim v  as Variant
pr.Get(1)
v = pr.pmat
v(0)=10.5
v(1)=14.5
pr.pmat = v
pr.Put(1)
```

This code loads Property 1, retrieves the entire array of real values into "v", updates the area and first moment of inertia (assuming the property was a bar), puts the entire array back, and saves the updated

Unpublished work. © 2025 Siemens

Property definition. While the code is slightly longer than simply accessing the individual properties, performance will be better using this approach.

Property Array Index Values

In general, most of the property data is simply stored in the various array properties. The location of the specific property data is defined in the following table.

For laminate properties, the ply information is stored on the layup referenced by the property.Property values are listed in the following tables:

- **Property Values for Rod, Bar, Tube, Link, Beam, and Spring Elements**
- **Property Values for DOF Spring, Curved Beam, Gap, Shear, Membrane, and Bending Elements**
- **Property Values for Plate, Plane Strain, Laminate, Mass, Mass Matrix, General Matrix Elements, Bearing Elements and Gear Elements**

Property Values for Rod, Bar, Tube, Link, Beam, and Spring Elements

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
Type	**1**	**2**	**3**	**4**	**5**	**6**
Flags						
0	cable		pipe		tapered	Bush orient. csys
1	compress. only gaps	shape type	closed ends		shape type	axial(1)/ torsion(0)
2		section method			section method_A	Bush Spring/ Damper Loc.
3					section method_B	Type Other(0) CBUSH(1) CBUSH1D(2)
4						CBUSH1D Shock

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
						Absorber Table (0) Equation(1)
5						CBUSH1D Spring Table (0) Equation(1)
6						CBUSH1D Damper Table (0) Equation(1)
7						
Values						
0	Area	Area	Dout	Ku_A	Area_A	Stiffness
1	Cable - Area I	I1	Din	Kv_A	I1_A	Damping
2		I2		Kw_A	I2_A	Bush Struc. Damping
3		I12		Kthu_A	I12_A	Bush Spring/ Damper Loc.
4	J	J		Kthv_A	J_A	Bush Trans. Stress Coeff
5	Ctors	K1,eff		Kthw_A	K1_A,eff	Bush Rot. Stress Coeff
6		K2,eff		Ku_B	K2_A,eff	Bush Trans. Strain Coeff

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
7	NSM	NSM	NSM	Kv_B	NSM_A	Bush Rot. Strain Coeff
8	Initial Tension	Yf_A1	Pipe Pressure	Kw_B	Yf_A1	Mass
9	Init. Slack	Zf_A1		Kthu_B	Zf_A1	
10	Allow. Ten. Stress	Yf_A2		Kthv_B	Yf_A2	Bush DOF1 Structural Damping
11		Zf_A2		Kthw_B	Zf_A2	Bush DOF2 Structural Damping
12		Yf_A3			Yf_A3	Bush DOF3 Structural Damping
13		Zf_A3			Zf_A3	Bush DOF4 Structural Damping
14		Yf_A4			Yf_A4	Bush DOF5 Structural Damping
15		Zf_A4		link type: 1 = Struct. 2 = Cond. 3 = Conv. 4 = Advect. 5 = Rad.	Zf_A4	Bush DOF6 Structural Damping
16					Yoff_A	CBUSH1D Shock Absorber CV Tension
17					Zoff_A	CBUSH1D Shock Absorber CV Compression

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
18					Warp_A	CBUSH1D Shock Absorber EXPV Tension
19	Perimeter	Perimeter			Perimeter_ A	CBUSH1D Shock Absorber EXPV Compressi on
20				cond. type 0 = Bulk 1 = Calc.	Area_B	CBUSH1D Shock Absorber S(u) Table ID
21				Bulk cond.	I1_B	CBUSH1D Spring F(u) Table ID
22				Calc. Area	I2_B	CBUSH1D Damper F(v) Table ID
23				Calc. Length	I12_B	Damp Force vs Freq
24				Calc. Cond	J_B	CBUSH1D Shock Absorber Tension - S(u) Equation ID
25				Cond. Direct.	K1_B,eff	CBUSH1D Shock Absorber Compressi on - S(u)

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
						Equation ID
26					K2_B,eff	CBUSH1D Shock Absorber Tension - S'(u) Equation ID
27					NSM_B	CBUSH1D Shock Absorber Compression - S'(u) Equation ID
28					Yf_B1	CBUSH1D Spring Tension - F(u) Equation ID
29					Zf_B1	CBUSH1D Spring Compression - F(u) Equation ID
30				Conv. Area	Yf_B2	CBUSH1D Spring Tension - F'(u) Equation ID
31				Conv. Coeff.	Zf_B2	CBUSH1D Spring Compression - F'(u) Equation ID

Unpublished work. © 2025 Siemens

Property Values						
	ROD	BAR	TUBE	LINK	BEAM	SPRING/ DAMPER
32					Yf_B3	CBUSH1D Damper Tension - F(v) Equation ID
33					Zf_B3	CBUSH1D Damper Compression - F(v) Equation ID
34					Yf_B4	CBUSH1D Damper Tension - F'(v) Equation ID
35				Adv. Mass Flow	Zf_B4	CBUSH1D Damper Compression - F'(v) Equation ID
36				Adv. Spec. Heat	Yoff_B	Bush DOF1 Stiffness
37				Adv. Direct.	Z_offB	Bush DOF2 Stiffness
38					Warp_B	Bush DOF3 Stiffness
39					Perimeter_ B	Bush DOF4 Stiffness
40		Std Shape H or Radius (DIM1)		Rad. View Factor	Std Shape H or Radius (DIM1)	Bush DOF5 Stiffness
41		Std Shape W1 (DIM2)		Rad. View Fac. Func.	Std Shape W1 (DIM2)	Bush DOF6 Stiffness

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
42		Std Shape W2 (DIM3)		Rad. Area	Std Shape W2 (DIM3)	Bush DOF1 Damping
43		Std Shape T1 (DIM4)		Rad. Emiss.	Std Shape T1 (DIM4)	Bush DOF2 Damping
44		Std Shape T2 (DIM5)			Std Shape T2 (DIM5)	Bush DOF3 Damping
45		Std Shape T (DIM6)			Std Shape T (DIM6)	Bush DOF4 Damping
46		Std Shape SR1			Std Shape SR1	Bush DOF5 Damping
47		Std Shape SR2			Std Shape SR2	Bush DOF6 Damping
48		Std Shape SR3			Std Shape SR3	Bush DOF1 K vs. Frequency Function
49		Std Shape SR4			Std Shape SR4	Bush DOF2 K vs. Frequency Function
50		Std Shape Ori-ent			Std Shape Orient	Bush DOF3 K vs. Frequency Function
51		Std Shape RefLoc			Std Shape RefLoc	Bush DOF4 K vs. Frequency Function
52		Shape RefY			Shape RefY	Bush DOF5 K vs. Frequency Function
53		Shape RefZ			Shape RefZ	Bush DOF6 K vs. Frequency Function

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
54						Bush DOF1 F/V vs. Freq. Function
55					End B_Std Shape H or Radius (DIM1)	Bush DOF2 F/V vs. Freq. Function
56					B_Std Shape W1 (DIM2)	Bush DOF3 F/V vs. Freq. Function
57					B_Std Shape W2 (DIM3)	Bush DOF4 F/V vs. Freq. Function
58					B_Std Shape T (DIM4)	Bush DOF5 F/V vs. Freq. Function
59					B_Std Shape T2 (DIM5)	Bush DOF6 F/V vs. Freq. Function
60					B_Std Shape T (DIM6)	Bush DOF1 F vs. Disp. Function
61					B_Std Shape SR1	Bush DOF2 F vs. Disp. Function
62					B_Std Shape SR2	Bush DOF3 F vs. Disp. Function
63					B_Std Shape SR3	Bush DOF4 F vs. Disp. Function

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
64					B_Std Shape SR4	Bush DOF5 F vs. Disp. Function
65					B_Std Shape Ori-ent	Bush DOF6 F vs. Disp. Function
66					B_Std Shape RefLoc	Bush DOF1 Struc. Damping vs. Frequency
67					B_Shape RefY	Bush DOF2 Struc. Damping vs. Frequency
68					B_Shape RefZ	Bush DOF3 Struc. Damping vs. Frequency
69	(PE)Large Strain Flag	(PE)Large Strain Flag			(PE)Large Strain Flag	Bush DOF4 Struc. Damping vs. Frequency
70		(PE)Integration Type			(PE)Integrat ion Type	Bush DOF5 Struc. Damping vs. Frequency
71		(PE)Section			(PE)Section	Bush DOF6 Struc. Damping vs. Frequency
72						CBUSH1D Generalize d Tension -

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
						F(u,v) Equation ID
73						CBUSH1D Generalized Compression - F(u,v) Equation ID
74						CBUSH1D Generalized Tension - F'(u) Equation ID
75						CBUSH1D Generalized Compression - F'(u) Equation ID
76						CBUSH1D Generalized Tension - F'(v) Equation ID
77						CBUSH1D Generalized Compression - F'(v) Equation ID

PE. Signifies these options/values are found in the Property Extensions dialog box.

For Spring/Damper to Ground (41):

Unpublished work. © 2025 Siemens

Same as Spring/Damper, only Type must be set to 1 = CBUSH or 2 = CBUSH1D and Boolean to specify Axial or Tension should not be used, as it is unavailable in the User Interface.

Property Values for DOF Spring, Curved Beam, Gap, Shear, Membrane, and Bending Elements

Property Values (continued)						
	DOF SPRING	**CURVED BEAM**	**GAP**	**SHEAR**	**MEMBRANE**	**BENDING**
Type	**7**	**8**	**9**	**11/12**	**13/14**	**15/16**
Flags						
0			Has CSys			Top Fiber
1		shape type	ABAQUS GAP CONDUCTANCE Active			Bot Fiber
2	DOF_A	section method	ABAQUS GAP CONDUCTANCE is Pressure based			
3	DOF_B		ABAQUS GAP RADIATION Active			
Values						
0	Stiffness	Area	Gap,initial	T	T	T
1	Damping	I1	Stiff,comp.			
2		I2	Stiff,tens.			
3		I12	Stiff,trans.			
4		J	Mu,y			
5		K1,eff	Mu,z			
6		K2,eff	PreloadT			
7		NSM	Plane X	NSM	NSM	NSM
8		Yf_A1	Plane Y	F1,eff.fact.		Top Fiber
9		Zf_A1	Plane Z	F2,eff.fact.		Bot Fiber
10		Yf_A2	Width/Area	(9.1+) F3,eff.fact.		12I/T3

Unpublished work. © 2025 Siemens

Property Values (continued)						
	DOF SPRING	**CURVED BEAM**	**GAP**	**SHEAR**	**MEMBRANE**	**BENDING**
11		Zf_A2	MaxPenRat (Set value to -1 to uncheck Adaptive box)	(9.1+) F4,eff.fact.		
12		Yf_A3	MaxAdjRat			
13		Zf_A3	MinPenRat			
14		Yf_A4	ABAQUS GAP CONDUCTANCE Function defining Conductance, Gap Clearance, Temperature relationship.			
15		Zf_A4	ABAQUS GAP RADIATION Emissivity A			
16			ABAQUS GAP RADIATION Emissivity B			
17		R, bend rad	ABAQUS GAP RADIATION Function defining Effective View Factor, Gap Clearance relationship.			
18..39	20=Force vs. Freq.function 21=Damping vs. Frequency function22 = Force vs. Disp. function					
40		Std Shape H or R (DIM1)				

Unpublished work. © 2025 Siemens

Property Values (continued)						
	DOF SPRING	CURVED BEAM	GAP	SHEAR	MEMBRANE	BENDING
41		Std Shape W1 (DIM2)				
42		Std Shape W2 (DIM3)				
43		Std Shape T1 (DIM4)				
44		Std Shape T2 (DIM5)				
45		Std Shape T (DIM6)				
46		Std Shape SR1				
47		Std Shape SR2				
48		Std Shape SR3				
49		Std Shape SR4				
50		Std Shape Orient				
51		Std Shape RefLoc				
52		Shape RefY				
53		Shape RefZ				
54..58						
59					(PE)Tria3 BehaviorType	(PE)Tria3 Behavior Type
60					(PE)Tria6 Behavior Type	(PE)Tria6 Behavior Type
61					(PE)Quad4 Behavior Type	(PE)Quad4 Behavior Type
62					(PE)Quad8 Behavior Type	(PE)Quad8 Behavior Type

Unpublished work. © 2025 Siemens

Property Values (continued)						
	DOF SPRING	**CURVED BEAM**	**GAP**	**SHEAR**	**MEMBRANE**	**BENDING**
63..68						
69					(PE)Large Strain Flag	(PE)Large Strain Flag
70					(PE)Tria3 Int. Type	(PE)Tria3 Int. Type
71					(PE)Tria6 Int. Type	(PE)Tria6 Int. Type
72					(PE)Quad4 Int. Type	(PE)Quad4 Int. Type
73					(PE)Quad8 Int. Type	(PE)Quad8 Int. Type
74						
75						
76						
77						

PE. Signifies these options/values are found in the Property Extensions dialog box.

For DOF Spring to Ground (42):

Same as DOF Spring only DOF_B should not be set as it is unavailable in the User Interface.

Property Values for Plate, Plane Strain, Laminate, Mass, Mass Matrix, General Matrix Elements, Bearing Elements and Gear Elements

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
Type	**17/18**	**19/20**	**21/22**	**27**	**28**	**30**
Flags						
0	Top Fiber	Top Fiber	failure theory			Matrix Type (0 = Stiff, 1 =

Unpublished work. © 2025 Siemens

Property Values						
	ROD	**BAR**	**TUBE**	**LINK**	**BEAM**	**SPRING/ DAMPER**
						Damp, 2 = Mass)
1	Bot Fiber	Bot Fiber	options			
2			offset bottom surface			
Values						
0	Tavg,T1	T	Bottom Fiber		M11	K1,1
1	T2		NSM	Ixx	M12	K1,2
2	T3		Bond Shear	Ixy	M13	K1,3
3	T4		Ref Temp	Iyy	M14	K1,4
4			Damping	Izx	M15	K1,5
5				Iyz	M16	K1,6
6				Izz		K11,12
7	NSM	NSM		(2)Mx	M22	K2,2
8	Top Fiber	Top Fiber		Xoff,refCS	M23	K2,3
9	Bot Fiber	Bot Fiber		Yoff,refCS	M24	K2,4
10	12I/T3			Zoff,refCS	M25	K2,5
11	Ts/T			My	M26	K2,6
12	(PE)Comp. Direction			Mz		K10,11
13	(PE)Copress Allowable					K10,12
14	(PE)X Fact.				M33	K3,3
15	(PE)Y Fact.				M34	K3,4
16	(PE)Type				M35	K3,5
17	(PE)F1, eff.				M36	K3,6
18	(PE)F2, eff.					K9,10
19	(PE)F3, eff.			Effective Diameter		K9,11

Unpublished work. © 2025 Siemens

Property Values						
	ROD	BAR	TUBE	LINK	BEAM	SPRING/ DAMPER
20	(PE)F4, eff.			(1)Mass Functon		K9,12
21					M44	K4,4
22					M45	K4,5
23					M46	K4,6
24						K8,9
25						K8,10
26						K8,11
27						K8,12
28					M55	K5,5
29					M56	K5,6
30						K7,8
31						K7,9
32						K7,10
33						K7,11
34						K7,12
35					M66	K6,6
36						K7,7
37						K8,8
38						K9,9
39						K10,10
40						K11,11
41						K12,12
42						K1,7
43						K1,8
44						K1,9
45						K1,10

Unpublished work. © 2025 Siemens

Property Values						
	ROD	BAR	TUBE	LINK	BEAM	SPRING/ DAMPER
46						K1,11
47						K1,12
48						K2,7
49						K2,8
50						K2,9
51						K2,10
52						K2,11
53						K2,12
54						K3,7
55						K3,8
56						K3,9
57						K3,10
58						K3,11
59	(PE)Tria3 Behavior Type	(PE)Tria3 Behavior Type	(PE)Tria3 Behavior Type			K3,12
60	(PE)Tria6 Behavior Type	(PE)Tria6 Behavior Type	(PE)Tria6 Behavior Type			K4,7
61	(PE)Quad4 Behavior Type	(PE)Quad4 Behavior Type	(PE)Quad4 Behavior Type			K4,8
62	(PE)Quad8 Behavior Type	(PE)Quad8 Behavior Type	(PE)Quad8 Behavior Type			K4,9
63						K4,10
64						K4,11
65						K4,12
66						K5,7
67						K5,8
68						K5,9
69	(PE)Large Strain Flag	(PE)Large Strain Flag	(PE)Large Strain Flag			K5,10

Unpublished work. © 2025 Siemens

Property Values						
	ROD	BAR	TUBE	LINK	BEAM	SPRING/ DAMPER
70	(PE)Tria3 Int. Type	(PE)Tria3 Int. Type	(PE)Tria3 Int. Type			K5,11
71	(PE)Tria6 Int. Type	(PE)Tria6 Int. Type	(PE)Tria6 Int. Type			K5,12
72	(PE)Quad4 Int. Type	(PE)Quad4 Int. Type	(PE)Quad4 Int. Type			K6,7
73	(PE)Quad8 Int. Type	(PE)Quad8 Int. Type	(PE)Quad8 Int. Type			K6,8
74						K6,9
75		(PE)Layer				K6,10
76		(PE)Thickness				K6,11
77						K6,12

1. Signifies value only available in GUI when using the Advanced Thermal Interface Style.

2. To create a mass with the same values in all directions, Mx, My, and Mz must ALL be set, not only Mx, as can be done in GUI.

PE. Signifies these options/values are found in the Property Extensions dialog box.

For Laminate Properties, the follwing values specify the Failure Theory options:

flagI(0): 0 = None, 1 = Hill, 2 = Hoffman, 3 = Tsai-Wu, 4 = Max Stran, 5 = NEi Nastran/STRESS, 6 = NEi Nastran/LARC02, 7 = NEi Nastran/PUCK, and 8 = NEi Nastran/MCT

Property Values for Plate, Plane Strain, Laminate, Mass, Mass Matrix, General Matrix Elements, Bearing Elements and Gear Elements

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	**SLIDE LINE**	**AXISYM SHELL**	**WELD/ FASTENER**	**SOLID LAMINATE**	**BEARING**	**GEAR**
Type	**31**	**33**	**35/36**	**38**	**39/40**	**46**	**47**
0		Penetration (Unsym = 1, Sym = 0)		(9.1+)M-Set DOF (CWELD)		Type; 0=BEAR	
1				(9.1+)Spot Weld (CWELD)			
2				(10+)Absolute (CFAST)			
3				(10+)Use Material CSys (CFAST)			
Values							
0	Dout	Stiffness Scale	(8.2+)T	(10+)Diameter for both CWELD and CFAST		K11	Module of Gear
1	Din	Nonsliding Friction		(10+)Mass		K12	Teeth Gear A
2				(10+)Struct Damping	(11+)Bond Shear Stress Allowable	K13	Teeth Gear B
3				(10+)KTX	(11+)Reference Temperature	K14	Pressure Angle
4		Static Friction		(10+)KTY	(11+)Damping Coefficient	K15	Stiffness Coeff.
5				(10+)KTZ	(11+)Bond Normal Stress Allowable	K16	Pitch Angle Gear A
6				(10+)KRX	(11+)Ply/ Stack Direction.	K21	Pitch Angle Gear B

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
					0=Default, 1=XY(12), 2=XZ(13), 3=YX(21), 4=YZ(23), 5=ZX(31), 6=ZY(32)		
7	NSM			(10+)KRY		K22	Helix Angle Gear A
8				(10+)KRZ		K23	Helix Angle Gear B
9				(10+)CWELD/ CFAST toggle (0 = CWELD, 1 = CFAST)		K24	
10		Master Width				K25	
11		Slave Width				K26	
12						K31	
13						K32	
14						K33	
15						K34	
16						K35	
17	R, bend rad					K36	
18						K41	
19						K42	
20						K43	
21						K44	
22						K45	
23						K46	
24						K51	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
25						K52	
26						K53	
27						K54	
28						K55	
29						K56	
30						K61	
31						K62	
32						K63	
33						K64	
34						K65	
35						K66	
36						C11	
37						C12	
38						C13	
39						C14	
40						C15	
41						C16	
42						C21	
43						C22	
44						C23	
45						C24	
46						C25	
47						C26	
48						C31	
49						C32	
50						C33	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
51						C34	
52						C35	
53						C36	
54						C41	
55						C42	
56						C43	
57						C44	
58						C45	
59						C46	
60						C51	
61						C52	
62						C53	
63						C54	
64						C55	
65						C56	
66						C61	
67						C62	
68						C63	
69					(PE)Large Strain Flag	C64	
70					(PE)Brick8 Integration Type	C65	
71					(PE)Brick20 Integration Type	C66	
72						M11	
73						M12	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
74						M13	
75						M14	
76						M15	
77						M16	
78						M21	
79						M22	
80						M23	
81						M24	
82						M25	
83						M26	
84						M31	
85						M32	
86						M33	
87						M34	
88						M35	
89						M36	
90						M41	
91						M42	
92						M43	
93						M44	
94						M45	
95						M46	
96						M51	
97						M52	
98						M53	
99						M54	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
100						M55	
101						M56	
102						M61	
103						M62	
104						M63	
105						M64	
106						M65	
107						M66	
108						HYST	
109						AVIS	
iValues							
0						FK11	
1						FK12	
2						FK13	
3						FK14	
4						FK15	
5						FK16	
6						FK21	
7						FK22	
8						FK23	
9						FK24	
10						FK25	
11						FK26	
12						FK31	
13						FK32	
14						FK33	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
15						FK34	
16						FK35	
17						FK36	
18						FK41	
19						FK42	
20						FK43	
21						FK44	
22						FK45	
23						FK46	
24						FK51	
25						FK52	
26						FK53	
27						FK54	
28						FK55	
29						FK56	
30						FK61	
31						FK62	
32						FK63	
33						FK64	
34						FK65	
35						FK66	
36						CK11	
37						CK12	
38						CK13	
39						CK14	
40						CK15	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
41						CK16	
42						CK21	
43						CK22	
44						CK23	
45						CK24	
46						CK25	
47						CK26	
48						CK31	
49						CK32	
50						CK33	
51						CK34	
52						CK35	
53						CK36	
54						CK41	
55						CK42	
56						CK43	
57						CK44	
58						CK45	
59						CK46	
60						CK51	
61						CK52	
62						CK53	
63						CK54	
64						CK55	
65						CK56	
66						CK61	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
67						CK62	
68						CK63	
69						CK64	
70						CK65	
71						CK66	
72						FM11	
73						FM12	
74						FM13	
75						FM14	
76						FM15	
77						FM16	
78						FM21	
79						FM22	
80						FM23	
81						FM24	
82						FM25	
83						FM26	
84						FM31	
85						FM32	
86						FM33	
87						FM34	
88						FM35	
89						FM36	
90						FM41	
91						FM42	
92						FM43	

Unpublished work. © 2025 Siemens

Property Values (continued)							
	CURVED TUBE	SLIDE LINE	AXISYM SHELL	WELD/ FASTENER	SOLID LAMINATE	BEARING	GEAR
93						FM44	
94						FM45	
95						FM46	
96						FM51	
97						FM52	
98						FM53	
99						FM54	
100						FM55	
101						FM56	
102						FM61	
103						FM62	
104						FM63	
105						FM64	
106						FM65	
107						FM66	
108						HYST	
109						AVIS	

When two types are listed, the first number corresponds to a linear type, the second to a parabolic type. The following types are also available: Plot Only (10), Plot Plate (32), Axisymmetric (23/24), Solid (25/26), and Rigid (29) and Contact (34 - Obsolete and should not be used).

For Solid Laminate Properties (39/40):

layupID property: Laminate Definition - Layup

For Solid Properties (25/26):

flag(0): False = Align to CSys, True = Align to Element

Integration Network (must be a value from 0 to 3) = pval(0)

Unpublished work. © 2025 Siemens

Property Extensions for Solid Properties:

Large Strain Flag = pval(69)

Tetra4 Elements: Integration Type = pval(70); Behavior Type = pval(59)

Tetra10 Elements: Integration Type = pval(71); Behavior Type = pval(60)

Pyram5 Elements: Integration Type = pval(72); Behavior Type = pval(61)

Pyram13 Elements: Integration Type = pval(73); Behavior Type = pval(62)

Wedge6 Elements: Integration Type = pval(74); Behavior Type = pval(63)

Wedge15 Elements: Integration Type = pval(75); Behavior Type = pval(64)

Brick8 Elements: Integration Type = pval(76); Behavior Type = pval(65)

Brick20 Elements: Integration Type = pval(68); Behavior Type = pval(66)

Material Override = pval(77)

For Solid Cohesive Properties (43/44):

flag(0): False = Align to CSys, True = Align to Element

Thickness = pval(0)

Property Extensions for Solid Cohesive Properties:

Integration Type = pval(70)

Output Type = pval(76)

Matrix Scheme = pval(77)

Property Object Methods

The following methods are available in addition to the standard entity methods.

ComputeShape

Unpublished work. © 2025 Siemens

ComputeShape

(shear_center_offset, Warping, stress_recovery)

Description:

Note

This method is obsolete - for new applications you should use either `ComputeStdShape()` or `ComputeGeneralShape()`. If values for End B of a tapered beam must be calculated, then `ComputeStdShape2()` or `ComputeGeneralShape2()` must be used.

This method computes the properties for a standard shape.

Input:

`BOOL shear_center_offset`	If True, the shear center offset is computed.
`BOOL Warping`	If True, the warping constants are computed.
`BOOL stress_recovery`	If True, the stress recovery locations and cross section orientation that are currently defined in the object are used for the computation. If False, default stress recovery locations and orientation are used.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

To use this method, the property must be a type that uses a shape - Bar, Beam... Prior to using this method, you must load the appropriate properties (as shown in the table above) to define the type of shape and its dimensions. This method will not work with general shapes.

For bars, beams and curved beams, where shapes are available, `flagI(1)` defines the type of shape selected. Available values are:

1 = Rectangular Bar, 2 = Rectangular Tube, 3 = Trapezoidal Bar, 4 = Trapezoidal Tube, 5 = Circular Bar, 6 = Circular Tube, 7 = Hex Bar, 8 = Hex Tube, 9 = I, 10 = Channel, 11 = Angle, 12 = T, 13 = Z, 14 = Hat, 15 = General

Unpublished work. © 2025 Siemens

Example:
None

ComputeStdShape

ComputeStdShape **(shapeID, dimensions, sOrient, EvalMethod, shear_center_offset, Warping, stress_recovery)**	
Description:	
This method computes the properties for a standard shape.	
Input:	
`INT4 shapeID`	ID of one of the Standard Shapes (1 = Rectangular Bar, 2 = Rectangular Tube, 3 = Trapezoidal Bar, 4 = Trapezoidal Tube, 5 = Circular Bar, 6 = Circular Tube, 7 = Hex Bar, 8 = Hex Tube, 9 = I, 10 = Channel, 11 = Angle, 12 = T, 13 = Z, 14 = Hat) or... Nastran Shapes (16=Rod, 17=Tube, 18=L, 19=I, 20 = Channel, 21=T, 22=Box, 23=Bar, 24=Cross, 25=H, 26=T1, 27=I1, 28=Chan1, 29=Z, 30=Chan2, 31=T2, 32=Box1, 33=Hexa, 34=Hat, 35=Hat1).
`REAL8 dimensions[0..5]`	The dimensions that define the shape. Values are in the same order that they are shown in the shape dialog box.
`INT4 sOrient`	The orientation direction (0=Right, 1=Up, 2=Left, 3=Down). Ignored for the Nastran shapes - for these orientation is always *Right*.
`INT4 EvalMethod`	The method used to evaluate the section properties (0=Auto, 1=Original, 2=Alternate, 3=Nastran PBEAML).
`BOOL shear_center_offset`	If True, the shear center offset is computed.
`BOOL Warping`	If True, the warping constants are computed.

Unpublished work. © 2025 Siemens

`BOOL stress_recovery`	If True, the stress recovery locations and cross section orientation that are currently defined in the object are used for the computation. If False, default stress recovery locations and orientation are used.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Property type not set to a type that uses a shape.
Remarks/Usage:	

- The General shape is not available with this method. Use `ComputeGeneralShape()` instead.
- The Nastran PBEAML evaluator assumes thin-walled theory. If your shape is not *thin* it will produce an inaccurate representation of the shape.
- To use this method, the object must first be set to a type that uses shapes - for example a Bar or Beam. (for example `p.type = FET_L_BEAM`).
- If you use the *Alternate* eval_method, you should first set the Poisson's Ratio to be used in the computation (for example `p.pval(70) = 0.3`)
- Only End A properties can be computed with this method. Tapered beams are not supported.

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim p As femap.Prop
   Set p = App.feProp

   Dim d(5) As Double

   d(0) = 2.0
   d(1) = 1.0
   d(2) = 0.2
   d(3) = 0.1

   p.type = FET_L_BEAM
   p.ComputeStdShape( FSHP_NAS_L, d, FSOR_RIGHT,

    FSEV_PBEAML, True, True, True )
   p.Put(1)
End Sub
```


Unpublished work. © 2025 Siemens

ComputeStdShape2

ComputeStdShape2

(bAtEndAOrOverall, shapeID, dimensions, sOrient, EvalMethod, shear_center_offset, Warping, stress_recovery)

Description:

This method computes the properties for a standard shape at either End A or End B of a beam.

Input:

BOOL bAtEndAOrOverall	If True, only computes End A values. If False, only computes End B values and sets the Tapered flag.
INT4 shapeID	ID of one of the Standard Shapes (1 = Rectangular Bar, 2 = Rectangular Tube, 3 = Trapezoidal Bar, 4 = Trapezoidal Tube, 5 = Circular Bar, 6 = Circular Tube, 7 = Hex Bar, 8 = Hex Tube, 9 = I, 10 = Channel, 11 = Angle, 12 = T, 13 = Z, 14 = Hat) or... Nastran Shapes (16=Rod, 17=Tube, 18=L, 19=I, 20 = Channel, 21=T, 22=Box, 23=Bar, 24=Cross, 25=H, 26=T1, 27=I1, 28=Chan1, 29=Z, 30=Chan2, 31=T2, 32=Box1, 33=Hexa, 34=Hat, 35=Hat1).
REAL8 dimensions[0..5]	The dimensions that define the shape. Values are in the same order that they are shown in the shape dialog box.
INT4 sOrient	The orientation direction (0=Right, 1=Up, 2=Left, 3=Down). Ignored for the Nastran shapes - for these orientation is always *Right*.
INT4 EvalMethod	The method used to evaluate the section properties (0=Auto, 1=Original, 2=Alternate, 3=Nastran PBEAML).
BOOL shear_center_offset	If True, the shear center offset is computed.
BOOL Warping	If True, the warping constants are computed.
BOOL stress_recovery	If True, the stress recovery locations and cross section orientation that are currently defined in the object are used for the computation. If False,

Unpublished work. © 2025 Siemens

	default stress recovery locations and orientation are used.
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Property type is not `L_BEAM(5)` or `P_BEAM(37)`.
Remarks/Usage:	

- The General shape is not available with this method. Use `ComputeGeneralShape()` or `ComputeGeneralShape2()`, if End B values are needed, instead.
- The Nastran PBEAML evaluator assumes thin-walled theory. If your shape is not *thin* it will produce an inaccurate representation of the shape.
- To use this method, the object must first be set to a type `L_BEAM(5)` or `P_BEAM(37)`.
- If you use the *Alternate* EvalMethod, you should first set the Poisson's Ratio to be used in the computation (for example `p.pval(70) = 0.3`).

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()


   Dim p As femap.Prop
   Set p = App.feProp

   Dim d(5) As Double

   d(0) = 1.8
   d(1) = 0.9
   d(2) = 0.18
   d(3) = 0.09

   p.type = FET_L_BEAM
   p.ComputeStdShape2( False, FSHP_NAS_L, d, FSOR_RIGHT,

    FSEV_PBEAML, True, True, True )
   p.Put(1)
End Sub
```

ComputeGeneralShape

Unpublished work. © 2025 Siemens

ComputeGeneralShape

(surfaceID, vecDIR, sOrient, EvalMethod, shear_center_offset, Warping, stress_recovery)

Description:

This method computes the properties for a general shape.

Input:

`INT4 surfaceID`	ID of a surface to be used to compute the general shape.
`REAL8 vecDIR[0..2]`	A vector that defines the original surface Y axis.
`INT4 shape_orient`	The orientation direction (0=Right, 1=Up, 2=Left, 3=Down). Ignored for the Nastran shapes - for these orientation is always *Up*.
`INT4 eval_method`	The method used to evaluate the section properties (0=Auto, 1=Original, 2=Alternate).
`BOOL shear_center_offset`	If True, the shear center offset is computed.
`BOOL warping`	If True, the warping constants are computed.
`BOOL stress_recovery`	If True, the stress recovery locations and cross section orientation that are currently defined in the object are used for the computation. If False, default stress recovery locations and orientation are used.

Output:

None	None

Return Code:

`FE_BAD_TYPE`	Property type not set to a type that uses a shape.

Remarks/Usage:

- The Nastran PBEAML evaluator is not available for the general section.
- To use this method, the object must first be set to a type that uses shapes - for example a Bar or Beam (for example `p.type = FET_L_BEAM`).
- If you use the *Alternate* eval_method, you should first set the Poisson's Ratio to be used in the computation (for example `p.pval(70) = 0.3`).

Unpublished work. © 2025 Siemens

• Only End A properties can be computed with this method. Tapered beams are not supported.
Example:
None

ComputeGeneralShape2

ComputeGeneralShape2	
(bAtEndAOrOverall, surfaceID, vecDIR, sOrient, EvalMethod, shear_center_offset, Warping, stress_recovery)	
Description:	
This method computes the properties for a general shape at either End A or End B of a beam.	
Input:	
`BOOL bAtEndAOrOverall`	If True, only computes End A values. If False, only computes End B values and sets the Tapered flag.
`INT4 surfaceID`	ID of a surface to be used to compute the general shape.
`REAL8 vecDIR[0..2]`	A vector that defines the original surface Y axis.
`INT4 sOrient`	The orientation direction (0=Right, 1=Up, 2=Left, 3=Down). Ignored for the Nastran shapes - for these orientation is always *Up*.
`INT4 EvalMethod`	The method used to evaluate the section properties (0=Auto, 1=Original, 2=Alternate).
`BOOL shear_center_offset`	If True, the shear center offset is computed.
`BOOL Warping`	If True, the warping constants are computed.
`BOOL stress_recovery`	If True, the stress recovery locations and cross section orientation that are currently defined in the object are used for the computation. If False, default stress recovery locations and orientation are used.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_BAD_TYPE`	Property type not set to a type that uses a shape.
Remarks/Usage:	
• The Nastran PBEAML evaluator is not available for the general section. • To use this method, the object must first be set to a type that uses shapes - for example a Bar or Beam (for example `p.type = FET_L_BEAM`). • If you use the *Alternate* eval_method, you should first set the Poisson's Ratio to be used in the computation (for example `p.pval(70) = 0.3`).	
Example:	
None	

Matl

`Matl` **`( void )`**	
Description:	
None	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method loads the material that is referenced by the currently loaded property object.	

Unpublished work. © 2025 Siemens

Example:

This method must be used with a Set statement to define the Material object.

```
Sub Main()
Dim App As femap.model
Set App = feFemap()


Dim mt As Matl
Dim pr As Prop

Dim prID as Long
prID = 83

Set pr = App.feProp
pr.Get(prID)
Set mt = pr.Matl()

   App.feAppMessage(FCM_NORMAL, "Material: " _
     + Str$(mt.ID)+ " "+mt.title)


End Sub
```

This sequence creates the "mt" Material object with the material data that is referenced by property 83.

Layup

Layup

(void)

Description:	
None	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method loads the layup that is referenced by the currently loaded property object.
Example:
None

Thickness

`Thickness` **`( dAvgOrC1, dAtC2, dAtC3, dAtC4 )`**	
Description:	
This method returns the thickness of the property.	
Input:	
None	None
Output:	
`REAL8 dAvgOrC1`	The thickness or alternately the thickness at corner 1.
`REAL8 dAtC2`	The thickness at corner 2.
`REAL8 dAtC3`	The thickness at corner 3.
`REAL8 dAtC4`	The thickness at corner 4.
Return Code:	
`FE_OK`	
`FE_BAD_TYPE`	Property type does not have thickness.
`FE_NOT_AVAILABLE`	Laminate property does not reference a Layup.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Area

Area **(dAreaA, dAreaB)**	
Description:	
This method returns the area at EndA and EndB of the line element property.	
Input:	
None	None
Output:	
`REAL8 dAreaA`	The area at EndA.
`REAL8 dAreaB`	The area at EndB.
Return Code:	
`FE_OK`	
`FE_BAD_TYPE`	Property is not correct type.
Remarks/Usage:	
None	
Example:	
None	

Inertia

Unpublished work. © 2025 Siemens

Inertia

(dI1A, dI2A, dI12A, dJA, dI1B, dI2B, dI12B, dJB)

Description:

This function returns the Inertia properties of the line property.

Input:

None	None

Output:

`REAL8 dI1A`	Area Moment of Inertia for plane 1 EndA.
`REAL8 dI2A`	Area Moment of Inertia for plane 2 EndA.
`REAL8 dI12A`	Cross product of inertia at EndA.
`REAL8 dIJA`	Torsional constant EndA.
`REAL8 dI1B`	Area Moment of Inertia for plane 1 EndB.
`REAL8 dI2B`	Area Moment of Inertia for plane 2 EndB.
`REAL8 dI12B`	Cross product of inertia at EndB.
`REAL8 dIJB`	Torsional constant EndB.

Return Code:

`FE_OK`	
`FE_BAD_TYPE`	Property is not a line type.

Remarks/Usage:

None

Example:

None

Unpublished work. © 2025 Siemens

AreDuplicate

AreDuplicate **(nProp1, nProp2, ignoretitle)**	
Description:	
This function determines if two properties have identical values and includes an option to also consider the titles of the two properties.	
Input:	
`INT4 nProp1`	ID of a property.
`INT4 nProp2`	ID of a property to compare to the property specified in `nProp1`.
`BOOL ignoretitle`	Should title be ignored when determining if the 2 properties are duplicates (True=titles ignored, False=titles considered).
Output:	
None	None
Return Code:	
`FE_OK`	Properties are duplicates.
`FE_FAIL`	Properties are not duplicates (i.e., different).
`FE_NOT_EXIST`	At least one of the specified properties does not exist in the model.
Remarks/Usage:	
None	
Example:	
None	

GetLibraryOfType

Unpublished work. © 2025 Siemens

GetLibraryOfType

(libINDEX, libTYPE, LibFILE)

Description:

This function retrieves a property of a specified type from the Property library.

Input:

`INT4 libINDEX`	The index of the entity stored in the library. The first property of the specified type stored is `libINDEX = 0`, the second has `libINDEX = 1`, and so on. If you specify `libINDEX = -1`, a dialog box will be displayed showing all properties of the specified type in the library, and will allow the user to select the property to load.
`INT4 or enum libTYPE`	Use `zElementType` enumeration. Alternatively, enter value for Type of property: Rod=1, Bar=2, Tube=3, Link=4, Beam=5, Spring/ Damper=6, DOFSpring=7, CurvedBeam=8, Gap=9, PlotOnly=10, ShearPanel (Lin=11 Para=12), Membrane (Lin=13 Para=14), BendingOnly (Lin=15 Para=16), Plate (Lin=17 Para=18), PlaneStrain (Lin=19 Para=20), Laminate (Lin=21 Para=22), Axisymmetric (Lin=23 Para=24), Solid (Lin=25 Para=26), Mass=27, MassMatrix=28, Rigid=29, StiffnessMatrix=30, CurvedTube=31, PlotOnlyPlate=32, SlideLine=33, Contact=34, Axisymmetric Shell (Lin=35, Para=36), Para Beam=37, Weld/Fastener=38, Solid Laminate (Lin=39, Para=40)
`INT4 or enum libFILE`	Use `zLibraryFile` enumeration. Alternatively, enter value for Type of library: 0=Open Dialog Box, 1=Personal, 2=Shared, 3=Femap Standard Libraries

Output:

None	None

Return Code:

`FE_FAIL`	No Property was selected. Either the library is empty or does not contain any Properties of the matching type (e.g. Beam Properties).
`FE_CANCEL`	User canceled the dialog box to select a Property from the library.

Unpublished work. © 2025 Siemens

`FE_INVALID`	Library file was opened but isn't a valid library of the correct type.
`FE_NOT_EXIST`	The selected index in the library was not found.
`FE_NOT_AVAILABLE`	The library file could not be accessed. Either the filename is specified incorrectly in Femap, or there is some other permission or corruption problem with the file.
`FE_BAD_TYPE`	`libTYPE` value does not correspond to an available property type
Remarks/Usage:	
This method can be used instead of specifying the *Active Element Type* before using the `GetLibrary` method to load a property from the Property library.	
Example:	
None	

GetValueArray

`GetValueArray`	
`( numValue, nPropID, nPropIndex, dValue )`	
Description:	
This function returns an array of property values using any combination of property IDs and property array index values.	
Input:	
`INT4 numValue`	The number of entries in `nPropID` and `nPropIndex`.
`INT4 nPropID[0..numValue-1]`	The IDs of the properties.
`INT4 nPropIndex[0..numValue-1]`	The ID(s) of the property array index value(s).
Output:	
`REAL8 dValue[0..numvalue-1]`	The array of all the property values.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_BAD_DATA`	One or more `nPropIndex` values are outside the allowable range.
Remarks/Usage:	
See **Accessing Property Properties** and **Property Array Index Values** for more information about accessing property values using the property array index values.	
Example:	
See **GetValueArray** for an example using materials.	

PutValueArray

`PutValueArray`	
`( numValue, nPropID, nPropIndex, dValue )`	
Description:	
This function updates any number of property values on any number of properties.	
Input:	
`INT4 numValue`	The number of entries in `nPropID`, `nPropIndex`, and `dValue`.
`INT4 nPropID[0..numValue-1]`	The IDs of the properties.
`INT4 nPropIndex[0..numValue-1]`	The IDs of the property array index value(s).
`REAL8 dValue[0..numvalue-1]`	The property values.
Output:	
None	None
Return Code:	
`FE_NOT_EXIST`	One or more Properties being updated do not exist.
`FE_BAD_DATA`	One or more `nPropIndex` values are outside the allowable range.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

See **Accessing Property Properties** and **Property Array Index Values** for more information about accessing property values using the property array index values.
Example:
See **PutValueArray** for an example using materials.

PutValue

`PutValue` **`( nPropSetID, nPropIndex, dValue )`**	
Description:	
This function updates a single property value on any number of properties.	
Input:	
`INT4 nPropSetID`	The ID of set of property IDs.
`INT4 nPropIndex`	The ID of the property array index value.
`REAL8 dValue`	The property value.
Output:	
None	None
Return Code:	
`FE_BAD_DATA`	One or more nPropIndex values are outside the allowable range.
Remarks/Usage:	
See **Accessing Property Properties** and **Property Array Index Values** for more information about accessing property values using the property array index values.	
Example:	
None	

CopyArray

Unpublished work. © 2025 Siemens

CopyArray

(bUpdateReferences, bCopyToStartID, nCopyToStartID, numID, nOrigID, pnNewID)

Description:

This function copies any number of properties and allows you to select a starting ID for the copies or provide all IDs for all copies. It also allows you to choose if the *new property* should replace the *original property* anywhere the original property was referenced (i.e., element "n" would now reference copied property instead of original property).

Input:

`BOOL bUpdateReferences`	Flag to indicate if *copied property* should replace *original property* anywhere the *original property* was referenced. True = original property references are replaced with copied property.
`BOOL bCopyToStartID`	Flag controls IDs of *copied properties*. When True, uses value of `nCopyToStartID` as the first ID of the copies. If a property ID already exists in the range where new properties will be created, that ID is skipped and the property will be placed in the next empty ID. When False, then `pnNewID` becomes an input instead of an output and must be populated with the same number of entries as `nOrigID` and cannot contain any IDs of properties which already exist.
`INT4 nCopyToStartID`	Starting ID for copied properties.
`INT4 numID`	Number of original property IDs in `nOrig` array.
`INT4 nOrigID[numID-1]`	The IDs of the original properties.

Output:

`INT4 pnNewID[numID-1]`	The IDs of the newly created (copied) properties. If `bCopyToStartID` is False, this is treated as an input instead of an output, which contains the new IDs off all copied properties. It must have the same number of entries as `nOrigID` and cannot contain any IDs of existing properties.

Return Code:

`FE_NOT_AVAILABLE`	One or more of the properties to be created already exist.

Unpublished work. © 2025 Siemens

`FE_NOT_EXIST`	One or more of the properties being copied do not exist.
`FE_NO_MEMORY`	Unable to allocate memory to process this operation.
Remarks/Usage:	
None	
Example:	
None	

GetAllArray

`GetAllArray` **`( setID, numProp, entID, exist, proptype, mid, extramid, layer, color, layupID, refCSys, title )`**	
Description:	
This method retrieves arrays of values for a set of properties, including if property exists in the database.	
Input:	
`INT4 setID`	ID of the set that contains the property IDs to retrieve. If 0, retrieve all properties in the model.
Output:	
`INT4 numProp`	The number of properties for which data is being returned.
`INT4 entID[0..numProp-1]`	The IDs of the properties being retrieved.
`BOOL exist[0..numProp-1]`	True=Property exists, False=Property does not exist.
`INT4 proptype[0..numProp-1]`	Type of property (See *type* section in **Property Object Properties**).
`INT4 mid[0..numProp-1]`	The ID of the primary material for the property.
`INT4 extramid[0..(10*numProp)-1]`	The IDs of any extra materials referenced by the property.

Unpublished work. © 2025 Siemens

`INT4 layer[0..numProp-1]`	The ID of the layer for the property.
`INT4 color[0..numProp-1]`	The color of the property.
`INT4 layupID[0..numProp-1]`	The ID of a Layup that is referenced. Will be 0 except for laminate and solid laminate properties.
`INT4 refCSys[0..numProp-1]`	The ID of a Reference Coordinate System.
`STRING title[0..numProp-1]`	The title of the property.
Return Code:	
`FE_FAIL`	Unable to retrieve property info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve property info.
Remarks/Usage:	
None	
Example:	
None	

PutAllArray

`PutAllArray`
`( numProp, entID, bUpdateType, proptype, bUpdateMatl, mid, bUpdateExtraMatl, extramid, bUpdateLayer, layer, bUpdateColor, color, bUpdateLayup, layupID, bUpdateCSys, refCSys, bUpdateTitle, title )`
Description:
This method updates properties of multiple selected properties.
Input:

`INT4 numProp`	The number of properties for which data is being returned.
`INT4 entID[0..numProp-1]`	The IDs of the properties to update.

Unpublished work. © 2025 Siemens

`BOOL bUpdateType[0..numProp-1]`	If True, update proptype.
`INT4 proptype[0..numProp-1]`	Type of property (See type section in **Property Object Properties**). Only assigned if `bUpdateType`=True.
`BOOL bUpdateMatl[0..numProp-1]`	If True, update the mid properties.
`INT4 mid[0..numProp-1]`	The ID of the primary material for the property. Only assigned if `bUpdateMatl`=True
`BOOL bUpdateExtraMatl[0..numProp-1]`	If True, update extramid properties.
`INT4 extramid[0..(10*numProp)-1]`	The IDs of any extra materials referenced by the property. Only assigned if `bUpdateMatl`=True.
`BOOL bUpdateLayer[0..numProp-1]`	If True, update layer ID.
`INT4 layer[0..numProp-1]`	The ID of the layer for the property. Only assigned if `bUpdateLayer`=True.
`BOOL bUpdateColor[0..numProp-1]`	If True, update color.
`INT4 color[0..numProp-1]`	The color of the property. Only assigned if `bUpdateColor`=True.
`BOOL bUpdateLayup[0..numProp-1]`	If True, update layup ID.
`INT4 layupID[0..numProp-1]`	The ID of a Layup that is referenced. Will be 0 except for laminate and solid laminate properties. Only assigned if `bUpdateLayup`=True.
`BOOL bUpdateCSys[0..numProp-1]`	If True, update refCSys
`INT4 refCSys[0..numProp-1]`	The ID of a Reference Coordinate System. Only assigned if `bUpdateCSys`=True.
`BOOL bUpdateTitle[0..numProp-1]`	If True, update title.
`STRING title[0..numProp-1]`	The title of the property.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to store one or more property with the specified IDs.
`FE_NO_MEMORY`	Unable to allocate memory to update properties.
Remarks/Usage:	
None	
Example:	
None	

GetFlagArray

`GetFlagArray` **`( numValue, nPropID, nFlagIndex, nIntValue )`**	
Description:	
This method returns an array of property values using any combination of property IDs and property flag array index values.	
Input:	
`INT4 numValue`	The number of entries in the `nPropID` and `nFlagIndex` arrays. Both arrays must be the same size, so if you want to.
`INT4 nPropID[0..numValue-1]`	The IDs of the properties.
`INT4 nFlagIndex[0..numValue-1]`	The IDs of the property flag array index values.
Output:	
`INT4 nIntValue[0..numValue-1]`	The array of flag values that is returned.
Return Code:	
`FE_FAIL`	Unable to retrieve flag info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve flag info.

Unpublished work. © 2025 Siemens

FE_NOT_EXIST	One or more specified properties do not exist.
FE_BAD_DATA	One or more flag indices are invalid. Valid flag indices must be in the range [0..7].

Remarks/Usage:

Many of the flag values are really BOOLEANs, but are returned as INT4. In those cases, 0=False, 1=True.

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim pr As femap.Prop
   Set pr = App.feProp

   Dim propIDs(1) As Long
   Dim propFlags(1) As Long
   Dim vals As Variant

   ‘Recover Values for Flags 0 and 4 for Property 1
   propIDs(0) = 1
   propIDs(1) = 1
   propFlags(0) = 0
   propFlags(1) = 4

   rc = pr.GetFlagArray (2, propIDs, propFlags, vals)
End Sub
```

PutFlagArray

Unpublished work. © 2025 Siemens

PutFlagArray

(numValue, nPropID, nFlagIndex, nIntValue)

Description:

This method updates any number of property flag values on any number of properties.

Input:

`INT4 numValue`	The number of entries in `nPropID` and `nFlagIndex`.
`INT4 nPropID[0..numValue-1]`	The IDs of the properties.
`INT4 nFlagIndex[0..numValue-1]`	The IDs of the property flag array index values.
`INT4 nIntValue[0..numValue-1]`	The array of flag values to update/apply.

Output:

None	None

Return Code:

`FE_FAIL`	Unable to retrieve flag info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve flag info.
`FE_NOT_EXIST`	One or more specified properties do not exist.
`FE_BAD_DATA`	One or more flag indices are invalid. Valid flag indices must be in the range [0..7].

Remarks/Usage:

Many of the flag values are really BOOLEANs, but are returned as `INT4`. In those cases, 0=False, 1=True.

Example:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim pr As femap.Prop
```


Unpublished work. © 2025 Siemens

```
   Set pr = App.feProp

   Dim propIDs(1) As Long
   Dim propFlags(1) As Long
   Dim vals(1) As Long

   'Set Values for Flags 0 and 4 for Property 1 to 1 (True)
   propIDs(0) = 1
   propIDs(1) = 1
   propFlags(0) = 0
   propFlags(1) = 4
   vals(0) = 1
   vals(1) = 1

   rc = pr.PutFlagArray (2, propIDs, propFlags, vals)

End Sub
```

HasMatl

HasMatl

( )

Description:	
This function returns whether the property has an associated material.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_OK`	The property has an associated material.
`FE_FAIL`	The property does not have an associated material.
Remarks/Usage:	
None	
Example:	
None	

HasShape

`HasShape`
`( void )`
Description:
This function returns whether the property has an associated shape.
Input:

None	None
Output:	
None	None
Return Code:	
`FE_OK`	The property has an associated shape.
`FE_FAIL`	The property does not have an associated shape.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

Reference Objects

Reference objects correspond to the references created for imported Geometry, Analysis Model, and/or Analysis Results files. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Reference objects. For more information, see **Common Entity Properties and Methods**.

Use the `feReference` method of the Femap Application object to create Reference objects.

Reference Object Properties

Reports are always stored with a SetID=1, and the ID equal to the reference ID.

Property	Description
`STRING FileName[260]`	Full directory path and file name of Geometry, Analysis Model, or Analysis Results file (Maximum 260 Characters).
`STRING ParentFileName[260]`	Full directory path and file name of *parent* Geometry, Analysis Model, or Analysis Results file (Maximum 260 Characters).
`INT4 ReadType`	Type of file being referenced: 1=Geometry file, 2=Analysis Model file, 3=Analysis Results file.
`INT4 ReadFrom`	Program or translator which created the imported file: -1=None, 0=MSC NASTRAN, 1=PC NASTRAN, 2=PAL 2, 3=STARDYNE, 4=ANSYS, 5=ANSYS PC LINEAR, 6=ALGOR, 7=COSMOS, 8=PATRAN, 9=FEMAP Neutral, 10=DXF, 11=IGES, 12=CSV File, 13=UAI NASTRAN, 14=COSMIC NASTRAN, 15=STAAD, 16=ABAQUS, 17=WECAN (EMDAC), 18=SAP, 19=CDA SPRINT, 20=CAEFEM, 21=IDEAS, 22=ME NASTRAN, 23=ADINA, 24=GENESIS, 25=ODBC, 26=CSA NASTRAN, 27=SLA, 28=CFDESIGN, 29=ACIS, 30=PARASOLID, 31=LS-DYNA, 32=MARC, 33=SINDA, 34=STEP, 35=NEi NASTRAN, 36=MODEL SOLUTION, 37=NCODE (NOT USED), 38=CATIA, 39=PRO/ENGINEER, 40=VDA, 41=SOLID EDGE,

Unpublished work. © 2025 Siemens

Property	Description
	42=TMG, 43=NX (UNIGRAPHICS), 44=VR NASTRAN, 45=NX NASTRAN, 46=CATIA V5, 47=SOLIDWORKS, 48=TOSCA, 49=JT
`INT4 ReadTime`	Time the file was imported into Femap (in *Unix Time*).
`INT4 ReadFileTime`	Time the imported file was originally created (in *Unix Time*).

Reference Object Methods

DeleteAll

`DeleteAll`	
`( void )`	
Description:	
This method deletes all References currently in the model.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method does the same thing as clicking the **Remove All** button in the Reference Manager dialog box, which is accessed using the **PDM** → **References** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

Report Objects

Report objects correspond to the report formats in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Report objects. For more information, see **Common Entity Properties and Methods**.

Use the `feReport` method of the Femap Application object to create Report objects.

Report Object Properties

Reports are always stored with a `SetID=1`, and the ID equal to the report ID.

Property	Description
`STRING title`	Report Format title (maximum 79 characters).
`INT4 DataType`	The type of data included in this report: 7=Nodal Results, 8=Elemental Results.
`BOOL CenterHeader1`	Set to True to center the associated header.
`BOOL CenterHeader2`	
`BOOL DbSpHeader1`	Set to True to double-space the associated header.
`BOOL DbSpHeader2`	
`STRING Header1`	The formats associated with the various lines of the report. Refer to the **List → Output → Use Format** command for more information about how to define each of these formats. Maximum size is 130 characters.
`STRING Header2`	
`STRING ColumnHeader`	
`STRING Data1`	
`STRING Data2`	

Unpublished work. © 2025 Siemens

Property	Description
`STRING Data3`	
`STRING Data4`	

Report Object Methods

There are no Report object-specific methods. Only the standard entity methods are available.

Superelement Reference Objects

Superelement references work in conjunction with Nastran AnalysisMgr objects and are used to create references to external superelements in an analysis set. Note that superelement references are ignored in an analysis set if the `NasExtSEOutOn` property is set to TRUE in the analysis set.

Use the `feSEReference` method of the Femap Application object to create superelement reference objects.

Superelement Reference Object Properties

Superelement references are stored with the setID value equal to the ID of the analysis set.

In this table, you may see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`STRING ReferenceFile`	External Superelement file.
`INT4 ReferenceType`	Type of Superelement Reference. 0 = OP2, 1=OP4, 2=PCH.
`INT4 ReferenceID`	Superelement ID.
`STRING ASMFile`	ASM file. Only used when Reference Type is set to OP2 or OP4.
`BOOL Stiffness`	Only used when ReferenceType = PCH. Stiffness Matrix (K2GG).

Unpublished work. © 2025 Siemens

Property	Description
`BOOL Mass`	Only used when ReferenceType = PCH. Mass Matrix (M2GG).
`BOOL ViscousDamping`	Only used when ReferenceType = PCH. Viscous Damping Matrix (B4GG).
`BOOL StructuralDamping`	Only used when ReferenceType = PCH. Structural Damping Matrix (K42GG).
`BOOL LoadsMatrix`	Only used when ReferenceType = PCH. Loads Matrix (P2G).

`enum zAnalysisAssignForm AssignForm`	Form used for ASSIGN statements:
	`0..None = AAF_NONE`
	`1..FORMATTED = AAF_FORMATTED`
	`2..UNFORMATTED = AAF_UNFORMATED`
	`3..LITTLEENDIAM = AAF_LITTLEENDIAN`
	`4..BIGENDIAN = AAF_BIGENDIAN`

Solid Objects

Solid objects correspond to the solids and volumes in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for solids. For more information, see **Common Entity Properties and Methods**.

Use the `feSolid` method of the Femap Application object to create Solid objects.

Solid Object Properties

Solids are always stored with a SetID=1, and the ID equal to the Solid (or Volume) ID.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 Type`	The solid/volume type: 6=Volume, 39=Solid.
`INT4 attrPID`	The Property ID of the meshing attribute.
For Volumes Only	

`INT4 VolSurface[0..5]`
or
`VARIANT vVolSurface`
	The IDs of the surfaces used to define the volume.

`BOOL VolReverse[0..5][0..2]`
or
`VARIANT vVolReverse`
	Flags used to indicate how the surface parametric directions are reversed when defining the volume. True=Reversed.

`INT4 VolColor`	The color of the volume.
`INT4 VolLayer`	The layer associated with the volume.
`INT4 VolType`	The type/shape of the volume: 0=Brick,1=Wedge,2=Pyramid,3=Tetra.
For Solids Only	
`INT4 color`	The color of the solid.
`INT4 layer`	The layer associated with the solid.
`INT4 Engine`	The ID of the solid modeling engine used with this solid: 0=None, 1=Parasolid, 2=ACIS.
`INT4 Address`	The solid modeling engine geometry address.
`STRING title`	The title of the solid (maximum 79 characters).
`ENUM attrTopology`	Type of solid element: 6 = FTO_TETRA4, 7 = FTO_WEDGE6, 8 = FTO_BRICK8, 10 = FTO_TETRA10, 11 = FTO_WEDGE15, 12 =

Unpublished work. © 2025 Siemens

Property	Description
	FTO_BRICK20, 14 = FTO_PYRAMID5, 19 = FTO_PYRAMID13
`BOOL attrMidsideOnGeometry`	True or False for if elements are parabolic.
`REAL8 attrMaxMidsideAngle`	The max angle for midside nodes.

Solid Object Methods

Points

`Points` **`( numPoints, pointID )`**	
Description:	
This method returns a list of all of the points used to define this solid/volume.	
Input:	
None	None
Output:	
`INT4 numPoints`	The number of points in the pointID list.
`INT4 pointID[0..numPoints-1]`	The IDs of the points on the solid.
Return Code:	
`FE_FAIL`	Unable to get the point list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	

Unpublished work. © 2025 Siemens

Example:
None

PointsAsSet

`PointsAsSet` **`( nSetID, bClear )`**	
Description:	
This method returns all of the points used to define this solid/volume in a Set object.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Points.
`BOOL bClear`	If True, the Set is cleared prior to loading the Point IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the point list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

Curves

Unpublished work. © 2025 Siemens

Curves

(nCombinedMode, numCurves, curveID)

Description:

This method returns a list of all of the curves used to define this solid/volume.

Input:

`INT4 nCombinedMode`	0=List contains the basic curves that are on the solid, 1=Any curves that are on the solid that are in a Combined Curve are replaced by the ID of the Combined Curve, 2=List contains both the underlying and combined curves.

Output:

`INT4 numCurves`	The number of curves in the curveID list.
`INT4 curveID[0..numCurves-1]`	The IDs of the curves on the solid.

Return Code:

`FE_FAIL`	Unable to get the curve list. Possibly data is not stored or ID is not set correctly.

Remarks/Usage:

This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.

Example:

None

CurvesAsSet

CurvesAsSet

(nCombinedMode, nSetID, bClear)

Description:

Unpublished work. © 2025 Siemens

This method returns all of the curves used to define this solid/volume in a Set object.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic curves that are on the solid, 1=Any curves that are on the solid that are in a Combined Curve are replaced by the ID of the Combined Curve, 2=List contains both the underlying and combined curves.
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Curves.
`BOOL bClear`	If True, the Set is cleared prior to loading the Curve IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the curve list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

FreeCurvesAsSet

`FreeCurvesAsSet` **`( nCombinedMode, nSetID, bClear )`**
Description:
This method returns all of the free curves used to define this Solid in a Set object. Free curves are curves on a sheet solid that are only attached to one surface - the free edges of the sheet.
Input:

Unpublished work. © 2025 Siemens

`INT4 nCombinedMode`	0=List contains the basic curves that are on the solid, 1=Any curves that are on the solid that are in a Combined Curve are replaced by the ID of the Combined Curve, 2=List contains both the underlying and combined curves.
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Curves.
`BOOL bClear`	If True, the Set is cleared prior to loading the Curve IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the curve list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is Put to the database). The ID field must be properly specified.	
Example:	
None	

Surfaces

`Surfaces` **`( nCombinedMode, numSurfs, surfaceID )`**	
Description:	
This method returns a list of all of the surfaces used to define this solid/volume.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are on the solid, 1=Any surfaces that are on the solid that are in a Combined Surface are replaced by the ID of the Combined Surface, 2=List contains both the underlying and combined surfaces.

Unpublished work. © 2025 Siemens

Output:	
`INT4 numSurfs`	The number of surfaces in the surfaceID list.
`INT4 surfaceID[0..numSurfs-1]`	The IDs of the surfaces on the solid.
Return Code:	
`FE_FAIL`	Unable to get the surface list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

SurfacesAsSet

`SurfacesAsSet` **`( nCombinedMode, nSetID, bClear )`**	
Description:	
This method returns a list of all of the surfaces used to define this solid/volume.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are on the solid, 1=Any surfaces that are on the solid that are in a Combined Surface are replaced by the ID of the Combined Surface, 2=List contains both the underlying and combined surfaces.
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Surfaces.
`BOOL bClear`	If True, the Set is cleared prior to loading the Surface IDs.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	Unable to get the surface list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any solid/volume, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

SheetFacesAsSet

`SheetFacesAsSet`
`( nSetID, nInsideSolidSetID )`
Description:
This method returns a list of all of the surfaces which are *sheet faces* on a sheet solid or general body and also returns a list of all of the surfaces which are *sheet faces* inside a general body.
Input:

`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Surfaces which are *sheet faces* of a sheet solid or general body.
`INT4 nInsideSolidSetID`	The ID of the Set Object that will contain the IDs of the Surfaces which are *sheet faces* found on the interior (inside) of a general body.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the surface list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

None
Example:
None

WireCurvesAsSet

`WireCurvesAsSet` **`( nSetID, nInsideSolidSetID )`**	
Description:	
This method returns a list of all of the curves which are *wire solids* on a general body and also returns a list of all of the curves which are *wire solids* inside a general body.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Curves which are *wire solids* of a general body.
`INT4 nInsideSolidSetID`	The ID of the Set Object that will contain the IDs of the Curves which are *wire solids* found inside a general body.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the surface list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
None	
Example:	
None	

MassProp

Unpublished work. © 2025 Siemens

MassProp	
(area, volume, cg, inertia)	
Description:	
This function computes the mass properties of a solid.	
Input:	
None	None
Output:	
`REAL8 area`	The total surface area of the solid.
`REAL8 volume`	The volume of the solid (if closed).
`REAL8 cg[0..2]`	The location of the center of gravity of the solid.
`REAL8 inertia[0..9]`	The 3x3 inertia matrix stored in row order [0][0], [0][1], [0][2], [1][0]...[2][2].
Return Code:	
`FE_NOT_EXIST`	The solid that you have selected does not exist.
Remarks/Usage:	
The cg and inertia terms are in global rectangular coordinates.This method will only work with solids. Furthermore, it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

Export

Export
(fName)
Description:

Unpublished work. © 2025 Siemens

This function writes the current solid to an export (SAT or X_T) file.	
Input:	
`STRING fName`	The name of the file to create.
Output:	
None	None
Return Code:	
`FE_FAIL`	The current object does not contain a solid (must be a solid, not a volume).
Remarks/Usage:	
This method will only work with solids. Furthermore, it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

IsSolid

`IsSolid` **`( void )`**	
Description:	
This function indicates whether a solid is a true, valid solid.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	The solid is not a true solid - could be a sheet or wire.

Unpublished work. © 2025 Siemens

`FE_INVALID`	The current object does not contain a valid solid - volumes are not supported.
Remarks/Usage:	
This method will only work with solids, not volumes. Returns `FE_OK` if current object is a solid.	
Example:	
None	

IsSheet

`IsSheet` **`( void )`**	
Description:	
This function indicates whether a solid is a sheet solid.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	The solid is not a sheet solid - could be a solid body or wire.
`FE_INVALID`	The current object does not contain a valid solid - volumes are not supported.
Remarks/Usage:	
This method will only work with solids, not volumes. Returns `FE_OK` if current object is a sheet solid. A sheet solid is a collection of surfaces, possibly stitched, but which do not enclose a closed volume.	
Example:	
None	

IsWire

Unpublished work. © 2025 Siemens

IsWire	
(void)	
Description:	
This function indicates whether a solid is a wire solid.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_FAIL	The solid is not a wire solid - could be a solid body or sheet.
FE_INVALID	The current object does not contain a valid solid - volumes are not supported.
Remarks/Usage:	
This method will only work with solids, not volumes. Returns FE_OK if current object is a wire solid. A wire solid is a collection of curves, without surfaces.	
Example:	
None	

IsGeneral

IsGeneral	
(void)	
Description:	
This function indicates whether a solid is a general body.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	The solid is not a general body.
`FE_INVALID`	The current object does not contain a valid solid - volumes are not supported.
Remarks/Usage:	
This method will only work with solids, not volumes. Returns `FE_OK` if current object is a general body.	
Example:	
None	

Inside

`Inside`
`( vXYZ, nInsideCode )`
Description:
This method finds if a coordinate location lies “inside”, “outside”, or “on” a point, curve, or surface of the specified solid.
Input:

`REAL8 xyz[0..2]`	The Global Rectangular coordinates of the location to be tested.
Output:	
`INT4 nInsideCode`	0=Location is outside, 1=Location is inside, 2=Location is on a point, curve, or surface of the specified solid.
Return Code:	
None	None

`Remarks/Usage:`
To check if locations are in multiple solids being considered as a *single combined solid*, see **feSolidInsideMulti**.

Unpublished work. © 2025 Siemens

Example:
None

Facets

Facets **(numFacets, xyzFacet)**	
Description:	
This method returns the triangular facets used for display of solids.	
Input:	
None	None
Output:	
`INT4 numFacets`	The number of facets returned.
`REAL8 xyzFacet[0..9*numFacets-1]`	The coordinates of the corners of the facets. Each facet is a triangle, and therefore has 9 coordinates. They are arranged in xyzFacet as [X1,Y1,Z1,X2,Y2,Z2,X3,Y3,Z3....] for the first facet, followed by the second and so on.
Return Code:	
`FE_FAIL`	Not a solid, or unable to retrieve facets.
Remarks/Usage:	
This method will only work with solid faces that have facetting information. The surface information must be in the database. Only the ID from the current object is used.	
Example:	
None	

Nodes

Unpublished work. © 2025 Siemens

Nodes

(getBoundary, getInterior, numNodes, nodeID)

Description:

This method returns the nodes that are attached to this solid.

Input:

`BOOL getBoundary`	If True, the nodes attached to the solid boundary (the surfaces, curves and points) are selected.
`BOOL getInterior`	If True, the nodes attached to the interior of the solid (not on the surfaces, curves or points) are selected.

Output:

`INT4 numNodes`	The number of nodes returned.
`INT4 nodeID[0..numNodes-1]`	The IDs of the selected nodes which are attached to the solid.

Return Code:

`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the solid in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.

Remarks/Usage:

None

Example:

The following codes modifies the color of all nodes on Solid 4.

```
Sub Main
Dim App As femap.model
Set App = GetObject(,"femap.model")

Dim so As Object
Set so = App.feSolid
Dim nd As Object
Set nd = App.feNode
Dim rc As Long
Dim Count As Long
```


Unpublished work. © 2025 Siemens

```
Dim Nodes As Variant
 Dim i As Long

solidID = 4

rc = so.Get (solidID)
rc = so.Nodes(True, True, Count, Nodes)
For i = 0 To Count - 1
   nd.Get (Nodes(i))
   nd.Color = 4
   nd.Put (0)
Next i

App.feViewRegenerate(0)

End Sub
```

NodesAsSet

NodesAsSet

(getBoundary, getInterior, nSetID, bClear)

Description:	
This method returns the nodes that are attached to this solid in a Set.	
Input:	
`BOOL getBoundary`	If True, the nodes attached to the solid boundary (the surfaces, curves and points) are selected
`BOOL getInterior`	If True, the nodes attached to the interior of the solid (not on the surfaces, curves or points) are selected
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Nodes
`BOOL bClear`	If True, the Set is cleared prior to loading the Node IDs
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the solid in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.
Remarks/Usage:	
None	
Example:	
None	

Elements

`Elements`	
`( numElems, elemID )`	
Description:	
This method returns the elements that are attached to this solid.	
Input:	
None	None
Output:	
`INT4 numElems`	The number of elements returned.
`INT4 elemID[0..numElems-1]`	The IDs of the selected elements which are attached to the solid.
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the solid in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.

Unpublished work. © 2025 Siemens

Remarks/Usage:
Only elements that have all of their nodes in the solid will be returned by this method. Elements that simply touch the solid at one edge or one node will not be returned.
Example:
None

ElementsAsSet

`ElementsAsSet` **`( nSetID, bClear )`**	
Description:	
This method returns the elements that are attached to this solid in a Set.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Elements.
`BOOL bClear`	If True, the Set is cleared prior to loading the Element IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the solid in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.
Remarks/Usage:	
Only elements that have all of their nodes in the solid will be returned by this method. Elements that simply touch the solid at one edge or one node will not be returned.	
Example:	
None	

Unpublished work. © 2025 Siemens

BoundingBox

BoundingBox	
(Box)	
Description:	
This method returns the coordinates of a rectangular box the encloses this geometry.	
Input:	
None	None
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in Global Rectangular coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
`FE_INVALID`	Object does not contain valid geometry. This method only works for Parasolid-based geometry.
Remarks/Usage:	
This method always retrieves a box that is aligned with Global Rectangular coordinates, not an aligned box of minimum volume.	
Example:	
None	

InitializeAttributes

Unpublished work. © 2025 Siemens

InitializeAttributes	
(nPropID)	
Description:	
This method initializes a mesh attribute of a solid to a specific property.	
Input:	
`INT4 nPropID`	The ID of the property to initialize mesh attribute to.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to initialize.
Remarks/Usage:	
None	
Example:	
None	

Volume

Volume	
(volume)	
Description:	
This method returns the volume of the solid.	
Input:	
None	None
Output:	
`REAL8 volume`	The volume of the solid.

Unpublished work. © 2025 Siemens

Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

IsFacet

`IsFacet`
 	
`( void )`	

Description:	
This method indicates if the solid has any facet-based surfaces.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Solid contains one or more facet-based surfaces.
`FE_FAIL`	Solid is a *classic* Parasolid body containing no facet-based surfaces.
`FE_INVALID`	The current object does not contain a valid Parasolid Body - volumes are not supported.
Remarks/Usage:	
This method will only work with Parasolid solids, not volumes.	

Unpublished work. © 2025 Siemens

Example:

The following code determines if any surfaces on a solid are facet-based:

```
Sub Main
   Dim App as femap.model
   Set App = feFemap()

   Dim Sol As femap.Solid
   Set Sol = App.feSolid

   Dim Su As femap.Surface
   Set Su = App.feSurface

   Dim suSet As femap.Set
   Set suSet = App.feSet

   Dim showSet As femap.Set
   Set showSet = App.feSet

    'Does solid have any facet surfaces
   If Sol.SelectID("Pick Solid to Check") = FE_OK Then
        If Sol.IsFacet() = FE_OK Then
            'If the solid is all facet surfaces, put them in showSet
            If Sol.IsAllFacet() = FE_OK Then
               showSet.AddRule( Sol.ID, FGD_SURFACE_ONSOLID
            'If not, walk surfaces of solid and see which ones are
            Else
               suSet.AddRule( Sol.ID, FGD_SURFACE_ONSOLID)
               While Su.NextInSet( suSet.ID ) = FE_OK
```


Unpublished work. © 2025 Siemens

```
                    If Su.IsFacet() Then
                        showSet.Add( Su.ID )
                     End If
                Wend
            End If
        End If
    End If

     'if there are any faceted surfaces, show them
    If Not showSet.IsEmpty() Then
        showSet.Show(FT_SURFACE)
    End If
End Sub
```

IsAllFacet

IsAllFacet

(void)

Description:	
This method indicates if all the surfaces of the solid are facet-based surfaces.	
Input:	
None	None
Output:	
None	None
Return Code:	
FE_OK	All surfaces of this solid are facet-based.

Unpublished work. © 2025 Siemens

`FE_FAIL`	Not all of the surface of the solid are facet-based.
`FE_INVALID`	The current object does not contain a valid Parasolid Body - volumes are not supported.
Remarks/Usage:	
This method will only work with Parasolid solids, not volumes.	
Example:	
None	

IsFacet

`IsFacet` **`( void )`**	
Description:	
This method indicates if the solid has any facet-based surfaces.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Solid contains one or more facet-based surfaces.
`FE_FAIL`	Solid is a *classic* Parasolid body containing no facet-based surfaces.
`FE_INVALID`	The current object does not contain a valid Parasolid Body - volumes are not supported.
Remarks/Usage:	
This method will only work with Parasolid solids, not volumes.	
Example:	
None	

Unpublished work. © 2025 Siemens

Surface Objects

Surface objects correspond to the surfaces (including boundary surfaces) in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Surface objects. For more information, see **Common Entity Properties and Methods**.

Use the `feSurface` method of the Femap Application object to create Surface objects.

Surface Object Properties

Surfaces are always stored with a `SetID=1`, and the ID equal to the Surface ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`INT4 layer`	The ID of the layer associated with the surface.
`INT4 color`	The surface color.
`INT4 type`	The surface type: 0=Bilinear, 1=Ruled, 2=Revolution, 3=Coons, 4=Bezier, 5=Solid, 6=Not Used, 7=Boundary.

`INT4 StdCurve[0..5]`
or
`VARIANT vStdCurve`
	The IDs of the curves referenced by Bilinear, Ruled, Revolution, Coons and Bezier surfaces.

`BOOL StdCurveReverse[0..5]`
or
`VARIANT vStdCurveReverse`
	If True, the surface uses the corresponding `StdCurve` in a reversed orientation.

`INT4 Divisions[0..1]`	The number of divisions in the u and v parametric directions for display purposes.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vDivisions`	
`INT4 Engine`	The ID of the solid modeling engine used with this surface: 0=None, 1=Parasolid.
`INT4 Address`	The solid modeling engine geometry address.
`BOOL Suppressed`	True if this surface is suppressed (will not be meshed).
`INT4 InternalToBoundary`	If this surface is used to define a Combined Surface, then this property contains the ID of the Combined Surface, otherwise it is 0.
`REAL8 ScaleFactor`	The Solid Geometry Scale Factor.
Mesh Attributes	
`INT4 attrPID`	The Property ID of the meshing attribute.
`INT4 attrTopology`	The shape of elements that will be meshed onto the surface (2=Tri3, 3=Tri6, 4=Quad4, 5=Quad8).
`BOOL attrMaxQuads`	If True, the **Max Quads** option will be turned on, which will attempt to minimize the amount of triangles in a mesh.
`INT4 attrMesher`	The mesher that will be used (0=Auto,1=Subdivision, 2=Fast Tri, 3=3D Tri).
`INT4 attrMappedLevel`	Mapped Meshing Switch (0=Off,1=On).
`BOOL attrMapSubdivisions`	If True and using subdivision mesher, then additional mapped meshing may be done at the subdivision level.
`BOOL attrMapEqualOnly`	If True, only surfaces with equal numbers of elements on opposite sides will be mapped mesh.
`BOOL attrMapAltTri`	If True, and Mapped Tri meshing, the triangle cuts will alternate directions.
`BOOL attrMapRightBias`	If True and Mapped Tri meshing, the first cut will be *right* oriented, otherwise *left*.

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 attrMapAngleDeviation`	If Mapped Meshing, this value is the max angle deviation from 90 deg to determine a corner of the mapped mesh.
`REAL8 attrMapMinCornerAngle`	If Mapped Meshing, prevents mapped meshing if a mapped corner angle is less than this value.
`BOOL attrMidsideGeom`	If True, Midside nodes will be moved to (or toward) the geometry. If False, Midside nodes are at the midpoint of the element edge. This has no effect on planar surfaces.
`REAL8 attrMidsideAngle`	If Nonzero, then this limits the angle that an edge can be distorted as the midside node is moved toward the geometry.
`BOOL attrPostMeshCleanup`	If True, for the v10 meshers, after meshing has completed the mesh is scanned for certain patterns of elements that can be adjusted to improve the mesh.
`INT4 QuadMeshLayers`	The number of quad mesh layers to pave around holes.
`REAL8 attrQuadCutAngle`	Quad elements with angles that are further from 90 deg than this angle are split into triangles.
`INT4 attrMinBetween`	For the Subdivision mesher, ensures that there will be at least this number of elements along any cut between edges. Normally should be set to 1.
`REAL8 attrMaxAspect`	For the Subdivision mesher, the divisions are adjusted to attempt to get a max element aspect ratio less than this value.
`INT4 attrQuickCutNodes`	For the Subdivision mesher, surfaces with more than this number of elements along its edges are subdivided without looking at every possibility. Not used with v10 Mesher.
`BOOL attrVertexLoopsAsHardPoints`	If True, Use Internal Points as Mesh Locations will be activated.
`BOOL attrSmoothLaplacian`	If True, use Laplacian smoothing. If False, Centroidal.
`INT4 attrSmoothIter`	Number of Smoothing Iterations.
`REAL8 attrSmoothTolerance`	Tolerance value for smoothing. Smoothing will stop prior to the specified number of iterations if no nodes are moving further than this value.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL attrConnectEdgeNodes`	If True, automatically uses nodes on curves when meshing this surface (automatically connects to adjoining meshes).
`REAL8 attrConnectEdgeNodeTol`	Tolerance between nodal locations and mesh locations on edge for finding existing edge nodes to connect.
`INT4 attrOffsetFrom`	Location in element to offset from (0=Centerline, 1=Top Face, 2=Bottom Face).
`REAL8 attrOffset`	The meshing attribute defining the offset to be applied to elements.
`BOOL attrInitialized`	If True, the attributes have been initialized - probably because the surface has been meshed, or attributes have been defined.
Mesh Approach	
`INT4 approach`	The meshing approach for the surface: 0=Not Specified, 1=Free-Parametric, 2=Free-Planar, 3=Mapped-Four Corner, 4=Mapped-Three Corner, 5=Mapped-Three Corner Fan, 6=Link to Surface, 7=Fast Tri-Parametric, 8=Fast Tri-Planar.

`INT4 approachData[0..3]`
or
`VARIANT vapproachData`
	The data for various approaches. For the mapped approaches, these are the point IDs defining the *corners* of the mapped mesh. For the Link to Surface approach, `approachData[0]` is the ID of the surface to which this surface is linked.

Mesh Sizing	
`REAL8 MeshSizeInterior`	Growth factor on interior of surface.
`REAL8 MeshSizeChord`	Factor for curvature-based meshing.
For Mesh Points on Surface	

`INT4 MeshPoint[0..149]`
or
`VARIANT vMeshPoint`
	The IDs of points used as mesh points on the surface.

Unpublished work. © 2025 Siemens

Property	Description
For Boundary Surfaces only	

`INT4 BoundaryCurve[0..749]`
 	
or
 	
`VARIANT vBoundaryCurve`
	The IDs of the curves used to define the boundary.

`INT4 BoundaryFlag[0..749]`
 	
or
 	
`VARIANT vBoundaryFlag`
	An identifier corresponding to how each curve is used in the boundary. This flag should be compared against the following values to see if the corresponding conditions exist.
	
	0x00 - curve is used in a forward orientation
	0x01 - curve is used in a reverse orientation
	0x10 - curve is used as the start of a loop
	0x20 - curve is used as the end of a loop
	0x80 - curve is used in a loop that is part of a “hole” in the surface
	
	The outer loop of the boundary always comes first, starts with a curve marked with 0x10 (start of loop), and ends with a curve marked as 0x20 (end of loop). This is followed by any additional loops which define the holes in the surface. There can only be one outer loop.

`INT4 BoundaryMode`	The mapping of the boundary for meshing: 0=Planar/Interpolate Curves, 3=Map to Surface.
`INT4 BoundaryOnSurface`	The ID for the surface to use for mapping if BoundaryMode=3.

Surface Object Methods

Inside

Unpublished work. © 2025 Siemens

Inside	
Insid	
Description:	
This method finds if a parametric location lies inside the specified surface.	
Input:	
`REAL8 u`	The parametric coordinates of the location to be tested.
`REAL8 v`	
Output:	
None	None
Return Code:	
`FE_FAIL`	The location lies outside of the surface (or inside a hole).
Remarks/Usage:	
This method will work for any type of surface except combined boundary surfaces. Use `InsideXYZ` for those surfaces.	
Example:	
None	

InsideXYZ

InsideXYZ	
(xyz)	
Description:	
This method finds if a location lies inside the specified surface.	
Input:	
`REAL8 xyz[0..2]`	The Global Rectangular coordinates of the location to be tested.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	The location lies outside of the surface (or inside a hole).
Remarks/Usage:	
This method will work for any type of surface including combined boundary surfaces.	
Example:	
None	

ParamToXYZ

`ParamToXYZ` **`( u, v, xyz )`**	
Description:	
This method converts a parametric location (u,v) to a coordinate location (xyz).	
Input:	
`REAL8 u`	The parametric coordinates of the location being specified.
`REAL8 v`	
Output:	
`REAL8 xyz[0..2]`	The coordinates of the location in global rectangular.
Return Code:	
`FE_FAIL`	Unable to compute the location.
Remarks/Usage:	
This method will work for any type of surface.	

Unpublished work. © 2025 Siemens

Example:
None

XYZToParam

XYZToParam **(xyz, u, v)**	
Description:	
This method converts a coordinate location (xyz) to a parametric location (u,v).	
Input:	
`REAL8 xyz[0..2]`	The coordinates of the location in global rectangular.
Output:	
`REAL8 u`	The parametric coordinates of the location.
`REAL8 v`	
Return Code:	
`FE_FAIL`	Unable to compute the location.
Remarks/Usage:	
This method will work for any type of surface.	
Example:	
None	

CG

Unpublished work. © 2025 Siemens

CG	
(xyz)	
Description:	
This method finds the center of gravity of a surface.	
Input:	
None	None
Output:	
`REAL8 xyz[0..2]`	The coordinates of the center of gravity, in global rectangular.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will only work for solid surfaces. Furthermore, the surface must be `Put` to the model. The data in the current object is not used.	
Example:	
None	

Area

Area	
(area)	
Description:	
This method finds the area of a surface.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`REAL8 area`	The area of the surface.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will only work for solid surfaces. Furthermore, the surface must be `Put` to the model. The data in the current object is not used.	
Example:	
None	

ApproximateArea

`ApproximateArea` **`( area )`**	
Description:	
This method is similar to `Area( )`, however the area is not computed as precisely to improve performance.	
Input:	
None	None
Output:	
`REAL8 area`	The area of the surface.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database.
`FE_BAD_TYPE`	Surface type is not supported. This method only works with Solid and Combined Surfaces.
`FE_NOT_EXIST`	The surface does not exist.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method will only work for solid surfaces. Furthermore, the surface must be `Put` to the model. The data in the current object is not used.
Example:
None

Tangents

`Tangents`	
`( u, v, uTangent, vTangent )`	
Description:	
This method finds unit vectors in the directions of the tangents to the surface at a selected location.	
Input:	
`REAL8 u`	The parametric coordinates of the location being specified.
`REAL8 v`	
Output:	
`REAL8 uTangent[0..2]`	The tangent vector in the direction of the u parametric space of the surface.
`REAL8 vTangent[0..2]`	The tangent vector in the direction of the v parametric space of the surface.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will work for any surface.	
Example:	
None	

Unpublished work. © 2025 Siemens

Normal

Normal	
(u, v, normal)	
Description:	
This method finds a unit vector in the direction of the surface normal at a selected location.	
Input:	
`REAL8 u`	The parametric coordinates of the location being specified.
`REAL8 v`	
Output:	
`REAL8 normal[0..2]`	The vector in the direction of the surface normal.
Return Code:	
`FE_FAIL`	Unable to compute the normal. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will work for any surface.	
Example:	
None	

NormalAtXYZ

NormalAtXYZ
(nLocations, xyz, normal)
Description:
This method finds a unit vector in the direction of the surface normal at one or more locations.

Unpublished work. © 2025 Siemens

Input:	
`INT4 nLocations`	Number of locations specified in the xyz array.
`REAL8 xyz[0..3*nLocations-1]`	The Global Rectangular coordinates of the locations where you want normals computed. This array is stored [x0,y0,z0,x1,y1,z1...], and must contain `3*nLocations` entries.
Output:	
`REAL8 normal[0..3*nLocations-1]`	The vectors in the direction of the surface normals. Each set of 3 entries in this array correspond to the matching entries in the xyz array, and are the x,y and z vector components of the normal.
Return Code:	
`FE_FAIL`	Unable to compute the normals. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will work for any surface.	
Example:	
None	

ProjectOnto

`ProjectOnto`
`( initial, tolerance, projection )`
Description:
This method projects a location to the closest location that lies on a surface.
Input:

`REAL8 initial[0..2]`	The initial coordinates to be projected onto the surface.
`REAL8 tolerance`	The tolerance within which you will consider the projected location to be on the surface. The method stops when it finds a point that is at least

Unpublished work. © 2025 Siemens

	this close. This tolerance is only used for certain standard surface types.
Output:	
`REAL8 projection[0..2]`	The projected location, on the surface.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will work for any surface, but all boundary surfaces must be `Put` to the database before using this method.	
Example:	
None	

ProjectToClosest

`ProjectToClosest` **`( vOrig, vProj )`**	
Description:	
This method projects a location to the closest location that lies on a surface (i.e., Femap determines vector used for projection).	
Input:	
`REAL8 vOrig [0..2]`	The initial coordinates to be projected onto the surface.
Output:	
`REAL8 vProj [0..2]`	The projected location, on the surface.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database.

Unpublished work. © 2025 Siemens

Remarks/Usage:
Does the same thing as the `ProjectOnto` method, but always uses a Tolerance value of 1.0E-8.
Example:
None

ProjectAlongVector

`ProjectAlongVector` **`( bBothDirections, vAlong, vOrig, vProj )`**	
Description:	
This method projects a location along a specified vector to a location that lies on a surface (i.e., user supplies vector used for projection).	
Input:	
`BOOL bBothDirections`	When True, the location will be projected in both directions along the supplied vector to determine the location to use (i.e., if a location was inside a cylindrical surface, it could actually find 2 locations on the surface, and the one closest to the original location will be used). When False, the location will only be projected in the direction of the supplied vector.
`REAL8 vAlong [0..2]`	The vector supplied by the user to project along.
`REAL8 vOrig [0..2]`	The initial coordinates to be projected onto the surface.
Output:	
`REAL8 vProj [0..2]`	The projected location, on the surface.
Return Code:	
`FE_FAIL`	Unable to compute the location. Possibly wrong surface type, ID is not set, or data is not stored in database. Additionally, if the supplied vector never intersects the surface, no projected location can be determined.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

Solid

`Solid` **`( nSolidID )`**	
Description:	
This method returns the ID of the Solid that contains the current Surface.	
Input:	
None	None
Output:	
`INT4 nSolidID`	The ID of the Solid that references the current Surface.
Return Code:	
`FE_FAIL`	Unable to get the solid.
Remarks/Usage:	
This method will only work with Solid surfaces and Combined Surfaces that are defined on a Solid.	
Example:	
None	

BoundarySurfaces

Unpublished work. © 2025 Siemens

BoundarySurfaces **(numSurf, surfID)**	
Description:	
For a combined surface, this method returns a list of all of the surfaces used to define this surface.	
Input:	
None	None
Output:	
`INT4 numSurf`	The number of surfaces in the *surfID* list.
`INT4 surfID[0..numSurf-1]`	The IDs of the surfaces used to define the Combined Surface.
Return Code:	
`FE_FAIL`	Unable to get the surface list. Possibly data is not stored or this is not a Combined Surface.
`FE_NOT_AVAILABLE`	No surfaces available for this surface.
Remarks/Usage:	
This method will only work with Combined Surfaces.	
Example:	
None	

BoundarySurfacesAsSet

BoundarySurfacesAsSet **(nSetID, bClear)**
Description:
For a combined surface, this method returns a list of all of the surfaces used to define this surface in a Set object.
Input:

Unpublished work. © 2025 Siemens

`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Surfaces.
`BOOL bClear`	If True, the Set is cleared prior to loading the Surface IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the surface list. Possibly data is not stored or this is not a Combined Surface.
`FE_NOT_AVAILABLE`	No surfaces available for this surface.
Remarks/Usage:	
This method will only work with Combined Surfaces.	
Example:	
None	

AdjacentSurfaces

`AdjacentSurfaces` **`( nCombinedMode, numSurf, surfID )`**	
Description:	
This method returns a list of all of the surfaces that are adjacent to the current surface.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are adjacent (`FCC_OFF`), 1=Any surfaces that are adjacent that are in a Combined Surface are replaced by the ID of the Combined Surface (`FCC_ON`), 2=List contains both the underlying and combined surfaces (`FCC_BOTH`).
Output:	
`INT4 numSurf`	The number of surfaces in the *surfID* list.

Unpublished work. © 2025 Siemens

`INT4 surfID[0..numSurf-1]`	The IDs of the adjacent surfaces.
Return Code:	
`FE_FAIL`	Unable to find adjacent surfaces.
Remarks/Usage:	
This method will only work with Solid Surfaces or Combined Surfaces on Solids. It finds all of the curves on the current surface and then finds and returns all of the other surfaces that use one or more of those curves.	
Example:	
None	

AdjacentSurfacesAsSet

`AdjacentSurfacesAsSet` **`( nCombinedMode, nSetID, bClear )`**	
Description:	
This method returns a list of all of the surfaces that are adjacent to the current surface in a Set object.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic surfaces that are adjacent (`FCC_OFF`), 1=Any surfaces that are adjacent that are in a Combined Surface are replaced by the ID of the Combined Surface (`FCC_ON`), 2=List contains both the underlying and combined surfaces (`FCC_BOTH`).
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Surfaces.
`BOOL bClear`	If True, the Set is cleared prior to loading the Surface IDs.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Unable to find adjacent surfaces.
Remarks/Usage:	
This method will only work with Solid Surfaces or Combined Surfaces on Solids. It finds all of the curves on the current surface and then finds and returns all of the other surfaces that use one or more of those curves.	
Example:	
None	

Curves

`Curves` **`( nCombinedMode, numCurves, curveID )`**	
Description:	
This method returns a list of all of the curves used to define this surface.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic curves that are on the surface (`FCC_OFF`), 1=Any curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (`FCC_ON`), 2=List contains both the underlying and combined curves (`FCC_BOTH`).
Output:	
`INT4 numCurves`	The number of curves in the *curveID* list.
`INT4 curveID[0..numCurves-1]`	The IDs of the curves on the surface.
Return Code:	
`FE_FAIL`	Unable to get the curve list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	

Unpublished work. © 2025 Siemens

Example:
None

CurveLoops

CurveLoops **(nCombinedMode, numLoops, numCurves, curveID)**	
Description:	
This method returns a list of all of the curves used to define this surface. Unlike `Curves()` this method returns the curves in their order around each surface loop.	
Input:	
`INT4 nCombinedMode`	0=List contains the basic curves that are on the surface (`FCC_OFF`), 1=Any curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (`FCC_ON`), 2=List contains both the underlying and combined curves (`FCC_BOTH`).
Output:	
`INT4 numLoops`	The number of loops on the surface and in the *curveID* list. A loop is a connected set of curves. Any surface with no interior holes or other features only has one loop. Each hole in the surface adds another loop.
`INT4 numCurves`	The number of entries in the *curveID* list. This is not the number of curves since there is an extra 0 at the end of each loop.
`INT4 curveID[0..numCurves-1]`	The IDs of the curves on the surface. Each loop ends with a 0. If the ID is negative, this indicates that the curve is used in the reverse direction.
Return Code:	
`FE_FAIL`	Unable to get the curve list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

The ID list in curveID list looks like outer c1, outer c2...outer cn, 0, inner1 c1...inner1 cn, 0, ... innerN c1...innerN cn, 0

The 0 entries define the end of a loop. IDs are negative if a curve is used in reverse direction.

This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim feSurf As femap.Surface
   Set feSurf = App.feSurface
   Dim nLoops As Long
   Dim nCurves As Long
   Dim curveIDs As Variant
   Dim bFirstLoop As Boolean
   bFirstLoop = True
   Dim j As Long
   Dim cuSet As femap.Set
   Set cuSet = App.feSet
   While feSurf.Next = femap.FE_OK
       cuSet.Clear()
       bFirstLoop = True
   If feSurf.CurveLoops(FCC_ON, nLoops, nCurves, curveIDs ) = femap.FE_OK
Then
           For j=0 To nCurves -1
                If curveIDs(j) = 0 Then 'end of a loop
                   If bFirstLoop Then
                       Msg = "Outer Loop"
                       App.feAppMessage( FCM_NORMAL, Msg )
```


Unpublished work. © 2025 Siemens

```
                    bFirstLoop = False
                Else
                    Msg = "Inner Loop"
                    App.feAppMessage( FCM_NORMAL, Msg )
                End If
                cuSet.Show( FT_CURVE )
                cuSet.Clear()
            Else
                 If curveIDs(j) > 0 Then
                    cuSet.Add( curveIDs(j) )
                Else
                    cuSet.Add( -curveIDs(j) )
                End If
            End If
        Next
    End If
  Wend
End Sub
```

CurvesAsSet

CurvesAsSet

(nCombinedMode, nSetID, bClear)

Description:

This method returns a list of all of the curves used to define this surface in a Set object.

Input:

Unpublished work. © 2025 Siemens

`INT4 nCombinedMode`	0=List contains the basic curves that are on the surface (`FCC_OFF`), 1=Any curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (`FCC_ON`), 2=List contains both the underlying and combined curves (`FCC_BOTH`).
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Curves.
`BOOL bClear`	If True, the Set is cleared prior to loading the Curve IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the curve list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

Points

`Points` **`( nCombinedMode, numPoints, pointID )`**	
Description:	
This method returns a list of all of the points referenced by the surface.	
Input:	
`INT4 nCombinedMode`	When finding the list of points, this method first finds all curves on the surface, then points on those curves. 0=Uses the basic curves that are on the surface (`FCC_OFF`), 1=Any curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (`FCC_ON`),

Unpublished work. © 2025 Siemens

	2=Uses both the underlying and combined curves (FCC_BOTH).
Output:	
INT4 numPoints	The number of points in the *pointID* list.
INT4 pointID[0..numPoints-1]	The IDs of the points on the surface.
Return Code:	
FE_FAIL	Unable to get the point list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is Put to the database). The ID field must be properly specified.	
Example:	
None	

PointsAsSet

PointsAsSet
(nCombinedMode, nSetID, bClear)
Description:
This method returns a list of all of the points referenced by the surface in a Set object.
Input:

INT4 nCombinedMode	When finding the list of points, this method first finds all curves on the surface, then points on those curves. 0=Uses the basic curves that are on the surface (FCC_OFF), 1=Any curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (FCC_ON), 2=Uses both the underlying and combined curves (FCC_BOTH).
INT4 nSetID	The ID of the Set Object that will contain the IDs of the Points.

Unpublished work. © 2025 Siemens

`BOOL bClear`	If True, the Set is cleared prior to loading the Point IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the point list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified.	
Example:	
None	

EndPoints

`EndPoints` **`( numPoints, pointID )`**	
Description:	
This method returns a list of all of the points referenced by the surface at the ends of the defining curves.	
Input:	
None	None
Output:	
`INT4 numPoints`	The number of points in the pointID list.
`INT4 pointID[0..numPoints-1]`	The IDs of the points on the surface.
Return Code:	
`FE_FAIL`	Unable to get the point list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified. This is just like the `Points` method, except that function will return other points like the points at the center of an arc used to define the surface. This method will only return one pair of points (the endpoints) for each curve.
Example:
None

EndPointsAsSet

`EndPointsAsSet`	
`( nSetID, bClear )`	
Description:	
This method returns a list of all of the points referenced by the surface at the ends of the defining curves in a Set object.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Points.
`BOOL bClear`	If True, the Set is cleared prior to loading the Point IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to get the point list. Possibly data is not stored or ID is not set correctly.
Remarks/Usage:	
This method will work with any surface, but it uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database). The ID field must be properly specified. This is just like the Points method, except that function will return other points like the points at the center of an arc used to define the surface. This method will only return one pair of points (the endpoints) for each curve.	

Unpublished work. © 2025 Siemens

Example:
None

Planar

`Planar`	
`( base, normal )`	
Description:	
This method returns information about planar surfaces.	
Input:	
None	None
Output:	
`REAL8 base[0..2]`	The coordinates of a point on the surface.
`REAL8 normal[0..2]`	The vector that is normal to the plane.
Return Code:	
`FE_FAIL`	Surface is not planar, or could not evaluate the surface.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until they are `Put` to the database). **Note** This method has changed for consistency with other surface methods. Prior to Femap version 9.2 it did not return information about the plane. To access the old functionality, use `IsPlane( )`.	
Example:	
None	

Unpublished work. © 2025 Siemens

Cylindrical

Cylindrical **(radius, base, axis)**	
Description:	
This method returns information about cylindrical surfaces.	
Input:	
None	None
Output:	
`REAL8 radius`	The radius of the cylinder.
`REAL8 base[0..2]`	The coordinates of the center of the base of the cylinder.
`REAL8 axis[0..2]`	The components of the axis vector.
Return Code:	
`FE_FAIL`	Surface is not cylindrical, or could not evaluate the surface.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

Spherical

Unpublished work. © 2025 Siemens

Spherical	
(radius, center)	
Description:	
This method returns information about spherical surfaces	
Input:	
None	None
Output:	
`REAL8 radius`	The radius of the sphere.
`REAL8 center[0..2]`	The coordinates of the center of the sphere.
Return Code:	
`FE_FAIL`	Surface is not spherical, or could not evaluate the surface.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

Conical

Conical	
(radius, angle, base, axis)	
Description:	
This method returns information about conical surfaces.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
`REAL8 radius`	The radius of the cone in the plane of the base location.
`REAL8 angle`	The half angle of the cone.
`REAL8 base[0..2]`	A location along the axis of the cone.
`REAL8 axis[0..2]`	The vector along the axis of the cone.
Return Code:	
`FE_FAIL`	Surface is not conical, or could not evaluate the surface.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

Toroidal

`Toroidal`	
`( radius1, radius2, base, axis )`	
Description:	
This method returns information about toroidal surfaces.	
Input:	
None	None
Output:	
`REAL8 radius1`	The major radius of the toroid.
`REAL8 radius2`	The minor radius of the toroid.

Unpublished work. © 2025 Siemens

`REAL8 base[0..2]`	A location along the axis of the toroid.
`REAL8 axis[0..2]`	The vector along the axis of the toroid.
Return Code:	
`FE_FAIL`	Surface is not toroidal, or could not evaluate the surface.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

IsPlane

`IsPlane` **`( void )`**	
Description:	
This method indicates whether a surface is planar.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Surface is not planar, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a solid surface.
`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.

Unpublished work. © 2025 Siemens

Remarks/Usage:
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).
Example:
None

IsCylinder

`IsCylinder`	
`( void )`	
Description:	
This method indicates whether a surface is cylindrical.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Surface is not cylindrical, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a solid surface.
`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

Unpublished work. © 2025 Siemens

IsSphere

IsSphere	
(void)	
Description:	
This method indicates whether a surface is spherical.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Surface is not shperical, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a solid surface.
`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

IsCone

IsCone
(void)
Description:

Unpublished work. © 2025 Siemens

This method indicates whether a surface is conical.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Surface is not conical, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a solid surface.
`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

IsTorus

`IsTorus`
`( void )`
Description:
This method indicates whether a surface is toroidal.
Input:
None	None
Output:
None	None
Return Code:

Unpublished work. © 2025 Siemens

`FE_FAIL`	Surface is not toroidal, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a solid surface.
`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

IsBlend

`IsBlend` **`( blendtype, dRadius)`**	
Description:	
This method indicates whether a surface is a blend.	
Input:	
None	None
Output:	
`INT4 blendtype`	The Type of blend. 4002=cylinder, 4003=cone, 4004=sphere, 4005= torus, 4006=B-Surface, 4007=blend surface, 4009=swept, 4010=spun.
`REAL8 dRadius`	Blend Radius. For blends with two radii, it is always the smaller of the two.
Return Code:	
`FE_FAIL`	Surface is not a blend, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a solid surface.

Unpublished work. © 2025 Siemens

`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	
Example:	
None	

IsBoundingSolidRegion

`IsBoundingSolidRegion`	
`( void )`	
Description:	
This method indicates whether a surface is part of a solid body definition.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Bounds a solid volume. This will also be returned for any surface which is embedded within the *solid portion* of a general body.
`FE_FAIL`	Does not bound a solid volume.
`FE_BAD_TYPE`	Not a solid surface.
Remarks/Usage:	
This method will only work with solid surfaces, not surfaces created with the Femap Standard geometry engine. It uses the data stored in the database, not any changes made in the current object (until it is `Put` to the database).	

Unpublished work. © 2025 Siemens

Example:
None

Facets

Facets **(numFacets, xyzFacet)**	
Description:	
This method returns the triangular facets used for display of solid surfaces.	
Input:	
None	None
Output:	
`INT4 numFacets`	The number of facets returned.
`REAL8 xyzFacet[0..9*numFacets-1]`	The coordinates of the corners of the facets. Each facet is a triangle, and therefore has 9 coordinates. They are arranged in `xyzFacet` as [X1,Y1,Z1,X2,Y2,Z2,X3,Y3,Z3....] for the first facet, followed by the second and so on.
Return Code:	
`FE_FAIL`	Wrong type of surface, or unable to retrieve facets.
Remarks/Usage:	
This method will only work with solid faces that have facetting information. The surface information must be in the database. Only the ID from the current object is used.	
Example:	
None	

Nodes

Unpublished work. © 2025 Siemens

Nodes

(getBoundary, getInterior, numNodes, nodeID)

Description:

This method returns the nodes that are attached to this surface.

Input:

`BOOL getBoundary`	If True, the nodes attached to the surface boundary (the curves and points) are selected.
`BOOL getInterior`	If True, the nodes attached to the interior of the surface (not on the curves or points) are selected.

Output:

`INT4 numNodes`	The number of nodes returned.
`INT4 nodeID[0..numNodes-1]`	The IDs of the selected nodes which are attached to the surface.

Return Code:

`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the surface in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.

Remarks/Usage:

None

Example:

The following code modifies the color of all nodes on Surface 4.

```
Sub Main()
Dim App As femap.model
Set App = feFemap()

Dim su As Surface
Set su = App.feSurface


Dim nd As Node
Set nd = App.feNode
```


Unpublished work. © 2025 Siemens

```
Dim rc As Long
Dim Count As Long
Dim Nodes As Variant
Dim i As Long

surfID = 4

rc = su.Get (surfID)
rc = su.Nodes(True, True, Count, Nodes)


For i = 0 To Count - 1
   nd.Get (Nodes(i))
   nd.Color = 4
   nd.Put (0)

Next i
App.feViewRegenerate(0)

End Sub
```

NodesAsSet

NodesAsSet

(getBoundary, getInterior, nSetID, bClear)

Description:	
This method returns the nodes that are attached to this surface in a Set object.	
Input:	
`BOOL getBoundary`	If True, the nodes attached to the surface boundary (the curves and points) are selected.
`BOOL getInterior`	If True, the nodes attached to the interior of the surface (not on the curves or points) are selected.
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Nodes.
`BOOL bClear`	If True, the Set is cleared prior to loading the Node IDs.
Output:	
None	None

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to return nodes, possibly none exist.
`FE_NOT_EXIST`	No nodes were attached to the surface in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached nodes.
Remarks/Usage:	
None	
Example:	
None	

Elements

`Elements` **`( numElems, elemID )`**	
Description:	
This method returns the elements that are attached to this surface.	
Input:	
None	None
Output:	
`INT4 numElems`	The number of elements returned.
`INT4 elemID[0..numElems-1]`	The IDs of the selected elements which are attached to the surface.
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the surface in the manner you selected.

Unpublished work. © 2025 Siemens

`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.
Remarks/Usage:	
Only elements that have all of their nodes on the surface will be returned by this method. Elements that simply touch the surface at one edge or one node will not be returned. Similarly, solid elements with one face on the surface will not be returned.	
Example:	
None	

ElementsAsSet

`ElementsAsSet` **`( nSetID, bClear )`**	
Description:	
This method returns the elements that are attached to this surface in a Set object.	
Input:	
`INT4 nSetID`	The ID of the Set Object that will contain the IDs of the Elements.
`BOOL bClear`	If True, the Set is cleared prior to loading the Element IDs.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the surface in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Only elements that have all of their nodes on the surface will be returned by this method. Elements that simply touch the surface at one edge or one node will not be returned. Similarly, solid elements with one face on the surface will not be returned.
Example:
None

ElementsWithFaces

`ElementsWithFaces`	
`( numElems, elemID )`	
Description:	
This method returns the elements that have faces which are attached to this surface.	
Input:	
None	None
Output:	
`INT4 numElems`	The number of elements returned.
`INT4 elemID[0..numElems-1]`	The IDs of the selected elements which have faces that are attached to the surface.
Return Code:	
`FE_FAIL`	Unable to return elements, possibly none exist.
`FE_NOT_EXIST`	No elements were attached to the surface in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate list of attached elements.
Remarks/Usage:	
Only elements that have faces on the surface will be returned by this method. Solid elements with one face on the surface will be returned, as will planar elements meshed on the surface. Other elements that simply touch the surface at a corner, or solid elements touching just along an edge will not be returned.	
Example:	
None	

Unpublished work. © 2025 Siemens

ElementFaces

ElementFaces **(getPlaneElem, getSolidElem, requireElemAttach, numFaces, elemID, faceID)**	
Description:	
This method returns the element faces that are attached to this surface.	
Input:	
`BOOL getPlaneElem`	If True, planar elements with faces on the surface will be returned.
`BOOL getSolidElem`	If True, solid elements with faces on the surface will be returned.
`BOOL requireElemAttach`	If True, planar elements must actually have been meshed on the surface, not just have all of its nodes on the surface.
Output:	
`INT4 numFaces`	The number of elements and faces returned.
`INT4 elemID[0..numFaces-1]`	The IDs of the selected elements which have faces that are attached to the surface.
`INT4 faceID[0..numFaces-1]`	The IDs of the faces of the elements which are on the surface.
Return Code:	
`FE_FAIL`	Unable to return element faces, possibly none exist.
`FE_NOT_EXIST`	No faces were attached to the surface in the manner you selected.
`FE_NO_MEMORY`	Insufficient memory to allocate lists of attached elements/faces.
Remarks/Usage:	
The elemID and faceID arrays are stored in corresponding order . . . `elemID[i]` and `faceID[i]` together represent the element face that is actually attached to the surface.	

Unpublished work. © 2025 Siemens

Example:
None

BoundingBox

BoundingBox	
(Box)	
Description:	
This method returns the coordinates of a rectangular box the encloses this geometry.	
Input:	
None	None
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in Global Rectangular coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
`FE_INVALID`	Object does not contain valid geometry. This method only works for Parasolid-based geometry.
Remarks/Usage:	
This method always retrieves a box that is aligned with Global Rectangular coordinates, not an aligned box of minimum volume.	
Example:	
None	

BoundingBoxInCSys

Unpublished work. © 2025 Siemens

BoundingBoxInCSys **(nCSysID, Box)**	
Description:	
This method returns the coordinates of a rectangular box the encloses this geometry, in coordinates of the selected coordinate system.	
Input:	
`INT4 nCSysID`	ID of coordinate system to return minimum and maximum.
Output:	
`REAL8 Box[0..5]`	The dimensions of the bounding box, in "coordinate system specified with `nCSysID`" coordinates. Box[0] - Minimum X, Box[3] - Maximum X Box[1] - Minimum Y, Box[4] - Maximum Y Box[2] - Minimum Z, Box[5] - Maximum Z
Return Code:	
`FE_FAIL`	Unable to find a bounding box.
`FE_INVALID`	Object does not contain valid geometry. This method only works for Parasolid-based geometry.
Remarks/Usage:	
This method always retrieves a box that is aligned with the selected coordinate system, not an aligned box of minimum volume.	
Example:	
None	

BoundingSize

Unpublished work. © 2025 Siemens

BoundingSize	
(pvDim, pvLongDir, pvNormalDir)	
Description:	
This method finds the sizes of the smallest box that encloses the surface and the direction of the longest side and the shortest side.	
Input:	
None	None
Output:	
`VARIANT pvDim[0..2]`	The dimensions of the bounding box.
`VARIANT pvLongDir[0..2]`	The direction along the longest side.
`VARIANT pvNormalDir[0..2]`	The direction along the shortest side of the box (normal to a planar surface).
Return Code:	
`FE_FAIL`	Unable to compute the size. Possibly wrong surface type, ID is not set, or data is not stored in database.
`FE_INVALID`	Invalid surface type, this method only works with Solid and Combined surfaces.
Remarks/Usage:	
This method will only work for solid surfaces. Furthermore, the surface must be `Put` to the model. The data in the current object is not used.	
Example:	
None	

NormalBox

NormalBox
(box)
Description:

Unpublished work. © 2025 Siemens

This method finds a bounding box around the unit normals.	
Input:	
None	None
Output:	
`REAL8 box[0..5]`	The values of the normal bounding box, in Global Rectangular coordinates. Box[0] - Minimum dX, Box[3] - Maximum dX Box[1] - Minimum dY, Box[4] - Maximum dY Box[2] - Minimum dZ, Box[5] - Maximum dZ
Return Code:	
`FE_FAIL`	Unable to compute the normals. Possibly wrong surface type, ID is not set, or data is not stored in database.
Remarks/Usage:	
This method will work for any surface.	
Example:	
None	

MaxNormalDeviation

`MaxNormalDeviation` **`( dLimit, dAngle )`**	
Description:	
This method the maximum angle between the normal vectors on this surface.	
Input:	
`REAL8 dLimit`	A maximum angle that is acceptable. Normally set this to 0.0 - which means to return any angle variation. If you set this to some other positive number, then the method will fail if the angle is greater than the specified value.
Output:	

Unpublished work. © 2025 Siemens

`REAL8 dAngle`	The maximum angle between normal vectors.
Return Code:	
`FE_FAIL`	The maximum angle exceeded the limit you set (`dAngle` is invalid).
`FE_NOT_AVAILABLE`	Surface facetting information is not available to compute the normals.
`FE_NO_MEMORY`	Not enough memory to load all of the surface facets.
Remarks/Usage:	
This is an approximate method. It uses the normals at the corners of facets, therefore you are not guaranteed that the maximum deviation will not be slightly larger than the value computed.	
Example:	
None	

AngleAtPoint

`AngleAtPoint`
`( nPtID, dAngle, vBisector, vNormal )`
Description:
This method retrieves the internal angle, the surface normal vector, and a surface tangent vector that bisects the angle at a point on a surface boundary.
Input:

`INT4 nPtID`	The ID of the point to check.
Output:	
`REAL8 dAngle`	The angle, in the surface, formed by the edges meeting at the selected point.
`REAL8 vBisector[0..2]`	The surface tangent vector that bisects the angle.
`REAL8 vNormal[0..2]`	The surface normal at the point.

Unpublished work. © 2025 Siemens

Return Code:	
`FE_FAIL`	Unable to determine the requested information.
`FE_NOT_AVAILABLE`	The selected point was not found.
`FE_NO_MEMORY`	Not enough memory to load all of the surface facets.
Remarks/Usage:	
None	
Example:	
None	

HasPole

`HasPole` **`( void )`**	
Description:	
This method indicates whether a surface contains a pole.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Surface does not contain a pole, or could not evaluate the surface.
`FE_BAD_TYPE`	Surface is not a Parasolid surface.
`FE_NOT_AVAILABLE`	Surface object has not been loaded with valid data from the database.
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

MinRadiiOfCurvature

`MinRadiiOfCurvature`
 	
`( pdConcaveRadius, pdConvexRadius, pbIsPlanar )`	

Description:	
This method returns the minimum concave and convex radii of curvature of a surface.	
Input:	
None	None
Output:	
`REAL8 pdConcaveRadius`	The minimum radius of curvature on the concave side of the surface.
`REAL8 pdConvexRadius`	The minimum radius of curvature on the convex side of the surface.
`BOOL pbIsPlanar`	True if the surface is planar.
Return Code:	
None	None
Remarks/Usage:	
Use this method to reset all parameters, then adjust individual properties to suit your needs.	
Example:	
None	

RationalParamToXYZ

Unpublished work. © 2025 Siemens

RationalParamToXYZ	
(u, v, XYZ)	
Description:	
This method converts a parametric location (u,v) to a coordinate location (xyz) using rational uv space.	
Input:	
`REAL8 u`	The rational parametric coordinates of the location being specified. Parametric coordinates are between 0 and 1.
`REAL8 v`	
Output:	
`REAL8 XYZ[0..2]`	The coordinates of the location in global rectangular.
Return Code:	
`FE_FAIL`	Unable to compute the location.
Remarks/Usage:	
None	
Example:	
None	

ResetMeshAttr

ResetMeshAttr	
( )	
Description:	
This method resets the mesh attribute parameters to default values.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
None	None
Remarks/Usage:	
Use this method to reset all parameters, then adjust individual properties to suit your needs.	
Example:	
None	

Mesh

`Mesh` **`( propID, elemShape, bAllowMapped )`**	
Description:	
This method meshes the surface using the current meshing attributes and the values specified.	
Input:	
`INT4 propID`	Specifies the ID of the property to use for elements as they are created. If you want to use predefined meshing attributes, set `propID=0`. In this case, all selected surfaces must have meshing attributes defined or they will not be meshed. The `propID` property must be compatible with surface meshing.
`INT4 elemShape`	The shape of the elements to mesh. Must be 2=Tria3, 3=Tria6, 4=Quad4, 5=Quad8 or set to 0 to use the shape that is predefined on the meshing attributes.
`BOOL bAllowMapped`	If True, mapped meshing of the surfaces is allowed, otherwise it is not. This argument is ignored if `elemShape=0`.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_FAIL`	Could not mesh surfaces.
`FE_INVALID`	The value specified for elemShape is invalid.
`FE_NOT_EXIST`	The property specified does not exist.
`FE_NOT_AVAILABLE`	No surface has been loaded in the current object.

Remarks/Usage:

This method simply calls the application method `feMeshSurface2( )` with the ID of the current surface and without resetting meshing attributes. If you have made changes to the meshing attributes in the current surface object, you must call `Put( )` prior to calling this method or those changes will not be used.

Example:

This example meshes Surface 2 with Parabolic Quad elements that are offset by 0.1:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()
   Dim s As femap.Surface
   Set s = App.feSurface

   s.Get(2)
   s.ResetMeshAttr()
   s.attrOffset = 0.1
   s.attrOffsetFrom = FMOF_TOP
   s.Put(2)
   s.Mesh(1,FTO_QUAD8,True)
End Sub
```

CurvesAtPoint

`CurvesAtPoint`

`( nPtID, PrevCurveID, NextCurveID )`

Description:

This method retrieves the curve before and after a specified point on a surface boundary.

Input:

Unpublished work. © 2025 Siemens

`INT4 nPtID`	The ID of the point to check.
Output:	
`INT4 PrevCurveID`	ID of the curve before `nPtID`.
`INT4 NextCurveID`	ID of the curve after `nPtID`.
Return Code:	
`FE_FAIL`	Point was not associated to surface currently loaded in object.
`FE_NOT_AVAILABLE`	The selected point was not found.
Remarks/Usage:	
None	
Example:	

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Dim feSurf As femap.Surface
    Set feSurf = App.feSurface

    Dim fePoint As femap.Point
    Set fePoint = App.fePoint

    Dim ptSet As femap.Set
    Set ptSet = App.feSet

    Dim cuID1 As Long
    Dim cuID2 As Long

    If feSurf.SelectID( "Pick Suface..." ) =FE_OK Then
        If feSurf.type = FSU_BOUNDARY Then
            If fePoint.SelectID("Pick a Point on this Surface") = FE_OK
Then
                ' Make sure this point is on the surface
                ' Add all the points on the surface selected to ptSet
                ptSet.AddRule( feSurf.ID, FGD_POINT_ONSURFACE )
                If ptSet.IsAdded( fePoint.ID ) Then
                    If feSurf.CurvesAtPoint( fePoint.ID, cuID1, cuID2 ) =
FE_OK Then
                        App.feAppMessage( FCM_NORMAL, "Curve 1 is " + Str$
(cuID1) + " Curve 2 is " + Str$(cuID2 ) )
                    Else
                        App.feAppMessage( FCM_NORMAL, "Cound not find
curves at point..." )
```


Unpublished work. © 2025 Siemens

```
                    End If
                Else
                    App.feAppMessage( FCM_NORMAL, "Point not on surface,
exiting..." )
                End If
            End If
        Else
            App.feAppMessage( FCM_NORMAL, "Only Boundary Surfaces
Supported by this Call" )
        End If
    End If
End Sub
```

CommonCurvesAsSet

CommonCurvesAsSet

(nCombinedMode, nOtherCurveID, nSetID, bClear)

Description:

This method adds all curves which are common between the surface currently in the Surface object and another specified surface to a specified Set object. There are options to clear the set before adding the common curves and whether or not regular curves, combined curves, or both types should be added to the Set object.

Input:

`INT4 nCombinedMode`	When finding common curves, this method limits which curves on the surfaces will be considered. 0=Uses the basic curves that are on the surface (`FCC_OFF`), 1=Any basic curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (`FCC_ON`), 2=Uses both the underlying and combined curves (`FCC_BOTH`).
`INT4 nOtherSurfaceID`	ID of other surface used to determine if there are any common curves.
`INT4 nSetID`	ID of set to add common curves.
`BOOL bClear`	If True, the Set object will be cleared before the common curves are added. If False, the common curves are simply added to the specified Set object.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	No common curves were found between the surfaces.
Remarks/Usage:	
None	
Example:	
None	

FindMappedMeshingCorners

`FindMappedMeshingCorners` **`( nNumDesired, nNumFound, nCornerList )`**	
Description:	
This method returns a list of corner points for used to specify a mapped meshing approach.	
Input:	
`INT4 nNumDesired`	Number of corners to attempt to locate. Should be set to 0 (Automatic), 3, or 4.
Output:	
`INT4 nNumFound`	Number of corners found which may be used to set a mapped meshing approach.
`INT4 nCornerList [0..NumFound-1]`	The IDs of the appropriate corner points, in order, to properly set up a mapped meshing approach.
Return Code:	
None	None
Remarks/Usage:	
None	

Unpublished work. © 2025 Siemens

Example:
None

IsSuppressed

IsSuppressed **(void)**	
Description:	
Indicates whether or not the surface is suppressed.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Surface is suppressed.
`FE_FAIL`	Surface is not suppressed.
Remarks/Usage:	
None	
Example:	
None	

Curvature

Unpublished work. © 2025 Siemens

Curvature	
(vXYZ, pdRadius1, vCurveDir1, pdRadius2, vCurveDir2, vNormDir)	
Description:	
Returns curvature of surface in 2 directions based on specified coordinates.	
Input:	
`REAL8 XYZ[0..2]`	The coordinates of specified location in global rectangular.
Output:	
`REAL pdRadius1`	Radius of curvature in Direction 1.
`REAL8 vCurveDir1[0..2]`	The vector in the direction of Direction 1.
`REAL pdRadius2`	Radius of curvature in Direction 2.
`REAL8 vCurveDir1[0..2]`	The vector in the direction of Direction 2.
`REAL8 XYZ[0..2]`	The vector in the direction of the surface normal.
Return Code:	
`FE_BAD_TYPE`	Surface type is not supported for this method.
`FE_FAIL`	
Remarks/Usage:	
None	
Example:	
None	

IsMappedApproach

Unpublished work. © 2025 Siemens

IsMappedApproach	
(void)	
Description:	
Indicates whether or not the surface has a mapped meshing approach assigned.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Surface has mapped meshing approach assigned.
`FE_FAIL`	Surface does not have mapped meshing approach assigned.
Remarks/Usage:	
None	
Example:	
None	

IsMappable

IsMappable	
(bForceBlands, dMaxBlendRadius, dMergeAngle, pnNumEdges, pnCornerPoints, pdEdgeLength, pnNumEdgeCurves, pnEdgeCurves)	
Description:	
This function attempts to determine if a surface is appropriate for mapped meshing. If so, it provides corners and edges which will be used to apply a mapped mesh.	
Input:	
`BOOL bForceBlends`	If True, the command will see if a *forced* mapped mesh would be possible for a blend (fillet) with a

Unpublished work. © 2025 Siemens

	radius smaller than the `dMaxBlendRadius` value. If False, only a blend surface which fits other criteria for mapped meshing will return `FE_OK`.
`REAL8 dMaxBlendRadius`	Maximum Radius of Blend to consider for *forced* mapped meshing.
`REAL8 dMergeAngle`	Maximum Angle allowed between two curves for the curves to be considered one *edge* for mapped meshing purposes.
Output:	
`INT4 pnNumEdges`	Number of mapped edges, usually 4.
`INT4 pnCornerPoints [0..pnNumEdges - 1]`	Point IDs of the mapped corners, in order, starting with the end point of the first edge.
`REAL8 pdEdgeLength [0..pnNumEdges -1]`	Total length of each mapped edge.
`INT4 pnNumEdgeCurves`	Total number of curves on surface +1 to allow for 0 indicating last curve of last edge.
`INT4 pnEdgeCurves [0..pnNumEdgeCurves-1]`	IDs of the curves making up the edges. `+ID` indicates more curves exist for current edge. `-ID` indicates last curve on an edge. `0` is returned after last curve of last edge.
Return Code:	
`FE_BAD_TYPE`	Surface type is not supported for this method.
`FE_FAIL`	By default, surface is not appropriate for a mapped mesh.
Remarks/Usage:	
If a surface has a mapped meshing approach defined, this command will always return `FE_OK` for the surface.	
Example:	
None	

AddMeshPoint

Unpublished work. © 2025 Siemens

AddMeshPoint	
(nPointID)	
Description:	
Adds a *mesh point* to a surface using the ID of the point.	
Input:	
`INT4 nPointID`	ID of point to add as a mesh point on surface.
Output:	
None	None
Return Code:	
`FE_INVALID`	Max number of available mesh points for the surface has been exceeded.
`FE_FAIL`	Point does not exist.
Remarks/Usage:	
None	
Example:	
None	

RemoveMeshPoint

RemoveMeshPoint	
(nPointID)	
Description:	
Removes a *mesh point* from a surface using the ID of the point.	
Input:	
`INT4 nPointID`	ID of point to remove as a mesh point on surface.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_FAIL`	Point does not exist.
Remarks/Usage:	
None	
Example:	
None	

RemoveAllMeshPoints

`RemoveAllMeshPoints`
`( void )`
Description:
Removes all *mesh points* from a surface using the ID of the point.
Input:

None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

CountMeshPoint

Unpublished work. © 2025 Siemens

CountMeshPoint	
(void)	
Description:	
Returns the number of mesh points currently specified on a surface.	
Input:	
None	None
Output:	
None	None
Return Code:	
`count`	The number of mesh points specified on a surface.
Remarks/Usage:	
None	
Example:	
None	

IsMultiLoop

IsMultiLoop	
(void)	
Description:	
Indicates if a surface has any interior loops (for example, interior holes, slots, etc.).	
Input:	
None	None
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`FE_OK`	Surface has interior loops.
`FE_FAIL`	Surface does not have interior loops.
`FE_INVALID`	Number of loops cannot be properly calculated.
Remarks/Usage:	
None	
Example:	
None	

PointLoops

`PointLoops` **`( nCombinedMode, numLoops, numPoints, PointID )`**	
Description:	
This method returns the number of loops on a surface along with the IDs of the points used for each loop.	
Input:	
`INT4 nCombinedMode`	When finding loops, this method limits which curves on the surfaces will be considered. 0=Uses the basic curves that are on the surface (`FCC_OFF`), 1=Any basic curves that are on the surface that are in a Combined Curve are replaced by the ID of the Combined Curve (`FCC_ON`), 2=Uses both the underlying and combined curves (`FCC_BOTH`).
Output:	
`INT4 numLoops`	Total number of surface loops.
`INT4 numPoints`	Total number of points used by the `loops + numLoops`.

Unpublished work. © 2025 Siemens

`VARIANT PointID`	IDs of the points making up each loop. 0 is returned after the last point of each loop as a separator.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

InitMeshAttributes

`InitMeshAttributes`	
`( nPropID)`	
Description:	
This method initials the mesh attributes of a surface with a specified property.	
Input:	
`INT4 nPropID`	The ID of the property to initialize the mesh attributes of the surface.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

IsFacet

IsFacet	
(void)	
Description:	
Indicates whether or not the surface is faceted surface.	
Input:	
None	None
Output:	
None	None
Return Code:	
`FE_OK`	Surface is a faceted surface.
`FE_FAIL`	Surface is not a faceted surface.
`FE_INVALID`	The current object does not contain a valid Parasolid surface.
Remarks/Usage:	
None	
Example:	
None	

GetAllArray

GetAllArray
(setID, numSurface, entID, exist, surfacetype, layer, color, donotmesh, nInternalToBoundary, divisions, reverse, curves)
Description:

Unpublished work. © 2025 Siemens

This method retrieves arrays of values for a set of surfaces, including if a surface exists in the database.	
Input:	
`INT4 setID`	ID of the Set object that contains the surface IDs to retrieve. If 0, retrieve all surfaces in the model.
Output:	
`INT4 numSurface`	The number of surfaces for which data is being returned.
`INT4 entID[0..numSurface-1]`	The IDs of the surfaces returned.
`BOOL exist[0..numSurface-1]`	True=Surface exists, False=Surface does not exist.
`INT4 surfacetype[0..numSurface-1]`	Type of Surface (0=Bilinear, 1=Ruled, 2=Revolution, 3=Coons, 4=Bezier, 5=Solid, 6=Not Used, 7=Boundary).
`INT4 layer[0..numSurface-1]`	The ID of the layer for the surface.
`INT4 color[0..numSurface-1]`	The color of the curve.
`BOOL donotmesh[0..numSurface-1]`	True=surface is suppressed for meshing, False=surface not suppressed.
`INT4 nInternalToBoundary[0..numSurface-1]`	If nonzero, the ID of a Composite Surface where this surface is used.
`INT4 divisions[0..(2*numSurface)-1]`	The number of divisions in the u and v parametric directions for display purposes.
`BOOL reverse[0..(6*numSurface)-1]`	If True, the surface uses the corresponding curve in a reversed orientation. See Remarks/Usage.
`INT4 curves[0..(6*numSurface)-1]`	The IDs of curves used to define the surface. See Remarks/Usage.
Return Code:	
`FE_FAIL`	Unable to retrieve surface info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve surface info.
Remarks/Usage:	
For Solid surfaces, the reverse and curves arrays are not used or specified, as Solid surfaces cannot be edited by changing curves, therefore all values for *reverse* are False and all values for *curves* are 0.	

Unpublished work. © 2025 Siemens

Example:
None

PutAllArray

PutAllArray **(numSurface, entID, bUpdateLayer, layer, bUpdateColor, color, bUpdateDoNotMesh, donotmesh, bUpdateDivisions, divisions)**	
Description:	
This method updates properties of multiple selected surfaces.	
Input:	
`INT4 numSurface`	The number of surfaces for which data is to be updated.
`INT4 entID[0..numSurface-1]`	The IDs of the surfaces to update.
`BOOL bUpdateLayer[0..numSurface-1]`	If True, update Layer ID.
`INT4 layer[0..numSurface-1]`	The ID of the layer for the surface. Only assigned if `bUpdateLayer=True`.
`BOOL bUpdateColor[0..numSurface-1]`	If True, update Color.
`INT4 color[0..numSurface-1]`	The color of the curve. Only assigned if `bUpdateColor=True`.
`BOOL bUpdateDoNotMesh[0..numSurface-1]`	If True, update donotmesh property.
`BOOL donotmesh[0..numSurface-1]`	True=surface is suppressed for meshing, False=surface not suppressed.
`BOOL bUpdateDivisions[0..numSurface-1]`	If True, update divisions property.
`INT4 divisions[0..(2*numSurface)-1]`	The number of divisions in the u and v parametric directions for display purposes. Only assigned if `bUpdateDivisions = True`.

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store one or more surface with the specified IDs.
`FE_NO_MEMORY`	Unable to allocate memory to update surfaces.
Remarks/Usage:	
None	
Example:	
None	

GetAllAttrArray

`GetAllAttrArray`
`( setID, numSurface, entID, exist, pid, approach, approach_data )`
Description:
This method retrieves arrays of meshing attributes assigned to a set of surfaces.
Input:

`INT4 setID`	ID of the Set object that contains the surface IDs to retrieve. If 0, retrieve all surfaces in the model.
Output:	
`INT4 numSurface`	The number of surfaces for which data is being returned.
`INT4 entID[0..numSurface-1]`	The IDs of the surfaces returned.
`BOOL exist[0..numSurface-1]`	True=Surface exists, False=Surface does not exist.
`INT4 pid[0..numSurface-1]`	The ID of a Property assigned as a meshing attribute.
`INT4 approach[0..numSurface-1]`	The meshing approach for the surface: 0=Not Specified, 1=Free-Parametric, 2=Free-Planar,

Unpublished work. © 2025 Siemens

	3=Mapped-Four Corner, 4=Mapped-Three Corner, 5=Mapped-Three Corner Fan, 6=Link to Surface, 7=Fast Tri-Parametric, 8=Fast Tri-Planar.
`INT4 approach_data[0..(4*numSurface)-1]`	The data for various approaches. For the mapped approaches, these are the point IDs defining the *corners* of the mapped mesh. For the Link to Surface approach, `approachData[0]` is the ID of the surface to which this surface is linked.
Return Code:	
`FE_FAIL`	Unable to retrieve surface info.
`FE_NO_MEMORY`	Unable to allocate memory to retrieve surface info.
Remarks/Usage:	
None	
Example:	
None	

PutAllAttrArray

`PutAllAttrArray`
`( numSurface, entID, bUpdatePID, pid, bUpdateApproach, approach, approach_data )`
Description:
This method updates attributes of multiple selected surfaces.
Input:

`INT4 numSurface`	The number of surfaces for which data is being updated.
`INT4 entID[0..numSurface-1]`	The IDs of the surfaces to update.
`BOOL bUpdatePID[0..numSurface-1]`	If True, update the property meshing attribute.
`INT4 pid[0..numSurface-1]`	The ID of a Property assigned as a meshing attribute.

Unpublished work. © 2025 Siemens

`BOOL bUpdateApproach`	If True, update the approach and approach_data meshing attributes.
`INT4 approach[0..numSurface-1]`	The meshing approach for the surface: 0=Not Specified, 1=Free-Parametric, 2=Free-Planar, 3=Mapped-Four Corner, 4=Mapped-Three Corner, 5=Mapped-Three Corner Fan, 6=Link to Surface, 7=Fast Tri-Parametric, 8=Fast Tri-Planar. Only assigned if `bUpdateApproach=True`.
`INT4 approach_data[0..(4*numSurface)-1]`	The data for various approaches. For the mapped approaches, these are the point IDs defining the "corners" of the mapped mesh. For the Link to Surface approach, approachData[0] is the ID of the surface to which this surface is linked. Only assigned if `bUpdateApproach=True`.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store one or more surface with the specified IDs.
`FE_NO_MEMORY`	Unable to allocate memory to update surfaces.
Remarks/Usage:	
None	
Example:	
None	

HasConsistentSurfaceMesh

`HasConsistentSurfaceMesh`
`( void )`
Description:
This method checks if the surface has a consistent surface mesh.
Input:

Unpublished work. © 2025 Siemens

None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Table Data Objects

Table Data objects correspond to the Tables in your model. They are derived from Entity objects so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Table Data objects. For more information, see **Common Entity Properties and Methods**.

Use the `feTableData` method of the Femap Application object to create Table Data objects.

For an example of creating a new Table Data object, then populating the Table Data object with values, see **SetMultiCellDouble**.

Unpublished work. © 2025 Siemens

Table Data Object Properties

Table Data objects are always stored with a SetID=1 and the ID equal to the Table Data object ID.

Property	Description
STRING title	The table title (maximum 79 characters).
INT4 type	The Type of Data in the Table Data object (0=None, 1=Function, 2=Vector Function, 3=Load Set Combination, 4=Result Set Processing).
INT4 subtype	All of these refer to the same property and specify the type of Function or Vector Function in the Table Data object.
INT4 FunctionType	
INT4 VectorFunctionType	
INT4 Rows	The number of Rows and Columns in the Table Data object. These values are read-only and may only be set using the Initialize and/or Resize methods.
INT4 Columns	

Unpublished work. © 2025 Siemens

Table Data Object Methods

The following methods are available in addition to the standard entity methods.

Unpublished work. © 2025 Siemens

GetFunction

GetFunction **(nFunctionID)**	
Description:	
Loads a function into the current Table object	
Input:	
`INT4 nFunctionID`	The ID of the function to load
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to load the selected function.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutFunction

PutFunction **(nFunctionID)**	
Description:	
Saves the current Table object as a Function.	
Input:	
`INT4 nFunctionID`	The ID of the function to create/update
Output:	
None	None
Return Code:	
`FE_BAD_TYPE`	Either the current Table does not contain a valid Function Type or it does not contain two columns.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

CellRange

CellRange **(nMinRow, nMaxRow, nMinCol, nMaxCol)**	
Description:	
Finds the range of valid cells in the Table currently loaded in the object.	
Input:	
`INT4 nFunctionID`	The ID of the function to create/update
Output:	
`INT4 nMinRow`	The max/min Row and Column indices of the valid cells in the table.
`INT4 nMaxRow`	
`INT4 nMinCol`	
`INT4 nMaxCol`	
Return Code:	
`FE_FAIL`	No valid cells found.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SaveToFile

SaveToFile **(sFileName, bAppend)**	
Description:	
Saves the current table to a comma-separated text file.	
Input:	
`INT4 nFunctionID`	The ID of the function to create/update
Output:	
`STRING sFileName`	The max/min Row and Column indexes of the valid cells in the table.
`BOOL bAppend`	Option to allow appending to an existing file. If False, the file will be overwritten.
Return Code:	
`FE_NO_FILENAME`	Unable to open the specified file.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Initialize

Initialize **(nNumRow, nNumCol)**	
Description:	
Saves the current table to a comma-separated text file.	
Input:	
`INT4 nNumRow`	The number of Rows in the Table.
`INT4 nNumCol`	The number of Columns in the Table.
Output:	
None	
Return Code:	
`FE_OK`	Object is initialized.
Remarks/Usage:	
This method will clear any data/cell that was previously in the Table object.	
Example:	
See **SetMultiCellDouble** for more.	

Unpublished work. © 2025 Siemens

Resize

Resize **(nNumRow, nNumCol)**	
Description:	
Re-sizes an already populated Table object to the newly specified sizes.	
Input:	
`INT4 nNumRow`	The number of Rows in the Table.
`INT4 nNumCol`	The number of Columns in the Table.
Output:	
None	
Return Code:	
`FE_OK`	Object is initialized.
Remarks/Usage:	
Use this method to grow your table beyond the sizes specified in the Initialize method. Using the Resize method does not change the contents of already existing cells.	
Example:	
None	

Unpublished work. © 2025 Siemens

Clear

Clear	
(Void)	
Description:	
Clears the current object.	
Input:	
None	
Output:	
None	
Return Code:	
FE_OK	
Remarks/Usage:	
Calling this method removes all valid cells from the object.	
Example:	
None	

Unpublished work. © 2025 Siemens

SetCellDouble

SetCellDouble **(nRow, nCol, dValue)**	
Description:	
Assign a Real, Double-precision value to a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to update.
`INT4 nCol`	
`REAL8 dValue`	The value assigned to that cell
Output:	
None	
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetCellinteger

SetCellinteger **(nRow, nCol, nValue)**	
Description:	
Assign an integer value to a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to update.
`INT4 nCol`	
`INT4 nValue`	The value assigned to that cell
Output:	
None	
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetCellText

SetCellText **(nRow, nCol, sText)**	
Description:	
Assign a Text String value to a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to update.
`INT4 nCol`	
`STRING sText`	The text assigned to that cell
Output:	
None	
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetCellEquation

SetCellEquation **(nRow, nCol, sEqn)**	
Description:	
Specify a text string that contains an equation to assign to a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to update.
`INT4 nCol`	
`STRING sEqn`	The equation to assign to the cell
Output:	
None	
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetCellEntity

SetCellEntity **(nRow, nCol, nType, nID)**	
Description:	
Specify an Entity to assign to a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to update.
`INT4 nCol`	
`INT4 nType`	The Entity Type to assign to the cell. For entity information, see **Entity Types** .
`INT4 nID`	The ID of the Entity to assign to the cell
Output:	
None	
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetCellDouble

GetCellDouble **(nRow, nCol, dValue)**	
Description:	
Retrieve a Real, Double-precision value from a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to retrieve.
`INT4 nCol`	
Output:	
`REAL8 dValue`	The current value stored in the selected cell
Return Code:	
`FE_FAIL`	The selected cell does not have a value.
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetCellinteger

GetCellinteger **(nRow, nCol, nValue)**	
Description:	
Retrieve a Real, Double-precision value from a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to retrieve.
`INT4 nCol`	
Output:	
`INT4 nValue`	The current value stored in the selected cell
Return Code:	
`FE_FAIL`	The selected cell does not have a value.
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetCellText

GetCellText **(nRow, nCol, sText)**	
Description:	
Retrieve a Text String value from a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to retrieve.
`INT4 nCol`	
Output:	
`STRING sText`	The text string retrieved from that cell
Return Code:	
`FE_FAIL`	The selected cell does not have a value.
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
This method will return a text representation of cells specified with integer or real, double-precision values.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetCellEntity

GetCellEntity **(nRow, nCol, nType, nID)**	
Description:	
Retrieve an Entity from a specified cell.	
Input:	
`INT4 nRow`	The row and column of the cell to update.
`INT4 nCol`	
Output:	
`INT4 nType`	The Entity Type to assign to the cell. For entity information, see **Entity Types** .
`INT4 nID`	The ID of the Entity to assign to the cell
Return Code:	
`FE_FAIL`	The selected cell does not have a value.
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

SetMultiCellDouble

Unpublished work. © 2025 Siemens

SetMultiCellDouble

(nFirstRow, nLastRow, nFirstCol, nLastCol, dValues)

Description:

Assign Real, Double-precision values to a range of specified cells.

Input:

`INT4 nFirstRow`	The range of cells to assign values from the `dValues` array. `nFirstRow` must be less than or equal to `nLastRow` and `nFirstCol` must be less than or equal to `nLastCol`.
`INT4 nLastRow`	
`INT4 nFirstCol`	
`INT4 nLastCol`	
`REAL8 dValues[0..N]`	The Values to assign to the cells in the specified range. This array must contain: `(nLastRow-nFirstRow+1)*(nLastCol-nFirstCol+1)` values.

Output:

None	None

Return Code:

`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.

Remarks/Usage:

None

Example:

```
Sub Main
   Dim App As femap.model
   Set App = feFemap()

   Dim t As femap.TableData
```


Unpublished work. © 2025 Siemens

```
Set t = App.feTableData

 ' If creating a table, call this first to setup number of rows
 ' and columns
 ' ...it will clear all other data
t.Initialize( 3, 4 )

 ' Setup parameters after initializing
t.type = FTBL_VECTORFUNCTION
t.VectorFunctionType = FTBV_ACCELERATION
t.title = "This is a Vector Function"

 ' Create some data...these are simply demo variables that
 ' would never be valid
Dim v(11) As Double
For i=0 To 11
    v(i) = i
Next

 ' Load data into table...array must contain data in the
 ' order...
 ' (Row0,Col0), (Row0,Col1), (Row0,Col2)...
 ' (Row1,Col0),(Row1,Col1)......(RowN,ColN)
rc = t.SetMultiCellDouble( 0, 2, 0, 3, v )

 'Save the Table
t.Put(1)
```


Unpublished work. © 2025 Siemens

```
End Sub
```


Unpublished work. © 2025 Siemens

SetMultiCellInteger

SetMultiCellInteger **(nFirstRow, nLastRow, nFirstCol, nLastCol, nValues)**	
Description:	
Assign Integer values to a range of specified cells.	
Input:	
`INT4 nFirstRow`	The range of cells to assign values from the `dValues` array. `nFirstRow` must be less than or equal to `nLastRow` and `nFirstCol` must be less than or equal to `nLastCol`.
`INT4 nLastRow`	
`INT4 nFirstCol`	
`INT4 nLastCol`	
`REAL8 dValues[0..N]`	The Values to assign to the cells in the specified range. This array must contain: `(nLastRow-nFirstRow+1)*(nLastCol-nFirstCol+1)` values.
Output:	
None	None
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetMultiCellText

SetMultiCellText **(nFirstRow, nLastRow, nFirstCol, nLastCol, sTextValues)**	
Description:	
Assign text string values to a range of specified cells.	
Input:	
`INT4 nFirstRow`	The range of cells to assign values from the `dValues` array. `nFirstRow` must be less than or equal to `nLastRow` and `nFirstCol` must be less than or equal to `nLastCol`.
`INT4 nLastRow`	
`INT4 nFirstCol`	
`INT4 nLastCol`	
`STRING sTextValues[0..N]`	The text string values to assign to the cells in the specified range. This array must contain: `(nLastRow-nFirstRow+1)*(nLastCol-nFirstCol+1)` values.
Output:	
None	None
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

GetMultiCellDouble

GetMultiCellDouble **(nFirstRow, nLastRow, nFirstCol, nLastCol, dValues)**	
Description:	
Retrieve Real, Double-precision values from a range of specified cells.	
Input:	
`INT4 nFirstRow`	The range of cells to assign values from the `dValues` array. `nFirstRow` must be less than or equal to `nLastRow` and `nFirstCol` must be less than or equal to `nLastCol`.
`INT4 nLastRow`	
`INT4 nFirstCol`	
`INT4 nLastCol`	
Output:	
`REAL8 dValues[0..N]`	The Values retrieved from the cells in the specified range. This array will contain: `(nLastRow-nFirstRow+1)*(nLastCol-nFirstCol+1)` values
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
Cells that are in the valid range, but do not have a specified value will return a value of 0.0. Querying these cells will not cause an error in the Return Code.	
Example:	
None	

Unpublished work. © 2025 Siemens

GetMultiCellInteger

GetMultiCellInteger **(nFirstRow, nLastRow, nFirstCol, nLastCol, dValues)**	
Description:	
Retrieve Integer values from a range of specified cells.	
Input:	
`INT4 nFirstRow`	The range of cells to assign values from the `dValues` array. `nFirstRow` must be less than or equal to `nLastRow` and `nFirstCol` must be less than or equal to `nLastCol`.
`INT4 nLastRow`	
`INT4 nFirstCol`	
`INT4 nLastCol`	
Output:	
`REAL8 nValues[0..N]`	The Values retrieved from the cells in the specified range. This array will contain: `(nLastRow-nFirstRow+1)*(nLastCol-nFirstCol+1)` values
Return Code:	
`FE_INVALID`	The object has not been initialized.
`FE_NOT_AVAILABLE`	An invalid row/column has been specified.
Remarks/Usage:	
Cells that are in the valid range, but do not have a specified value will return a value of 0.0. Querying these cells will not cause an error in the Return Code.	
Example:	
None	

Unpublished work. © 2025 Siemens

Text Objects

Text objects correspond to the text entities in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for text. For more information, see **Common Entity Properties and Methods**.

Use the `feText` method of the Femap Application object to create Text objects.

Text Object Properties

Text entities are always stored with a `SetID=1`, and the ID equal to the Text ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description
`BOOL ModelPosition`	Method for positioning text (True=Model, False=Screen).
`BOOL AllViews`	Indicates if the text is visible in all Views - if False, only visible in *VisibleView*.
`BOOL DrawPointer`	If True, a pointer is drawn in addition to the text.
`BOOL PointerIs3D`	If True, point is in Model space, if False pointer is in View space
`BOOL DrawBorder`	If True, a border is drawn around the text.
`INT4 VisibleView`	The ID of a view where the text is visible. Only used if `AllViews`=False.
`INT4 layer`	The layer associated with the text.
`INT4 color`	The color of the text.
`INT4 BorderColor`	The color of the border drawn around the text. Only used if `DrawBorder`=True.
`INT4 BackColor`	The background color of the text. Only used if the `DrawBorder`=True.
`REAL8 TextPosition[0..2]`	The coordinates of the text. If `ModelPosition`=True, these are model

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vTextPosition`	coordinates in the Global Rectangular CSys. If `ModelPosition`=False, these are specified as a percent of the view location and `TextPosition[2]` is not used.

`REAL8 PointerPosition[0..2]`
 	
or
 	
`VARIANT vPointerPosition`
	The location of the tip of the pointer, specified in the same manner as TextPosition. Only used if `DrawPointer`=True.

`INT4 HorzJustify`	The horizontal justification method.
`INT4 VertJustify`	The vertical justification method.
`INT4 FontNumber`	The ID of the font for the text.
`STRING text`	The actual text string (limited to 511 characters). Contains carriage return and line-feed characters between lines of text.

Text Object Methods

GetFontData

`GetFontData` **`( pdPointHeight, pbBold, pbItalic, ppcName )`**	
Description:	
This function returns parameters from Text objects. Requires a `Get` before use.	
Input:	
None	None
Output:	

Unpublished work. © 2025 Siemens

`REAL8 pdPointHeight`	The height of the text.
`BOOL pbBold`	If True if text is Bold, otherwise returns False.
`BOOL pbItalic`	If True if text is Italicized, otherwise returns False.
`STRING ppcName`	Name of Font.
Return Code:	
`FE_OK`	Text object values retrieved.
Remarks/Usage:	
None	
Example:	
None	

SetFontData

`SetFontData`	
`( pdPointHeight, pbBold, pbItalic, ppcName )`	
Description:	
This function sets parameters for Text objects. Requires a `Put` after use to update Text object.	
Input:	
`REAL8 pdPointHeight`	The height of the text.
`BOOL pbBold`	If True if text is Bold. If False text is not Bold.
`BOOL pbItalic`	If True if text is Italicized. If False, text is not Italicized.
`STRING ppcName`	Name of Font.
Output:	

Unpublished work. © 2025 Siemens

None	None
Return Code:	
`FE_OK`	Text object values set.
Remarks/Usage:	
None	
Example:	
None	

TMGBC Objects

TMGBC objects correspond to the TMG-defined boundary condition data in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for TMBBC Objects. For more information, see **Common Entity Properties and Methods**.

Use the `feTMGBC` method of the Femap Application object to create TMGBC objects.

TMGBC Object Properties

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`INT4 i[0..263]`
 	
or
 	
`VARIANT vi`
	Integer array.

`REAL8 r[0..263]`
 	
or
 	
`VARIANT vr`
	Real array.

Unpublished work. © 2025 Siemens

Property	Description

`BOOL b[0..532]`
 	
or
 	
`VARIANT vb`
	Boolean array.

`INT4 c[0..242]`
 	
or
 	
`VARIANT vc`
	Character array, each entry is returned and accessed via an `INT4`.

TMGBC Object Methods

GetAll

`GetAll` **`( entID,  i, r, c )`**	
Description:	
This method is obsolete, therefore, `GetAll2` should be used instead. This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i[0..119]`	Notice, the size of the most of the arrays are smaller than in the property definition. `c` is returned in an integer array, even though it contains only char values.
`REAL8 r[0..73]`	

Unpublished work. © 2025 Siemens

`INT4 c[0..242]`	
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
Should no longer be used.	
Example:	
None	

PutAll

`PutAll` **`( entID,  i, r, c )`**	
Description:	
This method is obsolete, therefore, `PutAll2` should be used instead. This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i[0..119]`	Notice, the size of the most of the arrays are smaller than in the property definition. `c` is loaded in an integer array, even though it contains only char values.
`REAL8 r[0..73]`	
`INT4 c[0..242]`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	

Unpublished work. © 2025 Siemens

Should no longer be used.
Example:
None

GetAll2

GetAll2 **(entID, i, r, b, c)**	
Description:	
This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i`	Refer to the property descriptions for each of these items. `c` is returned in an integer array, even though it contains only char values.
`REAL8 r`	
`BOOL b`	
`INT4 c`	
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutAll2

PutAll2 **(entID, i, r, b, c)**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i`	Refer to the property descriptions for each of these items. `c` is loaded in an integer array, even though it contains only char values.
`REAL8 r`	
`BOOL b`	
`INT4 c`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
None	
Example:	
None	

TMGCtrl Objects

TMGCtrl objects correspond to the TMG-defined control data in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for TMGCtrl objects. For more information, see **Common Entity Properties and Methods**.

Unpublished work. © 2025 Siemens

Use the `feTMGCtrl` method of the Femap Application object to create TMGCtrl objects.

TMGCtrl Object Properties

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`INT4 i[0..167]`
or
`VARIANT vi`
	Integer array.

`REAL8 r[0..63]`
or
`VARIANT vr`
	Real array.

`BOOL b[0..297]`
or
`VARIANT vb`
	Boolean array.

`INT4 c[0..485]`
or
`VARIANT vc`
	Character array, each entry is returned and accessed via an `INT4`.

TMGCtrl Object Methods

Unpublished work. © 2025 Siemens

GetAll

GetAll **(entID, i, r, b, c)**	
Description:	
This method is obsolete, therefore, `GetAll2` should be used instead. This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i[0..53]`	Notice, the size of the most of the arrays are smaller than in the property definition. `c` is returned in an integer array, even though it contains only char values.
`REAL8 r[0..19]`	
`BOOL b[0..121]`	
`INT4 c[0..485]`	
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
Should no longer be used.	
Example:	
None	

PutAll

Unpublished work. © 2025 Siemens

PutAll **(entID, i, r, b, c)**	
Description:	
This method is obsolete, therefore, `PutAll2` should be used instead. This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i[0..53]`	Notice, the size of the most of the arrays are smaller than in the property definition.. `c` is loaded in an integer array, even though it contains only char values.
`REAL8 r[0..19]`	
`BOOL b[0..121]`	
`INT4 c[0..485]`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
Should no longer be used.	
Example:	
None	

GetAll2

Unpublished work. © 2025 Siemens

GetAll2	
(entID, i, r, b, c)	
Description:	
This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i`	Refer to the property descriptions for each of these items. `c` is returned in an integer array, even though it contains only char values.
`REAL8 r`	
`BOOL b`	
`INT4 c`	
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
None	
Example:	
None	

PutAll2

PutAll2	
(entID, i, r, b, c)	
Description:	
This method stores an entity with the specified ID.	

Unpublished work. © 2025 Siemens

Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i`	Refer to the property descriptions for each of these items. `c` is loaded in an integer array, even though it contains only char values.
`REAL8 r`	
`BOOL b`	
`INT4 c`	
None	None
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
None	
Example:	
None	

TMGInt Objects

TMGInt objects correspond to the TMG-defined integer data in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for TmbInt objects. For more information, see **Common Entity Properties and Methods**.

Use the `feTMGInt` method of the Femap Application object to create TMGInt Objects.

TMGInt Object Properties

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description

`INT4 i[0..49]`
 	
or
 	
`VARIANT vi`
	Integer array.

TMGInt Object Methods

GetAll

`GetAll` **`( entID,  i )`**	
Description:	
This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i`	Refer to the property descriptions for each of these items.
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutAll

PutAll **(entID, i)**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i`	Refer to the property descriptions for each of these items. `c` is loaded in an integer array, even though it contains only char values.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
None	
Example:	
None	

TMGOpt Objects

TMGOpt objects correspond to the TMG-defined option data in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for TMGOpt objects. For more information, see **Common Entity Properties and Methods**.

Use the `feTMGOpt` method of the Femap Application object to create TMGOpt objects.

Unpublished work. © 2025 Siemens

TMGOpt Object Properties

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`INT4 i[0..16]`
 	
or
 	
`VARIANT vi`
	Integer array.

`REAL8 r[0..1]`
 	
or
 	
`VARIANT vr`
	Real array.

`INT4 c[0..84]`
 	
or
 	
`VARIANT vc`
	Character array, each entry is returned and accessed via an `INT4`.

TMGOpt Object Methods

GetAll

`GetAll` **`( entID,  i, r, c )`**
Description:

Unpublished work. © 2025 Siemens

This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i`	Refer to the property descriptions for each of these items. `c` is returned in an integer array, even though it contains only char values.
`REAL8 r`	
`INT4 c`	
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
None	
Example:	
None	

PutAll

`PutAll` **`( entID,  i, r, c )`**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i`	Refer to the property descriptions for each of these items. `c` is loaded in an integer array, even though it contains only char values.
`REAL8 r`	

Unpublished work. © 2025 Siemens

`INT4 c`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
None	
Example:	
None	

TMGReal Objects

TMGReal objects correspond to the TMG-defined real data in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for TMGReal objects. For more information, see **Common Entity Properties and Methods**.

Use the `feTMGReal` method of the Femap Application object to create TMGReal objects.

TMGReal Object Properties

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Property	Description

`INT4 i[0..11]`
 	
or
 	
`VARIANT vi`
	Integer array.

`REAL8 r[0..55]`	Real array.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vr`	

TMGReal Object Methods

GetAll

`GetAll` **`( entID,  i, r )`**	
Description:	
This method retrieves an entity with the specified ID.	
Input:	
`INT4 entID`	ID of the entity to retrieve.
Output:	
`INT4 i`	Refer to the property descriptions for each of these items.
`REAL8 r`	
Return Code:	
`FE_FAIL`	The specified entity does not exist.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

PutAll

PutAll **(entID, i, r)**	
Description:	
This method stores an entity with the specified ID.	
Input:	
`INT4 entID`	The ID of the entity to store.
`INT4 i`	Refer to the property descriptions for each of these items.
`REAL8 r`	
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to store the entity with the specified ID.
Remarks/Usage:	
None	
Example:	
None	

Variable Objects

Variable objects correspond to the variables in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Variable objects. For more information, see **Common Entity Properties and Methods**.

Use the `feVar` method of the Femap Application object to create Variable objects.

Unpublished work. © 2025 Siemens

Variable Object Properties

Variables are always stored with a `SetID=1`, and the ID equal to an encoded value that is derived from the variable name. You can use the `GetVarID` method described below to find the ID.

If you use the properties to define a variable, it is your responsibility to update all of them in a compatible fashion. You cannot simply specify one or two of them and leave the rest blank. In addition, `VarName` must be defined to match the ID that you pass to `Put`. It is often easier to use the `Define` method below to create variables, and simply use the properties to retrieve their values.

Property	Description
`REAL8 Value`	The stored value of the variable.
`STRING VarName`	The name of the variable. Maximum 20 characters.
`STRING Equation`	The defining equation of the variable. Maximum 160 characters.

Variable Object Methods

GetVarID

`GetVarID` **`( varname )`**	
Description:	
This method retrieves the encoded ID that is associated with a specified variable name.	
Input:	
`STRING varname`	The name of the variable to compute the associated ID.
Output:	
None	None
Return Code:	

Unpublished work. © 2025 Siemens

`INT4 ID`	The ID associated with the name.
Remarks/Usage:	
You do not need to have a variable with the specified name to use this function.	
Example:	
None	

Evaluate

`Evaluate` **`( void )`**	
Description:	
This method evaluates the equation stored in the current Equation property. The result is returned, and used to update the Value property.	
Input:	
None	None
Output:	
None	None
Return Code:	
`REAL8 value`	The value of the equation property. Also stored in Value.
Remarks/Usage:	
None	
Example:	
None	

Define

Unpublished work. © 2025 Siemens

Define	
(varname, equation)	
Description:	
This method evaluates the equation stored in the current Equation property. The result is returned, and used to update the Value property.	
Input:	
`STRING varname`	The variable name to create or update. Maximum 20 characters.
`STRING equation`	The equation used to define the variable. Maximum 160 characters.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This method automatically computes the value and sets all properties for the variable. In addition, the variable is `Put` to the database so it is permanently retained.	
Example:	
None	

DefineReal

DefineReal	
(varname, value)	
Description:	
This method defines a variable and its associated real number.	
Input:	
`STRING varname`	The variable name to create or update. Maximum 20 characters.

Unpublished work. © 2025 Siemens

`REAL8 value`	The value to assigned to the variable.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Vector Objects

Vector objects correspond to defined vectors which can be stored in the database for future use in commands. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for Vector objects. For more information, see **Common Entity Properties and Methods**.

Use the `feVector` method of the Femap Application object to create Vector objects.

Vector Object Properties

Vectors are always stored with a `SetID=1`, and the ID equal to the reference ID.

Property	Description
`STRING title`	Title of saved vector (Maximum 79 Characters).

`REAL8 base[0..2]`
 	
or
 	
`VARIANT vBase`
	Vector base location in global rectangular coordinates.

`REAL8 dir[0..2]`	Components of a vector defining the vector direction.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vDir`	
`REAL8 length`	Length of vector.

Vector Object Methods

PutAll

`PutAll` **`( ID, sTitle, vBase, vDir, dLen  )`**	
Description:	
This method saves a vector to the database with all the available options using a single call.	
Input:	
`INT4 ID`	ID of vector.
`STRING title`	Title of vector (Maximum 79 Characters).
`REAL8 vBase[0..2]`	Vector base location in global rectangular coordinates.
`REAL8 vDir[0..2]`	Components of a vector defining the vector direction.
`REAL8 dLen`	Length of vector.
Output:	
None	None
Return Code:	
`FE_FAIL`	Unable to save vector.

Unpublished work. © 2025 Siemens

Remarks/Usage:
None
Example:
None

View Objects

View objects correspond to the views in your model. They are derived from Entity objects, so in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for View objects. For more information, see **Common Entity Properties and Methods**.

Use the `feView` method of the Femap Application object to create View objects.

All model orientation in Femap version 11.3 and above should be done using the View Orientation (ViewOrient) Object. For backwards compatibility, the orientation in the View Object may be used, as the `Get` function loads the orientation from the ViewOrient Object into the View Object and the `Put` function saves the orientation from the View Object in to the ViewOrient object.

For Femap version 2020.1 and above, control of the display of digits for real numbers in the graphics window is now controlled by properties with this format: `DO_(View Option)_(Digits Option)`.

There are some limitations:

1. Rotations should be applied using the `Rotate` method and not by direct access to the Rotation property data. Some access to the Rotation property data will work (for example, the Snap to Closest Orthogonal View API delivered with Femap, where all 3 values are set together).
2. You should be careful when accessing a View Object and a ViewOrient object at the same time as the backwards compatibility support in the `Put` function of the View Object will override data in the ViewOrient Object. If accessing both, the `Put` for the ViewOrient Object should always be done after the `Put` on the View Object.

View Object Properties

Views are always stored with a `SetID=1`, and the ID equal to the view ID.

In this table, you will see some properties that list two names - one of which is an array, the other a Variant. Refer to **Multiple Definitions of Array Properties** for more information.

Unpublished work. © 2025 Siemens

Property	Description
`STRING title`	View title (maximum 79 characters).
`INT4 Mode`	Drawing style: 0=Draw, 1=Feature, 2=Quick Hide, 3=Hide, 4=Free Edge, 5=Free Face. Should only be used when Enable Legacy XY Plotting option is turned on in the User Interface tab of **File** → `Preferences`: 6=XYvsID, 7=XYvsSET, 8=XYvsVALUE, 9=XYvs-POSITION, 10=XY of Function.

`REAL8 ViewLegendLocation[0..1]`
 	
or
 	
`VARIANT vViewLegendLocation`
	The X and Y positions are the relative distances, as a percentage of overall screen size, from the upper left hand corner of the *view legend area* to the upper left hand corner of the view where the view legend is being displayed. For instance, a value of X = 0 and Y = 0 positions the view legend as far to the left and as close to the top of the view as possible, while X = 100 and Y = 100 positions the view legend as for to the right and as close to the bottom as possible.

`INT4 ViewLegendJustification`	Justification option: 0=Left Justification, 1=Right Justification, 2=Center Justification, 3=Single Line.
`BOOL ViewLegendLockJustification`	Justification is locked (True) or free to change as the view legend is moved around the graphics window (False).
`INT4 Deformed`	Deformed style: 0=Off, 1=Deformed, 2=Animate, 3=Animate Multi-Case, 4=Arrow, 5=Trace, 6=Streamline.
`INT4 Contour`	Contour style: 0=Off, 1=Contour, 2=Criteria, 3=Beam Diagram, 4=IsoSurface, 5=Section Cut, 6=Contour Arrow.
`BOOL UndefAnimation`	True if undeformed animation.
`INT4 BC`	Constraint SetID, or -1 for active.
`INT4 Load`	Load SetID, or -1 for active.
`INT4 Group`	Group ID, -1 for active group, -2 for multiple groups, and 0 for all groups.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 LabelGroup`	Label Group ID, -1 for active group, and 0 for all groups.
`INT4 ContourGroup`	ID of Group containing Elements to be used for various contour style displays. -1 for active.
`INT4 bClear`	If True then clear the list of groups in the multi group display before setting the Hide/Show group IDs specified in Set Objects with IDs `nShowSetID` and `nHideSetID`.
`INT4 nShowSetID`	Set Object containing Group IDs to be set as Show in the Multi Group Display.
`INT4 nHideSetID`	Set Object containing Group IDs to be set as Hide in the Multi Group Display.
`INT4 OutputSet`	Output Set ID for post.
`INT4 FinalOutputSet`	Final Output Set ID for multi-case animation.
`INT4 OutputSetInc`	Output Set increment for multi-case animation.
`INT4 ContourVector1V2`	This property replaces `ContourVector1`. For more information see, **Working with Output in Femap v2020.1 and above**. First Output Vector ID in Select Multiple Contour Vectors dialog box (Same as ContourData).
`INT4 ContourVector2V2`	This property replaces `ContourVector2`. For more information see, **Working with Output in Femap v2020.1 and above**. Second Output Vector ID in Select Multiple Contour Vectors dialog box.
`INT4 ContourVector3V2`	This property replaces `ContourVector3`. For more information see, **Working with Output in Femap v2020.1 and above**. Third Output Vector ID in Select Multiple Contour Vectors dialog box.
`INT4 ContourDataV2`	This property replaces `ContourData`. For more information see, **Working with Output in Femap v2020.1 and above**.

Unpublished work. © 2025 Siemens

Property	Description
	Output Vector ID for contour.
`INT4 ContourVecAddlV2`	This property replaces `ContourVecAddl`. For more information see, **Working with Output in Femap v2020.1 and above**. Additional contour vector output ID. Same as ContourVector2.
`INT4 DeformDataV2`	This property replaces `DeformData`. For more information see, **Working with Output in Femap v2020.1 and above**. Output Vector ID for deformation.

`REAL8 PostTitlesLocation[0..1]`
 	
or
 	
`VARIANT vPostTitlesLocation`
	The X and Y positions are the relative distances, as a percentage of overall screen size, from the upper left hand corner of the *post titles area* to the upper left hand corner of the view where the post titles are being displayed. For instance, a value of X = 0 and Y = 0 positions the post titles as far to the left and as close to the top of the view as possible, while X = 100 and Y = 100 positions the post titles as for to the right and as close to the bottom as possible.

`INT4 PostTitlesJustification`	Justification option: 0=Left Justification, 1=Right Justification, 2=Center Justification, 3=Single Line.
`BOOL PostTitlesLockJustification`	Justification is locked (True) or free to change as the post titles are moved around the graphics window (False).

`REAL8 Rotation[0..2]`
 	
or
 	
`VARIANT vRotation`
	Rotation angles (See section **View Objects** for more information about specifying orientation)

`REAL8 PrevRotation[0..2]`
 	
or
 	
`VARIANT vPrevRotation`
	Previous rotation angles (ee **View Objects** for more information about specifying orientation).

Unpublished work. © 2025 Siemens

Property	Description

`REAL8 Center[0..2]`
 	
or
 	
`VARIANT vCenter`
	Location of view center (See **View Objects** for more information about specifying orientation).

`REAL8 Magnification`	Magnification factor (1=`AutoScale`).
`REAL8 Aspect`	View aspect ratio.
`BOOL RotationCenterOn`	Flag to use rotation center for dynamic rotation.

`REAL8 RotationCenter[0..2]`
 	
or
 	
`VARIANT vRotationCenter`
	Center of rotation for dynamic rotation.

`REAL8 RotationVector[0..2]`
 	
or
 	
`VARIANT vRotationVector`
	Axis of rotation for dynamic rotation.

`REAL8 PrevCenter[0..2]`
 	
or
 	
`VARIANT vPrevCenter`
	Previous center location.

`REAL8 PrevMagnification`	Previous magnification factor.
`BOOL ShowAllLayers`	Flag to display all layers.
`BOOL WindowTitleBar`	Flag to indicate title bar is on.
`INT4 WindowBorder`	Border mode.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL WindowEraseBack`	True if background should be erased.
`INT4 WindowBackColor`	Background Color ID.
`BOOL WindowShadeBack`	True if background should be shaded using second background color.
`INT4 WindowShadeBackOpt`	0=Vertical, 1=Horizontal, 2=Diagonal Up, 3=Diagonal Down, 4=Square Spot, 5=Circular Spot, 6=Horizon, 7=Bitmap, 8=Stretched Bitmap.
`INT4 WindowBackColor2`	Second Background Color ID - only used in render with `WindowShadeBack`=True.
`INT4 WindowBackColor3`	Third Background Color ID - only used in render with `WindowShadeBack`=True and `WindowShadeBackOpt`=6
`INT4 WindowShadeHorizon`	The location of the background horizon in percent of screen height.
`BOOL WindowLogo`	If True, and a logo bitmap is defined, it will be shown in this view.

`REAL8 LogoLocation[0..2]`
or
`VARIANT vLogoLocation`
	The location of the logo bitmap if it is shown in the view. In percent of screen.

`BOOL Draw[0..121]`
or
`VARIANT vDraw`
	View Option Draw flags.

`INT4 Label[0..121]`
or
`VARIANT vLabel`
	View Option Labeling.

`INT4 ColorMode[0..121]`	View Option Color Modes.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vColorMode`	
`INT4 ElementNoResultMode`	Controls Elements with no results option mode for Contour/Criteria Style. 0=Show, 1=Hide (additional options planned for future releases).

`INT4 Color[0..121]`
 	
or
 	
`VARIANT vColor`
	View Option View Color.

(Obsolete) `REAL8 PerspectiveDist`	Perspective distance (Use `PerspectiveAngle` instead).
`REAL8 PerspectiveAngle`	Specifies the Perspective Angle, which sets the value in the Angle field for the Perspective option of the **View** → **Options** command.
`REAL8 InterOccularDist`	Perspective separation.
`REAL8 ShrinkFactor`	Element shrink factor.
`REAL8 FeatureLineAngle`	Angle used to control where feature lines are displayed.
(Obsolete) `REAL8 CurveError`	Use Global `Info_FacetXXX` properties.
`REAL8 AmbientLight`	Ambient light value.

`REAL8 Light[0..2]`
 	
or
 	
`VARIANT vLight`
	Location of light source when Light Location/Type is set to Viewer, Viewer Spot, Model, or Model Spot.

`REAL8 SpotlightDist`	Spotlight distance.
`REAL8 SpotlightDir[0..2]`	Location of Light in Screen Coordinates when Light Location/Type is Screen or Screen Spot.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vSpotlightDir`	

`REAL8 AxisLocation[0..1]`
 	
or
 	
`VARIANT vAxisLocation`
	Location of screen axes.

`REAL8 ViewAxesSize`	Size of screen axis as a percentage.
`REAL8 ArrowsToLabel`	Percent of post arrows to label.
(Obsolete) `INT4 SurfaceDivisions`	Use surface faceting.
`REAL8 LoadVecLength`	Length of load vectors.
`REAL8 OtherVecLength`	Length of other vectors
`REAL8 LoadVectorMinScale`	Load vector minimum scale factor.
`BOOL CurveConnectWirebody`	If True and Curve Connectivity option is turned on, Wirebody entity curves will be displayed using the specified color by `CurveConnectWirebodyColor`.
`INT4 CurveConnectWirebodyColor`	Display color for Wirebody entity curves when Curve Connectivity option is turned on.
`BOOL CurveConnectFree`	If True and Curve Connectivity option is turned on, Free curves on surfaces will be displayed using the specified color by `CurveConnectFreeColor`.
`INT4 CurveConnectFreeColor`	Display color for Free curves on surfaces when Curve Connectivity option is turned on.
`BOOL CurveConnectManifold`	If True and Curve Connectivity option is turned on, Manifold curves on surfaces will be displayed using the specified color by `CurveConnectManifoldColor`.
`INT4 CurveConnectManifoldColor`	Display color for Manifold curves on surface when Curve Connectivity option is turned on.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL CurveConnectNonManifold`	If True and Curve Connectivity option is turned on, NonManifold curves on surfaces will be displayed using the specified color by `CurveConnectNonManifoldColor`.
`INT4 CurveConnectNonManifoldColor`	Display color for NonManifold curves on surface when Curve Connectivity option is turned on.
`REAL8 RenderLogDepthOffset`	Controls Mesh Edit / Free Edge / Aero Entities / Contact / Element / Boundary / Surface Depth Offset (0.001 to 1000, default is 1).
`REAL8 RenderPushForward`	Controls edge offset of filled entities such as surfaces and elements at an angle to the screen (0 to 500, default is 50).
`REAL8 RenderPushUnit`	Controls edge offset of filled entities such as surfaces and elements perpendicular to normal (0 to 500, default is 100).
`REAL8 RenderPushUndeformed`	Controls the screen normal offset of the undeformed model with respect to the deformed model (-100 to 100, default is -100).
`INT4 RenderPushLabel`	Controls the screen normal offset of labels. The value is a multiple of the font height (0 to 10, default is 1).
`REAL8 DeformedRelScale`	Deformed relative scale factor.
`REAL8 DeformedAbsScale`	Deformed absolute scale factor.
`INT4 AnimationFrames`	Number of animation frames.
`INT4 AnimationDelay`	Animation delay factor.

`INT4 UserContourLevel[0..11]`
or
`VARIANT vUserContourLevel`
	Use User Defined contours.

`REAL8 UserContourValue[0..11]`	Contour level value.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vUserContourValue`	
`INT4 UserContourCount`	Number of user defined levels.
`INT4 ContourLabelFreq`	Contour labeling frequency.
`INT4 ContourLabelDigits`	(OBSOLETE) For Femap version 2020.1 and above, use various Digits Control properties on the View Object to specify options for display of real numbers in the Graphics Window. Number of digits on contour labels (0 to 15.)
`INT4 ContourLegendFreq`	Contour legend labeling frequency.
`INT4 ContourLegendDigits`	(OBSOLETE) For Femap version 2020.1 and above, use the various Digits Control properties on the View Object to specify options for display of real numbers in the Graphics Window. Number of digits on contour legend (0 to 15).
`BOOL ContourLegendHorizontal`	Display contour legend horizontally (True) or vertically (False).
`BOOL ContourLegendReversed`	Display contour legend with values reversed (True) or in typical order (False).
`REAL8 ContourLegendShrinkPct`	Contour legend is shrunk to this percentage to avoid overlap with view legend, post title and logo bitmap. Must be a value between 1.0 and 100.0.
`REAL8 ContourLegendWidthPct`	Controls the thickness of only the color bar portion of the legend. This option can be used to make the overall width of the color bar thicker or thinner and must have a value between 10.0 and 200. (default is 100.0).
`BOOL ContourLegendTopLeftLabel`	Always have labels appear to the left of a vertical legend/above a horizontal legend (True) or appear right of a vertical legend/below a horizontal legend (False).

Unpublished work. © 2025 Siemens

Property	Description
`BOOL ContourLegendLockOrientation`	Orientation is locked (True) or free to change orientation as the contour legend is moved around the graphics window (False).

`REAL8 ContourLegendLocation[0..1]`
or
`VARIANT vContourLegendLocation`
	X and Y positions are the relative distances, as a percentage of overall screen size, from the upper left hand corner of the *legend area* (area encompassing the legend and values) to the upper left hand corner of the view where the legend is being displayed. For instance, a value of X = 0 and Y = 0 positions the legend as far to the left and as close to the top of the view as possible, while X = 100 and Y = 100 positions the legend as for to the right and as close to the bottom as possible.

`INT4 ContourColor[0..5]`
or
`VARIANT vContourColor`
	Contour Base Colors.

`REAL8 CriteriaMax`	Upper criteria limit.
`REAL8 CriteriaMin`	Lower criteria limit.
`BOOL ContourElemUnaveraged`	Show unaveraged element contours.
`BOOL ContourElemBreakLinearParabolic`	Don’t average across Linear/Parabolic boundary.
`BOOL ContourElemBreakProp`	Don’t average across properties.
`BOOL ContourElemBreakMatl`	Don’t average across materials.
`BOOL ContourElemBreakLayer`	Don’t average across layers.
`BOOL ContourElemBreakColor`	Don’t average across colors.
`BOOL ContourElemBreakGeom`	Don’t average across discontinuous geometry.
`REAL8 ContourElemBreakAngle`	Geometry break angle.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ContourDataSelection`	Sets Data Selection option in Contour Options. 0 = Contour Group, 1 = Visible Group(s), 2 = All Data/ Full Model.
`BOOL ContourCornerData`	Sets Use Corner Data option in Contour Options. True = On, False = Off.
`INT4 ContourDataConversion`	Specifies Data Conversion option in Contour Options. 0 = Average, 1 = Max Absolute Value, 2 = Min Absolute Value, 3 = Max Value, 4 = Min Value.
`BOOL SectionPlaneMode`	The mode for section displays - True for Multiple Section mode, false otherwise.
`BOOL Section1On`	True if Section plane 1 is on in Multiple Section mode.
`BOOL Section2On`	True if Section plane 2 is on in Multiple Section mode.
`BOOL Section3On`	True if Section plane 3 is on in Multiple Section mode.

`REAL8 SectionBase[0..2]`
or
`VARIANT vSectionBase`
	Origin for first cut plane.

`REAL8 SectionNormal[0..2]`
or
`VARIANT vSectionNormal`
	Vector normal for first cut plane.

`REAL8 Section2Base[0..2]`
or
`VARIANT vSection2Base`
	Origin for second cut plane.

`REAL8 Section2Normal[0..2]`	Vector normal for second cut plane.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vSection2Normal`	

`REAL8 Section3Base[0..2]`
 	
or
 	
`VARIANT vSection3Base`
	Origin for third cut plane.

`REAL8 Section3Normal[0..2]`
 	
or
 	
`VARIANT vSection3Normal`
	Vector normal for third cut plane.

`REAL8 SectionSpacing`	Spacing between cut planes.
`INT4 SectionCount`	Number of Cutting Planes.
`BOOL SectionMode`	Section cut mode.
`INT4 SectionCapMode`	Only used when both `SectionPlaneMode` and `SectionMode` are both set to False. Sets the Cap Option for the Cut Plane. 0=Negative Cap, 1=Positive Cap.
`REAL8 IsosurfaceValue`	Single IsoSurface value.
`INT4 TraceOpt`	Trace option.
`INT4 TraceIDV2`	This property replaces `TraceID`. For more information see, **Working with Output in Femap v2020.1 and above**. ID of output vector for trace plots.
`BOOL ContourVecHide`	True if contour vectors should be included in hidden line calculation.
`INT4 ContourVecMode`	Contour vector output relative to mode.

Unpublished work. © 2025 Siemens

Property	Description
`BOOL ContourVecAutoVec`	Automatically select contour arrow vectors from `ContourVector1`.
`BOOL ContourVecAutoOrient`	Automatically orient contour arrows based on selected contour vector(s).
`BOOL ContourVecMinVecOn`	Only display contour arrows above the value specified for `ContourVecMinVec`.
`REAL8 ContourVecMinVec`	Threshold value to display contour arrows.
`REAL8 ContourVecMinLen`	Minimum length between 0 and 100 for contour arrows. The minimum length specified represents the shortest arrow that will be drawn. Greater values will scale up from that length, to the maximum.

`INT4 ContourVecID1V2[0..2]`
or
`VARIANT vContourVecID1V2`
	These properties replace `ContourVecID1` and `vContourVecID1`. For more information see, **Working with Output in Femap v2020.1 and above**. IDs of output vectors for contour vector.

`INT4 ContourVecID2V2[0..2]`
or
`VARIANT vContourVecID2V2`
	These properties replace `ContourVecID2` and `vContourVecID2`. For more information see, **Working with Output in Femap v2020.1 and above**. IDs of output vectors for second contour vector.

`INT4 ContourVecID3V2[0..2]`
or
`VARIANT vContourVecID3V2`
	These properties replace `ContourVecID3` and `vContourVecID3`. For more information see, **Working with Output in Femap v2020.1 and above**. IDs of output vectors for third contour vector.

`BOOL ContourVecOn[0..2]`
or
`VARIANT vContourVecOn`
	Visibility flag for individual contour arrow.
	
	`ContourVecOn(0)`=Arrow 1
	`ContourVecOn(1)`=Arrow 2
	`ContourVecOn(2)`=Arrow 3
	
	When True, contour arrow will be displayed.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 ContourVecCSys`	Contour vector CSys.

`REAL8 ContourVecDir[0..2]`
 	
or
 	
`VARIANT vContourVecDir`
	Components of contour vector direction.

`INT4 ContourVecColor1`	Color of Contour Arrow 1 - Use enum for color for easier selection of color).
`INT4 ContourVecColor2`	Color for second contour vector.
`INT4 ContourVecColor3`	Color for third contour vector.
`INT4 ContourVecDigits`	(OBSOLETE) For Femap version 2020.1 and above, use various Digits Control properties on the View Object to specify options for display of real numbers in the Graphics Window. Number of digits in contour vector labels (0 to 15)
`INT4 ContourVecLength`	Length of contour vectors.
`INT4 StreamLineOpt`	Determines what location(s) are used to initiate streamlines (0 – uses `vStreamLineLocation`, 1 – uses `StreamLineID` as a point ID, 2 – uses `StreamLineID` as a group ID).
`INT4 StreamLineID`	ID of a point or group of points used to initiate a streamline.
`INT4 StreamLineRungeKuttaOrder`	Controls accuracy of streamline solution – the higher the value, the more accurate and slower the solution (1 to 4).
`REAL8 StreamLineTolerance`	Controls accuracy of streamline solution – the lower the value, the more accurate and slower the solution (1e-15 to 1).
`REAL8 StreamLineMaxLenFactor`	This is a multiplier times the model *diameter* – if the streamline is too long, calculation is terminated (1e-15 upwards).

Unpublished work. © 2025 Siemens

Property	Description
`REAL8 StreamLineMinSpeedPct`	This is a percentage of the maximum speed – if the speed gets too low, calculation is terminated (1e-15 to 100).
`REAL8 StreamLineMaxTime`	Not Used.

`REAL8 StreamLineLocation[0..2]`
 	
or
 	
`VARIANT vStreamLineLocation`
	XYZ location of start point of streamline if `StreamLineOpt` = 0.

`INT4 TransparencyPercent`	Transparency percentage applied to all entities (0=opaque,100=fully transparent).
`BOOL RotateAroundModelAxes`	If True, dynamic rotations are around the model axes. If False, screen axes (used if `RotateUseAxis` is true).
`BOOL RotateSingleAxis`	If True, dynamic rotations are around a single axis based on the predominant direction (used if `RotateUseAxis` is true).
`BOOL RotateUseAxis`	If True, single and model axis dynamic rotation is available.
`INT4 RotateCSys`	The ID of the Coordinate System used for rotations by the **View Rotate** command.

`INT4 UserContourPalette[0..63]`
 	
or
 	
`VARIANT vUserContourPalette`
	Colors for user contour palette.

`INT4 MaxUserContourPalette`	Number of colors stored in UserContour Palette.
`INT4 TransformDeformMode`	Controls destination of transformed deformation data: 0 = None 1 = Coordinate System given in `TransformDeformCSys` 2 = Nodal Output Coordinate System

Unpublished work. © 2025 Siemens

Property	Description
`INT4 TransformDeformCSys`	Destination coordinate system when `TransformDeformMode` = 1.
`BOOL TransformDeformX`	Control which components of the transformed deformation are displayed.
`BOOL TransformDeformY`	
`BOOL TransformDeformZ`	
`BOOL TransformDeformXInput`	Control which components of the transformed deformation are used for transformation.
`BOOL TransformDeformYInput`	
`BOOL TransformDeformZInput`	
`INT4 TransformNodalMode`	Controls contour/criteria destination coordinate system for transformed nodal output vectors: 0 = None 1 = Coordinate System given in `TransformNodalCSys` 2 = Nodal Output Coordinate System
`INT4 TransformNodalCSys`	Destination coordinate system when `TransformNodalMode` = 1.
`INT4 TransformPlateMode`	Controls contour/criteria destination for transformed plate element output vectors: 0 = None. 3 = Material Coordinate System. 4 = Vector defined by axis `TransformPlateDOF` of Coordinate System `TransformPlateCSys` projected onto plate. 5 = Vector defined by `TransformPlateVector` projected onto plate.
`INT4 TransformPlateCSys`	Destination coordinate system when `TransformPlateMode` = 4.
`INT4 TransformPlateDOF`	Axis of destination coordinate system when `TransformPlateMode` = 4.
`INT4 TransformPlateVector[0..2]`	Vector used when `TransformPlateMode` = 5.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vTransformPlateVector`	
`REAL8 TransformPlateTolerance`	Tolerance Angle value used when transforming plate output.
`INT4 TransformSolidMode`	Controls contour/criteria destination coordinate system for transformed solid element output vectors: 0 = None. 6 = Coordinate System given in `TransformSolidCSys`. 7 = Material Coordinate System.
`INT4 TransformSolidCSys`	Destination coordinate system when `TransformSolidMode` = 4.
`INT4 GlobalPlyLocation`	For laminate results, the global ply location to display for contour/criteria displays: 0=Location of Selected Output Vector, -1=Top Ply, -2=Bottom Ply, any other positive number is the ID of the Global Ply to display. Not used unless laminate results are selected.
`INT4 DeformRelativeToNode`	The `NodeID` to deform/animate around. You must also set the *label* value for this to be used.
`REAL8 BeamDiagramScale`	The scale factor for beam diagrams.
`INT4 ComplexSyncMethod`	Complex Results - Sets method for complex expansion (-1=Use Model Settings, 0=Selected Results - Ignore Phase Differences in View Override section, 1=Synchronize Phase in View Override section).
`REAL8 ComplexSyncPhase`	Complex Results - Static Display Phase Angle in View Override section.
`REAL8 ComplexSyncIncrement`	Complex Results - Animation Phase Increment in View Override section.
`REAL8 ClipPlaneOrigin[0..2]`	The coordinates of a point on the model clipping plane.

Unpublished work. © 2025 Siemens

Property	Description
or `VARIANT vClipPlaneOrigin`	

`REAL8 ClipPlaneNormal[0..2]`
 	
or
 	
`VARIANT vClipPlaneNormal`
	The vector components of the normal to the model clipping plane.

`BOOL BlankNodesOff`	If True, nodes associated with the blanked entity will not be blanked.
Digits Control - View Options - Load Vectors	
`INT4 DO_Loads_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_Loads_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_Loads_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_Loads_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_Loads_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001
`INT4 DO_Loads_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_Loads_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DO_Loads_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_Loads_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_Loads_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_Loads_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_Loads_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_Loads_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_Loads_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Digits Control - View Options - Constraint	
`INT4 DO_BCs_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_BCs_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_BCs_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_BCs_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_BCs_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DO_BCs_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_BCs_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_BCs_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_BCs_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_BCs_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_BCs_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_BCs_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_BCs_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_BCs_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Digits Control - View Options - Deformed Model	
`INT4 DO_Deformed_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_Deformed_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_Deformed_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_Deformed_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_Deformed_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0

Unpublished work. © 2025 Siemens

Property	Description
	1 = e+1 2 = e+01 3 = e+001
`INT4 DO_Deformed_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_Deformed_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_Deformed_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_Deformed_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_Deformed_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_Deformed_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_Deformed_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_Deformed_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_Deformed_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Digits Control - View Options - Contour/Criteria Style	
`INT4 DO_Criteria_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_Criteria_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_Criteria_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_Criteria_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DO_Criteria_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001
`INT4 DO_Criteria_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_Criteria_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_Criteria_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_Criteria_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_Criteria_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_Criteria_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_Criteria_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_Criteria_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_Criteria_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Digits Control - View Options - Contour/Criteria Legend	
`INT4 DO_ContourLegend_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_ContourLegend_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_ContourLegend_RoundToZeroValue`	Sets value for Round To Zero.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DO_ContourLegend_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_ContourLegend_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001
`INT4 DO_ContourLegend_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_ContourLegend_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_ContourLegend_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_ContourLegend_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_ContourLegend_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_ContourLegend_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_ContourLegend_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_ContourLegend_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_ContourLegend_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Digits Control - View Options - Beam Diagram	
`INT4 DO_BeamDiagram_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic

Unpublished work. © 2025 Siemens

Property	Description
`BOOL DO_BeamDiagram_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_BeamDiagram_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_BeamDiagram_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_BeamDiagram_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001
`INT4 DO_BeamDiagram_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_BeamDiagram_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_BeamDiagram_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_BeamDiagram_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_BeamDiagram_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_BeamDiagram_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_BeamDiagram_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_BeamDiagram_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_BeamDiagram_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Digits Control - View Options - Contour Arrow Options	
`INT4 DO_ContourArrow_Option`	Sets the Method in Digit Options:

Unpublished work. © 2025 Siemens

Property	Description
	1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_ContourArrow_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_ContourArrow_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_ContourArrow_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_ContourArrow_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001
`INT4 DO_ContourArrow_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_ContourArrow_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_ContourArrow_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_ContourArrow_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_ContourArrow_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_ContourArrow_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_ContourArrow_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.
`INT4 DO_ContourArrow_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_ContourArrow_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.

Unpublished work. © 2025 Siemens

Property	Description
Digits Control - View Options - Freebody Vectors	
`INT4 DO_FreeBody_Option`	Sets the Method in Digit Options: 1 = Exponent 2 = Significant Figures 3 = Decimal Places 4 = Femap Classic
`BOOL DO_FreeBody_RoundToZero`	Sets On/Off for Round To Zero.
`REAL8 DO_FreeBody_RoundToZeroValue`	Sets value for Round To Zero.
`INT4 DO_FreeBody_BeforeDecimalSeparator`	Sets the before Exponent Style: 0 = .123 1 = 0.123 2 = 1.123
`INT4 DO_FreeBody_MinimumExponentSize`	Sets the minimum exponent size: 0 = No E+0 1 = e+1 2 = e+01 3 = e+001
`INT4 DO_FreeBody_ExponentDigits`	Sets the number of digits for the Exponent method. Can be 2 to 15 digits.
`INT4 DO_FreeBody_SigFigs`	Sets the number of Significant Figures for the Significant Figure Method. Can be 1 to 15.
`INT4 DO_FreeBody_LeadingZeros`	Sets the Maximum Zeros after Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_FreeBody_TrailingZeros`	Sets the Maximum Zeros before Decimal Separator for the Significant Figure Method. Can be 0 to 10.
`INT4 DO_FreeBody_DecimalPlaces`	Sets the Number of decimal places for the Decimal Place Method. Can be 1 to 15.
`INT4 DO_FreeBody_MaxDigits`	Sets the Maximum number of digits for the Decimal Place Method. Can be 2 to 16. Must be greater than number of decimal places.
`INT4 DO_FreeBody_BaseExponent`	Sets the Base Exponent for the Decimal Place Method. Can be -30 to 30.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 DO_FreeBody_OriginalDigits`	Sets the Number of Digits for the Femap Classic Method. Can be 0 to 15.
`BOOL DO_FreeBody_Exponent`	Sets the On/Off for Force Exponent in the Femap Classic Method.
Model Data Contour and Discrete Value controls	
`BOOL ModelDataContourOn`	If True, turns on **Show Model Data Contour** in the Contour Data From section of Contour Model Data dialog box. In addition, this option must be set to True, along with `DiscreteValueOn`, for a Discrete Value plot to be displayed.
`INT4 ModelDataContourGroup`	ID of Group specified in Show On section of the Contour Model Data dialog box to limit display of selected model data. Specify 0 for all groups or -1 to use the active group. Also used for Discrete Value plots.
`BOOL ModelDataContourLabels`	If True, turns on the **Allow Labels** option in the **Show As** section of the Contour Model Data dialog box. Also used for Discrete Value plots.
`BOOL DiscreteValueOn`	If True, turns on the **On** option in the **Discrete Values** section of Contour Model Data dialog box. In addition, `ModelDataContourOn` option must be set to True for Discrete Value plot to be displayed.
`INT4 DiscreteValueSetID`	ID of Discrete Value Set to display.
Legacy XY Plot Options (Should not be used)	

`INT4 XYOutputSet[0..8]`
or
`VARIANT vXYOutputSet`
	Output Set IDs for each curve.

`INT4 XYSetRange[0..1]`
or
`VARIANT vXYSetRange`
	Output Set IDs for all XY curves.

Unpublished work. © 2025 Siemens

Property	Description

`INT4 XYOutputDataV2[0..8]`
 	
or
 	
`VARIANT vXYOutputDataV2`
	These properties replace `XYOutputData` and `vXYOutputData`. For more information see, **Working with Output in Femap v2020.1 and above**.
	
	Output Vector IDs for XY curves.

`INT4 XYOutputEntity[0..8]`
 	
or
 	
`VARIANT vXYOutputEntity`
	Entity ID (node or element) for each XY curve.

`INT4 Function[0..8]`
 	
or
 	
`VARIANT vFunction`
	Function IDs for XY plot curves.

`REAL8 XYXmin`	XY X axis range.
`REAL8 XYXmax`	
`REAL8 XYymin`	XY Y aAxis range.
`REAL8 XYymax`	

`REAL8 XYScale[0..8]`
 	
or
 	
`VARIANT vXYScale`
	Scale factor for associated XY curve.

`STRING XYTitle`	XY plot titles (maximum 79 characters).
`STRING XYSubtitle`	
`INT4 XYCSys`	CSys for XY vs Position.

Unpublished work. © 2025 Siemens

Property	Description
`INT4 XYDirection`	Direction for XY vs Position.
`INT4 XYXTics`	Number of tics along X axis.
`INT4 XYYTics`	Number of tics along Y axis.
`REAL8 PrevWindowLeft`	Previous sizing of XY plot in window.
`REAL8 PrevWindowTop`	
`REAL8 PrevWindowRight`	
`REAL8 PrevWindowBottom`	
`REAL8 PrevWindowXmin`	
`REAL8 PrevWindowXmax`	
`REAL8 PrevWindowYmin`	
`REAL8 PrevWindowYmax`	

View Object Freebody Interface

The following table shows properties that were part of the View Object Freebody Interface prior to Femap version 10.3. With that release, the entire methodology of Freebody Display was changed necessitating changes to the API. This table shows how to map the old API interface to the new one.

Property	Description	Now Controlled By
`BOOL FreebodyOn`	Flag indicating freebody is on.	View Object `Draw[FVI_FREEBODY]` Property.
`BOOL FreebodyLoad[0..5]`	Flags for load types for freebody.	Freebody Object `SumContributions` Property.

Unpublished work. © 2025 Siemens

Property	Description	Now Controlled By
or `VARIANT vFreebodyLoad`		
`INT4 FreebodyGroup`	ID of group to be used in freebody.	Freebody Object Group Property.

`BOOL FreebodyForceMom[0..1]`
 	
or
 	
`VARIANT vFreebodyForceMom`
	Show `forces[0]` and `moments[1]`.	Vector visibility: Freebody Object `ShowTotalVec` and `ShowNodalVec` Properties. Inclusion in summation vector calculation: Freebody Object `SumComponents` Property.

`BOOL FreebodyShowInternal`	Show internal freebody data.	No longer used.
`BOOL FreebodyShowSum`	Show summed data at each node.	Still Valid.
`BOOL FreebodyMinVecFlag`	Show vectors with a minimum length.	Still Valid.
`BOOL FreebodyComponents`	Show vector components.	Freebody Object `TotalVectorMode` and `NodalVectorMode` Properties.

`BOOL FreebodyXYZ[0..2]`
 	
or
 	
`VARIANT vFreebodyXYZ`
	Flags for X,Y,Z components.	Vector visibility: Freebody Object `ShowTotalVec` and `ShowNodalVec` Properties. Inclusion in summation vector calculation: Freebody Object `SumComponents` Property.

`BOOL FreebodyScaleMag`	Scale vectors based on their magnitude.	View Object `Draw[FVI_FREEBODY_VECTOR]` Property.
`BOOL FreebodyShowResultant`	Show resultant of summed freebody info.	Freebody Object `TotalVectorMode` Property.

Unpublished work. © 2025 Siemens

Property	Description	Now Controlled By
`BOOL FreebodyEntityColor`	Show freebody with entity colors.	View Object `ColorMode[FVI_NODAL_F]` and `ColorMode[FVI_NODAL_M]` Properties. Set to 3 for Entity Colors.
`INT4 FreebodyShowCSys`	CSys ID for freebody display.	Freebody Object CSys Property.
`REAL8 FreebodyShowMag`	Minimum magnitude vector to show.	Still Valid.

`REAL8 FreebodyShowLength[0..1]`
 	
or
 	
`VARIANT vFreebodyShowLength`
	Min/Max display length.	View Object `Freebody_Length`, `Freebody_UnitsPerLength`, `Freebody_Total_Force_Factor`, `Freebody_Total_Moment_Factor`, `Freebody_Nodal_Force_Factor` and `Freebody_Nodal_Moment_Factor` Properties.

`REAL8 FreebodyResultantLoc[0..2]`
 	
or
 	
`VARIANT vFreebodyResultantLoc`
	Location for resultant vector.	Freebody Object X, Y and Z Properties.

`INT4 FreebodyResultantColor`	Color for resultant vector.	Freebody Object `TotalVecColor` Property when Total Vector Color Mode is set to Freebody in the Active View.

The following table describes new View Object Properties that were added in Femap version 10.3 for controlling Freebody displays.

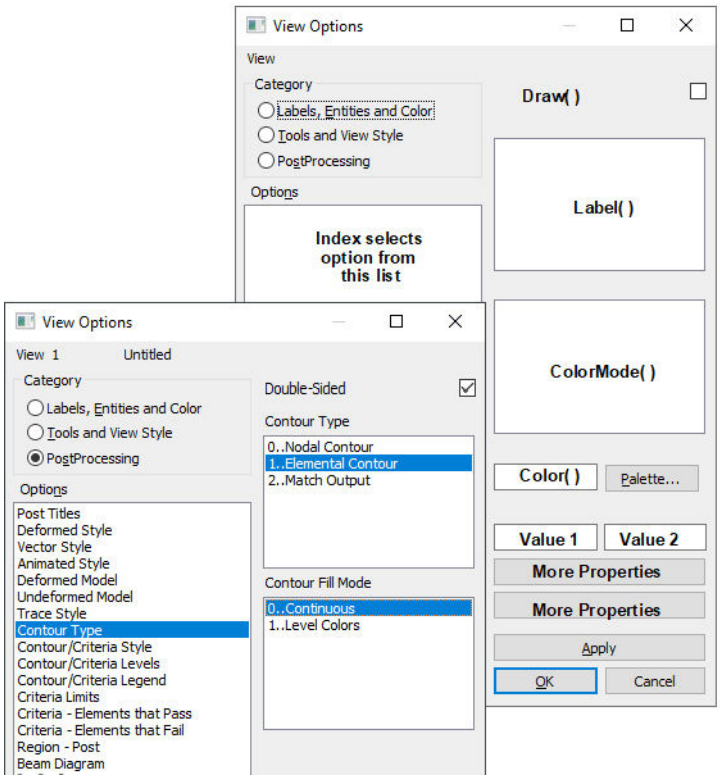
Unpublished work. © 2025 Siemens

Property	Description
`INT4 FreebodyLength`	Controls the baseline length for all Freebody Vectors.
`INT4 FreebodyUnitsPerLength`	Controls the number of units per baseline length when Freebody Vector scaling is enabled.
`REAL8 FreebodyTotalForceFactor`	Length Factor for Freebody Total Force Vector.
`REAL8 FreebodyTotalMomentFactor`	Length Factor for Freebody Total Moment Vector.
`REAL8 FreebodyNodalForceFactor`	Length Factor for Freebody Nodal Force Vectors.
`REAL8 FreebodyNodalMomentFactor`	Length Factor for Freebody Nodal Moment Vectors.
`INT4 FreebodyDigits`	Controls the number of digits on Freebody value labels. This is an alias for the `ColorMode[FVI_FREEBODY_VECTOR]` Property.

View Option Types

All of the View Option properties (Draw, Label, Color, ColorMode) require an index to access the desired values. These properties and their related indices correspond to the options that you can control in the user interface **View Options** command. The following figure shows how the API properties relate to the various controls in the dialog box.

Unpublished work. © 2025 Siemens



So, for example, to set the View to use Double-Sided, Elemental Contours, as shown above, you would set:

```
View.Draw( FVI_CONTOUR_TYPE ) = True
```

```
View.Label( FVI_CONTOUR_TYPE ) = 1
```

...and save the View object. Note that for `FVI_CONTOUR_TYPE`, the `Color( )` property is not used and does not appear in the dialog box. You should not query or set these unused properties. Doing so will not cause any current problem, but it will also not be useful and if those properties are used in future releases, it could cause errors.

The View Options dialog does have additional controls that are not linked to the `Draw( )`, `Label( )`, `ColorMode( )` or `Color( )` properties. In the figure above, they are labeled **Value 1**, **Value 2** and **More Properties**. Accessing these properties is described later, first however, the following table maps the index used by `Draw( )`, `Label( )`, `ColorMode( )` and `Color( )` to the names in the **View Options** command.

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Labels, Entities and Colors		
Label Parameters	0	FVI_LABEL
Coordinate System	1	FVI_CSYS
Point	2	FVI_POINT
Curve	3	FVI_CURVE
Combined Curve	95	FVI_COMPOSITE_CURVE
Curve - Mesh Size	24	FVI_CURVE_MESHSIZE
Curve/Surface Direction	39	FVI_CURVE_ACCURACY
Surface	4	FVI_SURFACE
Boundary	27	FVI_BOUNDARY
Solid	5	FVI_SOLID
Mesh Point	119	FVI_MESH_POINT
Text	6	FVI_TEXT
Node	7	FVI_NODE
Node - Perm Constraint	8	FVI_NODE_PERMCONSTRAINT
Element	9	FVI_ELEMENT
Element - Direction	10	FVI_ELEMENT_DIRECTION
Element - Offsets/Releases	11	FVI_ELEMENT_OFFSET
Element - Orientation/Shape	12	FVI_ELEMENT_ORIENT
Element - Beam Y-Axis	13	FVI_BEAM_YAXIS

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Element - Coordinate System	96	`FVI_ELEM_CSYS`
Element - Material Direction	115	`FVI_ELEM_MATL_DIR`
Element - Weld	90	`FVI_ELEMENT_WELD`
Element - Rigid	94	`FVI_RIGID`
Element - Cohesive	120	`FVI_ELEMENT_COHESIVE`
Element - Nastran General Matrix	125	`FVI_NASTRAN_GENERAL_MATRIX`
Element - Shell	121	`FVI_ELEMENT_SHELL`
Load Vectors	78	`FVI_LOAD_VECTORS`
Load - Body	100	`FVI_LOAD_BODY`
Load - Force and Bearing	14	`FVI_LOAD_FORCE`
Load - Moment and Torque	15	`FVI_LOAD_MOMENT`
Load - Temperature	16	`FVI_LOAD_THERMAL`
Load - Distributed Load	71	`FVI_LOAD_DISTRIBUTEDLOAD`
Load - Pressure	17	`FVI_LOAD_PRESSURE`
Load - Acceleration	18	`FVI_LOAD_ACCELERATION`
Load - Velocity	72	`FVI_LOAD_VELOCITY`
Load - Enforced Displacement	19	`FVI_LOAD_ENFORCEDDISP`
Load - Nonlinear Force	73	`FVI_LOAD_NONLINEARFORCE`
Load - Heat Generation	66	`FVI_LOAD_HEATGEN`

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Load - Heat Flux	67	FVI_LOAD_HEATFLUX
Load - Convection	68	FVI_LOAD_CONVECTION
Load - Radiation	69	FVI_LOAD_RADIATION
Load - Bolt Preload	93	FVI_BOLT_PRELOAD
Load - Fluid Tracking	81	FVI_LOAD_FLUIDTRACKING
Load - Unknown Condition	82	FVI_LOAD_UNKNOWN
Load - Slip Wall Condition	83	FVI_LOAD_SLIPWALL
Load - Fan Curve	84	FVI_LOAD_FANCURVE
Load - Periodic Condition	85	FVI_LOAD_PERIODIC
Constraint	20	FVI_CONSTRAINT
Constraint Equation	21	FVI_CONSTRAINT_EQUATION
Connector	79	FVI_CONNECTOR
Connection Region	91	FVI_CONNECTION_REGION
Combined - Eliminated Points	99	FVI_INTERIOR_POINT
Combined - Eliminated Curves	97	FVI_INTERIOR_CURVE
Combined - Eliminated Surfaces	98	FVI_INTERIOR_SURFACE
Monitor Point	123	FVI_MONITOR_POINT
Aero Panel	102	FVI_AERO_PANEL
Aero Mesh	103	FVI_AERO_MESH

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Aero Interference	108	`FVI_AERO_INTERFERENCE`
Aero Spline	104	`FVI_AERO_SPLINE`
Aero Control Surface	105	`FVI_AERO_SURFACE`
Kinematic Joint	126	`FVI_KINEMATIC_JOINT`
Load - Kinematic Driver	127	`FVI_DRIVER_LOAD`
Tools and View Style		
Free Edge and Face	22	`FVI_FREE_EDGE`
Shrink Elements	23	`FVI_SHRINK_ELEM`
Fill, Backfaces and Hidden	25	`FVI_FILL_ELEM`
Filled Edges	26	`FVI_FILLED_EDGES`
Feature Line	118	`FVI_FEATURE_LINE`
Curve Connectivity	124	`FVI_CURVE_CONNECTIVITY`
Graphics Options	77	`FVI_RENDER_OPTIONS`
Performance Graphics	116	`FVI_PERF_GRAPHICS`
Transparency	92	`FVI_TRANSPARENCY`
Shading	28	`FVI_SHADING`
Perspective	29	`FVI_PERSPECTIVE`
Axisymmetric Axis	87	`FVI_AXISYM_AXIS`
View Legend	31	`FVI_VIEW_LEGEND`
View Axes	32	`FVI_VIEW_AXES`

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Origin	33	FVI_ORIGIN
Workplane and Rulers	34	FVI_WORKPLANE
Workplane Grid	35	FVI_WORKPLANE_GRID
Group Clipping Planes	36	FVI_CLIPPING_PLANE
Symbols	37	FVI_SYMBOL
Preview	109	FVI_PREVIEW
View Aspect Ratio	38	FVI_ASPECT_RATIO
Model Clipping Plane	101	FVI_CLIP_PLANE
PostProcessing		
Post Titles	40	FVI_POST_TITLES
Deformed Style	41	FVI_DEFORMED_STYLE
Vector Style	42	FVI_VECTOR_STYLE
Animated Style	43	FVI_ANIMATED_STYLE
Deformed Model	44	FVI_DEFORMED
Undeformed Model	45	FVI_UNDEFORMED
Trace Style	74	FVI_TRACE_STYLE
Contour Type	80	FVI_CONTOUR_TYPE
Contour/Criteria Style	46	FVI_CONTOUR_STYLE
Contour/Criteria Levels	47	FVI_CONTOUR_LEVELS
Contour/Criteria Legend	48	FVI_CONTOUR_LEGEND

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Criteria Limits	49	`FVI_CRITERIA_LIMITS`
Criteria - Elements that Pass	50	`FVI_CRITERIA_PASS`
Criteria - Elements that Fail	51	`FVI_CRITERIA_FAIL`
Region - Post	122	`FVI_REGION_POST`
Beam Diagram	86	`FVI_BEAM_DIAGRAM`
IsoSurface	76	`FVI_ISOSURFACE`
IsoLine	88	`FVI_ISOLINE`
Streamline	89	`FVI_STREAMLINE`
Contour Arrow Style	75	`FVI_CONTOUR_VECTOR`
Contour Arrow Options	117	`FVI_CONTOUR_ARROW_OPT`
Freebody	114	`FVI_FREEBODY`
Freebody Node Markers	106	`FVI_FREEBODY_NODEMARKER`
Freebody Vectors	113	`FVI_FREEBODY_VECTOR`
Freebody Total Force	107	`FVI_FREEBODY_TOTALF`
Freebody Total Moment	110	`FVI_FREEBODY_TOTALM`
Freebody Nodal Force	111	`FVI_FREEBODY_NODAL_F`
Freebody Nodal Moment	112	`FVI_FREEBODY_NODAL_M`

The following table describes how to access *extra properties* used by some dialog options:

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
Labels, Entities and Colors		
Label Parameters	**Value1**, **Text Depth Offset**	`RenderPushLabel`
Load Vectors	**Value1**, **Load Len**	`LoadVecLength`
	Value2, **Min Scale**	`LoadVectorMinScale`
	Digits... button	Digit Options Dialog... See the **Digits Control - View Options - Load Vectors** section for information.
Constraint	**Digits...** button	Digit Options Dialog... See **Digits Control - View Options - Constraint** section for information.
Tools and View Style		
Shrink Elements	**Value1**, **Shrink To %**	`ShrinkFactor`
Feature Line	**Value1**, **Angle**	`FeatureLineAngle`
Curve Connectivity	**More-Connectivity Colors**	Curve Connectivity Colors Dialog...
	Wirebody toggle	`CurveConnectWirebody`
	Wirebody Color	`CurveConnectWirebodyColor`
	Free toggle	`CurveConnectFree`
	Free Color	`CurveConnectFreeColor`
	Manifold toggle	`CurveConnectManifold`
	Manifold Color	`CurveConnectManifoldColor`
	NonManifold toggle	`CurveConnectNonManifold`
Text in View Options	**Control**	**Property**

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
	NonManifold Color	`CurveConnectNonManifoldColor`
Graphics Options	**Value1, Offset Factor**	`RenderPushForward`
	Value2, Offset Units	`RenderPushUnit`
	More - Advanced Depth Control button	Advanced Depth Control Dialog...
	Mesh Edit / Free Edge / Aero Entities / Contact / Element / Boundary / Surface Depth Offset	`RenderLogDepthOffset`
	Filled Edge Depth Offset Factor	`RenderPushForward`
	Filled Edge Depth Offset Units	`RenderPushUnit`
	Text Depth Offset	`RenderPushLabel`
	Undeformed Depth Offset	`RenderPushUndeformed`
Transparency	**Value1, Percent**	`TransparencyPercent`
Shading	**Value1, % Ambient**	`AmbientLight`
	Value2, Distance	`SpotlightDist`
Text in View Options	**Control**	**Property**

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
	More - Light...	`Light` (when Light Location/Type is set to Viewer, Viewer Spot, Model, or Model Spot). `SpotlightDir` (when Light Location/Type is set to Model or Model Spot).
Perspective	**Value1**, **Distance**	`PerspectiveDist`
View Legend	**More - Custom Position...**	Select View Position - View Legend Dialog...
	X	`ViewLegendLocation(0)`
	Y	`ViewLegendLocation(1)`
	Justification option	`ViewLegendJustification`
	Lock Justification toggle	`ViewLegendLockJustification`
View Axes	**Value1**, **% Size**	
	More - Position...	Select View Position - View Axes Dialog...
	X	`AxisLocation (0)`
	Y	`AxisLocation (1)`
Symbols	**Value1**, **Load Len**	`LoadVecLength`
	Value2, **Other Vec**	`OtherVecLength`
View Aspect Ratio	**Value1**, **Aspect Ratio**	`Aspect`
Model Clipping Plane	**More - Clipping Plane**	Plane (Method) - Define Clipping Plane...
Text in View Options	**Control**	**Property**

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
	Origin for plane	`ClipPlaneOrigin`
	Normal direction of plane	`ClipPlaneNormal`
PostProcessing		
Post Titles	**More - Custom Position...**	Select View Position - Post Titles Dialog...
	X	`PostTitlesLocation(0)`
	Y	`PostTitlesLocation(1)`
	Justification option	`PostTitlesJustification`
	Lock Justification toggle	`PostTitlesLockJustification`
Deformed Style	**Value1, Scale %**	`DeformedRelScale`
	Value2, Scale Act	`DeformedAbsScale`
Vector Style	**Value1, Top Percent**	`ArrowsToLabel`
Animated Style	**Value1, Frames**	`AnimationFrames`
	Value2, Delay	`AnimationDelay`
Deformed Model	**Value1, Node ID**	`DeformRelativeToNode`
	Digits... button	Digit Options Dialog... See **Digits Control - View Options - Deformed Model** section for information.
Undeformed Model	**Value1, Depth Offset %**	`RenderPushUndeformed`
Text in View Options	**Control**	**Property**

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
Contour Type	**More - Contour Options**	Select Contour Options Dialog...
	Contour Group	`ContourGroup`
	Contour Type	`Label( FVI_CONTOUR_TYPE )`
	Data Conversion	`ColorMode( FVI_CONTOUR_STYLE )`
	No Averaging	`ContourElemUnaveraged`
	Property	`ContourElemBreakProp`
	Material	`ContourElemBreakMatl`
	Layer	`ContourElemBreakLayer`
	Color	`ContourElemBreakColor`
	Angle Between Check	`ContourElemBreakGeom`
	Angle Between Text	`ContourElemBreakAngle`
	Double-Sided Planar	`Draw( FVI_CONTOUR_TYPE )`
	Additional Element Data	`ContourVecAddl`
	Rendered Contours	`ColorMode( FVI_CONTOUR_TYPE )`
Contour/Criteria Style	**Value1, Label Freq**	`ContourLabelFreq`
Text in View Options	**Control**	**Property**

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
	Digits... button	Digit Options Dialog... See **Digits Control - View Options - Contour/ Criteria Style** section for information.
	More - Contour Options	Select Contour Options Dialog... See “Contour Type” in this table for more information
Contour/Criteria Levels	**Value1, Minimum**	UserContourValue(11)
	Value2, Maximum	UserContourValue(0)
	More - Set Levels	Contour/Criteria Levels Dialog...
	Specified Levels	UserContourLevel(1..10) for Level, UserContourValue(1..10) for Values
	Colors	ContourColor
	User Palette	MaxUserContourPalette, UserContourPalette
Contour/Criteria Legend	**Value1, Label Freq**	ContourLegendFreq
	Digits... button	Digit Options Dialog... See **Digits Control - View Options - Contour/Criteria Legend** section for information
	More - Custom Position...	Select View Position - Contour Legend Dialog...
	X	`ContourLegendLocation(0)`
	Y	`ContourLegendLocation(1)`
Text in View Options	**Control**	**Property**

Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
	Vertical/ Horizontal option	`ContourLegendHorizontal`
	Lock Orientation toggle	`ContourLegendLockOrientation`
	Reversed toggle	`ContourLegendReversed`
	Label Top/ Left toggle	`ContourLegendTopLeftLabel`
	More - Legend Size...	Enter Contour Legend Size Dialog...
	Length (%)	`ContourLegendShrinkPct`
	Width (%)	`ContourLegendWidthPct`
Criteria Limits	**Value1**, Minimum	`CriteriaMin`
	Value2, **Maximum**	`CriteriaMax`
Beam Diagram	**Value1**, **Scale %**	`BeamDiagramScale`
	Digits... button	Digit Options Dialog... See **Digits Control - View Options - Beam Diagram** section for information.
IsoSurface	**Value1**, **IsoSurface At**	`IsosurfaceValue`
IsoLine	**Value1**, **IsoLine At**	`IsosurfaceValue`
Streamline	**Value1**, **Max Length**	`StreamLineMaxLenFactor`
	Value2, **Min Speed**	`StreamLineMinSpeedPct`
Text in View Options	**Control**	**Property**

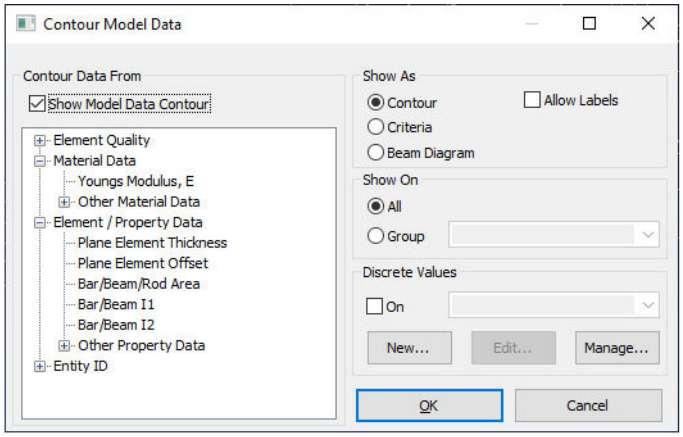
Unpublished work. © 2025 Siemens

Text in View Options	Control	Property
	More - Streamline Loc	`StreamLineLocation`
Contour Arrow Style	**Value1**, **Min Value**	`ContourVecMinVec`
	Value2, **Length**	`ContourVecLength`
Contour Arrow Options	**Value2**, **Min Length**	`ContourVecMinLen`
	Digits... button	Digit Options Dialog... See **Digits Control - View Options - Contour Arrow Options** section for information.
Freebody Vectors	**Value1**, **Units/Length**	`FreebodyUnitsPerLength`
	Value2, **Length**	`FreebodyLength`
	Digits... button	Digit Options Dialog... See **Digits Control - View Options - Freebody Vectors** section for information.
Freebody Total Force	**Value2**, **Factor**	`FreebodyTotalForceFactor`
Freebody Total Moment	**Value2**, **Factor**	`FreebodyTotalMomentFactor`
Freebody Nodal Force	**Value2**, **Factor**	`FreebodyNodalForceFactor`
Freebody Nodal Force	**Value2**, **Factor**	`FreebodyNodalMomentFactor`
Text in View Options	**Control**	**Property**

Model Contour Data Types

The **View** → **Advanced Post** → **Model Contour Data** command, which is used to create contour, criteria, or beam diagram plots of model data, opens the Contour Model Data dialog box:

Unpublished work. © 2025 Siemens



The types of properties and materials in the model determine which model data items appear in the **Contour Data From** section and can be plotted. To specify which model data item to display, use the `SetContourModelDataType` method (see **SetModelDataContourType**).

Some more commonly used items are available all the time, thus Femap provides descriptive enumerations (enums) which may make it easier to select the desired item instead of needing to reference the table below to determine the *index* value. These can be specified by only specifying the `zMDCType` argument for `SetContourModelDataType`.

For example, to select the **Plane Element Offset** item under **Element/Property Data**, simply specify the `zMDCType` with an *enum* and enter 0 for the other 3 arguments:

```
v.SetModelDataContourType(FVMDC_ELEM_PLATE_OFFSET,0,0,0)
```

For material items which are not as common, set `zMDCType` to `FVMDC_MATL_TABLE`, set the `zMaterialType` to a valid value (i.e., a type of material that exists in the model), then use the `iLocation` value to specify an array index location for the `mval` property on the Material object (See **Material Array Index Values**).

For example, to select **ISOTRIPIC Nu** (Poisson's Ratio), use the following:

```
v.SetModelDataContourType(FVMDC_MATL_TABLE,0,FMT_ISOTROPIC,6)
```

For property items which are not as common, set `zMDCType` to `FVMDC_PROP_TABLE`, set the `zElementType` to a valid value (i.e., a type of property that exists in the model), then use the `iLocation` value to specify an array index location for the `pval` property on the Property object (See **Accessing Property Properties**).

For example, to select "PLATE NS Mass/Area" (Nonstructural mass/area), use the following:

```
v.SetModelDataContourType(FVMDC_PROP_TABLE,FET_L_PLATE,0,7)
```


Unpublished work. © 2025 Siemens

The following table contains the enums and index values for the *commonly used* items available when selecting model data to display with **View → Advanced Post → Model Contour Data** command.

Text in View Options	Index	Constant
None Selected	0	`FVMDC_NONE_SELECTED`
Element Quality - Femap Checks		
Aspect Ratio	8	`FVMDC_ELEM_ASPECT_RATIO`
Taper	9	`FVMDC_ELEM_TAPER`
Alt Taper	10	`FVMDC_ELEM_ALT_TAPER`
Internal Angles	11	`FVMDC_ELEM_INTERNAL_ANGLE`
Skew	12	`FVMDC_ELEM_SKEW`
Warping	13	`FVMDC_ELEM_WARPING`
Nastran Warping	14	`FVMDC_ELEM_NASTRAN_WARPING`
Tet Collapse	15	`FVMDC_ELEM_TET_COLLAPSE`
Jacobian	16	`FVMDC_ELEM_JACOBIAN`
Combined Quality	17	`FVMDC_ELEM_COMBINED`
Explicit Time Step	18	`FVMDC_ELEM_EXPLICIT_TIME_STEP`
Element Quality - NX Nastran Checks (Quads)		
Quad Skew	19	`FVMDC_ELEM_NQSKEW`
Quad Taper	20	`FVMDC_ELEM_NQTAPER`
Quad Warp	21	`FVMDC_ELEM_NQWARP`
Quad IAMin	22	`FVMDC_ELEM_NQIAMIN`

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Quad IAMax	23	FVMDC_ELEM_NQIAMAX
Quad AR (Aspect Ratio)	47	FVMDC_ELEM_NQAR
Element Quality - NX Nastran Checks (Triangles)		
Tria Skew	24	FVMDC_ELEM_NTSKEW
Tria IA Max	25	FVMDC_ELEM_NTIAMAX
Tria AR (Aspect Ratio)	48	FVMDC_ELEM_NTAR
Tria EPLR	49	FVMDC_ELEM_NT6EPLR
Element Quality - NX Nastran Checks (Tetrahedrals)		
Tetra AR (Aspect Ratio)	26	FVMDC_ELEM_NTETAR
Tetra EPLR	27	FVMDC_ELEM_NTETEPLR
Tetra EPIA	50	FVMDC_ELEM_NTETEPIA
Tetra DetJ	28	FVMDC_ELEM_NTETDETJ
Element Quality - NX Nastran Checks (Hexahedrals/Bricks)		
Hex AR (Aspect Ratio)	29	FVMDC_ELEM_NHEXAR
Hex EPLR	30	FVMDC_ELEM_NHEXEPLR
Hex EPIA	51	FVMDC_ELEM_NHEXEPIA
Hex DetJ	31	FVMDC_ELEM_NHEXDETJ
Hex Warp	32	FVMDC_ELEM_NHEXWARP
Element Quality - NX Nastran Checks (Wedges)		
Penta AR	33	FVMDC_ELEM_NPENAR

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Penta EPLR	34	`FVMDC_ELEM_NPENEPLR`
Penta EPIA	52	`FVMDC_ELEM_NPENEPIA`
Penta DetJ	35	`FVMDC_ELEM_NPENDETJ`
Penta Warp	36	`FVMDC_ELEM_NPENWARP`
Element Quality - NX Nastran Checks (Pyramids)		
Pyr AR	37	`FVMDC_ELEM_NPYRAR`
Pyr EPLR	38	`FVMDC_ELEM_NPYREPLR`
Pyr EPIA	53	`FVMDC_ELEM_NPYREPIA`
Pyr Warp	39	`FVMDC_ELEM_NPYRWARP`
Pyr DetJ	40	`FVMDC_ELEM_NPYRDETJ`
Material Data		
Youngs Modulus, E	1	`FVMDC_MATL_E`
Items in Other Material Data (see above for explanation on how to use this constant with `SetModelDataContourType` to access these Material values)	2	`FVMDC_MATL_TABLE`
Element/Property Data		
Plane Element Thickness	3	`FVMDC_PROP_PLATE_THICKNESS`
Plane Element Offset	44	`FVMDC_ELEM_PLATE_OFFSET`
Bar/Beam/Rod Area	4	`FVMDC_PROP_LINE_AREA`
Bar/Beam I1	5	`FVMDC_PROP_LINE_I1`

Unpublished work. © 2025 Siemens

Text in View Options	Index	Constant
Bar/Beam I2	6	`FVMDC_PROP_LINE_I2`
Items in Other Property Data (see above for explanation on how to use this constant with `SetModelDataContourType` to access these Material values)	7	`FVMDC_PROP_TABLE`
Entity ID		
Element (ID)	41	`FVMDC_ELEM_ID`
Material (ID)	42	`FVMDC_MATL_ID`
Property (ID)	43	`FVMDC_PROP_ID`

View Object Methods

Initialize

`Initialize`	
`( void )`	
Description:	
This initializes a view object to the default settings.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
View Objects have many properties, and it is difficult and time consuming to initialize them all individually. This method initializes the object just like creating a new view in Femap with the default view settings. You can then selectively modify just the properties that you need.
Example:
None

GetMultiGroupList

`GetMultiGroupList`	
`( nGroups, nGroupID )`	
Description:	
This method retrieves the group ids used in the current view for the Multi Group Display.	
Input:	
None	None
Output:	
`INT4 nGroups`	Number of groups in `nGroupID`.
`INT4 nGroupID[0..nGroups-1]`	The group IDs that have been referenced in Multi Group display (Positive ID value = *Shown*, Negative ID value = *Hidden*).
Return Code:	
`FE_NO_MEMORY`	Unable to allocate memory for group list.
`FE_NOT_AVAILABLE`	No groups have be referenced in the Multi Group Display.
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetMultiGroupList

SetMultiGroupList **(bClear, nGroups, nGroupID)**	
Description:	
This method sets the group ids to be used in the current view for the Multi Group Display.	
Input:	
`INT4 bClear`	If True then clear the list of groups in the multi group display before adding groups in `nGroupID`.
`INT4 nGroups`	Number of groups in `nGroupID`.
`INT4 nGroupID[0..nGroups-1]`	The group IDs used in the Multi Group display (Positive ID value = *Show*, Negative ID value = *Hide*).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

SetMultiGroupListFromSets

SetMultiGroupListFromSets

(bClear, nShowSetID, nHideSetID)

Description:

This method sets the group ids in Set Objects to be used in the current view for the Multi Group Display.

Input:

`INT4 bClear`	If True then clear the list of groups in the multi group display before setting the Hide/Show group IDs specified in Set Objects with IDs `nShowSetID` and `nHideSetID`.
`INT4 nShowSetID`	Set Object containing Group IDs to be set as Show in the Multi Group Display.
`INT4 nHideSetID`	Set Object containing Group IDs to be set as Hide in the Multi Group Display.

Output:

None	None

Return Code:

None	None

Remarks/Usage:

None

Example:

Sample API that sets the current view to Multi Group Display with Groups 1, 3, 5 and 7 in Show mode, all other groups in Hide Mode.

```
Sub Main
    Dim App as femap.model
    Set App = feFemap()

    Dim feView As femap.View
    Set feView = App.feView

    Dim grSet As femap.Set
    Set grSet = App.feSet

    Dim feGroup As femap.group
```


Unpublished work. © 2025 Siemens

```
    Set feGroup = App.feGroup

    Dim viewID As Long

    If App.feAppGetActiveView( viewID ) = FE_OK Then
        If feView.Get( viewID ) = FE_OK Then
            grSet.Add(1)
            grSet.Add(3)
            grSet.Add(5)
            grSet.Add(7)
            feView.group = -2 ‘Set View to Multi Group Display
            feView.SetMultiGroupListFromSets( True, grSet.ID, 0 )
            feView.Put( viewID )
            feView.Regenerate()
        End If
    End If

End Sub
```

ClearMultiGroupList

ClearMultiGroupList

(void)

Description:	
This method clear the groups in the Multi Group Display.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

Unpublished work. © 2025 Siemens

Visible

Visible	
(visible)	
Description:	
This makes a view visible on the screen, or removes it from the screen.	
Input:	
`BOOL visible`	Set to True to make a view visible, False to remove it.
Output:	
None	None
Return Code:	
`FE_FAIL`	The operation failed. Either you asked to make a view visible that was already visible, or to remove a view that was not visible.
Remarks/Usage:	
This method accomplishes the same task as the **View** → **Activate** command.	
Example:	
None	

Redraw

Redraw	
(void)	
Description:	
Redraws the current view.	
Input:	
None	None

Unpublished work. © 2025 Siemens

Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view that is visible on the screen. Same as the **View** → **Redraw** command.	
Example:	
None	

Regenerate

`Regenerate`
 	
`( void )`	

Description:	
Regenerates the current view.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view that is visible on the screen. Same as the **View** → **Regenerate** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

IsLayerVisible

IsLayerVisible	
(layerID)	
Description:	
Indicates whether the selected layer is in the visible layer list.	
Input:	
`INT4 layerID`	ID of layer to check.
Output:	
None	None
Return Code:	
`FE_FAIL`	Selected layer is not in the visible layer list (FE_OK means layer is in the visible layer list).
Remarks/Usage:	
The current object must be loaded with a view that is visible on the screen. This command does not tell whether the visible layer list is actually being used. If you are displaying all layers, layers that are not in the visible layer list will still be reported as not visible, even though they are because the list is not used.	
Example:	
None	

SetLayerVisible

SetLayerVisible
(layerID, visible)
Description:
Adds or removes layers from the visible layer list.
Input:

Unpublished work. © 2025 Siemens

`INT4 layerID`	ID of layer to modify.
`BOOL visible`	Set to True to make the layer visible, False to remove it from the visible list.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view that is visible on the screen. In addition to this command, you must also change the `ShowAllLayers` property to make sure that the visible layer list is being used (instead of all layers).	
Example:	
None	

Rotate

`Rotate` **`( oX, oY, oZ, dX, dY, dZ, angle )`**	
Description:	
Rotates the current view around the specified axis.	
Input:	
`REAL8 oX`	The origin of the axis of rotation - in global rectangular coordinates.
`REAL8 oY`	
`REAL8 oZ`	
`REAL8 dX`	The components of the axis of rotation.
`REAL8 dY`	

Unpublished work. © 2025 Siemens

`REAL8 dZ`	
`REAL8 angle`	The angle of rotation, in degrees.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view from your model. This command simply updates the view object. To update the display in Femap, the view must be `Put` to the database, and then you must `Redraw`.	
Example:	
None	

AlignCSys

`AlignCSys`
`( csysID )`
Description:
Rotates the current view to be aligned with the XY plane of the selected coordinate system.
Input:

`INT4 csysID`	The ID of the coordinate system used for alignment.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

The current object must be loaded with a view from your model. This command simply updates the view object. To update the display in Femap, the view must be `Put` to the database, and then you must `Redraw`.
Example:
None

AlignVector

`AlignVector` **`( oX, oY, oZ, dX, dY, dZ )`**	
Description:	
Rotates the current view to be aligned looking along the specified vector.	
Input:	
`REAL8 oX`	The origin of the alignment axis - in global rectangular coordinates.
`REAL8 oY`	
`REAL8 oZ`	
`REAL8 dX`	The components of the axis of alignment.
`REAL8 dY`	
`REAL8 dZ`	
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view from your model. This command simply updates the view object. To update the display in Femap, the view must be **Put** to the database, and then you must **Redraw**.	

Unpublished work. © 2025 Siemens

Example:
None

Align2Vector

Align2Vector **(vZx, vZy, vZz, vXYx, vXYy, vXYz, bAutoXY)**	
Description:	
Rotates the current view to be aligned looking along the specified vector.	
Input:	
`REAL8 vZx`	The components of the axis of alignment (normal to the screen).
`REAL8 vZy`	
`REAL8 vZz`	
`REAL8 vXYx`	The components of a vector used to determine the X and Y (in screen) axes. The normal vector (vZx,vZy,vZz) is crossed with this vector to determine the Y vector, which is then crossed again to determine X.
`REAL8 vXYy`	
`REAL8 vXYz`	
`BOOL bAutoXY`	If True, the X and Y vectors are swapped so that the model better fits into the window shape.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view from your model. This command simply updates the view object. To update the display in Femap, the view must be `Put` to the database, and then you must `Redraw`.	

Unpublished work. © 2025 Siemens

Example:
None

AlignWorkplane

AlignWorkplane **(void)**	
Description:	
Rotates the current view to be aligned with the normal to the workplane.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view from your model. This command simply updates the view object. To update the display in Femap, the View must be `Put` to the database, and then you must `Redraw`.	
Example:	
None	

AutoscaleAll

AutoscaleAll **(redoBox)**
Description:

Unpublished work. © 2025 Siemens

Changes the magnification so that the current model is scaled to fit in the view.	
Input:	
`BOOL redoBox`	If set to True, then the imaginary box around the model that is used for autoscaling is recomputed. If set to False, the existing box is used.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view from your model. This command simply updates the view object. To update the display in Femap, the View must be `Put` to the database, and then you must `Redraw`.	
Example:	
None	

AutoscaleVisible

`AutoscaleVisible` **`( redoBox )`**	
Description:	
Changes the magnification so that the visible portions of the model are scaled to fit in the view.	
Input:	
`BOOL redoBox`	If set to True, then the imaginary box around the model that is used for autoscaling is recomputed. If set to False, the existing box is used.
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
The current object must be loaded with a view from your model that is currently visible in a window. This method differs from `AutoscaleAll` in that the current group, layers and display options are considered when scaling the view. This command simply updates the view object. To update the display in Femap, the View must be `Put` to the database, and then you must `Redraw`.
Example:
None

FitVisible

`FitVisible`

`( redoBox )`

Description:	
Changes the magnification so that the visible portions of the model are scaled to fit in the view, based on the model's current orientation.	
Input:	
`BOOL redoBox`	If set to True, then the imaginary box around the model that is used to fit the visible entities based on orientation is recomputed. If set to False, the existing box is used.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
The current object must be loaded with a view from your model that is currently visible in a window. This method differs from AutoscaleVisible in that the model's orientation is also taken into account. This command simply updates the view object. To update the display in Femap, the View must be `Put` to the database, and then you must `Redraw`.	
None	

Unpublished work. © 2025 Siemens

Example:
None

SetModelDataContourForm

SetModelDataContourForm **(zContourFormat)**	
Description:	
Specifies the **Show As** option in the Contour Model Data dialog box, which can be **Contour**, **Criteria**, or **Beam Diagram**.	
Input:	
`enum zContourFormat`	Specify an option for Show As, from these options: Contour = `FCF_CONTOUR(1)` Criteria = `FCF_CRITERIA(2)` Beam Diagram = `FCF_BEAM_DIAGRAM(3)`
Output:	
None	None
Return Code:	
`FE_FAIL`	Selected option for `zContourFormat` is not valid.
Remarks/Usage:	
Used in conjunction with `SetModelDataContourType` to display the desired model data as a contour, criteria, or beam diagram plot.	
Example:	
None	

GetModelDataContourForm

Unpublished work. © 2025 Siemens

GetModelDataContourForm **(zContourFormat)**	
Description:	
Determines the current option specified for **Show As** option in the Contour Model Data dialog box, which can be **Contour**, **Criteria**, or **Beam Diagram**.	
Input:	
None	None
Output:	
`enum zContourFormat`	Option currently set for **Show As**, which will be one of these options: Contour = `FCF_CONTOUR(1)` Criteria = `FCF_CRITERIA(2)` Beam Diagram = `FCF_BEAM_DIAGRAM(3)`
Return Code:	
`FE_FAIL`	Selected option for `zContourFormat` is not valid.
Remarks/Usage:	
Used in conjunction with GetModelDataContourType to determine many of the options currently being used for a plot of model data or discrete values.	
Example:	
None	

SetModelDataContourType

SetModelDataContourType **(zMDCType, zElementType, zMaterialType, ilocation)**
Description:
Selects the type of model data to display from the *tree structure* in the **Contour Data From** section of the Contour Model Data dialog box. This option can also be used for a Discrete Value plot.

Unpublished work. © 2025 Siemens

Input:	
`enum zMDCType`	Used to specify which model data item to display as a contour, criteria, beam diagram. See **Model Contour Data Types** for more information about selecting the appropriate model data item. If using any option other than FVMDC_MATL_TABLE or FVMDC_PROP_TABLE, all other arguments in this method are ignored.
`enum zElementType`	Only used when `zMDCType` is set to FVMDC_PROP_TABLE. Indicates the element/property type to use for the index value specified in `iLocation` (See **Element Types** in **Femap Constants**).
`enum zMaterialType`	Only used when `zMDCType` is set to FVMDC_MATL_TABLE. Indicates the material type to use for the index value specified in `iLocation` (See **"Material Types"** in **Femap Constants**).
`INT4 iLocation`	Only used when `zMDCType` is set to FVMDC_MATL_TABLE or FVMDC_PROP_TABLE. ID of the index in the Material Array Index (See **Material Array Index Values**) or Property Array Index (See **Property Array Index Values**) to use to create the plot.
Output:	
Return Code:	
`FE_FAIL`	Selected `zElemenType` or `zMaterialType` does not exist in the model.
Remarks/Usage:	
Used in conjunction with `SetModelDataContourForm` to display the desired model data as a contour, criteria, or beam diagram plot. To plot discrete values, set the `ModelDataContourOn` property on the View object to True and use `DiscreteValueSetID` to select an appropriate discrete value set.	
Example:	
None	

GetModelDataContourType

Unpublished work. © 2025 Siemens

GetModelDataContourType	
(zMDCType, zElementType, zMaterialType, ilocation)	
Description:	
Determines the type of model data from the *tree structure* in the **Contour Data From** section of the Contour Model Data dialog box currently being displayed. This option can also be used for a Discrete Value plot.	
Input:	
None	None
Output:	
`enum zMDCType`	Used to specify which model data item to display as a contour, criteria, beam diagram. See **Model Contour Data Types** for more information about selecting the appropriate model data item. If using any option other than `FVMDC_MATL_TABLE` or `FVMDC_PROP_TABLE`, all other arguments in this method are ignored.
`enum zElementType`	Only used when `zMDCType` is set to `FVMDC_PROP_TABLE`. Indicates the element/property type to use for the index value specified in iLocation (See **Element Types** in **Femap Constants**).
`enum zMaterialType`	Only used when `zMDCType` is set to `FVMDC_MATL_TABLE`. Indicates the material type to use for the index value specified in `iLocation` (See **"Material Types"** in **Femap Constants**).
`INT4 iLocation`	Only used when `zMDCType` is set to `FVMDC_MATL_TABLE` or `FVMDC_PROP_TABLE`. ID of the index in the Material Array Index (See **Material Array Index Values**) or Property Array Index (See **Property Array Index Values**) to use to create the plot.
Return Code:	
None	None
Remarks/Usage:	
Used in conjunction with `GetModelDataContourFrom` to determine many of the options currently being used for a plot of model data or discrete values.	

Unpublished work. © 2025 Siemens

Example:
None

GetFontData

`GetFontData`
 	
`( pdPointHeight, pbBold, pbItalic, ppcName )`	

Description:	
This function returns parameters from View objects. Requires a `Get` before use.	
Input:	
None	None
Output:	
`INT4 pdPointHeight`	The size of the text.
`BOOL pbBold`	If True if text is Bold, otherwise returns False.
`BOOL pbItalic`	If True if text is Italicized, otherwise returns False.
`STRING ppcName`	Name of Font.
Return Code:	
`FE_OK`	Values retrieved.
Remarks/Usage:	
None	
Example:	
None	

SetFontData

Unpublished work. © 2025 Siemens

SetFontData	
(pdPointHeight, pbBold, pbItalic, ppcName)	
Description:	
This function sets parameters on View objects. Requires a `Put` after use to update View object.	
Input:	
`INT4 pdPointHeight`	The size of the text.
`BOOL pbBold`	If True if text is Bold, otherwise returns False.
`BOOL pbItalic`	If True if text is Italicized, otherwise returns False.
`STRING ppcName`	Name of Font.
Output:	
None	None
Return Code:	
`FE_OK`	Values set.
Remarks/Usage:	
None	
Example:	
None	

SetClippingPlane

SetClippingPlane
(origin, direction)
Description:
Updates and activates the Clipping Plane for the View.

Unpublished work. © 2025 Siemens

Input:	
`REAL8 origin[0..2]`	The coordinates of a location on the Clipping Plane in Global Rectangular coordinates.
`REAL8 direction[0..2]`	The components of a vector that is normal to the Clipping Plane in Global Rectangular coordinates.
Output:	
None	None
Return Code:	
`FE_FAIL`	Either the origin or direction are not properly specified
Remarks/Usage:	
This method does several things. It sets the location and orientation of the clipping plane and also enables the plane and sets the clipping direction to Positive which matches the direction specified. All of these options can also be accessed and modified via properties on this object.	
Example:	
None	

ViewOrient Objects

As of Femap version 11.3, the data and methods of the View Object which control orientation should no longer be used, as that data is only used to provide backwards compatibility. Please see the overall description of the View Object (See **View Objects**) for restrictions on using the data and methods.

The ViewOrient objects provide access to the view orientation functionality using options set using various commands on the **View** menu. More specifically, provides access to the options found in the **View** → **Rotate** → **Advanced Rotate...** command. Many of the commands on the **View** → **Rotate...** menu simply set a combination of options in the Advanced Rotation Options dialog box to produce the desired behavior during dynamic rotation. They are derived from Entity objects, so, in addition to the properties and methods described here, all of the properties and methods defined for Entity objects are also available for ViewOrient objects, except the standard `Get` and `Put`. See **Common Entity Properties and Methods** for more information.

The `Get` method and `Put` methods are slightly different than standard, but require you use the ID of the View where you want to change the orientation options. Dynamic rotation for each View in your model is controlled by a corresponding ViewOrient object, which will have the same ID as the View (i.e., to manipulate the orientation options for View 1, use the `Get` method to load ViewOrient object with values from View 1, then use the `Put` method to make changes to the view).

Use the `feViewOrient` method of the Femap Application object to access ViewOrient Objects.

Unpublished work. © 2025 Siemens

ViewOrient Object Properties

Property	Description

`REAL8 Center[0..2]`
 	
or
 	
`VARIANT vCenter`
	Location of view center. This is set using the **View → Center** command in the User Interface.

`REAL8 Magnification`	Magnification factor (1=AutoScale). This is set using the **View → Magnify** command in the User Interface.
`BOOL ModelAlwaysInFront`	When True, the **Model Always in Front of Viewer** option in the **Rotation Axis** section of the Advanced Rotation Options dialog box will be turned on.
`BOOL AutoAspectRatio`	Flag to indicate if the Aspect Ratio of the view should be automatically calculated. When True, the **AutoAspect** check box for the **View Aspect Ration** option of the **View→ Options** command will be checked. When False, it will use the value specified for **AspectRatio**.
`REAL8 AspectRatio`	Specifies the Aspect Ratio for the view, which sets the value in the **Aspect Ratio** field for the **View Aspect Ratio** option of the **View→ Options** command.
`BOOL Perspective`	Flag to indicate if **Perspective** mode is turned on. When True, the **Option On** check box for the **Perspective** option of the **View → Options** command will be checked.
`REAL8 PerspectiveAngle`	Specifies the **Perspective Angle**, which sets the value in the **Angle** field for the **Perspective** option of the **View→ Options** command.
`INT4 RotationAxisOption`	Sets the **Rotation Axes** option in the Advanced Rotation Options dialog box: 0=View Axes, 1=Model Axes, 2=Coordinate System, 3=Vector.
`INT4 RotationCenterOption`	Sets the **Rotation Center** option in the Advanced Rotation Options dialog box: 0=View

Unpublished work. © 2025 Siemens

Property	Description
	Center, 1=Screen Center, 2=Coordinate System, 3=Location.
`INT4 RotationCoordSys`	ID of coordinate system to rotate about. Set in the **Rotation Axes** section in the Advanced Rotation Options dialog box. Only used when `RotationAxisOption` is set to 2 (Coordinate System).
`BOOL SingleAxis`	When True, the **Single Axis Rotation** option in the **Rotation Axis** section of the Advanced Rotation Options dialog box will be turned on.

`REAL8 RotationCenter[0..2]`
or
`VARIANT vRotationCenter`
	Center of rotation for dynamic rotation, which is set using the **Location** button in the **Rotation Center** section of the Advanced Rotation Options dialog box. Also, sets the base coordinates of the rotation vector. Only used when `RotationCenterOption` is set to 3 (Location).

`REAL8 RotationVector[0..2]`
or
`VARIANT vRotationVector`
	Axis of rotation for dynamic rotation, which is set using the **Vector** button in the **Rotation Center** section of the Advanced Rotation Options dialog box.

`INT4 SecondaryRotationAxesOption`	Sets the **Rotate Around** option in the **View → Rotate → Model** command, which controls how precise rotation to a particular orientation typically occurs. -1 (or any negative integer)=Screen Axis, 0=Basic Rectangular Coordinate System, 1=Basic Cylindrical Coordinate System, 2=Basic Spherical Coordinate System, 3 or above=ID of User-Defined Coordinate System.

ViewOrient Object Methods

SetEyeDirection

Unpublished work. © 2025 Siemens

SetEyeDirection	
(dEyeX, dEyeY, dEyeZ, dRightX, dRightY, dRightZ)	
Description:	
Sets the coordinates of the *eye direction* vector and *right direction* vector to align the view.	
Input:	
`REAL8 dEyeX`	X coordinate of Eye direction vector.
`REAL8 dEyeY`	Y coordinate of Eye direction vector.
`REAL8 dEyeZ`	Z coordinate of Eye direction vector.
`REAL8 dRightX`	X coordinate of Right direction vector.
`REAL8 dRightY`	Y coordinate of Right direction vector.
`REAL8 dRightZ`	Z coordinate of Right direction vector.
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
These values are set in the User Interface using the **View → Align → Eye and Directions** command.	
Example:	
None	

GetEyeDirection

Unpublished work. © 2025 Siemens

GetEyeDirection	
(dEyeX, dEyeY, dEyeZ, dRightX, dRightY, dRightZ)	
Description:	
Returns the coordinates of *eye direction* vector and *right direction* vector from the view.	
Input:	
None	None
Output:	
`REAL8 dEyeX`	X coordinate of Eye direction vector.
`REAL8 dEyeY`	Y coordinate of Eye direction vector.
`REAL8 dEyeZ`	Z coordinate of Eye direction vector.
`REAL8 dRightX`	X coordinate of Right direction vector.
`REAL8 dRightY`	Y coordinate of Right direction vector.
`REAL8 dRightZ`	Z coordinate of Right direction vector.
Return Code:	
None	None
Remarks/Usage:	
These values can be reviewed in the User Interface using the **View → Align → Eye and Directions** command.	
Example:	
None	

SetRotationAngles

Unpublished work. © 2025 Siemens

SetRotationAngles	
(dX, dY, dZ)	
Description:	
Set the orientation of the view currently loaded in the ViewOrient object by specifying Euler angles relative to the Basic Rectangular Coordinate System.	
Input:	
`REAL8 dX`	X angle in degrees (-179.9999 to 180.0).
`REAL8 dY`	Y angle in degrees (-179.9999 to 180.0).
`REAL8 dZ`	Z angle in degrees (-179.9999 to 180.0).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This function provides similar functionality to the **View → Rotate → Model** command when **Rotate Around** is set to **0..Basic Rectangular** and values are entered for X, Y, and/or Z. Also, if values are entered outside the expected range, they will automatically be converted to values within the range.	
Example:	
None	

GetRotationAngles

GetRotationAngles
(dX, dY, dZ)
Description:
Returns the orientation of the view currently loaded in the ViewOrient object in Euler angles relative to the Basic Rectangular Coordinate System.

Unpublished work. © 2025 Siemens

Input:	
None	None
Output:	
`REAL8 dX`	X angle in degrees (-179.9999 to 180.0).
`REAL8 dY`	Y angle in degrees (-179.9999 to 180.0).
`REAL8 dZ`	Z angle in degrees (-179.9999 to 180.0).
Return Code:	
None	None
Remarks/Usage:	
This function returns the X, Y, and/or Z values that are displayed when using the **View** → **Rotate** → **Model** command and **Rotate Around** is set to **0..Basic Rectangular**.	
Example:	
None	

SnapToAxes

`SnapToAxes` **`( void )`**	
Description:	
Automatically aligns the view to the closest orthogonal axes relative to current model orientation.	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This call is used in the **Custom Tools → Views → Snap to Closest Orthogonal View** command.
Example:
None

ViewRotation

`ViewRotation`	
`( qUseCenter, dX, dY, dZ )`	
Description:	
Turns on the **Model Always in Front of Viewer** option (`ModelAlwaysInFront`=True), sets Rotation Axes to View Axes (`RotationAxisOption`=0), and sets Rotation Center to View Center (`RotationCenterOption`=0). When `qUseCenter` Boolean is set to True, sets Rotation Center to Location (`RotationCenterOption`=3) and the additional arguments are used to provide X (`RotationCenter[0]`), Y (`RotationCenter[1]`), and Z (`RotationCenter[2]`) coordinates for the rotation center.	
Input:	
`BOOL qUseCenter`	When True, sets **Rotation Center to Location** in the Advanced Rotation Options dialog box and use the coordinates specified in dX, dY, and dZ as the location of the rotations center.
`REAL8 dX`	X coordinate of Rotation Center (only used when `qUseCenter`=True).
`REAL8 dY`	Y coordinate of Rotation Center (only used when `qUseCenter`=True).
`REAL8 dZ`	Z coordinate of Rotation Center (only used when `qUseCenter`=True).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This function provides similar functionality to the **View → Rotate → Rotate About View Center**; **View → Rotate → Rotate About Rotation Center**; and **View → Rotate → Rotate Around View Axis** commands.	

Unpublished work. © 2025 Siemens

Example:
None

ModelRotation

`ModelRotation` **`( qUseCenter, dX, dY, dZ )`**	
Description:	
Turns on the **Model Always in Front of Viewer** option (`ModelAlwaysInFront`=True), sets Rotation Axes to Model Axes (`RotationAxisOption`=1), and sets Rotation Center to View Center (`RotationCenterOption`=0). When `qUseCenter` Boolean is set to True, sets Rotation Center to Location (`RotationCenterOption`=3) and the additional arguments are used to provide X (`RotationCenter[0]`), `Y (RotationCenter[1])`, and `Z (RotationCenter[2])` coordinates for the rotation center.	
Input:	
`BOOL qUseCenter`	When True, sets **Rotation Center to Location** in the Advanced Rotation Options dialog box and use the coordinates specified in dX, dY, and dZ as the location of the rotations center.
`REAL8 dX`	X coordinate of Rotation Center (only used when `qUseCenter`=True).
`REAL8 dY`	Y coordinate of Rotation Center (only used when `qUseCenter`=True).
`REAL8 dZ`	Z coordinate of Rotation Center (only used when `qUseCenter`=True).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This function provides similar functionality to the **View → Rotate → Rotate About View Center**; **View → Rotate → Rotate About Rotation Center**; and **View → Rotate → Rotate Around Model Axis** commands.	

Unpublished work. © 2025 Siemens

Example:
None

CoordSysRotation

CoordSysRotation **(iCoordSys, qUseCenter, dX, dY, dZ)**	
Description:	
Turns on the **Model Always in Front of Viewer** option (`ModelAlwaysInFront`=True), sets Rotation Axes to Coordinate System (`RotationAxisOption`=2), sets the specified Coordinate System in the drop-down (`RotationCoordSys=iCoordSys`), and sets Rotation Center to Coordinate System (`RotationCenterOption`=2). When `qUseCenter` Boolean is set to True, sets Rotation Center to Location (`RotationCenterOption`=3) and the additional arguments are used to provide X (`RotationCenter[0]`), Y (`RotationCenter[1]`), and Z (`RotationCenter[2]`) coordinates for the rotation center.	
Input:	
`INT4 iCoordSys`	ID of Coordinate System to rotate around
`BOOL qUseCenter`	When True, sets **Rotation Center to Location** in the Advanced Rotation Options dialog box and use the coordinates specified in dX, dY, and dZ as the location of the rotations center.
`REAL8 dX`	X coordinate of Rotation Center (only used when `qUseCenter`=True).
`REAL8 dY`	Y coordinate of Rotation Center (only used when `qUseCenter`=True).
`REAL8 dZ`	Z coordinate of Rotation Center (only used when `qUseCenter`=True).
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	

Unpublished work. © 2025 Siemens

This function provides similar functionality to the **View → Rotate → Rotate Around Coordinate System** command.
Example:
None

VectorRotation

`VectorRotation` **`( dX, dY, dZ, dVX, dVY, dVZ )`**	
Description:	
Turns on the **Model Always in Front of Viewer** option (`ModelAlwaysInFront`=True), sets Rotation Axes to Vector (`RotationAxisOption`=3), and sets Rotation Center to Location (`RotationCenterOption`=3). The arguments are used to provide `X (RotationCenter[0])`, `Y (RotationCenter[1])`, and `Z (RotationCenter[2])` coordinates for the rotation center (base of vector) and `X (RotationVector[0])`, `Y (RotationVector[1])`, and `Z (RotationVector[2])` coordinates of the tip of the vector.	
Input:	
`REAL8 dX`	X coordinate of Rotation Center.
`REAL8 dY`	Y coordinate of Rotation Center.
`REAL8 dZ`	Z coordinate of Rotation Center.
`REAL8 dVX`	X coordinate of Rotation Vector.
`REAL8 dVY`	Y coordinate of Rotation Vector.
`REAL8 dVZ`	Z coordinate of Rotation Vector.
Output:	
None	None
Return Code:	
None	None

Unpublished work. © 2025 Siemens

Remarks/Usage:
This function provides similar functionality to the **View** → **Rotate** → **Rotate Around Vector** command.
Example:
None

RollThru

`RollThru` **`( void )`**	
Description:	
Turns off the **Model Always in Front of Viewer** option (`ModelAlwaysInFront`=False), sets Rotation Axes to View Axis (`RotationAxisOption`=0), and sets Rotation Center to Screen Center (`RotationCenterOption`=1).	
Input:	
None	None
Output:	
None	None
Return Code:	
None	None
Remarks/Usage:	
This function provides similar functionality to the **View** → **Rotate** → **Roll-Thru** command.	
Example:	
None	

GetFirstPathID

Unpublished work. © 2025 Siemens

`GetFirstPathID`	
`( piPathID )`	
Description:	
Gets the first ID of the stored play backs.	
Input:	
None	None
Output:	
`INT4 piPathID`	The ID of the first path.
Return Code:	
None	None
Remarks/Usage:	
None	
Example:	
None	

GetNextPathID

`GetNextPathID`	
`( piPathID )`	
Description:	
Gets the next ID of the stored play backs.	
Input:	
None	None
Output:	
`INT4 piPathID`	The ID of the next path.
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
None	
Example:	
None	

Spin

Spin **(nFrames, dX, dY, dZ, dVX, dVY, dVZ, pdFPS)**	
Description:	
This function measures the speed of dynamic rotation as well as spins the active model.	
Input:	
`INT4 nFrames`	Sets the number of Frames to spin. When `nFrames`=0 the model will remain spinning until you click anywhere in the interface or graphics window.
`REAL8 dX`	X coordinate of Rotation Center.
`REAL8 dY`	Y coordinate of Rotation Center.
`REAL8 dZ`	Z coordinate of Rotation Center.
`REAL8 dVX`	X coordinate of Rotation vector.
`REAL8 dVY`	Y coordinate of Rotation vector.
`REAL8 dVZ`	Z coordinate of Rotation vector.
Output:	
`REAL8 pdFPS`	Frames per second.
Return Code:	

Unpublished work. © 2025 Siemens

None	None
Remarks/Usage:	
A portion of these values are set in the User Interface using the **View** → **Rotate** → **Spin Around Vector** command.	
Example:	
None	

Unpublished work. © 2025 Siemens

Unpublished work. © 2025 Siemens

6. Femap Events

Events provide a way for your application to be notified when certain operations are happening in the Femap session. For example, by processing events, you will be notified when a new model is opened, a command is processed or Femap is shut down, and your program can then respond appropriately to that event.

Receiving and Processing Events

To receive and process events in your application, you must do the following:

1. Register your application with Femap as an add-in. You do this by calling the `feAppRegisterAddInPane` method (see **feAppRegisterAddInPane**). Event messages are only sent to applications that are registered as add-ins. When calling this method, you should specify the handle to the window that contains the message loop that will process the events.
2. Call the Windows API function `RegisterWindowMessage` to obtain the message identifier for the event message. The syntax of this call will be something like:

   ```
   EventMessage = RegisterWindowMessage ( "FE_EVENT_MESSAGE" );
   ```

   You must register the `FE_EVENT_MESSAGE` string to select the proper message.
3. Modify your window procedure to process the event messages that Femap sends. The format of these messages is described later in this section. You do not have to process any particular messages. Only process the messages that you need for your application.

   Note

   Femap sends the event messages to your application. Femap is not waiting for a response from you; it will go on processing whatever other commands or actions are requested.

Alternate Method - Event Callbacks

As an alternative to handling event messages, if you can build your event functionality as a stand-alone program file, basic script or executable, then you can register an event callback to call that code. This approach is often simpler if the code you want to execute can be stand-alone. Simply call `feAppEventCallback` (see **feAppEventCallback**) to hook your code to a particular event. After that every time the event occurs, your code will be called.

If the event that your callback processes uses the `lParam` value (like `FEVENT_COMMAND`), you can access it from your callback routine by using `EventLParam` described in **Global Integer Properties**.

Unpublished work. © 2025 Siemens

Format of Event Messages

All windows messages contain four parameters.

Parameter	Description
`hWnd`	The handle to the window where the message is being sent. Not really very important in this case.
`message`	The message type. This parameter will always be equal to the return value from `RegisterWindowMessage( FE_EVENT_MESSAGE )` for event messages. This is how you will be able to distinguish a Femap event message from other windows messages coming to your window.
`wParam`	For Femap event messages, this parameter indicates the type of event that has occurred. Refer to the various events for more information.
`lParam`	For Femap event messages, this parameter contains additional data for that event. Refer to the various events for more information.

The Femap Events

The following events are currently defined in Femap.

wParam	Event	Description
1	`FEVENT_INITIALIZE`	Femap is initializing for the first time. `lParam` is not used.
2	`FEVENT_NEWMODEL`	Starting a new model, or opening an existing model. Any previous model that was in use has now been closed. `lParam` is not used.
3	`FEVENT_ENDMODEL`	Current model has been closed. `lParam` is not used. To determine the identifier of the model about to be closed, use `FEVENT_CLOSINGMODEL`.
4	`FEVENT_SHUTDOWN`	Femap is being shut down. This event is issued twice during the shutdown process. The first event has `lParam`=0 and occurs prior to a question to save the model and

Unpublished work. © 2025 Siemens

wParam	Event	Description
		possibly abort the shutdown. The second event has `lParam`=1 and indicates Femap is definitely closing.
5	`FEVENT_COMMAND`	A command has been executed - either from the menu, or thru some other mechanism. `lParam` is the command number that was executed.
6	`FEVENT_PROGSTART`	Indicates that a program file is starting execution. lParam is not used.
7	`FEVENT_PROGEND`	Indicates that a program file is ending execution. lParam is not used.
8	`FEVENT_DRAWSTART`	Indicates that a redraw or regenerate is beginning. Neither this nor the `FEVENT_DRAWEND` event are issued every time a view is updated - they are only issued if a standard **Redraw** or **Regenerate** command are selected from the menu or the API, or when some other action or command triggers a full redraw or regenerate.
9	`FEVENT_DRAWEND`	Indicates that a redraw or regenerate is ending.
10	`FEVENT_ANALYSISEND`	Indicates that an analysis that Femap was monitoring has been completed.
11	`FEVENT_RESULTSEND`	Indicates that the automatic loading of results after an analysis finishes has completed.
12	`FEVENT_SWITCHMODEL`	Indicates that the active model has been switched to a different model. `lParam` contains an identifier of the new active model - the same value that is returned by `feAppGetModel()`. To determine which model is about to be left, use `FEVENT_LEAVINGMODEL`.
13	`FEVENT_SELECTOR`	Indicates when anything happens to the Selector (**Select** Toolbar). The value of `lParam` lets you know what has occurred: >0=An ID (`lParam` value) was selected in Single Pick mode

Unpublished work. © 2025 Siemens

wParam	Event	Description
		-1=Selection was updated in Multiple Pick mode -2=Related entities were added to the selection -3=A group was loaded into the selection -4=Selection was *Grown* -5=Selection was *Shrunk* -999=Selection was cleared
14	`FEVENT_UPDATEDSAVEDSET`	Indicates a *SavedSet* Set Object has been updated.
15	`FEVENT_LEAVINGMODEL`	Used in conjunction with `FEVENT_SWITCHMODEL` and indicates about to leave a model. `lParam` contains the identifier of the model being left.
16	`FEVENT_CLOSINGMODEL`	Used in conjunction with `FEVENT_ENDMODEL` and indicates closing a model. `lParam` contains the identifier of the model being closed.
17	`FEVENT_FILEOPENEND`	Indicates the File > Open command has completed.
18	`FEVENT_FILEIMPORTEND`	Indicates that the Geometry, Neutral File or Fibersim import commands have completed.

When processing the `FEVENT_COMMAND` event, you will need to know the command number that you are looking for. Although we do not publish these command numbers, it is easy to find them. Simply process the`FEVENT_COMMAND` event, run your application, switch to Femap and execute the command that you want to process. Remember the value of `lParam` that you receive with the `FEVENT_COMMAND` event that you receive. It will be the same each time you perform that command. Several examples are as follows:

Command	lParam
View → Redraw	2001
View → Regenerate	2002
Tools → Undo	1101
Tools → Redo	1102
File → Save	1003

Unpublished work. © 2025 Siemens

Command	lParam
File → Open	1002
File → Exit	1025

Using Femap Events

The program below shows an example of how Femap Events can be captured in your application. The code does not actually do anything after watching the events, it provides a framework that you can use to get started.

```
Global Const FEVENT_INITIALIZE = 1
Global Const FEVENT_NEWMODEL   = 2
Global Const FEVENT_ENDMODEL   = 3
Global Const FEVENT_SHUTDOWN   = 4
Global Const FEVENT_COMMAND    = 5
Global Const FEVENT_PROGSTART  = 6
Global Const FEVENT_PROGEND    = 7

Dim WM_FEMAP_MESSAGE   As Long
Dim PreviousWindowProc As Long

Dim hMainForm As Long
Dim hMainWnd As Long

Public femap As Object

Const GWL_WNDPROC              = (-4&)
Const GWL_HWNDPARENT           = (-8)

Public Declare Function RegisterWindowMessage Lib "user32" _
```


Unpublished work. © 2025 Siemens

```
    Alias "RegisterWindowMessageA" (ByVal lpString As String) As Long

Public Declare Function SetWindowLong Lib "user32" _
    Alias "SetWindowLongA" (ByVal hwnd As Long, _
    ByVal nIndex As Long, ByVal dwNewLong As Long) _
    As Long

Public Declare Function CallWindowProc Lib "user32" _
    Alias "CallWindowProcA" _
(ByVal lpPrevWndFunc As Long, ByVal hwnd As Long, ByVal Msg As Long, _
    ByVal wParam As Long, ByVal lParam As Long) As Long

‘ This function is the Window procedure that you must supply to
‘ override the default message handling

Public Function WindowProc(ByVal hwnd As Long, _
  ByVal uMsg As Long, ByVal wParam As Long, ByVal lParam As Long) _
  As Long

  Dim rc As Long

   If uMsg = WM_FEMAP_MESSAGE Then
       If wParam = FEVENT_SHUTDOWN Then
           Unload Form1
       End If
       If wParam = FEVENT_COMMAND Then
            'MsgBox ("Femap Command")
       End If
```


Unpublished work. © 2025 Siemens

```
        If wParam = FEVENT_PROGSTART Then
            MsgBox ("Program Start")
        End If
        If wParam = FEVENT_PROGEND Then
            MsgBox ("Program End")
        End If

    Else
        WindowProc = CallWindowProc(PreviousWindowProc, hwnd, uMsg, _
                            wParam, lParam)
    End If

End Function

‘ This function is called to initialize capturing the events

Public Function InitApp()
Dim rc As Long
    hMainForm = Form1.hwnd
    Set femap = GetObject(, "femap.model")
                ‘ For more control over window placement and style use
    ‘ feAppRegisterAddInPane( )
    rc = femap.feAppRegisterAddIn(True, hMainForm, hMainForm, True, 1)
    WM_FEMAP_MESSAGE = RegisterWindowMessage("FE_EVENT_MESSAGE")
    PreviousWindowProc = SetWindowLong(Form1.hwnd, GWL_WNDPROC, _
                                            AddressOf WindowProc)
End Function
```


Unpublished work. © 2025 Siemens

```
‘ This function is called to terminate capture


Public Function CloseApp()
   rc = SetWindowLong(hMainForm, GWL_WNDPROC, PreviousWindowProc)
   rc = femap.feAppRegisterAddIn(False, hMainForm, hMainForm, _
                                 True, 1)
End Function
```


Unpublished work. © 2025 Siemens

7. Proprietary and Restricted Rights Notice

This software and related documentation are proprietary to Siemens Digital Industries Software Inc.
© 2025 Siemens Digital Industries Software Inc. All Rights Reserved.
Siemens and the Siemens logo are registered trademarks of Siemens AG. NX is a trademark or registered trademark of Siemens Digital Industries Software Inc. or its subsidiaries in the United States and in other countries. All other trademarks, registered trademarks or service marks belong to their respective holders.
Contains portions or was derived from the RSA Data Security, Inc. MD5 Message-Digest Algorithm.

NeuQuant Neural-Net Quantization Algorithm

The following copyright refers only to the *bmp2raster.exe* executable distributed with Femap: NeuQuant Neural-Net Quantization Algorithm Copyright (c) 1994 Anthony Dekker NEUQUANT Neural-Net quantization algorithm by Anthony Dekker, 1994. See "Kohonen neural networks for optimal color quantization" in "Network: Computation in Neural Systems" Vol. 5 (1994) pp 351-367 for a discussion of the algorithm. See also **https://scientificgems.wordpress.com/stuff/neuquant-fast-high-quality-image-quantization/** Any party obtaining a copy of these files from the author, directly or indirectly, is granted, free of charge, a full and unrestricted irrevocable, world-wide, paid up, royalty-free, nonexclusive right and license to deal in this software and documentation files (the "Software"), including without limitation the rights to use, copy, modify, merge, publish, distribute, sub-license, and/or sell copies of the Software, and to permit persons who receive copies from any such party to do so, with the only requirement being that this copyright notice remain intact.

Intel Fortran Compiler Redistributables – Notice of LIMITATION OF LIABILITY

LIMITATION OF LIABILITY

Intel will not be liable for any of the following losses or damages (whether such losses or damages were foreseen, foreseeable, known or otherwise): (i) loss of revenue; (ii) loss of actual or anticipated profits; (iii) loss of the use of money; (iv) loss of anticipated savings; (v) loss of business; (vi) loss of opportunity; (vii) loss of goodwill; (viii) loss of use of the Materials; (ix) loss of reputation; (x) loss of, damage to, or corruption of data; or (xi) any indirect, incidental, special or consequential loss of damage however caused (including loss or damage of the type specified in this Section 7).

Intel's total cumulative liability to You, including for direct damages for claims relating to this Agreement, and whether for breach of contract, negligence, or for any other reason, will not exceed $100.

You acknowledge that the limitations of liability provided in this Section 7 are an essential part of this Agreement. You agree that the limitations of liability provided in this Agreement with respect to Intel will be conveyed to and made binding upon any customer of Yours that acquires the Redistributables.

Commercial Software: - 3DConnexion SDK (Library) - 2.0

3D input device development tools and related technology are provided under license from 3Dconnexion.
© 3Dconnexion 1992 - 2013. All rights reserved.

Contact information and Femap conventions

Contact information

Siemens DI Software Femap	
Web:	**https://support.sw.siemens.com/en-US/**

Customer Support Femap	
Phone:	(714) 952-5444, (800) 955-0000 (In US & Canada)
Web:	**https://support.sw.siemens.com/en-US/**

Conventions

Femap manuals use different fonts to highlight command names or input that you must type.

`a:setup`	Shows text that you should type.
OK, **Cancel**	Shows a command name or text that you will see in a dialog box.

Windows statement

Throughout this manual, you will see references to Windows. Windows refers to Microsoft® Windows 7, Windows 8, Windows 8.1, and Windows 10 and 11 (64-bit versions only). You will need one of these operating environments to run Femap for the PC. This manual assumes that you are familiar with the general use of the operating environment. If you are not, you can refer to the Windows User's Guide for additional assistance. Similarly, throughout the manual all references to Femap, refer to the latest version of our software.

Unpublished work. © 2025 Siemens